

Storage Systems

Organization, Performance, Coding,
Reliability and Their Data Processing

Alexander Thomasian



Storage Systems

Storage Systems

Organization, Performance, Coding, Reliability,
and Their Data Processing

Alexander Thomasian

Thomasian & Associates
Pleasantville, NY, United States



MK

MORGAN KAUFMANN PUBLISHERS

ELSEVIER

AN IMPRINT OF ELSEVIER

Morgan Kaufmann is an imprint of Elsevier
50 Hampshire Street, 5th Floor, Cambridge, MA 02139, United States

Copyright © 2022 Elsevier Inc. All rights reserved.

MATLAB® is a trademark of The MathWorks, Inc. and is used with permission.
The MathWorks does not warrant the accuracy of the text or exercises in this book.

This book's use or discussion of MATLAB® software or related products does not constitute endorsement or sponsorship by The MathWorks of a particular pedagogical approach or particular use of the MATLAB® software.

No part of this publication may be reproduced or transmitted in any form or by any means, electronic or mechanical, including photocopying, recording, or any information storage and retrieval system, without permission in writing from the publisher. Details on how to seek permission, further information about the Publisher's permissions policies and our arrangements with organizations such as the Copyright Clearance Center and the Copyright Licensing Agency, can be found at our website: www.elsevier.com/permissions.

This book and the individual contributions contained in it are protected under copyright by the Publisher (other than as may be noted herein).

Notices

Knowledge and best practice in this field are constantly changing. As new research and experience broaden our understanding, changes in research methods, professional practices, or medical treatment may become necessary.

Practitioners and researchers must always rely on their own experience and knowledge in evaluating and using any information, methods, compounds, or experiments described herein. In using such information or methods they should be mindful of their own safety and the safety of others, including parties for whom they have a professional responsibility.

To the fullest extent of the law, neither the Publisher nor the authors, contributors, or editors, assume any liability for any injury and/or damage to persons or property as a matter of products liability, negligence or otherwise, or from any use or operation of any methods, products, instructions, or ideas contained in the material herein.

Library of Congress Cataloging-in-Publication Data

A catalog record for this book is available from the Library of Congress

British Library Cataloguing-in-Publication Data

A catalogue record for this book is available from the British Library

ISBN: 978-0-323-90796-5

For information on all Morgan Kaufmann publications
visit our website at <https://www.elsevier.com/books-and-journals>

Publisher: Joe Hayton
Acquisitions Editor: Chris Katsaropoulos
Editorial Project Manager: Rafael G. Trombaco
Production Project Manager: Omer Mukthar
Designer: Mark Rogers

Typeset by VTeX



*Dedicated to the memory of my parents Sarkis and Gayaneh Thomasian and my
sister Lena Movsessian*

Contents

About the author	xvii	1.10.3 Honeywell Corp.	31
Preface	xix	1.10.4 Hewlett-Packard - HP Corp.	31
Acknowledgments	xxv	1.10.5 Digital Equipment Corp - DEC	33
Abbreviations and acronyms	xxvii	1.11 Computer company revenue rankings	34
		1.12 Computer structures book	34
		1.13 Computer family architectures - CFA	35
1. Introduction		1.14 Virtual memory and page replacement algorithms	36
1.1 Computer systems after WW II	2	1.15 Memory space fragmentation and dynamic storage allocation	38
1.2 High level programming languages - Fortran	4	1.15.1 Page replacement algorithms	38
1.2.1 A Programming Language - APL	5	1.15.2 Simplified analysis of a paging system	41
1.2.2 COMmon Business Oriented Language - COBOL	5	1.16 Analysis of thrashing in 2-phase locking - 2PL systems	42
1.2.3 IBM's PL/I programming language	6	1.17 CPU caches	43
1.2.4 Some early computer companies	7	1.18 Multiprogrammed computer systems	47
1.3 Effect of data representation on storage space requirements	8	1.19 Timesharing systems	47
1.4 Basic computer arithmetic	10	1.20 Mean response with FCFS and processor-sharing scheduling	49
1.5 Author's experience with IBM computers in 1970s	11	1.21 Analysis of open and closed queueing network models	51
1.5.1 IBM computers at Univ. of Tehran and IBM World Trade Corp. in Tehran, Iran	11	1.22 Bottleneck analysis and balanced job bounds	53
1.5.2 My experiences with IBM computers at Tehran Regional Electric Company	15	1.23 Performance analyses of I/O subsystems	54
1.5.3 Customer billing at TREC utility	16	1.24 Vector supercomputers	56
1.5.4 My experience with IBM computers at UCLA	18	1.25 Parallel computers	57
1.6 IBM's System 360 and its successors	19	1.25.1 The ILLIAC IV computer	57
1.6.1 US lawsuits against IBM and AT&T	21	1.25.2 Thinking Machines Connection Machine	59
1.6.2 Amdahl Corp. and plug compatible computers	21	1.25.3 Kendall Square Research's KSR-1	59
1.6.3 Radio Corporation of America - RCA	22	1.25.4 Goodyear Massively Parallel Processor - MPP	59
1.6.4 Electronic Data Systems - EDS and Perot Systems	22	1.25.5 MasPar	60
1.7 The IBM S/360 computer family	22	1.25.6 NCUBE	60
1.8 Operating systems associated with IBM mainframes	25	1.25.7 Meiko	60
1.9 Early computer companies possibly competing with IBM	27	1.25.8 SUPRENUM	60
1.9.1 Burroughs + UNIVAC = UNISYS	27	1.25.9 Parsytec	61
1.10 My experience at Burroughs Corp.	28	1.25.10 Intel Personal SuperComputer - iPSC	61
1.10.1 NCR - National Cash Register Corp.	30	1.25.11 IBM's BlueGene supercomputer	61
1.10.2 Control Data Corporation	30	1.25.12 Tesla Dojo supercomputer for AI training	61
		1.26 The future of supercomputing	61
		1.27 Microprocessor CPUs, GPUs, FPGAs, and ASICs	64

1.28 RISC-V and other microprocessors	67	2.3.17 Micro-Electro-Mechanical Systems - MEMS	112
1.29 The IBM PC and its compatibles	71	2.3.18 IBM Zurich millipede	113
1.29.1 Experience with IBM workstations	73	2.3.19 Phase Change Memory - PCM	114
1.30 Storage studies by Alan Jay Smith at Berkeley	74	2.3.20 Flash memories	115
1.31 Prefetching	76	2.3.21 Companies producing flash memories	117
1.32 Database buffers	77	2.3.22 Elevating commodity storage with the SALSA host translation layer	118
1.33 Checkpointing in processing large jobs	78	2.3.23 Flash SSD versus magnetic HDD pricing	120
1.34 Computer related rule of thumb	79	2.3.24 Pure Storage design of Purity	121
1.34.1 Amdahl rules in developing S/360 computers	79	2.3.25 Intel/Micron 3D_XPoint Optane Memory	123
1.34.2 Amdahl's law in the era of multicore	79	2.3.26 Processing In Memory - PIM	125
1.34.3 Amazon optimal configurations for x86-based EC2 instances	80	2.3.27 Universal memory technology - UltraRAM	125
1.34.4 Kung's law	81	2.3.28 Racetrack memory	126
1.34.5 Brooks' law	81	2.3.29 Optical storage	126
1.34.6 Patterson et al.'s roofline bound	81	2.3.30 Holographic memory	126
1.34.7 Gray's rules of thumb	82	2.3.31 M-DISC and storage longevity	126
1.34.8 Jim Gray's five minute rule	83	2.3.32 Persistent and NonVolatile Memory - NVM	127
1.34.9 Moore's law	84	2.3.33 Glass as a new storage medium	128
1.34.10 Wright's law	84	2.3.34 DNA based archival storage system	128
1.34.11 Dennard's law	85	2.4 Reliability studies of DRAM, HDDs, & flash SSDs	131
1.34.12 Huang's law for Graphics Processing Units - GPUs	85	2.4.1 Flash SSD reliability at Facebook, Google & NetApp	133
1.34.13 Grosch's law	86	2.5 Storage Networking Industry Association - SNIA	135
1.34.14 Kryder's law	86	2.5.1 Solid state storage performance	135
1.34.15 Subsecond response times	86	2.5.2 Persistent Memory Forum	135
1.35 Conclusions and summary	87	2.5.3 Computational storage	136
2. Storage technologies and their data		2.6 Big data and its sources	136
2.1 Evolution of recording material	90	2.7 Sources of storage content	138
2.2 Advertising and e-commerce	93	2.8 Ranking and description of media companies	139
2.3 Computer storage technologies	97	2.9 Sources of news: newspapers, radio and TV stations	141
2.3.1 Punched cards - Hollerith and IBM	97	2.9.1 Newspapers in US and worldwide	141
2.3.2 Punched paper tapes	98	2.9.2 TV networks in US	142
2.3.3 Handwriting recognition	100	2.10 Text editing and formatting languages	144
2.3.4 Delay line memories	101	2.11 Online books sources	145
2.3.5 Core memories	101	2.12 Free book download web sites	146
2.3.6 Semiconductor memories	103	2.13 Data, image, audio and video compression	147
2.3.7 Redundant array of Independent Memories - RAIM	103	2.13.1 Data compression	147
2.3.8 Magnetic Random Access Memory - MRAM	104	2.13.2 Huffman coding/encoding	147
2.3.9 Magnetic tapes and tape libraries	105	2.13.3 Lempel-Ziv - LZ encoding	148
2.3.10 An analytical model for a tape library	106	2.13.4 Arithmetic coding	149
2.3.11 Summary of a recent article on magnetic tapes	107	2.13.5 Miscellaneous topics on data compression	149
2.3.12 Origins of Hard Disk Drives - HDDs	108	2.13.6 Universal Resource Locator - URL shortener	150
2.3.13 HDD manufacturers	108	2.13.7 Image compression	151
2.3.14 Storage technologies expected to replace disk drives	110	2.13.8 Video/audio compression	152
2.3.15 Magnetic bubble memories	112	2.14 Main memory data compression	153
2.3.16 Charged Couple Devices - CCDs	112	2.15 Data deduplication in storage systems	157

2.15.1	Data deduplication at Microsoft	158	3.3.3	Linux support for disk scheduling	205
2.15.2	The Venti prototype at Bell Labs/Lucent	158	3.4	Disk scheduling for mixed workloads	205
2.15.3	Data Domain deduplication	159	3.5	Real time disk scheduling for multimedia	206
2.15.4	Datrium	159	3.6	Storage virtualization	207
2.15.5	Summary of a major survey on data deduplication	160	3.7	File placement on disk	208
2.16	Up and coming data deduplication companies	160	3.7.1	Anticipatory disk arm placement	209
2.17	Storage research at IBM's Almaden Research Center in 1990s	163	3.8	Disks with Shingled Magnetic Recording - SMR	210
2.18	Cleversafe and its information dispersal technology	167	3.9	Review of analyses of disk scheduling methods	211
2.19	Recent developments at IBM Research at ARC	170	3.10	Analytic studies of disk storage	213
2.20	Storage research at Hewlett-Packard - HP	171	3.11	Analysis of a zoned disk with the FCFS scheduling	213
2.21	Primary storage vendors and enterprise companies in 2020	172	3.11.1	Disk service time in zoned disks with FCFS scheduling	214
2.22	All-flash upstart storage companies	173	3.12	Performance analysis of the SCAN policy	215
2.23	Hyperconverged infrastructure for storage systems	174	3.13	Analysis of the SATF policy	217
2.24	Top enterprise storage backup players	177	3.13.1	Preliminary investigation of SATF	218
2.25	Data storage companies: up and coming storage vendors	178	3.13.2	First method for the analysis of SATF	219
2.26	Parallel file systems	179	3.13.3	Second method for the analysis of SATF	222
2.27	Cloud storage	179	3.14	Conclusions	222
2.27.1	Cloud computing price models	181	4.	Mirrored & hybrid arrays	
2.27.2	Storage as a service in cloud computing	184	4.1	Introduction to mirrored and hybrid disk arrays	223
2.27.3	Cloud storage elasticity and its benchmarking	184	4.2	Mirrored and hybrid disk array organizations	225
2.28	Jai Menon's predictions on the future of clouds	185	4.2.1	Basic Mirroring - BM	225
2.29	Cloud storage companies	187	4.2.2	Group Rotate Declustering - GRD	226
2.30	Distributed systems research related to clouds	189	4.2.3	Interleaved Declustering - ID	227
2.30.1	OceanStore	189	4.2.4	Chained Declustering - CD	227
2.30.2	Inktomi and CAP theorem	189	4.2.5	Dual striping	228
2.30.3	Replicated data	190	4.2.6	Logical volume and automatic storage management and GPFS	228
2.30.4	Sky Computing	191	4.2.7	LSI Logic RAID	229
2.31	Data encryption	191	4.2.8	Adaptive disk arrays	229
2.31.1	Data encryption for cloud storage	193	4.2.9	SSPiRAL (Survivable Storage using Parity in Redundant Array Layout)	229
2.32	Conclusions - predictions about storage systems	194	4.2.10	B-code	230
2.32.1	Resurgence in shared storage, but Fibre-Channel fades	194	4.2.11	Weaver codes	230
3.	Disk drive data placement and scheduling		4.2.12	Robust, Efficient, Scalable, Autonomous, Reliable - RESAR	231
3.1	The organization of Hard Disk Drives - HDDs	197	4.2.13	Multiway placement	231
3.2	Internal organization of files in UNIX	200	4.2.14	Classification of mirrored and hybrid disk arrays	231
3.3	Review of disk arm scheduling	201	4.3	Routing read requests in mirrored disks	231
3.3.1	Implementation of SATF	202	4.4	Shortening the tail for response times	233
3.3.2	Disk performance studies by Windsor Hsu and Alan Jay Smith at IBM ARC	204	4.5	Improving write performance in mirrored disks	235
			4.6	Disks with multiple R/W heads on a single and multiple arms	235
			4.7	Seek distances in single and mirrored disks	236
			4.8	Mirrored disk performance in normal, degraded, rebuild modes	238

4.8.1	Queueing theory for disk performance evaluation	238	5.2.2	Teradata Corp.	272
4.8.2	RAID1 performance in normal mode	240	5.2.3	National Cash Register Corp. - NCR	272
4.8.3	Uniform vs round-robin routing of read requests in mirrored disks	240	5.2.4	EMC Corp.	272
4.8.4	Completion times of multiple requests in RAID1 and RAID5	241	5.2.5	SUN Microsystems	273
4.8.5	Operation in degraded mode analysis	242	5.2.6	SUN ZFS file system and volume manager	273
4.8.6	Rebuild processing in RAID1	243	5.2.7	International Business Machines - IBM Corp.	274
4.9	Protecting against rare event failures in archival systems	245	5.2.8	Storage Technology Corp. - STK	276
4.10	RAIDP: ReplicAtion with IntraDisk Parity for cost effective storage of warm data	247	5.2.9	Western Digital Corp. - WDC	276
4.11	Remote mirroring for disaster recovery	248	5.2.10	Network Appliance - NetApp Corp.	277
4.12	RAID reliability analysis	249	5.2.11	HP storage products	277
4.12.1	Reliability expressions for mirrored and hybrid disk arrays	249	5.3	RAID classification and motivation	278
4.12.2	Reliability analysis of variants of RAID1 and RAID(4 + k) w/o repair	251	5.4	RAID0 and striping	278
4.12.3	Reliability analysis of variants of RAID1 and RAID(4 + k) w/o repair	251	5.5	RAID2	280
4.12.4	Performability analysis	254	5.6	RAID3	281
4.12.5	Shortcut Reliability Analysis - SRA	254	5.7	RAID4	281
4.12.6	Transient solutions of Continuous Time Markov Chains - CTMCs	256	5.8	RAID5	281
4.12.7	RAID reliability analysis with repair	257	5.8.1	RAID0 and RAID5 stripe unit size	282
4.12.8	Solving linear Ordinary Differential Equations - ODEs	258	5.8.2	Parity striping and load balancing	283
4.12.9	Simplified for RAID5 rebuild reliability analysis	259	5.8.3	Load balancing in disk arrays without striping	283
4.12.10	Enhanced reliability modeling of storage systems	260	5.9	RAID5 performance analysis in normal mode	284
4.12.11	Metrics for reliability estimation in storage systems	262	5.10	RAID(4 + k) disk arrays in normal and degraded mode	286
4.12.12	Expected Annual Fraction of Data Loss - EAFDL	262	5.11	Rebuild processing in disk arrays	288
4.12.13	Taking into account controller failures	263	5.11.1	Variations of rebuild processing in RAID5	289
4.13	Storage reliability research at IBM's Zurich Research Lab	264	5.12	Vacationing server model for rebuild processing	290
4.13.1	Analysis and simulation for reliability estimation	265	5.12.1	Effect of rebuild processing on external request response time	292
4.13.2	Automated Reliability Interactive Estimation System - ARIES	265	5.12.2	Rebuild unit sizes in RAID5	294
4.13.3	System AVailability Estimator - SAVE project at IBM Research	266	5.12.3	Permanent Customer Model - PCM and comparison with VSM	294
4.14	Conclusions	267	5.13	RAID5 sparing configurations for rebuild	295
5.	Redundant Arrays of Independent Disks - RAID		5.13.1	Rebuild by distributed sparing	295
5.1	Redundant Arrays of Inexpensive Disks	270	5.13.2	Rebuild by restriping	295
5.1.1	RAID prototypes at Berkeley and CMU	271	5.13.3	Rebuild with parity sparing	296
5.2	Early RAID products	271	5.14	IntraDisk Redundancy - IDR for higher reliability rebuild	296
5.2.1	Tandem Corporation	271	5.15	Disk scrubbing for higher reliability rebuild processing	298
			5.15.1	Disk scrubbing versus intradisk redundancy	298
			5.16	Predictive Failure Analysis - PFA	301
			5.17	Undetected disk errors and Silent Data Corruption - SDC	301
			5.18	Clustered RAID5 layouts	303
			5.18.1	Balanced Incomplete Block Designs - BIBDs	303
			5.18.2	PRIME data layout	305
			5.18.3	Relatively Prime - RELPR data layout	306
			5.18.4	Thorp permutation for load balancing in clustered RAID5	306

5.18.5	Clustered RAID with Nearly Random Permutations - NRP	306	6.3	Simple data entanglement layouts with high reliability	338
5.19	Clustered RAID designs by Walter Burkhard et al. at UCSD	307	6.4	Reed-Solomon codes	340
5.19.1	Almost Complete Address Translation - ACATS	307	6.5	A family of MDS block array codes with two parities	341
5.19.2	DATUM for tolerating multiple disk failures in disk arrays	309	6.6	Codes for correcting two erasures with independent parities	343
5.19.3	Permutation Development Data Layout - PDDL	309	6.7	Row-Diagonal Parity - RDP codes	346
5.19.4	Balancing loads by shifted parity group placements	311	6.8	Short write operations	347
5.20	Log-structured file systems and arrays	311	6.9	Additional reading	348
5.21	RAID6	315	7.	Saving power in disks, flash memories, and servers	
5.21.1	Supplementary Parity Augmentation - SPA	316	7.1	Introduction to power consumption in computer systems	350
5.22	Reed-Solomon coding for higher reliability	316	7.2	Saving battery power in laptop computers	350
5.22.1	RS coding summary and finite field arithmetic	316	7.3	Varying spindown threshold based on user behavior	350
5.23	Parity based MDS codes	318	7.4	Exploiting idleness in storage systems	351
5.24	RDP arrays and their optimal recovery	318	7.5	Making enterprise computers greener by protecting them better	352
5.25	EVENODD defined and efficient rebuild of a single disk	319	7.6	Policy optimization for dynamic power management	352
5.26	Blaum-Roth - BR code	321	7.7	Managing energy and server resources in hosting centers	353
5.27	X-code disk arrays and rebuild mode with one and two disk failures	321	7.8	Interplay of energy and performance for RAID running OLTP	353
5.27.1	Minimizing reconstruction overhead in MDS RAID	323	7.9	Dynamic speed control for server disk power management	354
5.27.2	Binary MDS array codes with optimal repair	324	7.10	Approaches to conserve disk energy in network servers	354
5.28	The RM2 disk array	324	7.11	Energy efficiency through burstiness	355
5.29	RAID7	326	7.12	Dempsey: a tool for modeling hard disk power consumption	356
5.30	Erasure coding for distributed storage	326	7.13	MAID - Massive Arrays of Idle Disks alternative to tape storage	356
5.30.1	Pyramid codes	327	7.13.1	The Copan MAID storage system	357
5.30.2	HDFS-Xorbas	328	7.13.2	Nexsan AutoMAID	357
5.30.3	Ceph erasure coding with LR code	329	7.14	Self-tuning power aware storage cache replacement algorithm	358
5.30.4	Hitchhiker erasure code	329	7.15	Popular Data Concentration - PDC	359
5.30.5	Hadoop Adaptively-Coded Distributed File System	329	7.16	Disk layout optimization for reducing energy consumption	359
5.30.6	HashTag Erasure Code - HTEC	329	7.17	Managing server energy and operational costs in hosting centers	360
5.31	ReGenerating codes	330	7.17.1	Improving energy savings while meeting performance goals in RAID	362
5.31.1	Pentagon and heptagon codes	331	7.18	Performance directed energy management for main memory and disks	363
5.31.2	Minimum Storage Regeneration - MSR codes	332	7.19	Exploiting redundancy to conserve energy in storage systems	363
5.32	Protection schemes for flash memories	333	7.20	Thermal disk drive design: challenges and possible solutions	364
5.32.1	Flash protection schemes	333	7.21	PARAID: the gear-shifting Power-Aware RAID	365
5.32.2	Differential RAID for flash memories	333			
5.32.3	Partial MDS - PMDS coding for SSDs	334			
5.33	Conclusions	335			
5.33.1	Seagate launches self-healing memory	336			
6.	Coding for multiple disk failures				
6.1	Introduction	337			
6.2	2-Dimensional string layouts	337			

7.22	DiskGroup: energy efficient disk layout for RAID1 systems	365	8.5	Combined shared disk and nothing systems	393
7.22.1	Power provisioning for a warehouse-sized computer	366	8.6	Parallel systems at IBM Research	393
7.23	Pergamum: replacing tape with disk-based archival storage	366	8.7	Interconnection networks in IBM's BlueGene/L	394
7.24	Energy efficient RAID - ERAID	368	8.8	Data allocation and transaction routing in multicomputers	395
7.25	Power reduction via write-offloading	369	8.9	Data allocation with a distributed relational databases	397
7.26	Redundant Arrays of Hybrid Disks - RAHD	369	8.10	Review of multicomputer Data Base Machines - DBMs	398
7.27	Achieving power-efficient, erasure-coded storage	369	8.10.1	Tandem Corporation	399
7.28	Effect of energy-saving schemes on disk reliability	370	8.10.2	Teradata Corporation	401
7.29	Mathematical model of disk reliability versus load and temperature	370	8.10.3	Stratus Technologies	402
7.30	Sample-Replicate-Consolidate mapping - SRCMap	370	8.10.4	Dell Corporation	402
7.31	Power Proportional Distributed File Systems - PPDFS	371	8.10.5	Inspur Systems	402
7.32	Dynamic locality improvement to increase effective storage performance	371	8.11	Benchmarking in various forms	402
7.33	Disk data reorganization for reducing energy consumption	373	8.11.1	Transaction Processing Council - TPC	402
7.34	File assignment with minimal variance of service time	373	8.11.2	Storage Performance Council - SPC	404
7.35	Striping-based Energy Aware - SEA placement	376	8.11.3	Standard Performance Evaluation Corporation - SPEC	404
7.36	PEARL: Performance, Energy, and ReLiability balanced dynamic data distribution	377	8.11.4	Benchmarking distributed databases	405
7.37	Power proportionality for data center storage	378	8.11.5	Mixed workload benchmarks: OLTP and OLAP	405
7.38	Economic evaluation of energy saving with reliability constraint	378	8.11.6	Sort benchmarks	406
7.39	Dynamic server provisioning for data center power management	378	8.12	Data Base Machines - DBMs and backend processors	407
7.40	Modeling the energy costs of I/O workloads	379	8.12.1	List of advantages and disadvantages	408
7.41	Energy proportionality is required in addition to energy efficiency	380	8.13	Head-per-track disks	410
7.42	SDD design tradeoffs from energy perspective	380	8.13.1	Rotating Associative Processor for Information Dissemination - RAPID	410
7.43	Green AI	381	8.13.2	Context Addressable Cellular System for Non-Numeric Processing - CASSM	410
7.44	Conclusions	382	8.13.3	Rotating Associative Relational Store - RARES	410
8.	Database parallelism, big data and analytics, deep learning		8.13.4	The Context-Addressed Segment-Sequential Storage - CASSS	410
8.1	Stonebraker's classification of computer systems	387	8.13.5	Relational Associative Processor - RAP	411
8.2	Comparison of systems from the viewpoint of CPU performance	388	8.13.6	Number of rotations required to read qualifying tuples from RAP	411
8.3	High performance network and channel-based interconnects for storage	388	8.14	Active disks projects	412
8.3.1	Intel Compute Express Link - CXL	391	8.14.1	Active disk project at CMU/PDL	413
8.4	Concurrency and coherency control in data sharing systems	393	8.14.2	Active disk project at UCSB/Maryland	413
			8.14.3	Intelligent disks - IDisks	413
			8.14.4	SmartSTOR project at IBM Almaden	414
			8.14.5	Smart disk cluster project at Northwestern U.	414
			8.14.6	Integrating server, storage and database stack at IBM Almaden	414
			8.14.7	Active disks summary	415

8.14.8	Active storage revisited	415	8.36	RAPID and Oracle AutoML: a fast and predictive AutoML pipeline	452
8.15	Multidimensional indices on disk, DRAM, and flash	415	8.37	Benchmarking automatic ML frameworks	453
8.15.1	Survey of multi-dimensional indices	415	8.38	Alibaba's X-engine	454
8.15.2	Combining dimensionality reduction with clustering	417	8.39	RocksDB with ultrafast data access	456
8.15.3	Dimensionality reduction via SVD, PCA, KLT	417	8.40	LightStore project at MIT	457
8.15.4	Clustering methods for dimensionality reduction	418	8.41	Pink: high-speed in-storage key-value store with bounded tails	457
8.15.5	CSVD to build an index for k -NN queries	419	8.42	BlueDBM: an appliance for big data analytics	457
8.15.6	Nearest-neighbors query processing for multiple clusters	420	8.43	WiSer highly available HTAP DBMS for IoT applications	458
8.15.7	Exact versus approximate k -NN queries	420	8.44	Raven RDBMS at Microsoft provides ML	459
8.15.8	Indices suited for dimensionality reduced data	421	8.45	Machine Learning data platform - MLdp	459
8.15.9	Ordered Partition - OP tree index	421	8.46	Databricks	460
8.15.10	Stepwise Dimensionality Increasing SDI-tree index	422	8.47	Fungible - a new storage architecture for big data	461
8.16	Implementing indices in flash memories	423	8.48	Network requirements for resource disaggregation	464
8.17	Redesign of relational databases by Stonebraker et al.	425	8.49	Deep learning and associated hardware	466
8.18	Parallel Data Base Machines - DBMs	428	8.50	GPU accelerated database systems	467
8.18.1	Almaden Research Backend Relational Engine - ARBRE	428	8.51	Graphics Processing Unit - GPU solutions	470
8.18.2	IBM's DB2 Parallel Edition	429	8.52	Field Programmable Gate Array - FPGA solutions	472
8.19	Google File System, Bigtable, and Spanner	429	8.53	Multichip modules	475
8.20	Microsoft Azure	431	8.54	Unified solutions	477
8.20.1	Amazon DynamoDB - ADDB	432	8.55	Power consumption in FPGAs and ASICs	479
8.20.2	Aurora database for processing continual inputs from sensors	432	8.56	Hybrid approaches to acceleration	480
8.21	IBM and other cloud service providers	433	8.57	Application Specific Integrated Circuit - ASIC	481
8.22	Distributed databases in cloud computing	434	8.58	Tensorflow and Tensor Processing Units - TPUs	482
8.23	SpringFS bridging agility and performance in elastic distributed storage	436	8.59	Increasing computational challenges	484
8.24	Snowflake cloud based data warehousing with SQL support	436	8.60	Quantum Neural Nets - QNNs	485
8.24.1	Storage architecture and provisioning in Snowflake	438	8.61	Data acceleration examples	485
8.24.2	Resource sharing in Snowflake	439	8.61.1	The Q100 data processing unit	485
8.25	Review of peer-to-peer computing	439	8.61.2	Many-core architecture for in-memory data processing at Oracle	486
8.26	Fast Array of Wimpy Nodes - FAWN	441	8.62	Cerebras wafer size chips vs GPUS	487
8.27	RAMCloud project at Stanford	442	8.62.1	The deep learning workload	487
8.28	How flash changes the design of database storage engines	444	8.62.2	Understanding performance	487
8.29	Hybrid Transaction Analytic Processing - HTAP	447	8.62.3	More silicon area means more space for compute cores	488
8.30	Intelligent page store for concurrent txn and query processing	447	8.62.4	More silicon area means more on chip memory	489
8.31	Oracle Exadata database machine	447	8.62.5	Fast communication Swarm fabric	489
8.32	Oracle in memory option or Database in Main Memory - DBIM	449	8.62.6	Co-designed with software for maximum utilization and usability	490
8.33	MemSQL/SingleStore	450	8.63	Conclusions	491
8.34	Amazon Aurora	451	9.	Structured, unstructured, and diverse databases	
8.35	Transaction processing in the cloud	451	9.1	Categories of file systems	494
			9.2	Mainframe count-key-data disk organizations	496

9.3 Hierarchical and network Data Base Management Systems - DBMSs	496	9.33 Streaming analytics	545
9.3.1 Information Management System - IMS	497	9.33.1 Aurora database to manage data streams	545
9.3.2 Transaction Processing Facility - TPF	497	9.34 Trill: a high-performance incremental query processor for diverse analytics	548
9.3.3 Network databases	498	9.35 Summary of Forrester Wave™ streaming analytics, Q3, 2109	549
9.4 Relational data model	498	9.36 Content stores	551
9.4.1 Methods to reduce the level of lock contention in OLTP	500	9.37 Multimodel databases	552
9.4.2 Informix	501	9.38 Main memory databases	553
9.4.3 Berkeley's INGRES relational database	502	9.39 Distributed file systems and object storage	554
9.4.4 Oracle's relational databases	502	9.40 Enterprise Backup and recovery software solutions	557
9.5 Ranking methodology for database engines	503	9.41 Analytics and Business Intelligence - ABI platforms	558
9.6 Overall ranking of all database types	504	9.41.1 Crunchbase finds business information about companies	559
9.7 Relational database management systems	504	9.42 Blockchain, Bitcoin, Ethereum	560
9.8 Object relational databases	507		
9.9 Data mining	509	10. Heterogeneous Disk Arrays - HDAs	
9.10 Data warehousing and OLAP	510	10.1 Introduction to RAID	565
9.10.1 Multi-dimensional OLAP - MOLAP	511	10.2 Data allocation in a Heterogeneous Disk Array - HDA	568
9.10.2 Relational OLAP - ROLAP	512	10.2.1 Balancing disk allocations	568
9.10.3 Hybrid OLAP - HOLAP	512	10.2.2 Allocation methods considered in this study	569
9.11 Distinct schools of thought in data warehouse design	513	10.3 Analytic justification for HDA	570
9.11.1 The Inmon method	513	10.3.1 Improved response time due to HDA	570
9.11.2 The Kimball method	514	10.3.2 The effect of disk failures on response time	571
9.11.3 Deciding factors for Kimball vs Inmon approaches	515	10.3.3 Shortcut reliability analysis of HDA	571
9.11.4 Conclusions on data warehousing	515	10.4 HDA data allocation experiment setup	571
9.12 Data lakes	515	10.4.1 Virtual array allocation requests	572
9.12.1 Major data lake types	516	10.4.2 Estimating virtual array widths in normal mode	572
9.13 Open source big data projects	518	10.4.3 Dealing with load increase in degraded mode	574
9.14 Semi-structured data and its model	519	10.5 Data allocation experiments	574
9.14.1 Pros and cons of semi-structured data format	520	10.5.1 Description of the experiment	574
9.15 Big data technology and the five Vs	520	10.5.2 Assumptions and parameter settings	575
9.16 Hadoop technology ecosphere	521	10.5.3 Comparison of allocation methods	576
9.16.1 Four steps on the workflow of an analytical application	522	10.5.4 Sensitivity analysis	579
9.17 Distributed batch vs inline processing	522	10.5.5 Clustered RAID5	580
9.18 NoSQL/non-relational databases	523	10.6 Rebuild processing in HDA	582
9.19 Key-value stores	524	10.7 RAID+ data layout based on Latin squares	583
9.20 Document stores	525	10.8 Related work	584
9.21 Time-series databases	527	10.9 Using utility functions to provision storage systems	587
9.22 Kubernetes and other containers	529	10.10 Conclusions	590
9.23 Graph databases	531		
9.23.1 Categories of graph models	532		
9.24 Object-oriented databases	533		
9.25 Search engines for text	534		
9.26 Web search engines	536		
9.27 Resource Description Framework - RDF	537		
9.28 Wide column stores	538		
9.29 Multivalue databases	541		
9.30 Native XML databases	542		
9.31 Realtime stream processing	543		
9.32 Event stores	543		

11. Hierarchical RAID - HRAID

11.1 Introduction to HRAID	593
11.2 Intranode & internode coding in HRAID	594
11.3 Concurrency control in HRAID	596
11.4 RAID IOPS with no disk failures	597
11.5 RAID IOPS with disk failures	598
11.6 HRAID response times	599
11.7 HRAID2/2 performance	602
11.8 RAID and HRAID reliability	603
11.9 Shortcut reliability analysis of HRAID	605
11.10 Simulation to estimate the MTDL	606
11.11 Multistep recovery in HRAID	610
11.12 Related work	615
11.13 Collective Intelligent Bricks - CIB or Icecube project at IBM	617
11.13.1 Reliability analysis of Icecube	618
11.14 Conclusions	620

12. Conclusions

Appendix	625
A.1 Books on topics related to storage	625
A.1.1 Computer organization, architecture and fault-tolerance	625
A.1.2 Operating systems	626

A.1.3 Storage systems	626
A.1.4 Coding for storage systems	626
A.1.5 Database systems and machine learning	627
A.1.6 High frequency trading	627
A.1.7 Performance and reliability analysis	627
A.1.8 Computer arithmetic and logic design	628
A.1.9 Algorithms and data structures	628
A.1.10 Peer to peer computing	628
A.1.11 Computer history via interviews	629
A.2 ACM, IEEE, USENIX, and their publications	629
A.3 Journals, conferences, and workshops dealing with storage systems	630
A.3.1 Workshops and conferences related to data base machines	632
A.4 Web sites for trade publications	633
A.4.1 Few interesting blogs	634
A.5 Storage research in industry	634
A.6 Storage research at universities	636
A.7 Funding agencies, national labs, and research institutes	636

Bibliography	641
Index	695

About the author

Alexander Thomasian after graduating from the Koushesh-Davitian Armenian High School in Tehran, Iran was admitted to the Univ. of Tehran to study Electrical Engineering. With a B.S.E.E. he first joined IBM World Trade Corp. as a Systems Engineer, before switching to Tehran Regional Electric Utility - TREC, which had half a million customers in 1970. With a PhD degree in Computer Science from Univ. of Calif. at Los Angeles, and before founding Thomasian & Associates consulting in Pleasantville, NY, he spent half of his time in academia: Case Western Reserve Univ. - CWRU, U. of Southern Calif. - USC, Univ. of Connecticut - UConn, and New Jersey Institute of Technology - NJIT and the other half in industry. He was a Sr. Staff Scientist at: Burrough (now Unisys) Corporate Center for Performance Modeling and Research Staff Member at IBM's T. J. Watson Research Center - TJWRC, with a one year assignment at Almaden Research Center. At NJIT he ran the *Integrated Systems Lab* with a half a dozen students. As an adjunct professor he taught graduate courses at Univ. of Calif. at Irvine, while at Burroughs and at Columbia Univ. while at IBM Research by invitation. He has taught graduate level courses on computer architecture performance analysis, databases and data mining over the years.

On assignment to IBM's Almaden Research Center he devised analytic and simulation models to predict the performance of IBMs RAID5 product under development. He continued research on storage systems at IBM TJWRC and later at NJIT where his storage research was funded by the US Nat'l Science Foundation - NSF, Hitachi Global Storage Technologies - HSGT, and AT&T's Virtual University Research Institute - VURI and then at Thomasian & Associates. He spent a year at Shenzhen Institutes of Advanced Technology in 2010-11 and was a Fulbright Fellow at the American Univ. of Armenia for the first half of 2015.

He is a Life Fellow of IEEE for fundamental contributions to the design and analysis of concurrency control methods and performance analysis of computer systems. At IBM he was recipient of an Outstanding Innovation Award and other awards related to his four patents. He was an editor of IEEE Transactions on Parallel and Distributed Systems and was on the program committees of numerous conferences. His monograph on database concurrency control was republished by Springer. He has 150 over publications in top journals and conferences, which are mainly listed here.

<https://dblp.org/pid/t/AlexanderThomasian.html>

Preface

During the 1960's, most data storage systems used with mainframes and minicomputers were the ones specified by individual computer manufacturers. Since those days, the industry has evolved into a worldwide assortment of computer companies offering systems for a large, diverse mix of applications. The impact on the market for computer storage systems has been huge - Immense markets, divergent host system requirements, short product lives for storage products, numerous storage company start-ups, with valiant attempts to achieve storage product standards and reliability.

Partial abstract for “How computer storage became a modern business” panel at Computer History Museum on March 9, 2005, also see (Katz, 2010).

<https://computerhistory.org/events/how-computer-storage-became-modern/>

The preface is organized as follows: (1) Why this book. (2) Text overview. (3) Intended audience. (4) Book overview. (5) Overview of book chapters. (6) Acknowledgments. (7) Abbreviations and acronyms.

Why this book?

Textbooks on computer organization and architecture do not sufficiently cover storage systems, which is especially a problem in view of rapid recent advancements in the field. There are books dealing with specific storage technologies, e.g., (Qureshi et al., 2011), but few books dealing with storage, especially from the viewpoint of the *Redundant Arrays of Inexpensive/Independent Disk - RAID* paradigm, e.g., (Jin et al., 2001), (Jacob et al., 2008).

This text has been written to fill this gap and much more, i.e., not only storage technologies, but also data storage formats, data compression, encryption and processed. Stored data includes text (books), images (photos), audio (music), video (movies). Data is generated by machines (measurement data) or humans on the internet (emails, tweets) is increasing exponentially.

<https://blog.microfocus.com/how-much-data-is-created-on-the-internet-each-day/>

There is also ephemeral data, which should be deliberately deleted after a specific period of time, has no archiving capability and uses full end-to-end encryption as required by mobile applications, such as WhatsApp.

<https://www.linkedin.com/pulse/just-what-ephemeral-data-phil-beckett/>

Desirable attributes for storage systems are high capacity, high recording density, low access latency, high access bandwidth, nonvolatility, low cost per GigaByte (GB), lengthy lifespan, and low power consumption. These attributes are not independent, e.g., higher linear recording densities lead to higher data transfer rates.

High storage capacity is achievable by magnetic tapes, especially as part of robotic libraries, and magnetic disks. Unless the *Massive Arrive of Idle Drives - MAID* is used rotating disks consume power, while tapes don't. Storage efficiency is increased via data compression and deduplication (used in archiving) and encryption for security and privacy.

RAID paradigm uses replication and *Erasur Coding - EC* for higher availability and durability. Redundancy is in the form of replications which doubles or triples storage requirements, while EC at its best with *Maximum Distance Separable - MDS* codes dedicates the capacity of k disks to tolerate k disk failures in *k-disk-failure tolerant - kDFT* arrays. Data and the codes can be held at distributed sites to deal with disasters such as earthquakes, floods, fires.

Extracting information from data for decision making is an area of intense research and development. Machine Learning - ML algorithms run on CPUs assisted by *Graphics Processing Units - GPUs*, *Field Programmable Gate Arrays - FPGAs*, Google's *Tensor Processing Units - TPUs*, which are *Application Specific Integrated Circuits - ASICs*.

https://en.wikipedia.org/wiki/Tensor_Processing_Unit

The text is kept up-to date by referring to recent publications, trade publications, company announcements, and blogs, which are accessible on the Internet. Readers are encouraged to follow the approach taken in the book in pursuing new developments by subscribing to appropriate publications, e.g., <https://www.theregister.com/>.

Text overview

Computers were initially designed for **computing**: for codebreaking, military, engineering, and scientific applications, hence emphasis was on high speed arithmetic and CPU performance expressed in MIPS and MFLOPS. I/O performance has gained importance with the advent of commercial data processing, but more recently with the advent of *High Performance computing - HPC*, data mining, and *Machine Learning - ML* applications, which require high bandwidth low latency access to data.

Most data processing used to be carried out on disk-resident files and later databases which have important advantages with respect to file system as discussed in many database textbooks, but for a quick summary also in:

<https://www.geeksforgeeks.org/advantages-of-dbms-over-file-system/>

Efficient data organization and algorithms for query processing were developed mainly for relational databases. New algorithms have been developed for NoSQL, multimodel, document, graph, *Key-Value - KV* stores. Current research is now focused on data processing on flash and other novel storage technologies.

The *Redundant Array of Inexpensive/Independent Disks - RAID* paradigm was proposed in (Patterson et al., 1988) to replace expensive, large form factor and high capacity disks of associated with mainframe computers with several low cost, small form factor and capacity disks. At this time all modern disks attain very high capacities at a small form factor with *Fixed Block Architecture - FBA*, while large form factor disks are not manufactured anymore.

Data replication in mirrored RAID1 disk arrays provides higher reliability and higher performance via parallel access to replicated data. Striping partitions large files into fixed sized strips which are placed on consecutive disks in wrap-around manner.

Erasure coding in RAID(4 + k), $k \geq 1$ dedicates k check strips per stripe to check strips, which need to be updated when a data strip is updated, i.e., the disk update disk load increases with k .

Blocks on a failed disk in RAID5 can be reconstructed on demand by *eXclusive ORing - XORing* the surviving blocks in a stripe, while rebuild is the systematic reconstruction of the blocks of a failed disk on a spare disk. Rebuild may fail due to a second disk failure, or when an unreadable sector known as *Latent Sector Error - LSE* is encountered, which is the more likely cause of rebuild failure.

Intra-Disk Redundancy - IDR and disk scrubbing are two methods to deal with LSEs. RAID6 with two check disks using *Reed-Solomon - RS* or parity based codes such as EVENODD and RDP to deal with two disk failures or one disk and an LSE. Parity based codes are desirable because they incur a lower computational cost than RS codes. Specialized *Local Redundancy Codes - LRC* have been developed for low cost recovery in distributed systems. *Intra-Disk Redundancy - IDR* and disk scrubbing are two low-cost methods to deal with LSEs.

Data Base Machines - DBMs use novel hardware and software methods to achieve higher performance for parallel processing. Novel data organizations and processing methods have been developed, such as columnar data. We discuss efforts at universities such as FAWN at CMU, RAMCloud at Stanford, Lightstore at MIT, and new hardware/software from Exadata from Oracle, Aurora from AWS, and Xengine from Alibaba. CPU performance is enhanced by GPUs, FPGAs, and ASICs, which are used by Google's *Tensor Processing Units - TPUs*.

Two chapters are dedicated to author's own research dealing with Heterogeneous Disk Arrays - HDAs and *Hierarchical RAID - HRAID* and related studies.

Intended audience and required background to read the book

The book is intended for students in computer science and engineering, electrical and industrial engineering, management information sciences, information science and technology students. IT professionals may benefit from reading this book to broaden their horizon. Readers exposed to Wintel (Windows on Intel processors) versions of UNIX, and Apple computers will benefit from learning about the variety of computers, including the latest developments.

The book should be of interest to those interested in the history of computing. I started my graduate studies the year Intel started introducing its microprocessors. I drop many names and have been lucky to have exchanged ideas with many of them, some through email.

In addition to historical material the book provides information about company rankings with emphasis on processors, storage, and software (databases - old and new, backup and recovery, containers). This aspect of the book will be of interest to MBAs and students studying MIS.

The reader should preferably have background in basic computer science and mathematics (discrete math, probability, and Markov chains), but may skip sections requiring such background which he/she finds difficult. Readers are encouraged to refer to the original material for details and illustrations.

There are many references not just to technical papers but also to trade publications available on the Internet. Wikipedia has also served as a major source of information.

Attention has been to the academic genealogy of researchers, the rise and also fall of companies.

Overview of book chapters

The book can be used as a reference book and even as an encyclopedia of sorts, so that I am not going to specify a prerequisites chart for reading the book, except that it is better to read Chapter 3, 4, and 5 in that order. Chapters 3 and 4 introduce the performance analysis used in Chapter 5 and Chapter 3 introduces reliability modeling.

Chapter 1: Introduction. Computers gained popularity in commercial, scientific, engineering, and military applications after WW II. IBM introduced the S/360 computer family in 1964 with Multiple Virtual Storage - MVS operating system, which is perhaps the only aspect of IBM that has survived as the z-series with 64 bit addressing, hardware assists, etc. Other computer companies have their own computer families and operating systems, although High Level Languages - HLLs were standardized. I discuss my brief experience as a systems engineer at IBM and then at an electric utility. Microprocessors introduced in 1970 and were later enhanced by GPUs and FPGAs. Parallel and vector computers for parallel computation. Virtual and cache memory organizations and their replacement algorithms are discussed. Queueing Network Models - QNMs and their solution methods, which were used in capacity planning are introduced. Rules of thumb in computing such as Moore and Amdahl laws are discussed.

Chapter 2: We discuss storage technologies starting with paper which allowed writing and later printing. Punch cards were used to store census data, company records and simple data processing. Magnetic tapes and disks (also optical) held data, images, audio and video. High bandwidth communications have resulted in a paradigm shift by downloading and streaming replacing CDs and DVDs. Tapes can only be accessed sequentially and hence can only be used for batch processing after sorting data records. Disks allow both sequential and random accesses via hashing or indexing. For high performance flash memories and other SCMs - Storage Class Memories - SCMs are replacing magnetic disks. Data compression and deduplication can be used in preserving storage space and network bandwidth utilization and encryption for data security and privacy. Many other storage technologies some of which are no longer pursued, while other are under intense investigation.

Chapter 3: Disk drive organization, data placement, and scheduling. Commodity disks with Fixed Block Architecture - FBA (fixed size sectors) replaced Count Key Data - CKD disks associated with IBM mainframes. Zoned bit recording - ZBR, Shingled Magnetic Recording - SMR are among schemes to increase disk capacity. Disk scheduling policies and data placement to minimize disk positioning time have been under investigation since mid 1960s. Shortest Access Time First - SATF scheduling is extended with a version providing lookahead and two priority classes. Analysis of queueing delays in disks with FCFS (with zoning) and SCAN policy is discussed. A review of performance evaluation of storage subsystems using queueing analysis and simulation is presented.

Chapter 4: Mirrored and hybrid arrays. Mirrored disks classified as RAID1 protect against disk failures by storing data twice, while hybrid arrays store the exclusive-or of two disks between them, allowing all two disk failures and half of consecutive three disk failures to be tolerated. RAID1 doubles disk access bandwidth. Access latency can be improved by data placement and disk scheduling, e.g., accessing the disk with shorter seek time. The fact that a disk failure in Basic Mirroring - BM results in the doubling of the load on surviving disk has led to several RAID1 configurations, which distribute the load of a failed disk over multiple disks for a balanced load. Shortcut reliability analysis shows these data layouts to be less reliable than BM. Hybrid disk arrays store original and XORed data blocks. Analytic and simulation methods to estimate RAID and more generally computer systems reliability are presented.

Chapter 5: Variation of RAID with emphasis on RAID(4 + k), $k = 1 : 3$ which use k check disks to protect against as many disk failures. Operation in normal, degraded, and rebuild modes is described and its performance analyzed. Clustered RAID5 uses parity group size G smaller than the number of disks to reduce disk overload when a disk fails. Balanced Incomplete Block Designs - BIBD and Nearly Random Permutations - NRP are two methods to place parity blocks to balance disk loads. Various methods for rebuild processing including the Vacationing Server Model - VSM and Permanent Customer Model - PCM are presented. IntraDisk redundancy - IDR and disk scrubbing methods to deal with Latent Sector Errors - LSEs, which may lead to unsuccessful rebuild are presented. RAID6 utilizes Reed-Solomon codes, but also parity based methods such as EVENODD and RDP to implement 2-Disk-Failure-Tolerant - 2DFT arrays. Coding technique applicable to distributed storage systems is also discussed.

Chapter 6: Coding methods. EVENODD, RDP, and X-code are Maximum Distance Separable - MDS codes in that they incur the minimum level of redundancy to tolerate two disk failures with parity coding. RDP is shown to outperform EVENODD in the number of XORs required. Two-dimensional codes are also discussed.

Chapter 7: Power reduction in storage systems and servers. Power consumption is a major problem for server and storage clouds. We start the discussion with disk spindown in notebooks to increase battery lifetimes. Server energy can be saved by lowering voltage and in the case of disks by lowering RPMs and spindown. Massive Arrays of Independent Disks - MAID paradigm spins down most disks, but leaves a few disk spinning for caching recently accessed data and for new updates. Hybrid disks with a flash memory or NonVolatile RAM - NVRAM cache to hold updates can be spundown to save energy, since flash memories with no mechanical parts consume less energy than disks. Power consumption can be reduced by temporal and spatial clustering of disk accesses.

Chapter 8: Database parallelism, big data, analytics, and deep learning. Early Data Base Machines - DBMs were mainframes running database management systems - DBMSs. Data mining on mainframes was considered too costly, since results obtained by data mining were considered of questionable value. The advent of powerful low-cost microprocessors allowed the building of DBMs affording a high degree of parallelism, such as the Teradata DBC/1012, which has been used for data mining and data warehousing. Active disks process higher priority disk accesses for OnLine Transaction Processing - OLTP, while processing data mining requests as no cost freeblock accesses. Disks with a processor per track capability, such as the Relational Associative Processor - RAP, are no longer feasible because of high track densities, but the concept of associating processing power has been applied to flash storage and DRAM. Systems combining OLTP and Online Analytic Processing - OLAP are discussed. We describe prototypes: FAWN at CMU, RAMCloud at Stanford, and Lightstore at MIT; Oracle's Exadata, AWS' Aurora, Alibaba's PolarDB, Fungible Data Center. That companies can roll their own systems optimized for their application as explained in the case of Alibaba is a new phenomenon, which at the other extreme has Amazon Web Service - AWS, which acts as a universal computing utility.

Chapter 9: We discuss structured, semi-structured, and unstructured data, which includes network, hierarchical, and relational databases, big data, Hadoop technology ecosphere, NoSQL databases, key-value stores, document stores, times series, graph, and object-oriented databases, web search engines, Resource Description Framework - RDF, wide column stores, native XML and JSON databases, realtime database, event stores, content store, multimodel databases, main memory databases. Novel applications such as streaming analytics and business intelligence platforms.

Chapter 10: Heterogeneous Disk Arrays - HDAs allow multiple Virtual Arrays - VAs to share disk space using multi-function controllers. The RAID level and organization of VAs is determined by reliability and workload requirements, e.g., RAID1 is suited for OLTP with high access rates to small data blocks, while RAID5 is better suited for parallel accesses to large files. Allowing variable RAID levels conserves disk bandwidth, because not all VAs need to be allocated with the highest redundancy level required by a subset of datasets.

Chapter 11: Hierarchical RAID - HRAID. HRAID is motivated by the storage bricks paradigm. Each Storage Node - SN has a controller implementing the intranode erasure code over SN's disks. HRAID uses a multilevel RAID paradigm with internode erasure coding. Disk failures are handled first by intranode redundancy and internode redundancy if intranode redundancy fails. Internode redundancy can also handle SN failures. Nodes may be provided with higher levels of redundancy RAID(4 + k), $k \geq 1$, and restriping is used for rebuild by overwriting check strips. An baseline array with no interdisk redundancy is considered for comparison purposes. Operations with and without intranode rebuild processing are considered. Simulation is used to determine the effect of varying HRAID parameters on the Mean Time to Data Loss - MTDDL.

Chapter 12: We discuss few topics not covered in the book to encourage readers to pursue them on their own.

Each book chapter is preceded by its table of contents, which should assist the reader in deciding what to read and what to skip.

Books related to storage systems listed in the Appendix can be classified as follows.

1. Computer organization and architecture and fault-tolerance
2. Operating systems
3. Storage systems
4. Coding for storage systems
5. Database systems, machine learning
6. High frequency trading
7. Performance and reliability analysis
8. Computer arithmetic
9. Algorithms and data structures
10. Peer to peer computing
11. Computer history

Aside from books there are the following sources are provided in the Appendix.

- Journals and conference proceedings by ACM, IEEE, USENIX, etc.
- Publication sources and self-publishing.
- Journals, conferences & workshops relevant to storage systems.
- Workshops and conferences related to DBMs - Data Base Machines.
- Web sites for trade publications.

Miscellaneous

I relied heavily on the following web sites in preparing this manuscript.

1. Google search engine: <https://www.google.com>, which was cofounded by Larry Page and Sergey Brin. I learned on 8/20/21, just before submitting the book, that Google had Scott Hassan as a third founder and he contributed a lot of initial code.
<https://www.nytimes.com/2021/08/20/technology/Scott-Hassan-Allison-Huynh-divorce.html>
2. Wikipedia cofounded by Larry Sanger and Jimmy Wales: <https://en.wikipedia.org> articles.
3. *Database and logic programming - dblp* initiated by Michael Ley to build book's bibliography.
<https://dblp.uni-trier.de>, <https://dblp.org>
4. Google's scholar web site provides citation counts: <https://scholar.google.com>
5. Free online book library: <https://book4you.org> or <https://1lib.us>

Other relevant web sites are: <https://www.researchgate.net/>, <https://semanticscholar.org>

I have provided the *universal resource locators - urls* inline, rather than in the reference list for readers convenience. Most urls were accessed recently, so download dates are not given to save space. Graphs are not reproduced, except those appearing in author's papers. Some tables are reproduced with copyright permission for reader convenience.

Author affiliations are provided to the extent known to me. A web site dealing with genealogy in mathematical sciences is:

<https://www.genealogy.math.ndsu.nodak.edu/>

For example, Garth Gibson was a PhD student of David Patterson graduated from *U. Calif. at Berkeley - UCB* in 1991. Patterson was a PhD student of David Frederic Martin (rip) at UCLA. Patterson retired from UCB in 2016 to join Google. PhD Theses at the EECS Dept. at UCB are available here: <https://www2.eecs.berkeley.edu/Pubs/Dissertations/>, but Gibson's award winning PhD thesis was only published as a book (Gibson, 1992). Gibson joined the CS Dept. at *Carnegie-Mellon U. - CMU*, where he founded the *Parallel Data Laboratory - PDL* at CMU, <https://www.pdl.cmu.edu>, but he now heads the Vector Institute in Toronto, Canada.

<https://vectorinstitute.ai/>

Academic genealogy is also provided here:

<https://academictree.org/computerscience/>

The author would appreciate reader emails to comment on book contents and reporting mistakes and typos with (page#, line#) to alexthomasian@gmail.com. Please add "storagebook" in the subject line. I intend to maintain the Errata which I will happily share with contacting readers, provided they specify their backgrounds and affiliations.

The book cover is that of Armenia's holy Mount Ararat, with 7th century Zvartnots Cathedral in the foreground.

https://www.ancient.eu/Zvartnots_Cathedral/

This view can be seen on the highway leading to Yerevan's Zvartnots Int'l Airport.

*"I've spent too many years at war with myself
The doctor has told me it's no good for my health
To search for perfection is all very well
But to look for heaven is to live here in hell."*
"Consider me gone" by Sting in "The dream of the blue turtles"

Acknowledgments

The teachers of Tehran's Armenian Kooshesh-Davitian school (now defunct), where one hour per day was dedicated to teach Armenian language, history, etc. (1952-64).

Univ. of Tehran and Calouste Gulbenkian Foundation scholarship for the first and last two years of U. Tehran education (1964-68).

IBM System Engineers Massih Ettefagh and Jafar Namazi at IBM's *World Trade Corp.* - *WTC* in Tehran who involved me in their projects (1968-69).

Sahak Sahakian (rip) hired me at *Tehran's Regional Electric Company* - *TREC* to lead in modernizing its large scale billing system (1969-71).

Algirdas Avizienis supported my PhD studies at the Computer Science Dept. at *Univ. of Calif. at Los Angeles* - *UCLA* and served as my PhD advisor. Dr Maniel Vineberg for supervising my masters project.

<http://avizienis.info>

Leonard Kleinrock for his excellent queueing theory courses at UCLA, which were the most helpful in my further research career.

<https://www.lk.cs.ucla.edu/index.html>

Richard Muntz distributed his class notes and an early draft of (Williams and Bhandiwad, 1976) in his 212B course. These served as a starting point for my research in queueing network modeling.

Chuck Rose (rip) for hiring me as an assistant professor in the System Engineering Dept. at *Case Western Reserve Univ.* - *CWRU* in Cleveland, OH (1977-79).

George Bekey reporting to Sr VP and Dean of Engineering Zohrab Kaprielian (rip) hired me in the EE-Systems Dept. at *Univ. Southern Calif.* - *USC* on Melvin Breuer's (rip) recommendation (1979-1983).

PhD students at USC: Paul Bay, Behzad Nadji, and In Kyung Ryu tirelessly collaborated in joint research.

Jeffrey Buzen as editor of *Computer Performance* helped me publish my work in a timely manner.

Kayvan Kiamanesh at Alpha-Data Corp. who funded a performance analysis study of multiheaded disk arms in 1983.

Blaine Gaither hired me as a Sr. Staff Scientist at the Corporate Center for Performance Modeling - CCPM of Burroughs Corp. - now Unisys (1983-84).

Steve Lavenberg (rip) hired me as a Research Staff Member in his Systems Analysis Dept. at IBM's T.J. Watson Research Center (1985-1998).

Harut Barsamian (rip) at *Univ. of Calif. at Irvine* - *UCI* approved my tutorial proposal for Int'l Conf. on Distr. Computing Systems - ICDCS'89. Erhard Rahm collaborated with me on the initial versions of the tutorial.

Collaboration with Peter Franaszek and John Robinson at IBM Research led to an OIA - Outstanding Innovation Award for the patent (Franaszek et al., 1993) and a top-rated RAID patent (Franaszek et al., 1996).

Robert Morris and Jai Menon hosted my assignment at IBM's Almaden Research Center - ARC to lead a RAID performance study.

Vittorio Castelli (*Amazon Web Services* - *AWS*) and Chung-Sheng Li (Accenture, PwC) collaborated with me in a NASA project at IBM Research which led to two patents (Castelli et al., 1997a,b) and (Castelli et al., 2003).

Joseph Leung (rip) who was hired as the chair of the Computer Science Department at *New Jersey Institute of Technology* - *NJIT* to improve its ranking hired me as a Full Professor as a step in that direction.

Yoav Raz (rip) and Amnon Namad hosted an extended version of my SIGMETRICS'98 tutorial on RAID performance analysis at EMC Corp. in Hopkinton, MA, December 2000.

Yavuz Oruc on assignment at *National Science Foundation* - *NSF* from U. Maryland approved funding for my proposal on performance analysis of *2-Disk Failure Tolerant* - *2DTF* arrays (2002-2005).

Spencer W. Ng (Sandisk, retired) at *Hitachi Global Storage Technologies* - *HGST* provided funding for my research on evaluating RAID5 rebuild processing at NJIT in 2005.

Behzad Nadji (Blackberry/Research in Motion, retired) as director of AT&T Research provided funding for my research at NJIT via VURI - Virtual U. Research Inst. project in 2006.

NJIT PhD students Chunqi Han, Yue Li*, Lijuan (Catherine) Zhang*, and Jun Xu, and masters students Alex Branzoi, Gang Fu, Chang (Linda) Liu, Chintan Shah, Saman Zarandioon assisted me with my research on storage systems and high-dimensional indexing (*).

Mario Blaum (IBM *Almaden Research Center* - ARC) coauthored a paper on RAID1 reliability analysis (Thomasian and Blaum, 2006) and a survey on *2-disk-failure-tolerant* - 2DFT disk arrays (Thomasian and Blaum, 2009).

Student Jose Pagan (<https://www.XKL.com>) installed Sun Solaris on a Dell Desktop (2007).

Lionel Ni (*Hong Kong U. of Science and Technology* - HKUST) serving as a director at *Shenzhen Inst. Science and Tech.* - SIAT hosted my stay as an Outstanding Int'l Visiting Scientist (2010-2011). Dr Yujie Tang (algomau.ca) with a masters from Harbin U. assisted me in my research.

Yuhui Deng at Jinan U. in Guangzhou invited me to present my research.

Fulbright fellowship award to teach at *American Univ. of Armenia* - AUA (AUA.am) for the first half of 2015. I collaborated with Victor Ohanyan at AUA/*Yerevan State Univ.* - YSU and was invited to lecture at YSU by Samvel Shokourian. I was befriended by cryptography expert Gurgen Khachatryan (<http://www.qdsc.us/QDSC/GK.html>). All three are members of National Academy of Sciences of the Republic of Armenia.

<https://www.sci.am>

Tamer Ozsu (U. Waterloo) as *Editor in Chief* - EIC of ACM Book Series encouraged me to submit a book to ACM, but stepped down in May 2019.

Chris Katsaropoulos, Sr. Acquisitions Editor at Elsevier approved my book proposal in November 2020. I am working with Elsevier's publishing team Rafael G. Trombaco (Sao Paulo, Brazil) and Omer Mukthar (Chennai, India).

Numerous persons have responded to my questions: Mario Blaum (IBM ARC), Ilias Iliadis (IBM ZRL), Dieter Gawlick (Oracle), Randy Katz (Univ. of Calif. at Berkeley), Harold Stone (IBM Research, NEC, consultant), Joshua Knight (IBM Research, retired), Ken Shirriff. <http://www.righito.com/>

Physical therapist Sue Ock Glynn, PhD at Burke Rehabilitation center in White Plains, NY dealt with my BPPV or vertigo condition expertly.

My dedicated and loving wife Ling Li Thomasian took care of most mundane matters, while I was working on the book, especially as after the pandemic started. I owe her an apology for not being helpful with the household chores. Our son Chenglin tried to be less annoying with his interruptions during the book writing.

Abbreviations and acronyms

Some abbreviations provided here are well known in *United States* - *USA* where the book was written, but then US constitutes 4.25% of world population at this time.

<http://srv1.worldometers.info/world-population/us-population/>

Powers of ten used in our discussions are as follows:

kilobyte - KB (3), megabyte - MB (6), gigabyte - GB (9), TB - terabyte (12), petabyte - PB (15), exabyte - EB (18), zettabyte - ZB (21), yottabyte - YB (24), and brontobytes - BB (10), googol (100), googolplex (googol).

0-9

2DFT 2-Disk-Failure-Tolerant.

2PC 2-Phase Commit.

2PL 2-Phase Locking.

8b/10b Encoding for DC balance.

A

AC/DC Alternating/Direct Current.

ACID Atomicity, Consistency, Isolation, and Durability.

AES Advanced Encryption Standard.

AFD Advanced Format Drive.

AFR Annual Failure Rate.

AHCI Advanced Host Controller Interface.

AMAT Average Memory Access Time.

AMD American Micro Devices (Corp.).

ANN Artificial Neural Network.

ANSI American National Standards Institute.

API Application Program Interface.

APL A Programming Language.

ARC Adaptive Replacement Cache.

ARC Almaden Research Center (IBM).

ARM Advanced RISC Machine.

ARR Annual Replacement Rate.

ATA Advanced Technology Attachment (IBM PC).

AWS Amazon Web Services.

B

b bit.

B byte.

BBS Bulletin Board System.

BCH Bose-Chaudhuri-Hocquenghem (code).

BER Bit Error Rate.

BI Business Intelligence.

BIBD Balanced Incomplete Block Design.

BM Basic Mirroring.

C

CAP Consistency, Availability, and Partition Tolerance.

CAS Content Addressable Storage.

CAS Chinese Academy of Sciences.

CBIR Content Based Image Retrieval.

CC Concurrency Control.

CNN Convolutional Neural Network.
CCD Charge-Coupled Device.
CD Chained Declustering.
CDC Control Data Corporation.
CDN Content Delivery Network.
CEO Chief Executive Officer.
CEP Complex Event Processing.
CIA Central Intelligence Agency.
CIB Collective Intelligent Bricks (IBM/ARC).
CISC Complex Instruction Set Computer.
CMR Conventional Magnetic Recording.
CMOS Complementary Metal Oxide Semiconductors.
CMS Conversational Monitor Systems.
COW Copy On Write.
CPI Cycles per Instruction.
cpi characters per inch (magnetic tape).
CPU Central Processing Unit.
CRC Cyclic Redundancy Code.
CRM Customer Relationship Management.
CSCAN Cyclical SCAN.
CSM Central Server Model.
CSMA/CD Carrier Sense Multiple Access/Collision Detection.
CSCAN Cyclical SCAN.
CTO Chief Technical Officer.
CTMC Continuous Time Markov Chain.
CTO Chief Technical Officer.
CUDA Compute Unified Device Architecture.
CVE Common Vulnerabilities and Exposures.
CVS Concurrent Version System.
CWRU Case Western Reserve University.
CXL Compute Express Link.

D

DAC Disk Array Controller.
DAS Direct Attached Storage.
DASD Direct Access Storage Device.
DBC/M Data Base Computer/Machine.
DBMS Data Base Management System.
DDL Data Definition Language.
DEC Digital Equipment Corporation.
DES Data Encryption Standard.
DFTL Demand-based Flash Translation Layer.
DFVS Dynamic Frequency and Voltage Scaling.
DIMM Dual Inline Memory Module.
DL Deep Learning.
DMA Direct Memory Access.
DML Data Manipulation Language.
DNN Deep Neural Network.
DoD Department of Defense.
DOR Disk Oriented Rebuild.
DoW Dedicate on Write.
DR Disaster Recovery.
DRAM Dynamic Random Access Memory.
DS Decision Support.
DSA Dynamic Storage Allocation.
DSA Domain Specific Architecture.
DSN Dispersed Storage Network.
DSS Decision Support Systems.
DTM Dynamic Thermal Management.
DTMC Discrete Time Markov Chain.
DVFS Dynamic Voltage and Frequency Scaling.

E

EAFDL Expected Annual Fraction of Data Loss.

EC Erasure Coding.
ECC Error Checking and Correcting (memory technology).
ECKD Extended Count Key Data (IBM DASD).
ECL Emitter Coupled Logic.
EDF Earliest Deadline First.
EDS Electronic Data Systems.
EPIC Explicitly Parallel Instruction Computing (microprocessor).
EPFL École Polytechnique Fédérale de Lausanne.
ERP Enterprise Resource Planning.
ES Enterprise Storage.
ETL Extract, Transform, Load.
EXCP EXecute Channel Program.

F

FAB Federated Array of Bricks (HP).
FAST File and Storage Technologies (conference).
FAT File Access Table (Microsoft).
FAWN Fast Array of Wimpy Nodes (PDL/CMU).
FBA Fixed Block Architecture.
FBI Federal Bureau of Investigation (U.S.).
FCFS First-Come, First-Served.
FCP Fibre Channel Protocol (SCSI).
FFS Fast File System (UNIX).
FGCS Fifth Generation Computer System.
FHE Fully Homomorphic Encryption.
FIFO First-In, First-Out.
F/J Fork/Join (queueing system).
FLOP Floating Point Operations per Second.
FLP Floating Point arithmetic.
FPGA Field-Programmable Gate Array.
FLP Floating Point (numbers).
FSCK File Systems Consistency Check.
FTL Flash Translation Layer.

G

GA Genetic Algorithm.
gcd greatest common divisor.
GE General Electric.
GIGO Garbage-In, Garbage-Out.
GPFS Global Parallel File System (IBM).
GPR General Purpose Registers (IBM S/360).
GPU Graphical Processing Unit.
GRD Group Rotate Declustering (mirroring).
GUI Graphical User Interface.

H

HAMR Heat-Assisted Magnetic Recording.
HBM High Bandwidth Memory.
HCI Human Computer Interaction.
HCI Hyper-Converged Infrastructure.
HDA Heterogeneous Disk Array.
HDA Hybrid Disk Array.
HDD Hard Disk Drive.
HDFS Hadoop Distributed File System.
HLL High Level Language.
HOAP Hybrid Operational Analytic Processing.
HPE Hewlett-Packard Enterprise.
HPL Hewlett-Packard Labs.
HRAID Hierarchical RAID.
HTAP Hybrid Transaction Analytic Processing.
HTTP HyperText Transfer Protocol.

I

IaaS Infrastructure as a Service.
IBM International Business Machines.
IDA Information Dispersal Algorithm.
IDE Intelligent/Integrated Data Electronics Interface.
IDEMA Int'l Disk drive Equipment and Materials Association.
ID Interleaved Declustering (mirroring).
IDA Information Dispersal Algorithm.
IDC Int'l Data Corp.
IDR Intra-Disk Redundancy.
IMS Information Management System (IBM).
I/O Input/Output.
IOPS I/Os Per Second.
IoT Internet of Things.
ISA Instruction Set Architecture.
ISAM Index Sequential Access Method.
ISP Instruction Set Processor.
IWARP Internet Wide Area RDMA Protocol.

J

JBOD Just a Bunch of Disks.
JPEG Joint Photographic Experts Group.
JPL Jet Propulsion Labs.
JSON JavaScript Object Notation.
JSQ Join the Shortest Queue.

K

kDFT k Disk Failure Tolerant.
KLT Karhunen-Loève Transform.
KV Key-Value (store).
KWH KiloWattHour.

L

LAMP Linux, Apache, MySQL, PHP.
LAN Local Area Network.
LBA Logical Block Address (on disk).
lcm least common multiple.
LDPC Low Density Parity-Check.
LFS Log-structured File System.
LFU Least Frequently Used.
L.P. Limited Partnership.
LRC Local Recoverable code.
LRU Least Recently Used.
LSA Log-Structured Array.
LSE Latent Sector Error.
LSM Log Structured Merge (trees).
LST Laplace Stieltjes Transform.
LTO Linear Tape Open.
LUN Logical Unit Number.
LVM Logical Volume Management.
LZ Lempel-Ziv (data compression).

M

MAID Massive Arrays of Independent Disks.
MAMR Microwave Assisted Magnetic Recording.
MCC Microelectronics and Computer Consortium.
MD5 Message Digest 5 (algorithm).
MDS Maximum Distance Separable.
MEMS MicroElectroMechanical System.
MFLOPS Mega Floating Point Operations Per Second.
MFT Multiprogramming with a Fixed Number of Tasks.
MIMD Multiple-Instructions, Multiple Data stream computer.
MIPS Millions of Instructions Per Second.

MIPS Microprocessor with Interlocked Pipeline Stages.
ML Machine Learning.
MLC Multi-Level Cell (flash memory).
MOLAP Multidimensional OLAP.
MOLS Mutually Orthogonal Latin Squares.
MPEG Moving Picture Experts Group.
MPI Message Passing Interface.
MPL MultiProgramming Level.
MPP Massively Parallel Processing.
ms millisecond.
MSR Minimum Storage Regeneration.
MSRP Maximize Service Revenue and Profit.
MSS Mass Storage Systems.
MTBF Mean Time Between Failures.
MTTR Mean Time to Repair.
MTTF Mean Time to Failure.
MTTDL Mean Time to Data Loss.
MULTICS Multiplexed Information and Computing Service.
μs microsecond.
MULTICs Multiplexed Information and Computing Service.
MVA Mean Value Analysis.
MVCC Multi-Version Concurrency Control.
MVS Multiple Virtual Storage (IBM OS).
MVT Multiprogramming with a Variable Number of Tasks.
MXT Memory eXpansion Technology (IBM).

N

NAS Network Attached Storage.
NASA National Aeronautics and Space Administration.
NASDAQ National Association of Securities Dealers Automated Quotation.
NBS National Bureau of Standards (now NIST).
NCQ Native Command Queueing.
NCR National Cash Register (Corp.).
NetApp Network Appliance (Corp.).
NFS Network File System.
NFT Non Fungible Token.
NIC Network Interface Card.
NIST National Institute of Science and Technology.
NLP Natural Language Processing.
NMSE Normalized Mean Squared Error.
NN Neural Network.
NoSQL Not only SQL.
NSA National Security Agency (U.S.).
NSF National Science Foundation (U.S.).
NRP Nearly Random Permutations.
ns nanosecond.
NUMA Non-Uniform Memory Access.
NVRAM/NVS NonVolatile RAM/Storage.
NYU New York University.

O

OCC Optimistic Concurrency Control.
ODM Original Design Manufacturer.
OEM Original Equipment Manufacturer.
OLAP OnLine Analytic Processing.
OLTP OnLine Transaction Processing.

P

PaaS Platform as a Service.
PASTA Poisson Arrivals See Time Averages.
payg pay-as-you-go.
PCA Principal Component Analysis.
PCIe Peripheral Component Interconnection express.
PCM Permanent Customer Model (queueing theory).

PCM Phase Change Memory.
PCM Plug Compatible Machine.
PCM Pulse Code Modulation.
PDC Popular Data Concentration.
pdf probability density function.
PDF Probability Distribution Function.
PDF Portable Document Format.
PFA Predictive Failure Analysis.
PFF Page Fault Frequency.
PFS Parallel File System (Panasas).
PIM Processing in Memory.
P.K. Pollaczek-Khintchine (queueing formula).
plc public limited company.
PL/I Programming Language/I.
pmf probability mass function.
PMR Perpendicular Magnetic Recording.
POSIX Portable Operating System Interface.
POWER Performance Optimized With Enhanced RISC (IBM).
PS Personal Storage.
PUE Power Usage Effectiveness.
PVFS Parallel Virtual File System.

Q

QC Quantum Computing.
QLC Quad Level Cell.
QNM Queueing Network Model.
QoS Quality of Service.

R

RAID Redundant Array of Independent/Inexpensive Disks.
RAIM Redundant Array of Independent Memories.
RAM Random Access Memory.
RAMAC Random Access Method of Accounting and Control.
RAIL Redundant Array of Independent Libraries.
RAP Relational Associative Processor.
RCW ReConstruct Write.
RDBMS Relational Data Base Management System.
RDF Resource Description Framework.
RDMA Remote Direct Memory Access.
RDP Rotated Diagonal Parity (NetAPP).
RDS Relational Data Service (Amazon).
RGC ReGenerating Code.
RISC Reduced Instruction Set Computer.
RLE Run-Length Encoding.
RMF Resource Management Facility.
RMS Root Mean Square (error).
RMW Read-Modify-Write.
ROLAP Relational OLAP.
RPC Remote Procedure Call.
RPM Rotations Per Minute (disk).
RPO Rotational Positioning Optimization.
RPS Rotational Position Sensing.
RS Reed-Solomon (code).
RSA Rivest-Shamir-Adleman (public key encryption).
RU Rebuild Unit (RAID5).
RVA RAMAC Virtual Array (IBM).
R/W Read/Write (disk head).

S

SaaS Software as a Service.
SAN Storage Area Network.
SAS Serial Attached SCSI.
SCU Santa Clara University.

SATA Serial Advanced Technology Attachment.
SATF Shortest Access Time First.
SCM Storage Class Memory.
SCSI Small Computer System Interface.
SDC Silent Data Corruption.
SDC System Development Corporation.
SDS Software Defined Storage.
SEC/DED Single Error Correcting/Double Error Detecting.
SGI Silicon Graphics Inc.
SHA Secure Hash Algorithm.
SIS Single Instance Storage (deduplication).
SIMD Single-Instruction, Multiple-Data stream computer.
SJF Shortest Job First.
SLA Service Level Agreement.
SLC Single Level Cell (flash memory).
SMART Self-Monitoring Analysis and Reporting Technology.
SMF Systems Management Facility.
SMR Shingled Magnetic Recording.
SMP Symmetric MultiProcessor.
SMS Short Message Service.
SMTP Simple Mail Transfer Protocol.
SNIA Storage Networking Industry Association.
SOAP Simple Object Access Protocol.
SOR Stripe Oriented Rebuild.
SPA Supplementary Parity Augmentation.
SPARC Scalable Processor ARChitecture.
SPC Storage Performance Council.
SPEC Standard Performance Evaluation Corporation.
SPOF Single Point Of Failure.
Spooling Simultaneous Peripheral Operations On-Line
SPTF Shortest Processing Time First.
SRAM Static Random Access Memory.
SRC Systems Research Center (DEC).
SSB Star Schema benchmark.
SSD Solid State Drive/Device.
SSF Shortest Seek First.
SSP Storage Service Provider.
SSRC Storage Systems Research Center (UCSC).
SSTF Shortest Seek Time First.
STK StorageTek, Storage Technology Corp.
SU Stripe Unit or Strip.
SVD Singular Value Decomposition.
SVL Silicon Valley Lab. (formerly STL).
SWP Small Write Penalty.

T

TCO Total Cost of Ownership.
TCQ Tag Command Queueing.
TIFF Tagged Image File Format (Corp.).
TIP Transparent Informed Prefetching.
TJWRC T. J. Watson Research Center (IBM).
TLB Translation Lookaside Buffer (virtual memory).
TMR Triple Modular Redundancy.
TNS Tandem NonStop System.
TPC Transaction Processing Council.
TPM Traditional Power Mode.
TPF Transaction Processing Facility (IBM).
TPS Transactions Per Second.
TPU Tensor Processing Unit.
TSO Time-Sharing Option (IBM).
TSO Time-Stamp Ordering method (concurrency control).
txn transaction.

U

U University.
UBER Uncorrectable Bit Error Rate.
UC U. California.
UCB UC Berkeley.
UCI UC Irvine.
UCLA UC at Los Angeles.
USC U. Southern California.
UCSC UC at Santa Cruz.
UCSD UC at San Diego.
UDE Undetected Disk Errors.
UDP User Datagram Protocol.
UIUC U. Illinois at Urbana-Champaign.
UML Unified Modeling Language.
UPS Uninterruptible Power Supply.
URE Undetectable Read Error.
URL Universal Resource Locator.
USB Universal Serial Bus.
USP User Service Platform.
USPTO US Patent and Trademark Office.
URL Universal Resource Locator.
UTM Urchin Tracking Module.
UWE Undetectable Write Error.

V

VA Virtual Array.
VAX Virtual Address eXtension (DEC).
VBR Variable Bit Rate.
VCN Voice Coil Motor.
VD Virtual Disk.
VLIW Very Long Instruction Word.
VoD Video on Demand.
VM Virtual Machine.
VM Virtual Memory system.
VMS Virtual Memory System (DEC OS).
VSA Virtual Storage Appliance.
VSAM Virtual Storage Access Method.
VSM Vacationing Server Model.

W

WAL Write Ahead Logging.
WDL Wait-Depth Limited (locking method).
WDM Wavelength Division Multiplexing (also DWDM - dense).
WORM Write Once Read Many.
WSE Wafer Scale Engine.
W-SPTF Window-based SPTF.
WWW World Wide Web.

X

XA eXtended Architecture (IBM).
XOR eXclusive-OR.

Y

YARN Yet Another Resource Negotiator.
YCSB Yahoo! Cloud Serving Benchmark.
YFQ Yet another Fair Queueing.

Z

ZBR Zoned Bit Recording.
ZLR Zero Latency Read.

Chapter 1

Introduction

Computer systems after WW II in Section 1.1
High level language Fortran in Section 1.2
A Programming Language - APL in Subsection 1.2.1
Common Business Oriented Language - COBOL in Subsection 1.2.2
IBM's PL/I programming language in Subsection 1.2.3
Early computer companies in Subsection 1.2.4
Effect of data representation on storage space requirements in Section 1.3
Basic computer arithmetic in Section 1.4
Author's experience with IBM computers in Section 1.5
Computers at Univ. of Tehran and IBM Iran in Subsection 1.5.1
Computers at *Tehran's Regional Electric Company - TREC* in Subsection 1.5.2
Customer billing at TREC in Subsection 1.5.3
My experience with IBM computers at UCLA in Subsection 1.5.4
IBM's System 360 and its successors in Section 1.6
US lawsuits against IBM and AT&T in Section 1.6.1
Amdahl Corp. and plug compatible computers in Subsection 1.6.2
Radio Corporation of America - RCA in Subsection 1.6.3
Electronic Data Systems - EDS and Perot Systems in Subsection 1.6.4
The IBM S/360 computer family in Section 1.7
Operating systems associated with IBM mainframes in Section 1.8
Early computer companies competing with IBM in Section 1.9
Burroughs + UNIVAC = Unisys in Subsection 1.9.1
My experience at Burroughs Corp. in Section 1.10
National Cash Register - NCR Corp. in Subsection 1.10.1
Control Data Corporation - CDC in Subsection 1.10.2
Honeywell Corp. in Subsection 1.10.3
Hewlett-Packard - HP Corp. in Subsection 1.10.4
Computer company revenue rankings in Section 1.11
Computer structures textbook in Section 1.12
Computer family architectures - CFA in Section 1.13
Virtual memory and page replacement algorithms in Section 1.14
Memory space fragmentation and dynamic storage allocation in Section 1.15
Page replacement algorithms in Section 1.15.1
Analysis of thrashing in 2-phase locking - 2PL systems in Section 1.16
CPU caches in Section 1.17
Multiprogrammed computer systems in Section 1.18
Timesharing systems in Section 1.19
Mean response with FCFS and processor-sharing scheduling in Section 1.20
Analysis of open and closed queueing network models in Section 1.21
Bottleneck analysis and balanced job bounds in Section 1.22
Performance analyses of I/O Subsystems in Section 1.23
Vector supercomputers in Section 1.24
Parallel computers in Section 1.25
The ILLIAC IV computer in Subsection 1.25.1
Thinking Machines' Connection Machine in Subsection 1.25.2

Kendall Square Research's KSR-1 in Subsection [1.25.3](#)
Goodyear Massively Parallel Processor - MPP in Subsection [1.25.4](#)
MasPar in Subsection [1.25.5](#)
NCUBE in Subsection [1.25.6](#)
Meiko in Subsection [1.25.7](#)
SUPRENUM in Subsection [1.25.8](#)
Parsytec in Subsection [1.25.9](#)
Intel Personal SuperComputer - iPSC in Subsection [1.25.10](#)
IBM's BlueGene supercomputer in Subsection [1.25.11](#)
Tesla Dojo supercomputer for AI training in Subsection [1.25.12](#)
The future of supercomputing in Section [1.26](#)
Microprocessor CPUs, GPUs, FPGAs, and ASICs in Section [1.27](#)
RISC-V and other microprocessors in Section [1.28](#)
The IBM PC and its compatibles in Subsection [1.29](#)
Experience with IBM workstations in Subsection [1.29.1](#)
Storage studies by Alan Jay Smith at Berkeley in Section [1.30](#).
Prefetching in Section [1.31](#)
Database buffers in Section [1.32](#)
Checkpointing in processing large jobs in Section [1.33](#)
Computer related rules of thumb in Section [1.34](#)
Amdahl's rules in Subsection [1.34.1](#)
Amdahl's law in multicore era in Subsection [1.34.2](#)
Amazon optimal configurations for x86-based EC2 instances in Subsection [1.34.3](#)
Kung's law in Subsection [1.34.4](#)
Brooks' law in Subsection [1.34.5](#)
Roofline model in Subsection [1.34.6](#)
Gray's rules in Subsection [1.34.7](#)
Gray's five minute rule in Subsection [1.34.8](#)
Moore's rule in Subsection [1.34.9](#)
Wright's law in Subsection [1.34.10](#)
Dennard's law in Subsection [1.34.11](#)
Huang's law for Graphics Processing Units - GPUs in Subsection [1.34.12](#)
Grosch's law in Subsection [1.34.13](#)
Kryder's law in Subsection [1.34.14](#)
Subsecond response times in Subsection [1.34.15](#)
Conclusions and summary in Section [1.35](#)

1.1 Computer systems after WW II

An outline of history of computing hardware starting with calculators is given below.

https://en.wikipedia.org/wiki/History_of_computing_hardware

The reader is also referred to Appendix M "Historical Perspectives and References" in (Hennessy and Patterson, 2017).

Early computers were built to carry out arithmetic operations and hence their name. Early "computers" were human beings, mostly women, which were organized into teams to carry out repetitive calculations on calculators.

Two parallel teams could be used to crosscheck results. The 2016 "Hidden Figures" book was turned into a movie about the role of African American female "computers" at NASA, who did more than just computing.

<https://www.history.com/news/human-computers-women-at-nasa>

https://en.wikipedia.org/wiki/Hidden_Figures

Computers for codebreaking were developed during WW II and this included the Colossus computer developed in Bletchley Park in 1943. Alan Turing was a mathematician who devised machines to decipher the German code Enigma. Alan Turing The Enigma in a biographical book by Andrew Hodges, which served as source for the movie "The Imitation Game".

https://en.wikipedia.org/wiki/Colossus_computer

https://en.wikipedia.org/wiki/Alan_Turing

Early computers were classified scientific and commercial categories, but there were also general purpose computers. Scientific computers did fast *FLloating Point - FLP* arithmetic but little *Input/Output - I/O*, while commercial computers did decimal arithmetic and provided higher output - I/O bandwidth to read/write data from magnetic tapes and disks.

David Kuck who contributed software to the ILLIAC IV computer at *U. Illinois at Urbana-Champaign - UIUC* authored “Structures of Computer and Communications” (Kuck, 1979), which starts with the early history of IBM, which turned out to be the most influential company in the second half of the 20th century. According to (Hennessy and Patterson, 2017). In a keynote address at the 15th Annual International Symposium on Computer Architecture - ISCA (1988) he commented that “Also, I/O needs a lot of work”, but his comment was preceded by Seymour Cray in a public lecture in 1976 “I/O certainly has been lagging in the last decade”.

“IBM founder Thomas J. Watson Sr. worked as a door-to-door salesman for *National Cash Register - NCR Corp.* using a horse and buggy carrying sales items, including coffins which he offered to grieving families. He was fired from NCR in 1914 when he got drunk and his buggy was stolen (hence IBM’s no drinking policy). Watson used his experience as salesman at NCR to join *Computing Tabulating Recording - CTR*, https://en.wikipedia.org/wiki/Computing-Tabulating-Recording_Company which was renamed *International Business Machines - IBM* in 1924. https://en.wikipedia.org/wiki/History_of_IBM

Watson Sr. divided the company into IBM US led by Thomas J. Watson, Jr. and IBM *World Trade Corp. - WTC* led by his younger son, Arthur K. Watson in 1949. By 1970 IBM WTC produced half of IBM’s revenue. <https://www.ibm.com/ibm/history/ibm100/us/en/icons/ibmworldtrade/> Watson Jr. authored the autobiography “Father, Son, & Company” (Watson and Petre, 2000). In 2017 the number of IBM employees in India exceeded US, hence the claim that IBM stands for “Indian Business Machines”.

IBM’s System/360 computer family announced in July 1964 is perhaps the most important product announcement in history.

https://en.wikipedia.org/wiki/IBM_System/360

System/360 computers were designed to cover the full range of applications, from small to large, commercial, scientific, and even real time. The IBM System/360 delivered between 1965 and 1978 led to *Plug Compatible Machines - PCMs* with similar *Instruction Set Architectures - ISAs*.

Extensions of the System/360 were used in successive computer families including the current z-series.

https://en.wikipedia.org/wiki/Instruction_set_architecture

An article in praise of the IBM System/360 was written on the 50th anniversary of its announcement.

https://www.theregister.co.uk/Print/2014/04/07/ibm_s_360_50_anniversary/

Adjusted for inflation IBM was the most valuable company in history in 1967. It was among the top ten till 2013, but is now (8/12/21) ranked #116 at \$12.23B.

<https://www.huntedhead.com/2013/03/25/the-worlds-most-valuable-companies-throughout-history-5-ibm/>

<https://www.statista.com/statistics/263264/top-companies-in-the-world-by-market-capitalization/>

Apple in 2021 exceeded the \$2 trillion valuation threshold, while Microsoft, Amazon, and Alphabet/Google far exceed \$1 trillion.

https://en.wikipedia.org/wiki/List_of_public_corporations_by_market_capitalization

A key advantage of System/360 computers besides sharing peripherals was their compatibility in that they shared the same *Instruction Set Architecture - ISA*, i.e., compilers and assembler language programmers dealt with the same programming interface, the same instruction set and registers. IBM’s low-end System/360 models used software routines or microprograms to carry out *FLloating Point - FLP* arithmetic.

Higher end models System/360s were too costly for small universities and engineering companies. As a successor to the IBM 1620 computer IBM introduced the single-user IBM 1130 computer in 1965 with a Fortran compiler. It was the least expensive computer at the time.

https://en.wikipedia.org/wiki/IBM_1130

An IBM 1620 was available to students at U. Tehran, but it seems to me that it was also used for student registration by producing id cards to which a passport photograph was attached. Following a successful compilation of a Fortran program the 1620 would punch out the object code on card deck. This deck with added appropriate subroutines and input data was fed back to the card reader for program execution.

Aryamehr university founded in 1996 rented an IBM 1130 computer equipped with a slow removable magnetic disk, which in spite of being slow was obviated the need for handling card decks.

https://www.ibm.com/ibm/history/exhibits/1130/1130_intro.html

Aryamehr U. was renamed Sharif U. of Technology. after the Shah of Iran whose nickname was Aryamehr (Light of the

Aryans) was deposited in 1979. Sharif U. later acquired a CDC Cyber 174 computer, which was a modern version of CDC 6600.

https://en.wikipedia.org/wiki/CDC_Cyber

Dr Mohammad Ali Mojtahedi, the principal of Alborz High School in Tehran (1944-1979) and also lecturer at U. Tehran was asked by the Shah in 1965 to recruit Iranian faculty worldwide to establish a new university specializing in science and technology as described in “Mojtahedi and the Founding of the Arya-Mehr University of Technology” (Zarghamee, 2011). Iranian students from Sharif U. and U. Tehran have been very successful in pursuing PhD degrees in US and Europe. This has included Maryam Mirzakhani, the only woman to win the Fields Medal in Mathematics for under forties.

https://en.wikipedia.org/wiki/Fields_Medal

https://en.wikipedia.org/wiki/Maryam_Mirzakhani

The IBM 1800 computer had the same ISA as the 1130 and could support multiple users via timesharing.

https://ethw.org/IBM_1800

The duplexed 1800 was a highly reliable computer for data acquisition and process control (Harrison et al., 1981).

https://en.wikipedia.org/wiki/IBM_1800_Data_Acquisition_and_Control_System

Early computers with magnetic core memories had small memory sizes because of the high cost of core memories to keep overall costs down. Computers with limited memory size used secondary storage as an extension of the main memory with appropriate planning by the programmer, before virtual memory systems were implemented, see Section 1.14. Overlay programming allowed programs larger than the main memory to be run on the 1130.

[https://en.wikipedia.org/wiki/Overlay_\(programming\)](https://en.wikipedia.org/wiki/Overlay_(programming))

The main program A after calling subroutine B first would call subroutine C next which would overlay B , resulting in a total memory requirement $|A| + \max\{|B|, |C|\}$, where $|A|$ denotes A 's size, rather than $|A| + |B| + |C|$.

Consider the multiplication of two square $N \times N$ matrices A and B yielding C , i.e. $C = A \cdot B$. For larger values of N the $3N^2$ memory words would be required to hold the three matrices in main memory. The memory requirement could be reduced to $3N$ words, assuming A is stored row first and B column first on disk. C is computed one row at a time, which is then written to disk by We reading successive rows in A and then successive columns in B to compute one row in C . Fetching row $A_{i,*}$ and a column $B_{*,j}$, $1 \leq j \leq N$ from disk would allow computing.

$$C_{i,j} = \sum_{k=1}^N A_{i,k} \times B_{k,j}, \quad 1 \leq i \leq N \quad 1 \leq j \leq N.$$

Virtual memories are discussed in Section 1.14.

1.2 High level programming languages - Fortran

John Backus developed the *Formula translation - Fortran* high-level programming language at IBM in 1950s, which simplified the programming of numerical calculations tremendously, thus popularizing computers for scientific and engineering applications.

<https://en.wikipedia.org/wiki/Fortran>

Fortran uses FLP numbers which obviate the need for scaling for numerical calculations and integers for indexing arrays. Integer variables in Fortran start with letters I-N by default, while other letters are used for FLP variables. Both are stored as 32 bit words, but there are also halfword (16 bit) integers. Higher FLP precision is achieved with double precision representation and requires two words. Fortran had provisions to save memory space. The EQUIVALENCE statement allows multiple arrays to share memory space, while the COMMON statement is used to pass arguments from the main program to a subroutine, with no need for copying them.

Backus later participated in developing ALGOL and is responsible for the *Backus-Naur Form - BNF* to formalize Algol's syntax. Upon receiving the Turing award in 1977 Backus wrote a paper on *Functional Programming - FP* (Backus, 1978).

https://en.wikipedia.org/wiki/John_Backus

Backus who retired from IBM in 1991, was invited back to IBM's *Almaden Research Center - ARC* in 1993 to celebrate the Charles Draper Prize from the *National Academy of Engineering - NAE*. He gave his speech facing the wall, complaining that IBM did not provide him with adequate resources to implement FP. What follows is based on *Engineering and Technology Wiki - ETWH*.

https://ethw.org/John_Backus

“An FP interpreter was distributed with the 4.2BSD Unix operating system. Backus spent the latter part of his career developing *Functional Level - FL*, a successor to FP. FL was an internal IBM research project and many of the language’s innovatively important ideas were later implemented in Iverson’s J programming language.” The design of FL is given below.

<https://theory.stanford.edu/~aiken/publications/trs/FLProject.pdf>

1.2.1 A Programming Language - APL

Functional programming has similarities to APL leased developed at IBM by Kenneth E. Iverson and released in 1966.

[https://en.wikipedia.org/wiki/APL_\(programming_language\)](https://en.wikipedia.org/wiki/APL_(programming_language))

APL can specify arithmetic operations on arrays succinctly, e.g., the addition of the elements of two vectors with equal lengths A and B can be expressed as $C \leftarrow A + B$.

APL set was used by Algirdas Avizienis at UCLA to specify computer arithmetic algorithms.

More importantly, the instruction set of IBM System/360 computers is specified succinctly in (Falkoff et al., 1964) and in “Computer Architecture: Concepts and Evolution” (Blaauw and Jr., 1997).

In 1975 IBM released the 5100 portable computer, which was an early personal computer running APL.

https://en.wikipedia.org/wiki/IBM_5100

https://en.wikipedia.org/wiki/History_of_personal_computers

APL2 released in 1984 was a major APL extension supporting nested non-rectangular arrays.

Wai-Mee Ching developed an APL compiler at IBM Research. He also parallelized APL to run on the IBM *Research Parallel Processor Prototype - RP3* computer (Pfister et al., 1985) as described in (Ching and Ju, 1991). As noted in (Gottlieb et al., 1986) the RP3 was based on the Ultracomputer proposed at by Allan Gottlieb et al. at *New York U. - NYU*.

<https://en.wikipedia.org/wiki/Ultracomputer>

The Ultracomputer interconnection network had a combining feature for synchronization operations, which were expected to rarely happen but slowed down the network for all accesses. RP3 nodes made from *Research OPD Micro Processor - ROMP* processors, where *OPD - Office Product Division*, had no FLP hardware.

When RS/6000 was announced it had more FLP computing power in one engine than RP3 had in the octant, even with added FLP hardware according to RP3 architect Greg Pfister. Only one octant (1/8 of a full system) was completed and the RP3 built at a cost of \$40 Million was placed in large crates to be mothballed. Wai-Mee Ching later defined and implemented ELI, a language with full APL functionality, but not requiring a special keyboard with Greek letters.

[https://en.wikipedia.org/wiki/ELI_\(programming_language\)](https://en.wikipedia.org/wiki/ELI_(programming_language))

<http://fastarray.appspot.com/index.html>

1.2.2 COmmon Business Oriented Language - COBOL

COBOL was developed in 1960 to deal with the high cost of programming military and commercial applications by the US government.

<https://en.wikipedia.org/wiki/COBOL>

COBOL was a team effort led by Grace Hopper, inventor of Flowmatic and Jean Sammet who authored “Programming Languages: History and Fundamentals” in 1969. This book was used as a textbook at UCLA’s computer science department in 1970s.

https://en.wikipedia.org/wiki/Jean_E._Sammet

As an invited speaker to UCLA Sammet posed the question: “Why are all COBOL formulas preceded with the verb COMPUTE?” and the answer was that all statements start with a command to tell the computer what to do. COBOL variables can have lengthy names to make the programs self-documenting. Richard Wexelblat’s “History of Programming Languages” complemented Sammet’s book (Wexelblat, 1981). His U. Pennsylvania PhD in Computer Science in 1965 is the first.

IBM Fellow Jim Brady architected a highly parallel system to automate Year 2000 - Y2K compliance for COBOL programs. This system featured a self-modification process that allowed the system to evolve from an 80% to a 99.98% conversion rate in the first 4 months of production allowing the system to convert five billions lines of COBOL code per month with a 0.000004% error rate.

<https://www.crunchbase.com/person/jim-brady-2#section-overview>

A lazy fix of the Y2K bug led to problems in 2020.

<https://www.newscientist.com/article/2229238-a-lazy-fix-20-years-ago-means-the-y2k--is-taking-down-computers-now/>

COBOL programs use decimal arithmetic, where the number of digits for variables is specified by the programmer based on the expected maximum value. Founding members of successful companies gain wealth through their stock holdings after the company's *Initial Public Offering - IPO*. This paradigm led to the current wealthiest individuals: John Bezos who holds 11.1% stake in Amazon, He was temporarily overtaken by Elon Musk.

<https://www.investopedia.com/articles/investing/012715/5-richest-people-world.asp>

The highest salary in US is almost \$600,000.000, which takes eleven digits, including cents.

<https://www.bloomberg.com/graphics/2020-highest-paid-ceos/>

The proposed hourly minimum wage is currently \$15.00 per hour, \$600 per week, or $\approx$ \$30,000K per year, which requires seven digits.

1.2.3 IBM's PL/I programming language

IBM developed the procedural block structured *Programming Language/I - PL/I* in conjunction with its System/360 computers in 1960s. PL/I combines features from Fortran to specify formulas, Algol's block structure and dynamic data allocation, and COBOL in dealing with file structures. IBM encouraged its commercial System/360 customers to use PL/I rather than COBOL in order to make it difficult to migrate to non-IBM computers, which mostly did not run PL/I.

PL/I was used to write the *Multiplexed Information and Computing Service - MULTICS* operating system described in (Organick, 1972) to run on a *GeneralElectric - GE-645* computer.

<https://en.wikipedia.org/wiki/Multics>

IBM attempted to turn away from assembly language to higher level languages as early as 1965, and was making substantial use of PL/S by 1969, e.g. in MVS. PL/S was considered a trade secret at the time and was not available to customers.

https://en.wikipedia.org/wiki/IBM_PL/S

The C programming language developed at Bell Labs

[https://en.wikipedia.org/wiki/C\(programming_language\)](https://en.wikipedia.org/wiki/C(programming_language))

is similar to PL/I in that it is a high level programming language which was used in developing an operating system UNIX in 1970s.

<https://en.wikipedia.org/wiki/Unix>

Variables declared within begin blocks and procedures or subroutines are allocated memory space dynamically when entered and deallocated upon exit PL/I similarly to C/C++ allows dynamic data allocation/deallocation for CONTROLLED and BASED variables.

https://en.wikibooks.org/wiki/Software_Engineers_Handbook/Language_Dictionary/PLI/storage_classes

C uses new/free statements are substitutes for PL/I's ALLOC/DEALLOCATE statements. Pointers are used to locate instances of structures comprised of several variable, one of which is a pointer to build linked lists for discrete event simulation.

PL/I became available on many computers, even those of *Digital Equipment Corp. - DEC*. The many versions of PL/I are discussed below.

<https://en.wikipedia.org/wiki/PL/I>

Simulation programs for modeling queueing systems allocate storage dynamically to hold the attributes of arriving jobs or customers. Storage is deallocated when the job is completed. In a single server queue system with arrival rate λ and service rate $\mu > \lambda$ and server utilization factor $\rho = \lambda/\mu < 1$, the queue-length varies but remains finite. If $\mu > \lambda$ and $\rho > 1$ then the queue-length increases with simulation time (T_{sim} : Queue-length = $(\lambda - \mu)T_{sim}$). This will eventually lead to memory space exhaustion and a system crash. Memory leaks due to failure to deallocate memory space for completed jobs leads to memory space exhaustion even for $\rho < 1$ for lengthier simulations.

https://en.wikipedia.org/wiki/Memory_leak

Matrix elements are stored row- or column-major-order depending on the programming language.

https://en.wikipedia.org/wiki/Row-_and_column-major_order

This should be taken into account in accessing arrays to take advantage of spatial locality, since this will improve the cache hit rate and reduce program runtime, see Section 1.17.

This difference should also be noted when subroutine calls are made from one programming language to another. "Numerical Recipes: The Art of Numerical Computing" was initially written in Fortran, but later became available in C, C++, MATLAB®, Python, etc.

<http://www.nr.com>

MULTICS was a joint project between Bell Labs, *General Electric -GE*, and *Project Mathematics and Computation - MAC* at MIT delivered in 1967 was led by Fernando Corbato, who was later awarded the Turing award for his work on

time- and resources-sharing computers. It was preceded by MIT's *Compatible Time-Sharing System - CTSS* which ran on an IBM 709 in 1961.

https://en.wikipedia.org/wiki/Compatible_Time-Sharing_System

https://en.wikipedia.org/wiki/Fernando_J._Corbat%C3%B3

<http://groups.csail.mit.edu/mac/projects/mac/>

GE later sold its computer business to Honeywell, but under the direction of Jack Welch GE tried to buy it back, but this attempt was blocked by the *European Union - EU*.

<http://content.time.com/time/business/article/0,8599,166732,00.html>.

The current Honeywell Corp. is a result of its merger with Allied Signals in 2002.

<https://en.wikipedia.org/wiki/Honeywell>

One level store was an innovation associated with MULTICS, which provided a very large address space accommodating large files (Organick, 1972). It also introduced hierarchical file system, and ring-oriented security, which is an extension of two CPU modes known as slave/master or user versus kernel, supervisor, or privileged mode. The CPU can only execute I/O instructions in master mode. This makes data on disks less susceptible to corruption.

https://en.wikipedia.org/wiki/CPU_modes

Ten most popular programming languages are as follows.

<https://dev.to/javinpaul/top-10-most-popular-programming-languages-and-their-creators-59e1>

With Wikipedia and [WordPress.com](https://www.wordpress.com) among big sites running on server-side PHP code, it remains one of the most commonly used scripting languages for building websites and web applications, according to Stack Overflow – even if it is considered one of the “most-dreaded” languages to use.

<https://en.wikipedia.org/wiki/PHP>

1.2.4 Some early computer companies

Prime computer company was founded in 1972 by William Poduska with other participants of the Multics project.

https://en.wikipedia.org/wiki/Prime_Computer

The slowness of Prime computers made them eligible for export by *Coordinating Committee for Multilateral Export Controls - CoCom* regulations, which came into effect after WW II.

https://en.wikipedia.org/wiki/Coordinating_Committee_for_Multilateral_Export_Controls

but this did not help and the company was defunct by 1999.

Poduska left Prime to start Apollo Computer in 1980, two years before Sun Microsystems was founded.

https://en.wikipedia.org/wiki/Apollo_Computer

In the period 1980-87 Apollo was the largest manufacturer of network workstations allowing demand paging over the network. Apollo was acquired by Hewlett-Packard in 1989 and gradually closed down in the next decade.

Data General - DG was founded by engineers from *Digital Equipment Corp - DEC* most notably Edson de Castro and Henry Burkhardt III.

https://en.wikipedia.org/wiki/Data_General

DG released the AViiON (a play on DG's NOVA II) series of UNIX servers originally with the Motorola 88K RISC processors,

https://en.wikipedia.org/wiki/Motorola_88000

which allowed multiprocessing with *Non-Uniform Memory Access - NUMA*.

https://en.wikipedia.org/wiki/Non-uniform_memory_access

DG was the only major customer of the Motorola 88K line and when Motorola stopped its production in favor of the PowerPC (a joint effort with IBM and Apple) DG switched to Intel x86 processors, but had to compete with vendors such as Sequent producing similar computers. DG later produced CLARiiON SCSI RAID, which made it a takeover target by EMC in 1999. Tracy Kidder's 1981 bestseller “The Soul of a New Machine” is about DG (Kidder, 2000).

Sequent Computer Systems was founded in 1983 in Beaverton, OR, was acquired by IBM in 1999 for \$810 million.

https://en.wikipedia.org/wiki/Sequent_Computer_Systems

Sequent's first computers in 1984 were the Balance 8000 with 19 MHz National Semiconductor NS32032 processors with small write-through caches. The next series Symmetry in 1987 was based on Intel 80386 with 2-30 processors with write-back caches. Given that *Shared Memory Processor - SMP* was being integrated in microprocessors, Sequent started producing *cache coherent Non-Uniform Memory Architecture - ccNUMA* running *Dynamic Unix - Dynix*. Sequent became defunct in 1999.

1.3 Effect of data representation on storage space requirements

IBM System/360 computers provide binary and decimal fixed point representations. Binary numbers can be written compactly as hexadecimal numbers with A:F standing for 10:15. Binary numbers are used for Floating Point - FLP number representation, but there is also a decimal FLP representation.

Decimal numbers occupy 4 or 8 bits in packed and unpacked representations, respectively. The eight bit unpacked representation per decimal digit is half as efficient as packed decimal, since the high order 4-bits specify that this is a number not a character. The maximum two byte number with unpacked decimal is 99 and 999 with packed decimal. In packed decimal representation the last 4-bits specify the sign (C for positive and D for negative). For unpacked representations the high order four bits of the last byte represent the sign.

In the case of half- or full-word binary numbers the highest order bit is set to 0 for positive and 1 for negative. The range of binary numbers is divided in half, with half numbers designated as positive and the other half negative as shown below where the last two rows are two's and one's complement numbers

0	1	2	3	4	5	6	7
0	1	2	3	-4	-3	-2	-1
0	1	2	3	-3	-2	-1	-0

A problem with ones complement arithmetic is that it has a negative zero, which tests as zero. Another problem is end around carry, so that an overflow bit should be added back. https://en.wikipedia.org/wiki/Ones%27_complement

A similar convention is used for the sign bit of FLP numbers. With 2's complement number representation the largest positive number is $2^{15} - 1 = 32767$. The value of a negative number is obtained by flipping all bits and adding one, hence the smallest negative number is -2^{15} . The largest 2's complement number is $2^{31} - 1 = 2,147,483,647$, so that the formatted representation requires 2.5 times more bytes. When two numbers with the same sign are added overflow is detected if the sum has a different sign. Subtraction is attained by first flipping the sign of the subtrahend (the number to be subtracted) and adding it to the minuend.

Floating Point - FLP numbers are used to represent very small and large numbers with a fraction multiplied by powers of two or sixteen (in the case of System/360).

https://en.wikipedia.org/wiki/Floating-point_arithmetic

The 1985 IEEE 754 FLP standard has a binary and decimal version. The motivation for the standard was to make it easier to provide portable, robust mathematical software.

https://en.wikipedia.org/wiki/IEEE_754

https://en.wikipedia.org/wiki/Decimal_floating_point

IEEE Standard 754-2019 is under discussion reviving old ideas like block FLP numbers. (David G. Hough. The IEEE Standard 754: One for the history book. IEEE Computer. Dec 2019).

https://en.wikipedia.org/wiki/Block_floating_point

A study at IBM considers EFloat FLP number format with 4 to 6 additional bits of precision and a wider exponent range than the existing FLP formats. The EFloat format encodes frequent exponent values and signs with Huffman codes to minimize the average exponent field width.

<https://arxiv.org/abs/2102.02705>

Since internal representation varies with different architectures, unformatted I/O is limited in its portability. You can use unformatted I/O to write data out quickly for subsequent input to another Fortran program running on a machine with the same architecture. FLP and fixed point numbers will appear as eight hexadecimal numbers. Formatted I/O in the case of fixed point binary and FLP numbers incurs CPU processing to format the numbers into printable character strings, i.e., unpacked decimal format. There is a loss in accuracy when an inadequate number of fractional digits of FLP numbers are printed, but unformatted numeric files can be more easily ported to other systems using the same number representation.

Endianness

"Endianness" of binary numbers differs across systems:

<https://en.wikipedia.org/wiki/Endianness>

The term comes from Jonathan Swift's book "Gulliver's Travels", where the Lilliputian king required his citizens (the

Little-Endians) to break their eggs on the little-end. The Big-Endians were rebels who broke their eggs on the big end. The term was applied in computer architecture by Danny Cohen at *U. Southern California's Information Science Institute - USC/ISI*.

<https://www.internethalloffame.org/inductees/danny-cohen>

Big Endian Byte Order: The most significant byte (the “big end”) of the data is placed as the byte with the lowest address. The rest of the data is placed in order in the next three bytes in memory. If data is a 32-bit unsigned integer then the *Most Significant Byte - MSB* is the one for the largest powers of two: bits 31-24.

Little Endian Byte Order: The least significant byte (the “little end”) of the data is placed at the byte with the lowest address. The rest of the data is placed in order in the next three bytes in memory. The *Least Significant Byte - LSB* byte is the one for the smallest powers of two: bits 7-0.

Intel Fortran can convert unformatted data from Little-endian-to Big-endian.

Single and double precision (binary) FLP numbers in IBM's System/360 used one or two 32 bit words, respectively. The 32 bit FLP number in System/360 has a sign bit, seven bits for the exponent, which is radix 16, and 24 bit fraction or mantissa. An excess 128 representation is used for the exponent, so the exponents are in the range +127 to -128. This arrangement allows two FLP numbers to be compared as if they are two's complement fixed point binary numbers. Double precision number just have a longer mantissa with $24 + 32 = 56$ bits. System/360 Model 85 introduced in 1968 offered extended-precision 128-bit quadruple-precision FLP numbers.

Universal Numbers - UNUM FLP representation introduced by John Gustafson in (Gustafson, 2015) is specified in Table 1.1.

<https://www.nextplatform.com/2019/07/08/new-approach-could-sink-floating-point-computation/>

TABLE 1.1 POSITS new floating point representation.

	IEEE 754 floating point	Posits: UNUM version 3.0
Bit length	{16,31,64,128}	fixed length but arbitrary
Sign	sign-magnitude ($-0 \neq 0$)	two's complement ($-0 = +0$)
exponent	fixed length (biased binary)	variable length (Golomb-Rice)
Fraction map	$\phi(x) = 1 + x$	$\phi(x) = 1 + x$
Infinities	$\{-\infty, +\infty\}$	$\pm\infty$ (single point at infinity)
NaNs	many (nine quadrillion)	one
Underflow	gradual (subnormals)	gradual (natural)
Overflow	$1/\text{FLP_TRUE_MIN} = \infty$ (oops!)	never (exception: $1/0 = \pm\infty$)
Base	{ 2,10 }	$2^{2^m} \in \{2, 4, 16, 256, \dots\}$

The *American Standard Code for Information Interchange - ASCII* code with 7 bits was later converted to 8 bits.

<https://en.wikipedia.org/wiki/ASCII>

The 8-bit *Extended Binary Coded Decimal Interchange Code - EBCDIC* was introduced by IBM in 1964 in conjunction with System/360 computers, but was adopted by other computer companies.

<https://en.wikipedia.org/wiki/EBCDIC>

Letters in EBCDIC are not assigned consecutive codes, because of the influence of IBM punched cards.

Differences between EBCDIC and ASCII codes are given below.

<http://www.differencebetween.net/technology/communication-technology/difference-between-ebcdic-and-ascii/>Unicode can

have 8, 16, and 32 bit characters and accommodate the alphabets of many languages with 17×2^{16} code points.

<https://en.wikipedia.org/wiki/Unicode>

The *Unicode Transformation Format - UTF8* developed by Rob Pyke and Ken Thompson in 1992 constitutes 91% of the *World Wide Web - WWW*

<https://en.wikipedia.org/wiki/UTF-8>

Quantization which constrains continuous numbers to a discrete set of integers can be used to improve storage efficiency. Simulation programs can produce huge volumes of FLP data, which could be quantized to an 8-bit representation to save

transmission bandwidth and storage space (Lindstrom and Isenburg, 2006). In some applications the exponents in the output stream may be the same or vary in a narrow range, so that it can be specified by a few bits. A new institute has been established to address massive data demands from upgraded *Large Hadron Collider - LHC*, which in 2026 will produce one billion particle collisions per second.

https://www.nsf.gov/news/news_summ.jsp?cntn_id=296456&WT.mc_id=USNSF_51&WT.mc_ev=click

The use of low precision number representations in the context of deep learning is discussed in Chapter 8.

1.4 Basic computer arithmetic

Fast computer arithmetic is important because it has a major effect on computer performance. Addition is used to computer addresses which are usually specified as a base and displacement. An index value incremented in a register is added in addressing arrays.

I took the graduate level computer arithmetic course taught by Algirdas Avizienis in the Spring quarter of 1972. Avizienis had done his PhD research with James Robertson at UIUC, who is best known for his division method. indexAvizienis, Algirdas

<http://www.avizienis.info/index-en.html>

Avizienis had excellent class notes on computer arithmetic, but did not turn them into a textbook. He invented the totally-parallel add/subtract algorithm that reduces the time to add/subtract two operands with any number of digits to the time needed to add two digits. His invention was based on signed-digit arithmetic described in 1726 by John Colson in *Philosophical Transactions of the Royal Society*.

https://en.wikipedia.org/wiki/Signed-digit_representation

Antonin Svoboda before joining the Computer Science Department at UCLA had proposed the *Residue Number System - RNS*, which is based on Chinese Remainder Theorem.

https://en.wikipedia.org/wiki/Chinese_remainder_theorem

Svoboda was a professor in Czechoslovakia, which has since been split into the mainly catholic Check Republic and mainly Russian orthodox Slovakia. An advantage of RNS was parallel computer arithmetic and hence speedup. Svoboda came to US at the start of WW II to return when the war ended and then again in 1964 to stay and join UCLA. Svoboda had disclosed RNS to US scientists at a Moscow conference, so RNS was treated as a US state secret, so when Harvey Garner reinvented RNS as part of his PhD thesis at U. Michigan (Garner, 1959) he was investigated by *Federal Bureau of Investigations - FBI* for disclosing a state secret.

<https://www.computer.org/profiles/antonin-svoboda>

Two's complement arithmetic for n-bit addition is carried out using a *Ripple-Carry Adder - RCA* consisting of n *Full-Adders - FAs* with three inputs x_i , y_i , and c_i , where c_i is the carry from the lower-bit FA and produces two outputs: the sum bit: s_i and the carry bit to the higher-order bit position: c_{i+1}

$$s_i = (a_i \oplus b_i) \oplus c_i, \quad c_{i+1} = a_i \cdot b_i + (a_i \oplus b_i) \cdot c_i, \quad 0 \leq i \leq n-1.$$

In the case of 2's complement arithmetic $c_0 = 0$. The sum is attained only when the carry bit is propagated to the highest position. A single FA can be used as part of a sequential circuit (Kohavi and Jha, 2009) to produce one sum and one carry bit per each clock cycle, but a combinatorial circuit is faster and there are several methods to speed up addition using more gates, such as *Carry-Lookahead Adders - CLAs*.

The add/subtract operation in FLP numbers takes multiple steps: Use the difference between two exponents to shift right the fraction of the number with smaller exponent according to the difference before adding/subtracting the fractions and normalize the resulting number if necessary.

Multiplication of FLP numbers adds the exponents, but subtracts the excess, while division subtracts the exponents but adds back the excess.

Multiplication can be speeded up by using multiplier recoding of binary numbers into $\{-2, -1, 0, +1, +2\}$, which results in more efficient multiplication: a single bit shift and complementation if negative. In effect this is base-4 arithmetic and the number of multiplications is halved.

Carry Save Adders - CSAs achieve speed by deferring the propagation of carry bits to the last stage position. The number of sums produced by a multiplication is reduced from three to two by each adder. A CLA should be used in the last stage with two operands left to add.

Division by multiplication method was implemented by the IBM 360/91 computer, referred to as antibiological, since in biology cells multiply by division, rather than vice-versa (Anderson et al., 1967). To divide the fraction N/D represent the

fractional part of the denominator D as $1 - y$ with $y < 1$. Multiply numerator and denominator by $(1 + y)(1 + y^2)(1 + y^4) \dots$ yields $D = 1 - y^{2^k} \rightarrow 1$ and after k steps and the hence numerator converges to the quotient.

Pipelining is beneficial when dealing with vectors (Kogge, 1981), (Cragon, 1996). The computation is partitioned to equi-delay stages, partial results are moved from stage to stage per clock cycle and this applies to inputting new operands.

The IBM System/360 Model 44 intended for scientific computing allowed variable-length mantissas for FLP arithmetic. It provided a rotary switch on the front panel to set the number of digits in the mantissa (fraction) of double precision FLP numbers to 32, 40, 48, or 56. The setting improved computation speed when higher precision was not required.

https://en.wikipedia.org/wiki/IBM_System/360_Model_44

David Patterson (UC Berkeley, Google) has advocated lower precision FLP arithmetic for deep learning which is discussed in Chapter 8.

<https://spectrum.ieee.org/view-from-the-valley/computing/hardware/david-patterson-says-its-time-for-new-computer-architectures-and-software-languages>

Pentium Intel FDIV bug discovered in 1994 was attributed to missing entries in the lookup table used for Robertson's radix 4 division.

https://en.wikipedia.org/wiki/Pentium_FDIV_bug

Spectre, Meltdown, Foreshadow, Rowhammer, Spoiler are computer bugs discussed in (Markatos et al., 2019). *Common Vulnerabilities & Exposures - CVEs* is a list of entries containing an identification number, a description, and at least one reference for publicly known cybersecurity vulnerabilities. About 110,000 bugs in operating systems are listed in a Mitre Corp.'s CVE repository and 50 are added every day.

<http://cve.mitre.org>

Security flaws in Intel chips are discussed below.

<https://arstechnica.com/information-technology/2020/03/5-years-of-intel-cpus-and-chipsets-have-a-concerning-flaw-thats-unfixable/>

Advanced Micro-Devices - AMD processors released between 2011-19 are vulnerable to two new attacks.

<https://www.zdnet.com/article/amd-processors-from-2011-to-2019-vulnerable-to-two-new-attacks/>

Computer arithmetic constitutes a chapter in most books on computer organization and architecture, but there are several books dedicated to this topic which are listed in the Appendix, which also address the issue of achieving low power consumption in arithmetic.

1.5 Author's experience with IBM computers in 1970s

This section is semi-autobiographical and covers my experience with IBM computers at U. Tehran, IBM Iran, Tehran Regional Electric Company, UCLA, and IBM T. J. Watson Research Center.

1.5.1 IBM computers at Univ. of Tehran and IBM World Trade Corp. in Tehran, Iran

After graduating from U. Tehran with a *Bachelor of Science in Electrical Engineering - BSEE* degree, I was hired at its high voltage lab. The professor in charge was interested in the corona discharge effect, i.e., loss of power through radiation in high voltage *Alternating Current - AC* transmission.

https://en.wikipedia.org/wiki/Corona_discharge

A sample experiment was to measure the voltage required to generate a spark as the distance between two electrified spheres was varied. Corona discharge allows charge to continuously leak off the conductor into the air, leading to power loss in high voltage AC transmission lines.

https://en.wikipedia.org/wiki/Electric_power_transmission#/media/File:Electricity_grid_simple-_North_America.svg

Thomas Alva Edison's early power generation stations in Manhattan, which was based on *Direct Current - DC* had a limited distribution range.

<http://edison.rutgers.edu/power.htm>

<https://edisontechcenter.org/HistElectPowTrans.html>

Longer distance electric transmission became possible with polyphase AC generators invented by Nikola Tesla.

https://en.wikipedia.org/wiki/Nikola_Tesla

The war of currents between Edison and Tesla is discussed below.

https://en.wikipedia.org/wiki/War_of_the_currents

High Voltage Direct Current - HVDC does not have the corona discharge effect and is used for long distance data transmission and to interconnect unsynchronized AC grids. HVDC converters performed the conversion between AC and DC.

https://en.wikipedia.org/wiki/High-voltage_direct_current

I was walking home from U. Tehran when I met my classmate Bijan Zargar, who told me that the IBM office in Tehran offers aptitude tests as part of its hiring process. I took the test that very afternoon and was given an offer soon thereafter after a face-to-face interview. I could start work right away, since as the only son of a father over sixty I was exempt from military service. The paperwork to get the exemption document took two years, however, so I was unable to leave the country to attend IBM courses in Athens and Rome.

As part of my studies at U. Tehran there were a few lectures in Fortran programming, a Fortran programming manual was not available and we had limited access to the IBM 1620 computer at the university to practice programming.

Fortran programs at the time were written on special forms with 80 columns mimicking the number of column on IBM punch cards. The forms were submitted for keypunching, the program was run and the resulting printout was returned to students. The one day turnaround time was too high to learn programming.

The first five columns of 80 column forms were used to specify to five digit numeric labels, which served as targets for GO TO (branch) statements or CONTINUE statements to bracket DO loops, column 6 was used to indicate continuation of lengthy statement on the following line, columns 7-72 were used to specify Fortran statements, and columns 73-80 indicated an abbreviated program name and sequence numbers. Fortran was an early language supporting complex number arithmetic.

https://en.wikipedia.org/wiki/Complex_data_type

At IBM I was asked to assist a programmer at the *National Iranian Oil Company* - NIOC in debugging a Fortran program.

https://en.wikipedia.org/wiki/National_Iranian_Oil_Company

An arithmetic exception was occurring in a nineteen line formula. My proposal was to break down the formula into shorter formulas and to print partial results to pinpoint the operation which caused the error. An alternative was to check the core dump of the program, but this approach required knowledge of System/360 assembler language programming.

In 1970s IBM's WTC was located in a multistorey building on Pahlavi (Vali Asr) Ave, just north of Takht Jamshid Ave, where the US Embassy operated till 11/4/1979. The reader should note that street names in Iran were changed after the 1979 revolution, which led to the overthrow of the Pahlavi Dynasty and the establishment of the Islamic Republic of Iran.

IBM World Trade Corp. offices in Tehran

In early 1970s IBM WTC had several departments, which served the following functions. A similar organization for IBM WTC was followed in other countries:

0. Service Bureau: It was located on the ground level and rented time on its IBM 1410 computer, which was a second generation decimal computer introduced in early 1960s. The computer had six tape drives, no disks, a card reader and a 1403 printer.

https://en.wikipedia.org/wiki/IBM_1400_series

<http://bitsavers.org/pdf/ibm/1410/>

IBM was forced to discontinue its service bureau business in 1973 due to a *Control Data Corp.* - CDC lawsuit discussed later.

<https://www.nytimes.com/1973/01/16/archives/ibm-to-sell-unit-to-control-data-in-settling-suits-16million-will.html>

1. Sales and Systems Engineering - SSE Department: I was hired as a systems engineer into department, which was important in that it generated revenue by leasing computers and peripheral equipment, so that IBM's income did not fluctuate drastically year-to-year. IBM provided the software to run the computers, serviced the equipment and provided education.

NCR was the only company competing with IBM in Iran at that time, but was later joined by UNIVAC which solely sold equipment to Iran's military. The Tehran branch office had detailed documentation o NCR computers to assist salesmen to point out their weaknesses. IBM salesmen were given a substantial bonus if they reached or exceeded their annual quotas and became members of the so-called 100% club. This included trips to the US, even Hawaii.

System engineers assisted salesmen in selecting computers according to the application and the size of the business, but most salesmen offered computers exceeding customer needs to get higher commissions "Nobody gets fired for buying from IBM" was the motto followed by companies for a long time.

<https://www.forbes.com/sites/duenablomstrom/2018/11/30/nobody-gets-fired-for-buying-ibm-but-they-should/>

The SSE department consisted mostly from Iranians with degrees from US or *United Kingdom - UK*, but there were two Parsi Indians, and a Finn. With a degree from U Tehran I was an exception. Most systems engineers had degrees in various fields of engineering, since a computer science degree was offered at very few universities in 1960s.

2. Customer engineering Department: This department dealt with installing, maintaining, and repairing hardware and installing systems software at a later time. Customer engineers were required to have a degree in electrical engineering and according to schoolmate Varouzhan Harikian it involved extensive training in Europe. Computer rooms were heavily air conditioned to dissipate the heat generated by computer circuits and used raised floors for the thick cables supplying power and interconnecting units.

<https://www.pinterest.com/pin/483503709964490460/>

3. Education department: The courses were mainly offered to IBM customers, but occasionally to batches of newly-hired system engineers. More advance courses were held in Europe.

I took two courses at IBM Iran: Sales Application School which discussed the various aspects of hypothetical *Ideal Milk Bucket - IMB* company. There was a cash penalty for saying “IBM” during this class. Concepts such as *Economic Order Quantity - EOQ* for inventory control were introduced in the course. We were asked to make sales calls to managers and taught tricks such as leaving something behind for an excuse to come back.

After leaving IBM to join TREC I took a course on *System/360 Basic Assembler Language - BAL*, which included debugging programs using core dumps, i.e., a printout of program’s main memory content in hexadecimal at the point a job encountering an exception, such as zerodivide. At a later point programming languages such as PL/I specified the statement where an error leading to program termination occurred, so that checking coredumps became unnecessary. I learned PL/I at a brief course offered by a systems engineer who had taken a course in UK and then studied it using the PL/I reference manual.

4. Library held IBM Manuals. For revisions modified and new pages were sent to be inserted into folders, instead of reprinting whole manuals, which were usually several hundred pages long. A full-time librarian kept the manuals up-to-date by inserting modified pages into folders. Software bugs were reported on microfiche.

5. Human Resource - HR and Accounting Departments. Accounting billed customers. Salaries at IBM were confidential and a person from HR guarded the 1403 printer as the salary checks were being printed.

Iran as an oil rich country had the most advanced IBM computers in the Middle East such as the IBM S/360 Model 75 at NIOC. In 1960s and 1970s as second generation 1410 computers were being replaced by 3rd generation System/360s, the 1400s were refurbished and shipped to India.

The oil industry in Iran

National Iranian Oil Consortium - NIOC evolved from Anglo Persian Oil Company with the following members: NIOC, *British Petroleum - BP*, French/Total, Dutch/Shell and US/ExxonMobile.

https://en.wikipedia.org/wiki/National_Iranian_Oil_Company

https://en.wikipedia.org/wiki/Nationalization_of_the_Iranian_oil_industry

The Consortium agreement was signed after Iran’s prime minister Mossadegh who nationalized the oil was overthrown in a 1953 coup-d’etat organized by US and UK. A positively received documentary about this operation with code name Ajax is:

https://en.wikipedia.org/wiki/Coup_53

The Shah who had fled to Rome met there with US and UK intelligence officials, who helped him to regain the throne (Kinzer, 2004). Interestingly Iran was paid 25% of oil profits versus the 50% paid to Saudis.

https://en.wikipedia.org/wiki/The_Consortium_Agreement_of_1954

The consortium’s computer center was at Abadan, which was the site of a large refinery and had an IBM System/360 Model 75, which was the largest IBM computer between Rome and Tokyo.

https://en.wikipedia.org/wiki/Anglo-Persian_Oil_Company

All facilities in Abadan were destroyed during the Iraq/Iran war, which was started by Iraq in September 1980 and

lasted eight years.

https://en.wikipedia.org/wiki/Iran%E2%80%93Iraq_War

The Shah of Iran left Iran again in February 1979 this time fleeing to Egypt. The Islamic Revolution in 1979 led to new oil agreement by the new government. It has been said the Shah was overthrown for refusing to renew the consortium pact.

https://en.wikipedia.org/wiki/Iranian_Revolution

The taking hostage of US embassy staff by Iranian “students” in October 1979, which lasted 444 days led to the deterioration of Iran’s relationship with the US and other countries and led to US and *United Nations* - UN led sanctions.

https://en.wikipedia.org/wiki/Sanctions_against_Iran

Argo is a movie based on the Canadian escapade that took place during the Iran hostage crisis in 1979 and 1980.

[https://en.wikipedia.org/wiki/Argo_\(2012_film\)](https://en.wikipedia.org/wiki/Argo_(2012_film))

US sanctions were dropped after the *Joint Comprehensive Plan of Action* - JCPOA was signed on 7/20/15 and reinstated on 5/8/18 when President Trump withdrew from the plan.

<https://www.armscontrol.org/factsheets/JCPOA-at-a-glance>

With Democrats in the White House after 1/21/2021 Iran is expecting US sanctions to be lifted. On 8/11/21 the JCPOA negotiations were in a standstill.

<https://www.linkedin.com/pulse/jcpoa-still-alive-jorge-morales-pedraza/>

In the meantime it has a 25 year agreement with China for exporting oil.

<https://time.com/5872771/china-iran-deal/>

IBM ceased its operation in Iran after the 1979 revolution and was replaced by several companies.

<http://millennialmainframer.com/2014/03/mainframes-in-iran/iran-tells-oil-consortium-pact-will-not-be-renewed-companies.html>

Upon joining IBM I was given educational material for self-study on the basics of computing followed by Fortran programming. Each module was followed by questions and the reader could proceed only after answering all questions correctly, but was otherwise asked to read clarifying material. After I had completed the Fortran module I was assigned to work with Massih Ettefagh, who had a degree in Civil Engineering from UK.

My first assignment was to write a program for the statistical Chi-square test.

https://en.wikipedia.org/wiki/Chi-squared_test

I used the IBM 1130 computer at the computer center of Aryamehr U. to run my program. I later realized that I could have used the *Statistical Package for Social Sciences* - SPSS.

<https://en.wikipedia.org/wiki/SPSS>

Another assignment was setting up the input data to apply *Linear Programming* - LP to a chicken feed problem using IBM’s LP package. The goal was to determine the least expensive combination of ingredients meeting the minimum nutritional requirements for chickens. I used an early edition of Saul Gass’ “Linear programming: Methods and Application” (Gass, 2010) to familiarize myself with LP. Years later, as part of my PhD thesis I applied LP to determine the maximum throughput of a team-service multiserver queueing system where jobs require multiple servers simultaneously in a multiserver queueing system (Thomasian, 1978, 2014b).

I assisted a French structural company in Tehran

<http://www.soletanchefreyssinet.com/>

in setting up the input to a structural design program for frames developed at MIT, which was distributed by IBM as a type III customer contributed program.

https://en.wikipedia.org/wiki/IBM_Type-III_Library

Stress involved calculations on large matrices and given the small main memory size of the 1130 (16 Kilowords) at Aryamehr U where I ran my programs, temporary results were written and read back from a slow disk, resulting in multi-hour runtimes. The IBM 1130 was the first system from IBM to use removable disk packs, the IBM 2315. It was this mechanism and disk pack design that was taken up by DEC and other computer manufacturers at the time to use in their systems.

The 1130 had a 2315 pack with 4 sectors per track and 200 tracks per surface giving a combined total of 512 kilowords, each word was 16 bits, of storage (1 MB). It also had 3 spare tracks per surface and if a bad sector was detected, the whole track would be mapped out to one of the spares. The head mechanism used a ratchet design.

I developed a set of programs for inventory control and order expediting for an engineering firm with an IBM 1130 computer, but which did not have skilled programmers for the applications at hand. A major problem was that Fortran

did not have provisions for character string manipulation, I used calls from Fortran to assembler subroutines for string manipulation. There are provisions for strings in Fortran standard library.

https://stdlib.fortran-lang.org/page/specs/stdlib_strings.html

Jaffar Namazi as the Systems Engineer assigned to *Tehran's Regional Electric Company - TREC* introduced me to Sahak Sahakian, who was an Assistant Minister of Economy, who later became TREC's director. Iran's law at the time prohibited non-Moslems from holding ministerial positions. Stricter laws probably prevail in the *Islamic Republic of Iran - IRI*.

Sahakian who had an MBA from Stanford U had good ideas in running TREC and increased its revenue considerably. This was a matter of data cleansing, e.g., checking the multiplier associated with the meters, which were wrongly set to the default value one, possibly due to bribes.

https://en.wikipedia.org/wiki/Data_cleansing

Sahakian asked me to produce a table showing the growth of a fund for varying interest rates, something that can be accomplished nowadays with a calculator.

<https://en.wikipedia.org/wiki/Calculator>

<https://bestcalculators.net/best-programmable-calculator-reviews/>

I wrote a Fortran program and ran it on the IBM System/360 Model 30 at Tehran's Statistics Institute, which was a subsidiary of the Ministry of Economics. Results with single precision FLP numbers were not sufficiently accurate and the program ran very slowly with double precision FLP arithmetic. I reran the program on the 1410 decimal computer at IBM's service bureau. Accurate results were printed at full speed.

1.5.2 My experiences with IBM computers at Tehran Regional Electric Company

Sahakian as TREC director offered me a position in his company, probably after getting IBM's approval first. I accepted the offer because of the much higher salary and that this would allow me to save money at a faster rate to pursue my graduate studies.

TREC was planning to replace its IBM 1410 computer with the more modern but more expensive System/360 Model 40, but the acquisition was delayed because TREC programmers were slow in converting its 1410 Autocoder billing programs into PL/I.

TREC's 1410 was damaged irreparably due to a water leak from a broken water pipe, so TREC was forced to take delivery of the Model 40 computer to reinstate its billing operation for its industrial and residential customers.

https://en.wikipedia.org/wiki/IBM_System/360_Model_40

Since PL/I programs for billing were not ready yet the Model 40 was initially used in emulation mode, as if it is a 1410.

https://ethw.org/IBM_System/360

The 7-track feature enables the 2400 tape units in to process tape compatible with other IBM computers that utilize such tape units as the 727, 729, or 7330; which read and write tape in the *Binary Coded Decimal - BCD* format. To implement this feature, a seven-track read/write head is installed in the 2400 tape unit, replacing the nine-track read/write head, and the seven-track compatibility feature is installed in the control unit. The control is then capable of operating with both seven- and nine-track tape units. Reading or writing may be done at densities of 200, 556, or 800 bytes per inch. Odd or even parity checking is provided. Interblock gaps are approximately 0.75 inch. Character density and type of parity checking are selected by the modifier bits in the mode set command byte. The translator of the seven-track compatibility feature is bidirectional; when set on, it translates eight-bit bytes from main storage to six-bit *Binary Coded Decimal* tape characters, and vice-versa. The translator is set on or off by a mode set control command.

http://bitsavers.org/pdf/ibm/2803_2804/A22-6866-4_2400_Tape_Unit_2803_2804_Tape_Controls_Component_Description_Sep68.pdf

With some assistance by Namazi I led the effort in rewriting 1410's Autocoder programs into PL/I. Data cleansing was required because of characters appearing in numeric fields of the MasterFile - M/F. Some of the PL/I programs were poorly written being copied from the original Autocoder programs into PL/I.

The next step was to move the M/F of the 500,000 residential customer records from tape to disk, organized as *Index Sequential Access Method - ISAM* files, which allowed both sequential and random accesses to records. Half a million customer records could not be held on a single disk and ISAM files could not span disk boundaries, so the M/F was split into six ISAM files. The six files had to be declared in PL/I programs dealing with the M/F.

ISAM files at TREC were organized according to the order meters were read with a two month billing cycle with forty days for meter reading. New housing developments in Tehran resulted in a large number of consecutive records to be added

to the M/F. Since ISAM does not do automatic file balancing this resulted in unbalanced trees due to overflow records. The solution was to recreate each of the six ISAM files periodically, by M/F records to tapes before recreating ISAM files.

IBM's *Virtual Sequential Access Method* - VSAM files (Lovelace et al., 2013), which was motivated by main memory based AVL trees (Adelson-Velskii and Landis, 1962), which is self-balancing, were not available to us at the time. VSAM files are better known and B+ trees are used for indexing in databases (Ramakrishnan and Gehrke, 2002). Experimental results show that B+ trees are subject to "waves of misery" after index creation (Glombiewski et al., 2019).

Sahakian had negotiated a \$60M loan with the World Bank to place underground cables in Tehran.

https://en.wikipedia.org/wiki/World_Bank

His earlier request for interest rate calculations may have been to determine how long would it take to pay off the loan. It is not clear to me why Iran as a wealthy oil country needed this loan on which it had to pay interest.

Laying underground cables would have required digging up sidewalks in Tehran, a densely populated city of three million in 1970, which would have resulted in a major inconvenience to its population. Sahakian asked me to estimate the time to complete the project based on time estimates for various activities and precedence relationships. I drew a large but repetitive *Program Evaluation and Review Technique* - PERT diagram

https://en.wikipedia.org/wiki/Program_evaluation_and_review_technique

and used it to specify the input to the *Critical Path Method* - CPM package available for IBM 1130 computers.

https://en.wikipedia.org/wiki/Critical_path_method

Sahakian invited *Prime Minister* - PM Amir-Abbas Hoveida to show him the impressive PERT diagram and the results of the run. He asked me to make additional runs, so I had to go to Arymehr U. to make the runs and missed the presentation. I think Sahakian wanted me to be away for two reasons: firstly, that he has surrounded himself with Armenians and secondly I might have given honest but politically incorrect answers to Hoveida's questions. Hoveida served as PM for 13 years till the overthrow of Pahlavi dynasty in 1979 and was executed that year.

https://en.wikipedia.org/wiki/Amir-Abbas_Hoveida

PERT usually requires multiple followup runs to reflect the actual progress on the project, but none were undertaken.

At TREC I attempted to apply *Load Flow Analysis* - LFA to Tehran's 63 KV electric grid.

https://en.wikipedia.org/wiki/Power-flow_study

The LFA program distributed by IBM was written in assembler at MIT (a type III program distributed by IBM). The program was written to optimize performance by overlapping CPU and I/O processing on IBM 2311 disk drives, which had been replaced at TREC with larger 2314 drives. I was unable to modify the program written with low-level I/O commands for the 2311 to run on 2314s. This is something accomplishable by Googling.

IBM Iran and *Brown Boveri* - BB had a contract to utilize IBM 1800 computers for computerized control of Tehran's 63 KV electric grid in 1971.

https://en.wikipedia.org/wiki/Brown,_Boveri_%26_Cie

Aforementioned Massih Ettefagh was sent to Switzerland to collaborate with BB engineers. He told me that he was unhappy because BB had assigned its junior employees to work on the project. I do not think that the project was completed successfully. An experienced TREC engineer who had visited Armenia told me that Armenian engineers use slide rules to control their electric grid at a no cost. In 1988 BB merged with the Swedish company resulting in *ASEA Brown Boveri* - ABB.

<https://new.abb.com>

An interesting article on this topic is as follows.

<https://www.nytimes.com/2011/10/26/business/energy-environment/behind-the-power-grid-humans-with-high-stakes-jobs.html>

1.5.3 Customer billing at TREC utility

TREC half a million residential customer records in 1970 were initially held on forty tapes according to the forty working day y meter reading and bimonthly billing cycle. This was done for ease of tape handling rather than tape capacity, since $500,000/40 = 12,500$ on the average records per tape would utilize a small fraction of tape capacity.

It should be noted that the population of Tehran tripled from one million in 1950 to 1970 and tripled again to 9.259 million in 2021.

<http://worldpopulationreview.com/world-cities/tehran-population/>

There were two unique keys associated with each M/F record: (1) A routing number combining the meter reading day (1:40) and the region of the city (1:15). (2) A unique account number assigned at the time the customer applied for electric service at one of fifteen branch offices.

To access customer records using the customer number another ISAM file was used to determine customer's routing number, but a direct, hashed (or PL/I's REGIONAL(1)) file would have been more appropriate for this purpose.

When I joined TREC there was no program to add new customers to the M/F. there were thousands of new records to be added and there sorting according to routing showed there are many duplicates. This was because branches had sent the information of new customers several times following their complaints. The program I wrote to add new customer records into the M/F at appropriate positions based on the routing number detected and removed many duplicates and could even generated unique routing numbers when necessary.

Meter readers manually punched the reading onto IBM cards using a handheld device. With a total of 500,000 meters to be read in 40 days required 833 cards for each one of fifteen districts per working day. With ten meter readers per district would require 83 meter readings per day.

The punched cards returned to the computer center were sorted according to routing number and used to update the M/F with the latest readings. The electric bill was calculated based on the difference with the previous reading and therefore only one meter reading was sufficient per record. However to detect incorrect readings, the consumption for the last twelve months (six cycles) was used in a hi-lo test to detect incorrect readings.

A separate file holding electric bills was produced at the same time to be printed. The printed bills were distributed by mail and payments were to be made at certain banks.

More complex billing systems charge less for off-peak electricity consumption, e.g., midnight to 8 am, to encourage users to do their laundry and vacuuming past midnight. A solution would be to have meters that advance slower during off-peak hours.

Automatic Meter Reading - AMR use wireless transmission to the appropriate stations.

https://en.wikipedia.org/wiki/Automatic_meter_reading

A 900 MHz Wireless Mesh Network may be used for this purpose.

https://en.wikipedia.org/wiki/Mesh_networking

The billing and payments could also be done via the Internet. The utility company (Con-Edison in my case) emails a monthly statement. Users may provide a credit card number to the utility company for automatic payments (autopay).

Switching from tape to an ISAM organization allowed random accesses to enable online customer enquiries from the branches by phone to operators at TREC's computer center or using IBM's *Customer Information Control System - CICS*, which would allow direct access to the M/F without human intervention. I left Iran in 1971 and do not know if the latter plan was accomplished

Six IBM 2314 disk drives were required to hold the 500,000 records The 2314 disk drives had 7294 bytes per track.

https://www.ibm.com/ibm/history/exhibits/storage/storage_2314b.html

https://en.wikipedia.org/wiki/History_of_IBM_magnetic_disk_drives

With 275 byte records setting the blocking factor to thirteen allowed two 3575 byte physical blocks per track.

https://en.wikipedia.org/wiki/History_of_IBM_magnetic_disk_drives

The 2314 had 200 cylinders, 20 tracks per cylinder and 7294 bytes per track, hence ≈ 29 MegaByte - MB.

ISAM files could not span disk boundaries, so that six ISAM files were "DECLARED" in the PL/I programs accessing the M/F. The six files corresponded to the following meter reading days (1-7, 8-14, 15-21, 22-28, 29-35, 36-40). Files had to be "OPENED" for access and "CLOSED" after the processing was finished. Higher concurrency could be achieved by opening files in SHARED rather than EXCLUSIVE mode whenever possible.

Blocking factor to improve disk capacity utilization for ECKD drives

Track sizes for different IBM disk drives with the largest size 56,664 Bytes are as follows.

<http://ibmmainframes.com/references/disk.html>

Blocking stores multiple logical blocks as a single physical block. Blocking allows more efficient data storage on disk, because it alleviates the interrecord gaps associated with the ECKD format. Blocking also facilitates prefetching which is beneficial for sequential accesses. As far as records in a block are concerned there is no need to move records into a workspace for processing them and records can be accessed and updated in the buffer by advancing a pointer.

To illustrate the effectiveness of blocking consider logical records that are 100 bytes long and the ECKD layout that adds another one hundred byte gap per logical record, so that disks whose track size is 56,664 bytes can hold $\lfloor 56,664/200 \rfloor = 283$ records. With a blocking factor of ten: $\lfloor (56664/1100) \rfloor \times 10 = 510$ records. A 100 blocking

factor yields 100: $\lfloor 56664/10100 \rfloor \times 100 = 500$ records, and a blocking factor 79: $\lfloor 56664/8000 \rfloor \times 79 = 553$ records. Putting all records in a single block not exceeding the track size thus maximizes storage efficiency. A blocking factor of 565 results in a 56600 byte physical block size, which yields the maximum number of records. IBM used to provide tables for different disk drives to find good layouts for different record sizes. Note that blocking is also beneficial for magnetic tape and reduces the number of inter-records gaps, but more importantly the number of commands issued to read the tape.

It should be emphasized that ECKD disk drives are not manufactured anymore, but are virtualized in IBM's z-series mainframes running z/OS, which is an MVS descendent.

Double-buffering in PL/I allows prefetching physical blocks into one buffer, while the records in the other buffer are being processed by the CPU by advancing a pointer. This is an instance of CPU/IO overlap and the time it takes to process the file is the number of physical blocks to be read times $\max(T_{CPU}, T_{I/O})$.

Using pointers to access and update successive records obviates the need to copy them which is particularly time-consuming when slow core memories were in use.

1.5.4 My experience with IBM computers at UCLA

UCLA in early 1970s had two IBM 360/91 computers. One at the *Brain Research Institute - BRI* and the other at Math Sciences Building attached to the Engineering I Building, which was available to engineering students and also ran the *University Records System Access - URSA* timesharing system. Thelma Estrin (rip) helped organize BRI's Data Processing Lab. In 1980 she joined her husband Gerald Estrin (rip), who was possibly the chair of the CS department at the time, and son in law (rip). Their daughter Deborah Estrin (now at Cornell Tech) who is also a computer scientist joined the department from *U. Southern Calif. - USC* when her husband died. The Engineering VI building was inaugurated in 2018.

<https://samueli.ucla.edu/engineering-vi-grand-opening-new-building-heralds-new-era-of-growth-for-ucla-samueli/>

The 360/91 in the Math Science Building also ran a timesharing system for APL. IBM electric typewriters with keyboards extended with Greek letters were used for APL programming. An online tutorial allowed a person to learn APL in a couple of hours. I later learned that APL handles recursion, which facilitates the formulation of problems such as integer partitioning, which is the number of ways of writing integers as a sum, e.g., $3 = 1+1+1 = 1+2$.

[https://en.wikipedia.org/wiki/Partition_\(number_theory\)](https://en.wikipedia.org/wiki/Partition_(number_theory))

When I started my graduate studies at UCLA in September 1971 I was surprised to find out that IBM punch cards are still used for submitting programs and data. A dozen keypunch machines were available to students to prepare their programs and there was a fast card reader in an adjacent room to submit jobs. The printer in the computer room fed its printouts through a horizontal opening slot in the wall to the room with the card-reader to make results immediately available to users who had submitted jobs. Short programs were run almost instantaneously, while larger programs were run in roughly *Shortest Job First - SJF* order based on an estimated CPU time and memory usage provided by the user.

As part of my PhD thesis I used the *Recursive Queue Analyzer - RQA* developed at U. Michigan for solving *Continuous Time Markov Chains - CTMCs* with a finite number of states in one dimension and an infinite number of states indicating the number of customers. The CTMC had a repeating pattern in the second dimension, which allowed the truncated CTMC to be specified compactly (Wallace and Rosenberg, 1966). Power iteration was used to compute state probabilities (Bolch et al., 2006), (Stewart, 1989). I applied RQA to analyze the team-service queueing system, where customers request a varying number of servers in a multiserver queueing system.

The computer science program at UCLA was quite selective in that undergrad students had to pass two programming courses: Eng. 10 and 20, before being allowed to take upper division. The programming assignments in the latter course were quite challenging and kept students up all night.

Robert Braden managed programming of the math department's 360/91 (1968-74). In his interview at U. Minnesota he mentions the superiority of IBM Systems/360 Model 91 over the CDC 6600 (larger memory size), but criticizes the primitive code of its operating system (OS/MVT).

<https://conservancy.umn.edu/handle/11299/172263>

He mentioned computer's slow OS/MVT scheduler at UCLA and that students at his undergrad class were able to improve it. "We had this supercomputer, a 360/91, which is capable of 10 million operations a second but it had the ponderous IBM operating system, which meant that transition from job to job took 30 seconds." Braden had students in a systems programming course develop Quickrun scheduler. It seems that the slow scheduling was due to the *Job Entry Subsystem - JES*.

https://en.wikipedia.org/wiki/Job_Entry_Subsystem_2/3

The thick bundles of wires interconnecting hardware units of the System/360 Model/91's *Central Processing Unit - CPU* developed shorts over the years making repair impossible.

1.6 IBM's System 360 and its successors

The line of IBM mainframes from 1952 to present is as follows: IBM 1400 series, 700/7000 series, S/360260.174, S/370, S-370-XA (extended architecture), 308X, S/3090, S/390, zSeries 900, 800, 990, and 890, z/architecture - System z9, z10, zEnterprise System (z196, zEC12, z13, z14, z15, z16).

https://en.wikipedia.org/wiki/IBM_System/360

https://en.wikipedia.org/wiki/IBM_Z#IBM_zSeries_family

The latest in these series: the z15 was announced in 2019.

<https://www.ibm.com/us-en/marketplace/z15>

Telum will be the central processor chip for the next generation IBM Z and LinuxONE systems. it can be used in preventing fraud in real-time delivering in-txn inference in real time.

https://www.ibm.com/blogs/systems/ibm-telum-processor-the-next-gen-microprocessor-for-ibm-z-and-ibm-linuxone/?adoper=160926_1_PB

IBM Telum also provides improvements for performance, security, and availability.

The microprocessor contains 8 processor cores, clocked at over 5 GHz, with each core supported by a redesigned 32 MB private level-2 cache. The level-2 caches interact to form a 256 MB virtual Level-3 and 2 GB Level-4 cache. Along with improvements to the processor core itself, the 1.5x growth of cache per core over the z15 generation is designed to enable a significant increase in both per-thread performance and total capacity IBM can deliver in the next generation IBM Z system.

Telum also features significant innovation in security, with transparent encryption of main memory. Telum's Secure Execution improvements are designed to provide increased performance and usability for Hyper Protected Virtual Servers and trusted execution environments, making Telum an optimal choice for processing sensitive data in Hybrid Cloud architectures.

The predecessor IBM z15 chip was designed to enable industry-leading seven nines availability for IBM Z and LinuxONE systems. Telum is engineered to further improve upon availability with key innovations including a redesigned 8-channel memory interface capable of tolerating complete channel or DIMM failures and designed to transparently recover data without impact to response time. The improvement in reliability is probably based on the *Redundant array Of Independent Memories - RAIM* paradigm (Lastras et al., 2011). The z16 announced on 8/123/21 is discussed here.

<https://www.nextplatform.com/2021/08/23/ibm-bets-big-on-native-inference-with-big-iron/>

A historical discussion how building the System/360 mainframe nearly destroyed IBM, but proved to be the most successful product launch ever and changed the course of computing is provided in (Cortada, 2019a), which is previewed in (Cortada, 2019b).

IBM S/360 introduced by IBM in 1964 had six initial Models 30, 40, 50, 60, 65, and 70.

https://en.wikipedia.org/wiki/IBM_System/360

The Model 20 introduced later could be used as a tape-to-print device for spooling. Northrop-Page was a US company which was building a microwave network in Iran in 1970s, rented the night shift for the IBM 360/40 at TREC to produce tapes, which were then taken for printing to its premises with a Model 20 for spooling. Northrop had purchased Page Corp. in 1959.

https://en.wikipedia.org/wiki/Northrop_Corporation

https://en.wikipedia.org/wiki/Page_Communications_Engineers

Northrop's dealings with Iran in 1970s are discussed below.

<https://www.nytimes.com/1976/02/23/archives/northrop-rebate-reported-in-iran-as-an-atonement-2-million-is-said.html>

A spooling capability was provided with an OS/360 MFT partition (at highest memory addresses) dedicated to spooling.

<https://en.wikipedia.org/wiki/Spooling>

IBM S/360 and S/370 computers covering the 1960-75 period are described in (Pugh et al., 2003), which states that "no new product offering had a greater impact on the computer industry than the IBM System/360". The key to the system's success was compatibility among its many models, which allowed a business to move its applications to higher 360 and follow-up models as the business grew. IBM achieved its highest valuation in 1967. The S/360 was so successful that IBM's

own effort to replace it ten years later with the *Future Systems - FS* project failed.

https://en.wikipedia.org/wiki/IBM_Future_Systems_project

A timeline of IBM's *Advanced Computer Systems - ACS* to replace the S/360 is provided by Mark Smotherman at Clemson U.

https://people.cs.clemson.edu/~mark/acs_timeline.html

Complementary Metal Oxide Semiconductors - CMOS technology,

<https://en.wikipedia.org/wiki/CMOS>

used for manufacturing microprocessor chips for personal computers were much slower than bipolar *Emitter Coupled Logic - ECL* technology used in mainframes,

but a disadvantage of ECL was high power consumption and the need for cooling.

https://en.wikipedia.org/wiki/Emitter-coupled_logic

This situation changed over time in that both mainframes and PCs started relying on CMOS.

<https://www.elprocus.com/difference-between-nmos-cmos-technology/>

<http://www.rfwireless-world.com/Terminology/Difference-between-TTL-ECL-CMOS.html>

IBM converted its mainframe technology from ECL to CMOS at the turn of 20th century.

https://www.chronline.com/news/ibm_numbers_bipolars_days_with_g5_cmos_mainframes/

The importance of power consumption in computer systems is discussed in Chapter 7.

Due to its success the IBM S/360 led to *Plug Compatible Mainframes - PCMs*

https://en.wikipedia.org/wiki/Plug_compatible.

Amdahl Corp. was the first in this category founded by IBM S/360 architect Gene Amdahl.

https://en.wikipedia.org/wiki/Amdahl_Corporation

Fujitsu took over Amdahl in 1967. Hitachi had a complicated role in mainframes.

https://en.wikipedia.org/wiki/Hitachi_Data_Systems

Comparex was founded in Leipzig in 1990 and taken over by SoftwareOne in February 2019.

<https://en.wikipedia.org/wiki/Comparex>

<https://en.wikipedia.org/wiki/SoftwareOne>

IBM z-series computers are still in production. The need for Cobol running on mainframe computers was on the rise even before the COVID-19 crisis.

<https://spectrum.ieee.org/tech-talk/computing/software/mainframes-programming-language-cobol-news-coronavirus>

The latest Fujitsu mainframe in 2018 had a 20% improved performance.

<https://www.fujitsu.com/global/about/resources/news/press-releases/2018/0417-01.html>

Robert X. Cringely authored "The Decline and Fall of IBM: End of an American Icon" (Cringely, 2014) and made the following presentation in March 2020.

<https://www.youtube.com/watch?v=l4b1D1vWRnc>

Cringely's predictions following the stepping down of Ginni Rometti as CEO, which led to her succession by Arvind Krishna are quite dire.

<https://betanews.com/2020/02/03/my-first-prediction-for-2020-ibm/>

Red-Hat was acquired by IBM and its former CEO Jim Whitehurst was appointed IBM president and was supposed to succeed Krishna in a few years, but left IBM in July 2021 looking for a new chance to run something, since probably Arvind refused to step down.

1 <https://www.cringely.com/>

According to the following announcement on 10/8/2020.

<https://www.reuters.com/article/ibm-divestiture-idUSKBN26T1V1>

IBM will list its IT infrastructure services unit, which provides technical support for 4,600 clients in 115 countries and has a backlog of \$60 billion, as a separate company (NewCo?) by the end of 2021. The new company will have 90,000 employees and its leadership structure will be decided in a few months, *Chief Financial Officer - CFO* James Kavanaugh. IBM which currently has more than 352,000 workers, expects \$5 billion in expenses related to the separation and operational changes.

In spite of the IBM's decline there are the following positive aspects, which are quoted from a blog provided by Precisely, formerly Syncsort.

<https://www.precisely.com/blog/mainframe/9-mainframe-statistics>

1. Mainframes, more specifically IBM Z systems, are used by 71% of Fortune 500 companies.
2. Mainframes handle 90% of all credit card txns.

3. Mainframes handle 68% percent of the world's production IT workloads, but account for only 6% of IT costs.
4. IBM's z15 system can support up to 40 TB of memory (TB = 10^{12} bytes).
5. A z13 mainframe can survive an 8.0 Richter scale earthquake, as verified by a test complete with a video.
6. Mainframes handle 30 billion business txns each day, i.e., credit card payments, stock trades, and other business-critical txns.
7. Revenue from z-System sales increased 62% year-over-year with the 2019 release of the z15 system.
8. As of 2019, 96 of the world's largest 100 banks, 9 out of 10 of the world's largest insurance companies, and 23 out of the 25 largest retailers in the US use mainframes.
9. The median mainframe programmer salary is about \$76,282 according to PayScale, which is \$13,298 higher than the median salary for computer programmers in general. Programmers at Facebook average \$150K.

1.6.1 US lawsuits against IBM and AT&T

IBM computers were initially leased and the hardware and software and maintenance was bundled, i.e., the customer paid the high price for hardware and the software, service (maintenance) and education came for free. Provided with a free PL/I compiler users would be less motivated to buy a COBOL compiler, although the IBM S/360 at TREC came with a COBOL compiler as well. IBM was forced to unbundle software in 1969 following US governments antitrust lawsuit (Pugh, 2002).

https://www.wikiwand.com/en/History_of_IBM

https://en.wikipedia.org/wiki/History_of_IBM#1968-1984_Multiple_Government_and_Private_Antitrust_Complaints

In 1975 an antitrust lawsuit US vs IBM, was filed but withdrawn in 1982 after generating 30 million pages of paperwork.

<http://www.historyofinformation.com/detail.php?id=923>

American Telegraph & Telephone - AT&T was formed in late 19th century to become a telephone monopoly known as Ma Bell. US vs AT&T anti-monopoly lawsuit was successful and led to its breakup in 1984 into seven *Regional Bell Operating Companies* - RBOCs known as Baby Bells.

https://en.wikipedia.org/wiki/United_States_v._AT&T

(1) Ameritech, acquired by SBC in 1999, now part of AT&T Inc.

(2) Bell Atlantic (now Verizon Communications), which acquired GTE in 2000.

(3) BellSouth, acquired by AT&T Inc. in 2006

(4) NYNEX, acquired by Bell Atlantic in 1996, now part of Verizon Communications.

(5) Pacific Telesis, acquired by SBC in 1997, now part of AT&T Inc.

(6) *Southwestern Bell Corp.* - SBC acquired AT&T Corp. in 2005 taking its name.

https://en.wikipedia.org/wiki/AT&T_Corporation

(7) US West, acquired by Qwest in 2000, which in turn was acquired by CenturyLink in 2011.

Bell Labs was established by Alexander Graham Bell in 1925.

https://en.wikipedia.org/wiki/Bell_Labs

Initially its ownership was split between AT&T and Western Electric. In 1996 AT&T spun-off Bell Labs with most of its equipment manufacturing business to form Lucent. A small number of researchers were retained to create AT&T Labs, which was headed by Behzad Nadji till January 2006 at which point AT&T was acquired by SBC. In 2006 Lucent signed a merger with Alcatel and stopped doing basic research in 2008. In 2015 Nokia acquired Alcatel Lucent for \$16.6 Billion. Former manager Bell Labs manager, Narain Gehani who later joined NJIT wrote a book in praise of Bell Labs (Gerhani, 2003), has received unflattering reviews.

<https://www.amazon.com/gp/customer-reviews/R1F1W5QJOGNV46>

<https://ubiquity.acm.org/article.cfm?id=764016>

<https://www2.dmst.aueb.gr/dds/pubs/Breview/2004-CR-Bell/html/review.html>

1.6.2 Amdahl Corp. and plug compatible computers

Amdahl Corp. was founded by the main System/360 architect Gene Amdahl assisted by Japanese Fujitsu Corp. in 1970 was the first to introduce *IVM S/350 Plug Compatible Machines* - PCMs.

https://en.wikipedia.org/wiki/Plug_compatible

https://wikipedia.org/wiki/Gene_Amdahl

https://en.wikipedia.org/wiki/Amdahl_Corporation

The success of the company was partially attributable to an antitrust ruling that allowed computer users other than IBM to buy its software at reasonable prices. Until 1969 IBM software came bundled with its hardware, i.e., the software was provided for free with the purchase of hardware. In 1969 IBM was forced to unbundle its software from its hardware (Grad, 2002), allowing buyers of Amdahl computers to have access to IBM S/360 software, which made Amdahl computers viable. At its peak Amdahl had 24% of the mainframe market. Amdahl staff included Alexandre Brandwajne, who later joined *U. Calif. at Santa Cruz - UCSC*, Makoto Kobayashi, a student of Domenico Ferrari at *UC Berkeley - UCB* who ran simulations to optimize processor designs, John Colgrove who with Dieter Gawlick (now at Oracle) developed a RAID5 disk array used in performance studies by UCB researchers (Chen et al., 1990). Colgrove later founded Purestorage.

<https://www.purestorage.com>

Amdahl customers bought their peripherals from IBM. Amdahl attempted alliances with Memorex and later with StorageTEK, but the latter was vetoed by Fujitsu.

<http://www.1000bit.it/ad/bro/amdahl/Amdahl-First-25-Years.pdf>

The decline of IBM mainframe sales in 1990s affected Amdahl revenue and profitability and it was taken over by Fujitsu, which continued producing mainframes for the Japanese market.

<https://en.wikipedia.org/wiki/Fujitsu>

In 2018 Fujitsu was ranked #4 as an *Information Technology - IT* services, but is no longer in the top ten: 1. Microsoft, 2. IBM, 3. Accenture, 4. Oracle, 5. SAP, 6. HP, 7. Tata Consultancy, 8. Cognizant, 9. Capgemini, 10. Infosys.

<https://alertify.eu/top-10-information-technology-it-companies/>

I attended *Int'l Symp. on Computer Architecture - ISCA'76* at St Petersburg, FL to present (Thomasian and Avizienis, 1976). Gene Amdahl presented a keynote speech at conference dealt with the importance of maintaining architectural compatibility. IBM had dealt with this issue by providing the 360 computer family with a unified ISA and software and microcode for simulation (via software) and emulation (via microcode) across earlier computer families.

<http://ed-thelen.org/comp-hist/emulation.html>

1.6.3 Radio Corporation of America - RCA

RCA was founded as a subsidiary of *General Electric - GE* in 1919, but in 1932 became an independent company as part of an anti-trust lawsuit. RCA was re-acquired by GE in December 1985.

<https://en.wikipedia.org/wiki/RCA>

RCA's Spectra computers were S/360 compatible, except for privileged instructions.

https://en.wikipedia.org/wiki/RCA_Spectra_70

1.6.4 Electronic Data Systems - EDS and Perot Systems

Ross Perot, a very successful IBM salesman, in 1962 founded *Electronic Data Systems - EDS*, an IT services company which was acquired by HP for \$13.9 Billion in 2008.

https://en.wikipedia.org/wiki/Electronic_Data_Systems

When HP split into HP and *HP Enterprise - HPE* in 2015 the low-margin EDS was merged with *Computer Sciences Corp. - CSC* to create DXC Technology.

https://en.wikipedia.org/wiki/Computer_Sciences_Corporation

<https://www.dxc.com/>

<https://www.zdnet.com/article/dxc-technology-to-emerge-from-csc-and-hpe-enterprise-services-merger/>

It was announced on 12/1/2020 that HPE is moving to Houston, TX.

Ross Perot founded Perot Systems in 1988, which was another IT services company. It was acquired by Dell for \$3.9Bn in 2009 as Dell Services.

https://en.wikipedia.org/wiki/Perot_Systems

Dell sold Perot Systems to *Nippon Telephon & Telegraph - NTT* in 2016 at a loss of \$800M to buy EMC.

<https://arstechnica.com/information-technology/2016/03/dell-to-sell-perot-systems-unit-to-ntt-data-at-loss-of-800-million/>
<https://group.ntt/en/> <https://hello.global.ntt/>

1.7 The IBM S/360 computer family

Computers in the same family have the same *Instruction Set Architecture - ISA*, the same registers and instruction set available to assembly language programmers and compilers.

https://en.wikipedia.org/wiki/Instruction_set_architecture

ISAs of several important computers are provided below.

https://en.wikipedia.org/wiki/Comparison_of_instruction_set_architectures

Computers with the same ISA can share software such as the OS, compilers, assemblers, link editors, and utility programs, such as sorting.

Instruction sets evolved with time. In the case of S/360 a loop using 256 byte string transfers with a move character - MVC instructions was extended with *Move Character Long* - *MVCL* instruction which could transfer 2^{24} bytes in S/360 (IBM Corp., 1967). Unlike other instruction MVCL can be interrupted.

Operating systems for the S/360 had different main memory requirements providing higher capabilities with larger main memory sizes. The S/360 Model 30 and Model 40 could run IBM S/360 *Disk Operating System* - *DOS*, which provided *Simultaneous Peripheral Operations OnLine* - *SPOOLing*, i.e., printing from tape or disk files, but otherwise limited multiprogramming. A Model 40 with 64 KB memory could additionally run OS/MFT (multiprogramming with a fixed number of tasks) with three partitions, where the partition in highest memory addresses would be dedicated to spooling, a second partition could be used for running application programs and a third partition for program development.

The CPUs in the S/360 family varied widely in their organization. Models 30, 40, 50, and 65 were differed from the viewpoint of their datapath widths: 8, 16, 32, 64 bits, control store size, clock rate, which is the inverse of *Read-Only-Memory* - *ROM* cycle time, memory capacity, *Millions of Instructions Per Second* - *MIPS* for commercial and scientific applications, and their prices in 1964 in (Hennessy and Patterson, 2019). Microprograms were held in ROM whose cycle time affected the MIPS.

The S/360 had sixteen 32-bit *General Purpose Registers* - *GPRs* to hold binary numbers and eight (resp. four) registers for single (resp. double) precision *FLloating Point* - *FLP* numbers. Temporary results were held in registers, but CPU cycles were saved in S/360 Model 91 by forwarding FLP results directly from one arithmetic unit to the input buffer of another unit via the *Common Data Bus* - *CDB*, thus bypassing registers specified by the compiler in a form of decompiling, e.g., in the case of $X = A \times B + C$ temporary result $A \times B$ was sent to the input buffer of the ADDER, rather than an FLP register. The IBM System/360 Model 91 supported out-of-order instruction execution using the Tomasulo algorithm, which used register renaming (Shen and Lipasti, 2013).

https://en.wikipedia.org/wiki/Register_renaming

Instruction scheduling in modern processors is discussed in (Shen and Lipasti, 2013).

To address operands in main memory instructions used four bits to specify the GPR holding the base address and 12 bits to specify the displacement. In S/360 only 24 bits, out of 32 bit words were used for addressing, i.e., 2^{24} bytes or 4 MB for maximum memory size. Main memory sizes were limited in 1964 due to the high cost of magnetic core memories.

In the Model 20 low-end S/360 computers the sixteen 32 bit GPRs and four 64 bit FLP registers were mapped to reserved low-end main memory locations. This allowed shorter instructions with bits to specify a register, rather than 20 bits to specify addresses in main memory.

IBM S/360 computers were able to run programs developed for IBM's earlier generation computers, such as the IBM 1400 series decimal computer for commercial applications.

https://en.wikipedia.org/wiki/IBM_1400_series

The 1401 appeared in 1959 followed by the 1410 in 1960.

https://en.wikipedia.org/wiki/IBM_1401.

1400 computers used *Binary Coded Decimal* - *BCD* characters with six bits ($B, A, 8, 4, 2, 1$), preceded by C (odd parity check) and M (word mark), i.e. $C B A 8 4 2 1 M$. Word marks allow accessing a field at either end.

IBM 1400 computers used the low-level autocoder language, but also Fortran and COBOL.

<https://en.wikipedia.org/wiki/Autocoder>

Autocoder code could be inserted in Fortran programs to speedup inner loops. 1400 computers used magnetic tapes for compiling, sorting, as well as storing data files.

There are two working 1410s at the Computer History Museum in Mountain View, CA.

<https://www.computerhistory.org/>

The video of the 1401 compiling a Fortran program using 63 passes was sent to me by Dr Ken Shirriff,

<https://www.youtube.com/watch?v=uFQ3sajIdaM>

a former student of John Ousterhout at then at UC Berkeley, see e.g., (Shirriff and Ousterhout, 1994). Ken recently fixed a 1403 printer at museum.

<https://hackaday.com/2018/09/20/fixing-an-ibm-1401-computer-to-get-it-printing-again/>

Ken's blog is as follows.

<http://www.righto.com/>

Maurice Wilkes at Cambridge U. in *United Kingdom - UK* designed the *Electronic Delay Storage Automatic Calculator - EDSAC* in 1945.

https://en.wikipedia.org/wiki/Maurice_Wilkes

In 1951 Wilkes developed microprogramming to control the CPU via “microprograms”, which were held in a ROM - Read-Only Memory. More complex instructions require the processing of a sequence of microinstructions specifying *Register Transfer - RT* and *Arithmetic Logic Unit - ALU* operations. Microprogramming is based on a very simple computer inside the computer, which fetches microcode to process operations specified by an opcode (Agrawala and Rauscher, 2014). Microprograms have been classified as horizontal vs vertical, with horizontal microprograms invoking several operations in parallel (Habib, 1988), (Agrawala and Rauscher, 2014).

FLP arithmetic was implemented via subroutines in S/360 Model 30 and microprogramming in the Model 40. Microprogramming became less important with increased complexity of computer hardware and the simpler instruction set of RISC computers

Low-end S/360s such as IBM S/360 Model 30 and 40 used simulation via software routines and emulation via microcode of 1400 series computers, respectively.

<http://ed-thelen.org/comp-hist/emulation.html>

High-end 360s emulated IBM 7000 series computers for scientific computing. The IBM 7090 was a 2nd generation 36-bit computer that was a transistorized version of the first generation 709 vacuum tube computer.

https://en.wikipedia.org/wiki/IBM_7090

The IBM S/360 Model 195 had the same organization as the S/360 360/91 introduced in 1967, but with a more advanced technology to replace it in 1969.

https://en.wikipedia.org/wiki/IBM_System/360_Model_195

There was also a 370/195, but it did not have the virtual memory feature of other 370s.

Sequential program execution is disrupted by conditional or unconditional branches, which result in a slowdown in execution, since instructions other than those already in the instruction lookahead registers in the Model 91 or (instruction) cache may be required to continue the execution.

Branch prediction is discussed in most books on computer architecture, e.g., (Shen and Lipasti, 2013), (Hennessy and Patterson, 2017), but also a monograph dedicated to this topic (Cragon, 1991). Speculative execution is possible with adequate hardware resources and both sides of a conditional branch may be taken and even executed, but only results on the taken side need be retained.

The IBM S/360 ISA was published in the *Principles of Operation - PrincOps* manual in 1964 did not specify the instruction to assist sorting (IBM Corp., 1967), but this instruction was published later on page A53 in Appendix A.

<http://publibfi.boulder.ibm.com/epubs/pdf/dz9zr008.pdf>

Syncsort, renamed Precisely, MFX sort program outperformed IBM’s possibly utilizing this hidden instruction.

[https://en.wikipedia.org/wiki/Precisely_\(company\)](https://en.wikipedia.org/wiki/Precisely_(company))

Nowadays Precisely provides software for much more than sorting, i.e., big data processing.

<https://www.precisely.com/>

In 1965 IBM announced the 2314 disk drives, which used the same technology as 2311s but had a higher capacity.

https://www-03.ibm.com/ibm/history/exhibits/storage/storage_2314.html

Memorex founded in 1961 produced disk drives and tapes. As early as 1966 it shipped a disk pack and in 1968 IBM compatible disk drive. Memorex which also produced consumer products ran an ad in 1982: “Is it live, or is it Memorex?”. It was acquired by Burroughs Corp. in 1981 and became defunct in 1996.

<https://en.wikipedia.org/wiki/Memorex>

Storage Technology Corp. - StorageTEK was unsuccessful in producing an IBM S/360 compatible mainframe, but instead produced tape libraries.

https://en.wikipedia.org/wiki/Storage_Technology_Corporation

StorageTEK later developed the Iceberg disk array could emulate IBM’s ECKD DASD (Friedman, 1995), a topic elaborated in Chapter 5. StorageTEK also developed *Solid State Drives - SSDs*,

https://en.wikipedia.org/wiki/Solid-state_drive.

Early SSDs were based on *Dynamic Random Access Memory - DRAM* backed up by *Uninterruptible Power Supply - UPS*.

https://en.wikipedia.org/wiki/Uninterruptible_power_supply

DRAM SSDs have been replaced with flash based SSDs due to the drop in flash SSD prices and larger capacity flash memories Table 1.2, which are discussed in more detail in Chapter 2.

TABLE 1.2 Improvement of SSD characteristics over time, *Nimbus data DC100, **Samsung PM1643.

Year	1991	2018	Improvement
Capacity	20 MB	100TB*	5 million
Price	\$50/MB	\$0.372/GB**	134,408

The IBM S/360 had special I/O commands called *EXecute Channel Program - EXCP* that can only be executed in supervisor mode by OS routines. There were three types of I/O channels: (1) multiplexor (for slow devices), (2) selector, and (3) block multiplexer for multiplexing fast devices.

https://en.wikipedia.org/wiki/IBM_System/360

The following Redbook serves as z15 introduction and its Section 2.4 describes the I/O System Structure (Lascu et al., 2020). A more ip-to-date reference is IBM Redbook - IBM z15 (8562) Technical Guide. SG24-8852-00 (August 2020)

<https://www.redbooks.ibm.com/redbooks/pdfs/sg248852.pdf>

A program that issues a read request to disk due to a page fault or to access to a file or database page is blocked awaiting its completion. The *Program Status Word - PSW* (the program counter, condition codes, etc.) and GPRs and FLP registers for the blocked program are saved to allow its resumption when the I/O is complete. When a program is blocked, the job scheduler determines the next job to run. When the I/O is completed an I/O interrupt occurs, which makes the blocked job eligible to run. The job scheduler determines which blocked job should be restarted. Interrupts occur at the completion of instruction execution in S/360, but this is not the case for the MVCL instruction, as noted earlier.

1.8 Operating systems associated with IBM mainframes

A detailed description of S/360 operating systems is given at:

https://en.wikipedia.org/wiki/OS/360_and_successors

Additional OSs have been developed in conjunction with S/360 successor z-systems, with the z perhaps indicating the end of the line for S/360 computers.

<https://www.ibm.com/it-infrastructure/z/os>

z/OS basics are discussed in an IBM Redbook (Ebbers et al., 2011).

A timeline of operating systems which includes those from IBM is provided here.

https://en.wikipedia.org/wiki/Timeline_of_operating_systems

z/OS is a successor to *Multiple Virtual Storage - MVS* OS.

<https://en.wikipedia.org/wiki/MVS>

OS/360 MVS/MFT and MVS/MVT (multiprogramming with a fixed/variable number of tasks) were two versions of MVS. OS/360 MFT ran on TREC's S/360 Model 40 with 64 KB memory, while OS/MVT ran at UCLA's Model/91 with 2 GB main memory.

z/Linux is a platform for cloud and analytics. A single mainframe can run thousands of virtual Linux servers concurrently.

z/VM hypervisor can run thousands of Linux on z virtual machines on one mainframe and host z/OS, z/VSE and z/TPF.

z/VSE is a descendent of DOS/360 for lower end S/360 computers.

z/TPF Transaction Processing Facility is a special-purpose OS to process very high volumes of short *transactions - txns*, which is discussed further in Chapter 9.

In June 1970 IBM announced virtual memory support for S/360 followup S/370 computers

https://en.wikipedia.org/wiki/IBM_System/370

which led Burroughs Corp. to announce "Today IBM announces yesterday".

There were protection keys associated with 2K main memory blocks in early S/360s. Program utilizing those blocks were assigned that key (Madnick and Donovan, 1974). Accessing a main memory area with a different key resulted in a protection exception, which was different from an addressing exception. The advantage of Virtual Memory is that each user process has its own virtual space and hence page table, making it impossible to modify the address space of another program. This topic is discussed further in the context of mapping virtual addresses to main memory.

IBM in 1972 announced the *Virtual Machine - VM/370* system (Parmelee et al., 1972).

[https://en.wikipedia.org/wiki/VM_\(operating_system\)](https://en.wikipedia.org/wiki/VM_(operating_system))

VM was usually coupled with *Conversational Monitor System - CMS* and could also run other operating systems such as MVS and even itself.

According to (Popek and Goldberg, 1973, 1974) VM was implemented in a limited number of third generation computer systems, e.g. CP-67 on the IBM 360/67.

https://en.wikipedia.org/wiki/Popek_and_Goldberg_virtualization_requirements

Empirical studies have shown that not all computers such as the PDP-10 can support VMs. A model of a third-generation-like computer system is developed and formal techniques are used to derive precise sufficient conditions to test whether such architectures can support VMs. Hypervisors or VMs are discussed in (Smith and Nair, 2005).

<https://en.wikipedia.org/wiki/Hypervisor>

IBM 3090 announced in 1985 was a follow-up to the IBM, 370, 370/XA, ESA/370, 3033 and 308X according to N. S. Prasad. IBM Mainframe Architecture and Design, McGraw-Hill, 1989.

<https://archive.org/details/ibmmainframesarc00pras>

Wikipedia has comprehensive web pages on IBM computers.

https://en.wikipedia.org/wiki/IBM_System/360/newline https://en.wikipedia.org/wiki/IBM_System/370

https://en.wikipedia.org/wiki/IBM_System/390

https://en.wikipedia.org/wiki/IBM_3090

https://en.wikipedia.org/wiki/IBM_Z

In September 1990 IBM announced S/390 or ES/9000 with 31 bit addressing (Plambeck, 1989).

https://en.wikipedia.org/wiki/IBM_System/390

It was succeeded with the 64 bit z-architecture in 2000 (Plambeck et al., 2002).

<https://en.wikipedia.org/wiki/Z/Architecture>

Before 64 bit addressing was introduced an expanded memory was used as an extension of main memory, which was limited to 2 Gigabytes.

https://en.wikipedia.org/wiki/Z/Architecture#IBM_mainframe_expanded_storage

The expanded memory could also be used for inter-system communication (Rahm, 1992). The z-architecture is explained in “ABCs of z/OS System Programming” (Salla and Pughton, 2018).

Expanded storage was accessible as 4 KB blocks. Synchronous I/O with busy waiting was used in the case of data requests issued to expanded storage, since it incurred a short delay. Due to 64 bit addressing z/OS removed the need for Expanded Storage, so that all memory in z/OS is central storage. The move page instruction moves a single page, but there are instructions to move multiple pages. One MB pages were introduced in 2009 in conjunction with z10.

<https://en.wikipedia.org/wiki/Z/Architecture>

The z14 uses a 5.2 MHz microprocessor and its technical details are as follows.

[https://en.wikipedia.org/wiki/IBM_z14_\(microprocessor\)](https://en.wikipedia.org/wiki/IBM_z14_(microprocessor))

<https://www.ibm.com/us-en/marketplace/z14/details>

The ten cores implement a *Complex Instruction Set - CISC* z/Architecture with a superscalar, out-of-order pipeline. The z14 micro has 28 KB I-L1 and 28 KB D-L1 per core caches and 2 MB I-L2 and 4 MB D-L2 per core, and a 128 MB L3 shared cache (I = instruction, D = data).

It is argued in (Hennessy and Patterson, 2019) that microprocessors with the RISC organization outperform those with the CISC organization. While performance is important, the ability to run legacy software is more important, hence the concept of computer families and plug compatibility as expressed by Amdahl in his ISCA’76 keynote. The same argument applies to the Intel x86 processors that mostly runs Microsoft software.

IBM z-mainframes are based on the z-architecture rather than a RISC architecture, because it would be too costly to rewrite the software for IBM mainframes. Articles on IBM z14 appeared in March-May 2018 issue of *IBM J. R & D* 62.

[https://en.wikipedia.org/wiki/IBM_z14_\(microprocessor\)](https://en.wikipedia.org/wiki/IBM_z14_(microprocessor))

<https://www.gartner.com/doc/reprints?id=1-53ILPYJ&ct=180614&st=sg>

The latest Z15 IBM mainframe was announced in September 2019. The following article clarifies how much is being done on IBM mainframes.

<https://www.zdnet.com/article/ibm-z15-release-shines-light-on-how-much-is-still-being-run-on-a-mainframe/>

It is said that 85% of all credit cards, 29 billion ATM txns per year, 92% of the top 100 banks, 12 billion passenger flights are booked using IBM Z computers each year, etc.

Four criteria explain why IBM z continues to be a fit for core banking platforms is as follows:

1. **Continuous availability:** 83% of clients report near 100% availability.
2. **Secure computing platform.**
3. **Supports heavy and data intensive workload.**

4. Proven platform.

IBM renamed the descendants of AS/400 computers, after they were re-based on IBM Power processors, i-series

https://en.wikipedia.org/wiki/IBM_System_i

https://en.wikipedia.org/wiki/IBM_Power_Systems In August 2020 IBM introduced the Power10 core, which adheres to the new Power instruction set version 3.1, which was open sourced under the OpenPower Foundation last August. The Power10 core has over 200 new instructions which are aimed at making the microarchitecture more efficient at certain functions and adding new functions. A breakdown of the important new instructions and data formats supported in the Power10 core:

<https://www.nextplatform.com/2020/08/18/ibm-brings-an-architecture-gun-to-a-chip-knife-fight/>

The Power10 computer architecture feature highlights are as follows:

<https://en.wikipedia.org/wiki/POWER10>

Prefix architecture: Greatly expanded opcode space,

Program Counter - PC-relative addressing,

Matrix Th Assist - MMA instructions for SIMD processing and AI inference.

RISC friendly 8B instructions, including modified and new opcodes. 4B prefix and 4B suffix.

New instructions and datatypes (1) New scalar instructions for control flow and operational symmetry. (2) New SIMD instructions for AI, throughput and data manipulation.

Advanced systems features and ease of use: (1) Storage management, (2) debug, (3) Hot/cold page tracking, (4) Adaptive power management.

Adaptive Energy Scale: Adaptive power management.

Security for cloud: Transparent isolation and security for enterprise cloud workloads.

Inception clustering is the capability to access up to 2 PetaBytes by a Power10 processor not in just its own memory. It is noted that a similar capability was announced in conjunction with HP's The Machine. <https://www.nextplatform.com/2021/08/13/the-power-of-power10s-memory-inception-clustering/>

The connectivity of flash memory to S/390 - System Z is explained in (Tzortzatos, 2013). "Bus attached" flash storage feature *Flash Express - FE* introduced in 2012-13 for System Z, which is faster than DS8800 disk array with 60 GB cache and shows improved performance when paging can be done to FE. The firmware called *Logical PARTitions - LPAR* can virtualize the available physical FE. The preferred unit of transfer is 1 MB not 4 KB. The physical implementation of FE involves two PCIe bus attached flash storage modules so that if one fails, no data is lost.

1.9 Early computer companies possibly competing with IBM

Burroughs was one of the five mainframe companies known as the BUNCH: Burroughs, *UNIVersal Automatic Computer - UNIVAC* - Sperry Rand, *National Cash Register - NCR*, Control Data, and Honeywell which competed with up. With *General Electric - GE* and *Radio Corp. of America - RCA* they constituted the seven dwarfs:

<http://www.dvorak.org/blog/ibm-and-the-seven-dwarfs-dwarf-one-burroughs/>

1.9.1 Burroughs + UNIVAC = UNISYS

Burroughs Adding Machines Company was founded in 1886 and lasted a century till the merger with UNIVAC Corp. to form Unisys in 1986.

https://en.wikipedia.org/wiki/Burroughs_Corporation

<https://en.wikipedia.org/wiki/UNIVAC>

In 2018 Unisys had less than 22,000 employees, a revenue of \$2.83 billion and \$75.5 million earnings.

<https://en.wikipedia.org/wiki/Unisys>

The functional design of computers was critically reviewed and a new approach proposed in the context of Burroughs 5000 computer in (Barton, 1987). The B5000 was based on Algol programming language and used Polish string programs, https://en.wikipedia.org/wiki/Polish_notation

a new arrangement of control registers, and automatic allocation of temporary storage within expressions and procedures.

Burroughs stack computers had a segmented virtual memory. The highest level of the stack was held in fast registers. The organization of Burroughs B5700/B6700 computers is discussed in detail in (Organick, 1973).

https://en.wikipedia.org/wiki/Stack_machine

https://en.wikipedia.org/wiki/Burroughs_large_systems

A Burroughs 6700 was used as a front-end to the ILLIAC IV using fourteen synchronized disks to load programs and data (Hord, 1990). The slow I/O negated the speedup afforded by the ILLIAC IV parallelism, which was due to Amdahl's law in parallel computers as discussed in Section 1.34.1.

https://en.wikipedia.org/wiki/Amdahl%27s_law

Society for Worldwide Interbank Financial telecommunications - SWIFT was founded in Brussels, Belgium in 1973 to support 239 banks in fifteen countries. It started to establish common standards for financial txns and a shared data processing system and worldwide communications network designed by Logica and implemented by Burroughs. Fundamental operating procedures, rules for liability, etc., were established in 1975 and the first message was sent in 1977.

https://en.wikipedia.org/wiki/Society_for_Worldwide_Interbank_Financial_Telecommunication#cite_note-14

Logica was acquired by Canada-based CGI Group in 2012.

https://en.wikipedia.org/wiki/CGI_Inc.

1.10 My experience at Burroughs Corp.

I visited the Burroughs plant in Pasadena in 1980s which manufactured B-Series computers, described later, on the first floor of a one storey building, while employee offices were in the basement. This arrangement obviated the need for an elevator to move heavy equipment. Having outside office windows, a requirement in Germany, is not followed in US. No windows were available at the IBM T. J. Watson Research Center at Yorktown Heights, which was its postal address, but was actually in Millwood, NY. The three-storey crescent-like structure was designed by Finish architect Eero Saarinen and completed in 1961.

https://en.wikipedia.org/wiki/Eero_Saarinén

https://en.wikipedia.org/wiki/Thomas_J._Watson_Research_Center

IBM scientists at that time were supposed to spend their time in labs doing experiments in physics, chemistry, etc. Private offices were provided for writing patents and reports. Such research has been mostly discontinued at IBM. A windowed corridor on the two higher floors extended the full length of the building had white boards for technical brainstorming.

Blaine Gaither in Pasadena was the pointman to hire employees for the new *Corporate Center for Performance Modeling - CCPM* in Santa Ana, CA. Headhunter Christa Berlin acted as an intermediary calling me at *U. Southern Calif. - USC* asking me to contact Blaine. I found out later that my name was given to Burroughs along with eleven others by Mani K. Chandy at CalTech, who had declined the Burroughs offer to become CCPM director. I was not interested in the that position either, since it would entail frequent trips to Burroughs headquarters, which was in Detroit at the time. Gaither's personal preference for director's position was Pat Artis, since he could "sell ice to Eskimos" and hence would ensure CCPM funding.

<https://www.linkedin.com/in/pat-artis-b291b527/>

Santa Ana, CA was a desirable location since it was within driving distance from USC and that would allow me to continue supervising my three PhD students there.

W. Michael Blumenthal who had served as Secretary of Treasury in Jimmy Carter's cabinet had joined Burroughs Corp. as CEO in 1980 after Carter lost the presidential bid against Ronald Reagan.

https://en.wikipedia.org/wiki/W._Michael_Blumenthal

Carter's reelection failure can be attributed to the US embassy hostages in Tehran and his failure to secure hostage release by negotiation or through Operation Eagle Claw.

https://en.wikipedia.org/wiki/Operation_Eagle_Claw

Desert 1 is a comprehensive documentary about this operation.

<https://chicago.suntimes.com/movies-and-tv/2020/8/20/21375930/desert-one-review-iran-rescue-mission-documentary-operation-eagle-claw-barbara-kopple-movie>

In 1984 Blumenthal visited Southern California to boost employee morale in 1984. At a dinner gathering, having read "In Search of Excellence", first published in 1982 (Peters and Waterman, 1982), I asked him whether he expects Burroughs to be listed in the book. It didn't and instead the merger led to a rapid downfall of Unisys.

I joined Burroughs Corp. without researching its financial fitness. This was not easy before the advent of the Internet, but could be found in the financials of NY Times, *Wall Street Journal* – *WSJ*, etc. I assume trade publications such as ComputerWorld, Datamation would not write negative articles about Burroughs, which was an advertiser.

I had a positive impression of Burroughs Corp. based on descriptions of its computers. Famous computer scientist Edsger W. Dykstra was still a Burroughs Fellow at U. Texas at Austin in 1983, but he retired soon thereafter as a Schlumberger

Fellow also at Austin (1984-1999).

https://en.wikipedia.org/wiki/Edsger_W._Dijkstra

At Burroughs I found out that the company is losing business to IBM, since it was not able to meet the mainframe performance expectations by its customers. In 1986 when Burroughs bought UNIVAC computers for \$4.8 billion, the merged Unisys was supposed a stronger company and was advertised as “the synergy resulting from the power of two”.

The problem was the two companies had overlapping products and that Burroughs and UNIVAC mainframes were competitors in the small market for non-IBM mainframes, while interest in mainframe computers was declining in general. Currently Unisys is an IT services company that also supports Linux platforms.

<https://en.wikipedia.org/wiki/Unisys>.

<https://www.unisys.com/>

Burroughs in 1980s had several computer lines:

A series was a Burroughs stack computers described in (Organick, 1973).

https://en.wikipedia.org/wiki/Burroughs_large_systems

A series computers provided a Fortran compiler. I used the QNM of Buzen’s *Central Server Model - CSM* proposed in (Buzen, 1973) to compare the throughput of the A17 with a 17% faster CPU than its predecessor, possibly A15. There was a negligibly small improvement in performance (throughput) for the debit-credit like workload I modeled. According to this note the Burroughs A17 competed with IBM 3090/600E six-way mainframes in 1988 and rented for \$200K/month.

https://techmonitor.ai/technology/burroughs_announces_new_top_end_a17_mainframe

Burroughs mainframe computers were different from IBM S/360 mainframes in that supposedly they used packet rather than circuit switching in transmitting data from disks to main memory, but I have not seen documents on this matter.

Burroughs engineers who left the company to join HP developed its HP 3000 stack computer.

https://en.wikipedia.org/wiki/HP_3000.

The B series: B2500, B3500, B4500, which were decimal machines like the IBM’s 1400 series and used COBOL and assembler programming.

https://en.wikipedia.org/wiki/Burroughs_Medium_Systems

B1000 series was started in 1970s and included the B1700, B1899, and B1900. A notable idea in the B1700 was closing the semantic gap, the control store was writable and the microprogram loaded would be geared to the program to be executed (Wilner, 1972).

https://en.wikipedia.org/wiki/Burroughs_B1700

In 1988 UNISYS bought *Convergent Technologies Inc. - CTI*, after which it became its Network Systems Division. CTI was formed in 1979 by former Intel and Xerox PARC employees and its system ran both UNIX and MS-DOS.

https://en.wikipedia.org/wiki/Convergent_Technologies

J. Presper Eckert and John Mauchly built the *Electronic Numerical Integrator and Computer - ENIAC* at U. Pennsylvania’s Moore School of Electrical Engineering in 1943-1946.

<https://en.wikipedia.org/wiki/ENIAC>

John von Neumann engaged with the team of engineers who built the ENIAC to later write (Burks et al., 1963).

<https://www.ias.edu/electronic-computer-project>

Eckert and Mauchly formed a company which underwent several transitions described below culminating in UNIVAC.

<https://en.wikipedia.org/wiki/UNIVAC>

UNIVAC/Sperry Rand’s third generation computers (1960-1980) are described in (Gray and Smith, 2001). Earlier Sperry computers are described by the same authors in (Gray and Smith, 2004).

My students at *Case Western Reserve U. - CWRU* used the UNIVAC 1108 to develop simulation programs of a multi-programmed computers with storage allocation for batch processing using ALGOL for the operating systems course.

The computer center was located on the first floor of Crawford Hall at the northern edge of the CWRU campus located at the Univ. Circle

<https://www.universitycircle.org/> Donald Knuth was an undergrad student at CWRU Given a deck of cards holding a compiler to reproduce, he returned a deck with an improved compiler.

https://en.wikipedia.org/wiki/Donald_Knuth

William Lynch did OS research, but then joined Chi Corp in Cleveland OH.

<https://chicorporation.com/company/our-history/>

Ted Glaser was another important person associated with CWRU, who in spite of blindness made major contributions to computer science.

<https://www.nap.edu/read/4779/chapter/21>

1.10.1 NCR - National Cash Register Corp.

National Cash Register - NCR Corp. was founded in 1884.

https://en.wikipedia.org/wiki/NCR_Corporation

It is better known for point of sale terminals, but it has also produced several computers. NCR was acquired by AT&T in 1991 for \$7.4 billion, itself acquired Teradata Corp. later that year.

<https://www.teradata.com/>

In January 1993 NCR produced NCR 6298 RAID5 disk array, which is mentioned in (Chen et al., 1994b). AT&T spun off NCR in 1997 and NCR spun-off Teradata in 2007.

The Teradata DBC/1012 computer can be traced back to the doctoral thesis of Philip M. Neches at *Calif. Inst. of Technology* - CalTech. The storage capacity of DBC/1012 was obviously one TeraByte - TB or 10^{12} bytes.

<https://en.wikipedia.org/wiki/Teradata>

The first Teradata computer with an innovative mirrored disk organization was delivered to Walmart in 1983 in time for Christmas shopping.

<https://www.teradata.com/About-Us/Our-History>

An “Introduction to Teradata” and its documentations are provided here.

<https://docs.teradata.com/reader/xrmOxQIXa0ETFi5IPxHJaQ/uiY4wJQifXwZ8ih87eQLw>

<https://www.info.teradata.com/download.cfm?ItemID=1001259>

1.10.2 Control Data Corporation

Control Data Corp. - CDC was formed by a group of engineers after WW II.

https://en.wikipedia.org/wiki/Control_Data_Corporation

The CDC 6600 with a 100 ns CPU cycle time was the world’s fastest computer between 1964-69,

https://en.wikipedia.org/wiki/CDC_6600.

http://wiki.sjs.org/wiki/index.php/History_of_Computers_-_CDC_6600

more specifically three times faster than the IBM Stretch computer, whose clock cycle time varied from 300 to 600 ns across units.

https://en.wikipedia.org/wiki/IBM_7030_Stretch

Stretch was world’s fastest computer between 1961-64, but was considered a failure because it was only 30-40 times not hundred times faster than its predecessor.

<http://archive.computerhistory.org/resources/text/IBM/Stretch/pdfs/05-16/102632616.pdf>

<https://www.brightwork.com/blog/project-failures-ibms-stretch-computer>

The IBM 7950 known as Harvest which was based on the Stretch computer was delivered in 1962 to the *National Security Agency* - NSA where it operated until 1976. It was designed by James H. Pomerene and built by IBM for cryptanalysis. Pomerene later joined IBM Research where he became an IBM Fellow.

<https://history.computer.org/pioneers/pomerene.html>

The CDC 6600 was also faster than the IBM 360/91 announced in 1967 with a 60 ns cycle time.

https://en.wikipedia.org/wiki/IBM_System/360_Model_91

Thirty years after the introduction of the S/360 91 Michael Flynn at Stanford wrote an article about supercomputing at IBM (Flynn, 1998).

The CDC 7600 was a successor to the CDC 6600 and was the world’s fastest computer 1969-1975.

https://en.wikipedia.org/wiki/CDC_7600

An interesting aspect of the CDC 6600/7600 computers was the ten peripheral units, which had their own registers but shared an *Arithmetic Logic Unit* - ALU on a time-multiplexed basis (Thornton, 1970).

The CDC Cyber 70 and 170 series based on the architecture of the CDC 6600 and 7600 supercomputers, respectively. CDCs move from discrete electronic components and core memory to integrated circuits and semiconductor memory. The 200 series based was based on the CDC STAR-100 supercomputer discussed in Section 1.24.

https://en.wikipedia.org/wiki/CDC_Cyber

IBM had two post Stretch computer projects: X and Y, which are described in (Smotherman et al., 2016). The goal of project X at Poughkeepsie NY was to build a computer 10-20 times faster than the Stretch computer, while the goal of Project Y assigned to IBM Research was a hundred times faster computer.

IBM announced the S/360 model 92 which was supposed to be faster than the CDC 6600. This impacted CDC/6600 sales but such a computer was never delivered. This IBM technique called *Fear, Uncertainty, and Doubt* - FUD led to a

successful CDC lawsuit against IBM. CDC was awarded \$100M plus IBM's Service Bureau Business, which is listed under CDC on p. 62 in (Reilly, 2003).

Los Alamos National Lab - LANL developed the atomic bomb as part of the Manhattan Project in mid-1960s.

https://en.wikipedia.org/wiki/Manhattan_Project

<https://www.lanl.gov>

It embarked on a search for a new supercomputer to fulfill the growing need for computing power. The rivalry between IBM and CDC for this project is described in (Lewis, 2017).

In 1973 CDC announced the *Storage Module Drive* - SMD standard for disk drives, which was different from IBM's.

<http://www.computerhistory.org/storageengine/cdc-disk-drive-departs-from-ibm-standards/>

https://en.wikipedia.org/wiki/Storage_Module_Device

What is left of CDC is Ceridian, which is *Human Resource* - HR software provider.

<https://en.wikipedia.org/wiki/Ceridian>

1.10.3 Honeywell Corp.

Minneapolis based Honeywell is a major player in process control.

<https://en.wikipedia.org/wiki/Honeywell>

General Electric - GE which was involved in developing the MULTICS timesharing system sold its computer business to Honeywell in 1970.

<https://en.wikipedia.org/wiki/Multics>

Honeywell Bull was formed when Honeywell spun off its information systems division as a joint venture with France's Bull and Japan's NEC Corporations.

<https://www.nytimes.com/1987/06/10/business/honeywell-bull-s-new-computers.html>

https://en.wikipedia.org/wiki/Groupe_Bull.

Honeywell was taken over by the larger Allied Signals in 1999, but retained its name.

<https://en.wikipedia.org/wiki/Honeywell>

In 2000 *General Electric* - GE CEO Jack Welch's attempt to acquire Honeywell was blocked by the *European Union* - EU. During Welch's 1981-2001 twenty year tenure as head of GE, its revenue went up five fold and its valuation reached \$417 billion. Upon Welch's death NY Times wrote "The manager of the century" has died.

<https://www.nytimes.com/2020/03/02/business/jack-welch-died.html>

1.10.4 Hewlett-Packard - HP Corp.

Stanford U. *Electrical Engineering* - EE graduates Bill Hewlett and David Packard started HP in a Palo Alto garage in 1935. HP was incorporated in 1947 and went public in 1953.

<https://en.wikipedia.org/wiki/Hewlett-Packard>

HP entered the computer business with the HP 3000 computer, which was influenced by Burroughs stack computers.

https://en.wikipedia.org/wiki/HP_3000

HP's early history which deals with the HP 3000 is given here.

<https://web.archive.org/web/20061010164804/>

http://www.3k.com/index_papers_hp3000_history.html

Very Long Instruction Word - VLIW computers used compilers to pack several operations per instruction.

https://en.wikipedia.org/wiki/Very_long_instruction_word

VLIW chips were first delivered by Transmeta Corp. in 2000.

<https://www.computerworld.com/article/2593626/vliw-microprocessors.html>

HP collaborated with Intel in developing the Itanium microprocessor with an *Explicitly Parallel Instruction Computing* - EPIC IA-64 ISA (Sharnghani and Arora, 2000).

<https://en.wikipedia.org/wiki/IA-64>

https://en.wikipedia.org/wiki/Explicitly_parallel_instruction_computing

http://www.cs.virginia.edu/~skadron/cs854_uproc_survey/spring_2001/itanium.htm

Developing an efficient compiler for EPIC was an issue of extracting *Instruction Level Parallelism* - ILP.

<https://news.ycombinator.com/item?id=14331342>

The compiler for IA-64, i.e., Intel Itanium Architecture was successfully completed at Intel's Compiler Lab (Dulong et al., 1999, 2001). A pre-silicon software development environment called SoftDV was developed for this purpose.

The Itanium chip was succeeded by a 64 bit x86 chip, because the compiler technology did not deliver what was expected from the VLIW paradigm.

<https://en.wikipedia.org/wiki/Itanium>

Itanium is suitable for scientific computing, but not in running OS code, which involves frequent branching. Some systems spend a considerable amount of time running the OS, but of course this is not the case for computational workloads. Intel Itanium is finally dead.

<https://www.techspot.com/news/90622-intel-itanium-finally-dead.html>

Under the leadership of Carly Fiorina HP acquired Compaq, which had earlier acquired Tandem and *Digital Equipment Corp.* - DEC in 2002 in a stock swap worth \$25 billion, forming the second largest computer company in revenue after IBM at the time.

<https://www.cnet.com/news/hp-to-acquire-compaq-for-25-billion/>

Under the leadership of Meg Whitman HP was split into two divisions in 2015.

1. HP dealing with printers and personal computers with trading symbol HPQ.
2. *HP Enterprise - HPE*.

In 2016 HPE was split into a division dealing with computers and the other with IT services, which was HP's EDS acquisition called DXC Technology.

<https://arstechnica.com/information-technology/2016/05/hp-splits-again-as-hewlett-packard-enterprise-spins-off-it-services/>

<https://www.dxc.com>

HP's research center in Palo Alto is known as *HP Laboratories - HPL*.

was founded by Bernard M. Oliver, who developed *Pulse-Code Modulation - PCM* with John R. Pierce and Claude Shannon, considered the father of mathematical theory of communication (Shannon, 1949).

<https://www8.hp.com/us/en/hp-labs/research/overview.html>

https://en.wikipedia.org/wiki/Bernard_M._Oliver

Pierce who is known as father of communication satellites, left Bell Labs to join CalTech as a professor.

http://oralhistories.library.caltech.edu/98/1/OH_Pierce_J.pdf

HP's research activities in storage systems were impacted when John Wilkes moved to Google. His research at HP is described in Chapter 2.

A major area of research is 3D printing, Print Adjacencies & Microfluidics, Security, and Digital Manufacturing.

<https://www8.hp.com/us/en/hp-labs/research/overview.html>

Xerox Corp. attempt to take over HP's printer and personal computer business failed in 2020.

<https://www.cnbc.com/2020/03/31/xerox-ends-its-hostile-takeover-bid-for-hp.html>

The web page for HPE labs is

<https://www.hpe.com/us/en/hewlett-packard-labs.html>

The following projects are pursued at HPE research labs.

<https://www.labs.hpe.com/next-next>

1. **Dot Product Engine - DPE:** <https://www.labs.hpe.com/next-next/dot-product-engine>
2. **Impossible Computing:** <https://www.labs.hpe.com/next-next/impossible-problems>
- 3/4. **Toward a new data-based economy - Part One/Two:** <https://www.labs.hpe.com/next-next/data-based-economy>
<https://www.labs.hpe.com/next-next/the-smart-city-is-the-data-economy-made-manifest>
5. **The ethics of AI:** <https://www.labs.hpe.com/next-next/ai-ethics>
6. **Beyond bitcoin:** <https://www.labs.hpe.com/next-next/blockchain>
7. **Extreme computing:** <https://www.labs.hpe.com/next-next/exascale>
8. **Memory in the middle, Driven Computing - MDC:** <https://www.labs.hpe.com/next-next/mdc>
 The Machine is an ongoing HP project
<https://www.hpl.hp.com/research/systems-research/themachine/>
 The MDC project was presented in (Keeton, 2017).
<https://www.hpe.com/us/en/newsroom/blog-post/2017/05/memory-driven-computing-explained.html>
9. **The rise of common sense analytics/Distributed Mesh Computing - DMC:**
<https://www.labs.hpe.com/next-next/dmc>
10. **Tiny lasers - big gains:** <https://www.labs.hpe.com/next-next/energy>
11. **Rethinking Security:** <https://www.labs.hpe.com/next-next/security>

12. Neuromorphic computing: <https://www.labs.hpe.com/next-next/brain>

Neuromorphic computing is also an area of research at *Argonne National Labs - ANL*
<https://www.hpcwire.com/2021/08/13/argonne-turns-to-insects-for-neuromorphic-computing/>
 motivated by Intel's Loihi chip
<https://www.hpcwire.com/2020/03/18/intels-neuromorphic-chip-scales-up-and-it-smells/>

1.10.5 Digital Equipment Corp - DEC

DEC was founded by Ken Olsen and Harlan Anderson with a \$70K budget in Maynard, MA in 1957.

https://en.wikipedia.org/wiki/History_of_Digital_Equipment_Corporation

https://en.wikipedia.org/wiki/Digital_Equipment_Corporation

Both were affiliated with MIT's Lincoln Lab and involved in Whirlwind vacuum tube computer project, which served as a flight simulator for the Navy.

https://en.wikipedia.org/wiki/Whirlwind_I

After considering other alternatives the Whirlwind design finally converged on a magnetic core memory, which was perfected by Jay Forrester.

https://en.wikipedia.org/wiki/Jay_Wright_Forrester

The first twenty years at DEC are described by Gordon Bell.

<http://gordonbell.azurewebsites.net/digital/timeline/tmlnhome.htm>

<http://gordonbell.azurewebsites.net/digital/dec%201957%20to%20present%201978.pdf>

DEC in 1988 was the second largest computer company behind IBM, with \$11 billion in revenue and over 120,000 employees. DEC had very successful product lines: PDP-8, PDP-10 (36 bits), PDP-11, VAX clusters, and 32 and 64 bit Alpha microprocessors.

DEC's Dileep Bhandarkar and Douglas Clark had earlier compared RISC and CISC processors (Bhandarkar and Clark, 1991) and Bhandarkar later wrote (Bhandarkar, 1996) describing DEC's sophisticated Alpha microprocessor. Ending of Microsoft support for Alpha microprocessors in 1998 led to Alpha's demise.

https://en.wikipedia.org/wiki/DEC_Alpha

According to Clayton Christensen at Harvard Business School it was the inflexibility of the business model they had so long relied upon that led to DEC's downfall: His comments on disruptive innovation are given below.

<https://sloanreview.mit.edu/article/lessons-from-ken-olsen-and-digital-equipment-corp/>

"Digital Equipment Corp. had microprocessor technology, but its business model could not profitably sell a computer for less than \$50,000. The technology trapped in a high-cost business model had no impact on the world, and in fact, the world ultimately killed Digital. But IBM Corp., with the very same processors at its disposal, set up a different business model in Florida that could make money at a \$2,000 price point and 20% gross margins and changed the world."

Computers such as the 16-bit *Programmable Data Processor - PDP-11*, whose address space could not be expanded had a limited lifetime. *Virtual Address eXtension - VAX* was a 32-bit extension of the PDP-11.

<https://en.wikipedia.org/wiki/VAX>

VAX was a *Complex Instruction Set Computer - CISC* with an orthogonal instruction set, which means that the add instructions were available for all addressing modes and this should have facilitated compiler writing.

https://en.wikipedia.org/wiki/Orthogonal_instruction_set

The VAX 11/780 computer introduced in 1977 was designed by Bill Strecker, a doctoral student of Gordon Bell at CMU. Since I had submitted a paper to the *Int'l Symp. on Computer Architecture - ISCA'76*, I was asked to review another submitted paper for review, i.e., (Strecker, 1976). The paper reported trace-driven simulation results to obtain optimum parameters for the cache size, associativity, block size, replacement algorithm, and write strategy. I rated the paper as not worthy of publication, in comparison with (Liptay, 1968), which was a more convincing study to design of the cache for the IBM S/360 Model/85. The paper was not only accepted but it won the best paper award!

Before its demise DEC had several research centers.

https://en.wikipedia.org/wiki/DEC_Systems_Research_Center

DEC was acquired by Compaq Corp. in 1998 and in 2002 Compaq was taken over by HP.

1.11 Computer company revenue rankings

List of largest software companies has not varied much year to year.

https://en.wikipedia.org/wiki/List_of_the_largest_software_companies

The ranking of worlds largest technology companies in 2020 is listed in Table 1.3 based on revenue. this is a dynamic list and the readers should refer to the web page for the latest ranking, e.g., IBM ranked #9 was ranked #12 on 8/14/21.

https://en.wikipedia.org/wiki/List_of_largest_technology_companies_by_revenue

TABLE 1.3 World's technology companies ranked by total revenues in millions of dollars for respective fiscal years ending in 2019.

Rank	Country/Company	Revenue \$B	Employees	Headquarters
1	US/Apple Inc.	260,174	137,000	Cupertino, CA
2	S. Korea/Samsung	197,707	287,439	Suwon, SK
3	Taiwan/Foxconn	178,869	757,404	New Taipei City
4	US/Alphabet Inc.	161,857	118,899	Mountain View, CA
5	US/Microsoft	125,843	144,000	Redmond, WA
6	China/Huawei	124,316	194,000	Shenzhen, CN
7	US/Dell Tech.	92,154	165,000	Round Rock, TX
8	Japan/Hitachi	80,639	301,056	Tokyo, Japan
9	US/IBM	77,147	383,056	Armonk, NY
10	Japan/Sony	75,972	111,700	Tokyo, Japan
11	US/Intel	71,965	110,800	Santa Clara, CA
12	US/Facebook	70,697	44,842	Menlo Park, CA
13	Japan/Panasonic	68,897	259,385	Osaka, Japan
14	US/HP Inc.	58,613	56,000	Palo Alto, CA
15	China/Tencent	54,613	62,885	Shenzhen, CN
16	SK/LG Electronics	553,464	74,000	Seoul, SK
17	US/Cisco 2019	51,904	75,900	San Jose, CA
18	China/Lenovo	50,716	63,000	Beijing, China

As of 2020, Fortune lists Amazon (revenue of \$280.522 B in the retailing sector rather than the technology sector. It was ranked #3 in 2017 and is ranked #2 following Walmart with \$120.83 B retail revenue in 2018. Walmart is #1 at \$387.66 B according to *National Retail Federation - NFR* ranking.

<https://nrf.com/resources/top-retailers/top-100-retailers/top-100-retailers-2019>

1.12 Computer structures book

“Computer Structures” by Gordon Bell (DEC, CMU, Microsoft, retired)

https://en.wikipedia.org/wiki/Gordon_Bell

and Alan Newell (CMU, rip) is an encyclopedic description of early computers (Bell and Newell, 1971), which had a second edition with third author Daniel Siewiorek (Siewiorek et al., 1981).

Bell was with CMU 1966-72 but returned to DEC to develop VAX clusters. After retiring from DEC in 1983 he founded Encore computers, which was a multiprocessor with a snooping cache.

https://en.wikipedia.org/wiki/Encore_Computer

In 2002 Compro Computer Services acquired Encore Real Time Computing and continues its support of *Systems Engineering Lab - SEL*,

https://en.wikipedia.org/wiki/Systems_Engineering_Laboratories

Gould and Encore Real Time Computing products offer an upgrade path with the *Legacy Computer Replacement System-LCRS* hardware simulator.

<http://www.compro.net/web/>

Bell was a founding member of Ardent Computer in 1986 and remained at Ardent until it merged with Stellar in 1989, to become Stardent Computer. After serving a few years at NSF he joined Microsoft Corp. and his books and articles are

posted at:

<https://gordonbell.azurewebsites.net/>

ACM has an award in *High Performance Computing - HPC* named after Bell.

<https://awards.acm.org/bell>

Book's first edition reproduced John von Neumann's 1946 report at Princeton's *Institute of Advanced Studies - IAS* to the US army for stored program computers (Burks et al., 1963), which defines the von Neumann architecture.

https://www.ias.edu/sites/default/files/library/Prelim_Disc_Logical_Design.pdf

https://en.wikipedia.org/wiki/Von_Neumann_architecture

The Harvard architecture differs from the von Neumann architecture in that it provides separate memories for instructions and data.

https://en.wikipedia.org/wiki/Harvard_architecture.

Computers with separate instruction and data caches are now considered in the latter category, e.g., z14 and z15 IBM mainframes have separate L1 and L2 cache levels, but L3 caches hold both data and instructions.

There are also modified Harvard architectures allowing the contents of the instruction memory cache to be modified.

https://en.wikipedia.org/wiki/Modified_Harvard_architecture

Application of self-modifying code are discussed below, but the popularity of this method to save memory space has diminished over the years.

https://en.wikipedia.org/wiki/Self-modifying_code

Self-modifying code was used in IBM's early computers and IBM's *Selective Sequence Electronic Computer - SSEC* was the first computer capable of treating its own stored instructions exactly like data, modifying them, and acting on the result (Bashe et al., 1981). The IBM S/360 and its successors have an EXECUTE (EX) instruction, which overlays the second byte of a target instruction with the low-order 8 bits of R1 (*General Purpose Register - GPR-1*). This provides the effect of self-modification although the actual instruction in storage is not altered.

Kiviat graphs, radar charts, or star plots have been used in the book to represent various performance metrics of computer systems (Ferrari, 1978).

https://en.wikipedia.org/wiki/Radar_chart

<https://www.itl.nist.gov/div898/handbook/eda/section3/starplot.htm>

Little space is devoted in (Siewiorek et al., 1981) to storage systems. Chapter 9 describes the segmented virtual memory in Burroughs B5000 systems, which allows relocatable program and data segments and a drum for backup storage (Barton, 1987). Chapter 10 discusses the one level Atlas computer at U. Manchester, UK, which also used a drum as an extension to main memory (Kilburn et al., 1962).

Processor Memory Switch - PMS diagrams are used in the book to specify high-level computer organization. These diagrams however do not provide sufficient detail, e.g., that the S/360s uses circuit switching for I/O, requiring the elements of disk to memory path to be free for data transmission to occur.

1.13 Computer family architectures - CFA

According to (Huck and Flynn, 1989) the effectiveness of the execution of an application depends upon (1) the compiler, (2) the instruction set, (3) the implementation of the instruction set. The book provides a study of the effectiveness of instruction sets, which is motivated by the 1970s US Army funded study to evaluate the *Computer Family Architectures - CFAs* from the viewpoint of their ISA - Instruction Set Architecture. After a pre-screening with respect to some features deemed absolutely necessary the three finalists were the IBM S/360, the PDP-11, and the Interdata 8/32. The book extends Hauck's PhD thesis with the analysis of VAX/CMS, HP Precision Architecture - PA, and Architectures Workbench data.

Three performance metrics were used to determine the efficiency of the three architectures (Fuller et al., 1977).

S: The size of the memory required for a test program.

M: The number of bytes transferred between the main memory and processors for test program execution.

R: The number of bytes transferred between registers for test program execution. This measure does not take into account register bypassing via forwarding as in IBM 360/91, which has been implemented by several microprocessors.

The average performance of the architectures on twelve test programs are given in Table 1.4.

Interdata 8/32 followed by the PDP-11 and the S/370 was the winner. Interdata was founded in 1966 and its ISA was loosely based on the IBM S/360.

<https://en.wikipedia.org/wiki/Interdata>

Perkin-Elmer which was founded by Richard Perkin and Charles Elmer in 1937 as an optical design and consulting company took over Interdata in 1973.

TABLE 1.4 Quantitative comparison of three architectures.

Architecture	S	M	R
PDP-11	1.00	0.93	0.94
IBM S/370	1.21	1.27	1.29
Interdata 8/32	0.83	0.85	0.83

<https://en.wikipedia.org/wiki/PerkinElmer>

Perkin-Elmer spun off Concurrent Computer Corporation as its computing division in 1985.

<https://www.concurrent.com/>

<https://www.encyclopedia.com/books/politics-and-business-magazines/concurrent-computer-corporation>

The PDP-11 was a 16-bit minicomputer produced by DEC.

<https://en.wikipedia.org/wiki/PDP-11>

Its 16-bit logical addresses did not allow it to develop large applications and its Unibus serving memory accesses and I/O transfers was also a performance bottleneck. *Virtual Address eXtension* - VAX was a 32-bit CISC - *Complex Instruction Set Computer* minicomputer extension of the PDP-11.

<https://en.wikipedia.org/wiki/VAX>

It could be used for timesharing using the VMS Operating system and its successor OpenVMS.

<https://en.wikipedia.org/wiki/OpenVMS>.

A list of recent computer architectures is given by Mark Smotherman at Clemson U.

<https://people.cs.clemson.edu/~mark/architects.html>

A book describing *High-Level Language* - HLL computers was edited by Yaohan Chu at U. Maryland (Chu, 1975). Chu's provides four types of HLL computers: (1) von Neumann architecture, (2) syntax-oriented architecture, (3) indirect execution architecture, and (4) direct execution architecture. It is stated in (Ditzel and Patterson, 1980) that HLL computer systems are desirable, because of their very compact code, e.g., $a = x + y$ is more compact than

$x \ y \ + \ \rightarrow \ a,$

which is more compact than

load r1 x add r1 y st r1 a

This is also true of Java bytecode, which is postfix polish, so every instruction knows that one or more of its arguments is on the stack and those arguments do not have to be named.

https://en.wikipedia.org/wiki/Java_bytecode

A "Retrospective on High-Level Language Computer Architecture" in ISCA'80 was written by David Ditzel and David Patterson (Patterson and Ditzel, 1980).

<https://courses.cs.washington.edu/courses/cse548/05wi/files/Ditzel-Retrospective-on-HLL-Computer-Architecture.pdf>

Interestingly Ditzel coauthored with Patterson the famous *Reduced Instruction Set Computer* - RISC paper (Patterson and Ditzel, 1980).

The following discussion lists all three families of Burroughs computers as HLL. They are discussed in Section 1.10.

https://en.wikipedia.org/wiki/High-level_language_computer_architecture

The simplest reason for the lack of success of HLL computer architectures is that from 1980 optimizing compilers resulted in much faster code and were easier to develop than implementing a language in microcode.

1.14 Virtual memory and page replacement algorithms

Virtual memory systems were developed to extend main memory size with space on secondary storage devices (Denning, 1970). Virtual memories worked based on paging or segmentation.

In paging main memory is partitioned into fixed sized 4 KB blocks to fit 4 KB disk resident pages. Page sizes have grown with increasing main memory sizes. Logical addresses have two parts: page number and displacement. The page number (p) constitutes the high order address bits and there are x low order bits such that $2^x = \text{pagesize}$. Table lookup can be used for the mapping $p \rightarrow (1 : f)$, where f is the number of frames. The virtual memory paradigm allows the running of a program with more pages than frames. With the advent of larger main memories whole programs are held in main memory, so paging of programs was not an issue anymore. According to (Buzen, 1978) IBM's MVS operating system swaps whole program address spaces in and out, rather than one page at a time.

Addresses generated in a segmentation system are a tuple (s, d) , where s is the segment number and d is the displacement. The segment table has two entries per segment: (base, limit). s determines the base in the segment table, which added to d is the address in main memory. $d < \text{limit}$ to protect against addressing errors.

Pages or segments held in main memory are based on the memory reference pattern of active programs.

<https://techdifferences.com/difference-between-paging-and-segmentation-in-os.html>

Segments are variable in size and introduce external fragmentation, while pages are fixed size and introduce internal fragmentation.

A minimal number of pages called the working set should be loaded into main memory to run the program efficiently (Denning, 1970). When the program references a page, which is not already in main memory, the page is loaded into main memory if there are empty frames. Otherwise the page replacement algorithm finds a page to be overwritten. If the page to be replaced is dirty it should be saved on disk first, but the system may write dirty pages to disk in advance to maintain a sufficient number of clean pages in main memory.

A dirty page in main memory can be saved on a disk by specifying the following parameters to the *Direct Memory Access - DMA* controller. (1) The address of a page in main memory. (2) A page address on disk. (3) Action: Write. Similar steps are required to load a page from disk onto an empty or a clean page. Dirty pages can be saved to disk in advance to ensure that a sufficient number of frames are available for loading.

In a multiprogrammed computer system which runs multiple programs, a local page replacement policy replaces a page belonging to the program causing the page fault. A global page replacement policy may overwrite the pages of another program.

The key feature of the Atlas computer installed at U. Manchester in 1962 was its one level store combining a fast core memory and a drum with fixed page sizes.

[https://en.wikipedia.org/wiki/Atlas_\(computer\)](https://en.wikipedia.org/wiki/Atlas_(computer))

Interesting features of Atlas are as follows

- (1) 48-bit word size. A word could hold one floating-point number, one instruction, two 24-bit addresses or signed integers, or eight 6-bit characters.
- (2) A fast possibly *Carry Lookahead Adder - CLA* to minimize carry propagation time.
- (3) 24-bit (2 million words, 16 million characters) address space that embraced supervisor store, V-store, fixed store and the user store
- (4) 16K words of core store (equivalent to 96 KB), featuring interleaving of odd/even addresses.
- (5) 8K words of read-only memory called fixed store which contained the supervisor and extracode routines.
- (6) 96K words or 576 KB drum store, split across four drums but integrated with the core store using virtual memory.
- (7) 128 high-speed index registers used for address modification.
- (8) Peripherals such as magnetic tape and *Direct Memory Access - DMA* facilities.
- (9) Peripheral control through V-store addresses (memory-mapped I/O), interrupts and extracode routines, by reading and writing special wired-in store addresses.
- (10) An associative or content-addressable memory of page address registers to determine whether the desired 512 word page was in core store.
- (11) Instruction pipelining

Magnetic drum memory was invented by Austrian Gustav Tauschek in 1932, preceding magnetic disks by twenty years.

https://en.wikipedia.org/wiki/Drum_memory

Drums are cylindrical and have fixed head associated with each track so that there is a latency to access desired data blocks.

Performance analysis and scheduling in drums are discussed in (Fuller and Baskett, 1975), (Fuller, 1975). Drums were used in early time-sharing systems for swapping whole programs, but were replaced by disks organized as concentric cylinders thus with multiple tracks per cylinder providing a higher capacity. Disks are slower because do not provide dedicated *Read/Writes - R/W* heads. The R/W heads are attached to a disk arm which is moved to the cylinder holding the data on demand. This movement is called a seek. The manufacturing of drums ceased in the 1970s.

The Burroughs B5000 which used variable sized segments used a drum in 1961, but was followed up by the B5500 which used disks.

https://en.wikipedia.org/wiki/Burroughs_large_systems

The B5000 can then be therefore considered the first commercial virtual memory machine.

The operating system selects one or more of the sizes supported by the architecture. The larger pages are known as “huge pages” in Linux, “superpage” in FreeBSD, and “large pages” in Microsoft Windows. This allows for “the best of both worlds” reducing the pressure on the *Translation Lookaside Buffer - TLB*, which is a cache that holds page table

entries to speed-up processing, since otherwise given addresses generated by the CPU a memory access would be required to determine its address in main memory. This would double the number of memory references. Starting with the Pentium Pro, and the AMD Athlon, x86 processors support 4 Million Byte pages (called Page Size Extension).

This topic is addressed in (Guvenilir, 2019) and many references are provided below.

[https://en.wikipedia.org/wiki/Page_\(computer_memory\)](https://en.wikipedia.org/wiki/Page_(computer_memory))

1.15 Memory space fragmentation and dynamic storage allocation

Segmentation introduces external fragmentation and incurs overhead in finding sufficiently large enough empty memory space to allocate a segment. In the case of segmentation following the completion of a process its space is deallocated and larger memory spaces can be created by relocating programs in main memory by compaction.

Dynamic Storage Allocation - DSA algorithms such as best and first fit are discussed in (Knuth, 1997b). IBM researchers proposed preallocated popular block sizes for improved storage allocation efficiency for operating system use (Bozman et al., 1984). There is a vast literature on DSA (Wilson et al., 1995). Experiments to measure the cost of DSA in large C and C++ programs are reported in (Detlefs et al., 1994). Two books on garbage collection are (Jones and Lins, 1996), (Jones et al., 1996). Fragmentation also arises on disk. An early work dealing with this topic reduces the level of fragmentation by choosing the fraction of the contents estimated to have the greatest impact on the free space distribution followed by relocating data blocks to more favorable locations (Franaszek and Considine, 1979).

Disk defragmenter programs for Windows OS are run periodically to improve disk performance.

[https://en.wikipedia.org/wiki/Disk_Defragmenter_\(Windows\)](https://en.wikipedia.org/wiki/Disk_Defragmenter_(Windows))

When disk space is fragmented parts of a file system are allocated noncontiguously in empty disk spaces, so that there is a degradation in performance since accessing a file may incur multiple rotational latencies and seeks.

1.15.1 Page replacement algorithms

Many virtual memory replacement policies have been proposed and evaluated using analysis, and trace-driven simulation. In the case of a program whose size, rather working set, exceeds main memory size the page fault rate decreases with the number of available page frames and drops to zero once all pages of the working set are loaded into main memory. Not all program routines are always exercised and there is no need to load them into main memory unless they are referenced, e.g., subroutines that are invoked to deal with rare conditions. The replacement algorithm is important for buffers holding file or database pages, which are larger than program size and may be orders of magnitude larger than buffer size.

Virtual memory page replacement algorithms are implemented using auxiliary data structures, a doubly linked list (dequeue) or a hashtable or hashmap, which has a pointer to the block.

https://en.wikipedia.org/wiki/Page_replacement_algorithm

A list of page replacement algorithms is as follows

FIFO page replacement algorithm always replaces the oldest page or the page that has been in memory for the longest time. FIFO may utilize a two-sided dequeue with entries added from one end and removed from the other. Referenced pages are moved to the back of the dequeue and this is where newly referenced pages are added.

Second Chance or CLOCK considers candidate pages for removal in a round robin matter. A page that has been accessed between successive inspections is not replaced. The algorithm is implemented by adding a “second chance” bit to each memory frame and every time the frame is considered the bit is reset and otherwise the page is replaced.

Least Recently Used - LRU moves the entry of a referenced page to the bottom of a stack and the page at the top of the stack is the one to be replaced, unless it is dirty, in which case it has to be saved first.

Laszlo Belady at IBM discovered that certain page replacement algorithms, such as FIFO and second chance can cause more page faults as the memory size is increased (Belady, 1966).

https://en.wikipedia.org/wiki/B%C3%A9l%C3%A1dy%27s_anomaly

FIFO has this anomaly. For example in processing the reference string {0 1 2 3 0 1 4 0 1 2 3 4} with a buffer size 3 there are nine page faults, while there are ten page faults when the buffer size is four. An optimal algorithm called MIN is proposed in this study, but its specification is complex and is omitted here.

Stack algorithms were proposed by researchers at IBM's San Jose Research Center, which later moved to the *Almaden Research Center - ARC* (Mattson et al., 1970). During my one year temporary assignment at IBM/ARC starting April 1993 I was offered Mattson's office shortly after his retirement. It still held his papers and patents. The *inclusion property* of the stack algorithm states that data items appearing in larger cache sizes also appear in smaller ones. The deficiency associated with FIFO and second chance are obviated and this makes it possible to carry out one pass processing for varying cache

sizes. Examples of stack algorithms are *Least Frequently Used* - *LFU*, *LRU*, *Random*, and not practically feasible *Optimal*, which replaces pages referenced farthest in the future.

Proposed in (Robinson and Devarakonda, 1990) is *Frequency-Based Replacement* - *FBR* algorithm for managing caches used for disk blocks by a file or *Data Base Management System* - *DBMS*. The replacement algorithm uses the concept of maintaining reference counts in which locality has been “factored out” and replacement choices are made using a combination of reference frequency and block age. Simulation results based on file system references showed that this algorithm can offer up to 34% performance improvement over *LRU*, where the improvement is expressed as the fraction of the performance gain achieved between *LRU* replacement and the theoretically optimal policy in which the reference string must be known in advance. The implementation complexity and efficiency of this algorithm is comparable to *LRU*.

LRU-K database buffering algorithm described by Elizabeth and Patrick O’Neil (rip) at U. Mass., Boston and Gerhard Weikum (U. Saarland, Austria, Max Planck Inst.). *LRU-K* keeps track of the times of the last K references to popular database pages and uses this information to estimate the interarrival times of references (O’Neil et al., 1993). The optimality of *LRU-K* is shown in (O’Neil et al., 1999). Simulation experiments show that *LRU-K* by discriminating between pages based on their access frequencies outperforms other buffering algorithms. Furthermore, the *LRU-K* algorithm adapts in real time to changing patterns of access.

Early Eviction LRU - EELRU was proposed in (Smaragdakis et al., 1999) by Yannis Smaragdakis (U. Athens), Scott F. Kaplan (U. Mass., Amherst), and Paul R. Wilson at U. Texas at Austin. *LRU* exhibits anomalies for certain access patterns. *EELRU* uses how each page has been referenced with respect to others to carry out an online cost benefit analysis. *EELRU* cannot be a constant factor worse than *LRU*, while *LRU* can be worse than *EELRU* by a factor equal to the number of pages in main memory.

Adaptive Replacement Cache - ARC algorithm maintains two *LRU* pages lists: *L1* and *L2* (Megiddo and Modha, 2003, 2004). *L1* maintains pages that have been seen only once recently, while *L2* maintains pages that have been seen at least twice recently. *ARC*’s time overhead per request is independent of cache size, while its space overhead slightly exceeds *LRU*.

CLOCK with Adaptive Replacement - CAR described in (Bansal and Modha, 2004) is inspired by *ARC* and inherits virtually all advantages of *ARC* including its high performance, but does not serialize cache hits behind a single global lock. *Car* has several advantages over *CLOCK*:

- (i) it is scan-resistant;
- (ii) it is self-tuning and it adaptively and dynamically captures the “recency” and “frequency” features of a workload;
- (iii) it uses essentially the same primitives as *CLOCK*, and, hence, is low-complexity and amenable to a high-concurrency implementation;
- (iv) it outperforms *CLOCK* across a wide-range of cache sizes and workloads.

Working Set page replacement algorithm for allocating main memory space for programs and associated data was proposed by Peter Denning at MIT (Denning, 1968b, 1970).

<https://denninginstitute.org>

A program’s working set $W(t, T)$ at time t is the set of pages referenced in the time interval $(t - T, t)$. Relations between the average working-set size, the missing-page rate and the interreference-interval distribution are derived using an efficient algorithm in (Denning, 1968a). *Independent-Reference Model* - *IRM* in which page references are statistically independent, is used to assess the effects of interpage dependencies on working-set size observations. Under general assumptions, working-set size is shown to be normally distributed.

Damped Working Set - DWS algorithm resists sudden expansion of working set size and exhibits far greater stability in the number of page frames allocated to an active process (Smith, 1976a). It is motivated by the fact that the working set paging algorithm has the disadvantage that during changes of locality large numbers of pages are accumulated in memory unnecessarily. Analysis is used to determine *DWS*’s effectiveness.

Page Faulty Frequency - PFF replacement algorithm proposed at UCLA in (Chu and Opderbeck, 1972). *PFF* is an approximation to the working set policy.

- (1) The system keeps track of the *PFF* for each process, which is the number of faults divided by the number of references. A window or a weighted calculation is required since recent page faults are of interest. It is too expensive to calculate the number of references, so it is approximated by virtual time.
- (2) If the *PFF* is low, some frames are freed, otherwise more frames are allocated to this process.
- (3) Either raise the number of frames if using a local policy; or do not evict its frames using a global policy.
- (4) If the *PFF* is too high for all processes then reduce the *MPL*.

An analytic study to compare the WS and PFF algorithms is reported in (Gupta and Franklin, 1978). It is shown the WS algorithm is generally less sensitive to changes in program behavior. Results are also provided relating to performance sensitivity to selected algorithm parameters.

Statistical multiplexing

Bell Labs engineers faced the challenge to getting more bandwidth out of its network infrastructure. *Statistical Multiplexing - SM* was proposed by Wesley Chu, who later joined UCLA, to efficiently share the channel by allowing variability in resource allocation (Chu, 1978). SM is discussed in “A history of computer communications 1968-1988” in Section 5.4 on statistical multiplexing (Pelkey, 2019).

<http://www.historyofcomputercommunications.info/Book/5/5.4WesleyChuStatistical%20Multiplexer66-75.html>

Paul Green at IBM Research developed *Dense Wavelength Division Multiplexing - DWDM*, which works by combining and transmitting multiple signals simultaneously at different wavelengths on the same fiber.

<https://www.laserfocusworld.com/fiber-optics/article/16554159/ibm-introduces-dense-wavelengthdivisionmultiplexing-technology>

This IBM research effort was spun-off to Tellabs.

<https://www.tellabs.com/about>

<https://en.wikipedia.org/wiki/Tellabs>

The history of DWDM is provided in an interview with Paul Green, who has authored (Green, 1992).

https://ethw.org/Oral-History:Paul_Green

XKL is a company that uses SM in the context of DWDM. It was founded by Len Bosack who in 1984 cofounded Cisco with his partner and now ex-wife.

<https://www.xkl.com/its-time-for-your-reintroduction-to-statistical-multiplexing-download-our-ebook-today>

VMIN is a variable space page replacement algorithm, which has the property that no other page replacement algorithm can outperform it according to the chosen performance criterion (Prieve and Fabry, 1976). After each page reference, VMIN removes the page just referenced *if and only if* - iff the page will not be referenced again for more than R/U time units, where R is the cost of a page fault and U is the cost of keeping one page in main memory for one reference time. Varying R/U generates a performance curve and VMIN is optimal in that it resides on a line in the average-memory-size/page-fault-rate plane for each value of R/U below which no replacement algorithm can have a performance point. One can view VMIN as a DWS - Damped Working Set type algorithm described earlier, which uses a forward-looking window of length R/U . The following ranking of page replacement algorithms in (Prieve and Fabry, 1976).

$OPT < LRU < CLOCK < FIFO$.

An analysis of the optimal, look-ahead demand paging algorithms is given by AJS in (Smith, 1976b). Smith's other works are described in Section 1.30. Program behavior is described by the IRM - Independent Reference Model and the LRU stack model, via a discrete time Markov chain, which is used to calculate the theoretical minimum number of page faults for a program representable by either of these models in either a fixed or variable size memory. The behavior of optimal look-ahead and optimal realizable demand paging algorithms are compared, and it is observed that look-ahead paging demonstrates an inherent advantage sufficient to account for the differences observed between currently implemented demand paging algorithms and theoretically optimal algorithms.

An approximate analytic model for predicting the buffer hit probability under the LRU and FIFO buffer replacement policies using IRM is reported in (Dan and Towsley, 1990). Results from these models are compared to exact results for models developed in (King, 1971). For small values of the buffer size (B), and the total number of items sharing the buffer (D). Analytic and simulation results are compared for large values of B and D . Two applications of the analyses are provided.

Generalized Clock - GCLOCK buffer replacement policy uses a circular buffer and a weight associated with each page brought into the buffer to decide which page to replace. An approximate analysis of GCLOCK is presented in (Nicola et al., 1992). The analysis was validated via simulation for various workloads with data access skew and in all considered cases the error is less than 1%.

A simple analysis for the transient buffer hit probability for a system starting with an empty buffer with IRM is presented in (Dan et al., 1993). It is shown that the expected buffer hit probability when the buffer becomes full is virtually identical

to the steady state buffer hit probability with LRU. The method is generalized to estimate the transient behavior of the LRU policy starting with a non-empty buffer. It is shown that this method can be used to estimate the effect of a load surge on the buffer hit probability.

Early studies of page replacement algorithms are discussed in (Coffman and Denning, 1973) and (Spirn, 1977), but also textbooks on Operating Systems listed in the Appendix.

https://en.wikipedia.org/wiki/Page_replacement_algorithm

Research in this area by Alan Jay Smith is reported in Section 1.30.

Domenico Ferrari also at UC Berkeley conducted research on program restructuring for improved performance (Lau and Ferrari, 1983). An early study on program restructuring was conducted in (Hatfield and Gerald, 1971). The aim was to ensure that sectors of a program that are needed within a relatively short time of one another are in the same virtual page. Given large memory sizes page replacement algorithms for programs are no longer an issue.

The term *thrashing* was coined by Peter Denning in the context of virtual memory systems denoting severe performance degradation due to excessive paging (Denning, 1968a). Given that the probability of a page fault per unit of virtual execution time is m and each page fault takes T time units to service them. Efficiency $e(m)$ is defined as elapsed virtual time divided by elapsed virtual plus paging time: $e(m) = 1/(1 + mT)^{-1}$. Thus efficiency is the fraction of time the CPU is processing jobs.

Figure 4 in (Denning, 1968a) plots $e(m)$ versus $0 \leq m \leq 1$ for $T = 10^i$, $0 \leq i \leq 4$, where $T = 10,000$ milliseconds is typical of disk access time in 1968. In the case of a multiprogrammed computer system with virtual memory the page fault rate is exacerbated when multiple programs compete for memory space. One solution to the problem is to reduce the degree of multiprogramming so that more space is available for each program. This will result in a reduction in the paging rate with reasonable page replacement algorithms.

1.15.2 Simplified analysis of a paging system

Multiprogramming can be used to increase system throughput due too increased CPU/IO overlap, but the page fault-rate increases with higher *MultiProgramming Levels* - *MPLs* which result in smaller memory space allocations per program. Figure 9.5 in Section 9.5 in (Lazowska et al., 1984) shows the program lifetime, i.e., the time between page faults. A given program CPU time can be used to determine the number of page faults it generates. We repeat here the analysis of a simple paging model to determine the MPL yielding the maximum throughput.

<https://homes.cs.washington.edu/~lazowska/qsp/>

There are 300 frames available for running jobs. The performance of the computer system is evaluated by varying the number of jobs as $2 \leq M \leq 8$. The program lifetime is set to an increasing function of CPU time between page faults versus allocated page frames as shown in Figure 9.5 in (Lazowska et al., 1984).

Equipartition page allocation with $MPL = 5$ allocates $300/5 = 60$ page frames to each job. It follows from the lifetime function that with this allocation each job will experience one page fault every 9 msec of CPU time. Since the average CPU service time of a job is 3 seconds, each job will experience $3000/9 = 333$ page faults. Given that the access time of the paging disk is 12.5 msec then the loading (service demand) per program at the paging disk is $12.5 \times 333 = 4.16$ sec.

The resulting QNM has five jobs and four devices: the cpu, file disk 1 and 2, and the paging disk with service demands or loadings (3, 8, 2, 4.16). Figure 9.6 on the chapter shows that the throughput increases with increases MPL and reaches its peak at $MPL = 5$, but decreases beyond this point, which is the onset of thrashing.

The analysis for solving the closed QNM using MVA is given below:

```
/* Input parameters: N devices, M jobs,  $D_n$ ,  $1 \leq n \leq N$  service demands. */
for n = 1 to N do  $Q_n(0) = 0$  /* Set queue lengths for m = 0 to zero */
for m = 1 to M do /* vary concurrency */
{ for n = 1 to N do {  $R_k(m) = D_n[1 + Q_k(m - 1)]$  } /* mean response time at node n */
 $T(m) = m / \sum_{n=1}^N R_k(m)$  /* system throughput with m jobs
for n = 1 to N do  $Q_k(m) = T(m)R_k(m)$ .
```

1.16 Analysis of thrashing in 2-phase locking - 2PL systems

Txns in 2PL systems acquire locks on demand and all locks are released when the txn completing its execution commits (Gray and Reuter, 1992). The earlier analysis in (Thomasian and Ryu, 1991) was motivated by Jim Gray's early straw-man analysis of deadlock probabilities which is given in Section 7.11.5 (Gray and Reuter, 1992). We extended Gray's analysis to variable size txns and took into account the fact that for a deadlock to occur the txn holding the requested lock should be blocked.

A simple analysis of database lock contention clarifying the thrashing phenomenon in 2PL is presented in (Thomasian, 1993). Size k txns request k exclusive locks randomly from the D database objects consist of $k + 1$ phases where each of the first k phases leads to a lock request and the last phase to the release of all locks. This is a closed system and completed txns are replaced with txns of size j with probability f_j , where $\sum_{j \geq 1} f_j = 1$. Deadlocks are rare and have a negligible effect on performance, In the simulation study aborted txns to resolve deadlocks are replaced with a txn with same sequence of lock requests.

Txns encountering lock conflicts are blocked and remain blocked until the blocking txn completes or aborts to resolve a deadlock releasing all of its locks. Given that we only consider exclusive lock requests txns can be blocked by a single txn, while in the case of shared locks a txn requesting an exclusive lock may be blocked by several txns and will remain blocked until all of them are completed or aborted. Only exclusive locks are considered here to simplify the discussion.

Lock contention results in txn blocking and a drop in the mean number of active txns denoted as $\bar{M}_a$. This is the only form of performance degradation considered, since deadlocks are rare as shown in (Thomasian and Ryu, 1991) and have a negligible effect on performance in the form of wasted processing and increased response time of aborted and restarted txns.

The throughput characteristic $t(m)$, $1 \leq m \leq M_{max}$ as determined by hardware resource contention due too increased concurrency is a nondecreasing function of m (Lazowska et al., 1984). When a fraction β of txns are blocked the mean number of active txns is $\bar{M}_a = M(1 - \beta)$ and the system throughput taking into account lock contention is $T(M) = t(\bar{M}_a)$. Thrashing occurs as the number of concurrent txns is increased beyond a certain point and $\bar{M}_a$ decreases as M is increased.

Locks are also requested uniformly over the lifetime of a txn, from D objects (pages) in the database, so that at low lock contention levels txns holds $k/2$ locks on the average. Given that $k \ll D$ the probability that a txn requests the same lock is negligibly small. Lock conflicts occur when a txn requests an exclusive lock held by another txn, so that the probability of lock conflict per lock request is the ratio of the mean number of locks held in exclusive mode by other $M - 1$ txns in the systems and the number of database pages with individual locks.

$$p_c \approx \frac{(M - 1)(k/2)}{D}.$$

When a fraction f_s of lock requests are shared it can be showed that the effective database size is $D_{eff} \approx D/(1 - f_s)^2$ (Tay et al., 1985), but shared locks complicate the estimation of the delay due to lock conflicts since txns encountering lock conflicts are blocked. as quantified in (Thomasian and Ryu, 1991).

We consider small txns (small k) so that txns only block one txn. There might be a chain of txns where only the first txn is active and holds the lock. The last txn requesting the lock remains blocked until all txns preceding it are completed or aborted. We are interested in mean txn waiting time overall txns. At higher lock contention levels result in multiple levels of blocked txns and there is a snowball effect that leads to thrashing.

We denote the mean delay for a txn of size k blocked by an active txn as W_1 . with its residence time in the system. The probability of conflict increases with the number j of locks held be a txn: $j/(k(k + 1)/2)$, where the denominator is a normalization factor. Given that the duration of each step is s the $(k - j)s$ is the waiting time. The duration of each step (s) is selected such that the throughput with one txn is the inverse of mean txn residence time in the systems $t(1) = 1/((k + 1)s)$. It follows:

$$W_1 \approx \sum_{j=1}^k \frac{2j}{k(k + 1)} (k - j)s \approx \frac{k}{3}s.$$

The probability of blocking at the i^{th} level in the chain of blocked txns is $P_b(i) = \beta^i$, $i > 1$, so that the probability of blocking with an active txn is $P_b(1) = 1 - \sum_{i \geq 1} \beta^i$. When T_2 is blocked by an already blocked txn T_1 as in $T_2 \rightarrow T_1 \rightarrow T_0$ then due to the progress made by T_0 since it caused a lock conflict, the mean delay incurred for two levels of blocking is: $W = (1 - \beta)W_1 + \beta(W_1 + W_1/2) = W_1(1 + 0.5\beta)$.

Generalizing to multiple levels:

$$W = \sum_{i \geq 1} P_b(i)(i - 0.5)W_i. \quad (1.1)$$

Let $\alpha = kp_c W_1 / R(M)$ denote the fraction of time txns are blocked by an active txn, and $\beta = kp_c W / R(M)$ the fraction of time txns are blocked, where $R(M)$ is the mean residence time of txns in the system.

$$R(M) = kp_c W + (k + 1)s(\overline{M}_a),$$

where $s(\overline{M}_a) = [\overline{M}_a / t(\overline{M}_a)](k + 1)$.

Multiplying both sides of Eq. (1.1) by $k p_c /$ and simplifying we get:

$$\beta = \alpha(1 + 0.5\beta + 1.5\beta^2 + 2.5\beta^3 + \dots)$$

Summing the series as if it has an infinite number of terms for $\beta < 1$ leads to:

$$f(\beta) = \beta^3 - (1.5\alpha + 2)\beta^2 + (1.5\alpha + 1)\beta - \alpha = 0.$$

Using the usual substitution $\beta = \gamma + \alpha/2 + 2/3$ to eliminate β^2 term in the cubic equation yields:

$$g(\gamma) = \gamma^3 - q\gamma - r = 0,$$

where q and r are functions of α . The solution to the above equation can be expressed directly as a function of trigonometric functions (see Appendix B in (Thomasian, 1993)).

The discriminant for $g(\gamma)$ is $\Delta = 27r^2 - 4q^3 < 0$, which has three real roots for $\alpha < \alpha^* = 0.226$, where α^* is the only root of the discriminant:

$$h(\alpha) = \alpha^3 + \frac{4}{5}\alpha^2 + \frac{28}{15}\alpha - \frac{64}{135} \leq 0.$$

The smallest root γ_1 yields $\beta_1 = \gamma_1 + \alpha/2 + 2/3$, which determines the fraction of blocked txns.

$$\alpha < \beta_1 < \beta_2 \leq 1 \leq \beta_3.$$

For $\alpha > \alpha^*$ the txn processing system is susceptible to thrashing as also verified via simulation, which also shows that for the same α the variability in txn sizes accelerates the time to thrashing.

The single parameter α determines the level of lock contention, which is the product of mean number of lock conflicts per txn ($k p_c$) and the normalized mean waiting time with respect to active txns ($\alpha = W_1 / R$). Given β the mean number of active txns is $\overline{M}_a = (1 - \beta)M$ and the system throughput is $T(M) = t(\overline{M}_a)$ as noted earlier.

1.17 CPU caches

Programs access data from main memory via the cache, which is smaller and faster. This is because the main memory is based on *Dynamic Random Access Memory - DRAM*, whose speed is increasing slowly at about 10% per year. High-speed *Static RAM - SRAM* is used for higher levels of multi-level caches to overcome main memory latency. SRAM is built with transistors using the same technology as processor chips, so that SRAM density is improving at the same rate as processors, about 50% per year.

CPU caches which are smaller than main memory take advantage of the spatial and temporal locality of reference principles, which were first enunciated in the context of virtual memories (Denning, 1970). Spatial locality is mainly due to the fact instructions and data tend to be accessed sequentially, as in the case of elements of a matrix in the case of data.

Temporal locality is attributable to instructions in a loop, which are executed repeatedly. The 360/91 did not have a cache, but had multiple instruction registers acting as an instruction cache which could be used for prefetching from a 16-way interleaved memory. No further instruction fetching was required in the case of tight loops and this allowed successive elements of arrays to be fetched from the 16-way interleaved memory without disruptive instruction fetches. The IBM S/360 Model 91 had a 60 ns cycle time so that at its best with its pipelined FLP - Floating Point units it could deliver 16.7 *million Instructions Per Second - MIPS*, i.e., at the same rate as instructions were decoded. This was also possible with pipelined. CPU cache prefetching is discussed in a forthcoming book (Lotfi-Kamran and Sarbazi-Azad, 2022).

A CPU cache holding instructions and data was first introduced in conjunction with the IBM S/360 Model 85 computer in 1968 (Liptay, 1968). Given that the CPU cache determines the execution rate of instructions, a very fast determination whether a requested instruction or data is in the cache is required. In direct-mapped caches an appropriate number of low-order bits beyond the bits to specify locations in the cache blocks, are used to map a data block in main memory to a cache frame. The high order bits are then compared for a match. A hardware circuit with a few flip-flops can be used to implement the *Least Recently Used - LRU* policy in set associative caches. In fully-associative caches a block can be mapped into any frame in the cache, while in set-associative cache blocks can be mapped to one if m sets using lower order bits and then the using the higher order tag bits. CPU caches are discussed in most book on computer organization and architecture, some of which are listed in the Appendix. One third of (Jacob et al., 2008) is dedicated to caches and a 2-way set associative memory is illustrated in detail in (Harris and Harris, 2012).

<https://www.sciencedirect.com/topics/computer-science/set-associative-cache>

The IBM S/360 Model 85 had a sector cache with three different sizes 16, 24, or 32 KB, while the main memory size varied from 512K to 4096 KB (Liptay, 1968). Each one KB cache line consisted of sixteen 64B blocks. Frames in the caches could hold any line from main memory with an LRU policy. The identity of cache lines could be determined using a fully-associative paradigm. Because of the main memory width, four 16B transfers were required to transfer a 64B block. The sector referenced by the CPU was fetched first and the remaining sectors were fetched next in round-robin manner. Probability of finding referenced data was observed to be 96.8% for the traces under study and the performance with respect to an ideal system with a main memory with cache speed was 81%.

A detailed design of a sector cache is given in (Rothman and Smith, 1999), where the following points are noted.

1. A sector would typically be evicted from the cache before all of its blocks are loaded, and thus a large fraction of the cache capacity would remain unused.
2. The only way to have a very large cache capacity with a relatively small number of tag bits is to make the cache lines very large, but the problem in that case is that every miss requires that a large block be fetched in its entirety.
3. With a sector cache, it is possible to fetch only portions of a line (blocks), and thus the time to handle a miss and the bus traffic can both be significantly reduced.

Although it is likely that sector caches will have higher miss ratios than normal caches, there is the possibility that when timing is considered, the sector cache will be found to have better performance.

Various aspects of CPU cache memories are investigated in (Smith, 1982) under the following categories:

1. the cache fetch algorithm,
2. demand versus prefetch,
3. cache placement and replacement algorithms,
4. cache line size,
5. store-through versus copy-back updating of main memory,
6. cold-start versus warm-start miss rates,
7. multicache consistency,
8. the effect of I/O through the cache,
9. the behavior of split data/instruction caches,
10. cache size.

Modern microprocessors provide multiple cores on one chip, where each core can handle 2-4 threads or processes (Shen and Lipasti, 2013).

[https://en.wikipedia.org/wiki/Multithreading_\(computer_architecture\)](https://en.wikipedia.org/wiki/Multithreading_(computer_architecture))

Three multithreading options are:

1. coarse - run another thread if a thread stalls due to a cache miss,
2. interleaved by alternating in issuing instructions from different threads,
3. exploit parallelism available across multiple threads to decrease the waste associated with unused issue slots.

Modern CPUs have three levels of caches referred to as L_1 , L_2 , and L_3 . Not all systems use the inclusion property which mean that the contents of level L_i are cached in L_{i+1} . The inclusion through write-through is applied in some systems from L_2 to L_3 , which is much larger than L_2 . On board caches are available on most microprocessor chips, but given multicore chips an onboard L_2 cache is also provided.

Maintaining coherency between the caches and the data generated or consumed by I/O devices is challenging, with different solutions trading off hardware versus software complexity. The optimal approach for I/O data coherence depends on application and system characteristics and might require a combination of techniques (Hennessy and Patterson, 2017), (Culler et al., 1999).

<https://www.sciencedirect.com/topics/engineering/cache-coherence>

The IBM z14 mainframe has ten 2-way cores and z15 twelve cores with 2-way *Simultaneous MultiThreading* - SMT with separate instruction and data caches at L_1 and L_2 level and a shared L_3 cache.

[https://en.wikipedia.org/wiki/IBM_z14_\(microprocessor\)](https://en.wikipedia.org/wiki/IBM_z14_(microprocessor))

[https://en.wikipedia.org/wiki/IBM_z15_\(microprocessor\)](https://en.wikipedia.org/wiki/IBM_z15_(microprocessor))

The z-15 has a 128 KB I-L1 and 128 KB D-L1 caches per core and a 4 MB I-L2 and 4 MB D-L2 per core and an L_3 256 MB shared cache. It may be that the L_3 cache size in z15 was doubled based on measurements in z14.

Direct cache access for high bandwidth Network such as PCIe-express and 10 Gb/s Ethernet enable unprecedented levels of I/O bandwidths in mainstream platforms. In traditional architectures main memory latency alone can limit processors from matching 10 Gb/s inbound network I/O traffic. A platform-wide method called *Direct Cache Access* - DCA is proposed in (Huggahalli et al., 2005) to deliver inbound I/O data directly into processor caches. A significant reduction in memory latency and memory bandwidth for I/O intensive network I/O applications for benchmarks such as SPECWeb99 for performance evaluation of *World Wide Web* - WWW (retired 2005).

<https://www.spec.org/web99/>

Cache misses resulting in accesses to a lower level of the memory hierarchy entail the cache miss penalty. The access frequency at each storage device can be used to estimate the overall performance of a storage hierarchy. Expressions for average access time are given in (Hennessy and Patterson, 2017). In the case of two levels the *Average Memory Access Time* - AMAT is as follows.

$$AMAT = (1 - \text{Miss_rate}) \times \text{Hit_time} + \text{Miss_rate} \times (\text{Miss_penalty} + \text{Hit_time})$$

$$AMAT = \text{Hit_time} + \text{Miss_rate} \times \text{Miss_penalty}.$$

Multiprogramming has a detrimental effect on cache performance, since the cache misses by the next program overwrite the instructions and data of a previously executing program. Sharing of caches by several programs is investigated in (Thiébaud and Stone, 1987) by developing an analytical model for cache-reload transients and compares the model to observations based on several address traces. The cache-reload transient is the set of cache misses that occur when a process is reinitiated after being suspended temporarily.

Symmetric Multiprocessors - SMPs utilizes two or more identical processors connected to a shared main memory.

https://en.wikipedia.org/wiki/Symmetric_multiprocessing

Low-end SMP's consisted of microprocessors connected to a single bus to achieve more computing power at a low cost. Sequent computers used 2-30 microprocessors.

https://en.wikipedia.org/wiki/Sequent_Computer_Systems

Elxsi was a superminicomputer with a 25 nanosecond backpanel bus.

<https://en.wikipedia.org/wiki/Elxsi>

Two other companies in this category were Convex and Trilogy Systems.

https://en.wikipedia.org/wiki/Convex_Computer

Trilogy was a Silicon Valley company which became known as a "crater", i.e., consuming huge amounts of venture capital, to later implode, leaving nothing for its investors.

https://en.wikipedia.org/wiki/Trilogy_Systems

A recent attempt to implement wafer scale integration has been successful. Cerebras is discussed in some detail in Chapter 8.

<https://www.nextplatform.com/2020/10/23/is-there-a-wafer-scale-revolution-on-the-horizon/>

Solyndra solar panel manufacturer was another crater company, which received an over \$500 million loan from the Obama administration before going bust.

<https://www.washingtontimes.com/news/2015/aug/26/solyndra-misled-government-get-535-million-solar-p/>

Cache coherence in multiprocessors can be attained via snooping or directory based methods.

https://en.wikipedia.org/wiki/Cache_coherence

There is a performance degradation due to the cache coherence problem. Write propagation with snoopy protocols is implemented by either of the following two methods.

1. Write-invalidate: In case of a write operation to a cached location, the cache controller invalidates its own copy.

2. Write-update: When a write operation is observed to a location held by the cache, the controller updates its own copy.

Directory based methods are more robust and are used in high performance multiprocessors.

https://en.wikipedia.org/wiki/Directory-based_cache_coherence

A comprehensive textbook on parallel computer architecture hardware and software is (Culler et al., 1999).

Cache access in virtual memory systems

A page table in main memory provides the translation from virtual to physical addresses. Given that programs generate virtual memory addresses translation from virtual to physical addresses would double the number of main memory accesses. The inefficiency is obviated by taking advantage of the principle of locality using the *Translation Lookaside Buffer - TLB*, which is a cache for recent main memory references (Bhattacharjee and Lustig, 2018).

https://en.wikipedia.org/wiki/Translation_lookaside_buffer

TLB caches translate virtual to physical address TLBs are accessed very frequently and their misses are costly. There is the issue of whether the CPU cache can be addressed with virtual or physical addresses. A survey of techniques for architecting TLBs and address translation is reported in (Mittal, 2017).

<http://faculty.iitr.ac.in/~sparshfec/>

The following discussion is based on the following web page by Gene Cooperman at Northeastern U.

<https://course.ccs.neu.edu/com3200/parent/NOTES/cache-basics.html>

https://en.wikipedia.org/wiki/CPU_cache

Physically Indexed, Physically Tagged - PIPT caches use the physical address for both the index and the tag. While this is simple and avoids problems with aliasing, it is slow, as the physical address must be looked up first, which could involve a TLB miss and access to main memory, before that address can be looked up in the cache.

Virtually Indexed, Virtually Tagged - VIVT caches use the virtual address for both the index and the tag. This caching scheme can result in much faster lookups, since the *Memory Management Unit - MMU* does not need to determine the physical address for a given virtual address. VIVT suffers from aliasing problems, where different virtual addresses may refer to the same physical address. The result is that such addresses would be cached separately despite referring to the same memory, causing coherency problems. Although solutions to this problem exist they do not work for standard coherence protocols.

Another problem is homonyms, where the same virtual address maps to several different physical addresses. It is not possible to distinguish these mappings merely by looking at the virtual index itself, though potential solutions include: flushing the cache after a context switch, forcing address spaces to be non-overlapping, tagging the virtual address with an address space ID. Additionally, there is a problem that virtual-to-physical mappings can change, which would require flushing cache lines, as the VAs would no longer be valid. All these issues are absent if tags use physical addresses (VIPT).

Virtually Indexed, Physically Tagged - VIPT caches use the virtual address for the index and the physical address in the tag. The advantage with respect to PIPT is lower latency, as the cache line can be looked up in parallel with the TLB translation, however, the tag cannot be compared until the physical address is available. The advantage over VIVT is that since the tag has the physical address, the cache can detect homonyms. Theoretically, VIPT requires more tags bits because some of the index bits could differ between the virtual and physical addresses (for example bit 12 and above for 4 KB pages) and would have to be included both in the virtual index and in the physical tag. In practice this is not an issue because, in order to avoid coherency problems VIPT caches are designed to have no such index bits (e.g., by limiting the total number of bits for the index and the block offset to 12 for 4 KB pages); this limits the size of VIPT caches to the page size times the associativity of the cache.

Physically Indexed, Virtually Tagged - PIVT caches are often claimed in literature to be useless and non-existing. The MIPS R6000 implemented in extremely fast ECL technology is the sole implementation of this cache type. The R6000 is implemented in ECL, which is an extremely fast technology not suitable for large memories such as a TLB. The R6000 solves the issue by putting the TLB memory into a reserved part of the second-level cache having a tiny, high-speed TLB “slice” on chip. The cache is indexed by the physical address obtained from the TLB slice. However, since the TLB slice only translates those virtual address bits that are necessary to index the cache and does not use any tags, false cache hits may occur, which is solved by tagging with the virtual address.

1.18 Multiprogrammed computer systems

Large-scale mainframe computers used multiprogramming, which allowed the computer's CPU and main memory and disks be shared among several jobs. The CPU and disks are active resources shared in time-multiplexed basis, while programs simultaneously share main memory space which is a passive resource. Jobs in early systems belonged to the following categories: batch processing, timesharing, and *OnLine Transaction Processing - OLTP*.

Batch jobs tend to require large amounts of main memory and considerable CPU processing. The performance metric of interest for batch jobs is throughput, but turnaround or response time is more important from individual user's viewpoint. The response time can be improved by using the *Shortest Job First - SJF* or *Shortest Processing Time First - SPTF* scheduling. When preemptions are allowed there is the *Shortest Remaining Processing Time First - SRPT*. From a queueing theory viewpoint there is also SEPT and SERPT, where E stands for the expected value and R for remaining (Section 3.9 in (Kleinrock, 1976)). A practical method to implement SPT is for the user to specify CPU time based on previous runs.

BEST/1 is a capacity planning tool based on a *Queueing Network Model - QNM* for multiple workloads processed on a mainframes running IBM's MVS operating system (Buzen, 1978). BEST/1 was developed in 1970s at BGS cofounded by Jeff Buzen, a former Harvard U. professor, who developed a convolution algorithm for analyzing product-form closed QNMs (Buzen, 1973) as part of his PhD thesis (Buzen, 1971). BGS was bought by BMC for \$285M.

MVS and its extensions run on IBM mainframes. Hardware measurements are used to obtain device utilizations, while software tools measure job service demands at devices, job throughputs and mean response times. The latter are also used for model validation.

The workloads and their priorities used by capacity planning packages BEST/1 (Buzen, 1978) and MAP described in (Lazowska et al., 1984).

Priority 1: Online Txn Processing - OLTP: In addition to txn service demands or loadings, the arrival rate of txns is required for performance analysis. Several types of txns are considered: trivial, short, intermediate, long. Txns start execution at the highest execution at a lower priority,

Priority 2: Time Sharing: IBM provides the *Time-Sharing Option - TSO* which in addition to service demands requires the number of active users and their think time. Foreground/Background - FB scheduling is used for timesharing requests, which after receiving each quantum of CPU time are moved to a lower priority queue and stay at the lowest priority queue as batch jobs until completion.

Priority 3: Batch: In addition to service demands the analysis requires the MPL - multiprogramming level or the degree of concurrency. The MPL is constrained by the memory requirements of batch jobs.

Alex Birman at IBM TJWRC developed a QNM package called Argus for capacity planning of IBM mainframes in 1980s. but Argus was withdrawn by IBM even after its manual was published, since as computer manufacturer IBM would be responsible to deliver the performance predicted by Argus.

Logical partitioning was a form of hardware-based virtualization, which was introduced by Amdahl Corp. under the name *Multi-Domain Facility - MDF* in 1984, IBM's version of MDF was *Logical PARTitioning - LPAR* in 1988, and Hitachi's version under the name *Multiple Logical Partition Feature - MPLF*.

https://en.wikipedia.org/wiki/Logical_partition

Logical partitioning is discussed in Section 9.3 in (Smith and Nair, 2005).

<https://books.google.com/books?id=JPhQw41vD2MC&pg=PA458#v=onepage&q&f=false>

IBM in 2018 announced software for IBM Z Decision Support for Capacity Planning V2.1.0 accelerates capacity planning and modeling processes, optimizing resources and improving service levels.

https://www.ibm.com/common/ssi/ShowDoc.wss?docURL=/common/ssi/rep_ca/8/897/ENUS218-528/index.html

1.19 Timesharing systems

Timesharing systems share the computational power of a single computer by users of dumb terminals, thin clients, and personal computers.

https://en.wikipedia.org/wiki/Thin_client

A recent paper on serverless computing is (Castro et al., 2019), which discusses variants of cloud computing.

Edge computing is a distributed computing paradigm that brings computation and data storage closer to the location where it is needed to improve response times and save bandwidth. It is a topology rather than a technology.

https://en.wikipedia.org/wiki/Edge_computing

Early timesharing systems are discussed in two early books (Watson, 2007), (Wilkes, 1975) and below.

<http://www.cs.cornell.edu/wya/AcademicComputing/text/earlytimesharing.html>

Notable timesharing systems are listed below.

<https://en.wikipedia.org/wiki/Time-sharing>

CTSS - *Compatible Time Sharing System* became operational on an IBM 709 vacuum tube computer at MIT in 1961.

https://en.wikipedia.org/wiki/Compatible_Time-Sharing_System

One memory bank was dedicated to user programs and the other to the OS. The address space of the running program was saved on the drum, while the address space of a scheduled job was loaded into main memory to be served for another time quantum. The 709 drum which can load the main memory in a quarter of a second, as the system switched from one user to another following the CPU time quantum of 200 msec.

Berkeley Timesharing System was a pioneering time-sharing OS implemented between 1964 and 1967 at the UC Berkeley. It was designed as part of Project Genie and marketed by *Scientific Data Systems* - SDS 940 computer system.

https://en.wikipedia.org/wiki/Scientific_Data_Systems

It was the first commercial time-sharing which allowed general-purpose user programming.

https://en.wikipedia.org/wiki/Berkeley_Timesharing_System

Hardware projects at Berkeley are summarized here.

<https://people.eecs.berkeley.edu/~pattsrn/Arch/prototypes2.html>

In his PhD thesis Alan Scherr used a *Machine-Repairman Model* - MRM to predict the throughput in interactions per second and mean user response time in CTSS (Scherr, 1967). An early history of CTSS (1961-73) appears here.

<https://multicians.org/thvv/compatible-time-sharing-system.pdf>

Response time in timesharing systems is important, since subsecond response times according to (Thadhani, 1981) has the added benefit of reducing user think times and this results in shortened time per interaction and reduction in the completion time of the task at hand.

Machine repairman model applied to timesharing computer systems

The analysis of MRM using the M/M/1/M queueing model appears in most books on queueing theory (Kleinrock, 1975, 1976). Given N active users with think time Z , a server that can process requests at a rate $\mu = 1/\bar{x}$, where $\bar{x}$ is the mean service time of requests per interaction.

Given k requests at the server the state probabilities of the system can be determined by solving the following state equilibrium equations and $\sum_{k=0}^M p_k = 1$.

$$(M - k)\lambda p_k = \mu p_{k-1}, k \geq 1 \text{ with } p_0 = \left[\sum_{k=0}^M \left(\frac{\lambda}{\mu}\right)^k \frac{M!}{(M - k)!} \right]^{-1}$$

Using Little's result (Kleinrock, 1975) the mean request response R satisfies $N = (Z + R)\lambda$, where the system throughput is the rate at which the server completes requests when it is not idle.

$$\lambda = \mu(1 - p_0),$$

The mean response time follows from an application of Little's result. $N = (R + Z)\lambda$, which yields the mean user response time:

$$R = \frac{N}{\mu(1 - p_0)} - Z.$$

R can be approximated by two intersecting lines:

(1) for low utilizations $R \approx \bar{x}$, i.e., R remains almost flat,

(2) at heavier loads with $p_0 \approx 0$: $R \approx N\bar{x} - Z$.

The two lines intersect at the *saturation point*: $N^* = 1 + Z/\bar{x}$, which is the maximum number of users that can be accommodated with little performance (response time) degradation due to queueing.

Upon joining IBM Scherr got involved in developing IBM's *Time Sharing Option* - TSO.

https://en.wikipedia.org/wiki/Time_Sharing_Option

He was a member of a 1971 task force that created the *Multiple Virtual Storage* - MVS OS, whose variants are used today

to run IBM mainframes.

<https://en.wikipedia.org/wiki/MVS>

<https://en.wikipedia.org/wiki/Z/OS>

Scherr's accomplishments at IBM are listed below.

<https://history.computer.org/pioneers/scherr.html>

MTS - Michigan Terminal System developed at U. Michigan in 1967 was based on an IBM S/360-67.

https://en.wikipedia.org/wiki/Michigan_Terminal_System

TOPS-10 System (Timesharing Operating System-10) was launched in 1967 for the DEC PDP-10 computer.

<https://en.wikipedia.org/wiki/TOPS-10>

The DECSYSTEM-20 was a 36-bit DEC PDP-10 mainframe computer running the TOPS-20 operating system

<https://en.wikipedia.org/wiki/DECSYSTEM-20>

Its development had started in 1973 based on TENEX an OS for the PDP-10. developed at *Bolt Beranek & Newman - BBN* Inc. in Cambridge, MA (Bobrow et al., 1972).

<https://opost.com/tenex/hbook.html>

Many factors were involved in the decision to start TOPS-20 development, but primarily DEC wanted a new virtual memory OS for the KL10 processor. The TOPS-20 was shipped in January 1976.

<https://en.wikipedia.org/wiki/TOPS-20>

Later DEC developed a *Virtual Memory System - VMS* for *Virtual Address eXtension - VAX-780* computers.

<https://en.wikipedia.org/wiki/VAX>

The VAX-cluster computer with an FDDI interconnect was highly fault-tolerant, with an uptime exceeding sixteen years according to (Kronenberg et al., 1986), since a processor could be repaired, while the others were available to users.

<https://en.wikipedia.org/wiki/VMScluster>

In early 1980s USC provided about forty terminals to access its PDP-10 with probably a TOPS-20 OS. At peak hours students queued to get access to terminals. The waiting time for students needed to be determined by a higher level queueing system, where the service time was the time it took for students to complete their task and release the terminal. The situation could be improved by allowing a time maximum, which is a scheme for sharing terminals provided at local libraries.

UCLA's timesharing system called *University Records System Access - URSA* was used for student and administrative use and ran an IBM 360/91. The URSA major and minor constellations stand for large and small bears.

<https://www.constellation-guide.com/constellation-list/ursa-major-constellation/>

UCLA's mascot is a bear and its students athletic teams are called Bruins. John Wooden was UCLA's legendary basketball coach, known as the *Wizard of Westwood*, whose team won national championships in 10 out of 12 years in 1960s and 70s.

https://en.wikipedia.org/wiki/John_Wooden

1.20 Mean response with FCFS and processor-sharing scheduling

Early time-sharing systems were multiprogrammed using time-multiplexing at a fine time quantum (q). With $q \rightarrow 0$ we have a *Processor Sharing - PS* system (Kleinrock, 1976). The response time of a job with service time x and Poisson arrivals with rate λ is $R_{PS}(x) = x/(1 - \rho)$, where $\rho = \lambda\bar{x} < 1$ is the server utilization. Given that the service *probability density function - pdf* is $f_X(x)$ the mean response time is:

$$R_{PS} = \int_0^{\infty} R_{PS}(x) f_X(x) dx = \frac{\bar{x}}{1 - \rho}.$$

The wasted rather than waiting time is then $W_{PS} = R_{PS}(x) - \bar{x} = \rho\bar{x}/(1 - \rho)$.

With *First-Come, First-Serve - FCFS* scheduling the mean response time is given as the sum of mean service time $\bar{x}$ and mean waiting time given by the Pollaczek-Khintchine formula for M/G/1 queues (Kleinrock, 1975). For *Poisson Arrivals See Time Averages - PASTA* the mean waiting time incurred by an arriving request is the sum of the mean waiting time in the queue, which is the product of mean queue length ($\bar{N}_q$) and mean service time and the delay incurred by the customer in service provided that the server is busy. This delay is the residual service time of the request in service $\bar{x}_r = \bar{x}^2/(2\bar{x})$ (Kleinrock, 1975), where the i^{th} moment of service time is $\bar{x}^i = \int_{x=0}^{\infty} x^i f_X(x) dx$. It follows that:

$$W = \bar{N}_q \bar{x} + \rho \bar{x}_r,$$

noting that $\bar{N}_q = \lambda W$ and simplifying we obtain the *Pollaczek-Khintchine - P.K.* mean waiting time formula.

$$W_{FCFS} = \frac{\lambda \bar{x}^2}{2(1-\rho)} = \frac{\rho \bar{x}(1+c^2)}{2(1-\rho)},$$

where given the variance $\sigma_x^2 = \bar{x}^2 - (\bar{x})^2$, the coefficient of variance is $c^2 = \sigma_x^2 / (\bar{x})^2$. The mean response times is $R = E_{M/G/1} + \bar{x}$. It can be shown that PS improves mean response time with respect to FCFS for $c^2 > 1$: $R_{PS} < R_{FCFS}$. It was shown in early studies that the CPU burst time has a hyperexponential distribution with $c > 1$:

$$b(x) = p\mu_1 e^{-\mu_1 x} + (1-p)\mu_2 e^{-\mu_2 x} \text{ with } 0 < p < 1 \text{ and } \mu_1 \neq \mu_2.$$

This is the reason that PS is recommended in time-sharing systems.

We next discuss approximating closed M/G/1//N queues with an infinite source M/G/1 queue following the discussion in (Buzen and Goldberg, 1974).

In an M/G/1//N queue there are N sources with mean think time $1/\lambda$ which generate requests for a single server, whose i^{th} moment of service time is $\bar{x}^i$ its *probability density function - pdf* $b(x)$ and its *Laplace Stiltjes Transform - LST*

$$\mathcal{B}^*(s) = \int_0^\infty e^{-sx} b(x) dx.$$

Mean response time R_N is obtained by Lajos Takacs and the derivation is given in Takagi's Queueing Systems, volume II.

$$R_N = \frac{N}{\mu} - \frac{1}{\lambda} + [\lambda \sum_{n=0}^{N-1} \binom{N-n}{n} \frac{1}{H_n}]^{-1}$$

$H_0 = 1$ and $H_n, 1 \leq n \leq N$ is given by

$$H_n = \prod_{i=1}^n \frac{\mathcal{B}^*(i\lambda)}{1 - \mathcal{B}^*(i\lambda)}.$$

M/G/1 with infinite sources overestimates the mean response time for the same throughput. Table 1.5 provides the minimum number of sources (N) to attain a 5% error level for four distributions with mean one: constant, Erlang-2, exponential, and Hyperexponential-2 (with equal branching probabilities and branch means 0.5 and 1.5).

TABLE 1.5 Minimum number of sources N to attain 5% error when using the P.K. M/G/1 formula.

Distr./ ρ	.1	.2	.3	.4	.5	.6	.7	.8
Constant	2	3	6	10	17	32	61	160
Erlang-2	2	5	9	16	27	49	101	200+
Exponent'l	3	6	12	21	38	70	142	200+
Hyperexp.	4	8	16	27	48	93	188	200+

The following approximation for M/G/1//N has been proposed.

$$\frac{R_{M/G/1} - R_{M/G/1//N}}{R_{M/G/1//N}} = (1 + c_X^2) \frac{R_{M/M/1} - R_{M/M/1//N}}{R_{M/M/1//N}}$$

Even lower mean response times for short are attainable by multilevel feedback queueing system developed in conjunction with the MIT's MULTICS OS developed by Fernando Corbato provides an improvement over PS.

https://en.wikipedia.org/wiki/Multilevel_feedback_queue

Jobs initially enter the queue at level one and join level two which has a lower priority than the first level, and so on, the longer lower priority jobs do not interfere with shorter jobs. *MultiLevel Feedback Queues - MLFQs* are discussed in the OS book (Arpaci-Dusseau and Arpaci-Dusseau, 2015).

<http://pages.cs.wisc.edu/~remzi/OSTEP/cpu-sched-mlfq.pdf>

Feedback queues are analyzed in (Kleinrock, 1976), (Nuyens and Wierman, 2008), (Harchol-Balter, 2013).

Shortest Attained Service time First - SASF is an extreme multilevel feedback queueing system with quantum size $q \rightarrow 0$ and an infinite number of levels. The mean waiting time as a function of service time is given as:

$$W_{SASF(t)} = \frac{t\rho_{<t}}{1 - \rho_{<t}} + \frac{W_{FCFS_{<t}}}{1 - \rho_t}.$$

The service time PDF is $B(t)$ and the service time is truncated at t .

$$\rho_{<t} = \lambda \int_0^t \tau dB(\tau) + t[1 - B(t)].$$

The mean waiting time versus time is given by Fig. 7.13 on p. 289 which shows that SASF outperforms FCFS and SJF for very short service times. R. R. Muntz, Resource allocation, in *Software Systems Principles: A Survey*, Peter Freeman, Science Research Associates, 1975 (Eq. for $W_{SASF(<t)}$ appears as kindly corrected by Prof. Mor Harchol-Balter at CMU). Harchol noted that the performance of SASF is actually very similar to *Shortest-Remaining-Processing-Time - SRPT* (Kleinrock, 1976), which is always optimal, but requires knowledge of job sizes.

1.21 Analysis of open and closed queueing network models

To motivate the discussion we first consider an open system with jobs arriving at rate Λ . Job service demands or loadings at the N nodes of the QNM denoted by D_n , $1 \leq n \leq N$ are the product of the mean number of times (v_n) a job visits node n and the mean service time per visit ($\bar{x}_n$), so that $D_n = v_n \bar{x}_n$, $1 \leq n \leq N$. We first present the analysis of a single node.

Analysis of an M/M/1 queueing system

M/M/1 is a single server queue with exponential service times (with rate μ) and Poisson arrivals (with rate λ), which are served from a FCFS queue. The system can be modeled as birth-death process, where the state S_k has k requests at the system (queue plus server). The state transitions corresponding to births and deaths for $k \geq 0$ are as follows:

$$S_k \xrightarrow{\lambda} S_{k+1}, \quad S_{k+1} \xrightarrow{\mu} S_k, \quad k \geq 0.$$

The steady state transition probability equations with $\rho = \lambda/\mu < 1$ denote the server utilization factor, which due to Little's result (Kleinrock, 1975) is the mean number of request at the server. This is also the fraction of time the server is busy, since the probability of no requests at the server is shown to be $p_0 = 1 - \rho$, which holds for other single-server queues.

$$\lambda p_k = \mu p_{k+1} \text{ or } p_{k+1} = \rho p_k, \quad k \geq 0 \text{ or } p_n = \rho^k p_0, \quad k \geq 1.$$

where $\rho = \lambda/\mu$ is the server utilization factor according to Little's result (Kleinrock, 1975). The sum of probabilities converges for $\rho < 1$, leading to $p_0 = 1 - \rho$ and hence p_k follows the geometric distribution.

$$p_k = (1 - \rho)\rho^k, \quad k \geq 0,$$

The mean number of requests in the system is $\bar{k} = \rho/(1 - \rho)$. The mean system response time is $R = \bar{k}/\lambda = \bar{x}/(1 - \rho)$, i.e., R grows in proportion to $(1 - \rho)^{-1}$.

We next consider a QNM with N single server nodes, where requests are routed probabilistically from node to node with FCFS scheduling. It can be shown that even when there is feedback and arrivals are not Poisson, the system state probability according to Jackson's theorem (Kleinrock, 1975) can be expressed as the product of state probabilities of individual nodes, as if they have Poisson arrivals.

https://en.wikipedia.org/wiki/Jackson_network

Given that v_n is the mean number of visits to node n and $\bar{x}_n$ is the mean service time per visit, then the service demand at node n is $D_n = v_n \bar{x}_n$. Given that the utilization factors of the servers at the nodes are $\rho_n = \Lambda D_n < 1$, $1 \leq n \leq N$ the

system state probability is given by:

$$P[\underline{k}] = \prod_{n=1}^N (1 - \rho_n) \rho_n^{k_n},$$

where $\underline{k} = (k_1, k_2, \dots, k_N)$. This is because each node behaves as an M/M/1 queueing system with arrival rate Λ and service rate $\mu_n = 1/D_n$. The mean overall response time in the open QNM is then

$$R_{overall} = \sum_{n=1}^N R_n \text{ where } R_n = \frac{\bar{x}_n}{1 - \rho_n}.$$

A fixed number of jobs in a closed QNM is achieved with an infinite backlog of batch jobs resulting in a completed job being immediately replaced by another job. The system state of a closed QNM with a total number of K jobs and k_n jobs at device n is given as:

$$\underline{k} = (k_1, k_2, \dots, k_N) \text{ and } \sum_{n=1}^N k_n = K.$$

The solution to closed networks is obtained by dividing the state probabilities by the normalization constant $G(K)$.

$$G(K) = \sum_{\sum_{n=1}^N k_n = K} (D_1^{k_1} \times D_2^{k_2} \dots D_N^{k_N}).$$

The probability that the system is at state $\underline{k}$ is then:

$$P[\underline{k}] = \frac{1}{G(K)} (D_1^{k_1} \times D_2^{k_2} \dots D_N^{k_N}).$$

The number of terms in the summation is $\binom{N+K-1}{N-1}$ (Kleinrock, 1975), e.g., 1001 for $N = 5$ and $K = 10$.

Analysis of a closed QNM with K jobs and N single-server stations

The throughput $T(K)$ can be computed using Buzen's convolution algorithm (Buzen, 1971), (Williams and Bhandiwad, 1976), (Thomasian and Nadji, 1981a), (Lavenberg, 1983).

/* Computing $G(K)$ in a QNM with K jobs and N nodes */

$G(0) = 1$ and $G(k) = 0$ for $1 \leq k \leq K$,

for $n = 1$ to N do {

for $k = 1$ to K do {

$G(k) = G(k) + D_n G(k - 1)$ }

$T(K) = G(K - 1) / G(K)$ system throughput.

$R(K) = K / T(K)$: mean residence time due to Little's result.

for $n = 1$ to N do

$U_n(K) = D_n T(K)$, utilization of the n^{th} server.

$Q_n = \sum_{k=1}^K D_n^k G(K - k) / G(K)$ mean queue-length at n^{th} queue.

end

In the case of state-dependent servers, the probability that device n is in a state with k_n jobs is: $F_n(k_n) = D_n^{k_n} / \prod_{k=1}^{k_n} \alpha(k_n)$. In the case of a multiserver queue with k_{max} servers

$$\alpha(k) = \begin{cases} k & \text{for } k \leq k_{max} \\ k_{max} & \text{for } k \geq k_{max}. \end{cases}$$

For stations with state dependent servers $G(k)$ is computed using the convolution formula

$$G_n(k) = \sum_{i=0}^k G_n(k-i)F_n(i).$$

In the case of delay servers there is no queueing, since there are as many servers as jobs, so it is as if there are an infinite number of servers: $\alpha(k) = k, k \geq 1$.

The number of terms increases rapidly with the number of job classes (C), which differ in their service demands, i.e., K_c jobs in class c the number of states is (Lavenberg, 1983):

$$\prod_{c=1}^C \binom{K_c + N - 1}{N - 1}.$$

In the case of a balanced QNM with $D_n = D, 1 \leq n \leq N$

$$T(K) = \frac{G(K-1)}{G(K)} = \frac{\binom{K+N-2}{K-1} D^{K-1}}{\binom{K+N-1}{K} D^K} = \frac{K}{(K+N-1)D}. \quad (1.2)$$

The mean job residence time is $R(K) = K/T(K) = (K+N-1)D$, which increases linearly with M .

This result can be shown easily using *Mean Value Analysis - MVA* by noting that in a closed balanced QNM with N single-server nodes and K jobs the number of jobs observed by an arrival at the stations is $N-1$ jobs, so that in a balance QNM: $Q(K) = (K-1)/N$ (Lazowska et al., 1984). The mean response time at the node is

$$r(K) = D(1 + \frac{K-1}{N}).$$

The mean residence time in the system with N nodes is

$$R(M) = Nr(k) = D(N+K-1).$$

The throughput is then $T(K) = M/R(M) = M/(M+N-1)D$.

Multiprogramming was introduced to improve CPU utilization and hence system throughput by taking advantage of CPU-I/O overlap. Multiple jobs are loaded into main memory and when a job makes an I/O request the CPU switches from that job to another ready-to-run job via context switching. With the advent of fast secondary memories context switching may not be required.

Multitasking is a technique to speedup up the execution time of a single program provided it exhibits parallelism. An analysis of multitasking in the context of a QNM is provided in (Thomasian and Bay, 1986). Such a feature is provided by the PL/I programming language. Even in a uniprocessor speedup is possible due to CPU/I/O overlap. Multitasking in UNIX is accomplished using the FORK statement.

[https://en.wikipedia.org/wiki/Fork_\(system_call\)](https://en.wikipedia.org/wiki/Fork_(system_call))

1.22 Bottleneck analysis and balanced job bounds

Asymptotic Job Bounds - AJB presented in (Lazowska et al., 1984) is based on bottleneck analysis described in (Muntz and Wong, 1974).

Consider a QNM with N nodes with m_n servers at node n . There is a single job type with service demands $D_n, 1 \leq n \leq N$ at the nodes. The maximum system throughput at node n is achieved when its utilization increases to one, so that the maximum arrival rate or throughput sustained by the node is $\lambda_n^{max} = m_n/D_n$.

In the case of a closed QNM with K jobs the system throughput $T(K)$ increases with K . Given that $\rho_n = T(M)D_n$, the maximum system throughput is the minimum of maximum throughputs at the N nodes:

$$T_{max} = \min \left(\frac{m_1}{D_1}, \frac{m_2}{D_2}, \dots, \frac{m_N}{D_N} \right).$$

At low MPL there is little contention among the jobs at the devices of the computer system and the system throughput is the inverse of the sum of service demands or $T(1) = 1/\sum_{n=1}^N D_n$, so that in a plot of $t(k)$ versus k at low MPLs $t(k)$ is obtained by connecting $(0, 0)$ to $(1, T(1))$. The system throughput which is concave lies the asymptotic and balanced job bound which is obtained below.

The Arrival Theorem in closed QNMs is the basis of *Mean Value Analysis - MVA*, which states that a request at each node encounters a mean queue length as if there is one less job in the closed system (Lavenberg, 1983), (Lazowska et al., 1984). With multiple job classes one less job in the job class of the arriving job.

In the case of balanced loads the mean queue-length at each node is $(K - 1)/N$, so that the mean residence time at the node is $r(K) = D[1 + (K - 1)/N]$, which multiplied by N yields the mean job residence time $R(K) = D(N + K - 1)$. $R(K)$ increases linearly with M and the system throughput following Little's result (Kleinrock, 1975) is $T(K) = K/R(K) = K/[(K + N - 1)D]$. $T(K)$ is a concave function of K and as $K \rightarrow \infty : T(\infty) = T_{max} = 1/D$.

In a QNM with unequal service demands an upper bound to system throughput referred to as *Balanced Job Bound - BJB* is obtained in (Zahorjan et al., 1982), (Lazowska et al., 1984), where $D_{avg} = \sum_{n=1}^N D_n/N$ and it is proven in (Zahorjan et al., 1982) that this is indeed an upper bound, but it should be noted that this bound only applies to single server queues and that $T(K) < D_{max}$ for larger values of K , hence:

$$T(K) = \min\left(\frac{1}{D_{max}}, \frac{K}{K + N - 1} \frac{1}{D_{avg}}\right).$$

Given, e.g., $D_{max} = 2D_{avg}$ the crossover point is $K \geq N - 1$.

An approximate method to estimate the throughput of a closed QNMs with unequal service demands using Taylor series expansion with respect to the mean service demand with one and two job classes is reported in (Thomasian and Nadji, 1981b) and (Nadji and Thomasian, 1984).

Given the throughput characteristic: $T(k), k \geq 1$, we can analyze the performance of the system processing jobs with Poisson arrivals, but can take into account the maximum MPL due to memory size constraints by setting $T(k) = T(K_{max})$ for $k \geq K_{max}$, where $K_{max}M$ is the maximum MPL.

The system can be analyzed using a birth-death model with birth rate λ and death rate $T(k)$ provided $\lambda < T(K_{max})$ (Kleinrock, 1975). This is an example of hierarchical modeling, where results from a lower model are used in a higher level model (Lazowska et al., 1984).

Computer system performance with job arrivals and limited MPL

The job arrival rate is λ and the throughput characteristics $T(k), 1 \leq k \leq K_{max}$, where K_{max} is the maximum MPL and $\lambda < T(K_{max})$.

```
P = S = Q = 0, p0 = 1, ε = 10-6. /* Initializations */
for k > 1 do {
  pk = pk-1λ / T(k),
  P += pk until pk < ε, /*sum probabilities for normalization */
  S += k × pk /* mean number of jobs in the system */
  R = S / (P × λ). /* mean response time */
  Q += max(k - Kmax, 0) × pk /* mean number of enqueued jobs */
  W = Q / (Pλ). /* mean waiting in memory queue */ }
```

1.23 Performance analyses of I/O subsystems

The material in this section is mostly of historical interest, because of the transition from variable block size *Extended Count Key Data - ECKD* disk organizations to fixed sized sectors and the introduction of RAID. The techniques reported here might be applicable to the analysis of other queueing systems, however.

Jeff Buzen PhD thesis at Harvard U. was the efficient analysis of closed product-form *Queueing Network Models - QNMs*, more specifically the *Central Server Model - CSM* (Buzen, 1971, 1973). The central server is the CPU and the disks peripheral processors. Jobs after CPU processing access disks before resuming additional CPU processing. In the case of closed QNMs, a job complete at the CPU is immediately replaced by a new job, since it is assumed that there is an infinite backlog of batch jobs.

Buzen's analysis uses the convolution algorithm which allowed single- and multi- and state-dependent servers, where the overall service rate increases sublinearly. The categories of nodes which can be included in product-form QNMs was extended by the *Baskett, Chandy, Muntz, Palacios - BCMP* theorem, which extends Jackson's theorem, allowing four types of nodes with multiple classes of jobs (Baskett et al., 1975), (Lavenberg, 1983), (Lazowska et al., 1984):

- (1) Single and multiservers or state-dependent queues with the same negative exponential service times and FCFS scheduling.
 - (2) Processor-Sharing.
 - (3) Last-Come, First-Served Preemptive resume queue.
 - (4) Infinite or no queueing delay servers.
- Options (2:4) allow general service times.

Jobs arrivals to node i (if any) are Poisson and transition node i to node j with probability $p_{i,j}$ or leave the network with probability $1 - \sum_{j=1}^N p_{i,j} > 0$, where N is the number of QNM node numbered 1 : N .

Buzen's convolution algorithm was extended from a single to multiple job classes in (Reiser and Kobayashi, 1975). The use of generating functions to derive computational algorithms was presented in (Williams and Bhandiwad, 1976) and (Thomasian and Nadji, 1981a).

At a later point Jeff Buzen et al. cofounded BGS, which introduced the BEST/1 package for analyzing QNMs for capacity planning IBM mainframes and its *Plug Compatible Machines - PCMs* running the MVS operating system (Buzen, 1978). A software tool called Crystal was developed to extract modeling parameters such as job service demands using *Software Measurement Facility - SMF* and *Resource Measurement Facility - RMF* for hardware utilization (Buzen, 1978).

The *Mean Value Analysis - MVA* method for analyzing product-form QNMs was developed ten years later at IBM Research by Martin Reiser (rip) and Steven Lavenberg (rip) (Reiser and Lavenberg, 1980). Martin Reiser takes most of the credit for MVA in (Reiser, 2000).

MVA was used by Ed Lazowska (U. Washington) and John Zahorjan (U. Washington), G. Scott Graham (U. Toronto, rip), and Ken Sevcik (U. Toronto, rip) to develop the MAP performance analysis tool, which similarly to BEST/1 was developed for IBM mainframes (Lazowska et al., 1984). The I/O subsystem for the I/O subsystem of an IBM mainframe in 1980s is given by Figure 10.1 in Chapter 10 of *Quantitative System Performance - QSP* book (Lazowska et al., 1984). <https://homes.cs.washington.edu/~lazowska/qsp/>

The CPU sends commands via two I/O channels, which are connected to two out of three disk controllers, there are five *heads of strings - hos* for I/O buses with multiple disks attached to each bus. This configuration and related analysis is only of historical interest at this point.

Early queueing analysis of disk systems such as (Wilhelm, 1977) took advantage of the commonality of disk models, such as channel interference and concurrent seeks. IBM 2314 and 3330 disks were considered in this study. Comparison with simulation results showed the approximation to be quite accurate. Another analysis of a realistic disk model is reported in (Hunter and Jacobs, 1982).

A general model of disk storage systems equipped with *Rotational Position Sensing - RPS* is reported in (Zahorjan et al., 1978). Rotations are missed if two disks which do not have buffers are ready to transmit over the same channel simultaneously. It is shown in this study that the number of missed rotations waiting for the channel is not geometrically distributed and an approximate formula for the mean and variance of the number of rotations is derived.

The analysis model of an I/O subsystem in which devices are accessed from multiple CPUs and/or via alternative channel and control unit paths is reported (Bard, 1980). The model estimates access response times, given access rates for all CPU-device combinations. Central to the model's algorithm is the estimation of the RPS miss probabilities by means of the *Maximum Entropy Principle - MEP*.

https://en.wikipedia.org/wiki/Principle_of_maximum_entropy

Transmission networks are analyzed in (Bard, 1982) using the MEP again to determine whether a path to a given node is free when that node is ready to transmit. Both dynamic and fixed path selection mechanisms are treated and approximate methods for large networks are proposed. Consider disk k , which was issued an I/O request, it will be ready to reconnect to transmit data with probability: P_k [path busy] = P_k [hos busy] + P_k [hos free & controller busy] + P_k [hos free & controller free & channel busy]

The basic model of a block Amdahl mainframes was developed in at Amdahl Corp. in (Brandwajn, 1981). The goal was to represent the missed reconnection effect together with its influence on the overall performance of a DASD subsystem. The approach taken is that of step-wise decomposition and analysis with a fixed number of users. Computational algorithms are provided. The use of the basic model as a building block for more complex systems is illustrated in the context of a symmetrical two-CPU system with shared disks.

The expected disk performance improvement due to *Dynamic Path Reconnection - DPR* provided by *eXtended Architecture - XA* of IBM mainframes is investigated in (Brandwajn, 1983a). This feature provides the ability for a disk access to be resumed and completed on a path different from the initiating path. Considered are two double pathing connection schemes:

- (1) switched substrings (IBM 3380 Dynamic Path Selection).
- (2) Dual ported devices (Amdahl 6280 Dynamic Performance Pathing).

A queueing model is used to obtain missed revolutions due to busy transfer paths. The numerical results indicate a substantial reduction in the average I/O time due to DPR.

A throughput-driven approach to the performance analysis of DASD with a block-multiplexor channel system with multiple transfer paths is presented in (Brandwajn, 1983b). The approach relies on a step-wise analysis of shared components within the data transfer path: head of string, control unit, channel. The service time of an I/O request is obtained by analyzing each blocking point separately using a classical loss-system model. The method is relatively simple and applicable to a large number of configurations.

Expressions for RPS reconnect delays in three basic cases: (1) single path, (2) multiple path with static reconnect, (3) multiple path with dynamic reconnect are presented in (Buzen and Shum, 1987).

Elizabeth Shriver (HP, Bell Labs, rip), Arif Merchant (IBM, NEC, HP, Google), John Wilkes (HP, Google) at HP

An analytic model for disk drives, which takes into account request sizes, spatial locality, readahead and request re-ordering is reported in (Shriver et al., 1998). The analysis is carried out using disk drive components (queues, caches, and the disk) and a workload transformation technique for composing them.

1.24 Vector supercomputers

Pipelined vector supercomputers appeared in 1970s and reached maturity in 1980s.

https://en.wikipedia.org/wiki/Category:Vector_supercomputers

The CDC *S*Trings of binary digits that made up ARray - STAR-100.

https://en.wikipedia.org/wiki/CDC_STAR-100

and Texas Instrument's - TI's Advanced Scientific Computer - ASC

https://en.wikipedia.org/wiki/TI_Advanced_Scientific_Computer

were presented at IEEE COMPCON'72 in San Francisco. The TI ASC had a matrix multiply instruction, but given Fortran loops implementing a matrix multiply it is doubtful there was any compiler that could invoke the instruction.

Two books dedicated to pipelined computers are (Kogge, 1981), (Cragon, 1996) The reader is also referred to Appendix G in (Hennessy and Patterson, 2017) by Krste Asanovic.

CDC had the world's fastest computers the CDC 6600 and 7600, but when the development of the CDC 8600 ran into problems in 1972,

Seymour Cray left CDC to form Cray Research, which in 1976 produced the Cray-1, worlds fastest and most commercially successful supercomputer of the time (Cray Research, 1976).

https://en.wikipedia.org/wiki/CDC_8600

<https://en.wikipedia.org/wiki/Cray-1>

The Cray-1 provides eight 64 word register files, which need to be preloaded before their data is processed. Note similarity to RISC processors which only operate on data preloaded into registers. The Cray supercomputer was the first to deliver 20-60 MFLOPS (Russell, 1978).

A novel feature of Cray-1 was chaining, e.g., $d_i \leftarrow a_i \times b_i + c_i$, $1 \leq i \leq 64$ is computed by sending temporary results from the pipelined multiplier: $t_i \leftarrow a_i \times b_i$ to a pipelined adder to be added to c_i , rather than storing temporary results in a register file. This is possible because pipelining proceeds with the same cycle time. Chaining has the advantage of saving main memory bandwidth, which is a limited resource in vector supercomputers, since no register files would be required for temporary results. Note that the compiler for IBM S/360 Model 91 computer dedicated registers for temporary results, but they were bypassing during execution. A similar paradigm was possibly followed in Cray-1, so there was no actual saving in the number of registers.

An analysis of the Cray-1 computer is provided in (Sites, 1978) by Richard Sites, who later participated in defining the architecture of the Alpha microprocessor at DEC (Sites, 1993).

Convex computers originally called PARSEC was formed in Richardson, TX in 1982 It was marketed as a vector superminicomputer and had the same organization as Cray-1, except that it used the CMOS technology, which is slower

and consumes less power than ECL, but lends itself to *Very Large Scale Integration - VLSI* (Mead and Conway, 1980).

https://en.wikipedia.org/wiki/Convex_Computer

Convex's Exemplar parallel computers was based on HP's *Precision Architecture - PA* RISC processors.

<https://en.wikipedia.org/wiki/PA-RISC>

HP acquired Convex in 1995.

https://en.wikipedia.org/wiki/Convex_Computer

As part of his PhD thesis the author proposed the *Shared Computing Resource for Array Processing - SCRAP*, which also advocated chaining (Thomasian and Avizienis, 1976). A SCRAP-like system was studied in (Hwang and Ni, 1980). They authors also wrote a survey of pipelined vector computers (Hwang et al., 1981).

Cray gave a talk at HP Labs in Palo Alto, CA in summer 1996 on the same day I made a presentation there. HP was probably interested in buying Cray Corp., but Cray Corp. was instead acquired by *Silicon Graphics Inc - SGI*.

https://en.wikipedia.org/wiki/Silicon_Graphics

SGI was founded in 1981 the initial system was based on the Geometry Engine developed by Clarke and Marc Hannah at Stanford U. The company focused on 3D imaging, SGI workstations used SGI's ARC's firmware, instead of Microsoft's *Basic Input/Output System - BIOS*.

<https://en.wikipedia.org/wiki/BIOS>

https://en.wikipedia.org/wiki/SGI_Visual_Workstation

<https://www.sgi.com>

Cray set up a new company, *Seymour Roger Cray - SRC* Computers, and started the design of a massively parallel machine. The new design concentrated on communications and memory performance, the bottleneck that hampered many parallel designs. Design had just started when Cray died suddenly in a car accident. SRC Computers carried on development and specialized in reconfigurable computing.

https://en.wikipedia.org/wiki/Seymour_Cray

SRC uses a combination of Intel processors and Altera Stratix IV GX530s FPGAs.

<https://www.nextplatform.com/2015/05/28/the-other-cray-launches-cpu-fpga-hybrids/>

In 2009 SGI was acquired by Rackable which took its name. It was announced on May 17, 2019 that HPE is buying Cray for \$1.3 billion.

<https://en.wikipedia.org/wiki/Cray>

1.25 Parallel computers

The successive failure of parallel processing companies in late 80s and early 90s led Jim Gray at Microsoft to suggest that "Hennessy and Patterson should move *Massively Parallel Processing - MPP* to Chapter 11 in their book" (Hennessy and Patterson, 2017). Note that Chapter 11 is a term for bankruptcy.

A comprehensive text on parallel computer architecture which is a follow-up to (Hennessy and Patterson, 2017) by David Culler (UCB), Jaswinder Pal Singh (Princeton) with Anoop Gupta (Stanford, Microsoft, Seekout) is (Culler et al., 1999).

A list of parallel computers is as follows.

1.25.1 The ILLIAC IV computer

The ILLIAC IV was a parallel computer conceived by Daniel Slotnick at Westinghouse Electric, whose first attempt was the Solomon computer with 1024 bit-serial *Processing Elements - PEs* (a long acronym and a reminder that King Solomon supposedly had a thousand wives). At the point when funding dried the largest prototype was a 10×10 computer and this also led Slotnick to join *U. Illinois at Urbana-Champaign - UIUC*,

where he obtained funding for the ILLIAC IV computer from *Advanced Research Projects Agency - ARPA* renamed *Defense ARPA - DARPA*

<https://www.darpa.mil>

The goal of ARPANET was allowing countrywide access to supercomputers such as the ILLIAC IV in the case of a nuclear attack by the *Union of Soviet Socialist Republics - USSR*, which was the major rival to the US at the time.

<https://en.wikipedia.org/wiki/ARPANET>

This was to ensure access to supercomputing power, even if one or more computer sites were destroyed, something that was deemed not unlikely in the cold war era.

The US government argued that the threat of extinction is inaccurate, because there were not enough nuclear weapons at the time. The opposite is argued in the movie "On the beach" where the residents of Australian coastal city spared by the

initial nuclear exchange of WW III realize they are going to die like the rest of the world as radiation spreads.

[https://en.wikipedia.org/wiki/On_the_Beach_\(1959_film\)](https://en.wikipedia.org/wiki/On_the_Beach_(1959_film))

“The Day After” is made for TV movie in 1983, which considers the effects of a nuclear exchange between US and Russia on a midwestern city near US missile silos.

https://en.wikipedia.org/wiki/The_Day_After

The Burroughs Corp. was commissioned to build the ILLIAC IV massively parallel computer designed by Slotnick, but only the first quadrant with 64 *Processing Elements* - PEs was completed and placed at UIUC.

https://en.wikipedia.org/wiki/ILLIAC_IV

<http://ed-thelen.org/comp-hist/vs-illiac-iv.html>

Following antiwar protests in 1970s at the university this quadrant was transferred to *National Aeronautics and Space Administration* - NASA's Ames Center in California.

It is noted in Slotnick's Wiki that “Ironically, when the ILLIAC IV project was moved to Ames Research Center, the computer could only be accessed by telephone: (415) 329-0740.”

https://en.wikipedia.org/wiki/Daniel_Slotnick

<https://www.nasa.gov/ames>

Each ILLIAC IV quadrant had its own control unit which decoded instructions to drive PEs, which had *Floating Point Units* - FPUs. The ILLIAC IV is called a *Single Instruction Multiple Data* - SIMD computer based on Michael Flynn's classification (Flynn, 1972). Flynn's taxonomy for sequential case *Single Instruction, Single Data* - SISD and multicomputers *Multiple Instruction, Multiple Data* - MIMD. Cray 1 with multiple arithmetic pipelines operating on a single stream of data may be considered a *Multiple Instruction-Single Stream* - MISD computer.

ILLIAC IV's interconnection network facilitated two dimensional finite difference method, which can be expressed as follows for the $n + 1^{st}$ iteration:

https://en.wikipedia.org/wiki/Finite_difference_method

$$u_{i,j}^{n+1} = u_{i,j}^n + C \left(u_{i+1,j}^n + u_{i-1,j}^n + u_{i,j+1}^n + u_{i,j-1}^n \right) - 4u_{i,j}^n.$$

Note that the ILLIAC VI's interconnection network which connects each PE to its NWSE neighbors with wraparound is suited for this computation. The *Burroughs Scientific Processor* - BSP with $2^n + 1$ allowed matrix diagonals to be accessible conflict-free (Kuck and Stokes, 1982).

Main memory interleaving places successive data blocks in consecutive memory modules, whose number is usually a powers of two. In the case of 2^n modules the low order n bits determine the memory module to be accessed, while the higher order bits determine the row in main memory. There is a provision to access consecutive memory modules on successive bus cycles to fill a cache line in computers with caching. High-order bit interleaving can also be used to select memory banks.

The ILLIAC IV did not have an I/O subsystem per se and according to Figure 10 in (Bouknight et al., 1972) a Burroughs B6700 served as its I/O controller, refer to Section 1.9.1.

Another major shortcoming of the ILLIAC IV was not taking into account fault-tolerance in its design. The ILLIAC IV might have failed mainly due to PE failures, since they constituted a large fraction of its hardware. Failed PEs were rolled out and replaced with spares. The supply of spare PEs would be exhausted due to frequent PE failures.

http://bitsavers.informatik.uni-stuttgart.de/pdf/univOfIllinoisUrbana/illiac/ILLIAC_IV/B-176-B_ILLIAC_IV_Technical_Summary_Apr68.pdf

A very low *Mean Time To Failure* - MTTF was recorded at the time, which may have been due to the use of a new technology - thin-film memories.

<http://ed-thelen.org/comp-hist/navy-thin-film-memory-desc.html>

Checkpointing, which is discussed in Section 1.33 may have been used to ensure program progress in presence of failures.

ILLIAC IV led to several successful SIMD machines discussed below.

<https://en.wikipedia.org/wiki/SIMD>

<https://arstechnica.com/features/2000/03/simd/>

Supercomputers save lives by simulating nuclear weapons

A major application of supercomputers is nuclear weapon design, which was the reason for student protests at UIUC. There is also the issue of using nuclear weapon potency as a result of aging to obviate nuclear weapon tests. Atmospheric nuclear tests were used initially, because the disastrous cancer causing effects of such tests were not known initially. The Partial Test Ban Treaty banned nuclear testing in the atmosphere, underwater, and in outer space. Underground tests by countries trying to hide their experiments were detectable by seismic monitoring methods.

https://en.wikipedia.org/wiki/Nuclear_detonation_detection_system

The need for even subterranean testing was obviated by using supercomputers to determine the potency of nuclear weapons.

The movie “The Conqueror” about Genghis Khan - the founder of the Mongol empire in 1206 -

[https://en.wikipedia.org/wiki/The_Conqueror_\(1956_film\)](https://en.wikipedia.org/wiki/The_Conqueror_(1956_film))

https://en.wikipedia.org/wiki/Genghis_Khan

with John Wayne and Susan Hayward was shot in 1956 at St. George Utah, which had been the site of a nuclear test.

https://en.wikipedia.org/wiki/John_Wayne

https://en.wikipedia.org/wiki/Susan_Hayward

The main actors and 91 of the cast, out of a crew totaling 220 died or developed some form of cancer by the end of 1980.

<https://www.rt.com/op-ed/517760-toxic-legacy-nuclear-weapons/>

Radioactive material produced by nuclear reactors has been used maliciously in the form of depleted uranium (U238 versus U235, natural Uranium), which is denser than lead (Pb in Mendeleev’s periodic table.

https://en.wikipedia.org/wiki/Periodic_table and effective in penetrating tank armor. Depleted Uranium emits highly poisonous alpha particles affecting the population in conflict areas. Disposal of nuclear waste similarly to chemical waste remains a challenging problem.

1.25.2 Thinking Machines Connection Machine

Thinking Machines - *TM* which produced several versions of the *Connection Machine* - *CM* was the third company to register an internet domain (think.com) in May 1985, preceded by Symbolics, which built Lisp machines

<https://en.wikipedia.org/wiki/Symbolics>

and Bolt, Beranek and Newmann [BBN.com](http://www.bbn.com), which built Arpanet’s *Interface Message Processor* - *IMP*. BBN became a fully own subsidiary of Raytheon www.raytheon.com in 2009. At this time there are 340 million registered domain names.

TM became profitable in 1989 in part because of its DARPA contracts. It sold \$65 million worth of hardware and software, making it the market leader. In 1993 the world’s four fastest computers were from TM, but it filed for Chapter 11 in 1994.

https://en.wikipedia.org/wiki/Thinking_Machines_Corporation

1.25.3 Kendall Square Research’s KSR-1

KSR-1 was a *Cache-Only Memory Architecture* - *COMA* (Culler et al., 1999).

https://en.wikipedia.org/wiki/Kendall_Square_Research

KSR was one of supercomputer companies that went out of business along with others in earlier nineties.

1.25.4 Goodyear Massively Parallel Processor - MPP

Goodyear’s MPP was based on the earlier STARAN array processor (Batcher, 1974), which had 4x256 1-bit *Processing Elements* - *PEs*. The MPP was a 128x128 2-dimensional array of 1-bit PEs. Actually 132x128 PEs with a 4x128 configuration added for fault tolerance to substitute for up to 4 rows (or columns) of processors in the presence of problems.

https://en.wikipedia.org/wiki/Goodyear_MPP

1.25.5 MasPar

Maspar was an SIMD superminicomputer company founded in 1987 by Jeff Kalb, a DEC VP in charge of integrated circuits, who had people building a supercomputer based on the Goodyear MPP supercomputer, which was enhanced as follows.

<https://en.wikipedia.org/wiki/MasPar>

- 4 bits rather than 1 bit per processor.
- Increased connectivity from 4 to 8.
- A global interconnect instead of a full crossbar switch.

1.25.6 NCUBE

NCUBE was founded in 1983 in Beaverton, OR by Intel employees frustrated with Intel's reluctance to enter the parallel computing market, although Intel released the iPSC/1 in the same year. The first NCUBE's hypercube machines was released in 1985, N2 in 1989, N3 in 1995, and N4 in 1999.

<https://en.wikipedia.org/wiki/NCUBE>

At a later point NCUBE instead of high performance computing turned to *Video-on Demand* - VOD selling 100 VoD systems delivering 17,000 streams. NCUBE was affected badly by the dot-com downturn in 2001 and was acquired in 2005.

1.25.7 Meiko

The Transputer designed by INMOS Corp. in Bristol, UK was intended for parallel computing.

<https://en.wikipedia.org/wiki/Transputer>

Transputers used the programming language occam, based on the *Communicating Sequential Processes* - CSP calculus.

[https://en.wikipedia.org/wiki/Occam_\(programming_language\)](https://en.wikipedia.org/wiki/Occam_(programming_language))

Although the aggregate bandwidth for special case data shuffling could be very high, a major drawback of the Computing Surface architecture was poor I/O bandwidth for general data shuffling. This made the Meiko Computing Surface uneconomic for many applications.

https://en.wikipedia.org/wiki/Meiko_Scientific

1.25.8 SUPRENUM

This German effort at Krupp Atlas Elektronik was an MIMD supercomputer with distributed hardware. It used 32 bit Motorola micros with chips for FLP and memory management (Table 1.6).

<https://en.wikipedia.org/wiki/SUPRENUM>

Suprenum led to several successful companies such as Pallas.

<https://www.pallas.com/>

TABLE 1.6 Comparison of the Suprenum supercomputer with its contemporaries. *ES* - Evans and Sutherland, *CM* - Connection Machine.

System	# of nodes	Peak Mflops
Meiko CS-1	16	960
Suprenum-1	256	5120
Intel iPSC/860	128	7680
CM-2	65536	24000
CM-200	65536	32000
CM-5	1024	131000
KSR-1	64	2560
ES-1	3	640
Myrias SP-2	128	64

1.25.9 Parsytec

This German company like Meiko was based on Transputers. Its computers were used for industrial applications such as pattern recognition.

<https://en.wikipedia.org/wiki/Parsytec>

1.25.10 Intel Personal SuperComputer - iPSC

iPSC/1 was called the first parallel computer built from commercial off-the-shelf parts. It was delivered with 32 nodes to *Oak Ridge National Laboratory - ORNL* in 1985.

https://en.wikipedia.org/wiki/Intel_iPSC

1.25.11 IBM's BlueGene supercomputer

The Blue Gene/L supercomputer was influenced by Transputer and had the following aspects:

- Trading the speed of processors for lower power consumption by using low frequency and low power embedded PowerPC cores with FLP accelerators. With low performance chips the system could achieve better power efficiency for applications requiring a large number of nodes.
- Dual processors per node with two working modes: (1) co-processor mode where one processor handles computation and the other handles communication; (2) virtual-node mode, where both processors are available to run user code, but the processors share both the computation and communication load.
- *System-on-a-Chip* - SOC design so that all node components were embedded on one chip, with the exception of 512 MB external DRAM.
- A large number of nodes in increments of 1024 up to 65,536.
- Three-dimensional torus interconnect with auxiliary networks for global communications (broadcast and reductions), I/O, and management. A 3d torus in Cartesian coordinates is given as follows:

<https://mathworld.wolfram.com/Torus.html>

$$(c - \sqrt{x^2 + y^2})^2 + z^2 = a^2$$

with parametric equations $x = (c + a \cos(v)) \cos(u)$, $y = (c + a \cos(v)) \sin(u)$, $z = a \sin(v)$.

- Lightweight OS per node for minimum system overhead.

Interconnection networks for BlueGene are discussed in Chapter 8.

1.25.12 Tesla Dojo supercomputer for AI training

The description that follows is based on the first two sections of this web page.

<https://cleantechnica.com/2021/08/22/teslas-dojo-supercomputer-breaks-all-established-industry-standards-cleantechnica-deep-dive-part-1/>

Substitute 2-4 for the first four parts.

Tesla's Andrej Karpathy revealed in a presentation he Tesla's made in June, 2021 that the largest cluster is made up of NVIDIA's new A100 GPUs, but in the section we are mostly interested in Tesla's Dojo chip, which is listed in Table 1.7.

A high performance training node: 64b Superscalar CPU according to Table 1.8 is organized as follows:

4-way Multi-threaded scalar CPU

4 matrix multiply units

Register file, 1.25 MB SRAM, Sync, 2 Ld/St

<https://cleantechnica.com/files/2021/08/Training-Node-2.png>

1.26 The future of supercomputing

Newest supercomputers consist of thousands of interconnected microprocessors equipped with GPUs and/or FPGAs capable of processing huge volumes of data.

https://en.wikipedia.org/wiki/Graphics_processing_unit

https://en.wikipedia.org/wiki/Field-programmable_gate_array

TABLE 1.7 Top 10 supercomputers in the world doing FP32.

Rank	Name	Petaflops	Location
1	Fuguka	442.01	Japan
2	Summit	148.60	Tennessee
3	Sierra	94.64	California
4	Sunway TaihuLight	93.	China
5	Tesla GPU Cluster 3	≈90	California
6	Dojo	≈68.75	California
7	Perlmutter	64.59	California
8	Selene	63.46	California
9	Tianhe-2A	61.40	China
10	JUWELS	44.12	Germany

TABLE 1.8 Computer memory comparison, speed in nanoseconds.

Name	Size	Speed	Location	Shared
L1 cache	32-64 KB	0.5-1 ns	core/node	No
L2 cache	256 KB-2 MB	3-4 ns	core/node	No
L3 cache	10-256 M	10 ns	chip	Yes
DRAM	1-512 GB	60	motherboard	Yes
Hard drive	1G-B08TB	> 100	motherboard	Yes

While China is a major presence in the list of TOP 500 computers using the High Performance LinPack program, a different ranking is obtained using *High Performance Conjugate Gradients - HPCG* benchmark, which unlike LINPACK which operates on dense data structures, HPCG processes sparse data structures.

<http://www.machinedesign.com/community/there-s-new-test-town-ranking-supercomputers>

Chinese supercomputers are discussed in (Hamsoth, 2016)

<https://www.nextplatform.com/2016/06/20/look-inside-chinas-chart-topping-new-supercomputer/>

<https://www.cnet.com/news/china-surpasses-us-in-supercomputer-usage-on-top-500-list/>

US companies such as Intel/AMD and Nvidia dominate the underlying GPU technology and Xilinx FPGAs

<https://www.fool.com/investing/2016/07/02/intels-supercomputer-chips-get-ko-ed-by-china-and.aspx>

In 2014, the US Dept. of Energy - DoE initiated a multiyear collaboration between ORNL, ANL, and LLNL based on the *Collaboration of Oak-Ridge, Argonne, and Livermore - CORAL*. The next major phase in the DoE's scientific computing roadmap is described in (Hanson, 2020).

The IBM CORAL systems are based on a fundamentally new data-centric architecture, where compute power is embedded everywhere data resides, combining powerful CPUs with GPUs optimized for scientific computing and artificial intelligence workloads. The IBM CORAL systems were built on the combination of POWER9 CPUs, 6th-generation NVIDIA GPUs, and 5th-generation Mellanox InfiniBand. An overview of the system solutions deployed at ORNL and LLNL is provided in several articles on various aspects of CORAL in 6/20 issue of IBM J. R&D.

On June 8, 2018 IBM announced the world's fastest computer.

<https://www.nytimes.com/2018/06/08/technology/supercomputer-china-us.html>

<https://www.theverge.com/circuitbreaker/2018/6/12/17453918/ibm-summit-worlds-fastest-supercomputer-america-department-of-energy>

[https://en.wikipedia.org/wiki/Summit_\(supercomputer\)](https://en.wikipedia.org/wiki/Summit_(supercomputer))

IBM is a participant with Summit supercomputer with a Power9 microprocessor, which is at *Oak Ridge National Laboratory - ORNL Leadership Computing Facility - LCF*.

<https://www.olcf.ornl.gov/olcf-resources/compute-systems/summit/>

CORAL-2 initiative will take \$1.8 billion, for the development of at least two new exascale supercomputers to be deployed at DoE National Laboratories in the 2021-2023 timeframe.

<https://www.hpcwire.com/off-the-wire/us-announces-1-8-billion-supercomputing-initiative/>

Oak-Ridge Leadership Computing Facility - OLCF has been home to several supercomputers.

<https://www.olcf.ornl.gov/tag/hpc/>

The Cray XK7 Titan was disassembled on 8/1/2020.

<https://www.olcf.ornl.gov/2019/06/28/farewell-titan/>

Summit is an exascale computer to be delivered 2021. The tensor core technology in Summit's NVIDIA Volta GPUs speeds mixed-precision calculations technique that is beneficial to training networks in machine learning in which a high degree of numerical precision is not always required.

Frontier will be built by chip designer AMD and supercomputer manufacturer Cray for ORNL by 2021 for the DOE will deliver 1.5 Exaflops to perform advanced calculations in areas like nuclear and climate research.

<https://www.olcf.ornl.gov/2019/05/07/no-scaling-back-doe-cray-amd-to-bring-exascale-to-ornl/>

<https://www.theverge.com/2019/5/7/18535078/worlds-fastest-exascale-supercomputer-frontier-amd-cray-doe-oak-ridge-national-laboratory>

<https://finance.yahoo.com/news/hp-enterprise-nears-deal-buy-015349971.html>

DoE has revealed three exascale-class supercomputers. Argonne's National Laboratory's Aurora was the first system, announced in March 2019 and is expected to deliver over 1 ExaFLOPS performance. Aurora is being developed by Intel to advance clean fusion research.

<https://www.intel.com/content/www/us/en/now/your-data-on-intel/aurora-exascale-supercomputer-to-advance-clean-fusion-research-video.html>

Aurora will be delayed because of delays in Intel's 7 nm chips.

<https://www.tomshardware.com/news/us-governments-aurora-supercomputer-delayed-due-to-intels-7nm-setback>

Oak Ridge National Lab - ORNL's Frontier supercomputer, powered by AMD's Epyc 'Milan' processors and Radeon Instinct MI200 graphics, was unveiled in May 2019 and is on-track to deliver 1.5 ExaFLOPS performance in 2021.

Cray has a contract to build an exascale system EL Capitan to be installed at *Lawrence Livermore National Lab - LLNL* in 2022 based on a Shasta platform and slingshot interconnect.

El Capitan system that is set to hit 2 ExaFLOPS in 2023 using AMD's Epyc "Genoa" CPUs and AMD CDNA GPUs.

<https://www.nextplatform.com/2020/05/05/hpes-ungaro-on-delivering-exascale-for-the-masses/>

<https://www.top500.org/news/cray-unveils-shasta-its-supercomputer-platform-for-the-exascale-era/>

Japanese Fugaku supercomputer introduced on 6/1/2020 uses Fujitsu ARM based microprocessors and a toroidal network. <https://www.anandtech.com/show/15869/new-1-supercomputer-fujitsus-fugaku-and-a64fx-take-arm-to-the-top-with-415-petaflops> A list of world's supercomputers given in (cf. Table 1.9) was extracted from the following web page.

[https://en.wikipedia.org/wiki/Fugaku_\(supercomputer\)](https://en.wikipedia.org/wiki/Fugaku_(supercomputer))

<https://en.wikipedia.org/wiki/TOP500>

Italy's *Ministry of Education, University, and Research - MUIR*, National Institute of Nuclear Physics, and International School of Advanced Studies unveiled Leonardo, a new supercomputer to be constructed at CINECA capable of exascale computing for drug discovery, space exploration, and weather modeling. Nvidia will supply Ampere-based graphics cards and Mellanox HDR 200 GB/s Infiniband networking to Leonardo to deliver up to 10 exaflops of half-precision floating-point (FP16) performance.

<https://venturebeat.com/2020/10/15/nvidia-will-supply-hardware-for-leonardo-a-10-exaflop-supercomputer/>

Performance and cost comparison chart for computers ranked #1 in TOP500 is given in Table 1.9.

TABLE 1.9 Performance and cost comparison chart for computers ranked #1 in TOP500.

Name	Start	End	Performance (PFLOPS)	Cost (Million\$)	TOP500 Ranking	CPU/GPU Vendor	CPU	OS
Fugaku	2020	-	415	1213	6/20 1st	Fujitsu	A64FX	Linux-custom
Summit	2018	-	148	300	6/18-11/19 1st	IBM, NVIDIA	POWER9, Tesla	Linux (RedHat)
Sierra	2018	94	94	300	11/18-11/19 2nd	same	same	same
Sunway	2016	-	93	280	6/16-11/17 1st	NRCPC	SW26010	Linux (Raise)
K	2011	2019	10	1045	6/11-11/11 1st	Fujitsu	SPARC64 VIIIfx	Linux

The current list of worlds ten fastest computers is as follows.

<https://www.networkworld.com/article/3236875/embargo-10-of-the-worlds-fastest-supercomputers.html>

Plans for future supercomputers are listed below.

<https://spectrum.ieee.org/computing/hardware/will-china-attain-exascale-supercomputing-in-2020>

In May 2019, the Chinese government took a three-pronged approach with its exascale efforts, pitting the *National University of Defense Technology - NUDT*, the *National Research Center of Parallel Computer - NRPC* Engineering and Technology, and server maker Sugon come up with three distinct exascale designs relying on completely unique components. A first look at China's Sunway exascale supercomputer is given here.

<https://www.nextplatform.com/2021/02/10/a-sneak-peek-at-chinas-sunway-exascale-supercomputer/>

1.27 Microprocessor CPUs, GPUs, FPGAs, and ASICs

The advent of lower-cost microprocessors and associated microcomputers in the form of personal computers and servers killed off minicomputer companies, especially in US's New England area in 1980s, but included the Sun Microsystems in the Bay Area.

<https://en.wikipedia.org/wiki/Minicomputer>

https://ethw.org/Rise_and_Fall_of_Minicomputers

Intel which is the number one microprocessor company had a humble beginning with its 4 bit Intel 4004 in November 1971, which was intended to serve as the computational engine for hand-held electronic calculators.

https://en.wikipedia.org/wiki/Intel_4004

https://en.wikipedia.org/wiki/List_of_Intel_microprocessors#Intel_4004

The evolution of Intel microprocessors is listed below.

<https://www.businessnewsdaily.com/10817-slideshow-intel-processors-over-the-years.html>

Intel microprocessors combined with Windows OS known as Wintel dominate the field. Wintel's success can be ascribed to IBM's Personal computer standard which utilized Intel microprocessors and Microsoft's DOS operating system as discussed in Section 1.29.

Advanced Micro Devices - AMD is the 2nd largest supplier of x86 compatible microprocessors.

https://en.wikipedia.org/wiki/Advanced_Micro_Devices

When in 1981 IBM created its PC it wanted Intel's x86 processors under the condition that Intel also provides a second-source manufacturer for its patented x86 microprocessors and that led Intel and AMD entered into a 10-year technology exchange agreement. The terms of the agreement were that each company could acquire the right to become a second-source manufacturer of semiconductor products developed by the other; that is, each party could "earn" the right to manufacture and sell a product developed by the other.

AMD is a fabless company relying on Global Foundries. which was created by the divestiture of the manufacturing arm of AMD.

<https://en.wikipedia.org/wiki/GlobalFoundries>

The Emirate of Abu Dhabi is the owner of the company through its subsidiary *Advanced Technology Investment Company - ATIC*.

Hector Ruiz who was AMD's CEO wrote "Slingshot" (Ruiz, 2013) about Intel's unfair competition against AMD keeping its share to a few percentage points that led to a lawsuit. US's next supercomputer which will be able to process 1.5 quintillion (10^{18}) calculations per second will be built by Cray with AMD microprocessors.

<https://www.theverge.com/2019/5/7/18535078/worlds-fastest-exascale-supercomputer-frontier-amd-cray-doe-oak-ridge-national-laboratory>

There has been a turnaround in AMD fortunes with the help of CEO Lisa Su, a former IBMer.

https://en.wikipedia.org/wiki/Lisa_Su

A recent comparison of Intel and AMD processors is given below.

<https://www.techradar.com/news/a-reddit-user-charted-just-how-much-amd-is-beating-intel-right-now>

<https://www.extremetech.com/computing/325848-amd-x86-cpu-market-share-soars-hits-14-year-high>

AMD x86 CPU Market Share Soars, Hits 14-Year High as shown in Tables 1.10 and 1.11.

AMD EPYC server processors bring a distinct advantage in both dual and single socket configurations, with up to 32 high-performance Zen cores with 64 threads (2 threads per core), 8 memory channels and 128 lanes of PCIe. With the flexibility to choose from 8 to 32 cores, EPYC enables the user to deploy the right hardware platforms to meet workload needs ranging from virtualized infrastructure to large-scale big-data and analytics platforms and machine learning applications. AMD EPYC provides breakthrough processing power balanced with the industry-leading memory and I/O capacity needed to substantially eliminate performance bottlenecks.

TABLE 1.10 Overall x86 CPU share.

Overall x86 CPU share Includes IOT & SoC	21 Q2 share	21 Q1 share	20 Q2 share	Change quarter	Change year
Intel	77.5%	79.3%	81.7%	−1.8%	−4.2%
AMD	22.5%	20.7%	18.3%	+1.8%	+4.2%

TABLE 1.11 Server CPU share excluding IOT.

Server CPU share Includes IOT & SoC	21 Q2 share	21 Q1 share	20 Q2 share	Change quarter	Change year
Intel	90.5	91.1%	94.2	−0.6%	−3.7%
AMD	9.5%	8.9%	5.8%	+0.6%	+3.7%

The design of a vector microprocessor (Torrent-0) is presented in (Asanovic, 1998). Intel AVX-2 uses 256 bit registers which hold eight 32 bit integers with added SIMD instructions.

https://en.wikipedia.org/wiki/Advanced_Vector_Extensions

Linux creator Linus Torvalds stated on 7/13/20. “I hope AVX-512 dies a painful death, and that Intel starts fixing real problems instead of trying to create magic instructions to make benchmarks look good.”

<https://www.zdnet.com/article/linus-torvalds-i-hope-intels-avx-512-dies-a-painful-death/>

The following discussion comparing CPUs, FPGAs, GPUs, and ASICs is based on the following.

<https://www.arrow.com/en/research-and-events/articles/fpga-vs-cpu-vs-gpu-vs-microcontroller>

CPU: Overview: Traditional sequential processor for general-purpose applications. Processing: Single- and multi-core *MicroController Unit - MCU*, and *MicroProcessing Unit - MPU*, plus specialized blocks: *Floating Point Unit - FPU*, etc. Programming: OSes, APIs run numerous *High-Level Languages - HLLs*; assembly language. Peripherals: Wide choice of analog and digital peripherals in MCUs; MPUs include digital bus interfaces Strengths: Versatility, multitasking, ease of programming Weaknesses: OS capability adds high overhead; optimized for sequential processing with limited parallelism

FPGA Overview: Flexible collection of logic elements and *Intellectual Property - IP* blocks that can be configured and changed in the field. Processing: Configured for application; *System on a Chip - SoC* includes hard or soft IP cores (e.g., ARM). Programming: Traditionally *Hardware Description Language - HDL* (Verilog, *VHSIC-Very High Speed Integrated Circuit HDL*; newer systems include C/C++ via openCL & SDAccel. Peripherals: *Systems on a Chip - SoC*, include many transceiver blocks, configurable I/O banks. Strengths: Configurable for specific application; configuration can be changed after installation; high performance per watt; accommodates massively parallel operation; wide choice of features: DSPs, CPUs. Weaknesses: Relatively difficult to program; second-longest development time; poor performance for sequential operations; not good for FLP operations

A history of FPGAs is given here.

<https://web.archive.org/web/20070412183416/>

<http://filebox.vt.edu/users/tmagin/history.htm>

https://en.wikipedia.org/wiki/Field-programmable_gate_array

FPGAs tend to be slower, less energy efficient and less functional than their fixed *Application Specific Integrated Circuit - ASIC* counterparts.

https://en.wikipedia.org/wiki/Application-specific_integrated_circuit

FPGAs need on average 40 times as much area, draw 12 times as much dynamic power, and run at one third the speed of corresponding ASIC implementations. More recently, FPGAs such as the Xilinx Virtex-7 or the Altera Stratix 5 have come to rival corresponding ASIC and Application-Specific Standard Part by providing significantly reduced power usage, increased speed, lower materials cost, minimal implementation real-estate, and increased possibilities for re-configuration “on-the-fly”. A design that included 6 to 10 ASICs could be achieved using only one FPGA.

Advantages of FPGAs include the ability to re-program when already deployed “in the field” to fix bugs and often include shorter time to market and lower non-recurring engineering costs. Vendors can also take a middle road via FPGA prototyping: developing their prototype hardware on FPGAs, but manufacture their final version as an ASIC so that it can no longer be modified after the design has been committed.

Microsoft’s use of FPGAs to speedup Bing queries and reduce search power consumption is described here.

<https://www.enterprisai.news/2014/09/03/microsoft-using-fpgas-speed-bing-search/>

Xilinx, Inc. is an American technology company that develops highly flexible and adaptive processing platforms. Xilinx sells a broad range of FPGAs, *Complex Programmable Logic Devices* - CPLDs, design tools, intellectual property and reference designs. Xilinx customers represent just over one-half of the entire programmable logic market, at 51%.

Xilinx Deep Learning Solution on EPYC® Processors are discussed in the following AMD White Paper.

<https://www.amd.com/system/files/documents/xilinx-deep-learning-solution-on-amd-epyc-processors.pdf>

AMD acquisition of Xilinx for \$35 Billion on 10/27/2020, but this acquisition is in jeopardy.

<https://www.amd.com/en/press-releases/2020-10-27-amd-to-acquire-xilinx-creating-the-industry-s-high-performance-computing>

<https://seekingalpha.com/amp/article/4445289-xilinx-amd-deal-spread-blows-out>

GPU Overview: Originally designed for graphics, but now used in a wide range of computationally intensive applications. Processing: Thousands of identical processor cores. Programming: OpenCL & Nvidia's CUDA API allow general-purpose programming (e.g., C, C++, Python, Java, Fortran) Peripherals: Very limited; e.g., only cache memory. Strengths: Massive processing power for target applications, video processing, image analysis, signal processing. Weaknesses: High power consumption, not suited to some algorithms; problems must be reformulated to take advantage of parallelism, but API frameworks provide abstraction.

Compute Unified Device Architecture - CUDA is a parallel computing platform and API created by Nvidia.

<https://en.wikipedia.org/wiki/CUDA>

Nvidia's CUDA C++ programming guide is given below.

<https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html>

It allows software developers and engineers to use a CUDA-enabled GPUs for general purpose processing – an approach termed GPGPU - General-Purpose computing on GPUs. The CUDA platform is a software layer that gives direct access to the GPU's virtual instruction set and parallel computational elements, for the execution of compute kernel.

GPU-accelerated database systems: Survey and open challenges (Breß et al., 2014b)

Abstract

“The vast amount of processing power and memory band-width provided by modern graphics cards make them an interesting platform for data-intensive applications. Unsurprisingly, the database research community identified GPUs as effective co-processors for data processing several years ago. In the past years, there were many approaches to make use of GPUs at different levels of a database system. In this paper, we explore the design space of GPU-accelerated DBMSs. Based on this survey, we present key properties, important trade-offs and typical challenges of GPU-aware database architectures, and identify major open challenges. Additionally, we survey existing GPU-accelerated DBMSs and classify their architectural properties. Then, we summarize typical optimizations implemented in GPU-accelerated DBMSs. Finally, we propose a reference architecture, indicating how GPU acceleration can be integrated in existing DBMSs.” Efficient stream processing on modern hardware is discussed in (Zeuch et al., 2019).”

ASIC Overview: Application Specific Integrated Circuits are custom circuits optimized for the end application. Processing: Application-specific: may include third-party IP cores. Programming: Application-specific: TensorFlow open-source framework for Google's *Tensor Processing Units* - TPU; CPU manufacturers (e.g., Intel) include tools with new ASIC releases Peripherals: Tailored to application: may include industry-standard functions (USB, Ethernet, etc.) Strength: Custom-designed for application with optimum combination of performance and power consumption. Weaknesses: Longest development time; high cost; cannot be changed without redesigning the silicon, refer to Table 1.12.

Intel and AMD face competition from a 96-way ARM microprocessor, which has four threads per core, hence 384-way. Marvell acquired a small startup (Cavium) which developed the ThunderX3 microprocessor.

<https://www.techradar.com/news/intel-and-amd-face-an-armed-onslaught-from-a-96-core-cpu-monster>

The *Scalable Processor Architecture* - SPARC microprocessors developed by David Patterson and Carlo Sequin at UC Berkeley was adopted by SUN Microsystems and Fujitsu and served as the engine for the Solaris UNIX based OS (Hall and Barry, 1991). Sun Microsystems was acquired by Oracle in 2010,

https://en.wikipedia.org/wiki/Sun_acquisition_by_Oracle

so Sparc microprocessors are produced by Oracle and Fujitsu, but also others.

<https://en.wikipedia.org/wiki/SPARC>

TABLE 1.12 Options available for various applications *Tensor Processor Unit - TPU*.

Applications	CPU	FPGA	GPU	ASIC	Comments
Vision & image processing		X	X	X	FPGA may give way to ASIC in high-volume applications
AI Training			X		GPU parallelism well-suited for processing TB data sets in reasonable time
AI Inference	X	X	X	X	FPGAs leading; high-end CPUs (e.g., Intel's Xeon) and GPUs (e.g., Nvidia's T4) address this market
High speed search	X	X	X	X	Microsoft's Bing uses FPGAs; Google uses TPU ASIC; CPU needed for coordination & control
Industrial motor control	(X)	X		X	Many motor-control MCUs and ASICs available; FPGAs offer a quick-turn ASIC alternative
Supercomputer HPC	X		X		Majority of TOP500 supercomputers uses some combination of CPUs and GPUs
General purpose computing	X		(X)		CPU most versatile, flexible option; GPUs beginning to perform some tasks
Embedded control		X			CPUs (→ MCU) dominant in low-cost, space-constrained, low-power, mobile applications
Prototyping, low volume		X			FPGAs best choice for high-end applications; also pre/post-silicon and firmware development

In parallel John Hennessey at Stanford U. developed *Microprocessor without Interlocked Pipeline Stages - MIPS*

https://en.wikipedia.org/wiki/MIPS_architecture

MIPS computer systems was formed in 1984 and was acquired by *Silicon Graphics Inc. - SGI* in 1988. At this time MIPS Technologies is a US based fabless semiconductor design company owned by a *Venture Capital - VC* Company.

https://en.wikipedia.org/wiki/MIPS_Technologies

DLX (DeLuxe) instruction set which is an improvement on MIPS was developed by Patterson and Hennessey.

<https://en.wikipedia.org/wiki/DLX>

and is mainly used for educational purposes.

<https://www.csee.umbc.edu/courses/undergraduate/411/spring96/dlx.html>

MMIX(= 2009) is a 64-bit *Reduced Instruction Set Computer - RISC* architecture designed by Donald Knuth, which is an extension of the MIX hypothetical computer used in his *The Art of Computer Programming - TAOCP* book series. MIX's model number is 1009, which was derived by combining the model numbers and names of several commercial machines in late 1960s. Knuth was assisted by John Hennessey (MIPS architecture) and Richard L. Sites (Alpha architecture).

<https://en.wikipedia.org/wiki/MMIX>

Hennessey and Patterson shared the ACM annual Turing award in 2017.

https://en.wikipedia.org/wiki/Turing_Award

The award was started in 1966 and is considered to be the Nobel prize in computing, but seems biased towards American computer scientists.

https://stats.areppim.com/stats/stats_turingxnation.htm

<https://www.acm.org/media-center/2018/march/turing-award-2017>

A talk related to this award is (Hennessey and Patterson, 2018) and (Hennessey and Patterson, 2019) is a paper written for this occasion. The list of Turing award winners is given below.

<https://amturing.acm.org/byyear.cfm>

1.28 RISC-V and other microprocessors

David Patterson has advocated an open-source RISC architecture, referred to as RISC-V (Patterson and Waterman, 2017).

<https://en.wikipedia.org/wiki/RISC-V>

RISC-V has features to reduce cost, but also power consumption. The RISC-V ISA received the best instruction set award from the Linley Group, "the industry's leading experts in semiconductor markers".

<https://www.linkedin.com/company/the-linley-group/>

<https://linleygroup.com/pressdetail.php?The-Linley-Group-Announces-Winners-of-Annual-Analysts-Choice-Awards-85>

Linley Group's *MicroProcessor Report* - MPR is available online to subscribers.

<https://www.linleygroup.com/mpr/>

Given restrictive US trade policies the RISC-V foundation was moved to Switzerland in November 2019. RISC V is gaining popularity because RISC-V ISA is provided under open source licenses that do not require fees to use.

<https://en.wikipedia.org/wiki/RISC-V>

A web site describing RISC-V history is as follows.

<https://riscv.org/about/history/>

A list of companies producing RISC-V processors is as follows.

<https://www.datacenterknowledge.com/hardware/companies-pushing-open-source-risc-v-silicon-out-edge>

The following news item regarding RISC V calls it the Linux of chip world.

<https://www.zdnet.com/article/risc-v-the-linux-of-the-chip-world-is-starting-to-produce-technological-breakthroughs/>

A recent microprocessor design using RISC-V has a clock speed of 5 Ghertz, which is above recent top-of-the-line Intel Xeon server chip, E7, running at 3.2 Ghertz. Yet the novel RISC-V chip uses 1 W at 1.1 volts, less than 1% of the power of Intel Xeon. IBM mainframes 5-Ghertz liquid-cooled product takes 100 W. The chip is faster than the Intel and the Samsung chips *Embedded Microprocessor Consortium* - EMBC's CoreMark.

<https://www.eembc.org/coremark/>

Micro Magic, Inc. announced the world's highest performance/power 64-bit RISC-V core at 110,000 CoreMarks/W. According to its co-founder: "After achieving the world's fastest RISC-V at 5 GHz and 13,000 CoreMarks we achieved another milestone by producing over 8,000 CoreMarks at 3 GHz while consuming less than 70 mW.

<http://www.micromagic.com/>

A recent article lists RISC-V as the latest opportunity to change computing.

<https://www.zdnet.com/article/risc-v-ceo-the-biggest-opportunity-to-change-computing-since-the-1980s/>

Vector processing is provided by microprocessors today directly or supplemented GPUs such as those by Nvidia.

<https://en.wikipedia.org/wiki/Nvidia>

Top ten companies developing GPUs and their ranking is listed below.

<https://roboticsandautomationnews.com/2017/08/11/top-10-graphics-processing-unit-manufacturers-nvidia-clearly-in-the-lead/13709/>

<https://www.tomshardware.com/reviews/gpu-hierarchy,4388.html>

Exasol is a parallelized RDBMS which runs on a cluster of standard computer hardware servers. Following the *Single Program Multiple Data* - SPMD model, on each node the identical code is executed simultaneously. The data is stored in a column-oriented way and proprietary in-memory compression methods are used. The company claims that tuning efforts are not necessary since the database includes some kind of automatic self-optimization (automatic indices, table statistics, and distributing of data).

<https://en.wikipedia.org/wiki/Exasol>

<http://www.dbms2.com/2008/08/16/exasol-technical-briefing/>

<https://www.exasol.com/en/v7/>

Exasol uses GPUs whose advantages are listed below.

<https://www.exasol.com/en/resource/what-are-gpus-and-why-do-data-scientists-love-them/>

Nvidia is about to acquire ARM from Japan's Softbank for \$40B, creating "World's Premier Computing Company for the Age of AI".

<https://nvidianews.nvidia.com/news/nvidia-to-acquire-arm-for-40-billion-creating-worlds-premier-computing-company-for-the-age-of-ai>

A major concern from the Chinese firms is that Britain's biggest chip designer could restrict access of its architecture to Chinese businesses.

<https://sputniknews.com/business/202010211080839541-huawei-to-join-forces-with-chinese-tech-companies-to-limit-nvidia-arm-buyout-deal-amid-us-trade-war/>

<https://www.cnn.com/amp/2021/08/04/nvidias-40-billion-bid-for-arm-could-be-in-jeopardy.html>

This is considered a boon for RISC-V, which wants to incorporate in Switzerland.

<https://www.enterpriseai.news/2020/10/23/nvidia-arm-deal-a-boon-for-risc-v/>

https://www.theregister.com/2019/11/26/riscv_foundation_switzerland/

EU's SiPearl in Paris is planning to enter the CPU fray with an ARM architecture.

<https://www.techradar.com/news/mighty-and-surprising-cpu-rival-to-intel-and-amd-to-come-next-year>

Microprocessors are the computing engine in servers, desktops, notebook or laptop computers, tablets, sensors for *Internet of Things - IoT* and smartphones and watches.

https://en.wikipedia.org/wiki/Internet_of_things

Mobile applications (apps) are programs which run on mobile devices.

https://en.wikipedia.org/wiki/Mobile_app

Five programming languages to develop apps are BuildFire.js, Python, Java, PHP, C++. PHP Hypertext Preprocessor (a recursive acronym)

Microprocessors serve as embedded computers in appliances and cars to the extent that cars have been called computers on wheels.

https://en.wikipedia.org/wiki/Embedded_system

High end cars such as the BMW and Mercedes Benz have as many as 100 processors onboard.

<https://www.embedded.com/electronics-blogs/significant-bits/4024611/Motoring-with-microprocessors>

With 130 billion microprocessors produced by 2019 *Advanced RISC Machine - ARM* architecture is the most popular computer architecture occupying 75% of the 32 bit RISC CPU market.

https://en.wikipedia.org/wiki/ARM_architecture

Low power consumption of ARM microprocessors is important for mobile computing.

The origin on *Acorn RISC Machine - ARM* micros lies in the British Acorn Computing.

https://en.wikipedia.org/wiki/Acorn_Computers

Microprocessor processing power is expressed by various benchmarks such as *Standard Performance Evaluation Corporation - SPEC*.

<http://spec.org/>

Higher MIPS in succeeding microprocessor generations was initially achievable by simply speeding up the CPU clock by using a higher voltage, which however results in more power consumption and generates more heat.

<https://electronics.stackexchange.com/questions/186158/why-does-increased-voltage-increase-processor-performance>

After a point of diminishing returns in speeding up microprocessors by increasing their cycle time was reached a higher processing power was achieved by using increased transistor counts to adopt more sophisticated architectures and multiple cores per chip (Shen and Lipasti, 2013).

Cache coherency limits the performance of multicore systems. Different techniques can be applied to prevent cache incoherence. A survey of different techniques to improve cache performance has been presented here with focus on parameters like cache miss rate, average memory access time, execution time, energy, area, scalability. The survey indicates the need for a cache coherency model that supports scalability with minimum latency (Joshi and Ramasubramanian, 2015).

<https://www.comsol.com/blogs/havent-cpu-clock-speeds-increased-last-years/>

https://en.wikipedia.org/wiki/Multi-core_processor

Multithreading is the ability for a single CPU to execute multiple processes or threads concurrently.

[https://en.wikipedia.org/wiki/Multithreading_\(computer_architecture\)](https://en.wikipedia.org/wiki/Multithreading_(computer_architecture))

If one of the threads encounters a cache miss the other threads can continue their execution. Note similarity to multiprogramming which was introduced to deal with page faults. This is referred to as *Coarse MultiThreading - CMT*.

In *Interleaved Multi-Threading - IMT* processors execute instructions from different processes. There is less chance of dependency stalls, i.e., one instruction in one pipelining stage needing an output from an older instruction in the pipeline.

In *Simultaneous Multi-Threading - SMT* a superscalar processor issues instructions from multiple threads every CPU cycle. This is because any single thread has a limited amount of instruction-level parallelism, this type of multithreading tries to exploit parallelism available across multiple threads to decrease the waste associated with unused issue slots.

Multiple threads can interfere with each other when sharing hardware resources such as caches or TLBs. As a result, execution times of a single thread are not improved and can be degraded, even when only one thread is executing, due to lower frequencies or additional pipeline stages that are necessary to accommodate thread-switching hardware.

As explained in Intel's *Hyper-Threading Technology - HTT* Technical User's Guide, 2003 HTT is the same as SMT.

<https://web.archive.org/web/20100821074918/>

http://cache-www.intel.com/cd/00/00/01/77/17705_htt_user_guide.pdf

3.7 GHz is an upper limit for air-cooled Intel microprocessors and more expensive liquid cooling is required for 8.5-9 GHz. Intel has made its new 5 GHz chip available on auctions.

<https://arstechnica.com/gadgets/2019/01/intel-reported-to-have-a-new-5ghz-chip-thats-literally-priceless/>

Microprocessor for the IBM z-14 ad z-15 also provides 5.2 GHz.

[https://en.wikipedia.org/wiki/IBM_z15_\(microprocessor\)](https://en.wikipedia.org/wiki/IBM_z15_(microprocessor))

Annapurna Labs is an Israeli fabless semiconductor company that designs custom ARM-based SoCs and other ASICs. https://en.wikipedia.org/wiki/Application-specific_integrated_circuit

In early 2015, the company was acquired by Amazon for around \$350-\$375M. Annapurna Labs has produced 7 nm, second generation Graviton Microprocessor.

<https://www.hpcwire.com/2019/12/03/aws-launches-7nm-2nd-gen-graviton-arm-cpu/>

AWS Graviton (Alpine AL73400) is a 16-core ARMv8 SoC designed for Amazon's own infrastructure. The chip was first unveiled during Amazon's AWS re:Invent 2018. These processors are offered as part of Amazon's EC2 A1 instances. The Graviton features 16 Cortex-A72 cores organized as four quad-core clusters, all operating at 2.3 GHz.

https://en.wikichip.org/wiki/annapurna_labs/alpine/al73400

Taiwan Semiconductor Manufacturing Company - TSMC is a IC foundry with facilities in North America, Europe, Japan, China, and South Korea.

<https://www.tsmc.com/english>

<https://www.icinsights.com/news/bulletins/Five-Semiconductor-Companies-Hold-53-Of-Global-Wafer-Capacity/>

It is expected that TSMC will fall behind Samsung by 2024.

<https://seekingalpha.com/article/4392173-tsmc-to-fall-behind-intel-samsung-2024> Worldwide wafer capacity leaders are listed in Table 1.13.

TABLE 1.13 Worldwide wafer capacity leaders in Kilo wafers/month.

2019 rank	2018 rank	Company	Hdqrtrs Region	Capacity Dec-2018	Capacity Dec-2019	Yr/Yr change	Worldwide share
1	1	Samsung	S. Korea	1,934	2,935	0%	15%
2	2	TSMC	Taiwan	2,439	2,505	3%	12.8%
3	3	Micron	US	1,685	1,841	7%	9.4%
4	4	SK Hynix	S. Korea	1,630	1,743	7%	8.9%
5	5	Kioxia/WD	Japan	1,361	1m406	3%	7.2%

The history of CPUDB which led to (Danowitz et al., 2012a,b) is provided below.

<http://cpudb.stanford.edu/>

“For years Stanford University’s VLSI Research Group collected and maintained a unique spreadsheet of commercial microprocessor characteristics. Over the years these data points were useful in a variety of research talks and publications. Unfortunately, the lack of a central repository for this database made it difficult to both share the data with everyone, and enhance it with outside contributions... until now.”

AMD Accelerated Processing Units - APUs

The AMD Fusion project (renamed *Heterogeneous System Architecture - HSA* in 2012) was to develop a SoC that combines a CPU with a GPU on a single die started in 2006 and was moved forward by that years AMD's acquisition of graphics chipset manufacturer ATI. The project reportedly required three internal iterations of the Fusion concept to create a product deemed worthy of release. The project was delayed due to technical difficulties of combining a CPU and GPU on the same die at a 45 nm process. There were also conflicting views on the role of the CPU and GPU within the project.

The 4th generation APU, based on the Zen CPU architecture and the Vega graphics architecture was released by HP in 2017.

https://en.wikipedia.org/wiki/AMD_Accelerated_Processing_Unit

Tegra is a system on a chip (SoC) series developed by Nvidia for mobile devices such as smartphones, personal digital assistants, and mobile Internet devices. The Tegra integrates an ARM CPU, GPU, northbridge, southbridge, and memory controller onto one package. Early Tegra SoCs are designed as efficient multimedia processors, while more recent models emphasize performance for gaming and machine learning applications, without sacrificing power efficiency. Nvidia announced the AGX Orin family in December 2019. Tegra contains 17 billion transistors and 12

ARM Hercules cores and is capable of 200 INT8 TOPs @ 65 W.

<https://en.wikipedia.org/wiki/Tegra>

On Oct. 24, 2020 Huawei introduced Kirin 5 nm, 5G SoC with 15.3 billion transistors. It has 8 processors: 1 Cortex A77 @ 3.13 GHz, 3 Cortex A77s @ 2.54 GHz, 1 Cortex A55 @ 2.05 GHz. 2 *Neural Processing Units* - *NPU*s with big core and 1 *NPU* with small core,

https://en.wikichip.org/wiki/neural_processor

GPU with 24 cores Mali G78. Balong 5000 modem, LPDDR 5/4X. HiFi Audio, ISP 6.0m UFS, 4K HDR video, Mobile secure processor. The article states that Kirin is 10% faster vs Qualcomm Snapdragon 865+.

https://www.phonearena.com/news/huawei-kirin-9000-5nm-5g-soc_id128007

An alphabetic list of microprocessor companies and their microprocessors is listed below.

https://en.wikipedia.org/wiki/List_of_microprocessors

Generations of microprocessors are given below.

<https://www.elprocus.com/microprocessor-history-and-brief-information-about-its-generations/>

1.29 The IBM PC and its compatibles

The IBM *Personal Computer* - *PC* which announced in August 1982 was developed in twelve months under the guidance of Philip Don Estridge.

https://www.ibm.com/ibm/history/exhibits/pc25/pc25_birth.html

https://en.wikipedia.org/wiki/IBM_Personal_Computer

IBM's PC division used to employ about 10,000 employees in Boca Raton, FL, which is home to *Florida Atlantic U.* - *FAU*.

<https://www.fau.edu>

https://www.ibm.com/ibm/history/exhibits/vintage/vintage_4506VV8004.html

Estridge and several of his colleagues died in a plane crash in Dallas in 1985, which most certainly had a negative effect on further PC developments.

<https://www.nytimes.com/1985/08/05/us/philip-estridge-dies-in-jet-crash-guided-ibm-personal-computer.html>

A history of personal computers from various companies appears below.

https://en.wikipedia.org/wiki/History_of_personal_computers

As an IBM *Research Staff Member* - *RSM* I was given an IBM PC AT - *Advanced Technology* for home use in 1990. It ran the *Microsoft Disk Operating System* - *MS/DOS*. IBM paid for a telephone landline so that I could work from home by dialing into IBM mainframes running VM/CMS. This was to encourage IBM employees to work from home after hours.

In early 1990s there were two IBM mainframes at Yorktown, a 4-way and a 2-way machine. At one point I would use all six processors to run simulations to validate the accuracy of the analysis for deadlock probabilities reported in (Thomasian and Ryu, 1991). Such simulation runs were lengthy since deadlocks are rare. My runs did not incur additional cost since computers were rarely turned off.

Idle computer cycles are used to compute mathematical constants such as π to great accuracy. The record for the number of computed π digits is 31.4 trillion and it took 121 days on 25 virtual machines on a Google cloud using y-cruncher and storing 170 TB of data (posted on 3/14/2019).

<https://science.slashdot.org/story/19/03/14/146228/google-smashes-the-world-record-for-calculating-digits-of-pi>

As of 1/29/2020 the record is 50 trillion digits taking 304 days using four Intel Xeon's and 48 HDDs.

<http://www.numberworld.org/y-cruncher/>

https://en.wikipedia.org/wiki/Chronology_of_computation_of_%CF%80

https://en.wikipedia.org/wiki/Approximations_of_%CF%80

Pi calculated to 62.8 trillion digits' with a pair of 32-core AMD Epyc chips, 1 TB RAM, 510 TB disk space. Switzerland's Univ. of Applied Sciences Graubund broke the world record after 108 days and 9 hours. A pair of 32-core AMD Epyc 7542 processors powered the uni's rig. AMD states the CPU cores spend most of their time at 2.9 GHz, can burst to 3.4 GHz, have 128 MB L3 cache and happily run 64 threads apiece. A server with 1 TB of RAM was also employed, with Ubuntu Linux 20.04. A *Just a Bunch of Disks* - *JBOD* with 38 7200 RPM hard disks, each with 16 TB capacity.

I used the PC as a dumb terminal, although it could have been used to develop C/C++ programs. All my programs written at IBM Research up to a certain point were written in PL/I to run on the VM/CMS system. In hindsight I wish I had written my programs in C or C++ and also used the Latex for text formatting instead of IBM's SCRIPT language. both of which were portable and would have allowed me to rerun my programs and extend the latex papers after I left IBM.

Iomega zip drives were used to transport files between disconnected personal computers, before the advent of the *universal serial bus - usb*, *Secure Digital - SD* drives, and Wi-Fi.

<https://www.premiumusb.com/blog/differences-between-usb-flash-drive-vs-sd-card>

<https://en.wikipedia.org/wiki/Wi-Fi>

The 100 MB capacity of the Iomega zip drives was a major improvement over 1.44 MB floppy disk drives, but zip drives turned out to be unreliable and would fail suddenly, which resulted in a drop in zip drive sales and Iomega stock price fell fifty-fold from \$100 to \$2.

https://en.wikipedia.org/wiki/Zip_drive

EMC bought Iomega in 2008 and in 2013 started a joint venture with Lenovo, hence LenovoEMC stands for Iomega.

<https://en.wikipedia.org/wiki/LenovoEMC>

IBM was a vertically integrated company developing all important hardware and software itself, but only purchasing a few components such as transformers. This changed with the IBM PC project code-named Chess, since it disclosed detailed product specifications.

<https://arstechnica.com/gadgets/2017/06/ibm-pc-history-part-1/>

<https://arstechnica.com/gadgets/2017/07/ibm-pc-history-part-2/>

The IBM PC and its compatibles used Intel x86 microprocessors with Microsoft's MS/DOS operating system, which was later replaced with Microsoft Windows. They were called Wintel machines with the software coming from Microsoft and the hardware from Intel. The latest version of Windows following Windows 10 was announced on 6/24/2021 at 11am. This following manual describes the components of the IBM Personal Computer XT.

www.retroarchive.org/dos/docs/ibm5160techref.pdf

The IBM PC was successful because of IBM reputation for quality and its size was an assurance that the IBM PC will become a standard.

https://en.wikipedia.org/wiki/IBM_Personal_Computer

IBM OS/2 introduced in 1987 was supposed to be a replacement to MS/DOS running on the PS/2.

<https://www.howtogeek.com/688970/what-was-ibms-os2-and-why-did-it-matter/>

A deciding factor in the battle of OS/2 2.0 (\$195) and Windows 3.1 (\$150) in 1992 was that consumers perceived OS/2 as a product specifically for IBM's more expensive PS/2, Windows 3.1, however, could run on cheaper, mass-market machines discussed in what follows.

HP, Dell, Compaq, Gateway 2000, Acer entered the PC market with IBM PC compatibles.

Gateway founded in a South Dakota farm used a direct-sale method for its under \$1000 PCs, which was a low price at the time. Gateway later moved to Los Angeles and was acquired by the Taiwan's Acer company in 2007.

https://en.wikipedia.org/wiki/Gateway,_Inc.

Compaq was founded by three *Texas Instruments - TI* engineers in 1982.

https://en.wikipedia.org/wiki/Texas_Instruments

TI was a leader in semiconductor technology and the first to develop calculators in mid-1960s.

<https://www.nytimes.com/2019/03/07/obituaries/jerry-merryman-dead.html?action=click&module=Well&pgtype=Homepage§ion=Obituaries>

Compaq cofounder and CEO Rod Canyon claims in (Canyon, 2013) that at the time Compaq was founded PC software was not standardized, but that his team made it possible. Compaq later bought Tandem and DEC, but was taken over by HP for \$25 billion in 2002.

<https://www.cnet.com/news/hp-to-acquire-compaq-for-25-billion/>

Dell was created by a U. Texas at Austin student Michael Dell in his dorm-room. He dropped out of school to focus full-time on his fledgling business, The company produced the first computer the Turbo PC in 1985, which sold for \$795. The company grossed more than \$73 million in its first year of operation.

<https://en.wikipedia.org/wiki/Dell>

Given that IBM PC business was losing money with its unsuccessful third generation PS/2 PC line, with OS/2,

https://en.wikipedia.org/wiki/IBM_Personal_System/2

IBM sold its x86 PC business to Lenovo in 2005 for \$1.75 billion, while retaining 18.9% partial ownership.

<https://www.cnet.com/news/ibm-sells-pc-group-to-lenovo>

Market shares of personal computer vendors since 1996 are given here and the ranking for 2020 is reproduced in Table 1.14.

https://en.wikipedia.org/wiki/Market_share_of_personal_computer_vendors

TABLE 1.14 Top 6 vendors by number of units shipped, 2020. HP acquired Compaq in 2002. Asus was ranked #1 2016-19.

Rank	Manufacturer	Market Share
1	Lenovo	24.9%
2	HP	21.2%
3	Dell	16.4%
4	Apple	8.2
5	Asus	6.0
6	Acer	5.9%

Acer and Asus are both Taiwanese multinationals.

<https://en.wikipedia.org/wiki/Asus>

https://en.wikipedia.org/wiki/Acer_Inc.

IBM had invested billions of dollars into developing high-end x86 servers with some of the big iron power, reliability, flexibility of its higher-end servers, but at the same time, companies like Google charted a new course in which reliability was provided by software that could withstand the failure of a few systems. What was worse for traditional server makers, companies like Google and Facebook make their own servers rather than buying them already manufactured.

<http://newsdesignlogo.blogspot.com/2014/01/an-expected-deal-to-sell-off-its-intel.html> Ten years after the PC later IBM sold its Wintel server business also to Lenovo.

<https://www.cnet.com/news/ibm-sells-its-x86-server-business-to-lenovo-for-2-3-billion/>

<https://en.wikipedia.org/wiki/Lenovo>

1.29.1 Experience with IBM workstations

I was given a *RISC Technology Personal Computer - RT/PC* in 1991, which was equipped with a 10 MHZ ROMP micro-processor, which was introduced in 1986 with “not enough power, too high a price, and too late” by industry watchers.

https://en.wikipedia.org/wiki/IBM_RT_PC

https://en.wikipedia.org/wiki/IBM_6150_RT

It also made my office at H4-B30 at IBM’s leased building on Skyline Drive in Hawthorne, NY uncomfortably hot. IBM terminated the lease of the five-story building in 2005 and remaining staff were moved to IBM’s Yorktown building. The Hawthorne building is now leased to the Westchester Medical Center.

<https://www.westchestermedicalcenter.org/>

I gave the workstation to a summer student, since it ran AIX IBM’s version of UNIX which is popular in academia. The student shared a large room with other students where heat could be dissipated.

At a later point in 1996 I was given an *RS/6000 Performance Optimized With Enhanced RISC - POWER* workstation introduced in 1990 for use in a NASA-sponsored project.

<https://en.wikipedia.org/wiki/RS/6000>

https://en.wikipedia.org/wiki/IBM_POWER_microprocessors

The goal of the project was to index satellite images characterized by 60-dimensional vectors extracted from image textures. *Content Based Image Retrieval - CBIR* using *k*-nearest neighbor search. A multi-dimensional indexing structure does not work beyond 16 dimensions so that a combination of dimensionality reduction and clustering was used to build the index as described in Chapter 8.

The RS/6000 workstation came with a large color monitor, which was nice to have in the early days of the *World Wide Web - WWW*. Yahoo! displayed indices, e.g., a list of universities, which helped the user with accessing the desired university. At a later point Google provided a powerful search engine which is nowadays used in > 90% of queries.

https://en.wikipedia.org/wiki/Timeline_of_web_search_engines

The Internet Archive called the Wayback Machine, allows the user locate web sites which are no longer available.

<https://web.archive.org>

Before joining the NASA-sponsored project I took a two-day AIX course, which was IBM's version of UNIX at IBM's Learning Center in Thornwood, NY. AIX was developed for RS/6000 workstations, but later extended to run on System Z mainframes.

https://en.wikipedia.org/wiki/IBM_AIX

IBM mainframes also run Linux, which is standardized but AIX outperforms Linux by 5-10% according to this web page.

<https://www.linkedin.com/pulse/aix-vs-linux-hossam-moghazy/>

In 2019 IBM acquired RedHat for \$34 Billion, which provides Linux for cloud services.

IBM sold the Learning Center property to a catholic monastery, which also went out of business.

<http://www.catholicplaces.org/legionaries.htm>

IBM used to have numerous education centers throughout USA and worldwide, e.g., at La Hulpe, Belgium. As interest in IBM systems and software diminished most of these centers were shuttered. Today Learnquest provides training for IBM, Lenovo, Microsoft, Infor employees. Learnquest courses are listed here,

https://www.learnquest.com/vendor-training.aspx?tp_id=19

e.g., this is the Learnquest course on IBM storage systems.

<https://www.learnquest.com/course-detail-v3.aspx?cnum=SS01G>

At U. Conn. at Stamford which I joined for two years as part of my IBM early retirement plan, I was given a Wintel computer to log onto an IBM mainframe with VM operating system at U. Conn. Storrs. I started writing an *National Science Foundation - NSF* proposal which I completed and submitted after joining *New Jersey Institute of Technology - NJIT*.

At NJIT I asked a masters student to develop and run simulations of fork/join queueing systems to extend the approximations reported in (Thomasian and Tantawi, 1994). The runs on NJIT's *Silicon Graphics Inc. - SGI* "supercomputer" would not complete. It turned out the computer was reset at midnight to kill runaway programs with endless loop. It took us a while to figure out what is happening to our program.

The *Compute Express Link - CXL* is being developed to supersede the PCIe bus and is envisaged by its developers as making pools of memory (DRAM + *Storage Class Memory - SCM*) sharable between CPUs and also GPUs; but not *High Bandwidth Memory - HBM*.

<https://blocksandfiles.com/2021/03/25/cxl-and-the-developing-memory-hierarchy/>

https://en.wikipedia.org/wiki/High_Bandwidth_Memory

1.30 Storage studies by Alan Jay Smith at Berkeley

Alan Jay Smith - AJS at UCB carried out substantial studies on computer performance. His work on page replacement algorithms was discussed earlier. His research with Windsor W. Hsu is discussed in Chapter 3.

It is observed in (Smith, 1978a) that in a batch environment few disk files are "opened" per job and are read or written sequentially and that with many disk and low MPL there are few active files per disk. Sequential access to the files may have resulted from the fact that they were not indexed or hashed (Gray and Reuter, 1992). IBM's DOS and or MVS operating systems allocate files on contiguous disk cylinders, so that disk arm movement is rarely required for sequential disk accesses. This conforms with earlier measurement of disk arm movements lengths in a 2314 disk storage facility carried out by William Lynch at *Case Western Reserve U. - CWRU* that disk arms do not move for 63% of accesses (Lynch, 1972).

Disk seeks can be reduced by using multiple arms, so that if there are several open files on a given spindle, there will be a dedicated arm for each open file. At most two arms per disk have appeared in products, most notably Seagate's multi-arm disk technology, with each arm accessing half of disk cylinders.

<https://www.extremetech.com/computing/260946-seagates-new-multi-actuator-technology-double-hard-drive-speeds>

Trace driven simulation is used in (Smith, 1978a) to determine the effectiveness of both buffers and multiple arms. It is shown that a minimum total buffer capacity of twenty tracks is adequate in buffered disks.

Sequential accesses are an inherent characteristic of database and file systems. An algorithm which selectively prefetches data blocks is developed in (Smith, 1978b). The number of blocks prefetched is chosen by using an empirical run length distribution and conditioning on the observed number of preceding sequential block references. The optimal number of blocks to prefetch is estimated as a function of the following costs. (1) misses, (2) fetching additional data blocks at fault times, (3) prefetched blocks that are not referenced. Trace data taken from an operational IMS database system is analyzed and it is shown that anticipatory fetching of data can lead to significant improvement in turnaround time. A bibliography on paging and related topics is given in (Smith, 1978c).

https://en.wikipedia.org/wiki/IBM_Information_Management_System

Analysis of long term file reference patterns for application to algorithms for file migration to *Mass Storage Systems* (MSSs) is presented in (Smith, 1981b). In most large computer installations files are moved between online disks and MSS either by the system and/or by the user. Presented and analyzed are long term file reference data for the construction of algorithms for file migration. The use of the online user datasets at the *Stanford Linear Accelerator Center - SLAC* is examined. The following conclusions are drawn for files accessed sufficiently frequently:

- (1) about a third show declining rates of reference during their lifetime,
- (2) of the remainder, about 5% show correlated interreference intervals,
- (3) these intervals measured in days are more skewed than would occur with the Bernoulli process.

A large number of topics are covered in (Smith, 1981a): block size choice, data set placement, disk arm scheduling, rotational scheduling, compaction, fragmentation, I/O multipathing and file data structures.

A bibliography on file and I/O system optimization and related topics is discussed in (Smith, 1981c). The file system, and the components of the computer system associated with it (disks, drums, channels, mass storage, tapes and tape drives, controllers, I/O drivers, etc.) used to comprise a very substantial fraction of the cost of most computer systems. Summarized is the research on these two topics:

- (1) Cache disks which buffer recently accessed tracks.
- (2) File migration between disks and MSS in order to effectively maintain online a much larger amount of information than the disks can hold.

Automatic file migration is investigated in (Smith, 1981b). Files are moved dynamically between MSS and disks in response to user reference patterns over a long period. A replacement algorithm is developed and evaluated for selecting files to be moved from disk to MSS. Algorithms that work well are based on both the file size and the time since the file was last accessed. The best realizable algorithms utilize the distribution of the times between file references. Acceptable results are obtained by selecting for replacement files whose size multiplied by time to most recent reference is maximal. Comparisons are made with a number of standard algorithms developed for paging, such as Working Set, VMIN, and *Generalized OPT- GOPT*. Sufficient information (parameter values, fitted equations) is provided so that algorithms may be easily implemented on other systems.

Caching is required to alleviate the mismatch between CPU and disk speeds. Disk cache miss ratios and design considerations are studied via trace-driven simulations in (Smith, 1985). An 8 MB cache captures 80-90% of all I/O requests for the workload under consideration. The following cache parameters are considered: cache location, storage controller vs main memory, size, migration algorithms, block sizes, and other issues: access time, bandwidth, multipathing, technology, consistency, error recovery. Experiments are based on trace-driven simulation.

Measurements on machines and program analysis was used to predict program running time in (Saavedra and Smith, 1995):

- (1) developing a high level program to measure the design and performance of the cache and *Translation Lookaside Buffer - TLB* units;
- (2) using those measurements, along with published miss ratio data to improve the accuracy of runtime predictions;
- (3) using analytic tools and measurements to study and compare the design of several machines.

The design and performance of the cache and TLB for ten machines is described.

The results of a variety of trace-driven simulations of disk cache designs in timesharing and database systems are presented in (Zivkov and Smith, 1997). Computed are miss ratios, run lengths, traffic ratios, cache residency times, degree of memory pollution and other statistics for a variety of designs, varying block size, prefetching algorithm and write algorithm. It is determined that sequential prefetching produces a significant improvement in the miss ratio. Copy-back writing decreases write traffic relative to write-through by more than 50%; periodic flushing of the dirty blocks increased write traffic only slightly compared to write-back for large cache sizes. Block sizes larger than a track have been determined not to be useful. Write-allocate had little effect compared to no-write-allocate. No-write allocate: data at the missed-write location is not loaded to cache, and is written directly to the backing store. In this approach, data is loaded into the cache on read misses only.

Workload modeling by an author who researched file system design and scheduling in parallel systems is discussed in (Feitelson, 2015).

Out of core algorithms

External memory or out-of-core algorithms were designed to process data that is too large to fit into a computer's main memory at one time. Such algorithms are optimized to efficiently fetch and access data stored on disk, tape, or accessible via a computer network.

In the case of out-of-core computations, disk storage is treated as a lower level in the memory hierarchy. A comprehensive review of out-of-core algorithms is given in (Vitter, 2001). An external memory model similar to the *Random Access Machine - RAM* model, but with a cache in addition to main memory is proposed in (Aggarwal and Vitter, 1988).

https://en.wikipedia.org/wiki/Random-access_machine

https://en.wikipedia.org/wiki/Jeffrey_Vitter

The model captures the fact that read and write operations are much faster in a cache than in main memory, and that reading long contiguous blocks is faster than reading random blocks. The running time of an algorithm in the external memory model is defined by the number of memory reads and writes.

1.31 Prefetching

Prefetching in computer systems is implemented at multiple levels. The disk and main memory interface is considered here. A primitive form of prefetching is double buffering available to file systems in programming languages such as COBOL and PL/I and DBMSs, which allows the next block to be accessed from disk, while the current block is being processed by the CPU.

https://en.wikipedia.org/wiki/Multiple_buffering

Prefetching to onboard disk cache is carried out by the disk controller, e.g., when a disk block is read the remaining sectors on the track are cached. Prefetching can be carried out by the cache associated with the RAID controller. Finally prefetching can be initiated by the DBMS. Prefetching from disks is discussed in detail in Section 22.3 in (Jacob et al., 2008).

Aside from prefetching studies by Alan Jay Smith at UCB (Smith, 1978b), two other major studies of were are follows.

- (1) Princeton U.: Pei Cao (C), Ed Felten (F), Anna Karlin (K), and Kai Li (L).
- (2) Parallel Data Lab at CMU: R. Hugo Patterson (P), Fay Chang (Ch), Andrew Tomkins (T), and Garth Gibson (G).

The implementation and performance of application-controlled file caching replacement decisions are presented by Pei Cao et al. in (Cao et al., 1994a). Ordinarily processes are not allowed to control the cached pages of traditional file systems, while the kernel's LRU-SP algorithm (described below) fairly allocates cache space among processes. This combination can improve performance significantly: the number of block I/Os by 80% and elapsed time by 45%.

Caching and prefetching to improve the performance of file systems in an integrated manner are studied by the aforementioned and Anna Karlin in (Cao et al., 1995). Optimal integrated strategies for prefetching and caching must satisfy certain properties. Traces of file accesses are used to evaluate conservative and aggressive approaches and it is shown that the two integrated prefetching and caching strategies are close to optimal and reduce the running time of applications by up to 50%.

As the performance gap between disks and processors continues to increase effective utilization of the file cache becomes increasingly important. Application-controlled file caching and prefetching can apply application-specific knowledge to improve file cache management (Cao et al., 1996). Presented is the design, implementation, and performance of a file system that integrates application-controlled caching, prefetching, and disk scheduling. In a two-level cache management strategy: the kernel uses the LRU-SP (Least-Recently-Used with Swapping and Placeholders) policy to allocate blocks to processes, while each process integrates application-specific caching and prefetching based on the controlled-aggressive policy. Each process also improves its disk access latency by submitting prefetches in batches so that the requests can be scheduled to optimize disk access performance. Measurements show that this combination of techniques greatly improves the performance of the file system and the measured running time is reduced by 3% to 49% with a 26% average for single-processors.

Exploiting application level hints about future I/O accesses to reduce file read latencies by caching and prefetching is investigated in (Patterson et al., 1993). Systems that disclose high-level knowledge can transfer optimization information across module boundaries. *Transparent Informed Prefetching - TIP* provides techniques for converting the high throughput of new technologies such as RAID and *Log-Structured File Systems - LFS* into lower latency for applications. Experiments show that TIP yields reduced execution time of up to 30% (resp. 13%) for applications obtaining data from a remote file server (a single local disk). These experiments indicate that greater performance benefits will be available when TIP is integrated with low level resource management policies.

It is shown in (Patterson et al., 1995) how to use hints, i.e., application-disclosed access patterns to expose and exploit I/O parallelism and to dynamically allocate file buffers among competing demands:

- (1) prefetching hinted blocks,
- (2) caching hinted blocks for reuse, and
- (3) caching recently used data for unhinted accesses.

The study estimates the impact of alternative buffer allocations on application execution time and applies cost-benefit

analysis to allocate buffers where they will have the greatest impact. Experiments are carried out on a 150 MHz DEV Alpha microprocessor for VAX computers equipped with fifteen disks running Digital's OSF/1 in 1992.

https://en.wikipedia.org/wiki/Alpha_21064

Informed prefetching reduces the execution time of various applications by 20-83%.

TIP and caching manager takes advantage of application-generated hints to better utilize the file cache and I/O resources (Patterson et al., 1995). The TIP benchmark suite is evaluated with three real-world, disk-bound applications without any manual modifications achieves 29%, 69% and 70% reductions in execution time with data files striped over four disks, improving performance by the same amount as manually-hinted prefetching for two out of three applications. Simulation is used to estimate performance as affected by the widening gap between processor and disk speeds.

Informed prefetching and caching based on application disclosure of future I/O accesses can dramatically reduce the execution time of I/O-intensive applications according to (Tomkins et al., 1997). The paper shows how to incorporate adaptivity to disk load into the TIP2 system, which uses cost-benefit analysis to allocate global resources among multiple processes. TIPTOE (TIP with Temporal Overload Estimators - TOE) delivers 7% lower execution time compared to LRU-SP allocation scheme, also modified to include adaptive prefetching.

Aggressive prefetching is an effective technique for reducing the execution times of disk-bound applications by manipulating large and infrequently used uncached data was proposed in (Chang and Gibson, 1999). While automatic prefetching approaches based on static analysis or historical access patterns are effective for some workloads, they are not as effective as programmer-inserted prefetching for applications with irregular or input-dependent access patterns.

The authors also proposed a scheme to dynamically analyze an application and predict its future I/O accesses, while the processor is stalled due to an I/O. The approach used is to speculatively pre-execute the application's code in order to discover and issue hints for its future read accesses. Coupled with an aggressive hint-driven prefetching system, this automatic approach could be applied to arbitrary applications and should be particularly effective for those with irregular and input-dependent access patterns. The SpecHinty binary modification tool was designed and implemented to transform UNIX application binaries to perform speculative execution and issue hints.

1.32 Database buffers

Allocating database buffers in main memory is discussed in (Tuel, 1976). Large database buffers allocated in virtual memory, which are not backed up by real memory may result in the double paging anomaly when a page loaded into the buffer is paged out and is referenced again.

Three models with different assumptions are analyzed in (Lang et al., 1977b) in the context of a database buffer in a paging environment. The analysis is valid for arbitrary database and buffer sizes, and the I/O cost is obtained in terms of the miss ratio, the buffer size, the number of main memory pages available for the buffer, and relative buffer and database costs. The effect of main memory replacement algorithms on the number of main memory page faults is reported by the above authors in (Fernández et al., 1978).

The impact of RDBMs buffer managers on performance was investigated in (Sacco and Schkolnick, 1982).

- (1) Pages of a long sequential data scan will go to the top of the stack, causing the pages used by other processes possibly short txns to be flushed out of the buffer.
- (2) LRU behaves badly when a process cycles through a larger number of pages that cannot fit in the buffer and the result is internal thrashing with every reference causing a page fault.
- (3) A problem occurs in multiuser systems when one process uses pages more rapidly than another that has a looping behavior as it references pages. Even if the set of pages that are re-referenced inside the loop may be smaller than the buffer size, the "stealing" of pages by the first process effectively reduces the available frames in the buffer for the second process. This reduction can be such that the second process generates a page fault on every request. This effect is called external thrashing.

The Hotset algorithm exploits query behavior so that a set of pages that are accessed repeatedly form a hotset (Sacco and Schkolnick, 1982), (Sacco and Schkolnick, 1986). Admission control does not let a query into the system unless its hotset fits in memory. If the access plan is executed within a buffer whose size exceeds the size of the hot sets, processing will be fast, as the pages that are re-referenced in a loop will stay in the buffer. On the other hand, if the buffer size is below the value required to contain a hot set, the number of page faults required grows rapidly.

The implementation of a database buffer manager as a component of a DBMS and the interface between higher DBMS layers and the buffer manager is described in (Effelsberg and Härder, 1984). A difference between virtual memory paging and buffer management is the fixing of pages in the buffer to prevent uncontrolled replacement. Three basic tasks have to be performed by the buffer manager: search, allocation of frames, and page replacement.

Buffer management strategies for relational databases are discussed in (Chou and DeWitt, 1985). DBMIN is based on a model of relational query behavior: the *Query Locality Set Model - QLSM*. Like the hot-set model QLSM can predict future reference behavior and avoids potential problems of the hotset model by separating the modeling of reference behavior from any particular buffer management algorithm. The performance evaluation methodology for evaluating buffer management algorithms in a multiuser environment combines trace and distribution driven simulation models. Using this model DBMIN is compared with four traditional buffer replacement algorithms, the Working Set (Denning, 1968a) and Hotset algorithm. The design and implementation of a database buffer is described in (Chou et al., 1985).

1.33 Checkpointing in processing large jobs

Batch jobs require large amounts of CPU and I/O processing. The performance metric for batch jobs is throughput, i.e., number of jobs completed per unit time. The long running time of batch jobs may exceed computer's *Mean Time To Failure - MTTF*. Checkpointing solves this problem by saving the state of a computation periodically. This allows the computation to make progress by resuming from its last checkpoint if the computer crashes.

https://en.wikipedia.org/wiki/Application_checkpointing

A coordinated checkpointing algorithm was proposed in (Chandy and Lamport, 1985), which is based on global state detection accomplished using the *two-phase commit - 2PC* protocol (Gray and Reuter, 1992), (Ramakrishnan and Gehrke, 2002).

Checkpointing performance can be improved by latency hiding, which is accomplished by buffering, so that the process continues its operation while the checkpoint is being taken (the improvement is illustrated in Figure 6 in (Chandy and Lamport, 1985)).

With copy-on-write buffering the protection of all pages is set to read-only at checkpoint time. The application is resumed and a separate thread starts storing pages to the checkpoint. After a page is stored its protection is set to read-write. If there is an access violation the offending page is copied into a buffer and the protection is set to read-write. The checkpointing thread checkpoints the copy from the buffer and deallocates the page after it is done.

Memory exclusion is not to checkpoint read-only variables. Checkpoint compression lowers checkpointing overhead if it takes less time than the time saved for checkpointing.

Efficient checkpointing in MIMD computers is reported in a PhD thesis by James Plank supervised by Kai Li.

<http://web.eecs.utk.edu/~jplank/plank/papers/thesis.html>

<http://web.eecs.utk.edu/~jplank/>

An overview of checkpointing in uniprocessor and distributed systems, focusing on implementation and performance is given in (Plank, 1997b). With a single computer, the checkpoint saves global variables, the program stack, the registers and the program counter to disk, but in the case of message passing system there is the issue of obtaining a *consistent checkpoint* as shown in Figure 3 in (Plank, 1997b).

Rollback recovery techniques in message-passing systems that do not require special language constructs are surveyed in (Elnozahy et al., 2002a). Such techniques are classified into checkpoint-based and log-based. The former relies solely on checkpointing for system state restoration and can be coordinated, uncoordinated, or communication-induced. Log-based protocols combine checkpointing with logging of nondeterministic events, encoded in tuples called determinants. Depending on how determinants are logged, log-based protocols can be pessimistic, optimistic, or causal.

Rollback-recovery via checkpoint-restart has been used with reasonable success for applications running from hours to months in the past two decades. There are several systems and publicly available libraries to support checkpointing, which can be used by programmers, unless they provide own checkpointing support. The projections predict that unlike current practice the effect of rollback-recovery may play a more prominent role in how systems may be configured to reach the desired performance level. System planners may have to devote additional resources to enable rollback-recovery and the current practice of using “cheap commodity” systems to form large-scale clusters may face serious obstacles.

Checkpointing for petascale systems and the future of practical rollback-recovery is discussed in (Elnozahy and Plank, 2004). Conclusions of this study are discussed in Table 1.15. Petascale systems which can process 10^{15} FLOPS

https://en.wikipedia.org/wiki/Petascale_computing

<https://en.wikipedia.org/wiki/FLOPS>

Checkpointing applied to optimistic Concurrency Control

Restart-oriented *Concurrency Control - CC* methods, such as optimistic CC outperform blocking-oriented methods, such as standard two-phase locking in a high data contention environment at the cost of wasted CPU processing due to restarts.

TABLE 1.15 Summary of conclusions according to Table 1 in (Elnozahy and Plank, 2004) (trans. - transient), (perm. - permanent).

System node size	Scalability required	Resilience to trans. fail.	Nodes devoted spare cycles	Resilience to perm. fail.	Relative cost for 1PFLOP/sec
Large	100s	Best	Few	Poor	Highest
Mid size	1,000s	Good	Average	Acceptable	High
Small	10,000s	Acceptable	Average	Good	Moderate
Smallest	100,000s	Poor	Many	Best	Cheapest

There is a tradeoff between the checkpointing overhead and wasted processing. In the case of restart-oriented kill-based optimistic CC method performance can be improved by using checkpointing in the form of low cost savepoints in volatile main memory (Thomasian, 1995b). The possibility of data conflict and txn restart increases with its age, since the txn has accessed more objects. The analysis shows significant improvement in performance at high data conflict levels is attainable when accesses to database hot-spots are deferred.

1.34 Computer related rule of thumb

Rules of thumb proposed by prominent computer scientists and engineers are discussed in this section. These are mostly eponymous laws named after the person who stated them.

https://en.wikipedia.org/wiki/List_of_eponymous_laws

1.34.1 Amdahl rules in developing S/360 computers

Gene Amdahl as a key architect of IBM S/360 used the following laws in implementing S/360.

https://en.wikipedia.org/wiki/Gene_Amdahl

The longevity of the S/360 architecture (with modifications) is a testimony to the soundness of design, but this does not mean that these rules are applicable to computer design today. A major difference with early computers is multiprocessing, CPU caches, low latency flash memories, applications accessing huge data volumes, etc.

Balanced System Law: One byte of main memory per instruction per second, e.g., one MB main memory for a one *Millions of Instructions Per Second* computer. Setting $\alpha = \text{MB/MIPS} = 1$ results in a balanced system.

Amdahl's I/O Law I: One bit of I/O per second for each instruction per second, e.g., 1 MbPS for 1 MIPS.

Amdahl's I/O Law II: Programs do one I/O per 50K instructions and for sequential I/O the number of instructions is higher.

Amdahl's Parallel Processing Law to discredit parallel computing at a time when IBM did not produce parallel computers (Amdahl, 1967). Amdahl restated the law half a century later (Amdahl, 2013).

https://en.wikipedia.org/wiki/Amdahl%27s_law

The execution time of a program which is executed sequentially takes T time units: $T = T_s + T_p$, The portion taking T_s time units can only be executed sequentially, while the other portion taking T_p time units can be executed at a degree of parallelism p , Let $f = T_p/T$ and $T_s = (1 - f)T$ then the execution time of the program is $(1 - f)T + fT/p$ and the speedup:

$$\text{Speedup} = \frac{1}{1 - f + f/p},$$

With $f = 0.5$ and $p \rightarrow \infty$, $S_{max} \rightarrow 2$, i.e., the speedup has an upper limit of two regardless of the degree of parallelism.

There are several variations of Amdahl Law, most notably Gustafson law (Gustafson, 1988).

https://en.wikipedia.org/wiki/Gustafson%27s_law

1.34.2 Amdahl's law in the era of multicore

It is noted in (Hill and Marty, 2008) that in developing his law Amdahl assumed that a fraction f of a program's execution time is infinitely parallelizable with no overhead, while the remaining fraction, $1 - f$, was totally sequential With n

TABLE 1.16 Upper bounds on speedup, $f = 0.99$.

#BCEs	Basic	Symmetric	Asymmetric	Dynamic
16	14	14	14	<16
64	38	39	49	<60
256	72	89	166	<223
1024	91	161	531	<782

processors or cores in a multicore system the speedup is

$$S_{basic}(f, n) = [1 - f + f/n]^{-1}.$$

We first assume that a multicore chip can contain at most n *Base Core Equivalents* - BCEs. A chip may be constrained to n BCE due to limited memory or power or a combination. Given that the performance of a single-BCE core is 1, it is assumed that architects can expend the resources of r BCEs to create a rich core with sequential performance $perf(r)$.

The three systems considered in this study are:

- (1) Symmetric with sixteen 1 BCE cores.
- (2) Symmetric with four 4 BCE cores.
- (3) Asymmetric one 4 BCE and twelve 1 BCE cores.

In symmetric case the chip uses one core to execute sequentially at performance $perf(r)$. It uses all n/r cores to execute in parallel at performance $perf(r)*n/r$.

$$S_{symmetric}(f, n, r) = \left[\frac{1-f}{perf(r)} + \frac{f \cdot r}{perf(r)n} \right]^{-1}.$$

In asymmetric multicore case the chip uses the one core with more resources to execute sequentially at performance $perf(r)$. In the parallel fraction, however, it gets performance $perf(r)$ from the large core and performance 1 from each of the $n-r$ base cores:

$$S_{asymmetric}(f, n, r) = \left[\frac{1-f}{perf(r)} + \frac{f}{perf(r) + n - r} \right]^{-1}.$$

A dynamic multicore gets performance n using all base cores in parallel.

$$S_{dynamic}(f, n, r) = \left[\frac{1-f}{perf(r)} + \frac{f}{n} \right]^{-1}.$$

Upper bounds to speedup are given in (Table 1.16). $perf(r) = \sqrt{r}$ is used in numerical results and graphs.

1.34.3 Amazon optimal configurations for x86-based EC2 instances

The new Amazon EC2 (Elastic Cloud Compute) M6i instances announced on 8/16/21 are supposed to provide a better balance of compute, memory, storage and network resources than M5i (see Table 1.17). They are built on the AWS Nitro System, which is a combination of dedicated hardware and a lightweight hypervisor that works to offload traditional virtualization functions, ensuring the instances deliver more performance with higher availability.

<https://siliconangle.com/2021/08/16/amazon-debuts-new-generation-x86-based-workhorse-instances/>

The Amazon EC2 M6i instances are available in nine sizes with 2, 4, 8, 16, 32, 48, 64, 96, and 128 virtual CPU options.

The Amazon EC2 M6i instances are the highest performance Intel-based Amazon EC2 instances. They support up to 128 virtual central processing units (vCPUs) per instance, with a 4:1 ratio of memory to vCPU. The M6i instances support up to 40 Gbps bandwidth to Amazon EBS (Elastic Block Store - SDD or HDD) in the largest size, and up to 10 Gbps in the four smallest sizes, which is more than double that of the previous-generation M5 instances. The M6i instances also support up to 50 Gbps networking, twice that of M5 instances.

Customers can also enable Elastic Fabric Adapter (EFA) on the 32xlarge size of M6i. These instances also provide up to 20% higher memory bandwidth per vCPU compared with M5 instances, allowing customers to efficiently perform real-time analysis for data-intensive artificial intelligence/machine learning, gaming and high performance computing applications.

TABLE 1.17 Amazon EC2 offerings, EBS=SSD or HDD.

Name	vCPUs	Memory GiB	Network bandwidth Gbps	EBS throughput Gbps
m6i.large	2	8	up to 12.5	up to 10
m6i.xlarge	4	16	up to 12.5	up to 10
m6i.2xlarge	8	32	up to 12.5	up to 10
m6i.4xlarge	16	64	up to 12.5	up to 10
m6i.8xlarge	32	128	12.5	10
m6i.12xlarge	48	192	18.755	15
m6i.16xlarge	64	256	25	20
m6i.16xlarge	96	384	37.5	30
m6i.16xlarge	128	512	50	40

1.34.4 Kung's law

The law is related to H. T. Kung's research on systolic arrays.

https://en.wikipedia.org/wiki/Systolic_array

To balance an array of p linearly connected PEs for performing matrix computations such as matrix multiplication and matrix triangularization, the size of each PE's local memory must grow linearly with p . Thus, the larger the array is, the larger each PE's local memory must be (Kung, 1986).

1.34.5 Brooks' law

Fred Brooks with Gene Amdahl and Gerritt Blaauw was a former architect of S/360 (Amdahl et al., 1964), (Blaauw and Jr., 1997) and the author of "Mythical Man-Month" (Brooks, 1995), where he made this observation about software project management: "adding manpower to a late software project makes it late", i.e., each additional person makes the project take more rather than less time.

https://en.wikipedia.org/wiki/Brooks%27s_law

1.34.6 Patterson et al.'s roofline bound

The roofline model (Williams et al., 2009) provides a performance estimation to programmers and architecture designers by using bottleneck analysis to visually predict realistic performance estimations. The original model expresses the maximum performance of an application running on an architecture as a function of the *Computational Performance* - CP of the architecture and the reachable peak I/O bandwidth (BW). The CP consists of the maximum number of GFLOPs, that the processor or the functional unit is able to achieve.

The *Computational Intensity* - CI relates the CP and the BW, reflects the complexity of the algorithm, being the number of operations executed per byte accessed from memory. This model identifies the CP or the I/O BW as limiting factors. Therefore, the maximal attainable performance is always below the roofline obtained from both parameters:

$$\text{Attainable performance} = \min(CP, CI \times BW)$$

Terms and performance metrics related to the Roofline model are as follows:

https://en.wikipedia.org/wiki/Roofline_model

The work W denotes the number of operations performed by a given kernel or application and refers to *Floating Point Operations* - $FLOPs$, Work W is a property of the given kernel or application and thus depends partially on the platform characteristics.

The memory traffic Q denotes the number of bytes of memory transfers incurred during the execution of the kernel or application. In contrast to W , Q is heavily dependent on the properties of the chosen platform, such as for instance the structure of the cache hierarchy.

The arithmetic intensity I is the ratio of the work W to the memory traffic Q , i.e., $I = W/Q$, denoting the number of operations per byte of memory traffic. When the work W is expressed as FLOPs, the resulting arithmetic intensity I will be the ratio of FLOPs to total data movement (FLOPs/byte).

The naive Roofline is obtained by applying simple bound and bottleneck analysis. There are only two parameters, the peak performance (π) and the peak bandwidth (β) of the specific architecture, and one variable, the arithmetic intensity. The peak performance, in general expressed as GFLOPS, can be usually derived from architectural manuals, while the peak bandwidth, that specifically references DRAM bandwidth is instead obtained via benchmarking. The resulting plot, in general with both axes in logarithmic scale, is then derived by the following formula:

$$P = \min \begin{cases} \pi \\ \beta \times I \end{cases}$$

where P is the attainable performance, π the peak performance, β the peak bandwidth, I is the arithmetic intensity. The point at which the performance saturates at the peak performance level π is defined as ridge point, which offers insight on the machine's overall performance, by providing the minimum arithmetic intensity required to be able to achieve peak performance, and by suggesting at a glance the amount of effort required by the programmer to achieve peak performance.

A given kernel or application is then characterized by a point given by its arithmetic intensity I (on the x-axis). The attainable performance P is then computed by drawing a vertical line that hits the Roofline curve. Hence, the kernel or application is said to be memory-bound if $I \leq \pi/\beta$. Conversely, if $I \geq \pi/\beta$, the computation is said to be compute-bound.

There is a similarity to asymptotic or bottleneck analysis in (Muntz and Wong, 1974), which is discussed in Section 1.22.

1.34.7 Gray's rules of thumb

Jim Gray and Prashant Shenoy at the turn of the century enunciated the following storage and memory rules (Gray and Shenoy, 2000). Almost twenty years later it would be interesting to check if these rules apply.

1. Things get four times better every three years.
2. Extra bit of addressing needed every 18 months.
3. Storage capacities are increasing hundred-fold per decade.
4. Throughput for storage devices increasing ten-fold per decade.
5. Disk data cools ten-fold per decade.
6. Disk page size increases five-fold per decade.
7. . Nearline Tape/Online Disk/RAM storage costs are approximately 1/3/300.
8. In ten years DRAM will cost what disks cost today.
9. A single person can administer a million dollars of disk storage: 30 TB.

New generations of DRAM appearing every 3.3 years are four times larger than the previous generations, so that DRAM prices decline 50% per year. Memory prices over the years are given below.

<https://jcmit.net/memoryprice.htm>

Hard disk costs per GB up to 2009 are given at

<https://mkomo.com/cost-per-gigabyte>

Curve-fitting yields the following cost per GB:

$$\text{cost} = 10^{-0.2502(\text{year}-1980)+6.304}$$

Several terabyte drives have recently broken the \$0.10/gigabyte barrier, making the next milestone \$0.01/gigabyte, or \$10/terabyte.

Using IO density as design methodology at NetApp

IO Density is the measurement of IO generated over a given amount of stored capacity expressed as IOPS/TB. In other words, IO Density measures how much performance can be delivered by a given amount of storage capacity. With a single 7.2K RPM drive by rule of thumb it can deliver around 50 IOPS at 20 ms response time. Consider, however, that 7.2K RPM drives today can range anywhere from 1 TB to 8 TB in size. The ability of the drive to deliver 50 IOPS does not change with its size. As the size of the drive increases, the IOPS/TB ratio worsens, 50 IOPS/TB with 1 TB drive and 6.25 IOPS/TB with an 8 TB drive.

Using OnCommand Insight's IO Density report was used at NetApp to identify the range of service levels, i.e., IOPS/TB, that the apps actually needed. This was used to create a storage catalog based on three standard service levels:

Value: Services workloads requiring between 0 and 512 IOPS/TB.

Performance: Services workloads requiring between 512 and 2048 IOPS/TB.

Extreme: Services workloads requiring between 2048 and 8192 IOPS/TB.

<https://blog.netapp.com/blogs/the-importance-of-io-density-in-delivering-storage-as-a-service-part-1/>

The next step deals with placing IO limits on volumes to prevent applications from stepping on the performance resources of other applications within the same storage array. A further discussion is given below.

<https://blog.netapp.com/blogs/the-role-of-qos-in-delivering-storage-as-a-service-part-2/>

1.34.8 Jim Gray's five minute rule

This rule was first enunciated in a 1986 Tandem technical report and published in (Gray and Putzolu, 1987).

<http://www.hpl.hp.com/techreports/tandem/>

The rule of thumb is to decide whether to keep a page in main memory or release its space and read it back from disk when required. With the 1986/87 parameters the rule recommended that disk pages reused every 5 minutes or less should be cached in main memory.

https://en.wikipedia.org/wiki/Five-minute_rule

There was a counterpart one-minute rule for sequential access: "Cache sequentially accessed disk pages that are re-used every 1 minute or less".

The five minute rule was subsequently revised in 1997 (Gray and Graefe, 1997) twice again two times again after Gray's untimely death in 2007 in (Graefe, 2007, 2008), and 2017 (Appuswamy et al., 2017, 2019) to reflect changes in technology. The cost-benefit of computation is expressed as follows in (Graefe, 2007):

$$\text{Break_Even_Interval_in_Seconds} = (\text{Pages_Per_MB_of_RAM} / \text{Accesses_Per_Second_Per_Disk}) \times (\text{Price_Per_Disk_Drive} / \text{Price_Per_MB_of_RAM})$$

2007 data yields approximately a 90-minute interval for magnetic-disk-to-DRAM caching, 15 minutes for SSD-to-DRAM caching and 2 1/4 hours for disk-to-SSD caching, which means that all active data will remain in cache.

In 2017 the five-minute rule for DRAM-HDD has evolved into a four-hour rule, and the break-even interval for SSD-HDD is one day. It is concluded that all performance critical data will reside only in DRAM and SSD and HDDs will hold infrequently accessed data. This suggests that HDD and SSD vendors should target different tiers and optimization points, namely, IOPS for the performance of SSD, and high capacity cost/GB for HDDs (Tables 1.18 and 1.19).

TABLE 1.18 The evolution of page size for which the 5-minute rule holds across four decades based on appropriate price, performance, and page size values.

Year	1987	1997	2007	2018
Break even (4 KB page)	100 sec	9 min	1.5 hour	4 hour
Page size (5 min interval)	1 KB	8 KB	64 KB	512 KB

TABLE 1.19 The evolution of the break-even interval across four decades based on appropriate price, performance, and page size values.

Tier	1987	1997	2007	2018
DRAM-SSD	-	-	15 min	5 min
SSD-HDD	-	-	2.25 h	1.5 day

This talk on the five minute rule was given at the *Data Intensive Applications and Systems - DAIS* workshop (Appuswamy et al., 2017).
<http://www.adms-conf.org/2017/presentations/five-minute-rule-adms.pdf>

1.34.9 Moore's law

Robert Noyce and Gordon Moore founded Intel in 1968 as N.M. Electronics.

<https://en.wikipedia.org/wiki/Intel>

Moore's law enunciated in 1965 states that "the number of transistors per square inch on integrated circuits double each year".

https://en.wikipedia.org/wiki/Moore%27s_law

$$\# \text{transistors} = a^{\text{year}-1959}, \quad a = 2 \text{ initially } a = 1.59 \text{ now.}$$

The following is a 1996 term paper posted by Jim Gray on Moore's law, which was written as a term paper at ManTech U. in 1996.

http://jimgray.azurewebsites.net/moore_law.html

Bill Joy at Sun Microsystems according to (Patterson et al., 1987) stated in 1986:

$$\text{MIPS} = 2^{\text{year}-1984}$$

Regarding the ending of Moore's Law it has been argued in (Hennessy and Patterson, 2018) that *Domain Specific Architectures - DSA's* are a solution. Also relevant is the following article by Norman Jouppi et al. at Google in (Jouppi et al., 2018).

<https://cacm.acm.org/magazines/2018/9/230571-a-domain-specific-architecture-for-deep-neural-networks/fulltext>

The web page for the *Center for Domain Specific Computing - CDSC* led by Jason Cong at UCLA is:

<https://cdsc.ucla.edu/architectures/>

Two discussions of the ending of Moore's law are

<https://engineering.cmu.edu/news-events/news/2018/02/12-fuchs-nature-electronics.html>

<https://www.technologyreview.com/s/615226/were-not-prepared-for-the-end-of-moores-law/>

The delay in achieving 10 ns circuits at Intel is discussed in the following article.

<https://www.nytimes.com/2020/03/01/technology/intel-culture-robert-swan.html>

More recently a delay was announced in Intel's transitioning to 7 ns circuits.

<https://www.tomshardware.com/news/intel-announces-delay-to-7nm-processors-now-one-year-behind-expectations>

IBM's effort in this area is discussed here.

<https://spectrum.ieee.org/nanoclast/semiconductors/nanotechnology/ibms-3billion-research-project-has-kept-computing-moving-forward>

The following two papers appeared in IEEE Spectrum discussing Moore's law.

<https://spectrum.ieee.org/nanoclast/semiconductors/processors/the-accelerator-wall-a-new-problem-for-a-post-moores-law-world>

<https://spectrum.ieee.org/semiconductors/design/the-good-the-bad-and-the-weird-3-directions-for-moores-law>

The following article on Moore's law states that a "50-fold increase in transistor density is possible by 2030"

<https://www.futuretimeline.net/blog/2020/08/31-moores-law-future-timeline-2030.htm>

While silicon technology scaled exponentially over the past few decades, the breadth and depth of the software stack scaled at an unprecedented rate, bringing inefficiencies and leaving plenty of room for increasing performance. Raja Koduri's keynote explored opportunities in hardware-software co-design from the edge to the cloud to fully leverage the transistors that are left behind by the incredible pace of software change at the top of the stack.

<https://www.youtube.com/watch?v=QPHIXbDmIbU>

1.34.10 Wright's law

Theodore Wright an American aeronautical engineer argued in a 1936 seminal paper "Factors Affecting the Costs of Airplanes" (Wright, 1936), in which he noted that airplane production costs decreased at 10 to 15 per cent rate for every doubling of production numbers. This insight is also called the Experience Curve, since manufacturers learn through experience and become more efficient. A comparison of Moore's and Wright's law is given below.

<https://ark-invest.com/wrights-law/>

<https://ark-invest.com/articles/analyst-research/wrights-law-2/>

Given Y = cumulative average time (or cost) per unit X = cumulative number of units produced a = time (or cost) required to produce 1st unit b = slope of the function Wright's law formula is $Y = aX^b$.

Wright's law is compared with Moore's law and several others in (Nagy et al., 2012).

1.34.11 Dennard's law

Robert Dennard invented DRAM in 1956 as shown on a white board.

https://en.wikipedia.org/wiki/Robert_H._Dennard

The Dennard or MOSFET scaling states that as transistors get smaller their power density stays constant and the power use stays in proportion with area: both voltage and current scale (downward) with scaling.

https://en.wikipedia.org/wiki/Dennard_scaling

Scaling n -fold yields $O(n^2)$ transistors which run $O(n)$ faster.

According to the following web page power in CMOS chips can be modeled as:

<https://www.rambus.com/blogs/understanding-dennard-scaling-2/>

$$P = Q/CV^2 + VI_\ell.$$

Q is the number of transistors, f the frequency, C the capacitance, V the operating voltage, and I_ℓ the leakage current.

<https://www.rambus.com/blogs/understanding-dennard-scaling-2/>

With Dennard's scaling rules the total chip power for a given area size stayed the same from process generation to process generation. With a scaling factor of $\sqrt{2}$ feature size shrank at a rate of $1/S$ (the scaling ratio), transistor count doubled (Moore's Law) and the frequency increased by 40 every two years. With feature sizes below 65 nm, these rules could no longer be sustained, because of the exponential growth of the leakage current. Post-Dennard scaling leads to a power increase of $s^2 = 2$ per generation for the same die area. At the same time, utilization of a chip's computing resources decreases with a rate of $1/s^2$ per generation.

This means that at runtime large quantities of transistors on the chip have to be switched off completely, operated at lower frequencies, or organized in completely different or more energy efficient ways. For a given chip area energy efficiency can only be improved by 40% per generation. This dramatic effect, called dark silicon, already can be seen in current multicore process generations and will heavily affect future multicore and many-core processors.

The laws of Dennard scaling broke down between 2005-2007 according to a general industry consensus. As Hochschule confirms, because threshold and operating voltage cannot be scaled any longer, it is no longer possible to keep the power envelope constant from generation to generation and simultaneously achieve potential performance improvements. Intel's Mark Bohr remains bullish about the future of scaling in a post-Dennard world. In a 30 year retrospective in 2014 on Dennard's MOSFET scaling paper he wrote:

"It is commonly recognized that following the simple scaling rules described by Dennard and his team back in 1974 is now no longer a sufficient strategy to meet future transistor density, performance and power requirements. But ours is a very inventive industry and new transistor technologies such as strained silicon, high-dielectrics, metal gates and multiple-gate devices have been or will be introduced to continue scaling. So although the letter of Dennard's Law can no longer be followed, it has gotten us very far over the past 30 years and the spirit is alive and well in transistor R&D facilities around the world."

Dennard's law is closely related to Koomey's law, which states that performance per watt has doubled every 1.57 years.

https://www.phonearena.com/news/performance-per-watt-is-becoming-the-most-important-metric-for-chips_id133709

More details are provided in a presentation in 2003 by Norman Jouppi at Google, "The Future Evolution of High-Performance Microprocessors".

<https://web.stanford.edu/class/ee380/Abstracts/060927-jouppi-slides.pdf>

1.34.12 Huang's law for Graphics Processing Units - GPUs

(Jensen) Huang's law states that the performance of GPUs will more than double every two years. It was coined by Christopher Mims from *Wall Street J.* - *WSJ* on 9/19/2020. The hypothesis is subject to questions about its validity.

https://en.wikipedia.org/wiki/Huang%27s_law

On 6/18/20 it was announced that Nvidia is acquiring ARM for \$40 Billion.

<https://finance.yahoo.com/news/nvidias-acquisition-arm-proves-boon-063208683.html>

According to *National Association of Securities Dealers Automated Quotation - NASDAQ* the Nvidia Corp. (NVDA) \$40 million bid to acquire UK-based semiconductor producer ARM is under threat as the UK government is assessing the possibility of monopoly in the market.

<https://www.nasdaq.com/articles/nvidias-%2440-bln-arm-acquisition-under-threat-2021-08-04>

1.34.13 Grosch's law

ACM President Herb Grosch in 1953 stated that: "Computer performance increases as the square of the cost".

https://en.wikipedia.org/wiki/Grosch%27s_law

According to BusinessWeek a similar law was expressed two years earlier by Seymour Cray "Computers should obey a square law — when the price doubles, you should get at least four times as much speed." Grosch's law was observed to hold for different computer categories, but not across microprocessors, minicomputers and supercomputers. It has been said that Grosch's law follows the pattern with which IBM priced its computers at the time.

Scalability rules

A reviewer of (Abbott and Fisher, 2016) states: "Scalability Rules provides an essential set of practical tools and concepts anyone can use when designing, upgrading, or inheriting a technology platform. It is very easy to focus on an immediate problem and overlook issues that will appear in the future. This book ensures strategic design principles are applied to everyday challenges." The book enunciates principles for scaling web sites. Rule 30 is quoted below as an example.

Remove business intelligence from transaction processing

What, When, How, and Why.

What: Separate business systems from product systems and product intelligence from database systems.

When to use: Anytime you are considering internal company needs and data transfer within, to, or from your product.

How to use: Remove stored procedures from the database and put them in your application logic. Do not make synchronous calls between corporate and product systems.

Why: Putting application logic in databases is costly and represents scale challenges. Tying corporate systems and product systems together is also costly and represents similar scale challenges as well as availability concerns.

Key takeaways: Databases and internal corporate systems can be costly to scale due to license and unique system characteristics. As such, we want them dedicated to their specific tasks. In the case of databases, we want them focused on txns rather than product intelligence. In the case of back-office systems (business intelligence), we do not want our product tied to their capabilities to scale. Use asynchronous transfer of data for business systems.

1.34.14 Kryder's law

Seagate's Mark H. Kryder expressed Kryder's law in the *Scientific American* in 2005.

https://en.wikipedia.org/wiki/Mark_Kryder

"Inside of a decade and a half, hard disks had increased their capacity 1,000-fold, a rate that Intel founder Gordon Moore himself has called "flabbergasting." In 2009, Kryder projected that if hard drives were to continue to progress at their then-current pace of about 40% per year, then in 2020 a two-platter, 2.5-inch disk drive would store approximately 40 TB (terabytes) and cost about \$40. The largest capacity HDDs in 2017 are listed as follows.

<https://hddmag.com/top-x-largest-hard-drives/>

1.34.15 Subsecond response times

In 1981 Arvind J. Thadani published, "Interactive user productivity," IBM Systems Journal, 20 (4), 407-423. The requirement for computer response time was set to be 400 milliseconds, not 2 seconds which had been the previous standard. Such

a “subsecond” response time exceeded the Doherty threshold and use of such applications were deemed to be “addicting” to users.

<https://lawsofux.com/doherty-threshold/>

Note that subsecond response times affect the hardware and software of timesharing systems.

1.35 Conclusions and summary

The discussion in this chapter is biased towards IBM, because the author was affiliated with the company twice and using its software, most notably PL/I.

We provide the reader with a history of computing. This was a period of high creativity in computer design and fierce competition, which led to the disappearance of many companies and their computer lines. IBM System/350 mainframe follow-ups introduced in mid-1960s have maintained their role in running critical applications.

We discuss various topics in computer architecture and operating systems: number representation, basic computer arithmetic, virtual memory, caches, TLBs, multiprogrammed computers, timesharing computers, pipelined vector machines, parallel processing computers, microprocessors.

Advanced RISC Machines - ARM microprocessors occupy 75% of the 32 bit chip market and tend to run the Linux operating system. Intel dominates the 64-bit computer market with Microsoft’s Windows 10 OS.

<https://en.wikipedia.org/wiki/X86>

American Micro Devices - AMD has showed a turnaround in competing with Intel in 2019.

<https://www.extremetech.com/computing/296453-welcome-to-the-second-golden-age-of-amd>

<https://www.extremetech.com/computing/325848-amd-x86-cpu-market-share-soars-hits-14-year-high>

AMD is planning a merger with Xilinx and GPU maker NVIDIA with ARM.

IBM invented disk drives in 1950s and remained a major player until it sold its *Hard Disk Drive - HDD* business to Hitachi in 2002. The HDD market is now dominated by Seagate, *Western Digital Corp. - WDC*, and Toshiba. Leading storage companies are Dell/EMC, *HP Enterprise - HPE*, *Network Appliance - NetApp*, IBM, Oracle, Lenovo, Fujitsu, and Huawei.

<https://www.enterprisestorageforum.com/products/20-top-enterprise-data-storage-vendors.html>

There is a transition towards flash memories led by Intel, Micron, Samsung. Sandisk/WDC, SK Hynix. Purestorage is a leading storage company in this area. More recently Intel has introduced Optane memories, which are fast and nonvolatile.

Novel computing devices such as tablets and smartphones are produced by Apple, Samsung, Google, Huawei. Android (by Google) and IOS (by Apple) are the most popular smartphone/tablet operating systems.

Networking leaders are: Cisco, Huawei, HPE/Arista, Juniper, VMware, Aruba (HPE), Silver Peak, Aryaka, Nutanix, Dell/EMC.

Chapter 2

Storage technologies and their data

Evolution of recording material in Section 2.1
Advertising and e-commerce in Section 2.2
Computer Storage Technologies in Section 2.3
Punched paper tape and cards in Section 2.3.1
Punched paper tapes in Subsection 2.3.2
Handwriting recognition in Section 2.3.3
Delay lines for storage in Section 2.3.4
Core memories in Section 2.3.5
Semiconductor memories in Section 2.3.6
Redundant Array of Independent Memories - RAIM in Subsection 2.3.7
Magnetic Random Access Memory - MRAM in Section 2.3.8
Tape libraries and tertiary storage in Section 2.3.9
An analytical model for a tape library in Subsection 2.3.10
Summary of a recent article on magnetic tape in Subsection 2.3.11
Hard Disk Drives - HDDs in Subsection 2.3.12
HDD manufacturers in Subsection 2.3.13
Storage technologies replacing magnetic disk drives in Subsection 2.3.14
Magnetic bubble memories in Subsection 2.3.15
Charged Coupled Devices - CCDs in Subsection 2.3.16
Micro-Electro-Mechanical Systems - MEMS in Subsection 2.3.17
IBM Zurich Millipede in Subsection 2.3.18
Phase Change Memory - PCM in Subsection 2.3.19
Flash memories in Subsection 2.3.20
Companies producing flash memories in Subsection 2.3.21
Elevating commodity storage with the SALSA host translation layer in Subsection 2.3.22
Flash SSD versus magnetic HDD pricing in Subsection 2.3.23
Pure Storage design of Purity in Subsection 2.3.24
Intel/Micron 3D_XPoint Optane Memory in Subsection 2.3.25
Processing in Memory - PIM in Subsection 2.3.26.
Universal memory technology - ULTRARAM in Section 2.3.27
Racetrack memory in Section 2.3.28
Optical storage in Section 2.3.29.
Holographic memory in Section 2.3.30
M-DISC and storage longevity in Section 2.3.31
Persistent and NonVolatile Memory - NVM in Section 2.3.32
Glass as a new storage medium for cloud storage in Section 2.3.33
DNA based archival storage system in Section 2.3.34
Reliability studies of DRAM, HDDs, & Flash SSDs in Section 2.4
Flash SSD reliability at Google & NetApp in Section 2.4.1
Storage Networking Industry Association - SNIA in Section 2.5
Solid state storage performance in Subsection 2.5.1
Persistent Memory Forum in Subsection 2.5.2
Computational storage at SNIA in Subsection 2.5.3
Big data and its sources in Section 2.6
Sources of storage content in Section 2.7

Ranking and description of media companies in Section 2.8
 Sources of news: newspaper, radio, TV in Section 2.9
 Newspapers in US and worldwide in Subsection 2.9.1
 TV networks in US in Subsection 2.9.2
 Text editing and formatting in Section 2.10
 Online book sources in Section 2.11
 Free book download web sites in Section 2.12
 Data image, audio, and video compression methods in Section 2.13
 Data compression in Subsection 2.13.1
 Huffman encoding is discussed in Subsection 2.13.2
 Lempel-Ziv encoding in Subsection 2.13.3
 Arithmetic encoding in Subsection 2.13.4
 Miscellaneous topics on data compression in Subsection 2.13.5
 Universal Resource Locator - URL shortener in Subsection 2.13.6
 Image compression in Subsection 2.13.7
 Audio/video compression in Subsection 2.13.8
 Main memory data compression in Subsection 2.14
 Data deduplication in storage systems in Section 2.15
 Data deduplication at Microsoft in Subsection 2.15.1
 The Venti prototype at Bell Labs/Lucent in Subsection 2.15.2
 Data Domain deduplication in Subsection 2.15.3
 Datrium in Subsection 2.15.4
 Summary of a major survey on data deduplication in Subsection 2.15.5
 Data deduplication companies in Section 2.16
 Storage research at IBM's Almaden Research Center in Section 2.17
 Cleversafe and its information dispersal technology in Section 2.18
 Recent developments at IBM Research at ARC in Section 2.19
 Storage research at HP in Section 2.20
 Primary storage vendors and enterprise companies in Section 2.21
 All-flash upstart storage companies in Section 2.22
 Hyperconverged infrastructure for storage systems in Section 2.23
 Top enterprise storage backup players in Subsection 2.24
 Data storage companies: up and coming storage vendors in Section 2.25
 Parallel file systems are discussed in Section 2.26
 Cloud storage in Section 2.27
 Cloud computing price models in Subsection 2.27.1
 Storage as a Service in Subsection 2.27.2
 Cloud storage elasticity and its benchmarking in Subsection 2.27.3
 Jai Menon's predictions on the future of clouds in Section 2.28
 Cloud storage companies in Subsection 2.29
 Distributed systems research related to clouds in Section 2.30
 OceanStore in Subsection 2.30.1
 Inktomi and CAP theorem in Subsection 2.30.2
 Replicated data is discussed in Subsection 2.30.3
 Sky Computing in Subsection 2.30.4
 Data encryption is discussed in Section 2.31
 Predictions about storage systems in Section 2.32

2.1 Evolution of recording material

A historical review of writing material since early times is presented in (Goda and Kitsuregawa, 2012). A major development in the 2nd century B.C. - Before Christ was the invention of the papermaking process in China, which was a major improvement over writing on clay and wooden chips. Papyrus made in 4th century BC Egypt was also used for writing,

<https://en.wikipedia.org/wiki/Papyrus>

while sheep skin or parchment was developed for writing in Pergamon, a Greek city now in part of Turkey.

<https://en.wikipedia.org/wiki/Parchment>

Printing on woodblock was invented in China during the Tang Dynasty (618-906) AD - *Anno Domini* - after birth of Christ.

<http://www.silk-road.com/artl/printing.shtml>

According to IranicaOnline Encyclopedia (see below) this form of printing reached Iran.

Gutenberg's printing press on paper in Europe in mid-15th enabled the speedy reproduction of books mechanically.

https://en.wikipedia.org/wiki/Johannes_Gutenberg

In addition to books printing technology allowed mass dissemination of news via newspapers.

https://simple.wikipedia.org/wiki/Town_crier

A list of the oldest newspapers is as follows.

https://en.wikipedia.org/wiki/List_of_the_oldest_newspapers

Armenians and printing in the Middle East

An early printing press in the Middle East was established by an Armenian priest in 1637 in New Julfa near Isfahan, which was Iran's capital city during the Saffavid dynasty.

<http://www.iranicaonline.org/articles/cap-print-printing-a-persian-word-probably-derived-from-hindi-chapna-to-print-see-turner-no>

https://wikivisually.com/wiki/New_Julfa

New Julfa was built by Saffavid dynasty's Shah Abbas to house Armenians he forcefully relocated from the city of Julfa in Nakhijevan, which is at Iran's border with Turkey. Armenians in (old) Julfa had already established business ties with India and beyond, but this trade flourished further from New Julfa.

<http://www.iranicaonline.org/articles/julfa-index>

Prior to the introduction of the printing press books, bibles were patiently copied by Armenian monks in monasteries, but when a printing press was installed in New Julfa monks complained about losing their jobs. Handwritten texts had the advantage of being accompanied with colorful paintings known as Illuminated Manuscripts.

https://en.wikipedia.org/wiki/Armenian_illuminated_manuscripts

An exhibition on 5000 years of Armenian art was held in 2018-2019 at the Metropolitan Museum of Art in New York City with an accompanying publication.

<https://www.metmuseum.org/press/exhibitions/2018/armenia>

Calligraphy is an art form practiced in Arabic, Chinese and other languages. Tiles in Al Hambra Palace at Granada Spain had calligraphic writings in Arabic with intricate patterns, which might have inspired Maurits Escher's etchings.

https://en.wikipedia.org/wiki/M._C._Escher

Calligraphy should be contrasted with the current practice of texting with two thumbs.

Types of typesetting are described below.

<https://en.wikipedia.org/wiki/Typesetting>

The pre-digital era had manual typesetting, hot metal typesetting, and phototypesetting.

<https://whattheythink.com/articles/55522-day-typesetting-industry-died/>

Large printing presses printed newspapers in large quantities, which are folded and put on trucks for distribution is illustrated in the 2018 movie "The Post" about Watergate break-in, which led to President Nixon's impeachment in 1970s.

[https://en.wikipedia.org/wiki/The_Post_\(film\)](https://en.wikipedia.org/wiki/The_Post_(film))

The process of digital printing is described here.

https://en.wikipedia.org/wiki/Digital_printing

IBM 1130 computers had a typewriter for displaying results, but there was also an 80 *line per minute* - *lpm* printer. The IBM 1403 line printer associated with the IBM S/360 mainframes could print 1400 lpm with a 240 character chain rotated at high speed to place the appropriate character in the right position for printing.

https://en.wikipedia.org/wiki/IBM_1403

Computers can be used to drive on a variety of printers: 3D, inkjet, laser, thermal, etc.

[https://en.wikipedia.org/wiki/Printer_\(computing\)](https://en.wikipedia.org/wiki/Printer_(computing))

https://en.wikipedia.org/wiki/Laser_printing
https://en.wikipedia.org/wiki/Inkjet_printing
https://en.wikipedia.org/wiki/3D_printing

Rapid dissemination of information has always been a major problem. An early version of Pony express used by Assyrians was adopted by Persians (550-330 BC).

<http://www.flowofhistory.com/units/pre/2/FC15>

Pony Express used in the US in the years preceding the civil war allowing coast-to-coast message delivery in ten days.

https://en.wikipedia.org/wiki/Pony_Express

Pony express in US was later replaced by railroads, automobiles, and airplanes.

United States Postal Service - *USPS* is an independent agency of the executive branch of the US federal government, which provides postal service in the US.

https://en.wikipedia.org/wiki/United_States_Postal_Service

<https://www.usps.com>

Rules of mail exchange between countries are regulated by the Universal Postal Union signed in 1874, which is now part of *United Nations* - *UN*.

https://en.wikipedia.org/wiki/Universal_Postal_Union

The telegraph, teletype, telephone, and the fax improved the speed of news dissemination. Telegraphy which uses dots and dashes of the Morse code invented by Samuel Morse in 1830s.

https://en.wikipedia.org/wiki/Morse_code

The two most frequently used letters E and T are represented by a single code, a dot and a dash, respectively, Note similarity to Huffman coding (Huffman, 1952), (Sayood, 2017).

https://en.wikipedia.org/wiki/Huffman_coding

A patent battle between Nikola Tesla and Marconi for the invention of radio went on for years and was only settled by US Supreme Court six months after Tesla's death in 1943, while Marconi died in 1937. All of Marconi's radio patents were invalidated and awarded to Tesla.

<https://teslauniverse.com/nikola-tesla/articles/tesla-invented-radio-not-marconi>

The Internet has its origin in the efforts to build and interconnect large computers in US.

https://en.wikipedia.org/wiki/History_of_the_Internet

U.S. Defense Department's *Advanced Research Projects Agency* - *ARPA* funded ARPANet, which became a precursor to Internet since 1969 There were similar efforts in UK and France.

<https://en.wikipedia.org/wiki/ARPANET>

The volume of mail is reduced by email which can be sent/received from home, but this requires *Internet Service Providers* - *ISPs* subscription.

https://en.wikipedia.org/wiki/Internet_service_provider

America OnLine - *AOL* was an American web portal and online service provider based in New York City, which went online in Nov. 1985.

<https://en.wikipedia.org/wiki/AOL>

AOL was acquired by Verizon Communications for \$4.4 billion in June 2015, but in May 2021 Verizon sold Yahoo and AOL to Apollo for \$5 billion.

https://en.wikipedia.org/wiki/Apollo_Global_Management

WhatsApp, is an American freeware, cross-platform centralized messaging and voice-over-IP (VoIP) service owned by Facebook, Inc. It allows users to send text messages and voice messages, make voice and video calls, and share images, documents, user locations, and other content.

<https://en.wikipedia.org/wiki/WhatsApp>

WeChat is a free messaging alternative to text messaging and supports sending text, photos, videos, and more. It is available on iOS, Android, Windows Phone, Nokia S40, Symbian, Blackberry phones, and Mac OS X. It has multiple functions such as minigames, WeChat Pay and moments. It was developed and launched in China and is now a very popular and common app to have on a smartphone.

<https://en.wikipedia.org/wiki/WeChat>

Biden's democrat administration has distanced itself from Trump's attempt to ban WeChat.

<https://abcnews.go.com/Business/wireStory/us-distances-trump-attempts-ban-wechat-75829270>

WeChat and WhatsApp are compared here

<https://www.webnotes.com/wechat-vs-whatsapp-comparison>

There are about 300 underwater submarine cables that contribute to 99% of international data traffic.

<https://99percentinvisible.org/article/underwater-cloud-inside-cables-carrying-99-international-data-traffic/>

Satellite communication networks are another means of communication.

https://en.wikipedia.org/wiki/Communications_satellite

Satellite internet providers provide access in rural areas.

<https://www.reviews.org/internet-service/best-satellite-internet-providers/>

The Terragraph technology using 60 GHz V-band is being promoted by Facebook.

<https://spectrum.ieee.org/telecom/internet/facebooks-60ghz-terragraph-technology-moves-from-trials-to-commercial-gear>

<https://terragraph.com/>

Apple is also working on internet satellites for direct device communication.

<https://www.bloomberg.com/news/articles/2019-12-20/apple-has-top-secret-team-working-on-internet-satellites>

<https://www.popularmechanics.com/technology/infrastructure/a30445411/apple-satellites-iphone/>

56 kbit/s is the data rate of a classical single channel digital telephone line in North America. The figure of 56 kbit/s is derived from its implementation using the same digital infrastructure used since the 1960s for digital telephony in the public switched telephone network, which uses a sampling rate of 8,000 Hz for *Pulse Coded Modulation - PCM* audio with 8-bit audio bit depth to encode analogue signals into a digital stream of 64,000 bits/s.

PCM converts analog audio signals represented by waveforms into digital audio signals with no compression. In the T-carrier systems used in the US and Canada, a technique called bit-robbing uses Tanenbaum and Wetherall (2013), in every sixth frame, the least significant bit for *Channel Associated Signaling - CAS*.

https://en.wikipedia.org/wiki/Robbed-bit_signaling

<https://en.wikipedia.org/wiki/T-carrier>

This effectively renders the lowest bit of the 8 speech bits unusable for data transmission, and so a 56 kbit/s line uses only 7 of the 8 data bits $\times 7 \text{ bits} = 56 \text{ kbit/s}$.

In Oct. 2007 AT&T completed the next-generation IP/*Multi-Protocol Label Switching - MPLS* backbone network,

https://en.wikipedia.org/wiki/Multiprotocol_Label_Switching

which was world's largest deployment of 40-Gigabit Connectivity and a 17 Terabit Network was planned. The highest *Optical Carrier - OC-3840* is a network line with transmission speeds of up to 200 Gbit/s.

https://en.wikipedia.org/wiki/Optical_Carrier_transmission_rates

Using a new type of optical fiber, researchers at the *Technical Univ. of Denmark - DTU* have transmitted data over a single optical fiber at a speed of 43 terabits per second (43 Tbps) to set a new data transmission world record. This beats the previous record of 32 Tbps set at Germany's Karlsruhe Institute of Technology.

<https://newatlas.com/dtu-world-record-data-transmission-43tbps/33214/> The record is held by the 44.2 Tbps chip from Australian universities.

<https://www.monash.edu/news/articles/australian-researchers-record-worlds-fastest-internet-speed-from-a-single-optical-chip>

2.2 Advertising and e-commerce

Bulky Sears Roebuck catalogs appeared more than a century ago. Customers could order items specified in the catalog by filling forms and mailing them back to Sears.

<http://www.searsarchives.com/catalogs/history.htm>

A lot of local advertising "junk mail" is still being delivered via mass mailings, which includes publications such as the "Pennysaver". The mailings are from local entities, such as supermarkets advertising sales and are sorted by address for lower delivery cost with the recipients name listed as Resident. Mass mailings from department stores such as Macy's are being replaced by websites, which also appear as banners on whatever the user is accessing on the web. In addition to unsolicited phone calls known as spam, there are unsolicited emails advertising various forms of insurance, etc. The TV is quickly replacing the radio in advertising, which remains a cheap medium for political dialog.

Besides USPS *United Parcel Service - UPS* deliver parcels <https://www.ups.com>,

Federal Express - FedEx <https://www.Fedex.com>,

DHL international German carrier uses the initials of its founders. <https://www.dhl.com/us-en/home.html>

<https://en.wikipedia.org/wiki/DHL>

Amazon provides free delivery under certain conditions. According to former president Trump, Amazon's profitability can be ascribed to the low rates charged by USPS. It has been said that more searches start at [Amazon.com](https://www.amazon.com) than [google.com](https://www.google.com). Many items such as furniture are delivered free of charge by Wayfair

<https://www.wayfair.com>

provided their price exceeds \$49. Carvana <https://www.carvana.com> and Carmax <https://www.carmax.com> are two companies providing car delivery, which can be returned after 24 hours allowing test driving. Delivery of food has recently become popular, especially in the age of the corona virus. Instead of delivery by car and drones, four wheeled robots have been considered for this purpose.

Mailing lists were created for disseminating information via *Bulletin Board Systems-BBSs*.

https://en.wikipedia.org/wiki/Bulletin_board_system

One such mailing list is ACM SIGMOD's DBWorld and another one is that of ACM SIGMETRICS.

<https://research.cs.wisc.edu/dbworld/>

<https://www.sigmetrics.org/maillinglist.shtml>

The *World Wide Web* - WWW or the Web is a distributed information system where documents and other web resources are identified by *Uniform Resource Locators* - URLs, which may be interlinked by hypertext,

https://en.wikipedia.org/wiki/World_Wide_Web

<https://en.wikipedia.org/wiki/Hypertext>

To access documents on the Web the user needs to know the URL, e.g., that of New York Times to access it directly, otherwise they need to invoke a search engine.

<http://www.nytimes.com>

The web browser is a software application for retrieving, presenting and traversing information resources on the Web. The first web browser was developed by Tim Berners-Lee at European Center for Nuclear Research or *Conseil Europe'en pour la Recherche Nucléaire* - CERN in 1991.

<https://home.cern/science/computing/birth-web>

https://en.wikipedia.org/wiki/History_of_the_web_browser

<https://en.wikipedia.org/wiki/CERN>

Recently Berners-Lee has launched a campaign to fix the web.

<https://www.techspot.com/news/82910-tim-berners-lee-launches-plan-fix-web.html>

The Mosaic browser was developed at *National Center for Supercomputing Applications* - NCSA at *U. Illinois at Urbana-Champaign* - UIUC

https://en.wikipedia.org/wiki/National_Center_for_Supercomputing_Applications

[https://en.wikipedia.org/wiki/Mosaic_\(web_browser\)](https://en.wikipedia.org/wiki/Mosaic_(web_browser))

Marc Andreessen with Eric Bina developed the Netscape browser in 1993.

<https://en.wikipedia.org/wiki/Netscape>

Netscape at a later point developed Netscape navigator which did not share code with Mozilla, a free software community founded in 1998 by members of Netscape. Mozilla's current products include the Firefox web browser, Thunderbird e-mail client, Bugzilla bug tracking system, Gecko layout engine, Pocket "read-it-later-online" service, and others.

<https://en.wikipedia.org/wiki/Mozilla>

The first browser war was led by Microsoft against Netscape Navigator when it introduced the *Internet Explorer* - IE, which came bundled with Microsoft's PC software (1995-2001).

https://en.wikipedia.org/wiki/Browser_wars

Following Netscape Navigator's decline, Netscape open-sourced its browser code via the non-profit Mozilla Foundation, i.e., Mozilla Firefox.

<https://en.wikipedia.org/wiki/Firefox>

Google's Chrome introduced in 2008 is the most popular web browser at this time and can operate with most operating systems. Chromebooks are available from Dell, Acer, and Samsung. Chrome OS was announced by Google in 2009 based on a version of Linux. Its applications and user data reside in the cloud and hence primarily runs web applications at CERN.

https://en.wikipedia.org/wiki/Chrome_OS

To summarize, the most commonly used browsers are: Google Chrome, Mozilla Firefox,

<https://en.wikipedia.org/wiki/Firefox>

Microsoft's Internet Explorer & Edge, and Apple's Safari.

<https://www.nationthailand.com/news/30328504>

Mozilla is lightweight in that it uses 30% of the main memory used by Chrome.

On July 27, 2021 there are 1,877,108,632 web sites of which less than 200 Million are active.

<http://www.internetlivestats.com/total-number-of-websites/>

Scrolling down yields the number of web sites by year and the year important web site were launched (until 2018). There are about 138 Million dot.com domains.

Department of Commerce's [commerce.gov](https://www.commerce.gov) *National Telecommunications and Information Administration* - NTIA has extended Verisign's contract to run the internet registry until 2024.

<https://en.wikipedia.org/wiki/Verisign>

There is also the issue of dotcom domain pricing.

https://www.theregister.co.uk/2018/11/02/dotcom_domains_pricing/

The annual registration cost was \$7.85 in 2020 to increase to \$10.29 in 2024.

Many search engines such as Lycos, Ask Jeeves (now Ask) were once popular.

http://www.thehistoryofseo.com/The-Industry/Short_History_of_Early_Search_Engines.aspx

Search engines and the year they were born. Other details are provided at the web site.

Excite 1993

Yahoo! 1994

WebCrawler 1994

Lycos 1994

I-inforbook 1994

altavista 1995

Inktomi 1996

Ask Jeeves 1997 (now Ask)

Google 1997

MSN serach 1998

Overture, goto.com 1998

AllTheWeb 1999

Alta Vista was an early search engine developed by DEC, which was taken over by Yahoo! after DEC's demise.

<https://en.wikipedia.org/wiki/AltaVista>

Yahoo! search engine was overtaken by Google, which provides superior search capability, partially ascribed to its enhanced pagerank algorithm, but also the underlying indexing.

<https://en.wikipedia.org/wiki/PageRank>

The Google search engine has undergone many changes over the years.

https://en.wikipedia.org/wiki/Google_Search

Hummingbird is Google's latest search paradigm.

https://en.wikipedia.org/wiki/Google_Hummingbird

<https://www.entrepreneur.com/article/229926>

A list of currently popular search engines is as follows.

<https://www.searchenginejournal.com/alternative-search-engines/#close>

Seventeen alternatives to Google search engine are as follows:

<https://www.searchenginejournal.com/alternative-search-engines/#close>

1. <http://www.bing.com/> Microsoft Bing has a rewards program

2. <https://www.yandex.com> 45% Russian internet users, also used in Belarus, Kazakhstan, Turkey, and Ukraine,

3. <https://search.creativecommons.org/> CC search is accessible from here

<https://microsoftedge.microsoft.com/addons/Microsoft-Edge-Extensions-Home>

4. <https://swisscows.com/>

5. <https://duckduckgo.com/>

Tired of being tracked online? We can help. Get seamless privacy protection on your browser for free with one download: (1) private search, (2) tracker blocking, (3) site encryption

Twitter founder Jack Dorsey recommends this search engine which unlike Google does not trace users and provides the same response regardless user profile.

<https://sputniknews.com/science/201911301077444255-duck-you-google-twitter-ceo-dorsey-reveals-he-ditched-tech-giants-search-and-now-uses-duckduckgo/>

6. <https://www.startpage.com/> The world's most private search engine. Don't be tracked online. Protect your personal data. No collecting or selling search history. No 3rd party ad trackers or cookies. Search and browse anonymously.

7. <https://www.searchencrypt.com/home>

8. <https://gibiru.com/>

Uncensored private search.

9. <https://www.onesearch.com/>

Private and encrypted.

10. <https://www.wiki.com/>

Limit search to Wikipedia, encyclopedias etc.

11. <https://boardreader.com/>

12. <https://search.givewater.com/> Aims to have a positive impact on the world by empowering users to solve the issue of poor quality water and ineffective sanitation across the developing world.

13. <https://ekoru.org/> 56M Kg CO₂ has been saved by using Ekoru.

14. <https://www.ecosia.org/> Search the web to plant trees.

15. <https://twitter.com/home>

16. <https://www.slideshare.net/>

17. <https://archive.org> This is a non-profit library of millions of free books, movies, software, music, websites, and more.

Internet search is facilitated by web crawling.

https://en.wikipedia.org/wiki/Web_crawler

The world wide web has grown tremendously, so it is difficult to build a complete index. The indices are held on numerous machines, where search is invoked concurrently. The Google search engine exploiting parallelism responds to queries in less than half a second, e.g., 2,740 results in 0.36 s.

<https://www.searchtechnologies.com/blog/web-data-mining-tools-techniques>

According to Google's funfacts Google had 1.4 billion web pages in its index on 6/15/2011.

Twitter is a popular means of mass communication.

<https://en.wikipedia.org/wiki/Twitter>

Tweeting was used excessively by Pres. Donald Trump, which allowed him to bypass *MainStream Media* - *MSM*, such as NY Times.

https://en.wikipedia.org/wiki/Mainstream_media

which he called "fake news" because of the bias towards the democratic party and the "woke culture", which originally referred to awareness about racial prejudice and discrimination, but subsequently came to encompass an awareness of other issues of social inequality, e.g., gender and sexual orientation. The *Black Lives Matter* - *BLM* movement was a major contributor to woke movement as great amounts of money was raised after viewing George Floyd's head under police's knee.

https://en.wikipedia.org/wiki/Black_Lives_Matter

Twitter on January 2021 banned President Trump's access to its platform, following his speeches which allegedly led to the invasion of the capitol on 1/6/2021.

https://en.wikipedia.org/wiki/2021_United_States_Capitol_attack

Several protests have used Twitter in particular to organize demonstrations.

https://en.wikipedia.org/wiki/Twitter_Revolution

Notably the 2010-11 election protest in Iran US State Dept asked Twitter to defer its scheduled maintenance to allow protesters to continue using Twitter.

<https://www.aljazeera.com/news/2010/2/11/timeline-iran-after-the-election>

Reddit is a website posting user-generated content- photos, videos, links, and text and discussions of this content in what is essentially a *Bulletin Board System* - *BBS*.

<https://en.wikipedia.org/wiki/Reddit>

Reddit was cofounded by Alexis Ohanian and Steve Huffman in 2005, after they attended a talk by Paul Graham.

[https://en.wikipedia.org/wiki/Paul_Graham_\(programmer\)](https://en.wikipedia.org/wiki/Paul_Graham_(programmer))

Ohanian has been in the news as the husband of Serena Williams. She failed to achieve her 24th Grand Slam being beaten by younger Naomi Osaka in Australia semifinals on 2/18/21, but is still trying. Ohanian has authored "Without Their Permission: How the 21st Century Will Be Made, Not Managed", which debuted as #4 in NY Times National Best Seller list

<https://alexisohanian.com/>

Content Delivery Networks - *CDNs* speed up information dissemination by caching web data at multiple sites to provide geographical proximity and higher data availability.

https://en.wikipedia.org/wiki/Content_delivery_network

Akamai which is a major CDN was co-founded by Daniel Lewin and MIT Professor F. Thomson Leighton.

https://en.wikipedia.org/wiki/Daniel_Lewin

https://en.wikipedia.org/wiki/Akamai_Technologies

Lewin was a passenger on the skyjacked American Airline's flight from Boston to Los Angeles on 9/11/2001, which was

sky-jacked and hit the North Tower of *World Trade Center - WTC* in New York City. WTC constructed in 1973 is captured in movies made till 2001.

[https://en.wikipedia.org/wiki/World_Trade_Center_\(1973%E2%80%932001\)](https://en.wikipedia.org/wiki/World_Trade_Center_(1973%E2%80%932001))

Cloudflare provides a global CDN with unique performance capabilities and a strong focus on security. The company has a global infrastructure built from the ground up using only next-generation, high-performance equipment -no legacy software or hardware. The result is the Cloudflare is easy to set up, affordable, and built to continuously adapt and integrate emerging technologies to meet future needs of everyone's Internet properties.

<https://www.cdnplanet.com/cdns/cloudflare/>

A comparison of Cloudflare with Akamai is as follows.

<https://www.cdnplanet.com/compare/akamai/cloudflare/>

A list of the best CDN providers is as follows.

<https://hostingtribunal.com/best/cdn-providers/#gref>

Google's parent-company Alphabet third quarter 2018/19/20 earnings were \$33.7 \$38.9 \$37.3 billion, respectively.

<https://www.investopedia.com/alphabet-earnings-4584987>

Alphabet's market value has exceeded one trillion dollars, which is also the case for Apple, which actually exceeds \$ 2 trillion. Microsoft, and Amazon. Google "reads" user emails and displays ads for products related to the ongoing email conversation and this is how it makes a profit while providing the gmail service freely. Google profits when a user clicks on ads displayed by Google. Alphabet owns YouTube which provides 9% of its revenue through advertising. LinkedIn's revenue increased from \$6.8 B in 2019 to \$8.1 B in 2020 by 19.6%. How LinkedIn makes money is explained below.

<https://www.investopedia.com/ask/answers/120214/how-does-linkedin-lnkcd-make-money.asp>

Here are some details for Q3 in 2020.

Earnings per share: \$16.40 vs \$11.29 expected, according to Refinitiv estimates.

Revenue: \$46.17 billion vs \$42.90 billion expected, according to Refinitiv estimates.

Google Cloud: \$3.44 billion vs. \$3.32 billion estimated according to StreetAccount.

YouTube ads: \$5.04 billion vs. \$4.39 billion estimated, according to StreetAccount.

Traffic Acquisition Costs - TAC: \$8.17 B vs. \$7.66 B according to StreetAccount.

<https://www.cnbc.com/2020/10/29/alphabet-googl-earnings-q3-2020.html>

TAC consists of payments made by Internet search companies to affiliates and online firms that direct consumer and business traffic to their websites. TAC is a major source of revenue for Internet search firms such as Google.

<https://www.investopedia.com/terms/t/traffic-acquisition-cost-tac.asp>

Google Ads (formerly AdWords) is an online advertising platform developed by Google, where advertisers bid to display brief advertisements, service offerings, product listings, or videos to web users. It can place ads both in the results of search engines like Google Search and on non-search websites, mobile apps, and videos Google Ads is Alphabet Inc's main source of revenue, contributing \$134.8 billion in 2019.

https://en.wikipedia.org/wiki/Google_Ads

Google has published "Beyond last click: Understanding your consumers' online path to purchase insights and learnings from Google's clickstream research" for its advertisers explains conversion as something that happens when a user completes a "valuable" action on the advertiser site, such as a purchase or at least a request for additional information. A followup report is:

https://services.google.com/fh/files/misc/analytics_360_data_driven_attribution_feature_brief.pdf

2.3 Computer storage technologies

Computer storage technologies in approximate chronological order are reviewed in what follows.

2.3.1 Punched cards - Hollerith and IBM

Punched cards and paper tape were early media used for recording data.

https://en.wikipedia.org/wiki/Punched_card

Punched cards have a long history and were used by Charles Babbage in his analytic engine.

https://en.wikipedia.org/wiki/Charles_Babbage

Famous poet Lord George Byron's daughter Ada Byron Lovelace is credited with writing the first computer program, which was to calculate Bernoulli numbers for Charles Babbage, whom she had met in Italy.

<https://twobithistory.org/2018/08/18/ada-lovelace-note-g.html>

https://en.wikipedia.org/wiki/Bernoulli_number

Incidentally, the Ada programming language was originally designed for the US *Dept. of Defense - DoD* 1977-1983 to supersede over 450 programming languages used by the DoD at that time.

[https://en.wikipedia.org/wiki/Ada_\(programming_language\)](https://en.wikipedia.org/wiki/Ada_(programming_language))

The 1790 US census has been conducted every ten years ever since.

https://en.wikipedia.org/wiki/United_States_Census

The 1880 US census took eight years to complete and Hollerith cards were developed for the 1890 census to ensure its timely completion.

https://en.wikipedia.org/wiki/Herman_Hollerith

Hollerith's punched cards came with tabulating unit record machines.

https://en.wikipedia.org/wiki/Tabulating_machine

In 2020 the census is conducted by filling forms distributed by mail or preferably answering questions on a web site, which was my choice.

<https://2020census.gov> The punched card format adopted by IBM had eighty columns and twelve rows.

https://en.wikipedia.org/wiki/Punched_card

The upper three rows are called zones and numbered 12, 11, and 0. Row 0 was followed by rows 1:9 can be used to specify 0-9. Letters of the alphabet and punctuation symbols are represented with two holes, one in a zone row and the other in rows 1:9.

A disadvantage of punched cards was that a new card was required to correct mistakes.

https://en.wikipedia.org/wiki/Write_once_read_many

This may be the reason that IBM sold cards in boxes of 2000.

Paper tapes and cards are *Write Once Read Many - WORM* storage devices, in that written data cannot be modified, but WORM devices have the advantage that their data cannot be tampered. IBM 3363 was an optical WORM drive which appeared in 1987.

https://en.wikipedia.org/wiki/Write_once_read_many

<https://www.computerhistory.org/collections/catalog/102671161>

Organizations such as banks and other organizations used to hire key-punch operators to use a typewriter like keyboard to keypunch cards. There was a verification step to detect and correct errors on punch cards.

https://en.wikipedia.org/wiki/Two_pass_verification

Card readers could rapidly read tall stacks of cards transferring the data onto magnetic tapes or disks for further processing. Cards whose data was uploaded could be discarded or saved briefly for backup.

Various operations on card decks could be processed using Unit Record Equipment, e.g., the weekly salary could be computed as a product of the hourly wage times hours worked after being collated with cards which held the employee record. These cards could then be used to print salary checks.

https://en.wikipedia.org/wiki/Unit_record_equipment

Direct data entry onto magnetic tape was developed by *Mohawk Data Sciences - MDS* Corp. in mid-1960s, eliminating the need for punched cards, but did not gain popularity due IBM influence and insistence on selling punched cards.

https://en.wikipedia.org/wiki/Mohawk_Data_Sciences_Corporation

Some banks provided tellers with special forms to record deposits withdrawals, and transfers. The forms were later scanned by an *Optical Character Reader - OCR* devices.

https://en.wikipedia.org/wiki/Optical_character_recognition

The IBM 1287 developed at Rochester, MN in 1966 was probably the OCR machine being used in Tehran at the Industrial Bank.

https://en.wikipedia.org/wiki/Timeline_of_optical_character_recognition

2.3.2 Punched paper tapes

Punched (perforated) paper tape consists of a long strip of paper, Such tapes were used starting with the 19th century to control programmable looms and generally *Computer Numerical Control - CNC*.

https://en.wikipedia.org/wiki/Numerical_control

Paper tapes at remote sites have been used for recording measurements,

which are then transmitted to a central computer.

https://en.wikipedia.org/wiki/Punched_tape

Paper tapes are 0.1 mm thick and the two most common widths are 11/16 inch for 5 bit codes, and 1 inch (25.4 mm) for 6-8 bit codes. Hole spacing are 0.1 inch (2.54 mm) and data holes are 0.072 inches (1.83 mm) in diameter; with 0.046

inches (1.17 mm) feed holes. Number of character representations for varying number of rows are given here.

<http://www.chilton-computing.org.uk/acl/literature/chapman/p015.htm>

I dealt with a situation where paper tape was used for recording the volume of oil pumped at a well outside Tehran. Such paper tapes were periodically transmitted to a central site, via a tape reader connected to a modem attached to a phone line.

Circuit, message, packet switching, and virtual cut-through

Consider node A connected to node C via the intermediate node B. In the case of circuit switching the path from A to C is reserved via short reservation messages before transmission is started. The transmission time (t) is message length divided by line bandwidth. We assume that the distances are short, so that the propagation delay is negligible.

Transmitting the tape from A to B and then immediately from B to C each take t time units, so message-switching would require $2t$ time units.

If the message is broken into n packets, then the transmission time of each packet would be $p = t/n$. If packets received at node B are transmitted immediately to C then the transmission time of $n - 1$ packets is overlapped and the total transmission time is then $(n + 1)p = t + p$, which is almost one half of $2t$.

Paul Baran at Rand Corp. is credited with the invention of packet switching.

https://en.wikipedia.org/wiki/Packet_switching

RAND Corp. was created as a think tank within Douglas Aircraft Company in 1948. <https://www.rand.org>

In 1955 RAND spun off the *System Development Corp.* - SDC to help write the software for the *Semi-Automatic Ground Environment* - SAGE project in 1955.

https://en.wikipedia.org/wiki/Semi-Automatic_Ground_Environment

RAND supposedly referred to itself as *Research And No Development* - RAND. SDC was bought by Burroughs Corp. in 1980, but was later sold-off.

https://en.wikipedia.org/wiki/System_Development_Corporation

RAND has been involved in assessing the viability of war efforts and has developed simulators for this purpose and defining US foreign policy.

<https://www.rand.org/topics/wargaming.html>

<https://astutenews.com/2020/10/rand-and-the-malevolent-encirclement-of-russia/?cn-reloaded=1>

IBM's *System's Network architecture* - SNA predetermined a fixed path called a virtual-circuit from source to destination based on network loads, while *Burroughs Network Architecture* - BNA used dynamic routing decisions at each node based on outgoing channel utilizations. These are referred to as datagram (connectionless) and connection-oriented modes.

https://en.wikipedia.org/wiki/Packet_switching

With dynamic routing packets following different paths arrive out of order and there is a reassembly delay to reconstitute a message.

In virtual circuits all the packets of a message follow the same path and arrive at the destination node one after another, so that a packet reassembly delay is not incurred, but there is an initial delay in establishing the virtual circuit. Increased buffer space requirements due to reservation are expected in this case.

The *Virtual Cut-Through* - VCT is an improvement to packet switching (Kermani and Kleinrock, 1979). When the destination of an arriving packet is determined from its header and the outgoing channel is free then the forwarding of the packet is started right away even before the packet is fully received.

WormHole Switching - WHS is a variant of the VCT that avoids the need for large buffer spaces. In WHS a packet is transmitted between the nodes in units of *Flow units* - *flits*, the smallest units of a message on which flow control can be performed.

[https://en.wikipedia.org/wiki/Flit_\(computer_networking\)](https://en.wikipedia.org/wiki/Flit_(computer_networking))

The header flit of a message contains all the routing information and all the other flits contain data. The flits of the message are transmitted through the network in a pipelined fashion. Trailing flits follow the header flit(s) contiguously. When the header flit is blocked, then all the trailing flits occupy the buffers at the intermediate nodes. The main advantage of WHS derives from the pipelined message flow. Some implementations, however, require storage of multiple flits at each node to improve routing performance. The reduction of buffer requirements at each node has a major effect on the cost and size of multicomputer systems. The main disadvantage of WHS comes from the fact

that only the header flit has the routing information. If the header flit cannot advance in the network due to resource contention, all the trailing flits are also blocked along the path and these blocked messages can block other messages. This chained blocking can also lead to deadlock where messages wait for each other in a cycle and hence no message can advance any further. Prevention of deadlock is one of the main issues in WHS and is usually accomplished by a suitable choice of routing function that selectively prohibit messages from taking all the available paths, thus preventing cycles in the network. Selection of a routing algorithm is thus a major issue in wormhole-switched network.

A comprehensive comparison between virtual cut-through and wormhole routers for cache coherent *Network on-Chips* - NoC was carried out at China's *National Univ. of Defense Technology - NUDT* in (Wang et al., 2014). Since the minimum buffer size of VCT switching is larger than that of the wormhole one, the VCT switching is traditionally regarded as an inefficient NoC flow control type. With the scaling of semiconductor technology which reduces the criticality of buffer size for VCT switching may make it a promising NoC flow control candidate. The comprehensive comparison between the VCT and wormhole switching. Based on detailed RTL-level implementations, the hardware costs with both deterministic and adaptive routing were evaluated. Compared with the worm-hole switching VCT shortens the critical path by up to 27% and induces less area and power overheads. Furthermore, the allocator in VCT routers exhibits a better scalability in area overheads. Thus, the VCT router is an efficient NoC flow control type.

2.3.3 Handwriting recognition

There have been major advances in this field via handwriting recognition, such that scribbling on a tablet screen is converted into text.

https://en.wikipedia.org/wiki/Handwriting_recognition

https://www.theregister.com/2020/08/05/samsung_note20_launch/

Short-hand or stenography is an abbreviated symbolic writing of speech, which can then converted to longhand.

<https://en.wikipedia.org/wiki/Shorthand>

There are several shorthand standards. A stenotype is a specialized keyboard for speedy recording used in courts.

The need for shorthand is diminishing with the emergence of speech recognition (Beigi, 2011).

https://en.wikipedia.org/wiki/Speech_recognition

<https://www.ibm.com/cloud/watson-speech-to-text>

IBM Watson® Speech to Text technology enables fast and accurate speech transcription in multiple languages for a variety of use cases, including but not limited to customer self-service, agent assistance and speech analytics. Get started fast with our advanced machine learning models out-of-the-box or customize them for your use case.

Closed captions make speech accessible to the deaf or hearing impaired and also generally when the speech is not clear due to the accent of the speaker. There are speech recognition technologies to automatically create video captions.

https://en.wikipedia.org/wiki/Closed_captioning

Federal Communications Commissions - FCC has regulations for closed captioning.

<https://www.fcc.gov/consumers/guides/closed-captioning-television>

Subtitles may be used to provide the translation of a foreign language. It has been argued that subtitles are preferable to dubbing, but that depends on the quality of dubbing and many “movie goers” prefer dubbing to subtitles. A primitive form of translations was to pause the film with a screen of recent conversations.

<https://www.lingq.com/blog/subtitles-versus-dubbing>

The top five subtitle translators are listed here

<https://www.aiseesoft.com/resource/subtitle-translator.html>

Mixed Excitation Linear Prediction - MELP used for speech coding was invented at GaTech (McCree and Barnwall, 1995). https://en.wikipedia.org/wiki/Mixed-excitation_linear_prediction

Speech coding involves a trade-off of different aspects including bit-rate, speech quality, delay (frame size and lookahead), computational complexity, robustness to different speakers and languages, robustness to different background noises, channel error robustness, and also codec state recovery in the face of packet loss. Since the MELP's lower rates (600 and 1200 bit/s) are supersets of the 2400 bit/s rate, the algorithm complexity (e.g. in MIPS) is about the same for all rates. The lower rates use increased frames and lookahead, as well as codebook size, therefore they require more memory.

Voice synthesis such as reading text with a natural voice is a valuable tool which has been used by speech-impaired persons such as Stephen Hawkins or just to allow the listener to proceed with his tasks, e.g., painting, while listening to text being read to him.

https://en.wikipedia.org/wiki/Speech_synthesis

Note this is different from audiobooks which are prerecorded by actors.

<https://en.wikipedia.org/wiki/Audiobook>

2.3.4 Delay line memories

There are several versions of delay line memories: (1) mercury, (2) magnetorestrictive, (3) piezoelectric, (4) electric.

https://en.wikipedia.org/wiki/Delay_line_memory

Maurice Wilkes' *Electronic Delay Storage Automatic Calculator - EDSAC* at Cambridge U. in 1949, was the first full-size stored-program computer. It had 512 35-bit words of memory stored in 32 mercury delay lines, each 576 bits. The first UNIVAC computer shipped to the U.S. Census Bureau in 1951 included seven 1.5 KB memory units with an average access time of 222 microseconds. Each unit held 18 mercury-filled tubes weighing 800 lbs in total.

<https://www.computerhistory.org/storageengine/edsac-computer-employs-delay-line-storage/>

2.3.5 Core memories

Non-volatile core memories consist of small magnetic toroids positioned in squares.

https://en.wikipedia.org/wiki/Magnetic-core_memory

The number of cores in two dimensions were set to be a power of two to make full use of addressing bits. Vertical and horizontal wires are used to select a core and a diagonal wire for sensing for reads and to write.

Magnetic cores required expensive manual manufacturing, which was carried out in Taiwan to keep costs down. Magnetic cores were slow, typically 1.0 microsecond access time. They were non-volatile, so that their contents remained intact if there was a power loss. In spite of their costs they were the first to be used as the main memory of general purpose computers by IBM and other companies. A good discussions of core memories is provided in (Matick, 1975, 1977).

Core memories were later replaced with inexpensive and faster *Dynamic Random Access Memories - DRAM*, which yielded themselves to mass fabrication.

https://en.wikipedia.org/wiki/Dynamic_random-access_memory

A blog by Ken Shirrif details core memories used by IBM S/360 computers, noting that a 128 KB core memory weighed 610 pounds. <http://www.righto.com/>

Core memories were later replaced with inexpensive and faster *Dynamic Random Access Memories - DRAM*, which yielded themselves to mass fabrication.

https://en.wikipedia.org/wiki/Dynamic_random-access_memory

Nonvolatility was an advantage that core memories held with respect to DRAM. In addition core memories were resilient to radiation and hence computers with core memory were more suitable for space exploration. One such computer was the *Jet Propulsion Labs - JPL's Self Testing and Repair -STAR* computer (Avizienis et al., 1971), which was planned for use for a ten year old mission to Pluto, which was canceled later.

<https://history.nasa.gov/computers/Ch5-5.html> Pluto was finally reached in 2015 with New Horizons mission launched in 2006. The transmissions reached earth by the end of 2020.

<https://www.space.com/18377-new-horizons.html>

According to 2/11/21 press release HPE accelerates space exploration with first ever in-space commercial edge computing and AI capabilities.

<https://www.hpe.com/us/en/newsroom/press-release/2021/02/hewlett-packard-enterprise-accelerates-space-exploration-with-first-ever-in-space-commercial-edge-computing-and-artificial-intelligence-capabilities.html>

Edge computing is a distributed computing paradigm that brings computation and data storage closer to the location where it is needed, in this case to the space station, to improve response times and save bandwidth.

https://en.wikipedia.org/wiki/Edge_computing

Triple modular redundancy - TMR versus Simplex/TMR

An interesting aspect of the JPL STAR computer was its utilization of TMR. Three modules were used to do the same computation and their outputs were voted in by a highly reliable comparator. When a module failed the TMR system continues operation in duplex mode, until one of two active modules failed. When there is a disagreement it is not clear the output of which module can be trusted and the system is considered failed.

Given that $r = e^{-\lambda t}$ is the reliability of each module and assuming a highly reliable voter:

$$R_{TMR}(t) = r^3 + 3r^2(1 - r) = 3r^2 - 2r^3$$

The MTTF of a single module (or Simplex systems) is given by a negative exponential distribution with a constant failure rate is given as (Trivedi, 2002):

$$\begin{aligned} \text{MTTF}_S &= \int_0^\infty e^{-\lambda t} dt = \frac{1}{\lambda} \\ \text{MTTF}_{TMR} &= \int_0^\infty R_{TMR}(t) dt = \frac{3}{2\lambda} - \frac{2}{3\lambda} = \frac{1}{3\lambda} + \frac{1}{2\lambda} = \frac{5}{6\lambda}. \end{aligned}$$

where a second derivation is provided based in the fact that the MTTF is the sum mean times leading to the first and second failure:

TMR is more reliable than a single module up to the crossover point: t_c .

$$3r^2 - 2r^3 \geq r \text{ or } 2r^2 - 3r + 1 \leq 0.$$

The solution $r = 1/2$ leads to $t_c \leq \ln(2)/\lambda$, i.e., TMR is more reliable than a simplex system up to t_c .

TMR/Simplex is an alternate approach which randomly turns off one of two surviving modules to save power, but it turns out this increases the MTTF, which is the sum of its time to failures in its two operating phases with three and one modules.

$$\text{MTTF}_{TMR/S} = \frac{1}{3\lambda} + \frac{1}{\lambda} = \frac{4}{3\lambda}.$$

$R_{TMR-S}(t)$ can be obtained as the product of the *Laplace Stieltjes Transforms - LSTs* of two exponentials with parameters $\lambda_1 = 3\lambda$ and $\lambda_2 = \lambda$.

$$\mathcal{H}(s) = \frac{\lambda_1}{s + \lambda_1} \times \frac{\lambda_2}{s + \lambda_2}.$$

Inverting the LST yields the density function, which can be used to obtain the density function for TMR/S reliability. A higher MTTF is not the best reliability metric as was illustrated earlier. In cases when the reliability expression is available the reliability at the end of the desired lifetime of the system, e.g., five years is more interesting. This requires the estimating of the distribution of time to failure, which is not always easy. An approach used by some researchers is to obtain the MTTF and then assume that the time to system failure is exponentially distributed: $R(t) = e^{-t/\text{MTTF}}$. This approach is unreliable since with this approximation a single module has a higher reliability than TMR for all $t > 0$.

An Wang the inventor of core memories was paid \$500K for his invention by IBM.

https://en.wikipedia.org/wiki/An_Wang

IBM also paid MIT \$13M for Jay Forester's work on perfecting core memories as part of the Whirlwind project at MIT.

https://en.wikipedia.org/wiki/Jay_Wright_Forester

DYNAMO was a simulation language developed under the direction of Jay Wright Forrester at the MIT's Computation Center and was used for the system dynamics simulations of global resource-depletion reported in the Club of Rome's Limits to Growth studies.

https://en.wikipedia.org/wiki/Club_of_Rome

<https://www.clubofrome.org/about-us/history/>

The club was cofounded by Mihajlo Mesarovic a member of Systems Eng. Dept at Case Western reserve U. - CWRU.

https://en.wikipedia.org/wiki/Mihajlo_D._Mesarovic

Wang Institute of graduate studies, which operated from 1979 to 1988, attracted researchers such as David Lomet from IBM, who later joined Microsoft.

https://en.wikipedia.org/wiki/Wang_Institute_of_Graduate_Studies

Wang also founded Wang Labs which went out of business in spite of its innovative products, such as calculators, word processors, and minicomputers.

https://en.wikipedia.org/wiki/Wang_Laboratories

2.3.6 Semiconductor memories

Varieties of semiconductor memories are discussed in (Jacob et al., 2008).

https://en.wikipedia.org/wiki/Semiconductor_memory

Semiconductor memories replaced core memories, because of their higher density, lower cost, lower access time, lower power consumption, although they are volatile.

Main memories today are based on DRAM technology.

https://en.wikipedia.org/wiki/Dynamic_random-access_memory

DRAM invented by IBM's Robert H. Dennard allows higher density, since it uses a **single** rather than 4-6 transistors of *Static RAM* - *SRAM*.

https://en.wikipedia.org/wiki/Robert_H._Dennard

Dennard is shown below with his invention drawn on a white-board.

<https://www.prnewswire.com/news-releases/robert-h-dennard-dram-inventor-and-scaling-pioneer-to-receive-semiconductor-industrys-top-honor-300877816.html>

SRAM is faster and used for CPU caches (Jacob et al., 2008).

https://en.wikipedia.org/wiki/Static_random-access_memory

The difference between *Dual Inline Memory Module* - *DIMM* and *Single Inline Memory Module* - *SIMM* is explained in (Jacob et al., 2008).

<https://en.wikipedia.org/wiki/DIMM>

DIMM contains one or several RAM chips on a small circuit board with pins that connect it to the computer motherboard. The DIMM stores each data bit in a separate memory cell. DIMMs use a 64-bit data path, since processors used in personal computers have a 64-bit data width.

<https://www.anandtech.com/show/15912/ddr5-specification-released-setting-the-stage-for-ddr56400-and-beyond>

Modern microprocessors and other VLSI *Systems on Chip* - *SOCs* are characterized by high leakage currents and require increasing amounts of embedded memories, which are implemented as six transistor (6T) SRAM microcells (there are also 4T versions). Such cells have several advantages such as fast access and reliable data retention, their disadvantage is that each cell occupies a large area and exhibits several leakage paths.

Gain-Cell embedded DRAM - *GC-eDRAM* circumvents these limitations, while remaining compatible with CMOS technologies, or more accurately *Complementary-Symmetry Metal-Oxide-Semiconductor* - *COS-MOS*, which is a type of *Metal-Oxide-Semiconductor Field-Effect Transistor* - *MOSFET* fabrication process that uses complementary and symmetrical pairs of p-type and n-type MOSFETs for logic functions. CMOS technology is used for constructing *Integrated Circuit* - *IC* chips, including microprocessors, microcontrollers, memory chips (including CMOS BIOS), and other digital logic circuits. CMOS technology is also used for analog circuits such as image sensors (CMOS sensors), data converters, RF circuits, etc.

<https://en.wikipedia.org/wiki/CMOS>

An internal feedback mechanism is used to increase the data retention time. A 2 kilobit memory macro was implemented in low power 65 nm CMOS technology and this results in a 3-fold improvement in retention time, a 5-fold reduction in retention power, and a 40% reduction in bitcell area with respect to 6T SRAM (Bonetti et al., 2020). Researchers at *Ecole Polytechnique Federale de Lausanne* - *EPFL* and Bar Ilan U. in Israel have developed a new type of embedded memory that takes up half as much space as traditional memory and uses less energy to store a given amount of data.

There is also a *non volatile SRAM* - *nvSRAM* (Xie, 2014), which extends the functionality of basic SRAM by adding non-volatile storage such as an EEPROM to the SRAM chip. In operation, data is written to and read from the SRAM portion with high-speed access; the data in SRAM can then be stored into or retrieved from the non-volatile storage at lower speeds when needed.

<https://en.wikipedia.org/wiki/NvSRAM>

2.3.7 Redundant array of Independent Memories - RAIM

RAIM is a new paradigm for main memory protection developed in conjunction with IBM's z-series computers (Meaney et al., 2012).

<https://www.ibm.com/community/z/wp-content/uploads/sites/14/2020/04/sysdevblog-4cde-ibm2520enterprise2520raim.pdf>

https://en.wikipedia.org/wiki/Redundant_array_of_independent_memory

Abstract

The IBM zEnterprise system introduced a new and innovative redundant array of independent memory (RAIM) subsystem design as a standard feature on all zEnterprise servers. It protects the server from single-channel errors such as sudden control, bus, buffer, and massive dynamic RAM (DRAM) failures, thus achieving the highest System z[®] memory availability. This system also introduced innovations such as DRAM and channel marking, as well as a novel dynamic cyclic redundancy code channel marking. This paper describes this RAIM subsystem and other reliability, availability, and serviceability features, including automatic channel error recovery; data and clock interface lane calibration, recovery, and repair; intermittent lane sparing; and specialty engines for maintenance, periodic calibration, power, and power-on controls.

Error control codes called diff-MDS are described in (Lastras et al., 2011).

Abstract

In this article we describe a class of error control codes called “diff-MDS” codes that are custom designed for highly resilient computer memory storage. The error scenarios of concern range from simple single bit errors, to memory chip failures and catastrophic memory module failures. Our approach to building codes for this setting relies on the concept of expurgating a parity code that is easy to decode for memory module failures so that a few additional small errors can be handled as well, thus preserving most of the decoding complexity advantages of the original code while extending its original intent. The manner in which we expurgate is carefully crafted so that the strength of the resulting code is comparable to that of a Reed-Solomon code when used for this particular setting. An instance of this class of algorithms has been incorporated in IBM’s zEnterprise mainframe offering, setting a new industry standard for memory resiliency.

Chipkill is IBM’s trademark for a form of advanced *Error Checking and Correcting - ECC* computer memory technology that protects computer memory systems from any single memory chip failure as well as multi-bit errors from any portion of a single memory chip.

<https://en.wikipedia.org/wiki/Chipkill>

One simple scheme to perform this function scatters the bits of a Hamming code ECC word across multiple memory chips, such that the failure of any single memory chip will affect only one ECC bit per word. This allows memory contents to be reconstructed despite the complete failure of one chip. Typical implementations use more advanced codes, such as a *Bose-Chaudhuri-Hocquenghem - BCH* code, that can correct multiple bits with less overhead.

https://en.wikipedia.org/wiki/BCH_code

Chipkill is frequently combined with dynamic bit-steering, so that if a chip fails (or has exceeded a threshold of bit errors), another, spare, memory chip is used to replace the failed chip. The concept is similar to that of RAID, which protects against disk failure, except that now the concept is applied to individual memory chips.

A 2009 paper using data from Google’s datacentres (Schroeder et al., 2009) provided evidence demonstrating that in observed Google systems, DRAM errors were recurrent at the same location, and that 8% of DIMMs were affected each year. “In more than 85% of the cases a correctable error is followed by at least one more correctable error in the same month”. DIMMs with chipkill error correction showed a lower fraction of DIMMs reporting uncorrectable errors compared to DIMMs with error correcting codes that can only correct single-bit errors.

2.3.8 Magnetic Random Access Memory - MRAM

What follows is the abstract from a book on MRAM (Dieny et al., 2017).

“MRAM is poised to replace traditional computer memory based on CMOS. MRAM will surpass all other types of memory devices in terms of nonvolatility, low energy dissipation, fast switching speed, radiation hardness, and durability. Although toggle-MRAM is currently a commercial product, it is clear that future developments in MRAM will be based on spin-transfer torque, which makes use of electrons’ spin angular momentum instead of their charge. MRAM will require an amalgamation of magnetics and microelectronics technologies. However, researchers and developers in magnetics and in microelectronics attend different technical conferences, publish in different journals, use different tools, and have different backgrounds in condensed-matter physics, electrical engineering, and materials science. This book is an introduction to MRAM for microelectronics engineers written by specialists in magnetic materials and devices. It presents the basic phenomena involved in MRAM, the materials and film stacks being used, the basic principles of the various types of MRAM (toggle and spin-transfer torque; magnetized in-plane or perpendicular-to-plane), the back-end magnetic technology, and recent developments toward logic-in-memory architectures. It helps bridge the cultural gap between the microelectronics and magnetics communities.”

2.3.9 Magnetic tapes and tape libraries

A technical discussion of magnetic tape and disk technology is given by IBM's Richard Matick in (Matick, 1975, 1977).
https://en.wikipedia.org/wiki/Magnetic_tape_data_storage

The first tape was 1/2 inch wide and was developed by computer pioneers John Eckert, Jr. and John Mauchly in 1951 in conjunction with the UNIVAC computer.

<https://www.history.com/this-day-in-history/univac-computer-dedicated>

IBM's Model 726 was the first commercial digital tape storage system that could store about 1.1 MB per reel.

https://en.wikipedia.org/wiki/IBM_729

Nine track tapes had eight data bits and a parity bit per column

https://en.wikipedia.org/wiki/9_track_tape

with a 800, 1600, and 6250 *characters per inch - cpi* density, yielding 22.5 MB, 45 MB and 175 MB capacity respectively on a standard 2,400 feet (730 meter) tape.

<https://www.ibm.com/it-infrastructure/storage/tape>

Since most errors occur in bursts more sophisticated error correction methods than a single parity bit per column are required (Peterson and Weldon, 1972), (MacWilliams and Sloane, 1977), (Fujita, 2006).

https://en.wikipedia.org/wiki/Burst_error-correcting_code

Early magnetic tapes required manual mounting/dismounting.

https://en.wikipedia.org/wiki/Magnetic_tape_data_storage

In the case of early tape reels two vacuum columns were provided to ensure that the tape is not torn as the tape is pulled forward or stopped.

Linear Tape Open - LTO, also known as the LTO Ultrium format is a powerful, scalable, adaptable open tape format that is optimized for high capacity, maximum storage density and performance. Since the first LTO products were brought to market, over 5.6 million drives, 351,732,245 cartridges and 370,870 Billion GB of media capacity have been shipped, making LTO Ultrium the most successful tape format in history.

<https://www.lto.org/technology/what-is-lto-technology/>

The LTO tape drives in 2000 held 100 GB uncompressed, 200 GB compressed, and 20/40 MB/s transfer rate and in their LTO-9 in Sept. 2000 18 TB uncompressed and 45 TB compressed and 400/1000 MB/s transfer rate.

https://en.wikipedia.org/wiki/Linear_Tape-Open

Tapes are several hundred meters in length and it takes 12.5 hours to write an LTO-9 tape, Twelve generations of LTO, have been announced with LTO-12 144 native and 360 TB compressed capacity.

There is a quest for higher tape recording, e.g., 200 Gbits per square inch.

https://www.eetimes.com/document.asp?doc_id=1332110#

IBM Zurich Research Lab - ZRL's effort is described in (Argumedo et al., 2008). Magnetic disks can attain even higher densities, e.g., 1.34 TB.inch², which may be the reason that disks are replacing tapes (Gray, 2002).

[https://en.wikipedia.org/wiki/Areal_density_\(computer_storage\)](https://en.wikipedia.org/wiki/Areal_density_(computer_storage))

Tapes have the advantage of not consuming power when not in use, while modern disks unlike removable disks are always mounted so that no manual intervention is required, but they can be spun down to save energy, as in the case of *Massive Arrays of Mostly Idle Disks - MAID* (Colarelli and Grunwald, 2002). This led to energy efficient Copan storage (Guha, 2006), which is discussed in Chapter 7.

Tapes are nowadays used for archiving and backup, but in the early days of computing they held the files in data processing such as sorting and even compiling Fortran on the IBM 1410. Sorting is discussed in detail in Knuth's third volume on "Sorting and Searching" (Knuth, 2005).

Consider a tape holding customer records of an electric utility in ascending order of a unique customer id in a Mster/File - M/F. Customer payments are first sorted according to this id, before updating the using a merge-like operation on a periodic basis. The tape for old M/F can be kept for backup along with the file used for updating it. If the system crashes for any reason the update program can be rerun using the input tape. Tapes are labeled externally to identify them, but there is always the possibility of a wrong tape being mounted manually. An electronic label written in tape could be used to check if the correct tape is mounted.

<https://www.ecma-international.org/publications-and-standards/standards/ecma-13/>

Tape libraries have been termed tertiary as opposed to secondary storage. Manual intervention is not required in the case of automated tape libraries or *Mass Storage Systems - MSSs*, such as the StorageTek automated cartridge system (Larson et al., 1987), the *National Center for Atmospheric Research - NCAR* mass storage system described in (Nelson et al., 1987).

https://en.wikipedia.org/wiki/Tape_library

Analysis of long term file reference patterns for application to file migration algorithms is reported in (Smith, 1981b).

The IBM 3850 discontinued in 1986 used robotic arms to store and retrieve 15 TB tape cartridges.

https://en.wikipedia.org/wiki/IBM_3850

IBM TS4500 Tape Library has a maximum capacity with LTO-8 / TS1155: 263.25 PB / 278 PB. A modern tape cartridge can hold 15 TB, A single 278 PB robotic tape library has the capacity of 400 million *Compact Disks* - *CDs*.

<https://www.ibm.com/products/ts4500>

https://en.wikipedia.org/wiki/Compact_disc

The coupling of multiple small and inexpensive jukeboxes together to create larger tertiary storage systems called *Redundant array of Independent Libraries* - *RAIL* was explored at IBM ARC (Ford et al., 1998). RAIL has performance and availability characteristics superior to conventional tertiary storage systems for almost the same \$/MB cost. The writing of small blocks requires two tapes to be mounted, the data and parity tape, as in RAID4. Presented is the architecture of prototypes StarFish I and II, which is a *Log-Structured Library Array* - *LSLA* on top of a RAIL architecture allowing compression and elimination of the *Small Write Penalty* - *SWP*.

The *Linear File Tape System* - *LFTS* adds metadata to a tape, to provide a file system interface to data on tape.

<https://www.snia.org/education/what-is-lfts>

2.3.10 An analytical model for a tape library

An analytical model to evaluate the performance of a tape library system is presented by IBM *Zurich Research Lab* - *ZRL* researchers in (Iliadis et al., 2016). The analysis takes into account the number of cartridges and tape drives as well as different mount/unmount policies to determine the mean waiting time for tapes. The accuracy of the model is confirmed by validation against measurements. Earlier work on this topic is also reviewed in the paper.

A tape library system consists of tape drives, robot arms, a storage rack for the tape cartridges, and a cartridge control unit. To serve a request, a robot arm fetches the appropriate tape cartridge from the storage rack and delivers it to a free tape drive. The tape drive unit mounts the tape, positions the head to the desired file and transfers its data. To free a tape drive, a robot arm unmounts the tape cartridge and returns it to the storage rack. Tape read/write requests specify the cartridge id, the position of the data block in the cartridge, and the size of the block. Requests submitted for cartridges are queued and served according to a scheduling policy. The hierarchical scheduling algorithm used to ensure fairness and avoid starvation is as follows:

Upper level: Cyclic or round-robin scheduling among the queues (mounting cartridges).

Lower level: A FCFS policy for serving requests within a queue (reading from a mounted cartridge).

Improving system performance by scheduling batches of requests in cartridges to minimize seek times is a desirable option not considered in the study.

When all requests for a cartridge are served (exhaustive service), and there are still pending requests at some other, non-mounted cartridge, the cartridge is unmounted and another cartridge with pending requests is mounted. If, however, there are no other pending requests to any other non-mounted cartridge, the cartridge can either remain mounted in anticipation of future requests or be unmounted so as to save time when future requests arrive for other non-mounted cartridges. Two mount/unmount policies deployed in this context are:

Always-Unmount - AU: A tape cartridge is immediately unmounted upon completion of all pending requests for it in anticipation of the next request arriving for another non-mounted cartridge.

Not-Unmount - NU: A tape cartridge remains mounted upon completion of all pending requests for it in anticipation of the next request arriving for this same currently mounted, but idle cartridge.

The performance analysis of the resulting polling system, which relies on (Takagi, 1986) with state-dependent queues is quite complex and beyond the scope of this discussion.

For the IBM RS4500 tape library system $c = 72$, $d = 12$, so that $n = 60$ (IBM Corp., 2016). It is assumed that $a = d$, i.e., there is no contention for robot arms. Parameters used in this study are given in Table 2.1.

Predicting the performance of a tape library system is key to efficiently dimension it, possibly in the context of multi-tiered storage systems that include tape libraries. The effect of two mount/unmount policies was analytically assessed. For light loads, the AU - Always-Unmount policy yields a mean delay that is lower than that of the NU - Not-Unmount policy, with the difference becoming negligible as the load increases. The effect of the number of robot arms was assessed by means of simulation shows that given fast robot arms multiple arms do not provide a significant performance improvement, but multiple arms may be necessary to attain higher availability.

TABLE 2.1 Range of values of different parameters according to Table II in (Iliadis et al., 2016).

Parameter	Range	Definition
c	20-720	number of cartridges
d	2-12	number of tape drives
a	1,2,6,12	number of arms
M	15 s	mount time
U	77 s	unmount time
s	60 s	seek time
Q	834 MB	mean request size
Q2	2.8 GB	standard deviation
bw	360 MB/s	bandwidth
n	12,20,60	# cartridges per tape drive

2.3.11 Summary of a recent article on magnetic tapes

The volume of data being recorded is increasing 30-40% per year, while HDD capacity is increasing at half that rate according to Mark Lantz of advanced tape technologies at IBM's ZRL (Lantz, 2018). The solution is to keep most of the world's data on tape which incurs one sixth of the cost to keep the same amount of data on disks. Tape in the form of reel-to-reel, cartridge, and cassette tapes is disappearing from consumer-level products.

The offline nature of tape is a defense against software bugs, for example in 2011 when a software flaw at Google resulted in deleting the email messages of 40,000 Gmail accounts, although these data was stored on HDDs across multiple data centers. The data backed up on tape was then used to restore the lost data.

<https://gmail.googleblog.com/2011/02/gmail-back-soon-for-everyone.html>

Early disks and tapes had about the same density, but disks surpassed tapes due to the fact that HDD sales produced more revenue. The real density of HDDs is about hundred times of tape density. For comparison, the world's largest HDDs are about twice the physical size of a Sony tape cartridge are the 60 TB Seagate SSD or 12 TB HGST helium-filled HDD. The largest commercially available tapes only store 15 TB.

Researchers at IBM's ZRL have collaborated with Fujifilm and Sony to achieve higher tape densities with 32 track recording. Tracks are about 100 nanometers wide, i.e., 246,000 tracks per inch and there is also the issue of increasing horizontal density, 818,000 bits per inch. With an areal recording density of 201 gigabits per square inch. With educed thickness an 1140 meter tiny tape cartridges can record 330 TB.

<https://arstechnica.com/information-technology/2017/08/ibm-and-sony-cram-up-to-330tb-into-tiny-tape-cartridge/>

The new cartridges, when they're eventually commercialized, will be significantly more expensive because of the complex manufacturing process. Likewise, a new tape drive costing several thousand pounds would be required. The latest breakthrough in this area is a 580 TB tape cartridge.

The rundown of IBM's tape density records is given in Table 2.2.

<https://gizmodo.com/a-new-breakthrough-in-tape-storage-could-squeeze-580-tb-1845851499>

TABLE 2.2 Rundown of IBM's tape storage density records, μm = micrometer, BF = Barium Ferite, SM = Sputtered media.

Year	2006	2010	2014	2015	2017
Cartridge capacity TB	8	35	154	220	330
# books stored	9M	35M	154M	220M	330M
Track width (μm)	1.5	0.45	0.177	0.140	0.103
Linear density kbits/inch	400	518	600	680	818
Tape material	BF	BF	BF	BF	SM
Tape thickness (μm)	6.1	5.9	4.3	4.3	4.7
Tape length (meters)	890	917	1255	1255	1098

The *Information Storage Technology Consortium - ISTC*, which includes HPE, IBM, Oracle, and Quantum and academic research groups have released an “International Magnetic Tape Storage Roadmap”, which predicts that real density will reach 91 (resp. 200) GB per square inch by 2025 (resp. 2028).

2.3.12 Origins of Hard Disk Drives - HDDs

Reynold Johnson at IBM San Jose, CA authored “A Proposal for Rapid Random Access File” in 1953, which quoted the 1949 patent “Magnetic Memory Device” by Jacob Rabinow at the *National Bureau of Standards - NBS*, renamed *National Institute of Science and Technology - NIST*. <https://www.nist.gov>
<http://history.computer.org/pioneers/rabinow.html>

Johnson’s proposal led to the first commercial disk drive named *Random Access Method of Accounting and Control - RAMAC* by IBM, which could store only 3.75 MegaBytes - MB on fifty large disks according to (Sierra, 2012):

“The first Direct Access Storage Device - DASD RAMAC 350 was conceived and designed by IBM personnel in downtown San Jose, CA in early 1950s. The feasibility models were a hodge-podge of components available at the time: the spindles were obtained from juke boxes, the disks were aluminum pizza plates with a hole in the middle, the magnetic recording materials (liquid) were poured from paper drinking cups and deposited manually, the first read/write heads were assembled by former watchmakers, etc. Some of the early servomotors consisted of coils obtained from hi-fi loudspeakers whose cones had been removed. Today, those motors are still called “voice” coil motors, a term that, although historically correct, is rather anachronistic since there are no “voices” (or music!) to be heard. Also, the coils are designed today especially for disk drives, not for loudspeakers.”

IBM accomplishments in disk storage technology are listed here:

https://www-03.ibm.com/ibm/history/exhibits/storage/storage_chrono20.html

A general discussion of storage systems is:

<http://www.computerhistory.org/timeline/memory-storage/>

An introduction to IBM’s DASD and file systems is presented in (Matick, 1975, 1977), (Bohl, 1981), (Sierra, 2012), (Jacob et al., 2008). An up-to-date discussion of magnetic disk drives was authored by Spencer W. Ng (IBM Research, Hitachi Global Storage Technologies - HGST, SanDisk, retired) as part of “DRAM, Cache, Disk” (Jacob et al., 2008). A more recent article asks “What is the Future of Disk Drives, Death or Rebirth?” (Deng, 2011). This question is answered by Eric Brewer et al. in (Brewer et al., 2016).

“Disks form the central element of Cloud-based storage, whose demand far outpaces the considerable rate of innovation in disks. Exponential growth in demand, already in progress for 15+ years, implies that most future disks will be in data centers and thus part of a large collection of disks. We describe the “collection view” of disks and how it and the focus on tail latency, driven by live services, place new and different requirements on disks. Beyond defining key metrics for data-center disks, we explore a range of new physical design options and changes to firmware that could improve these metrics.”

2.3.13 HDD manufacturers

Established ADSTAR as a wholly owned subsidiary In 1992 IBM’s Storage Products businesses comprising eleven sites in eight countries were combined into one division named *ADvanced STorage And Retrieval - ADSTAR*, which had a gross profit of about \$440 M and a revenue of \$6.11 B or just 7.2% in April 1993 was an admission by IBM that it no longer provided a strategic advantage by providing proprietary devices for its mainframe products and that it was being positioned to be sold off. IBM Chairman John Akers’ business strategy was reversed by Lou Gerstner who replaced Akers in April 1993, but the disk drive business continued to be troubled, declining to \$3 billion in 1995. The ADSTAR division was dismembered thereafter.

<https://en.wikipedia.org/wiki/ADSTAR>

ADSTAR profitability should be compared with top high tech companies listed in Table 2.3 in profitability.

<https://www.visualcapitalist.com/how-big-tech-makes-their-billions-2020/>

Hitachi Global Storage Technologies - HGST was a merger of Hitachi and IBM’s storage businesses in 2003. HGST was acquired by *Western Digital Corp. - WDC* in 2012, which also acquired Sandisk, third largest flash manufacturer in 2/15.

TABLE 2.3 Net incomes and capitalizations versus revenue for leading technology companies in billions of USD.

Company	Revenue (2019)	Net Income (2019)	Market Cap (7/20)
Apple	260.2	55.2	1580
Amazon	280.5	11.6	1440
Alphabet	161.9	34.3	1020
Microsoft	125.8	39.2	1560
Facebook	70.8	18.5	665
Combined	899.2	158.8	6240

Western Digital intends to exit storage systems business.

<https://www.anandtech.com/show/14890/western-digital-to-exit-storage-systems-sells-off-intelliflash-division>

A list of HDD manufacturers includes WDC, Seagate, etc.

<https://www.ranker.com/list/the-best-hard-drive-manufacturers/computer-hardware>

The increase in areal density on HDDs has slowed from 40% a year to 10-15%, which is due to the superparamagnetic limit, i.e., the size of magnetic grains. <https://en.wikipedia.org/wiki/Paramagnetism>
Perpendicular Magnetic Recording - PMR delivers three times the storage capacity of *Longitudinal Magnetic Recording - LMR*.

https://en.wikipedia.org/wiki/Perpendicular_recording

Seagate is the leader in *Heat Assisted Magnetic Recording - HAMR*, while WDC developed *Microwave Assisted Magnetic Recording - MAMR* to allow smaller magnetic grains.

<https://www.wdc.com/about-wd/newsroom/press-room/2017-10-11-western-digital-unveils-next-generation-technology-to-preserve-and-access-the-next-decade-of-big-data.html>

Twenty TB HDDs were expected using MAMR in 2020. Western Digital and Seagate are involved in developing *Energy Assisted Magnetic Recording - EAMR* drives. Seagate with HAMR expects to gain advantage over WDC and Toshiba.

<https://blocksandfiles.com/2020/03/12/seagate-hamr-hard-drive-roadmap/> .

WDC projects that the 2023 HDD *Total Available Market - TAM* will grow to \$27.5 B from \$24.9 B.

<https://www.forbes.com/sites/tomcoughlin/2018/12/09/energy-assisted-magnetic-recording-for-enterprise-storage/#3aaa8a77cd82>

Having invested into various EAMR technologies, WDC discovered that transition to HAMR is hard, since it requires the introduction of new heads as well as all-new platters, whereas MAMR needs new heads and slightly different platters. While working on MAMR, WDC found that it was possible to only partially implement the technology and still get the desired result – an increase in areal density.

In fact WDC's Ultrastar HC550 HDDs (and 20 TB drives) will be the first products to use this type of EAMR technology that is currently called *Energy-enhanced Perpendicular Magnetic Recording - EPMR*. WDC's 18 TB Sans SMR Ultrastar DC HC550 3.5-inch HDD, which relies on company's 6th generation helium-filled HelioSeal platform.

<https://www.anandtech.com/show/14869/western-digital-announces-18-tb-eamr-hard-drive>

The two key improvements are nine platters (both for 16 TB and 18 TB versions) and EAMR technology, which enables WDC to build 2 TB platters without using SMR.

The Ultrastar DC HC550 also includes several other innovations, top and bottom attached disk clamps, RVFF sensors, humidity sensors, and other ways to boost reliability and ensure consistent performance. The drives are rated for a 550 TB/annual workload, a 2.5 million hours MTBF, and are covered by a five-year limited warranty.

Seagate has developed *Multi Actuator technology-MACH.2*, which doubles IOPS in a single hard HDD, by using two independent actuators. <https://www.seagate.com/our-story/introducing-mach2/>

Showa Denko Corp. - SDK

The following discussion is based on the following article in Forbes by Tom Coughlin.

<https://www.forbes.com/sites/tomcoughlin/2020/02/12/80-tb-hard-disk-drives>

Showa Denko corporation is the largest independent manufacturer of HDD magnetic media, which it sells to all HDD manufacturers.

<https://www.sdk.co.jp/english/>

In 2/19 SDK announced that it will begin shipment of newly developed 3.5-inch HDD media for MAMR. This product was adopted by Toshiba Electronic Devices and Storage Corporation for use in its MAMR HDDs with 18 TB for near-line storage applications. SDK said that this media has a 2 TB per disk storage capacity on an aluminum substrate. The company said that it was also working on HAMR media. This MAMR product was their 10th generation *conventional magnetic recording* - CMR with perpendicular recording.

On 2/6/2020 SDK said that it has developed media for HAMR. The company says that it has worked out a new structure of magnetic layers and new ways of controlling temperature during HDD media production. Because of the high temperatures used HAMR media is very different from conventional HDD media and is often an iron, rather than cobalt-based alloy. Cooperation with Seagate on HAMR was announced on 6/10/21.

<https://www.sdk.co.jp/english/news/2021/41220.html>

According to SDK with conventional recording technologies current HDD storage areal densities are about 1.14 TB/in², but 5-6 TB/in² can be achieved using HAMR-based recording methods. If the same number of disks are used as in current 3.5-inch form factor near-line products (8-9 disks) this would enable HDD capacities of 70-80 TB. Seagate has announced that it will provide HAMR HDDs with 20 TB by the second half of 2020. These drives may use Seagate's internally manufactured HAMR media, but may also use SDK media. Toshiba is expected to use SDK 2 TB MAMR disks in its 18 TB HDDs this year.

WDC is shipping what it called ePMR HDDs with 18 and 20 TB capacities this year (20 TB is with shingled magnetic recording). In the future, WDC said it will use MAMR and HAMR technology for its high capacity HDDs.

2.3.14 Storage technologies expected to replace disk drives

Garth Gibson projected in (Gibson, 1992), which was based on his UCB PhD thesis that DRAM will replace HDDs in a few years. Graphs in Figure 2.1a favoring DRAM show a crossover point in HDD versus DRAM storage cost in the period 2001-7, but graphs in Figure 2.1b favoring HDDs go beyond 2020. DRAM's weakness is its volatility, which could be dealt with using an *Uninterruptible Power Supply* - UPS.

https://en.wikipedia.org/wiki/Uninterruptible_power_supply

The introduction of flash and *Storage Class Memories* - SCMs has resulted in a major storage landscape change (Burr et al., 2008), (Freitas and Wilcke, 2008). It is speculated that magnetic disk drives will be replaced by one or more of the SCMs presented in (Burr et al., 2008), which have random and sequential performance orders of magnitude better than disks. A 100- to 1,000-fold advantage for SCM in terms of the data center space and power is required till 2020.

The abstract for emerging nonvolatile storage technologies given in (Kryder and Kim, 2009) is as follows:

"There are numerous emerging nonvolatile memory technologies, which have been proposed as being capable of replacing HDDs. In this paper, the prospects for these alternative technologies to displace HDDs in 2020 are analyzed. In order to compare technologies, projections were made of storage density and performance in year 2020 for both hard disks and the alternative technologies, assuming the alternative technologies could solve their remaining problems and assuming that hard drives would continue to advance real density at a pace of about 40% per year, which would result in a two-disk 2.5-in disk drive that stores approximately 40 TBs and costs about \$40."

Given in Tables 2.4 and 2.5 are the characteristics of NVM technologies based on Kryder and Kim (2009), where a major conclusion is that competing with hard drives on a cost per terabyte basis will be challenging for any solid state technology, because the *International Technology Roadmap for Semiconductors* - ITRS lithography roadmap limits the density that most alternative technologies can achieve. Those technologies with the best opportunity have a small cell size and the capability of storing multiple bits per cell. *Phase Change RAM* - PCRAM and *Spin Transfer Torque RAM* - STTRAM appear to meet these criteria. PCRAMs are being marketed by at least one supplier and therefore appear to be closer to practical realization. On the other hand, STTRAMs would appear to have a performance edge assuming they, too, can be brought to market with multiple bits per cell. Although there are technologies that are not limited by the lithography roadmap and thus have greater areal density potential, they tend to be further from practical realization."

TABLE 2.4 Characteristics of alternative NVM technologies (Part I).

Device type	HDD	DRAM	NAND Flash	FRAM	MRAM	STTRAM	PCRAM	NRAM
Maturity	Product	Product	Product	Product	Product	Prototype	Product	Prototype
Density	400Gb/in ²	8Gb/chip	64 GB/chip	128 Mb/chip		2 MB/chip	512/chip	NA
Cell size	(2/3)F ²	6F ²	4F ²	6F ²	20F ²	3F ²	5F ²	5F ²
MLC capability	No	No	4 bits/cell	No	2 bits/cell	4 bits/cell	4 bits/cell	No
Energy/bit	NA	2pJ		10nJ	2pJ	0.2pJ	100pJ	10pJ
Access time (W/R)	9.5/8.5ms	10/10ns	200/25us	50/75ns	12/12ns	10/10ns	100/20ns	10/10ns
Endurance/retention	NA	10 ¹⁶ /64ms	10 ⁵ /10yr	10 ¹⁵ /10yr	10 ¹⁵ /10yr	10 ¹⁵ /10ys	10 ⁵ /10y	10 ¹⁶ /10yr

TABLE 2.5 Characteristics of alternative NVM technologies (Part II).

Device type	RRAM	CBRAM	SEM	Polymer	Molecular	Racetrack	Holographic	Probe
Maturity	Research	Prototype	Prototype	Research	Research	Research	Product	Prototype
Present density	64Kb/chip	2M/chip	128Mb/chip	128b/chip	12Kb/chip	NA	NA	NA
Cell size	6F ²	6F ²	4F ²	6F ²	6F ²	NA	NA	NA
MLC capability	2nits/cell	2bits/cell	No	2bits/cell	N	12bits/cell	NA	NA
Energy/bit	2pJ	2pJ	13pJ	NA	NA	2pJ	NA	100pJ
Access time (W/R)	10/20m	60/50ns	100/20ns	30/30ns	20/20ns	10/10ns	3.1/5.4ns	10/10ns
Endurance/retention	10 ⁵ /10yr	10 ⁶ /months	10 ⁹ /days	10 ⁴ /months	10 ⁵ /months	10 ¹⁶ /10yr	10 ⁵ /50yr	10 ⁵ /

References to alternative NVM technologies in the tables are as follows. *Phase Change RAM - PCRAM* (Qureshi et al., 2011), (Li and Chen, 2011), (Kim et al., 2014), (Gallo et al., 2019). Probe memory (Vettiger et al., 2000), Holographic memory (Ashley et al., 2000), Racetrack memory (Tomasello et al., 2014).

Forbes' Tom Loughlin has projected that emerging memory revenue will grow to \$44 B 2031.

<https://www.forbes.com/sites/tomcoughlin/2021/07/15/emerging-memories-44b-revenue-projected-by-2031/?sh=58c7cb99575f>

3D XPoint with brand name Optane discussed later could reach revenues of over \$20 B by 2031,

https://en.wikipedia.org/wiki/3D_XPoint

Magnetoresistive RAM - MRAM and *STT-RAM* revenues will grow to about \$1.7 Meanwhile, embedded ReRAM and MRAM will compete to replace the bulk of embedded NOR and SRAM in SoCs, fueling even greater revenue growth.

Resistive RAM - ReRAM is based on a simple three-layer structure of a top electrode, switching medium and bottom electrode. The resistance switching mechanism is based on the formation of a filament in the switching material when a voltage is applied between the two electrodes.

A comparison of several non-volatile memories is as follows.

nvSRAM: Technique: Non-volatile elements along with SRAM, which does not require memory backup (Hamacher et al., 2012).

<https://en.wikipedia.org/wiki/NvSRAM>

State retention: 20 years.

Endurance: unlimited.

Store mechanism: Autostore initiated when Vcc power down detected.

Power up data restore.

Non-volatile data is made automatically available in SRAM.

Substitution with SRAM: nvSRAM can be substituted for SRAM with minor board modification to add external capacitor.

Soldering: Standard *Surface Mount Technology* - *SMT* used.

Speed (best): 15-45 ns.

BBSRAM: Technique: Lithium energy source for power when external power is off.

Data retention: 7 years depends on battery and ambient temperature.

Store mechanism: Chip enable should be held high to prevent inadvertent reads/writes.

Power up data restore: SRAM will switch from battery to Vcc.

Substitution with DRAM: Provision for battery necessitates board redesign to accommodate bigger size for battery.

Soldering: Reflow solder cannot be done with battery installed as batteries may explode.

Speed (best): 70-100 ns.

Ferroelectric RAM: Technique: Has a ferroelectric crystal between electrodes to form a capacitor. The moment of atoms on application of electric fields is used to store data.

Data retention: 10 years.

Endurance: Unlimited.

Power up data restore: Static operation.

Data is stored in nonvolatile part only.

Soldering Standard SMT used.

Speed: 55 ns.

Magnetorestrictive RAM - MRAM: Technique: Similar to Ferroelectric RAM. but the atoms align themselves in the direction of external magnetic force.

Data retention: 20 years.

Endurance: Unlimited.

Power up data restore: Same as above.

Substitution with SRAM: Pin-to-pin compatible with existing SRAMs.

Soldering: Standard SMT used.

Speed: 35 ns.

2.3.15 Magnetic bubble memories

Magnetic bubbles are capable of carrying digital information and applying bubble logic.

https://en.wikipedia.org/wiki/Bubble_memory

Bubbles move synchronously under a global drive field, but also yield to local control by electric currents. In late 1970s they were considered a major contender for a high-density, nonvolatile, electronically accessed memory because of their high density, simple fabrication, low power dissipation, and nonvolatility.

Bubble logic in conjunction with bubble memory was much slower than its semiconductor counterpart. Bubble logic requires only simple additional controls, and the pipelined versions only add to the path length, without decreasing the data flow rate. The nonvolatility of bubble logic is an asset for guaranteed performance in hostile environments.

Bubble logic and the manipulation of bubble information is discussed in (Chen and Chang, 1978). The book (Chen, 1978) deals with the organization of magnetic bubble memories and sorting and searching. Record placement and access in magnetic bubble memories was studied in (Wong, 1980, 1983).

2.3.16 Charged Couple Devices - CCDs

A CCD is an *Integrated Circuit* - *IC* containing an array of linked, or coupled, capacitors. Under the control of an external circuit, each capacitor can transfer its electric charge to a neighboring capacitor. CCD sensors are a major technology used in digital imaging. CCDs were used in the *CASSM Data Base Machine* - *DBM* for non-numeric processing (Copeland et al., 1973). CCDs are discussed in (Matick, 1975, 1977).

https://en.wikipedia.org/wiki/Charge-coupled_device

2.3.17 Micro-Electro-Mechanical Systems - MEMS

It is noted in (Carley et al., 2000) at the turn of the century that disk drive capacities are increasing at 100% rate per year outpacing the 60% for semiconductor memories. While the latter density has decreased since 2000 disk storage remains the most cost-effective non-volatile storage with a reasonable access time compared to magnetic tape. The improvement in disk access time is minimal as discussed in (Gray and Shenoy, 2000).

MEMS is the latest attempt to deal with the access-time and entry-cost barriers which uses a hybrid approach combining the best of both semiconductor memories and disk drives. The following discussion is based on (Carley et al., 2000) which was published as part of a large project at CMU/PDL.

<https://www.pdl.cmu.edu/MEMS/index.shtml>

MEMS adopts the parallel wafer-fabrication process to keep unit costs low and adopted recording heads using mechanical positioning to address data stored on thin-film material instead of the lithographic fabrication of semiconductor memories. Like disk drives, the device has recording heads and a recording media surface that moves. However, the heads are actually MEMS probe tips fabricated in a parallel wafer-level manufacturing process. The CMU prototype employs magnetic storage media much like that used by disk drives, but the media surface instead of rotating moves linearly in the X and Y directions to seek the appropriate data. Data access is accomplished by moving the media at a constant velocity in the Y direction while data is read or written by the stationary probe tips. This design avoids problems with “stiction,” or the force required to begin moving one body whose surface is in contact with a fixed surface, that occurs in rotating bearings at very small geometries.

The media is deposited on a large (8 mm × 8 mm × 500 micrometer) square plate (media sled) and is held above the probe-tip array by a network of springs. A force is applied to the sled using electrostatic actuators, and such reciprocating motion is usually limited to a small fraction of the size of the structure, typically being 10% or less of the suspension/actuator length, a single probe tip “sweeps” only on the order of 1% of the media sled. All of the media area can be addressed by using a large array of probe tips, as long as the media sled moves in X and Y, by the pitch of the probe-tip array. A large array of probe tips also provides a significant increase in the data rate and reliability for the overall system.

Nearly all other approaches employ either passive or active independent positioning of probe tips in Z. IBM Millipede probe-based mass-storage system uses a 32 × 32 array of probe tips, each placed on the end of a cantilever (Vettiger et al., 2000).

2.3.18 IBM Zurich millipede

Millipede is the *Atomic Force Microscope* - AFM-based data storage that has a potentially ultrahigh density, terabit capacity, small form factor, and high data rate. Its potential for ultrahigh storage density has been demonstrated by a new thermomechanical local-probe technique to store and read back data in very thin polymer films. With this new technique, 30–40-nm-sized bit indentations of similar pitch size have been made by a single cantilever/tip in a thin (50-nm) PolyMethylMethAcrylate (PMMA) layer, resulting in a data storage density of 400–500 Gb/in². High data rates are achieved by parallel operation of large two-dimensional (2D) AFM arrays that have been batch-fabricated by silicon surface-micromachining techniques.

The VLSI of micro/nanomechanical devices (cantilevers/tips) on a single chip leads to the largest and densest 2D array of 32 × 32 (1024) AFM cantilevers with integrated write/read storage functionality ever built. Time-multiplexed electronics control the write/read storage cycles for parallel operation of the Millipede array chip. Initial areal densities of 100–200 Gb/in² have been achieved with the 32 × 32 array chip, which has potential for further improvements. In addition to data storage in polymers or other media, and not excluding magnetics, we envision areas in nanoscale science and technology such as lithography, high-speed/large-scale imaging, molecular and atomic manipulation, and many others in which Millipede may open up new perspectives and opportunities.

MEMS device characteristics and data layout

The MEMS media sled is organized into rectangular regions each storing $N \times M$ bits (2000 × 2000 bits in the model) accessible by a single probe tip. The smallest accessible data unit is a “tip sector” consisting of servo information (10b) and encoded data + error correction (80b = 8B encoded data). Multiple tip sectors are grouped into logical sectors, similar to a SCSI disk’s logical blocks. Unlike conventional disks, however, multiple probe tips access the media in parallel; many tip sectors can be read or written simultaneously.

To organize the low-level media contents, each bit is identified by the triple $\langle x, y, \text{tip} \rangle$ where $\langle x, y \rangle$ represents bit coordinates within the region addressable by $\langle \text{tip} \rangle$. Each active tip reads or writes data within a column of bits called a

“tip track.” Drawing an analogy with disk drives, each bit with identical values for $\langle x \rangle$ is called a “cylinder.” Therefore, a cylinder consists of all bits accessible by any tip without moving the sled along the X axis. Because power dissipation limits the number of probe tips that can be active simultaneously, cylinders are divided into tracks (all bits within a cylinder that can be read or written by concurrently active tips). Multiple simultaneously active tips allow logical sectors to be “striped,” or spread across tip sectors (in multiple tip tracks) to reduce access time.

Parameters and performance MEMS-based storage prototype at CMU are given in Tables 2.6 and 2.7.

TABLE 2.6 Parameters for system-level characterization of the media sled.

Parameter	Value
Sled mobility X & Y	100 micrometer
Bit cell width (area)	40 nm ²
Number of tips	6,400
Simultaneously active tips	1,280
Top sector length	80 bits
Servo overhead	10bits
Dice capacity	4.9 GB
Sled acceleration	82g
Per tip data rate	700 kb/s

TABLE 2.7 Performance of the MEMS-based storage system prototype at CMU.

Performance	Time (ms)
Avg/Max service time	0.67/1.92
Avg/Max seek time	0.53/0.73
Avg/Max X-seek time	0.52/0.73
Avg/max Y-seek time	0.7335/0.73
Settling time	0.22
Avg/max per request turnaround time	0.06/0.50

Total time of real applications for disks and MEMS-based storage is given in Table 2.8.

TABLE 2.8 Total time on real applications for disks and MEMS-based storage.

Benchmark	Storage Device	Compute Time	I/O Time	Total Time
POSTMARK	Atlas 10K	10	730	740
	Superdisk	10	400	410
	MEMS store	10	215	225
TPC-D (query 4)	Atlas 10K	3	27	30
	Superdisk	3	11	14
	MEMS store	3	7	10

2.3.19 Phase Change Memory - PCM

PCM is projected to be the top candidate for replacing disk drives, since it has a small cell size and can store multiple bits per cell (Qureshi et al., 2011).

https://en.wikipedia.org/wiki/Phase-change_memory

PCM is compared with other technologies in Table 2.9. It has been argued that all two-terminal non-volatile-memory devices, including PCM, should be considered memristors. However, this terminology has been challenged and the potential applicability of memristor theory to any physically realizable device is open to question.

<https://en.wikipedia.org/wiki/Memristor>

TABLE 2.9 Relative performance of memory technologies, including PCM.

	DRAM	NAND flash	Memristor	PCM
Byte addressable	Yes	No	Yes	Yes
Capacity	1X	4X	2-4X	2-4X
Latency	1X	400X	3-5X	3-5X
Endurance	10^{16}	10^4	10^6	$10^6 - 10^8$

The following discussion is based on the following article (Mirhoseini et al., 2012).

<http://news.rice.edu/2012/06/07/rice-ucla-slash-energy-needs-for-next-generation-memory/>

An optimization framework that exploits asymmetries in PCM read/write to minimize the number of bit transitions, which in turns yields energy and endurance efficiency. PCM uses heat-sensitive materials to store data as ones and zeros by changing the material resistance. The electronic properties of the material change from low resistance to high resistance when heat is applied to alter the arrangement of atoms from a conducting crystalline structure to a nonconducting glassy structure.

Writing data on PCM takes a fraction of the time required to write on flash memory, and the process is reversible but asymmetric; creating one state requires a short burst of intense heat, and reversing that state requires more time and less heat. The new encoding method is the first to take advantage of these asymmetric physical properties. One key to the encoding scheme is reading the existing data before new data is written.

Using a combination of programming approaches, the researchers created an encoder that can scan the “words” - short sections of bits on the card - and overwrite only the parts of the words that need to be overwritten. One part of the method is based on dynamic programming (Bellman, 2003), which starts from small codes that we show to be optimal, and then builds upon these small codes to rapidly search for improved, longer codes that minimize the bit transitions. The second part of the new method is based on *Integer-Linear Programming - ILP*, a technique that can find optimal solutions.

https://en.wikipedia.org/wiki/Integer_programming

The more complex the solution, the longer ILP takes to find the optimal solution, so the team found a shortcut by using dynamic programming to create a cheat sheet of small codes that could be quickly combined for more complex solutions.

Relative performance of memory technologies based on (DeBrabant et al., 2014) is given in Table 2.9.

2.3.20 Flash memories

Preliminary facts about flash storage are as follows.

- Flash was invented by Toshiba Corp. in 1984.
- Both NAND and NOR Flash technologies have been implemented.
- NOR Flash is *Single Level Cell - SLC*, while NAND is *Multiple Level Cell - MLC*.
- Flash devices can sustain a finite number of writes/erasures or *Program/Erase - P/E* cycles.
- MLC which stores twice the information of SLC wears out at a faster rate.
- NOR flash technologies are byte and NAND technologies are block addressable.
- NAND flash provides much higher recording density than NOR technology, hence of interest.
- Each NAND block has a limit of 100,000 rewrites, but that can be increased to 200K-3M rewrites.
- Flash memories have higher read than write data rates by a factor of three.
- For small blocks Flash devices can support 30 times more IOPs.
- For sequential I/O Flash provides sustained data rate of 220 MB per sec.
- Depending on the radial track location 15K RPM disks deliver 73-125 MB/sec on innermost vs outermost tracks.
- For small blocks flash provides 1.5 ms response time, versus 9 ms for 15K RPM disk drives.
- Error-correcting codes for *Triple Level Cell - TLC* flash memories are reported in (Yaakobi et al., 2012).

NAND and NOR flash memories are compared in Table 2.10.

TABLE 2.10 NAND vs NOR flash memories (Chang and Kuo, 2005).

	NAND	NOR
Density	High	Low
Read/write	Page-oriented	Bitwise
eXecute In Place (XIP)	No	Yes
Read/write/erase	Moderate/fast/fast	Very fast/slow/very slow
Cost per bit	Low	High

RAID memories based on SSD flash technology attain higher performance than HDD RAIDs, have smaller footprint, and lower power consumption.

<https://www.computerweekly.com/feature/SSD-RAID-essentials-What-you-need-to-know-about-flash-and-RAID>.

Most SSDs used buses such as SATA, SAS or Fibre Channel for interfacing with the rest of a computer system. Since SSDs became available in mass markets, SATA has become the most typical way for connecting SSDs in personal computers. However SATA was designed primarily for interfacing HDDs and has been observed to be increasingly inadequate for SSDs, which improved in speed over time. Within five years of mass market mainstream adoption (2005–2010) many SSDs were already held back by the comparatively slow data rates available for HDDs. Unlike HDDs, some SSDs are limited by the maximum throughput of SATA.

RAM-resident tables are used for flash memory management in which one single granularity size is used for both address translation and space management. As high-capacity flash-memory is becoming affordable more than ever, the dilemma of how to manage the RAM space or how to improve the access performance is emerging for many vendors.

A tree-based management scheme which adopts multiple granularities in flash-memory management is (Chang and Kuo, 2005). The objective is to not only reduce the runtime RAM footprint, but also manage the housekeeping for write workloads. The proposed method was evaluated under realistic workloads, where significant advantages over existing approaches were observed, in terms of the RAM space, access performance, and flash-memory lifetime.

This study makes use of AVL self-balancing trees (Adelson-Velskii and Landis, 1962), which are discussed in most books on data structures (Cormen et al., 2009), (Knuth, 2005). In an AVL tree the heights of the two child subtrees of any node differ by at most one; if at any time they differ by more than one, rebalancing is done to restore this property. Lookup, insertion, and deletion all take $O(\log n)$ time in both the average and worst cases, where n is the number of nodes in the tree prior to the operation. Insertions and deletions may require the tree to be rebalanced by one or more tree rotations.

Investigated in (Stoica et al., 2009) is the issue of evaluating and repairing write performance on flash devices. More specifically how a DBMS can take full advantage of flash memory as persistent storage. A flash aware data layout: *append and pack*, stabilizes device performance by eliminating random writes. The impact of append and pack on OLTP workload performance is assessed using both an analytical model and micro-benchmarks. It is concluded that significant improvements can be achieved for real workloads.

There are two fundamental differences between the SSD and RAID internal structures.

Different logical/physical mapping mechanisms: For RAID storage, logical blocks are statically mapped to “fixed” physical locations, which is determined by its *Logical Block Number - LBN*. For an SSD, a logical block is dynamically mapped to physical flash memory, and this mapping changes during runtime. Particular attention should be paid to a unique data layout problem in SSDs. Poor data layouts can cause undesirable performance degradation, while such a problem would not happen in RAID with its static mapping.

Different physical natures: RAID on magnetic disks has random access performance an order of magnitude lower than sequential access. For SSDs built on flash memories such a performance gap is much smaller, and thus are less sensitive to access patterns.

Without any moving parts, parallelizing I/O operations on SSDs could make random accesses capable of performing comparably to or even better than sequential accesses, while this is unlikely to happen in RAID. Extracting parallelism from HDDs, e.g., reading multiple tracks or intradisk parallelism poses difficulties discussed in (Gurumurthi et al., 2009). This is because data transfers from the Read/Write heads attached to the same disk arm cannot be coordinated simultaneously. RAID3 disk arrays utilize several synchronized disks to achieve higher data transfer rates.

Internal parallelism of flash SSDs is discussed in (Chen et al., 2016). Experiments and analysis shows that exploiting internal parallelism of SSDs can not only substantially improve I/O performance but also may lead to some surprising side

effects and dynamics. With parallel I/Os, SSD performance is no longer highly sensitive to access patterns (random or sequential), but rather to other factors, such as data access interferences and physical data layout, so that many of our prior understandings about SSDs also need to be reconsidered, e.g., with parallel I/Os write performance could outperform reads and is largely independent of access patterns, which is opposite to our long-existing common understanding about slow random writes on SSDs.

There is a strong interference between concurrent reads and writes as well as the impact of physical data layout to parallel I/O performance. Case studies of typical data-intensive application show that exploiting internal parallelism is the key to enhancing application performance, and more importantly, it also fundamentally changes the equation for optimizing applications. This calls for a reconsideration of aspects in application and system designs.

Experimental studies on SSDs and the interaction between internal and external parallelism in SSD-based RAID storage lead to recommendations for effectively exploiting internal parallelism are presented. Various forms of flash SSD parallelism are as follows:

Channel-Level Parallelism: Since the data bus is the main performance bottleneck, SSDs usually incorporate multiple channels to connect the controller to the flash memories. Each channel can operate independently and simultaneously.

Package-Level Parallelism: In order to optimize resource utilization, a channel is usually shared among multiple flash memory packages. Since each flash memory package can be operated independently, operations on flash memory packages attached to the same channel can be interleaved. As so, the bus utilization can be optimized.

Die-Level Parallelism: A flash memory package often includes two or more dies or chips. Each die can be selected individually and execute a command independent of the others to increase throughput.

Plane-Level Parallelism: A flash memory chip is typically composed of two or more planes. Flash memories by SAM-SUNG and MICRON support simultaneously performing the same operation, e.g., read/write/erase on multiple planes. Some flash memories provide cache mode to further parallelize medium access and bus transfer.

Analytic models for SSD flash write performance are presented in (Desnoyers, 2014).

“Flash SSDs update data by writing a new copy, rather than overwriting old data, causing prior copies of the same data to be invalidated. These writes are performed in units of pages, while space is reclaimed in units of multi-page erase blocks, necessitating copying of any remaining valid pages in the block before reclamation. The efficiency of this cleaning process greatly affects performance under random workloads; in particular, in SSDs the write bottleneck is typically internal media throughput, and write amplification due to additional internal copying directly reduces application throughput. We present the first nearly-exact closed-form solution for write amplification under greedy cleaning for uniformly distributed random traffic, validate its accuracy via simulation, and show that its inaccuracies are negligible for reasonable block sizes and over-provisioning ratios. In addition we also present the first models which predict performance degradation for both Least-Recently-Written - LRW cleaning and greedy cleaning under simple non-uniform traffic conditions; simulation results show the first model to be exact and the second to be accurate within 2%. We extend the LRW model to arbitrary combinations of random traffic, and demonstrate its use in predicting cleaning performance for real-world workloads. Using these analytic models, we examine the strategy of separating “hot” and “cold” data, showing that for our traffic model such separation eliminates any loss in performance due to non-uniform traffic. We then show how a system which segregates hot and cold data into different block pools may shift free space between these pools in order to achieve improved performance, and how numeric methods may be used with our model to find the optimum operating point, which approaches a write amplification of 1.0 for increasingly skewed traffic. We examine on-line methods for achieving this optimal operating point, and show a control strategy based on our model which achieves high performance for a number of real-world block traces.”

2.3.21 Companies producing flash memories

SanDisk founded in 1988 by Eli Harari, Sanjay Mehrotra and Jack Yuan, starting producing a 2.5 inch flash based SSD for IBM with 20 MB capacity priced at about \$1000. Sandisk which in early 2015 was the third largest producer of flash storage was acquired by *Western Digital Corp.* - WDC in May 2016.

Micron developed the QLC NAND technology that can store 4 bits per cell and is geared for an environment where data needs to be read quickly, but not rewritten repeatedly. TLC which stores three bits per cell, MLC which stores two bits per cell, and SLC which stores one bit per cell.

<https://www.micron.com/products/advanced-solutions/qlc-nand>

Intel's new drive combines Intel Optane memory H10 and Intel QLC 3D NAND storage to deliver with mixed random read/write speeds at low queue depths. The Intel Optane memory H10 is an ideal OS drive delivering fast boot and application launch and multitasking.

The intelligent Intel *Rapid Storage Technology* - *RST* driver working behind the scenes recognizes and remembers content needed for important and frequent tasks, adapting to user behavior.

<https://www.intel.com/content/www/us/en/architecture-and-technology/rapid-storage-technology.html>

It was announced on 10/19/2020 that Intel is selling most of its NAND flash memory business, with the exception of its fast Optane technology to S. Korea's **SK Hynix** for \$9 billion.

<https://www.engadget.com/intel-flash-sk-hynix-optane-022355772.html>

Violin Memory founded in 2005 is associated with Toshiba.

https://en.wikipedia.org/wiki/Violin_Memory

Violin does not use SSDs, but instead a proprietary *Flash Fabric Architecture* - *FFA*, which is a mesh of NAND flash dies, modules and a proprietary switched architecture for fault tolerance. A good writeup on Violin RAID is as follows.

<http://storagegaga.com/tag/vraid/>

Interconnections in a Violin memory are shown below.

<http://storagegaga.com/wp-content/uploads/2012/03/Violin-vXM-Architecture1.gif>

Nimble Storage was founded in January 2008 by Varun Mehta and Umesh Maheshwari. It has been an HP subsidiary since 2017.

https://en.wikipedia.org/wiki/Nimble_Storage

<https://www.hpe.com/us/en/storage/nimble.html>

IBM acquired **Texas Memory Systems - TMS** in 2012.

https://en.wikipedia.org/wiki/IBM_FlashSystem

DSSD flash company was acquired by **EMC** in 2014 for \$1 B. The coding scheme used by the 2d flash memory organization is described in Chapter 5.

<https://corporate.delltechnologies.com/en-us/newsroom/announcements/2014/05/20140505-06.htm>

In September 2016 Dell closed \$60 B merger with EMC.

<https://www.wsj.com/articles/dell-closes-60-billion-merger-with-emc-1473252540>

In December 2016 Bill Moore who cofounded DSSD with Jeff Bonwick (Bonwick, 2016) left Dell/EMC and the company stopped shipping DSSD hardware.

https://www.theregister.co.uk/2016/12/09/dssd_president_quits_dell_emc/

List of flash memory controller manufacturers in Table 2.11.

https://en.wikipedia.org/wiki/List_of_flash_memory_controller_manufacturers

CompactFlash - *CF* is a flash memory mass storage device used mainly in portable electronic devices format was specified by SanDisk which manufactured in 1994.

<https://en.wikipedia.org/wiki/CompactFlash>

Secure Digital - *SD* is a proprietary non-volatile memory card format developed by the SD Association (SDA) for use in portable devices.

https://en.wikipedia.org/wiki/SD_card

2.3.22 Elevating commodity storage with the SALSA host translation layer

SoftwAre Log Structured Array - *SALSA* is a translation layer that executes on the host that supports a wide range of single- and multi-device optimizations allowing unmodified applications to better utilize commodity storage. Because Salsa is implemented in software it can adapt to specific workloads as developed by a group at IBM/ZRL led by Evangelos Eleftheriou (Ioannou et al., 2018). SALSA demonstrate significant benefits using microbenchmarks and case studies based on three applications: MySQL, the Swift object store, and a video server.

SALSA is a *Software Defined Storage* - *SDS* system that acts as the *Translation Layer* - *TL* of a log-structured architecture to manage a shared pool (Rosenblum and Ousterhout, 1992), (Menon, 1995). It can be configured to use workload-specific policies for each application using the storage. SALSA can be used to run unmodified applications by exposing a Linux block device that can be either used directly, or mounted by a traditional Linux file system. SALSA contributes a novel host TL architecture and supports different media and allows optimizing for different objectives. Specifically:

- SALSA achieves substantial performance and durability benefits by implementing the TL on the host for single- and multi-device setups. For MySQL databases on SSDs SALSA outperforms the raw device by 1.7X on one SSD and by 35.4X on a software RAID5 array.
- SALSA makes efficient use of storage by allowing application-specific policies. A SALSA policy tailored to the Swift object store on SMRs that outperforms the raw device by up to a factor of 6.3X.

TABLE 2.11 List of flash memory controller manufacturers, CD=Compact Flash, SD=SD card.

Name	Based in	Status	NVMe contr.	SATA contr.	CF/SD contr.
Fusion-io SanDisk/WDC	US	Captive	Yes	Yes	No
Greenliant Syst.	US	Independent	No	Yes	Yes
Hyperstone	Germany	Independent	No	Yes	Yes
Indilinx Toshiba/Koxia	S. Korea	Captive	Yes	Yes	No
Intel	US	Captive	Yes	Yes	No
IntelliProp	US	Independent	Yes	Yes	No
InnoGrit	China	Independent	Yes	Yes	Yes
JMicron/Maxiotek	Taiwan	Independent	Yes	Yes	Yes
Marvell	US	Independent	Yes	Yes	No
Mio	China	Independent	Yes	Yes	No
Novachips	S. Korea	Independent	Yes	Yes	No
Phison	Taiwan	Independent	Yes	Yes	Yes
Realtek	Taiwan	Independent	Yes	Yes	N/A
Samsung	S. Korea	Captive	Yes	Yes	No
SandForce/Seagate	US	Captive	Yes	Yes	No
Silicon Motion	Taiwan	Independent	Yes	Yes	Yes
Starblaze	China	Independent	Yes	No	No
STEC/HGST/WDC	US	Captive	Yes	Yes	No
Kioxia/Toshiba	Japan	Captive	Yes	Yes	Yes
VIA Tech.	Taiwan	Captive	Yes	Yes	No
FADU	S. Korea	Captive	Yes	No	No
Kraftway	Russia	Independent	Yes	No	No

The OpenStack Object Store project, known as Swift, offers cloud storage software so that you can store and retrieve lots of data with a simple API. It's built for scale and optimized for durability, availability, and concurrency across the entire data set. Swift is ideal for storing unstructured data that can grow without bound.

<https://wiki.openstack.org/wiki/Swift>

<https://wiki.openstack.org/wiki/Swift>

- SALSA decouples space management from storage policy enabling it to accommodate different applications, each with its own policy, using the same storage pool. This allows running MySQL and Swift on the same storage with high performance and a low overhead.
- SALSA embraces storage diversity by supporting multiple types of devices. We present how we combine SMRs and SSDs to speedup file retrieval for a video server workload (where adding an SSD improves read performance by 19.1X), without modifying the application.

Three principal design choices made by SALSA are as follows:

1. It is log-structured, which allows it to deal with storage idiosyncrasies. By only writing big sequential segments, SALSA renders the drive's *Garbage Collection* - GC irrelevant, as its task becomes trivial. When space runs out, SALSA does its own GC.
2. SALSA supports multiple storage types and can combine them to build hybrid systems.
3. SALSA follows a modular design so that it can be used as a framework for implementing a variety of policies and algorithms, enabling adaptability to different requirements. From the perspective of the user, SALSA exposes block devices that can be used transparently by applications, e.g., by running a database directly on the device, or by creating a filesystem and running an application on top.

An important benefit of SALSA is that it does not require any application modifications. There are two main components in SALSA: the *Storage Capacity Manager - SCM*, which is responsible for managing the underlying storage, and one or more controllers that operate on the SCM. SCM is a common substrate that implements storage provisioning to controllers, GC, and other common functions. Controllers are responsible for implementing the storage policy, performing I/O, and mapping the logical (application) space to the physical space (L2P).

Salsa's SCM identifies the following devices *NonVolatile Memory - NVM*, Flash, HDD, SMR conventional and sequential. While SALSA supports multiple front-ends in the case of Linux kernel front-end each controller exposes a block device, which maintain a mapping between user-visible *Logical Block Addresses - LBAs* and backend *Physical Block Addresses - PBAs*. LSA controllers map LBAs to PBAs, with a flat array of 32 or 64 bits for each entry. Larger blocks (or pages) require less space for the table, but lead to I/O amplification for writes smaller than the page size, e.g., read-modify-write operations for writes.

For SSDs, the default pagesize is 4 KB, allowing us to address 16 TB (64 ZB for 64 bit entries) storage. For SMR drives, the default page size is 64 KB. Note that the page size has to be a multiple of the SCM grain size, in order to maintain interoperability with the GC and allocators. The mapping table is maintained in-memory, with an overhead of 4B per LBA, e.g., 1 GB DRAM per 1 TiB of storage for 4 KB pages, 512 MiB for 8 KiB. A back-pointer table of PBA-to-LBA mappings is maintained per segment for GC and restore operations, and it is either always in-memory or is constructed on-demand by scanning the LBA-to-PBA table, based on a run-time configuration parameter.

Accesses and updates to the mapping table are done in a thread-safe lock-free manner using compare-and-swap. A read operation will typically read the table, perform a read I/O operation to fetch the necessary data, and return them to the user. A write operation will allocate new space, perform a write I/O operation to write user data to this space, and update the table entry. A relocation operation on a PBA will read the PBA-to-LBA back-pointer, check that the LBA stills maps to the PBA in question, read the data, allocate new space, write the valid data to a new location, and update the table only if the LBA still maps to the relocated PBA.

For sequential segments of host-managed SMRs we force the allocated pages to be written sequentially to the drive, to avoid drive errors. We do so via a thread that ensures that all writes to these segments happen in-order. This is not required for other storage types (e.g., SSDs), and we do not use the I/O thread for them. Average raw throughput versus Salsa is given in Table 2.12.

TABLE 2.12 Average throughput (MB/s) and standard deviation for two random workloads on an SSD: 100% writes and 80/20% reads/writes. Block size is 4 KB.

Throughput	W:100	R:80%	W:20%
Raw	15.9±0.2	50.9±0.9	12.6 ±0.2
SALSA	37.7±0.9	72.5±6.2	18.1±1.5

2.3.23 Flash SSD versus magnetic HDD pricing

Flash SSDs vs HDDs are compared in following sources.

<https://blocksandfiles.com/2021/01/25/wikibon-ssds-vs-hard-drives-wrights-law/>

<https://wikibon.com/qlc-flash-hamrs-hdd/>

It is argued that the crossover between SSDs and HDDs prices can be accelerated by Wright's law, also discussed in Chapter 1. The Wikibon report has applied Wright's Law to NAND and HDD production, which predicts cost-saving efficiencies will be made in line with increased production: consumer and enterprises SSD NAND flash products, with its beneficial effects on pricing, conversely, the scope for production-related efficiencies for HDDs falls as fewer disk drives are produced. The outcome is that SSD \$/TB will drop faster than HDD \$/TB.

Wikibon projects that flash consumer SSDs become cheaper than HDDs on \$/TB basis by 2026 and that innovative storage and processor architectures will accelerate the migration from HDD to NAND flash and tape using consumer-grade flash. Flash is lower cost over 5 and 10 years for almost all file-based workloads than HDD, when storage management and space are factored in. Flash has already overtaken HDDs in total storage petabytes shipped. Total flash 435 ExaBytes - EB in 2020, compared to 310 EB for HDD.

Members of the HDD preservation society say HDDs will retain their current \$/TB advantage by increasing capacity through HAMR and MAMR technology, which will enable a disk platter to have much smaller bits and more tracks on the platter surfaces. That drives up areal density and so lowers the cost/TB. According to this viewpoint, SSDs will retain

their 5X or greater cost premium despite lowering their own cost/TB because HDD capacity is getting less expensive at the same or a greater rate. It is argued that there is simply isn't enough flash manufacturing capacity to match the millions of petabytes of disk drive manufacturing capacity that exists today.

"Clearly SSD price/GB will get closer to HDD price/GB over time. However, the raw number of *ExaBytes* - *EB* needed for a "wholesale switch" from nearline HDD to SSD is far too large for the NAND flash industry to contemplate. The capital investment needed to generate the EB required is prohibitively expensive."

HDD and Flash Storage pricing projections are provided in Table 2.13.

TABLE 2.13 Historical HDD & SSD Prices 2013:20.

	2013	2014	2015	2016	2017	2018	2019	2020
HDD \$/TB	60	53	45	35	30	26.8	23.9	22.2
SSD \$/TB	2,220	1,293	840	538	493	378	195	128
HDD Yr %		-12	-15	-22	-14	-11	-11	-7.2
SSD Yr %		-42	-35	-36	-8	-23	-48	-34

Projected HDD and Flash Storage pricing projections up to 2020 are also provided. Flash SSD is compared with consumer and enterprise disks in Table 2.14,

TABLE 2.14 Flash SSD versus consumer and enterprise disks.

Storage technology	Consumer Disk	Enterprise Disk	SSD
\$/GB	0.05	0.50	1.00
Capacity	3-4 TB	0.2-2 TB	500 GB
Sequential R/W	50 MB/s	200 MB/s	400 MB/s
IOPS (4K)	80-120	200	11.5-40K
Read latency	12 ms	6ms	0.15ms
Write latency	13 ms	6ms	0.05ms
Worst case	15 ms	10 ms	100ms

Largest NAND flash memory manufacturers as of the first quarter of 2019 are as follows:

https://en.wikipedia.org/wiki/List_of_flash_memory_controller_manufacturers

1. Samsung - 29.9%
2. Kioxia (formerly Toshiba) - 20.2%
3. Micron Technology (Crucial) - 16.5%
4. Western Digital (SanDisk) - 14.9%
5. SK Hynix - 9.5%
6. Intel - 8.5%

<https://www.statista.com/statistics/275886/market-share-held-by-leading-nand-flash-memory-manufacturers-worldwide/>

2.3.24 Pure Storage design of Purity

Pure Storage was founded by James Colgrove in 2009 and came out of stealth mode in August 2012.

<http://www.purestorage.com>

https://en.wikipedia.org/wiki/Pure_Storage

Colgrove was responsible for a RAID5 disk array with Dieter Gawlick at Amdahl Corp., which is discussed in Chapter 5. The idea to found Pure Storage to Colgrove following a stint at Sutter Hill Ventures, where he realized the potential for flash memories to build disk arrays. https://en.wikipedia.org/wiki/Sutter_Hill_Ventures

Colgrove started as a systems programmer at AT&T, moved to Amdahl, Symantec, and Veritas, before Purestore.

Pure Storage is a leader in Gartner Magic Quadrant. It competes with its NVMe-based products, FlashArray//X and the lower-cost, lower-performance FlashArray//C. Pure Storage is completely channel-driven

<https://www.saleshacker.com/channel-sales-direct-sales-strategy/>

and operates 2/3 in North America and 1/3 internationally.

It caters to the enterprise market and across all verticals, offering its modern data experience based on simplicity, flexibility and *AI for IT Operations* - *AIOps* capabilities.

https://en.wikipedia.org/wiki/Artificial_Intelligence_for_IT_Operations

Cloud Block Store offers a cloud-native storage solution for workloads deployed in AWS.

IT Operations Analytics - *ITOA* method retrieves, analyzes, and reports data for IT operations. *ITOA* may apply big data analytics to large datasets to produce business insights. In 2017 Gartner predicted that 15% of enterprises will use IT operations analytics technologies.

https://en.wikipedia.org/wiki/IT_operations_analytics

Pure Storage has standardized on its internally developed NVMe drive technology supported by nondisruptive migrations and customer-friendly business programs. Customers report high satisfaction due to ease of use and flexible cloud-like consumption programs. The FlashArray//C is a QLC 3D NAND-capable array offering more aggressive price points and higher capacities compared to FlashArray//X, and is offered as an alternative to HDD-based and hybrid storage arrays.

Ethan Miller (U. Calif. Santa Cruz) is a part-time researcher at Pure Storage.

<https://www.linkedin.com/in/ethanmiller/>

The following presentation describes the organization and operation of Pure Storage's Purity (Miller, 2014).

<https://storageconference.us/2014/Presentations/Miller.pdf>

Pure Storage can handle multiple databases such as SQL server, MongoDB and Oracle's *Real Application Cluster* - *RAC*, which allows multiple computers to run Oracle RDBMS software simultaneously, while accessing a single database.

https://en.wikipedia.org/wiki/Oracle_RAC

In *Real Application Cluster* - *RAC*, the write-txn must take ownership of the relevant area of the database: typically, this involves a request across the cluster interconnection network to transfer the data-block ownership from another node to the one wishing to do the write. This takes a few to tens of milliseconds compared to single database-node using in-memory operations. For many types of applications, the time spent coordinating block access across systems is low relative to the many operations on the system, and *RAC* will scale comparably to a single system. Moreover, high read-transactional databases as in data-warehousing work very well under *RAC*, since there is no need for ownership-transfers. Resource mastering in the cluster can be obtained in a maximum of three hops (owner-master-requestor). This makes Oracle *RAC* horizontally scalable with many nodes.

Purity takes advantage of recent developments to exploit flash memories, such as data deduplication and compression achieving a 3-10-fold reduction in data volume. Pure Storage adopts the technique proposed in (Gupta et al., 2009) that in addition to temporal and spatial locality certain values are accessed preferentially. Given evidence they design a *Content Addressable Storage* - *CAS* the *CA-SSD* design employs enhancements to the *FTL* via minimal additional hardware. *CA-SSD* is able to reduce average response times by about 59-84% compared to traditional SSDs. Various aspects of Purity are compared to a disk array in Table 2.15.

TABLE 2.15 Comparison of Purity and a disk array, Table 1 in (Colgrove et al., 2015).

Metric	Purity	Disk Array	Improvement
Peak IOPS@32KB	200K	65K	3.08X
Latency	1 msec	5 ms	5X
Usable capacity	40 TB	15 TB	1.6X
RACK Units - RUs	8	28	3.5X
Installation	4 hrs	40 hrs	10X
Power	1240 W	3500 W	2.82X
Annual Power Cost	\$13,034	\$36,792	2.82X
\$/GB	\$5	\$18	3.6X
IOPS/RU	25,000	2321	10.7X
IOPS/W	161	1.6	8.6X
IOPS/\$	1	0.144	6.9X

Purity stores data in segments each of which is striped across multiple SSDs, using a high-performance software implementation of Reed-Solomon coding using SIMD instructions to ensure that no data is lost even if two SSDs fail. In addition, Purity can leverage the parity pages within each SSD; *FTL* can quickly recover a single corrupted page without the need to

TABLE 2.16 Three different memory and storage products built on Intel Optane technology.

	Intel Optane Persistent Memory	Intel Optane DC SSD with Intel Memory Drive Technology	Intel Optane DC SSD
Interface	Memory channel	PCIe bus	PCIe bus
Capacity	512 GB per DIMM	1.5 TB per SSD	1.5 TB per SSD
Intel Platform	2nd gen Xeon proc.	any	any
Function	App Direct Mode Memory mode, volatile mem Persistent mem.	Volatile memory	Persistent mem.
Form factor	DIMM	U.2, M.2, AIC	U.2, M.2, AIC
OS	Windows, Linux VMware ESXi	Linux	Linux

read data from the other drives in the segment. A segment is composed of one *Allocation Unit - AU* from each of the drives across which it is striped; the specific drives may be different for each segment and are chosen when the segment is written. AU which are 8 MB in current systems, are the minimum allocation granularity for an SSD. Within a segment, each SSD is written atomically in write units, which are currently 1 MB. A horizontal stripe of write units across the segment, called a segio, accumulates compressed user data from the front, and accumulates log records from the back. When the two sections meet, the segio is completed and marked for flush to SSD. Writing log entries is less expensive than rewriting entire data sections.

2.3.25 Intel/Micron 3D_XPoint Optane Memory

3D-Xpoint is an NVM technology developed jointly by Intel and Micron Technology.

https://en.wikipedia.org/wiki/3D_XPoint

The Intel Optane Technology announced in 2015 and introduced in 2019 is a unique combination of Intel Optane memory media with Intel's advanced system memory controller, interface hardware, and software IP. Optane holds more data than DRAM, it is faster than flash and magnetic hard disks and it is non-volatile.

<https://www.intel.com/content/www/us/en/products/docs/storage/optane-technology-brief.html>

https://www.theregister.co.uk/2019/09/26/intel_optane_data_center_performance/

Intel claims that Optane is the first major memory and storage breakthrough in 25 years, which combines low latency, high endurance, QoS, and high throughput that allows the creation of solutions to remove data bottlenecks, and unleash CPU utilization. With Intel Optane technology, data centers can deploy bigger and more affordable datasets to gain new insights from large memory pools. Three different memory and storage products built on Intel Optane technology are given in Table 2.16.

Top ten reasons to deploy Optane technology in the data center according to Intel are as follows:

<https://www.content.shi.com/SHIcom/ContentAttachmentImages/SharedResources/Pdfs/Intel/intel-032320-optane-top-10-reasons.pdf>

- 1. Access Data Faster for Greater Insights.** Organizations need faster, more efficient, and more affordable ways to consume, process, and extract useful insights from high volumes of data they encounter. The amount of data is not only growing, but it is constantly evolving. Access to much more data closer to the CPU, which means faster computing of real-time analytics, txns, reservations, and other use cases that require predictably fast read-response times.
- 2. Boosting the Bottom Line.** More work gets done with the same servers when compared to alternative solutions. The savings can be used to reduce costs or expand capabilities and services.
 - Bigger, more affordable memory.
 - Cost-optimized for storage consolidation.
 - More scale per server.
 - Faster insight and larger memory pools.

- High endurance.
- Improve overall system with balanced cost, capacity, and performance.

3. Innovate with Flexible Architecture. Intel's products available to data center architects are as follows:

- Intel Xeon Scalable processors optimize interconnectivity with a focus on speed without compromising data security.
- Intel Optane DC Persistent Memory is connected to the memory bus, and represents a new class of memory and storage technology architected to extract further value from data. It can be used with the Intel Optane DC SSD P4800X as a fast-cache storage tier.
- Intel Optane DC SSDs help to eliminate data center storage bottlenecks and allow bigger, more SSD DC P4800X affordable data sets. They can accelerate applications, reduce txn costs for latency-sensitive workloads, and improve overall data center efficiency and performance.
- Intel QLC 3D NAND SSDs are reducing the cost for NAND SSD.

4. Bridge the Memory and Storage Gap.

- (a) Today's storage technologies leave a technology gap in data storage tiers. Memory is great from a performance standpoint, since it is close to the processor on the memory bus, but it's expensive and the ability to scale capacity for larger working sets is limited.
- (b) NAND brings capacity and is inexpensive relative to DRAM, but it is not on the memory bus and therefore does not approach the performance of memory.
- (c) This leaves data center architecture gaps when trying to balance capacity, performance, and cost considerations. Intel Optane technology and Intel QLC 3D NAND SSDs enable system architects to break through the bottlenecks in the working data caching tier, while cost-effectively increasing the capacity for data storage. These unique technologies bridge the gaps between localized hot data (near the CPU) and larger capacity data.

5. Accelerate Response and Performance. Comparison is between Optane SSD DC P4800X vs 3D NAND SSDs.

Breakthrough performance Higher throughput faster movement of data. Reads and writes at bit level, unlike pages and blocks for NAND. This means no garbage collection for faster write times. Performance does not decay under stress. Up to 6X faster, queue depth 1, 4K 70/30 R/W IOPS.

Predictably fast service. High QoS on mixed workload benchmarks. In an environment of fast growing data and demanding requirements, data centers must deploy solutions that enable predictably fast service. Up to 60x better, 63x faster, 99% QoS.

Responsive under load. With NAND-based SSDs, random write operations can add significant delay to the read operations. Intel Optane technology maintains consistent response times regardless of the write throughput applied to the drive. Provides fast service under any load even at low queue depth where most applications generate storage workloads. Up to 63X faster response time.

6. Cache Data Efficiently. Fast storage or cache refers to the tiering and layering, which enable a better memory-to-storage hierarchy. Intel Optane technology's combination of extremely low and predictable latency and high endurance allows it to function much more efficiently as a caching device than NAND-based solutions.

A system built on Intel Xeon Scalable processors, using Intel Optane SSD DC P4800X drives for the caching layer delivered up to 3X better price/performance levels than previous generation systems and storage media.

7. Expand memory: Intel Optane DC persistent memory is an innovative new tier in the memory hierarchy that is applicable to workloads across cloud, in-memory computing, and storage. This new application of Intel Optane technology in a DIMM form factor allows large amounts of data to be moved closer to the CPU - providing fast access, processing, and analysis in real time comparable to DRAM at low latencies. The DIMM also offers higher capacity and data persistence, allowing data to remain in memory even when power cycles occur. The flexibility of Intel Optane DC persistent memory enables system architects and designers to accelerate industry-trending In-Memory Databases, and support more Virtual Machines and larger memory for each.

8. Handle the Breadth of Storage Workloads. With Intel Optane SSD DC P4800X, each server can do more across applications. This data center SSD is ideal for working data and real-time data stored in large volumes; highly random storage bound applications; and random workloads at low queue depths, which is where the majority of activity occurs in real-world scenarios. These include: Storage, Cloud and VMS, Database, AI/Analytics. HPC, Communications.

9. Fully Connected Platform. Data defines the future, and Intel's unique position as a technology leader and breadth of capabilities can help customers deliver something new, or reimagine what they do, only better. This takes memory and platform innovation. Intel's end-to-end integrated architecture optimizes the Intel Xeon processor, Intel Optane technology and Intel 3D NAND technology to create an efficient data center that can move data faster, store more data, and process everything from cloud to the edge. Platform-connected capabilities link the compute and storage pools to efficiently manage storage at scale, accelerate applications, and simplify systems.

10. Experience the Intel Difference. Intel, building on 50 years of experience, is redesigning the fabric of how new systems are architected, building storage directly into the DNA of new system designs. The combination of Intel Optane DC Persistent Memory with the performance-optimized Intel Optane SSDs and next-generation cost-optimized Intel 3D NAND SSDs with QLC technology is redefining the storage hierarchy.

On 12/16/2020 Intel announced a new wave of Optane and 3D NAND SSDs.

<https://www.anandtech.com/show/16318/intel-announces-new-wave-of-optane-and-3d-nand-ssds>

A recent study of sensitivity of Optane performance for different workloads is reported in (Wu et al., 2019). The authors analyze the Optane SSD and formalize an “unwritten contract”, Violating this contract can result in 11x worse read latency and limited throughput (only 20% of peak bandwidth) regardless of parallelism. Rule 1 is to obtain low latency, Optane SSD users should issue small requests and maintain a small number of outstanding IOs.

According to 1/17/2021 Intel has killed off Optane high speed SSDs for the consumer market.

<https://www.techradar.com/news/intel-has-killed-off-its-speedy-optane-ssds-so-if-you-want-one-buy-it-quickly>

Intel will instead focus on the new Optane Memory H20 with SSD for the client market segment.

2.3.26 Processing In Memory - PIM

PIM is motivated by the fact that the energy consumed by data movement exceeds the energy required by computation (Ghose et al., 2018, 2019). It has the advantage that some of the computational capability is moved from the CPU to main memory (Ghose et al., 2018, 2019). Two proposed mechanisms in support of PIM are as follows:

1. *In Memory PoInter Chasing Accelerator - IMPICA* performs virtual-to-physical address translation with no need for a TLB - Translation Lookaside Buffer.
2. LazyPIM maintains cache coherence between PIM processing logic and CPU cores without sending coherence requests for every memory access.

In the era of AI and cognitive computing, there is a pressing need for energy-efficient computing systems for highly data-centric AI related applications. It is becoming increasingly clear that to build efficient cognitive computers require a transition to novel architectures where memory and processing are better collocated. Computational memory is one such approach where the physical attributes and state dynamics of memory devices are exploited to perform certain computational tasks in place with very high areal and energy efficiency. PCM devices based on materials such as $\text{Ge}_2\text{Sb}_2\text{Te}_5$ are appealing for computational memory owing to their multi-level storage capability and potential scalability.

Presented by researchers at IBM/ZRL in (Gallo et al., 2019) is a brief overview of the work towards enabling energy-efficient brain-inspired computing using PIM. A tutorial on brain-inspired computing using PCM is as follows:

<https://aip.scitation.org/doi/full/10.1063/1.5042413>

2.3.27 Universal memory technology - UltraRAM

The Universal memory technology - ULTRARAM was developed at U. Leicester, UK (Tizno et al., 2019).

https://www.theregister.co.uk/2020/01/27/universal_memory/

It is room-temperature, low-voltage, non-volatile, with compound-semiconductor memory cells.

“Whilst the different forms of conventional (charge-based) memories are well suited to their individual roles in computers and other electronic devices, flaws in their properties mean that intensive research into alternative, or emerging, memories continues. In particular, the goal of simultaneously achieving the contradictory requirements of non-volatility and fast, low-voltage (low-energy) switching has proved challenging. Here, we report an oxide-free, floating-gate memory cell based on III-V semiconductor heterostructures with a junctionless channel and non-destructive read of the stored data. Non-volatile data retention of at least 104 seconds in combination with switching at ≤ 2.6 ts is achieved by use of the extraordinary 2.1 eV conduction band offsets of InAs/AlSb and a triple-barrier resonant tunneling structure.

The conduction band is the band of electron orbitals that electrons can bounce up into from the valence band when energized. At the point when the electrons are in these orbitals, they have enough energy to move freely in the material.

This movement of electrons makes an electric current flow.

<https://byjus.com/jee/conduction-band/>

The combination of low-voltage operation and small capacitance implies intrinsic switching energy per unit area that is 100 and 1000 times smaller than DRAM and Flash respectively. The device may thus be considered as a new emerging memory with considerable potential.”

2.3.28 Racetrack memory

Racetrack or *Domain Wall Memory* - DWM was conceived by Stuart Parkin at IBM/ARC in 2002.

<https://www.ibm.com/ibm/history/ibm100/us/en/icons/racetrack/>

https://en.wikipedia.org/wiki/Racetrack_memory

Research on racetrack memories has been renewed at *New York U. - NYU* on 5/11/20.

https://en.wikipedia.org/wiki/Racetrack_memory

<https://www.popularmechanics.com/technology/design/a32388892/racetrack-memory-computer-data-storage/>

“While today’s hard drives use a motor to move a spindle across the platters, reading and writing data, racetrack memory does the opposite. The material stays in place as the data, itself, moves across the substrate - no mechanical parts required. Here’s how it works: A tiny magnetic quasiparticle, called a skyrmion, carries the data. External stimuli, like the pulse of a current, can move the skyrmions, which spin as if curled up in a ball. The resulting balls of skyrmions represent pieces of data, and the computer can move them quickly in sync with electrical pulses. Because skyrmions are small, move at high speed, and require little energy, it makes the computing process faster and more energy efficient, while also storing more data in a given amount of space than other techniques can.”

A technical discussion appears in (Tomasello et al., 2014).

<https://www.nature.com/articles/srep06784>

2.3.29 Optical storage

Optical disks date back to Alexander Graham Bell in 1884. Four generations of modern optical disks are as follows:

https://en.wikipedia.org/wiki/Optical_disc

1. *Compact Disks* - CDs for audio with 700 MB on 12 cm disks.
2. *Digital Versatile Disks* - DVDs for movies with 4.7 GB on 12 cm disks.
3. Blu-ray discs with up to 400 GB in an experimental setting.
4. One TB optical disks for archival storage.

<https://en.wikipedia.org/wiki/Blu-ray>

Optical disc technologies like Bluray store and read data by making changes to a thin film made of layers of polycarbonate and thin films glued together. This medium has a long, but limited lifetime.

https://en.wikipedia.org/wiki/Optical_disc

A laser encodes data by creating alternating marks and spaces on the surface of the thin film. High capacity optical discs use multiple thin film layers sandwiched together. The limit on density is that each layer needs to have sufficient transmittance to allow the laser beam to write and read the deeper layers. This is about 4-8 layers, which limits the maximum capacity of a disc to less than 1 TB.

2.3.30 Holographic memory

Holographic storage consists of two interfering laser beams, where one beam is the address line and the other encodes data pages to be written inside a block of photorefractive material.

https://en.wikipedia.org/wiki/Holographic_data_storage

Data is written and read in multi-megabyte pages. The cost per GB for holographic storage is not competitive with other storage technologies, which is the reason it is not deployed commercially.

https://en.wikipedia.org/wiki/Holographic_data_storage

2.3.31 M-DISC and storage longevity

Table 2.17 dealing with data longevity is based on

<https://blog.storagecraft.com/data-storage-lifespan/>

M-DISC is an archival-quality storage solution that preserves photos, videos, music, and documents for 1,000 years or more. Unlike hard drives, flash drives, and other writable media, that can lose data, M-DISC has been designed to protect your information from degradation and loss for centuries.

<http://mdisc.com/>

Lifespans for miscellaneous storage media are given in Table 2.17.

TABLE 2.17 Lifespans for miscellaneous storage media.

Magnetic data (tapes)	Up to 10 years
Nintendo cartridge	10-20 years (equipped with battery)
Floppy disk	10-20 years
CDs and DVDs	5-10 unrecorded, 2-5 recorded
Blu-Ray	Not certain, probably over 2-5 recorded
M-Disc	1,000 years (optical storage, theoretically)
Hard disk	3-5 years
Flash storage	5-10 years or more (depends on write cycles)

2.3.32 Persistent and NonVolatile Memory - NVM

Comparison of performance characteristics of NVM technologies relative to DRAM and NAND Flash is given in Table 2.18.

UCSD has a *Non-Volatile Systems Lab* where Steven Swanson leads research on this topic.

<https://cmrr.ucsd.edu/research/faculty-profiles/swanson.html>

Joy Arulraj (GaTech) and Andrew Pavlo (CMU) have coauthored a book on non-volatile memory databases (Arulraj and Pavlo, 2019).

<https://db.cs.cmu.edu/projects/nvm/>

TABLE 2.18 Comparison of performance characteristics of NVM technologies relative to DRAM and NAND Flash.

	DRAM	NAND Flash	Memristor	PCM
Byte-Addressable	Yes	No	Yes	Yes
Capacity	1X	4X	2-4X	2-4X
Latency	1X	400X	3-5X	3-5X
Endurance	10^{16}	10^4	10^6	$10^6 - 10^8$

The Prolegomenon (a critical or discursive introduction to a book) is an OLTP database for NVM storage (DeBrabant et al., 2014). The authors explore NVM-only and NVM+ DRAM architectures and for each architecture memory-oriented and disk-oriented OLTP DBMSs. Evaluation of software systems using NVM was challenging in 2014 due to lack of actual hardware. In this study, we use a NVM hardware emulator developed by Intel Labs.

In the NVM-only architecture the DBMS uses NVM exclusively for its storage. A memory-oriented and disk-oriented DBMS both run entirely on NVM storage. For the former the authors used the H-Store DBMS (Kallman et al., 2008) and for the latter MySQL with the InnoDB storage engine.

<https://en.wikipedia.org/wiki/InnoDB>

In a disk-oriented DBMS, the system's internal data is divided into in-memory and disk-resident components. The DBMS maintains a buffer pool in memory to store copies of pages retrieved from the database's primary storage location on disk. The emulator's *Non-Uniform Memory Access - NUMA* interface stores the DBMS's buffer pool in the byte-addressable NVM storage, while its data files and logs are stored in NVM through the *Persistent Memory File System - PMFS* interface.

Running the YSCB and TPC-C benchmarks shows that memory-oriented systems are better-suited to take advantage of NVM and outperform their disk-oriented counterparts. In both NVM-only and NVM + DRAM architectures the throughput of the memory-oriented systems decreases as workload skew is decreased, while the throughput of the disk-oriented architectures increases as workload skew is decreased. It is concluded that neither system is ideally suited for NVM and a new system combining disk-oriented and memory-oriented systems with a lightweight recovery scheme designed to utilize the non-volatility of NVM is required.

2.3.33 Glass as a new storage medium

The ultimate goal for cloud storage is very low cost storage, low access latency, and high throughput. Microsoft Research at Cambridge, UK and the OptoElectronics Research Center at U. Southampton, UK are exploring whether quartz glass as the mass storage in the cloud. Limitations of current storage systems are in three categories (Anderson et al., 2018):

Entanglement: Read and Write throughputs do not scale independently. The aggregate write throughput should support the data being ingested to the systems. The read throughput should support the read demand, which is a function of the volume of stored data.

Dropping Reliability: Data becomes less reliable over time in the case of magnetic disks. There is the possibility of whole device or media failures in the form of *Latent Sector Errors - LSEs* (Bairavasundaram et al., 2007), (Schroeder et al., 2010a) and Silent Errors (Hafner et al., 2008). Data longevity can be insured by copying data to new storage media every few years based on the *Annual Failure Rate - AFR*.

Constrained Workloads: Users partition their data between different tiers based on access characteristics and transfer the data between tiers if these requirements change. Required is a mass storage system that leaves the data in situ and is low cost, with acceptable performance across a wide range of access characteristics to free the users from having to make these choices.

Glass storage is referred to as *eternal memory*, since it can store 360 TB for billions of years.

https://en.wikipedia.org/wiki/5D_optical_data_storage

The five dimensions consist of the size, orientation and the three-dimensional position of the nanostructures.

Microsoft has produced a 2.5" *Write-Once, Read Many - WORM* disk which holds 75.6 TB.

<https://www.sciencetimes.com/articles/27396/20200921/petabyte-hard-disk-drive-glass.htm>

2.3.34 DNA based archival storage system

DeoxyriboNucleic Acid - DNA storage encodes and decodes binary data to and from synthesized strands.

https://en.wikipedia.org/wiki/DNA_digital_data_storage

DNA has enormous potential as a storage medium, because of its high storage density: one exabyte per cubic millimeter (EB/mm³, which is eight orders of magnitude denser than tape. DNA storage is also long lasting with a half-life of 500 years. Its practical use is currently severely limited because of its high cost and very slow read and write times.

According to (Bornholt et al., 2016) a DNA strand is a linear sequence of four nucleotides: adenine (A), cytosine (C), guanine (G), and thymine (T).

DNA sequences are conventionally represented starting with the 5' and 3' nucleotide ends. The interactions between different strands are predictable based on sequence. Two single strands can bind to each other and form a double helix if they are complementary: A in one strand aligns with T in the other, and likewise for C and G. The two strands in a double helix have opposite directionality (5' end binds to the other strand's 3' end), and thus the two sequences are the "reverse complement" of each other.

<https://profiles.nlm.nih.gov/spotlight/sc/feature/doublehelix>

Two strands do not need to be fully complementary to bind to one another. Such partial complementarity is useful for applications in DNA nanotechnology and other fields, but can also result in undesired "crosstalk" between sequences in complex reaction mixtures containing many sequences.

Given data the DNA synthesizer affects a DNA pool as shown in figures in (Bornholt et al., 2016) Random access is achieved by mapping a key to a pair of *polymerase chain reaction - PCR*. At write time, those primers are added to the strands. At read time, those same primers are used in PCR to amplify only the strands with the desired keys.

The obvious approach to store binary data in DNA is to encode the binary data in base 4, producing a string of $n/2$ base 4 digits from a string of n binary bits. The base-4 digits can then be mapped to DNA nucleotides, e.g., 0, 1, 2, 3 to A, C, G, T, respectively. The 8 bit binary string 01110001 maps to the base-4 string 1301(4), and then to the DNA sequence. However, DNA synthesis and sequencing processes are prone to a wide variety of errors: substitutions, insertions, and deletions of nucleotides, requiring a more careful encoding. The encoding of the word "Polya;" is given in Fig. 2.1,

The likelihood of some forms of error can be reduced by encoding binary data in base 3 instead of base 4, Each ternary digit maps to a DNA nucleotide based on a rotating code that avoids repeating the same nucleotide twice. This encoding avoids homopolymers - repetitions of the same nucleotide - that significantly increase the chance of sequencing errors. Because base 3 is not a multiple of base 2, mapping directly between the bases would be inefficient: 6 ternary digits ($3^6 = 729$) can store 9 bits of data ($2^9 = 512$), but waste 217 possible states. Instead, we use a Huffman code (Huffman, 1952) that maps each binary byte to either 5 or 6 ternary digits. For example, the Huffman code maps the binary string

-	P	o	l	y	a	;
Base 4 data	1100	1233	1230	1201	1201	0323
Base 3 Huffman code	12011	02110	02101	01112	01112	222031
DNA nucleotides	GCGAG	TGAGT	ATCGA	TGTCT	AGAGC	ATGTGA

FIGURE 2.1 Translating binary data to DNA nucleotides via a Huffman code.

01100001 to the base-3 string 01112₃. The rotating nucleotide encoding maps this string to the DNA sequence. The code maps more common ASCII characters to 5 digit strings offering compression benefits for textual data.

Another practical issue with representing data in DNA is that current synthesis technology does not scale beyond sequences of low hundreds of nucleotides. Data beyond the hundreds of bits therefore cannot be synthesized as a single strand of DNA. In addition, DNA pools do not offer spatial isolation and so a pool contains data for many different keys which are irrelevant to a single read operation. Isolating only the molecules of interest is non-trivial, and so existing DNA storage techniques generally sequence the entire solution, which incurs significant cost and time overheads.

A more robust design would provide redundancy at the data encoding level. This section discusses existing work on encodings for redundancy, highlights the reliability-density trade-off implicit in encoding design and presents a new encoding with similar reliability to and yet higher density than existing work.

Early work in DNA storage used encodings simpler than the one we describe above. Text can be translated to DNA by means of a simple ternary encoding: each of the 26 English characters and a space character maps to a sequence of three nucleotides drawn from A, C, and T (so exactly $3^3 = 27$ characters can be represented). This encoding suffers substantial overheads and poor reliability for longer messages.

The Goldman encoding splits the input DNA nucleotides into overlapping segments to provide fourfold redundancy for each segment (Goldman et al., 2013). Each window of four segments corresponds to a strand in the output encoding. It was used to successfully recover a 739 kB message. The encoding is used as a baseline because it is the most successful published DNA storage technique. It also offers a tunable level of redundancy by reducing the width of the segments and therefore repeating them more often in strands of the same length, e.g., if the overlapping segments were half as long they would be repeated in eight strands instead of four.

While the Goldman encoding provides high reliability, it also incurs significant overhead: each block in the input string is repeated four times. A simple new encoding that provides similar levels of redundancy to prior work, but with reduced overhead uses the *exclusive OR* - XOR operation at the strand level. The XOR $A \oplus B$ of the payloads A and B of two strands produces a new payload and so a new DNA strand. The address block of the new strand encodes the addresses of the input strands that were the inputs to the XOR; the high bit of the address is used to indicate whether a strand is an original payload or an XOR strand. This encoding provides its redundancy in a similar fashion to RAID5: any two of the three strands A , B , and $A \oplus B$ are sufficient to recover the third.

The reliability of this encoding is similar to that of Goldman. The authors successfully recovered objects from both encodings in a “wet lab” experiment.

<https://vivadifferences.com/wet-lab-vs-dry/>

However, the theoretical density of this encoding is much higher than Goldman, where in their encoding each nucleotide repeats (up to) four times, in the new encoding each nucleotide repeats an average of 1.5 times. In practice, the density difference is lower, due to the overheads of addressing and primers that are constant between the two encodings. Simulation results the XOR encoding to be twice as dense as that of Goldman, and for all practical DNA synthesis and sequencing technologies, it provides equivalent levels of reliability.

Recent work in approximate storage shows that many applications do not need high-precision storage for every data structure. For example, while the header data of a JPEG file is critical to successful decoding, small errors in the payload are tolerable at the cost of some decoding imprecision.

A key advantage of XOR encoding is that the level of redundancy is tunable at a per-block granularity. For critical data high redundancy is provided by pairing critical blocks with many other blocks: if A is critical, produce $A \oplus B$, $A \oplus C$, etc. On the other hand, for blocks that are less critical, redundancy can be further reduced: instead of including only two blocks in an XOR, we can include n , such that any $n - 1$ of the n blocks is sufficient to recover the last, at an average density overhead of $1/n$.

In addition to improving density, tunable redundancy has a significant effect on performance. Both DNA synthesis and sequencing are slower and more error-prone with larger datasets, and this error does not always grow linearly in size. It is often economically viable to synthesize smaller DNA pools with more accurate technology, while larger pools are out of reach. Tunable redundancy allows the storage system to optimize the balance between reliability and efficiency.

The encodings in (Bornholt et al., 2016) ignores special properties of DNA synthesis and sequencing that make choosing an optimal encoding more complex. In particular, errors in synthesis and sequencing are not uniform: they vary according to location within a strand, and different sequences of nucleotides are more susceptible to undesired reactions.

Synthesis error grows with the length of the strand being synthesized; nucleotides towards the end of the strand are more likely to be incorrect. Some of these errors are easily ignored, because they result in truncations, where the product strand is not of the correct length. Other errors are more insidious, with substitution errors being particularly common.

An improved version of our encoding would tolerate variable substitution errors by not aligning the strands directly. For example, rather than computing $A \oplus B$, we might instead compute $A \oplus B'$, where B' is B in reverse. Since nucleotides at the end of a strand are more error-prone, reversing one strand ensures that the average quality is (roughly) constant along the strand, so that each nucleotide has at least some redundancy information stored in a high-reliability position.

Discussed in (Dimopoulou et al., 2020) is a recent study on the use of DNA as a means of digital data storage to deal with the problem of storage capacity and durability based on the belief that DNA is appropriate for the efficient storage of digital data, which can outperform by far the existing storage means. In Table 2.19 DNA is shown as the bottom level of storage hierarchy. It is said DNA can store about 215 petabytes in a single gram of DNA, which is inline with the great chemical stability of the DNA molecules. The main challenge lies in the restrictions imposed by the biological procedures of DNA synthesis (writing) and DNA sequencing (reading), which are involved in the encoding process and introduce significant error in the encoded sequence. Furthermore, the high expenses for DNA synthesis and sequencing yield the need for finding solutions for reducing the total cost for storing data into DNA, i.e., \$7000 to synthesize 2 MB of data into DNA, and another \$2000 to read it by sequencing.

TABLE 2.19 DNA storage is shown as the bottom level of the storage hierarchy.

Technology	Access time	Durability
Flash	ms	5 years
HDD	10 ms	5 years
Tape	minutes	15-30 years
DNA storage	tens of hours	centuries

Several works tackle the problem of digital data storage onto DNA, but all of these approaches mainly try to transcode a bit stream onto a DNA sequence without taking into account the original input data characteristics. In earlier work the authors have presented a new quaternary (fourth order) encoder which is suitable for the proposed work-flow. A more detailed theoretical study of the efficiency and the advantages of our encoding algorithm comparing it with the existing state of the art encoding methods for DNA data storage.

DNA data storage can be described by the general workflow of five main sub-processes. The first part is the encoding of a sequence of digital symbols into a quaternary sequence of A, T, C and G. Then the encoded sequences need to be synthesized into DNA. As the DNA synthesis is error free under the constraint that the strands to be synthesized are no longer than 200 *nucleotides - nts*, the encoded data has to be cut into smaller chunks (oligos) using a formatting procedure in which special headers should also be added denoting the order of the information in the initial long sequence. Then the oligos are synthesized into DNA in vitro and stored into special capsules which prevent it from contacts with water and oxygen, to achieve reliable storage for hundreds of years.

Data is read back using special machines called *sequencers*. A main drawback of the sequencing part is the fact that it is prone to errors and there are some particular ill-cases that produce high error rates. These can be avoided by respecting the following rules in the encoding of the DNA sequence:

1. No homopolymer runs, i.e., the repetition of the same symbol > 3 times should be avoided
2. $\%A, T \leq \%G, C$, i.e., the content of G and C should not be greater than the percentage of As and Ts. A good ratio being 1.5 AT for 1 GC.
3. No repetitions of short patterns

By avoiding the above cases the sequencing will be more reliable but not error free. The addition of extra redundancy using PCR amplification (a specific biological process) that uses a special enzyme to create many copies of the synthesized strands, may improve the reliability of the sequencing.

<https://www.promega.com/resources/guides/nucleic-acid-analysis/pcr-amplification/>

Furthermore the sequencers themselves are adding extra redundancy as they are producing copies while reading the strands.

The sequencing results in many copies of the original synthesized oligos many of which will contain errors. Thus, there is a need to select which oligos are the less corrupted. Under the hypothesis that the most requested oligos will be the most representative ones, it is those sequences that will be selected for the decoding. In addition to this selection, error correction can further improve the quality of the reconstructed information. Finally using the inverse procedure of the one followed during the encoding, the initial information is retrieved.

The main idea is the creation of codewords from a set of duplets (pairs of symbols), which create an acceptable sequence when assembled in a longer strand. More precisely, the codewords are constructed by selecting elements from the following dictionaries:

$$C_1 = \{AT, AC, AG, TA, TC, TG, CA, CT, GA, GT\} \quad C_2 = \{A, T, C, G\}$$

Then, the code C is created by selecting elements from C_1 and C_2 . Codewords of an even length are constructed only by selecting $\ell/2$ pairs from dictionary C_1 . Codewords of an odd length are constructed by selecting $(\ell - 1)/2$ pairs from C_1 also adding a symbol from C_2 at the end of the codeword. To ensure that the code does not create homopolymers, the dictionary C_1 does not contain pairs of the same symbol. This means that the pairs AA, TT, CC, and GG are omitted as their consecutive repetition could create homopolymers. Furthermore, to keep the C and G percentage lower or equal to the one of A and T the pairs GC and CG are also not included. To verify this last claim we have computed the evolution of the A, T and G, C percentages created using our code for a source of symbols that follows a Gaussian distribution, in function of the variance σ^2 of the source. As expected, the amount of GC content tends to 40% while the one of AT tends to 60% when the source becomes uniform (big values of σ). Comparison to previous works referred in the paper are listed in Table 2.20.

TABLE 2.20 Comparison to previous works referred in the paper (coding potential in bits/nt), nt=nucleotide.

Parameter	Church [1]	Goldman. [2]	Grass [4]	Bornholt [3]	Blawat [5]	Erlich. [6]	Citing
Input data (MB)	0.65	0.75	0.08	0.15	22	2.15	0.26
Coding potential	1	1.58	1.78	1.58	1.6	1.98	1.6
Redundancy	1	4	1	1.5	1.13	1.07	1
Error correction	No	Yes	Yes	No	Yes	Yes	No

“Can We Store the Whole World’s Data in DNA Storage?” is the title of a paper by David Du et al. at Univ. Minnesota, which also provides a lengthy list of references to recent work.

https://www.usenix.org/system/files/hotstorage20_paper_li_0.pdf

2.4 Reliability studies of DRAM, HDDs, & flash SSDs

Garth Gibson studied failure rates in his PhD thesis at UCB (Gibson, 1992). He estimated HDD MTTFs based on HDD returns to manufacturers, not a very accurate measure, because not all broke HDDs are returned.

An comprehensive review of HDD technology was carried out at Quantum Corp. (Yang and Sun, 1999).

https://en.wikipedia.org/wiki/Quantum_Corporation

Quantum Corp. in 2000 sold its HDD business to Maxtor, which itself was acquired by Seagate in 2001.

Reliability and security of RAID storage systems using SATA disk drives was investigated in (Hughes and Murray, 2005). Disk storage systems usually operate 24×7 , so the historical failure data are operating (power-on-hours) failure rates. These range from 0.3–3% per year (Hughes et al., 2002). A typical drive specification goal today for both SATA or SCSI/Fibre channel is 0.7% annual failure rate. Another manufacturer specified reliability factor is “five-year drive life”. This has recently become a drive manufacturer warranty guarantee. There is a contradiction here, since 1,200,000 hour MTBF means that 50% of the drives will survive for 137 years, far more than their design life of five. The actual reliability goal and its test validation is to meet the 0.7% AFR for 3–5 years. Lower failure rates are expected in enterprise-class drives and systems with professional handling, cooling, vibration, and power. Failure types, occurrence frequency, and primary stress condition are given in Table 2.21.

Bianca Schroeder (CMU, U. Toronto) with Gibson carried out a more comprehensive study of disk failures which are reported in (Schroeder and Gibson, 2007a,b).

U. Wisconsin’s Lakshmi N. Bairavasundaram assisted by researchers at NetApp reported *Latent Sector Errors - LSEs* in (Bairavasundaram et al., 2007).

Bairavasundaram with Andrea and Remzi Arpaci-Dusseau (U. Wisc.), Garth Goodson (CMU), and Bianca Schroeder (U. Toronto) reported on data corruption in the storage stack in (Bairavasundaram et al., 2008).

TABLE 2.21 Failure types, occurrence frequency, and primary stress condition.

Failure mode	Description	Frequency	Stress Condition
Head-Disk interference	Head crash	15.5%	Operating
No problem found	Returned drive tests ok	15.0%	—
Recording heads	Complex nanotech devices	14.5%	Operating
Post manufacture	Drive handling damage	10.1%	Non-Op
Circuit board “PCB”	Many IC components	8.5%	Operating
Head or disk corrosion	Causes disk HDI or defects	7.7%	Non Op
Head assembly “E-Block”	Wires, preamp, coil	6.8%	Operating
Head Disk Assembly	Mechanics, Electrical, Voice coil	3.9%	Operating
Disk defects	Causes HDI or read errors	2.6%	Operating
Drive firmware	Internal operating system	1.9%	Operating
Head-disk stiction	Disk won’t spin up in drive	1.3%	Non-Op
Spindle bearing	Disk spin bearing	1.1%	Operating
Contamination inside drive	Foreign gases or chemicals	0.7%	Op/Non-Op

Yuanyuan Zhou (now at UCSD) with students at UIUC and Arkady Kanevsky at Intel analyzed storage logs collected from about 39,000 storage systems commercially deployed at various customer sites over a period of 44 months including about 1,800,000 disks hosted in about 155,000 storage shelf enclosures (Jiang et al., 2008b,a). Some of our major findings include:

1. In addition to disk failures that contribute to 20-55% of storage subsystem failures, other components such as physical interconnects and protocol stacks also account for significant percentages of failures.
2. Each individual storage subsystem failure type and storage subsystem failure as a whole exhibit strong self-correlations and “bursty” patterns.
3. Storage subsystems configured with redundant interconnects experience 30-40% lower failure rates than those with a single interconnect.
4. Spanning disks of a RAID group across multiple shelves provides a more resilient solution for storage subsystems than within a single shelf.

Schroeder et al. (U. Toronto) investigated *Latent Sector Errors - LSEs* in HDDs in (Schroeder et al., 2010b,a).

Schroeder (U. Toronto) with Eduardo Pinheiro and Wolf-Dietrich Weber at Google published a report on DRAM errors in (Schroeder et al., 2009, 2011).

“What bugs live in cloud storage” is investigated by Haryadi S. Gunawi and Mingzhe Hao et al. (U. Chicago) in (Gunawi et al., 2014). A study concerned with variations in performance in HDD and SSD performance was conducted by Mingzhe Hao a student of Andrew A. Chien (U. Chicago) at NetApp (Hao et al., 2016). The study was conducted over 450,000 disks and 4,000 SSDs over 87 days for an overall total of 857 million (disk) and 7 million (SSD) drive hours. It was observed that over 0.2% (resp. 0.6%) of time an HDD (resp. SSD) is 2 times slower than its peers. Given this performance fault-tolerant systems are proposed, e.g., blocks of a slow disk can be reconstructed on-demand by engaging the parity disks.

Disk consistency checker such as *file system consistency check - fsck* is used to detect and repair corrupted files affected by failures, errors, and bugs.

<https://en.wikipedia.org/wiki/Fsck>

Checkers are rarely run and operate slowly causing significant downtime for large scale storage systems. Addressed in (Ma et al., 2015) is a *fast fsck - ffsck* supported by a modified *third extended file system - ext3* journaling file system for Linux,

https://en.wikipedia.org/wiki/Journaling_file_system

<https://en.wikipedia.org/wiki/Ext3>

which can scan and repair at full sequential bandwidth of the underlying device. fsck is applied to FreeBSD’s FFS file system and its checker as well.

Data Domain’s RAIDshield monitors disk errors and takes steps to protect against them (Ma et al., 2015) This was a five year study covering about one million six SATA disk models. RAIDShield monitors the health of each disk and replaces disks that are predicted to fail imminently, eliminating 98% of triple disk errors, which are 80% of all RAID6 failures. A

method was designed and simulated using the joint failure probability to quantify and predict how likely a RAID group is to face multiple simultaneous disk failures,

A recent study addresses making disk failure predictions using SMARTer (Lu et al., 2020). It is noted that disks are the most commonly replaced hardware components. A total of 380K HDDs are considered in this study. The proposed *Machine Learning - ML* method predicts disk failures with 0.95 F-measure and 0.95 *Mathews Correlation Coefficient - MCC* for ten days prediction horizon on the average.

<https://en.wikipedia.org/wiki/F-score>

In statistical analysis of binary classification, the F-score or F-measure is a measure of a test's accuracy. The highest possible value of an F-score is 1.0, indicating perfect precision and recall, and the lowest possible value is 0, if either the precision or the recall is zero. Given TP: number of True Positives, FP: number of False Positives, FN: number of False Negatives:

$$F_1 = 2 \frac{\text{precision} \times \text{recall}}{\text{precision} + \text{recall}} = \frac{TP}{TP + \frac{1}{2}(FP + FN)}.$$

The original formula for *Matthews Correlation Coefficient - MCC* was:

https://en.wikipedia.org/wiki/Matthews_correlation_coefficient

$$N = TN + TP + FN + FP$$

$$S = \frac{TP + FN}{N} \quad P = \frac{TP + FP}{N},$$

$$MCC = \frac{TP/N - S \times P}{\sqrt{PS(1-S)(1-P)}}$$

Backblaze report on “three drives that never failed”

<https://www.tomshardware.com/amp/news/backblaze-2q21-reliability-report-three-drives-never-failed>

Backblaze storage backup company published its Drive Stats for Q2 2021 report yesterday, and despite an overall increase in failure rates, the company found three HDDs that didn't fail once.

<https://www.backblaze.com/>

But don't take that as a ringing (or would it be spinning?) endorsement quite yet. There are some important caveats regarding the reliability of these particular drives.

The models in question are the Seagate ST6000DX000, HGST HUH721212ALE600, and Western Digital WUH721816ALE6L0. Of those, only the Seagate drives have been in use for over two years, and the company only gathered stats for 886 of them. That pales in comparison to the 177,935 drives that were examined for this report.

The Seagate drives' performance is still notable, however, because those HDDs have been in use for 74 months on average with a lifetime failure rate of 0.92 percent. The WD and HGST drives have been in use for three and 21 months, respectively, so their 0 (WD) and 0.41 percent (HGST) failure rates could rise as they continue to age.

Those rates are still low compared to the average. Backblaze said the annual failure rate for all of its drives rose from 0.81 percent to 1.01 percent year-over-year. “The increase is within our confidence interval,” the company said, “but bears watching going forward.” It didn't attribute the rise in drive failures to anything in particular.

2.4.1 Flash SSD reliability at Facebook, Google & NetApp

The first large-scale study of flash-based SSD reliability in the field was reported (Meza et al., 2015) by CMU researchers at Facebook data centers over nearly four years and millions of operational hours in order to understand failure properties. The SSD characteristics considered were

1. the amount of data written to and read from flash chips;
2. how data is mapped within the SSD address space;

3. the amount of data copied, erased, and discarded by the flash controller;
4. flash board temperature and bus power;
5. data written by the OS to flash-based SSDs does not always accurately indicate the amount of wear induced on flash cells due to optimizations in the SSD controller and buffering employed in the system software.

The following observations are made:

- 1: SSDs go through several distinct failure periods: – early detection, early failure, usable life, and wearout.
- 2: The effect of read disturbance errors is not a predominant source of errors in the SSDs.
- 3: Sparse data layout across an SSD's physical address space (e.g., non-contiguously allocated data) leads to high SSD failure rates; dense data layout (e.g., contiguous data) can also negatively impact reliability under certain conditions, likely due to adversarial access pattern
- 4: Higher temperatures lead to increased failure rates, but do so most noticeably for SSDs that do not employ throttling techniques.
- 5: The amount of data reported to be written by the system software can overstate the amount of data actually written to flash chips, due to system-level buffering and wear reduction techniques.

The study conducted by Bianca Schroeder et al. the reliability characteristics of flash SSDs (Schroeder et al., 2016) covering several million drive days, ten different drive models, different flash technologies (MLC, eMLC, SLC) over 6 years of production use in Google's data centers. The study came to a number of unexpected conclusions:

1. The *Raw Bit Error Rate - RBER* grows at a much slower rate with wear-out than the exponential rate commonly assumed and are not predictive of uncorrectable errors or other error modes.
2. Widely used metric *Uncorrectable Bit Error Rate - UBER* is not a meaningful metric, since no correlation was observed between the number of reads and the number of such errors.
3. The higher-end SLC drives are more reliable than MLC drives within typical drive lifetimes.
4. Compared to HDDs, SSD flash drives have a significantly lower replacement rate in the field, however, they have a higher rate of uncorrectable errors.

Bianca Schroeder et al. report a study of file system reliability in SSDs in (Jaffer et al., 2019, 2020). It is stated that failure characteristics have changed such that firstly partial failures are an order of magnitude higher for SSDs and secondly FTL is prone to bugs during power faults. The study is concerned with how ext4 (followup to ext3), Btrfs, F2FS deal with partial drive errors. The following are investigated: (1) Can they detect errors? (2) Can they recover from errors? (3) Can fsck - the system checker fix errors?

SSD faults and their manifestation with respect to Read I/O **R**, Write I/O **W**, Corruption **C**, Shorn write **S**, and Lost Write **L** are as follows:

Uncorrectable bit corruption: **R**.

Incomplete Program Operation: **S**, **L**.

Dropped Writes: **R**, **W**, **C**.

Silent Bit Corruption: **C**.

Misdirected Writes: **R**, **C**.

Shorn Writes: **S**.

FTL Metadata Corruption: **R**, **W**, **C**.

Incomplete Erase Operation: **C**.

Bianca Schroeder et al.'s study of flash memories in (Maneas et al., 2020), which covered 1.4 million drives from three different manufacturers, 18 different models, 12 different capacities, and four flash technologies, (SLC, cMLC (consumer-class), eMLC (enterprise-class), and 3D-TLC. Data came NetApp's automated telemetry, including drive replacements and the reason, bad blocks, usage, drive age, firmware versions, drive role, used as data, parity, or spare, etc.

The drives are installed in NetApp enterprise storage systems running ONTAP OS,

<https://en.wikipedia.org/wiki/ONTAP>

with data center mounting, cooling, and power conditioning. The SSDs are performing in near-ideal environments, rather than notebooks or business PCs, so reliability differences can be attributed the SSDs, not customer infrastructure. The findings are as follows:

1. Drives with less usage experience higher replacement rates.
2. Infant mortality actually rises over the first year of field use before starting to decline.
3. The most costly SLC - Single Level Cell drives are not more reliable than MLC - Multiple Level Cell drives, while the newest high density 3D-TLC (triple level cell) drives have the highest overall replacement rate, which may be that higher density cells exhibit more failures.

Key observations are as follows:

1. Not all SSDs offer the same reliability and the *Annual Replacement Rates ARR* ranges from 0.07%-1.2% with mean 0.22%.
2. Drive models from the same manufacturer, with the same flash technology, age, and similar capacity, can have ARRs varying dramatically from 0.53% percent for one 15 TB drive versus 1.13% its 15.3 TB stable mate.
3. Bad block provisioning is quite generous and even after several years, the average of consumed spare blocks is less than 15%. Even the drives at the 99.9 percentile of consumed spare blocks have consumed only 33%.

Flash wear out is a non-issue and even after 2-3 years, less than 2% of the rated life is consumed on average. Even the drives in the 99.9th percentile have consumed only 33 percent of their rated life.

NetApp is interested in detecting anomalous drive behavior to take corrective action. A few major fault types lead to most replacements, ranked from most serious to least below.

1. In almost 33% of the failures, the SCSI layer saw a hardware error reported by the SSD, that triggered immediate SSD replacement and data reconstruction. Such errors might be caused by ECC errors in drive's DRAM. Total drive failure - no response - is rare, less than 1% of all failures.
2. Almost 14% of drive replacements are caused by lost writes, where the ECC code on the 4k block doesn't match up with the data. Once a drive has several of these errors, it is marked failed.
3. Aborted commands lead to another 14% of drives being replaced. If the host sends a write, but the data is not written, that is marked as an aborted command.
4. The least serious failures, in about 25% of the cases, are where drives are replaced out of an abundance of caution, based on predictive and threshold data. The drive hasn't yet failed, but the metrics are concerning, so the system initiates a disk copy so the user never sees a problem.

2.5 Storage Networking Industry Association - SNIA

SNIA is a non-profit organization consisting of IT companies. Its mission is to lead the storage industry in developing and promoting vendor-neutral architectures, standards and educational services that facilitate the efficient management, movement and security of information. SNIA publishes videos, webcasts, blogs and white papers on the following topics.

<https://www.snia.org/about>

(1) *Cloud Storage Technologies Initiative - CSTI*, (2) *Conformance Testing Program*, (3) *Data Protection*, (4) *Ethernet Storage*, (5) *Green Storage*, (6) *Linear Tape File System - LTFS*, (7) *Long Term Retention - LTR*, (8) *Software Defined Storage - SDS*, (9) *Solid State Storage - SSS*, (10) *Storage Management - SMI*, (11) *Storage Security*.

2.5.1 Solid state storage performance

SSDs behave very differently than HDDs. *Storage Performance Council - SPC* considers both SSDs and HDDs in its benchmarks, with SSD being the winner not just in performance. The origins of SPC are discussed in Chapter 8. <https://www.spcresults.org>

The *Real World Storage Workload - RWSW* performance test specification provides standardized analysis of in-situ target server application storage performance and standardized comparison and qualification of datacenter storage when using Reference IO Capture Workloads as the test stimuli in RWSW tests. <https://www.snia.org/rws>

The *Solid State Storage - SSS* Performance Test Specifications developed by the SNIA *Technical Work Group - TWG* define how to measure SSD performance in an accurate repeatable manner and enables comparison between different manufacturers' products.

The *SSS Initiative - SSSI Workload I/O Capture Program - WIOCP* collects contemporary I/O operation performance metrics for analysis by SNIA TWGs to gain insights into workload I/O characteristics, key performance metrics, and SSSI design opportunities and considerations

2.5.2 Persistent Memory Forum

SNIA has a *Compute, Memory, and Storage Initiative - CMSI* initiative.

<https://www.snia.org/forums/cmsi>

which includes Persistent Memory and *Non-Volatile Dual In-line Memory Modules - NVDIMMs*.

<https://en.wikipedia.org/wiki/NVDIMM>

There are four types of NVDIMM implementations:

NVDIMM-F: DIMM with flash storage. System users need to pair the storage DIMM alongside a traditional DRAM DIMM.

NVDIMM-N: DIMM with flash storage and traditional DRAM on the same module. The computer accesses the traditional DRAM directly. In the event of a power failure, the module copies the data from the volatile traditional DRAM to the persistent flash storage, and copies it back when power is restored. It uses a small backup power source for this.

NVDIMM-P: will enable computer main memory to be persistent. It can share the *Double Data Rate 4 - DDR4* Synchronous DRAM. It is a high bandwidth (“double data rate”) interface DIMM interconnect with DRAM DIMMs. DDR5 differs from DDR4 in that it is intended to reduce power consumption.

https://en.wikipedia.org/wiki/DDR4_SDRAM

<https://www.te.com/usa-en/products/connectors/sockets/memory-sockets/dimm-sockets/ddr4-sockets.html?tab=pgp-story>

NVDIMM-X: DDR4 DIMM with NAND Flash and volatile DRAM on the same module, developed by Xitore.com.

SNIA’s *Special Interest Group - SIG* accelerates awareness and adoption of persistent memory technologies in the marketplace through education, providing input to standards organizations, and development of vendor-agnostic user perspective case studies, best practices, and vertical industry requirements to help end user customers understand how products and solutions can meet performance, cost, and efficiency goals.

<https://www.snia.org/technology-focus/persistent-memory>

SNIA is driving technical activities to define how the industry will use the new persistent memory solutions coming to market. The Persistent Memory *Technical Working Group - TWG* delivers specifications describing the behavior of a common set of software interfaces that provide access to NVM.

The NVM Programming Model Specification describes behavior used by applications and kernel components to access emerging features for traditional block NVM (SSDs), and a new programming model for persistent memory – NVM hardware designed to be treated by software similarly to system memory. Two articles by Andy Rudoff on this topic in *USENIX* ;login are as follows,

https://www.usenix.org/system/files/login/articles/08_rudoff_040-045_final.pdf

https://www.usenix.org/system/files/login/articles/login_summer17_07_rudoff.pdf

2.5.3 Computational storage

SNIA has recently formed a *Technical Working Group - TWG* on computational storage. Samsung’s *Key-Value SSD - KVSSD* is a new type of SSD that offloads indexing and space management to the storage device by adopting a key-value interface.

The SNIA TWG’s “Computational Storage Architecture and Programming Model v0.3 rev 1”, defines recommended behavior for hardware and software that supports computational storage.

<https://www.snia.org/computational>

What happens when “Compute meets Storage?” is given here.

https://www.snia.org/sites/default/files/SDC/2019/presentations/Computational/Shadley_Scott_Adams_Nick_What_Happens_when_Compute_meets_Storage_Computational_Storage_TWG.pdf

Computational Storage is becoming necessary with increasing volume of generated data (32 ZB - Zettabyte (10^{21} bytes) in 2018 to 175 ZB in 2025) The Digitization of the World from Edge to Core is discussed by Seagate authors in 2018.

<https://www.seagate.com/files/www-content/our-story/trends/files/idc-seagate-dataage-whitepaper.pdf>

The latter figure was predicted by Tom Loughlin.

<https://www.forbes.com/sites/tomcoughlin/2018/11/27/175-zettabytes-by-2025/#5831ec1a5459>

2.6 Big data and its sources

In 2011 IBM estimated that three EBs are generated each day. In 2012 experts predicted the amount of data generated annually to increase 4300% by 2020 and enterprises are responsible for storing 80% of this data.

<https://www.seagate.com/tech-insights/big-data-analytics-master-ti/>

UCB’s Hal Varian calls computer enabled data gathering “computer mediated txns”.

<https://people.ischool.berkeley.edu/~hal/Papers/2013/BeyondBigDataPaperFINAL.pdf>

A recent study at U. Southern California - USC revealed that humankind has stored 295 ExaBytes ($1 \text{ EB} = 10^{18}$ bytes) in the 1986-2007 period.

<https://www.computerworld.com/article/2513110/data-center/scientists-calculate-total-data-stored-to-date--295--exabytes.html>

Cisco *Global Cloud Index - GCI* estimates that nearly 850 Zettabytes of data will be generated by humans and machines taken together by 2021, up from 220 ZB generated in 2016. Report estimates that 10 percent of this data will be useful i.e. 85 Zettabytes, since ephemeral data is not stored.

<https://www.linkedin.com/pulse/edge-computing-opportunity-3x-cloud-has-just-begun-prashant-trivedi/>

Sources of big data according to the following web site are as follows:

<https://www.cloudmoyo.com/blog/data-architecture/what-is-big-data-and-where-it-comes-from/>

Media as big data source Media includes social media and interactive platforms, like Google, Facebook, Twitter, YouTube, Instagram, as well as generic media like images, videos, audios, and podcasts. These provide quantitative and qualitative insights on every aspect of user interaction, and valuable insights on consumer preferences and changing trends. Since it is self-broadcasted and crosses all physical and demographical barriers, it is the fastest way for businesses to understand their target audience, draw patterns and conclusions, and enhance their decision-making capabilities.

Machine data is generated by industrial equipment, sensors installed in machinery, and web logs tracking user behavior. This type of data is expected to grow exponentially as *Internet of Things - IoT* grows. Sensors such as medical devices to measure blood pressure, heartbeat, smart meters, road cameras, satellites, games and the rapidly growing IoT deliver high velocity, value, volume and variety of data. According to (Gao et al., 2020):

“Continued advancement of sensors has led to an ever-increasing amount of data of various physical nature to be acquired from production lines. As rich information relevant to the machines and processes are embedded within these “big data”, how to effectively and efficiently discover patterns in the big data to enhance productivity and economy has become both a challenge and an opportunity. This paper discusses essential elements of and promising solutions enabled by data science that are critical to processing data of high volume, velocity, variety, and low veracity, towards the creation of added-value in smart factories of the future.”

Transactional data includes inline/offline txns, invoices, payment orders, storage records, delivery receipts – all are characterized as transactional data yet data alone is almost meaningless, and most organizations struggle to make sense of the data that they are generating and how it can be put to good use.

Surveillance data based on phone conversations (audio), internet messages (text), and images and video from strategic locations (images and video).

<https://en.wikipedia.org/wiki/Surveillance>

NetApp and Dell are two companies that provide storage for surveillance videos.

<https://www.netapp.com/us/solutions/unstructured-data/video-surveillance.aspx>

<https://www.forbes.com/sites/tomcoughlin/2017/11/09/storage-for-surveillance/#46311bfd424f>

Top five sources of big data are listed below are quite similar to above list:

<https://www.allerin.com/blog/top-5-sources-of-big-data>

Media is the most popular source of big data and was discussed before. It provides valuable insights on consumer preferences and changing trends. By self-broadcasting it crosses all physical and demographical barriers and is hence the fastest way for businesses to get an in-depth overview of their target audience, draw patterns and conclusions, and enhance their decision-making.

Cloud data source. Cloud storage accommodates structured and unstructured data and provides business with real-time information and on-demand insights. As big data can be stored and sourced on public or private clouds, clouds provide an efficient and economical data source.

The web. The public web constitutes easily accessible big data, which is commonly available to all individuals and companies. Web services such as Wikipedia provide free and quick informational insights to everyone. The enormity of the Web ensures for its diverse usability and is especially beneficial to start-ups and small and medium enterprises, so that they do not have to develop their own big data infrastructure. Wikipedia co-founder Larry Sanger slams the site's left wing woke bias and claims its days of neutrality are over.

<https://www.dailymail.co.uk/news/article-9283061/Wikipedia-founder-Larry-Sanger-slams-sites-leftist-bias-claims-neutrality-gone.html>

IoT. IoT is mentioned earlier as machine data and includes big data generated, not only from computers and smartphones, but also from every device that can emit data, medical devices, vehicular processes, video games, meters, cameras, household appliances, etc.

Databases. This was earlier listed as transactional data. Businesses utilize databases to acquire relevant big data, which integrates the way for a hybrid data model and requires low investment and IT infrastructural costs. Furthermore, these databases are deployed for business intelligence purposes as well. These databases can then provide for the extraction of insights that are used to drive business profits.

The process of extracting and analyzing data among extensive big data sources is a complex and time-consuming process, which can be resolved if organizations encompass all the necessary considerations of big data, take into account relevant data sources, and deploy them in a manner which is well tuned to their organizational goals.

2.7 Sources of storage content

Audio, video, image, text are major contributors to storage occupancy.

Audio encodings (codec) and a list and Audio file formats are described below.

https://en.wikipedia.org/wiki/Audio_codec

https://en.wikipedia.org/wiki/List_of_codecs

https://en.wikipedia.org/wiki/Audio_file_format

Video file formats or encodings are listed here.

https://en.wikipedia.org/wiki/Video_file_format

https://en.wikipedia.org/wiki/Comparison_of_video_container_formats

A survey of video encryption is provided in (Shah and Saxena, 2011), but data encryption is discussed in Section 2.31.

Image file formats are standardized means of organizing and storing digital images. Images may be stored in an uncompressed format, a lossless or lossy compressed format, or a vector format. Image files are composed of digital data in one of these formats so that the data can be rasterized for use on a computer display or printer. Rasterization converts the image data into a grid of pixels. Each pixel has a number of bits to designate its color.

<https://en.wikipedia.org/wiki/Rasterisation>

Rasterizing an image file for a specific device takes into account the number of bits per pixel (the color depth) that the device is designed to handle.

https://en.wikipedia.org/wiki/Image_file_formats

A survey of video encryption is provided in (Shah and Saxena, 2011), but data encryption is discussed in Section 2.31.

There are a large number of file formats for text.

https://en.wikipedia.org/wiki/List_of_file_formats

This book was written using LaTeX file formatting language, developed by ACM Turing award recipient Leslie Lamport (SRI Int'l, DEC/Compaq, Microsoft),

<https://en.wikipedia.org/wiki/LaTeX>

which is based on Donald Knuth's Tex formatting language.

<https://en.wikipedia.org/wiki/TeX>

Applying LaTeX or Latexing latex files designated .tex produces a .dvi (device independent) file format if there are no errors. The dvips command converts a .dvi file to a .ps (postscript) file, which can then be displayed on the web or for printing. The ps2pdf command converts a .ps file to pdf, which is a common format for posting documents on the web. The pdflatex command can convert a .tex file directly to pdf. Postscript files are typically double the size of pdf files, but pdf is not the most space-efficient file format from the viewpoint of storage efficiency and transmission bandwidth requirements. <https://acrobat.adobe.com/us/en/acrobat/about-adobe-pdf.html> Software to read and produce pdf files are available freely from Adobe,

https://en.wikipedia.org/wiki/Adobe_Acrobat

but more advanced features are available at a fee. For example, smallpdf.com can be used to carry out certain functions on pdf files, such as removing certain pages.

SkyLaTeX is a tool designed to facilitate the academic journal and book proofing process. SkyLaTeX allows authors to insert corrections directly into the production LaTeX file, preview the corrected PDF after compilation, and to view the editor's queries and answer them. The workflow is as follows: What follows is based on a web page describing SkyLaTeX.

<https://vtex.lt/services/skylatex-proofing/>

1. After copyediting, typesetting and enrichments have been made, a unique link to the LaTeX production file is sent to the author.

2. By clicking this link, the author is directed to SkyLaTeX homepage where the LaTeX source file of the publication is shown on the left and the corresponding PDF preview on the right. The PDF layout fully meets house style and publisher requirements.
3. If authors want to correct anything, they can make changes directly into the LaTeX source file and compile it online.
4. To complete the final proof, the author simply clicks on the 'Validate and finish proofing' button.

DjVu files provide a great balance between quality and size. and are inherently smaller than PDF files, making them excellent candidates for use in a web environment, however, PDF files are capable of providing higher quality images, which is necessary in some cases.

<https://www.guidingtech.com/56915/gt-explains-djvu-format-pdf-differences/>

The *Rich Text Format - RTF* is a proprietary document file format developed by Microsoft Corp. 1987-2008. Most Microsoft Word processors are able to read and write some versions of RTF.

https://en.wikipedia.org/wiki/Rich_Text_Format

2.8 Ranking and description of media companies

The media industry covers a wide variety of areas: The following ranking by **market capitalizations** on 10/7/20 was updated on 2/20/2021, so that some ranking have changed.

<https://www.investopedia.com/stock-analysis/021815/worlds-top-ten-media-companies-dis-cmsa-fox.aspx>

1. **Netflix (NFLX):** \$234 B. Netflix founded in 1997 transferred itself from a DVD-by-mail business into a streaming giant. The company had 167 million subscribers as of the end of 2019. The majority of Netflix's revenues come from membership fees from streaming customers. It generated \$20 B in streaming revenue in 2020 and ≈\$300M in DVD rentals. Netflix is currently the world's leading internet television network with more than 100 million members in 190 countries watched 125 million hours. Netflix uses AWS for nearly all its computing and storage needs, including databases, analytics, recommendation engines, video transcoding, and more - hundreds of functions using more than 100,000 server instances on AWS.

<https://en.wikipedia.org/wiki/Transcoding>

A list of top 101 AWS customers is given here

<https://www.cloudzero.com/blog/netflix-aws>

NetFlix which is one of AWS's largest customers and its cost was \$9.6 M per month in 2019. <https://aws.amazon.com/solutions/case-studies/netflix/>

Also missing is DropBox, which started with AWS but is now building its own datacenter.

Early multimedia server design studies at IBM Research are reported in (Sitaram and Dan, 2000), which was written by two of its participants. Another book in streaming media server design is (Dashti et al., 2003). A comprehensive discussion of VoD is given below.

https://en.wikipedia.org/wiki/Video_on_demand A new development in this area is discussed here.

The Cisco Cloud Services Stack - CCSS

The following discussion is based on the following.

<https://www.cisco.com/c/en/us/solutions/collateral/service-provider/at-a-glance-c45-743988.html>

Cisco and Qwilt have partnered to deliver full-stack, turnkey, managed content delivery solution with carrier-grade infrastructure providing a new architecture and business model optimized for content delivery using edge nodes owned and operated by *Internet Service Providers - ISPs*. Content delivery is optimized within the network, which improves quality and reduces scaling costs. The business model provides new revenue for service providers collaborating with publishers. CCSS allows ISPs to leverage the efficiency of an edge cloud architecture to handle the growth of streaming media across their networks. This solution provides greater control and visibility so they can respond faster and more intelligently to changing network conditions, as well as enabling new business models, customers, and revenue. This new architecture means content delivery is optimized within the network, In summary CCSS achieves the following

- Improves content delivery *Quality of Experience - QoE* through deeply embedded edge delivery network nodes.
- Creates new business models and revenue through a flexible monetization platform.
- Accelerates time to revenue through a growing ecosystem of content publishers on the platform.

- Reduces infrastructure scaling costs through a cost-effective delivery model and edge computing architecture that optimizes network economics.
- Scale fast and efficiently as traffic grows and usage profiles evolve through flexible service definition, full solution control, and comprehensive visibility that speeds analysis and insights

Other CCSS advantages are as follows:

- Optimizes delivery of over-the-top content as well as managed content.
- The user is provided with the control and visibility needed to fully optimize content delivery network resources.
- Leverages the global network of content publishers already onboard Qwilt's *Content Delivery Sharing - CDS* service.
- Deploys the solution with no capital investment through a unique revenue sharing arrangement.
- Leverages open caching specifications that enable federated content delivery in ISP networks. These were developed by the *Streaming Video Alliance - SVA* and are being adopted by service providers and publishers around the world.
- Unlike other commercial and private *Content Delivery Networks - CDNs*, CCSS provides a full-stack, fully managed content delivery infrastructure with the resource control, performance visibility, configuration flexibility, and monetization capability exactly matching network conditions and business requirements and business requirements.

VOD services were only made possible as a result of two major technological developments: *Discrete Cosine Transform - DCT* video compression (Sayood, 2017)

https://en.wikipedia.org/wiki/Discrete_cosine_transform

and *Asymmetric Digital Subscriber Line - ADSL* data transmission.

https://en.wikipedia.org/wiki/Asymmetric_digital_subscriber_line

DSL enables faster data transmission over copper telephone lines than a conventional voiceband modem can provide. ADSL differs from the less common symmetric digital subscriber line (SDSL) in that it provides a greater bit rate toward the customer premises (downstream) than the reverse (upstream). Providers usually market ADSL as an Internet access service primarily for downloading.

DCT is a lossy compression technique that was first proposed by Nasir Ahmed in 1972, but later adapted into a motion-compensated DCT algorithm for video coding standards such as the H.26x formats from 1988 onwards and the MPEG formats from 1991 onwards. Motion-compensated DCT video compression significantly reduced the amount of bandwidth required for a television signal.

ADSL increased the bandwidth of data that could be sent over a copper telephone wire. ADSL increased the bandwidth of a telephone line from around 100 kbps to 2 Mbps, while DCT compression reduced the required bandwidth of a television signal from around 200 Mbps down to 2 Mbps. The combination of DCT and ADSL technologies made it possible to practically implement VOD services at around 2 Mbps bandwidth in the 1990s.

- 2. Walt Disney (DIS):** \$220 B. Disney was founded by Walter Disney in 1923 in Burbank, CA. It is now a large media and entertainment group with multiple subsidiaries and an international presence. Disney offer media networks, parks and resorts, studio entertainment, interactive media. Disney generated ≈\$70 B in revenues over the *Trailing Twelve Months - TTM*. Disney Plus had 57.5 million subscribers in mid 2020.
- 3. Comcast Corp (CMCSA):** \$210 B. Comcast established in 1963 in Philadelphia, Pa. is one of the largest global media, entertainment, and communications companies generating over \$108.9B TTM. Its business is run through six segments: Cable, high speed internet, TV, films, parks, and European sky.com which provides video, high-speed internet, voice, and wireless phone services.
- 4. AT&T (ATT):** \$203.65 B. AT&T's \$101 B acquisition of Time Warner Inc. in 2018, added Turner Broadcasting and *Home Box Office - HBO*, *Turner Network Television - TNT*, *Turner Broadcasting Systems -TBS*, *Cable News Network - CNN*, Cartoon Network, Warner Bros., DirecTV. In 2019 its revenue was \$219 B.
- 5. Charter Communications (CHTR):** \$87.4 B. Charter acquired Time Warner Cable in 2015 for \$87B giving it a significant media presence. The company operates under the Spectrum name, boasting 26 million customers. Its 2018 revenue was \$43.63 B.
- 6. Sony (SNE):** \$95 B. Sony in addition to electronics and music has other media properties. 2020 revenue \$76B.
- 7. Thomson Reuters Corp. (TRI):** \$40 B. Thomson Reuters is a Canada-based media company, has five segments: legal professionals, corporates, tax and accounting professionals, Reuters news, and global print. The Reuters news

business offers financial news via media organizations. Thomson Reuters revenue in 2019 was \$5.9B. It competes with Bloomberg L.P. whose revenue was \$10 B in 2018.

https://en.wikipedia.org/wiki/Bloomberg_L.P.

8. **ViacomCBS (VIAC):** \$17.5 B. Viacom merged with CBS in December 2019 to form ViacomCBS. (*Columbia Broadcasting System - CBS*). Viacom was established in 2005 in NY city and provides entertainment content for a global audience through digital, film, and television. Brands include Nickelodeon, TeenNick, Comedy Central, Spike TV, Paramount Pictures, MTV Films. 2019 revenue was \$27.81 B.
9. **Fox Corp. (FOX):** \$15.4 B. In 2017 Fox sold its entertainment business which included its 20th Century Fox television and film studios, Hulu (30%), cable channels such as FX, to Disney for \$71 B, Fox Corp. now includes the Fox News and Business, Big Ten Networks, and Fox Broadcasting Company. Fox Corp revenue in 2020 was \$12.3 B.
10. **DISH Network Corp. (DISH):** \$14.5 B. The Dish Network established in 1980 operates as a holding company and through its subsidiaries provides TV services through direct satellite broadcast, broadband services, movies, and games on demand for rental and sale. It was established in 1980. 2019 revenue was \$12.81 B.

2.9 Sources of news: newspapers, radio and TV stations

2.9.1 Newspapers in US and worldwide

Top ranking newspapers in the US are as follows:

<https://www.agilitypr.com/resources/top-media-outlets/top-15-daily-american-newspapers/>

- (1) The Wall Street Journal <https://www.wsj.com>
- (2) USA Today <https://www.usatoday.com>
- (3) Los Angeles Times, <https://www.latimes.com>
- (4) NY Times, <https://www.nytimes.com>
- (5) Houston Chronicle, <https://www.chron.com>

Wikipedia ranks USA Today as highest in circulation.

https://en.wikipedia.org/wiki/List_of_newspapers_in_the_United_States

This paper is delivered to all hotel rooms, perhaps unoccupied and unread.

Washington Post - *WAPo* owned by Jeff Bezos is ranked #8.

<https://www.washingtonpost.com>

A list of online newspaper archives is listed below.

https://en.wikipedia.org/wiki/Wikipedia:List_of_online_newspaper_archives

In the case of US newspapers there is a breakdown by state.

The NY Times switched to digital production before 1980, but started saving digital text only in 1980.

<https://www.nytimes.edu>

https://en.wikipedia.org/wiki/The_New_York_Times The Internet allows a reader to access multiple newspapers to obtain different viewpoints of the same news story. *Mainstream Media* - *MSM* is used to refer to large mass news media that influence millions.

https://en.wikipedia.org/wiki/Mainstream_media

Preceding the 2020 presidential election (Trump vs Biden) newspapers such as the NY Times and WaPo and TV stations such as CNN and MSNBC joined forces against the republican candidate Donald Trump. Joe Biden led the polls from the start and won the election after minimal campaigning. Voter fraud is suspected might have occurred in states where no photo-id is required and absentee voting.

Most newspapers require a subscription, but may allow a few free accesses per month. Most newspapers allow the reader to see the headlines and a few paragraphs. Public libraries in US provide access to local newspapers to their members. There are many free online newspapers requesting free-will donations.

<https://www.counterpunch.com>, <https://www.csmonitor.com>,
<https://www.dailybeast.com>, <https://huffpost.com> (Huffington Post),
<https://www.newsmax.com>, <https://www.politico.com>,
<https://veteranstoday.com>, <https://thehill.com/>,
nypost.com, <https://www.theepochtimes.com>

US news papers vary significantly in what is considered worth publishing, e.g., the Christian Science Monitor published the following article by Robert Reich (Secretary of Labor under three presidents) stating that US's biggest and only major jobs program is the U.S. military. Over 1,400,000 Americans are now on active duty; another 833,000 are in the reserves,

many full time. Another 1,600,000 Americans work in companies that supply the military with everything from weapons to utensils.

<https://www.csmonitor.com/Business/Robert-Reich/2010/0813/America-s-biggest-jobs-program-The-US-military>

Washington Post - *Wapo* is a newspaper owned by Amazon's Jeff Bezos (worth \$223.64 B - net worth vary in time) alternates with Elon Musk (worth \$162 B) as world's wealthiest person, even after sharing a portion of his wealth with his ex-wife MacKenzie Scott worth \$53 B is ranked #3 among women, while Steve Jobs' widow Lauren is ranked #6 among women billionaires.

<https://howmuch.net/articles/wealthiest-women-in-the-world>

Lauren recently bought majority interest in *The Atlantic* monthly, whose editor Jeffrey Goldberg wrote a negative article about Trump two months before the election.

https://en.wikipedia.org/wiki/The_Atlantic

<https://www.theatlantic.com/politics/archive/2020/09/trump-americans-who-died-at-war-are-losers-and-suckers/615997/>

Given Bill Gates net worth is \$131 B Melinda French Gates worth \$2.4 B is another female multi billionaire in the making.

Bezos detested Trump who perhaps correctly ascribed Amazon's success to low-cost USPS delivery rates and while Amazon was very profitable USPS was in the red and required bailout by US government. (as noted earlier).

https://en.wikipedia.org/wiki/The_Washington_Post

Saudi Jamal Khashoggi who had criticized Saudi crown prince and de-facto ruler *Mohammed bin Salman* - *MBS* in *Wapo* was assassinated and dismembered as he entered the Saudi consulate in Istanbul to get divorce papers. In fact he had married to an Egyptian hostess a few months earlier, perhaps because as a Moslem he was allowed multiple wives.

https://en.wikipedia.org/wiki/Jamal_Khashoggi

After being suffocated he was cut to pieces and his body was not found by Turkish police in the consulate. A mock trial held in Saudi Arabia found some officials guilty, but not MBS. When asked about Khashoggi Trump answered "America first", in other words his main concern was the profit from huge US military sales to Saudis, which created jobs in US and especially California. Biden administration declassified a report that found MBS guilty, but no action was taken against him.

<https://www.theguardian.com/world/2021/jan/19/biden-administration-to-declassify-report-into-khashoggi>

British newspapers and their web sites are already in English, but almost all major foreign newspapers have an English edition, although an American spelling is mostly used:

The Guardian <https://www.theguardian.com>

The Independent <https://www.independent.co.uk>

The Daily Mail <https://www.dailymail.co.uk>

Telegraph <https://news.bbc.co.uk>, <https://www.economist.com>

Non-English-speaking countries publish news in English for wider access, e.g.,

<https://www.france24.com> and *Le Monde* <https://www.mondediplo.com/> from France,

dw.de and *Der Spiegel* <https://www.spiegel.de/international/> from Germany,

rt.com Russia Today and *sputniknews.com*, <https://pravdareport.com> from Russia,

which have Ms. Margarita Simonyan as editor-in-chief,

Haaretz [haaretz.com](https://www.haaretz.com) and *Jerusalem Post* <https://www.jpost.com> from Israel,

<https://www.aljazeera.com> Aljazeera from Qatar,

Dailystar <https://dailystar.com.lb> *Dailystar* from Lebanon,

<http://www.presstv.com> has been seized by US with a message in Arabic! the same content is available at which is not seized <https://www.presstv.ir>, and <https://www.farsnews.ir/en> from Iran,

today.az, *report.az*, *newsaz* from Azerbaijan

<https://mirrorspectator.com/>, <https://asbraz.com> Armenia diaspora in US

<https://en.armradio.am/>, <https://www.lragir.am>

Public libraries in US subscribe to local newspapers, such as the *NY Times* in New York, but also provide online access to others such as *WaPo*.

https://go.gale.com/ps/start.do?prodId=STND&userGroupName=nysl_me_wls

Newspapers provide archival access to earlier issues, which are published in digital form. Adobe provides software for scanning documents into pdf.

<https://helpx.adobe.com/acrobat/using/scan-documents-pdf.html>

2.9.2 TV networks in US

TV networks produce a hours of content most of which is probably held on disk. This can be evidenced in conjunction to sports news where clips from previous events are provided on demand.

The *Public Broadcasting System - PBS* <https://www.pbs.org/> is an American public broadcaster and television program distributor, which started operation in 1970. It has 350 TV stations which are funded by a combination of member station dues, the Corporation for Public Broadcasting, National Datacast, pledge drives, and donations from both private foundations and individual citizens.

<https://en.wikipedia.org/wiki/PBS>,

https://en.wikipedia.org/wiki/List_of_PBS_member_stations

A similar strategy is used by the *National Public Radio - NPR*.

<https://www.npr.org/>

Commercial TV stations generate revenue by airing commercials, 8-10 minutes in half an hour. Infomercials are another form of paid advertisements. A recent example is Mike Lindell from [MyPillow.com](https://www.mypillow.com) renting hours of TV time on *One American News Network - OANN* to discuss perceived-run irregularities in the 2020 presidential election.

A more extreme case is the *Quality Value Convenience - QVC* is a 24 hour homes shopping network which was founded in 1986.

<https://en.wikipedia.org/wiki/QVC>

Cable TV uses subscriptions. There is a basic fee but also additional fees for premium channels such as *Home Box Office - HBO* and *Showtime*. The payment is made to the local *Internet Service Provider - ISP* provider, such as Verizon or Optimum.

https://en.wikipedia.org/wiki/List_of_broadband_providers_in_the_United_States Verizon provides discounts for bundles: \$75-\$20 = \$45 for 75/75 Gbps internet, 30% for the phone, and \$43.99 for TV (including the above movie channels. Add \$30 for Voice (a phone landline). Verizon has presence worldwide but mainly in US.

https://en.wikipedia.org/wiki/Verizon_Communications

The three major early TV networks in US were:

National Broadcasting Company - NBC sports radio 1927, TV since 1939, now owned by Comcast.

<https://en.wikipedia.org/wiki/NBC>

Columbia Broadcasting System - CBS radio 1929, TV since 1941, now owned by Viacom.

<https://en.wikipedia.org/wiki/CBS>

American Broadcasting Company - ABC, radio 1943-2007, TV since 1948, now owned by Disney.

https://en.wikipedia.org/wiki/American_Broadcasting_Company

Color TV was introduced in US in mid 1950s.

https://en.wikipedia.org/wiki/Color_television

The first color television was patented by Hovannes Adamian in 1908.

https://en.wikipedia.org/wiki/Hovannes_Adamian

Cable-Satellite Public Affairs Network - CSPAN was created in 1979 by the cable television industry as a nonprofit public service. CSPAN televises proceedings of the US federal government, as well as other public affairs programming.

<https://en.wikipedia.org/wiki/C-SPAN>

Cable News Network - CNN is a 24 hour news network founded by Ted Turner in 1980. CNN gained prominence by reporting the first and second Gulf War from Baghdad.

https://en.wikipedia.org/wiki/Media_coverage_of_the_Gulf_War

It is now owned by the Turner Broadcasting System division of WarnerMedia.

<https://en.wikipedia.org/wiki/CNN>

Fox Broadcasting Company - FOX was established by Rupert Murdoch and Barry Diller in 1986.

https://en.wikipedia.org/wiki/Fox_Broadcasting_Company

https://en.wikipedia.org/wiki/Rupert_Murdoch

FOX News was built by Roger Ailes (rip), a staunch republican, who was forced to resign in 2016 as a result of sexual harassment complaints.

https://en.wikipedia.org/wiki/Fox_News

https://en.wikipedia.org/wiki/Roger_Ailes

MSNBC is the news division of <https://nbcuniversal.com> and was formed in 1996. It showed favoritism towards Barack Obama and Democrat party and in general, with persistent attacks on Republican party.

<https://en.wikipedia.org/wiki/MSNBC>

Newsy is a 24 hour news channel since 2008 (no longer available on Verizon). www.newsy.com

One American News Network - OANN launched in 2013 is owned by Herring Networks founded by Richard Herring and is biased towards Republicans and Trump.

https://en.wikipedia.org/wiki/One_America_News_Network

<https://www.oann.com/>

Newsmax TV owned by Newsmax Media was established in 2014 by American journalist and Newsmax CEO Christopher Ruddy 35 million satellite subscribers through DirecTV, a satellite service and Dish Network, which provides wireless service.

https://en.wikipedia.org/wiki/Newsmax_TV

<https://en.wikipedia.org/wiki/DirecTV>

https://en.wikipedia.org/wiki/Dish_Network

A list of pay TV channels is as follows.

https://en.wikipedia.org/wiki/List_of_United_States_pay_television_channels

Streaming TV from Netflix, Hulu, among others is a relatively new development. They also produce original movies and TV series.

<https://www.inc.com/sarah-jackson/best-tv-streaming-services.html>

An example of alternative media (newsources, TV and print) is Democracy Now aired at 8am by Amy Goodman by several TV and radio stations.

[https://en.wikipedia.org/wiki/Alternative_media_\(U.S._political_left\)](https://en.wikipedia.org/wiki/Alternative_media_(U.S._political_left))

<https://www.democracynow.org>

Library of Congress - LOC serves as an archivist of TV programs.

<https://www.loc.gov/rr/mopic/tvcoll.html>

2.10 Text editing and formatting languages

Text editing languages listed below should be distinguished from text formatting languages. pico or nano, which are the same. vi and emacs which are compared below.

<http://www.reallylinux.com/docs/editors/editor.shtml>

<https://www.unixtutorial.org/reference/unix-text-editors>

Emacs development began during the 1970s at MIT's AI Lab, whose PDP-6 and 10 computers used the *Incompatible Timesharing System - ITS* OS that featured *Tape Editor and Corrector - TECO*. GNU Emacs describes itself as “the extensible, customizable, self-documenting, real-time display editor”. The most popular version is GNU Emacs was created by Richard Stallman for the GNU Project. GNU also delivered Linux, hence the recursive acronym “GNU’s Not Unix!”

<https://en.wikipedia.org/wiki/Emacs>

https://en.wikipedia.org/wiki/GNU_Project

Other text editors are listed here.

https://en.wikipedia.org/wiki/List_of_text_editors

<https://kinsta.com/blog/best-text-editors/>

Text formatting languages are almost as old as computers. Troff was developed by Jerome Saltzer at MIT in early 1960s in conjunction with MULTICS - Multiplexed Information and Computing Service (Organick, 1972).

<https://en.wikipedia.org/wiki/Multics>

Later Troff became available at Bell Labs for UNIX.

<https://en.wikipedia.org/wiki/Troff>

Stuart Madnick at MIT contributed to the design and development of IBM CP67/CMS OS, which stands for Control Program/Cambridge Monitor System.

<https://en.wikipedia.org/wiki/CP/CMS>

Madnick and Alan Moulton developed the SCRIPT text editing and formatting system for IBM (Madnick and Moulton, 1968).

Leslie Lamport developed the Latex formatting language based on Knuth's TEX developed for writing *The Art of Computer Programming - TAOCP* book series. Note that TEX is a typesetting language which gives the user more control.

https://en.wikipedia.org/wiki/Leslie_Lamport

https://en.wikipedia.org/wiki/Donald_Knuth

<https://en.wikipedia.org/wiki/TeX>

What You See Is What You Get - WYSIWYG is a system in which editing software allows content to be edited in a form that resembles its appearance when printed or displayed as a finished product, such as a printed document, web page, or slide presentation.

<https://en.wikipedia.org/wiki/WYSIWYG>

An advantage of Word with respect to Latex is that the user can see the formatted text as it is being input. Word does not provide its advanced capabilities of Latex for formatting mathematical expressions. A comparison of the two is given below.

https://openwetware.org/wiki/Word_vs._LaTeX

Some examples where Latex is superior are given here

<http://nitens.org/taraborelli/latex>

SkyLaTeX is an interesting interface provided by Elsevier for proofreading. The Latex file provided by the author with additional formatting commands especially in the case of tables and references appears on the left and can be “compiled” to create the corresponding PDF file appearing on the right. Both files are held at an Elsevier server.

<https://www.skylatex.com/proof/>

A book chapter written in Latex I submitted to Springer for publication in “Encyclopedia of Databases”, 2nd ed. (Liu and Özsu, 2018) was first converted to Word by the publisher before inclusion. This trend may be the result of widespread availability of Microsoft Word with Windows OS to utilize the workforce in countries with cheaper labor, e.g., India and Philippines, rather than Word’s superiority.

Gordon Bell’s personal digital store

In a 2001 article “A Personal Digital Store” (Bell, 2001), the author advocates the use a digital disk to hold all data encountered by a person. Table 2.22 based on (Gemmell et al., 2006) is a glimpse of Gordon Bell’s data up to 2006.

<http://totalrecallbook.com/>

TABLE 2.22 Gordon Bell’s content format circa Nov. 2006.

Item type	Number	Size (GB)
Video	1,303	61.3
Audio	5,083	12.2
Pictures	43,812	8.8
Tiff	3,832	7.9
Web pages	70,918	5.7
PDF	3,527	4.9
PPT	1,815	4.5
Doc & RTF	13,764	1.2
Other	5,046	0.9
Email text	97,271	0.3
Total	245,371	107.6

2.11 Online books sources

Jeff Bezos started [Amazon.com](https://www.amazon.com) by selling mail-order books from his garage. Amazon has expanded to offer competitive prices for many goods with an easy to use interface.

<https://www.amazon.com> Besides books Amazon currently sells many products from its multiple gigantic warehouses. A map of Amazon over 100 warehouses, called fulfillment centers is as follows.

<https://www.cnbc.com/2020/01/19/map-of-amazon-warehouses.html>

A very profitable aspect of Amazon is *Amazon Web Services* - AWS, whose earnings are twice of IBM and half of Amazon. Competition from Amazon led to the closing down of many mom and pop, brick and mortar stores and even the major chain Borders.

Self-publish eBooks and paperbacks for free with Kindle Direct Publishing,

<https://kdp.amazon.com>

Amazon tablets known as Fire are less expensive than the Google IPAD and Samsung tablets.

<https://www.amazon.com/Tablets/b?node=1232597011>

Another company publishing books is <https://www.bookbaby.com>

e-books which can be read online or downloaded for reading or printing have a long history.

<https://en.wikipedia.org/wiki/E-book>

Amazon acts a mediator between sellers and buyers while charging a fee. A large proportion of used books were published as K-12 and college textbooks. Amazon rents the more expensive textbooks and also provides a book publishing service.

<https://www.bookbaby.com/amazon-book-publishing/>

Amazon tablets known as Fire are less expensive than the Google IPAD and Samsung tablets.

https://en.wikipedia.org/wiki/Amazon_Fire_tablet Amazon also provides book self publishing.

<https://kdp.amazon.com>

Barnes & Nobles - B&N has survived in two formats: brick and mortar stores and e-commerce.

<https://www.barnesandnoble.com>

B&N's Nook tablet uses the Android mobile OS developed by Google.

https://en.wikipedia.org/wiki/Barnes_&_Noble_Nook

Google's Android is a best-selling OS worldwide on smartphones since 2011 and tablets since 2013. Android is based on Linux and runs on a 64 bit variants of x86 and ARM microprocessors. As of May 2017 it has over two billion active users, the largest installed base of any OS.

[https://en.wikipedia.org/wiki/Android_\(operating_system\)](https://en.wikipedia.org/wiki/Android_(operating_system))

Google has further developed Android TV, Android Auto for cars, and Wear OS for wrist watches, game consoles, digital cameras, PCs and other electronics

A list of mobile operating system and their ranking is given below.

https://www.webopedia.com/DidYouKnow/Hardware_Software/mobile-operating-systems-mobile-os-explained.html

2.12 Free book download web sites

There are numerous websites providing free books, which can be read online or downloaded and printed.

book4you.org founded in 2009 proclaims itself as the worlds largest ebook library, which holds the books in an elastic cloud, which is discussed i Chapter 8. It encourages donations from members, but a limited number of downloads are permitted per day without paying a fee.

https://en.wikipedia.org/wiki/Elastic_cloud_storage

with 8.4 million books and more than 80 million articles on mid-July 2021.

<https://book4you.org> <https://1lib.us/>

Book4you provides several formats for downloading books, e.g., a epub format book may be downloaded as a pdf file. There are websites such as zamzar.com, which can be used to convert book formats, when the original format cannot be displayed or printed. Some book formats are used for storage because they are more compact, e.g., R. Shanker's "Basic Training in Mathematics" requires 4,488,678 bytes with the djvu format versus 26,425,238 bytes using pdf, a six-fold improvement in storage space, but little difference in resolution.

<https://en.wikipedia.org/wiki/DjVu>

A list of free public domain book sources is as follows.

<https://ebookfriendly.com/free-public-domain-books-sources/>

1 - **Project Gutenberg** was started by Michael S. Hart at UIUC in 1971, on a Xerox Sigma V mainframe developed by *Scientific Data Systems - SDS* connected to Arpanet.

https://en.wikipedia.org/wiki/Project_Gutenberg

https://en.wikipedia.org/wiki/Scientific_Data_Systems

https://en.wikipedia.org/wiki/SDS_Sigma_series

It currently holds 60,0000 books in the following categories.

<http://www.gutenberg.org/wiki/Category:Bookshelf>

2 - **Europeana** <https://www.europeana.eu/portal/en>

3 - **Digital Public Library of America** <https://dp.la>

4 - **Internet Archive** is a non-profit library of millions of free books, movies, software, music, websites, and more. It was founded by Brewster Kahle in 1966.

https://en.wikipedia.org/wiki/Internet_Archive

<https://archive.org>

Wayback Machine is part of the Internet Archive, which has archived billions of pages since 2005.

https://en.wikipedia.org/wiki/Wayback_Machine

5 - **Open Library** is part of the Internet Archive.

<https://openlibrary.org/>

6-8 - Omitted.

9 - **Google** had a major effort to scan all published books, but fully scanned books are posted partially with missing pages to avoid copyright violations.

<https://books.google.com>

<https://www.edsurge.com/news/2017-08-10-what-happened-to-google-s-effort-to-scan-millions-of-university-library-books>

The answer to the question how much storage is required to store scanned documents is answered below.

<https://catalystsecure.com/blog/2016/07/ask-catalyst-how-much-storage-do-i-need-for-600000-scanned-documents-or-how-many-tiffs-in-a-gb/>

Tagged Image File Format - TIFF handles images and data within a single file. A good resolution is 300 *DPI - Dots Per Inch* For 400 GB of scanned single-page TIFFs the average page requires 65 KB, ranging between 54 and 75 KB. A single page pdf document produced from a Latex file with no figures is less than 10 KB.

Several sites such as [jstor.org](https://www.jstor.org/) for academic journals are provided at [ithaka.org](https://www.ithaka.org/). \$30 per article is charged for downloads.

2.13 Data, image, audio and video compression

Compression applied to data, images, audio and video is discussed in the following sections.

2.13.1 Data compression

Data compression has been classified into lossless or reversible and lossy or irreversible. Lossless data compression is applied to text files or some images such as radiological data, where loss of accuracy can be catastrophic as in the case of missed cancer tumors.

The data compression ratio is the size of uncompressed data divided by the size of compressed data.

https://en.wikipedia.org/wiki/Data_compression_ratio

Lossy compression such as JPEG for images or MP3 and Opus for audio can achieve much higher compression ratios at the cost of decreased fidelity.

Data compression can be applied to in transit, being transmitted and at rest, stored data. Transmitting compressed data will take less time and storing it will take less space. With compressed data in main memory fewer page faults will be encountered and this will result in a shorter runtime.

Three text compression methods: Huffman, Lempel-Ziv, and arithmetic coding are discussed in the following sections.

2.13.2 Huffman coding/encoding

Huffman encoding to achieve data compression was developed by David Huffman as part of an undergraduate project in a 1952 course taught by Robert Fano at MIT (Huffman, 1952). Fano was a student of Claude Shannon, who became the father of information theory by publishing “a mathematical theory of communication” in *Bell Systems Technical J. - BSTJ* (Shannon, 1949).

The Huffman algorithm assigns shorter codes to more frequently appearing symbols. Consider a text written with three characters a, b, and c, with frequencies $P(a) = 0.5$, $P(b) = 0.25$, $P(c) = 0.25$ then the encoding a as 1, b as 00, c as 01 is appropriate and requires so many bits per character on the average:

$$\bar{\ell} = 0.5 \times 1 + 2 \times (0.25 \times 2) = 1.5$$

Huffman coding which is bottom up outperforms the top-down Shannon-Fano coding.

https://en.wikipedia.org/wiki/Huffman_coding

https://en.wikipedia.org/wiki/Shannon%E2%80%93Fano_coding

and is described in most books on computer science algorithms (Cormen et al., 2009).

The information entropy provides the limit to lossless data compression by defining the relationship between the occurrence probability of information and redundancy. The entropy H of a random variable X with possible discrete values

$\{x_i, 1 \leq i \leq n\}$ and *probability mass function* - pmf $P(X)$ according to Shannon's theorem is given as:

$$\mathcal{H}(X) = - \sum_{i=1}^n P(x_i) \log_2 P(x_i).$$

For the above example:

$$\mathcal{H}(a) = -\log_2(0.5) = 1,$$

$$\mathcal{H}(b) = \mathcal{H}(c) = -\log_2(0.25) = 2$$

$$\mathcal{H}(a, b, c) = \mathcal{H}(a) \times 0.5 + \mathcal{H}(b) \times 0.25 + \mathcal{H}(c) \times 0.25 = 1.5 \text{ bits}$$

In the given example the Huffman encodings achieve the minimal encoding length, which is rarely the case.

2.13.3 Lempel-Ziv - LZ encoding

LZ77 and LZ78 or LZ1 and LZ2 are both dictionary coders described by Abraham Lempel and Jacob (Yaakov) Ziv (both Technion U. emeriti).

https://en.wikipedia.org/wiki/LZ77_and_LZ78

The two lossless data compression algorithms form the basis for many variations including LZW, LZSS, LZMA and several ubiquitous compression schemes, including GIF and the DEFLATE algorithm used in PNG and ZIP.

LZ77 encodes and decodes from a sliding window over previously seen characters. Decompression must always start at the beginning of the input. LZ78 decompression could allow random access to the input if the entire dictionary is known in advance, however, in practice the dictionary is created during encoding and decoding by creating a new phrase whenever a token is output.

LZ78 was extended by Terry Welch as Lempel-Ziv-Welch - LZW in (Welch, 1984), which provides efficient data compression (Solomon, 2014).

<https://en.wikipedia.org/wiki/Lempel%E2%80%93Ziv%E2%80%93Welch>

<https://www.geeksforgeeks.org/lzw-lempel-ziv-welch-compression-technique/>

LZ4r due to Rui Chen and Lihao Xu at Wayne State U. is a new high speed data compression algorithm in LZ77 family for general data. It achieves better compression ratio than comparable ones in the family, such as LZ4, Snappy and LZO.

<http://nisl.wayne.edu/Papers/Tech/lz4r-fms2020-poster.pdf> It was presented at the Flash Memory summit in 2020.

<https://www.flashmemorysummit.com>

Snappy is widely used in Google projects like Bigtable, MapReduce and in compressing data for Google's internal *remote Procedure Call* - RPC systems. Snappy for fast compression uses, zstd for read-intensive storage, and brotli if a user is waiting for the data or low-read-throughput storage compression.

Regarding Snappy versus LZ4 while LZ4 performs in general better than Snappy, in both compression ratio and speed for all common data compression corpora, such as silesia, enwik8, calgary, canterbury, Google has decided that its engineers can be more productive doing other work than an infrastructure level Snappy-to-LZ4 migration, but if the choice now between develop Snappy or just use LZ4, LZ4 would be choice.

[https://en.wikipedia.org/wiki/LZ4_\(compression_algorithm\)](https://en.wikipedia.org/wiki/LZ4_(compression_algorithm))

Zstandard was designed to give a compression ratio comparable to that of the DEFLATE algorithm (developed in 1991 and used in the original ZIP and gzip programs), but faster, especially for decompression. Tunable with compression levels ranging from negative 5 (fastest)[5] to 22 (slowest in compression speed, but best compression ratio). The zstd package includes parallel (multi-threaded) implementations of both compression and decompression.

<https://en.wikipedia.org/wiki/Zstandard>

A comparison of various compression methods is reproduced in Table 2.23.

https://openzfs.org/w/images/b/b3/03-OpenZFS_2017_-_ZStandard_in_ZFS.pdf

Note that LZ4 is fastest in data compression and decompression, but slightly outperformed by Snappy as far as the data compression ratio is concerned.

BCM is a high-performance file compressor that utilizes advanced context modeling techniques to achieve a very high compression ratio. BCM combines *Burroughs-Wheeler-Transform* - BWT (Burrows and Wheeler, 1994) and context prediction.

https://en.wikipedia.org/wiki/Burrows%E2%80%93Wheeler_transform

<https://github.com/encode84/bcm>

ENWIK8 dataset is based on Wikipedia and is typically used to measure a model's ability to compress data. It is shown in Table 2.24 that BCM-9 outperforms other methods.

TABLE 2.23 Ratio versus speed comparison for several data compression methods at 4.0 MHz.

Compressor	Ratio	Compress MB/s	Decompress MB/s
Zstd 1.1.3 (–1)	2.877	430	1110
Zlib 1.2.8 (–1)	2.743	110	400
Brotli 0.5.2	2.708	400	430
Quicklz 1.5.0	2.238	550	710
Lzo1x 2.0.9	2.108	650	830
Lz4 1.7.5	2.101	720	3600
Snappy 1.1.3	2.091	500	1650
Lzf 3.6	2.077	400	860

TABLE 2.24 Compressing the ENWOK8 dataset using different methods.

Method	Compressed Size
Original	100,000,000 bytes
GZIP-9	36,445,248 bytes
BZIP2-9	29,008,758 bytes
7-Zip Ultra	24,864,804 bytes
BCM-9	20,789,667 bytes

2.13.4 Arithmetic coding

Arithmetic coding was developed by Jorma Rissanen (rip) and Glen Langdon (UCSC, rip) at IBM/ARC (Rissanen and Jr., 1979), (Jr., 1984), (Rissanen and Mohiuddin, 1989).

https://en.wikipedia.org/wiki/Arithmetic_coding

Consider three symbols a, b, c each with probabilities 0.2, 0.5, and 0.3, respectively. The symbols are assigned ranges $a = (0,0.2)$, $b = (0.2,0.7)$, $c = (0.7,1.0)$. To code the three symbol message “bac”, the initial range for “b” is $(0.2,0.7)$, for “ba” it is transformed to $(0.2, 0.2 + 0.5 \times 0.2) = (0.2,0.3)$. and for “bac” it is further transformed to $(0.3 - 0.1 \times 0.3, 0.3) = (0.27,0.3)$.

It is adequate to specify a single number inside the interval for decoding. For (a,b,c) with initial intervals $(0.0,0.2,0.7,1.0)$ given 0.49 the first letter is “b”. Given “b” we have the following intervals for the second letter $(0.2,0.3,0.55,0.7)$. Since 0.49 is in the second interval therefore the second interval is also “b”. The intervals for the third letter are $(0.3,0.35,0.475,0.55)$. since 0.49 is in the third interval then the third letter is “c”. An infinite sequence of letters can be produced in this manner, so an end of string character is required.

2.13.5 Miscellaneous topics on data compression

A proposal to cascade LZ compression with arithmetic coding is presented by Yehoshua Perl et al. at NJIT in (Perl et al., 1991).

Balakrishna Iyer (rip) and David Wilhite considered LZ encoding in addition to *Run-Length Encoding - RLE*, which replaces sequences in which the same data value is repeated, with a count and the data value (Iyer and Wilhite, 1994). Experimentation with IBM’s IMS DBMS showed that LZ yield space savings of 83.08% versus 24.21% for RLE alone.

Sort order preservation is a desirable property for compressed data and was considered by Ali Zandi, Balakrishna Iyer (rip), and Glen Langdon (UCSC, rip) at IBM ARC in (Zandi et al., 1993), so that search can be conducted on compressed data.

A short list of books on data compression in chronological order is as follows.

James W. Storer. “Data Compression Methods and Theory” (Storer, 1988).

David Solomon. “The Complete Reference Paperback” (Solomon, 2014).

Khalid Sayood. “Introduction to Data Compression” (Sayood, 2017).

2.13.6 Universal Resource Locator - URL shortener

Long url's take more effort to type and may exceed the line-width in books, requiring them to be broken into two or more lines. URL shorteners create a super short, unique URL that will redirect to the specific website of choice.

<https://blog.hootsuite.com/what-are-url-shorteners/>

The shorter URL will typically include the shortener site's address, plus a combination of random letters, e.g.,

<https://animalplanet.com/tv-shows/puppy-bowl/full-episodes/puppy-bowl-xvi>

into the Hootsuite Ow.ly shortener, it would generate ow.ly/uK2f50AJDI6, by cutting down 48 letters.

URL shorteners work by creating a redirect to your long URL. Entering a URL into your internet browser sends an HTTP request to the web server to select a specific website.

There are several good reasons to use a URL shortener when sharing a link.

1. Suspicious looking, long links.
2. Linking to a specific page buried deep on a website, or are using *Urchin Tracking Module - UTM* parameters to track your visitors, you might find yourself wrestling with a lengthy URL.
UTM parameters are simply tags that you add to a URL, following a "?". When someone clicks on a URL with UTM parameters, those tags are sent back to your Google Analytics for tracking.
<https://neilpatel.com/blog/the-ultimate-guide-to-using-utm-parameters/>
3. Oversized URLs look clunky in social posts, shared via text, pasted into an email.
4. Long URLs are difficult to type and the user is bound to make a mistake unless copy and paste can be used.
5. Short urls make text look neat.
6. A custom URL shortener lets you brand your links and helps spread brand awareness.
7. Link shorteners allow tracking performance and offer tracking metrics, e.g., who clicked your link, where, and when - information that will help you calculate the campaign's ROI. To easily track source traffic, create a variety of shortlinks using different UTM parameters and combine this with other analytics tools - like Google Analytics and you're setting yourself up for marketing success.
8. Short URLs allow you to get the most out of character limits on social media Twitter has a strict character limit of 280, so keeping posts concise is key. Short URLs help avoid *Too Long; Did not Read - TL;DR* syndrome.
9. Shorter links are useful for *Instant Messaging - IM* or email, where longer links may be hard to read, or disrupted altogether by line breaks.
10. Shorter links are more memorable, snappier, and easy to remember.
11. A wedding announcement trying to collect submissions on a Google Form, which directs followers to <https://docs.google.com/forms/d/1ezpZnYSA08L3nlsLzgjKToA76qKYgFpKzW5FMaKLoxI> can be substituted with custom shortlink like bit.ly/submitRW, which is easy to understand and recall.
12. Recommend shortlink on business cards, print ads, or signs. If people can actually type into their browser with just a few keystrokes, you're more likely to earn some traffic.

Ten URL shorteners plus Google

ow.ly uses the <https://hootsuite.com/pages/owly> platform.

T.co [Twitter.com](https://twitter.com) features a built-in URL shortener that automatically reduces any long URL down to 23 characters.

bitly.com bitly.com

tinyurl tinyurl.com/

Rebrandly rebrandly.com

UseHyperlink <https://usehyperlink.com/>

tiny.cc <https://tiny.cc/>

bit.do <https://bit.do/>

clickmeter <https://www.clickmeter.com/>

shorte.st <https://shorte.st/>

cutt.ly <https://cutt.ly/>

clkim <https://clkim.com/>

goo.gl <https://firebase.google.com/products/dynamic-links/>

Link shorteners have recently been in the news. The *Federation of American Scientists - FAS.org* report accused Russian “hackers” from Sputnik Mundo of using link-shortening services such as bit.ly and Twitter to spread “vaccine-related malware across Latin America,”

<https://sputniknews.com/science/202012221081525862-cia-linked-nonprofit-accuses-russian-news-of-cookie-campaign-amid-sputnik-v-vaccine-rollout-effort/>

According to Twitter and Bitly however. “Having a link shortener protects users from malicious sites that engage in spreading malware, phishing attacks, and other harmful activity.”

2.13.7 Image compression

Companies providing image hosting save storage by compressing photos. A list of image compression methods is as follows.

<https://makeawebsitehub.com/free-photo-hosting/>

Fractal compression is a lossy compression method for digital images, which is best suited for textures and natural images and takes advantage of the fact that parts of an image often resemble other parts. Fractal algorithms convert these parts into “fractal codes”, which are used to recreate the encoded image. For satellite images satisfactory results have been attained by a 170:1 compression ratio.

https://en.wikipedia.org/wiki/Fractal_compression

Fractal compression first applies an affine transformation, any combination of a rotation, scale change, skew or translation. Contractive transformation makes things smaller and brings objects closer. The *Iterated Function System* repeating the transformations.

<https://www.i-programmer.info/babbages-bag/482-fractal-image-compression.html>

The search for self-similarities makes fractal compression quite expensive, so that it cannot be applied in real-time. The patent is held by Canadian Michael Barnsley and Alan Sloan (Barnsley and Sloan, 1989).

Joint Photographic Experts Group - JPEG: JPEG is a standard for compressing photographic images using the *Discrete Cosine Transform - DCT*.

https://en.wikipedia.org/wiki/Discrete_cosine_transform

<https://en.wikipedia.org/wiki/JPEG>

JPEG 2000: JPEG 2000 is JPEG’s successor format using wavelets.

https://en.wikipedia.org/wiki/JPEG_2000

Wavelet compression: Wavelet compression is well suited for image, video, and audio compression. Notable implementations are JPEG 2000, DjVu and ECW for still images, CineForm, and the BBC’s Dirac.

https://en.wikipedia.org/wiki/Wavelet_transform

JBIG2: JBIG2 is an image compression standard for bilevel images, which is suitable for both lossless and lossy compression. Only the bitmap of a representative character needs to be coded instead of coding the bitmaps of each occurrence of the same character.

<https://en.wikipedia.org/wiki/JBIG2>

Cartesian Perceptual Compression - CPC: It was designed for high compression of black-and-white raster document images for archival scans. It is used by [JSTOR.com](https://www.jstor.org/) for archiving academic journals.

https://en.wikipedia.org/wiki/Cartesian_Perceptual_Compression

DjVu: DjVu is a file format for scanned documents. DjVu was developed at AT&T Research by Yan LeCun et al. in 1996. It generates files that are smaller than PDF files. <https://en.wikipedia.org/wiki/DjVu>

PDF and DjVu delivering high resolution documents in very different ways: PDF primarily encodes graphics and text as vectorized data, whereas DjVu primarily encodes them as pixmap images. Bitmap uses one bit per pixel while pixmap uses several bits. The two methods are different in that PDF places the burden of rendering the document on the reader, whereas DjVu places that burden on the creator.

<https://en.wikipedia.org/wiki/Bitmap>

ICER: Used by the Mars Rovers is related to JPEG 2000 in its use of wavelets. Mars Perseverance Rover reached Mars on 2/18/21 after a seven month journey and is sending back images.

<https://www.nature.com/articles/d41586-021-00432-1>

Incidentally Russia had two rover missions to the Moon (see pp. 4-11).

https://s3.amazonaws.com/planetary/assets/tpr/pdf/tpr-1990-v10n4_200424_191843.pdf

but Russian missions to Mars were canceled as the Soviet Union collapsed, but the “Russian” Mars Rover was tested by its builder Alexander Kemurdjian in Death Valley, CA (see pp. 12-14).

https://en.wikipedia.org/wiki/Lunokhod_programme

<https://www.planetary.org/profiles/alexander-kemurdjian>

https://s3.naws.com/planetary/assets/tpr/pdf/tpr-1993-v13n1_200424_191730.pdf

Although *Amazon Web Services* - AWS does not publicly provide the details of S3’s technical design, Amazon S3 manages data with an object storage architecture, which aims to provide scalability, high availability, and low latency with 99.999999999

<https://www.amazon.com/s3/>

Simply protecting against a complete HDD failure such as with RAID, does not protect against the gradual failure of the bits stored on magnetic media. The combination of widely distributed erasure coded data (say with an 18/8 coding policy) and data scrubbing technology achieves 15 nines of data durability.

HAM: Hardware compression of color information is used in Amiga computers.

2.13.8 Video/audio compression

DVD specification provided a storage capacity of 4.7 GB for a single-layered, and 8.5 GB for a dual-layered, both single-sided discs.

Flash: Details are given at Adobe Flash Video File Format Specification Version 10.1.

https://en.wikipedia.org/wiki/Flash_Video

Moving Picture Experts Group - MPEG: MPEG-1 was set up in 1993 for moving pictures and audio for digital storage media at up to 1.5 Mbit/s. There is also Motion JPEG, MPEG-2, MPEG-4 H.261, followed by H.263, H.264/MPEG4-AVC.

<https://en.wikipedia.org/wiki/H.261>

Most DVDs which are about 4 GB store movies in MPEG-2 format (aka H.262). MPEG2 compresses the video by discarding the information in portions of the image that do not change from one frame to another, and saving only the portions of the image where new information is added. The MPEG4 compression mechanism is a bit more complicated compared to that of MPEG2, as it needs better algorithms to scan and determine which pixels can be discarded, to reduce the data even further. In summary:

1. MPEG2 is the encoding method for DVDs, while MPEG4 is the choice for portable devices and online use.
2. MPEG2 encoded video files are much bigger compared to MPEG4.
3. MPEG2 requires a lot more bandwidth for streaming compared to MPEG4.
4. MPEG2 produces better video quality compared to MPEG4.
5. MPEG2’s compression is much simpler compared to MPEG4.

A 5 TB Barracuda Pro from Seagate with *Conventional Magnetic Recording* - CMR and therefore hold a 1000 movies. The Barracuda Pro features a 7,200 RPM spindle speed with 256 MB cache, offering up to 250 MB/s in sustained sequential transfer rates

<https://www.seagate.com/internal-hard-drives/cmr-smr-list/>

The maximum required bitrate is 19.39/38.8 Mb/s for broadcast/cable TV. Given that 38.8 Mb/s is less than 5 MB/s a single disk can sustain 50 streams, Movies should be stored multiple times for higher availability and to spread disk load. Popular movies can be cached in DRAM,

Ogg Theora: This is a variable-bitrate, DCT - Discrete Cosine Transform based video compression scheme.

<https://en.wikipedia.org/wiki/Theora>

Sorenson video codec: Sorenson Media specializing in video encoding in 1995, licensed the Sorenson Vision codec to Apple for Quicktime 3.00 software. It was later replaced by H.264 format.

https://en.wikipedia.org/wiki/H.264/MPEG-4_AVC

The company was acquired in 2019 by TV ratings company Nielson.

https://en.wikipedia.org/wiki/Sorenson_Media

https://en.wikipedia.org/wiki/Nielsen_ratings

VC-1: Developed by Windows Media Video 9 by Microsoft in 2003.

<https://en.wikipedia.org/wiki/VC-1>

Music compression, *Advanced Audio Compression* - AAC - used by Apple Inc., ADPCM, ATRAC, Dolby AC-3, MP2, MP3, Musepack, Ogg, WMA (from Microsoft).

https://en.wikipedia.org/wiki/Data_compression#Audio

Speech coding compression has aspects which are discussed under Audio compression.

There is a trade-off between loss of audio quality and transmission or storage size depending on the application. For example, one 640 MB CD holds approximately one hour of uncompressed high fidelity music, less than 2 hours of music compressed losslessly, or 7 hours of music compressed in the MP3 format. Same 640 MB CD can typically store about 200 hours of speech.

2.14 Main memory data compression

Software-based main memory compression was first considered by Fred Douglass and John Ousterhout (Stanford) at UCB in (Douglass, 1993). The idea was to take advantage of faster CPUs to save memory space by keeping pages in main memory in compressed form. Compression allowed higher MPLs and fewer page faults.

Compressed caching was proposed at U. Texas at Austin in (Wilson et al., 1999). The scheme uses part of the available main memory to hold pages in compressed form. This effectively adds a new level to the virtual memory hierarchy, which bridges the gap between uncompressed RAM and disk. While previous studies had not shown a consistent benefit from the use of compressed virtual memory, a significant reduction of paging costs was observed in this study, becoming increasingly attractive at faster CPU speeds. However an additional coprocessor is used for data compression, but also data encryption. Aside from a novel compression algorithm it is adaptively determined how much memory should be compressed by keeping track of recent program behavior. This solves the problem of different programs, or phases within the same program, performing best for different amounts of compressed memory.

The size of the compressed region can be static or varied dynamically. Adaptive main memory compression proposed by researchers at ETH Zurich in (Tuduce and Gross, 2005), which is based on an application's execution profile to find uncompressed and compressed regions into which an application's working set fits. If such a size does not exist or compression hurts performance, compression is not applied.

A project to design and manufacture compression in main memory called Data Crusher was initiated by IBM engineers in Rochester, MN in early 1990s, which was home of IBM's *Advanced System - AS/400* computers.

https://www-03.ibm.com/ibm/history/exhibits/rochesterrochester_4010.html

IBM became the largest employer in Rochester, MN in 1948 due to Thomas J. Watson Jr's friendship with a veteran of WW II who was a resident of Rochester. The workforce at Rochester exceeded 8000 in 1980, but at this time the IBM presence in Rochester is all but gone.

<https://krocnews.com/from-rapid-growth-to-slow-decline-history-of-ibm-rochester/>

The data compression project at Rochester was taken over by IBM Research in Hawthorne, NY. The argument that sequential compression/decompression does not meet speed requirements was used to utilize four compressors/decompressors operating jointly to deal with a 1024 byte cache line as 256 B blocks.

DRAM prices in 1995 were high, but by the time the *Memory eXpansion Technology - MXT* project at IBM Research was completed in 2001 DRAM prices had dropped significantly.

<https://jcmr.net/memoryprice.htm>

There was also the fact that IBM customers demanded second-sourcing, i.e., the compression chip be available from a source other than IBM. x86 chips from AMD was another instance double sourcing. Several papers describing were published in a special issue of IBM J. of R&D (Abali et al., 2001), (Franszsek and Robinson, 2001), (Tremaine et al., 2001a).

IBM and ServerWorks collaborated in developing Pinnacle, a low-cost, single-chip memory controller (or north bridge) using MXT. Pinnacle was Intel Pentium III and Xeon bus-compatible chip that also did data compression (Tremaine et al., 2001b). Customers usually require second sourcing. In the electronics industry, a second source is a company that is licensed to manufacture and sell components originally designed by another company, the first source. The most notable case is AMD acting as a second source for Intel.

https://en.wikipedia.org/wiki/Second_source

The second source, if any, is not known for Pinnacle.

https://www.academia.edu/6375920/Pinnacle_IBM_MXT_in_a_Memory_Controller_Chip

The reader is referred to the diagrams in this paper which provides detailed overall organization and chip layouts in Figures 2 and 3.

Main memory management with data compression is complicated, because of the fact that compressed cache lines have variable lengths (Franszsek and Robinson, 1998), (Franszsek and Robinson, 2001). The system under consideration with a data compressor/decompressor (DC) and *Compressed RAM - C-RAM* is specified as:

CPU(s) ↔ L1(s) ↔ L2(s) ↔ L3 ↔ DC ↔ C-RAM

C-RAM's cache lines are dynamically decompressed to handle cache misses at L3. Compression/decompression, address translation, and memory management are performed by hardware, which has implications for the directory structure and storage allocation designs. The design of a C-RAM with the following properties is considered.

1. Logically, the memory consists of a collection of randomly accessible lines of fixed size L .
2. Internally, the i^{th} line is stored in a compressed format as $L(i) \leq L$ bytes. $L(i)$ may vary as the line is modified. Uncompressed lines are stored when $L(i) = L$.
3. The minimum access size (granule) is $g = 32$ bytes.
4. Memory accesses invoke a translation between a logical line address and an internal address. This correspondence is stored in a directory D contained in M . Translation, fetching and memory management within the C-RAM are carried out by a memory controller rather than by OS software.

The L3 line size is set to 1024 bytes, since a larger line size yields a better compression ratio, but the latency is also larger. Three techniques for managing a memory of the above type are:

1. Organizing to be a linear space, where variable-length intervals are allocated/deallocated;
Organizing as a collection of blocks of possibly multiple sizes, where space for a variable-length object is allocated as an integral number of such blocks;
2. Organizing as a collection of blocks, but permitting a variable amount of space to be allocated within a block. The first two methods require garbage collection due to space fragmentation or unfavorable distributions of block sizes. These processes can be time consuming and complex, and are not well-suited to applications where fast response times and simple processing are required.

Compressed lines are allocated as some integral number of blocks, with leftover pieces, termed fragments, from possibly multiple lines sharing an additional block. A line fragment shares a block with line fragments drawn from a cohort of other lines. The smaller the cohorts, the fewer the memory operations required to store a line, but the larger the potential fragmentation. This is also true for the number of line fragments permitted to share a block. A principal observation is that the size of cohorts and number of lines allowed to share a block can be quite small with low resulting fragmentation. The result is that the technique is of low complexity, and is suitable for implementation in hardware.

Virtual addresses are translated to real addresses by means of page tables, which vary depending on details of the processor architecture and OS. A real address corresponds to the location in the directory entry for the line. An important feature is that information pertaining to the blocks holding a compressed L3 line is stored at a location which can be determined, e.g. using a known offset, from the real address. Compressed lines that do not fit entirely within directory entries are stored using one or more fixed size 256 B blocks.

On an L3 cache miss, the memory controller and decompression hardware find the blocks allocated to store the compressed line and dynamically decompresses it to handle the miss. Similarly, when a new or modified line is stored, the blocks currently allocated to the line are made free, the line is compressed, and then stored in the C-RAM by allocating the required number of blocks. As this is done by hardware, under normal operation the fact that main memory is compressed is transparent to the OS. However, since memory is compressed, the logical total size of main memory can grow or shrink dynamically, and the OS must be modified to handle this.

Given 1 KB lines and 256 byte blocks, supposing each line compresses to 1:1024 bytes uniformly, the expected compressed line size is 512.5 bytes, i.e., the compression is almost exactly 50%. However, using some number of full blocks to store each line, since 1-4 blocks are equally likely, the expected number of blocks to store a line is 2.5, which has a compression of 62.5%, much worse than 50%. Making block sizes significantly smaller will result in a dramatic increase in directory size. Another approach to reducing storage fragmentation is to combine two or more fragments, that is, the "left-over" pieces in the last blocks used to store compressed lines, into single blocks. In order to make fragment combining feasible using a hardware-based memory controller, and in particular to provide a guarantee on the maximum latency, some constraint must be made on the sets of lines for which fragment combining is allowed. We will call a set of lines for which fragment combining is allowed a *cohort*. In order to have a small upper bound on the time required for directory scans, ideally the size of cohorts should be small.

There is the question of how many fragments can be combined in a block. From a hardware point of view, the simplest case is to allow only two fragments per block. However, if significant gains would be realized by allowing 3-way or greater combining, the increased complexity might be acceptable. Also, given a fragment and the sizes and locations of other fragments in the cohort for which combining is possible, which fragments should be chosen? Two options are to use *First-Fit - FF* or *Best-Fit* methods.

Optimal block size in MXT

The tradeoff involved in choosing a block size is that the larger the block size, the more wasted space there is due to fragmentation; the smaller the block size, the more space is required for the directory.

In the case of a dynamic-directory structure, the average compression should be set to a value which is expected to be close to that which will occur in practice; the resulting block size will be approximately optimal when the C-RAM is operating at this degree of compression.

Given a memory of total size M bytes, this memory contains a directory which, for each object gives the location of the blocks used to store the object. The directory consists of all data in the memory other than blocks. A simplistic “directory structure” is the following:

1. External tables provide a pointer to the first block used to store a given object;
2. Each block is extended with a pointer field, which is used to point to the next block used to store the object or null. The collection of all pointers is considered to be part of the directory.

Let q be the number of bytes used by the directory per block stored in the memory. For the previous simple design example, determining q is straightforward: q is just the length (in bytes) of a block pointer.

The tradeoff in finding an optimal value for the block size b is as follows: As b increases, there is more wasted space in fragments, since on a per-object basis each fragment is on the average a fraction $1/n$ of a full block; as b decreases, there is a larger total number of blocks, and therefore more overhead in directory space, since each block requires q bytes of directory space. An estimate of the optimal value of b can be found using a continuous approximation.

1. The total number of blocks in the memory is $M/(b + q)$.
2. The total number of objects T that can be stored in the memory as a function of b is given by the following:

$$T(b) = \frac{M/(b + q)}{p/b + 1/n} = \frac{Mnb}{(b + q)(np + b)}$$

It follows from this expression that the optimal value of b is given by $b_{opt} = \sqrt{npq}$. Setting $p = 512B$ (for 50% 1024 B line compression) and setting n to 4 (which is typical for two-way combining using first-fit or best-fit with $g = 32$ B blocks leads to $b_{opt} = \sqrt{4 \times 512 \times 9} = 128$.

Required memory for each MB of logical memory, given in KBs is given in Table 2.25. Comparison of data compression and net compression is given in Table 2.26. It took several of emails rounds to obtain the IBM copyright for the two tables.
/newline <https://www-01.ibm.com/contact/submissions/extsub.nsf/index>

TABLE 2.25 Required memory for each MB of logical memory, given in KBs, Table 9 in (Fraszcek and Robinson, 1998).

Block-size	AIX (active)	NT (boot)	NT (active)
256	436.9	581.2	459.4
128	406.8	555.3	427.3
64	423.4	576.0	446.0

[h]

TABLE 2.26 Comparison of data compression and net compression (%), including directory overhead, Table 10 in (Fraszcek and Robinson, 1998).

Block size	AIX (active)		NT (boot)		NT (active)	
	Data	Net	Data	Net	Data	Net
256	41.1	42.7	55.2	56.8	43.3	44.9
128	36.6	39.7	51.1	54.2	38.6	41.7
64	35.1	41.3	50.5	56.3	37.3	43.6

MXT has been extended to IBM Power servers under the name *Active Memory Expansion - AME* (Blaner et al., 2013). This is because of the heightened focus on computer security and IBM Power servers spending an increasing number of cycles on crypto functions.

Gennady Pekhimenko (U. Toronto) did his PhD at CMU where he proposes *Linearly Compressed Pages - LCPs* (Pekhimenko et al., 2013; Pekhimenko, 2016). It is argued that prior proposals for addressing performance degradation problem are either costly or energy inefficient. By compressing all cache lines within a page to the same size LCP avoids the performance degradation problem without requiring costly or energy-inefficient hardware. Evaluations using benchmarks show that the approach can significantly increase the effective memory capacity by 69% on average. The new mechanism considerably reduces the memory bandwidth requirements of most of the evaluated benchmarks (by 24% on average), and improves overall performance (by 6.1%/13.9%/10.7% compared to a baseline system that does not employ main memory compression. LCP also decreases energy consumed by the main memory subsystem by 9.5%.

The Compresso pragmatic main memory compression was developed at U. Texas Austin (Choukse et al., 2018) based on the arguments that larger memory capacity and higher memory bandwidth are required for better performance and energy efficiency and that hardware memory compression provides a solution without increasing system cost. It is claimed that current memory compression solutions face the following two challenges.

1. Keeping memory compressed requires additional memory accesses, possibly in the critical path affecting performance.
2. Changing the OS to take advantage of the increased capacity, and to handle incompressible data, which delays deployment. Similarly to MXT

Compresso is a hardware memory compression architecture that minimizes memory overheads due to compression, with no changes to the OS. New optimizations are proposed to reduce additional memory movement achieving a 1.85x compression for main memory on average, with a 24%/27% speedup over for single/multi-core systems.

IBM MXT was mostly non-intrusive, but required a few OS changes. Since the compression granularity was 1 KB, a memory read would result in 1 KB worth of data being decompressed and transferred. To address this, a large 32 MB L3 cache with a 1 KB line size was introduced. These features would significantly hinder the performance and energy efficiency of MXT in today's systems. Summary of related work based on Table V in (Choukse et al., 2018) is given in Table 2.27.

TABLE 2.27 Summary of related work based on Table V in (Choukse et al., 2018).

Related work	OS transparent	Hardware changes	Compression granularity	Cacheline packing	Data movement opt.	References
IBM-MXT	Partially	LLC, MC	1KB	N/A	N/A	(Tremaine et al., 2001a)
RMC	No	BST, MC	64B	LinePack	Light	(Ekman and Stenström, 2005)
LCP	No	TLBs, MC	64B	LCP	No	(Pekhimenko et al., 2013)
Buri	Partially	MC	64B	LCP	No	(Zhao et al., 2015)
DMC	Partially	MC	64B/1KB	LCP/N/A	No	(Kim et al., 2017)
Compresso	Yes	MC	64B	LinePack	Yes	(Choukse et al., 2018)

IBM POWER7 is the first POWER server to include on-chip hardware accelerators for symmetric shared key and asymmetric public key cryptography to support *Rivest Shamir Adleman - RSA* encryption,

https://simple.wikipedia.org/wiki/RSA_algorithm

and elliptic key cryptography and *Application Managed Encryption - AME*. Supported are *Advanced Encryption Standard - AES*, *Secure Hash Algorithm - SHA*, and *Message Digest 5 - MD5* algorithm as bulk cryptographic functions.

Data compression in z-series computers

zEnterprise Data Compression - zEDC Express is complementary to the hardware compression on the System z processor chip. Compression is invoked using the compression call (CMPSC) coprocessor instruction and a dictionary to compress data.

The *CSRCMPSC* macro Compresses data and Expands previously-compressed data.

https://www.ibm.com/support/knowledgecenter/SSLTBW_2.2.0/com.ibm.zos.v2r2.ieaa700/iea3a7_CSRCMPSC_Compress_and_expand_data.htm

The zEDC Express uses the DEFLATE format, is dictionary-less because the dictionary is included in the data stream. Hardware compression is optimized for short records, such as database rows, while zEDC is optimized for large sequential files. Deflate is a lossless data compression file format that uses a combination of *Lempel-Ziv-Storer-Szymanski - LZSS* and Huffman coding.

<https://en.wikipedia.org/wiki/Deflate>

CMPSC versus zEDC Express are compared below from different viewpoints.

Where available: *CMPSC*: On chip in every IBM eServer zSeries *zEDC*: PCIe adapter with IBM zEC12, zBC12 later models.

Maturity: *CMPSC* Decades of use by access methods and IBM DB2. *zEDC* Industry standard with decades of software support.

Where run: *CMPSC* Work performed jointly by the CPU and coprocessor *zEDC* Work performed by the PCIe adapter.

Format: *CMPSC* Proprietary compression format *zEDC* Standards-compliant (RFC1951).

Objects compressed: *CMPSC*: Rows in a database. *zEDC*: Large sequential data. QSAM and BSAM online sequential data, objects stored in a database.

Standard of data *CMPSC*: System z only. *zEDC*: Cross platform data exchange.

Users: *CMPSC*: VSAM for better disk use, DB2 for lower memory usage the majority of DB2 users currently compress their row, DFSMSHsm/ds. *zEDC*: QSAM/BSAM for better disk use and batch elapsed time improvement, IBM SMF for increased availability and online storage reduction, Java for high throughput standard compression java.util.zip, *Data Facility Storage Management Subsystem hierarchical storage manager - DFSMSHsm* and DFSMS data set services (DFSMSdss), Encryption Facility for z/OS for better industry data exchange, IBM Sterling Connect:Direct for z/OS for better throughput and link usage, Independent software vendor (ISV) support for increased client value.

Coming data compression encryption in z14 and z15

With the launch of IBM z14™ in 2017, IBM announced that its hardware included additional encryption features (CPACF) embedded in the processor chip, reducing the increase in CPU utilization to the low single digits; on average, around 2.6%.

This number was derived by measuring the MIPS overhead for 23 different clients when encrypting their z/OS datasets and represents the average of all values measured. The ability to encrypt data, both at rest and in flight, for a very low cost, was great news for customers concerned about data security.

A side effect of encrypting the data on z14 is that it renders storage-level compression ineffective, because once you've encrypted data, it is not readily compressed. A zEnterprise® Data Compression (zEDC) card could be used to compress the data before encryption, but that is an added expense.

On IBM z15™, with the Integrated Accelerator for zEnterprise Data Compression, the industry standard compression used by zEDC is now built into the z15 core, very much like encryption is with CPACF. Now customers can have the best of both worlds with compression and encryption (in that order) right on the processor cores. Encryption becomes even less expensive, since after compression, there is much less data to encrypt

<https://www.ibm.com/downloads/cas/AM1PYZBB>

According to Lihao Xu (Wayne State U.) it is well known to perform compression before encryption, not only to reduce size, but also to increase security degree of encryption, since compressed data has less redundancy. But in many cloud applications, they don't compress data stored on their system for enterprise users, because most of the users encrypt their data before uploading to storage, which makes sense. Azure alike platforms (Google, AWS, Box, Dropbox) don't control encryption part (and decryption part, as they don't know encryption key), but they do have strong incentive to compress them to reduce their storage costs. So this is a very practical problem if solved. Two references on this topic are (Johnson et al., 2004), (Klinc et al., 2012).

2.15 Data deduplication in storage systems

Data deduplication is known under different names: intelligent compression, *Single-Instance Storage - SIS*, data reduction and capacity optimized storage.

https://en.wikipedia.org/wiki/Data_deduplication

File compression methods such as Lempel-Ziv, which remove short repeated strings result in a limited compression ratio, while data deduplication which acts at a higher level tends to result in more significant reduction in storage space requirements.

Data deduplication is an important step during daily backup, since it saves storage by not archiving duplicate files. Deltas for slightly modified files may be saved instead of a whole modified file. Deduplication can be carried out at the level of blocks of data. Deduplication at a finer granularity may identify duplicates, which do not exist at a coarser level, as in the case of a file with minor modifications. Breaking data into standardized units is called chunking, which may be format aware or format agnostic. Chunks may be compared by hashing, byte by byte comparison, or delta differencing. A modified chunk can be reconstructed by applying deltas to a baseline chunk.

Delta differencing is an incremental approach to backup that only backs up blocks that have changed after the first full backup. The differences are recorded in files called deltas. The process involves examining a backup file set and locating the blocks that have changed since the last backup period.

<https://searchstorage.techtarget.com/definition/delta-differencing>

An email server in a large company may end up holding numerous copies of the same report that may have been to the employees of a company as an attachment. Storage space and network bandwidth could have been saved simply by posting the article on a web site and sending the *universal resource locator - url*, which serves as a pointer to the report and is obviously much shorter than the report. The url can be shortened further via url shortening, which was discussed in Section 2.13.6.

Early systems used disk to tape backup, but *Disk to Disk - D2D* backup is preferred, since it speeds up duplicate detection and backup and restore operations. “Tapes suck” is the name of a book by former CEO of Data Domain an early data deduplication company (Slootman, 2011).

Snowflake headed by Frank Slootman IPOed in Sept. 2020 and its share process doubled the same day. It was the highest valued software IPO ever and at the end of the day they were valued more than General Motors, Dell, and Uber combined. https://en.wikipedia.org/wiki/Snowflake_Inc.

Snowflake is discussed in some detail in Chapter 8.

Data deduplication can be client-, server-, or storage device-based, which determines who bears the processing load. Server-based deduplication allows cross correlation among multiple clients, while the third option allows deduplication of data in place. The first two approaches assist in preserving network bandwidth.

There is a distinction between inline and postprocess deduplication, depending on whether deduplication occurs before or after writing to disk.

Data deduplication has the advantage of reducing the storage footprint and *Total Cost of Ownership - TCO* by allowing more data to be stored, or allowing data to be stored on fewer disks thus reducing energy costs and reducing the amount of data sent across the network.

2.15.1 Data deduplication at Microsoft

An early instance of applying offline deduplication termed *Single Instance Storage - SIS* was carried out in the context of backing up Windows images at a remote install facility (Bolosky et al., 2000). A background process scans files checking for duplicates. A variant of copy-on-write, named copy-on-close, is used for protecting updates to shared files.

2.15.2 The Venti prototype at Bell Labs/Lucent

Venti was an inline or on-the-fly deduplication scheme for *Content Addressable Storage - CAS*, which was developed by Sean Quinlan (Google) and Sean Dorward (Google) at Bell Labs (Quinlan and Dorward, 2002). Venti uses a unique hash of a block’s contents as its identifier to achieve a write-once policy. Venti removes duplicate fixed-size data blocks by comparing fingerprints based on cryptographic hash functions applied to determine if two data chunks are identical.

Data deduplication uses hashing to determine a fingerprint, which is compared against others to determine whether a file or a block has already been stored. Consider two files consisting of five blocks each, which differ in one block. File level deduplication would require storing ten blocks, while block level deduplication requires only six blocks, i.e., storing the block that differs twice. Hashing is discussed in Knuth’s third volume (Knuth, 2005) and (Konheim, 2010).

National Institute for Standards and Technology - NIST (formerly *National Bureau of Standards - NBS*) developed the *Secure Hash Algorithm - SHA1* returning a 20 B hash value, which is more reliable than the earlier *Message Digest - MD-5*, which returns 16 bytes.

https://en.wikipedia.org/wiki/Secure_Hash_Algorithms

<https://en.wikipedia.org/wiki/MD5>

Given n files and b bit hash values the probability p that there will be one or more collision is:

$$p \leq \frac{n(n-1)}{2} \times \frac{1}{2^b}.$$

Venti writes files after compression with an LZ77 algorithm onto a log and the fingerprint in the index points to the block. The index using a disk-resident hashtable is shown in Figure 5 in (Quinlan and Dorward, 2002). The index is divided into fixed-sized buckets stored as a single disk block. Each bucket contains the index map for a small section of the fingerprint space. A hash function is used to map fingerprints to index buckets in a roughly uniform manner, and then the bucket is

examined using binary search. Provisioned with sufficient buckets the index hash table will be relatively empty and bucket overflows will be extremely rare. If a bucket does overflow, the extra entries are placed in an adjacent bucket. This structure is simple and efficient, requiring one disk access to locate a block in most cases.

Deduplication fails if identical data blocks appear shifted. This problem is overcome by an algorithm which computes the fingerprint as the string is shifted (Manber, 1994). The example, the fingerprint for a 50 byte substring is computed as follows ($t_1, t_2, \dots, t_{50}$) as follows:

$$F_1 = \left(\sum_{i=1}^{50} t_i p^{50-i} \right) \text{mod}(M),$$

where $p(i)$ is a prime number and $M = 2^{30}$. Computing fingerprints is started with the last $k = 8$ bits set to zero. The summation is best computed using Horner's method.

https://en.wikipedia.org/wiki/Horner%27s_method

The fingerprint F_2 starting with the second position can be computed as:

$$F_2 = (pF_1 + T_{51} - t_1 p^{49}) \text{mod}(M)$$

2.15.3 Data Domain deduplication

Data Domain was cofounded by Kai Li, Ben Zhu, and Brian Biles, while Li was on a sabbatical from Princeton U. in California in 2001. Hugo R. Patterson a former student of Garth Gibson joined as Chief Architect three months after the initial funding. Data Domain was acquired by EMC in 2011, which was itself acquired by Dell Corp. in 2015 to form Dell/EMC.

https://en.wikipedia.org/wiki/Dell EMC_Data_Domain

In 2004 Data Domain offered a disk-based substitute for the tape automation backup market plus deduplication reducing the volume of data. It is obviously more efficient to compare incoming large datasets against data stored on disk rather than tape, because of the random access capability associated with disks as noted in (Slootman, 2011).

Data Domain's implementation of deduplication is described in (Zhu et al., 2008) A main memory resident Summary Vector using a Bloom filter determines if a new segment is a duplicate or not (Bloom, 1970).

https://en.wikipedia.org/wiki/Bloom_filter

- Init() initializes an m -bit vector $[0, m - 1]$ to zero.
- Insert(fingerprint) uses independent hashing function $h_i(x)$, $1 \leq i \leq k$ to set appropriate bits to one.
- Lookup(fingerprint) uses the same hash function to determine if all the bits are set to one, in which case the segment is a duplicate with high probability.

After n elements are hashed the probability that a specific bit is zero is

$$\left(1 - \frac{1}{m}\right)^{kn} = e^{-kn/m}$$

The probability of a false positive is then:

$$\left(1 - \left(1 - \frac{1}{m}\right)^{kn}\right)^k = \left(1 - e^{-kn/m}\right)^k$$

In addition index fragments are cached, where each fragment indexes a set of chunks found together in the input.

2.15.4 Datrium

Datrium CEO Tim Page claims that his team has developed the only platform that provides deduplication, compression, erasure coding and data encryption – for data in transit and at rest. Windsor Hsu who had left IBM ARC to join Data Domain, joined Datrium as CTO, after Data Domain was purchased by Dell/EMC. Presentations by Datrium staff, including those by Hsu are provided at the company web site are no longer available. Hsu has since left Datrium to join the Alibaba Group as its Chief Storage Architect.

https://en.wikipedia.org/wiki/Alibaba_Group

Datrium claims to provide effective deduplication without affecting the performance of high-IOPS applications. The technology is based on a content-addressed architecture that is said to optimize data at rest, on premises or in the cloud, and data in motion among data centers and the public cloud.

According to Hugo Patterson Datrium is the first to build a system that combines industry-leading performance and scalability for primary workloads, built-in long-term retention of capacity-optimized application snapshots on affordable media, and bandwidth-optimized data mobility.

Datrium deduplication technology handles primary storage random IO and enables snapshot storage on less expensive media, such as disks and Amazon's S3 in the cloud, so that multitudes of snapshots (replicas) can be retained. Hence the claim that it frees enterprises from having to do backups. Datrium which offered *Disaster Recovery as a Service - DRaaS* was acquired by VMware on 7/15/2020 and www.datrium.com is no longer active.

2.15.5 Summary of a major survey on data deduplication

A comprehensive survey of data deduplication in (Paulo and Pereira, 2014) is summarized below.

Granularity: Whole file, fixed (block level), variable block sizes.

Locality:

Temporal locality: Chunks appear several times in a short period of time.

Spatial locality: Chunks appearing in some order once appear in the same order again.

No locality.

Timing: Inline deduplication occurs before data is written persistently, while offline deduplication is applied to data that is stored persistently.

Indexing: Three options are: Full, Partial, Sparse. Most systems summarize content by hashing before building the index as in the case of SIS - Single Instance Storage in Windows 2000 (Bolosky et al., 2000) and Venti (Quinlan and Dorward, 2002). Comparing fingerprints to find similar chunks presents a scalability problem and requires a large index. One solution is coalescing a group of similarity fingerprints into superfingerprints. Two matching superfingerprints indicate high resemblance between the chunks, thus scaling the index to a larger number of chunks as noted by Andrei Broder et al. in (Broder et al., 1997).

Sparse indexing for deduplication uses sampling and exploits the locality within backup streams is the method used by HP (Lillibridge et al., 2009). The goal is to resolve the chunk lookup disk bottleneck. A group of stored chunks are mapped by a single entry at the index, e.g., several chunks are grouped into a segment, which is identified by a similarity rather than an identity signature. A partial index indexes a subset of unique stored chunks (Guo and Efstathopoulos, 2011). An LRU policy is used to determine which signatures should be held in RAM.

Technique: Aliasing for data deduplication or chunk-based deduplication omits exact duplicates by using an indirection layer. Delta encoding eliminates duplicate content among two similar, but not fully identical chunks. Only the base chunk is fully stored, while the deltas or diffs to restore other chunks are stored separately.

Scope: Centralized, Global, Local. Global deduplicates across all nodes, as opposed to local.

Data deduplication performance can be improved by first applying a weak hash function followed by a strong hash function to attain higher confidence. Some systems ascertain that two chunks are identical by a bit by bit comparison.

Data deduplication metrics listed below, are given here:

http://wikibon.org/wiki/v/Wikibon_Data_De-duplication_Performance_Tables

Inline Backup Speed (TB/hr) – for solutions that have a single inline step.

Post-process Ingest Speed (TB/hr) – for solutions that ingest at a higher speed first, and then deduplicate.

Post-Process Deduplication Speed (TB/hr).

Daily Backup Capacity (TB/Day).

Raw Usable Capacity (TB) – the figure before protection, formatting and other subtractions, and does not include the impact of de-duplication.

2.16 Up and coming data deduplication companies

Atempo provides protection in several categories.

https://www.atempo.com/wp-content/uploads/2021/03/DATASHEET_en_TinaHyperstream_2021.pdf

Disaster Recovery for Servers

Lina - All-in-One Backup Protection for Your Workstations and Laptops

Physical and Virtual Server Protection Maximizing IT Infrastructure, Minimizing Downtime

Automated physical and virtual server protection backup to disk, tape or deduplicated storage. Rapid data recovery for minimum downtime for all virtual machines (VMs).

Miria - HPC and Large Data Management - Lustre & GPFS Storing, protecting, sharing and accessing data from multiple filesystems (GPFS, Lustre, NFS, CIFS ...) and multiple storages (disk, cloud, tape, object...) have created a number of challenges that it handles.

Migration for Hybrid NAS and File Storages

Synchronization for Hybrid NAS Data and File Storages File Syncing and Movement for Petabyte-Scale Storages Isilon, NetApp, Qumulo, DDN and More

Backup And Archive Media Assets

Barracuda Backup is a data recovery, restoration, and deduplication product from Barracuda Networks. It features data center backup support for email protection, network & application security, and general data protection.

<https://www.barracuda.com>

Clear Analytics is a business intelligence solution that enables nontechnical end users to perform analytics by leveraging existing knowledge of Excel coupled with a built in query builder.

<https://www.clearanalyticsbi.com/>

Dell/EMC Avamar is a hardware and software data backup and deduplication product. It provides protection and recovery through a complete software and hardware solution when paired with Dell EMC Data Domain for virtual environments, remote offices, enterprise apps, NAS servers, and desktops and laptops.

<https://www.delltechnologies.com/en-us/data-protection/data-protection-suite/avamar-data-protection-software.htm#>

Druva druva.com offers a SaaS platform for data protection across data centers, cloud applications, and endpoints. Securely backup and recover data with the scale and simplicity of the public cloud, and only pay for what you use, reducing your overall TCO. Enforce data governance and extract more intelligence with search and analytics to unlock the full value of your data.

<https://en.wikipedia.org/wiki/Druva>

Druva Phoenix is scalable, all-in-one backup, disaster recovery, archival and analytics. Simplifies data protection, dramatically reduce costs, and improves data visibility for today's complex information environments.

DupeCatcher is a free de-duplication tool from the maker of Cloudingo. Advertises itself as stopping Salesforce duplicates before they become a problem.

<https://www.dupecatcher.com/>

Exagrid EX series. All vendors using traditional inline approaches to data deduplication split data into 4 KB to 10 KB groups of byte "blocks." The backup software uses 64 KB to 128 KB fixed-length blocks. The challenge is that for every 10 TB of backup data, the tracking or "hash table" was one billion blocks. The hash table grows so large that it needs to be housed in a single front-end controller with additional disk shelves, an approach referred to as "scale-up." As a result, only capacity is added as data grows and since no additional bandwidth or processing resources are added, the backup window grows in length as data volumes increase. At some point, the backup window becomes too long and a new front-end controller is required, known as a disruptive and expensive "forklift upgrade." Byte-level deduplication allows for system scalability, known as "scale-out," but it requires an understanding of the format of every backup application, which limits the list of supported backup applications. An alternative approach is to use hyper-converged scale-out nodes with block-level deduplication. However, this approach is still burdened with the large hash table look-ups and, therefore, requires expensive flash storage to increase performance, which increases the price of the hardware.

ExaGrid uses zone-level deduplication, which breaks data into larger "zones" and then compares at the byte level. This approach allows for the best of all worlds.

1. The tracking table is 1,000th the size of the block-level approach and allows for full appliances in a scale-out solution. As data grows, all resources are added: processor, memory, and bandwidth as well as disk.
2. (2) The zone approach is backup application agnostic, allowing ExaGrid to support virtually any backup application.
3. ExaGrid's approach does not maintain a very large, ever-growing hash table and, therefore, avoids the need for expensive flash to accelerate hash table look-ups. ExaGrid's approach keeps the cost of the hardware low.

In summary, block-level deduplication drives a scale-up architecture that only adds disk as data grows, or with a scale-out node approach requires expensive flash storage to perform large hash table look-ups. Both approaches slow down backups and/or increase cost. ExaGrid's zone-level deduplication includes full server appliances in a scale-out hyper-converged solution without large hash table look-ups, which results in the fastest backup and restore performance at the

lowest price. ExaGrid's approach also supports a wide range of backup application support. This zone-level approach provides the best of all worlds: ExaGrid can work with any backup application and can easily scale, resulting in a fixed-length backup window regardless of data growth.

<https://exagrid.com/exagrid-products/data-deduplication/>

Falconstor Virtual Tape Library - VTL is the eponymous data center backup and recovery.

Falconstor FDS is a data management / deduplication solution.

Fujitsu Data Deduplication Appliance is a data deduplication solution.

Hitachi Protection Platform Backup Appliance was formerly offered by Sepaton with DeltaStor data deduplication software.

HPE StoreOnce is a backup and recovery hardware solution from HPE, which provides disk-based backup, deduplication, and long-term storage. StoreOnce offerings can support virtual and cloud environments for small business, mid-size and large organizations.

KLD Processing is a deduplication engine, designed to aid discovery.

<https://www.kldiscovery.com/>

Microsoft Data Protection Manager - DPM Appliance: In early 2015 Microsoft added data deduplication capability to their System Center 2012 R2 DPM.

<https://docs.microsoft.com/en-us/system-center/dpm/dpm-overview?view=sc-dpm-2019>

NEC HYDRAsTOR HS3-510 deduplication appliance for smaller businesses. HS *Virtual Appliance* - VA is a data deduplication solution for virtualized environments and HS8-5000 which is scale-out grid storage and deduplication appliance. HYDRAsTOR was also a scalable secondary storage.

NetApp FAS series systems offer a storage array system for enterprises. Using deduplication, data compression, and data compaction to increase storage efficiency.

<https://docs.netapp.com/ontap-9/index.jsp?topic=%2Fcom.netapp.doc.dot-cm-vsmg%2FGUID-9C88C1A6-990A-4826-83F8-0C8EAD6C3613.html>

Nexsan Data Deduplication Appliance from Imation offers data deduplication in their storage solutions.

Nexsan DeDupe SG is a disk based backup and data deduplication option, from Imation company Nexsan. Nexsan's new DeDupe SG 2.0, together with FalconStor's *File-interface Deduplication System - FDS* 2.0, combines increased availability, performance, and deduplication features with Nexsan's AutoMAID power reduction technology.

<https://www.crn.com/news/storage/222301227/nexsan-falconstor-join-forces-on-newest-backup-appliance.htm>

ObjectEngine formerly StorReduce was acquired by Pure Storage and relaunched in 2019 with the new name. It is a software defined, scale-out deduplication service in the cloud, which reduces object storage costs, speeds up transfer, and enables data reuse.

PowerProtect DD (formerly Dell EMC Data Domain) is a next-generation appliance replacing Dell EMC Data Domain. It is a suite of hardware appliances used for data protection, backup, storage and deduplication. PowerProtect appliance offerings are cloud-enabled and vary by organization size.

<https://www.delltechnologies.com/en-us/data-protection/powerprotect-dd-series.htm#scroll=off>

ProtecTier Data Deduplication Framework is a real-time compression appliance, which utilizes the Lempel-Ziv algorithm by IBM N series Storage environments provides up to 80% data compression, which increases storage efficiency with no performance degradation (IBM Corp. Int'l Tech. Support Org., 2016).

<http://www.redbooks.ibm.com/abstracts/sg247953.html?Open>

<https://www.ibm.com/support/knowledgecenter/en/nseries/com.ibm.nseries.hardware.doc/ipg/sof.htm>

Data compression can be applied after deduplication, since deduplication does not work well on compressed data.

Quantum DXi Series is Quantum's deduplication solution.

Quest DR Series is a backup and recovery solution, featuring data deduplication.

RingLead DMS Duplicate Prevention enforces perimeter protection around B2B databases to stop dirty data in real time, at the source, and consistently maintain and improve the health of data.

Spectra Logic's nTier is a data storage product line offers data deduplication.

StarDQ is a real time solution for cleansing, deduping, and enriching enterprise data. By integrating StarDQ Solution, organizations can cleanse, match and unify data across multiple data sources and data domains. According to the vendor, the goal is to ensure that data is a strategic, trustworthy, valuable asset, which can benefit organizations by enhancing decision making power, reducing expenses and facilitating seamless customer interaction.

tye.io is a data cleansing software for *Small and Medium Businesses - SMBs* that cleans data directly in a current technology stack. The vendor states that there's no training or migration required, no extra work involved. tye merges lists and cleans data for sales, *Customer relationship Management - CRM*, and marketing.

Veritas NetBackup Appliance is a storage and deduplication solution.

A comprehensive study of the past, present, and future of data deduplication by Dan Feng et al. at Huazhong U. with assistance from Hong Jiang at U. Texas at Arlington and Fred Douglass at EMC based on (Xia et al., 2016) in given Table 2.28.

TABLE 2.28 Data reduction ratios of large-scale real-world data sets.

Institutions	Workloads	Size	Data reduction schemes	Ratio
Microsoft	857 desktops (Meyer and Bolosky, 2011)	162 TB	file level dep = dupe internal users chunk level deduplication internal users (8K) file level deduplication across users chunk level deduplication across users	≈ 21% ≈ 42% ≈ 50% ≈ 68%
ditto	15 MS file servers (El-Shimi et al., 2012)	6.8 TB	file level deduplication chunk level deduplication (64 kB)	0-16% 15-90%
EMC	10,000 backup systems (Wallace et al., 2012)	700 TB	chunk level dedupe (8 KB)	69-93%
ditto	6 large backup datasets (Shilane et al., 2012)	33 TB	chunk level dedupe (8 kB) deduplication + delta compression deduplication + delta compression + GZIP	85-97% 66-82% 74-87%
Meinz U.	4 HPC data centers (Meister et al., 2012)	1212 TB	File level dedupe chunk level dedupe (8 KB)	5-10% 20-30%

State-of-the-art approaches to content-defined chunking for data deduplication are: Low Bandwidth network File System - LBFS (Muthitacharoen et al., 2001), Content based chunking algorithm (Eshghi and Tang, 2005), Regression chunking (El-Shimi et al., 2012), Fingerdiff mote Differential Compression - RDC (Teodosiu et al., 2006), SampleByte (Agarwal et al., 2010), Sdelta (Xia et al., 2014), Asymmetric extremum content defined chunking algorithm (Zhang et al., 2015), Leap-based CDC (Yu et al., 2015), Bimodal chunking (Kruus et al., 2010), Subchunk (Romanski et al., 2011), FBC (Lu et al., 2010), MHD (Zhou and Wen, 2013).

2.17 Storage research at IBM's Almaden Research Center in 1990s

The storage group at IBM/ARC in 1990s was led by Jai Menon, who had done his PhD at *Ohio State U. - OSU* with David K. Hsiao on performance evaluation of *Data Base Machines - DBMs*. On a one year assignment from IBM Research (April 1993-94) I collaborated with Menon in developing analytic and simulation models to compare the performance of IBM's RAID5 under development with NCR's 8298.

Jai Menon later became the director of IBM's storage and servers group, before joining EMC/Dell Corp., and later superconvergence pioneer Cloudistics, and now Fungible.

<https://www.fungible.com/leadership/>

As an aside, the background of my collaboration with Menon is as follows. Before attending the *Int'l Conf. on Distributed Computing Systems - ICDCS'92* to be held in Yokohama, Japan I contacted Masaru Kitsuregawa at U. Tokyo volunteering to give a talk while in Japan. I also asked him if I can bring along Jai Menon and Prof. Walter Burkhard from USCD, who were both attending the conference.

My talk at U. Tokyo covered concurrency control methods (Franaszek et al., 1992), while Jai Menon presented the performance analysis of RAID5 disk arrays (Menon, 1994). After his talk I told Jai privately that RAID5 performance in rebuild mode can be analyzed exactly using the *Vacationing Server Model - VSM* for M/G/1 queues (Takagi, 1991), (Thomasian, 2018a). I had applied VSM to the performance analysis of threshold scheduling of readers and writers in (Thomasian and Nicola, 1993).

I first learned about VSM via a homework problem in Leonard Kleinrock's course on queueing theory at UCLA in Fall 1973. His classnotes were later published as *Queueing Systems Vol. I & II* (Kleinrock, 1975, 1976) and VSM became problem 5.23 in Chapter 5 in Volume I.

In Feb. 1993 Menon's department head Robert Morris (Bell Labs, IBM, YongLooLin U., Singapore) contacted my department head Steve Lavenberg to ask him for a one year transfer to collaborate with Menon. Robert Morris was familiar

with my publications as discussed below. As program chair of ICDCS'93 held in Pittsburgh, PA he had asked me to handle the Performance Evaluation track.

While attending the ACM SIGMETRICS'84 conference held in Cambridge, MA Wing Shing Wong (Bell Labs, *Chinese U. Hong Kong - CUHK*) gave me a paper he had written with Robert Morris using analysis to compare the performance of *Optimistic Concurrency Control - OCC* and static locking. The latter analysis was similar to my earlier analysis in (Thomasian and Ryu, 1983).

USC PhD student In Kyung Ryu and I collaborated to generalize Wing's paper in two directions (Thomasian and Ryu, 1985), (Ryu and Thomasian, 1987).

1. a more realistic on-demand data access rather than accessing objects at once,
2. Allowing committed txns to kill/abort conflicting txns to alleviate wasted processing.

While the kill option alleviates wasted processing for memory resident data, further simulation studies showed that the die option is preferable if the data resides on disk and access invariance via buffer retention prevails (Franaszek et al., 1992).

Being assigned Dick Mattson's former office at IBM/ARC I had a productive year developing the analysis of the RAID5 disk array utilizing Allicat disk drives under development at IBM's *Storage Systems Division - SSD* (Thomasian and Menon, 1994).

John Lui a PhD student of Richard Muntz at UCLA who had coauthored a paper on RAID5 rebuild options at VLDB'90 (Muntz and Lui, 1990b) had joined IBM *Storage Systems Division - SSD* to develop RAID software. John stopped by my office at ARC to tell me that he is leaving IBM to join the *Chinese U. Hong Kong - CUHK*. In a chance encounter with IBM Fellow Michael Blasgen I was told that IBM's RAID effort has been unsuccessful in terms of both its hardware and software.

<https://www.linkedin.com/in/michaelblasgen/>

Computerworld's March 7, 1994 has an article on IBM's delay to ship its RAID5 leaving EMC free to gain marketshare. Table 2.29 is reproduced from this article.

<https://books.google.com/books?id=t5FTatxeQsYC&pg=PA16&lpg=PA16&dq=ibm+raid5+project+in+1990s#v=onepage&q&f=false>

In fact IBM shipped 9337 RAID5 products as early as 1992 according to the 1994 entry below for AS/400 and RS/6000 computers.

https://www.ibm.com/ibm/history/exhibits/storage/storage_chrono20.html

TABLE 2.29 RAID roll call - EMC has only host RAID subsystems in general availability.

Vendor	Product	Date shipped	Status
EMC Corp.	Symmetrix RAID1 supported	Late 1990	About 3000 units installed
IBM	Two RAID5 models planned	June 1994	Delayed September
STK	Iceberg	Late 1992	June
Amdahl	RAID-5 supplied by Encore Corp.	Third qtr	Letter of intent signed

IBM's failure to develop the RAID controller codenamed Hager is mentioned in (Chen et al., 1994b), but this was not mentioned to me when I presented performance results from my study to IBM/SSD management at San Jose and Tucson in March 1994. The delay in shipping the IBM RAID5 product for mainframes was already mentioned in Computerworld 3/1994.

It seems to that the delay in shipping the product was due to emulation of *Extended Count Key Data - ECKD* DASD on disks with fixed block architecture, which had been overcome by EMC in the context of its Symmetrix disk array. *Count Key Data - CKD* was the original S/360 track organization shown below.

https://en.wikipedia.org/wiki/Count_key_data#/media/File:CKD_Track_Format.png

The key is an optional field and in the case of blocking is the last key on the block, which is also the largest since records are sorted. An example given in (Jacob et al., 2008) postulates 30B for the count field plus 10B for gaps, to each record incurs a 50 B overhead.

ECKD disk format is emulated by *Disk Array Controller - DAC* on *Fixed Block Architecture - FBA* format disks with 512 byte sectors. This is because key mainframe software was written to operate on 3390 disks with ECKD format. The 512 byte sector was identified as an impediment to disk capacity in 1998 which was removed by the Advanced Format

4096-byte sectors. This increased the efficiency of the data surface area 5-13%, while increasing the strength of the ECC which in turn allowed higher capacity. The format standardized in 2005 by an industry consortium was incorporated after 2011 in new products from all HDD manufacturers.

https://en.wikipedia.org/wiki/Advanced_Format Larger 4096 byte sectors for fixed block disks allow stronger error correction algorithms for higher storage densities are called *Advanced Format Drives - AFDs* by IDEMA.

https://en.wikipedia.org/wiki/Advanced_Format

<https://www.seagate.com/tech-insights/advanced-format-4k-sector-hard-drives-master-ti/>

According to Figure 2.1 in (IBM Corp. Int'l Tech. Support Org., 1997) shows the track organization for an IBM 3310 disk with a *Fixed Block Architecture - FBA*. Each track has 33 sectors, the last one is a spare. The track starts with 14 byte zeroes, one Synch byte, one Flag byte (defective?), three bytes for addresses (sector, track, cylinder), 2 bytes for *Cyclic Redundancy Check - CRC*,

https://en.wikipedia.org/wiki/Cyclic_redundancy_check

16B (bytes) of zeroes, 1 synch byte, 512B for sector, 4B ECC, followed by 24B for next sector. In effect 28B per sector.

The *Error Correcting Code - ECC* is utilized to locate and correct correctable errors in each sector. The CRC is utilized to determine whether errors exist in the sector that may not have been identified and corrected using the ECC code. When such errors are identified, then an uncorrectable sector signal is generated.

In older drives, every sector is preceded by an ID field, which contains the ID information related to the immediately following data sector. Such information includes the sector's physical address (cylinder, head, and sector number); a flag to indicate the sector is split by servo; whether the sector is defective; and if defective, where the data is relocated to. The ID is like a mini sector, with its own sync field, ECC or CRC, and gaps. Unlike user's data sectors, its contents have a fixed format, and it is not accessible by the user. The presence of sector IDs represents a significant consumption of storage space on a drive's surface. The ID fields can occupy up to 10% of a track, while the gaps can take up another 5%. Hence, a track's capacity can increase by about 18% if the ID field can be dispensed with (Jacob et al., 2008). This idea was patented by IBM researchers in (Hetzler and Kabelac, 1993).

It is also possible that IBM managers at SSD were not anxious in introducing a RAID5 product, replacing IBM's expensive DASD with high profit margins with the less expensive RAID array, which could easily be developed by other companies.

Having failed in developing a RAID product IBM in 1997 became a reseller of StorageTek's Iceberg array renaming it *RAMAC Virtual Array - RVA* (IBM Corp. Int'l Tech. Support Org., 1997). Jai Menon performance analysis of *Log-Structured Array - LSA* in (Menon, 1995) may have been motivated by IBM's interest in buying StorageTek or at least reselling Iceberg. Instead StorageTek was acquired by Sun Microsystems in September 2005 for \$4.1 Billion, which was acquired by Oracle for \$7.4 Billion.

IBM had 70% of the storage revenue in 1992, but its storage revenue has dropped drastically since then. Things got worse during the tenure of Ambuj Goyal, who seems to have written his own profile.

<https://dora.iitk.ac.in/dora/ambuj-goyal-profile>

The following article written in conjunction with the IBM Edge Conf. in July 2013 discusses Goyal's view about storage that it should be all flash.

<https://www.datacenterdynamics.com/news/ibm-systems-and-technology-group-strategy-revealed/>

His statement about the need for disks does not apply to backup and archival storage, and hyperscalars as noted by Eric Brewer (UCB, Google) in (Brewer et al., 2016). Hyperscalars are companies like Google, Microsoft, Facebook, Alibaba and Amazon that rely on large disk arrays for their business and keep HDDs alive.

<https://blocksandfiles.com/2019/10/07/ssds-will-kill-hard-disks-in-enterprise/>

https://www.eetimes.com/document.asp?doc_id=1334865#

<https://www.plesk.com/blog/business-industry/hyperscalers-growth-cloud-infrastructure/>

https://en.wikipedia.org/wiki/Hyperscale_computing

It was announced in January 2014 that IBM is selling its Intel x-86 based server business to Lenovo for \$2.3 Billion.

<https://www.cnet.com/news/ibm-sells-its-rx86-server-business-to-lenovo-for-2-3-billion/>

In 2005 IBM sold its PC business to Lenovo and this led to major layoffs in its Boca Raton, FL plant. The humble beginnings of Lenovo founded in Beijing in 1988 which was initially named Legend are provided below.

<https://en.wikipedia.org/wiki/Lenovo>

At this time Lenovo's laptop is ranked #1.

<https://www.forbesmiddleeast.com/innovation/technology/lenovo-ranks-as-worlds-best-laptop-brand>

IBM's DS4800 can be configured as a RAID 0, 1, 3, 5, 10.

<http://www-07.ibm.com/storage/includes/content/disk/ds4000/ds4800/>

<http://www-07.ibm.com/storage/includes/content/disk/ds4000/ds4800/pdf/TSD01054USEN.pdf>

RAID3 which has also been referred to as byte level striping requires synchronized disk spinning.

<https://www.techopedia.com/definition/17274/raid-3>

In 2007 IBM bought the Israeli startup xiv storage founded by Moshe Yanai, who was previously responsible for the successful Symmetrix mirrored disk array at EMC. He spent his own \$3 Million to fund the company, which IBM bought for \$300 Million.

https://en.wikipedia.org/wiki/Moshe_Yanai

https://en.wikipedia.org/wiki/IBM_XIV_Storage_System

<http://www.redbooks.ibm.com/redbooks/pdfs/sg247659.pdf>

Number of CPU cores (min/max) - 108/180

Fibre Channel rates - 8 Gbps

iSCSI rates - 1 Gbps or 10 Gbps

Fibre Channel ports (min/max) - 16 / 24

1 or 10 Gbps iSCSI ports (min / max) - 14 or 8 / 22 or 12

Capacity per drive (nearline SAS) - 4 TB or 6 TB SSDs

Number of disk drives (min/max) - 108/180

Effective capacity per system (min / max)* - 354 TB / 970 TB

* The effective capacity values in this table are based on a 2x compression ratio.

Memory (min/max) - 864 GB/1,440 GB

Flash caching (min/max) - 7.2 TB/12 TB

Given other options with Flash memories IBM does not plan further extensions of xiv storage.

https://www.theregister.co.uk/2017/01/20/xiv_facing_extinction/

Infinidat introduced in 2011 is Moshe Yanai's latest endeavor.

<https://en.wikipedia.org/wiki/Infinidat>

According to Infinidat hyperscale storage is best achieved with disk drives, since they are cheaper and more reliable than flash memories.

<https://blocksandfiles.com/2019/11/07/infinidat-disk-drives-scale-better-than-all-flash-arrays/>

The following discussion of Infinidat follows the Gartner Group

<https://www.gartner.com/doc/reprints?id=1-IOI9TL9M&ct=190917&st=sb>

Infinidat continues to make innovative improvements to its hybrid array product portfolio with the goal of cost-effectively addressing enterprise storage performance requirements without transitioning customers to solid-state arrays. The economics of Infinidat's platform position it as a viable consolidation target for large data centers. Although, InfiniBox supports internal NVMe SSDs and is NVMe-oF- and SCM-compatible, a majority of its I/O activity is addressed by its relatively large DRAM footprint along with SSDs used as a cache. Hence, investing in these technologies is made optional. In May 2019, Infinidat announced its Elastic Data Fabric vision that aims to ensure seamless data mobility between the edge, the core and the cloud. Customers can also leverage InfiniVerse, a SaaS-based platform that supports predictive analytics and can centrally monitor and manage these systems. Infinidat also offers Capacity on Demand (COD) and InfiniBox FLX programs. In addition to Infinidat's storage appliances, enterprises can use its Neutrix Cloud managed storage service for primary storage and DR, located in close proximity to leading public cloud providers, such as AWS, Microsoft Azure and *Google Cloud Platform - GCP*.

Strengths: Infinidat scores high for customer satisfaction due to its high level of resiliency, stable microcode, ease of management, overall performance and all-inclusive pricing model. InfiniBox integrates with a broad range of hypervisors, containers, public cloud and backup vendors. InfiniBox facilitates a zero *Recovery Point Objective - RPO* disaster recovery solution that does not require three-site topologies, instead leveraging the InfiniSync appliance on the primary site, a rugged system that can replicate data to the secondary site via a cellular network.

Cautions Infinidat's nontraditional architecture may cause some I&O leaders to devote resources and time to validate performance claims with a comprehensive *Proof Of Concept - POC*. InfiniBox does not support deduplication, thus potentially increasing storage consumption for certain applications, such as virtual desktop environments. Infinidat does not offer an SSA solution, which may limit its consideration for customers demanding an SSA-only solution.

Vast Data

VAST Data funded in 2016 bets on flash technology.

<https://vastdata.com/>

<https://blocksandfiles.com/2018/11/20/is-vast-data-bullshit-or-a-magic-bullet/>

https://en.wikipedia.org/wiki/VAST_Data

VAST Data relies on a single QLC flash tier for its capacity storage in 2018 while NetApp intends to add QLC flash to its all-flash arrays next year. Pure Storage announced the QLC-using FlashArray//C in August this year. All three suppliers believe that a QLC capacity tier provides faster data access than nearline 7,200 RPM disk drives.

In 2018, TLC laptop flash, Intel's 3D XPoint and commodity NVMe fabrics will combine to provide an opportunity for a next generation file and object storage architecture that can beat the cost/capacity curve of spinning disks. The following advantages are listed.

1. Using next-gen data reduction tools that have never before been implemented in storage, VAST can dramatically reduce the footprint of even pre-compressed datasets up to 80% or more.
2. Large datasets will no longer be bound by the performance limits of archival storage, data is instantly accessible at Flash speeds.
3. The solution is up to 10X more dense than HDD-based solutions.
4. Customers can finally abandon the traditional HW refresh cycle created by HDDs – keep systems on the floor for 10 years or more.

<https://horizontechnology.com/news/data-center-hardware-refresh-cycles/>

IBM Spectrum storage

IBM has many storage products which are discussed below.

https://en.wikipedia.org/wiki/IBM_Storage

IBM Spectrum storage consists of several novel and renamed products.

https://en.wikipedia.org/wiki/IBM_Storage#IBM_Spectrum_Storage

IBM Spectrum Accelerate is based on the IBM XIV, a high-end disk storage system. IBM Spectrum Accelerate and XIV run the same base software stack.

IBM Spectrum Scale is software-defined storage for cloud and analytics. It was formerly known as *Global Parallel File Systems - GPFS* before IBM re-branded all storage products in 2015.

IBM Spectrum Virtualize is a block storage virtualization system. Because the IBM Storwize V7000 uses *SAN Volume Controller - SVC* code, it can also be used to perform storage virtualization in exactly the same way as SVC. Since mid-2012 it offers real time compression with no performance impact, saving up to 80% of disk utilization. SVC can be configured on a Stretched Cluster Mode, with automatic failover between two datacenters and can have SSD that can be used by EasyTier software to perform sub-LUN automatic tiering.

https://en.wikipedia.org/wiki/IBM_Storwize_family

IBM Spectrum Control provides infrastructure management for virtualized, cloud and software-defined storage.

IBM Spectrum Protect is a progression of the Tivoli Storage Management product that allows users to run any application designed for disk files against tape data without concern for the fact that the data is physically stored on tape.

IBM's storage group in 9/1/20 has 18 permanent members and several listed projects.

https://researcher.watson.ibm.com/researcher/view_group.php?id=4203

IBM plans to consolidate its storage lines.

<https://www.networkworld.com/article/3527219/ibm-consolidates-storage-products-under-a-single-brand.html>

2.18 Cleversafe and its information dispersal technology

In 2015 IBM acquired Chicago based Cleversafe Inc. for \$1.5 B. Cleversafe storage scales to exabytes - EB= 10^{18} B making IBM a leader in object storage market. The *Dispersed Storage Network - DSN* provides clients with low-cost, large scale active archives and unstructured data content stores.

https://www.theregister.co.uk/2016/12/14/ibms_clever_and_safe_acquisition_gets_it_object_storage_leadership/

The origins of the company and several related articles are provided here.

<https://en.wikipedia.org/wiki/Cleversafe>

Cleversafe utilizes the Michael Rubin's *Information Dispersal Algorithm - IDA* (Rabin, 1989). IDA places pieces of a dataset in different storage locations providing a method for storing data in different disks for redundancy purposes. IDA was adopted by Guillermo Alvarez et al. in the DATUM disk array (Alvarez et al., 1997). A file is split into k chunks and encoded as $n > k$ chunks. Any k chunks out of these n chunks can be used to reconstruct the original file.

http://wikibon.org/wiki/v/3_Tenants_of_Security_and_the_Role_of_Information_Dispersal

A new dispersal scheme called *All or Nothing Transform - AONT*-based on *Reed Solomon - RS* coding is described in (Resch and Plank, 2011). AONT-RS which forms the backbone of commercial dispersed storage system blends AONT with RS to achieve high security with low computational and storage costs.

In the case of Cleversafe data to be stored is broken into w bit words. The generator/dispersal matrix G with elements $g_{i,j}$ has n rows and k columns. This matrix is multiplied by a k -element vector D with elements d_j (called the data or message) to yield an n -element vector C with elements c_i for the codeword. Each element of the codeword is stored on a different storage node.

$$\begin{pmatrix} g_{0,0} & g_{0,1} & g_{0,2} & \dots \\ g_{1,0} & g_{1,1} & g_{1,2} & \dots \\ g_{2,0} & g_{2,1} & g_{2,2} & \dots \\ \vdots & \vdots & \vdots & \vdots \end{pmatrix} \times \begin{pmatrix} d_0 \\ d_1 \\ d_2 \\ \vdots \end{pmatrix} = \begin{pmatrix} c_0 \\ c_1 \\ c_2 \\ \vdots \end{pmatrix}$$

The dispersal matrix is constructed so that all combinations of k rows yield invertible matrices, which yields a technique to reconstruct D from any k surviving elements of the codeword: each row of G corresponds to the calculation of a codeword element. Created is a new $k \times k$ matrix A from the rows of G that correspond to the k surviving elements. We invert A and multiply A^{-1} by the surviving elements to yield D . The construction of G guarantees that A is invertible.

Adi Shamir's secret sharing algorithm encodes w bits of data in d_0 . The remaining elements of D are randomly chosen w -bit words. The matrix G is a Vandermonde matrix with $g_{i,j} = i^j$, which guarantees that any k rows are invertible so long as $n \leq 2^w$ (Shamir, 1979). Shamir's algorithm applied to a b -byte file requires nb bytes so that encoding requires $O(knb)$ XOR and multiplication operations.

The security guarantees of Shamir's algorithm are very strong – even with an infinite amount of computing power. Unless an attacker has possession of k words he cannot determine anything about the initial data. This is done without the necessity of storing encryption keys.

Rabin's IDA weakens the security, but improves both storage efficiency and performance. Each element of D now contains a word of data. Thus the storage requirement is nb/k bytes, improving both storage efficiency and encoding performance by a factor of k . Like Shamir, k elements of the codeword are required to reconstruct the original data. Since the security guarantees of Rabin are far less than Shamir he suggests a technique to generate the rows of G randomly, embed the row id's within each code-word element, then encrypt the codewords (Rabin, 1989). This requires storing an external encryption key, which does not solve the main problem of providing security without securely storing encryption keys.

Hugo Krawczyk (IBM Research, Algorand Foundation)

<https://algorand.foundation/about-us/who-we-are>

proposed a blending of Rabin and Shamir, by encrypting the data with a key-based encryption algorithm and then dispersing the encrypted data with an IDA and the key with a secret sharing scheme called *Secret Sharing Made Short - SSMS* (Krawczyk, 1993). The dispersal algorithm described in (Resch and Plank, 2011) also enriches Rabin's IDA with security, but unlike SSMS does so without secret sharing and with the integration of integrity checking for corruption.

The new dispersal algorithm: AONT-RS enriches Rabin's IDA in two ways.

First modification employs a variant of AONT as a $(s + 1, s + 1)$ threshold scheme over the data (Rivest, 1997). Data composed of s words of size w_A is encoded into $s + 1$ different words, so that none of the original words may be decoded unless all $s + 1$ encoded words are present, or an attacker possesses enough computing power to crack an encryption key. The key, however, is encoded with the data. If a file's size is b bytes, the performance of encoding is $O(b)$.

The benefits of the AONT are: (1) No external keys are necessary. (2) Little extra storage. (3) The computational requirements of the attacker may be a parameter of the encoding. (4) The performance is good.

The AONT works as follows. The data is composed of s words $(d_0, \dots, d_{s-1})$, each of which is w_A bits in length. A random key K is chosen, and each codeword c_i is calculated as:

$$c_i = d_i \oplus E(K, i + 1),$$

where E is a key-based encryption algorithm, such as *Advanced Encryption Standard - AES*. A final codeword, c_k , is calculated to be a function of K and a hash of the other codewords. AONT has computational security, which means that unless an attacker possesses all $s + 1$ codewords or can guess K , the attacker cannot get information about any word or data. The scheme is modified slightly by adding an extra word d_s called a canary, which has a known fixed value to check the integrity of the data when it is decoded.

We generate $(c_0, \dots, c_s)$ as described above and then calculate a hash h of the $s + 1$ codewords using a standard hashing algorithm such as SHA-256 having an output at least as long as K . We then calculate a final block c_{s+1} as: $c_{s+1} = K \oplus h$. <https://en.bitcoinwiki.org/wiki/SOA-256>

Second modification of Rabin's IDA employs a systematic erasure code instead of a non-systematic one. A systematic code is defined to be one where the codeword contains the original elements of D . Without loss of generality, the first k elements of C are equal to the elements of D : $c_i = d_i$ for $0 \leq i \leq k$. This means that the first k rows of G are $k \times k$ identity matrix. Employing a systematic erasure code instead of a non-systematic one improves performance, because it eliminates the need to encode the first k codewords.

Since many systems use values of k that are large relative to n (e.g. POTSHARDS' evaluation uses a (3,5) Shamir scheme (Storer et al., 2009)) the savings during encoding with a systematic erasure code are substantial. Moreover, when decoding, codeword elements that are equal to data elements do not have to be decoded, which improves performance further. The dispersal technique AONT-RS, as it is a combination of the AONT and RS coding with the intuition is that AONT is used for security and the dispersal for availability, proximity and fault-tolerance. This is unlike Shamir, Rabin and SSMS which use dispersal to achieve both functions.

A canary is appended to the data, and the data and canary are encrypted with a random key. A hash value of the encrypted data is computed. The hash value and random key are then combined via bitwise exclusive-or to form a difference, which is appended to the encrypted data to form the AONT package.

Once processed by AONT, the result is treated as normal input to a systematic IDA. The IDA splits the input into k slices formed directly from the input and computes $n - k$ codewords.

A concrete example

Suppose we have a 4 KB block of data, D to be split into 16 slices on 16 storage nodes so that we may reconstruct and verify the data so long as ten slices can be possessed.

Shamir: To apply his algorithm, we view the data as 4096 bytes: $d_0, \dots, d_{4095}$. Each of the 16 slices $S_0, \dots, S_{15}$ will also be composed of 4096 individual bytes $s_{i,0}, \dots, s_{i,4095}$ such that $s_{i,j}$ is a function of d_j and nine random bytes. Specifically,

$$s_{i,j}d_j \oplus \sum_{x=1}^9 (i+1)^x r_{j,x}$$

where $r_{j,x}$ is a random byte and arithmetic is over $\text{GF}(2^8)$. The total storage requirement is 64 KB.

Rabin: Pad D to 4100 bytes and partition it to ten slices $DS_i, 0 \leq i \leq 9$ Then calculate each of the 16 slices using RS coding on the individual bytes:

$$s_{i,j} = \sum_{x=0}^9 (i+1)^x d_{x,j}.$$

The total storage requirement is $16 \times (410 + 16) = 6.65$ KB.

AONT-RS: Add 34 additional bytes to the data, and we first view it as being composed of 257 16-byte words, $d_0, \dots, d_{256}$, where the first 256 words are the original data. Set d_{256} to be a 16-byte canary value. Choose K to be sixteen random bytes and set each as follows:

$$c_i = d_i \oplus E(K, i + 1),$$

where E is a standard encryption algorithm. Next calculate h to be a 16-byte hash of $(c_0, \dots, c_{256})$. Finally, set $c_{257} = h \oplus K$. The last 2 bytes are padding so that data can split ten-ways.

SSMS: With SSMS, select a random 16-byte encryption key and encrypt the data with an encryption algorithm such as AES. Disperse it using Rabin and disperse the key using Shamir. The total storage requirement is $16 \times (410 + 16) = 6.65$ KB.

Theoretical performance of AONT

The CPU time that it takes to encode D composed of S total bytes, with a $R \times C$ dispersal matrix is $\mathbf{Perf}(R, C, S) = O(RCS)$. If the encoding is implemented with standard finite field arithmetic, then:

$$\mathbf{PERF}(R, C, S) = \frac{S}{S} \left(\frac{(R-1)(C-1)}{\mathbf{MULT}} + \frac{R(C-1)}{\mathbf{XOR}} \right),$$

where **MULT** is the bandwidth of performing Galois Field multiplication and **XOR** is the bandwidth of performing XOR operations. This is because encoding becomes a series of dot products to create R coding slices each of whose size is S/C bytes. The difference in the number of multiplications vs. XORs arises because nearly all dispersal matrices have ones in their top rows and leftmost columns.

Implementations of RS coding differ in their performance characteristics. Using Cauchy Reed-Solomon coding (Blomer et al., 1995), which substitutes additional XOR operations for the multiplication and can improve performance significantly (Plank et al., 2009b). Additionally, AONT(S) is the time that it takes to perform the AONT on S bytes of data. The choice of w_A , encryption and hashing technique will all affect AONT(S). In general, though, it is $O(S)$ and is also easy to parallelize. Given the parameters k, n, b , $\mathbf{Perf}(R, C, S)$, and AONT(S) the performance of the three main dispersal algorithms and their storage requirements are given in Table 2.30.

TABLE 2.30 Running time and storage requirements of the three dispersal algorithms.

Algorithm	Running-time	Storage
Shamir	$\mathbf{Perf}(n, K, kb)$	nb
Rabin	$\mathbf{Perf}(n, k, b)$	$\frac{nb}{k}$
AONT-RS	$\mathbf{AONT}(b) + \mathbf{Perf}(n - k, k, b)$	$\frac{n(b+w_A)}{k}$

Information dispersal versus replication from the viewpoint of storage and bandwidth overhead are compared in Table 2.31.

TABLE 2.31 Comparison of dispersal and replication schemes (typical cases indicated by *).

DISPERSAL					
Width	Threshold	Rel. 9's	Storage overhead	Bandwidth overhead	Access choices
8	5	6	60%	60%	56
8	6	7	33%	33%	28
16	10	>16	60%	60%	8008*
16	12	11	33%	33%	1820
32	24	>16	33%	33%	11 million
64	56	>16	14%	14%	214 million
REPLICATION					
2	No	5	100%	100%	2
2	Yes	10	100%	100%	2
3	No	7	200%	200%	3
3	Yes	>16	200%	200%	3*

More recently IBM purchased Red Hat for \$34 B in Nov. 2018.

<https://www.forbes.com/sites/moorinsights/2018/11/07/ibm-raises-the-bar-for-storage-again/#6cc35e373427>

2.19 Recent developments at IBM Research at ARC

This discussion is based on the following downloaded on 9/14/20.

https://researcher.watson.ibm.com/researcher/view_group.php?id=4203

The head of global storage research is Lawrence Y. Chu, who with Richard Freitas presented a tutorial on “Solid State Storage Technology Design and Applications” a decade ago.

<https://static.usenix.org/event/fast10/tutorials/T2.pdf>

The features of *Storage Class Memories* - *SCMs* are listed as (1) Non-volatile, (2) Short access time - like DRAM, (3) Low cost per bit - disk like, (4) solid state (no moving parts). SCM blurs the distinction between **memory** (fast, expensive, volatile), and **storage** (slow cheap, nonvolatile),

Several members of ARC are involved in Docker, which is a *Platform as a Service* - *PAAS* product that uses OS-level virtualization to deliver software in packages called containers.

[https://en.wikipedia.org/wiki/Docker_\(software\)](https://en.wikipedia.org/wiki/Docker_(software))

A tutorial to the common components of the Docker ecosystem is as follows. by Justin Elingwood.

<https://www.digitalocean.com/community/tutorials/the-docker-ecosystem-an-introduction-to-common-components>

Mario Blaum continues research on codes for storage devices.

<https://www.linkedin.com/in/mario-blaum-b7609ba/>

2.20 Storage research at Hewlett-Packard - HP

Cambridge U. graduate John Wilkes joined HP Labs in Palo Alto, CA in 1982, where he led for successful research on storage systems. In 2008 Wilkes left HP to join Google after coauthoring a book on storage systems (Wilkes et al., 2008).

John Wilkes web page for his publications at HP is as follows.

<https://john.e-wilkes.com/work-publications.html>

Datamesh was an early project which led to publications in the period 1989-1992. According to Figure 1 in (Wilkes, 1992b) at a high level Datamesh consists of a fast reliable interconnect to three module types:

- (1) Disk modules consisting of a CPU and disk.
- (2) RAM modules providing a high capacity buffer.
- (3) LAN modules providing connectivity.

Several issues raised by this project are as follows:

Shortest Access Time First: Optimizing seek times as in the case of SSTF and SCAN policies (Denning, 1967) is not adequate, since average rotational delay compared to seek time is not negligible. SATF which selects the request with the minimum positioning time was proposed in (Jacobson and Wilkes, 1991).

Loge indirect disk: Loge reserves a small amount (3-5%) of the disk space as “free blocks”, and directing a write to the next rotationally-available free block produces much lower latency (English and Stepanov, 1992). Loge is much better at writes but slightly worse at reads than a regular disk.

Disk Shuffling: By using the observed access pattern to the stored data, and rearranging the data to optimize future accesses of the same type, it is possible to reclaim the lost performance (Ruemmler and Wilkes, 1991).

Wilkes led the design of AutoRAID combining RAID1 and RAID5 arrays in (Wilkes et al., 1996) which is discussed in Chapter 10 on *Heterogeneous Disk Arrays* - *HDAs*. HP’s Surestore product was based on AutoRAID, which is compared to conventional RAID arrays.

https://support.hpe.com/hpsc/doc/public/display?docId=emr_na-lpg75004

Pei Cao (Cisco, U. Wisconsin, Google) led the effort to develop the TickerTaip parallel RAID array (Cao et al., 1994b). The system was designed based on the argument that in traditional disk arrays all requests flow through a centralized controller, which is then a *Single Point of Failure* - *SPOF*. TickerTaip distributes controller functions across several loosely coupled processors and this results in better scalability, fault tolerance, and flexibility. A working example and distributed algorithms to compute parities are described in Chapter 5. Techniques for establishing request atomicity, sequencing, and recovery are also discussed.

The Hibernator project dealing with conserving power in storage systems (Zhu et al., 2005) is discussed in Chapter 7.

Wilkes led research in automating the design of storage systems which culminated in the *Disk Array Designer* - *DAD* in (Anderson et al., 2005). This is discussed in more detail in Chapter 10.

Specifications for the HP Smart array P440ar controller,

https://support.hpe.com/hpsc/doc/public/display?docId=emr_na-c04496202

which is a 12 Gb/s SAS RAID controller that provides enterprise-class storage performance, increased internal scalability with SAS Expander Card, and data protection for specific HPE ProLiant Gen9 rack and tower servers. It features eight internal physical links and delivers increased server uptime by providing advanced storage functionality, including online RAID level migration between any RAID levels with *Flash Backed Write Cache* - *FBWC*, global online spare, and pre-failure warning. The Gen9 family of controllers supports the HPE Smart Storage Battery, which can support multiple

Smart Array controllers without needing additional backup power sources for each controller, resulting in a simple upgrade process.

3PAR founded in 1999 was acquired by HP in 2010.

https://en.wikipedia.org/wiki/HPE_3PAR

Thin provisioning advocated by 3PAR uses *Dedicate on Write - DoW* rather than *Dedicate on Allocate - DoA* paradigm, reducing the number of required disks.

<https://web.archive.org/web/20061118125440/>

http://www.byteandswitch.com/document.asp?doc_id=35674

IBM's MVS os used DoA paradigm in conjunction file systems, requiring a set of contiguous cylinders to be allocated before a file was written.

Nimble Storage is a flash company founded in 2008 which was acquired by HP in April 2017.

https://en.wikipedia.org/wiki/Nimble_Storage

It is advertised as predictive storage which alleviates 86% of problems. A 99.9999% availability is guaranteed over the installed base and 54% of problems resolved are determined to be outside storage.

In 2015 HP split into *HP Enterprise - HPE*, which specializes in servers and storage systems and competes with Dell/EMC and IBM, while HP specializes in printers and *Personal Computers - PCs*. PCs come in two categories: those manufactured by HP and those acquired from Compaq in 2002.

<https://www.cnet.com/news/hp-to-acquire-compaq-for-25-billion/>

2.21 Primary storage vendors and enterprise companies in 2020

Data storage companies are market leaders and wield big influence. They provide SAN - Storage Area Network or NAS - Network Attached Storage technologies, and increasingly, hybrid cloud storage solutions. Top five in enterprise storage market are given in Table 2.32 followed by a list of companies.

TABLE 2.32 Top 5 Companies, Worldwide Total Enterprise Storage Systems Market, 3rd Quarter of 2020 Revenue in US\$ M). Source: IDC Worldwide Quarterly Enterprise Storage Systems Tracker 12/9/2020.

Company	3Q20 Revenue	3Q20 Market share	3Q19 Revenue	3Q19 Market share	3Q20/3Q19 Revenue growth
1. Dell Technologies	\$1,949.9	28.8%	\$2,077.2	30.2%	-6.1%
2. HPE/New H3C Group	\$733.4	10.8%	\$680.2	9.9%	7.8%
T3. NetApp	\$638.5	9.4%	\$651.7	9.5%	-2.0%
T3. Huawei	\$633.3	9.4%	\$511.8	7.4%	23.7%
T5. Hitachi	\$376.2	5.6%	\$419.4	6.1%	-10.3%
T5. IBM	\$312.3	4.6%	\$398.4	5.8%	-21.6
Rest of Market	\$2,128.4	31.4%	\$2,132.0	31.0%	-0.2%
Total	\$6,772.0	100.0%	\$6,870.8	100.0%	-1.4%

Dell/EMC Since the \$67 billion merger of EMC and Dell in 2016, the combined company leads the external enterprise storage systems market, with SAN and/or NAS according to technology analyst firm *Int'l Data Corp. - IDC*. Product lines include Isilon NAS storage, EMC Unity hybrid-flash storage arrays for block and file storage, SC series arrays and the enduring VMAX family of products.

Hewlett Packard Enterprise - HPE and its Chinese joint venture, the New H3C Group, has surpassed Dell EMC in the overall market for enterprise storage systems. Notable product lines include HPE 3PAR StoreServ midrange arrays, entry-level HPE StoreEasy Storage NAS systems and flash-enabled MSA Storage.

<https://www.hpe.com>

NetApp has introduced NVMe-over-Fabrics (FC-NVMe) support to its all-flash arrays and offering hybrid cloud data tiering support in its ONTAP storage software.

<https://www.NetApp.com>

IBM in 2017, the company announced NVMe-based storage to improve enterprise storage performance.

https://en.wikipedia.org/wiki/IBM_Storage

Hitachi Vantara. Hitachi combined *Hitachi Data Systems - HDS* with Pentaho and the Hitachi Insights Group forming a subsidiary in 2017 focusing on data integration, the IoT, big data analytics, and enterprise storage.

https://en.wikipedia.org/wiki/Hitachi_Data_Systems

Huawei is expanding product portfolio includes all-flash OceanStor Dorado V3 arrays with NVMe support and OceanStor 18000 V5 hybrid-flash storage systems.

<https://en.wikipedia.org/wiki/Huawei>

In 2007, Huawei began a joint venture with U.S. security software vendor Symantec Corp., Huawei Symantec, which aimed to provide end-to-end solutions for network data storage and security. Huawei bought out Symantec's share in the venture in 2012, since Symantec had fears that the partnership "would prevent it from obtaining US government classified information about cyberthreats".

Enterprise data storage companies that offer their own take on storage technologies for businesses.

Oracle offers more than business databases and related software products, it also sells ZFS Storage which traces back to the Sun Microsystems, along with Zero Data Loss Recovery Appliances

https://en.wikipedia.org/wiki/Oracle_Zero_Data_Loss_Recovery_Appliance

and StorageTek archival tape systems.

<https://www.oracle.com/storage/>

Lenovo has ties with EMC and resells IBM Storwize arrays, but also has Lenovo Storage S2200 and S3200 SAN hardware.

<https://www.lenovo.com/us/en/data-center/storage>

Fujitsu gathers its arrays, backup appliances, tape libraries and *Software-Defined Storage - SDS* offerings under the ETERNUS banner, which is company's vision of providing storage solutions for the entire data lifecycle.

<https://www.fujitsu.com/global/products/computing/storage/>

Western Digital is synonymous with hard drives, but the company also produces data center storage systems like Ultrastar for the data center via its *Hitachi Global Storage Technologies - HGST* subsidiary. Western Digital intends to exit dedicated storage systems business.

<https://www.anandtech.com/show/14890/western-digital-to-exit-storage-systems-sells-off-intelliflash-division>

Given Dell/EMC's top ranking we list its products and services in Table 2.33.

<https://en.wikipedia.org/wiki/Dell EMC>

TABLE 2.33 Dell/EMC products and services.

Product category	Products/Services
Information Storage	PowerMax, VMAX Family, VNX/VNXe Family, Isilon, Atmos, XtremIO, ScaleIO, Unity/Unity XT Family, PowerStore, ECS
Archiving, Backup, and Recovery	Avamar, DataDomain, NetWorker, RecoverPoint, Centera, SourceOne
Storage and Content Management	Service Assurance Suite, Appsync, PowerPath, ViPR SRM, ViPR Controller
Virtualization	VMware, VPLEX
Services	Consulting, Customer support, Education Services, Managed Services, Technology Services and Solutions
Security/Compliance	RSA Security, Dell SecureWorks
Cloud computing/Converged Infrastructure	VxBlock, VxRack, VxRail, VSPEX, Virtustream
Data Computing	Greenplum, Pivotal (majority owned by EMC)

A more up-to-date *International Data Corp - IDC* report on top storage companies in Table 2.34 is given based on this. <https://www.businesswire.com/news/home/20200609005796/en>

2.22 All-flash upstart storage companies

Such companies have leapfrogging traditional, disk-based storage systems to focus on flash-enabled arrays that help set new standards of application and database performance.

Pure Storage has FlashArray//X based on NVMe SSDs. Its founding and the Purity arrays in discussed in Subsection 2.3.24.

<https://www.purestorage.com/>

TABLE 2.34 Top 5 Companies, Worldwide Enterprise External OEM Storage Systems Market, Quarterly Revenues (USD millions) based on IDC Worldwide Quarterly Enterprise Storage Systems Tracker, 6/9/2020.

Company	1Q20 Revenue	1Q19 share%	1Q19 Revenue	1Q19 share%	1Q20/1Q19 growth%
1. Dell Technologies	2,162.0	33.2	2,355.9	33.2	−8.2
2. NetApp	715.7	11.0	894.9	12.6	−20.0
3. HPE/New H3C Group	646.2	9.9	778.2	11.0	−17.0
4. Hitachi	430.3	6.6	470.5	6.6	−8.5
T5. IBM	332.1	5.1	320.0	4.5	3.8
T5. Pure Storage	311.7	4.8	289.5	4.1	7.7
T5. Huawei	270.7	4.2	230.0	3.2	17.7
Rest of Market	1,652.9	25.3	1,766.5	24.9	−6.4
Total	6,521.7	100.0	7,105.4	100.0	−8.2

Violin Systems started as Violin Memory is an early bet on all-flash with RAM. Violin reemerging from bankruptcy focuses on high-performance Flash Storage, but also storage services and management software tools that support modern-day workloads.

<https://www.violinsystems.com/>

Tegile’s parent company was *Hitachi Global Storage Technologies - HGST*. It concerns itself solely with hybrid and all-flash storage systems, including NVMe-based arrays like its IntelliFlash N Series. The company, then known as Tegile Systems, was acquired by Western Digital in 2017 and by DDN in September 2019.

<https://en.wikipedia.org/wiki/IntelliFlash>

Kaminario focuses on delivering flash-friendly *Software Defined Storage - SDS* after leaving the hardware side of the business to Tech Data which sells Kaminario K2 arrays.

<https://www.techdata.com/>

2.23 Hyperconverged infrastructure for storage systems

Converged Infrastructure - CI solutions consolidate datacenter technologies so that they can be acquired, deployed, managed, and supported as a single system.

HyperConverged Infrastructure - HCI differs from CI in that storage, computer, and storage networking functions are collapsed into a single.

A guide to hyperconverged infrastructure for Data Center Architects is (Lowe, 2015).

VMware claims that HCI allows the TCO to be reduced by 40%. The software layer in HCI includes a *Virtual Storage Appliance - VSA* that runs on each node. A VSA is a storage controller that runs on a *Virtual Machine - VM* to create shared storage without the cost of additional hardware. VSA consolidates direct-attached capacity on each physical host and creates a virtual storage pool that looks like networked storage. VSAs are popular with small businesses that need shared storage for redundancy and *High Availability - HA* as well as with large enterprises that need to migrate data between disparate arrays.

VSA communicates with other VSAs over Ethernet links, forming a distributed file system across which VMs are run. HCI is accomplished via the following:

1. Blade consolidation,
2. Converged infrastructure,
3. HyperConverged HW-based appliance,
4. Converged software defined infrastructure,
5. Intelligent and composable fabric-based infrastructures.

Data center elements that can be integrated into HCI are:

1. Deduplication appliances.
2. SSD caches, all flash arrays.
3. Backup software.

4. Data replication.

Nutanix is recognized by Gartner Magic Quadrant as a leader in HCI Software.

<https://www.nutanix.com/go/gartner-magic-quadrant-for-hyperconverged-systems>

Nutanix and VMware owned a combined 93 percent of the global HCI software market in the 2nd quarter of 2020, according to Gartner. Nutanix CEO Dheeraj Pandey has said that it is HCI software not hardware that is the true technology driver.

<https://www.crn.com/slide-shows/data-center/nutanix-bests-vmware-as-hyperconverged-software-leader>

Strato-Scale Computing and Maxta provide software support. HPE SimpliVity is a proprietary hardware card in their HPE's solution, which provides write caching, data deduplication and compression. HPE's OmniStack Accelerator Card assists in complex storage operations. Michael Dell has pronounced that Dell will exceed Nutanix 6-fold.

<https://www.crn.com/news/data-center/dell-to-become-six-times-bigger-than-nutanix-says-michael-dell>

SNIA defines *Software Defined Storage - SDS* as virtualized pools of storage with data service characteristics that may be applied to meet the requirements specified through the service management interface. According to SNIA SDS includes the following:

<https://www.snia.org/sds>

Automation: Simplified management that reduces the cost of maintaining the storage infrastructure.

Standard Interfaces: APIs for the management, provisioning and maintenance of storage devices and services.

Virtualized Data Path: Block, file and/or object interfaces that support applications written for these interfaces.

Scalability: Ability to seamlessly scale the storage infrastructure without disruption in availability or performance.

Transparency: Enabling users to monitor and manage their own storage consumption against available resources and costs.

Standardized Storage Management Interface - SMI: SNIA's Storage Management Initiative Specification (SMI-S),

SDS as integral component of the *Software Defined Data Center - SSDC*.

SDS to enable DevOps: Tools to communicate their storage requirements through interfaces such as SNIA's *Cloud Data Management Interface - CDMI*.

Where Data Services Executed: In servers, storage appliances, or both, removing the historical boundaries of where they execute.

DevOps is a set of practices that combines software development and IT operations to shorten the systems development life cycle and provide continuous delivery with high software quality. DevOps is complementary with Agile software development; several DevOps aspects came from Agile methodology.

<https://en.wikipedia.org/wiki/DevOps>

"Agile is an iterative approach to project management and software development that helps teams deliver value to their customers faster and with fewer headaches. Instead of betting everything on a "big bang" launch, an agile team delivers work in small, but consumable, increments. Requirements, plans, and results are evaluated continuously so teams have a natural mechanism for responding to change quickly."

<https://www.atlassian.com/agile>

A list of companies which are involved in HCI is as follows:

Nutanix: Nutanix's Enterprise Cloud platform enables SDS in the data center. It supports a wide range of storage services (file, block, container and virtual machine), along with backup and disaster recovery orchestration.

<https://en.wikipedia.org/wiki/Nutanix>

Pivot 3 is an Austin, TX based company that offers flash and hybrid flash HCI systems with a number of desirable data services, including asynchronous replication, erasure coding, thin provisioning and more. The company's high-end, all-flash HCI appliances pack both NVMe SSDs and conventional SSDs to accelerate storage workloads.

<https://pivot3.com/>

HPE SimpliVity with its OmniCube building blocks for *Software Defined Data Center - SSDC*.

<https://www.geeksforgeeks.org/overview-of-software-defined-data-center-ssdc/>

SSDC was defined by VMware CTO Steve Herrod in referring to a virtualized data center in which all infrastructure elements networking, storage, CPU - Compute and security are virtualized and delivered as a *Infrastructure as a Service*. While Server virtualization was only providing compute service, SSDC extends virtualization from the basic compute to storage services and networking resources. In SSDC deployment, operation, provisioning and configuration are abstracted from hardware by enabling policy-driven automation or implementing software

intelligence of everything which not only speeds the resource delivery but also enhances efficiency. SSDC is also called as SSD (Software Defined Datacenter) or VDC (Virtual Data Center).

Some of the vendors who provide SSDC hardware and services are Amazon, Dell/VMware, Microsoft, Hewlett Packard Enterprise, Oracle, IBM and Citrix etc.

Components of SSDC:

Virtualized Compute which masks server resources and as a result administrators use Virtual Machines for running and using multiple different applications and operating system on a single server. So the users and organizations stay out of managing complicated server-resource details.

Virtualized Storage pools physical storage resources from multiple network storage devices which eliminates disconnected storage systems and gives an appearance of single storage device managed by a central hub. It gives a facility of flexibility and extensibility and scalability to make it more dynamic.

Virtualized Network combines network resources and allows provisioning and managing networks independently of the physical hardware/channels so that assigning and reassigning to a particular server becomes more flexible.

Companies should be ready before transitioning into SSDC:

1. Assess company collaboration towards adoption of newer technology.
2. DevOps teams have a great role in SSDC so access the capabilities & limitations of organization's DevOps team.
3. Monitor the basic software infrastructure of the organization.
4. Monitor the organization's need of compute and storage services.

Benefits of implementing SSDC:

1. Increases business productivity
2. Reduce costs
3. Faster IT Service Delivery
4. Improves infrastructure performance
5. Increase scalability
6. Provides more storage capacity
7. Improve business agility
8. Reduce IT resource usage
9. Automated deployment and management
10. Allows infrastructure and application modernization.

Challenges in implementing SSDC:

1. Standardization across teams is a problem
2. Needs increased DevOps skills
3. Switching over to the new environment of SSDC
4. Needs change in existing processes and workflows of organization

Its HCI technology provides a single software stack that combines multiple IT infrastructure products into one shared x86 resource pool. The size of a server is usually described as a number in "U". One rack unit is approximately 1.75 inches, 2 rack units are "2U" and so on. OmniCube collapses legacy stacks and functionality into hyper-converged x86 building blocks. OmniCube consists of one or more 2U building blocks based on commodity x86 systems, pre-configured with SSD, flash, HDD storage, and connected in an efficient scale-out manner. The system easily scales by adding additional OmniCubes to create very large-scale environments in small increments.

OmniCube features high availability and no *Single Point Of Failure - SPOF* and systems can be deployed across multiple locations, such as remote or branch offices. In addition to integrating server compute, storage and network switching features, HPE SimpliVity extends capabilities with centralized management of geographically distributed systems; built-in VM data protection and off-site backup. OmniCube's Data Virtualization Platform deduplicates, compresses, and optimizes data when it is first created with no impact on performance.

It encompasses all VMs running on OmniCube's globally federated HCI.

<http://www.techrepublic.com/blog/smb-technologist/simplivitys-omnicube-the-new-converged-data-center-hardware->

on-the-block/

<https://www.hpe.com/us/en/newsroom/blog-post/2017/03/hpe-completes-acquisition-of-simplivity.html>

<https://www.storageinfoworks.com/Differentiation.asp>

Efficient & Agile storage management in software defined environments

The following discussion reflects IBM's viewpoint based on (Alba et al., 2014), that the IT industry is experiencing a disruptive trend making the entire data center infrastructure software defined and programmable. This allows IT resources to be provisioned and optimized continuously according to a declarative and expressive specification of the workload requirements. *Software Defined Environments* - *SDEs* facilitate agile IT deployment and responsive data center configurations that enable rapid creation and optimization of value-added services for clients.

However, this fundamental shift introduces new challenges to existing data center management solutions. The focus is on the storage aspect of the IT infrastructure and investigate its unique challenges as well as opportunities in the emerging software defined environments. SDS solutions and its interaction with *Software Defined Compute* - *SDC* and *Software Defined Networking* - *SDN* are discussed.

Whereas failures of compute or network resources can be dealt with by repeating computations or resending data, stored data cannot be re-created in such a straightforward manner. Furthermore, data may have to remain persistent after the lifecycle of the workload. Storage resources should offer data protection, in addition to, for instance, availability or performance levels also applicable to the other types of resources. Persistent data also limits the mobility of workloads. An analogy with SDN is helpful to formally define the objectives associated with SDS. SDN is aimed at providing a software implementation for some or all of the networking control plane in standard servers.

The SDN approach can be divided into two dimensions, both of which also apply to SDS:

Horizontal dimension of global optimization across nodes, where control plane functions shift from a fully distributed model to a centralized model in order to enable global optimization across the nodes;

Vertical dimension of the level of software integration of the control and data planes.

Global tiering across multiple storage systems would be able to rebalance the load by moving data to a more idle storage system, possibly passing along the original tiering information of that data to improve placement within the new system. As an example of increasing levels of software integration applied to SDS, consider the typical RAID function: Initially, only a static configuration of *Logical Unit Number* - *LUNs* with preconfigured RAID levels is presented by the hardware (storage system) to the software then, software becomes capable of configuring LUNs with desired RAID levels through a programmatic interface to the hardware; finally, the RAID function is performed in software, eliminating the hardware availability restriction and enabling optimizations such as distributed RAID across nodes.

2.24 Top enterprise storage backup players

Veritas which had merged with cybersecurity expert Symantec in 2005 split from it in 2016 as a private company and focused on enterprise data protection with its NetBackup backup and recovery suite and the Flex Appliance, which can be deployed in minutes to provide on-demand backup, recovery, archiving and cloud tiering services using a microservices-based approach.

https://en.wikipedia.org/wiki/Veritas_Technologies

On 8/9/2019, Broadcom Inc. announced acquiring the Enterprise Security software division of Symantec for \$10.7 B. The remainder of the company was renamed NortonLifeLock.

Commvault provides data protection and information management to midsized to large businesses. It helps enterprises extract more from their information as it passes from primary to secondary storage. Commvault Hyperscale is available both as an appliance that integrates storage, compute, networking, backup and recovery, analytics and data lifecycle management, or as software that works with systems from Dell EMC, HPE, Cisco and others.

<https://en.wikipedia.org/wiki/Commvault>

Actifio is mainly known as a copy data management specialist, but its expertise in helping businesses optimize their complex storage environments enables the company to offer *Disaster Recovery as a Service* - *DRaaS*.

<https://www.actifio.com>

2.25 Data storage companies: up and coming storage vendors

Cobalt Iron offers a single dashboard to manage backup. The console provides the user the data needed to troubleshoot the system. Compass can be found in many global data centers, including AWS, Google Cloud, IBM Cloud, Alibaba and Microsoft Azure.

<https://www.cobaltiron.com/>

Benefits of Compass are as follows:

Analytics: Enabling automation, predictive fault avoidance, and prescriptive process improvement.

SaaS: Delivering 100% software currency for data protection across the enterprise.

Cloud: Supporting public, hybrid, and private cloud deployments

Virtualization: Delivering optimized server, storage, and network utilization

Pavilion Data Systems - PDS: NVMe over Fabric -NVMe-oF is a solution designed to enable faster speeds in data storage environments. PDS offers write performance at speeds up to 90 GB/s with 40 microsecond write latency. PDS's Swarm, decentralized data storage and distribution; which allows Pavilion to rebuild an SDD in just a few minutes. To round out its offering, the company offers encryption, thin provisioning and an array of data management tools.

<https://pavilion.io/company/>

Thin provisioning

"Thin provisioning uses virtualization to give the appearance of more physical resources than are actually available. In computing, thin provisioning involves using virtualization technology to give the appearance of having more physical resources than are actually available. If a system always has enough resource to simultaneously support all of the virtualized resources, then it is not thin provisioned. The term thin provisioning is applied to disk layer in this article, but could refer to an allocation scheme for any resource. For example, real memory in a computer is typically thin-provisioned to running tasks with some form of address translation technology doing the virtualization. Each task acts as if it has real memory allocated. The sum of the allocated virtual memory assigned to tasks typically exceeds the total of real memory.

The efficiency of thin or thick/fat provisioning is a function of the use case, not of the technology. Thick provisioning is typically more efficient when the amount of resource used very closely approximates to the amount of resource allocated. Thin provisioning offers more efficiency where the amount of resource used is much smaller than allocated, so that the benefit of providing only the resource needed exceeds the cost of the virtualization technology used."

https://en.wikipedia.org/wiki/Thin_provisioning

StorONE is based on SDS data storage solution that is as flexible as it needs to be to handle any number of storage tasks such as file, block or object storage. It can be used for virtual storage, or to handle cloud storage and other secondary storage, hybrid arrays or those ultra-fast all-flash arrays.

Coraid 's EtherDrive storage arrays consist of affordable hardware running efficient software connected to servers through an EtherDrive *Host Based Adapter - HBA*, which are Ethernet interface cards that use a driver to make the HBA appear as a simple SAS HBA.

<https://coraid.com/solutions.html>

DataDirect Networks - DDN DDN founded in 1998 is headquartered in Chatsworth, CA has about 650 employees. It is a provider of large storage systems for unstructured data and big data environments. DDN in 2018 purchased the virtualization focused storage company Tintri and in 2019 acquired SDS vendor Nexenta.

https://en.wikipedia.org/wiki/DataDirect_Networks

<https://www.ddn.com>

<https://tintri.com/>

Infotrend <http://www.infotrendworks.com/products.asp>

Panasas is a data storage company that creates *Network-Attached Storage - NAS* for technical computing environments. It was founded by Garth Gibson and William Courtright in 2004. Scalable performance of the *Panasas Parallel File System - PFFS* is discussed in (Welch et al., 2008).

<https://www.panasas.com>

<https://en.wikipedia.org/wiki/Panasas>

Other *Original Equipment Manufacturers - OEMs* do not themselves market the storage components they manufacture.

Pillar which produced flash memories was acquired by Oracle in 2011 and its hardware was used in conjunction with Oracle products such as Exadata. Oracle laid off almost all of Pillar's employees in August 2019.

https://www.theregister.co.uk/2019/08/16/oracle_lays_off_hundreds_of_works_as_pillar_data_closed/

A ranking of primary storage companies according to the Gartner Group in 2020 is given here:

<https://www.gartner.com/reviews/market/primary-storage>

Gartner's Magic Quadrant for Primary Storage covers vendors that offer dedicated products or product lines for *Solid-State Arrays - SSAs*, which cannot be combined or expanded with any form of HDD.

<https://www.gartner.com>

2.26 Parallel file systems

The following discussion is partially based on a review of parallel file systems at *Illinois Institute of Technology - IIT* in 2010 by Samuel Lang associated with *Argonne National Lab - ANL*, which is funded by the *Dept of Energy - DoE*.

<https://www.cs.iit.edu/~iraicu/teaching/CS595-F10/lecture07.pdf>

Lang was a member of a team which developed Darshan, a scalable HPC I/O characterization tool.

<https://www.mcs.anl.gov/research/projects/darshan/>

Parallel Virtual File System - PVFS: PVFS is based on the Vesta parallel file system developed at IBM Research by Peter Corbett (NetApp, HPE) and Dror Feitelson (Hebrew U.) (Corbett and Feitelson, 1996), but was rewritten later.

https://en.wikipedia.org/wiki/Parallel_Virtual_File_System

General Parallel File System - GPFS: GPFS was developed by Frank Schmuck and Roger Haskin at IBM ARC (Schmuck and Haskin, 2002).

https://en.wikipedia.org/wiki/IBM_General_Parallel_File_System

GPFS was based on Tiger Shark file system developed for multimedia and Vesta partitioned file system (Haskin, 1998).

GPFS was rebranded Spectrum Scale by IBM in 2015.

Lustre: Lustre is an open-source, distributed parallel file system software platform designed for scalability, high-performance, and high-availability.

<https://www.cse.buffalo.edu/faculty/tkosar/cse710/papers/lustre-whitepaper.pdf>

<http://lustre.org/resources/>

http://wiki.lustre.org/Main_Page

Ceph: Ceph is a free file system for object storage with object-level, block-level, and file-level interfaces. It is intended for distributed operation without a single point of failure, which is scalable to the exabyte level. Ceph was developed by Sage A. Weil as part of his PhD thesis at *U. California at Santa Cruz - UCSC* working with Scott A. Brandt, Ethan L. Miller, Darrell D. E. Long, and Carlos Maltzahn (Weil et al., 2006a).

[https://en.wikipedia.org/wiki/Ceph_\(software\)](https://en.wikipedia.org/wiki/Ceph_(software))

Ursa Minor: Ursa Minor is a cluster-based storage system that allows data-specific selection and on-line changes to, encoding schemes and fault models. Thus, different data types can share a scalable storage infrastructure, rather than suffering from "one size fits all" as noted in (Abd-El-Malek et al., 2005). Ursa-Minor was developed as part of Self* storage project at CMU's PDL to automate the storage management function in a storage brick environment described in (Ganger et al., 2003).

<http://www.pdl.cmu.edu/SelfStar/>

Panasas Parallel File System - PPFS: PPFS was developed at Panasas.com cofounded by Garth Gibson and William Courtright and described in (Welch et al., 2008). Rapid prototyping of RAID via RAIDframe is described in Courtright's PhD thesis (Courtright et al., 1996).

<http://www.pdl.cmu.edu/RAIDframe/>

2.27 Cloud storage

MIT's J.C.R. Licklider associated with the ARPANET project in the 1960s is credited with the invention of cloud computing, cloud storage being a special case.

https://en.wikipedia.org/wiki/Cloud_storage

NIST's *Computer Security Resource Center - CSRC* defines cloud computing as follows:

<https://csrc.nist.gov/publications/detail/sp/800-145/final>

“Cloud computing is a model for enabling ubiquitous, convenient, on-demand network access to a shared pool of configurable computing resources (e.g., networks, servers, storage, applications, and services) that can be rapidly provisioned and released with minimal management effort or service provider interaction.”

Two current major trends are:

1. Replacing workstations and personal computers with thin clients.
2. Ubiquitous or pervasive computing uses any device for work. The term was coined by Mark Weiser around 1988, during his tenure as Chief Technologist of the Xerox *Palo Alto Research Center - PARC*.

Due to a greater level of abstraction provided by cloud computing, the skillset required to manage and support the for IT will diminish.

https://en.wikipedia.org/wiki/Shadow_IT

Users need an internet connection to get access to computing and storage capacity of the cloud. CompuServe started offering its users a small amount of disk space in 1983, followed by AT&T PersonaLink services in 1990s. The term “cloud” first appeared in a 1996 Compaq Corp. document.

<https://www.technologyreview.com/s/425970/who-coined-cloud-computing/>

There are two basic types of virtualization technology in cloud computing:

Application virtualization delivers an application hosted by the cloud by high-grade virtual machines. It is cheaper because it is available to a large number of users. Note similarity to timeshared computer systems. Early time-shared computers are discussed in Chapter 1.

Server virtualization uses common physical hardware to host *Virtual Machines - VMs*. A physical host machine could have any number of VMs, which can be installed with their own OS and their own set of applications. Server virtualization consolidates a large number of physical machines and this increases in computing efficiency: lower space, maintenance, cooling, and electricity costs, besides the obvious reduction in procurement costs for the machines.

The following discussion of cloud computing price models is based on (Ruparelia, 2016), which justifies cloud computing using a resource sharing argument, e.g., while personal computers are affordable its price components are as follows: 10% processor, 60% memory, and 20% network bandwidth. Companies instead of buying equipment which is a *Capital Expenditure - CapEx*, can meet computing needs using an *Operating Expenditure - OpEx* by relying on cloud computing. Cloud computing provides service offerings for different abstraction levels.

Infrastructure as a Service - IaaS Hardware infrastructure (servers, storage, etc.) on a utility basis.

Platform as a Service - PaaS is the same as IaaS but includes the OS and any other core applications to enable user to install and run software. Pricing generally is on a utility basis.

Software as a Service - SaaS is as per PaaS but includes hosted applications that fulfill a function. The applications are used when needed and the cost of installing and maintaining the application and its supporting hardware infrastructure is avoided. Pricing is on a per-use basis.

Information as a Service - INaaS provides information relevant to an individual or corporation and to their businesses, business processes, or tasks. Pricing is usually on a consumption, per-use basis.

Business Process as a service - BPaaS fulfills a business function or replaces a business process in an organization. Typically combines *Business Process Outsourcing - BPO* with software as a service (SaaS). Pricing generally on a per use basis.

NIST proposes four distinct cloud deployment models:

Public cloud provides services to anyone having Internet access.

Private cloud provides services to a single entity, e.g., a government organization or a business enterprise from a private network.

Community cloud is shared by several entities.

Hybrid cloud is a conglomeration of other types of clouds, which might be necessary when a cloud service needs to access computing resources from other clouds a concept is known as cloud bursting.

There are four types of hybrid clouds:

1. consisting of only private clouds.
2. consisting of private and public clouds.
3. consisting of private, public, and community clouds.
4. consists of private, public, community, and hybrid clouds.

Cloud bursting is a configuration that's set up between a private cloud and a public cloud to deal with peaks in IT demand. If an organization using a private cloud reaches 100 percent of its resource capacity, the overflow traffic is directed to a public cloud so there's no interruption of services.

<https://Azure.microsoft.com/en-us/overview/what-is-cloud-bursting/>

Ubiquitous access through a network is important, because the user is not constrained by location. With increasing user base there is the danger of compromised security and network access being limited to a private network or a community of users. On-demand self-service or availability has two characteristics:

1. The service is available as long as the cloud is not down, hence cloud availability is important.
2. The number of user should not affect the user experience.

Five characteristics of cloud storage according to (Ruparelia, 2016) are as follows:

Broad network access: consuming service from anywhere.

On-demand self-service: consume service when user wants.

Resource pooling and virtualization: Pools the infrastructure, virtual platforms, and applications.

Rapid elasticity and horizontal scalability: sharing pooled resources.

Measured service: *pay as you go - payg* service.

Various cloud relationships are as follows: Encapsulation is one of two types: composition and aggregation. An aggregated cloud may consist of multiple SaaS cells, while a composite cloud may have Email, CRM, Database, Storage, Web hosting cell, etc.

Specific clouds perform distinct functions reusable by other clouds can and increase profits, which is the opposite of Gossen's First Law of Economics, and can be thought of as providing an increasing marginal utility with usage.

https://en.wikipedia.org/wiki/Gossen%27s_laws

2.27.1 Cloud computing price models

Price models determine the price users pay in order to receive the value of a product or service. Cloud service providers compute the costs of provisioning and operating a cloud service using a cost model, which is then converted into a price model depending on the cloud service provider's business model, marketing strategy, and revenue expectations. Cloud services are compared based on their price model in (Ruparelia, 2016):

Utility price model based on the usage of the service such as KiloWattHours - KWH in an electric utility.

Consumption price model where user pays for the computing resources they use, e.g., number of servers in *Amazon Web Services* - AWS and amount of storage in IaaS or PaaS.

Txn based price model is suitable under the following circumstances:

1. Txn volumes are known and predictable.
2. The business process is measured in discrete units to represent a txn.
3. The txn volume is tied to cost drivers.
4. When business processes are standardized and driven by txns.

Subscription based such as access to news web site, number of accesses during subscription period does not matter.

Service price model based on delivered benefit, *Service Level Agreement* - SLA, or money saved.

Fixed price model which may consist of a recurring and nonrecurring price.

Volume-based price model related to the number of users, amount of storage space, rate of txns

Tiered price model based on SLAs, volume, or amount spent, e.g., SLA users pay more for more stringent SLAs

Performance price model is benchmark-based and the price relies on benchmark results.

Outcome-based price models provide a bonus if an outcome is achieved and with some performance related models the provider is penalized if an SLA or benefit is not realized.

Business-linked price model Whereas outcome based models use metrics that measure the value of cloud computing, i.e., the contribution that cloud computing makes to *Key Performance Indicators* - KPIs affecting users business model.

<https://www.clearpointstrategy.com/18-key-performance-indicators/>

A mapping cloud computing objectives to business and financial KPIs is as follows:

- Cloud value model - Business KPI
- Operating expenditure - Cash flow
- Demand flexibility - Productivity
- Price flexibility - Cost efficiency

- Time-to-market - Cash-flow
- Location flexibility - Productivity
- Asset optimization - Return on investment (ROI)
- Profit margin flexibility - Profit margin

Gain share prices model can be combined with a penalty based performance model to create a hybrid performance model.

Marketing price model driven by marketing rather than performance. The goal is to attract as many customers as possible and to then monetize it to create a profit.

Premium price models are offered by SaaS companies. LinkedIn provides a free version, but a premium is to be paid for features, such as InMail to LinkedIn members. The LinkedIn algorithm is described here.

<https://blog.hootsuite.com/how-the-linkedin-algorithm-works-hacks/>

Razor-and-blades price model uses the analogy of cheap and expensive blades. Similarly printers may be cheap but cartridges expensive.

Hybrid price models The utility, service, and performance price models can be combined to produce hybrid price models, e.g., a subscription-based system that utilizes a tiered approach.

Seven cloud computing value models are as follows:

Time-to-market value model reduces users time to go to market by enabling him to transition from one computing environment to another at a faster pace.

Operating expense model The IT department would need to spend capital up-front in order to create and maintain the computing service. Extra capacity is required to meet SLAs. This expenditure could have been employed elsewhere to pursue a business opportunity. Conversely, should the actual demand exceed the compute capacity that you have invested capital in, the extra demand will either not be met or will translate to degraded SLAs. The end result is an opportunity loss to business, since the unmet demand will cause loss of business. The operating expenditure model of cloud computing, uses elasticity to avoid opportunity costs and losses.

User demand flexibility Every product or service provided by the business to customers has the following lifecycle: development, testing, launch, marketing and usage.

During each of these phases, differing demands are placed upon computing so that there are periods consisting of highs and lows over the entire life cycle.

Price flexibility Over time the general trend in price for computing: traditional or cloud computing is downward. There is also the downward trend due to economies of scale resulting from the adoption of new technologies as the older ones reach obsolescence.

Time-to-market agility model of cloud computing, reduces user time to go to market by enabling the user to transition from one computing environment to another at a faster pace. The user does not have to spend time on building a computing environment and at a short notice may get access to any number of computing resources available.

Location flexibility value model of cloud computing enables the user to access the computing environment from any location due to the ubiquitous nature of computing. This flexibility increases productivity and empowers user to have global reach.

Asset optimization value model of cloud computing is similar to the demand flexibility value model from a different perspective. Toward the end-of-life the user may be left with extra computing infrastructure that takes up needless space and so acts as a drain upon cashflow. All this assumes that the asset life cycle that affects investment follows the product life cycle allowing the user to use cloud computing optimize assets.

Profit margin When the user produces any good or service, value-added costs including overheads fall 25% every time the cumulative volume or accumulated experience doubles, i.e., a downward-sloping price experience curve that increases with experience.

Boston Consulting Group - BCGs experience curve

This is a hypothesis that costs follow a definite pattern, which is a function of accumulated production experience. What follows is based on a short article by BCG founder Bruce Henderson.

<https://www.bcg.com/en-us/publications/1968/business-unit-strategy-growth-experience-curve>

There are far reaching implications for business management. Given business competition as a system in equilibrium this hypothesis introduces a whole new set of relationships, which are of critical importance in establishing the stability

of competition. The whole of business policy is affected from manufacturing cost standards to export policy. The following conclusions are justifiable:

1. Reductions in cost of manufacturing and distribution should be predictable.
2. Purchased material prices should be stable.
3. New products should be priced as low as necessary to dominate their market segment to be sold.
4. Market share must be maintained at all costs as long as growth rate exceeds the anticipated rate of return.
5. The value of market share can be calculated with enough accuracy to permit determination of the return on investment as the result of any change in share.
6. Export potentials of a product can be approximated by relating comparative advantage derived from experience levels.
7. The cost trade-offs of low labor cost versus concentrated production can be approximated.

Financial metrics or yardsticks can be used to assess the value proposition of a cloud service. Four common financial metrics are: *payback method*, *Net Present Value - NPV*, *Return On Investment - ROI*, and *Time To Market - TTM*. Other metrics are *Economic Value Added - EVA*, *Return on Assets - ROA*, and *Return on Equity - ROE*, which are difficult to use when considering a single service, product, or project, because they typically aggregate the computations at a corporate level and so rely on other factors such as the company's tax rate and its corporate KPIs.

Payback method measures the time needed to recoup investment in a product or service. This method is better suited for capital expenditure because you can depreciate it over a number of years.

Return on Investment - ROI Whereas the payback method considers the time to recoup the investment, ROI uses the percentage of the investment amount that will be recouped. ROI is widely used in the IT industry to assess capital investments.

$$ROI = \frac{\text{Gain from investment}}{\text{Cost of investment}} - 1.$$

Unadjusted ROI assumes that all gains and costs are produced at the outset. To adjust for this anomaly, usually an NPV calculation is performed that discounts the time value of money to get a more realistic value for ROI.

Net Present Value - NPV When you have multiple cashflows occurring on a regular basis, those cashflows are called a stream of cashflows. When regular streams of cashflows are equal in value, the cashflows are known as annuities. Because of the effect of inflation and interest rates over time, a cashflow amount of \$1,000 next month is worth more than the same amount months later. NPV calculations are used to assess the present value of a series of cashflows. The NPV of an investment is the present value of all future benefits, such as cashflows, generated by the investment, net of initial costs, discounted over intervals of time.

<https://www.investopedia.com/terms/n/npv.asp>

Time to Market - TTM estimating the time when revenues will be obtained is another financial yardstick. Using traditional computing it would take a year to go to market with a new offering that you are developing, but with cloud computing it could take three months, so that the TTM will be nine months sooner. If the user earns \$20,000 per month with the new system, then the earlier TTM would represent additional inflows of \$180,000. TTM can be expressed as time as well as its equivalent in monetary terms.

Evolution of business process management techniques and technologies can be summarized as follows:

- (1975) total quality management - TQM,
- (1980) Just in Time - JIT,
- (1985) Lean methodology,
- (1990) Six sigma,
- (2000) Process architecture,
- (>2020) BPaaS and cloud computing.

Six Sigma strategies seek to improve the quality of the output of a process by identifying and removing the causes of defects and minimizing impact variability in manufacturing and business processes. It uses a set of quality management methods, mainly empirical, statistical methods, and creates a special infrastructure of people within the organization who are experts in these methods. Six-sigma implies 0.00034% defective.

https://en.wikipedia.org/wiki/Six_Sigma

2.27.2 Storage as a service in cloud computing

A recent study considers the storage landscape associated with prevalent storage technologies: tape, HDDs, and NAND components by presenting the past ten-year trends including both economic metrics (component revenue and bit shipments) and technology metrics (bit densities and cost per bit and manufacturing capacity) (Fontana, 2018). These are used to project influences on cloud strategy.

There was a drop to 10.4% from 12% of the total storage capacity shipped in the 4th quarter of 2019. Total shipped enterprise storage capacity across disk drives and SSDs was 169.79 EB, with disk drives up 83% per year on year to account for 144.1 EB.

<https://blocksandfiles.com/2020/02/17/enterprises-buy-more-disk-than-ssd-drive-capacity/>

Amazon Web Services - AWS Storage as a Service - SaaS with 20% of revenue) leads other companies according to Table 2.35.

<https://blocksandfiles.com/2021/02/16/aws-top-enterprise-storage-suppliers-by-revenue/>

TABLE 2.35 SaaS Market Share, Market Share in 3Q20 (revenue in US\$ millions).

Company	3Q20 Rev.	3Q20 share	3Q19 Rev.	3Q19 share	3Q20.3Q19
AWS(SaaS @ 20%)	2,230.0	-	1,647.2	-	29%
1. Dell technologies	1,949	28.9%	2077.2	30.2%	−6.1%
AWS (SaaS @ 15%)	1,740	-	1235.4	-	29%
Azure(SaaS @ 20%)	1,264	-	518.4%	-	59%
Azure(SaaS @ 15%)	948.3	-	388.8%	-	59%
T2 HPE	729.6	20%	680	10%	7.3%
T3 NetApp	638.5	9.5%	651.7	9.5%	−2.0%
T3 Huawei	633.3	9.4%	511.8	7.4%	23.7%
T5.Hitachi	376.2	5.6%	419.4	6.1%	−10.3%
T5.IBM	312.3	4.6%	398.4	5.8	−21.6
Rest of market	2,104.3	31.2%	2,132	31%	−1.3%

A clear trend is that on-premises storage vendors today represent not much more than half of the overall enterprise storage services, by revenue share, This share will likely shrink further.

2.27.3 Cloud storage elasticity and its benchmarking

In cloud computing, elasticity is defined as the degree to which a system is able to adapt to workload changes by provisioning and de-provisioning resources in an autonomic manner, such that at each point in time the available resources match the current demand as closely as possible.

[https://en.wikipedia.org/wiki/Elasticity_\(cloud_computing\)](https://en.wikipedia.org/wiki/Elasticity_(cloud_computing))

Several definitions of cloud elasticity are as follows:

Samuel Kounev et al. (Herbst et al., 2013):

“Elasticity is the degree to which a system is able to adapt to workload changes by provisioning and deprovisioning resources in an autonomic manner, such that, at each point in time, the available resources match the current demand as closely as possible.”

Open Data Center Alliance - ODCA, 2012 “[...] defines elasticity as “the configurability and expandability of the solution [. . .] Centrally, it is the ability to scale up and scale down capacity based on subscriber workload”.

http://www.opendatacenteralliance.org/docs/ODCA_Compute_IaaS_MasterUM_v1.0_Nov2012.pdf

P. Nell & T. Grance at NIST, 2011 “Rapid elasticity: Capabilities can be elastically provisioned and released, in some cases automatically, to scale rapidly outward and inward commensurate with demand. To the consumer, the capabilities available for provisioning often appear to be unlimited and can be appropriated in any quantity at anytime.”

Edwin Schouten at IBM, 2012 “Elasticity is basically a rename of scalability [. . .]” and “removes any manual labor needed to increase or reduce capacity.”

Rich Wolski CTO at Eycalptus, 2011 “Elasticity measures the ability of the cloud to map a single user request to different resources.”

Reuven Cohen, 2009 “Elasticity is “the quantifiable ability to manage, measure, predict and adapt responsiveness of an application based on real time demands placed on an infrastructure using a combination of local and remote computing resources.”

<https://www.forbes.com/sites/reuvencohen/?sh=4cf9f63f66cc>

An early benchmarking study of elasticity in Cloud computing is Weber et al. (2014).

The SPEC Cloud IaaS 2018 benchmark measures the performance of IaaS public and private cloud implementations.

http://www.spec.org/cloud_iaas2018/docs/faq.html

The benchmark stresses provisioning as well as runtime aspects of a cloud using I/O and CPU intensive cloud computing workloads. SPEC selected the social media NoSQL database txns and K-means clustering using map/reduce as two significant and representative workload types within cloud computing. The first workload uses the Yahoo Cloud Serving benchmark with the NoSQL Cassandra database. The second workload uses K-means from Intel Hiben Suite for Hadoop.

Each workload runs as a distributed application made up of 6 or 7 *Application Instances - AIs*. Adding AIs to the cloud stress the cloud's available resources (e.g., CPU, memory, disk, and network). The workloads continue to run until the test fails the quality of service (QoS) requirements. The tester can also limit the maximum number of AIs created during a run.

2.28 Jai Menon's predictions on the future of clouds

Some early cloud services were: *Amazon Web Services - AWS* in 2006, Microsoft's *Azure* 2010, IBM's *Public Cloud* 2011, and Google *Cloud Platform* 2012.

Jai Menon (IBM, Dell/EMC, Cloudistics, Fungible) has predicted three phases of cloud services in 2019.

<https://www.itweb.co.za/content/xnklOvzbYPkv4Ymz>

<https://medium.com/jai-menons-blog/the-future-of-cloud-is-clear-and-its-not-all-aws-c559604d3cfb>

First Phase - Cautious adoption:

Second Phase - Excessive exuberance:

Third Phase - Diversification: Customers began to need several private and public clouds.

Note similarity to Gartner's hype curve representing adoption and maturity of emerging technologies through five phases of its life cycle:

<https://www.gartner.com/en/research/methodologies/gartner-hype-cycle>

1. Technology trigger.
2. Peak of inflated expectations
3. Trough of disillusionment.
4. Slope of enlightenment.
5. Plateau of productivity.

With the advent of storage clouds there is no need for users to have high capacity disk storage on their desktops or notebooks. Described in (Lunt, 2018) are the steps required for establishing cloud storage.

<https://spectrum.ieee.org/computing/hardware/a-view-to-the-cloud>

1. Data is saved locally on a desktop or notebook.
2. Data is transmitted over the *Internet Service Provider - ISP* to the data center.
3. For reliability the data is stored at least twice (in the cloud).
4. The data is also backed up on magnetic tape. Offline tape storage has the advantage of not being deleted inadvertently by human errors.

A list of cloud companies is provided in Table 2.36.

<https://www.zdnet.com/article/top-cloud-providers-2019-aws-microsoft-azure-google-cloud-ibm-makes-hybrid-move-salesforce-dominates-saas/>

The following report was posted in December 2020. Quarterly cloud revenue in last column. Company revenues as of 11/25/20.

<https://cloudwars.co/special-report-2020/>

Microsoft: CEO Satya Nadella. Microsoft dominance is based on Office 365 Commercial, Azure Stack, Cosmos DB and LinkedIn and GitHub.

Amazon Web Services - AWS: CEO Jeff Bezos/Andy Jessy. Amazon is dominant in IaaS and has expanded into the database market by replacing Oracle databases with its own data-management tools, something that Larry Ellison thought is impossible. Netflix uses AWS as its platform. AWS accelerating revenue growth in cloud computing and

TABLE 2.36 Cloud rankings are based in quarterly cloud revenue in cloudwars report.

Company	Overall Rev	Cloud Rank	Cloud Rev
Microsoft	\$147.62B	1	\$15.2B
Amazon AWS	\$280.5B	2	\$11.6B
Salesforce	\$17.1B	3	\$5.42B
Google	\$89.46	4	\$3.4
SAP	\$33.84B	5	\$2.36B
Oracle	\$39.4B	6	\$2.2B (est.)
IBM	\$77.147B	7	\$6.00B
Workday	\$4.162B	8	\$969M
Servicenow	\$1.09B	9	\$1.09B
Adobe	\$12.87B	10	\$838M
Accenture	&44.3B	11	\$2.75B

competitive positioning is given on 12/22/20.

<https://techcrunch.com/2020/12/22/with-a-50b-run-rate-in-reach-can-anyone-stop-aws/>

Google Cloud CEO Thomas Kurian came from Oracle to remake Google. Note that cloud revenue is a small fraction of Google's revenue.

Adobe Chairman & CEO is Shantanu Narayen.

https://en.wikipedia.org/wiki/Adobe_Inc.

Salesforce CEO is Marc Bonioff. It is leader in SaaS category, but is facing competition from rivals redefining the category.

Oracle CEO Larry Ellison is moving company headquarters from Redwood City, CA to Austin, TX. Oracle internally developed cloud handles *Enterprise Resource Planning - ERP*

https://en.wikipedia.org/wiki/Enterprise_resource_planning

and Cloud *Human Capital Management - HCM* businesses. Oracle's Autonomous Database has received rave reviews.

<https://www.oracle.com/database/what-is-autonomous-database.html>

SAP CEO is Christian Klein. SAP develops software to manage business operations and customer relations.

<https://en.wikipedia.org/wiki/SAP>

It is known for its ERP software.

SAP's cloud business grew about 40% in Q4 2019 even without including the acquisition of Qaultrics, whose customer-experience expertise allows SAP to combine that with its ERP and operational-data expertise to offer a range of unmatched capabilities.

IBM CEO is Arvind Krishnan. IBM provides services across IaaS, PaaS and SaaS, emphasizing openness, cybersecurity, and expertise in hybrid cloud and multi-cloud. IBM cloud services was based on the acquisition of Softlayer in June 2013, although prior to that IBM had announced public and hybrid cloud services in 2011.

https://en.wikipedia.org/wiki/IBM_cloud_computing

IBM acquired RedHat Linux in July 2019 to enhance its hybrid cloud service for \$34 Billion. Red-Hat CEO James Whitehurst was made IBM president and was supposed to replace Krishna after he steps down, but it seems Krishna who is 59 (July 2021) decided to stay and Whitehurst left IBM.

<https://seekingalpha.com/article/4333975-ibms-main-asset-is-can-sell-red-hats-offerings>

ServiceNow CEO is Bill McDermott left SAP to join ServiceNow in Nov. 2019. The company founded in 2003 is headquartered in Santa Clara, CA develops a cloud computing platform to help companies manage digital workflows for enterprise operations, and was also recognized for innovation. The digital-workflow company continues to thrive with its unique approach toward changing how companies work rather than focusing on specific applications.

<https://en.wikipedia.org/wiki/ServiceNow>

Workday with co-CEOs Aneel Bhusri & Chano Fernandez is an American on-demand financial management and *Human Capital Management - HCM* software vendor. Workday was founded by PeopleSoft founder and chief strategist Aneel Bhusri following Oracle's hostile takeover in 2005. Workday is surging on the acquisition of Adaptive Insights and has the new-found ability to deliver an entire ERP suite. It is a flagship *Human Capital Management - HCM* but also ERP business and particularly Financial. The company claims its deployment methodology is reducing customer costs

significantly, and that it intends to push that new model out across the globe.

https://en.wikipedia.org/wiki/Workday,_Inc.

Accenture whose CEO is Julie Sweet is headquartered in Dublin, Ireland. Formerly it was known as Andersen Consulting (1989–2000). Accenture's ability to offer strategic guidance along with highly innovative technology has made it a rapidly emerging force in the cloud marketplace.

<https://en.wikipedia.org/wiki/Accenture>

Gartner's magic quadrant for cloud services in Sept 2020 is provided below.

<https://www.c-sharpcorner.com/article/top-10-cloud-service-providers/>

The following is a prediction of increase in revenue in following years.

<https://www.gartner.com/en/newsroom/press-releases/2020-07-23-gartner-forecasts-worldwide-public-cloud-revenue-to-grow-6point3-percent-in-2020>

Worldwide public cloud service revenue forecast in millions of USD in different categories are as follows:

Business Process Services (BPaaS) 2019/20/21/22 45,212/43,438/46,287/49,509

Application Infrastructure Service (PaaS) 2019/20/21/22 37,512/43,498/57,337/72,022

Application Services (SaaS) 2019/20/21/22 102,064/104,672/120,990/140,629

Management and security service (CMSS) 2019/20/21/22 12,386/14,663/16,089/18,387

System Infrastructure Services (IaaS) 2019/20/21/22 44,457/50,393/64,294/18,387

Desktop as a Service (DaaS) 2019/20/21/22 616/1,203/1,951/2,535

Total market 2019/20/21/22 242,699/257,867/306,948/364,062

Business Management Company - BMC which had acquired BGS in 1998.

<https://www.bizjournals.com/houston/stories/1998/02/09/newscolumn3.html>

BMC's *Multi-Cloud Service Management - MCSM* helps business services across multi-cloud, multi-device, and multi-channel environments with the highest accuracy and speed, and at the lowest cost. BMC makes it easy to streamline service delivery and meet user demands across the range of on-premises, cloud, and hybrid IT services. MCSM ensures smooth migrations, on how to assess what to migrate, what it will cost, and how to keep it secure. MCSM ensures the following:

<https://www.bmc.com/it-solutions/multi-cloud-management.html>

2.29 Cloud storage companies

A list of companies providing storage in the cloud is as follows.

<http://www.cloudstoragebest.com/cloud-storage-types/>

There are companies charging less than \$5 per month for unlimited storage.

Dropbox Standard Contract

\$12.50 / user / month, starting with 3 users

3 TB secure storage

Sharing and collaboration tools

120 days of file recovery

256-bit AES and SSL/TLS encryption

Smart Sync

Dropbox admin tools

Office 365 integration

Admin console and audit log

Granular sharing permissions

User and company-managed groups

Remote device wipe

Two-factor authentication (2FA)

Unlimited API access to security and productivity platform partners.

Amazon Web Services - AWS cloud storage service S3 was introduced in 2006, followed by Oracle Cloud Storage, and Microsoft Azure Storage. Openstack Swift is an object storage software, object storage systems like EMC Atmos, EMC ECS and Hitachi Content Platform.

Cloud storage systems are compared below.

<https://www.imore.com/which-cloud-storage-has-best-value>

The following discussion of the advantages of cloud storage.

1. Companies need to pay for storage they actually use, typically based on average consumption during a month. Cloud storage may not be less expensive, but incurs OpEx - Operating Expenses rather than CapEX - Capital Expenses.
2. Businesses using cloud storage can cut their energy consumption by 70% to make them look like a green business.
3. Organizations can choose between off/on-premises cloud storage options or a mixture. On-premises software is installed and runs on computers on the premises rather than at a remote facility such as a server farm or cloud. There are the following considerations:

Continuity of operations.

Disaster Recovery.

Security: Personally Identifiable Information - PII.

Health Insurance Portability and Accountability Act - HIPAA.

Sarbanes-Oxley - SARBOX financial compliance act.

Information Assurance/Computer Network Defense - IA/CND.

Records retention laws, regulations, and policies.

4. Storage availability and data protection requirement for object storage architecture.
5. Storage service provider carries out storage maintenance tasks, such as purchasing additional storage capacity allowing users to concentrate on their core business.
6. Cloud storage provides users with access to a broad range of resources and applications hosted in the infrastructure of another organization via a web service interface.
7. Cloud storage can be used for copying virtual machine images from the cloud to on-premises locations.
8. Cloud storage can deal with natural disasters using two or three backup servers at different locations.
9. Cloud storage can function as a central file server for organizations with multiple office locations.

Outsourcing data storage increases the attack surface area.

<https://www.sans.edu/cyber-research/security-laboratory/article/did-attack-surface>

1. The risk of unauthorized physical access to data increases with degree of replication, which depends on customer's service level. Encryption can be used to ensure confidentiality.
2. The number of people with access to the data, who could be compromised increases dramatically, since cloud storage companies have a much larger technical staff than a single company and this increases with degree of replication.
3. There is an increase in the number of networks over which the data travels. Instead of just a LAN or SAN data stored on a cloud requires a WAN.
4. Sharing storage and networks with many other users makes it possible for them to access your data via faulty actions or equipment, bugs, and malicious intent.

Considerations in choosing a storage cloud supplier

These include supplier stability, cloud accessibility, and other considerations.

Supplier stability

Outsourcing data storage to another company needs careful investigation, since a company may go out of business or the services it provides can change over time. Contracts become worthless when a company ceases to exist due to the following reasons:

1. Bankruptcy.
2. A major disaster, fire, flooding, earthquake.
3. Change of focus.
4. Purchased by another company.
5. Move to another country.

Cloud accessibility:

1. Depending on WAN bandwidth outsourced storage performance is likely to be lower than local storage.
2. Reliability and availability depends on *Wide Area Networks* - WANs and on the level of precautions taken by the service provider. Reliability is based on hardware as well as the algorithms used.
3. Increases with multiplicity of data storage.

Other concerns

1. Security of stored and in transit data is a concern when dealing with cloud storage providers.
2. Public agencies that must ensure the confidentiality of electronic records may encounter complications with using cloud computing and storage. The U.S. *Department of Defense - DoD* designated *Defense Information Systems Agency - DISA* maintains a list of records management products for records retention, *Personally Identifiable Information - PII*, and security (*Information Assurance - IA* requirements).
3. Cloud storage is a rich resource for both hackers and national security agencies.
4. Piracy and copyright infringement may be enabled by sites that permit filesharing.
5. The legal aspect, from a regulatory compliance standpoint, is a concern when storing files domestically and especially internationally.

2.30 Distributed systems research related to clouds

Leslie Lamport who has won the Turing award for his contributions to distributed systems has made the following comment about them:

<http://www.munzandmore.com/2016/ora/distributed-systems-failure-unknown-computer-leslie-lamport>

“The failure of a computer you did not even know existed can render your computer unusable.”

OceanStore and Inktomi at Berkeley and data replication are topics discussed in this section.

2.30.1 OceanStore

OceanStore was designed by John Kubiatowicz et al. at UCB for global-scale distributed storage utility infrastructure scalability and to automatically recover from server and network failures, incorporate new resources and to adjust to usage patterns (Rhea et al., 2001).

The OceanStore maintenance-free cloud storage has the following characteristics.

1. Multiple distributed resources act as one.
2. Fault tolerance through redundancy and data distribution.
3. Highly durable through the creation of versioned copies.
4. Eventually consistent with regard to data replicas according to AWS’s Werner Vogels (Vogels, 2008).
https://en.wikipedia.org/wiki/Eventual_consistency

Pond was an OceanStore prototype containing many of the features of a complete system such as:

1. Location independent routing,
2. Byzantine update commitment,
https://en.wikipedia.org/wiki/Byzantine_fault
3. Push-based update of cached copies through an overlay multicast network (Eriksson, 1994) and continuous archiving to an erasure-coded farm (Rhea et al., 2003).

Other examples of dispersed storage systems are as follows:

Pond which was an OceanStore prototype (Rhea et al., 2003),

PASIS by Garth Goodson, Jay Wylie, Greg Ganger, Michael Rieter at PDL/CMU (Goodson et al., 2004),

Glacier by Andreas Haeberlin, Alan Mislove, and Peter Druschel (Max Planck Inst.) at Rice U. (Haeberlen et al., 2005),

Gridsharing by Arun Subbiah and Douglas Blough (UCI) at GaTech (Subbiah and Blough, 2005),

Pergamum by Mark Storer, Kevin Greenan, and Ethan Miller at SSRC/UCSC (Storer et al., 2008),

POTSHARDS by the same team at UCSC (Storer et al., 2009),

Tahoe-Least Authority Files Store-LAFS. <https://tahoe-lafs.org/trac/tahoe-lafs>

2.30.2 Inktomi and CAP theorem

Inktomi was founded by Eric Brewer and Paul Gauthier at UCB in 1996.

<https://en.wikipedia.org/wiki/Inktomi>

The intent was to provide software for *Internet Service Providers - ISPs* Inktomi was acquired by Yahoo! in 2003 after the dot-com bubble burst in 2001 Yahoo! since 2019 is part Verizon Media and is merged with *America OnLine - AOL*.

<https://en.wikipedia.org/wiki/Yahoo!>

<https://en.wikipedia.org/wiki/AOL>

Based on his experience about distributed systems Brewer made the following observation at *Principles of Distributed Computing* - PODC'00 Conf. Brewer made an attempt to clarify the way we think about distributed systems, including the fault model, high availability, graceful degradation, data consistency, evolution, composition, and autonomy, by enunciating the Consistency, Availability and Partitioning-CAP theorem (Brewer, 2010):

Even the ones (distributed systems) that work, tend to be very fragile, they are hard to keep up, hard to manage, hard to grow, hard to evolve, and hard to program.

The CAP theorem states that networked shared data system can only have two out of the following three properties: It was stated twelve years later that by explicitly handling partitions, designers can optimize consistency and availability, thereby achieving some trade-off of all three features (Brewer, 2012).

<https://www.infoq.com/articles/cap-twelve-years-later-how-the-rules-have-changed>

Distributed system must reconcile differences between multiple copies of distributed data. (1) By exchanging updates between servers. (2) Choosing an appropriate final state when concurrent updates have occurred is called reconciliation. A widespread approach is “last writer wins” Werner Vogels at Amazon (Vogels, 2008), but “first writer wins” is also used.

2.30.3 Replicated data

Data may be replicated at multiple network nodes for performance and availability as noted in the PhD thesis of Hector Garcia-Molina at Stanford U. (Garcia-Molina, 1982). Eager replication keeps all replicas exactly synchronized at all nodes by updating them as part of an atomic transaction resulting in serializable execution (Ramakrishnan and Gehrke, 2002). Eager replication has poor performance, low throughput and increased transaction response times, because extra updates and messages are added to the transaction. Eager replication results in deadlocks whose frequency is estimated in (Gray et al., 1996b). It is not an option for mobile applications where most nodes are normally disconnected.

Mobile applications require lazy replication algorithms that asynchronously propagate replica updates to other nodes after the updating transaction commits. Lazy replication can be used even in continuously connected systems to improve response time, but may require reconciliation (Gray et al., 1996b).

Update anywhere-anytime-anyway transactional replication has unstable behavior as the workload scales up: a ten-fold increase in nodes and traffic results in a thousand fold increase in deadlocks or reconciliations. Master copy replication (primary copy) schemes reduce this problem. A simple analytic model demonstrates these results. If the txn profiles are not constrained, lazy-group schemes suffer from unstable reconciliation. A two-tier replication algorithm is proposed in (Gray et al., 1996b) that allows mobile (disconnected) applications to propose tentative update transactions that are later applied to a master copy. Commutative update transactions avoid the instability of other replication schemes. If the transaction profiles are not constrained, lazy-group schemes suffer from unstable reconciliation.

Discussion by Rockford Lhotka

Component-based Scalable Logical Architecture - CSLA.NET is a software development framework that helps in building reusable, maintainable object-oriented business layer for applications.

[CSLA.net](http://www.lhotka.net)

<http://www.lhotka.net/Article.aspx?id=890d3e3c-8a49-486c-ae48-a44e7e1f7844>

https://en.wikipedia.org/wiki/Component-based_Scalable_Logical_Architecture

The CSLA.NET architecture handles concurrency at the database level, rather than object level, since this would have required a distributed lock manager, which would have a large impact on performance, scalability and fault tolerance, but this is not so when addressing concurrency at the database level, since this allows us to take advantage of pre-existing functionality.

Only optimistic concurrency is supported, since pessimistic concurrency in a distributed environment requires a distributed lock manager. With optimistic concurrency there are two options:

Last person to write wins is easiest to implement, since it requires no extra work and provides the highest performance. Implementing “first write wins” has definite performance costs. There are three primary options and many variations:

Timestamp: A timestamp column need to be added to every database table. All business classes keep the timestamp value for the data in their object as a Private variable marked as <NotUndoable()>. If an object has data coming from multiple tables, there may be multiple timestamps. When you go to update or delete data you must first retrieve the existing timestamp values for each row of data before updating that row. The value in the object is compared to the one in the table and update occurs if they match.

Cyclical Redundancy Check-CRC: CRC is a numeric value calculated based on a set of bytes of data. When data is loaded from the database into an object the CRC is immediately calculated and the CRC is stored in the object. CRC is less invasive because it doesn't require any changes to the database schema, which is critical for preexisting databases. To update or delete data you must first retrieve the existing data from each row and use it to calculate a CRC. The user can then compare this CRC with the value in the object and if they match the update proceeds.

Field comparisons: This approach has the highest overhead, but is the more powerful than the timestamp or CRC approaches, because it allows interleaved updates, where users can concurrently update different fields of the same object without blocking each other.

2.30.4 Sky Computing

Sky Computing is proposed in (Stoica and Shenker, 2021) is made up of three layers: “a compatibility layer to mask low-level technical differences, an intercloud layer to route jobs to the right cloud, and a peering layer that allows clouds to have agreements with each other about how to exchange services.”

Compatibility layer will enable an application developer to move their app from one cloud to another.

Intercloud layer is one level up from compatibility layer. It will allow applications to run across multiple cloud providers – depending on user preferences, e.g., minimize cost, time, the need to process the data locally. The intercloud layer decides where to run your job.

Peering layer: Moving data into cloud is cheaper than moving it out, i.e., data gravity pricing. But Still, moving data from one cloud to another can still be the most cost-effective option, especially for jobs where the computation resources are much more expensive than the data transfer costs. If data is dynamically updated the incentives against job movement are most acute. Current pricing policies for exporting data are independent of the cloud the data might be going to. An alternative to be explored to date is for clouds to enter into reciprocal data peering arrangements, where they agree to allow free exporting of data to each other, and to connect with high-speed links.

The analogy between the Internet and Sky Computing is summarized in Table 2.37.

TABLE 2.37 Analogy between the Internet and Sky Computing.

Internet	Sky
Router	Server
Autonomous system	Datacenter / Availability zone
ISP Internet Service Provider	Cloud provider
Enterprise network	Private cloud
Internet protocol	Computability layer
BGP Border Gateway Protocol	Intercloud layer

Ali Khodsi the CEO of \$28 billion AI startup Databricks says sky computing is the future, with Amazon, Microsoft, and Google becoming more open cloud platforms. [newline <https://www.businessinsider.com/databricks-ceo-sky-computing-aws-microsoft-google-cloud-open-2021-8>

2.31 Data encryption

In cryptography, encryption is defined as the process of encoding information in such a way that only authorized parties can access it. <https://en.wikipedia.org/wiki/Encryption>

The Enigma machine was an encryption device developed in Germany, which was employed extensively during World War II.

https://en.wikipedia.org/wiki/Enigma_machine

The deciphering of Enigma started in Poland and reached fruition in England by a team led by mathematician Alan Turing.

https://en.wikipedia.org/wiki/Cryptanalysis_of_the_Enigma

https://en.wikipedia.org/wiki/Alan_Turing

The movie “The Imitation Game” deals with mathematician Alan Turing’s attempt to decipher Enigma.

https://en.wikipedia.org/wiki/The_Imitation_Game

The Russians had their own machine which was exhibited at the KGB Espionage Museum in New York City in 2019.

https://getpocket.com/explore/item/the-soviets-unbreakable-code?utm_source=pocket-newtab

<https://kgbespionagemuseum.org/>

The following discussion of data encryption is based on “Cryptography and Network Security: Principles and Practice” (Stallings, 2017a).

The *Data Encryption Standard - DES* was adopted in 1977 by NIST as the most widely used data encryption standard known as *Federal Information Processing Standard - FIPS*.

https://en.wikipedia.org/wiki/Federal_Information_Processing_Standards

The algorithm itself is referred to as the *Data Encryption Algorithm - DEA*. Data are encrypted in 64-bit blocks using a 56-bit key, so that the algorithm transforms 64-bit input in a series of steps into a 64-bit output. Same steps, with the same key, are used to reverse the encryption (Konheim, 2010).

The *Advanced Encryption Standard - AES* first published in 1998 was intended to replace DES for commercial applications. It uses a 128-bit block size and a key size of 128, 192, or 256 bits. Encryption is carried out with four separate functions:

1. byte substitution,
2. permutation,
3. arithmetic operations over a finite field,
4. XOR with a key.

https://en.wikipedia.org/wiki/Advanced_Encryption_Standard

Advances in Quantum Computing have recently revealed weaknesses in AES and this is leading to new proposals.

<https://www.bloomberg.com/news/articles/2021-02-07/a-swiss-company-says-it-found-weakness-that-imperils-encryption>

RSA Public Key Encryption - PKE was developed by Ronald Rivest (MIT), Adi Shamir (Weizman Inst.), Leonard Adleman (USC) in 1977.

[https://en.wikipedia.org/wiki/RSA_\(cryptosystem\)](https://en.wikipedia.org/wiki/RSA_(cryptosystem))

RSA allows authentication about the source of the message.

<https://cloudarchitectmusings.com/2018/02/08/encrypting-your-data-in-the-cloud-part-2-encryption-101/>

This led to a Turing award for its three authors in 2002 with an award ceremony at MIT in 2003.

<https://www.technologyreview.com/s/402303/rsa-turing-award-lecture/>

CryptDB developed at MIT by Raluca Ada Popa et al. executes SQL queries at the server over encrypted data and sends the results back to the client for decryption (Popa et al., 2011), (Popa et al., 2012). CryptDB has three novel ideas:

1. An SQL-aware encryption strategy that maps SQL operations to encryption schemes.
2. Adjustable query-based encryption, which allows CryptDB to adjust the encryption level of each data item based on user queries.
3. Onion encryption to efficiently change data encryption levels.

CryptDB has modest overhead. For example on the TPC-C benchmark on Postgres it reduces throughput by 27% compared to regular Postgres. Importantly, CryptDB does not change the innards of existing databases, as such, CryptDB is portable; and porting CryptDB to MySQL required changing only 86 lines of code.

Relevant to this discussion is *Fully Homomorphic Encryption - FHE*, which allows direct mathematical operations on the encrypted data. Upon decryption, the results will be correct.

https://en.wikipedia.org/wiki/Homomorphic_encryption

For example, a user may encrypted 2, 3, and 7 to a third party and ask him to add the first and second values, then multiply the result by the third value and return the result. The user can then decrypt that result and get 35.

The user will never have to share a key with the third party doing the computation; the data remains encrypted with a key the third party never received. So, while the third party performed the operations you asked it to, it never knew the values of either the inputs or the output.

FHE comes at a steep cost in additional compute power and memory resources required to operate on FHE-encrypted machine-learning models - roughly 40 to 50 times the compute and 10 to 20 times the RAM that would be required to do

the same work on unencrypted models.

<https://arstechnica.com/gadgets/2020/07/ibm-completes-successful-field-trials-on-fully-homomorphic-encryption/>
<https://www.engadget.com/facebook-analyze-encrypted-messages-ad-targeting-175739715.html>

2.31.1 Data encryption for cloud storage

Data encryption is important in a cloud storage environment, since the data is not held locally. In the case of storage systems we are dealing with data-at-rest, although similar data encryption methods are applicable to data in transit.

https://en.wikipedia.org/wiki/Data_at_rest

AWS provides server and client side security services (Li et al., 2013).

<http://docs.amazonwebservices.com/AmazonS3/latest/dev/UsingServerSideEncryption.html>

<http://aws.amazon.com/articles/2850096021478074>

Microsoft's Azure data security and encryption best practices are as follows.

<https://docs.microsoft.com/en-us/azure/security/azure-security-data-encryption-best-practices>

BitLocker is a full volume encryption feature included with Microsoft Windows versions starting with Windows Vista. It is designed to protect data by providing encryption for entire volumes. By default, it uses the AES encryption algorithm in cipher block chaining or XTS mode with a 128-bit or 256-bit key.

<https://en.wikipedia.org/wiki/BitLocker>

An overview of the bitlocker data encryption method available for Windows OS is given here.

<https://docs.microsoft.com/en-us/previous-versions/windows/it-pro/windows-server-2012-R2-and-2012/hh831713%28v%3dws.11%29>

A tutorial on data encryption on the web is provided by Ken Hui.

<https://cloudarchitectmusings.com/2018/02/08/encrypting-your-data-in-the-cloud-part-2-encryption-101/>

A table in the article by Ken Hui (Table 2.38) summarizes the encryption offered by Amazon S3, Azure blob storage, and Google cloud storage.

<https://cloudarchitectmusings.com/2018/03/09/data-encryption-in-the-cloud-part-4-aws-azure-and-google-cloud/>

All three providers use the 1998 AES256 symmetric key encryption, but the first two also use *Galois Counter Mode - GCM* for authenticated encryption by adding a unique tag to the ciphertext.

https://en.wikipedia.org/wiki/Advanced_Encryption_Standard

Compressing data before encryption not only makes for shorter messages to be transmitted or stored, but also improves security by reducing the redundancy in the plaintext and making cryptanalysis harder. There is even a patent from NetApp to do things in two steps.

A single-pass data compression and encryption method is given in (Paris and Mackay, 2012) Embodiments compress and encrypt data in a single pass to reduce inefficiencies that occur from compression and encrypting data separately. Typically, compression and encryption are implemented in separate functional units. This has a few disadvantages:

1. Encryption cannot make use of compression state to further secure the message.
2. Processed data is read and written twice.
3. Additional space, time, and resources are consumed.
4. It is more prone to potential cipher-attacks since the encryption stage is independent from compression.

Embodiments overcome these disadvantages by structuring these operations so that both compression and encryption is executed within the same processing loop. Thus:

1. Encryption is stronger due to the dependence on the compression state,
2. I/O buffers are accessed only once reducing overhead,
3. System footprint is reduced,
4. Cipher analysis is more complex since the decryption process cannot be separated from the decompression process.

The practice of concealing a message within another message or a physical object dates back to ancient Greeks. In computing/electronic contexts, a computer file, message, image, or video is concealed within another file, message, image, or video. The word steganography comes from Greek steganographia, which combines the words steganos meaning "covered or concealed" and -graphia meaning "writing"

<https://en.wikipedia.org/wiki/Steganography>

Modern steganography entered the world in 1985 with the advent of personal computers being applied to classical steganography problems. Development following that was very slow, but has since taken off, going by a large number of steganography software available:

TABLE 2.38 Data Encryption in the Cloud, AWS, Azure and Google Cloud by Ken Hui, KMI = Key Management Infrastructure, KMS = Key Management Service, CMK = Customer Master Key, HSM = Hardware Security Modules.

	Amazon S3	Azure Blob Storage	Google Cloud Storage
Encryption methods			
Server side encryption	Yes	Yes	Yes
Client side encryption	Yes with encryption client provided	Yes with encryption client provided	Allowed but no encryption client provided
Key management			
Symmetric key encryption	* AES-256 GCM * Used for both key and data encryption & with SSE-S3, SSE-KMS, and client side encryption (both options) * Used for data encryption with SSE-C	* AES-256 * Used for data encryption both server & client side encryption * Can be used for encryption with client side encrypt	* AES-256 GCM * Used for both key and data encryption
Asymmetric key encryption	* RSA Can be used for key encryption with client-side encryption using client side master key	* RSA * Used for key encryption with server-side encryption (both options) and with client side encryption	Can be used for client side encryption but no integration
Customer stored and managed	Yes for SSE-C and client side encryption using client side master key	Yes for client side encryption	*Yes for server side encryption with customer supplied KEK can be used for client-side encryption, but no integration provided
Cloud provider stored and customer managed (using their own KMI)	Yes for SSE-C and client Side encryption using client-side master key (using cloudHSM)		
Cloud provider stored and customer managed (Using cloud key mgmt service)	Yes for SSE-KMS and Client-side encryption using KMS managed CMK	Yes for server side encryption with customer-managed keys and client-side encryption (both using Azure key vault)	
Cloud provider stored and managed	Yes for SSE-S3	Yes for server-side with service managed keys	Default encryption method

2.32 Conclusions - predictions about storage systems

We conclude with the following four conclusions made by Jai Menon about storage systems in 2017.

<https://medium.com/jai-menons-blog/four-storage-predictions-7284788b45d1>

The rise of cloud, the emergence of integrated systems, and new interconnect technologies are all impacting storage. Menon's blog was discontinued after he joined Fungible.

<https://www.prnewswire.com/news-releases/former-dell-and-ibm-executive-joins-fungible-as-chief-scientist-300926044.html>

2.32.1 Resurgence in shared storage, but Fibre-Channel fades

Storage Area Networks - SANs provide shared block storage to multiple servers. There has been a trend away from SANs to *Direct Attached Storage* - DAS. Speculation about the death of SANs is as follows:

1. SAN latencies are too high for use with faster SSD storage.
2. SAN arrays are expensive and proprietary and often use purpose-built hardware and premium-priced drives.
3. SANs are complex to manage and those based on the *Fibre Channel* - FC standard require unique host bus adapters (HBAs), switches, and cables.

https://en.wikipedia.org/wiki/Fibre_Channel

They also use non-intuitive storage-centric management.

4. *virtual SANs* - vSANs do not have the above three disadvantages. vSAN software aggregates and shares direct attached storage across a cluster of compute servers.

There are the following four points:

- SANs have traditionally used the SCSI protocol over FC, but future SANs will use very low-overhead *NVMe extended to work on Fabrics* - NVMeoF protocol over Ethernet. With NVMeoF, so that there is no latency difference between local and remote SAN storage. E8 storage's SAN array supports 10 Million IOPS with latencies of 40–100 microseconds.
<https://e8storage.com/10x-performance-e8-storage-launches-first-ever-centralized-nvme-solution-for-enterprise-storage/>
- Future SAN arrays are being built as software on standard x86 servers (e.g. Hedvig, Formation data systems, Tintri) and there is no premium on drive pricing. Hedvig was purchased by Commvault.
<https://www.commvault.com/software-defined-storage>
- SAN arrays are now providing intuitive app-centric management (e.g. Tintri). This, combined with use of NVMeoF over Ethernet, addresses issue (3) above.
- Virtual SANs have disadvantages such as limited scalability, the approaches they use for high availability are inefficient compared to RAID, and they are very dependent on good data locality for performance.

Menon believes that shared storage SANs (NVMeoF over Ethernet) will see a resurgence and FC which has been synonymous with SANs, will fade away.

Emergence of a new storage hierarchy – persistent memory, flash, cloud

The current storage hierarchy from fastest to slowest is as follows:

1. server-attached NVMe Flash,
2. shared storage using write-intensive Flash SSDs,
3. read-intensive SSDs,
4. high-speed disk drives,
5. near-line disk drives,
6. tape or cloud.

It is predicted that the future storage hierarchy will be much simpler and consist of (1) persistent memory, (2) Flash and (3) Cloud. The following breakdown storage is provided in a recent article: 90% disk and 10% flash.

<https://blocksandfiles.com/2021/02/22/seagate-cto-says-ssds-will-not-kill-disk-drives/>

Persistent memory is an emerging new layer between DRAM and Flash accessible using memory protocols. It is non-volatile with latency close to DRAM (100s of nsecs), 4–5X higher capacity, and lower price than DRAMs. Many technologies are vying to be this persistent memory layer including 3D Xpoint from Intel/Micron, Resistive RAM from HPE/Sandisk, TLC PCM from IBM and various forms of NVDIMMs that are DRAM and Flash combinations. Too early to predict a winner given the performance, durability and date slippages with 3D Xpoint which was the front runner to me. Much work remains to be done on the software side with OS and file system support before this new persistent memory layer becomes a reality.

The next layer after Persistent memory will be Flash in the server or Flash in a SAN array With NVMe and NVMeoF.

Menon predicts that 15k RPM HDDs will not be available past 2017 and 10k RPM HDDs past 2018. Higher RPM disks are relevant for applications such as video distribution. In most other cases IOPS which is the inverse of mean rotational latency plus seek time is the more important parameter.

In the case of a 300 GB 7200 RPM drive which is four times bigger than a 73 GB 10K drive, quarter stroking can be used to obtain a high-performance 75 GB partition along with a 225 GB partition for archiving.

Quarter stroking reduces seek time by 6.68 ms we can actually shave 6.68 ms off the seek time and an average access time of 6.4 ms, which is faster than the 10K RPM “performance” drive. A similar discussion appears here.

<https://www.zdnet.com/article/how-higher-rpm-hard-drives-rip-you-off/>

Combined with the lower power, cooling and support needs of Flash, all-flash TCO is superior to hybrid Flash/disk TCO or all-disk TCO.

With the rapid drop in pricing for cloud storage the most future archival and backup storage being in the cloud. Cloud storage is most likely to be based on low cost disk drive technology, however a case can be made that next generation QLC Flash (4 bits per cell).

<http://storageconference.us/2016/Slides/RobPeglar.pdf>

A technology winner for cloud storage is yet to be determined.

To summarize, Menon calls Flash as the new disk, Cloud as the new tape, and a new persistent memory layer emerging between DRAM and Flash.

Stand-alone primary storage diminishes in importance

Customers select and purchase compute, storage and networking products separately. The dominant way to purchase primary storage continues to be as stand-alone storage. The problem with this 3-tier approach is that it can take several months of integration and testing before customer's application are able to run on separately purchased infrastructure.

Integrated systems where compute, storage and networking were purchased together emerged in 2007. First generation integrated systems were called *Converged Infrastructure - CI* systems. Separate products are pre-tested and pre-integrated by the vendor, reducing the time it takes a customer to run their first applications following a CI purchase. While CI systems have good performance and scalability, they are still too complex to deploy and manage.

Starting in about 2012, customers started to purchase 2nd generation HCI integrated systems. HCI uses clusters of compute servers with DAS storage and virtual SAN software. Unlike CI, which is really three pretested products sold together, HCI is truly a single product with unified management. They are much easier to deploy and manage than CI systems.

More and more, storage will be sold as part of an integrated system such as converged or hyperconverged or whatever they evolve to become. Due to the compelling ease of use and fast time to value of integrated systems, stand-alone storage products are expected to diminish in importance and gradually disappear.

New form of cloud integrated system

According to 451 Research's latest quarterly Voice of the Enterprise Survey of IT buyers, hyperconverged infrastructure is currently in use at 40% of organizations, and 451 Research analysts expect that number to rise substantially over the next two years. Gartner also predicts that HCI will be the dominant form of integrated system by 2020.

451research.com from *Standards & Poor - S&P* and Gartner are among those who believe that hyperconverged systems will be the dominant and perhaps ultimate form of integrated system for the foreseeable future.

Menon predicts that there will be other forms of integrated systems beyond HCI, which in spite of their advantages have limited scalability (16–32 nodes), inefficient resource consumption (storage and compute are coupled), inefficient *High Availability - HA* (replication or erasure-coding versus RAID), and dependence on data locality for good performance.

There is a third generation of integrated system that combines the performance and scalability of CI with the ease of use of HCI, while providing full cloud capability (self-service, multi-tenancy, metered service, elasticity, network delivered, hyper-scale). Menon's predicts they integrate an all-flash shared storage array as part of the integrated system. An example of such a system is the Cloudistics composable cloud platform.

<http://www.cloudistics.com>

Such integrated systems are different from HCI by allowing independent scaling of compute and storage, and they are different from CI because they are not the integration of three separate products, but are truly created as a single product with unified management and are different from both CI and HCI in their ability to provide the full cloud experience.

Note that Sky Computing is a more recent development.

Books related to storage systems are listed in the Appendix.

Chapter 3

Disk drive data placement and scheduling

The organization of hard disk drives in Section 3.1
Internal organization of files in UNIX in Section 3.2
Review of disk arm scheduling in Section 3.3
Implementation of Shortest Access Time First - SATF in Subsection 3.3.1
Disk performance studies by Windsor Hsu and Alan Jay Smith in Section 3.3.2
Linux support for disk scheduling in Subsection 3.3.3
Disk scheduling for mixed workloads in Section 3.4
Real time disk scheduling for multimedia in Section 3.5
Storage virtualization on disks in Section 3.6
Disk file placement in Section 3.7
Anticipatory disk arm placement in Section 3.7.1
Shingled Magnetic Recording - SMR in Section 3.8
Analyses of disk scheduling methods in Section 3.9
Analytic studies of disk storage in Section 3.10
Performance analyses of disk drives with FCFS scheduling in Section 3.11
Disk service time in zoned disks with FCFS scheduling in Subsection 3.11.1
Performance analyses of disk drives with SCAN scheduling in Section 3.12
Performance analyses of disk drives SATF scheduling in Section 3.13
Conclusions in Section 3.14

3.1 The organization of Hard Disk Drives - HDDs

Magnetic *Hard Disk Drives* - HDDs are described in (Ruemmler and Wilkes, 1994) and by Spencer W. Ng in (Ng, 1998), who coauthored “Cache, DRAM, Disk” (Jacob et al., 2008).

HDDs consist of one or more platters, which are connected to a shaft which rotates them together at a fixed angular speed specified by *Rotations Per Minute* - RPM. Data is recorded on concentric magnetic tracks, usually on both sides of a platter. Tracks with the same radius constitute a cylinder and can be accessed together by *Read/Write* - R/W heads on a disk arm. This is the reason why large files are usually written sequentially on successive disk tracks on the same cylinder before proceeding to the next cylinder. The time that it takes to move the disk arm to the appropriate cylinder from its current position is called seek time.

Computer disks utilize *Constant Angular Velocity* - CAV, which has the advantage with respect to *Constant Linear Velocity* - CLV with helical recording (on 33 or 45 RPM vinyl *Long Playing* - LP disks) in that the drive mechanism is easier to build and less expensive and allows switching from one cylinder to another without changing disk speed.

https://en.wikipedia.org/wiki/Constant_angular_velocity#/

https://en.wikipedia.org/wiki/Constant_linear_velocity

Music was recorded on disk, tapes, cartridges, cassettes, optical *Compact Disk* - CDs is being replaced by streaming into Ipods, but later into iPhones and iPads from iTunes or other sources. https://en.wikipedia.org/wiki/Music_download

https://en.wikipedia.org/wiki/Phonograph_record

<https://www.cdrom2go.com/blog/a-brief-history-of-recorded-music>

IBM was an early leader in developing magnetic disks. Large form-factor disks with the *Count Key Data* - CKD format were developed for the IBM S/360 mainframe computers and its successors became a standard. CKD disk format later evolved to *Extended CKD* - ECKD format (Bohl, 1981), (Wiederhold, 1987), (Sierra, 2012) (originally 1990), (Jacob et al., 2008).

https://en.wikipedia.org/wiki/Count_key_data

A catalog provides dataset attributes and the volumes on which they are located. Generation data groups make it possible to catalog successive updates. *Data Facility Storage Management Subsystem - DFSMS* manages space, and performs the essential data storage, program, and device management functions of the system.

https://www.ibm.com/support/knowledgecenter/SSLTBW_2.2.0/com.ibm.zos.v2r2.idai500/dgt3i511.htm

Volume Table of Contents - VTOC is a directory that locates datasets on a disk.

https://en.wikipedia.org/wiki/Volume_Table_of_Contents

It is the functional equivalent of the Microsoft DOS *File Access Table - FAT* or the UNIX root directory (Bach, 1986).

ECKD disk format is emulated by *RAID Disk Array Controller - DAC* on *Fixed Block Architecture - FBA* format disks with 512 byte sectors.

https://en.wikipedia.org/wiki/Disk_sector

This is because key mainframe software, especially z/OS, which is a descendant of the MVS operating system was written to operate on 3390 disks with ECKD format.

<https://en.wikipedia.org/wiki/MVS>

The advanced format 4096 byte sectors increased the efficiency of the data surface area by 5-13%, while increasing the strength of the ECC which in turn allowed higher capacity. The standardized format was incorporated in new *Hard Disk Drives - HDDs* by 2011.

https://en.wikipedia.org/wiki/Advanced_Format

The following steps are followed in accessing data.

Dealing with variable block sizes on a fixed block size architecture

1. The database buffer in main memory is checked to determine if the requested data block is cached.
2. An I/O request is issued if it is not cached.
3. An access to a single virtual 3390 disk block may result in accesses to multiple disks when its data spans them. This is referred to as *multiple exposure* and is a fork/join request to disks (Thomasian, 2014b). The DAC translates combines disk blocks to obtain the requested block. This inefficiency is prevented by using larger stripe unit or strip sizes or parity striping, i.e., no data striping at all, which was proposed in (Gray et al., 1990).
4. In the case of write the original block is split according to strip boundaries in RAID5 (Chen et al., 1994b). It is augmented with cached data to create cachable data blocks, which are saved in *Non-Volatile Storage - NVS*, before they are destaged in a batch as NVS space is being exhausted (Treiber and Menon, 1995),(Varma and Jacobson, 1998).

Several terabyte drives have recently broken the \$0.10/GB - GigaByte barrier, making the next milestone \$0.01/GB, or \$10/TB - TeraByte. Increasing disk capacities are expected to lead to higher access rates to individual disks, based on the fact that there is an inherent IO density associated with blocks, but it should be noted that very high capacity disks are used for archival storage and not high performance applications. In what follows we describe a related study at NetApp.

IO Density is the measurement of IO generated over a given amount of stored capacity expressed as IOPS/TB, i.e., IO Density measures how much performance can be delivered by a given amount of storage capacity. As a rule of thumb, a single 7.2K RPM drive can deliver around 50 IOPS (at 20 ms response time). Today the 7.2K RPM drives can range from 1 TB to 8 TB in size, but the ability of the drive to deliver 50 IOPS does not change with its size. As the size of the drive increases, the IOPS/TB ratio worsens, e.g., 6.25 IOPS/TB with an 8 TB drive.

Using OnCommand Insight's IO Density report helped NetApp identify the range of service levels, i.e., IOPS/TB, that the apps actually needed. This was used to we create a storage catalog based on three standard service levels.

<https://blog.netapp.com/blogs/the-importance-of-io-density-in-delivering-storage-as-a-service-part-1/>

Value: Services workloads requiring up to 512 IOPS/TB.

Performance: Services workloads requiring between 512 and 2048 IOPS/TB.

Extreme: Services workloads requiring between 2048 and 8192 IOPS/TB.

The next step places IO limits on volumes to prevent applications from stepping on the performance resources of other applications within the same storage array.

<https://blog.netapp.com/blogs/the-role-of-qos-in-delivering-storage-as-a-service-part-2/>

Increased cache sizes are made possible by larger DRAM and flash memories limit the increase in disk access rates to manageable values. Disk access rate can also be reduced by utilizing disk capacity partially, but this defeats the purpose of higher capacity disks. For example IBM's *Transaction Processing Facility* - *TPF* utilized a few cylinders on each disk to reduce disk loads and minimize seek time in what is called short stroking.

https://en.wikipedia.org/wiki/Transaction_Processing_Facility

Higher disk capacity is attained by *Zoned Bit Recording* - *ZBR*, where the linear recording density is held constant across disk tracks, but this is done over many tracks at a time to simplify the bookkeeping. This results in an increased number of sectors on outer tracks with respect to inner tracks, because of their larger circumference.

Shingled Magnetic Recording - *SMR* increases the recording density radially, by writing overlapping tracks as discussed in Section 3.8. The two methods can coexist.

<https://www.networkcomputing.com/data-centers/shingled-magnetic-recording-part-2-using-shingled-drives>

Movable disk arm has a *Read and Write* - *R/W* head per recorded disk surface. There is a head settling time following a seek to ensure that the target track has been reached before writing occurs (Ng, 1998).

The time for the required data blocks to rotate past the *R/W* head on the disk arm is called rotational delay or latency. For accesses to small blocks the mean latency is approximately half the disk rotation time ($T_r/2$).

The *Zero Latency Read* - *ZLR* capability (Ng, 1998), (Jacob et al., 2008) allows a block transfer to start at the first encountered sector of a requested block thus eliminating latency. *ZLR* has a significant impact on the transfer time of larger blocks, e.g., full tracks can be read in one disk rotation, regardless of where the *R/W* head lands on the track. The transfer time of an 8 KB (kilobyte) block with sixteen 512 byte sectors is sixteen times T_r divided by the number of sectors.

A new trend in disk technology is that the access time for up to c cylinders ($10 \leq c \leq 20$) is the same as head settling time as noted in (Schlosser et al., 2005). This has implications on placing sequentially accessed data by writing multiple tracks on one surface, before switching to the next surface as noted in (Shao et al., 2007). Track and cylinder skew and track alignment are also considerations to optimize sequential disk accesses as noted in (Schindler et al., 2002).

D-GRAID, a gracefully-degrading and quickly-recovering RAID storage array to improve system availability and ensure that most files remain available even when an unexpectedly high number of faults occur (Sivathanu et al., 2005). *D-GRAID* also recovers from failures quickly, restoring only live file system data to a hotspare. Both graceful degradation and live-block recovery are implemented in a prototype SCSI-based storage system demonstrating that file system functionality can be implemented on block-based interface.

Continuous Data Protection - *CDP* storage technology enables reverting the state of the storage to previous points in time. Four alternative architectures for supporting *CDP* in a storage controller are proposed in (Laden et al., 2007), and compared analytically with respect to both write performance and space usage overheads. Factors such as the degree of protection granularity and the temporal distance distribution of the given workload affect these overheads. The model allows predicting the *CDP* overheads for arbitrary workloads to determine the best architecture for a given scenario. The analysis is verified by running a prototype *CDP* enabled block device on both synthetic and traced workloads and comparing the outcome with our analysis between *CDP* overheads and a workload's temporal locality. One of the considered architectures is superior for workloads exhibiting high temporal locality w.r.t. granularity, whereas another of the architectures is superior for workloads exhibiting low temporal locality w.r.t. granularity. An *OLTP* workload and a file server workload and show which *CDP* architecture is superior for each workload at which granularities.

Disk seek time was improving slowly at an annual rate of less than 8% at the turn of the century as noted in (Gray and Shenoy, 2000). A similar result is reported in (Hsu and Smith, 2004) and different seek time characteristics for writes and reads is proposed. Disk seek time is higher for writes than reads due to increased head settling time (Ng, 1998).

Disk drives can be categorized according to their usage as *Personal Storage* - *PS* versus *Enterprise Storage* - *ES*. *ES* drives tend to have a higher performance than *PS* drives, e.g., 15,000 RPM for *ES* and 7200 for *PS*. Smaller diameters for platters are required for higher RPMs to lower power consumption, but then more platters are required to attain a higher disk capacity (Sankar et al., 2008).

The *Small Computer System Interface* - *SCSI* is used mainly for *ES*, while the *Intelligent/Integrated Data Electronics* - *IDE* interface or *Advanced Technology Attachment* - *ATA* is used for *PS*. *ATA* stands for *AT Attachment*, while *AT* stands for IBM's *Advanced Technology PC*.

https://en.wikipedia.org/wiki/IBM_Personal_Computer/AT.

Serial ATA - *SATA* was introduced to replace *Parallel ATA* - *PATA* in 2000.

https://en.wikipedia.org/wiki/Serial_ATA

https://en.wikipedia.org/wiki/Parallel_ATA#44-pin_variant

Differences between *SCSI* and *ATA* drives are discussed in (Anderson et al., 2003).

https://en.wikipedia.org/wiki/Hard_disk_drive_interface

There was a 60% increase in disk capacities preceding the turn of the century (Gray and Shenoy, 2000). This increased to 100% in the period 2003-2006, but decreased to 30% since then (Hennessy and Patterson, 2017). There is substantial increase in magnetic disk capacities as new technologies such as it Heat/Microwave/Energy Assisted Magnetic Recording - HAMR/MAMR/EAMR.

https://en.wikipedia.org/wiki/Heat-assisted_magnetic_recording

<https://www.forbes.com/sites/tomcoughlin/2018/12/09/energy-assisted-magnetic-recording-for-enterprise-storage/#3c85b5dcd82d>

To build a nine-platter 40 TB hard drive, Seagate needs to increase the areal density of its media to around 2. TB/in². Seagate projects 100 TB drives in 2030.

<https://www.tomshardware.com/news/seagate-technology-roadmap-2021>

There is a 40% annual improvement in disk transfer rates as a result of increasing linear recording densities and higher disk RPMs. The disk transfer rate of 100 MB/sec is an order of magnitude smaller than DRAM, but about the same as flash memories.

Online Transaction Processing - OLTP is a challenging workload from the viewpoint that transactions access randomly placed disk blocks, so that disk access time is high and is dominated by positioning time. This incurs a large delay in transaction response time. The I/O trace analysis of an airline reservation system was investigated at DEC *System Research Center - SRC* showing that disk accesses are mostly to small blocks: 96% to 4 KB blocks and 4% to 24 KB blocks (Ramakrishnan et al., 1992).

The transfer time for small data blocks is negligibly short with respect to disk positioning time, which is the sum of seek time and rotational latency. Accesses to randomly placed short blocks are inefficient in that a small fraction of time for data transfers constitutes useful work.

Improving the disk access time is important for OLTP applications, because it constitutes a significant fraction of transaction response time, which has stringent response time requirements, e.g., the maximum throughput for OLTP benchmarks is determined by the maximum throughput at a point, where 95th percentile of response time does not exceed a prespecified threshold.

Disk access bandwidth for accesses to randomly placed data is a limited resource in modern disk drives due to their mechanical nature. As shown in (Ruemmler and Wilkes, 1993) the working set of disk data in main memory has been shown to be a few percent of the disk capacity. I/O performance is best improved by eliminating disk I/Os by caching and prefetching. Caching in NVS at the DAC can be used to improve the efficiency of destaging. An onboard disk cache can also be used for prefetching.

Disk access time can be improved by providing multiple arms, as in the case of Seagate Mach-2.

<https://www.seagate.com/innovation/multi-actuator-hard-drives/>

<https://www.tomshardware.com/news/seagate-lists-dual-actuator-hdd-exos-2x14>

The utilization of DRAM and NAND flash memories for data storage is discussed in Chapter 8.

3.2 Internal organization of files in UNIX

UNIX files are described in Chapter 4 of “The Design of the UNIX Operating System” (Bach, 1986). An improved UNIX file system proposed at UCB in (McKusick et al., 1984) allows files of size 2^{32} B (byte) and minimum block size of 4 KB. All the blocks in the file are accessible with two levels of indirection. Certain files in the file systems are directories and contain pointers to files that may themselves be directories. Inodes are descriptors associated with files, which specify the ownership of the file and timestamps specify last modification. An inode may also contain references to indirect blocks containing further data block indices.

The disk is partitioned into areas called cylinder groups, which comprise several consecutive disk cylinders. Bit maps are used to specify available blocks in a cylinder group. A static number of inodes is allocated per cylinder group. By using 4 KB blocks the access efficiency is increased 4-fold with respect to 1 KB blocks, utilized initially.

The 4.4 BSD file system includes a new algorithm for allocating disk blocks to files by improving file clustering and reducing fragmentation, which improves the performance of sequential I/O.

A study of the allocation policy in the *Fast File System - FFS* is reported in (Smith and Seltzer, 1996). The file systems are artificially aged by replaying a workload similar to a real one. Experimental results show that read/write performance is improved by 32-20%.

I/O performance in UNIX file systems is improved via the *Co-locating Fast File System - C-FFS* described in (Ganger and Kaashoek, 1997). Introduced are two techniques: embedded inodes and explicit grouping. With embedded inodes most files are stored in the directory, removing indirection. With explicit grouping, the data blocks of multiple small files named

by a given directory are allocated adjacently and moved to and from the disk as a unit in most cases. Measurement results show a 50% improvement for reads and writes.

3.3 Review of disk arm scheduling

Since the introduction of HDDs numerous disk scheduling methods have been proposed. The chronological review provided in this section is mainly of historical interest, since flash systems are the future of high performance storage systems. We conclude with a discussion of disk scheduling in Linux.

FCFS disk scheduling

FCFS disk scheduling policy is the default policy and does well or lower disk utilizations, since short queue lengths do not provide opportunities for reordered processing of disk requests to attain higher processing efficiency. Evidence to the contrary is given in (Jacobson and Wilkes, 1991), where it is stated that even when disk utilizations are low queueing occurs due to bursty arrivals. When the processing of requests in a batch is completed before the next request arrives, then the mean waiting time for requests in a batch is one half the number of requests times mean disk access time for random accesses.

FCFS provides a rather poor performance in the case of accesses to small randomly placed data blocks, since the ratio of transfer time to service time is small due to positioning time. Reordering of the processing of requests can be used to reduce the mean processing time per request.

Based on data from an early timesharing system disk arms rarely move, more precisely only one third of accesses result in a seek (Lynch, 1972). This observation is true if data is accessed sequentially and there is a single active file per disk. Similar arguments are presented in (Smith, 1978a).

Seek distance and time can be reduced by placing frequently accesses data on contiguous disk cylinders. Middle disk cylinders have been considered for this purpose, since they can be accessed with reduced seek distance. In the case with ZBR placing files on outer disk cylinders is advisable, since they provide higher data transfer rates.

The high seek time in early disk drives served as the motivation for the *Shortest Seek Time First - SSTF* and the SCAN policies both proposed Peter Denning who did his PhD at MIT (Denning, 1967). denninginstitute.com

SSTF schedules enqueued requests according to seek distance from the cylinder for the current request. The seek time characteristic is an increasing function of seek distance. Denoting the seek distance by d curve-fitting has been used to obtain an expression for the seek time, e.g., $t_{seek}(d) = a + b\sqrt{d}$

Since SSTF is a greedy policy, there is possibility of starvation, i.e., some requests may encounter long delays and this may result in higher variability in disk response times. It is argued in (Wilhelm, 1976) that FCFS outperforms SSTF, but a rebuttal is presented in (Hofri, 1980), which provides an analysis of FCFS scheduling taking into account the correlation between successive requests using the *Matrix Geometric Method - MGM* by Marcel Neuts (Neuts, 1981). An M/G/1 queueing model was also used in (Wilhelm, 1976), but SSTF scheduling is evaluated using simulation in both studies.

SCAN or elevator scheduling moves the disk arm in alternate directions serving pending requests in that direction, so that unlike SSTF it is not prone to starvation. LOOK is more efficient than SCAN since it reverses the direction of a SCAN when there are no more requests in that direction (Merten, 1970). SCAN and LOOK visit the middle disk cylinders twice as often as innermost and outermost disk cylinders, so that the mean disk response time at these cylinders is lower than outer cylinders as noted in (Oney, 1975). Variations of the SCAN policy in addition to LOOK are as follows.

Cyclical SCAN - CSCAN returns the disk arm to one end after each disk scan, preferably outer disk cylinders in zoned disks, since they hold more data blocks than inner cylinders. This is to alleviate the bias of serving requests on middle disk cylinders twice, since otherwise they are scanned in both directions.

N-step SCAN is proposed in (Teorey and Pinkerton, 1972) to decrease the variance of individual waiting times while not significantly degrading throughput. Requests are grouped in sizes of N or less; that is, the minimum of N , and (b) the current queue length L whenever the previous group has been completely serviced. Lower variance results from the fact that once a group has been formed no reordering of requests is allowed.

FSCAN is a variation of SCAN with $N = 2$ queues (Coffman et al., 1972). Requests are served from one queue, while newly arrived requests are enqueued at the other queue. FSCAN along with N-Step-SCAN prevents “arm stickiness” unlike SSTF, SCAN, and C-SCAN. Arm stickiness in those other algorithms occurs when a stream of requests for the same track causes the disk arm to stop progressing at that track, preferring to satisfy the no-seek requests for the track it is on.

V(R) with $0 \leq R \leq 1$ ranges from $V(0) = \text{SSTF}$ to $V(1) = \text{SCAN}$, so that it provides a continuum of algorithms based on the value of R (Geist and Daniel, 1987). To bias the arm to continue its movement in the direction of the current scan, $d_{bias} = R \times C$, where C is the number of disk cylinders, is *subtracted* from the seek distance in that direction. The value of

R is varied in simulation studies to determine the value that minimizes the sum of the mean response time and a constant k times its standard deviation to approximate a percentile of response time. It is shown that for lower arrival rates SSTF is best, i.e., $R = 0$, while at higher arrival rates $R = 0.2$ provides the best performance.

Grouped Shortest Time First - GSTF is a combination of SCAN and *Shortest Seek First - SSF* policies (Seltzer et al., 1990). A disk is divided into groups of consecutive cylinders and the disk arm completes the processing of requests in the current group according to *Shortest Seek First - SSF*, before proceeding to the next group according to SCAN. When there is only one group we have SATF and with as many groups as cylinders we have SCAN.

Weighted Shortest Time First - WSTF multiplies positioning time by $F = \max(0, 1 - w/T_{max})$, where w is the waiting time of a request and T_{max} is the maximum waiting time (Seltzer et al., 1990). WSTF attempts to prioritize the processing of delayed requests to minimize the standard deviation of waiting time. For $T_{max} \rightarrow \infty$ WSTF becomes the same as SATF. WSTF was compared with other methods via simulation.

There is a similarity to time-dependent priority scheduling discussed in Section 3.7 in (Kleinrock, 1976).

https://www.lk.cs.ucla.edu/data/files/Kleinrock/a_delay_dependant_queue_discipline.pdf

According to this policy the priority of a request increases with its waiting time. So that the priority of the i^{th} job arriving at t_i is $p_i(t) = a_i(t - t_i)$, where t is the current time and a_i is determined according to the external priority of the i^{th} job. Job i will have the higher priority than job j at time t if $p_i(t) > p_j(t)$.

Shortest Access/Processing Time First - SATF/SPTF was proposed in (Jacobson and Wilkes, 1991) at HP Labs. It minimizes the access or positioning time, which is the sum of seek time and rotational latency (Jacobson and Wilkes, 1991). SATF is a nonpreemptive policy which has the desirable properties of *Shortest Job First - SJF* or *Shortest Processing Time - SPT* (Kleinrock, 1976).

The difficulty with implementing this approach is estimating positioning time, which is best done by the disk controller itself. Firmware has been used in (Lumb et al., 2000a) for freeblock scheduling, which processes low priority requests opportunistically incurring no or minimal disk access time.

3.3.1 Implementation of SATF

There are n pending requests at cylinders C_2^i , $1 \leq i \leq n$ after the R/W head completed processing a request on cylinder C_1 . Given the seek time characteristic: $t_{seek}(d)$ then the time to move the arm from the initial cylinder to the destination cylinder is:

$$t_s^i = t_{seek}(|C_1 - C_2^i|), 1 \leq i \leq n.$$

Given that the R/W head was initially at sector s_1 , after t_s^i ms the R/W head will land at

$$s_2^i = (s_1 + (T_r/t_s^i)S_i) \bmod S_i,$$

where S_i is the number of sectors at the i^{th} cylinder.

Given the angular position of the first sector encountered on the target track is s_2^i then its angular position is $\alpha_2^i = s_2^i/S_i$, $1 \leq i \leq n$.

If $\alpha_2^i > \alpha_1^i$ then the rotational latency is $t_\ell = (\alpha_2 - \alpha_1)T_r/(2\pi)$, else set $\alpha_2^i = \alpha_2^i + 2\pi$.

The i^{th} request with the smallest $t^i = t_s^i + t_\ell^i$ will be selected according to the SATF policy.

Sliding Window SPTF limits the number of requests considered for SPTF scheduling. Four scheduling algorithms are proposed in (Jacobson and Wilkes, 1991):

(i) *Short-Sighted Optimal Access Time - SS-OAT* algorithm, which finds the optimal schedule for a fixed number of requests.

(ii) *Sliding Window OAT - SW-OAT*, which considers all possible permutations to process requests within a fixed window to determine the shortest schedule. The scheduling step is repeated when the completed request is replaced by a new request. SPTF applied over the requests within the window outperforms it in (Riska et al., 2004).

(iii) Short-sighted greedy that considers the best among n requests.

(iv) Fully greedy which considers all of the requests in the queue.

The relative performance of the four methods are 1.84, 1.75, 1.73, and 1.00, which shows the benefit of considering all of the requests in the queue.

Batched Shortest Time First - BSATF has two queues. Requests are served from one queue, while new arrivals are enqueued in the other queue. This policy exhibits a poorer performance than SATF, since only one half of all requests are available for SATF scheduling.

Aged Shortest Access Time First Policy - ASATF, which is a variation to WSTF where $F = 1 - \mathcal{W}w$ was proposed in (Jacobson and Wilkes, 1991) and simulated in (Worthington et al., 1994). For large $\mathcal{W}$ ASATF is tantamount to FCFS scheduling and for $\mathcal{W} = 1$ it is the same as SATF. A detailed simulation study of this policy appears in (Hsu and Smith, 2004), which obtains the optimum value for $\mathcal{W}$.

Group Sweeping Scheduling - GSS was proposed by researchers at IBM TJWRC in (Chen et al., 1993) and (Yu et al., 1993). GSS divides the set of n streams into g groups, such that the streams in one group have similar deadlines. Individual streams within a group are served according to SCAN.

Yet-another Fair Queueing - YFQ is a disk scheduling algorithm proposed at Bell Labs (Bruno et al., 1999). The idea is to set aside portions of the disk bandwidth to guarantee file accesses with high throughput, low delay, and good fairness. YFQ was implemented as part of the Eclipse/Berkeley Software Development - BSD OS.

<https://en.wikipedia.org/wiki/BSD/OS>

Since such QoS guarantees hinder global disk scheduling optimizations the authors propose and evaluate several disk scheduling enhancements to promote global optimization.

Conditional Priority CP-SATF is an extension of SATF to allow one class of requests, e.g., reads, to be processed at a higher priority with respect to another class, e.g., writes (Thomasian and Liu, 2002a), (Thomasian and Liu, 2002b). SATF winner read requests are processed unconditionally, while an SATF winner write request is processed provided:

$$x_{write}^{best}/x_{read}^{best} < t, 0 \leq t \leq 1.$$

$t = 1$ corresponds to SATF, while for $t = 0$ write (low priority requests) are processed only when the queue for read (high priority read requests) is empty. The latter case is equivalent to the *Head-of-the-Line - HOL* nonpreemptive priority queueing discipline, which serves requests in FCFS order from each queue (Kleinrock, 1976). There is a degradation in performance, since the SATF scheduling policy is not being applied to all requests. Simulation results show that $t \approx 0.625$ improves the response time of high priority requests, at the cost of a small reduction in overall throughput.

SPTF with i Levels of lookahead-LA(i) was proposed in (Thomasian and Liu, 2002a,b). Consider the scheduling of the next request after a request is completed. Requested blocks are small and have short transfer times, so that only the positioning time matters.

SATF winner **A** request, whose processing will be followed by request **X**. **B** is another pending request with $T_B > T_A$, but which can be followed by requests Y such that $T_B + \alpha T_Y < T_A + \alpha T_X$. Since **X** or **Y** may not be processed at all, their service time is discounted with a factor $0 \leq \alpha \leq 1$. An SATF policy looking ahead two requests is denoted SATF-LA(2) and its cost is $O(n^2)$ versus $O(n)$ for SATF.

Shortest Adjusted Processing Time First (SAPTF) is a variation of SPTF to reduce the variance of waiting time was proposed in (Thomasian and Liu, 2004). There are multiple priority levels based on the waiting time of a given request. This is achieved by replacing $0 \leq t \leq 1$ in SPTF-CP(t) with t^i , where i is the priority level according to Table 3.1.

TABLE 3.1 Levels for waiting time and their thresholds t_i .

Waiting time range	Threshold
$0 < w \leq W$	$f_0 = 1$
$W < w \leq W + \sigma$	$f_1 = t$
$W + \sigma < w \leq W + 2\sigma$	$f_2 = t^2$
$W + 2\sigma < w \leq W + 3\sigma$	$f_3 = t^3$
$w > W + 3\sigma$	$f_4 = t^4$

Window-based SPTF so that only requests within a time window $(0, T)$ of the arrival time of the oldest request are considered for scheduling (Riska et al., 2004). T is adjusted according to the load with the goal of improving the standard deviation of response time, at the cost of a small sacrifice in its mean value. Trace-driven simulations show that WB-SPTF outperforms SPTF in overload mode. The effect of the number of requests considered for scheduling on performance is reported in (Riska and Riedel, 2003). The dynamic WB-SPTF algorithm handles overload situations, by adjusting the window size according to the load conditions of the system.

Window-Based SPTF - WB-SPTF similarly to SW-SPTF applies SPTF only over a subset of disk requests, i.e., a window scaled by time instead of the number of requests.

3.3.2 Disk performance studies by Windsor Hsu and Alan Jay Smith at IBM ARC

The performance impact of I/O optimizations and disk improvements on performance is presented in (Hsu and Smith, 2004). A summary of the paper follows.

Server and PC workloads to analyze the true performance impact of various I/O optimization techniques including read caching, sequential and opportunistic prefetching, write buffering, request scheduling, striping, and short-stroking, which writes data on outer higher capacity disk cylinders thus reducing both rotational latency and seek time.

Four basic disk technology improvement effects are: (1) faster seeks, (2) higher RPMs, (3/4) improved linear/radial (track) densities. Each effect is analyzed separately to determine the actual benefit. Historical rates of improvement and trend are used to determine the effect of disk technology scaling.

A methodology for replaying real workloads is developed to allow realistic I/O rate scaling.

Reducing the number of physical I/Os (via caching and prefetching) is more effective than efficiently performing I/Os. Sequential prefetching and write buffering reduce the average read and write response time by about 50% and 90%, respectively.

A robust method for improving performance is to use larger caches $\geq 1\%$ of used storage.

Disk technology improvement at the historical rate increases performance by about 8% per year if the disk occupancy rate is kept constant and by about 15% per year if the same number of disks are used.

The actual average seek time and rotational latency are, respectively, only about 35% and 60% of the specified values. Head positioning time dominates data transfer time, suggesting that to effectively utilize disk bandwidth data should be reorganized to allow more sequential accesses, as in the case of proximal I/O discussed below.

Diskseen is an improvement over current disk prefetch policies, which track access patterns at the level of files (Ding et al., 2007). It is pointed out that full performance improvements are achievable by prefetching for the following two reasons.

- (1) prefetch opportunities can only be detected by knowing the disk data layout,
- (2) non-sequential access of disk data is much slower than sequential access.

Diskseen tracks the locations and access times of disk blocks and based on analysis of their temporal and spatial relationships improves the sequentiality of disk accesses and overall prefetching performance. Implementation of Diskseen in the Linux 2.6 kernel shows that it can significantly improve the effectiveness of prefetching, reducing execution times by 20%-53% for micro-benchmarks and real applications such as *global regular expression print - grep*, *Concurrent Version System - CVS*, and *TPC-H benchmark*

<https://www.tpc.org>.

Diskseen with emphasis on prefetching is reported in (Jiang et al., 2013). A prefetcher seeks to accurately predict which data will be requested and load it ahead of the arrival of the corresponding requests. Current disk prefetch policies in major operating systems track access patterns at the level of file abstraction. While this is useful for exploiting application-level access patterns, for two reasons file-level prefetching cannot realize the full performance improvements achievable by prefetching.

- (1) certain prefetch opportunities can only be detected by knowing the data layout on disk, such as the contiguous layout of file metadata or data from multiple files.
- (2) nonsequential access of disk data (requiring disk head movement) is much slower than sequential access, and the performance penalty for mis-prefetching a randomly located block, relative to that of a sequential block, is correspondingly greater.

Native Command Queuing - NCQ is an implementation of SATF, which is included in the SATA 2 protocol specification to maximize disk throughput. With the advent of the SCSI protocol disk requests are enqueued at the disk controller, which carries out request scheduling instead of the OS level. The fact that the disk drive is not aware of the OS intent in scheduling disk requests is referred to as *expectation discord* in (Yu et al., 2010). A software mechanism to resolve this problem is proposed and evaluated in this work.

Most SCSI disk implemented *Tag Command Queueing - TCQ* and a disk may accept sixteen numbered requests and process them in any order to minimize access time. *Serial ATA - SATA* have NCQ that has a bit specifying whether it should report completion when media has been written (Yu et al., 2010). The tag's of completed requests are then returned (McKusick, 2012).

Proximal I/O described by NetApp researchers in (Schindler et al., 2011) allows disk drives to retire multiple I/Os in a single revolution and achieves seven-fold improvement in random I/O performance. Proximal I/O uses a flash memory-based write cache, which is 1% of disk capacity to aggregate random updates, until they have sufficient density to exploit proximal I/O.

D-SPTF - Distributed Shortest Positioning Time First investigated at CMU/PDL (Lumb and Golding, 2004) is a request distribution protocol for decentralized systems of storage servers, which exploits high-speed interconnects to dynamically select the server among those with a replica to service read requests. In doing so it simultaneously balances load, exploits the aggregate cache capacity, and reduces positioning times for cache misses. For network latencies up to 200 microseconds expected in storage clusters D-SPTF performs as well as would a hypothetical centralized system with the same collection of CPU, cache, and disk resources. D-SPTF achieves up to 65% higher throughput than popular decentralized approaches.

3.3.3 Linux support for disk scheduling

The discussion in this section is based on <https://www.vertica.com/docs/9.2.x/HTML/Content/Authoring/InstallationGuide/BeforeYouInstall/IOScheduling.htm?Highlight=scheduling>

The Linux kernel can use I/O schedulers to prioritize disk I/O. Most Linux distributions use the *Completely Fair Queuing - CFQ* by default, which gives input and output requests equal priority.

https://en.wikipedia.org/wiki/Completely_Fair_Scheduler

This scheduler is efficient in systems running multiple tasks that need equal access to I/O resources, but it can create a bottleneck when used on Vertica drives containing the catalog and data directories. It gives write requests equal priority to read requests, and its per-process I/O queues can penalize processes making more requests than other processes.

Instead of the CFQ scheduler, hosts can be configured to use either the Deadline or NOOP I/O schedulers for the drives containing the catalog and data directories.

The Deadline scheduler gives priority to read requests over write requests. It also imposes a deadline on all requests. After reaching the deadline, such requests gain priority over all other requests. This scheduling method helps prevent processes from becoming starved for I/O access. The Deadline scheduler is best used with HDDs and attempts to group requests for adjacent sectors on a disk, lowering the time the drive spends seeking.

https://en.wikipedia.org/wiki/Deadline_scheduler

The NOOP scheduler is simply a FIFO scheduler placing all input and output requests into a single queue. This scheduler is best used on *Solid State Drives - SSDs*. Because SSDs do not have a physical read head, no performance penalty exists when accessing non-adjacent sectors.

https://en.wikipedia.org/wiki/Noop_scheduler

Failure to use one of these schedulers for the Vertica drives containing the catalog and data directories can result in slower database performance. Other drives on the system (such as the drive containing swap space, log files, or the Linux system files) can still use the default CFQ scheduler.

<https://book4you.com/book/1254679/a9f0ca>

Heuristics for disk drive positioning in 4.3 BSD UNIX on disk drives are described in (Stevens, 1989).

3.4 Disk scheduling for mixed workloads

A mixed workload consists of requests to continuous and discrete objects. Continuous data requests have an implicit deadline associated with the delivery of the next data block, e.g., to ensure glitch-free viewing of streaming, as opposed to downloading. The *Earliest Deadline First - EDF* scheduling policy described in (Liu and Layland, 1973) is a natural choice, but results in a higher disk positioning time overhead and reduced throughput. Liu is the author of a book on Real Time Scheduling, 2000.

SCAN-EDF improves performance by using SCAN, while servicing requests with the same deadline was proposed by Narasimha Reddy (Texas A&M) and James Wyllie at IBM ARC (Reddy and Wyllie, 1993). A retrospective of the 1993 article was presented in (Reddy et al., 2005). A comparison of SCAN-EDF with other scheduling policies shows that SCAN-EDF combines the best features of both SCAN and EDF. Also studied is the impact of buffer size on the maximum number of supported video streams.

Elias Balafoutis et al. in (Balafoutis et al., 2003) considered a media server, which in addition to accesses to Continuous objects or (C-requests), provide access to Discrete objects or (D-requests). C-requests have to meet performance requirements, while discrete requests should be served with as small a mean response time as possible. One scheduling method serves C-requests according to SCAN and intervening D-requests according to either SPTF or OPT(N), where the latter determines the schedule which yields the lowest time after enumerating all $N!$ (N-factorial) schedules. **FAir Mixed-scan ScHeduling - FAMISH** ensures that all C-requests are served in the current round and that D-requests are served in FCFS order.

Advanced multimedia applications for digital libraries with mixed workloads with accesses to Continuous (C) and Discrete (D) data is investigated in Nerjes et al. (1997). A stochastic performance model for stochastic service guarantees using a combination of analysis and simulation is developed. A round based scheduling scheme with stochastic performance guarantees for continuous data requests bounds the probability that “glitches” occur and the probability that the response time exceeds a certain tolerance threshold.

Two categories of timing guarantees or *service levels* for I/O requests are deterministic and statistical. The latter category is meaningful for *Variable-Bit-Rate* - VBR rather than *Constant-Bit-Rate* - CBR streams, since the required I/O bandwidth for VBR varies and provisioning for worst case behavior will degrade performance inordinately.

https://en.wikipedia.org/wiki/Variable_bitrate

https://en.wikipedia.org/wiki/Constant_bitrate

A two level queueing model for disk scheduling was proposed in (Shenoy and Vin, 1998). This approach was adopted by Ravi Wijayarathne and Narasimha Reddy at Texas A&M in (Wijayarathne and Reddy, 2000) for supporting multiple QoS levels, where the first level queue acts as a scheduler, as well as an admission controller to a pool of requests from which requests are scheduled on disks.

Three request categories are considered in this study:

Periodic requests are CBR or VBR and require service at regular intervals of time (deterministic or statistical guarantee).

Interactive requests require quick service as for playing a video-game (best effort mean response time).

Aperiodic requests for regular file transfers. Guarantee of a minimum level of service or bandwidth, is only possible if the disks are not 100% utilized by other requests.

Leaky bucket control deals with bursts of interactive requests,

https://en.wikipedia.org/wiki/Leaky_bucket

e.g. a limited number of such requests are admitted per second into a pool from which requests are scheduled on disk (McDysan and Spohn, 1995). While the proposed techniques improve throughput significantly over other methods, further improvements seem possible by dynamic allocation and adaptive performance guarantees.

Scheduling with *Quality-of-Service* - QoS guarantees according to (Wijayarathne and Reddy, 2000) requires:

- (1) controlling and limiting the allocation of resources for different levels;
- (2) scheduling of resources to meet performance goals;
- (3) determining system design and its parameters;
- (4) investigating the tradeoffs between statistical and deterministic guarantees on throughput.

In spite of the randomness of disk service times, the scheduling problem in the deterministic case can be approached as follows: requests are grouped into rounds and served according to a seek optimizing policy, so that all requests will meet their deadline (with a worst case assumption for latency, full rotation).

3.5 Real time disk scheduling for multimedia

A cost-driven disk scheduling algorithm for multi-media object retrieval is discussed in (Ghandeharizadeh et al., 2003). Deadline is the absolute time by which a request must be served. The scheduler might lose a certain number of requests. To minimize total cost the scheduler might sacrifice a high priority request in favor of servicing several low priority requests. A higher priority is assigned to high quality streams with respect to low-priority streams and high priority requests have a higher cost in minimizing deadlines than low priority ones.

The algorithm utilizes a single queue for all requests and maximizes the number of high priority requests served while trying to accommodate as many low priority requests as possible.

Aperiodic and periodic requests are considered in this study. Periodic requests are denoted by two parameters (c , p), where p is the period and $c < p$ is the service time required in that period. Aperiodic requests arrive according to a Poisson process. By sizing the system with respect to peak system load the loss rate is maintained at a very low rate (one in a million). Admission control is used when the peak expected load is exceeded.

TABLE 3.2 Categorization of work on (disk) scheduling.

Related work	Priority	Scheduling	Deadline	Cost
(Andrews et al., 1996)		X		
(Abbott and Garcia-Molina, 1992)	X			
(Cheng and Gillies, 1996)		X		
(Chang and Garcia-Molina, 1997)		X		
(Carey et al., 1989)	X			
(Chen et al., 1993)		X		
(Dayal et al., 1988)	X		X	
(Chen et al., 1991)		X	X	
(Haritsa et al., 1993)	X		X	X
(Kamel et al., 2000)	X	X	X	
(Lin and Tarn, 1991)	X		X	
(Ramamritham and Stankovic, 1994)	X		X	
(Reddy and Wyllie, 1994)			X	X
(To and Hamidzadeh, 1999)				X
(Wijayarathne and Reddy, 1999)	X	X	X	

The algorithm provides a solution to an optimization problem with the following dimensions:

(1) priority, (2) seek time, (3) deadlines, (4) cost.

With R_0 at the head of the queues we have the following queue, which constitute several SCAN cycles: $R_0, R_1, \dots, R_n$. Requests are sorted according to the SCAN. When a new request R_{new} which fall between R_i and R_{i+1} arrives it is inserted between them. If the insertion results in an invalid state the algorithm performs the following steps to restore validity with the objective to satisfy as many conditions as possible.

- (1) R_{new} should not violate the deadline of a high priority request.
- (2) the total cost incurred as a result of dropped requests should be minimized.
- (3) Restoration of validity should not sacrifice bandwidth efficiency.

The algorithm is compared with a multipriority queue which serves high priority requests first. The queues have the following policies: (1) FIFO, (2) EDF, (3) CSCAN, (4) SCAN-EDF, which is utilized in this study because it was shown to be superior to CSCAN and EDF in (Reddy and Wyllie, 1993).

The total cost for $n = 2$ priority levels is given as $\sum_{i=1}^n C_i \times \text{loss}_i$, where loss_i is the number of priority requests which missed the deadline. The reader is referred to (Ghandeharizadeh et al., 2003) for experimental results.

Previous work in this area is summarized in Table 3.2.

3.6 Storage virtualization

Multi-Dimensional Storage Virtualization - MDSV allows multiple virtual disks, each with a distinct combination of capacity, latency and bandwidth requirements, to be multiplexed on a physical disk storage system with performance isolation as proposed in (Peng and Chiueh, 2008). Novel design and implementation techniques to solve the availability and fairness problems in MDSV are proposed. This discussion is applicable to *Heterogeneous Disk Arrays - HDAs* in (Thomasian and Xu, 2011c, 2016) in Chapter 10.

Storage Service Providers - SSPs use SAN/NAS technology to provide virtual disks with QoS guarantees. According to Tzi-cker Chiueh et al. the key challenge is to maximize the number of virtual resource units supported by exploiting the *Statistical Multiplexing - SM* nature of the request load (Gopalan et al., 2006). While a number of measurement-based admission control algorithms utilize SM along the bandwidth dimension, they do not satisfactorily exploit SM along the delay dimension to guarantee distinct per-virtual-unit delay bounds.

Delay Distribution Measurement - DDM admission control algorithm is based on measurement and supports virtual units with distinct probabilistic delay guarantees. When compared to deterministic admission control, DDM can potentially increase the number of admitted virtual units (and resource utilization) by up to a factor of three.

In 2009 Prof. Chiueh, who was a student of Randy Katz at UCB, moved from SUNY to *Industrial Technology Research Institute - ITRI* in Taiwan.

3.7 File placement on disk

File placement is complementary to disk arm scheduling. It can be based on reducing latency, seek time, and positioning time, which is a combination of the two.

Techniques to minimize rotational latency are proposed in (Ng, 1991). One solution is to store data blocks 180 degrees apart on tracks. A second solution is providing two sets of read-write heads opposite each other. The same effect can be attained by storing data 180 degrees apart on synchronized mirrored disks. This technique combined with striping is proposed by researchers at Princeton U. in (Yu et al., 2000).

The organ pipe organization minimizes the seek distances by placing files at a distance related to their access frequency from the middle disk cylinder (Wong, 1980, 1983).

Disk shuffling moves data around to achieve improved performance. Disk shuffling was first considered at HP Labs and is reported in (Ruemmler and Wilkes, 1991). This can be done by the disk driver as proposed in (Carson and Reynolds, 1989), (Vongsathorn and Carson, 1990). A count is kept of the number of requests directed to each shuffling quantum, which may be a cylinder, over a period of time, and these counts are used to drive the rearrangement. A map containing the current location of logical cylinders is stored in a known fixed place on disk, e.g. in the first physical cylinder. This map is updated every time the disk is shuffled, and read into main memory on power-up to alleviate disk accesses.

The rearrangement can be done during idle periods, to minimize the impact on normal operation. The straightforward organ pipe arrangement is not always optimal, e.g., a large sequentially-accessed file that used to be contiguous could end up split, thus increasing the average seek distance. By looking for such dependency patterns in accesses, the organ pipe arrangement can be adjusted to keep sequentially-accessed data close together. Using such techniques can result in a reduction of average seek distances by as much as 30% compared with straightforward organ-pipe shuffling (Carson and Vongsathorn, 1989).

An adaptive technique for reducing disk seek times is described in (Akyürek and Salem, 1995). The technique copies frequently referenced blocks from their original locations to reserved space near the middle of the disk. Reference frequencies are estimated by monitoring the stream of arriving requests. Trace-driven simulations show that seek times can be cut substantially by copying only a small number of blocks using this technique. The technique was implemented by modifying a UNIX device driver.

The 4.4BSD file system includes a new algorithm for allocating disk blocks to files, whose goal is to improve file clustering to increase the amount of sequential I/O for reading or writing files. Keith Smith and Margo Seltzer studied the effectiveness of this algorithm at reducing file system fragmentation by creating a program that artificially ages a file system by replaying a workload similar to that experienced by a real file system (Smith and Seltzer, 1996). The program was used to replay ten months of activity on two file systems that differed only in the disk allocation algorithms that they used. It was observed that the new allocation algorithm had approximately half the fragmentation of a similarly aged file system that used the traditional disk allocation algorithm. Measuring the performance difference between the two file systems by reading and writing the same set of files on the two systems showed that this decrease in fragmentation improved file write throughput by 20% and read throughput by 32%.

A good review of file placement is presented by Kang Shin et al. in (Huang et al., 2005). The authors propose data replication in free disk space to improve performance and reduce energy consumption based on shorter positioning times, but if updates are frequent there is the additional disadvantage that all replicas need to be updated. An anticipatory I/O scheduler is also considered in this study, while disk arm scheduling is considered to be orthogonal to this work.

There has been a proposal for dynamic reorganization of disk data called *Automatic Locality Improving Storage - ALIS* by Windsor Hsu a PhD student of Alan Jay Smith at UCB doing his research at IBM/ARC (Hsu et al., 2005).

Block ReORGanization - BORG is a self-optimizing storage system that performs automatic block reorganization based on the observed I/O workload as proposed by Raju Rangaswami et al. in (Bhadkamkar et al., 2009). BORG is motivated by three characteristics of I/O workloads:

1. non-uniform access frequency distribution,
2. temporal locality,
3. partial determinism in non-sequential accesses.

BORG manages a small, dedicated partition on the disk drive, with the goal of servicing a majority of the I/O requests from within this partition with significantly reduced seek and rotational delays. BORG is transparent to the rest of the storage stack, including applications, filesystem(s), and I/O schedulers, thereby requiring no or minimal modification to storage stack implementations. A Linux implementation of BORG was investigated using several real-world workloads, including individual user desktop environments, a web-server, a virtual machine monitor, and a version control SVN server. SVN is

a centralized version control system, which means that the version history is stored in a central server. These experiments comprehensively demonstrate BORG's effectiveness in improving I/O performance and its incurred resource overhead.

The block layer interface is considered to be ideal for block level reorganization for the following reasons.

1. Key temporal, block- and process-level attributes about disk accesses are available.
2. Operating at the block layer makes the solution independent of the file system layer above, allowing it the flexibility to support multiple heterogeneous file systems simultaneously.
3. New abstractions due to virtualization trends as well as *network-attached storage* - NAS and *Storage Area Network* - SAN environments can be supported.

BORG uses a disk-based cache as an extension of the main memory cache and this offers three additional advantages to main memory caching alone, prefetching, and I/O scheduling.

1. It is more effective as a cache (than main memory) because it offers a less expensive (and thus larger) as well as reliable caching solution, thus allowing data to be cache-resident for long periods of time.
2. The size of the disk-based cache can easily be configured.
3. Dynamically optimizing data layout based on access patterns within a disk-based cache provides the unique ability to make initially non-sequential data accesses more sequential.

Certain workloads, including application start-up, content indexing and web-page requests, exhibit a more non-sequential, but deterministic, access behavior. It is thus possible that the same set of data can be accessed sequentially by some applications and non-sequentially by others. Based on the observation that this may be a temporary phenomenon has been argued in (Akyürek and Salem, 1995) in favor of copying rather than shuffling of data. Copying retains original sequential layouts so a choice of location based on the observed access pattern may be possible. Reverting back to the original layout is straightforward. Similarly, rather than permanently disturbing the sequential layout of files, BORG operates on copies of blocks placed temporarily in an independent partition, optimizing for the current common case of access for each data block.

The analyzer summarizes the disk I/O workload. It splits the I/O trace obtained by the profiler into multiple I/O traces, one per process. Each process trace is used to build a directed process access graph $G_i(V_i, E_i)$, where vertices represent LBA ranges and edges a temporal dependency between two LBA ranges. The weight on an edge between vertices (u, v) represents the frequency of accesses from u to v . The directed and weighted graph represents repeated sequences of multiple non-sequential requests. Since multiple processes may access the same LBA, a single master access graph $G(V, E)$, that captures all available correlations into a single input for the reconfiguration planner is created in Table 3.3.

Application startup time improvement is given in Table 3.4.

3.7.1 Anticipatory disk arm placement

Anticipatory Arm Placement - AAP proposed in (King, 1990) with the goal to minimize the disk seek distance. Positioning the disk arm at the middle disk cylinder will reduce the seek distance from $C/3$ on the average to $C/4$ for requests uniformly distributed over all cylinders (King, 1990). In the case of mirrored disks the arms should be placed at cylinders $C/4$ and $3C/4$, which yields a mean seek distance of $C/8$. The seek distance for mirrored disks is discussed in (Bitton and Gray, 1988) in Chapter 4.

The analysis is extended to disks with *Zoned Bit Recording* - ZBR, mirrored disks, with uniform disk and hot spot accesses in (Thomasian and Fu, 2006). Rather than dealing with discrete values the capacity of disk cylinders is set to be proportional to the radius of its tracks. Cylinder capacities are obtained by a linear interpolation between the innermost and outermost cylinders. Denoting the radius of the innermost and outermost cylinder with R_i and R_o , then $R_i + \sqrt{(R_o^2 + R_i^2)/2}$ divides the disk into two parts with equal capacities. In mirrored disks the two arms should be placed at $R_i + \sqrt{(4R_i^2 + R_o^2)/3}$ and $R_i + \sqrt{(2(R_i^2 + 2R_o^2))/3}$.

AAP is effective for lower disk utilization, since positioning of the disk arm will result in an increased response time to external disk requests. Simulation results show that there is a crossover point, so that beyond a certain arrival rate AAP results in degraded performance, because the processing of external requests is delayed by the disk arm being repositioned.

The anticipatory disk scheduling method proposed in (Iyer and Druschel, 2001) defers the processing of a new request, until it is ascertained that there are no further pending requests in the area of disk at which the R/W head resides. The improvement in performance due to this method is explored in the context of Apache file servers. This demonstrates that a non-work-conserving scheduling due to forced idleness (Kleinrock, 1976) can improve performance in some cases.

TABLE 3.3 C is the most connected vertex and is chosen for placement first. Next, vertex B is placed after vertex C since it is connected by an outgoing edge and has the highest weight of all the edges connected to C. Next, vertex G is placed before vertex group $C \cup B$. The final sequence of vertices from the lowest LBA to the highest is: $L = [F, H, J, A, G, C, B, E, D]$.

Edge 1	Edge 2	Edge 3
$A \xrightarrow{8} B,$	$A \xrightarrow{5} D,$	$A \xrightarrow{7} E$
$B \xrightarrow{7} C,$	$B \xrightarrow{1} E,$	-
$C \xrightarrow{10} B,$	$C \xrightarrow{7} F,$	$C \xrightarrow{2} J$
$D \xrightarrow{2} A,$	$D \xrightarrow{6} H,$	-
$E \xrightarrow{6} A,$	$E \xrightarrow{9} D,$	-
$F \xrightarrow{6} I$	-	-
$G \xrightarrow{9} C$	-	-
$H \xrightarrow{3} E,$	$H \xrightarrow{8} I,$	-
$I \xrightarrow{9} E$	-	-
$J \xrightarrow{4} C,$	$J \xrightarrow{3} G,$	$J \xrightarrow{8} I$

TABLE 3.4 Application start-up time improvement, v = vanilla, B = BORG.

App	Start-up time		Rand. I/O%		Avg seek (#cyl)	
	V	B	V	B	V	B
firefox	3.71	2.32	2.7	1.2	132	37
oowriter	5.30	2.74	3.8	0.2	193	20
xemacs	7.26	2.72	2.1	0.3	87	9
acoread	6.20	2.65	4.6	0.1	39	9
eclipse	4.12w	1.52	2.5	0.3	198	29
gimp	3.62	3.66	2.5	2.1	102	63
ooimpress	5.18	1.97	2.7	0.3	61	39

3.8 Disks with Shingled Magnetic Recording - SMR

The capacity growth of magnetic disks slowed down at the turn of the century (Gray and Shenoy, 2000). New recording techniques are required if the data density of magnetic disks is to continue at its current 30-50% annual growth (Amer et al., 2010).

HDDs in 2011 could store 400 GB/in², while the limit was estimated to be about 1 Tb/in². *Shingled Magnetic Recording - SMR* is a solution to exceed this limit with the *least* intrusive departure from prevailing technology as noted above. SMR uses a write head with a stronger, but asymmetric magnetic field, which is made possible by the fact that writes require a much stronger magnetic field than reads.

<https://www.seagate.com/tech-insights/breaking-areal-density-barriers-with-seagate-smr-master-ti/>

SMR introduces challenges in determining and implementing the more effective data layouts. It imposes new limitations on the ability to perform random data updates. SMR writing takes advantage of the property that by overlapping tracks, but leaving a relatively small strip of a previously written track untouched, is still sufficient for reading the track with current read heads. A 2.3-fold density increase in capacity is possible. The overlapping-tracks architecture slows down the writing process, since writing to one track overwrites adjacent tracks, and requires them to be rewritten as well.

Shingled writing is currently the most attractive among considered options, because it is the easiest to implement at the device level. SMR trades the inconvenience of the inability to update in-place for a much higher data density by using a

different write technique that overlaps the currently written track with the previous track. Random reads are still possible on such devices, but writes must be done largely sequentially. The authors discuss possible changes to disk-based data structures that the adoption of SMR requires. The usefulness of NVRAM in the context of shingled disks is explored. I/O traces are used to demonstrate the surprising stability of written device blocks, e.g., that more than 93% of device blocks remain unchanged over a day, and that for more specialized workloads less than 0.5% of a shingled-write disk's capacity would be needed to hold randomly updated block.

Seagate started shipping device-managed SMR HDDs in September 2013, which hide SMR complexity by managing it in the firmware, presenting an interface like any other HDD, while other SMR devices are host-managed. There are three different ways that data can be managed on an SMR drive.

https://en.wikipedia.org/wiki/Shingled_magnetic_recording

Drive managed: The fact that the HDD uses SMR need not be known by the host.

Host managed: Such a device has to be written sequentially not to destroy existing data. The drive will not execute commands violating this protocol.

Host aware: This is a combination of Drive and Host Managed options according to *Storage Networking Industry Association - SNIA* experts (Dunn and Feldman, 2014).

Peter Desnoyers and his students at Northeastern U. have developed a simulator, which predicts the latency for SMR disk within a few percent and a very small *Root Mean Square - RMS* error (Shafaei et al., 2017).

SMR disks offer a plug-compatible higher-capacity replacement for conventional disks. For non-sequential workloads, these disks show bimodal behavior (Aghayev et al., 2017): a short period of high throughput followed by period of low throughput. The Linux ext4 file system modified significantly improves throughput in both modes. Benchmarking of four different drive-managed SMR disks from two vendors, showed that ext4-lazy achieves 1.7-5.4x improvement over ext4 on a metadata-light file server benchmark.

<https://en.wikipedia.org/wiki/Ext4>

On metadata-heavy benchmarks it achieves 2-13-fold improvement over ext4 on drive-managed SMR disks and conventional disks.

Storage vendors, including but reportedly not limited to Western Digital, have quietly begun shipping SMR in place of *Conventional Magnetic Recording - CMR* disks.

<https://arstechnica.com/gadgets/2020/04/caveat-emptor-smr-disks-are-being-submarined-into-unexpected-channels/>

SMR disks have multiple levels of caching – DRAM, then some CMR zones and finally shingled zones. Writes are to the CMR space and when the disk is idle the drive will rearrange itself in the background writing the CMR data into one of 10-200 shingled “zones”. They are all appendable, but if a sector within a zone needs changing, the entire zone must be rewritten and zones can be marked discarded.

When the CMR zone fills the drive may start appending to a SMR zone – and in doing so it slows down dramatically. If the drive stops to flush out the CMR zone, then the OS is going to see a pause, which might make the RAID5 controller have a timeout and assume that the disk has failed and initiate a rebuild process.

3.9 Review of analyses of disk scheduling methods

Disk performance evaluation can be carried out via measurement, simulation, or analysis. Power consumption in addition to performance is a consideration.

It is shown in (Ruemmler and Wilkes, 1994) that rather detailed specification of disks are required to accurately predict disk performance via trace-driven simulation. Disk I/O traces can also be used for disk workload characterization as in the case of the OLTP workload in (Ramakrishnan et al., 1992). Of interest is disk request size distribution, locality, arrival process. The characterized workload can be used in random number driven simulations to predict performance directly or to verify the accuracy of approximate queueing analyses based on simplifying assumptions, such as Poisson arrivals. The arrival rate of requests can be easily modified in random number driven simulations by reducing interarrival times in a trace-driven situation, but this may result in a contradiction where a disk request is issued, before a preceding request which determines its address is completed.

Table 3.5 is based on a survey of several workload characterization studies (Ganger, 1995).

Two of the traces come from HP systems running HP-UX™, a version of the UNIX™ OS (Ruemmler and Wilkes, 1993). Cello comes from a server at HP Labs used for program development, simulation, mail, and news. Snake is from a file server at UCB used primarily for compilation and editing. While these traces are actually two months long, a single week-long snapshot is considered. The other traces are from commercial VAX™ systems running the VMS OS (Ramakrishnan et al.,

TABLE 3.5 Basic characteristics of the disk request traces (Ganger, 1995).

Trace Name	Length (hours)	#of Disks	#of Requests	Average Size	Percent Reads	Percent Seq. Reads
Cello	168	8	3,262,824	6.3KB	46.0%	2.5%
Snake	168	3	1,133,663	6.7KB	52.4%	18.6%
Air-Rsv	9	16	2,106,704	5.1KB	79.3%	7.8%
Sci-TS	19.6	43	5,187,693	2.4KB	81.5%	13.8%
Order	12	22	12,236,433	3.1KB	86.2%	7.5%
Report	8	22	8,679,057	3.9KB	88.6%	3.8%

1992). Air-Rsv is from a txn processing environment with 500 agents. Sci-TS is from a scientific time-sharing environment in which analytic modeling software and graphical and statistical packages were used. Order and Report are from a machine parts distribution company. Order was collected during daytime hours, representing an order entry and processing workload. Report was collected at night, capturing a batch environment in which reports of the day's activities were generated.

Synthetic trace generators tend to be inadequate because they ignore or simplify feedback effects between individual request response times and subsequent arrivals. Also, they predict performance changes with I/O subsystem performance metrics, which do not correlate with changes in overall system performance. These problems are demonstrated in (Ganger and Patt, 1993).

A new approach to generate a synthetic disk workload based on the analysis of real disk traces is presented in (Goodman and Sayano, 1990). The workload generation model can capture the heavy-tailed behavior of real disk workload, a critical feature to reproduce disk subsystem congestion. The generator provides synthetic workload much more accurately than commonly-used models. Since the workload plays a critical role in performance evaluations, having a more accurate disk workload generator is important for storage researchers in order to obtain fair and unbiased performance predictions.

Some workloads exhibit a bursty arrival process as shown in (Hsu and Smith, 2003), which may lead to a worse disk performance than with Poisson arrivals with the same rate. A hybrid method using simulation is used in (Thomasian and Liu, 2004) to estimate the mean disk service time as a function of the number of requests available for scheduling. These are then used in a higher level analytical model with Poisson arrivals to estimate the mean disk response time in which case the mean response time is underestimated.

DiskSim used for simulation of disk scheduling policies in (Worthington et al., 1994) was developed by Yale Patt and his student Greg Ganger and Bruce Worthington at U. Michigan. DiskSim was brought to PDL/CMU by Greg Ganger, where it was further extended (Bucy et al., 2008).

<http://www.pdl.cmu.edu/DiskSim/index.shtml>

PDL researchers developed the DIXTRAC tool to extract disk parameters for DiskSim by exercising the disk rather than relying on disk specs.

<http://www.pdl.cmu.edu/Dixtrac/index.shtml>

Seagate's 15K RPM 146 GB Cheetah was the last disk whose characteristics were extracted in 2007.

<http://www.pdl.cmu.edu/DiskSim/diskspecs.shtml>

RAIDframe was developed CMU/PDL for rapid RAID prototyping and simulation (Courtright et al., 1996), (Courtright et al., 1997).

The Pantheon storage system simulator was developed at HP Labs in a project led by John Wilkes. It was originally intended to do performance modeling of parallel disk arrays. Over time it has been extended and generalized so that it now supports modeling of a wide range of I/O systems (Wilkes, 1996). It was used in the following projects at HP Labs:

- (1) TickerTAIP parallel RAID architecture (Cao et al., 1994c).
- (2) Analysis of idle-period detection and prediction algorithms (Golding et al., 1995b).
- (3) The HP AutoRAID advanced disk array technology (Wilkes et al., 1996).
- (4) Performance and availability analyses.

Proteus (Greek mythology) for reliability estimation by simulation was a portable simulation program to estimate data loss in large disk arrays. Proteus models mirrored disks, all RAID levels, and some two-dimensional arrays (Kao et al., 2013). It was used to predict the risk of data loss for RESAR disk array (Schwarz et al., 2016) described in Chapter 4. It is stated: "Our measurements have shown a surprisingly good agreement with the reliability figures obtained through analytical techniques and indicate that there is very little difference between the reliability values obtained assuming deterministic repair times (Kao et al., 2013).

Cloud Quality of Service Simulator for Reliability - CQSIM-R is a Monte-Carlo simulator for large scale storage systems, which takes into account the placement of data (Hall, 2016). A further study along these lines was conducted in (Zhang et al., 2017).

Performance analysis of disk scheduling policies other than FCFS with Poisson arrivals and independent service times is mathematically complex and requires unrealistic assumptions for the sake of tractability.

3.10 Analytic studies of disk storage

Queueing analyses of disk and drum storage systems are reviewed in (Coffman and Hofri, 1986, 1990).

The performance of IBM 2314 DASD is analyzed in (Abate et al., 1968). The analysis in (Thomasian and Menon, 1994, 1997) uses the technique presented in this work to approximate the seek time characteristic by line segments.

An analysis of the SCAN policy for reducing disk seek times is presented in (Coffman et al., 1972). A comparison of two disk scanning policies is presented in (Edward et al., 1973). A queueing analysis of the scan policy for moving-head disks is provided in (Oney, 1975). The expected performance of scanning disks is analyzed in (Coffman and Hofri, 1982). A class of FIFO queues arising in computer system analysis is reported in (Coffman and Hofri, 1978).

An analytical model to predict the throughput of RAID 1/0 arrays (mirrored disks) was reported in (Uysal et al., 2001) at HP (Uysal et al., 2001). Inputs to the model are:

- (1) request rate,
- (2) request sizes,
- (3) run count - number of requests made to contiguous addresses,
- (4) queue length - mean size of device queues,
- (5) amount of data accessed between consecutive accesses to same block.

There are three levels in the model: the cache, controllers, and disks. The model is validated for a state-of-the-art disk array for several synthetic workloads and array configurations. Predictions are accurate within 15% on the average) for the set of experiments conducted.

A comparison of the performance of some of the aforementioned disk scheduling methods using trace-driven and synthetic simulation is given in (Worthington et al., 1994), (Thomasian and Liu, 2002a,b, 2004), (Hsu and Smith, 2004).

Drums are easier to analyze since they are head per track devices, so that their service time is the sum of latency and transfer time. The *Vacationing Server Model - VSM* was utilized in (Fuller and Baskett, 1975), (Fuller, 1975).

Publications at Seagate Research Center are relevant to this discussion are (Riska and Riedel, 2006a,b).

3.11 Analysis of a zoned disk with the FCFS scheduling

FCFS constitutes a baseline scheduling policy against which other policies can be compared. This is the reason why most analytical studies of single disks and RAID arrays have dealt with FCFS scheduling. (Merchant and Yu, 1992), (Chen and Towsley, 1993), (Kuratti and Sanders, 1995), (Thomasian and Menon, 1994, 1997), (Chen and Towsley, 1996), (Merchant and Yu, 1996), (Thomasian et al., 2007a).

The FCFS disk scheduling policy with Poisson arrivals can be analyzed assuming that successive disk requests are independent random variables, which are the sum of seek, latency, and transfer time. The *Pollaczek-Khintchine - P.K.* formula for the mean waiting time in the M/G/1 queueing system (Kleinrock, 1975) requires the arrival rate of disk requests (λ) and the first two moments of disk service time with FCFS policy $E[T_{FCFS}^i]$, $1 \leq i \leq 2$.

The maximum disk throughput for FCFS is $\lambda_{FCFS}^{max} = E[T_{FCFS}]^{-1}$ and the mean waiting time is given by Eq. (3.1) (Kleinrock, 1975):

$$W = \frac{\lambda E[T_{FCFS}^2]}{2(1 - \rho)}, \quad (3.1)$$

where $\rho = \lambda E[T_{FCFS}] < 1$ is the disk utilization. The mean response time of the disk is $R = W + E[T_{FCFS}]$.

The variance of waiting time is (Kleinrock, 1975):

$$\sigma_w^2 = W^2 + \frac{\lambda E[T_{FCFS}^3]}{3(1 - \rho)}.$$

Assuming that the probability that the same cylinder is accessed is p and the remaining $C - 1$ cylinders are accessed uniformly then the probability that the seek distance is d is given as (Lavenberg, 1983):

$$p_D(d) = \begin{cases} p & \text{if } d = 0, \\ (1 - p) \frac{2(C-d)}{C^2} & \text{if } 1 \leq d \leq C - 1. \end{cases}$$

The transfer time is a function of the block size to be transferred, but this also depends on the placement of the block in zoned disks. Postulating that data blocks are uniformly placed over all disk cylinders is a pessimistic assumption, since there is a tendency to write on a subset of outer disk tracks, which is called short stroking. This reduces seek time and increases data transfer rates, since more sectors are read per unit time.

<https://www.tomshardware.com/reviews/short-stroking-hdd,2157.html> The moments of the seek time can be obtained for a given seek time characteristic: $t_{seek}(d)$, $1 \leq d \leq C - 1$, which has been extracted for several disks by the DIXTRAC tool for automated disk drive characteristic extraction at CMU's PDL.

<https://www.pdl.cmu.edu/Dixtrac/index.shtml>

For small block sizes that constitute a small fraction of the circumference of a track the latency is uniform over the disk rotation time. A more complex and accurate model for larger blocks is presented in (Thomasian et al., 2007a). There are two cases where the head lands inside the disk block to be accessed with probability q , which is the ratio of the number of sectors in the block and the number of sectors per track and outside the block with probability $1 - q$. Thus with probability q the latency equals one disk rotation and with probability $1 - q$ the latency is uniform over the remaining track sectors, whose number can be translated into time.

The transfer time equals the ratio of the number of sectors per block and the number of sectors per track times disk rotation time. The average transfer time over the blocks being accessed can be computed easily.

Disk service time can be expressed as

$$T_{FCFS} = T_s + T_\ell + T_x,$$

where T_s is the seek time, T_ℓ is the rotational latency, and T_x is the transfer time. Taking expectations from both sides the i^{th} moment of disk service time is given as:

$$E[T_{FCFS}^i] = E[(T_s + T_\ell + T_x)]^i.$$

The computation of the second moment $E[T_{FCFS}^2]$ leads to $E[T_s T_\ell]$ which equals $E[T_s]E[T_\ell]$, since T_s is independent from T_ℓ . The variance of T_{FCFS} can be expressed as the sum of the variances of its components, assumed to be independent (Trivedi, 2002):

$$\sigma_{FCFS}^2 = \sigma_s^2 + \sigma_\ell^2 + \sigma_x^2$$

For larger block transfers T_ℓ is not independent of T_x , we therefore obtain the sum of latency and transfer time: $T_\ell + T_x$ (Thomasian, 2006c).

3.11.1 Disk service time in zoned disks with FCFS scheduling

In a zoned disk assuming uniform accesses to all disk blocks, the probability that the disk arm accesses cylinder k is the ratio of the number of sectors on the cylinder $S_T(k)$ and the total number of sectors on a disk with C cylinders:

$$p_K(k) = \frac{\#_of_sectors_on_cylinder_k}{total_#_of_sectors_on_disk} = \frac{S_T(c)}{\sum_{c=1}^C S_T(k)}, \quad 1 \leq k \leq C.$$

Two ways to have a seek distance d from cylinder k are to seek to cylinder $k \pm d$. Given the probability of accessing various cylinders, it follows:

$$p_{D|K}(d|k) = \begin{cases} p_K(k) & d = 0 \\ p_K(k - d) + p_K(k + d) & d \neq 0. \end{cases}$$

The following two conditions are required: $k > d$ and $k + d \leq C$, i.e., $p_K(k) = 0$ for $k \leq d$ and $k + d > C$. Unconditioning on k using the equation for $p_K(k)$ we obtain:

$$p_D(d) = \sum_{k=1}^C p_{D|K}(d|k) \times p_K(k).$$

$$E[T_s^i] = \sum_{d=1}^{C-1} [t_{seek}(d)]^i p_D(d).$$

The rotational latency for accessing small blocks can be approximated by uniformly distributed points on the circumference of a track, i.e., T_ℓ is uniform over $(0, T_r)$, and its i^{th} moment is given as $E[T_\ell^i] \approx T_{rot}^i / (i + 1)$.

The transfer time of disk blocks with $S_B = j$ sectors per block on cylinder k is given as $T_x(j, k) = j T_{sector}(k)$, where $T_{sector}(k) = T_r / S_T(k)$ is the transfer time per sector on cylinder k . The i^{th} moment of transfer time for blocks with j sectors is given by:

$$E[T_x^i(j)] = \sum_{k=1}^C (T_x(j, k))^i p_K(k) = \sum_{k=1}^C (j T_{sector}(k))^i p_K(k).$$

Unconditioning on the number of sectors per accessed block yields:

$$E[T_x^i] = \sum_{\forall j} Pr[S_B = j] E[T_x^i | j],$$

where $Pr[S_B = j]$ is the probability that a request accesses j disk sectors.

The accuracy of this analysis has been validated against simulation results in (Thomasian et al., 2007a).

3.12 Performance analysis of the SCAN policy

An approximate analysis of the SCAN policy was reported by at IBM Research in (Polyzois et al., 1993). Given the number of disk requests to be processed is m and the requests are uniformly distributed over all disk blocks in a nonzoned disk (with an equal number of blocks per cylinder) with C cylinders, the analysis determines the number of accessed cylinders as:

$$\bar{n} = C[1 - (1 - \frac{1}{C})^m].$$

For large C and $m \ll C$ all requests will tend to be at different cylinders, so that $\bar{n} \approx n$. The seek time for successive accesses then uses an average seek distance $d = C/\bar{n}$ to compute the average seek time for each access with a formula of the form $E[T_s] = a + b\sqrt{d}$.

Elizabeth Varki (U. New Hampshire) and Arif Merchant (IBM, NEC, HP, Google)

An analysis of the SCAN policy is reported in (Varki et al., 2004) by estimating the mean seek distance of requests when a certain number of requests are available at a disk by approximating the mean seek distance as the ratio of maximum distance and disk_queue + 1, i.e., $\bar{d}[q] = (C - 1)/(q + 1)$. This is incorrect as noted in (Thomasian and Liu, 2005a), since for $q = 1$ we have $\bar{d}[1] = (C - 1)/2$, while the expected value is $\bar{d}[1] \approx (C - 1)/3$.

Define q as the number of requests in a closed system, where a completed disk request is immediately replaced by a new request. The mean seek distance $\bar{d}[q] \approx (C - 1)/(q + 2)$ follows from the fact that the mean seek distance of n uniformly distributed points sorted in increasing order on a unit line is $1/(n + 1)$, the variance $n/[(n + 2)(n + 1)^2]$. The discrepancy is due to the fact that with q disk requests we effectively have $q + 1$ requests, where the additional request is the one that just departed, leaving the disk arm in its current position. We use $\bar{d}[q]$ for the formula in (Varki et al., 2004) and $\bar{d}[q]$ for the corrected formula.

Using the mean seek distance to estimate the mean seek time is another weakness of the analysis in (Varki et al., 2004). In a similar vain substituting the *Multiprogramming Level - MPL* distribution with its average in multiprogrammed

computer systems, can lead to a significant error in estimating system throughput as shown in (Dowdy and Gordon, 1987). It is also shown in (Dowdy et al., 1984) that the throughput characteristic is concave in a closed QNM of single server queues. The seek time characteristic consists of a concave region for shorter seek times and a linear region for higher seek distances.

A detailed simulation of the IBM 18ES, 9.17 GB, 7200 RPM disk drive was used in (Thomasian and Liu, 2005a) to obtain the error in estimating seek distances for the SCAN policy. The simulator configures itself by accessing the database of extracted disk parameters. In Fig. 3.1 we plot the normalized mean seek distance obtained by simulation, to estimate $\tilde{d}[q]/C \approx 1/(q+1)$ and $\bar{d}[q]/C \approx 1/(q+2)$ versus q . It is seen that $\tilde{d}[q]/C$ overestimates the mean seek distance and that $\bar{d}[q]/C$ provides a much better approximation.

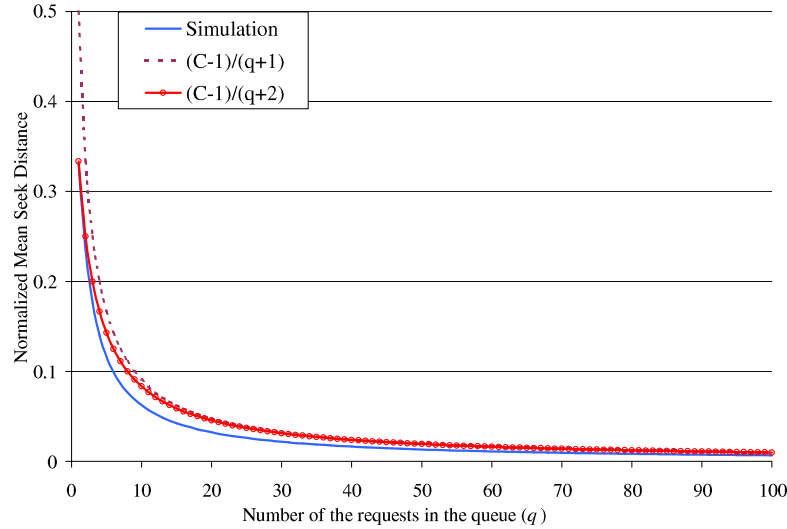


FIGURE 3.1 Normalized seek distance versus the number of requests in the queue.

We next consider the analysis of a single disk drive with the SCAN policy. The disk drive processes requests to small randomly placed disk blocks from a Poisson arrival stream with rate λ . The mean service time with q requests, denoted by $\bar{x}_{disk}[q]$, is the sum of the mean seek time with q requests: denoted by $\bar{t}_{seek}[q]$, the mean rotational latency approximated by half a disk rotation time. The transfer time is assumed to be negligible. Three different methods to compute $\bar{t}_{seek}[q]$ are:

- (1) Setting $\tilde{d}[q] = (C-1)/(q+1)$ (incorrect).
- (2) Using simulation to obtain the mean seek distance $\bar{d}[q]$, although $(C-1)/(q+2)$ was shown to be a good estimate.
- (3) Using simulation to obtain $\bar{t}_{seek}[q]$ directly.

In Fig. 3.2 we plot the mean response time versus the arrival rate normalized with the mean service time of the FCFS policy: $\lambda \bar{x}_{FCFS}$. Very accurate results are obtained with method (3), which is tantamount to obtaining the seek distance distribution $p_d[q]$, $1 \leq d \leq C-1$ and applying it to $seek_time[d]$ to compute $\bar{t}_{seek}[q]$. Results are less accurate with method (2) and the method in (Varki et al., 2004) is incorrect.

Experimental results show that as the number of requests available for scheduling increases, the mean service time for SCAN decreases much more slowly than with SATF (see Section 3.3.1). For the IBM 18ES $p \approx 1/8$ in q^{-p} , while $0.14 \leq p \leq 0.15$ for other disk drives, whose characteristics are given at the PDL web site.

The analysis of SCAN with the assumption that requests are uniformly distributed over all cylinders is an approximation for zoned disks. In a zoned disk, in theory, q requests would partition the disk to $q+1$ equal capacity regions, so that more requests will be concentrated at outer disk cylinders, since they hold more data than inner cylinders (Thomasian and Fu, 2006). The requests at the outer disk cylinders will be closer together and the requests at the inner cylinders will be further apart. In effect some seek times will be shorter and some longer than a nonzoned disk, so the effect of zoning on overall mean seek time with SCAN is not significant, but a higher variance in seek time is expected.

Analysis of mean response time for C-SCAN

The following analysis is based on (Cady et al., 2011), which uses the following notation.

r_{min} & r_{max} radius of innermost & outermost track.

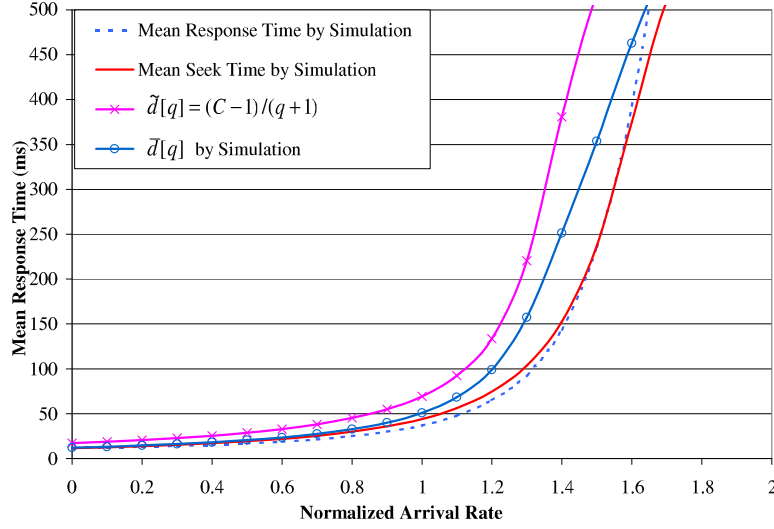


FIGURE 3.2 Mean response time versus normalized arrival rate.

σ the speed of read-write head.

τ_0 the time to backtrack in C-SCAN.

$E[M]$ mean head moving time.

$E[P]$ mean processing time.

$E[T]$ mean access time of a random request.

N_Q number of jobs in the system not being processed at a random time (in the queue).

λ arrival rate of requests.

S the sum of rotational latency and transfer time.

$\rho = \lambda E[S]$ the system load.

The analysis for $E[M]$ is based on (Coffman et al., 1972). $E[M]$ varies from SCAN to C-SCAN and in the latter case

$$E[M] = \frac{1}{2}[\tau_0 + (r_{\max} - r_{\min})\sigma]$$

Given *Poisson Arrivals See Time Averages - PASTA* (Wolff, 1982) we have: $E[S_e] = E[S^2]/(2E[S])$, which is the mean residual of S :

$$E[P] = \rho E[S_e] + E[N_Q]E[S] + E[S] = \rho E[S_e] + \rho E[T] + (1 - \rho E[S]).$$

$$E[T] = E[M] + E[P] = E[M] + \rho E[S - e] + \rho E[T] + (1 - \rho E[S])$$

$$(1 - \rho E[T] = E[M] + \rho E[S_e] + (1 - \rho)E[S].$$

The average time for C-SCAN is then:

$$E[T] = \frac{E[M]}{1 - \rho} + \frac{\rho}{1 - \rho} \frac{E[S^2]}{2E[S]} + E[S]$$

The analysis is extended to ZBR and validated against simulation. The errors of the approximation are bounded as:

$$\frac{E[M] + \rho E[S_c]}{1 - \lambda \min_r E[S(r)]} \leq E[T] \leq \frac{E[M] + \rho E[S_c]}{1 - \lambda \max_r E[S(r)]}$$

3.13 Analysis of the SATF policy

The disk transfer time of a given data block is determined by the track on which it is allocated, so that this time can be reduced by placing larger blocks on outer tracks in ZBR disks. This is disadvantageous from the viewpoint of some disk scheduling policies.

Given the position of the arm after completing a request, the disk controller can rather accurately estimate the time required to process a request from the queue according to (Ng, 2001). This feature makes it possible to implement the SATF or SPTF policy, also known as *Rotational Positioning Optimization - RPO*.

No distinction is made between SATF and SPTF in our discussion and we assume that both minimize disk positioning time, since the transfer time is negligible for small disk blocks. SATF has similarities to the *Shortest-Job-First - SJF* policy, which utilizes a priori knowledge of job service times to prioritize the processing of a request with the smallest service time. SJF minimizes the mean response time compared to all other nonpreemptive job scheduling policies (Kleinrock, 1976). SATF is non-work-conserving queueing discipline, since service times vary according to the order of disk access arrivals.

SATF has been considered in numerous studies and has been shown to outperform the SCAN policy (Seltzer et al., 1990), (Jacobson and Wilkes, 1991), (Worthington et al., 1994), (Thomasian and Liu, 2002a,b, 2004), (Hsu and Smith, 2004). SATF is implemented in some disk controllers with the *SCSI - Small Computer Systems Interface*, which can hold up to 128 requests.

<https://en.wikipedia.org/wiki/SCSI>

The rest of this section is organized as follows. After a preliminary investigation of SATF, we propose two methods to evaluate its performance. Variations of SATF to attain better performance and preventing starvation but also overload are also discussed.

3.13.1 Preliminary investigation of SATF

We simulated the SATF scheduling policy to determine, whether the next request to be processed is based on seek time or rotational latency. In other words we are interested in knowing which component of service time has a dominant effect for different disk characteristics. Simulation results for two disk drives with very different characteristics, which are given in Table 3.6, are shown in Fig. 3.3. The mean disk service time $\bar{x}_{disk}$, which would be the mean service time with FCFS is used for normalizing the arrival rate: $\lambda_n = \lambda \bar{x}_{disk} = \lambda E[T_{FCFS}]$.

TABLE 3.6 Disk parameters used in our experiments.

Company	Quantum	Seagate	IBM	Seagate
Disk	Atlas 10K	Cheetah 9LP	Ultrastar 18ES	Elite
Capacity	9.1 GB	9.1 GB	9.1 GB	1.4 GB
RPM	10,025	10,045	7,200	5,400
Diameter (in inches)	3.5	3.5	3.5	5.25
# of Band	24	11	55	14
Max/Min Sectors/Track	334/229	254/167	390/247	85/61
Mean Service Time (ms)	8.70	8.65	11.55	17.78

The graphs show that at low arrival rates both the seek time and rotational latency have the minimum value among the requests available for disk scheduling. As the arrival rate increases the seek time dominates the rotational latency (note that the RPM for one disk is twice that of the other disk). There is a crossover point at normalized arrival rate $\lambda_n \approx 1.75$ beyond which there is a high probability that neither seek time, nor rotational latency alone determine the SATF request.

The fractions of the various cases are related to each other as follows:

$$\text{MinSeekTime} + \text{MinRotationalLatency} - \text{MinBoth} + \text{Neither} = 1.$$

In order to analyze SATF we need to determine the minimum of the sum of two random variables: seek time and rotational latency, which is difficult to determine, since it is a function of the number of outstanding disk requests, but also the current position of the read/write head.

A simulation to estimate the first two moments of disk service time with SATF yielded: $\bar{x}_{SATF}^i$, $i = 1, 2$ and also the mean waiting time of requests W_{SATF} at an arrival rate λ . Unlike the FCFS policy these moments are functions of the queue-length, and hence the arrival rate λ . The Pollaczek-Khintchine formula (3.1) for the M/G/1 queueing system which holds for the FCFS queueing policy does not hold for SATF, since SATF is non-work-conserving discipline.

Fig. 3.4 shows simulation results for the mean response with the SATF and FCFS policies versus the arrival rate for the four disk drives in Table 3.6. The simulator uses detailed disk characteristics given at Diskspecs web site. It is observed that SATF outperforms FCFS by a wide margin, since FCFS saturates when λ approaches $\lambda_{FCFS}^{Max} = E[T_{FCFS}]^{-1}$.

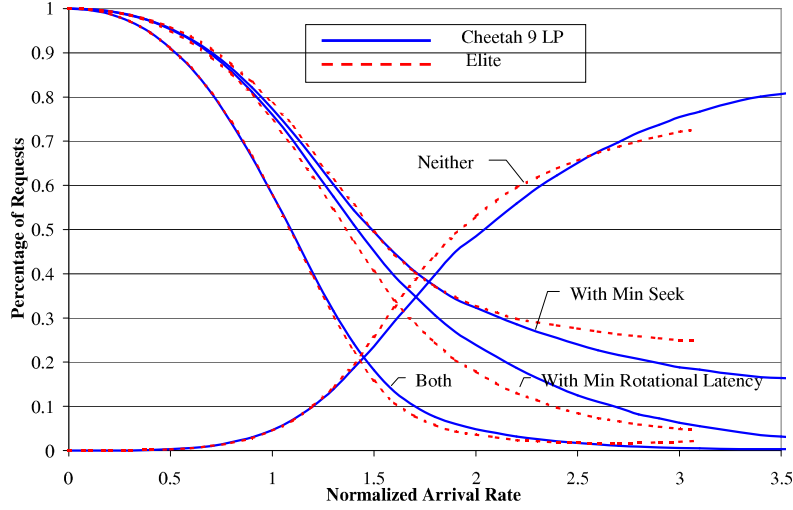


FIGURE 3.3 Dominance of seek time and rotational delay with SATF.

The SATF policy can attain a much higher throughput than FCFS, since starting with the same mean service time as FCFS at low arrival rates and short queue-lengths, its mean service time decreases with increasing queue-lengths, a relationship which is characterized in this study.

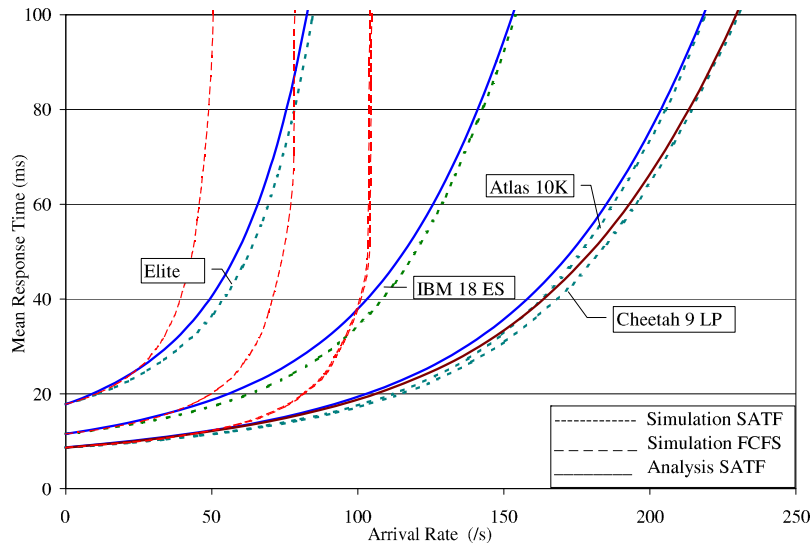


FIGURE 3.4 The mean response times for various disks versus the arrival rate (the mean response times with asymptotic values correspond to Elite, IBM18ES, ATLAS10K, and Cheetah9LP disk drives respectively).

3.13.2 First method for the analysis of SATF

We apply hierarchical modeling (Thomasian and Gargeya, 1984) for analyzing disk performance with the SATF policy, which is quite similar to the analysis used for the SCAN policy. A more detailed discussion is given here, especially for the higher level analysis. Similarly to SCAN, the lower level model uses simulation to obtain the mean service time of disk requests for varying number of requests in a closed model. At the higher level we assume Poisson arrivals with rate λ , so that a birth-death model (Kleinrock, 1975) is applicable, where the state is determined by the number of requests (m) and the service rate varies with the inverse of mean service time with m requests.

We simulated each one of the four disks considered in this study with a fixed number of disk requests available for SATF scheduling. Upon the completion of each disk request, a new randomly placed request was inserted into the queue of pending requests, before the disk scheduling step was repeated. The number (m) is varied according to $1 \leq m \leq M$ to

obtain the mean disk service time $\bar{x}_{SATF}[m]$. M is in fact the maximum number of requests that can be held at the disk and are accessible to the SATF scheduler of the disk controller. We have set $M = 100$, although $M = 128$ would have been a better upper limit for modern disk drives.

Fig. 3.5 is a plot of the disk service times normalized with respect to $\bar{x}_{disk} = E[T_{FCFS}]$: $x[m] = \bar{x}_{SATF}[m]/\bar{x}_{disk}$ versus m . The mean service time characteristic $x[m]$ is a decreasing function of m , which does not reach a limiting value even at $M = 100$.

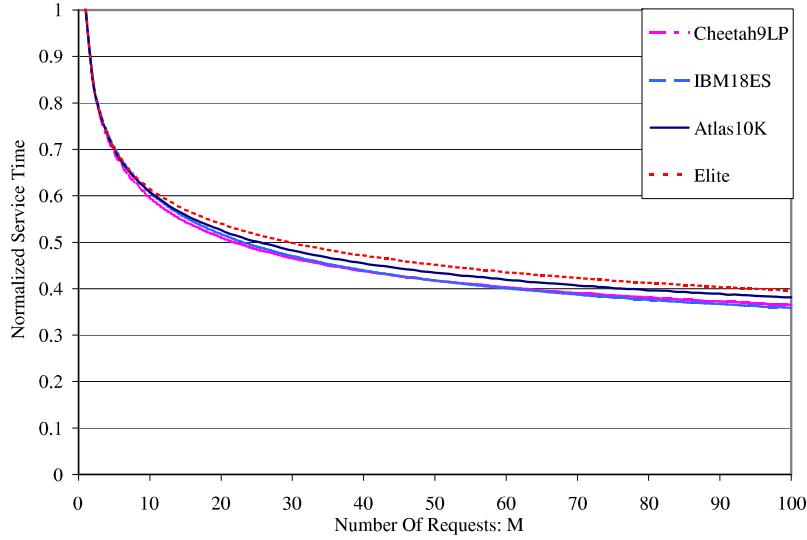


FIGURE 3.5 Normalized mean service time versus number of requests.

Assuming state-dependent service times are exponentially distributed then the state-dependent service rates are inverses of service times: $\mu[m] = 1/\bar{x}_{SATF}[m]$.

For an arrival rate λ we use a birth-death model to obtain the mean disk response time (Kleinrock, 1975). The steady-state equations are given as follows:

$$\lambda P[m-1] = \mu[m]P[m], \quad 1 \leq m \leq M_{max}.$$

The value of M_{max} is not known a priori, but is selected such that the accumulated probability mass approaches one. For $M_{max} \geq M$ for $m \geq M$ we set $\mu[m] = \mu[M]$, i.e., the performance of SATF does not improve beyond M , since this is the maximum number of disk requests available for SATF scheduling. A steady-state solution exists and the system will not be saturated provided $\mu[M_0] > \lambda$ for $M_0 \leq M$ (Kleinrock, 1975).

The computation of R_{SATF} is given below.

COMPUTATION OF MEAN RESPONSE TIME IN SATF

Input: Arrival rate λ , normalized access time $x[m]$.

Output: Mean response time R .

```

m = 0           // initialization
P[0] = 1.0      // initialization
S = 0.0        // to sum probabilities
while (P[m] ≥ ε) do
  m ++
  P[m] = λ P[m - 1] × x[m]
  S += P[m]      // sum of probabilities
  N += m P[m]    // sum number in system
end

```

$$\begin{aligned}
S+ &= \bar{x}_{disk}S + P[0] \\
\bar{N} &= \bar{x}_{FCFS} \times \bar{N}/S && // \text{mean number of requests} \\
R_{SATF} &= \bar{N}/\lambda && // \text{Little's result.}
\end{aligned}$$

We carried out this experiment for the four disk drives in Table 3.6 and plotted the mean response times obtained by analysis alongside simulation results in Fig. 3.4. It is observed that the mean response time obtained by analysis and simulation match quite well.

A byproduct of the closeness of the graphs for the mean disk service time characteristic $x[m]$, $1 \leq m \leq M$, in Fig. 3.5 is that given the mean service time ($E[T_{FCFS}]$) of another drive we can calculate R_{SATF} at a given arrival rate.

Curve-fitting to the normalized mean disk service time characteristic $x[m]$, $1 \leq m \leq M$ in the figure yields:

$$x[m] \approx m^{-p}, \quad 1 \leq m \leq M,$$

which is a good match for all four disks under consideration, as shown in Table 3.7. The standard error is given as follows:

$$\hat{\sigma}^2 = \sum_{i=1}^M (Y_i - \hat{Y}_i)^2 / (M - 1),$$

where M is the number of points ($N = M = 100$ in our study).

TABLE 3.7 SATF curve-fitting results with the four disks.

Disk_id	Value of p	Standard Error
IBM18es	0.2181	0.0005
Elite	0.2098	0.0014
Atlas10K	0.2142	0.0074
Cheetah9LP	0.2249	0.0006

We conclude that disk service time with SATF is roughly proportional to the inverse of the fifth root of the number of requests, e.g., at $m = 32$ the mean disk service time is halved. Results reported in (Anderson et al., 2003) concur with this result.

Before concluding this section it is worthwhile to note that the second moment of disk response time can be obtained using $\bar{N}(\bar{N} - 1) = \lambda^2 R^2$ (Kleinrock, 1975).

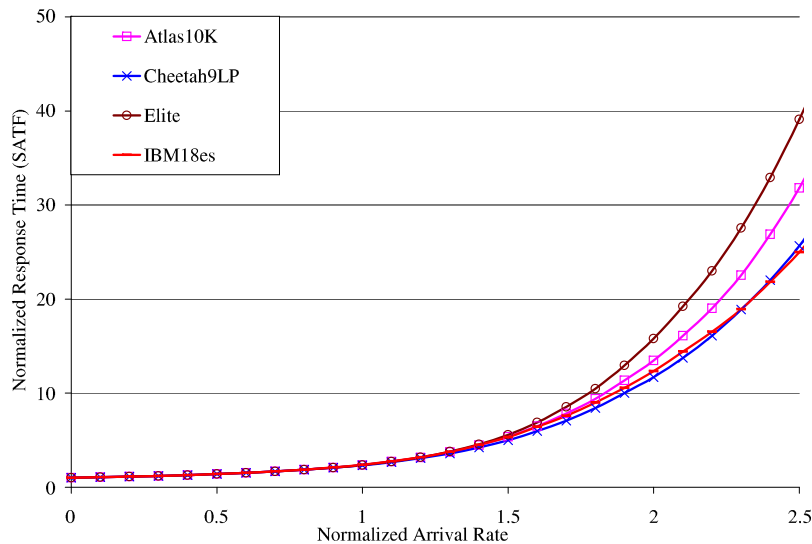


FIGURE 3.6 Normalized mean disk response time versus normalized arrival rate.

3.13.3 Second method for the analysis of SATF

Fig. 3.6 is a plot of the normalized mean response time characteristic (R_n) versus the normalized arrival rate (λ_n) for the four disk drives under consideration. The normalization for each disk drive is with respect to its own mean service time ($\bar{x}_{disk}$), which is listed in Table 3.6.

We observe that the normalized mean disk response times for the four disks are almost indistinguishable up to $\lambda_n = 1.5$, i.e., 1.5 times the arrival rate sustainable by FCFS. This leads to another method to estimate the mean response time, which is based on denormalizing both the mean response time and the arrival rate with respect to the mean service time of yet another disk drive.

3.14 Conclusions

There is a paradigm shift in that disks are used as high capacity storage systems for archiving and backup as noted in (Brewer et al., 2016), where disk capacity and transfer rate, rather than disk access time is important. Flash and even DRAM re used for high performance database applications, making most disk scheduling methods of historical interest only.

Chapter 4

Mirrored & hybrid arrays

Introduction to mirrored and hybrid disk arrays in Section 4.1
Mirrored and hybrid disk array organizations in Section 4.2
Performance improvement by routing of disk requests in Section 4.3
Shortening the tail for response times in Section 4.4
Improving write performance in mirrored disks in Section 4.5
Disks with multiple R/W heads on a single and multiple arms in Section 4.6
Seek distances with mirrored disks in Section 4.7
Mirrored disk performance in normal, degraded, and rebuild modes in Section 4.8
Queueing theory for disk performance evaluation in Subsection 4.8.1
RAID1 performance in normal mode in Subsection 4.8.2
Uniform vs round-robin routing of read requests in mirrored disks in Subsection 4.8.3
Completion times of multiple requests in RAID1 and RAID5 in Subsection 4.8.4
Operation in degraded mode in Subsection 4.8.5
Rebuild processing in RAID1 in Subsection 4.8.6
Protecting against rare event failures in archival systems in Section 4.9
RAIDP: ReplicAtion with Intradisk Parity for cost effective storage in Section 4.10
Remote mirroring for disaster recovery in Section 4.11
RAID reliability analysis in Section 4.12
Reliability analysis of mirrored and hybrid disk arrays in Section 4.12.1
Reliability analysis of variants of RAID1 and RAID(4 + k) w/o repair in Subsection 4.12.2
Performability analysis in Subsection 4.12.4
Shortcut reliability analysis in Subsection 4.12.5
Transient solutions of Continuous Time Markov Chains - CTMCs in Subsection 4.12.6
RAID reliability analysis with repair in Subsection 4.12.7
Solving linear Ordinary Differential Equations - ODEs in Subsection 4.12.8
Simplified RAID5 rebuild reliability analysis in Subsection 4.12.9
Enhanced reliability modeling of storage systems in Subsection 4.12.10
Metrics for reliability estimation in storage systems in Subsection 4.12.11
Expected Annual Fraction of Data Loss - EAFDL in Subsection 4.12.12
Taking into account controller failures in Section 4.12.13
Storage reliability research at IBM's Zurich Research Lab in Section 4.13
Conclusions in Section 4.14

4.1 Introduction to mirrored and hybrid disk arrays

Redundant Arrays of Inexpensive Disk - RAID organization for disk arrays was introduced in a seminal paper by David Patterson, Garth Gibson, and Randy Katz at Univ. Calif. at Berkeley (Patterson et al., 1988). RAID introduced a five level classification of disks array, of which RAID1 with mirrored disks was an early form of achieving highly reliable storage, which had been appeared in two commercial products. *Hybrid Disk Arrays* - HDAs store the *eXclusive OR* - XORed data blocks, but maintain the same redundancy level as mirrored disks.

Erasur Coding - EC incurs a much lower redundancy level than RAID1 with one parity disk in RAID5 and two check disks in RAID6 to protect against one and two disk failures, respectively.

RAID1 which doubles the data access bandwidth in high performance systems, such as *Tandem NonStop* - TNS (Tandem Database Group, 1989), which used *Basic Mirroring* - BM with two disks cross-connected to two processors to tolerate

disk as well as processor failures. This discussion may not apply to current Tandem computers, emulate the early systems.
https://en.wikipedia.org/wiki/Tandem_Computers
https://www.ceengineering.net/resources/tandem_nonstop_history.pdf

Elucidation by Tandem CE Engineering - Gary Hall 6/11/21

In respect of storage, the core Tandem features introduced from the outset of the platform in the mid-1970's - RAID1 disk mirroring, dual disk controllers/dual ported drives (multiple-path access to data), and the read-after-write data integrity check - have been retained throughout the platform's development/history. And these core features continue with the latest HPE NonStop X-Series.

Generic HPE SAN RAID product were incorporated into the 'Tandem' product portfolio over the course of the last decade but, ultimately, regardless of whatever level of RAID/data replication the generic HPE SAN is capable of, data is presented to the native Tandem O/S as a set of mirrored pair disks (a 'volume' in Tandem parlance).

Furthermore, the core Tandem storage features can even be replicated, verbatim, on a version of the NonStop O/S which has been ported to a 'virtualized' environment (independent of infrastructure).

This is not the full story, however, and the above is a bit of an over simplification as a large proportion of modern Non-Stop systems run Tandem's Open System Services (OSS) which effectively provides a virtualized O/S (LINUX/UNIX like and often visible to the user as a web server/service) running on top of the Tandem O/S (Guardian) where the Tandem O/S is the 'hypervisor' and through which the hardware storage configuration can be presented to the data in a myriad of different configurations.

Teradata DBC/1012 database computer with 10^{12} Byte or one *TeraByte* - TB capacity used mirroring with an *Interleaved Declustering* - ID organization, so that the primary data on each disk in a cluster of disks is replicated in the secondary areas of the remaining disks in that cluster. This results in a lower load increase than in the case of BM if a disk fails. Teradata DBC/1012 is also a *Data Base Machines* - DBM, which is discussed in Chapter 8.

EMC's Symmetrix is a successful early RAID1 product simulating *Extended Count Key Data* - ECKD format disks used by IBM mainframes (Bohl, 1981), (Wiederhold, 1987), (Sierra, 2012) on small form factor commodity disks with *Fixed Block Architecture* - FBA with 512 byte sectors.

https://en.wikipedia.org/wiki/EMC_Symmetrix

Symmetrix provides dynamic hot spares to save the contents of a failed/about to fail disk on it. When a bad drive is replaced by a new drive data is copied back from the dynamic spare to the new drive and the dynamic spare is released and becomes available for use again.

With permanent sparing data is copied to the spare and configuration changes are made to change the spare name to the failed volume name. When bad drive is replaced, it becomes part of the spare pool. The advantage of permanent sparing is that copying is required once while it has to be done twice with dynamic sparing. With increased disk capacities copying takes many hours.

<https://gestaltit.com/exclusive/devang/emc-symmetrix-permanent-sparing/>

There are more than two disks in RAID1/0 arrays, which consists of mirrored RAID1 disks with BM at the lower level, incorporated into a RAID0 array at the higher level. RAID0 uses striping which partitions large files into fixed size stripe units or strips to balance disk loads (Chen et al., 1994b), but in this case striping is applied to both mirrored pairs.

Three-way replication has been used by the *Hadoop Distributed File System* - HDFS, since it provides reliability and parallel access to disk drives.

<http://en.dwhwiki.info/concepts/hdfs/hdfs>

Hadoop writes data on Racks, which are physical collections of Datanodes in a Hadoop cluster (30 to 40). With the help of Racks Namenode chooses the closest Datanode to achieve maximum performance while reading/writing data which reduces network traffic. A Rack can have multiple Datanodes storing the file blocks and their replica's. The Hadoop itself automatically writes a particular file block in two different Datanodes in a rack.

<https://www.geeksforgeeks.org/hadoop-rack-and-rack-awareness/>

Rack awareness policies are as follows.

- (1) There should not be more than 1 replica on the same Datanode.
- (2) More than 2 replicas of a single block is not allowed on the same Rack.
- (3) The number of Racks used inside a Hadoop cluster must be smaller than the number of replicas. HDFS places one

replica on the local node, another replica on a different node at the local Rack, and the last replica on a different node at a different Rack. This policy improves write performance, while not impacting data reliability or read performance.

DiskReduce developed at the *Parallel Data Laboratory - PDL* at CMU initially writes three copies of data to accommodate HDFS, but then converts triplicated data to RAID5 or RAID6 formats since as data becomes stale it is not frequently accessed anymore (Fan et al., 2009).

Solid State Disks - SSDs based on flash technology provide shorter access times, consume less power than *Hard Disk Drives - HDDs*, are nonvolatile and are less costly than DRAM, which can be made nonvolatile by providing *Uninterruptible Power Supply - UPS* at an extra cost.

https://en.wikipedia.org/wiki/Uninterruptible_power_supply

Flash memories and other technologies providing nonvolatility are discussed in Chapter 2.

HDDs are being gradually replaced by SSDs and even DRAM for high performance applications, but only SSDs are discussed in conjunction with mirroring in this chapter for the following reasons:

1. Mirroring has been implemented mainly in the context of HDDs.
2. Mirroring is less suitable for flash memories because of the higher cost and that efficient coding methods are available for SSDs.
3. More HDDs are being shipped than SSDs as discussed below.

380 PetaBytes (PB = 10^{15} B) of HDDs were shipped versus 28 PB of NAND Flash in 2012, while this figure in 2018 increased to 780 PB for HDDs and 176 PB for NAND flash, according to IBM's Robert E. Fontana (Fontana, 2018). The revenue for HDD shipments was \$26.1. vs \$56.5 Billion for NAND, which is due to the higher cost of flash memories.

Over 90% of enterprise drive shipments today are HDDs, according to *Int'l Data Corp. - IDC*, but by 2025, HDDs will comprise almost 20% of drive shipments, while SSDs will gain market share. The total growth in data created will result in massive sales of both HDDs and SSDs.

<https://www.backblaze.com/blog/hdd-vs-ssd-in-data-centers/>

SSDs outsell HDDs by a factor of 2.6 in Q2 of 2021. Many interesting items are provided in this report.

<https://www.tomshardware.com/news/client-ssds-outsell-client-hdds-by-2-6-times-in-q2>

In this chapter we review the more influential papers scattered over a large number of journals and conferences, which cover mirrored disk arrays.

4.2 Mirrored and hybrid disk array organizations

There have been numerous proposals for RAID1 organizations, which are improvements to BM to attain a more balanced disk load when a disk fails, but RAID1 reliability is another important consideration discussed in Section 4.12.

As noted earlier HDAs store redundant data in the form of XORed data blocks. This results in a higher reliability, but this is at the cost of updating data. In Section 4.2.14 we list commonalities among considered disk arrays.

4.2.1 Basic Mirroring - BM

Basic Mirroring - BM is the original and simplest form of disk mirroring, which replicates data on two identical disks, but the data can also be replicated on heterogeneous storage media with equal storage capacity.

For higher volumes of data there are M pairs of mirrored disks with BM organization, so that the total number of disks is $N = 2M$. RAID1/0 is a hierarchical RAID organization with mirrored pairs at the lower level and RAID0 at the higher level. Striping across mirrored pairs is optional, but will result in a balanced load. Up to M disk failures can be tolerated, as long as the failed disks are not pairs, e.g., the probability of data loss due to a second disk failure is $1/(2M - 1)$.

RAID0/1 mirrors two RAID0 arrays, which are considered as a superdisk with M disks as shown in Fig. 4.1. The failure of any disk results in the failure of the superdisk and recovery may not be considered at the level of individual failed disks. Up to M disk failures can be tolerated as long as they are all on one side. The probability of data loss due to a second disk failure is: $M/(2M - 1) > 0.5$.

RAID1/0 and RAID0/1 are examples of nested RAID levels,

https://en.wikipedia.org/wiki/Nested_RAID_levels

whose reliability was analyzed in (Thomasian, 2006a) and is repeated in Section 4.12.

BM like other mirrored organizations doubles disk access bandwidth for read requests, but if a disk fails then the read load of the surviving disk is doubled. This is especially a problem if the data is not striped and the mirrored pair is heavily loaded. The resulting disk overload may reduce the throughput of an *OnLine Transaction Processing - OLTP* system Most

Primary Disks				Secondary Disks			
D_1	D_2	D_3	D_4	D_5	D_6	D_7	D_8
A	B	C	D	A'	B'	C'	D'
E	F	G	H	E'	F'	G'	H'
I	J	K	L	I'	J'	K'	L'
M	N	O	P	M'	N'	O'	P'

FIGURE 4.1 RAID0/1 with $N = 8$ disks. Primed blocks replicate unprimed blocks.

performance studies of RAID1 have been conducted in the context of the BM configuration, an exception is (Thomasian and Xu, 2008).

4.2.2 Group Rotate Declustering - GRD

GRD proposed in (Chen and Towsley, 1996) is a RAID0/1 array with mirroring at the higher level and RAID0 at the lower. The RAID0 on one side is backed up by a RAID0 whose strips are shifted by one, so that the failure of a disk on either side will result in the distribution of its loads across all disks at the other side

On one side are M striped primary disks and on the right M secondary disks whose data is rotated from row to row as shown in Fig. 4.2.

Primary Disks				Secondary Disks			
D_1	D_2	D_3	D_4	D_5	D_6	D_7	D_8
A	B	C	D	A'	B'	C'	D'
E	F	G	H	H'	E'	F'	G'
I	J	K	L	K'	L'	I'	J'
M	N	O	P	N'	O'	P'	M'

FIGURE 4.2 Group Rotate Declustering with $N = 8$ disks. Shifted primed blocks replicate unprimed blocks.

GRD has the advantage that upon the failure of a disk on either side its read load is distributed evenly over the M disks at the other side. GRD can tolerate up to M disk failures on one side, but data loss occurs if a second disk fails at the other side, so that the probability of data loss similarly to RAID0/1 is $M/(2M - 1) > 0.5$. The fraction of GRD requests routed can be adjusted to balance disk loads as discussed in Section 4.8.

Two variants of data placements for GRD are described in (Chen and Towsley, 1996).

1. Disk space is halved by allocating primary data on outer cylinders and secondary data on inner cylinders. With *Zoned Bit Recording - ZBR* outer disk cylinders with a higher perimeter hold more sectors than inner cylinders, so that primary data will occupy fewer cylinders than secondary data, which implies shorter seeks in accessing randomly placed primary blocks.
2. Primary (resp. secondary) data can be allocated at upper (resp. lower) tracks of a disk cylinder but this is restricted to disks with an even number of tracks per cylinder, since based on disk capacity requirements both surfaces of a platter need not be used.

RAID-X proposed in (Hwang et al., 2002) is shown in Fig. 4.3. Data strips on primary disks are placed diagonally in secondary areas.

D_1	D_2	D_3	D_4
B_0	B_1	B_2	B_3
B_4	B_5	B_6	B_7
B_8	B_9	B_{10}	B_{11}
M_9	M_6	M_3	M_0
M_{10}	M_7	M_4	M_1
M_{11}	M_8	M_5	M_2

FIGURE 4.3 RAID-x organization with $M = 4$ disks in one cluster.

4.2.3 Interleaved Declustering - ID

The ID organization N disks into c clusters with $n = N/c$ disks per cluster as shown in Fig. 4.4. Half of the capacity of each disk is dedicated to primary data and the other half to secondary data. The primary data on each disk is partitioned into $n - 1$ chunks of equal size, which are stored at the other $n - 1$ disks of the cluster.

Cluster 1				Cluster 2			
D_1	D_2	D_3	D_4	D_5	D_6	D_7	D_8
A	B	C	D	E	F	G	H
b_3	a_1	a_2	a_3	f_3	e_1	e_2	e_3
c_2	c_3	b_1	b_2	g_2	g_3	f_1	f_2
d_1	d_2	d_3	c_1	h_1	h_2	h_3	g_1

FIGURE 4.4 Interleaved Declustering with $N = 8$ disks, $c = 2$ clusters, and $n = 4$ disks per cluster. Capital letters denote primary data and small letters subsets of secondary data.

At most one disk per cluster can fail and with c clusters the maximum number of disk failures that can be tolerated is $I = c$, while up to one half of disks can fail for the other RAID1 organizations discussed in this chapter. The advantage of ID over BM is that the read load of a failed primary data is distributed over $n - 1$ disks.

The *Striped Mirroring Disk Array (SMDA)*, which is described in (Jin and Hwang, 2000) has a layout similar to ID.

4.2.4 Chained Declustering - CD

The CD organization was proposed in (Hsiao and DeWitt, 1990) in conjunction with the *Gamma Data Base Machine - DBM* (DeWitt et al., 1990), which is discussed in Chapter 8. Given N disks CD divides disk space equally between primary and secondary data, so that primary data on disk D_i is replicated as secondary data on disk $D_{i+1 \bmod(N)}$ as shown in Fig. 4.5. CD can tolerate up to M disk failures, as long as there are no consecutive disk failures. Data can be accessed from its primary or secondary areas, similarly to ID and GRD organizations.

D_1	D_2	D_3	D_4	D_5	D_6	D_7	D_8
$\frac{1}{2}A$	$\frac{1}{2}B$	$\frac{1}{2}C$	$\frac{1}{2}D$	$\frac{1}{2}E$	$\frac{1}{2}F$	$\frac{1}{2}G$	$\frac{1}{2}H$
$\frac{1}{2}h$	$\frac{1}{2}a$	$\frac{1}{2}b$	$\frac{1}{2}c$	$\frac{1}{2}d$	$\frac{1}{2}e$	$\frac{1}{2}f$	$\frac{1}{2}g$

FIGURE 4.5 Chained declustering with $N = 8$ disks. Primary (resp. secondary) blocks are in capital (resp. small) letters. The read load is evenly distributed between the primary and secondary copies.

CD can be extended to spread secondary data over the next n disks, which will result in a more balanced load, but is less reliable than the original CD, since data loss will occur if the two failed disks are within n disks of each other, hence the probability of data loss is higher.

As shown in Fig. 4.6 in the case of disk failures fractional routing of read requests is required to attain a balanced read load. Only load balancing with a single disk failure is considered in (Hsiao and DeWitt, 1990). The effect of more than one

D_1	D_2	D_3	D_4	D_5	D_6	D_7	D_8
$1 \times A$	$0 \times B$	$\frac{1}{7}C$	$\frac{2}{7}D$	$\frac{3}{7}E$	$\frac{4}{7}F$	$\frac{5}{7}G$	$\frac{6}{7}H$
$\frac{1}{7}h$	$0 \times a$	$1 \times b$	$\frac{6}{7}c$	$\frac{5}{7}d$	$\frac{4}{7}e$	$\frac{3}{7}f$	$\frac{2}{7}g$
$1 \times A$	$0 \times B$	$1 \times C$	$0 \times D$	$\frac{1}{5}E$	$\frac{2}{5}F$	$\frac{3}{5}G$	$\frac{4}{5}H$
$\frac{1}{5}h$	$0 \times a$	b	$0 \times c$	$1 \times d$	$\frac{4}{5}e$	$\frac{3}{5}f$	$\frac{2}{5}g$
$1 \times A$	$0 \times B$	$\frac{1}{3}C$	$1 \times D$	$0 \times E$	$\frac{1}{4}F$	$\frac{1}{2}G$	$\frac{3}{4}H$
$\frac{1}{4}h$	$0 \times a$	$1 \times b$	$\frac{2}{3}c$	$0 \times d$	$1 \times e$	$\frac{3}{4}f$	$\frac{1}{2}g$
$1 \times A$	$0 \times B$	$\frac{1}{3} \times C$	$\frac{2}{3}D$	$1 \times E$	$0 \times F$	$\frac{1}{3}G$	$\frac{2}{3}H$
$\frac{1}{3}h$	$0 \times a$	$1 \times b$	$\frac{2}{3}c$	$\frac{1}{3}d$	$0 \times e$	$1 \times f$	$\frac{2}{3}g$

FIGURE 4.6 Balancing loads in CD after D_2 , and then either D_4 , D_5 or D_6 fail.

disk failures on system performance was investigated in (Thomasian and Xu, 2008). Computation of routing probabilities amounts to solving a set of linear equations to balance disk loads are outlined below.

Routing is used to balance disk loads as specified above. With a single disk failure the read load per disk increases to $8/7$. With two disks failures the load increase depends on the distance of the failed disks, If disks are one apart: D_2 and D_4 , then maximum disk loads is $6/5 = 1.2$. If D_2 and D_5 fail, the load increase is $5/4 = 1.25$. When disks are farthest apart: D_2 and D_6 , disk loads are $4/3 = 1.33$.

We use the failure of D_2 and D_5 in specifying the steps required to determine the fractions of access to primary and secondary data to balance disk loads. Given that a on D_2 is unavailable, $f_A = 1$, but f_h on D_1 is yet to be determined. On D_8 $f_H = 1 - f_h$ and f_g is yet to be determined. On D_7 $F_G = 1 - f_g$ and f_f is yet to be determined. On D_6 $f_F = 1 - f_f$ and $f_e = 1$, since data block E on D_5 is not available. Equating the loads on D_1 , D_8 , D_7 , D_6 we have the following three equations to determine three unknowns: $1 + f_h = 1 - f_h + f_g = 1 - f_g + f_f = 1 - f_f + 1$. It follows $f_g = 2f_h$, $2f_g = f_f + f_h$, $f_g = 2f_f - 1$, which yields: $f_h = 1/4$, $f_g = 1/2$, and $f_f = 3/4$. Disk load increases from one to $5/4$.

The Petal disk array utilizes striping and the CD organization as shown in Figure 5 in (Lee and Thekkath, 1996), where alternate rows are dedicated to primary and secondary data, respectively.

4.2.5 Dual striping

Dual striping in mirrored disks which combines large and small stripe sizes to improve performance is proposed in (Merchant and Yu, 1995). Database queries require table scans and OLTP workloads accessing small data blocks, access data from the large and small chunks, respectively.

Small stripes reduce data access skew for OLTP applications, while fewer seeks are required when table scans are processed over large stripes. The ID organization also utilizes two strip sizes, where secondary strips are $c - 1$ times smaller than primary stripes, where c is the number of disks in a cluster. It is not clear whether the finer striping granularity in Fig. 4.7 will result in a more balanced load for realistic workloads.

R0	R1	R2	...	R9
	r0.0	r0.1	...	r0.8
	r0.9	r0.10	...	r0.17
	⋮	⋮	...	⋮
r1.0		r1.1	...	r1.8
r1.9		r1.10	...	r1.17
⋮		⋮	...	⋮

FIGURE 4.7 Dual striping goes further than ID organization by splitting secondary data blocks further.

The analysis is based on the *Permanent Customer Model - PCM* in (Boxma and Cohen, 1991). Requests to small disk blocks are from a Poisson arrival process and queries corresponding to permanent customers access large blocks at each disk (one query per disk). There is an infinite backlog of queries and a completed query is replaced by a new query immediately.

4.2.6 Logical volume and automatic storage management and GPFS

Logical Volume Management - LVM is a feature provided by some operating systems, which in addition to RAID0 supports RAID1/0 and RAID5.

https://en.wikipedia.org/wiki/Logical_volume_management

There are multiple implementations of LVM by various companies.

Automatic Storage Management - ASM is a feature that provides striping and replication at the database level by Oracle.

https://en.wikipedia.org/wiki/Automatic_Storage_Management

The *IBM Global Parallel File Systems - GPFS* project, which led to Spectrum Scale in 2015 had its origins in the Tiger Shark file system by Roger Haskin and Frank Schmuck at IBM ARC as an alternative or a supplement to RAID (Haskin, 1998), (May, 2001), (Schmuck and Haskin, 2002).

https://en.wikipedia.org/wiki/IBM_Spectrum_Scale

GPFS supports replication, which is implemented at the file system level.

4.2.7 LSI Logic RAID

Alden Wilner at LSI Logic patented the LSI Logic RAID, which provides protection against disk failures by storing the XOR of pairs of *Data Disks* - *Ddisks* on a *Parity disk* - *Pdisk* (Wilner, 2001).

The *Data out-Degree* - *DoutD* is the number of parities associated with each data element and the *Parity in-Degree* - *PinD* is the number of data elements *eXclusive OR* - *XOR*ed to compute parities, so that $DoutD = PinD = 2$ in this case. With $N = 2M = 8$ disks there are four *Ddisks* and four *Pdisks* denoted by *D* and *P*, respectively, so that $P_{i,i+1 \bmod M} = D_i \oplus D_{(i+1) \bmod M}$, $1 \leq i \leq 4$, as shown below:

$$(D_1, P_{1,2}, D_2, P_{2,3}, D_3, P_{3,4}, D_4, P_{4,1}).$$

LSI Logic RAID can tolerate all two disk failures and one half of three consecutive disk failures when the middle disk is a *Pdisk*, but not otherwise. If the three disks $(D_1, P_{1,2}, D_2)$ fail then recovery can proceed as follows: $D_1 = D_4 \oplus P_{4,1}$, $D_2 = D_3 \oplus P_{2,3}$, and $P_{1,2} = D_1 \oplus D_2$.

An OLTP workload with read and write accesses to small data blocks will result in an unbalanced disk load if all updates are uniform over data blocks are carried out as *Read-Modify-Wrote* - *RMW* accesses. This load can be balanced by using a combination of *RMW* and *ReConstruct Write* - *RCW* accesses, which read remaining blocks in the parity group to compute the parity (Thomasian, 2005a). The performance of LSI RAID with *RMW* versus *RCW* was investigated in (Thomasian and Tang, 2012).

Alternatively, disk loads can also be balanced as shown below by shifting the strips right from row-to-row, where *D* and *P* represent strips in the first row, and $\bar{D}$ and $\bar{P}$ represent the shifted strips in the second row. The disadvantage of this layout is that it does not tolerate consecutive three disk failures.

$$(D_1, P_{1,2}, D_2, P_{2,3}, D_3, P_{3,4}, D_4, P_{4,1}).$$

$$(\bar{P}_{4,1}, \bar{D}_1, \bar{P}_{1,2}, \bar{D}_2, \bar{P}_{2,3}, \bar{D}_3, \bar{P}_{3,4}, \bar{D}_4).$$

4.2.8 Adaptive disk arrays

The *Self-Adaptive Disk Array* - *SADA* was proposed in (Pâris et al., 2006). The starting point is a RAID0/1 array with four disk pairs: $A_1, A_2, B_1, B_2, C_1, C_2, D_1, D_2$, which hold copies of datasets *A*, *B*, *C*, and *D*. If B_1 fails extra protection is provided for *B* by setting $A_1 = A \oplus B$. If D_1 fails the system sets $C_2 = C \oplus D$. If D_2 fails the system sets $A_1 = (A \oplus B) \oplus (C \oplus D)$ and $C_2 = D$. In effect this is a RAID5 array with one parity disk. Finally, if B_2 fails the system XORs *A* from A_2 , *C* from C_1 , and *D* from C_2 with A_1 so that $A_1 = B$ and we have an unprotected array.

4.2.9 SSPiRAL (Survivable Storage using Parity in Redundant Array Layout)

SSPiRAL was proposed by Thomas Schwarz (Marquette U.), Jehan Francois Paris (U. Houston), Darrell Long (UCSC), and Ahmed Amer (U. Santa Clara) in (Schwarz et al., 2008).

LSI Logic RAID is SSPiRAL(4+4,2) with four *Ddisks* and four *Pdisks* and $DoutD = PinD = 2$, as shown in Fig. 4.8. SSPiRAL extends LSI RAID to $DoutD = PinD = 3$, so that each *Ddisk* is protected by three *Pdisks* and vice-versa. SSPiRAL(4+4,3) with $N = 2M = 8$ disks, four *Ddisks* and four *Pdisks* is shown in Fig. 4.9.

D_1	D_2	D_3	D_4	D_5	D_6	D_7	D_8
d_1	$p_{1,2}$	d_2	$p_{2,3}$	d_3	$p_{3,4}$	d_4	$p_{4,1}$

FIGURE 4.8 SSPiRAL(4+4,2) with four *Ddisks* and four *Pdisks* with $PinD = DoutD = 2$.

D_1	D_2	D_3	D_4	D_5	D_6	D_7	D_8
d_1	d_2	d_3	d_4	$d_1 \oplus d_2 \oplus d_3$	$d_2 \oplus d_3 \oplus d_4$	$d_3 \oplus d_4 \oplus d_1$	$d_1 \oplus d_2 \oplus d_3$

FIGURE 4.9 SSPiRAL(4+4,3) with $PinD = DoutD = 3$.

For SSPiRAL(4+4,3) all disk failures up to three can be tolerated. There are $\binom{8}{4} = 70$ possibilities for four disk failures and data loss occurs in 14 cases (Schwarz et al., 2008), as follows:

- (1) A *Ddisk* and the 3 *Pdisks* in which it participates (4 cases).
- (2) Two out of four *Ddisks* and the two *Pdisks* in which they both participate (6 cases).
- (3) Three out of four *Ddisks* and the *Pdisk* in which all three participate (4 cases). If all four *Ddisks* fail they can be reconstructed using *Pdisks*, e.g., $\{d_1 \oplus d_2\} = \{d_3 \oplus d_4 \oplus d_1\} \oplus \{d_2 \oplus d_3 \oplus d_4\}$ and $d_4 = \{d_1 \oplus d_2\} \oplus \{d_4 \oplus d_1 \oplus d_2\}$.

4.2.10 B-code

B-code and its dual proposed by Lihao Xu et al. at CalTech stores parities associated with data in columns (Xu et al., 1999). It was proposed by

RM2 proposed in (Park, 1995) and X-code proposed in (Xu and Bruck, 1999) are two *2-Disk-Failure Tolerant - 2DFT* arrays with horizontal parities, where the former is not *Maximum Distance Separable - MDS* and the latter is (Thomasian and Blaum, 2009). B-codes like Weaver codes, discussed below, store data on the same device (Hafner, 2005b). In the case of $n = 3$ bits per column and $\ell = 2n = 6$ columns, we have $\hat{B}_6$, which is a dual of B_6 code, as shown below based on Fig. 1 in (Xu et al., 1999):

$$\begin{pmatrix} a_1 & a_2 & a_3 & a_4 & a_5 & a_6 \\ a_2 + a_3 & a_3 + a_4 & a_4 + a_5 & a_5 + a_6 & a_6 + a_1 & a_1 + a_2 \\ a_4 + a_6 & a_5 + a_1 & a_6 + a_2 & a_1 + a_3 & a_2 + a_4 & a_3 + a_5 \end{pmatrix}$$

When the four columns 3-6 are erased they can be reconstructed in the following manner. $a_3 = (a_2 + a_3) \oplus a_2$, $a_4 = (a_3 + a_4) \oplus a_3$, $a_5 = (a_5 + a_1) \oplus a_1$, $a_6 = (a_4 + a_6) \oplus a_4$.

Properties of B-codes and their duals are listed in (Xu et al., 1999). This code is MDS since by placing the 6 data bits in first two columns we have four parity columns and so four column failures can be tolerated.

4.2.11 Weaver codes

Weaver codes proposed in (Hafner, 2005b) have three design features:

- (a) data and parity blocks on the same strip,
- (b) constrained parity in-degree,
- (c) balance and symmetry.

There are n strips per stripe and each strip holds r data elements and q parity elements. For $\text{Weaver}(n, k, t)$ parity elements have an in-degree k and data elements have an out-degree t . Denoting the *greatest common divisor - gcd* of k and t with $m = \gcd(k, t)$, $r = k/m$ and $q = t/m$ are minimal. The efficiency of the code is $e = r/(r + q) = k/(k + t)$. Since $k \leq t$ the maximum efficiency of this code is 50%. $\text{Weaver}(n, t, t)$ codes with 50% efficiency are a special category of interest in this study.

Table 1 in the paper is a partial listing of *Parity Defining Sets - PDS* for $\text{WEAVER}(n, t, t)$ codes, where the PDS is $\kappa(j) = \kappa_1(j) + s \bmod(n)$, e.g., the second row for $t = 3$ in the table: $\kappa_1(0) = (1, 2, 4)$ and $s = 2$, so that the parities corresponding to d_j are at $(j + 3, j + 4, j + 6) \bmod(n)$. LSI Logic RAID is $\text{Weaver}(n, 2, 2)$ and SSPiRAL is $\text{Weaver}(n, 3, 3)$, but unlike Weaver codes they place parities on separate devices.

D_1	D_2	D_3	D_4
d_1	d_2	d_3	d_4
$d_2 \oplus d_3$	$d_3 \oplus d_4$	$d_4 \oplus d_1$	$d_1 \oplus d_2$

FIGURE 4.10 Data layout for a Weaver-like (4,2,2) with $N = 4$ disks.

$\text{Weaver}(4, 2, 2)$ shown in Fig. 4.10. can tolerate two disk failures, e.g., if disks D_1 and D_2 fail then $d_1 = d_4 \oplus (d_4 \oplus d_1)$, $d_2 = d_1 \oplus (d_1 \oplus d_2)$, but unlike LSI Logic RAID three disk failures cannot be tolerated.

$\text{Weaver}(8, 3, 3)$ with $N = 8$ disks with $\text{PoutD} = \text{PinD} = 3$ is shown in Fig. 4.11.

D_1	D_2	D_3	D_4	D_5	D_6	D_7	D_8
d_1	d_2	d_3	d_4	d_5	d_6	d_7	d_8
$d_3 \oplus d_4 \oplus d_6$	$d_4 \oplus d_5 \oplus d_7$	$d_5 \oplus d_6 \oplus d_8$	$d_6 \oplus d_7 \oplus d_1$	$d_7 \oplus d_8 \oplus d_2$	$d_8 \oplus d_1 \oplus d_3$	$d_1 \oplus d_2 \oplus d_4$	$d_2 \oplus d_3 \oplus d_5$

FIGURE 4.11 $\text{Weaver}(8, 3, 3)$ data layout with $N = 8$ disks and $t = 3$.

Since a data block and associated parities for $\text{Weaver}(8, 3, 3)$ appear at four different disks all three disk failures can be tolerated. Consider the failure of D_1, D_4, D_6 , so that d_1 's parity is only available at D_7 . To recover d_1 we need to recover d_4 , since d_2 is available. We have the following steps $(d_4 \oplus d_5 \oplus d_7) \oplus d_5 \oplus d_7 \rightarrow d_4$ and $(d_7 \oplus d_8 \oplus d_2) \oplus d_7 \oplus d_8 \rightarrow d_2$. Finally, $d_2 \oplus d_4 \oplus (d_1 \oplus d_2 \oplus d_4) \rightarrow d_1$.

$\text{Weaver}(8, 3, 3)$ can tolerate four disk failures, as long as they do not involve a data strip and its parities. Recovery is possible when the first four disks in Fig. 4.11 fail, as follows:

$$\begin{aligned}
d_7 \oplus d_8 \oplus (d_7 \oplus d_8 \oplus d_2) &\rightarrow d_2, \\
d_2 \oplus d_5 \oplus (d_2 \oplus d_3 \oplus d_5) &\rightarrow d_3, \\
d_8 \oplus d_3 \oplus (d_8 \oplus d_1 \oplus d_3) &\rightarrow d_1, \\
d_1 \oplus d_2 \oplus (d_1 \oplus d_2 \oplus d_4) &\rightarrow d_4.
\end{aligned}$$

Note that for $N = 8$ disks and four disk failures there are $\binom{8}{4} = 70$ possibilities.

Weaver codes are also presented by James Plank in a FAST'05 tutorial (Plank, 2005).

<http://web.eecs.utk.edu/~jplank/plank/papers/FAST-2005.html>

4.2.12 Robust, Efficient, Scalable, Autonomous, Reliable - RESAR

Thomas Schwarz et al. proposed exabyte scale storage RESAR, which places disklets into two different parity groups - with one parity disklet in each stripe (Schwarz et al., 2016). In fact RESAR is RAID5/1, i.e., a mirrored system where each side is a RAID5 with M disks (Schwarz et al., 2016). When a disk fails its mirror is accessed first and if this fails data is reconstructed using the RAID5 paradigm.

“A RESAR-based layout with 16 data disklets per stripe has about 50 times lower probability of suffering data loss in the presence of a fixed number of failures than a corresponding RAID6 organization”. The simulation to estimate the probability of data loss for 100,000 disks required 10,000 hours or 417 days.

4.2.13 Multiway placement

Three-way versions of chained, group rotate, and standard mirroring is proposed in (Shang et al., 2011). The *Shifted Declustering - SD* provides optimal parallelism, since the data replicas are distributed evenly. The SD layout has a similarity to (Alvarez et al., 1998), which is discussed in Chapter 5.

4.2.14 Classification of mirrored and hybrid disk arrays

In BM and GRD data is replicated on independent disks, while ID and CD configurations share disk space between primary and secondary data.

Data on mirrored disks may be striped to balance disk loads. The strips usually have the same size, but different strip sizes have also been considered as in the case of interleaved declustering and dual striping.

The effect of disk failures on disk array throughput is quantified in Section 4.8.5. In what follows we consider the effect of disk failures on read requests, whose processing has the most effect on throughput degradation.

Basic Mirroring: BM. The read load on the mirroring disk is doubled and the throughput for the disk pair is halved for read requests.

Group Rotate Declustering: GRD. For $k \leq M = N/2$ failed disks one side the decrease in throughput is $2(M - k)/(2M)$. Routing for load balancing is required, similarly to RAID1/0.

Interleaved Declustering: ID. With a single disk failure per cluster, the throughput for read requests is reduced by $(n - 1)/n$ or conversely the load on surviving disks in a cluster is increased by $n/(n - 1)$. Request routing for load balancing is not required.

Chained Declustering: CD. The decrease in read throughput is $(N - k)/N$, where $k \leq N/2$, as long as the k failed disks are not adjacent. Routing for load balancing is required.

4.3 Routing read requests in mirrored disks

We summarize routing strategies in mirrored disks with BM as proposed in (Towsley et al., 1990).

1. Single Queue - SQ: Update requests can be processed only when both disks are idle, while read requests can be processed at both disks or only the primary disk.

1.1 Primary/Secondary - PSSQ: Disks are designated as primary and secondary.

(i) **Serial PSSQ - S-PSSQ** allows only one request to be processed at a time.

(ii) **Concurrent PSSQ - C-PSSQ** allows concurrency between read and update requests.

A read request can start service at the primary disk.

1.2 Equitable SQ - ESQ: Disks are treated equally.

Concurrent Read ESQ - CR-ESQ: Allows reads to be processed concurrently. Only one update is allowed to be processed at any one time.

Concurrent Read Update - CRU-ESQ: Allows concurrency between reads and updates, but an update is still required to wait until both disks are available. A read request may proceed as soon as a disk becomes available.

Minimum Read - MR-ESQ: Similarly to S-PSSQ each request waits until the previous request is completed. Read requests are processed at both disks, but when the first read request completes, abort the other request. It should be noted that disk accesses cannot be preempted at all stages, especially during seeks.

2. Multiple Queue Policies:

Distributed MQ - DMQ: Maintain a separate queue for each disk. An update request generates write requests for each queue. Read requests are routed randomly to balance disk loads.

Shortest Queue - DMQ - SQ-DMQ: Route read requests to the shortest queue. Reads can also spawn requests at both queues. There are two variations of **Minimum Read - MR** with and without preemption. Abort is initiated when a read request begins service or is completed.

Common MQ - CMQ: The request at the head of the queue starts processing as soon as either disk is free. Update requests are spawned at both disks and are enqueued at the busy disks.

RAID1 performance is compared with RAID5 using an M/G/1 queueing model in (Chen and Towsley, 1996). Three RAID1 configurations considered with FCFS disk scheduling: BM, CD, and GRD, where GRD is shown to provide the best performance for small and large I/O environments. Three routing policies are considered for RAID1: *Random Join - RJ*, *Shortest Queue - SQ*, and *Minimum Seek - MS*. Local disk scheduling policies can additionally be applied at each disk.

Routing policies described in (Towsley et al., 1990) are analyzed in (Towsley and Chen, 1991) using the *Matrix Geometric Method - MGM* developed by Marcel Neuts (rip) in (Neuts, 1981). The processing of read and write requests in mirrored disks has similarities to their scheduling in replicated databases, which is summarized in Table 1 provided by Randolph Nelson (ox.com) and Bala Iyer (rip) at IBM Research in (Nelson and Iyer, 1985), which is repeated in (Thomasian, 2014b).

Routing of disk accesses in RAID1 is classified according to queue organization in (Thomasian, 2006b):

Private Queue - PQ with Immediate Routing: Based on a routing criterion read requests are sent to one of the disks, while write requests are sent to both disks.

Shared Queue - SQ with Deferred Routing: Read requests held at SQ are routed to the first disk that becomes available.

Hybrid Queue - HQ: Read requests held in SQ are routed to PQs after some delay to ensure that it will be included in disk scheduling, e.g., by SATF.

Two examples of static routing to private queues are: (1) uniform routing (with equal probabilities), (2) round-robin routing. It is shown (Thomasian, 2006b) and Section 4.8 that with Poisson arrivals and exponential service times that round-robin routing yields a lower mean response time with respect to uniform probabilistic routing.

Affinity Based Routing - ABR to reduce disk access time is investigated by Chunqi Han (HGST, Data Domain, Nimble Storage, WeRide.ai) and Alex Thomasian in Thomasian and Han (2005a). ABR divides disks to inner and outer disk blocks, which are delineated by an *Logical Block Address - LBA* referred to as the pivot point. Requests with LBAs lower than the pivot point are sent to outer tracks and vice-versa. With $2s + 1$ disk sectors and pivot point at s in zoned disks this would entail outer data blocks occupying fewer cylinders than inner data blocks and hence shorter seek times and access times for outer data (Jacob et al., 2008). Several criteria have been used in (Thomasian and Han, 2005a) to determine the pivot point. Numerical results show that pivot point selected based on balanced disk utilizations achieves about the same performance as the one based on minimum mean response time. This can be justified by the *Pollaczek-Khintchine - PK* formula given by Eq. (4.3). The transposed mirror organization, which switches the data on inner and outer tracks achieves the best performance at heavy loads.

Join the Shortest Queue - JSQ policy is optimal when the service rates are fixed or non-decreasing according to (Whitt, 1986). JSQ may not be effective for disk scheduling, since the disk queue length is not a good measure of outstanding work. Given the initial LBA and number of blocks to be accessed by each enqueued request the response time of the requests to be routed with SATF scheduling can be calculated given disk drive characteristics. It is best to select the disk that reduces the mean disk service time over all pending requests, which is tantamount to minimizing the completion time of all requests (Thomasian, 2006b).

Given the LBA accurate timing emulators can be used to estimate disk service time with respect to its current position as reported in Lumb et al. (2000a).

For random disk accesses, performance is mainly determined by the local disk scheduling policy rather than the routing policy to the PQ - Private Queue. SATF scheduling with SQ - Shared Queue applied to $n > 1$ replicated disks provides more opportunities than PQ in improving performance, since more requests are available for SATF scheduling (Thomasian, 2006b). There are no controllers for mirrored disks, which could serve requests from a shared queue.

4.4 Shortening the tail for response times

According to this study at Google (Dean and Barroso, 2013) variability of response time arise for different reasons:

Shared resources by different or with the same applications for CPU cores, processor caches, memory and network bandwidth.

Daemons ordinarily use only limited resources on average, but when scheduled generate hiccups;

Global resource sharing applications running on different machines contending for resources such as network switches and shared file systems.

Maintenance activities exemplified by data reconstruction in distributed file systems, periodic log compactions in storage systems, and periodic garbage collection in languages can cause periodic spikes in latency;

Queueing in servers and network switches affects the variability.

Hardware trends affecting variability are as follows:

Power limits for modern CPUs allow them to temporarily run above their average power envelope, but throttling if this activity is sustained for a long period;

Garbage collection in flash SSDs can increase read latency by a factor of hundred with even a modest level of write activity.

Energy management: Powersaving modes in many types of devices save considerable energy, but add additional latency when moving from inactive to active modes.

A common technique for reducing latency in large scale online services is to parallelize sub-operations across many different machines, where each sub-operation is colocated with its data. Parallelization happens by fanning out a request from a root server to a large number of leaf servers and merging responses via a request-distribution tree. These sub-operations must all complete within a strict deadline for the service to feel responsive. Variability in the latency distribution of individual components is magnified at the service level; for example, consider a system where each server typically responds in 10 ms but with a 99th percentile latency of one second. If a user request is handled on just one such server, one user request in 100 will be slow.

Measurements from a real Google service that is logically similar to this idealized scenario; root servers distribute a request through intermediate servers to a very large number of leaf servers. Table 4.1 shows the effect of large fanout on latency distributions. Techniques that concentrate on slow outliers can yield dramatic reductions in overall service performance.

TABLE 4.1 Individual leaf request finishing times for a large fanout service tree (measured from root node of the tree).

Percentiles	50	95	99
One random leaf finishes	1ms	5ms	10ms
95% of all leaf requests finish	12ms	32ms	70ms
100% of all leaf requests finish	40ms	87ms	140ms

Hedged requests. A simple way to curb latency variability is to issue the same request to multiple replicas and use the results from whichever replica responds first. In the case of “hedged requests” a client first sends one request to the most appropriate replica, but then sends a secondary request after some brief delay. The client cancels remaining outstanding requests once the first result is received. Load can be increased modestly by deferring sending a secondary request until the first request has been outstanding for more than the 95th percentile expected latency for this class of requests, while substantially shortening the latency tail. A Google benchmark that reads the values for 1,000 keys stored in a BigTable table distributed across 100 different servers, sending a hedging request after a 10 ms delay reduces the 99.9th percentile latency for retrieving all 1,000 values from 1,800 ms to 74 ms, while sending just 2% more requests. The overhead of hedged requests can be further reduced by tagging them as lower priority than the primary requests.

Tied requests. The hedged-requests technique also has a window of vulnerability in which multiple servers can execute the same request unnecessarily. That extra work can be capped by waiting for the 95th percentile expected latency before issuing the hedged request, but this approach limits the benefits to only a small fraction of requests. Permitting more aggressive use of hedged requests with moderate resource consumption requires faster cancellation of requests. A common source of variability is queueing delays on the server before a request begins execution. For many services, once a request is actually scheduled and begins execution, the variability of its completion time goes down substantially.

Allowing a client to choose between two servers based on queue lengths at enqueue time exponentially improves load-balancing performance over a uniform random scheme according to (Mitzenmacher, 2001), but advocated in this work is not choosing but rather enqueueing copies of a request in multiple servers simultaneously. The servers are allowed to communicate updates on the status of these copies to each other. We call requests where servers perform cross-server status updates “tied requests.” The simplest form of a tied request has the client send the request to two different servers, each tagged with the identity of the other server (“tied”). When a request begins execution, it sends a cancellation message to its counterpart. The corresponding request, if still enqueued at the other server is aborted immediately.

Google’s implementation of this technique in the context of its cluster-level distributed file system is effective at reducing both median and tail latencies. Table 4.2 lists the times for servicing a small read request from a BigTable where the data is not cached in memory but must be read from the underlying file system; each file chunk has three replicas on distinct machines. The table includes read latencies observed with and without tied requests for two scenarios: The first is a cluster in which the benchmark is running in isolation, in which case latency variability is mostly from self-interference and regular cluster-management activities. Sending a tied request that does cross-server cancellation to another file system replica following 1 ms reduces median latency by 16% and is increasingly effective along the tail of the latency distribution, achieving nearly 40% reduction at the 99.9th-percentile latency. In the second scenario there is also a large, concurrent sorting job running on the same cluster con-tending for the same disk resources in the shared file system. Although overall latencies are somewhat higher due to higher utilization, similar reductions in the latency profile are achieved with the tied-request technique discussed earlier. The latency profile with tied requests while running a concurrent large sorting job is nearly identical to the latency profile of a mostly idle cluster without tied requests. Tied requests allow the workloads to be consolidated into a single cluster, resulting in dramatic computing cost reductions. In both Table 4.2 scenarios, the overhead of tied requests in disk utilization is less than 1%, indicating the cancellation strategy is effective at eliminating redundant reads.

TABLE 4.2 Read latencies in a BigTable service benchmark.

	Mostly idle cluster		With concurrent terasort	
	no hedge	tied request after 1 ms	no hedge	tied request after 1 ms
50%ile	19 ms	16 ms (−16%)	24 ms	19 ms (−21%)
90%ile	38 ms	29 ms (−24%)	56 ms	38 ms (−32%)
99%ile	67 ms	42 ms (−37%)	108 ms	67 ms (−38%)
99.9%ile	98 ms	61 ms (−38%)	159 ms	108 ms (−32%)

An alternative to the tied-request and hedged-request schemes is to probe remote queues first, then submit the request to the least-loaded server. It can be beneficial but is less effective than submitting work to two queues simultaneously for three reasons:

- (1) load levels can change between probe and request time;
- (2) request service times can be difficult to estimate due to underlying system and hardware variability;
- (3) clients can create temporary hot spots by all clients picking the same least-loaded server at the same time.

The *Distributed Shortest Positioning Time First - D-SPTF* protocol proposed at CMU/PDL dynamically distribute requests in decentralized storage servers (Lumb and Golding, 2004). For 10-200 microsecond network latencies D-SPTF performs as well as a centralized system. D-SPTF achieves up to 65% higher throughput than popular decentralized approaches and may adapt to heterogeneous server capabilities.

DSPTF sends a request to one server and the request is forwarded to replicas only if the initial server does not have it in its cache and uses cross-server cancellations. In more complex coding schemes where a primary request is sent to the machine with the desired data block and if no response is received after a brief delay a collection of requests is issued to a subset of the remaining replication group sufficient to reconstruct the desired data, with the whole ensemble forming a set of tied requests. Note, too, the class of techniques described here is effective only when the phenomena that causes variability does not tend to simultaneously affect multiple request replicas, such uncorrelated phenomena are expected to be rather common.

Presented in (Joshi et al., 2017) is a study that helps in understanding the efficiency of redundancy in reducing latency. Different redundancy strategies in terms of the number of redundant tasks and the time when they are issued and canceled are compared for different service time distributions. The log-concavity of the task service time creates a dichotomy of when adding redundancy helps. If the service time distribution is log-convex (i.e., log of the tail probability is convex), then adding maximum redundancy reduces both latency and cost. And if it is log-concave (i.e., log of the tail probability

is concave), then less redundancy, and early cancellation of redundant tasks is more effective. Based on these insights a general redundancy strategy that achieves a good latency-cost trade-off for an arbitrary service time distribution. This work also generalizes and extends some results in the analysis of fork-join queues.

4.5 Improving write performance in mirrored disks

Write anywhere policy on either or both mirrored disks has been used to minimize disk arm movement and to reduce the susceptibility to data loss in systems without an NVRAM (Solworth and Orji, 1991, 1993). A directory keeps track of blocks written anywhere on one disk and to allow efficient sequential access data is updated later on the primary disk.

A two-phase method for scheduling mirrored disks is proposed in (Polyzois et al., 1993). The two disks alternate in processing read and write requests. While one disk processes read requests, the other disk destages dirty blocks held in *NonVolatile Storage* - NVS. Destaging in batch mode uses CSCAN (SCAN in one direction) scheduling. It is unlikely that the two phases will have equal durations and there will be a perfect overlap in read and write processing. Several refinements to this method were therefore addressed in (Thomasian and Liu, 2005b), which similarly to (Polyzois et al., 1993) was conducted using simulations with synthetic workloads. Refinements introduced in this work with respect to the 2-phase method were as follows.

- (1) Eliminate idleness by processing write requests, instead of just relying on freeblock scheduling, i.e., doing useful work during disk rotations (Lumb et al., 2000a).
- (2) Given that rotational latency can take considerable time, and SATG outperforms SCAN and CSCAN, use SATF instead of CSCAN for destaging. Destage time can be minimized by considering permutations of outstanding requests when their number is not very large.
- (3) Defer destaging dirty blocks if the number of enqueued read requests exceeds a threshold. This provides more opportunities for dirty blocks to be overwritten and more efficient destaging due to adjacent blocks.

A duplexed NVRAM, e.g., a DRAM protected by UPS - Uninterruptible Power Supply is as reliable as magnetic disks according to a study at IBM/ARC reported in (Menon and Cortney, 1993), which allows a fast write capability, i.e., a write is considered completed as soon as the data is written to the duplexed NVRAM.

The writing of modified data blocks to disk can be deferred, allowing reads to be processed at a higher priority than writes, which improves response time for OLTP txns. Overwriting of dirty blocks to NVRAM obviates unnecessary destages to disk. Deferred destaging in batches in LBA order reduces positioning time. Results of trace analysis at IBM/ARC reported in (Treiber and Menon, 1995) were used to quantify the number overwritten dirty blocks and the proximity of destaged requests (the number and distance of dirty blocks on the same track) in the performance analysis reported in (Thomasian and Menon, 1997). Destage is initiated when the buffer becomes full or when it is filling rapidly as suggested in (Varma and Jacobson, 1998).

An NVRAM improves performance by allowing deferred destaging of dirty blocks in OLTP systems, but is not necessary for recovery in systems with *Write Ahead Logging* - WAL of before and after images of modified data (Gray and Reuter, 1992), (Ramakrishnan and Gehrke, 2002). The before images are used in undoing the updates of aborted txns, while the after images allow the NoForce logging policy, i.e., a txn may commit without forcing dirty pages to disk.

4.6 Disks with multiple R/W heads on a single and multiple arms

Disks with two *Read/Write* - R/W heads on one arm, which are at a fixed track distance d from each other are analyzed in (Calderbank et al., 1984), (Manolopoulos and Kollias, 1989). The *Nearer-Server* - NS rule of such a system is analyzed in two cases in (Calderbank et al., 1984):

- (1) both heads have to be kept on the surface of the disk.
- (2) this restriction is relaxed.

It was shown that an optimization with respect to d yields an expected seek distance, which is slightly less than that of a system with two independent arms and a single controller which allows the movement of one arm at a time.

It is shown in (Manolopoulos and Kollias, 1989) that for a disk with C cylinders the optimal distance between the two heads is $d = C/2 - 1$, which reduces the seek distance from $C/3$ to $C/6$. A disk with a two heads at a fixed distance d is studied by simulation of an actual system in (Page and Wood, 1981).

Scheduling of a disk with two arms, each with a single R/W head is studied in (Hofri, 1983). The closest head in terms of number of cylinders is used to serve an incoming request, while the other head is moved to a better position in anticipation of the next request. If the accessed cylinder is at $a < C/2$ then the other head is moved somewhere between a and C and if $a > C/2$ the head is positioned between the first cylinder and a . More specifically it is shown in (Manolopoulos and Vakali,

1991) that in the first case the head should be positioned at $a + \lfloor (2(C - a)/3) \rfloor$ and otherwise at $\lfloor a/3 \rfloor$. The analysis yields a seek distance $5C/36$, which was given without proof in (Hofri, 1983).

The optimal placement for two-headed disk systems is the camel arrangement, i.e., two consecutive organ-pipes (Manolopoulos and Kollias, 1990). There are $2^{N/2+1}$ optimal camel arrangements for a disk with $N = 2(2n + 1)$ cylinders.

A discrete model of two headed disks is presented in (Carson et al., 1992).

Conner Corp. has a patent for multiple actuator Chinook disk (Squires et al., 1994).

https://en.wikipedia.org/wiki/Conner_Peripherals/#Performance_issues_and_the_Chinook_dualactuator_drive

A dual actuator logging disk architecture with one head dedicated to reading and the other to logging in regions with free disk sectors is proposed in (Chandy, 2007). Seagate has more recently introduced multiactuator disk technology, but each arm accesses half of the tracks.

<https://www.seagate.com/our-story/introducing-mach2/>

Improving disk performance via latency reduction and *Rotational Position Sensing* - RPS miss delay is addressed in (Ng, 1991). Three methods are proposed to improve latency, which are also applicable to non-RPS disks:

Synchronized 180° apart mirrors: Moving the disk arm seeking to the same cylinder on both disks. Since disks are synchronized half a rotation apart this results in a rotational latency reduction from $T_r/3$ to $T_r/4$, where T_r denotes disk rotation time.

Data duplicated 180° apart on one disk: Two copies of the same data are placed 180° apart on disk tracks.

Dual actuators on a single disk: The two actuators are placed opposite each other and move together to the same track to reduce rotational latency.

Given a fixed capacity disk array analytical models are developed in (Yu et al., 2000) to determine the combination of striping, mirroring, and rotational data replication, which yields the best performance, i.e., with reduced seek times and rotational delays, for a given workload and disk characteristics. The effectiveness of the configuration models are tried on a prototype.

A taxonomy of disk parallelism is provided in (Sankar et al., 2008). In **DASH**, **D**isk stack is the number of platters constituting a cylinder, **A**rm assembly is the number of arms, **S**urface is the number of surfaces, and **H**ead (number of R/W heads per arm). A conventional disk is then D1 A1 S1 H1. Air turbulence affects the vibration of platters and heads, which makes it not possible to transfer data simultaneously from multiple tracks. A cost benefit analysis of intradisk parallelism is reported in this paper, which also discusses power consumption.

4.7 Seek distances in single and mirrored disks

The analysis for the seek distance for reads with k -way replication is given in (Bitton and Gray, 1988). Gray's interest in mirrored disks was due to his affiliation with Tandem Corp. which utilized mirrored disks. Based on the fact that for a disk with C cylinders the *probability mass function* - pmf for the seek distance with uniform accesses to disk cylinders is given as:

$$P[X = i] = \begin{cases} \frac{2(C-i)}{C^2}, & 1 \leq i \leq C-1, \\ \frac{1}{C}, & i = 0. \end{cases}$$

It follows

$$P[X \geq i] = (1 - \frac{i}{C})^2$$

With a degree of data replication k the mean seek distance in tracks is:

$$P[\min(X_1, X_2, \dots, X_k) \geq i] = \prod_{j=1}^k P[X_j \geq i]$$

Using substitution $x = i/C$ the Reimann integral in a continuous domain yields:

https://en.wikipedia.org/wiki/Riemann_integral

$$E[X_R] = \frac{1}{C^{2k}} \sum_{i=1}^{C-1} (1 - \frac{i}{C})^{2k} \approx \int_0^1 (1-x)^{2k} dx = \frac{1}{2k+1}, \quad (4.1)$$

e.g., for $k = 2$ the seek distance is $C/5$.

An extension of the analysis for writes uses the maximum function to take into account that writes on both disks should be completed: To derive the expected seek distance for writes note that

$$P[\max(X_1, X_2, \dots, X_k) \leq i] = \prod_{j=1}^k P(X_j \leq i) = \left[\frac{2}{C^2} \sum_{j=1}^i (C - i)^k \right]^k = \frac{i^k}{C^{2k}} (2C - i - 1)^k$$

$$E[X_W] = \sum_{i=1}^{C-1} \left[1 - \frac{i^k}{C^{2k}} (2C - i - 1)^k \right] \approx C(1 - I_k), \quad I_k = \frac{2k}{2k+1} \frac{2k-2}{2k-1} \dots \frac{2}{3}, \quad (4.2)$$

e.g., for $k = 2$, $E[X_W] = \frac{7}{15}C = 0.46C$. The mean service time for writes is $E[X_W]$ plus half a disk rotation.

The cumulative distribution and density function for the minimum and maximum seek distances for disk systems with two independent arms are compared with one-arm systems using continuous functions (Chien, 1993).

The analysis in (Bitton and Gray, 1988) does not take into account the effect write synchronization, i.e., R/W heads being positioned on the same track after a write, so that there is no improvement with shortest seek distance routing. The read seek distance is underestimated and the write seek distance is overestimated. This effect is considered in (Lo and Matloff, 1992).

A more detailed analysis in (Vakali and Manolopoulos, 1997) by Athena Vakali with Yannis Manolopoulos at Aristotle U. Thessaloniki in (Vakali and Manolopoulos, 1997), which takes into account disk scheduling and the number of disk cylinders. It is shown that previous analyses are accurate for a large number of cylinders and a small number of disks, which is usually the case. The same researchers compare seek distances in mirrored disks without and with cylinder replication (multiple copies of a cylinder on a disk), under various placement policies with uniform and normal distribution of requests (Vakali and Manolopoulos, 2000). It is concluded that cylinder replication yields a significant improvement.

Seek optimization in RAID1 configurations is investigated in Bachmat and Lam (2005), where a distinction is made between online and offline algorithms. Online algorithms know the current position of the disk arm, while offline algorithms choose for each copy $0 \leq i \leq d - 1$ of each file $0 \leq j \leq L - 1$ a probability $p_{i,j}$ that a read request from file j will be serviced by the i^{th} copy of the file. Mirrored disks such as ID, GRD, and CD are referred to as semistructured in this study and shown to outperform BM as far as seek distances are concerned. This can be attributed to the fact that data at the two disks in BM are mirrored exactly, so that both disks introduce the same delay, but this is not true otherwise. A list of studies for seek optimization is also provided.

The seek distance problem is considered in (Calderbank et al., 1985), where two servers move along C positions on a straight line or in a circle. Requests for service from one of the C positions with uniform distribution are served from a FCFS queue one at a time. The processing of the n^{th} request is not started until the processing of the $n - 1^{st}$ request is completed. Processing the n^{th} request entails determining which server to move, moving it to the requested position, and then performing the service.

The *Nearer-Server - NS* rule is optimal for a circle, with an expected normalized server motion $5/36 \approx 0.13889$. The average distance over an interval (straight line) is 0.1625, while the optimal policy is shown to be 0.1598, so that NS is within $0.1625/0.1598 = 1.69\% < 2\%$ of the optimal policy. It is better than $C/6 = 0.1666C$ for the partition rule which dedicates two servers: one server dedicated to cylinders $(1, C/2)$ and another to $(C/2 + 1, C)$.

The *Anticipatory Arm Placement - AAP* in single and mirrored non-zoned disks is presented in (King, 1990). With the seek distance in the range $(0,1)$, the arm should be placed at the middle disk cylinder, reducing the seek distance from $1/3$ to $1/4$. In mirrored disks the two arms should be placed at $1/4$ and $3/4$ so that the seek distance is $1/8$.

AAP in the context of nonzoned and zoned disks with uniform disk accesses over all cylinders as well as a provision for hot spots is considered in (Thomasian and Fu, 2006). To simplify the analysis in zoned disks, rather than dealing with discrete values for the number of sectors per track in each zone, the capacity of each track is set to be proportional to the radius. Denoting the radius of the innermost and outermost cylinder with R_i and R_o , then $R_i + \sqrt{(R_o^2 + R_i^2)}/2$ divides the disk into two parts with equal capacities and the two arms should be placed at $R_i + \sqrt{(4R_i^2 + R_o^2)}/3$ and $R_i + \sqrt{(2(R_i^2 + 2R_o^2))}/3$. AAP is detrimental for higher disk utilizations, since positioning of the disk arm may delay external requests. Simulation results show a crossover point and beyond a certain arrival rate AAP results in degraded performance.

Predictive disk arm movement based on the history of past requests is undertaken in (Mumolo, 1999). Trace-driven simulations have shown a reduction of 11%-23% in seek time. Memory requirements and computational complexities of several prediction algorithms are specified. Reducing file systems latency using a predictive approach is also relevant to this discussion (Griffioen and Appleton, 1994).

Rearranging data on disk can be used to reduce seek distances, as discussed for single disks in Chapter 3. A study that applies cylinder remapping in both single and mirrored disks is reported in (Geist et al., 1994). Trace analysis of cylinder request streams exhibits strong Markovian dependence, which are used in expressing the mean seek distance for both single and mirrored disks. Simulated annealing optimization (Kirkpatrick et al., 1983) is used to find permutations reducing seek distance. The simulated annealing algorithm is specified in (Press et al., 2007). The computational costs with a disk with 1760 cylinders are reduced by carrying out the optimization on clusters of 40 disk cylinders. The optimal permutations for two disks are non-identical.

4.8 Mirrored disk performance in normal, degraded, rebuild modes

I/O trace analysis of an airline reservation system reported in (Ramakrishnan et al., 1992) determined that most disk accesses are to small randomly placed disk blocks of size 4 KB (96%) and 24 KB (4%). This workload is considered in several studies of OLTP workload performance. Note that accesses to large small blocks is dominated by positioning time.

We analyze RAID1 performance in normal, degraded, and rebuild modes with an OLTP workload, which is also considered in analyzing RAID5 performance in Chapter 5.

Disk access delay is a major contributor to txn response time in OLTP workloads and percentiles of response time are used to determine the maximum throughput in benchmarks. Disk accesses can be improved by caching or prefetching, which are discussed in Section 4.1.

4.8.1 Queueing theory for disk performance evaluation

An M/G/1 queueing model with Poisson arrivals (M) and general service times (G) at a single server (Kleinrock, 1975) has been adopted in several analytic studies of RAID: (Merchant and Yu, 1992, 1995, 1996), (Chen and Towsley, 1993), (Thomasian and Menon, 1994, 1997), (Chen and Towsley, 1996), (Thomasian et al., 2004), (Thomasian, 2005b), (Thomasian et al., 2007a), (Thomasian and Xu, 2008).

According to the M/G/1 queueing model the requests arrival process is Poisson with rate λ . Disk service times have a general distribution whose i^{th} moment is $\bar{x}^i$, variance $\sigma_X^2 = \bar{x}^2 - (\bar{x})^2$, and coefficient of variation squared $c_X^2 = \sigma_X^2 / (\bar{x})^2$. The mean residual service time is $\bar{x}_r = \bar{x}^2 / (2\bar{x})$. The *Laplace Stieltjes Transform - LST* of waiting time is $\mathcal{B}^*(s) = \int_0^\infty b(x)e^{-sx}dx$ (Kleinrock, 1975). The disk utilization factor is $\rho = \lambda\bar{x}$, which according to Little's result is the mean number of requests at the server (Kleinrock, 1975), which is also the fraction of time the server is busy, which should be less than one to ensure finite queue-lengths.

The mean queue-length according to Little's result is $\bar{N}_q = \lambda W$, where W is the mean waiting time in the queue. Given that *Poisson Arrivals See Time Averages - PASTA* (Wolff, 1982), the mean waiting time with FCFS scheduling is the sum of the waiting time in the queue (product of mean queue-length and mean residual service time) and the delay introduced by a server (probability server is busy and $\bar{x}_r$).

$$W = \bar{N}_q \bar{x} + \rho \bar{x}_r.$$

Noting that $\bar{N}_q = \lambda W$ yields the mean waiting time with FCFS scheduling, which is known as the Pollaczek-Khintchine - P.K. formula (Kleinrock, 1975):

$$W = \frac{\lambda \bar{x}^2}{2(1 - \rho)} = \frac{\rho \bar{x}(1 + c_X^2)}{2(1 - \rho)}, \quad \rho < 1. \quad (4.3)$$

The second moment of waiting time is (Kleinrock, 1975)

$$\overline{w^2} = 2W^2 + \frac{\lambda \bar{x}^3}{3(1 - \rho)}. \quad (4.4)$$

There is another P.K. formula for the LST of waiting time, which can be inverted to obtain its distribution $\mathcal{W}^*(s)$, which can be differentiated to obtain $\overline{w^k}$.

$$\mathcal{W}^*(s) = \frac{s(1-\rho)}{s-\lambda+\lambda\mathcal{B}^*(s)} \quad (4.5)$$

The k^{th} moment of waiting time is given by Takacs recurrence formula (Kleinrock, 1975).

$$\overline{w^k} = \frac{\lambda}{1-\rho} \sum_{i=1}^k \binom{k}{i} \frac{\overline{x^{i+1}}}{i+1} \overline{w^{k-i}} \quad (4.6)$$

The mean and k^{th} moment of response time is given as:

$$R = W + \bar{x}, \quad \overline{r^k} = \sum_{i=0}^k \binom{k}{i} \overline{w^{k-i}} \overline{x^i} \quad k \geq 1. \quad (4.7)$$

The variance of response time is the sum of the variances of waiting time and service time, since the two are independent for the FCFS scheduling policy.

$$\sigma_R^2 = \sigma_W^2 + \sigma_X^2.$$

The squared coefficient of variation of disk response times is given in (Thomasian and Menon, 1997).

$$c_R^2 = \frac{\sigma_R^2}{R^2} = \frac{c_X^2 + \frac{\rho s_X}{3(1-\rho)} + \frac{\rho(1+c_X^2)}{4(1-\rho)^2}}{1 + \frac{\rho(1+c_X^2)}{1-\rho} + \frac{\rho^2(1+c_X^2)^2}{4(1-\rho)^2}}. \quad (4.8)$$

$s_X = \overline{x^3}/(\bar{x})^3$ is the coefficient of skewness of service time, the third moment divided by the mean cubed.

Similarly the response time distribution can be obtained by inverting the response time LST which is the product of LSTs for service time and waiting time:

$$\mathcal{R}^*(s) = \mathcal{B}^*(s)\mathcal{W}^*(s).$$

In the case of the exponential service times $c_X^2 = 1$ and the mean response time of the M/M/1 queue is:

$$R_{M/M/1} = \frac{\bar{x}}{1-\rho}, \quad \rho < 1. \quad (4.9)$$

With FCFS scheduling the waiting time is independent of service time, so that the variance of response time is given as follows:

$$\sigma_R^2 = \sigma_W^2 + \sigma_X^2.$$

This equation does not hold for other disk scheduling policies such as SATF, which similarly to *Shortest Job First - SJF* take into account request service time in scheduling jobs, since waiting time is not independent of service time.

Plotting the mean response time R versus $0 \leq \rho < 1$ by varying λ .

$$R_{M/G/1} = \begin{cases} \bar{x} & \text{for small value of } \lambda \\ \infty & \text{as } \lambda \rightarrow 1/\bar{x} \end{cases}$$

The advantage of an open model is that the performance given as mean response time versus λ , while the number of active sources and their “think time” would be required otherwise. There is also the difficulty of analyzing closed models, which is the topic of H. Takagi’s 2nd volume on Queueing Systems. The mean response time in an M/G/1/N queueing systems with N sources with exponentially distributed think time and general service times which is based on (Buzen and Goldberg, 1974) is given in Chapter 1.

Percentiles of response times in M/G/1 can be obtained from the distribution of response time $R(t)$, e.g., the 90th percentile is obtained by solving $R(t_{90}) = 0.9$, where $R(t) = \int_0^t r(x)dx$, where $r(x)$ is the density function of response time obtained by inserting $\mathcal{R}^*(s)$. Empirical equations for response time percentiles of the form $t_{90} \approx R_{M/G/1} + k\sigma_R$ are given in (Allen, 1990).

4.8.2 RAID1 performance in normal mode

This section is based on (Thomasian and Xu, 2008). The arrival rate to a RAID0/1 disk array with $N = 2M$ disks is Λ . Assuming that disk loads are balanced due to striping, the load at each disk pair is $\lambda = \Lambda/M$. The fraction of read (resp. write) requests is f_r (resp. $f_w = 1 - f_r$). The i^{th} moment of read and write accesses is $\bar{x}_r^i$ and $\bar{x}_w^i$. Assuming read requests are routed uniformly over two BM disks the disk utilization due to read and write requests is:

$$\rho_r = ((\lambda f_r)/2)\bar{x}_r \text{ and } \rho_w = \lambda f_w \bar{x}_w.$$

The maximum arrival rate per disk is

$$\lambda_{normal}^{max} = [f_r/2)\bar{x}_r + f_w \bar{x}_w]^{-1}.$$

The fraction of read and write accesses to disks is $f'_r = (f_r/2)/(f_r/2 + f_w)$ and $f'_w = 1 - f'_r$, so that the i^{th} moment of disk access time is:

$$\bar{x}_d^i = f'_r \bar{x}_r^i + f'_w \bar{x}_w^i,$$

The mean disk utilization is:

$$\rho = \lambda \bar{x}_d = \lambda(f_r/2\bar{x}_r + f_w \bar{x}_w) = \rho_r + \rho_w.$$

If read and write requests are processed in FCFS order the mean waiting time is:

$$W_{M/G/1} = \frac{\lambda \bar{x}_d^2}{2(1 - \rho)}. \quad (4.10)$$

The mean response time of read requests, which affect mean txn response time can be reduced by processing them at a higher priority than writes. Given the mean residual service times for reads and writes:

$$\bar{x}'_r = \frac{\bar{x}_r^2}{2\bar{x}_r} \text{ and } \bar{x}'_w = \frac{\bar{x}_w^2}{2\bar{x}_w}$$

and disk utilizations ρ_r and ρ_w for reads and writes, the mean waiting time for read requests is obtained by applying PASTA again:

$$W_{M/G/1}^r = \bar{N}_q^r \bar{x}_r + \rho_r \bar{x}'_r + \rho_w \bar{x}'_w.$$

Noting that $\bar{N}_q^r = (\lambda f_r) \bar{x}_r W_r = \rho_r W_r$ and simplifying (Kleinrock, 1976):

$$W_{M/G/1}^r = \frac{\lambda \bar{x}_d^2}{2(1 - \rho_r)} = \frac{\rho \bar{x}(1 + c_x^2)}{2(1 - \rho_r)}. \quad (4.11)$$

The improvement due to prioritizing read requests is: $W_{M/G/1}^r / W = (1 - \rho)/(1 - \rho_r)$, e.g., 3-fold decrease for $\rho = 0.8$ and $\rho_r = 0.4$.

In addition to increasing throughput, mirroring improves the response time for read requests, since they can be processed in parallel. To quantify this effect we assume that all disk requests are reads ($f_r = 1$) the arrival rate to a mirrored pair is 2λ , and disk access time is exponentially distributed with mean $\bar{x} = 1/\mu$. With uniform routing each disk is subjected to Poisson arrivals with rate λ (Kleinrock, 1975) and the per disk utilization is $\rho = \lambda \bar{x}$.

The mean response time in normal mode is $R_{normal} = \bar{x}/(1 - \rho)$ and with one failed disk $R^{degraded} = \bar{x}/(1 - 2\rho)$, e.g., $\rho = 0.4$, $R^{degraded} = 5\bar{x}$ versus $R_{normal} = 1.67\bar{x}$, which is a three-fold increase.

4.8.3 Uniform vs round-robin routing of read requests in mirrored disks

With uniform routing according to the splitting property of Poisson processes the arrival rate to each disk is $0.5 \times 2\lambda = \lambda$ (Kleinrock, 1975).

With round robin routing the interarrival time to each disk is an Erlang-2 distribution (Kleinrock, 1975), which is the sum of two exponentials, so that mean, second moment, and coefficient of variation of interarrival time are:

$$\bar{t} = 2 \times (1/(2\lambda)), \sigma_t^2 = 2 \times (1/(4\lambda^2)), \text{ and } c_a^2 = 1/2.$$

With everything else being equal, a smaller c_a^2 is expected to result in a smaller W , but this is not generally true (private communication Lajos Takacs (rip) at *Case Western Reserve U. - CWRU* in 1978). More specifically for the same mean interarrival time and service rate in GI/M/1 queues (Kleinrock, 1975) a smaller coefficient of variation for interarrival time does not necessarily imply a smaller mean waiting time (W). A counterexample to this effect is given in (Thomasian, 2014a).

The mean waiting time for GI/M/1 is: $W_{E-2} = \sigma \bar{x} / (1 - \sigma)$, where $\sigma = \mathcal{A}^*(\mu(1 - \sigma))$ and $\mathcal{A}^*(s)$ is the LST of interarrival time. In the case of the Erlang-2 distribution:

$$\mathcal{A}^*(s) = \left[\frac{2\lambda}{s + 2\lambda} \right]^2$$

Setting $s = \mu(1 - \sigma)$, factoring out $1 - \sigma$, we obtain a quadratic equation in σ , whose smaller (negative root) is meaningful.

$$\sigma^2 - (1 + 4\rho)\sigma + 4\rho^2 = 0, \text{ so that: } \sigma = \frac{1}{2}(1 + 4\rho - \sqrt{1 + 8\rho}).$$

Given $W_{M/M/1} = \rho \bar{x} / (1 - \rho)$, it is easy to see that $W_{MI/G/1} < W_{M/M/1}$ since $\sigma < \rho$.

Mirrored disks with a shared queue for processing read requests can be modeled as an M/M/2 queueing system with FCFS scheduling. Disk utilizations are $\rho = 2\lambda \bar{x} / 2 = \lambda \bar{x}$ and mean response time $R_2 = \bar{x} / (1 - \rho^2)$ (Kleinrock, 1975). Setting $\rho = 0.9$ $R_2 \approx 5\bar{x}$, while $R_1 = 10\bar{x}$ if requests were uniformly routed to two disks.

To attain lower response times the same read request can be issued to both disks. The first completed request is then accepted, but the other requests are allowed to proceed to completion. Disks modeled as M/M/1 queues have an exponentially distributed response times, so that $R_{min} = R(2\rho)/2 = \bar{x} / (2(1 - 2\rho))$. With n -way replication $R_{min}^n = \bar{x} / (n(1 - n\rho))$. Sending redundant disk accesses is beneficial provided $R > R_{min}^n$ or $\rho < 1/(n + 1)$.

Fork/join queueing system

The completion time of writes on both mirrored disks is represented as *Fork/Join - F/J* queueing system. A 2-way F/J queueing system with Poisson arrivals and exponential service time is analyzed exactly in (Flatto and Hahn, 1984), which leads to the following mean response time (Nelson and Tantawi, 1988), (Thomasian, 2014b):

$$R_2^{F/J} = R_1(1.5 - \frac{\rho}{8}) \text{ where } R_1 = \frac{\bar{x}}{1 - \rho} = (\mu - \lambda)^{-1}.$$

Curve-fitting against simulation results was used in (Nelson and Tantawi, 1988) to obtain an approximate expression for k -way F/J queueing system with Poisson arrivals and exponential service times.

$$R_K^{F/J}(\rho) \approx \left[\frac{H_K}{H_2} + (1 - \frac{H_K}{H_2}) \frac{4\rho}{11} \right] R_2^{F/J}(\rho), \quad 2 \leq K \leq 32. \quad (4.12)$$

This mean response time holds if all requests are writes, but otherwise with interfering read requests the write response time is higher and closer to $R_2^{max} = H_2 R = 1.5R$, i.e., as long as the total disk utilization remains equal to ρ . An approximate expression for the mean response time in n -way F/J queueing systems with Poisson arrivals and exponential service times is obtained using simulation in (Nelson and Tantawi, 1988). A similar attempt with general service times is reported in (Thomasian and Tantawi, 1994). A survey of F/J and related queueing systems is given in (Thomasian, 2014b).

4.8.4 Completion times of multiple requests in RAID1 and RAID5

Of interest in mirrored disks (RAID1) is the time to complete writes to both disks, while in the case of RAID5 it is the completion times of writing data and parity blocks. The completion time is the expected maximum of corresponding response times.

The response time is approximated by a Coxian distribution in (Chen and Towsley, 1993), which is applicable to cases when the coefficient of variation (c_R) is smaller or larger than one. Disk service time is approximated by an Erlang distribution in (Kuratti and Sanders, 1995), which restricts the analysis to certain workload and disk characteristics. An Erlang distribution for response times is used in (Thomasian and Menon, 1994, 1997), since for the workload and disk characteristics under consideration (small randomly distributed disk accesses $c_R^2 \leq 1$ in Eq. (4.8)) A similar approach is used in

(Merchant and Yu, 1995). An extreme values distribution (David and Nagaraja, 2003) in (Merchant and Yu, 1996) and (Thomasian et al., 2007a).

To compute R_n^{max} we first approximate the response times of its components with the extreme-value distribution

$$P[Y < y] = \exp[e^{-\frac{y-a}{b}}]$$

The mean is $\bar{Y} = a + \gamma b$ and the variance $\sigma_Y^2 = (\pi b)^2/6$. The parameters a and b can be obtained by setting $b = \sqrt{6\sigma_Y^2}/\pi$ and $a = \bar{Y} - \gamma b$. The maximum of n random variables with the extreme value distribution is:

$$Y_n^{max} = (a + \gamma b) + b \ln(n),$$

where $\gamma \approx 0.57721$ is the Euler-Mascheroni constant. It follows

$$R_n^{max} = R + \sqrt{6/\pi} \sigma \ln(n)$$

While easy to compute the approximation is not accurate and its accuracy can be improved by dividing second term of the above expression by 1.27 (Thomasian et al., 2007a).

The $k > 1$ -stage Erlang distribution ($\mathcal{E}_k$) with parameter μ at each stage (Kleinrock, 1975) has been used in (Thomasian and Menon, 1994, 1997) to approximate the response time distribution, since for $\rho \rightarrow 0$ $c_R^2(\rho) \rightarrow c_X^2$ and for $\rho \rightarrow 1$ $c_R^2(\rho) \rightarrow 1$, i.e., an exponential distribution, regardless of the service time distribution.

$$F(t) = 1 - e^{-\mu t} \sum_{j=0}^{k-1} \frac{(\mu t)^j}{j!}, \quad t > 0, \quad k \geq 1, \quad (4.13)$$

which has a mean $\bar{x} = k/\mu$, variance $\sigma_X^2 = k/\mu^2$, and CV $c = 1/\sqrt{k} < 1$.

The number of stages for $\mathcal{E}_k$ is determined as $k = 1/\lceil c_{R(\rho)}^2 \rceil$ and each stage has a rate $\mu = k/R(\rho)$. $R_K^{max}(\rho)$ for $\mathcal{E}_k$ is given as follows in (Thomasian and Menon, 1997),

$$R_K^{max}(\rho) = \int_0^\infty \left[1 - \prod_{i=1}^K \left(1 - e^{-\mu_i t} \sum_{j=0}^{k_i-1} \frac{(\mu_i t)^j}{j!} \right) \right] dt. \quad (4.14)$$

In the special case $K = 2$ and different response times we have:

$$R_2^{max}(\rho) = \sum_{i=1}^2 R_i(\rho) - \sum_{m=0}^{k_1-1} \sum_{n=0}^{k_2-1} \binom{m+n}{m} \frac{\mu_1^m \mu_2^n}{(\mu_1 + \mu_2)^{m+n+1}}, \quad (4.15)$$

where $\mu_1 = k_1/R_1(\rho)$ and $\mu_2 = k_2/R_2(\rho)$.

4.8.5 Operation in degraded mode analysis

When one of two mirrored disks fails in BM the read load on the surviving disk is doubled. The mean disk service time is $\bar{x}_d^{degr} = f_r \bar{x}_r + f_w \bar{x}_w$ and $\lambda_{max}^{degr} = 1/\bar{x}_d^{degr}$. The maximum request rate per second drops 2-fold for $f_r = 1$, but this drop is less significant for a large f_w . RAID1/0 tolerates up to $k \leq M$ disk failures as long as failed disks are not pairs. Given that disk loads are balanced due to striping: $R_r^{overall} = (k/M)R_r^{degr} + (1 - k/M)R_r^{norm}$.

In the case of ID with n disks we assume that the read load is evenly distributed between primary and secondary data blocks, so that the normalized load at each disk in normal mode is one, since one half of the load at each disk is due to primary data and the other half due to secondary data of the other $n - 1$ disks in that cluster: When a disk fails the load on remaining disks increases by $1/(n - 1)$, so that for ID $n = 8$ the load increase is 14.2%, versus 50% for BM.

Denoting the load in normal mode on primary and secondary disks in GRD by one, the load of secondary disks increases by k/M when k primary disks fail. We reduce the load on primary disks with k disk failures by a fraction $1 - \alpha_k$ of read requests, so that a fraction α_k is routed to secondary disks. To balance disk loads on both sides we set $1 - \alpha_k = \alpha_k + k/M$, hence $\alpha_k = (M - k)/2M$, so that with $k = M$ failures the load on primary disks is zero and secondary disks receive twice the load.

Routing of requests can be used to balance disk loads due to reads in CD as illustrated in Fig. 4.6 (Hsiao and DeWitt, 1990) in Section 4.2. Since CD can tolerate multiple disk failures the routing probabilities should be adjusted, as shown by the last two rows of the figure.

As a simple example we consider balanced disk loads in normal mode with $f_r = 1$. D_3 load will be doubled while the loads on remaining disks will increase to 6/5. In case D_7 fails the loads of D_5 - D_6 and D_8 - D_1 will increase to 3/2 of normal load. This is an instance when given a single spare disk the rebuilding D_2 or D_4 should be prioritized with respect to D_7 .

4.8.6 Rebuild processing in RAID1

Both RAID1 with *Basic Mirroring* - BM and RAID5 are k -Disk-Failure-Tolerant - k DFT with $k = 1$, i.e. $k - 1$ is the maximum number of disk failures that can be tolerated and both are susceptible to data loss if a second disk fails. Rebuild processing may be started immediately if a spare disk is provided. RAID0/1 differs from RAID5 in that it can tolerate the failure of M out of $2M$ disks, as long as they are not pairs.

Disk failures detection by *Predictive Failure Analysis* - PFA reduces rebuild overhead by copying the contents of a failing disk onto a spare disk.

<https://www.computerworld.com/article/2846009/the-5-smart-stats-that-actually-predict-hard-drive-failure.html>

Rebuild time can be minimized if the processing of external disk requests is quiesced. This is unacceptable because of the high the cost of downtime for most lines of business, so rebuild processing is carried out while the disk array continues its operation.

According to Gartner Group in June 2020, the average cost of IT downtime is \$5,600 per minute. Because there are so many differences in how businesses operate, downtime, at the low end, can be as much as \$140,000 per hour; \$300,000 per hour on average, and as much as \$540,000 per hour at the higher end.

The time to read the contents of an idle disk can be approximated as: $T_{copy}(\rho) \approx N_{track} \times T_r$, where N_{track} is the number of disk tracks, T_r is the disk rotation time. Extra delays due to track and cylinder skew are ignored (Ng, 1998), (Jacob et al., 2008).

Alternatively the disk capacity is known. SATA HDDs have a 100-200 MB/s transfer rate, which varies from inner to outer disk tracks.

https://en.wikipedia.org/wiki/Hard_disk_drive_performance_characteristics

The time to copy a disk is disk capacity divided by average transfer rate. SATA HDDs have a 100-200 MB/s transfer rate, which varies from inner to outer disk tracks due to zoning. Disk capacity is not fully utilized, but is questionable whether such knowledge can be exploited.

Given that the disk utilization due to user requests in normal mode is ρ , the disk utilization in degraded mode with all read requests is $\rho^{degr} = 2\rho$ and the fraction of time that the surviving disk in BM pair is idle and available rebuild processing is $1 - 2\rho$.

The analysis of RAID1 in rebuild mode is a special case of the analysis of RAID5 arrays using the *Vacationing Server Model* - VSM (Takagi, 1991) for the M/G/1 queueing model reported in (Thomasian and Menon, 1994, 1997). The analysis obtains rebuild processing time and the effect of rebuild processing on the mean response time of external disk requests.

According to VSM, the server starts taking vacations when the queue of external requests is empty. Vacations correspond to reading successive *Rebuild Units* - RUs, which may be tracks from the surviving disk to be written onto a spare disk. There are two types of vacations:

Type one vacations invoke a seek to access the next track to be rebuilt and a disk rotation to read the track.

Type two vacations read consecutive tracks without incurring a seek. Track skew is usually negligible in this case.

While the analysis in (Thomasian and Menon, 1994) takes into account two vacation types, at this point we only consider one type of rebuild requests, whose i^{th} moment is $\bar{v}^i$ and whose residual vacation time is $\bar{v}_r = \bar{v}^2/(2\bar{v})$. Given $\bar{v}^i = T_r^i$ then $\bar{v}_r = T_r/2$, one half of disk rotation time.

The external workload accesses randomly placed small disk blocks to read or write them. We ignore the difference between the access time of read and write requests by setting: $\bar{x}_r \approx \bar{x}_w = \bar{x}$. Given that the processing of first request is delayed by $\bar{v}_r$, then the first disk request has duration $\bar{y} = \bar{x} + \bar{v}_r$ and the modified busy period, called a delay cycle (Kleinrock, 1976) is:

$$\bar{d} = \frac{\bar{y}}{1 - \rho} = \frac{\bar{x} + \bar{v}_r}{1 - \rho}. \quad (4.16)$$

A simple derivation of this equation for VSM is given in (Thomasian, 2018b), but repeated in Chapter 5, where we also repeat the more detailed analysis in (Thomasian and Menon, 1994) with two vacation types.

The mean time to the arrival of the first request after the completion of a busy period is $1/\lambda$ so that we define a cycle as: $T_{cycle} = \bar{d} + 1/\lambda$.

The mean number of RUs/tracks read per idle period is obtained as follows. The probability that a request arrives during rebuild, which is assumed to be a disk rotation time is $p = 1 - e^{-\lambda T_r} \approx \lambda T_r$, so that the distribution of the number rebuilds per idle period is $P_n = p(1 - p)^{n-1}$ with a mean $\bar{n} = 1/p$ (Trivedi, 2002). Given a disk with N_{track} tracks, rebuild time is:

$$T_{rebuild} = N_{track} \times T_{cycle} / \bar{n}.$$

The read load on the surviving disk is reduced as rebuild progresses, which results in a reduction of the load on the surviving disk and acceleration of rebuild processing. This effect can be taken into account by carrying out the analysis in stages for increasing fractions of rebuilt data on a spare disk.

The mean waiting time in an M/G/1 queue with VSM (W_{VSM}) is given as Eq. (2.14a) in Chapter 2 in (Takagi, 1991). A simple derivation of this equation is given in (Thomasian, 2018b) as follows:

$$W_{VSM} = \frac{\lambda \bar{x}^2}{2(1 - \rho)} + \frac{\bar{v}^2}{2\bar{v}} = W_{M/G/1} + \frac{\bar{v}^2}{2\bar{v}}. \quad (4.17)$$

Eq. (11) for W_{VSM} in (Bachmat and Schindler, 2002) is based on Eq. (2.40a) in Chapter 2 in (Takagi, 1991) for the case when the first external request has an exceptional service time, which is not the case here.

Rebuild time is also estimated using the *Permanent Customer Model - PCM* which processes two types of requests (Boxma and Cohen, 1991):

Ordinary customers: Arrive according to a Poisson process to randomly placed disk blocks, which are served in FCFS order together with permanent requests.

Permanent Customers: There is single permanent requests which rejoin the tail of the queue of ordinary requests upon completing service.

PCM was adopted to analyze rebuild processing in mirrored disks in (Merchant and Yu, 1994). Permanent requests represent the reading of a track, while ordinary requests are accesses to read and update randomly placed disk blocks. Unlike VSM PCM cannot model that there are two types of rebuild reads.

PCM and VSM rebuild in mirrored disks in strict order of succeeding tracks and greedy order are compared in (Bachmat and Schindler, 2002). Greedy rebuild is intended to reduce rebuild time by out-of-order processing of track reads, i.e., by reading the closest unread track to the R/W head.

Note that greedy rebuild is not applicable to RAID5, because of excessive buffer space requirements, since rebuild units read from different disks will rarely belong to the same stripe to allow reconstruction. Out of order reading also complicates read redirection which is not addressed.

VSM incurs fewer seeks than PCM in reading successive tracks, since the probability that there are interim arrivals and the R/W head moves after reading the current track is lower for VSM than PCM:

$$P_{VSM} = 1 - e^{-\lambda T_r} \text{ and } P_{PCM} = 1 - e^{-\lambda(T_r + N_q \bar{x})},$$

where T_r is disk rotation time.

As shown in Figures 3(a) and 3(b) in (Bachmat and Schindler, 2002) rebuild time with VSM is lower than PCM, and except for the highest arrival rates $R_{PCM} > R_{VSM}$. The greedy policy shows very little improvement for R_{VSM} , which significantly outperforms R_{PCM} . This is because VSM processes rebuild requests at a lower priority than external requests, while PCM processes them at the same priority. It is not explained in (Bachmat and Schindler, 2002) why the VSM greedy policy outperforms PCM for Seagate Cheetah disks, but is slightly outperformed for Quantum Atlas disks. This effect is contrary to simulation results reported in (Fu et al., 2004a).

Rebuild processing can be parallelized for ID, GRD, and CD RAID1 organizations. This is because the contents of a disk are spread over multiple disks. In the case of CD organization the rebuilding of a failed disk D_i can be parallelized by copying from $D_{i \pm 1}$ disk. $n - 1$ -fold parallelism is possible with the ID organization for reading secondary data to rebuild primary data. In the case of GRD with $N = 2M$ disks, M disks can participate in rebuild processing. At low disk utilization the spare disk will constitute a bottleneck and the reading rate from source disks should be throttled.

TABLE 4.3 Parameters used in reliable archiving study.

Parameter	Meaning
m	data disklets in a reliability stripe
k	parity disklets in a reliability stripe
$n = m + k$	number of disklets in a reliability stripe
r	number of reliability stripes in a uber group
D	total number of disks
L	number of disklets per disk
$N = L \times D$	total number of disklets
$d = r \times n$	number of disklets per uber group
$u = \frac{D \times L}{n \times s}$	total number of uber-groups
S	number of uber parities
f	number of failed disks
$U = \lceil \frac{D \times L \times s}{n \times r} \rceil$	#number of disks storing uber parities

4.9 Protecting against rare event failures in archival systems

Digital archives necessitating stronger reliability measures than RAID to avoid data loss from device failure. Mirroring for high volume data is too expensive over time. A compromise solution that uses multi-level redundancy coding to reduce the probability of data loss from multiple simultaneous device failures. This approach handles failures of one or two devices efficiently, while still allowing the system to survive rare-event, larger-scale failures of four or more devices.

Rare failures in archival systems are dealt with in (Wildani et al., 2009). by Avani Wildani, Thomas J. E. Schwarz (USC, Marquette U.), Ethan Miller and Darrell Long at UCSC by splitting each disk into fixed size disklets, which are used to construct reliability stripes. To protect against rare event failures, reliability stripes are grouped into large uber (super) groups, each of which has a corresponding uber parity, which is only used to recover data when disk failures overwhelm the redundancy in a single reliability stripe. Uber-parity can be stored on a variety of devices such as NVRAM and always powered-on disks to offset write bottlenecks, while still keeping the number of active devices low. Our calculations of failure probabilities found that the addition of uber-groups allowed the system to absorb many more disk failures without data loss. Through discrete event simulation, it was determined that adding uber-groups only negatively impacts performance when these groups need to be used for a rebuild. Since rebuilds using uber parity occur very rarely, they minimally impact system performance over time. Finally, we showed that robustness against rare events can be achieved for under 5% of total system cost.

The parameters used in this study are summarized in Table 4.3.

Robustness is the probability of data loss given a certain number of disk failures. Robustness calculations avoid several problems that occur when modeling the reliability of disk arrays, e.g. that disk failures are correlated.

Assuming there are f failed disks at any point. Since disklets in a stripe come from different disks, the number x of unavailable disklets in a randomly chosen stripe is binomially distributed:

$$p_x(f, n, D) = \binom{n}{x} \frac{f!(D-n)!(D-f)!}{D!(D-f-n+x)!(f-x)!}.$$

Reliability stripes are protected against disklet un-availability through parity disklets. The probability that all stripes remain accessible without resorting to uber parities after f disk failures is:

$$P_S(k, f, n, D) = \sum_{i=0}^k p_i(f, n, D)$$

Let f_i denote the number of failed disks in stripe i . The uber parities can recover all data as long as

$$\sum_{i=1}^r (f_i - k) \leq s.$$

Let S denote the set of tuples $\vec{f} = (f_1, f_2, \dots, f_r)$, which satisfies the above summation. The probability that an Uber group has failed is

$$P_{Uber}(s, k, f, n, D) = \sum_{\vec{f} \in S} \prod_{i=1}^r p_{f_i}(f, n, D).$$

The probability that an uber group has failed for $k = 2$ is

$$P_{Uber}(2, 2, f, n, D) = \binom{n}{1} p_4(f, n, D) P_S(k, f, n, D)^{r-1} + \binom{n}{1} p_3(f, n, D) P_S(k, f, n, D)^{r-1} + \binom{n}{2} p_3(f, n, D)^2 P_S(k, f, n, D)^{r-2} + P_S(K, f, n, D)^r$$

Assuming that the uber parity is stored on a device that never fails the probability of data loss is given by p_{dl} , which is the probability that at least one uber group has lost data.

$$p_{dl}(s, k, f, n, D, r, L) = 1 - (P_{Uber}(s, k, f, n, D, r, L))^u$$

In the case when uber parities are stored on the same kind of disks as the disks in the archive it is assumed there is only one uber parity per uber group. The probability that x of the uber parity disk have failed is:

$$p_{uf} = \binom{U}{x} \binom{D}{f-x} \binom{U+D}{f}^{-1}$$

If x uber disks have failed then ux/U groups have no protection, whereas $(1 - x/U)u$ do. An uber group with no data protection has not suffered data loss with probability $p_S(k, f, n, D)^r$. The probability of data loss is given by p_{dl} the probability that at least one uber-group has lost data

$$p_{dl}(s, k, f, n, D, r, L) = 1 - \sum_{x=0}^f p_{uf} \times p_S^{uxr/U} \times p_{uber}^{(1-x/U)u}.$$

Data loss probabilities for the given configuration are given in Table 4.4.

TABLE 4.4 Data loss probabilities. Case 1: No uber failures, Case 2: Uber failures. $k = 1$, $s = 1$, $n = 16$, $D = 1024$, $r = 16$, $L = 64$.

f	Case 1	Case 2
1	0.	0.
2	0.00159525	0.00160774
3	0.0286551	0.0265257
4	0.106224	0.0998437
5	0.248078	0.237821
6	0.442996	0.431747
7	0.651256	0.642184
8	0.82319	0.8177
9	0.930476	0.928003
10	0.979671	0.978861
11	0.995754	0.995567
12	0.99939	0.99936
13	0.999942	0.999939
14	0.999996	0.999996
15	1.	1.

Adding uber parities is a good proposition, because simulation, we see up to a four-fold improvement in *Mean Time to Data Loss - MTDDL* after adding uber parities, which is more robust than RAID and requires fewer disks.

Simply discrete event simulation program in Python was used in obtaining the MTDDL for two cases with and without uber disks.

<https://simpy.readthedocs.io/en/latest/>

4.10 RAIDP: ReplicAtion with IntraDisk Parity for cost effective storage of warm data

Examples of systems that triple-replicate warm data are Amazon’s DynamoDB, Facebook’s Haystack, Google File System - GFS, Windows Azure Storage - WAS, Microsoft’s Flat Datacenter Storage - FDS, Hadoop Distributed File System - HDFS.

The advantages of replication for warm data are listed as follows:

Better for reads: (1) load balancing, (2) parallelism, (3) avoids degraded reads (failed disks with erasure coding).

Better for writes: Lower synch latency.

Better for reads and writes: 1. Increased sequentiality.

2. Avoids encoding CPU processing used.

3. Lower repair traffic.

Distributed storage systems often triplicate data to reduce the risk of permanent data loss, thereby tolerating two simultaneous disk failures. This price can be reduced by utilizing erasure coding, but this optimization is usually only applied to cold data, because erasure coding affects performance for warm data. RAIDP maintains only two replicas, rather than three, but it increases durability by utilizing small disk “add-ons” for storing intra-disk erasure codes that are local to the server but fail independently from the disk (Rosenfeld et al., 2020).

By carefully laying out the data, the add-ons allow RAIDP to recover from simultaneous disk failures. RAIDP retains much of the benefits of replication, trading off some performance and availability for substantially reduced storage requirements, networking overheads, and their related costs. RAIDP implemented in HDFS shows that baseline RAIDP achieves performance close to that of HDFS with only two replicas, and performs within 21% of the default triplicating HDFS with an update-oriented variant, while halving the storage and networking overheads and providing similar durability.

Given N disks there are $N - 1$ chunks per disks and with disk capacity S the superchunk size is S/N . Two other architectural considerations are:

Mirroring: Superchunks are 2-replicated

Sharing: Any two disks share at most one superchunk A RAIDP layout is given in Fig. 4.12.

D_1	D_2	D_3	D_4	D_5	D_6	D_7
1	2	3	4	5	6	7
7	1	2	3	4	5	6
8	9	10	11	12	13	14
13	14	8	9	10	11	12
15	17	18	18	19	20	21
19	20	21	15	16	17	18

FIGURE 4.12 Example RAIDP Layout of superchunks across the disks. If D_1 fails then its load is distributed across remaining disks.

First-row superchunks are replicated in the second row, third-row superchunks are replicated in the fourth row, and that, generally, i -th row superchunks are replicated in the $i + 1$ row for every odd i . The replication shift between rows is gradually increasing: the replica shift between the first and second rows is exactly one column, the shift between the third and fourth row is exactly two columns, and, generally, the shift between the pair of rows $2i - 1$ and $2i$ is i columns.

Lstor replaces a third replica disk with 2 Lstors, which need to be cheap, fast, and fail separately from disk.

Storage: Enough to maintain parity.

Processing: Microcontroller for local machine independence

Power: Several hundred Amps for 2–3 min from small supercapacitor to read data from the Lstor Commodity 2.5 inch 4 TB disk for storing an additional replica costs \$100: 66% more than a conservative estimate of the cost of two Lstors.

It is assumed that Lstors have the following properties:

1. Disk failures occur separately from Lstor failures.

2. The Lstor capacity is that of a superchunk to keep parity information, plus a small journal.
3. Parity information can be stored on the Lstor at least as quickly as the data can be stored on the associated disk.
4. The Lstor's journal is fast. It can be read from and written to at a high enough speed such that the overhead is negligible as compared to the corresponding disk I/O. It can be implemented using DRAM equipped with a battery.
5. The Lstor parity and journal content are persistent. For the RAIDP layout in Fig. 4.12 the Lstor for D_1 will hold $1 \oplus 7 \oplus 8 \oplus 13 \oplus 15 \oplus 19$.

To illustrate the use of Lstor in recovering from two disk failures consider five disks with associated LSTORs. The letters indicate superchunks.

$D_0 : a, d; L_0 : a \oplus d$.

$D_1 : a, b, c; L_1 : a \oplus b \oplus c$.

$D_2 : b, c; L_2 : b \oplus c$.

$D_3 : d, e; L_3 : d \oplus e$.

$D_4 : c; L_4 : c$.

If D_2 and D_3 fail then e is lost, but it can be reconstructed as $L_2 \oplus b_1$ or $L_3 \oplus d_0$.

4.11 Remote mirroring for disaster recovery

There are four common solutions for dealing with disasters: (1) tape backup, (2) remote mirroring, (3) site failover, and (4) site reconstruction. For Seneca Minwen Ji, Alistair Veitch, and John Wilkes at HP Labs in (Ji et al., 2003) set out to design an asynchronous mirroring system that could exploit a low-speed *Wide Area Network* - WAN link by batching writes in an order-preserving manner. Seneca's goal is to make the data available and keep each copy consistent despite disk array, Seneca box, network failures, and both temporary and permanent site outages.

https://en.wikipedia.org/wiki/Seneca_the_Younger

Synchronous mirroring (sync): The secondary receives and applies each write before the write is considered completed at the primary. This scheme provides great resilience to data loss, but has poor write performance and high network cost.

Write-order preserving asynchronous mirroring (async): Writes on the primary are propagated to the secondary at the speed of the interconnect. There is an improvement in performance with respect to synchronous mirroring.

Batched asynchronous mirroring with write absorption (asyncB): Repeated writes to the same data are coalesced to reduce bandwidth cost. Batches of updates from the primary are periodically propagated to the secondary.

The usual assessment criteria for remote mirroring designs include data availability, data (reliability), performance degradation in normal use, and the cost of operation due to inter-site network traffic. The following issues arise. (1) the fault-coverage model; (2) how closely synchronized the copies are (divergence); (3) how updates are propagated; (4) when updates are acknowledged; (5) where the data duplication is performed.

While (Ji et al., 2003) is concerned with the Seneca remote mirroring system, its Table 1 which gives examples of remote mirroring policies is reproduced below, where **D**ivergence, **P**ropagation order, **W**here done.

Veritas Volume Replicator **D**: none / IO-count-bounded / log-space-bounded / unbounded (bitmap) **P**: lock-step / in-order asynchronous / in-order asynchronous / out of order **W****D**: host.

IBM's Peer-to-Peer Remote Copy (PPRC) **D**: none / unbounded (full copy) **P**:lock-step / out of order **W**: array.

IBM's Extended Remote Copy (XRC) **D** space-bounded / unbounded (full copy) **P** in order asynchronous /out of order-host + disk array.

EMC Symmetrix Remote Data Facility SRDF-1 **D** Semisynchronous: none / unbounded (per-track bitmap) **P** lock-step / out of order **W**disk array.

EMC SRDF 2 Adaptive Copy track-count-bounded (track "bitmap") **P** in order / out-of-order **W** disk array.

EMC SRDF-3 **D** Adaptive Copy: track-count-bounded / unbounded (track "bitmap") **PO** out of order / out-of-order **W** disk array.

NetApp SnapMirror **D** unbounded (file system structure) **P** atomic in-order asynchronous batched, with overwrites **W** file server.

HP XP continuous access-1 **D** synchronous: none / unbounded (bitmap) **P** lock-step / out of order **W** disk array.

HP XP continuous access-2 **D** asynchronous: cache-space-bounded / unbounded (bitmap) **PO** in-order asynchronous / out of order **W** disk array.

HP Continuous Access Storage Appliance (CASA) same two cases as above.

Seneca (Ji et al., 2003) **D** time-bounded batches / log-space-bounded / unbounded (bitmap) **P** grouped, atomic in-order asynchronous batched, with overwrites / (same) / out of order **W** duplexed SAN appliance.

IBM's XRC for MVS mainframes drain cached data from disk array NVRAM cache, writes it to a disk log, and transmits it to a remote site.

4.12 RAID reliability analysis

Garth Gibson estimated the time to failure based on the length of time users took to return their broken disks to the manufacturer (Gibson, 1992). A follow up study with Bianca Schroeder determined the relative frequency (percentage) of hardware replacements for ten most frequently replaced components for three systems named as PC1, COM1, COM2 (Schroeder and Gibson, 2007a,b).

HPC1 components: Hard drive 30.6, Memory 28.5, Misc/Unknown 14.4, CPU 12.4, PCI motherboard 4.9, Controller 2.9, QSW 1.7, Power supply 1.6, MLB 1.0, SCSI BP 0.3.

COM1 components: Power supply 34.8, Memory 20.1, Hard drive 18.1, Case 11.4, Fan 8.0, CPU 2.0, SCSI Board 0.6, NIC Card 1.2, LV Power Board 0.6, CPU heatsink 0.6.

COM2 components: Hard drive 49.1, Motherboard 23.4, Power supply 10.1, RAID card 4.1, Memory 3.4, SCSI cable 2.2, Fan 2.2, CPU 2.2, CD-ROM 0.6, Raid Controller 0.6.

The following discussion considers only hard drives, although other components may lead to storage system failures. Disk controller in addition to disk failures are taken into account in Subsection 4.12.13. *Latent Sector Errors - LSEs* are discussed in Chapter 5.

This section consists of the following subsections.

Reliability expressions for mirrored and hybrid disk arrays in Subsection 4.12.1.

Reliability analysis of variants of RAID1 and RAID(4+k) w/o repair in Subsection 4.12.2.

Performability analysis in Subsection 4.12.4.

Shortcut Reliability Analysis - SRA in Subsection 4.12.5

Transient solutions of Continuous Time Markov Chains - CTMCs in Subsection 4.12.6

RAID reliability analysis with repair in Subsection 4.12.7

Solving linear Ordinary Differential Equations - ODEs in Subsection 4.12.8

Shortcut method for RAID5 rebuild reliability analysis in Subsection 4.12.9

Enhanced reliability modeling of storage systems in Subsection 4.12.10

Metrics for reliability estimation in storage systems in Subsection 4.12.11

Expected Annual Fraction of Data Loss - EAFDL in Subsection 4.12.12

Taking into account controller failures in Section 4.12.13

4.12.1 Reliability expressions for mirrored and hybrid disk arrays

The reliability function is the complement of the probability distribution of time to failure: $R(t) = 1 - F(t)$ (Trivedi, 2002; Trivedi and Bobbio, 2017). In our discussion we will be mainly concerned with disk reliabilities, rather than *Disk Array Controllers - DACs*, interconnects, power supplies, cooling fans, etc., since we are interested in the relative reliability of various RAID configurations, rather than estimating system reliability.

The Weibull distribution $R(t) = e^{-(\delta t)^\alpha}$ has been determined to be a good fit for the time to disk failure. The parameter $\alpha > 0$ is a shape factor, for $\alpha = 1$ we have an exponential distribution and the failure rate increases (resp. decreases) with t for $\alpha > 1$ (resp. $\alpha < 1$) (Trivedi, 2002).

The exponential distribution has been used for RAID reliability analysis by researchers at U. Calif. at Berkeley (Patterson et al., 1988), (Gibson, 1992), because of its mathematical tractability, hence $R(t) = e^{-\delta t}$.¹

For a positive random variable $x > 0$ with *probability density function - pdf* $f(x)$ and *Probability Distribution Function - PDF* $F(x) = \int_0^x f(y)dy$, the mean can be determined using integration by parts (Trivedi, 2002):

$$\bar{x} = \int_0^\infty xf(x)dx = \int_0^\infty [1 - F(x)]dx.$$

The *Mean Time to Failure - MTTF* for an exponential distribution is then:

$$\text{MTTF} = \int_0^\infty R(t)dt = \int_0^\infty e^{-\delta t} dt = \frac{1}{\delta}.$$

1. We use δ for the failure rate for reliability modeling, since λ was used for the arrival rate of requests in queueing analysis.

Given that for a single disk $MTTF_1 = 10^6$ hours, for $n = 10000$ disks is $MTTF_{10K} = 100$ hours or four days. In the case of the exponential distribution the failure rate with n components is:

$$R_n^{rep}(t) = R^n(t) = e^{-n\delta}.$$

It follows that the time to the first fail which will lead to data loss for n disks is:

$$MTTF_{n-disk} = \frac{MTTF_{disk}}{n}.$$

Given that for a single disk $MTTF_1 = 10^6$ hours, for $n = 10000$ disks is $MTTF_{10K} = 100$ hours or four days.

The expected value of the minimum of n independent random variables with a Weibull distribution with parameters δ and α is:

$$\overline{\min(X)} = \frac{\Gamma(1 + \frac{1}{\alpha})}{n\delta}.$$

The Gamma function is defined as: $\Gamma(x) = \int_0^\infty t^{x-1} e^{-t} dt$ and for positive integers it is given as $\Gamma(x) = (x-1)!$ (Trivedi, 2002).

In the case of n -way disk replication all n disks should fail for data loss to occur:

$$R_n^{rep} = 1 - [1 - R(t)]^n.$$

$$R_2^{rep}(t) = 1 - (1 - R(t))^2 = 2R(t) - R^2(t).$$

In the case of an exponential distribution the *Mean Time to Failure - MTTF* is:

$$MTTDL_2^{rep} = \int_0^\infty R_2^{rep}(t) dt = \frac{2}{\delta} - \frac{1}{2\delta} = \frac{1.5}{\delta}.$$

The MTTDL of a disk array with n replicated disks is the sum of times it operates with $0 \leq jn - 1$ disk failures:

$$MTTDL = \sum_{j=0}^{n-1} T_j$$

In the case of the exponential distribution

$$MTTF_n^{rep} = \sum_{j=0}^{n-1} \frac{1}{(n-j)\delta} = \frac{H_n}{\delta}, \text{ where } H_n \text{ is the Harmonic sum.} \quad (4.18)$$

A point of diminishing returns in improving the MTTDL is soon reached as n increases.

Let $X_1, \dots, X_n$ be n independent random variables with exponential distribution all with parameter δ_i and we want to determine

$$X(n) = \max_{1 \leq i \leq n} (X_i).$$

For the exponential distribution $F(t) = 1 - e^{-\delta t}$, the distribution of the maximum $F_n(t)$, of $X(n)$ can be expressed as follows (Trivedi, 2002).

$$F_n(t) = \text{Prob}[X(n) \leq t] = \prod_{i=1}^n (1 - e^{-\delta_i t})$$

Given that the variables $(X_i)_{i \in n}$ are *i.i.d.* with parameter δ the expectation of $X(n)$ is given as:

$$E[X(n)] = \int_0^\infty [1 - F_n(t)] dt = \int_0^\infty [1 - (1 - e^{-\delta t})^n] dt \quad (4.19)$$

It follows

$$E[X(n)] = \frac{1}{\delta} \sum_{i=1}^n \binom{n}{i} (-1)^{i-1} \frac{1}{i} = \sum_{i=1}^n \frac{1}{i}.$$

Induction is used in (Ohanyan and Thomasian, 2016) to show that this is the Harmonic sum (Trivedi, 2002).

Transformation of random variables is used in Section 4.6.2 in (Trivedi, 2002) to evaluate Eq. (4.19). Let $u = 1 - e^{-\delta t}$ then $dt = du/[\delta(1-u)]$

$$E[X(n)] = \frac{1}{\delta} \int_0^1 \frac{1-u^n}{1-u} du = \frac{1}{\delta} \sum_{i=1}^n \int_0^1 u^{i-1} du = \frac{1}{\delta} \sum_{i=1}^n \frac{1}{i} = \frac{1}{\delta} H_n.$$

$E[X(n)]$ can be derived using the definition of T_i as the i^{th} order statistic (Trivedi, 2002), (David and Nagaraja, 2003): $E[T_2 - T_1] = \frac{1}{(n-1)\delta}$, where $T_1 = 1/(n\delta)$ is the smallest random variable. Then by induction:

$$E[T_n] = E[T_1 + \sum_{i=1}^{n-1} (T_{i+1} - T_i)] = \sum_{i=0}^{n-1} \frac{1}{(n-i)\delta} = \frac{H_n}{\delta}.$$

<https://mikespivey.wordpress.com/2013/05/13/expectedmaxexponential/>

4.12.2 Reliability analysis of variants of RAID1 and RAID(4 + k) w/o repair

Maximum Distance Separable - MDS Erasure Codes - ECs utilize k check disks to protect against k disk failures, which is the minimum redundancy level (Thomasian and Blaum, 2009). Thus EC incurs less redundancy than replication, even when the code is not MDS. MDS RAID(4 + k) arrays are k -Disk-Failure-Tolerant - k DFT, where $k \geq 1$ is the number of redundant disks.

The reliability expression for RAID5 1DFT arrays with N disks without repair is the sum of probabilities with zero and a single disk failure.

$$R_{RAID5(N)}(t) = R^N(t) + N R^{N-1}(t)[1 - R(t)] = N R^{N-1}(t) - (N-1)R^N(t). \quad (4.20)$$

In the case of $R(t) = e^{-\delta t}$:

$$MTTDL_{RAID5(N)} = \int_0^\infty R_{RAID5(N)}(t) dt = \frac{N}{(N-1)\delta} - \frac{N-1}{N\delta} = \frac{2N-1}{N(N-1)\delta} = \frac{1}{N\delta} + \frac{1}{(N-1)\delta}. \quad (4.21)$$

The MTTDL is expressed as the sum of duration of the two operating stages of RAID5 with no and single disk failure, which yields the MTTDL bypassing the integration of reliability expression.

4.12.3 Reliability analysis of variants of RAID1 and RAID(4 + k) w/o repair

We consider a CTMC whose states are denoted by $\mathcal{S}_i$, where $0 \leq i \leq I$ denotes the number of failed disks and I is the maximum number of failed disks that can be tolerated in an array with N disks. Let $A(N, i)$ denote the number of cases that i failed disks that do not lead to data loss, hence the probability that the i^{th} disk failures does not lead to data loss is obtained by dividing by the total number of cases:

$$p_i = \frac{A(N, i)}{\binom{N}{i}}, 1 \leq i \leq I, \quad (4.22)$$

and $q_i = 1 - p_i$, $2 \leq i \leq I$ for the probability that a transition from $\mathcal{S}_i$ leads to data loss.

The rates of transitions not leading to data loss are given as follows:

$$\mathcal{S}_{i-1} \rightarrow \mathcal{S}_i \text{ with rate: } (N-i+1)p_i\delta, \quad 1 \leq i \leq I.$$

$$\mathcal{S}_{i-1} \rightarrow \mathcal{S}_F \text{ with rate: } (N-i+1)q_i\delta, \quad 2 \leq i \leq I.$$

For RAID1 with $N = 2M$ disks, $I = M$ for BM and most RAID1 organizations, and $I = c$ for the ID organization, since only one disk failure per cluster is allowed. Generally, $A(N, i) = \binom{N}{i}$, $i = 0, 1$ and $A(N, i) = 0$ for $i > I$. Setting $r = R(t) = e^{-\lambda t}$, RAID reliability can be expressed as:

$$R_{RAID}(N) = \sum_{i=0}^I A(N, i) r^{N-i} (1-r)^i. \quad (4.23)$$

In the case of RAID1/0 with $N = 2M$ disks, which is a RAID0 array with M virtual disks each one of which is a RAID1, as few as two disk failure may lead to data loss if they are pairs, but up to M disk failures can be tolerated “on one side” in the best case.

$$A(N, i) = \binom{M}{i} 2^i, \quad 0 \leq i \leq M. \quad (4.24)$$

For n -way replication with $M = N/n$ groups of n -way replicated disks:

$$A(N, i) = \binom{M}{i} n^i, \quad 0 \leq i \leq M.$$

In the case of RAID0/1 and GRD up to M disks can fail on one side or the same virtual disk:

$$A(N, i) = 2 \binom{M}{i}, \quad 0 \leq i \leq M. \quad (4.25)$$

If it was possible to remove the side with a single failed disk then the system continues its operation with M disks, the reliability function would be:

$$R_{GRD} = r^{2M} + 2Mr^{2M-1}(1-r) + r^M.$$

which can be used to obtain the MTDDL.

ID with c clusters and $n = N/c$ disks per cluster, can have only one disk failure per cluster and any one of the n disks in a cluster can fail.

$$A(N, i) = \binom{c}{i} n^i, \quad 0 \leq i \leq c. \quad (4.26)$$

The expression for $A(N, i)$ for CD is derived in (Thomasian and Blaum, 2006) and also the Appendix in (Gafsi and Biersack, 2000).

$$A(N, i) = \binom{N-i-1}{i-1} + \binom{N-i}{i}, \quad 1 \leq i \leq M. \quad (4.27)$$

Let v_i denote the number of visits to state $\mathcal{S}_i$. When repair returning the system to its initial state is not allowed $v_0 = 1$ for $\mathcal{S}_0$, then v_i can be computed using the probability p_i given by Eq. (4.22).

$$v_i = v_{i-1} p_i, \quad v_i = \prod_{j=0}^i p_j, \quad 1 \leq i \leq I. \quad (4.28)$$

A closed form expression for $A(N, i)$ for LSI Logic RAID is not available, but the reliability expression using enumeration for $N = 8$ was reported in (Thomasian and Tang, 2012)

$$A(8, i) = \binom{8}{i}, \quad 0 \leq i \leq 2, \quad A(8, 3) = \binom{8}{3} - 4 = 52,$$

since there are four cases where the Ddisk is in the middle, so that $q_3 = 4/56 = 1/14$. There are $\binom{8}{4} = 70$ configurations for four disk failures, but according to the enumeration given in Table 2 in (Thomasian and Tang, 2012) only 25 cases lead to data loss, so that $q_4 = 25/70 = 5/14$, which differs from $q_4 = 16/65$ given in Section 3.1.2 in (Schwarz et al., 2008).

The reliability analyses of SSPiRAL(4+4,3) disk array in (Schwarz et al., 2008) uses a *CTMC Continuous Times Markov Chain*, which yields the reliability expression that is used to determine the probability of data loss at the end of its economic lifespan, say five years. We have the following transitions

$$\mathcal{S}_i \rightarrow \mathcal{S}_{i+1}, \text{ with failure rate } (8-i)\delta, 0 \leq i \leq 3$$

$$\mathcal{S}_{i+1} \rightarrow \mathcal{S}_i, \text{ with repair rate } i\mu, 0 \leq i \leq 2.$$

Finally, $(1/5)^{th}$ of failures with rate 5δ from $\mathcal{S}_3$ lead to data loss ($\mathcal{S}_{DL}$) and $(4/5)^{th}$ to $\mathcal{S}_4$. There are two transitions from $\mathcal{S}_4$: with rate 4δ to $\mathcal{S}_{DL}$ and 4μ to $\mathcal{S}_3$. In other words SSPiRAL(4+4,3) tolerates up to three failed disks. Eighty percent of four disk failures are tolerated for $N = 8$, so that $v_4 = 4/5$ and $A(8, 4) = (4/5)\binom{8}{4} = 56$.

The analysis has the following shortcomings:

- The repair rate is set proportional to the number of failed disks and does not take into account potential hardware bottlenecks, such as the effect of the limited bandwidth of the interconnect on concurrent data transfers.
- Conversely with sufficient bandwidth disk rebuilds can proceed concurrently.
- An infinite supply of spare disks is postulated.
- Repair rates are exponentially distributed and as a new disk fails due to the memoryless property of this distribution it is as if the rebuild process at all disks under repair is restarted.
- The analysis does not take into account LSEs, which are the main cause of rebuild failure in disk arrays (Iliadis et al., 2011).

The reliability of RAID(4 + k), $k \geq 1$ arrays with N disks, which can tolerate up to k disk failures is:

$$R_{RAID(4+k)}(N) = \sum_{i=0}^k \binom{N}{i} r^{N-i} (1-r)^i. \quad (4.29)$$

In Fig. 4.13 we plot the reliabilities for various RAID1 organizations and RAID5/6/7 and LSI RAID for $N = 8$ disks versus time normalized with respect to MTTF. For small values of t , RAID7 which tolerates all three disk failures has the highest reliability, while LSI Logic RAID is second best since it tolerates half of three disk failures. For smaller values of t RAID6 is more reliable than BM, which is the most reliable of mirrored disk organizations, but there is a crossover point since BM can tolerate more than two disk failures. RAID5 has the lowest reliability since it can only tolerate single disk failures.

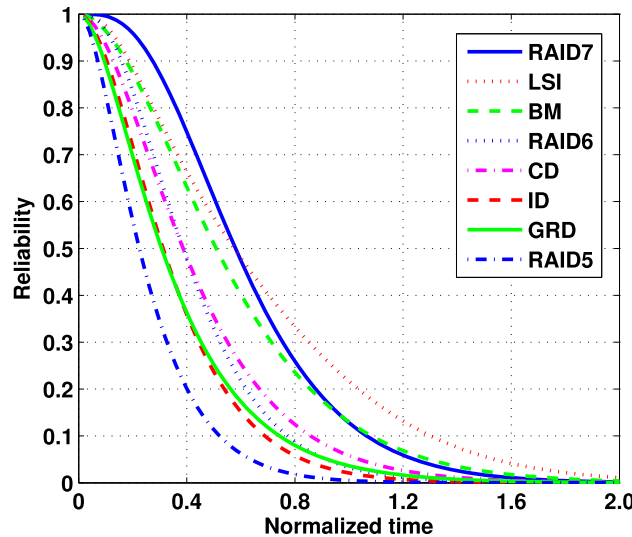


FIGURE 4.13 Reliability versus time normalized with respect to disk MTTF (Thomasian and Tang, 2012).

The MTDDL of RAID systems with N disks with no repair can be obtained by noting that the mean holding time in state $\mathcal{S}_i$ with i disk failures is $H_i = [(N-i)\delta]^{-1}$, which is the inverse of the exponential failure rate at that state. Note that

in a system without repair all states are visited once $v_i = 1, \forall i$:

$$MTTDL_{RAID(N)} = \sum_{i=0}^M v_i H_i = \sum_{i=0}^M \frac{v_i}{(N-i)\delta}. \quad (4.30)$$

A better measure than the MTTDL in RAID systems is its reliability at the end of its useful lifetime: t_u : $R_{RAID}(t_u)$. To justify this point consider a single system with failure rate $r(t) = e^{-\delta t}$ with $MTTF_{simplex} = 1/\delta$, while in the case of a *Triple Modular Redundancy - TMR* system, where at least 2-out-of-3 components are required for the correct operation of the system:

$$R_{TMR}(t) = r^3(t) + 3r^2(t)(1-r(t)) = 3r^2(t) - 2r^3(t),$$

so that $MTTF_{TMR} = 5/(6\delta)$. Setting $R_{TMR}(t) = R_{simplex}(t)$ yields the crossover point $t_c = \ln 2/\delta \approx 0.693/\delta$, and $R_{TMR}(t) > R_{simplex}(t)$ for $t < t_c$.

TABLE 4.5 Rows two and three provide the MTTDL for $N = 8$ disks with $MTTF = \delta^{-1}$, where δ is the failure rate. The second row has the MTTDLs as a ratio and the third row the MTTDL as a fraction. The first term in the approximate reliability expressions in Section 4.12.5, where ε is the unreliability.

RAID5	BM	CD	GRD	ID	RAID6	LSI	RAID7	SSP	R8
$\frac{15}{56\delta}$	$\frac{163}{280\delta}$	$\frac{379}{840\delta}$	$\frac{3}{8\delta}$	$\frac{61}{168\delta}$	$\frac{73}{168\delta}$	$\frac{521}{840\delta}$	$\frac{638}{840\delta}$	$\frac{701}{840\delta}$	$\frac{743}{840\delta}$
$0.268\delta^{-1}$	$0.582\delta^{-1}$	$0.451\delta^{-1}$	$0.375\delta^{-1}$	$0.363\delta^{-1}$	$0.435\delta^{-1}$	$0.620\delta^{-1}$	$7595\delta^{-1}$	$0.8345\delta^{-1}$	$0.8848\delta^{-1}$
$\binom{N}{2}\varepsilon^2$	$\frac{N\varepsilon^2}{2}$	$N\varepsilon^2$	$\frac{N(N-1)\varepsilon^2}{4}$	$\frac{N(N-c)\varepsilon^2}{2c}$	$\binom{N}{3}\varepsilon^3$	$\frac{N}{2}\varepsilon^3$	$\binom{N}{4}\varepsilon^4$	$\frac{1}{5}\binom{8}{4}\varepsilon^5$	$\binom{N}{5}\varepsilon^5$

MTTDLs for RAID1, RAID5-8, LSI, SSPiRAL and Weaver codes are given in Table 4.5. RAID5 arrays are less reliable than mirrored disks, since they cannot tolerate more than one disk failure. Mirrored disks in increasing order of increasing MTTDL are ID, GRD, CD, BM. LSI Logic RAID is more reliable than RAID6 because it tolerates all two disk failures, but also one half of three disk failures. SSPiRAL(4+4,3) tolerates all three disk failures and most four disk failures, which includes all Ddisk failures. Weaver(8,3,3) is less reliable than RAID8, since it does not tolerate all four disk failures.

4.12.4 Performability analysis

The performability measure combines the failure process with performance (Haverkort et al., 2001). Performability is defined in (Thomasian and Xu, 2008) as the number of I/O requests that are processed by a fully utilized disk array from the beginning of its operation (with no failed disks) to the point that data loss occurs. Disk repairs were not considered in this study. Performability is obtained by summing the product fraction of visits to states with i disk failures (v_i) and the durations of the interval $(1/[(N-i)\delta])$ times and the maximum throughput during that interval denoted by $T_{max}^i = (N-i)/\bar{x}_d$, where $\bar{x}_d$ is the mean disk service time:

$$\mathcal{P} = \sum_{i=0}^I \frac{v_i T_{max}^i}{(N-i)\delta}. \quad (4.31)$$

Note similarity to Eq. (4.30) to compute the MTTDL. The performability of various RAID organizations is compared in (Thomasian and Xu, 2008). A similar approach was used in (Thomasian and Tang, 2011a) to compute performability in *Hierarchical RAID - HRAID* described in Chapter 11.

4.12.5 Shortcut Reliability Analysis - SRA

SRA was introduced in (Thomasian, 2006c) by expressing the reliability of a single component (disk) as $r = 1 - \varepsilon$, where $\varepsilon \ll 1$. Assuming that the disk failures follow an exponential distribution with $MTTF = 10^6$ hours or 114 years, after three 3 years the reliability drops to $R(3) \approx 1 - 3/114 = 0.975$, so that $\varepsilon = 0.025$. Note that the exponential distribution is not required for this analysis.

The reliability of n -way replication which allows at most $n - 1$ disk failures is given as:

$$R_{n-way}(n) = 1 - (1 - r)^n \approx 1 - \varepsilon^n.$$

For the BM configuration we utilize the expression for $A(N, i)$ given by Eq. (4.24), but only retain the ε^2 term, which is sufficient for comparing reliabilities.²

$$R_{BM}(N) \approx r^N + Nyr^{N-1}(1-r) + 4\binom{N/2}{2}r^{N-2}(1-r)^2 \approx 1 - \frac{N}{2}\varepsilon^2. \quad (4.32)$$

For GRD or RAID0/1 instead of using $A(N, i)$ in Eq. (4.25), we consider an alternative method. Each of two mirrored virtual disks consists of M disks and $R_M = r^M$, so that:

$$R_{GRD}(N) = 1 - (1 - R_M)^2 = 2r^M - r^{2M} \approx 1 - \frac{N(N-1)}{4}\varepsilon^2. \quad (4.33)$$

In the case of ID we use the expression for $A(N, i)$ given by Eq. (4.26).

$$\begin{aligned} R_{ID}(N) &= r^N + Nyr^{N-1}(1-r) + \frac{c(c-1)}{2}n^2r^{N-2}(1-r)^2 \\ &\approx 1 - \frac{(N(N-1) - c(c-1)n^2)}{2}\varepsilon^2. \end{aligned} \quad (4.34)$$

For $c = N/2$, ID is equivalent to BM.

In the case of CD the expression for $A(N, i)$ is given by Eq. (4.27), but it is sufficient to argue that out of $\binom{N}{2}$ two disk failures, there are N consecutive two disk failures that lead to data loss, so that $A(N, 2) = N(N-1)/2 - N = N(N-3)/2$.

$$R_{CD}(N) \approx r^N + Nr^{N-1}(1-r) + \frac{N(N-3)}{2}r^{N-2}(1-r)^2 \approx 1 - N\varepsilon^2. \quad (4.35)$$

Note that BM is the most reliable among the four mirrored disk organizations followed by CD, while ID is more reliable than GRD since $N(2/c - 1) < 1$.

LSI Logic RAID tolerates all single and double disk failures: $A(N, i) = \binom{N}{i}$, $0 \leq i \leq 2$, but only half of three adjacent disk failures, so that $A(N, 3) = \binom{N}{3} - N/2$. We have:

$$R_{LSI}(N) \approx 1 - \frac{N}{2}\varepsilon^3. \quad (4.36)$$

Based on the previous discussion for SSPIRAL disk array:

$$R_{SSP}(8) \approx 1 - 14\varepsilon^5. \quad (4.37)$$

The reliability expression for k DFT RAID($k+4$), $k \geq 1$ MDS arrays is given in (Thomasian et al., 2012) as:

$$R_{RAID(4+k)} \approx 1 - \binom{N}{k+1}\varepsilon^{k+1} + (k+1)\binom{N}{k+2}\varepsilon^{k+2} - \dots, \quad (4.38)$$

but only the first (negative) term is required in approximations. In what follows we set $N = 8$, which yield:

$$R_{RAID5}(8) = 1 - 28\varepsilon^2, \quad R_{RAID6}(8) = 1 - 56\varepsilon^3, \quad R_{RAID7}(8) = 1 - 70\varepsilon^4, \quad R_{RAID8}(8) = 1 - 56\varepsilon^5.$$

The following conclusions can be drawn:

1. RAID5 is less reliable than the four RAID1 organizations, since RAID1/0 arrays can tolerate up to $M = N/2$ disk failures, as long as they are not mirrors.
2. LSI LOGIC RAID is more reliable than RAID6, since the latter in addition to all two disk failures tolerates half of three disk failures. RAID7 is more reliable than LSI RAID since it tolerates all three disk failures.
3. RAID8 is more reliable than SSPIRAL(4+4,3), since the latter cannot tolerate $1/5^{th}$ of four disk failures.

The above discussion is summarized in Table 4.5.

We next consider a disk array with two nested reliability levels, RAID level 1 and 5, and $N = 2M$ disks. The approximate reliability of a RAID5/1 and RAID1/5 array can be determined by substituting the expression for RAID5 into the reliability expression for RAID1, and vice-versa.

$$R_{R1} = 2R_{R5} - R_{R5}^2, \quad R_{R5} = R_{R1}^M + MR_{R1}^{M-1}(1 - R_{R1}), \quad R_{5,1} \approx 1 - \frac{1}{2}M(M-1)\varepsilon^4.$$

2. We have corrected the first part of Eq. (7) in (Thomasian, 2006c).

$$R_{R5} = R_{R1}^M + M R_{R1}^{M-1} (1 - R_{R1}), R_{R1} = 1 - (1 - R_{R5})^2, R_{1/5} \approx 1 - \frac{1}{4} M^2 (M - 1)^2 \epsilon^4.$$

Although both arrays may fail due to four disk failures, it follows that RAID5/1 with mirroring at the higher level is more reliable than RAID1/5 which is a RAID5 consisting of mirrored disks. This is because any two disk failures on both sides of RAID5/1 will result in data loss, while two specific RAID1 arrays should fail in the case of RAID1/5. The analysis for RAID6/1 and RAID1/6 is reported in (Thomasian, 2006c).

4.12.6 Transient solutions of Continuous Time Markov Chains - CTMCs

We follow (de Souza e Silva and Gail, 2000) in specifying methods to obtain the transient analysis of the behavior of fault-tolerant systems. This analysis is also presented in (Trivedi and Bobbio, 2017).

We are concerned with a CTMC $X = X(t)$, $t \geq 0$ with finite state space $\mathcal{S} = \{s_i : i = 1, \dots, M\}$ and infinitesimal generator

$$Q = \begin{pmatrix} -q_1 & q_{1,2} & \dots & q_{1,M} \\ q_{2,1} & -q_2 & \dots & q_{2,M} \\ \vdots & \vdots & \ddots & \vdots \\ q_{M,1} & q_{M,2} & \dots & -q_M \end{pmatrix}$$

where $q_i = -\sum_{j \neq i} q_{i,j}$. The transition function is an $M \times M$ matrix $\Pi(t)$, which satisfies:

$$\Pi(t)_{i,j} = P\{x(t' + t) = s_j | X(x') = s - I\}$$

and both backward and forward Kolmogorov equations

$$\Pi'(t) = Q\Pi(t), \quad \Pi'(t) = \Pi(t)Q$$

The vector of state probabilities $\pi(t) = [\pi_1(t) \dots \pi_M(t)]$ can be computed from the transition function:

$$\pi(t) = \pi(0)\Pi(t)$$

Several solution methods are outlined in the paper.

(1) Solution based on Ordinary Differential Equations - ODEs

Given the ODE $y'(t) = f(t, y)$ the simplest approximation technique for obtaining a solution to an ODE is Euler's method. The observation interval $(0, t)$ is divided into subintervals of length h , and then an approximate value of $y((n + 1)h)$ is obtained from $y(nh)$ by approximating the derivative at the point $(y(nh), nh)$ with the difference quotient $[y((n + 1)h) - y(nh)]/h$. Given

$$\Pi(t) = [I + Qh]\Pi(t - h),$$

where $\Pi(0) = I$ and the matrix $\Pi(t)$ can be obtained recursively from the above equation.

Using Taylor series expansion:

$$\Pi(t + h) = [I + hQ + \frac{h^2}{2!}Q^2 + \frac{h^3}{3!}Q^3 + \dots]\Pi(t)$$

The usual Runge-Kutta method can be used to solve this equation.

(2) Solutions based on the exponentials of a matrix

$$\Pi(t) = e^{Qt} \text{ where } e^{Qt} = \sum_{n=0}^{\infty} \frac{(Qt)^n}{n!}$$

Several methods to compute the exponentials of a matrix are outlined in the paper.

(3) Laplace transform method

Given that $\Pi^*(s)$ is the Laplace Transform of $\Pi(t)$ we have

$$(sI - Q)\Pi^*(s) = I.$$

$$\Pi^*(s) = \frac{1}{s} \left(I - \frac{Q}{s} \right)^{-1} = \sum_{n=0}^{\infty} \frac{Q^n}{s^{n+1}}.$$

$$\Pi(t) = \sum_{n=0}^{\infty} \frac{(Qt)^n}{n!} = e^{Qt}.$$

(4) **Krylov subspace method**

(5) **Randomization or Uniformization**

This method was introduced in (Grassmann, 1991).

[https://en.wikipedia.org/wiki/Uniformization_\(probability_theory\)](https://en.wikipedia.org/wiki/Uniformization_(probability_theory))

For a CTMC with transition rate matrix Q , the uniformized *Discrete Time Markov Chain - DTMC* transition matrix P is defined as

$$p_{i,j} = \begin{cases} q_{i,j}/\gamma & \text{if } i \neq j \\ (1 - \sum_{j \neq i} q_{i,j})/\gamma & \text{if } i = j. \end{cases}$$

The γ uniform rate parameter is chosen such that $\gamma \geq \max_i |q_{i,i}|$. In matrix notation $P = I + \frac{1}{\gamma}Q$.

For a starting distribution $\pi(0)$, the distribution at time t , $\pi(t)$ is computed by (Reibman and Trivedi, 1988; Reibman et al., 1989).

$$\pi(t) = \pi(0) \sum_{n=0}^{\infty} P^n \frac{(\gamma t)^n}{n!} e^{-\gamma t}$$

In practice this series is terminated after finitely many terms.

Several other issues discussed in the article are stiff models, infinite state space models, computing the mean and higher moments, distribution, approximations, application examples.

4.12.7 RAID reliability analysis with repair

In what follows we specify the analysis of the RAID5 failure process based on (Gibson, 1992). The operation of a RAID5 array with $N + 1$ disks can be represented by three states based on the number (i) of failed disks: $\mathcal{S}_i$, $0 \leq i \leq 2$. If the disk failure rate is δ and the repair rate μ then we have the following transitions.

$$\mathcal{S}_0 \xrightarrow{(N+1)\delta} \mathcal{S}_1 \quad \mathcal{S}_1 \xrightarrow{N\delta} \mathcal{S}_2 \quad \mathcal{S}_1 \xrightarrow{\mu} \mathcal{S}_0.$$

The system can be described by the following set of linear equations:

$$\frac{dP_0(t)}{dt} = -(N+1)\delta P_0(t) + \mu P_1(t).$$

$$\frac{dP_1(t)}{dt} = (N+1)\delta P_0(t) - (\delta + \mu)P_1(t),$$

$$\frac{dP_2(t)}{dt} = N\delta P_1(t).$$

State probabilities can be obtained by obtaining the eigenvalues as specified in Section 4.12.8 or using Laplace-Transform methods. Both lead to:

$$P_0(t) = \frac{(N\delta + \mu + \zeta)e^{\zeta t} - (N\delta + \mu + \eta)e^{\eta t}}{\zeta - \eta}$$

$$P_1(t) = \frac{(N+1)\delta}{\zeta - \eta} (e^{\zeta t} - e^{\eta t})$$

$$\begin{aligned}
P_2(t) &= 1 - P_0(t) - P_1(t) \\
\zeta, \eta &= \frac{1}{2}[-((2N+1)\delta + \mu) \pm \sqrt{\lambda^2 + \mu^2 + 2(2N+1)\delta\mu}], \\
R(t) &= P_0(t) + P_1(t) = 1 - P_2(t) = \frac{\zeta e^{\eta t} - \eta e^{\zeta t}}{\zeta - \eta}, \\
\text{MTTDL} &= \int_{t=0}^{\infty} R(t)dt = \frac{(2N+1)\delta + \mu}{N(N+1)\delta^2} \approx \frac{\mu}{N(N+1)\delta^2} = \frac{MTTF^2}{N(N+1)MTTR}. \quad (4.39)
\end{aligned}$$

4.12.8 Solving linear Ordinary Differential Equations - ODEs

$$\frac{d\mathbf{x}}{dt} = \mathbf{A}\mathbf{x}$$

(i) Compute the eigenvalues and eigenvectors of the coefficient matrix $\mathbf{A}$

$$\mathbf{A} = \begin{pmatrix} a_{1,1} & \dots & a_{1,n} \\ a_{2,1} & \dots & a_{2,n} \\ \vdots & \ddots & \vdots \\ a_{n,1} & \dots & a_{n,n} \end{pmatrix}$$

(ii) Use the eigenvalues and eigenvectors of $\mathbf{A}$ to construct the diagonal matrix $\mathbf{D}$ and change of basis matrix $\mathbf{C}$

$$\mathbf{D} = \mathbf{C}^{-1}\mathbf{A}\mathbf{C} \Leftrightarrow \mathbf{A} = \mathbf{C}\mathbf{D}\mathbf{C}^{-1}$$

$$\mathbf{D} = \begin{pmatrix} \lambda_1 & 0 & \dots & 0 \\ 0 & \lambda_2 & \dots & 0 \\ \vdots & \ddots & \ddots & \vdots \\ 0 & \dots & \dots & \lambda_n \end{pmatrix}$$

$$\frac{d\mathbf{z}}{dt} = \mathbf{D}\mathbf{z} \Rightarrow \mathbf{z} = \begin{pmatrix} c_1 e^{\lambda_1 t} \\ \vdots \\ c_n e^{\lambda_n t} \end{pmatrix}$$

(iv) The solution of the original system will be

$$\mathbf{x} = \mathbf{C}\mathbf{z}$$

The general solution of the following system of differential equations.

$$\frac{dx_1}{dt} = x_1 + x_2.$$

$$\frac{dx_2}{dt} = 4x_1 + x_2.$$

The matrix formulation of the problem

$$\begin{pmatrix} \frac{dx_1}{dt} \\ \frac{dx_2}{dt} \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 4 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$$

The eigenvalues of $\mathbf{A}$ can be found as follows

$$\det(\mathbf{A} - \lambda \mathbf{I}) = 0, \quad \lambda^2 - 2\lambda - 3 = 0 \Rightarrow \lambda = -1, 3$$

The eigenvalues and eigenvectors of $\mathbf{A}$ are as follows

$$\mathbf{C} = \begin{pmatrix} 1 & 1 \\ 2 & -2 \end{pmatrix} \quad \mathbf{D} = \begin{pmatrix} 3 & 0 \\ 0 & -1 \end{pmatrix}$$

The general solution of $\frac{dy}{dt} = \mathbf{A}y$ is:

$$\mathbf{y} = \mathbf{C}\mathbf{z} = \begin{pmatrix} c_1 e^{3t} + c_2 e^{-t} \\ 2c_1 e^{3t} - 2c_2 e^{-t} \end{pmatrix} = c_1 e^{3t} \begin{pmatrix} 1 \\ 2 \end{pmatrix} + c_2 e^{-t} \begin{pmatrix} 1 \\ -2 \end{pmatrix}$$

4.12.9 Simplified for RAID5 rebuild reliability analysis

A shortcut method to obtain the RAID5 MTDDL with N disks with failure rate δ and repair rate μ provides insight into the rebuild process. Operation of the systems is specified by a 3-state CTMC:

$\mathcal{S}_N$: no failed disks, $\mathcal{S}_{N-1}$: single failed disk, $\mathcal{S}_{N-2}$: two failed disks and data loss.

The transition rates are

$$\mathcal{S}_N \rightarrow \mathcal{S}_{N-1}: N\delta, \quad \mathcal{S}_{N-1} \rightarrow \mathcal{S}_{N-2}: (N-1)\delta, \quad \mathcal{S}_{N-1} \rightarrow \mathcal{S}_N: \mu.$$

The time to transition from $\mathcal{S}_N$ to $\mathcal{S}_{N-2}$ is the MTDDL.

At CTMC state $\mathcal{S}_{N-1}$ there are two competing transitions with exponential rates: rebuild is successful when the repair process finishes before another disk fails, otherwise there is data loss. The probability of a successful rebuild is the ratio of transition rates $p_s = \mu/(\mu + (N-1)\delta)$ (Kleinrock, 1975). The distribution of successful rebuilds is $P_{succ} = (1 - p_s)p_s^k$, $k \geq 0$, which has a mean

$$v_N = \frac{p_s}{1 - p_s} = \frac{\mu}{N\delta} = \frac{MTTF}{(N-1)MTTR}.$$

The MTDDL is dominated by the time spent at $\mathcal{S}_N$, which is exponentially distributed with mean holding time $\mathcal{H}_N = MTTF/N$, while the time spent in $\mathcal{S}_{N-1}$ is small, since $\mu \gg (N-1)\lambda$. Multiplication by v_N yields the MTDDL.

$$MTDDL_{RAID5} \approx v_N \mathcal{H}_N = \frac{\mu}{N(N-1)\lambda^2} = \frac{MTTF^2}{N(N-1)MTTR}. \quad (4.40)$$

For $N = 2$ we have the MTDDL for RAID1.

The analysis in (Chen et al., 1994b), referred as the Chen model, provides the following MTDDL for RAID6 arrays:

$$MTDDL_{RAID6} \approx \frac{MTTF^3}{N(N-1)(N-2)MTTR^2}, \quad (4.41)$$

which assumes that the repair-rate with one or two disk failures is μ and this transition leads directly to a fault-free state. If RAID(4 + k) tolerates k failures a generalization of Chen's formulas yields:

$$MTDDL_{RAID(4+k)}^C = \frac{MTTF^{k+1}(k-1)!}{N!MTTR^k} \quad (4.42)$$

The Chen model in fact computes the time to first failure, while the Angus reliability model continues repair after data loss occurs (Angus, 1998). Another difference of the Angus model is that the repair rates are set proportional to the number of failed components (disks). In fact after data loss occurs a completely different and recovery process using backup tapes

may be required.

$$\text{MTTDL}_{RAID(4+k)}^A \approx \frac{\text{MTTF}^{N-k+1}}{k \binom{N}{k} \text{MTTR}^{N-k}} \sum_{i=0}^k \binom{N}{i} \left(\frac{\text{MTTR}}{\text{MTTF}} \right)^i \approx \frac{\text{MTTF}}{(N-k) \times \binom{N}{k}} \left(\frac{\text{MTTF}}{\text{MTTR}} \right)^{N-k}. \quad (4.43)$$

Given that $\text{MTTR} \ll \text{MTTF}$ the summation is equal to one. It is easy to see that the $\text{MTTDL}_A / \text{MTTDL}_C \approx k!$.

A more realistic MTTDL can be obtained by allowing a few spares. In a RAID5 disk array with N disks, one of which is a spare, we have four states $\mathcal{S}_{N-i}$, $0 \leq i \leq 3$, where $\mathcal{S}_{N-2}$ designates a successful rebuild on the spare disk and $\mathcal{S}_{N-3}$ is the failed state. Given $v_N = v_{N-1} = 1$ and the transition probabilities: $\mathcal{S}_{N-1} \rightarrow \mathcal{S}_{N-2}$: $p_s = \mu / (\mu + (N-1)\delta)$ and we have $v_{N-2} = p$, so that the MTTDL is:

$$\text{MTTDL}_1^{(1)} = \frac{1}{N\delta} + \frac{1}{(N-1)\delta} + \frac{p}{(N-2)\delta} \approx \frac{2+p}{N\delta}, \quad (4.44)$$

where the superscript: (1) denotes a single spare disk. The MTTDL with j spares is:

$$\text{MTTDL}_{RAID5}^{(k)} = \frac{1}{N\delta} + \sum_{i=0}^j \frac{p^i}{(N-1-i)\delta}.$$

The contribution to MTTDL decreases with increasing k since $p < 1$. The limitation on the number of spare disks is not a problem if additional spare disks are reordered as the supply is depleted (see Fig. 5.22 in (Gibson, 1992)).

Early analyses of RAID reliability with repair dealt with whole disk failures, as reported in (Gibson, 1992). These analyses were later extended to include the effect of *Latent Sector Errors - LSEs*, but also the effectiveness of *IntraDisk Redundancy - IDR* and disk scrubbing to mask and eliminate LSEs by Ilias Iliadis et al. at IBM's *Zurich Research Lab - ZRL* (Iliadis et al., 2011). Adjusting the disk failure rate has been used to take account these complications. The analysis by Ajay Dholakia et al. in (Dholakia et al., 2008), which takes into account LSEs yields:

$$\text{MTTDL}_{RAID5} \approx \frac{(2N-1)\delta + \mu}{N\lambda[(N-1)\lambda + \mu P_{uf}]} \quad (4.45)$$

which extends the analysis of (Gibson, 1992) by multiplying μ with P_{uf} , which is the probability of an uncorrectable disk failure.

The direct path approximation method investigated at IBM ZRL in (Iliadis and Venkatesan, 2015) is used to estimate the MTTDL in some interesting cases, such as mirrored RAID5's or RAID5/1:

$$\text{MTTDL}_{R5/1} \approx \frac{\mu^3}{3M(M-1)\delta^4} = \frac{\text{MTTF}^4}{3M(M-1)\text{MTTR}^3} \quad (4.46)$$

This method is applicable to Weibull and Gamma distributions and has been extended to multiple shortest paths.

Using Eq. (4.40) a heuristic hierarchical reliability modeling approach was proposed in (Xin et al., 2003) to estimate the MTTDL in RAID5/1.

$$\text{MTTDL}_{R1} \approx \frac{\text{MTTDL}_{R5}^2}{2\text{MTTR}}, \quad (4.47)$$

$$\text{MTTDL}_{R5} \approx \frac{\text{MTTF}_{disk}^2}{M(M-1)\text{MTTR}}, \quad (4.48)$$

$$\text{MTTDL}_{R5/1} \approx \frac{\text{MTTF}^4}{4M(M-1)\text{MTTR}^3}. \quad (4.49)$$

4.12.10 Enhanced reliability modeling of storage systems

This study was conducted at NetApp by Jon G. Elerath and Michael Pecht at U. Maryland (Elerath and Pecht, 2007). It is claimed that the model corrects errors associated with the common assumption that system failure times to failure follow a homogeneous Poisson process. Separate generalized failure distributions are used to model catastrophic failures and usage

TABLE 4.6 Range of average Read Error Rates - RER, RR - Read Rate, Low RR 1.35×10^9 , High RR 1.35×10^{10} , EPH - Error Per Hour.

	Error rate/byte	Low RR	High RR
Low	8×10^{-15}	1.08×10^{-5} EPH	1.085×10^{-40} EPH
Medium	8.0×10^{-14}	1.08×10^{-4} EPH	1.08×10^{-3} EPH
High	3.2×10^{-13}	4.32×10^{-4}	4.32×10^{-3}

dependent data corruptions for HDDs. Catastrophic failure restoration is represented by a three-parameter Weibull, so the model can include a minimum time to restore as a function of data transfer rate and HDD storage capacity. Data can be scrubbed as a background operation to eliminate corrupted data that, in the event of a simultaneous catastrophic failure, results in double disk failures. Field-based times to failure data and mathematic justification for a new model are presented. Model results have been verified and predict between 2 to 1,500 times as many *Double Disk Failures - DDFs* as that estimated using the current MTDDL method.

It is pointed out that while disk failure rates and repair rates are exponentially distributed in a RAID5 disk array with $N + 1$ disks, analysis yields

$$MTDDL = \frac{(2N + 1)\lambda + \mu}{N(N + 1)\lambda^2} \approx \frac{\mu}{N(N + 1)\lambda^2} = \frac{MTTF_{disk}^2}{N(N + 1)MTTR_{disk}}$$

The four component-related distributions required for this model are (1) time to operational failure, (2) time to restore such a failure, (3) time to generation of a latent defect, (4) time to scrub HDDs for latent defects. The simulations in this paper use a three-parameter Weibull distribution with the following density function:

$$F(t) = \frac{\beta}{\eta} \left(\frac{t}{\eta}\right)^{\beta-1} e^{-\left(\frac{t}{\eta}\right)^\beta},$$

where γ is the location parameter, η is the characteristic life, and β the shape parameter.

Time to operational failure: A Weibull failure distribution with a slightly increasing failure rate is used with $\beta = 1.12$ and $\eta = 461,386$ hours. These parameters are from a field population of over 120,000 HDDs that operated for up to 6,000 hours each.

Time to restore model recognizes that there is a minimum time before which the probability of being fully restored is zero. *Fibre Channel - FC* HDDs can sustain up to 100 MB/s data transfer rates, although 50 MB/sec is more common. The data-bus to which the RAID group is attached has only a 2 GB/s capability. Thus, in a RAID group of 14, a 144 GB HDD on an FC interface will require a minimum of three hours with no other I/O to reconstruct the failed HDD.

A 500 GB, Serial ATA HDD on a 1.5 Gb data-bus will require 10.4 hours to read all other HDDs and reconstruct a replaced HDD. The added I/O associated with continuing to serve data will lengthen the time to restore an operational failure. Some operating systems place a limit on the amount of I/O that takes place during reconstruction, thereby assuring reconstruction will complete in a prescribed amount of time. This results in a maximum reconstruction time. The minimum time of six hours is used for the location parameter. The shape parameter of 2 generates a right-skewed distribution, and the characteristic life is 12 hours.

Time to Latent defect - TTLd NetApp completed a study in late 2004 on 282,000 HDDs used in RAID. The read error rate (RER), averaged over three months, was 8×10^{-14} errors per byte read. At the same time, another analysis of 66,800 HDDs showed a RER $\approx 3.2 \times 10^{-13}$ errors per byte. A more recent analysis of 63,000 HDDs over five months showed a much improved 8×10^{-15} errors per byte read. Refer to Table 4.6.

Time to Scrub - TTScrub: Latent data corruptions can occur any time the disks are spinning, but can be eliminated by “background scrubbing.” During scrubbing data is read and compared to the parity. If they are consistent, no action is taken. If they are inconsistent, the corrupted data is recovered and rewritten to the HDD. If the media is defective, the recovered data is written to new physical sectors on the HDD and the bad blocks are mapped out. Scrubbing is a background activity performed on an as-possible basis so it does not affect performance. If not scrubbed, the period of time to accumulate latent defects starts when the HDD first begins operation in the system. The latent defect rate is assumed to be constant with respect to time ($\beta = 1$) and is based on the error generation rate and the hourly data transfer rate. As with full HDD data reconstruction, the time required to scrub an entire HDD is a random variable that depends on the HDD capacity and the amount of foreground activity. The minimum time to cover the entire HDD is based on capacity and foreground I/O.

The operating system may invoke a maximum time to complete scrubbing. The shape parameter $\beta = 3$, which produces a Normal shaped distribution after the delay set by the location parameter, γ .

4.12.11 Metrics for reliability estimation in storage systems

Metrics to estimate storage reliability are compared in (Greenan et al., 2010). The study compares the MTDDL, *Bit Half-Life - BHL* (Rosenthal, 2008, 2010a,b) DDFpKRG - Double-Disk Failures per 1000 Reliability Groups (Elerath and Pecht, 2007), Data Loss events per Petabyte Year (DALOPY) (Hafner and Rao, 2006), and NOMDL_t.

MTDDL and DALOPY are calculated via Markov models. BHL is calculated by finding the time at which a bit has a 0.50 probability of failure, which is difficult to calculate via simulation and can be estimated using a Markov model. For BHL, this time is calculated for the entire system instead of a single bit. DDFpKRG and NOMDL are computed using Monte Carlo simulation.

If the time of data loss is F and the mission time is t , then the simulator implements the function, $I(F < t) = \{0, 1\}$ (0 is no data loss, 1 is data loss). That is, the system either had a data loss event within the mission time or not. Many iterations of the simulator are required to get statistically meaningful results. The discussion is summarized in Table 4.7.

TABLE 4.7 Qualitative comparison of different storage reliability metrics.

	Meaningful	Understandable	Calculable	Comparable	Result
MTDDL			X		37.60 years
BHL					26.06 years
DDF pKRG	X	X	X		183 DDFs
DALOPY	X	X	X		3.32 DL per (PB × Year)
NOMDL _{10y}	X	X	X	X	14.41 B lost per TB

$$R(t) = 1 - \sum_{i=1}^N \frac{I(F_i < t)}{N}$$

4.12.12 Expected Annual Fraction of Data Loss - EAFDL

Defined in (Iliadis and Venkatesan, 2014) EAFDL is the fraction of stored data that is expected to be lost by the system annually. A general methodology to obtain the EAFDL metric analytically, in conjunction with the MTDDL metric for various redundancy schemes and for a large class real-world failure time distributions such as Weibull and gamma is provided.

The reliability of a replication-based storage system under clustered, declustered, and symmetric data placement schemes. It is shown that the declustered placement scheme offers superior reliability in terms of both metrics. Previous work has used simulation to evaluate the magnitude of data loss, but this is the first work to analytically assess it, and the first to present a general theoretical framework for this purpose.

TABLE 4.8 Notation of system parameters in (Iliadis and Venkatesan, 2014).

Parameter	Definition
n	number of storage devices
c	amount of data stored in each device
r	replication factor
k	spread factor of the data placement scheme
b	reserved rebuild bandwidth per device
$1/\delta$	mean time to failure of a storage device
$U = nc/r$	amount of user data stored in the systems
$1/\mu = c/b$	time to read c bytes at rate b from a device

The parameters used in this study are given in Table 4.8.

r : Replication factor.

e : Exposure level—maximum number of copies that any data has lost.

T_i : Cycles (operational and repair periods).

P_{DL} : Probability of data loss in repair period.

U : Amount of user data in system.

Q : Amount of data lost upon a first device failure.

$$MTTDL \approx \sum_{i=1}^m E[T_i] \approx \frac{E[T]}{P_{DL}}, \quad EAFDL = \frac{E[Q]}{E[T] \times U}$$

To justify the EAFDL measure consider clustered and declustered data placements. In cluster data placement the k blocks on a device are replicated in another device, while in declustered data placement the k blocks are distributed over k nodes as shown in Fig. 4.14.

n_1	n_2	...	n_{k+1}
D_1	D_1	...	$\vdots$
D_2	D_2	...	$\vdots$
$\vdots$	$\vdots$	$\vdots$	$\vdots$
D_k	$D - k$	...	...

n_1	n_2	...	n_{k+1}
D_1	D_1	...	$\vdots$
D_2	...	D_2	$\vdots$
$\vdots$	$\vdots$	$\vdots$	$\vdots$
D_k	...	...	D_k

FIGURE 4.14 Clustered and declustered data layout with $k + 1$ nodes.

In both cases two node failures will lead to data loss, so that they have the same MTTDL, but two node failures in the declustered case will result in a lower data loss.

$$MTTDL \approx \begin{cases} \left(\frac{b}{\delta c}\right)^{r-1} \frac{1}{n\delta} \\ \left(\frac{b}{2\delta c}\right)^{r-1} \frac{(r-1)!}{n\delta} \prod_{e=1}^{r-2} \left(\frac{n-e}{r-e}\right)^{r-e-1} \end{cases}$$

$$EAFDL \approx \begin{cases} \left(\frac{\delta c}{b}\right)^{r-1} \lambda \\ \left(\frac{2\delta c}{b}\right)^{r-1} \frac{\delta}{(r-1)!} \prod_{e=1}^{r-1} \left(\frac{r-e}{n-e}\right)^{r-e} \end{cases}$$

4.12.13 Taking into account controller failures

Three mirrored disk configurations: C_i , $1 \leq i \leq 3$ are considered in (Ng, 1987).

(a) C_1 has a single controller for duplexed disks.

(b) C_2 has one controller per disk and incoming reads are routed to both disks.

(c) C_3 has two controllers crossconnected to disk.

The MTTDL is determined by a CTMC with three states:

$\mathcal{S}_0$: no failed disks.

$\mathcal{S}_1$: single failed disk.

$\mathcal{S}_2$: both disks or the control unit has failed.

For example, given that disk and controller failure rates are δ_d and δ_c and the disk and controller repair rates are μ_d and μ_c . The transition rates for C_1 are:

$$\mathcal{S}_0 \rightarrow \mathcal{S}_1 : 2\delta_c, \quad \mathcal{S}_1 \rightarrow \mathcal{S}_0 : \mu_c, \quad \mathcal{S}_1 \rightarrow \mathcal{S}_2 : \delta_d + \delta_c, \quad \mathcal{S}_0 \rightarrow \mathcal{S}_2 : \delta_d.$$

This and similar CTMCs can be solved to estimate the MTTDL and the availability, which is the fraction of time data can be accessed.

The *Crosshatch Disk Array - CDA* proposed by Spencer W. Ng at IBM/ARC in (Ng, 1994a) has N^2 disks, which are doubly connected to horizontal and vertical buses attached to $2N$ controllers. Parity groups are defined over diagonal disks, so data loss occurs with:

- (1) Double disk failures,
- (2) single disk plus double controller failures,
- (3) quadruple controller failures.

Reliability analysis is used to show that the MTDDL attained by CDA exceeds the MTDDL attained with less complex organizations.

4.13 Storage reliability research at IBM's Zurich Research Lab

In this section we briefly review recent research at IBM's *Zurich Research Labs - ZRL*. Efficient replica maintenance and placement to attain high availability and data durability has been one of the topics of research. This has led to implementations such as Carbonite, which deal with efficient replica maintenance in distributed systems as discussed in (Chun et al., 2006). <https://www.carbonite.com/>

Given that there are n nodes and user data blocks are replicated r times. Vinodh Venkatesan, Ilias Iliadis et al. have considered three data placements methods in (Venkatesan et al., 2010).

- (a) Declustered: r data replicas are placed randomly at the nodes.
- (b) Clustered: nodes are divided into disjoint sets of r nodes on which data is replicated.
- (c) k-Clustered: the n nodes are divided into disjoint sets of k nodes called clusters.

Each of these clusters is an independent storage system with k nodes with a declustered placement scheme. No data block in one cluster is replicated in another cluster. Analytic and simulation results show that for a replication factor of two all placements have an MTDDL within a factor of two.

Clustered Placement - CP and *Declustered Placement - DP* is considered in (Venkatesan et al., 2011). DP spreads replicas across all other nodes, while a minimum number of nodes are used by CP. The average lifetime of a node is set to be of the order of $\delta^{-1} = 10^5$ hours and given c bytes per node and a rebuild bandwidth b it takes $c/b = 10$ hours to rebuild a node, so that $\delta c/b \ll 1$. Given that the probability that the system experiences data loss is P_{DL} , then

$$MTDDL \approx [n\delta P_{DL}]^{-1}.$$

Eq. (4) in (Venkatesan et al., 2011) leads to P_{DL} .

For $r = 2$, $MTDDL_{clus} \approx b/(nc\delta^2)$ and $MTDDL_{declus} \approx b/(2nc\delta^2)$, so that the MTDDL is inversely proportional to the number of nodes.

For $r = 3$, $MTDDL_{clus} \approx b^2/(nc^2\delta^3)$ and $MTDDL_{declus} \approx (n-1)b^2/(4nc^2\delta^3)$, which is almost independent of the number of nodes.

Data storage reliability under rebuild bandwidth constraints is discussed in (Venkatesan et al., 2012). An increased degree of replication increases parallelism for rebuild, but this is so if sufficient bandwidth is available for this purpose.

The effect of CP and DP with two reliability metrics: *Expected Annual Fraction of Data Loss - EAFDL* and MTDDL is investigated in (Iliadis and Venkatesan, 2014). There are n storage devices and $r = 2, 3$ is the replication factor. CP (resp. DP) replicates data at the other $r - 1$ (resp. $n - 1$ devices). Rebuild time is obtained taking into account the disk capacity (c) and the amount of user data $U = nc/r$. The reserved bandwidth per device is b so that time to read a disk is c/b . Given that mean time to device failure is $1/\delta$ for $r = 2$:

$$MTDDL_{CP} = \frac{b}{nc\delta^2} \text{ and } MTDDL_{DP} = \frac{b}{2nc\delta^2}.$$

$$EAFDL_{CP} = \frac{\delta^2 c}{b} \text{ and } EAFDL = \frac{2\delta^2 c(n-1)}{b}.$$

A study comparing erasure coding with replication in the context of peer-to-peer systems built to provide storage durability by taking advantage of network bandwidth, storage capacity, and computational resources is (Weatherspoon and Kubiatowicz, 2002). It is shown that erasure coding provides an MTDDL many orders of magnitude higher than replicated systems, while utilizing an order of magnitude less bandwidth and storage.

4.13.1 Analysis and simulation for reliability estimation

Analytic and simulation software packages developed to assess the reliability, availability, and serviceability of computer systems is surveyed in (Jr. and Malek, 1988). Approaches based on stochastic models, i.e., Markov Chains are emphasized in our discussion. Table 1 in the paper provides the application of the tool, input, models, and model solution methods. This survey is dated and (Trivedi and Bobbio, 2017) provides a more recent discussion.

The wrteip for CARE III reliability modeling package has many examples which also appear in (Trivedi and Geist, 1981; Trivedi, 1982, 2002).

Software reliability models are discussed below re less relevant to our discussion.

https://en.wikipedia.org/wiki/List_of_software_reliability_models

4.13.2 Automated Reliability Interactive Estimation System - ARIES

The ARIES reliability analysis package was developed at UCLA by Ying-Wah (Peter) Ng under the supervision of Algirdas Avizienis (Ng and Avizienis, 1980).

The tool was initially written in APL, which was later translated to C for broader distribution. The following input parameters are used:

N : initial number of modules in the active configuration.

D : number of degradations allowed in the active configuration.

S : number of spare modules.

Ca : coverage for recovery from active module failures.

Cd : coverage for recovery from spare module failures.

λ : failure rate of one active module.

μ : failure rate of one spare module ($\mu = \lambda$ is spare powered).

Y : sequence of allowed degradations of the active configuration.

CY : coverage vector for transitions into degraded configuration.

It is shown in (Ng, 1976) that the reliability function of a closed fault-tolerant system has the form.

$$R(t) = \sum_{\forall i} A_i e^{-\sigma_i t} \quad (4.50)$$

Let i indicate a nonfailed state and σ_i the failure rate at that state. A_i can be expressed a function of state parameters and can be quite complex. The *Mean Time to First Failure - MTFF* easily follows from Eq. (4.50)

$$\text{MTFF} = \int_{t=0}^{\infty} R(t) dt = \sum_{\forall i} \frac{A_i}{\sigma_i}.$$

$R(t)$ has a specific form

$$R(t) = X(t) \mathcal{A} W(t)$$

$$X(t) = (e^{Y[0]\lambda t}, \dots, e^{-Y[D]\lambda t}) \text{ with } Y[0] = N$$

$$W(t) = (1, e^{-\mu t}, \dots, e^{-S\mu t})$$

λ and μ are the failure rate of active and spare components, which are powered up and down, respectively.

$$\mathcal{A} = \begin{pmatrix} A_{S,0}^0 & \dots & A_{S,0}^D \\ \vdots & & \\ A_{S,S}^0 & \dots & A_{S,S}^D \end{pmatrix} \quad (4.51)$$

Two more parameters are required for modeling repairable systems

M = number of repairman or repair facilities.

Ψ = repair rate of one repairman.

4.13.3 System Availability Estimator - SAVE project at IBM Research

Previous reliability modeling effort at IBM Research by Willard Bouricius and Willard Carter appears in (Bouricius et al., 1971). This study defines the coverage parameter denoted by c as the probability that the system can recover after a failure.

The CTMC of duplex systems where the system state is specified as S_i where i is the number of nonfailed processors. Processors fail with rate δ and are repaired with rate μ . We have the following transitions:

$$S_2 \xrightarrow{2\delta c} S_1, S_2 \xrightarrow{2\delta(1-c)} S_0, S_1 \xrightarrow{\mu} S_2$$

SAVE developed at IBM *Thomas J. Watson Research Center - TJWRC* had an analytic component with similarities to ARIES described in (Blum et al., 1994). It is noted in (de Souza e Silva and Gail, 2000) that realistic models of such systems are often not amenable to analysis using conventional analytic or numerical methods (de Souza e Silva and Gail, 2000).

Simulation as an alternative method to estimate availability in the context of the SAVE project was suggested in (Conway and Goyal, 1986). The unified framework for simulating Markovian models of highly dependable systems is described in (Goyal et al., 1992). Fast simulation methods based on importance sampling are relevant to such systems (Mazonka, 2016), since failures are rare events. Efficient simulation techniques for estimating steady-state quantities in models of highly dependable computing systems with general component failure and repair time distributions are considered.

The regenerative method of simulation for steady state estimation can be used when the failure time distributions are exponentially distributed, but when the failure times are generally distributed the regenerative structure is lost and the new approach reported in (Nicola et al., 1993) exploits a ratio representation for steady-state quantities in terms of cycles that are no longer independent and identically distributed. A splitting technique is used in which importance sampling is used to speed up the simulation of rare system failure events during a cycle, and standard simulation is used to estimate the expected cycle length. Experimental results show that the method is effective in practice. Techniques for fast simulation of models of highly dependable systems are reviewed in (Nicola et al., 2001) by Victor Nicola (U. Twente, emeritus), Perwez Shahabuddin (Columbia U., rip), Marvin Nakayama (NJIT) at IBM TJWRC.

The use of simulation to evaluate these models requires that the simulation frequently observes system failures, to accurately estimation of dependability measures of these models. This renders ordinary simulation impractical for evaluating such systems to overcome this problem, simulation techniques based on importance sampling have been developed, to estimate dependability measures efficiently in Markov and non-Markov models of highly dependable systems. When importance sampling works well, simulation run lengths can be reduced by several orders of magnitude when estimating transient as well as steady-state dependability measures.

The following example is provided in Fig. 1 in (Nicola et al., 2001):

Two sets of processors with four processors per set (to 4-way multiprocessors).

Two sets of controllers with two controllers per set.

Four clusters of disks each consisting of four disks.

The multiprocessors are connected to all controllers.

The pair of controllers on the left and right each have two connections to the disk clusters on their side.

In a disk cluster, data are replicated so that one disk can fail without affecting the system (as in RAID5). The “primary” data on a disk are replicated so that 1/3 is on each of the other 3 disks in the same cluster. Thus one disk in each cluster can be inaccessible without losing access to the data, as in the interleaved declustering method.

It is assumed that when a processor of a given type fails it has a 0.01 probability of causing the operating processor of the other type to fail. Each unit in the system has two failure modes which occur with equal probability. The failure rates (per hour) are 1/1000 for processors, 1/20 000 for controllers, and 1/60 000 for disks. The repair rates (per hour) are 1 for all mode 1 failures and 1/2 for all mode 2 failures. The simulation yields the following results: $0.7820 \times 10^{-7} \pm 3.8\%$ for the steady-state unavailability estimate and MTTF of $0.2161 \times 10^8 \pm 7.5\%$.

Symbolic Hierarchical Automated Reliability and Performance Evaluator - SHARPE is a tool developed by Kishor Trivedi et al. at Duke U. (Sahner et al., 1996).

<https://sharpe.pratt.duke.edu/>

Hierarchical modeling is used to evaluate the availability of redundant private cloud in (de S. Matos et al., 2017), where it is noted that in addition to CTMCs other tools for reliability and availability analysis of computer systems are *Reliability Block Diagrams - RBDs*, fault-trees (O'Connor and Kleyner, 2012), *Stochastic Petri Nets - SPNs*, *Markov Reward Models - MRM* (Trivedi and Bobbio, 2017).

A recent survey of approaches for availability estimation in (Endo et al., 2016) leads to the following statistics: experimentation 28, simulation 3, quantitative 1, modeling 0. There are few publications modeling large-scale systems such as storage clouds where simulation plays a larger role.

4.14 Conclusions

RAID1 arrays have a 100% redundancy in disk capacity, which is higher than the $100k/N\%$ for RAID($4 + k$) arrays with N disks, but this redundancy is acceptable for high performance applications in view of doubled bandwidth. RAID1 has two advantages over RAID($4 + k$) arrays:

- (1) Updating small data blocks requires writing the modified block to two disks, while updates in RAID($4 + k$) invoke SWP - small write penalty, where each writes requires $2(k + 1)$ accesses if data was not cached.
- (2) RAID1 doubles disk access bandwidth to small data blocks.
- (3) RAID($4 + k$) allows parallel reading of strips when accessing large chunks of data. This is also possible with RAID1/0 arrays with striping.
- (4) RAID($4 + k$) can efficiently write large files as full stripe writes, where check blocks are computed on the fly as data blocks on consecutive disks are written.

We survey the numerous techniques to improve the processing of read and write requests in RAID1. We consider data and anticipatory arm placement to improve performance. Multiarm disks improve performance without replicating data, which is a new development with multiterabyte disks.

Chapter 5

Redundant Arrays of Independent Disks - RAID

Redundant Arrays of Inexpensive Disks in Section 5.1
RAID prototypes at Berkeley and CMU in Subsection 5.1.1
Early RAID products in Section 5.2
Tandem Corp. in Subsection 5.2.1
Teradata Corp. in Subsection 5.2.2
National Cash Register - NCR Corp. in Subsection 5.2.3
EMC Corp. in Subsection 5.2.4
Sun Microsystems in Subsection 5.2.5
SUN ZFS file system and volume manager in Subsection 5.2.6
International Business Machines - IBM Corp. in Subsection 5.2.7
Storage Technology Corp. - STK in Subsection 5.2.8
Western Digital Corp. - WDC in Subsection 5.2.9
Network Appliance - NetApp Corp. in Subsection 5.2.10
HP storage products in Subsection 5.2.11
RAID classification and motivation in Section 5.3
RAID0 and RAID striping in Section 5.4
RAID2 in Section 5.5
RAID3 in Section 5.6
RAID4 in Section 5.7
RAID5 in Section 5.8
RAID0 and RAID5 stripe unit size in Section 5.8.1
Parity striping and load balancing in Section 5.8.2
Load balancing in disk arrays without striping in Subsection 5.8.3
Performance analysis of RAID5 disk arrays in Section 5.9
RAID(4+k) disk arrays in Subsection 5.10
Rebuild processing in disk arrays in Section 5.11
Variations of rebuild processing in RAID5 in Subsection 5.11.1
Vacationing server model for rebuild processing in Subsection 5.12
Effect of rebuild processing on external request response time in Subsection 5.12.1
Rebuild unit sizes in Subsection 5.12.2
Permanent Customer Model - PCM and comparison with VSM in Subsection 5.12.3
RAID5 sparing configurations for rebuild in Section 5.13
Rebuild by distributed sparing in Subsection 5.13.1
Rebuild by restriping in Subsection 5.13.2
Rebuild by parity sparing in Subsection 5.13.3
IntraDisk Redundancy - IDR for higher reliability rebuild in Section 5.14
Disk scrubbing for higher reliability in rebuild processing in Section 5.15
Disk scrubbing versus intradisk redundancy in Section 5.15.1
Predictive Failure Analysis - PFA in Section 5.16
Undetected disk errors and silent data corruption in Section 5.17
Clustered RAID5 layouts in Section 5.18
Balanced Incomplete Block Designs - BIBDs in Subsection 5.18.1
PRIME data layout in Subsection 5.18.2

Relatively Prime - RELPR data layout in Subsection 5.18.3
 Thorp permutation for load balancing in clustered RAID5 in Subsection 5.18.4
 Clustered RAID with Nearly Random Permutations - NRP in Subsection 5.18.5
 Clustered RAID designs by Walter Burkhard et al. at UCSD in Section 5.19
 Almost Complete Address Translation - ACATS in Subsection 5.19.1
 DATUM for tolerating multiple disk failures in disk arrays in Subsection 5.19.2
 Permutation Development Data Layout - PDDL in Subsection 5.19.3
 Balancing loads by shifted parity group placements in Section 5.19.4
 Log-structured file systems and arrays in Section 5.20
 RAID6 in Section 5.21
 Supplementary Parity Augmentation - SPA in Subsection 5.21.1
 Reed-Solomon coding for higher reliability in Section 5.22
 Reed-Solomon - RS code performance in Subsection 5.22.1
 Parity based MDS codes in Section 5.23
 RDP Arrays and their optimal recovery in Section 5.24
 EVENODD defined and efficient rebuild of a single Disk in Section 5.25
 Blaum-Roth code in Section 5.26
 X-code disk arrays and rebuild with one and two disk failures in Section 5.27
 Minimizing reconstruction overhead in MDS RAID in Section 5.27.1
 Binary MDS array codes with optimal repair in Section 5.27.2
 The RM2 disk array in Subsection 5.28
 RAID7 in Section 5.29
 Erasure coding for distributed storage in Section 5.30
 Pyramid codes in Subsection 5.30.1
 HDFS-Xorbas in Subsection 5.30.2
 Ceph erasure coding with LR Code in Subsection 5.30.3
 Hitchhiker erasure code in Subsection 5.30.4
 Hadoop Adaptively-Coded Distributed File System in Section 5.30.5
 HashTag erasure code - HTEC in Subsection 5.30.6
 ReGenerating codes in Section 5.31.
 Pentagon and heptagon codes in Subsection 5.31.1
 Minimum storage regeneration - MSR Codes in Subsection 5.31.2
 Protection schemes for flash memories in Section 5.32
 Flash protection schemes in Subsection 5.32.1
 Differential RAID for flash memories in Subsection 5.32.2
 Partial MDS - PMDS coding for SSDs in Subsection 5.32.3
 Conclusions in Section 5.33
 Seagate launches self-healing memory in Section 5.33.1

5.1 Redundant Arrays of Inexpensive Disks

The five-level *Redundant Array of Inexpensive/Independent Disks* - RAID classification RAID1:5 was proposed by Garth Gibson (CMU, Vector Inst.) and professors David Patterson (Google) and Randy Katz at *U. California at Berkeley* - UCB (Patterson et al., 1987, 1988). The term Independent replaced Inexpensive on later publications such as (Chen et al., 1994b). “A personal recollection of how storage became a system” was written twenty years later (Katz, 2010) based on a panel discussion on this topic held at the Computer History Museum in 2005 (Katz, 2010).

<https://computerhistory.org/events/how-computer-storage-became-modern/>

Interestingly Fred Kurzweil Jr. published a paper on small disk arrays at CompCon’88 (Kurzweil, 1988).

Gibson’s PhD thesis which received ACM’s best dissertation award in 1991 covers two main topics (Gibson, 1992):

1. estimating the time to disk failure based on broken disk returns to manufacturers,
2. developing Markovian reliability models for several RAID organizations after approximating the time to disk failure with an exponential distribution (Trivedi, 2002).

Gibson and his coauthors of (Patterson et al., 1988) coauthored a paper with Martin Schulze on RAID reliability modeling (Schulze et al., 1989).

Reliability modeling is discussed in Chapter 4.

5.1.1 RAID prototypes at Berkeley and CMU

Researchers at Berkeley developed two prototypes:

RAID I or RAID the First which was evaluated by Ann Chevernak/Drapeau and Randy Katz (GeorgiaTech, Institute-ISI/USC, Aerospace Corp.) and Randy Katz at UCB in (Chervenak and Katz, 1991). RAID II or RAID the Second was evaluated by Peter Chen (U. Michigan) et al. in (Chen et al., 1994a). The two RAID arrays are compared in Table 7 in (Chen et al., 1994b), which is reproduced as Table 5.1.

TABLE 5.1 The performance of RAID II compared to RAID I.

	Disk array read performance	Disk array write performance	Write performance degradation
RAID I	2.4 MB/s	1.2 MB/s	50%
RAID II	20.9 MB/s	18.2 MB/s	13%
RAID II speedup	8.7	15.2	.

Gibson joined *Carnegie-Mellon U. - CMU* in 1991, where he founded the *Parallel Data Laboratory - PDL*, <https://www.pdl.cmu.edu>

but he left CMU in January 2018 to join the Vector Institute in Toronto.

<https://vectorinstitute.ai>

Greg Ganger a former student of Yale Patt (U. Texas) at U. Michigan is the current PDL director.

Gibson developed three prototypes of the Scotch Parallel File system with the following goals (Gibson et al., 1995):

1. to reduce the cost/performance penalty of maintaining redundant data;
2. to develop an extensible software framework for rapid prototyping;
3. to reduce RAID complexity and automate error-handling;
4. to extend file systems to exploit parallel storage;
5. to extend RAID to distributed parallel computing environments.

PDL research has shifted from storage systems towards big data and *Machine Learning - ML*.

<https://www.pdl.cmu.edu/Publications/index>

5.2 Early RAID products

Early RAID products were based on mirroring classified as RAID1 according to the RAID classification Patterson et al. (1988). RAID1 organizations are discussed in Chapter 4, so we only discuss here companies with RAID1 products.

5.2.1 Tandem Corporation

Tandem Corp. was founded in 1974 to build fault-tolerant computers.

https://en.wikipedia.org/wiki/Tandem_Computers

Basic Mirroring - BM was implemented by Tandem starting with NonStop I or T/16 in 1978.

The *Tandem NonStop - TNS* SQL computer was introduced in 1987 (Tandem Database Group, 1989).

https://en.wikipedia.org/wiki/NonStop_SQL

Compaq Corp., a maker of IBM *Personal Computer - PC* compatibles acquired Tandem Corp. in 1997 and *Digital Equipment Corp. - DEC* in 1998. Compaq was acquired by *Hewlett-Packard-HP Corp.* led by Carly Fiorina, who transcended from secretary to CEO, in 2002 for \$25 billion.

<https://www.cnet.com/news/hp-to-acquire-compaq-for-25-billion/>

Tandem computers are still successful products sold by HP and are also discussed in Chapter 4, since they utilized mirrored disks.

<https://www.hpe.com/us/en/servers/nonstop.html>

HP was split into two divisions HP producing printers and personal computers, its own and those made by Compaq with symbol HPQ at *NY Stock Exchange - NYSE*) and *Hewlett-Packard Enterprise - HPE* with symbol HPE at NASDAQ). HPE has Tandem computers in addition to its own servers.

Tandem started with a proprietary microprocessor, followed by DEC's Alpha microprocessor, then HP's Itanium (VLIW processor), and finally Intel x86. Itanium which is discussed in more detail in Chapter 1 has been discontinued. A recent article discusses the HP and HPE is as follows.

<https://www.nextplatform.com/2021/03/03/changing-of-the-guard-for-hpc-and-big-iron-at-hpe/>

5.2.2 Teradata Corp.

Teradata DBC/1012 computer in its 1984 implementation used a new form of mirroring called *Interleaved Declustering - ID*, which is an improvement over BM in that the data on each disk is partitioned and duplicated over multiple disks (Staff, 1985). This results in the load of a failed disk to be distributed across multiple disks.

https://en.wikipedia.org/wiki/DBC_1012

Teradata was acquired by *National Cash Register - NCR Corp.* in 1989, which itself was acquired by *American Telephone and Telegraph - AT&T Corp.* in 1991. Both acquisitions were reversed and AT&T was acquired by Southern Bell of Austin, TX in 2006, which took its name.

<https://www.teradata.com/About-Us/Our-History>

The DBC/1012 RAID1 mirrored disk organization is discussed in Chapter 4 since its uses mirroring and Chapter 8 since it is also a *Data Base Machine - DBM*.

5.2.3 National Cash Register Corp. - NCR

According to (Chen et al., 1994b) an early commercially available RAID was the NCR 6298, which supported RAID levels 0, 1, 3, 5. The system had a higher cost per GB than others and disk formatting for RAID3 configuration took time. Some details are given in a Computerworld article.

<https://books.google.com/books?id=heyWMcI8pQUc&pg=PA89&dq=ncr+6298&hl=en&sa=X&ved=2ahUKEwjW973f9rDIAhUNmuAKHfcuCykQ6AEwAHoECAAQAg#v=onepage&q=ncr%206298&f=false>

5.2.4 EMC Corp.

EMC Corp. was founded by Michael Egan, Roger Marino, and John Curly in 1970.

<https://en.wikipedia.org/wiki/Dell EMC>

EMC Corp. became a successful storage company after it hired Moshe Yanai in 1987,

https://en.wikipedia.org/wiki/Moshe_Yanai

who in 1990 produced the Symmetrix mirrored disk array,

which had the capability to emulate IBM *Extended Count Key Data - ECKD* drives, such as the 3390.

https://en.wikipedia.org/wiki/EMC_Symmetrix.

Symmetrix using the straightforward mirroring paradigm made EMC a multibillion dollar company. Symmetrix was first adopted by risk-taking airline industry to lower operational costs. The Symmetrix's *Disk Array Controller - DAC* could connect via block multiplexor channels to IBM mainframes, which have remained as the main engines for high performance database applications till now

The author gave a talk at EMC at Hopkinton, MA in December 2000 on RAID performance analysis, based on an expanded version of his SIGMETRICS'98 tutorial. The attendees included Israeli engineers, but also engineers who had joined from DEC. Dell merged with EMC in 2016 leading to Dell EMC.

<https://en.wikipedia.org/wiki/Dell EMC>

Data Domain, a data deduplication company cofounded by Kai Li (Princeton U.) in 2001 was acquired by EMC in 2011.

[https://en.wikipedia.org/wiki/Data_Domain_\(corporation\)](https://en.wikipedia.org/wiki/Data_Domain_(corporation))

The *Reliable Array of Independent Nodes - RAIN* system described in (Bohossian et al., 2001) had led to the founding of Rainfinity in 1998 in Pasadena, CA by CalTech's Jehoshua (Shuki) Bruck (formerly with IBM) and his students Vasken Bohossian (<https://human-engine.com>) and Lihao Xu (Wayne State U.), who received a \$2K patent award from CalTech.

Rainfinity's *File Management Appliance - FMA* provides virtualization solutions for heterogeneous *Network Attached Storage - NAS*.

https://en.wikipedia.org/wiki/Network-attached_storage

NAS is different from *Storage Area Networks* - *SANs* in that it provides file level rather than block level access to storage devices.

https://en.wikipedia.org/wiki/Storage_area_network

EMC acquired Rainfinity for about \$100M in 2007.

<https://www.networkworld.com/article/2313441/emc-acquires-nas-virtualization-vendor-rainfinity.html>

5.2.5 SUN Microsystems

In 1979 Xerox donated Alto computers produced at its *Palo Alto Research Center* - *PARC* to several computer science departments including Stanford. Ethernet had also been invented at Xerox's by Robert Metcalfe to network computers, which was only 10 Mbits/s when originally released in 1980.

<https://en.wikipedia.org/wiki/Ethernet>

Inspired by the Alto workstation the initial design of *Stanford U. Network* - *SUN* workstation was configured by Forest Baskett using low-cost components with the intent to build a personal workstation for *Computer-Aided Design* - *CAD*. It was a 3M computer: 1 MIPS processor, 1 MB memory, and a 1 Megapixel raster scan bit-map graphics display. The Altos were connected using Ethernet to form several *Local Area Networks* - *LANs*. The project name was derived from the initials of the campus' *Stanford University Network* - *SUN*, which became the name of the company.

SUN Microsystems was founded in 1982 by three Stanford U. graduates: Vinod Khosla, Andy Bechtolsheim, and Scott McNeally. https://en.wikipedia.org/wiki/Sun_Microsystems

Andy developed high-end workstations based on Motorola 68K microprocessors. Berkeley's UNIX guru Bill Joy, who joined SUN later is also considered a SUN founder. SUN introduced the *Scalable Processor ARChitecture* - *SPARC* based on *Reduced Instruction Set Computer* - *RISC* microprocessor designed at UCB by David Patterson and Carlo Sequin.

<https://en.wikipedia.org/wiki/SPARC>

As of September 2017, the latest commercial high-end SPARC processors are 64 bit Fujitsu's SPARC64 XIIIs.

<https://talesfromthedatacenter.com/2017/10/sparc-and-solaris-the-past-and-the-future/>

SUN workstations with the UNIX-based Solaris operating system were popular in 1990s due to their graphics capability for CAD, but their popularity waned as Wintel boxes started delivering similar capabilities at a lower price.

At the turn of the century Sun delivered discounted workstations to NJIT at a low cost. The workstations were connected to a storage node, so that user files were accessible from any workstation on campus. When I left NJIT in 2007 Solaris operating system was still open source and my student Jose Pagan assisted me in installing Solaris on my Dell Desktop. After several years as more and more disk files appeared to be corrupted I started storing files twice as in the case of IRON file system (Prabhakaran et al., 2005). At a later point when I bought an HP desktop Solaris was no longer available so Ubuntu OS was installed. I have updated to Ubuntu 14.04 and a newer version of Latex.

<https://wiki.ubuntu.com/Releases>

James Gosling at SUN developed the Java programming language in 1995.

[https://en.wikipedia.org/wiki/Java_\(programming_language\)](https://en.wikipedia.org/wiki/Java_(programming_language)).

Oracle sued Google for using Java in developing the Android mobile OS and Google appealed the case in January 2019.

https://en.wikipedia.org/wiki/Oracle_America,_Inc._v._Google,_Inc.

https://en.wikipedia.org/wiki/Google_v._Oracle_America

The Supreme Court ruled 6-2 on 4/5/21 that Google's use of the Java APIs was within the bounds of fair use, reversing the Federal Circuit Appeals Court ruling.

A team of programmers at Sun was also responsible for the *Network File System* - *NFS* in 1984.

https://en.wikipedia.org/wiki/Network_File_System.

The *Parallel NFS* - *PNFS* protocol was developed by Panasas to allow the NFS protocol to process file requests in parallel.

SUN Microsystems whose rise is described in (Hall and Barry, 1991) was affected badly by the dot-com bubble at the end of the century,

since many companies whose web sites were based on SUN servers went out of business.

https://en.wikipedia.org/wiki/Dot-com_bubble

SUN was bought by Oracle in April 2009 for \$5.6 billion.

5.2.6 SUN ZFS file system and volume manager

ZFS file system and volume manager was developed in conjunction with SUN's Solaris OS (Karlsson, 2006) by Matthew Ahrens and Jeff Bonwick in 2001. After Oracle acquired Sun Microsystems, the Solaris OS lost its open source status it,

and ZFS followed at which point ZFS developers left Oracle in September 2013 to create the OpenZFS project.

<https://en.wikipedia.org/wiki/OpenZFS>

A summary of ZFS features based on Wiki is as follows: <https://en.wikipedia.org/wiki/ZFS>

Pooled storage: Pooled storage combines features of a file system and a volume manager, so that ZFS files can span multiple drives and storage space can be expanded by adding drives. Pooled data storage is provided via collections of disks called Zpools, which can be optimized for capacity, performance or reliability. ZFS also handles partitioning and formatting.

Copy-On-Write - CoW: A valuable feature of ZFS in that it updates data by writing it to a different block pointed to by the metadata. Otherwise once the write is complete the overwritten data is lost. With CoW if the system crashes while the write is in progress the old data will be preserved. This also means that the system does not need to run the *File Systems Consistency Check - FSCK* after a system crash. <https://en.wikipedia.org/wiki/Fscck>

Snapshots: ZFS retained old data creates snapshots of the file system through time as changes are made. File writes using ZFS are transactional. Time sliders feature provides file snapshots at regular intervals.

Data integrity verification and automatic repair: ZFS data integrity is achieved by using a Fletcher-based checksum <https://software.intel.com/en-us/articles/fast-computation-of-fletcher-checksums> or an SHA-256 hash throughout the file system tree. Blocks are checksummed and the checksum saved in the pointer to that block, rather than at the actual block itself.

https://en.wikipedia.org/wiki/Fletcher%27s_checksum

Remote Direct Memory Access - RDMA supports zero-copy networking by enabling the network adapter to transfer data from the network directly to application memory and vice-versa. This eliminates the need to copy data between application memory and OS data buffers. A shortcoming of this strategy is that the target is not notified of the completion of a data transfer.

RAID-Z: ZFS can handle RAID with no extra software or hardware. It is a variation of RAID5 designed to overcome the write hole error, with data and parity becoming inconsistent after a crash, such as power loss. RAIDZ uses variable width stripes to get around this problem, so that every write is effectively a full-stripe write. Together with ZFS's copy on write characteristic, this eliminates the RAID5 write holes.

Huge Storage potential: ZFS is a 128 bit file system.

Data Scrubbing: ZFS checks the integrity of data using checksums and detects silent data errors and correct errors whenever possible.

Installing ZFS. This is beyond the scope of our discussion.

RAID-Z is expensive, because of the number of drives required to add storage space.

The effect of disk and memory corruption on Sun ZFS file system data integrity was investigated by Remzi and Andrea Arpaci-Dusseau et al. at (U. Wisc.) in (Zhang et al., 2010). Fault injection was used to show that ZFS can tolerate a wide range of disk faults,

https://en.wikipedia.org/wiki/Fault_injection

but is less resilient to memory corruption evaluated in (Schroeder et al., 2009, 2011), which can lead to corrupt data being returned to applications. The analysis reveals the importance of considering both the memory and disks in the construction of truly robust file and storage systems.

An end-to-end data integrity approach for storage systems is considered by Arpaci-Dusseau et al. in (Zhang et al., 2013c) to enable components along the I/O path to alter their protection scheme to meet the performance and reliability demands of the system. The new concept is applied to the *Zettabyte File System - ZZFS*. An analytical framework for protection is developed, which is compared with the same protection scheme for all components.

5.2.7 International Business Machines - IBM Corp.

IBM S/360 mainframes introduced in mid-1960s turned out to be one of the most successful computer families in history, which has survived since 1964 (Cortada, 2019b,a).

https://en.wikipedia.org/wiki/IBM_System/360 UCB's Michael Stonebraker in a keynote speech at the *Int'l Conf. on Data Eng - ICDE'86* called IBM an iron-monger, but IBM was changed to an IT services company under the leadership of Louis Gerstner (1993-2002).

IBM Global Business Services - GBS is organized into three growth platforms which contain seven service lines:

- 1-Digital Strategy & Interactive: Digital Strategy, IBM iX
- 2-Cloud Application Innovation: ADM Innovation, Cloud Application Migration Services, Next Gen EA

- 3-*Cognitive Process* - CP Transformation: CP Decision Support, CP Re-Engineering, CP Process Services

Acquisitions

- 4-PricewaterhouseCoopers (PwC) Consulting, 2001
- 5-Bluewolf, 2016
- 6-Resource/Ammirati, 2016
- 7-Promontory Financial Group, 2016

IBM *Global Technology Services* - GTS provides IT services for clients around the world. GTS has expanded its capabilities in hybrid cloud and is applying automation and Watson in its services delivery, especially through the IBM Services Platform with Watson launched in July 2017.

On 10/8/20 IBM announced it was spinning off the *Managed Infrastructure Services* - MIS unit of its GTS division into a new public company called Kyndryl.

<https://www.ibm.com/case-studies/ibm-global-technology-services>

The effect of the 1988 Berkeley RAID paper (Patterson et al., 1988)

The RAID paper, i.e., (Patterson et al., 1988), in addition to a classification was a proposal to replace large high capacity mainframe disks known as DASD (Bohl, 1981), (Wiederhold, 1987), Sierra (2012) with *Extended Count Key Data* - ECKD format, which is an extension of *Count Key Data* - CKD format.

https://en.wikipedia.org/wiki/Count_key_data

with *Fixed Block Architecture Disks* - FBA disks.

https://en.wikipedia.org/wiki/Fixed-block_architecture

The 3390 Model 9 DASD in 1993 was IBM's last *Single Large Expensive Disk* - SLED just five years after (Patterson et al., 1988) was published.

https://en.wikipedia.org/wiki/History_of_IBM_magnetic_disk_drives.

Ken Ouchi at IBM was the author of a patent for a RAID5-like disk arrays in the context of ECKD drives in 1977, a dozen years before (Patterson et al., 1988).

<https://patents.justia.com/patent/4092732>

ECKD disk drives would have made the implementation difficult. What is described in the patent to compute the parity over the strips of a large block as they are being written on successive disks. At a later point Ouchi coauthored a patent with Mario Blaum on *2-Disk failure Tolerant* - 2DFT arrays (Blaum and Ouchi, 1994). An interview with Ken Ouchi at the Computer History Museum appears below.

<http://www.computerhistory.org/collections/catalog/102740119>

Ouchi's parities could be applied at the level of equal sized records on a per file basis given the Count-Key-Data organization, where the parity is applied to the data portion.

Given that it is known that if parameter k is coprime with n then $i \times k \bmod n$, generates all values 0 to $n - 1$ in a periodic manner as i increases. The parameter k is called a "generator" of the $\bmod n$ addition group. With n disks and a parity group size k which is prime wrt n a RAID5 disk array can be constructed. By choosing the parity group size k to be mutually prime with respect to the number of disks n a RAID5 layout is achievable.

An IBM 2-page technical disclosure bulletin enunciating the RAID5 concept (Lawlor, 1981) and a RAID5 patent for AS/400 system at Rochester, MN is as follows Clarke et al. (1988).

The IBM AS/400 computers, now known as the i-series provide RAID1 and RAID5 AS/400 Disk Storage Topics and Tools. SG24-5693-00 /newline <http://www.redbooks.ibm.com/redbooks/pdfs/sg245693.pdf>

IBM CEO John Akers combined IBM storage businesses in eight countries in 1992 to form *ADvanced STorage And Retrieval* - ADSTAR in preparing to sell it off. <https://en.wikipedia.org/wiki/ADSTAR>

IBM's financial losses and a major drop in its stock price led to the replacement of John Akers with Louis Gerstner from Nabisco Corp. in 1992.

https://en.wikipedia.org/wiki/Louis_V._Gerstner_Jr.

<https://www.nytimes.com/1993/03/22/business/rjr-leader-in-ibm-spotlight.html>

In fact Wall Street's choice had been Lou Gerstner's older brother Richard Gerstner, who was an IBM executive. Richard did not take the job because he suffered from Lyme's disease.

<https://patch.com/new-york/mineola/richard-t-gerstner-lyme-disease-activist-73>

Gerstner reversed Akers decision to sell ADSTAR, but this was done by his successor Sam Palmisano who sold the storage business to Hitachi in 2003 for \$2.05 billion and this led to *Hitachi Global Storage Technologies* - HGST.

<https://en.wikipedia.org/wiki/HGST>

At that point half of IBM's storage employees were transferred to HGST, but were mostly laid off after five years. Two such employees were Mario Blaum, who is back at IBM as a contractor, and Spencer Ng, later at SanDisk (retired).

The IBM market capitalization went up from \$29B to \$168B during Gerstner's tenure, but the company fell behind with respect to other computer companies.

<https://companiesmarketcap.com/ibm/marketcap/>

Under Gerstner IBM grew its Global Consulting Business.

<https://www.ibm.com/services/careers/professionals>

in 2002 purchasing the consulting arm of *PricewaterhouseCoopers* - *PwC*.

<https://www.cnet.com/news/ibm-grabs-consulting-giant-for-3-5-billion/>

<https://en.wikipedia.org/wiki/PricewaterhouseCoopers>

A presentation summarizing the fall of IBM during Sam Palmisano's and Ginny Rommetti's tenure is given below.

<https://www.youtube.com/watch?v=l4b1D1vWRnc>

5.2.8 Storage Technology Corp. - STK

STK was founded by four former IBM engineers Jesse Aweida, Juan Rodriguez, Thomas S. Kavanagh, and Zoltan Herger in 1969, which is the same year as the founding of Amdahl Corp. https://en.wikipedia.org/wiki/Storage_Technology_Corporation

Unlike Amdahl STK was not successful in producing *Mainframe Compatible Processors* - *MCPs*.

In 1992 STK introduced the Iceberg RAID6 disk array (Friedman, 1995), with two check strips per stripe with *Reed-Solomon* - *RS* coding (Reed and Solomon, 1960), (MacWilliams and Sloane, 1977). RAID6 is motivated by the fact that RAID5 rebuild may fail due to a second disk failure or when a *Latent Sector Error* - *LSE* is encountered, which is the more likely event. (Blaum et al., 1994)

The fact that StorageTek utilized the *Log-Structured File Systems* - *LFS* paradigm (Rosenblum and Ousterhout, 1992) discussed in Section 5.20 may be attributed to fact that it simplified the mapping of ECKD files to FBA disks. The term *Log-Structured Array* - *LSA* was coined in (Menon, 1995), which analyzes RAID5 with LFS so that small writes are replaced by full stripe writes (Chen et al., 1994b).

As noted in (Chen et al., 1994b) the IBM's effort to develop RAID5 failed in early 1990s. IBM temporarily became a reseller of the Iceberg disk arrays under the name *RAMAC Virtual Array* - *RVA* which is described in an IBM 1997 Redbook (IBM Corp. Int'l Tech. Support Org., 1997). The predicted *StorageTek* - *STK* merger with IBM in 1998 did not materialize, https://www.theregister.co.uk/1998/10/29/big_blue_storagetek_cuddle_up/ but instead STK which is better known for its automated tape libraries,

<http://www.fundinguniverse.com/company-histories/storage-technology-corporation-history/>

was acquired by Sun Microsystems in 2005 for \$4.1 billion.

<https://www.infoworld.com/article/2669714/sun-completes--4-1-billion-storagetek-acquisition.html>

Oracle Corp. by acquiring SUN in 2010 for \$7.4 B also acquired STK.

https://en.wikipedia.org/wiki/Oracle_Corporation

<http://www.oracle.com/us/corporate/press/018363>

5.2.9 Western Digital Corp. - WDC

WDC is a storage company founded in 1970 in San Jose,

https://en.wikipedia.org/wiki/Western_Digital

which acquired HGST for \$4.8 billion in 2012.

In 2016 WDC acquired SanDisk founded in 1988 to produce flash memories for \$19 billion.

<https://en.wikipedia.org/wiki/SanDisk>

WDC settled an SMR HDD for false advertising class action lawsuit admitting that SMR is not suitable for RAID storage. "Aside from generally slower performance, that can also cause excessive rebuild times (resilvering) in the targets NAS use cases, particularly if they use ZFS RAID arrays."

<https://www.tomshardware.com/news/wd-moves-to-settle-smr-hdd-false-advertising-class-action-lawsuit>

5.2.10 Network Appliance - NetApp Corp.

Network Appliance - NetApp Corp., which produces data hardware and software was founded by David Hitz, James Lau, and Michael Malcolm (former U. Waterloo professor) in 1992.

<https://en.wikipedia.org/wiki/NetApp>

Hitz's autobiography "How to castrate a bull" emphasizes management and marketing issues at NetApp, including the firing of Malcolm (Hitz and Walsh, 2009).

Most early NetApp systems utilized *Direct Attached Storage - DAS*, where the storage system is only accessible to the host to which it is attached. NetApp initially adopted *Network Attached Storage - NAS*,

https://en.wikipedia.org/wiki/Network-attached_storage

but later introduced *Storage Area Networks - SANs* for enterprise customers using high performance RAID.

https://enilvester.wikipedia.org/wiki/Storage_area_network

Rotated Diagonal Parity - RDP was a NetApp invention to implement RAID, which is discussed in Section 5.24.

5.2.11 HP storage products

Bill Hewlett and Bill Packard founded started HP in a Palo Alto Garage in 1939.

<https://en.wikipedia.org/wiki/Hewlett-Packard>

John Wilkes joined HP Labs in Palo Alto, CA in 1982 after graduating from UK's Cambridge U. He led a large and successful group for storage research at HP with many visiting students.

http://www.hpl.hp.com/news/2002/oct-dec/john_wilkes.html

Before joining Google in 2008, he coauthored a book on storage systems based on his work at HP (Wilkes et al., 2008). John Wilkes at Google has maintained a web page for his earlier publications at HP as well as new ones at Google.

<https://john.e-wilkes.com/work-publications.html>

Wilkes led the design of AutoRAID combining RAID1 and RAID5 arrays in (Wilkes et al., 1996) which is discussed in Chapter 10 on *Heterogeneous Disk Arrays - HDAs*. HP's Surestore product was based on AutoRAID.

Specifications for the HP Smart array are provided below.

https://support.hpe.com/hpsc/doc/public/display?docId=emr_na-c04496202

The HPE Smart Array P440 Controller is a Flexible Smart Array, 12 Gb/s SAS RAID controller that provides enterprise-class storage performance, increased internal scalability with SAS Expander Card, and data protection for specific HPE ProLiant Gen9 rack and tower servers. It features eight internal physical links and delivers increased server uptime by providing advanced storage functionality, including online RAID level migration between any RAID levels with *Flash Backed Write Cache - FBWC*, global online spare, and pre-failure warning. The Gen9 family of controllers supports the HPE Smart Storage Battery, multiple Smart Array controllers are supported without needing additional backup power sources for each controller, resulting in simple upgrade process.

3PAR is a storage company founded in 1999 was acquired by HP in 2010.

https://en.wikipedia.org/wiki/HPE_3PAR

Thin provisioning advocated by 3PAR uses *Dedicate on Write - DoW* rather than *Dedicate on Allocate - DoA* paradigm, reducing required disk space requirements and disk numbers.

<https://web.archive.org/web/20061118125440/>

http://www.byteandswitch.com/document.asp?doc_id=35674

DoA paradigm was used in conjunction file systems, which required a set of contiguous cylinders for the MVS OS.

Nimble storage founded in 2008 was acquired by HPE in 2017 for \$1.09 Billion to become a subsidiary of HP.

https://en.wikipedia.org/wiki/Nimble_Storage

It is advertised as predictive storage which makes 86% of problems disappear. A 99.9999% availability is guaranteed over the installed base and 54% of problems resolved are determined to be outside storage. InfoSight applies data science to identify, predict, and prevent problems across infrastructure layers as described below to achieve six nines availability shown in Figure 1 in the following booklet.

<https://m.softchoice.com/web/newsite/documents/partners/hpe/Nimble-Six-Nine-White-Paper.pdf>

HP was split into *HP Enterprise - HPE* and HP in 2015 and is moving to Houston, TX.

5.3 RAID classification and motivation

The original five level RAID classification in (Patterson et al., 1988) was extended to seven levels with the addition of RAID0 and RAID6 in (Chen et al., 1994b) by Peter Chen (U. Mich.) and Edward K. Lee (DEC SRC, Data Domain, Tintri, Applatix), Garth Gibson (CMU/Vestor Inst.), David Patterson (Google), and Randy Katz at *U. Calif at Berkeley - UCB*

RAID1 is based on disk mirroring, RAID2 on Hamming codes, RAID3,4,5 on single parities, RAID0 stripes data, i.e., large datasets are partitioned into strips which are placed on consecutive disks with wraparound. The strips in a row across all disks constitute a stripe. RAID6 dedicates two strips in each stripe (row) to check strips, which are based on Reed-Solomon coding or parity codes.

The motivation for RAID was to replace expensive, large form-factor *Direct Access Storage Devices - DASD* associated with IBM mainframes and its plug compatibles, referred to as *Single Large Expensive Disks - SLEDs* in (Patterson et al., 1988), with smaller, inexpensive high volume commodity fixed block disks associated with personal computers in 1980s. SLEDs are no longer manufactured and the **I** in RAID now stands for **I**ndependent (Chen et al., 1994b).

Given that multiple low-cost, less reliable disks replace large capacity highly reliable DASD resulted in reduced reliability, which necessitated fault-tolerance through replication and erasure coding such as parity. Another novel aspect of the RAID proposal was striping to avoid unbalanced loads across disks. Full stripe writes are efficient because they allow parity and generally check strips to be computed on the fly, e.g., parity strips by XORing successive strips in the case of RAID5. Striping is discussed in more detail in Section 5.4.

IBM's MVS operating system and its followups issue I/O instructions called *Channel Command Words - CCWs* to DASD with *Extended Count Key Data - ECKD* format (Bohl, 1981). Command translation was to be implemented by the *RAID Disk Array Controller - DAC* using its cache as a workspace to convert from ECKD to fixed format blocks and vice-versa. An alternative solution would have been to modify MVS's I/O routines to handle *Fixed Block Architecture - FBA* disks, but that requires running an alternative OS such as Linux.

https://en.wikipedia.org/wiki/Linux_on_IBM_Z

IBM z-systems mainframes run AIX and Linux (Gracin et al., 2007).

https://en.wikipedia.org/wiki/Linux_on_z_Systems

IBM's Linux1 server runs Linux but not z/OS and is hence cheaper (Hurwitz et al., 2019). "Set up Linux on IBM System z for Production" is discussed in 2013.

<https://www.redbooks.ibm.com/redpieces/abstracts/sg248137.html>

The following explanation was provided by former IBM researcher Joshua Knight whether a hardware modification is required to access fixed block disks on mainframes. There is a z-Linux native device driver for *SCSI Fibre Channel Protocol - FCP*.

https://en.wikipedia.org/wiki/Fibre_Channel_Protocol

so devices can be defined on a Fibre Channel attached to a storage box as a z/VM guest. Then there is the support which allows z/VM to access FCP disks (Webb et al., 2007), which means that a z/VM system can survive without a storage system that supports ECKD disks. It is also possible to have any z/VM zLinux guest use the same subsystem to access the FCP disks, but with a significant performance penalty.

Installations that want to run z/VM primarily as a host for z/Linux guests can use non-IBM FCP storage, but most installations do not use the z/VM FCP support for z/Linux guests and simply let the z/Linux guests "direct attach" to the FCP storage.

5.4 RAID0 and striping

There is a susceptibility to load imbalance when a frequently accessed file on a SLED is placed on a smaller-capacity, slow disk drive. This may be dealt with by utilizing striping to balance disk loads (Chen et al., 1994b). Striping partitions large files into fixed size *Stripe Units - SUs* or strips and places them modulo N , which is the number of disks to balance their load. RAID0 does not provide redundancy to protect against disk failures.

If disks and rows/strips are numbered starting with zero, then the i^{th} strip is placed at disk/column $j = i \bmod N$ and row $k = \lfloor \frac{i}{N} \rfloor$. A more complex mapping is required to take into account check strips. Striping allows data access parallelism for reading/writing large files. Large files can be written as full stripe writes in RAID5 and generally RAID(4 + k), $k \geq 1$, so that check strips are computed as data strips are being written (Chen et al., 1994b).

The author collaborated with John Silvester (*U. Southern California - USC*, emeritus) in carrying out a capacity planning study of IBM mainframes at Lockheed Corp. in Burbank, CA (Silvester and Thomasian, 1981). The goal of the study

reported was to improve performance and to save money by utilizing the BEST/1 capacity planning tool from BGS Corp. (Buzen, 1978). BGS cofounded by Jeff Buzen with Robert P. Goldberg and Harold S. Schwenk hence BGS in 1975 was acquired by BMC in 1998 for \$285 million. <https://www.bmc.com>
<https://www.bizjournals.com/houston/stories/1998/02/09/newscolumn3.html>

Lockheed provided us measurements from the large computer center with four mainframes using MVS's tools, which could be used to build a BEST/1 model (Buzen, 1978). This was before the advent of BGS's Crystal model building tool. It turned out that only a few disks had a significant utilization and the rest with very low utilization were modeled as delay servers (Silvester and Thomasian, 1981).

There were several studies of disk load imbalance or access skew in the context of *Just a Bunch of Disks* - JBOD. An investigation of I/O performance by Jeff Buzen and Annie Shum at BGS Corp. showed a slow variation in disk load imbalance (Buzen and Shum, 1990).

An empirical study of variations in DASD volume activity was also reported by Bruce McNutt at IBM's SSD - Storage Systems Division (McNutt, 1986).

The discussion of striping in (Salem and Garcia-Molina, 1986) mentions striping in an early version of UNIX and the striping study at IBM TJWRC (Kim, 1985).

Striping was evaluated via simulation by Miron Livny (U. Wisc. at Madison) and Setrag Khoshafian (<https://Khoshconsulting.com>) and Haran Boral (rip) at *MicroElectronics and Computer Technology Corp.* - MCC in Austin, TX (1982-2000) (Livny et al., 1986).

https://en.wikipedia.org/wiki/MicroElectronics_and_Computer_Technology_Corporation

An analysis of synchronized disk interleaving is reported in (Kim, 1986) using the M/G/1 queueing model (Kleinrock, 1975). Only a single disk needs to be considered because of the symmetry arising from synchronized nature of disks.

The analysis of asynchronous disk interleaving is reported in (Kim and Tantawi, 1991). Disk requests spanning all disks are served *one request at a time*, so that requests at all disks have to be completed before the next request is issued, which is an instance of a fork/join queueing system (Thomasian, 2014b). The mean access time is the expected maximum of the access times at n disks. Each access time is the sum of exponentially distributed seek times and uniformly distributed rotational latencies. Given the mean (μ_X) and standard deviation (σ_X) for disk service times, the expected value of the maximum service time for an n -way system is given using an approximation in "Order Statistics" (David and Nagaraja, 2003):

$$E[Max(X_n)] \approx \mu_X + \sigma_X G(n), \quad G(n) \leq \frac{n-1}{\sqrt{2n-1}}.$$

The analysis determines the mean service time for processing disk accesses. Mean fork/join response times could have been determined by determining the second moment of disk access times. With Poisson arrivals the analysis could be extended by using the *Pollaczek-Khintchine* - P.K. formula to determine the mean response time of requests (Kleinrock, 1975), which is also given in Chapter 1.

An experimental study of striping to reduce access skew is reported by Greg Ganger (CMU), Bruce Worthington (Microsoft), Robert Hou, and Yale Patt (U. Texas at Austin) at U. Michigan in (Ganger et al., 1993) by running the TPC-B benchmark on a 14-disk array. The mean queue-length metric for **O**=original, **S32**: striped with 32 KB strips, **R**: Random, and **S64**: striped with 64 KB strips are given in Table 2 in the paper. The queue-length for **O** varies in the range (0-11), for **S32** it is 0.07, for **R**: it is 0.036, and for **S64** it is 0.076-0.081. It is interesting to note that random allocation outperforms S32 two-fold. The justification for mean queue-lengths follows from the P.K. formula for mean waiting time in M/G/1 queues (Kleinrock, 1975).

Given that RAID0 does not provide redundancy to deal with disk failures or unreadable blocks, the single pass restore method is proposed by Caetano Sauer, Theo Harder, and Goetz Graefe (Google, Madison) at U. Kaiserslautern to increase restore bandwidth and substantially decreases disk repair time (Sauer et al., 2019). Media failures may leave the database systems unavailable for several hours until recovery is complete, which is more important in the case of high performance OLTP systems. Instant restore goes further permitting read/write access to any data on a device undergoing restore – even data not yet restored – by restoring individual data segments on demand. Thus, the restore process is guided primarily by the needs of applications, and the observed mean time to repair is reduced drastically. Instant restore may be preceded by a reconstruction of the damaged block by accessing corresponding blocks in RAID5.

The single pass restore technique is incrementally implemented on a system starting with the traditional ARIES design for logging and recovery by researchers at IBM/ARC (Mohan et al., 1992). Experiments show that the txn latency perceived after a media failure can be cut down to less than a second. The net effect is that a few "nines" of availability are added to the system using simple and low-overhead software technique.

If a block on disk is unreadable in a RAID5 environment the I/O system will try to recover an unreadable block first, before the DBMS software which issued an I/O access is invoked.

5.5 RAID2

RAID2 is a proposal for byte level striping on disks with synchronized spindle rotations, which allows reading from multiple disks concurrently to yield a higher data transfer rate. It should be noted that the smallest unit of data transfer from FBA disks is a 512 byte sector, and a similar comment applies to RAID3 arrays where byte level striping has been advocated.

RAID2 was proposed in (Park and Balasubramanian, 1986) by implementing the Hamming code to detect and correct disk failures. The Hamming code was proposed by Richard Hamming at Bell Labs (Hamming, 1950), (Peterson and Weldon, 1972). A disk array based on the Hamming code was used in conjunction with the *Connection Machine CM-2* (Gibson, 1992) from Thinking Machines.

https://en.wikipedia.org/wiki/Connection_Machine

Its one of the several parallel computer companies that went out of business in the Boston area in early 1990s.

The DataVault was Thinking Machines' *Mass Storage System - MSS*, storing 5 to 10 GB of data with transfer rates of 40 MB/sec. Eight DataVaults could be operated in parallel for a combined data transfer rate of 320 MB/s. Each DataVault unit stored its data in an array of 39 individual disk drives with data was spread across the drives. Each 64-bit data chunk received from the I/O bus was split into two 32-bit words. After verifying parity, the DataVault controller added 7 bits of *Error Correcting Code - ECC* and stored the resulting 39 bits on 39 individual drives. Subsequent failure of any one of the 39 drives would not *impair* reading of the data, since the ECC code allows any single bit error to be detected and corrected.

Although operation is possible with a single failed drive, three spare drives were available to replace failed units until they are repaired. The ECC codes permit 100% recovery of the data on any one failed disk, allowing a new copy of this data to be reconstructed and written onto the replacement disk. Once this recovery is complete, the data base is considered to be healed.

<https://en.wikipedia.org/wiki/DataVault>

An example of Hamming with seven bits is given in Fig. 5.1.

Position	1	2	3	4	5	6	7
Binary	001	010	011	100	101	110	111
Parities	p1	p2	d1	p4	d2	d3	d4
p1	X		X		X		X
p2		X	X			X	X
p4				X	X	X	X

FIGURE 5.1 Four data bits protected by three parity bits p1, p2, and p4. Parity bits are computed over data bits that have a one in that position in their binary representation.

Errors in p1 and p4 mean d2 in position 5 is in error and can be corrected by using either parity.

https://en.wikipedia.org/wiki/Hamming_code

Generally p_{2^i} , $i \geq 0$ protects data bits whose address has a "1" in the i^{th} position. Applying parity checking resulting in all zeros indicates no error, while otherwise the binary number (p_{16} , p_8 , p_4 , p_2 , p_1) indicates the position of the incorrect bit.

Given r parity bits up to $n = 2^r - 1$ data bits can be protected, and the message length is $k = 2^r - 1 + r$. For example, if $r = 4$ then $n = 15$

$$p_1, p_2, d_1, p_4, d_2, d_3, d_4, p_8, d_5, d_6, d_7, d_8, d_9, d_{10}, d_{11}, p_{16}, d_{12}, d_{13}, d_{14}, d_{15}$$

RAID2 with fifteen data disks requires five parity disks as shown in Fig. 5.2.

-	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
-	p1	p2	d1	p4	d2	d3	d4	p8	d5	d6	d7	d8	d9	d10	d11	p16	d12	d13	d14	d15
p1	X		X		X		X		X		X		X		X		X		X	
p2		X	X			X	X			X	X			X	X		X	X	X	
p4				X	X	X	X					X	X	X	X					X
p8								X	X	X	X	X	X	X	X					
p16																X	X	X	X	X

FIGURE 5.2 Disk numbers and their contents. There are $r = 5$ parities and a maximum number of $2^r - 1 = 31$ data bits can be protected, but only 15 data bits are shown.

Hamming codes have a minimum distance of three, i.e., they can detect and correct any single bit error and detect two non-self-identifying errors. When the errors are self-identifying, e.g., it is known which disks have failed in a disk array, we can do better and correct them too. This was noted in (Gibson, 1992) this effect was ignored in the design of Thinking Machine's CM-2 (Thinking Machines Corp., 1987).

The extended Hamming code with an additional overall parity is *Single Error Correcting/Double Error Detecting - SEC/DED*.

5.6 RAID3

RAID3 similarly to RAID2 stripes into small strips across $N - 1$ disks, while the parity strip is stored on N^{th} disk. Spinning disks are synchronized and large data blocks are read/written from/to N disks concurrently to achieve high bandwidths.

If one of RAID3 data disks is broken its contents can be restored on the fly by XORing corresponding blocks (sectors) from $N - 1$ non-failed disks. The RAID3 disadvantage is low bandwidth in accessing small data blocks.

5.7 RAID4

RAID4 is similar to RAID3 in that it uses a single parity disk for data protection, but differs because of its large data blocks which span multiple sectors. The parity disk holds the *exclusive OR* - *XOR* of the strips at the other disks. When a data disk fails, its data blocks can be reconstructed on demand by XORing the $N - 1$ corresponding blocks on surviving disks. Blocks can be reconstructed on demand during degraded RAID5 operation and systematically, one block after another, in rebuild mode.

Placing all parities on one of N disks in RAID4 results in a bottleneck for write-intensive workloads to small blocks, since the write load of the parity disk will be the sum of the write loads at data disks or $N - 1$ -fold higher if update loads are balanced. Writes to large blocks can be handled efficiently as full stripe writes, where the parity is computed as consecutive data strips are written to disks

A major disadvantage of RAID4 is that the bandwidth of the parity disk is wasted, since it will not be utilized by read accesses.

5.8 RAID5

Rotated parity disk arrays classified as RAID5 resolve the shortcoming associated with RAID4 in that the bandwidth parities are placed in repeating right to left diagonals, which is referred to as the left symmetric layout (Lee and Katz, 1993a), (Chen et al., 1994b). This allows all disks to be utilized for read requests thus resulting in an increased disk bandwidth. In addition the write load is balanced by evenly distributing parities across disks. This is because updating a data block in RAID5 requires updating the corresponding parity block as well.

Shown in Fig. 5.3 are the data layouts for RAID0, RAID1, RAID5, and RAID6.

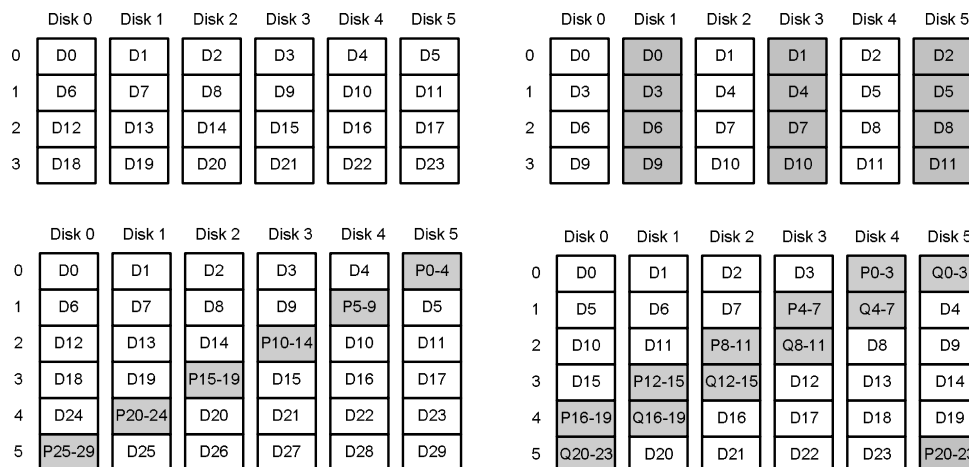


FIGURE 5.3 RAID0, RAID1, RAID5, and RAID6 layouts.

0	1	2	3	4	0	1	2	3	P0	0	1	2	3	4
5	6	7	8	9	4	5	6	7	P1	5	6	7	8	9
10	11	12	13	14	8	9	10	11	P2	10	11	12	13	14
15	16	17	18	19	12	12	12	15	P3	15	16	17	18	19
20	21	22	23	24	16	17	18	19	P4	P4	P3	P2	P1	P0
P0	0	1	2	3	P0	0	1	2	3	0	1	2	3	P0
4	P1	5	6	7	7	P1	4	5	6	10	11	P2	13	14
8	9	P2	10	11	10	11	P2	8	9	P4	21	12	23	14
12	13	14	P3	15	13	14	15	P3	12	20	31	22	P6	24
16	17	18	19	P4	16	17	18	19	P4	30	P8	32	33	34
0	1	2	3	P0	0	1	2	3	P0	5	6	7	P1	9
4	5	6	P1	7	5	6	7	P1	4	15	P3	17	18	19
8	9	P2	10	11	10	11	P2	8	9	25	16	27	18	P5
12	P3	13	14	15	15	P3	12	13	14	35	26	P7	28	29
P4	16	17	18	19	P4	16	17	18	19	P9	36	37	38	39

FIGURE 5.4 This figure is based on (Lee and Katz, 1993a). There are $N = 5$ disks per array. Starting with RAID0 and RAID4 in the top left hand corner we show various RAID5 layouts: Flat left symmetric; right asymmetric; right symmetric; extended left symmetric; (second and third layouts in right column), left asymmetric; left symmetric.

Eight parity data layouts for RAID5 were proposed by Edward K. Lee and Randy Katz in (Lee and Katz, 1993a), where their performance is also investigated. Several of these layouts are shown in Fig. 5.4. Data organizations which allow parallel access to N consecutive strips in a disk array with N disks maximize throughput for large sequential accesses. Note that this data layout is an improvement over the RAID5 layout in Fig. 5.3 in that by simply renumbering strips allows full bandwidth access to N disks. The flat left symmetric, right asymmetric, and left symmetric, and extended left symmetric also fall into this category.

The RAID5 array at Amdahl Corp.

An early performance comparison of RAID1 and RAID5 arrays was carried out by Peter Chen and the authors of (Patterson et al., 1988) as reported in (Chen et al., 1990). Not surprisingly the investigation showed that for large data blocks RAID5 outperforms RAID1, but for small data blocks in applications such as OLTP RAID1 is better, since it doubles the bandwidth for accessing data. Similar conclusions are reached via an analytic study in (Thomasian and Xu, 2011a).

According to Dieter Gawlick who was at Amdahl Corp. (1964-68),

<https://www.linkedin.com/in/dietergawlick/>

the performance study in (Chen et al., 1990) was carried out on a striped disk array with parity protection, which he codeveloped with John Colgrove based on a request from NASA - *National Aeronautics and Space Administration* at Ames, CA.

The array was to be used to record one megabytes per second generated by a Cray supercomputer, while at that time the default OS, i.e., Unix, could only handle about 60 KB/sec. John Colgrove has proceeded to found PureStorage which utilizes flash technology. Details on Pure Storage are provided on Chapter 2.

[PureStorage.com](https://www.purestorage.com)

5.8.1 RAID0 and RAID5 stripe unit size

The size of the *Striping Unit* - SU size to get the highest performance from RAID0 was addressed in (Chen and Patterson, 1990). It is concluded that the SU size depends on only two parameters.

1. the number of outstanding disk requests, which is a composite parameter in that according to Little's law $\bar{N}_q = \lambda W$.
2. mean positioning time multiplied by disk data transfer rate.

The following two equations are derived for SU size in *KiloBytes* - *KB*.

$$\text{SU size} = S \times (\text{average pos. time}) \times (\text{data transfer rate}) \times (\text{degree of conc.} - 1) + 0.5 \text{ KB, where } S \approx 1/4.$$

The following formula applies when the degree of concurrency is unknown:

$$\text{Compromise SU size} = Z \times (\text{average pos. time}) \times (\text{data transfer rate}), \text{ where } Z \approx 2/3.$$

Since a good choice for SU size depends on average request sizes it is proposed by Gerhard Weikum (Max Planck Inst.) and Peter Zabback (Apple) in (Weikum and Zabback, 1992) that SU sizes should be file-specific rather than the same for all files. Intuitively speaking a small strip size is appropriate for a file with high access rate to small records and a large strip size for files with large sequential accesses to blocks. A method for tuning file-specific SUs based on average request size and the throughput requirements of the application is proposed. Experiments with a synthetic benchmark exhibited significant performance gains. In another study two stripe sizes were used per disk for mirrored data depending on whether the data is accessed in large chunks by queries or small blocks by transactions (Merchant and Yu, 1995).

Zabback later collaborated with Jai Menon at IBM ARC to develop a RAID configuration tool based on three steps (Zabback et al., 1996):

1. Collecting I/O traces while running typical applications.
2. Analyzing the traces to determine the application workload characteristics.
3. Using a simulator to evaluate different RAID configurations by running workload traces.

An ad hoc performance model of RAID0 taking into account queueing and fork-join synchronization is formulated in (Lee and Katz, 1993b). Logical requests are broken up into independent physical disk requests, which must all complete to satisfy the original request. Approximate equations are derived for disk utilization and throughput, and request response times. The analytic model is validated via simulation. It is shown that the optimum strip size, which maximizes throughput and minimizes response time is given as:

$$\text{OPT_SU} = \sqrt{\frac{P \times X(L - 1 + \gamma)SZ}{N}}$$

P is the positioning time, and X is the disk data transfer rate. N is the number of disks and there are L processes generating n -way disk accesses, where SZ is the request size. Error analysis yields the correction factor $\gamma = 0.15$. Queueing-theoretic approaches for analyzing fork/join queueing systems are surveyed in open models in (Thomasian, 2014b).

Research on striping in RAID0 to RAID5 to determine how the choice of SU varies with the number of disks (N) is reported in (Chen and Lee, 1995). Synthetic workloads are used in this study to derive design rules for striping with varying amounts of workload information. The design rules are validated with real workload traces. As would be expected it is determined that for read-intensive workloads there is no difference in the optimal SU size for RAID0 and RAID5, while for write intensive workloads the optimal SU size for RAID5 is four times smaller than for read-intensive workloads. The optimal SU for reads in RAID5 varies inversely with the number of disks, but the optimal SU for writes varies with the number of disks. Overall for workloads with an unspecified mix of reads and writes is independent of N . In the absence of specific workload information the SU size for a wide range of N values is:

$$1/2 \times \text{average positioning time} \times \text{disk transfer rate}.$$

5.8.2 Parity striping and load balancing

Parity striping preserves the original data layout to ensure efficient sequential accesses to large files. This data layout for RAID5 proposed by Jim Gray et al. at Tandem Corp. dedicates $1/N^{th}$ of disk capacities to parity strips and places them contiguously on N disks in (Gray et al., 1990).

In the case of a write intensive workload to small randomly placed blocks placing parities at the middle disk cylinders will improve performance by reducing seek distances. When a disk fails the parities can be overwritten to reconstruct the contents of a failed disk, resulting in a RAID0 array, but otherwise a spare disk can be used for this purpose.

5.8.3 Load balancing in disk arrays without striping

The load imbalance associated with nonstriped disk arrays can be dealt with load balancing tools such as the *Placement Optimization Program* - POP developed at IBM T.J. Watson Research Center - TJWRC (Wolf, 1989). POP obtains mean

disk response times assuming Poisson arrivals and exponential disk service times to obtain a file allocation that minimize the overall mean response time.

The arrival rate to the i^{th} disk is λ_i and the disk service times are assumed to be exponentially distributed with mean $\bar{x}_i$ so that the mean response time at disk i is $R_i = \bar{x}_i / (1 - \rho_i)$, where $\rho_i = \lambda_i \bar{x}_i$ is the disk utilization. The mean response times over N disks can then be expressed as a weighted function of individual disk response times:

$$R = \sum_{i=1}^N \frac{\lambda_i}{\Lambda} R_i, \text{ where } \Lambda = \sum_{i=1}^N \lambda_i.$$

It is assumed that the rate of requests to disk files is known and the arrival rate to disk i is the sum of the arrival rates to files which are assigned to the disk, POP considers two heuristic optimization methods to tweak file allocations to minimize the overall mean response time rather than carrying out a costly reallocation of all files.

An analytic study to compare RAID5 performance with striping versus RAID5 with parity striping is reported in (Chen and Towsley, 1993). RAID5 striping usually assumes large enough strips to accommodate most disk requests with a single disk access, rather than each logical request accessing multiple disks whose number is given by a probability distribution postulated by the authors. In the case of parity striping large disk accesses are satisfied by sequentially reading/writing blocks from/to a single disk, so parity striping is more susceptible to access skew.

Allocation of data to minimize the mean response time of secondary storage devices in the context of a closed *Central Server Model - CSM* was considered in (Buzen and Chen, 1974).

Disk load balancing to minimize mean response time using an open queueing model with Poisson arrivals and general service times was investigated in (Piepmeier, 1975). There are N disk whose service times (mean and variance) vary across the disks. The total arrival rate of requests is fixed $\Lambda = \sum_{i=1}^N \lambda_i$, but λ_i can be varied. The P.K. formula for mean waiting time for M/G/1 models and the Lagrange multiplier method to obtain the allocation that minimizes overall mean response time. The optimal allocation does not utilize all N disks.

https://en.wikipedia.org/wiki/Lagrange_multiplier

The load allocation policy described in (Georgiadis et al., 2004) was developed at IBM TJWRC by Leonid Georgiadis (U. Thessaloniki, Greece), Christos Nikolaou (rip) and the author in (Georgiadis et al., 2004) for allocating load on N heterogeneous processors with different speeds. The *Min-Max Policy - MMP* described in (Ibaraki and Katoh, 1988) is used in this study to minimize the maximum expected response time. The optimization goal is achieved by routing requests to the fastest $M \leq N$ processors, with rates that equalize their mean response times, so that the slower servers are not be utilized.

5.9 RAID5 performance analysis in normal mode

We start with the performance analysis in normal, degraded and rebuild modes. Two cases are considered XOR capability at the disks or at the *Disk Array Controller - DAC*, which equipped with a cache which is partially non-volatile.

To update a data block d_{old} with d_{new} in RAID5 the following steps are required if the disks have an XOR capability.

1. The disk holding the data block after receiving the new value of the block (d_{new}) and its address reads the old data block (d_{old}) and computes $d_{diff} = d_{old} \oplus d_{new}$, sends d_{diff} and its address to the parity disk (via the DAC), overwrites d_{old} with d_{new} after one disk rotation,
2. Parity disk receives d_{diff} and its address, accesses p_{old} , computes $p_{new} = p_{old} \oplus d_{diff}$, overwrites p_{old} with p_{new} after one disk rotation.

There are two *Read-Modify-Write - RMW* accesses.

If disks do not have an XOR capability the DAC given d_{new} and its address accesses d_{old} and p_{old} , if they are not cached, d_{diff} is computed by the DAC, XORed with p_{old} to yield p_{new} , which with d_{new} is sent to the respective disk to be written.

The writing of a single block in RAID5 thus incurs four disk accesses if the p_{old} and d_{old} data blocks are not cached. This is referred to as the *Small Write Penalty - SWP* in (Chen et al., 1994b).

A list of techniques to improve performance in updating small disk blocks is as follows.

Cache Modified Blocks in NVRAM The method proposed in (Menon and Courtney, 1993) allows *fast writes*, i.e., a disk write is considered completed as soon as the data is written onto the NVRAM. A duplexed NVRAM is shown in this work to be as reliable as magnetic disk storage in this study.

Caching dirty blocks and destaging them in batches improves efficiency of writing dirty blocks to disk. A comprehensive analysis of disk I/O traces at IBM/ARC in (Treiber and Menon, 1995) showed the spatial locality of destaged blocks, e.g., the occurrence of two blocks at a short distance on the same track. Another advantage of caching in NVRAM is that dirty blocks are overwritten several times, before they are destaged, hence reducing the number of disk accesses for writing dirty blocks. Both effects are taken into account in the queueing analysis of distributed sparing in (Thomasian and Menon, 1997). Destaging dirty blocks as the buffer gets filled is proposed in (Treiber and Menon, 1995), while an alternate scheme which starts destaging based on the rate the buffer is filling (Varma and Jacobson, 1998).

A periodic update policy for dirty blocks is analyzed in (Carson and Setia, 1992) to obtain the average response time for read requests. Read and write load, the cache-hit ratio, and the disk scheduler's ability to reduce service time are taken into account. The design criteria can be used to choose among competing cache write policies. It is concluded that the bulk load generated by the periodic destage policy results in overload which severely affects read response times, but an effective use of the disk cache and disk scheduling can alleviate this problem in a narrow operating range.

Floating Parity method proposed and analyzed by researchers at IBM/ARC (Menon et al., 1993) allocates empty space for parity blocks on disk tracks, so that parity blocks can be written with reduced rotational latency.

Parity logging scheme was proposed and evaluated by researchers at CMU/PDL (Stodolsky et al., 1994). The differences between the old and modified data blocks are logged onto disk, sorted periodically according to increasing *Logical Block Addresses - LBAs* of parity blocks to be updated, and then applied to the parity blocks using a SCAN disk scheduling policy.

Disk architecture for Composite Operations - DACO: is a proposal by Mingqiang Li (IBM, Lenovo, Huawei, Kanzhun Ltd.) and Jiwu Shu at Tsinghua U. to place a write head followed by a read head to "eliminate disk rotation time" (Li and Shu, 2010). Shortcomings of this approach discussed in (Thomasian, 2010b, 2012a) are summarized below.

1. Implementing the R/W heads for DACO is unsurmountably difficult.
2. Adding this feature to disk drives will make them less cost-competitive.
3. Write holes are prevented by computing check blocks at the DAC, which does not utilize DACO.
4. Rotation time is smaller for high-performance enterprise disks where DACO is most pertinent.
5. A small fraction of manufactured disk drives are used as components in RAID(4 + k), $k \geq 1$ arrays.
6. There are numerous methods to deal with the small write penalty, most notably taking advantage of NVS.
7. The mean disk response times for read accesses can be improved by processing them at a higher priority than update requests, even when disks have a high utilization.
8. Scheduling updates adjacent requests in batches reduces disk utilization.
9. DACO cannot assist XOR operations involving multiple disks, such as reconstructing missing blocks and rebuild processing of failed disks.
10. DACO may not be able to assist with *Intra-Disk Redundancy - IDR* and disk scrubbing, while microprocessors associated with disks are capable of implementing more complex operations, such as those associated with active disk technology.
11. The efficiency of RMW accesses can be improved by adopting alternative disk organizations, such as a disk drive with multiple arms.
12. There is a trend towards high performance data storage in NVRAM or even DRAM which makes DACO associated with HDDs irrelevant.

Comments on DACO by Peter Desnoyers at Northeastern U.

A similar technique is used for disk formatting, which takes advantage of a known offset between the read and write heads to accurately write the sync pulses. However, the geometry does not seem to work for this new application. Observing the head assembly off a dead drive it appears that the actual head (i.e. the part produced by IC fab techniques) is much smaller than 1 mm in the longitudinal direction. With a 3.5" outer track (280 mm) holding about 2 million bytes, 1 mm is about 7000 bits or less than two 4 KB sectors. In other words, one would need a larger head than is used today in order to be able to read a full sector (in order to ECC it before applying an update), before the first bit of the sector reaches the write head. indexDesnoyers, Peter

Head skew would be the other geometric problem. On the disk drive under consideration the head swing is roughly 1/2 the length of the arm, or 1/4 its length to either side of the center. If the read and write heads are spaced about 4 KB apart longitudinally, i.e. about 0.5-0.6 mm - then if they line up on the same track in the middle, their relative distance from disk center will differ by about 0.15 mm. With current track widths running well under 100 nm, that equals thousands of tracks.

To summarize, providing nonvolatile storage to cache writes, which allows deferred optimized batch destaging is the preferred method.

5.10 RAID(4 + k) disk arrays in normal and degraded mode

Arrays with k check disks out of N disks are referred to as RAID(4 + k), e.g. RAID5 has one check disk ($k = 1$). To write a data block in the case of RAID(4+k) disk array $k + 1$ reads are required to access data and check blocks followed by $k + 1$ writes, $2(k + 1)$ disk accesses altogether. The i^{th} check block is modified by d_{diff} multiplied by the coefficient $incr_i$ (Li and Shu, 2010), corresponding to the coefficients of Vandermonde matrix (Plank, 1997a), (Plank and Ding, 2005). The Vandermonde matrix is discussed in more detail in Section 5.22.1.

https://en.wikipedia.org/wiki/Vandermonde_matrix

After p_{old}^i is read the disk computes

$$p_{new}^i = p_{old}^i \oplus d_{diff} \oplus incr_i,$$

which overwrites p_{old}^i after one disk rotation.

Due to striping the load for reading and updating data blocks at all disks is balanced. The left symmetric organization for check blocks results in a balanced parity update load at all disks (Chen et al., 1994b), (Thomasian and Blaum, 2009). The load per disk is a weighted sum of the loads due to read and write requests.

Let f_r denote the fraction of reads and $f_w = 1 - f_r$ the fraction of writes. Parities are computed at the DAC so writes are processed as *Single Reads - SRs* and *Single Writes - SWs*. The maximum throughputs RAID0:DAC0(k+4):RAID(k+4):RAID(k+4)P per disk is the inverse of the disk loads at full disk utilization.

$$\begin{aligned}\lambda_{RAID0}^{max} &= [f_r \bar{x}_{SR} + f_w \bar{x}_{SW}]^{-1}, \\ \lambda_{DAC0(k)}^{max} &= [f_r \bar{x}_{SR} + (k + 1) f_w \bar{x}_{SW}]^{-1}, \\ \lambda_{RAID(k+4)}^{max} &= [f_r \bar{x}_{SR} + (k + 1) f_w \bar{x}_{RMW}]^{-1}, \\ \lambda_{RAID(k+4)P}^{max} &= [f_r \bar{x}_{SR} + (k + 1) f_w (\bar{x}_{SR} + \bar{x}_{SW})]^{-1}.\end{aligned}$$

The maximum throughputs for the various cases can be obtained by postulating disk service time parameters. The OLTP workload issues accesses to 4 KB blocks, whose transfer time is negligible. The disk service time is determined by disk positioning time, which is the sum of seek time and rotational latency approximated by one half of disk rotation time. We postulate 10,000 RPM disks with $T_{rot} = 6$ milliseconds (ms) and mean seek time set at 4 ms, so that $\bar{x}_{SW} \approx \bar{x}_{SR} = 10$ ms and $\bar{x}_{RMW} = 16$ ms.

In the four cases we have the following *I/Os Per Second - IOPS* per disk:

$$\begin{aligned}\lambda_{RAID0}^{max} &= 100, \\ \lambda_{DAC0(k+4)}^{max} &= 100[1 + f_w k]^{-1}, \\ \lambda_{RAID(k+4)}^{max} &= 100[1 + f_w (0.6 + 1.6k)]^{-1}, \\ \lambda_{RAID(k+4)P}^{max} &= 100[1 + f_w (1 + 2k)]^{-1}.\end{aligned}$$

To illustrate the degradation in throughput we consider RAID6 and assuming a large cache R:W = 1:1 and $f_w = 0.5$:

$$\lambda_{RAID0}^{max} = 100, \lambda_{DAC06}^{max} = 50, \lambda_{RAID6}^{max} \approx 34.5, \lambda_{RAID6P}^{max} \approx 28.6.$$

Disk service time $\tilde{x}_d$ is the sum of three random variables representing the times for the three components of mechanical disk arm movement: (1) seek time $\tilde{x}_s$, (2) latency $\tilde{x}_\ell$, and (3) transfer time $\tilde{x}_t$ plus disk controller time, which is negligible. Components of disk service time are estimated in Chapter 3.

The mean time to read a disk block is the expected value of the sum of its three components:

$$\bar{x}_d = \bar{x}_s + \bar{x}_\ell + \bar{x}_t.$$

Assuming that the three mechanical components of disk service time are independent the variance of service time is (Trivedi, 2002):

$$\sigma_x^2 = \sigma_s^2 + \sigma_\ell^2 + \sigma_t^2.$$

The second moment of disk service time is then $\overline{x_d^2} = \sigma_d^2 + (\overline{x_d})^2$.

Let $\overline{x}_{SR}$ and $\overline{x}_{SW}$ denote the mean service time for *Single Read - SR*, *Single Write - SW*, and *Read-Modify-Write - RMW* requests, then $\overline{x}_{SR} = \overline{x}_d$, $\overline{x}_{SW} = \overline{x}_d + T_h$, and $\overline{x}_{RMW} = \overline{x}_{SR} + T_r$, where T_h and T_r are the head settling time and disk rotation time, respectively.

The fraction of logical read and update requests is denoted by f_r and $f_w = 1 - f_r$, respectively. The maximum throughputs for RAID(4 + k) per disk with RMW and SR and SW requests are given as follows:

$$\lambda_{RAID(4+k)}^{max} = [f_r \overline{x}_{SR} + (k+1) f_w \overline{x}_{RMW}]^{-1},$$

$$\lambda_{RAID(4+k)}'^{max} = [f_r \overline{x}_{SR} + (k+1) f_w (\overline{x}_{SR} + \overline{x}_{SW})]^{-1}.$$

In fact if SW requests are not processed immediately following the SR request and there is an intervening disk request then $\tilde{x}_{RMW} \approx \tilde{x}_{SR} + \tilde{x}_{SW}$.

$$\overline{x^2}_{RMW} = \overline{x^2}_{SR} + \overline{x^2}_{SW} + 2\overline{x}_{SR}\overline{x}_{SW}.$$

Otherwise with no intervening disk requests: $\overline{x}_{RMW} \approx \overline{x}_{SR} + \overline{x}_{SW}$.

We utilize the M/G/1 queueing model with Poisson arrivals and general distribution for service times to estimate the mean disk response time (Kleinrock, 1975). The Poisson arrival process is a good approximation for SR requests, since they are generated by OLTP applications processed at a high degree of concurrency, but not so for write requests, which are cached and destaged in batches (Thomasian et al., 2007a). Writes are processed at a lower priority and write response times do not affect txn response times. The effect of locality in destaging writes based on I/O trace analysis of the workload of an IMS database are reported in (Treiber and Menon, 1995) and taken into account in (Thomasian and Menon, 1997).

Let Λ denote the arrival rate of logical read and write requests to the disk array. In normal mode the arrival rate of read requests to each disk is $\lambda_R^n = f_r \Lambda / N$ and the arrival rate for RMW requests is $\lambda_W^n = 2f_w \Lambda / N$, since both data and parity blocks need to be updated. Since each write request in RAID5 requires the updating of data and parity blocks disk utilizations are as follows:

$$\rho = \rho_{SR} + \rho_{RMW} = \lambda f_R \overline{x}_{SR} + 2\lambda f_W \overline{x}_{RMW}.$$

In case disks do not have an XOR capability to allow RMW requests then independent SR and SW requests are issued and hence:

$$\rho = \lambda f_R \overline{x}_{SR} + 2\lambda f_W (\overline{x}_{SR} + \overline{x}_{SW})$$

If RMW requests are allowed:

$$\rho = \lambda f_R \overline{x}_{SR} + 2\lambda f_W \overline{x}_{RMW}.$$

The mean waiting times with FCFS scheduling is (Kleinrock, 1976).

$$W = \frac{W_0}{1 - \rho}, \quad W_0 = \frac{1}{2} [\lambda f_R \overline{x^2}_{SR} + 2\lambda f_W \overline{x^2}_{RMW}], \quad (5.1)$$

where W_0 is the delay encountered due to the request in service at the disk.

The mean response time of SR requests processed at a higher priority than RMW can be obtained by substituting ρ with the disk utilization solely due to SR requests in Eq. (5.1).

$$\rho_{SR} = \lambda f_R \overline{x}_{SR}.$$

Given $\rho_{SR} = 0.2$ and $\rho_{RMW} = 0.4$, $W_{FCFS} / W_{Priority} = (1 - 0.2) / (1 - 0.6) = 0.8 / 0.4 = 2$, i.e., a two-fold improvement. The mean response time for SR requests is given as $R_{SR} = \overline{x}_{SR} + W$.

Full stripe writes can be processed efficiently as RAID(4 + k), $k \geq 1$, since the check strips can be computed on the fly. This is the reason why full stripe writes were adopted in the Iceberg disk array. The RCW - ReConstruct Write method is an approximation to full stripe writes when the majority of strips in a stripe are updated. The remaining strips in the stripe

are accessed and used to compute check blocks. RCW performance is compared with the RMW method in (Thomasian, 2005a).

A scheme for variable-scope parity protection was patented in (Frasaszek et al., 1996) and evaluated in (Frasaszek and Robinson, 1997). In this study parity applies to arbitrary subsets in each stripe.

RAID(4 + k) arrays can continue operation with up to k failed disks, but there is a degradation in disk response times, which is due to the increased load resulting from on demand reconstruction of missing blocks. The read load on surviving disks doubles with a single disk failure in RAID5 and in general increases $k + 1$ -fold in the case of RAID(4 + k) with k failed disks. The load increase with mixed read and write requests in clustered RAID5 and RAID6 arrays is quantified in (Thomasian, 2005a), but the methodology is extended to RAID7 in (Thomasian et al., 2012).

In degraded mode the read load on the $N - 1$ surviving RAID5 disks is doubled, because each surviving disk accesses its own data blocks and participates in a $N - 1$ -way fork/join requests to reconstruct the data on a failed disk:

$$\rho_R = \frac{2\Lambda f_r}{N}$$

In the case of write requests there are three cases ($\bar{d}$ and $\bar{p}$ indicate a failed block).

1. The probability that data and parity block are both available is $P[d \wedge p] = (N - 2)/N$. In this case two RMW requests will be required for data and parity blocks.
2. The probability that the data block is on the failed disk is $P[\bar{d}] = 1/N$. d_{new} is XORed with $N - 2$ data blocks read by SRs to compute p_{new} , which is written by an SW.
3. The probability that the parity block is on the failed disk is $P[\bar{p}] = 1/N$. There is no need to compute the parity and d_{new} is written to disk as an SW.

The cost per disk for writes is a weighted sum of the three cases, which should be multiplied by $\Lambda f_w/N$ to obtain the disk utilization.

$$\rho_W = \frac{\Lambda f_w}{N} \left[\frac{2(N - 2)}{N} \bar{x}_{RMW} + \frac{1}{N} ((N - 2)\bar{x}_{SR} + \bar{x}_{SW}) + \frac{1}{N} \bar{x}_{SW} \right].$$

The M/M/1 queueing system for analyzing RAID(4 + k) disk arrays with read requests, the mean disk response time with zero failures is given as $R_0 = \bar{x}_d/(1 - \rho_0)$, where the subscript denotes the number of failed when all requests are reads then with i disk failures $\lambda_i = (i + 1)\lambda_0$. The degradation in response time with one and two disk failures in RAID6 is: $R_1/R_0 = 2.5$ and $R_2/R_0 = 9$. A lower performance degradation can be achieved by adopting the clustered RAID5 paradigm.

5.11 Rebuild processing in disk arrays

Rebuild processing in RAID5 is important for the following reasons:

1. RAID5 disk arrays in degraded mode, with one failed disk, are susceptible to data loss if a second disk fails before rebuild is completed, so that the duration of this phase should be shortened as much as possible, while providing an acceptable response time for external disk requests.
2. RAID5 disk arrays have a poor performance when operating in degraded mode, which is another reason why rebuild processing should be started right away after a disk failure and its duration should be shortened without severely affecting the response time of external disk requests.
3. RAID5 with a single failed disk cannot deal with encountered unreadable blocks, i.e., *Latent Sector Errors - LSEs* (Bairavasundaram et al., 2007), (Schroeder et al., 2010b,a).
4. The rebuild time for very high capacity disks can exceed one day, so that judicious methods should be utilized to complete the rebuild in a timely manner.

While RAID5 is in degraded mode each disk processes its own requests as well as fork-join requests on behalf of the failed disk. *Clustered RAID - CRAID* or parity declustering proposed in (Muntz and Lui, 1990b) to reduce the load of RAID5 disk arrays operating in degraded mode by setting the parity group size G to a value smaller than N , the number of disks. The increased load for read requests in RAID5 is given by the declustering ratio: $\alpha = (G - 1)/(N - 1) < 1$, i.e., smaller than 100% which would have been incurred otherwise. Disk loads for read requests are increased by a factor of $1 + \alpha < 2$ with clustering, rather than doubled without clustering.

The *Rebuild Unit - RU* is the smallest unit of rebuild processing which has a fixed size. with the advent of zoning *Zoned Bit Recording - ZBR* an appropriate RU size is a track, so that all rebuild transfers take the same amount of time, which is one disk rotation. In the case of ZBR the number of sectors per track increases from inner to outer tracks and this results in increased track capacity (Ng, 1998). This complicates buffer management in rebuild processing. It is appropriate to set the RU size to an appropriate fraction of SU size, since the mean waiting time of external disk requests is positioning time plus data transfer time.

Two variations of rebuild processing are considered in (Holland and Gibson, 1992; Holland et al., 1994; Holland, 1994).

Stripe-Oriented Rebuild - SOR proceeds one stripe at a time in a non-clustered RAID5 and one parity group at a time in clustered RAID5. This is provided that the *Rebuild Unit - RU* has same size as an SU, but this is not generally true. This method does not fully utilize disk bandwidth, since it reads RUs via fork/join requests with the associated synchronization delay to rebuild one RU at a time (Thomasian, 2014b).

Disk-Oriented Rebuild - DOR reads data from surviving disks concurrently. As in SOR RUs are read at a lower priority than external disk requests, i.e., the reading of RUs at each disk is started when the queue of external disk requests is empty. Successive RUs are read until an external disk request arrives, at which point the reading of the current RU is completed, but no new RUs are read.

DOR can be analyzed using the *Vacationing Server Model - VSM*, where successive RUs are read until the first external request arrives. After a disk completes processing external requests and an idle period starts then the disk restarts reading successive RUs. The first track read for rebuild requires a seek, while the following reads do not. We have a queueing model with multiple vacations with variable durations (Doshi, 1986). The analysis is also given in “Vacations and Priority Systems” (Takagi, 1991).

Fig. 5.5 shows the processing of external requests constituting the periods, while rebuild processing is carried out during idle periods on surviving disks by reading successive RUs. The reading of the first RU in each idle period requires a seek and hence takes more time. The RU size is set to be a track and with *Zero latency Read - ZLR* capability the reading of successive tracks will take one disk rotation. Rebuild processing is stopped when an external disk request arrives.

For a disk array with N disks allocate N RU-sized blocks in the DAC buffer: $N - 1$ RU buffers for reading from surviving disks and the N^{th} buffer holds the RU obtained by XORing $N - 1$ RUs, which is to be written to the spare disk. The sizes of the buffers can be varied as rebuild progresses according to track size. It is beneficial to start rebuild with the outermost track, since more data is read in one disk rotation. Even if disk loads are balanced reading from some disks can proceed faster than others, so a number of buffers need to be dedicated in each column. The number of rows required for RUs is $i_{max} - i_{min} + 1$, where i is the stripe number. If all n rows for a disk are occupied then reading from that disk is stopped and this will result in rebuild processing slowdown. In fact once all strips in a parity group size are available, their space can be released after they are XORed and the resulting block is written.

A fault injection study was used in (Brown and Patterson, 2000) to provoke situations to compromise RAID5 availability. Various aspects of fault-injection are discussed here.

https://en.wikipedia.org/wiki/Fault_injection

Experiments were carried out with software RAID systems shipped with Linux, Solaris 7 Server, and Windows 2000 Server. Single fault and multiple fault microbenchmarks were considered. The methodology was shown to be powerful enough not only to quantify the impact of various failure conditions on the availability of these systems, but also to unearth their design philosophies with respect to transient errors and recovery policy.

Linux's software RAID implementation takes a totally opposite approach to the management of transient faults than Solaris and Windows in that the Linux implementation shuts down a disk in a controlled manner at the first error, rather than wait to see if the error is transient. In contrast, Solaris and Windows ignore most transient faults and thus these systems are substantially more robust to transients than the Linux system. Note that both Windows and Solaris do log the transient errors even if not acting upon them.

5.11.1 Variations of rebuild processing in RAID5

The following variations of rebuild processing in RAID5 are proposed in (Muntz and Lui, 1990b):

Baseline rebuild reconstruct the contents of a failed disk on a spare disk. The missing RU is materialized by XORing RUs as they become available and the RU is written to the spare disk. External accesses to missing blocks are processed by issuing fork/join requests to corresponding blocks on surviving disks.

Read redirection is an obvious improvement with respect to baseline rebuild by keeping track of materialized blocks on the spare disk and processing requests intended for the failed disk by accessing these blocks. The three advantages of read redirection are:

1. The load on surviving disks is lowered as rebuild progresses, which results in improved response times on the surviving disks.
2. The response time for accesses to the failed disk improves, since fork-join requests tend to take more time due to the synchronization effect.
3. The lowered disk utilizations will result in a reduced rebuild time, so that RAID5 will be less susceptible to a second disk failure.
4. With lower rebuild time fewer external disk requests will be affected by rebuild processing.

Piggybacking was an attempt to improve rebuild time by writing data blocks reconstructed on demand on the spare disk (Muntz and Lui, 1990b). The problem with this method is that rebuilding one of many blocks on a disk track does not result in a reduced rebuild time. In fact for low values of $\alpha = (G - 1)/(N - 1)$ the spare disk may become a bottleneck and this effect is exacerbated by piggybacking. Furthermore piggybacking does not reduce rebuild time as shown via simulation in (Holland and Gibson, 1992), (Holland et al., 1994; Holland, 1994), which contradicts the analysis in (Muntz and Lui, 1990b).

The simulation study of piggybacking reported in (Fu et al., 2004a) instead of reconstructing one block, reconstructs the whole track on which the block resides. This option results in an increased response time for user requests, but reduces rebuild time as would be expected.

Hot mirroring was proposed at U. Tokyo in (Mogi and Kitsuregawa, 1996) to improve RAID5 performance in normal mode as well as rebuild mode. Hot data blocks are stored on mirrored disks, so that this method is similar to HP's AutoRAID array (Wilkes et al., 1996).

The preemption of rebuild reads can be used to improve the response time of user requests. A split-seek option, which abandons a rebuild read after a seek is completed if there is an external disk request pending is considered in (Thomasian and Menon, 1994). Preemptions of rebuild read requests during the latency and transfer phases are evaluated via simulation in (Thomasian, 1995a). The improvement in user response times should be weighed against the increase in rebuild time, i.e., even more disk requests will encounter an increased response time if the rebuilt period is elongated.

Another scheme to minimize rebuild time is to provide several anchor points serving as starting points for the rebuild process. For example, placing anchor points at cylinders $C/4$ and $3C/4$ will reduce the mean seek distance from $C/3$ to $C/8$ cylinders. Simulation studies in (Holland, 1994) have shown negligible improvement in performance, which is at the cost of increased buffer space requirement.

The anchors for rebuild may be based on current hotspots in the workload. The *Popularity-based multi-threaded Reconstruction Optimization - PRO* method proposed in (Tian et al., 2007) by Lei Tian (U. Nebraska, Tintri), Hong Jiang (U. Texas at Arlington), Dan Feng (Huazhong U.) et al. in adjusts the anchors based on hotspots. This has two advantages:

1. Carrying out rebuild reads in disk areas being accessed by external workloads minimizes the seek distance.
2. Rebuilding the hot spot areas results in reducing the disk load on surviving disks more quickly than would be possible otherwise.

I/O workload outsourcing for boosting RAID5 reconstruction performance by setting up a surrogate RAID, which may be a RAID1 or RAID5 was proposed in Suzhen Wu (Xiamen U.) and the author of (Tian et al., 2007) in (Wu et al., 2009).

Rebuild time can be reduced by associating empty/full tags with disk blocks, so that the rebuilding of empty blocks can be bypassed. *Heterogeneous Disk Arrays - HDAs* discussed in Chapter 10 delineate disk space for *Virtual Arrays - VAs* and rebuild processing of heavily utilized arrays can be prioritized,

5.12 Vacationing server model for rebuild processing

RAID5 disks are modeled as independent M/G/1 queue with Poisson arrivals with rate λ (Kleinrock, 1975). Disk requests have a general service time with $\bar{x}^i$ denoting the i^{th} moment, so that the utilization factor is $\rho = \lambda \bar{x}$, which is also the mean number of requests being served according to Little's result and also the fraction of time the server is busy. ρ should be less than one to ensure that infinite queues are not formed.

An M/G/1 queueing system alternates between busy periods during which requests are served with mean duration $\bar{g}$ and idle periods determined by the interarrival time $\bar{i} = 1/\lambda$. It follows from its definition of the utilization factor $\rho = \bar{g}/(\bar{g} + \bar{i})$,

which yields the mean busy period:

$$\bar{g} = \frac{\bar{x}}{1 - \rho}. \quad (5.2)$$

The delay cycle is similar to a busy period and consists of an initial delay (Y_0), rather than the service time of a regular request, which leads to a delay busy period (Y_b) (Kleinrock, 1976). The delay cycle is the sum: $Y_c = Y_0 + Y_b$. The *Laplace-Stieltjes transform - LST* of delay cycle can be obtained similarly the busy period (Kleinrock, 1975).

$$G_c^*(s) = G_o^*(s + \lambda - \lambda G^*(s)).$$

When the first request is an ordinary request

$$G^*(s) = G[s + \lambda - \lambda G^*(s)],$$

whose i^{th} moment is $\bar{g}^i = -dG^{*(i)}(s)/ds^i|_{s=0}$. The first moment using an intuitive argument was given by Eq. (5.2), but a similar argument is used in (Thomasian, 2018c) to obtain the mean delay busy period:

$$\rho = \frac{\bar{d}_v - \bar{v}_r}{\bar{d}_v + 1/\lambda} \implies \bar{d}_v = \frac{\bar{x} + \bar{v}_r}{1 - \rho}. \quad (5.3)$$

According to VSM with multiple vacations with varying durations, upon the completion of a busy period the server takes an ordered sequence of vacations with different distributions $\mathcal{V}_i, i \geq 1$ (Doshi, 1986), (Takagi, 1991). The first vacation in Fig. 5.5 takes longer, since it requires a seek to the first *Rebuild Unit - RU*, e.g., a track, to be read beyond the point, where the rebuild process was last interrupted, while succeeding RUs/tracks can be read one after the other without incurring a seek.

The PDF - Probability Distribution Function of the i^{th} vacation $\mathcal{V}_i$ is denoted by $V_i(t)$ and given that q_i is the fraction of type i vacations, the PDF of overall vacation time is:

$$V(t) = \sum_{i \geq 1} q_i V_i(t). \quad (5.4)$$

The j^{th} moment of type i vacations is

$$\gamma_{i,j} = \int_0^\infty t^j V_i(t) dt.$$

The probability that an external request arrival in time interval $(0, t)$ is $P[arrival] = 1 - e^{-\lambda t}$ and the probability that there are no arrivals is $P[no_arrival] = e^{-\lambda t}$. The probability that i vacations are taken can be specified using the notation for LSTs in (Kleinrock, 1975):

$$p_i = [1 - \int_0^\infty dV_i(t)] \prod_{j=1}^{i-1} \int_0^\infty e^{-\lambda t} dV_j(t) = [1 - V_i^*(\lambda)] \prod_{j=1}^{i-1} V_j^*(\lambda). \quad (5.5)$$

The mean number of vacations per idle period is $\bar{J} = \sum_{j \geq 1} j p_j$ and the fraction of type i vacations is $q_i = \sum_{j \geq i} p_j$.

Fig. 5.5 shows rebuild processing using VSM with two types of vacations: $\mathcal{V}_1$ involving a seek followed by the read of the first track after a busy period, and $\mathcal{V}_2$ reading of successive tracks until an external arrival occurs. ZLR eliminates latency and allows reading of a track to start with the first encountered sector, so that the latency is negligible. We ignore track and cylinders skews, which amount to the transfer time of a few sectors, while the reading of tracks takes one disk rotation, which has hundreds of sectors. The probability that an arrival occurs during the i^{th} vacation is:

$$p_1 = 1 - V_1^*(\lambda) \text{ and } p_i = [1 - V_2^*] V_1^*(\lambda) [V_2^*(\lambda)]^{i-2}, i \geq 2. \quad (5.6)$$

The mean number of vacations with two vacation types is:

$$\bar{J}_2 = 1 + \frac{V_1^*(\lambda)}{1 - V_2^*(\lambda)}. \quad (5.7)$$

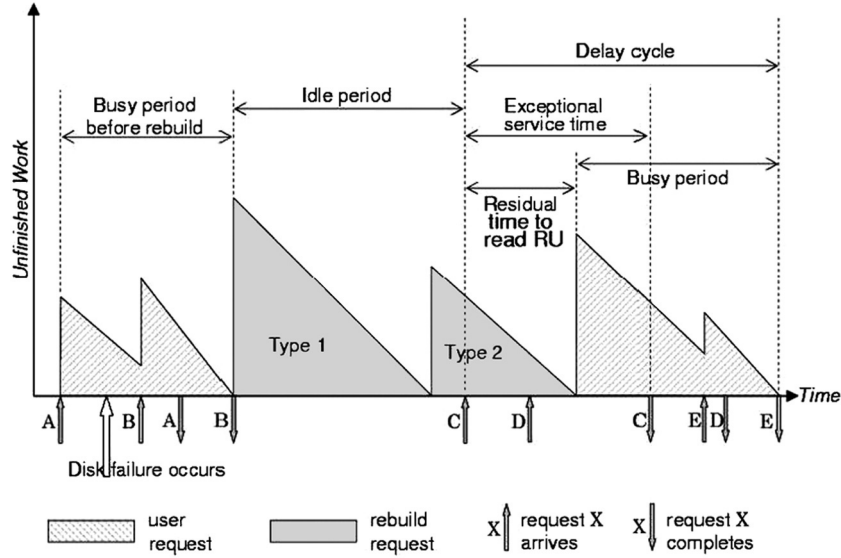


FIGURE 5.5 Key parameters associated with rebuild processing (Thomasian et al., 2007b).

The fraction of type 1 and 2 vacations is as follows:

$$q_1 = \frac{1}{1 + \frac{V_1^*(\lambda)}{1 - V_2^*(\lambda)}} = \frac{1 - V_2^*(\lambda)}{1 - V_2^*(\lambda) + V_1^*(\lambda)} \text{ and } q_2 = 1 - q_1. \quad (5.8)$$

The overall LST $V^*(s)$ is a weighted sum of LSTs of different vacation types:

$$V^*(s) = \frac{(1 - V_2^*(\lambda))V_1^*(s) + V_1^*(\lambda)V_2^*(s)}{1 - V_2^*(\lambda) + V_1^*(\lambda)} \quad (5.9)$$

The j^{th} moment of vacation time is then:

$$\gamma_j = \sum_{i \geq 1} q_j \gamma_{i,j} = \frac{(1 - V_2^*(\lambda))\gamma_{1,j} + V_1^*(\lambda)\gamma_{2,j}}{1 - V_2^*(\lambda) + V_1^*(\lambda)} \quad (5.10)$$

The mean residual vacation time required for estimating mean waiting time is $\bar{v}_r = \gamma_1$.

5.12.1 Effect of rebuild processing on external request response time

Let σ denote the fraction of failed disk blocks that have been reconstructed on a spare disk. When read redirection is in effect the arrival rate of reads in rebuild mode on surviving disks is:

$$\rho_R = \lambda_R(\sigma)\bar{x}_{SR} = \frac{\Lambda f_R(2 - \sigma)}{N}\bar{x}_{SR}. \quad (5.11)$$

The cost for writes can be given similarly to degraded mode by taking into account σ , so that the disk utilization for write requests is:

$$\rho_W = \frac{\Lambda f_w}{N} \left[\left(\frac{2(N-2)}{N} + \sigma \right) \bar{x}_{RMW} + \left(\frac{1}{N} - \sigma \right) ((N-2)\bar{x}_{SR} + \bar{x}_{SW}) + \left(\frac{1}{N} - \sigma \right) \bar{x}_{SW} \right].$$

Taking into account the fact that *Poisson Arrivals See Time Averages - PASTA* enunciated in (Wolff, 1982) the mean waiting time of external disk requests in an M/G/1 queueing system with vacations, whose i^{th} moment is $\bar{v}^i$ and whose mean residual vacation time we have:

$$W_{VSM} = \bar{N}_q \bar{x} + \rho \bar{x}_r + (1 - \rho) \bar{v}_r. \quad (5.12)$$

The first term is the mean waiting time in the queue, which is the product of mean queue length ($\bar{N}_q$) and mean service time ($\bar{x}$), the second term is the mean delay caused by an in-progress request at the server, and the third term the mean delay caused by vacations.

The mean delay at a busy server is the mean residual service time: $\bar{x}_r = \bar{x}^2/(2\bar{x})$ and arrivals during an idle period are delayed by the mean residual vacation time: $\bar{v}_r = \bar{v}^2/(2\bar{v})$. The above equation can be simplified by noting that according to Little's result $\bar{N}_q = \lambda W_{VSM}$ (Kleinrock, 1975) and hence:

$$W_{VSM} = \frac{\lambda \bar{x}^2}{2(1 - \rho)} + v_r = W_{M/G/1} + v_r, \quad (5.13)$$

i.e., all requests rather than just the first one is affected by the mean residual vacation or rebuild time (Menon, 1994). In the case of VSM the first vacation $\mathcal{V}_\infty$ involves a seek following the transfer time of the first track. The second vacation is the transfer time of successive disks, which requires a disk rotation. The response time for user requests can be reduced by the split-seek option, which can be modeled by two vacation types where the first vacation is a seek and the second and all successive vacations are disk rotations corresponding to the reading of successive tracks.

In the case of two vacation types $V_1^*(s) = X_{seek}^*(s) \times X_{track}^*(s)$ and $V_2^*(s) = X_{track}^*(s)$. In the case of three vacation types $V_1^*(s) = X_{seek}^*(s)$, $V_2^*(s) = X_{track}^*(s)$ and $V_i^*(s) = X_{track}^*(s)$, $i \leq 3$. The analysis in the latter case in (Thomasian and Menon, 1994) can be simplified by noting that two types of vacations suffice, since $V_2^*(s) = V_i^*(s)$, $i \geq 3$.

The split-seek option is taken a step further in (Thomasian, 1995a), which allows preemption during the latency and even during data transfer stages. The smaller RU resulting from such preemptions results in an increased rebuild time, so that more external requests are processed while rebuild is in progress. Preemptions are inefficient since multiple visits involving seeks are required to the same track to read all sectors. A new metric proposed in this work is defined in this work to compare the efficiency of rebuild options.

“excess cumulative mean response time with respect to case with no disk failures”

While preempting rebuilds reduces response times it elongates rebuild time, so that more requests are processed in degraded mode.

Relevant to this discussion is “systems support for preemptive disk scheduling” (Dimitrijevic et al., 2005).

“Allowing higher-priority requests to preempt ongoing disk IOs is of particular benefit to delay-sensitive and real-time systems. In this paper, we present Semi-preemptible IO, which divides disk IO requests into small temporal units of disk commands to improve the preemptibility of disk access. We first lay out main design strategies to allow preemption of each component of disk access: seek, rotation, and data transfer, namely, seek-splitting, JIT-seek, and chunking. We then present the preemption mechanisms for single and multidisk systems: JIT-preemption and JIT-migration. The evaluation of our prototype system showed that Semi-preemptible IO substantially improved the preemptibility of disk access with little loss in disk throughput and that preemptive disk scheduling could improve the response time for high-priority interactive requests.”

The rate of read requests at the start of rebuild processing is twice the rate in normal mode at surviving disks. This rate drops gradually until it returns to normal mode as a result of read redirection at which point the spare disk replaces the failed disk completely. The variation in disk utilization due to read redirection in computing $T_{rebuild}(\rho)$ can be taken into account by computing rebuild processing time over k intervals (Thomasian and Menon, 1994).

$$T_{rebuild}(\rho) = \sum_{i=1}^k T_{rebuild}(\rho_i), \quad \rho_i = [2 - \frac{2i-1}{2k}] \rho. \quad (5.14)$$

Note that rebuild times diminish over successive periods as more blocks are reconstructed on the spare disk due to read redirection.

The reading of surviving disks for rebuild takes approximately the same time at all disks, since disk loads are balanced due to striping and uniform parity placement, such as the left symmetric parity organization (Lee and Katz, 1993a). The coefficient of variation of rebuild time increases with the arrival rate, but remains below 3% in the case of distributed sparing as shown in Figure 4 in (Thomasian and Menon, 1997).

Since the writing of the RUs is pipelined with disk reading with dedicated sparing, rebuild time can be approximated by the time to complete the reading time of disks. This is because the spare disk is not a bottleneck in dedicated sparing, but

this might not be the case with CRAID. Performance in rebuild mode can be improved by reducing the load on the spare disk below of what it would have been due to read redirection based on the progress of rebuild processing.

Rebuild time increases rapidly with higher disk utilization (ρ) due to external disk requests. Given that $T_{rebuild}(0)$ is the time for reading an idle disk, curve-fitting against simulation in the case of a read-only workload yields (Fu et al., 2004a)

$$T_{rebuild}(\rho) = \frac{T_{rebuild}(0)}{1 - \beta\rho} \text{ with } \beta = 1.75.$$

The analysis of VSM with dedicated sparing in (Thomasian and Menon, 1994) is extended to distributed sparing in (Thomasian and Menon, 1997). An iterative solution is required to take into account the effect of rebuild writes, whose rate is $N - 1$ times smaller than rebuild reads, while both are affected by the external disk load. Three methods to analyze RAID5 rebuild processing are reported in (Thomasian et al., 2007b).

5.12.2 Rebuild unit sizes in RAID5

Rebuild processing in degraded mode further affects the response time of external disk requests. In what follows we discuss the effect of RU size on response time of external disk requests using DOR and rebuild time, which can be analyzed using VSM - vacationing server queueing model.

The tradeoff between the response time to external disk requests and rebuild time was investigated in a simulation study reported in (Hou et al., 1993). The RU size was increased from a 4K block, to a track, to a cylinder.

A 4 KB block is not a suitable RU size, since only the transfer time for the reading of the block constitutes useful work and the track has to be visited as many times as many times as the are blocks on the track each time incurring unproductive positioning time. The number of sectors per track increased considerably since this paper was written.

A track or multiple tracks are suitable RUs, because of ZLR - Zero Latency Read capability allows the reading of a track to be started at any disk sector, so that it takes one disk rotation (Ng, 1998), (Jacob et al., 2008).

Setting the RU to be a track was not an issue when the paper was written, but it complicates buffer management with ZBR - Zoned Bit Recording, because of variable track capacities. Plus this does not conform with the original definition that RUs have a fixed size.

Assuming that other factors, such as the bandwidth of buses and the hardware/software mechanism for computing parities do not slow down rate of transmission for disk blocks then rebuild time can be approximated by the time to read a disk, which is elongated by the processing of user requests. Rebuild time will be increased when the spare disk becomes a bottleneck, which is the case with *Clustered RAID - CRAID* (Muntz and Lui, 1990b).

5.12.3 Permanent Customer Model - PCM and comparison with VSM

PCM was proposed and analyzed in (Boxma and Cohen, 1991). It was adopted in several studies of RAID5 and RAID1 rebuild processing (Merchant and Yu, 1992, 1994, 1995, 1996).

PCM allows a single rebuild read to be inserted into the queue of pending disk requests as soon as another rebuild read is completed. Given that disk requests are processed in FCFS order, the arrival of disk requests while a rebuild read is enqueued and being processed will result in more interruptions in rebuild reads than with VSM. A comparison of VSM versus PCM is as follows:

1. VSM improves the response time of external disk requests, since rebuild reads are processed at a lower priority i.e., only when the queue of external disk requests is empty.
2. Rebuild processing with VSM takes less time than PCM, since more rebuild reads are processed one after the other and hence incur fewer seeks. This can be explained by the fact that PCM rebuild reads after spending time in the queue have a higher probability of being intercepted by user requests, so that there are fewer opportunities for the processing of consecutive rebuild read requests. The probability that rebuild requests are intercepted by user requests arriving with rate λ with VSM and PCM rebuild are given in (Fu et al., 2004a):

$$P_{VSM} = 1 - e^{-\lambda \bar{x}_{RU}}, \quad P_{PCM} = 1 - e^{-\lambda(\bar{x}'_{RU} + W_{RU})},$$

x_{RU} and x'_{RU} are the times to read an RU using the VSM and PCM methods, which are treated as constants set equal to their mean values: $\bar{x}_{RU}$ and $\bar{x}'_{RU}$ to simplify the discussion. W_{RU} is the mean waiting time in the queue for rebuild request in PCM. Since $\bar{x}'_{RU} + W_{RU} > \bar{x}_{RU}$ implies $P_{VSM} < P_{PCM}$.

3. Given that rebuild time with VSM takes less time than rebuild time with PCM fewer external requests are processed while rebuild processing is in progress.

Instead of reading one track at a time, the RU can be set to more than one track. Increasing the number of tracks and more generally using a larger RU size is expected to reduce rebuild time, but this is at the cost of increased response time for external disk accesses as shown by the analysis. The effect of varying the RU size is investigated in Section 5.12.2.

5.13 RAID5 sparing configurations for rebuild

Upon a single disk failure, the RAID5 rebuild process reads RUs from surviving disks, which are in the same parity group as an RU on a broken disk, XORs them to reconstruct the missing RU which is then written onto a spare disk drive in dedicated sparing (Menon, 1994), (Thomasian and Menon, 1994). A hot spare allows the rebuild process to be started right away, thus reducing the window of vulnerability to data loss due to a second disk failure.

Given that the MTTF of modern drives may be as high as a million hours, the possibility of a second failure during rebuild is rather small, unless disk drive failures are correlated, e.g., they belong to the same manufacturing batch with a common fault. It is much more likely than the rebuild process will not be able to complete due to unreadable disk sectors if a *Latent Sector Errors - LSEs* is encountered.

Bypassing unreadable sectors is an option not discussed in earlier studies. It is possible that the LSE is located in an unallocated disk area and even if allocated the area which is not referenced again.

An advantage of dedicated sparing is that the spare can be shared among several arrays in (Ng, 1994a). As failed disks are replaced by a new disk, it becomes the new spare. Markov chain modeling is used to show that sharing spare disks has little effect on *Mean Time to Data Loss - MTDDL*, but has a significant effect on the mean time to service calls for arrays, which affects the operational cost of the array, i.e., *Total Cost of Ownership - TCO*. Disk replacement may be carried out periodically and in batches to reduce replacement cost. Permanent versus dynamic sparing in the context of EMC's Symmetrix disk arrays in Chapter 4 is permanent to this discussion.

5.13.1 Rebuild by distributed sparing

The wasted bandwidth of the spare drive in dedicated sparing is eliminated by adopting distributed sparing, which is shown in Fig. 5.6, i.e., allocating enough empty space on each drive to hold the data of a failed drive (Menon and Mattson, 1992a), (Thomasian and Menon, 1997). The rebuild process with distributed sparing is slowed down due to the fact that rebuild reads contend with rebuild writes for disk bandwidth of surviving disk, but disk loads remain balanced.

D_1	D_2	D_3	D_4	D_5	D_6	D_1	D_2	D_3	D_4	D_5	D_6	D_1	D_2	D_4	D_5	D_6
d_1	d_2	d_3	d_4	$p_{1:4}$	s_1	d_1	d_2	X	d_4	$p_{1:4}$	s_1	d_1	d_2	d_4	$p_{1:4}$	d_3
d_6	d_7	d_8	$p_{5:8}$	s_2	d_5	d_6	d_7	X	$p_{5:8}$	s_2	d_5	d_6	d_7	$p_{5:8}$	d_8	d_5
d_9	d_{10}	$p_{9:12}$	s_3	d_{11}	d_{12}	d_9	d_{10}	X	s_3	d_{11}	d_{12}	d_9	d_{10}	$p_{9:12}$	d_{11}	d_{12}
d_{13}	$p_{13:16}$	s_4	d_{14}	d_{15}	d_{16}	d_{13}	$p_{13:16}$	X	d_{14}	d_{15}	d_{16}	d_{13}	$p_{13:16}$	d_{14}	d_{15}	d_{16}
$p_{17:20}$	s_5	d_{17}	d_{18}	d_{19}	d_{20}	$p_{17:20}$	s_5	X	d_{18}	d_{19}	d_{20}	$p_{17:20}$	d_{17}	d_{18}	d_{19}	d_{20}

FIGURE 5.6 RAID5 disk array with distributed sparing with $N = 6$ disks. Left to right: RAID5 before failure, with failed D_3 disk indicated by X, and after rebuild. The blocks are denoted as data (d), parity (p), spare (s), and unavailable (x).

5.13.2 Rebuild by restriping

Restriping involves overwriting check blocks by recovered data. In the case of RAID7 with three check blocks P, Q, R undergoes the following transformations, by overwriting R, Q, and P check blocks resulting in:

$$\text{RAID7(P,Q,R)} \rightarrow \text{RAID6(P,Q)} \rightarrow \text{RAID5(P)} \rightarrow \text{RAID0}.$$

Restriping was proposed by K.K. Rao (IBM, Wells-Fargo), James Lee Hafner (IBM, rip), and Richard Golding (IBM, consultant) at IBM/ARC (Rao et al., 2006, 2011) in the context of intelligent bricks project also known as IceCube, which was led by Winfred Wilcke at IBM/ARC (Wilcke et al., 2006).

IBM later washed hands of the startup that resulted from this project.

<https://searchstorage.techtarget.com/news/1263835/IBM-washes-hands-of-IceCube-storage-project>

5.13.3 Rebuild with parity sparing

In parity sparing there are two RAID5 arrays with N_1 and N_2 disks. When a disk fails, the SUs of one of the parities are used as spare areas for the other array. Parity sparing in Fig. 5.7 based on a paper by Jai Menon and Richard Mattson shows the merging of two arrays (Menon and Mattson, 1992b).

Given that the first disk in the second array fails we set $p_{1,10} = p_{1,5} \oplus p_{6,10}$ followed by $p_{6,10} = d_6$. Note that $p_{1,10}$ has replaced $p_{1,5}$ and now covers all disks in the first row. A similar transformation is applied to all rows except the last row, where $p_{56,60}$ is lost. We replace $p_{51,55}$ with $p_{51,60} = p_{51,55} \oplus (\bigoplus_{i=1}^5 d_{55+i})$.

RAID6 with $N = N_1 + N_2$ drives is a better starting point than two RAID5 disk arrays, because it provides a higher reliability, but this is at a higher update cost. When the first disk fails in RAID6 the array first reverts to RAID5 by using the Q parities as spare areas. After a second disk failure, RAID5 can revert to RAID0 by using the P parities as spare areas.

Copyback mode is applicable to distributed sparing (Menon and Mattson, 1992a), (Thomasian and Menon, 1997), parity sparing (Menon and Mattson, 1992b), and restriping, where data from the failed disk is copied to standby/spare disk to return the system to normal mode.

D1	D2	D3	D4	D5	D6	D7	D8	D9	DA	DB	DC
d_1	d_2	d_3	d_4	d_5	$p_{1,5}$	d_6	d_7	d_8	d_9	d_{10}	$p_{6,10}$
d_{11}	d_{12}	d_{13}	d_{14}	p_{11-15}	d_{15}	d_{16}	d_{17}	d_{18}	d_{19}	$p_{16,20}$	d_{20}
d_{21}	d_{22}	d_{23}	$p_{21,25}$	d_{24}	d_{25}	d_{26}	d_{27}	d_{28}	$p_{26,30}$	d_{29}	d_{30}
d_{31}	d_{32}	$p_{31,35}$	d_{33}	d_{34}	d_{35}	d_{36}	d_{37}	$p_{36,40}$	d_{38}	d_{39}	d_{40}
d_{41}	$p_{41,45}$	d_{42}	d_{43}	d_{44}	d_{45}	d_{46}	$p_{45,50}$	d_{47}	d_{48}	d_{49}	d_{50}
$p_{51,55}$	d_{51}	d_{52}	d_{53}	d_{54}	d_{55}	$p_{55,60}$	d_{56}	d_{57}	d_{58}	d_{59}	d_{60}
d_1	d_2	d_3	d_4	d_5	$p_{1,10}$	X	d_7	d_8	d_9	d_{10}	d_6
d_{11}	d_{12}	d_{13}	d_{14}	$p_{11-20,5}$	d_{15}	X	d_{17}	d_{18}	d_{19}	d_{16}	d_{20}
d_{21}	d_{22}	d_{23}	$p_{21,30}$	d_{24}	d_{25}	X	d_{27}	d_{28}	d_{26}	d_{29}	d_{30}
d_{31}	d_{32}	$p_{31,40}$	d_{33}	d_{34}	d_{35}	X	d_{37}	d_{36}	d_{38}	d_{39}	d_{40}
d_{41}	$p_{41,50}$	d_{42}	d_{43}	d_{44}	d_{45}	X	d_{46}	d_{47}	d_{48}	d_{49}	d_{50}
$p_{51,60}$	d_{51}	d_{52}	d_{53}	d_{54}	d_{55}	X	d_{56}	d_{57}	d_{58}	d_{59}	d_{60}

FIGURE 5.7 Two RAID5 arrays with $N = 6$ disks each, before/above and after/below double lines after disk failure and reconstruction, with the strips in failed disk D7 in the second array denoted by F. The parity block in the first RAID5 is applied to all blocks after the two arrays are merged. The blocks are denoted as data (d), parity (p), spare (s), and unavailable (X).

5.14 IntraDisk Redundancy - IDR for higher reliability rebuild

As noted earlier rebuild may be unsuccessful due to two reasons:

1. A second disk fails before rebuild is completed, but this is unlikely unless disk failures are correlated, e.g., disks are from the same faulty batch.
2. *Latent Sector Errors - LSEs* encountered during rebuild are the main cause of unsuccessful rebuilds in RAID5.

<https://www.datrium.com/wp-content/uploads/2017/10/1FT-Myth-and-Facts.pdf>

<https://storagemojo.com/2008/02/18/latent-sector-errors-in-disk-drives/>

The fact that 4% of RAID5 rebuilds were unsuccessful led Mario Blaum et al. at IBM ARC to develop the *2-Disk-Failure-Tolerant - 2DFT EVENODD* disk array with two parities, which requires 50% less XORs than *Reed-Solomon - RS* codes for fifteen disks (Blaum et al., 1995). More importantly according to (Iliadis et al., 2011) the probability of unsuccessful rebuilds due to LSEs is two orders of magnitude higher than a second disk failure.

Intra-Disk Redundancy - IDR is a novel method patented as US7823011 to efficiently deal with LSEs developed by Ajay Dholakia, Ilias Iliadis et al. at IBM/ZRL, which applies redundancy at the level of disk sectors to mask LSEs (Dholakia et al., 2008). Dholakia later transferred to Lenovo as part of IBM's sale of its x86-based server business in 2014.

<https://www.cnet.com/news/ibm-sells-its-x86-server-business-to-lenovo-for-2-3-billion/>

IDR is based on segments of length $\ell = n + m$ sectors, where n are data sectors and m are check sectors. There are S bits per sector and the probability that a bit is in error is P_{bit} . The probability of an uncorrectable sector error is

$$P_s = 1 - (1 - P_{bit})^S.$$

When no coding is applied ($m = 0$), the probability that a segment is in error is:

$$P_{seg}^{none} = 1 - (1 - P_s)^\ell.$$

There are $n_d = C_d/(\ell S)$ segments on a disk with C_d sectors. The probability of an uncorrectable failure for a k DFT array in critical mode with k failed disks is:

$$P_{uf}^{(k)} = 1 - (1 - P_{seg})^{\frac{(N-k)C_d}{\ell S}},$$

where the exponent denotes the number of disk segments in critical rebuild mode: $k = 1$ for RAID5 and $k = 2$ for RAID6.

The *Single Parity - SP*, the *Reed Solomon - RS*, and *Interleaved Parity - IP* codes are analyzed in (Dholakia et al., 2008). The probability that a segment encounters an unrecoverable error is given as:

$$P_{seg}^{SP} = [1 + \frac{(\ell - 2)G_2 - 1}{\bar{B}}]P_S,$$

$$P_{seg}^{RS} \approx P_{seg}^{IP} = [1 + \frac{(\ell - m - 1)G_{m+1} - \sum_{j=1}^M G_j}{\bar{B}}]P_S.$$

The mean burst length is $\bar{B} = \sum_{j=1}^{\infty} j b_j$, where b_j is the probability of a burst of length j and $G_m = \sum_{j=m}^{\infty} b_j$. It is interesting to note that IP delivers the same probability of error as the more complex RS method.

The probability that the critical mode in RAID5 ends, because of another disk failure or unrecoverable error is denoted P_{fhr} . The probability of a successful rebuild is then:

$$1 - P_{fhr} = (1 - P_{fr})(1 - P_{uf}^1),$$

where $1 - P_{fr}$ is the probability of no disk failure during rebuild and $1 - P_{uf}^1$ is the probability that no uncorrectable segment is encountered. MTDDL taking into account uncorrectable segment errors is derived in (Dholakia et al., 2008).

$$MTDDL = \frac{(2N - 1)\lambda + \mu}{N\lambda[(N - 1)\lambda + \mu P_{uf}^1]} \quad (5.15)$$

Numerical results have shown that RAID5 with IDR attains the same MTDDL as RAID6. While IDR incurs negligibly longer data transfers due to extra check sectors, this overhead is negligible compared to the extra disk access required by RAID6 with respect to RAID5 (Thomasian et al., 2007a).

Unsuccessful rebuild in RAID5 due to LSEs

We follow the discussion in (Leventhal, 2009), which estimates the probability of unsuccessful rebuilds in RAID5 due to LSEs.

The rebuild of a failed disk in a 100 GB disk array requires the successful reading of $100 \times 10^9 / 512 \approx 10^{11} / 500 \approx 2 \times 10^8 = 200$ million sectors.

A bit error rate of one in 10^{14} bits implies that one sector in 24 billion sectors cannot be correctly read. The probability of reading all sectors successfully is then: $(1 - 1/(24 \times 10^9))^{200 \times 10^6} = 99.2\%$.

This means that on average 0.8% of disk failures would result in data loss due to uncorrectable bit errors.

Disk capacity increases two-fold every two years according to Kryder's law

https://en.wikipedia.org/wiki/Mark_Kryder

so that a 10 TB RAID with 20 billion sectors was considered with a lower bit error rate of one in 10^{16} bits, the probability of unsuccessful rebuilds remains the same as the previous example: $(1 - 1/(2.4 \times 10^{12}))^{(2.0 \times 10^{10})} = 99.2\%$.

Larger disk capacities are made possible by the increased areal recording density, which has resulted in an increase in the time required to read the contents of a disk for rebuild, e.g., it takes 40 hours to read a 4 TB 7200 RPM disk (Huawei Technologies, 2014). At low external disk loads rebuild time can be approximated by the time to read the contents of a disk, which is the number of disk tracks times disk rotation time, if the disk is quiesced. The number of disk tracks can be estimated as the ratio of copying time and rotation time, which is 8.33 milliseconds (ms) for 7200 RPM disks. $40 \times 3600 \times 1000 / 8.33 = 17.28$ million disk tracks.

Mean track capacity equals the ratio of disk capacity and number of tracks.

$4 \times 10^{12} / (17.28 \times 10^6) = 231,481$ bytes.
The average transfer rate in MB/second is: $4 \times 10^{12} / 40 \times 3600 \approx 278$ MB/sec.

5.15 Disk scrubbing for higher reliability rebuild processing

Disk scrubbing is the periodic reading of disks to detect and fix LSEs using the redundancy provided by the RAID5 paradigm. The advantage of disk scrubbing is that sector errors are detected and corrected early, before they can hinder the completion of the rebuild process. Disk scrubbing also has the advantage that it is a software solution which does not require hardware modification.

Disk scrubbing was proposed by Hannu Kari et al. at Texas A&M U. (Kari et al., 1994), who later published a PhD thesis at Helsinki Inst. of Technology (Kari, 1997), where he proposed four *Disk Scrubbing Algorithms - DSA*:

Original DSA: Start a scrubbing request when the disk is idle after a wt time delay.

Adaptive DSA: Adjusts wt based on system activity, i.e., increase wt for heavier loads and decrease wt for lighter loads.

Simplified DSA: Maintain a queue for user and another queue for scanning requests. After the user queue has been empty for a sufficiently long interval a scanning request is issued.

Periodic DSA: Instead of starting DSA right away, issue it periodically, e.g., once a day.

DSA with the *Verify command* transfers read data into internal disk buffers, where the consistency of data is checked.

The reliability analysis (Dholakia et al., 2008) is extended in (Iliadis et al., 2008) to incorporate the effect of disk scrubbing, where it is shown numerically that for realistic parameters IDR provides an MTDL close to the no sector failures case, while disk scrubbing performs well for a very low rate of external requests, which allows more disk scrubbing to be initiated.

The analysis of disk scrubbing reported in (Iliadis et al., 2011) is summarized here. The probability of an error due to a write is P_w and writes constitute a fraction r_w of disk accesses with rate h . The probability of error in reading a sector without scrubbing is $P_e = r_w P_w$. The probabilities of sector failure for deterministic scrubbing and random (exponentially distributed) scrubbing period are:

$$P_s = [1 - (1 - e^{-hT_s}) / (hT_s)] P_e \text{ and } P_s = [hT_s / (1 + hT_s)] P_e, \quad (5.16)$$

where T_s denotes the mean scrub period. It follows that deterministic scrubbing is preferable, since random scrubbing has double the value for P_s . Furthermore $P_s \leq P_e \leq P_w$.

The effectiveness of disk scrubbing and IDR for dealing with LSEs is investigated in (Schroeder et al., 2010b,a), but the conclusions are contradicted in (Iliadis et al., 2011), which concludes that IDR is preferable to disk scrubbing.

5.15.1 Disk scrubbing versus intradisk redundancy

This section is a summary of the discussion in (Iliadis et al., 2011) provided by Ilias Iliadis at ZRL.

Point 1. Consider the read/write process:

- a read attempt is equivalent to a scrubbing operation, because if data is corrupted, a rebuild process is initiated to recover this data.

- latent errors are generated only by write operations, which may leave behind a latent error with a given probability.

Point 2. The disk scrubbing scheme is considered to be independent of the read/write process.

In particular, we consider the entire spectrum of scrubbing schemes that are independent of the read/write process.

A general class of scrubbing schemes is considered that includes the sequential/deterministic and random schemes as special cases. The scrubbing scheme employed determines the arbitrarily distributed interval S between successive checks of a given sector. Expected value of S or $E(S)$ is the scrubbing period T_s . In this context, the scrubbing intervals are allowed to vary. This formulation covers an unlimited number of particular implementations. Two such instances are the deterministic and the random scrubbing schemes. In particular, the distribution of S is

- fixed, i.e. $S = T_s$, for the deterministic scrubbing scheme: starting at one end of the disk and continue to the other end refers to the sequential scrubbing scheme, which is a special case of the deterministic scrubbing scheme.

- exponential, for the random scrubbing scheme.

Clearly, in the case of a sequential scrubbing scheme, the scrubbing period T_s is the time required for a complete check of all sectors of a disk. The frequency at which any given sector is checked is inversely proportional to the scrubbing period T_s .

It is proved that under very general conditions for the read/write process of a uniformly distributed random workload, the deterministic scrubbing scheme is the most efficient of all the scrubbing schemes belonging to this class. We obtain the MTDL for the deterministic scrubbing as a function of T_s and of the workload h (read/write operations per sector per day).

Of interest is a scrubbing scheme, whereby the scrubbing operations are performed during the periods when the disk queue is empty. In this case the interval S between successive checks of a given sector would depend on the workload, and the independence assumption of point 2 would no longer hold. So it seems that this scrubbing scheme does not fall within the general class considered in (Iliadis et al., 2011). Nevertheless, an approximation could be obtained by assuming independence and trying to derive the distribution of the corresponding interval S . Given that there are no scrubbing operations taking place during the busy period of the disk, in this case would hold that $S > B$, where B is the busy period of the disk queue (Kleinrock, 1975).

Given N disks for a p DFT array the storage efficiency - se is $se^{(RAID)} = (N - p)/N$. Given segments of size ℓ with m parity segments per segments $se^{(IDR)} = (\ell - m)/\ell$. It follows $se^{(RAID)+(IDR)} = se^{(RAID)} \times se^{(IDR)}$.

The number of sectors per disk is given as $S_D = \lfloor C_d/S_s \rfloor$, where C_d is the capacity of the disk and S_s sector size (in bytes). The number of data sectors in a RAID array: S_d is given as the product of S_D and the appropriate se .

The scrubbing process identifies unrecoverable sector errors at an early stage and attempts to correct them. The scrubbing process is periodic in that sectors are scrubbed at fixed intervals. Two different disk scrubbing schemes are considered: a sequential (or deterministic) and a random one. According to the sequential scheme, the scrubbing process reads and checks sectors in order, starting from the first and then ending at the last. The process then starts over by checking the first sector. The rate at which sectors are checked is specified a priori. This rate and the disk capacity determine the scrubbing period, i.e., the time required for a complete check of all sectors of a disk. Note that any arbitrarily given sector is checked regularly, with the intervals between successive checks being fixed and equal to the scrubbing period. Note that this also holds in the case of a deterministic, not necessarily sequential, scrubbing scheme that ensures that the whole disk is scrubbed within the scrubbing period. For this reason, we refer to the deterministic scheme in the remainder of the paper. The frequency at which any arbitrarily given sector is checked is inversely proportional to the scrubbing period. According to the random scheme, the scrubbing process checks randomly chosen sectors. Note that here, as in the deterministic scheme, the number of sectors checked in a scrubbing period is equal to the number of sectors contained in a disk. However, in contrast to the deterministic scheme, the random scheme does not guarantee that all sectors of a disk are checked in a scrubbing period. It may well be that some sectors will be checked multiple times, and other sectors not at all.

The rate of I/O requests per disk is $\sigma < \sigma_{max}$, where the latter is obtained using the small write penalty for I/O requests for OLTP. The load on a data sector is $h = \bar{S}_r \sigma / S_d$, where S_d is the number of sectors and $\bar{S}_r$ is the mean size of a disk request in sectors. Given that the fraction of writes is r_w , the rate of read and write requests per sector is: $h_r = (1 - r_w)h$ and $h_w = r_w h$.

The probability of a sector write resulting in an error is the ratio of the number of sectors written in error and the average size of a write request:

$$P_w = \frac{\bar{E}_w}{S_{r,w}},$$

i.e., the ratio of the number of sectors written with errors and the average size of a write request.

Postulated in (Iliadis et al., 2011) is the following sequence of reads and writes on a sector:

$$R_1, W_1, W_2, W_3, E_1, R_2, W_1, W_2, \dots$$

Contiguous R_i and W_i intervals put together constitute R_p 's and W_p 's, so that a read cycle is $R_p = \sum_{i=1}^{N_r} R_i$, where N_r is the number of read operations and R_i the interval following the i^{th} read. It follows $E(R_p) = E(N_r)E(r)$ and similarly $E(W_p) = E(N_w)E(W)$. The long term ratio of write operations is given by: $\tilde{r}_w = E(N_w)/(E(n_r) + E(N_w))$. The probability that an arbitrary point falls within a write cycle is:

$$\phi_w = \frac{E(W_p)}{E(R_p) + E(W_p)} = \frac{\tilde{r}_w E(W)}{(1 - \tilde{r}_w)E(R)} = \tilde{R}_w E(W).$$

The probability that the last operation on a sector was a write (P_{lw}) multiplied by P_w yields the probability of unrecoverable sector error $P_s = P_{lw} P_w$.

$P_{lw}(\text{no scrubbing}) = \phi_w$, the probability of an unrecoverable error without scrubbing is

$$P_e = \phi_w P_w = \frac{E(W_p)}{E(R_p) + E(W_p)} P_w.$$

Let Y represent the residual time of a write interval, so that $F_Y(x) = \int_x^\infty F_W(x)dx/E(W)$. Let S_i denote a typical scrubbing interval and $W_{PS,i}$ the duration of write cycles that fall within a scrubbing period, the proportion of time in a write cycle is then $\phi_w = E(W_{ps})/E(S)$.

W_{ps} is effectively reduced by Y or $S > Y$, so that unrecoverable errors are within an effective write cycle:

$$E(W'_{ps}) = E(W_{ps}) - \phi_w E(\min(Y, S))$$

The probability that a point falls within an effective write cycle is:

$$\phi_{ws} = \frac{E(W'_{ps})}{E(S)} = \phi_w \left(1 - \frac{E(\min(Y, S))}{E(S)}\right).$$

It can be argued that the last write operation falls within W'_{ps} , so that $\phi_{lw} = \phi_{ps}$.

With appropriate substitutions

$$P_s = P_{lw} P_w = \left(1 - \frac{E(\min(Y, S))}{ES}\right) P_e$$

where $f_Y(x) = F_W(x)/E(W)$ is the residual time of the write interval.

It follows that $P_s \leq P_e \leq P_w$.

Theorem 7.3 in (Iliadis et al., 2011) states that for all scrubbing schemes with a give average scrubbing period $E(S)$, the deterministic scrubbing scheme is the most efficient one, resulting in the minimum $P_{s,min}$ given by:

$$P_{s,min} = \left(1 - \frac{\int_0^{E(S)} F_Y(x)dx}{E(S)}\right) P_e,$$

where $F_Y(x) = F_W(x)/E(W)$ is the residual pdf of W and $F_Y(x) = \int_x^\infty f_Y(x)dx$.

Unrecoverable errors will be corrected if the residual scrubbing period $\cap S < W$, where $f_{\cap S}(x) = F_S(x)/E(s)$. It is shown that

$$E_s = P(\cap S < W) = \frac{E(\min(W, S))}{E(S)}.$$

It is proven when W intervals are exponentially distributed $E_s = 1 - P_s/P_e$.

As the scrubbing frequency is increased $E_s \leftarrow 1$ and $P_s = 0$.

When R and W have the same distribution and hence the same mean $P_e = \tilde{r}_w P_w$ rather than reads and writes following Poisson arrivals, which was a requirement in an earlier work.

The rate of read and write requests is

$$\tilde{h} = \frac{E(N_r) + E(N_w)}{E(R_p) + E(W_p)} = [(1 - \tilde{r}_w)E(R) + \tilde{r}_w E(W)]^{-1}.$$

Proposition 7.9 states that when the Reading and Writing intervals are identically and exponentially distributed then the probability of an unrecoverable sector error for the deterministic scrubbing scheme is given as:

$$P_s = \left(1 - \frac{1 - e^{-\tilde{h}T_s}}{\tilde{h}T_s}\right) P_e \quad E_s = \frac{1 - \tilde{h}T_s}{\tilde{h}T_s}$$

where $P_e = \tilde{r}_w P_w$ and $\tilde{r}_w = E(N_w)/[E(N_r) + E(N_w)]$ and E_s is given by Proposition 7.10. When R and W intervals are identically and exponentially distributed for the random scrubbing scheme.

$$P_s = \frac{\tilde{h}T_s}{1 + \tilde{h}T_s} P_e, \quad E_s = \frac{1}{1 + \tilde{h}T_s}. \quad (5.17)$$

For small values of $\tilde{h}T_s \ll 1$ for deterministic scrubbing

$$P_s^{det} = \frac{1}{2} P_e \tilde{h} T_s, \quad P_s^{exp} = P_e \tilde{h} T_s. \quad (5.18)$$

5.16 Predictive Failure Analysis - PFA

Self-Monitoring Analysis and Reporting Technology - SMART is a diagnostic method to predict disk failures, termed *Predictive Failure Analysis - PFA*.

<https://en.wikipedia.org/wiki/S.M.A.R.T.>

Previous work in this area is reviewed in (Pinheiro et al., 2007), where they identify SMART parameters that correlate highly with disk failure. The contents of a disk, whose failure may be imminent, are copied onto a spare disk before it fails. The failing disk may be used to process external disk requests and the complications of rebuild processing are avoided if the copying of the disk is completed before it fails.

Disk consistency checker such as *file system consistency check - fsck* is used to detect and repair corrupted files affected by failures, errors, and bugs. <https://en.wikipedia.org/wiki/Fsck>

Checkers are rarely run and operate slowly causing significant downtime for large scale storage systems. Addressed in (Ma et al., 2015) is a fast fsck - *ffsck* supported by a modified *third extended file system - ext3*, which is a journaling file system for Linux, <https://en.wikipedia.org/wiki/Ext3>

https://en.wikipedia.org/wiki/Journaling_file_system

which can scan and repair at full sequential bandwidth of the underlying device. fsck is applied to FreeBSD's FFS file system and its checker as well.

Data Domain's RAIDshield monitors disk errors and takes steps to protect against them (Ma et al., 2015). In a five year study covering about one million six SATA disk models RAIDShield monitored the health of each disk and replaced disks that were predicted to fail imminently, eliminating 98% of triple disk errors, which are 80% of all RAID6 failures. A method was designed and simulated using the joint failure probability to quantify and predict how likely a RAID group is to face multiple simultaneous disk failures.

A recent study addresses making disk failure predictions using SMARTer (Lu et al., 2020). It is noted that disks are the most commonly replaced hardware components. A total of 380K HDDs were considered in this study, The proposed ML method predicts disk failures with 0.95 F-measure https://en.wikipedia.org/wiki/F1_score and 0.95 *Mathews Correlation Coefficient - MCC* https://en.wikipedia.org/wiki/Matthews_correlation_coefficient for ten days prediction horizon on the average. Equations for MCC and the F-measure are given in Chapter 2.

The reliability measures in today's disk drive-based storage systems focus predominantly on protecting against complete disk failures. Previous disk reliability studies have analyzed empirical data in an attempt to better understand and predict disk failure rates. LSEs go undetected until the corresponding disk sectors are accessed. A major study to analyze the impact of LSEs, observe trends, and explore their implications on the design of reliability mechanisms in storage systems was conducted at NetApp by Lakshmi N. Bairavasundaram (U. Wisc., NetApp, Datrium), Garth R. Goodson (CMU, NetApp, Netero, Freshworks), Shankar Pasupathy (NetApp), Jiri Schindler (CMU, EMC, HPE, NetApp, Tranquil Data) at NetApp and is reported in (Bairavasundaram et al., 2007).

NetApp Active IQ is a digital advisor that simplifies the proactive care and optimization of NetApp storage. Fueled by telemetry data from a highly diverse installed base, it uses advanced AI and ML techniques to uncover opportunities to reduce risk and improve the performance and efficiency of the storage environment. It then provides the prescriptive guidance and actions to make it happen. This "actionable intelligence" simplifies storage administration and leads to higher availability, improved security, and higher-performing storage.

<https://www.netapp.com/services/support/active>

5.17 Undetected disk errors and Silent Data Corruption - SDC

Silent Data Corruption - SDC manifests itself as undetectable errors in data. The causes of *Undetected Disk Errors - UDEs* and their effects on data integrity are discussed in (Hafner et al., 2008), which describes basic techniques at various software layers in the I/O stack that have been applied to address this problem and solutions that can be integrated into the RAID subsystem.

https://en.wikipedia.org/wiki/Data_corruption#SILENT

UDE types are classifiable as *Undetectable Write/Read Errors - UWEs/UREs*. UWEs are in the form of dropped or phantom writes and off-track or misdirected writes resulting in stale data. UREs are in the form of *Error Correcting Code*

- ECC miscorrects, resulting in reading of corrupted data. UREs are also in the form of off-track reads, which result in reading of stale or corrupted data.

The paper presents technologies that can be used to address data corruption problems caused by UDEs, as well as other causes of data corruption. Technologies are divided into different classes depending on the software layer and the type of problem they attempt to address.

Checking at middleware or application software layers: Some file systems checksum 4-8 KB data chunks and store them separately. When the data chunk is read the checksum is recomputed and compared. A mismatch indicates corruption of either the data or the checksum, although the former is more likely because of its larger size.

This method was utilized by ZFS (Karlsson, 2006) and the *IRON - Internal ROBustNess* file system described in (Prabhakaran et al., 2005), which incurs minimal time and space overheads via in-disk checksumming, replication, and parity is shown to greatly enhance file system robustness.

Detection of Errors by the Storage System: Errors introduced along the datapath between the storage system and the application may not be detected. This class has several subclasses with respect to different powers of detection and correction with associated performance impacts and performance/cost tradeoffs.

By itself data and parity scrubbing cannot detect UDEs. A parity scrub provides an additional step beyond a data scrub.

1. A data scrub in addition to reading all of the data blocks recomputes the parity and compares this computed parity with the stored parity on disk. A miscompare implies a data corruption error. Data scrubs and parity scrubs do not always provide for recovery, and a comparison does not always provide a guarantee that no data corruption has occurred.
2. When metadata is colocated with data and metadata contains checksums, a read of the data and metadata can detect data corruption errors, especially those introduced by firmware, software, or the memory bus. When the metadata contains some form of logical addressing of the data, a read can detect off-track writes but only at the offset target location, where data was incorrectly overwritten. These methods are usually combined with a data scrub to enhance its effectiveness as discussed in (Sundaram, 2006).

https://atg.netapp.com/wp-content/uploads/2018/08/TECH_ONTAP-Private_Lives_of_Disk_Drives.pdf

3. The *Small Computer System Interface - SCSI* write-verify disk command rereads data after it is written to ensure that it was written correctly. This works for dropped write errors but cannot necessarily detect off-track writes, because the tracking algorithm, which has already decided that the correct track is being used, may not move the head from the current track. This also has a response-time performance impact, because a full disk spin is required to make the check and return the status is returned to the host. This method has a simple recovery algorithm which rewrites the data to the same or a different location on disk.
4. Similarly to the write-verify approach, the verification is instead handled in a write cache external to the disk drive. Fresh data is held in the cache even after being written to disk. When the data needs to be evicted from the cache, the disk is first read and compared with the cache copy. The eviction occurs only if the data compare is correct. This has the same simple recovery algorithm as the write-verify approach.

Write verification scheduled immediately after WRITE or *Read After Write - RAW* degrades performance and is hence unattractive. To reduce this performance penalty *Idle Read After Write - IRAW* was proposed by researchers at Seagate in (Riska and Riedel, 2008). The idea is to retain the written content in the disk cache and verify it once the disk drive becomes idle. Trace-driven evaluation of IRAW shows its feasibility and that disk idleness can be utilized for WRITE verification with minimal effect on user performance.

5. The metadata may be a checksum or version number, which is a sequence number or timestamp is stored and managed within the storage system itself. The second copy of the metadata is stored in memory for fast comparison and access, but must be flushed to disk to maintain detection capabilities after system crashes.
6. The metadata is stored as RAID stripe. It then has a locality of reference that can be used to mitigate some of the disk I/O overheads of separate locations. The advantages of this approach are lower I/O and bandwidth penalties for most operations; the disadvantage is potential loss of detection under certain failure scenarios.

SDCs may go undetected until a system or application malfunction occurs. The likelihood of UDEs increases with disk densities and the use of lower reliability desktop-class drives in enterprise storage systems. The incorporation of UDE detection and correction into storage systems is necessary to prevent increasing numbers of data corruption and data loss events.

A major problem with SDC is that data errors propagate during data rebuild. Techniques to deal with SDCs are described in by Mingqiang Li and Patrick Lee (Columbia U.) at *Chinese U. of HK - CUHK* (Li and Shu, 2010), (Li and Lee, 2014).

5.18 Clustered RAID5 layouts

CRAID was proposed in (Muntz and Lui, 1990b), and utilized in (Ng and Mattson, 1992, 1994), and (Holland and Gibson, 1992), (Holland et al., 1994; Holland, 1994), (Merchant and Yu, 1992, 1996), and (Fu et al., 2004a).

Without clustering the utilization of the spare disk on which data is being rebuilt remains below the utilization of surviving disks, until the rebuild process is completed at which point due to striping all disk utilizations become equal. In the case of CRAID the spare disk may become a bottleneck, which is because rebuild generates a stream of writes for RUs at rate $(N - 1)/(G - 1) = 1/\alpha$. The backlog of RUs to be written to the spare disk can be reduced by modifying the fraction of reads redirected to the spare disk to balance disk loads (Muntz and Lui, 1990b).

It is easy to see that the read load to small data blocks on the $N - 1$ surviving disks in a RAID5 with N disks is doubled when a disk failure occurs, which is because each surviving disk in addition to its read load processes is accessed to reconstruct blocks on the failed disk. The following expression for the load increase in processing read and write requests to small blocks in RAID5 with a single disk failure is given in (Ng and Mattson, 1992):

$$\frac{U_{faulty}}{U_{faultfree}} = \frac{N}{N - 1} + \frac{(N - 2)f_R + (N - 8)f_W}{(N - 1)(f_R + 4f_W)}. \quad (5.19)$$

where f_r is the fraction of reads and $f_w = 1 - f_r$ is the fraction of writes. Note that the load on surviving disks is doubled for $f_r = 1$.

The placement of parities to balance update load in RAID5 is stated in (Muntz and Lui, 1990b) as a combinatorial design problem (Colbourn and van Oorschot, 1989). Considered is the simple case with n disks and parity group size k with $k < n$, so that there are $\binom{n}{k}$ different configurations. A sample design is provided in Fig. 5.8.

Disk0	Disk1	Disk2	Disk3	Disk4
D0.0	D0.1	D0.2	P0	P1
D1.0	D1.1	D1.2	D2.2	P2
D2.0	D2.1	D3.1	D3.2	P3
D3.0	D4.0	D4.1	D4.2	P4
D5.0	D5.1	P5	D5.2	D6.2
D6.0	D6.1	P6	P7	D7.2
D7.0	D7.1	D8.1	P8	D8.2
D8.0	D9.0	D9.1	P9	D9.2
D10.0	P10	D10.1	D10.2	D11.2
D11.0	P11	D11.1	D12.1	D12.2
D12.0	P12	P13	D13.1	D13.2
D13.0	D14.0	P14	D14.1	D14.2
P15	D15.0	D15.1	D15.2	D16.2
P16	D16.0	D16.1	D17.1	D17.2
P17	D17.0	D18.0	D18.1	D18.2
P18	P19	D19.0	D19.1	D19.2

FIGURE 5.8 Data layout of a clustered array with $N = 5$ and $G = 4$ (Holland, 1994).

A limited version of BIBD with $\alpha \approx 0.5$ was proposed by Narasimha Reddy (IBM/ARC, Texas A&M U.) and Prithviraj Banerjee in (Reddy and Banerjee, 1991).

https://en.wikipedia.org/wiki/Prith_Banerjee

5.18.1 Balanced Incomplete Block Designs - BIBDs

Spencer W. Ng and Richard/Dick Mattson at IBM/ARC proposed the BIBD parity layout for clustered RAID5 in (Ng and Mattson, 1992, 1994) by referring to Marshall Hall's "Combinatorial Theory's list of designs in its Appendix I (Hall, 1986). The patent they filed in August 1990 on the application of BIBD data layouts was granted in 1993 (Mattson and Ng, 1993). The 1992 paper was published before the patent was granted. According to Mehrdad Dastoori formerly at *US Patent and Trademark Office - USPTO* "A patent application remains valid as long as a paper describing it was not published more than a year ago".

Fig. 5.9 based on an example given in these works shows a grouping of N distinct objects into b blocks, such that each block contains G objects, each object occurs in exactly r blocks, and each pair of objects appears in exactly L blocks. Only three out of five variables are free, because of the following two relationships:

$$bG = Nr \text{ and } r(G - 1) = L(N - 1).$$

For $N = 10$, $G = 4$, and $L = 2$, $r = 2 \times 9/3 = 6$, and $b = N \times r/G = 10 \times 6/4 = 15$.

Disk #	1	2	3	4	5	6	7	8	9	10
Parity Groups #	1	1	2	3	1	2	3	1	2	3
	2	4	4	4	5	6	7	5	6	7
	3	6	5	5	8	8	8	9	10	11
	4	7	7	6	10	9	9	12	12	12
	8	9	10	11	11	11	10	14	13	13
	12	13	14	15	13	14	15	15	15	14

FIGURE 5.9 BIBD data layout with $N=10$ disks and parity group size $G=4$ from Ng and Mattson (1994).

BIBD designs are not available for all values of N and G , e.g., layouts for $N = 33$ with $G = 12$ do not exist, but designs with $G = 11$ and $G = 13$ can be used instead (Holland et al., 1994).

Smaller BIBD designs are presented in (Schwabe and Sutherland, 1994, 1996) extending results in (Bose, 1939) and (Hanani, 1975) for the case when the number of disks is a prime number. Approximately balanced layouts are also introduced, which allow small imbalances in the distribution of parity units.

Mark Holland who was a summer student at IBM/ARC in early 1990s might have been inspired by the ongoing work on BIBD at IBM/ARC in undertaking his PhD research at CMU (private communication from Spencer Ng). Holland studied the problem of improved CRAID designs by rearranging the placement of data units to achieve better parallelism, while maintaining large write optimization (Holland and Gibson, 1992), (Holland et al., 1994; Holland, 1994).

The time requirement to compute a BIBD-based mapping is reasonably small, although the space requirement depends on the size of the BIBD table, which in turn depends on CRAID parameters (n and k).

Six desirable properties for BIBD layouts are listed in (Holland et al., 1994).

- 1. Single failure correcting:** The strips in the same stripe, i.e., parity group, are mapped to different disks to make recovery possible.
- 2. Distributed parity:** All disks have the same number of check units, so that disk loads in updating data blocks are balanced.
- 3. Distributed recovery workload:** Balanced load in failed mode, the reconstruction workload should be balanced across all disks.
- 4. Efficient mapping:** The functions mapping a file system's logical block address to the physical disk addresses can be computed with little computing and memory resources.
- 5. Large write optimization:** Each stripe contains a contiguous interval of user data to process a write of all $k - 1$ data units.
- 6. Maximal parallelism:** All disks are involved in reading large blocks of data.

A layout is ideal if it satisfies all six properties, but no previously published declustered layout is ideal. The BIBD-based layouts presented in (Holland et al., 1994) and (Schwabe and Sutherland, 1996) satisfy the first three properties. Since ideal layouts are possible only for very limited choices of k and n specified below, there has been interest in layouts where these properties hold approximately.

In what follows we follow the notation in (Alvarez et al., 1998), which was written by Guillermo Alvarez (HP, consultant), Walter Burkhard (UCSD, emeritus), Larry Stockmeyer (IBM/ARC, UCSC, rip), and Flaviu Cristian (IBM/ARC, UCSD, rip) at UCSD (Alvarez et al., 1998). The four parameters for a declustering layout according to the number of disk drives is $n \geq 2$, the number of stripes (parity groups) b , the number of strips per stripe or parity group size $k \leq n$, and the number of strips per disk r . Note the equality $bk = nr$ holds.

Each stripe consists of data and check units. The number of data units per stripe is m and the number of check units is $f = k - m$, but only $f = 1$ corresponding to RAID5 is considered. The word stripe is tantamount to parity group in the discussion.

The *Greatest Common Divisor* - GCD of x and y is the largest g such that $g|x$ and $g|y$, where $x|y$ means that x divides y .

Fact 1. If y and n are relatively prime and i and j are integers with $1 \leq i - j \neq n - 1$ then $iy \neq jy$.

Lemma 2. If a layout has the maximal parallelism property, then there is a way to name the disks $0 : n - 1$ such that $\text{disk}(a) = a \bmod n$ for every address a of client data.

Lemma 3. If a layout has the large write optimization and maximal parallelism properties, then the mapping of strips to disks is given by $\text{disk}((s, u) = ((k - 1)s + u) \bmod n$ for all s and u with $0 \leq u \leq k - 1$.

Lemma 4. If L' is a placement-ideal layout and if k is obtained by repeating L' any number of times, then L is placement ideal.

Theorem 5. There is a placement-ideal layout with parameters n, k, r , if and only if - iff $k|r$ and $(n - 1)|k(r - 1)$ and one of the following holds: (1) $k = n$, (2) $k = n - 1$, (3) $k = 2$, (4) $k = 3$ and $n = 5$ (5) $k = 4$ and $n = 7$ The layout for case (5) is illustrated in Fig. 5.10.

Disk0	Disk1	Disk2	Disk3	Disk4	Disk5	Disk6
$d_{0,0}$	$d_{0,1}$	$d_{0,2}$	$d_{1,0}$	$d_{1,1}$	$d_{1,2}$	$d_{2,0}$
$d_{2,1}$	$p_{2,2}$	$d_{3,0}$	$d_{3,1}$	$d_{3,2}$	$d_{4,0}$	$d_{4,1}$
$d_{4,2}$	$d_{5,0}$	$d_{5,1}$	$d_{5,2}$	$d_{6,0}$	$d_{6,1}$	$p_{6,2}$
p_1	p_6	p_4	p_2	p_0	p_5	p_3

FIGURE 5.10 Ideal placement for $n = 7$ and $k = 4$ with seven parity groups.

Lemma 6. If a layout L has a maximal parallelism on n -blocks, then its parallel read time transfers satisfy $\tau_L(o) \leq \lceil o/n \rceil + 1$ for all o .

Theorem 7. For every prime n , every with $2 \leq k \leq n$, every f with $0 \leq f < k$ and every r such that $k(n - 1)|r$, the layout Prime is f -failure correcting, has optimally distributed check information, has optimally distributed reconstruction for a single failure, has large write optimization, and has parallel read count $\tau_L(o) \leq \lceil o/n \rceil + 1$ for all o .

Lemma 8. For the Relatively Prime - RELPR layout with parameters n, k, r , where $r = k\phi(n)$ any two disks share at most $k(k - 1)$ stripes. $\phi(n)$ in number theory is Euler's totient function that counts the positive integers up to a given integer n that are relatively prime to n , e.g., $\phi(6) = 2$ since there are two such numbers: 1 and 5.

https://en.wikipedia.org/wiki/Eulers_totient_function

<https://www.doc.ic.ac.uk/~mrh/330tutor/ch05s02.html>

Theorem 9. For every n , every k with $2 \leq k \leq n$, ever f with $0 \leq f < k$, every r such that $(k\phi(n))|r$, the layout RELPR is f -failure correcting, has optimally distributed check information, has large write optimization, has reconstruction workload bounded above by $(k - 1)/\phi(n)$ for a single failure, and has a parallel; read count $\tau(o) \leq \lceil o/n \rceil + 1$ for all o .

A data layout with different strides between consecutive stripe units was proposed by Larry Stockmeyer in (Stockmeyer, 1994) and rediscovered by Alvarez and Burkhard at UCSD. If s denotes the stride and data unit a is mapped to disk d , then unit $a + 1$, will be mapped to disk $(d + s) \bmod n$.

Near ideal data layouts are proposed by Alvarez and Burkhard collaborating with Larry Stockmeyer and Flaviu Christian in (Alvarez et al., 1998).

5.18.2 PRIME data layout

The PRIME data layout shown in Fig. 5.11 relaxes the parallelism property slightly and is very close to ideal for prime values of n . For a layout L the parallel read count $\tau_L(o)$, the parallel read count, to be the maximum number of data units that any disk must supply when reading any o consecutive units of client data. For any access size $1 \leq o \leq bm$, $\tau_L \geq \lceil o/n \rceil$. A layout has optimal parallelism if there is equality for o . It follows from Lemma2 that a layout has maximal parallelism iff it has optimal parallelism.

Mapping of client data units a to array locations: $z = \lfloor \frac{a}{mn} \rfloor$

$y = (z \bmod (n - 1) + 1, \text{disk}(a) = a \bmod n, \text{offset}(a) = \lfloor a/n \rfloor + fz$

To tolerate f failures we need $0 \leq i < f$

$\text{check-disk}_i(a) = (((\lfloor a/m \rfloor)_1 m + i).y) \bmod n$

$\text{check-offset}_i(a) = (z + 1)j - f + i$

D0.0	D0.1	D1.0	D1.1	D2.0
D2.1	D3.0	D3.1	D4.0	D4.1
C4.0	C2.0	C0.0	C3.0	C1.0
C1.1	C4.1	C2.1	C0.1	C3.1
D5.0	D6.1	D5.1	D7.0	D6.0
D7.1	D9.0	D8.0	D9.1	D8.1
C9.0	C8.0	C7.0	C6.0	C5.0
C6.1	C5.1	C9.1	C8.1	C7.1
D10.0	D11.0	D12.0	D10.1	D11.1
D12.1	D13.1	D14.1	D13.0	D14.0
C14.0	C10.0	C11.0	C12.0	C13.0
C11.1	C12.1	C13.1	C14.1	C1.01
D15.0	D17.0	D16.1	D16.0	D15.1
D17.1	D19.1	D19.0	D18.1	D18.0
C19.0	C16.0	C18.0	C15.0	C17.0
C16.1	C18.1	C15.1	C17.1	C19.1

FIGURE 5.11 PRIME data layout, with $n = 5$, $k = 4$, $f = 2$. Cs.i denotes check unit i of stripe s.

D0.0	D0.1	D1.0	D1.1	D2.0	D2.1
D5.1	D3.0	D3.1	D4.0	D4.1	D5.0
C2	C5	C0	C3	C1	C4
D6.0	D8.1	D8.0	D7.1	D7.0	D6.1
D11.1	D11.0	D10.1	D10.0	D9.1	D9.0
C8	C10	C7	C9	C6	C11

FIGURE 5.12 RELPR structure for $n = 6$, $k = 3$, $f = 1$, with $b = 6 \times 2 = 12$.

5.18.3 Relatively Prime - RELPR data layout

PRIME may allow two strips from the same stripe on the same disk if n is not prime. One solution is to restrict all strides to be relatively prime to n , hence the *Relatively Prime - RELPR* data layout, which is shown in Fig. 5.12. The number of stripes in the layout is $b = n\phi(n)$.

Note that these data layout with all the parities placed on a separate row, allows maximum parallelism in accessing data. Other data layouts with horizontal parities are X-code (Xu and Bruck, 1999), which is MDS and incurs the minimum level of redundancy, and RM2 (Park, 1995) which is not.

5.18.4 Thorp permutation for load balancing in clustered RAID5

An alternative to BIBD is based on Thorp's shuffle (Merchant and Yu, 1992, 1996), which is described in "Seminumerical Algorithms" (Knuth, 1997a) and was proposed Arif Merchant (IBM/NEC/HP/Google) and Philip Yu (IBM/UIc) at IBM/TJWRC.

The Thorp shuffle cuts a deck of cards into two equal half decks A and B and then the following is repeated until no cards are left: "Pick deck A or B randomly and drop its bottom card and then drop the bottom card from the other half deck atop it". The computational complexity is $O(\log(N))$ (Knuth, 1997a), but the overall time complexity of computing block addresses is $O(\log(B) + \log(N))$, where B is the number of blocks per disk, which can be quite large.

The Thorp shuffle unlike BIBD can generate layouts for arbitrary N and G , but the examples given in (Merchant and Yu, 1996) are restricted to $N/G = 2^i$, $0 \leq i \leq 3$ with $G = 8$. As stated in (Holland et al., 1994) the Thorp shuffle has the disadvantage of incurring high computational cost per block access, while according to (Merchant and Yu, 1996) the computation takes 14 microseconds on an SGI computer at hand. On the other hand BIBD computes physical disk addresses by table lookup and a few simple arithmetic operations.

5.18.5 Clustered RAID with Nearly Random Permutations - NRP

A simple approach to randomly permute the elements of an array was proposed in (Fu et al., 2004b) and (Thomasian and Blaum, 2009) by Alex Thomasian et al. (NJIT) and Spencer Ng (HGST).

Disk array space is organized as an $M \times N$ matrix, where M is the number of stripes or rows and N is the number of disks or columns. Parity groups of size $G < N$ are placed sequentially on successive rows of the matrix, so that the i^{th} parity group PG_i occupies strips iG through $[iG + (G - 1)] \bmod(N)$. In the case of RAID5 (resp. RAID6) the P (resp. P and Q) parities can be consistently assigned as the last two strips in each parity group.

The initial allocation with $N = 10$ and $G = 4$ for RAID5 is shown in Fig. 5.13, where $p_{i:j}$ stands for the parity over data strips d_i through d_j . Given $G \leq N$ the *Least Common Multiple* - LCM is $L = \text{LCM}(N, G) = 20$. $r = \text{LCM}(N, G)/G = 2$ rows are required to allocate $L/G = 5$ parities in this case. Only one half of the columns with this initial layout hold parities, so that the parity update load will not be balanced. The problem is alleviated by randomizing the placement of blocks on the disks, so that approximately the same number of check blocks are allocated per disk.

The R rows or stripes on disk are subdivided onto R/r superrows, which are numbered consecutively. The stripe or row number I , possibly added to a constant, is used as a seed to a pseudo-random number generator to obtain a random permutation of $\{0, 1, \dots, N - 1\}$, given as $\underline{P}_I = \{P_0, P_1, \dots, P_{N-1}\}$. A random permutation may be generated using Algorithm 235 (Durstensfeld, 1964). The new method requires computing $N - 1$ uniformly distributed random variables.

Shuffle array $a[i]$, $i = 0, \dots, n-1$.

for $i := n \text{ step } -1$ until 2 do

{ $j = \text{entier}(i \times \text{random} + 1)$;

$b = a[i]$; $a[i] := a[j]$; $a[j] := b$ }

If $N \bmod(G) = 0$ then a random permutation is applied one row at a time, since $r = 1$, and otherwise it is applied to $r > 1$ rows at a time.

Given the random permutation $\underline{P}_1 = \{1, 10, 8, 7, 3, 2, 6, 4, 5, 9\}$ for the first row in Fig. 5.14 and since $\ell = \text{LCM}(10, 4)/10 = 2$, the same permutation is repeated on the second row as well, which ensures strips of a PG straddling two rows are mapped onto different columns or disks, (this is the case for d_6, d_7, d_8 , and $p_{6:8}$). This makes it possible to apply the RAID5 paradigm for recovery and to access all the data strips in the parity group in parallel. Given N , the strip size, and the RAID level, we can determine the row number I for a given block number. The strip in which the block resides is obtained by applying the permutation using I as a seed.

Disk#	1	2	3	4	5	6	7	8	9	10
Parity	D_1	D_2	D_3	$P_{1:3}$	D_4	D_5	D_6	$P_{4:6}$	D_7	D_8
groups	D_9	$P_{7:9}$	D_{10}	D_{11}	D_{12}	$P_{10:12}$	D_{13}	D_{14}	D_{15}	$P_{13:15}$
(initial	D_{16}	D_{17}	D_{18}	$P_{16:18}$	D_{19}	D_{20}	D_{21}	$P_{19:21}$	D_{22}	D_{23}
allocation)	D_{24}	$P_{22:24}$	D_{25}	D_{26}	D_{27}	$P_{25:27}$	D_{28}	D_{29}	D_{30}	$P_{27:30}$

FIGURE 5.13 Initial allocation of parity groups before permutation ($N = 10$, $G = 4$).

Disk#	1	2	3	4	5	6	7	8	9	10
Final	D_1	D_5	D_4	$P_{4:6}$	D_7	D_6	$P_{1:3}$	D_3	D_8	D_2
allocation	D_9	$P_{10:12}$	D_{12}	D_{14}	D_{15}	D_{13}	D_{11}	D_{10}	$P_{13:15}$	$P_{7:9}$

FIGURE 5.14 Permuted data blocks with nearly random permutation method ($N = 10$, $G = 4$). Note that only first two rows are shown.

5.19 Clustered RAID designs by Walter Burkhard et al. at UCSD

Three CRAID designs by Walter Burkhard (emeritus) and his students at UCSD: ACATS, DATUM, and PDDL are discussed in this section.

5.19.1 Almost Complete Address Translation - ACATS

Almost Complete Address Translation - ACATS proposed in (Schwarz and Burkhard, 1996) is deterministic, run-time efficient, and provides the best or almost the best possible incremental rebuild workload. ACATS provides protection against single disk as well as single string failures and provides a framework in which the RAID5 organization logically appears as RAID4, facilitating distributed sparing.

ACATS does not require the existence of a suitably configured block design or a pseudo-random number generator; it is applicable to arbitrarily configured orthogonal disk arrays. It is claimed that ACATS is faster than declustering schemes

(0,0)	(1,0)	(2,0)	(3,0)	(4,0)	object 0
(0,1)	(1,1)	(2,1)	(3,1)	(4,1)	
(0,2)	(1,2)	(2,2)	(3,2)	(4,2)	
(1,1)	(2,2)	(3,0)	(4,1)	(0,0)	object 1
(1,2)	(2,0)	(3,1)	(4,2)	(0,1)	
(1,0)	(2,1)	(3,2)	(4,0)	(0,2)	
(2,1)	(3,0)	(4,2)	(0,0)	(1,2)	object 2
(2,2)	(3,1)	(4,0)	(0,1)	(1,0)	
(2,0)	(3,2)	(4,1)	(0,2)	(1,1)	
(3,1)	(4,1)	(0,0)	(1,0)	(2,0)	object 3
(3,2)	(4,2)	(0,1)	(1,1)	(2,1)	
(3,0)	(4,0)	(0,2)	(1,2)	(2,2)	
(4,2)	(0,0)	(1,1)	(2,2)	(3,1)	object 4
(4,0)	(0,1)	(1,2)	(2,0)	(3,2)	
(4,1)	(0,2)	(1,0)	(2,1)	(3,0)	
(0,0)	(1,2)	(2,1)	(3,1)	(4,0)	object 5
(0,1)	(1,0)	(2,2)	(3,2)	(4,1)	
(0,2)	(1,1)	(2,0)	(3,0)	(4,2)	

FIGURE 5.15 ACATS data layout with $n = 5$ strings and $m = 3$ disks per string.

that use pseudo-random permutations and achieves better uniformity of disk loads during a disk rebuild; it is simpler than schemes based on block designs.

1. The objects comprising a parity group must reside on distinct physical strings.
2. The check, spare, and client data objects must be evenly distributed over all physical disks within the array.
3. The workload during string reconstruction must be evenly spread over the disks of the remaining strings. Similarly, the workload during a single disk rebuild must be evenly spread over the disks on strings other than the string containing the failed disk.
4. The mapping from logical to physical addresses must be reasonably efficient and the space to store the algorithm and data must be small.

Logical addresses are specified as a triple (s, d, t) , where s is the logical string number, d is the disk number, and t is the logical object number. The example considers $n = 5$ strings and $m = 3$ disks per string. Objects are specified as (s, d) , where s is the string and d the disk number. Logical string s is mapped to physical string $(s + t) \bmod n$, logical disk d (of logical string s) is mapped to physical disk

$$\left((s + t) \bmod n \vec{d}_{(s+t) \bmod n} * \vec{t} + d \bmod m \right)$$

$\vec{d}_0 = (0, 0)$, $\vec{d}_2 = (0, 2)$, $\vec{d}_3 = (1, 0)$, $\vec{d}_4 = (1, 1)$, Fig. 5.15 shows the first few objects on fifteen disks.

The cyclicity of the allocation is specified the period, which is 45 for in this case. Table 5.2 gives periods for other combinations on n and m . Defining level as $\ell = 1 + \lfloor \log_m(n - 1) \rfloor$, determines the period given as $\text{lcm}(m^\ell, n)$. For details the reader is referred to (Schwarz and Burkhard, 1996).

TABLE 5.2 ACATS periods for layouts of n strings and m disks per string.

#strings	n = 4	n = 5	n = 6	n = 7	n = 8	n = 9	n = 10
m = 3	36	45	18	63	72	27	270
m = 4	4	80	48	112	16	144	80
m = 5	20	5	159	175	200	225	50
m = 6	36	1080	216	252	178	216	1080
m = 7	28	35	42	7	3392	441	490

5.19.2 DATUM for tolerating multiple disk failures in disk arrays

DATUM data layout for tolerating multiple disk failures was proposed by Guillermo Alvarez (HP, IBM/ARC, consultant), Walter Burkhard (emeritus), and Flaviu Cristian (IBM, UCSD, rip) at UCSD (Alvarez et al., 1997) masks any number of failures while requiring an optimal amount of redundant storage space. Reconstruction accesses are spread uniformly over disks in the presence of failures without needing large layout tables in controller memory.

DATUM is based on the *Information Dispersal Algorithm - IDA* (Rabin, 1989), *IDA - Information Dispersal Algorithm* which has an efficient VLSI implementation (Bestavros, 1990). With assistance from Wong Wing Shing and Danny Chen Bestavros visiting Bell Labs authored a report on the performance analysis of IDA (Bestavros, 1991).

IDA encodes a sequence $\mathcal{F} = (d_1, d_2, \dots, d_m)$ into a sequence $\mathcal{E} = (e_1, e_2, \dots, e_m, e_{m+1}, \dots, e_{m+f})$, such that $\mathcal{E} = \mathcal{F}T$, where T is an $m \times (m + f)$ matrix:

$$T = \begin{pmatrix} 1 & 0 & \dots & t_{1,m+1} & t_{1,m+2} & \dots & t_{1,m+f} \\ 0 & 1 & \dots & t_{2,m+1} & t_{2,m+2} & \dots & t_{2,m+f} \\ \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & t_{m,m+1} & t_{m,m+2} & \dots & t_{m,m+f} \end{pmatrix}$$

Space is allocated uniformly over all disks to avoid bottlenecks. Disks that store strips of a given stripe are represented by a k -combination: $c = \langle X_1, \dots, X_k \rangle$, where $X_i \in [0, n - 1]$ is a disk identifier, furthermore: $X_i < X_{i+1}$, $1 \leq i \leq k$ and $nc = \text{num_combs} = \binom{n}{k}$. Given the address of a strip we need to compute the set of disks that have parts of the corresponding stripe for on-the fly reconstruction.

The total order $\succ$ over the set of combinations is $\langle X_1, \dots, X_k \rangle \succ \langle Y_1, \dots, Y_k \rangle$ if and only if either $X_k > Y_k$ or there is a $j < k$ such that $X_i = Y_i$ for $k \geq i \geq j$ and $X_j > Y_j$. Given a k -combination $c = \langle X_1, \dots, X_k \rangle$ then:

$$\text{loc}(\langle X_1, \dots, X_k \rangle) = \binom{X_k}{k} + \binom{X_{k-1}}{k-1} + \dots + \binom{X_1}{1}$$

The following algorithm computes the k -combination in array c , given its position.

```
invloc(position)
for (int i=k; i >= 1; i--)
  l=i
while choose(l,i) <= j
  l=l+1;
c[i]=l-1;
position=position-choose(l-1,i);
return(c)
```

Stripe unit addr as seen by the user is contained in stripe number $\text{str} = \lfloor \text{addr}/m \rfloor$ of the DATUM array since each stripe has m data units and the existence of redundant units is transparent to the user. Since there are more stripes than k -combinations str is mapped into the disk of k -combination

$\text{invloc}(\text{str} \bmod \text{numb_combs})$.

Given $c = \langle X_1, \dots, X_k \rangle$ corresponding to a stripe m of disks hold data strips and remaining $f = k - m$ hold redundant units. We take disks in positions

$$\lfloor \text{str}/nc \rfloor f, (\lfloor \text{str}/nc \rfloor f + 1) \bmod k, \dots, (\lfloor \text{str}/nc \rfloor f + f - 1) \bmod k$$

Fig. 5.16 shows the first iteration for $n = 4$, $k = 3$, $m = 2$, so that $f = k - m = 1$, i.e., a single failure is tolerated.

IDA is described in Chapter 2 in conjunction with Cleversafe acquired by IBM.

5.19.3 Permutation Development Data Layout - PDDL

PDDL was proposed in (Schwarz et al., 1999) by Thomas Schwarz (Marquette U.), Jesse Steinberg, and Walter Burkhard (emeritus) at UCSD exhibits excellent performance for both light and heavy workloads and has a straightforward space and run-time efficient implementations providing excellent parallel access performance and reduced incremental loads during

Offset	Disk0	Disk1	Disk2	Disk3
0	S0,R	S0,U0	S0,U1	S1,U3
1	S1,R	S1,U2	S2,U4	S2,U5
2	S2,R	S3,R	S3,U6	S3,U7
3	S4,U8	S4,R	S4,U9	S5,U11
4	S5,U10	S5,R	S6,R	S6,U13
5	S6,U12	S7,U14	S7,R	S7,U15
6	S8,U16	S8,U17	S8,R	S9,R
7	S9,U18	S9,U19	S10,U21	S10,R
8	S10,U20	S11,U22	S11,U23	S11,R

FIGURE 5.16 Sample layout for $f = 1$. $Sn.Um$ Data unit with address m , contained in stripe n . $Sn.R$ Redundant unit of stripe n .

degraded operation. PDDL can easily accommodate multiple failure tolerant redundancy schemes and incurs minimum address translation overhead compared to all other proposed declustering layouts.

Considered is a data layout for n disks with g stripes of width k and one spare: $n = gk + 1$. In addition to the six criteria expressed in (Holland et al., 1994) two more criteria are added:

7. Distributed sparing.
8. Maximal degraded read parallelism.

With $n = 7$ disks the first column represents spare strips and column 4 and 7 are check strips as shown in Fig. 5.17.

D0	D1	D2	D3	D4	D5	D6	D0	D1	D2	D3	D4	D5	D6
S	A0	A1	PA	B0	B1	PB	S	A0	A1	B0	PA	PB	B1
S	C0	C1	PC	D0	D1	PD	D1	S	C0	C1	D0	PC	PD
S	E0	E1	PE	F0	F1	PF	PF	F1	S	E0	E1	F0	PE
S	G0	G1	PG	H0	H1	PH	PG	PH	H1	S	G0	G1	H0
S	I0	I1	PI	J0	J1	PJ	J0	PI	PJ	J1	S	I0	I1
S	K0	K1	PH	L0	L1	PL	K1	L0	PK	PL	L1	S	K0
S	M0	M1	PM	N0	N1	PN	M0	M1	N0	PM	PN	N1	S

FIGURE 5.17 Virtual versus Physical RAID level 4.

```
int permutation[] = { 0, 1, 2, 4, 3, 6, 5 }
int virtual2physical( int disk , int offset )
{
return ( ( permutation[disk] + offset ) % 7 )
}
```

Satisfactory base constructions can be obtained using the construction in (Bose, 1939).

1. Determine a primitive element ω , which allows non-zero elements to be expressible as powers of ω modulo n .
2. Distribute all non-zero elements in round-robin fashion into g subsets:

$$B_1 = \{\omega^0, \omega^g, \dots, \omega^{g(k-1)}\}$$

$$B_2 = \{\omega^1, \omega^{g+1}, \dots, \omega^{g(k-1)+1}\}$$

$\vdots$

$$B_g = \{\omega^{g-1}, \omega^{2g-1}, \dots, \omega^{gk-1}\}$$

The base permutation is a concatenation of B 's. In the case of nonprime n satisfactory constructions are given in (Furino et al., 1996).

For $n = 7$ and $g = 2$, $\omega = 3$ is a primitive element, which yields $B_1 = \{1, 2, 4\}$ and $B_2 = \{3, 6, 5\}$. The RAIDFrame simulation package (Courtright et al., 1996), (Courtright et al., 1997) was used to determine the *Disk Working Set - DWS* sizes, which is the number of disks that perform at least one physical access to process a logical access. Compared were PDDL, DATUM, PRIME, Parity Declustering, and RAID5.

5.19.4 Balancing loads by shifted parity group placements

We propose a data allocation layout as follows. We place the strips of a PG of size G on consecutive strips of a disk array with N disks. The first $G - 1$ elements of a PG are data strips denoted by D and parity strip is denoted by P , which are computed by XORing the preceding $G - 1$ data blocks.

$iG \bmod N, i \geq 1$ generates all values $0 : N - 1$ in a periodic manner. The parameter G coprime or relatively prime with N , is referred to as a “generator” of the mod N addition group. Therefore the parities will appear in different columns.

The number of strips in the tables is $L = \text{LCM}(N, G)$. The number of rows in the table is $r = L/N$ and the number of parity groups $k = L/G$. The GCD (Greatest Common Divisor) of N and G : $m = \text{GCD}(N, G)$ is also important in our discussion. We first consider $N = 10, G = 3$ in Fig. 5.18.

Disk #	1	2	3	4	5	6	7	8	9	10
PGs	D_1	D_2	$P_{1:2}$	D_3	D_4	$P_{3:4}$	D_5	D_6	$P_{5:6}$	D_7
(initial	D_8	$P_{7:8}$	D_9	D_{10}	$P_{9:10}$	D_{11}	D_{12}	$P_{11:12}$	D_{13}	D_{14}
allocation)	$P_{13:14}$	D_{15}	D_{16}	$P_{15:16}$	D_{17}	D_{18}	$P_{17:18}$	D_{19}	D_{20}	$P_{19:20}$

FIGURE 5.18 Allocation for $N = 10$ and $G = 3$. Three rows are required since $r = \text{LCM}(10, 3)/10 = 3$. There is no need for shifting since $m = \text{GCD}(10, 3) = 1$.

We next consider the case $N = 10$ and $G = 4$ in Fig. 5.19.

Disk #	1	2	3	4	5	6	7	8	9	10
Parity	D_1	D_2	D_3	$P_{1:3}$	D_4	D_5	D_6	$P_{4:6}$	D_7	D_8
groups	D_9	$P_{7:9}$	D_{10}	D_{11}	D_{12}	$P_{10:12}$	D_{13}	D_{14}	D_{15}	$P_{13:15}$
(initial	D_{17}	D_{18}	$P_{16:18}$	D_{19}	D_{20}	D_{21}	$P_{19:21}$	D_{22}	D_{23}	D_{24}
allocation)	$P_{22:24}$	D_{25}	D_{26}	D_{27}	$P_{25:27}$	D_{28}	D_{29}	D_{30}	$P_{27:30}$	D_{31}

FIGURE 5.19 Allocation of parity groups in Fig. 5.13 with $N = 10, G = 4$. Two rows are required since $r = \text{LCM}(10, 4)/10 = 2$ and a single shift is required since $r = \text{GCM}(10, 4) = 2$. There are five parity groups.

In a final example we consider $N = 15$ and $G = 6$ in Fig. 5.20.

Disk #	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Groups	D_1	D_2	D_3	D_4	D_5	$P_{1:5}$	D_6	D_7	D_8	D_9	D_{10}	$P_{6:10}$	D_{11}	D_{12}	D_{13}
0 shift	D_{14}	D_{15}	$P_{11:15}$	D_{16}	D_{17}	D_{18}	D_{19}	D_{20}	$P_{16:20}$	D_{21}	D_{22}	D_{23}	D_{24}	D_{25}	$P_{21:25}$
Groups	D_{27}	D_{28}	D_{29}	D_{30}	$P_{26:30}$	D_{31}	D_{32}	D_{33}	D_{34}	D_{35}	$P_{31:35}$	D_{36}	D_{37}	D_{38}	D_{39}
1 shift	D_{40}	$P_{36:40}$	D_{41}	D_{42}	D_{43}	D_{44}	D_{45}	$P_{41:45}$	D_{46}	D_{47}	P_{48}	D_{49}	D_{50}	$P_{46:50}$	D_{51}
Groups	D_{53}	D_{54}	D_{55}	$P_{51:55}$	D_{56}	D_{57}	D_{58}	D_{59}	D_{60}	$P_{56:60}$	D_{61}	D_{62}	d_{63}	d_{64}	d_{65}
2 shifts	$P_{61:65}$	D_{66}	D_{67}	D_{68}	D_{69}	D_{70}	$P_{66:70}$	D_{71}	D_{72}	D_{73}	D_{74}	D_{75}	$P_{71:75}$	D_{76}	D_{77}

FIGURE 5.20 $N = 15$ and $G = 6$, so that $r = \text{LCM}(15, 6)/15 = 2$ and $k = \text{GCM}(15, 6) = 3$, so that two shifts are required. There are five parity groups in two rows.

5.20 Log-structured file systems and arrays

The Iceberg disk array implemented the *Log-Structured File System - LFS* paradigm in 1994, which provided outboard data compression, compaction, log structured files, and RAID6 2-Disk-Failure-Tolerant - 2DFT array.

https://en.wikipedia.org/wiki/Storage_Technology_Corporation

The justification for LFS stated in (Ousterhout and Douglass, 1989), (Rosenblum and Ousterhout, 1992) by John Ousterhout (Stanford U.), Fred Douglass (<http://www.douglass.org/fred/>), and Mendel Rosenblum (Stanford U.) at UCB (Ousterhout and Douglass, 1989), (Rosenblum and Ousterhout, 1992) is that many applications are I/O-limited and unable to benefit from improvements in CPU speeds. Reads are satisfied by increased buffer sizes in main memory, while writes constitute most of the disk loads.

A more detailed description and evaluation of LFS is presented in (Rosenblum and Ousterhout, 1992; Rosenblum, 1995). LFS speeds up both file writing and crash recovery by writing all modifications to disk sequentially in a log-like structure. Indexing information is added to the log so that files can be read back efficiently. Disk space is subdivided into segments and a segment cleaner moves files from mostly empty segments to mostly full segments. The empty segments are then available for writing modified files. Simulation can be used to demonstrate the efficiency of a simple cleaning policy based on cost and benefit.

The primary area of work on Sprite file systems was the building of a new *Network File System - NFS*, which made heavy use of local client-side caching in order to improve performance. After opening the file and some initial reads, the network is only used on-demand, and most user actions occur against the cache. Another notable features include support for *Single System Image - SSI* on computer clusters and the introduction of Tcl scripting language also originated.

[https://en.wikipedia.org/wiki/Sprite_\(operating_system\)](https://en.wikipedia.org/wiki/Sprite_(operating_system))

The Sprite LFS prototype outperformed current Unix file systems by an order of magnitude for small-file writes, while matching or exceeding UNIX performance for reading or writing large files. With cleaning overhead included Sprite LFS can use 70% of the disk bandwidth for writing, whereas UNIX file systems typically use five-to-ten percent due to seek and latency times.

Informal analysis of LFS

The following analysis follows Chapter 6 in “The Fractal Nature of Data Reference” (McNutt, 2002). by Bruce McNutt’s (IBM’s *Storage Systems Division - SSD*, retired) provides an informal derivation relating the mean number of times a data item is moved (M) to the fraction of utilized storage (u) in LFS.

A first-cut approximation to M chosen for simplicity rather than realism is as follows:

$$M = \frac{0.5}{1 - u} - 1, u > 0.5 \quad (6.1)$$

The realism of the model is improved by taking into account:

1. The impact of transient patterns of data reference within the workload.
2. Algorithms taking into account such patterns.

A data item that is first written belongs to generation 1, if a data item lasts long enough it enters generation 2, etc. Given that f represents the fraction of data items in any generation that move to the next generation, an enumeration of all possible outcomes for the life cycle of a given item is as follows.

1. The item never moved before becoming invalidated with probability $1 - f$.
2. The item moved exactly i times: $f^i (1 - f)$.

The average number of moves is then:

$$M = \frac{f}{1 - f}. \quad (6.2)$$

Assuming that the utilization of a segment declines at a linear rate

$$u = \frac{1 + f}{2}. \quad (6.3)$$

Substituting f from Eq. (6.3) in Eq. (6.2) yields Eq. (6.1).

LFS performance is affected by background I/O load for defragmentation, i.e., free space consolidation, which depends on several factors, including the storage capacity, the current write I/O rate, and the distribution of free space in the array. As noted earlier, files can be compressed because they are not written in place, otherwise there is the problem that a file may not compress as well in successive compressions.

Online data compression in LFS, which is described in (Burrows et al., 1992), was developed by Michael Burrows (Google) et al. at DEC’s *Systems Research Center - SRC* in Palo Alto, CA The block size for compression was varied from 1 KB to 32 KB yielding a compression ratio ranging from 68% to 51% as shown in Table 5.3 (Burrows et al., 1992).

TABLE 5.3 Percent compression ratio (CR) vs input block size (from Figure 1 in (Burrows et al., 1992)).

Block size	1 KB	2 KB	4 KB	8 KB	16 KB	32 KB
CR percent	68	63	59	55	53	51

Several compression algorithms including Lempel-Ziv in (Ziv and Lempel, 1977) were tried in this study. https://en.wikipedia.org/wiki/LZ77_and_LZ78
In addition to the *Compression Ratio* - CR, compression/decompression speed is another performance metric in this study.

Explanation by Michael Burrows

The *Compression Ratio* - CR in that table is output size divided by input size, so smaller numbers indicate more effective compression.

Compression typically becomes more effective with increased block size, because compression is all about prediction, which improves with better statistics. Larger block sizes provide more opportunity to gather statistics.

The influence of block size on compression effectiveness, becomes more pronounced with higher-order compression schemes, which use more symbols of context when predicting the next symbol. This is because the number of possible contexts increases exponentially with the context length. The chance of getting reliable statistics for a long context is quite small for small block sizes, so compression is correspondingly less effective for small block sizes.

As a more operational answer: the algorithm used for Table 2 ("Algorithm 2" in the paper) uses a trivial hash of the last three symbols to predict the next. The predictions are just the symbols seen most recently for each hashed context; they are stored in a table indexed by the hash value. The longer the block, the greater the fraction of symbols that benefit from the entries in the hash table populated earlier in the block. Three is a modest context length when compared with the most effective compression schemes, but is still large enough for there to be many contexts, and thus for the block size to have a significant influence on CR.

A performance analysis of *Log Structured Arrays* - LSA with a single parity disk and compared its performance with RAID5 in (Menon, 1995). <https://slideplayer.com/slide/4666204/>

Derivation of the write batch size to minimizes read response time

While LFS maximizes disk write operations, read operations can experience long queueing delays when batched writing is in progress. Scott Carson and Sanjeev Setia (GMU) analytically derived the write batch size that minimizes read response time, while retaining the benefit of write batching in (Carson and Setia, 1994). This study is related to authors' earlier work in (Carson and Setia, 1992).

Read and write requests arrive with rates λ_R and λ_W to the disk cache. Assuming that reads have a fixed cache hit ratio h_r the rate of requests to disk is $\lambda_r = (1 - h_r)\lambda_R$.

Let T be the length of interval between two updates. The number of dirty blocks written at the update interval is $\bar{A} = \lambda_w T = (1 - h_w(T))\lambda_W T$. It is assumed that the number of blocks written out every T second is always $\bar{A}$; thus the partial batch has $p = \bar{A} \bmod c$ blocks.

The mean service time for writing a batch size c is

$$\bar{s}_w = \overline{sk} + \overline{rot} + \overline{tr(c)}$$

where sk is the seek time, rot is the rotational latency, and $tr(c) = kc$ is the transfer time, with k denoting the number of 4K blocks written per millisecond.

The average service time for writing a partial batch is:

$$\bar{s}_p = \overline{sk} + \overline{rot} + \overline{tr(p)}$$

The average read service time is given by

$$\bar{s}_r = \overline{sk} + \overline{rot} + \overline{tr(1)}$$

The variance for a disk operation $o = \in (w, p, r)$, which are full and partial batch write or a read

$$\sigma_o^2 = \sigma_{sk}^2 + \sigma_{rot}^2$$

Given that the variance is the same in the three cases the subscript will be dropped. The coefficient of variation of service time is then: $C_{s_o}^2 = \sigma^2 / \bar{s}_o^2$.

Using the equation for head of the line priority queueing model in (Kleinrock, 1976) the mean response time for higher read requests is:

$$\bar{R} = \bar{s}_r + \frac{\rho_r \bar{s}_r (1 + C_{s_r}^2) + \rho_w \bar{s}_w (1 + C_{s_w}^2) + \rho_p \bar{s}_p (1 + C_{s_p}^2)}{2(1 - \rho_r)}$$

where $\rho_w = \lfloor \frac{\bar{A}}{c} \rfloor \times \frac{\bar{s}_w}{T}$ and

$$\rho_p = \begin{cases} \frac{s_p}{t} & \text{if } \bar{A} \bmod c > 0 \\ 0 & \text{otherwise} \end{cases}$$

Simulations that relax the assumptions of the analytic model demonstrate that the analytic result can be used to advantage in a practical setting.

It is assumed that c is a real number and there are no partial batch writes.

$$R(\bar{c}) = \bar{s}_r + \frac{\rho_r \bar{s}_r (1 + C_{s_r}^2)}{2(1 - \rho_r)} + \frac{\lambda_w}{2(1 - \rho_r)} \left[\frac{D^2 - \sigma^2}{c} + k^2 c + 2Dk \right]$$

where $D = \bar{s}k + r\bar{o}t$.

$$\frac{\partial R(\bar{c})}{\partial c} = \frac{\lambda_w}{2(1 - \rho_r)} \left[k^2 - \frac{D^2 + \sigma^2}{c^2} \right] = 0$$

which yields

$$c^* = \sqrt{\frac{D^2 + \sigma^2}{k^2}}$$

Interestingly the batch size that minimizes the read response time is independent of the read and write loads and only depends on the characteristics of the disk.

A performance study of LFS was reported as part of an investigation at IBM *Storage System Division - SSD* in (McNutt, 1994). A directory indicates the physical location of each logical object and the location of the most recent copy for objects written more than once. Storage fragmentation is dealt with by continuous garbage collection by shifting the location of stored data to release unused storage for reuse. A mathematical model of garbage collection shows how collection load relates to the utilization of storage and the amount of update locality present in the pattern of updates. A realistic statistical model of updates, based upon trace data analysis is applied. In addition, alternative policies are examined for determining which data areas to collect.

A new algorithm for choosing segments for garbage collection in LFS and LSA was proposed at IBM/ARC and compared against greedy and cost-benefit algorithms via simulation (Menon and Stockmeyer, 1998). The basic idea is that segments which have been recently filled by writes should not be considered for garbage collection until they achieved a certain age threshold. This is because as time progresses data in such segments is less likely to be updated, making old segments eligible for garbage collection. Among eligible segments the algorithm chooses segments that yield the most free space. Also given is a method to determine the optimal age-threshold under certain assumptions about the workload. This study was conducted before IBM became an Iceberg reseller under the name RAMAC Virtual Array - RVA (IBM Corp. Int'l Tech. Support Org., 1997). A related study is reported by Henry Butterworth (IBM/Hursley, UK) in (Butterworth, 1999).

A storage management system that has the ability to adapt to the data access characteristics of the application that uses it based on collection and analysis of runtime statistics is presented in (Chee et al., 1998). This feature is especially useful in the storage management layer of database systems, where applications exhibit relatively predictable access patterns. Adaptive reorganization is performed by the storage management system in a manner that optimizes the access patterns

of the system for which it is used. The log-structured storage system that naturally caters for write optimization, with the addition of a statistics collection mechanism to determine data access patterns of applications. The storage system can serve as a testbed for a variety of statistics analysis and clustering mechanisms. Higher level application-specific data clustering mechanisms can be used to override the storage system's low-level clustering mechanisms. In addition, the analysis techniques and reorganization scheme can be used in other storage systems. Performance results from our prototype show potential response time speedups of up to 83% over the basic log-structured file system in the best case, using a combination of storage reorganization and prefetching.

The *Zebra Network File System - NFS* was specified and implemented in (Hartman and Ousterhout, 1995) at UCB by John Hartman (U. Arizona), a student of John Ousterhout (UCB, Stanford U.) at UCB (Hartman and Ousterhout, 1995). The throughput of the distributed file systems is increased by striping the file data across multiple servers. Rather than striping each file separately Zebra combines strips from multiple sources, which provides high performance for writing small files as well as for reading and writing of large files. Zebra also writes parity information in each stripe in the style of RAID5 disk arrays; this increases storage costs slightly, but allows the system to continue operation while a single storage server is unavailable. The prototype implementation of Zebra built using the Sprite OS provided 4–5 times the throughput of the standard Sprite file system or NFS for large files and a 15–300% improvement for writing small files. A log-structured interface to a cluster of storage devices implemented as Swarm is described in (Muntz and Lui, 1990a).

Three adaptive methods to improve LFS performance are considered at UCB in (Matthews et al., 1997).

- (1) choosing the segment size to match disk and workload characteristics,
- (2) modifying the cleaning policy to adapt to changes in disk utilization,
- (3) using cached data to lower cleaning costs.

Data is reorganized to match read patterns to improve read response time. Trace-driven simulations on a combination of synthetic and measured workloads are used to demonstrate that these extensions to LFS can significantly improve its performance.

LFS is enhanced with a statistics collection mechanism to determine data access patterns of applications. Higher level application-specific data clustering mechanisms can be used to override the storage system's low-level clustering mechanisms (Chee et al., 1998). Performance results from the prototype showed potential response time speedups of up to 83 percent over the basic LFS in the best case, using a combination of storage reorganization and prefetching.

5.21 RAID6

StorageTek was the first company to develop a RAID6 disk array with two check strips per stripe with *Reed-Solomon - RS* coding in 1994 (Reed and Solomon, 1960), (MacWilliams and Sloane, 1977). The *Disk Array Controller - DAC* for Iceberg disk array was appropriately named Penguin (Friedman, 1995). RS codes are *Maximum Distance Separable - MDS* in that they incur the minimum level of redundancy, e.g., $k = 2$ strips per stripe to tolerate two disk failures. To balance disk loads the P and Q check strips were laid out in right to left diagonals as shown in Fig. 5.3.

Iceberg additionally implemented the *Log-structured File System - LFS* paradigm (Rosenblum and Ousterhout, 1992), whose extension to RAID5 was named *Log-Structured Array - LSA* in (Menon, 1995).

STK's Iceberg had sixteen disks: 13 data, two check, and a hot spare. The cache controller managed a 64 to 512 MB battery-backed cache. The controller used an 8 MB *NonVolatile Storage - NVS* cache to keep track of free space within the array, data mapping between logical IBM 3380/3390 disks and physical blocks within the array, and reclaiming disk space that is no longer in use. The following comments are applicable to Iceberg:

1. Compressing data results in the doubling of the effective storage capacity of the disk array.
2. Compressed data cannot be written into its original space, since after modification it may not be compressible to the same size.
3. Data chunks are accumulated into large segments which are written as full stripes reducing update overhead.
4. The cache is used as a staging area for converting variable length blocks associated with the *Extended Count Key Data - ECKD* format of IBM mainframe disks into *Fixed Block Architecture - FBA* format and vice-versa.

NetApp's argument for the viability of RAID6

NetApp argues that RAID6 does not incur a higher storage overhead than RAID5, since rather than two $7 + 1$ RAID5 arrays a $14 + 2$ RAID6 array will do.

http://ntptest.typepad.com/dave/2006/05/why_double_prot.html

Considering disks with reduced reliability $r = 1 - \varepsilon$, asymptotic reliability analysis yields the following expressions for RAID5 and RAID6 (Thomasian, 2006a):

$$R_5 \approx 1 - [N(N-1)/2]\varepsilon^2 \text{ and } R_6 = 1 - [2N(2N-1)(2N-2)/6]\varepsilon^3.$$

For $N = 7$ the reliability of two RAID5 arrays is:

$$R_5^2 \approx 1 - N(N-1)\varepsilon^2 \approx 1 - 42\varepsilon^2 \text{ versus } R_6 \approx 1 - 364\varepsilon^3,$$

i.e., RAID6 is much more reliable since it can tolerate all double disk failures, but the two RAID5 arrays can only tolerate two disk failures if they are not in the same RAID5. For updates RAID5 has the advantage of requiring four disk accesses versus six for RAID6.

5.21.1 Supplementary Parity Augmentation - SPA

RAID5 with SPA was proposed as part of a PhD research at Huazhong U. In addition to the **P** parity as the N^{th} disk has an **S** parity at its $N/2^{th}$ disk. The **P** parity covers all disks, while the **S** parity covers the first half of disks. The goal of the **S** parity is to reduce update overhead, shorten rebuild time, by reducing the number of disks involved in rebuild, and increase MTDDL with respect to RAID5 at the increased cost two parity updates for half of the disks (Tian et al., 2011).

Although SPA has the same disk space redundancy level as RAID6, unlike RAID6 it can only deal with only one half of all possible double disk failures, i.e., when the position of one failed disk precedes the **S** parity and the other follows. Ignoring LSE's the MTDDL for SPA is higher than RAID5, but lower than RAID6. Half of the disks require updating two check disks, hence the update overhead is lower than RAID6. An advantage of SPA is that half of the disks need to be read for rebuild as compared with RAID5. This may lead to a higher MTDDL than RAID5, since fewer LSEs are encountered. A more detailed review of SPA appears in (Thomasian, 2012b), which implies its publication in *ACM Trans. on Storage - TOS* was not warranted.

5.22 Reed-Solomon coding for higher reliability

Reed-Solomon - RS codes are due to Irving Reed at *U. Southern Calif. - USC* and Gustave Solomon (JPL consultant) (Reed and Solomon, 1960).

https://en.wikipedia.org/wiki/Reed%E2%80%93Solomon_error_correction

RS codes use computationally costly finite field arithmetic.

https://en.wikipedia.org/wiki/Finite_field_arithmetic

The notable history of the USC Viterbi School of Engineering 1906 - 2014, was compiled by former EE-Systems Dept department chair George and Mrs (rip) Bekey (Bekey and Bekey, 2015) and is attributed to Dean Zohrab Gabrielian.

<https://viterbischool.usc.edu/history/>

James Plank's (U. Tennessee, Knoxville) paper on implementing RS codes (Plank, 1997a) incorrectly specified the Vandermonde matrix. A mistake that was corrected eight years later in (Plank and Ding, 2005). Plank gave a tutorial on erasure codes in storage systems at USENIX FAST'05 conference (Plank, 2005). A second tutorial with Cheng Huang (Microsoft) was presented at FAST'13 (Plank and Huang, 2013). My paper citing (Plank, 1997a) to the FAST'03 Conf. was rejected because of this mistake, but was published later as (Thomasian et al., 2004, 2007a).

5.22.1 RS coding summary and finite field arithmetic

RS is a *Maximum Distance Separable - MDS* code, which encodes data from n disks onto m disks. This means that m erasures are tolerated. RS codes operate on words with w bits, where $n + m \leq 2^w$.

Eight bits are used if $n + m \leq 256$ and short binary (16 bit) numbers if $n + m \leq 65,536$.

$$\begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ B_{1,1} & B_{1,2} & B_{1,3} & B_{1,4} & B_{1,5} \\ B_{2,1} & B_{2,2} & B_{2,3} & B_{2,4} & B_{2,5} \\ B_{3,1} & B_{3,2} & B_{3,3} & B_{3,4} & B_{3,5} \end{pmatrix} \times \begin{pmatrix} D_1 \\ D_2 \\ D_3 \\ D_4 \\ D_5 \end{pmatrix} = \begin{pmatrix} D_1 \\ D_2 \\ D_3 \\ D_4 \\ D_5 \\ C_1 \\ C_2 \\ C_3 \end{pmatrix}$$

B is a Vandermonde matrix, which uses finite or Galois field arithmetic. If D_1 , D_4 , and C_2 fail remove corresponding rows from matrix B , rename it B' , and invert B'^{-1} . The data disks can be obtained as

$$D = B'^{-1} \times (D_2, D_3, D_5, C_1, C_3).$$

Galois Field - GF arithmetic

GF(2^w) has elements $(0, 1, 2, \dots, 2^w - 1)$, e.g., Galois field elements for $w = 3$ are $(0, 1, 2, 3, 4, 5, 6, 7)$.

GF addition simply XORs binary representation $(3 + 2) = 11 \oplus 10 = 1$, $(5 + 5) = 0$, $(7 + 3) = 4$.

Multiplication and division are hard to explain, but can be determined by table lookup.

Multiplication in a finite field is modulo an irreducible polynomial used to define the finite field, which is set to $g(x) = x^8 + x^4 + x^3 + x + 1$.

Consider the multiplication of $53 = x^5 + x^4 + x^2 + 1 = 32 + 16 + 4 + 1$ with its inverse which yields 1 in Rijndael, as proposed by Vincent Rijmen and Joan Daemen for *National Standard of Science and Technology - NIST* in 2001 and is known as the *Advanced Encryption Standard - AES*.

https://en.wikipedia.org/wiki/Advanced_Encryption_Standard

$$\begin{aligned} (x^6 + x^4 + x^2 + 1)(x^7 + x^6 + x^3 + x) = \\ x^{13} + x^{12} + x^{11} + x^{10} + x^9 + x^8 + x^6 + x^5 + x^3 + x^2 + x \\ \text{mod}(x^8 + x^4 + x^3 + x + 1) = 1 \end{aligned}$$

The multiplication table for Galois field arithmetic can be specified as follows:

	0	1	2	3	4	5	6	7
0	0	0	0	0	0	0	0	0
1	0	1	2	3	4	5	6	7
2	0	2	4	6	3	1	7	5
3	0	3	6	5	7	4	1	2
4	0	4	3	7	6	7	2	1
5	0	5	1	4	2	7	3	6
6	0	6	7	1	5	3	2	4
7	0	7	5	2	1	6	4	3

Cauchy RS codes are due to Johanness Blomer et al. at *ICSI - International Computer Science Institute* at Berkeley, CA (Blomer et al., 1995). Two improvements:

1. RS coding is implemented using a Cauchy matrix instead of a Vandermonde matrix, so the inversion cost is $O(n^2)$.
2. Projection converts Galois Field multiplications into XORs.

5.23 Parity based MDS codes

Parity based MDS parity codes to protect against two disk failures, which are less computationally costly than RS codes are discussed below in chronological order of their introduction:

EVENODD: Developed by Mario Blaum (IBM, HGST, IBM), Jim Brady (IBM, retired), Jehoshua Bruck (IBM, CalTech), and Jai Menon (IBM, Dell/EMC, Cloudera, Fungible) at IBM/ARC (Blaum et al., 1994, 1995), (Blaum et al., 2001). Coinventor Jim Brady has authored a book on his experience as computer systems architect at IBM (Brady, 2018). A new class of low-density MDS array codes for tolerating double disk failures in disk arrays with lower encoding and decoding complexity than EVENODD are discussed in (Fujita and Sakaniwa, 2007).

X-code: Developed by Lihao Xu (Wayne State) and Jehoshua/Shuki Bruck at CalTech (Xu and Bruck, 1999) is discussed in Section 5.27.

Rotated Diagonal Parity - RDP: was developed and patented by Peter Corbett (IBM, NetApp, HPE) and Atul Goel et al. at NetApp (Corbett et al., 2004), (Corbett et al., 2006) in Section 5.24.

The processing required for erasure coding has been investigated in the context of open source erasure coding libraries in (Plank et al., 2009a).

Optimizing storage solutions using Intel’s intelligent storage acceleration library is described in (Le, 2014).

A more recent discussion in (Chen and Xu, 2019) lists erasure coding libraries supporting RS codes and their programming languages.

Jerasure (Plank et al., 2014) and Intel’s ISA-L are written in C,

<https://software.intel.com/en-us/intel-cplusplus-compiler-17.0-user-and-reference-guide>

James Plank after consulting for StreamScale on erasure coding developed open-source software with Kevin Greenan at *U. Calif. at Santa Cruz - UCSC*, but StreamScale’s legal challenge led to his withdrawal from research in this field.

<https://www.techdirt.com/articles/20141115/07113529155/patent-troll-kills-open-source-project-speeding-up-computation-erasure-codes.shtml>

Zfec was written in Python (O’Whielacronx, 2012).

BackBlaze is an open-source RS erasure code written in Java (Beach, 2015).

<https://www.backblaze.com/blog/reed-solomon>

There has been interest in introducing redundancy to increase reliability in distributed storage systems. The BackBlaze storage vault takes 17 pieces of the same size and adds three additional check pieces so that there are a total of 20 pieces. The data could be processed at the rate of 149 MB/sec on 3.7 MHz Intel Xeon.

BackBlaze is also a data storage provider, which offers two products:

B2 Cloud Storage - An object storage service similar to Amazon’s S3.

Computer Backup - An online backup tool that allows Windows and macOS users to back up their data to offsite data centers.

<https://www.backblaze.com/blog/backblaze-hard-drive-stats-q1-2020/>

5.24 RDP arrays and their optimal recovery

The paper describing *Rotated Diagonal Parity - RDP* code developed at *Network Appliance - NetApp Corp.* won the best paper award at FAST’04 Conf. (Corbett et al., 2004). RDP is an MDS 2DFT code like EVENODD which outperforms EVENODD as discussed in Section 11.2. RDP was patented two years later (Corbett et al., 2006).

RDP arrays have a controlling parameter $p > 2$, which is a prime number. With $p + 1$ disks, in each row $p - 1$ blocks hold data, one block holds a horizontal parity, and another block diagonal parity as shown in Fig. 5.21, which is based on (Corbett et al., 2004).

Disk0	Disk1	Disk2	Disk3	P	Q
d_0	d_1	d_2	d_3	p_4	q_0
d_1	d_2	d_3	d_4	p_0	q_1
d_2	d_3	d_4	d_0	p_1	q_2
d_3	d_4	d_0	d_1	p_2	q_3

FIGURE 5.21 Storage system with RDP code with $p = 5$. The P disk holds the row parities and the Q disk the diagonal parities with that number. Note that the parities cannot be computed without horizontal parities being computed first.

The parity strips in column 5 are computed as the XOR of the data strips in that row, e.g., $p_4 \leftarrow d_0 \oplus d_1 \oplus d_2 \oplus d_3$.

Updating a strip would require updating the corresponding strips of horizontal and diagonal parities. A full stripe write requires updating four strips on the Q disk which would be very costly.

This problem is alleviated by selecting the prime number $p = 2^n + 1$, which allows defining diagonal parities within a group of 2^n stripes. This allows the block size for RDP to be the usual system block size divided by 2^n . For example, if the system's disk block size is 4096 B we can select $p = 17$, so that 16 RDP 256 B blocks per strip. This allows full stripe writes to be processible efficiently. A similar scheme was used in EVENODD (Blaum et al., 1994, 1995).

The optimal recovery method of a single disk failure for the RDP code reported by Liping Xiang et al. at *Chinese U. Hong Kong - CUHK* in (Xiang et al., 2010), (Xiang et al., 2011) reduces the number of blocks accessed to reconstruct a failed disk, but does not take into account the optimization in the previous paragraph, hence it is applicable to distributed storage systems, but of no interest to storage systems.

Considered in this work is a larger example with $p = 7$ to illustrate the reduction in number of disk accesses in rebuilding a single disk, e.g., Disk0. Rebuild can be simply carried out by using parity disks, i.e., Disk6 only, but using both parities results in a reduction in the number of accessed blocks as shown in Fig. 5.22.

Disk0	Disk1	Disk2	Disk3	Disk4	Disk5	Disk6	Disk7
X	P		B	B	B	P	
X	P	B	B	B	B	P	
X	B	B	B	P	P	P	
X	Q	Q					Q
X	Q					P	Q
X				D	Q	Q	Q

FIGURE 5.22 RDP array with $p = 7$. Reducing the number of disk reads for recovering Disk0. P/Q/B blocks accessed to utilize horizontal, diagonal, and both parities. Note that Disk1 and Disk6 access five blocks and there are disks accessing 3 and 4 blocks. 24 blocks are altogether accessed.

A scheme that balances the disk loads as much as possible is given in Fig. 5.23.

Disk0	Disk1	Disk2	Disk3	Disk4	Disk5	Disk6	Disk7
X	Q		Q				Q
X		Q				P	Q
X	B	P	P	P	B	B	
X				Q	Q		Q
X	P	P	B	B	P	B	
X	P	B	B	P	B	P	

FIGURE 5.23 Load balancing in recovering Disk0. P/Q/B blocks accessed to utilize horizontal, diagonal, and both parities. Note that Disk1 and Disk6 access five blocks and there are disks accessing 3 blocks. Four blocks are accessed at six disk and three at one, 27 blocks are accessed altogether.

Conventional recovery in RDP using horizontal parities requires $(p - 1)^2$ reads, while optimal recovery requires

$$3(p - 1)^2/4$$

reads, which is an approximately 25% saving in the number of disk accesses.

Recovery in NetApp RDP is also illustrated in (White et al., 2013).

It is interesting that Xiang et al. (2010) and (Xiang et al., 2011) were published on top conference and a journal without reviewers noting that the small granularity used in computing check symbols.

5.25 EVENODD defined and efficient rebuild of a single disk

EVENODD coding with a prime number of disks $p = 5$ is shown in Fig. 5.24. The horizontal parities appearing on Disk5 are simply computed by XORing the blocks in the row.

$$\bigoplus_{j=0}^4 d_{i,j}, 0 \leq i \leq 3$$

The EVENODD adjustor symbol h is computed by XORing the parity blocks in the $p - 1^{st}$ diagonal, which is column 4 in the figure below.

$$h = \bigoplus_{j=1}^{p-1} d_{p-1-j,j}.$$

Disk0	Disk1	Disk2	Disk3	Disk4	Disk5	Disk6	Redundancy matrix for RM2
$d_{0,0}$	$d_{0,1}$	$d_{0,2}$	$d_{0,3}$	$d_{0,4}$	$d_{0,5}$	$d_{0,6}$	
$d_{1,0}$	$d_{1,1}$	$d_{1,2}$	$d_{1,3}$	$d_{1,4}$	$d_{1,5}$	$d_{1,6}$	
$d_{2,0}$	$d_{2,1}$	$d_{2,2}$	$d_{2,3}$	$d_{2,4}$	$d_{2,5}$	$d_{2,6}$	
$d_{3,0}$	$d_{3,1}$	$d_{3,2}$	$d_{3,3}$	$d_{3,4}$	$d_{3,5}$	$d_{3,6}$	

FIGURE 5.24 EVENODD coding with $p = 5$. Disk0-4 are data disks, Disk5 and Disk6 horizontal and diagonal parity disks (Figure 3 in (Xiang et al., 2011)).

Diagonal parities are computed as follows

$$d_{j,p+1} = \{d_{i,r} | i + r = j \bmod p\} \oplus h.$$

More specifically we have:

$$d_{0,6} = d_{0,0} \oplus d_{3,2} \oplus d_{2,3} \oplus d_{1,4} \oplus h$$

$$d_{1,6} = d_{0,1} \oplus d_{1,0} \oplus d_{3,3} \oplus d_{2,4} \oplus h$$

$$d_{2,6} = d_{0,2} \oplus d_{1,1} \oplus d_{2,0} \oplus d_{3,4} \oplus h$$

$$d_{3,6} = d_{0,4} \oplus d_{1,3} \oplus d_{2,2} \oplus d_{3,1} \oplus h$$

A possible practical implementation of small write operations is to let each symbol be an 8-bit byte, and $m = 257$ (a Fermat prime number). Therefore, we have an array of up to 259 disks, more than enough for present and future applications. Note that the array does not have to have 259 disks (this is just the maximum number); if it has fewer disks the remaining columns should be treated as having zeros. Each column of the array consists of 256 bytes, i.e., half a sector. In this case, a small write operation consists of writing a whole column. Thus, the two redundant columns will be modified accordingly. Say, each symbol $a_{i,j}$, $0 \leq i \leq m - 2$ in column j , $0 \leq j \leq m - 1$ is replaced by r_i . Then, we have to do the following modifications in the redundant symbols:

$$a_{i,m} \leftarrow a_{i,m} \oplus a_{i,j} \oplus r_i$$

$$a_{i,m+1} \leftarrow a_{i,m+1} \oplus a_{(i-j)_m,j} \oplus r_{(i-j)_m} \oplus a_{m-1-j,j} \oplus r_{m-1-j}.$$

That is, when a sector is updated, the two corresponding redundant sectors are also updated according to the two equations. Namely, this implementation has the advantage that every small write involves updates in only two other disk sectors.

Rebuild for array codes in distributed storage systems with an EVENODD configuration is considered in (Wang et al., 2010), but in what follows we base our discussion on (Xiang et al., 2011).

Disk0	Disk1	Disk2	Disk3	Disk4	Disk5	Disk6
X	Q			Q		Q
X			Q	Q		Q
X	P	B	B	B	P	
X	B	B	B	P	P	

FIGURE 5.25 Reducing number of reads to recover Disk0 of EVENODD code. Note that 16 block accesses are required, versus 20 using only horizontal parity blocks.

Disk0	Disk1	Disk2	Disk3	Disk4	Disk5	Disk6
X			Q	Q		Q
X	P	B	B	B	P	
X	B	B	B	P	P	
X	Q	Q				Q

FIGURE 5.26 Load balancing in Recovering Disk0 with EVENODD code. Note that 16 block accesses are required and number of blocks accessed per disk varies between 2 and 3.

Recovery in EVENODD minimizing the number of accesses is shown in Fig. 5.25 and a balanced version is shown in Fig. 5.26.

EVENODD requires $p(p-1)$ blocks to be read using parity blocks, while optimally $(p-1)(3p+1)/4$ blocks need be read, so that with large p there is approximately 25% reduction in block accesses.

$$\frac{(p-1)(3p+1)/4}{p(p-1)} = \frac{3}{4} + \frac{1}{4p}.$$

5.26 Blaum-Roth - BR code

The BR codes are a family of *Low Density Parity Codes - LDPC* codes (Gallagher, 2003) theoretically optimal in that regard for binary (XOR) codes (Blaum and Roth, 1999). These codes are combinatorial in nature and are horizontal codes with two parity strips. They are defined by two parameters p , a prime, and n , the number of data strips in the stripe with $n \leq p$.

To describe a code with a Generator Matrix, the packet size is set to one bit. Therefore each data and coding strip is a w -bit word and that their concatenation, which is labeled $(D|C)$ is a wn -bit word called the codeword. The Generator Matrix G has wk rows and wn columns and a specific format: $G = (I|H)$, where I is a $wk \times wk$ identity matrix. Encoding adheres to the following equation:

$$(D|C) = D.G = D.(I|H)$$

Specifying the matrix H is sufficient to specify the code. BR codes are best described by their systematic generator matrices:

$$\left| \begin{array}{c|c|c|c|c|c} I_{p-1} & 0_{p-1} & \dots & 0_{p-1} & I_{p-1} & I_{p-1} \\ 0_{p-1} & I_{p-1} & \dots & 0_{p-1} & I_{p-1} & Q_{p-1}^{(1)} \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\ 0_{p-1} & 0_{p-1} & \dots & I_{p-1} & I_{p-1} & Q_{p-1}^{(n-1)} \end{array} \right|$$

where I_{p-1} is a $(p-1)$ dimensional identity matrix, and 0_{p-1} is a $(p-1)$ dimensional all zero matrix and for $1 \leq j \leq n-1$ Defines is $Q_{p-1}^{(k)} = (a_{i,j}^{(k)})_{0 \leq i,j \leq p-2}$

$$a_{i,j}^{(k)} = \begin{cases} 1 & \text{if } j \neq p-1-k \text{ and } i-j = k \bmod p \\ 1 & \text{if } j = p-1-k \text{ and either } i-k-1 \text{ or } 2i = k-1 \bmod p \\ 0 & \end{cases}$$

For example for $p=3$ the generator matrix has the form (Hafner et al., 2004). The reader is referred to (Hafner et al., 2005) for a discussion of matrix methods for lost data reconstruction in erasure codes.

$$\left| \begin{array}{c|c|c|c|c|c} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{array} \right|$$

Generalized expanded Blaum-Roth codes and their efficient encoding/decoding are presented in (Wu et al., 2020).

5.27 X-code disk arrays and rebuild mode with one and two disk failures

X-code differs from other MDS arrays in that it places the parity strips horizontally in two rows rather than vertically. There is a similarity in that respect to the RM2 code (Park, 1995), which is not MDS.

Given a prime number of N disks data is organized as segments consisting of $N \times N$ strips. Rows or stripes are indexed as $0 : N-1$, rows $0 : N-3$ hold data strips, row $N-2$ holds P parities, and row $N-1$ Q parities. Each data block is protected by two parity groups, which are diagonals with ± 1 slopes.

Figs. 5.27 and 5.28 show the data layouts for $N = 7$. $B_{i,j}$ denotes a data block at the i^{th} row (stripe) and the j^{th} column (disk), which may be a strip. PG memberships are specified by the number for the corresponding $p(i)$ and $q(i)$ parities. The two parities are independent and given that $\langle X \rangle_N \equiv X \bmod N$ then $p(i)$ and $q(i)$ for $0 \leq i \leq N - 1$ are computed as follows:

$$p(i) = B_{N-2,i} = \sum_{k=0}^{N-3} B_{k,(i-k-2)_N}, \quad q(i) = B_{N-1,i} = \sum_{k=0}^{N-3} B_{k,(i+k+2)_N}.$$

Row	D_0	D_1	D_2	D_3	D_4	D_5	D_6
0	$B_{0,0}^2$	$B_{0,1}^3$	$B_{0,2}^4$	$B_{0,3}^5$	$B_{0,4}^6$	$B_{0,5}^0$	$B_{0,6}^1$
1	$B_{1,0}^3$	$B_{1,1}^4$	$B_{1,2}^5$	$B_{1,3}^6$	$B_{1,4}^0$	$B_{1,5}^1$	$B_{1,6}^2$
2	$B_{2,0}^4$	$B_{2,1}^5$	$B_{2,2}^6$	$B_{2,3}^0$	$B_{2,4}^1$	$B_{2,5}^2$	$B_{2,6}^3$
3	$B_{3,0}^5$	$B_{3,1}^6$	$B_{3,2}^0$	$B_{3,3}^1$	$B_{3,4}^2$	$B_{3,5}^3$	$B_{3,6}^4$
4	$B_{4,0}^6$	$B_{4,1}^0$	$B_{4,2}^1$	$B_{4,3}^2$	$B_{4,4}^3$	$B_{4,5}^4$	$B_{4,6}^5$
5	$B_{5,0}^0$	$B_{5,1}^1$	$B_{5,2}^2$	$B_{5,3}^3$	$B_{5,4}^4$	$B_{5,5}^5$	$B_{5,6}^6$
6							

FIGURE 5.27 Diagram showing parity groups for $p(i)$, $0 \leq i \leq 6$ at row 5 with slope $=+1$ and $N = 7$ disks. The P parities for blocks $B_{i,j}$, $0 \leq i, j \leq 4$ are given, $B_{5,j} = p(j)$, $0 \leq j \leq 6$.

Row	D_0	D_1	D_2	D_3	D_4	D_5	D_6
0	$B_{0,0}^5$	$B_{0,1}^6$	$B_{0,2}^0$	$B_{0,3}^1$	$B_{0,4}^2$	$B_{0,5}^3$	$B_{0,6}^4$
1	$B_{1,0}^4$	$B_{1,1}^5$	$B_{1,2}^6$	$B_{1,3}^0$	$B_{1,4}^1$	$B_{1,5}^2$	$B_{1,6}^3$
2	$B_{2,0}^3$	$B_{2,1}^4$	$B_{2,2}^5$	$B_{2,3}^6$	$B_{2,4}^0$	$B_{2,5}^1$	$B_{2,6}^2$
3	$B_{3,0}^2$	$B_{3,1}^3$	$B_{3,2}^4$	$B_{3,3}^5$	$B_{3,4}^6$	$B_{3,5}^0$	$B_{3,6}^1$
4	$B_{4,0}^1$	$B_{4,1}^2$	$B_{4,2}^3$	$B_{4,3}^4$	$B_{4,4}^5$	$B_{4,5}^6$	$B_{4,6}^0$
5							
6	$B_{6,0}^0$	$B_{6,1}^1$	$B_{6,2}^2$	$B_{6,3}^3$	$B_{6,4}^4$	$B_{6,5}^5$	$B_{6,6}^6$

FIGURE 5.28 Diagram showing parity group for $q(i)$, $0 \leq i \leq 6$ at row 6 with slope $=-1$ and $N = 7$ disks. The Q parities for blocks $B_{i,j}$, $0 \leq i, j \leq 4$ are given, $B_{6,j} = q(j)$.

Note that the *Parity Group* - PG size is $N - 1 = 6$ blocks for both parities and either PG can be accessed to reconstruct a block, so that the load increase in degraded mode is the same as RAID6.

The load increase in the case of x-code with two disk failures exceeds RAID6 with RS coding and EVENODD and RDP with small symbol sizes (Thomasian and Xu, 2011b).

Table 5.4 provides the recovery path and the read cost for all unavailable data blocks with $N = 7$ and distance $d = 1$, i.e., two adjacent failed disks: D_0 and D_1 . Given that blocks reconstructed in a previous step are held in a buffer and need not be re-accessed from disk, the number of accesses required to reconstruct the blocks on D_0 are: 5, 9, 13, 17, and 21, whose sum is 65, i.e., 13 accesses per disk on the average. The extra load per segment at surviving disk for both failed disks is 26. Given the five accesses per disk on its own blocks, the access cost on surviving disks is 31. There is 6.2-fold disk load increase and the disk overload is $26/5 = 5.2$.

It seems that the access cost will decrease with d , but this is not so in the considered examples. To avoid the worst case the $N \times N$ segments may be rotated to achieve the average cost over all distances.

Motivated by the discussion in (Wang et al., 2010) and (Xiang et al., 2010, 2011) an improvement in the number of accessed blocks to reconstruct blocks on a single X-code disk is considered in (Thomasian and Xu, 2011b). Note the scheme to save block accesses is applicable to x-code arrays, but not EVENODD and RDP.

Assuming that disk D_0 has failed, rather than reconstructing all blocks using P or Q parities, number of accesses can be saved reconstructing 4 blocks using P and the other 3 blocks using Q parities.

$B_{0,0}$ using $p(2)$: $B_{5,1}$, $B_{4,2}$, $B_{3,3}$, $B_{2,4}$, $B_{1,5}$

$B_{1,0}$ using $p(3)$: $B_{5,3}$, $B_{4,4}$, $B_{3,5}$, $B_{2,6}$, $B_{0,1}$

TABLE 5.4 Read cost for failed disks 0 and 1 with $N = 7$ and $d = 1$.

Block	Recovery path	Cost
(0,0)	p(2)	5
(1,0)	q(6) + p(3)	9
(2,0)	p(2) + q(5) + p(4)	13
(3,0)	q(6) + p(3) + q(4) + p(5)	17
(4,0)	p(2) + q(5) + p(4) + q(3) + p(6)	21
(0,1)	q(6)	5
(1,1)	p(2) + q(5)	9
(2,1)	q(6) + p(3) + q(4)	13
(3,1)	p(2) + q(5) + p(4) + q(3)	17
(4,1)	q(6) + p(3) + q(4) + p(5) + q(2)	21

$B_{2,0}$ using $p(4)$: $B_{5,4}, B_{4,5}, B_{3,6}^4, B_{1,1}, B_{0,2}^5$

$B_{5,0}$ using $p(0)$: $B_{4,1}^6, (B_{3,2}, B_{2,3}, B_{1,4}^7, B_{0,5})$

$B_{3,0}$ using $q(2)$: $B_{4,1}^{6'}, B_{6,2}, B_{0,4}, B_{1,5}^{1'}, B_{(2,6)}^{3'}$

$B_{4,0}$ using $q(1)$: $B_{6,1}, B_{0,3}, B_{1,4}^{7'}, B_{2,5}, B_{3,6}^{4'}$

$B_{6,0}$ using $q(0)$: $B_{0,2}^{5'}, B_{1,3}, B_{2,4}, B_{3,5}^{2'}, B_{4,6}$

Repeated accesses to the same strip are specified by unprimed superscripts in a first occurrence and a primed subscript in a second occurrence. Rather than $N \times (N - 2) = 35$ accesses, **28** blocks are accessed, provided seven blocks are cached after they are accessed, which amounts to a 20% decrease in the number of accessed blocks. Each one of seven blocks is reused only once. Appropriate ordering of block reconstructions can be used to minimize cache requirements. For example, the reconstruction of $B_{1,0}$ and $B_{2,0}$ may be used to fetch $B_{3,5}$ and $B_{0,2}$, which can be used in reconstructing $B_{6,0}$. The space allocated to these strips can be released after $B_{6,0}$ is reconstructed.

With two failed disks a cache of size $N(N - 2)$ blocks may be used to minimize rebuild cost on a per segment basis.

A lower bound of X-code single disk recovery for one disk failure is given by Theorem 1 in (Xu et al., 2014c):

$$A_{min} = \frac{1}{4}(3N^2 - 8N + 13),$$

which for $p = N = 7$ yields $A_{min} = (147 - 56 + 13)/4 = \mathbf{26}$, while without the optimization $N(N - 2) = 35$ accesses would be required to access the data blocks. The algorithm *Minimum Disk Read Recovery - MDRR* is presented. to achieve this.

Counting blocks might not be a good measure of disk access time, since multiple consecutive blocks accessed sequentially can be accessed efficiently.

5.27.1 Minimizing reconstruction overhead in MDS RAID

An erasure coded storage system is partitioned into stripes, which are maximal sets of disk blocks that are independent in terms of redundancy relations. Each block is partitioned into a fixed number of elements: w , which are fixed-size units of data or parity information. Most of existing works still take the stripe-level recovery as their only consideration, but a considerable performance improvement is possible by considering a group of rotated stripes. To seize this potential improvement, in this paper problem of single failure recovery in the stack-level is systematically studied.

Two recovery mechanisms based on greedy algorithm to seek for the near-optimal solution: Balanced Priority - BP-Scheme and Search Time Priority - STP-Scheme are considered in (Fu et al., 2016). The results in Table 5.5 show that BP-Scheme gains 3.4% to 38.9% (the average is 21.2%) higher recovery speed than Khan's Scheme and 3.4% to 34.8% (the average is 19.1%) higher recovery speed than Luo's U-Scheme, while STP-Scheme owns 3.4% to 46.9% (the average is 25.15%) and 3.4% to 41.1% (the average is 22.3%) higher recovery speed than Khan's Scheme (Khan et al., 2012) and Luo's U-Scheme (Luo and Shu, 2013), respectively.

TABLE 5.5 The time overhead of different recovery schemes over different erasure codes.

Erasure code	k	w	Khan's scheme	Luo's U-scheme	BP-scheme	STP-scheme
Blaum-Roth ($m = 2$)	14	16	9 s	11 s	12 s	4 s
RDP ($m = 2$)	14	16	42 s	66 s	44 s	4 s
EVENODD ($m = 2$)	14	16	682 s	653 s	682	5 s
Liberation ($m = 2$)	14	17	16 s	31 s	18 s	3 s
STAR ($m = 3$)	13	16	257 s	379 s	250 s	5 s
Generalized RDP ($m = 3$)	13	12	3060 s	6230 s	3069 s	5 s
Cauchy-RS ($m = 4$)	16	10	>24 h	24 h	23 h	15 s
Cauchy-RS ($m = 5$)	15	10	>24 H	>24 h	>24 h	15 s

5.27.2 Binary MDS array codes with optimal repair

The abstract for (Hou and Lee, 2020) is as follows:

“Consider a binary maximum distance separable (MDS) array code composed of an $m \times (k + r)$ array of bits with k information columns and r parity columns, such that any k out of $k + r$ columns suffice to reconstruct the k information columns. Our goal is to provide optimal repair access for binary MDS array codes, meaning that the bandwidth triggered to repair any single failed information or parity column is minimized. In this paper, we propose a generic transformation framework for binary MDS array codes, using EVENODD codes as a motivating example, to support optimal repair access for $k + 1 \leq d \leq k + r - 1$, where d denotes the number of nonfailed columns that are connected for repair; note that when $d < k + r - 1$, some of the chosen d columns in repairing a failed column are specific. In addition, we show how our transformation framework applies to an example of binary MDS array codes with asymptotically optimal repair access of any single information column and enables asymptotically or exactly optimal repair access for any column. Furthermore, we present a new transformation for EVENODD codes with two parity columns such that the existing efficient repair property of any information column is preserved and the repair access of parity column is optimal.”

5.28 The RM2 disk array

Chan-Ik Park (PosTech U., S. Korea) had a visiting position at IBM TJWRC when he published (Park, 1995).

<http://cpmi.postech.ac.kr/welcome/>

He also submitted a patent with his hosts on an efficient method for providing fault tolerance against double device failures (Baylor et al., 1999), which is discussed next.

An $N \times N$ Redundancy Matrix - RM is used to define the data layout in RM2 has elements $RM_{i,j}$ given in Table 5.6.

- $RM_{i,j} = -1$: A parity unit of disk j belongs to the parity group i .
- $RM_{i,j} = k$, $1 \leq k \leq M - 1$ Indicates that the k -th data unit of disk j belongs to parity group i .
- $RM_{i,j} = 0$.

TABLE 5.6 Redundancy Matrix for RM2.

$$\begin{pmatrix} -1 & 0 & 0 & 0 & 2 & 1 & 1 \\ 2 & -1 & 0 & 0 & 0 & 2 & 1 \\ 1 & 2 & -1 & 0 & 0 & 2 & 1 \\ 1 & 1 & 2 & -1 & 0 & 0 & 2 \\ 2 & 1 & 1 & 2 & -1 & 0 & 0 \\ 0 & 2 & 1 & 1 & 2 & -1 & 0 \\ 0 & 0 & 2 & 1 & 1 & 2 & -1 \end{pmatrix}$$

RM2 is a 2DFT array with horizontal parities with a redundancy ratio p , which is achieved by dedicating one row or stripe out of $M = 1/p$ rows in a disk array to parity strips (Park, 1995). According to Theorem 2 in (Park, 1995) the number of disks in RM2 disk arrays should satisfy: $N \geq 3M - 2$. For $p = 1/3$ or $M = 3$ the smallest number satisfying the

inequality is $N = 7$ and for $p = 1/7$ or $M = 7$ the smallest number is $N = 19$. It is easy to see that RM2 with $N = 7$ has a higher redundancy ratio than RAID6, since $1/3 > 2/7$.

Coding is applied to M rows at a time, which are referred here as segments. There are two parity groups associated with each parity block, each consisting of $2(M - 1)$ data blocks plus the parity block. For example, in Fig. 5.29 which gives the data layout for $N = 7$ and $M = 3$ parity strip P_0 protects a parity group consisting of four strips: $(D_{0,6}, D_{0,1}, D_{0,4}, D_{0,3})$, with the first two in row 2 and the other two in row 3, and P_1 protects $(D_{0,1}, D_{1,2}, D_{1,4}, D_{1,5})$, etc.

D_0	D_1	D_2	D_3	D_4	D_5	D_6
P_0	P_1	P_2	P_3	P_4	P_5	P_6
$D_{2,3}$	$D_{3,4}$	$D_{4,5}$	$D_{5,6}$	$D_{6,0}$	$D_{0,1}$	$D_{1,2}$
$D_{1,4}$	$D_{2,5}$	$D_{3,6}$	$D_{4,0}$	$D_{5,1}$	$D_{6,2}$	$D_{0,3}$

FIGURE 5.29 RM2 Layout with $M = 3$ and $N = 7$. The seven disks are specified as $D_i, 0 \leq i \leq 6$. The parity strips as $P_i, 0 \leq i \leq 6$. Data strips $D_{i,j}, 1 \leq i, j \leq 6$ are protected by two parity strips P_i and P_j at different disks from each other and $D_{i,j}$.

A data block on a failed disk can be reconstructed by accessing the remaining blocks in either of its two parity groups. For two disk failures we need to build a recovery path. To access a block $d_{2,3}$ in strip $D_{2,3}$ assuming that D_0 and D_2 has failed we proceed as follows utilizing the parity group associated with p_2 . $d_{2,3} = p_2 \oplus d_{2,5} \oplus d_{2,6} \oplus d_{1,2}$. If D_3 fails instead of D_2 then $d_{2,3}$ on D_0 cannot be reconstructed using p_3 directly, since $d_{3,6}$ is not available. $d_{3,6}$ can be reconstructed using p_6 , so we have the following two steps:

- (1) $d_{3,6} = p_6 \oplus d_{5,6} \oplus d_{0,6} \oplus d_{6,2}$.
- (2) $d_{2,3} = p_3 \oplus d_{3,4} \oplus d_{3,6} \oplus d_{0,3}$.

Therefore, the recovery path for block $d_{2,3}$ has a length of two. With disks D_0 and D_1 failed, the path to reconstruct $d_{1,4}$ has a length of three:

$$d_{2,5} \rightarrow d_{2,3} \rightarrow d_{3,4} \rightarrow d_{1,4}.$$

Figure 7 in (Thomasian et al., 2007a) plots disk utilizations with one disk failure for a workload consisting of uniform disk accesses to small blocks. It is observed that disk loads are unbalanced, but load balance can be achieved by rotating consecutive disk segments.

Figure 8(a) and Figure 8(b) show the load imbalance with two disk failures with $(D_0$ and $D_1)$ and $(D_0$ and $D_9)$ failed. The load increase is higher when failed disks are adjacent. The load imbalance can be avoided by rotating successive segments.

The Balanced RM2 improved data placement proposed by Chan-Ik Park et al. in (Kim et al., 2004) achieves a more balanced load when disk failures occur and a 5-20% improvement in performance when two disk failures occur with respect to an unbalanced RM2 as shown in Fig. 5.30.

D_0	D_1	D_2	D_3	D_4	D_5	D_6
P_0	P_1	P_2	P_3	P_4	P_5	P_6
$D_{1,6}$	$D_{2,0}$	$D_{3,1}$	$D_{4,2}$	$D_{5,3}$	$D_{6,4}$	$D_{0,5}$
$D_{2,5}$	$D_{3,6}$	$D_{4,0}$	$D_{5,1}$	$D_{6,2}$	$D_{0,3}$	$D_{1,4}$

FIGURE 5.30 Balanced RM2 Layout with $M = 3$ and $N = 7$. The seven disks are specified as $D_i, 0 \leq i \leq 6$ and their parity strips as $P_i, 0 \leq i \leq 6$. Data strips $D_{i,j}, 1 \leq i, j \leq 6$ are protected by two parity strips P_i and P_j at different disks from each other and $D_{i,j}$.

According to Table 1 in (Kim et al., 2004) the maximum load imbalance with distance 1 and 2 between failed disks is 4 and 2 with the original RM2 and 2 and 1 with the balanced RM2. The load imbalance is one in all other cases.

Described in US Patent (Baylor et al., 1999) is a methodology to construct vertical codes. Besides the strip size, the codes have only one defining parameter, an even number N for the number of disks (columns). The number of rows is always $r = N/2$, thus the storage redundancy is $100/r\%$. A construction for $N = 8$ with parity elements in the last two rows motivated by this patents is given in (Hafner et al., 2004). Each data element touches two parity elements in the last row. On the left we show the mapping from each data element into its first parity element by labeling the data element by the lower case letter of its parity element label; in the right we do the same for the second parity element as shown in Fig. 5.31:

S0	S1	S2	S3	S4	S5	S6	S7	S0	S1	S2	S3	S4	S5	S6	S7
x	w	v	x	v	t	u	u	y	y	y	z	x	v	x	w
v	v	t	t	u	s	w	t	w	z	w	y	y	w	z	x
t	s	s	s	t	u	s	s	u	x	z	u	z	z	v	y
S	T	U	V	W	X	Y	Z	S	T	U	V	W	X	Y	Z

FIGURE 5.31 Code proposed in (Baylor et al., 1999) where each strip is covered by two horizontally placed strips.

5.29 RAID7

RAID7 arrays with three check strips per stripe using RS or specialized parity codes are expected to gain popularity according to (Leventhal, 2009). A different RAID7 was trademarked by Storage Computer Corp. based on RAID3 and RAID4 and additional proprietary hardware.

EVENODD was extended to tolerate three disk failures in (Blaum et al., 2001). There is also the STAR coding scheme developed by Cheng Huang (Microsoft), Lihao Xu, Jiangqiang Luo (Wayne State U.) in (Huang and Xu, 2005), (Luo et al., 2010). *RAID Triple Parity - RTP* is an extension of RDP (Corbett et al., 2004), extended to tolerate two disk failures in (Blaum, 2006) and in (Goel and Corbett, 2012).

The STAR code as shown in Fig. 5.32 is an MDS code, which extends the EVENODD code with a third column (Huang and Xu, 2005). A decoding scheme to tolerate simultaneous disk failures and silent errors is described in (Luo et al., 2010). Erasure code consists of $k \leq p$ data disks and $m = 3$ parity disks. There are $r = p - 1$ rows.

Disk0	Disk1	Disk2	Disk3	Disk4	Disk5	Disk6	Disk7
A	E	D	C	B			A
B	A	E	D	C			B
C	B	A	E	D			C
D	C	B	A	E			D
E	D	C	B	A			E

FIGURE 5.32 STAR code generating parity column III (the last column) STAR with $p = 5$ data disks and $r = p - 1 = 4$ rows. The symbols in first four rows are used to compute the check blocks in the last column. Last row is imaginary.

The symbol $a_{p-1, p+2}$ is the adjuster $S_2 = \bigoplus_{j=0}^{p-1} a_{(j-1)p, j}$. The symbols in the last column are computed as follows:

$$a_{i, p+2} = S_2 \oplus \left(\bigoplus_{j=0}^{p-1} a_{(i+j)p, j} \right)$$

The decoding complexity of the STAR code is much lower than those of comparable codes; and this is meaningful for storage systems that need higher reliability. Given in Figure 8 in (Huang and Xu, 2005) is the throughput performance for the decoding procedure with $r = n - k$ erasures. EVENODD achieves twice the throughput of the RS code for $r = 2$ and STAR attains twice of the throughput of RS code for $r = 3$.

The study also compares practical performance of three open-source or public domain erasure coding libraries, namely Jerasure and Intel's ISA-L for RS code, and a STAR code implementation. The goal of this paper is to provide data storage practitioner a guideline when they choose a proper erasure code for storage applications and systems that need high performance in encoding and decoding operations in the order of GBs/s. Additionally (Chen and Xu, 2020) identifies a practical technique that can further improve decoding performance of RS code greatly for both Jerasure and ISA-L for the most frequent disk failure pattern, i.e., one disk failure.

5.30 Erasure coding for distributed storage

In what follows we follow the survey of erasure coding in distributed storage by researchers at *Indian Institute of Science - IIS* at Bangalore (Balaji et al., 2018). Several practical codes are discussed in two categories: *Locally Recoverable Codes - LRC* and *Regenerating Codes - RGC*.

The importance of dealing with failures in cloud systems is illustrated in Table 5.7. In some cases it involves loss of data, as in the case of Google gmail, which was not a hardware failure and could not be fixed using the methods discussed in this section.

TABLE 5.7 Examples of transient failures in different cloud services.

Cloud service	Failure reason	Duration	Date
Google Gmail	Software bug	4 days	Feb 27-Mar 2, 2011
Google Search	Programming error	40 minutes	Jan 31, 2009
Amazon S3	Gossip protocol blowup	6-8 hours	July 20, 2008
Microsoft Azure	Malfunction in Windows Azure	22 hours	Mar 13-14, 2008

5.30.1 Pyramid codes

Basic and generalized pyramid codes introduced at Microsoft Research are described in (Huang et al., 2007). Pyramid codes have a higher overhead than erasures codes, but provide a much lower recovery cost, which can be deduced from Fig. 5.33. Microsoft's Azure is an *Local Reconstruction Code - LRC* which tolerates all three disk failures, but the repair of as many as four failed disks is possible according to (Huang et al., 2012). Consider twelve data blocks subdivided into two groups of six blocks with local parities and two global parities.

d_1	d_2	d_3	d_4	d_5	d_6	$c_{1,1}$
d_7	d_8	d_9	d_{10}	d_{11}	d_{12}	$c_{1,2}$
c_2	c_3	c_4				

FIGURE 5.33 A pyramid code with 12 data and 5 check disks. Note $c_{1,1}$ and $c_{1,2}$ provide parity protection for first and second half of the disks in the array.

A smaller example is provided here to save space.

$$(x_0, x_1, x_2), p_x, (y_0, y_1, y_2), p_y, c_0, c_1)$$

Consider the failure of x_0 , x_2 , and y_1 . Repair proceeds by repairing y_1 using p_y and x_0 and x_2 can then be repaired using c_0 and c_1 . Recovery with four disk failures is possible in 86% of cases, but is quite complex.

LRC can tolerate arbitrary 3 failures by choosing the following α 's and β 's for the smaller example.

$$\begin{aligned} q_{x,0} &= \alpha_0 x_0 + \alpha_1 x_1 + \alpha_2 x_2, \\ q_{x,1} &= \alpha_0^2 x_0 + \alpha_1^2 x_1 + \alpha_2^2 x_2, \\ q_{x,2} &= x_0 + x_1 + x_2 \\ q_{y,0} &= \beta_0 y_0 + \beta_1 y_1 + \beta_2 y_2, \\ q_{y,1} &= \beta_0^2 y_0 + \beta_1^2 y_1 + \beta_2^2 y_2, \\ q_{y,2} &= y_0 + y_1 + y_2 \end{aligned}$$

The LRC equations are as follows:

$$p_0 = q_{x,0} + q_{y,0}, \quad p_1 = q_{x,1} + q_{y,1}, \quad p_x = q_{x,2} p_y = q_{y,2}$$

The values of α s and β s are selected, so that LRC can decide all information-theoretically decodable four failures.

1. None of four parities fails: The failures are equally divided between the two groups. We have the following matrix.

$$G = \begin{pmatrix} 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ \alpha_i & \alpha_j & \beta_s & \beta_t \\ \alpha_i^2 & \alpha_j^2 & \beta_s^2 & \beta_t^2 \end{pmatrix} \quad \text{Det}(G) = (\alpha_j - \alpha_i)(\beta_t - \beta_s)(\alpha_i + \alpha_j - \beta_s - \beta_t)$$

2. Only one of p_x and p_y fails: Assume p_y fails. The remaining three failures are two in group x and one in group y . We have three equations with coefficients given by:

$$G' = \begin{pmatrix} 1 & 1 & 0 \\ \alpha_i & \alpha_j & \beta_s \\ \alpha_i^2 & \alpha_j^2 & \beta_s^2 \end{pmatrix} \quad \text{Det}(G') = \beta_s(\alpha_j - \alpha_i)(\beta_s - \alpha_j - \alpha_i)$$

3. Both p_x and p_y fail. The remaining two failures are divided between the two groups.

$$G'' = \begin{pmatrix} \alpha_i & \beta_s \\ \alpha_i^2 & \beta_s^2 \end{pmatrix} \quad \text{Det}(G'') = \alpha_i \beta_s (\beta_s - \alpha_i)$$

To ensure all cases are decodable all three matrices should be non-singular.

The repair savings of an $(n = 16, k = 12, r = 6)$ LR code with that of an $[n = 16, k = 12]$ RS code in the Azure production cluster are demonstrated in (Huang et al., 2012). The $(n = 18, k = 14, r = 7)$ LR code in Windows Azure Storage has repair degree comparable to that of an $[9, 6]$ RS code, but its storage overhead is 1.29 versus 1.5 for the RS code. This code has reportedly resulted in the savings of millions of dollars for Microsoft.

<https://www.microsoft.com/en-us/research/blog/better-way-store-data/>

The research in network codes for distributed storage systems in this area has been led by Alexandros Dimakis (U. Southern Calif. - USC, U. Texas at Austin), collaborating with Jehoshua (Shuki) Bruck at CalTech (Dimakis et al., 2007), (Dimakis et al., 2010). A survey of network codes for distributed storage is given in (Dimakis et al., 2011).

A $(4, 2)$ MDS code is exemplified by an EVENODD code shown in Fig. 5.34, which can tolerate any two disk failures.

Disk1	Disk2	Disk3	Disk4
A_1	B_1	$A_1 + B_1$	$A_2 + B_1$
A_2	B_2	$A_2 + B_2$	$A_1 + A_2 + B_2$

FIGURE 5.34 An MDS $(4, 2)$ code which can tolerate two disk failures.

Assuming both data disks (Disk1 and Disk2) are broken their contents can be reproduced using the blocks in Disk3 and Disk4 as follows:

$$(A_2 + B_2) + (A_1 + A_2 + B_2) \rightarrow A_1$$

$$A_1 + (A_1 + B_1) \rightarrow B_1$$

$$(A_1 + B_1) + (A_2 + B_1) \rightarrow A_1 + A_2$$

$$(A_1 + A_2) + (A_1 + A_2 + B_2) \rightarrow B_2$$

$$B_2 + (A_2 + B_2) \rightarrow A_2$$

A formal approach on reconstructing data disks is given in Chapter 6.

5.30.2 HDFS-Xorbas

HDFS Xorbas was implemented by Alexandros Dimakis (USC, U. Texas at Austin) et al. for Facebook. This is an LRC - Local Recovery Code build on top of an RS code by adding extra local parities as described in (Sathiamoorthy et al., 2013). The experimental evaluation of Xorbas was carried out in Amazon EC2 and a Facebook cluster, in which the repair performance of $(n = 16, k = 10, r = 5)$ LR code was compared against a $[14, 10]$ RS code. Xorbas reduced disk I/O and repair network traffic compared to RS codes. In addition to the four RS parity blocks associated with ten file blocks, LRC associates local parities as shown below:

$$S_1 = c_1 X_1 + c_2 X_2 + c_3 X_3 + c_4 X_4 + c_5 X_5$$

$$S_2 = c_6 X_6 + c_7 X_7 + c_8 X_8 + c_9 X_9 + c_{10} X_{10}$$

$$S_3 = c'_5 S_1 + c'_6 S_2 \text{ with } c'_5 = c'_6 = 1.$$

With three local parities the storage overhead is $17/10$, but S_3 need not be stored by choosing $c_i, 1 \leq i \leq 10$ satisfy the additional alignment equation $S_1 + S_2 + S_3 = 0$. It is shown that this code has the largest possible distance ($d = 5$) for this given locality ($r = 5$) and blocklength ($n = 16$). This is an LRC $(10, 6, 5)$ code.

The reliability analysis of the system with the birth-death model is given in (Ford et al., 2010). Chunks fail independently with rate λ_i and recover with rate ρ_i . The failure transition rates are $S_i \rightarrow S_{i+1}: \lambda_i, 0 \leq i \leq 4$ and repair transition rates $S_i \leftarrow S_{i+1}: \rho_i, 0 \leq i \leq 3$. Starting with state s and given that the system can tolerate r failures we are interested in the time to failure (the time to reach state $r - 1$),

$$\text{MTTF} = \frac{1}{\lambda} \left(\sum_{k=0}^{s-r} \sum_{i=0}^k \left(\frac{\rho^i}{\lambda^i} \right) \frac{1}{(s-k+i)_{i=1}} \right) \quad (5.20)$$

where $(a)_{(b)} \stackrel{\text{def}}{=} a(a-1)(a-2)\dots(a-b+1)$. Since recovery takes much less time than time to failure ($\rho \gg \lambda$) the MTTF can be approximated as:

$$MTTF \approx \frac{\rho^{s-r}}{\lambda^{s-r+1}} \frac{1}{(s)_{(s-r+1)}} + O\left(\frac{\rho^{s-r-1}}{\lambda^{s-r}}\right)$$

A comparison of three redundancy methods based on (Sathiamoorthy et al., 2013) is given in Table 5.8.

TABLE 5.8 Comparison of three redundancy schemes.

Scheme	Storage overhead	Repair traffic	MTTDL (days)
3-replication	2x	1x	2.31E+10
RS(10,4)	0.4x	10x	3.31E+13
LRC(10,6,5)	0.6x	5x	1.21E+15

5.30.3 Ceph erasure coding with LR code

A second distributed storage system that has an LRC plug-in is Ceph (Weil et al., 2006a).

<http://docs.ceph.com/docs/master/rados/operations/erasure-code-lrc/>

5.30.4 Hitchhiker erasure code

The Hitchhiker erasure code by K. V. Lakshmi (UCB, CMU), Kannan Ramchandran et al. at UCB (Rashmi et al., 2014) is used to implement HJDFS coding in (Rashmi et al., 2017), where it was used to evaluate its performance on a cluster at Facebook. The Hitchhiker code has now been incorporated into Apache Hadoop.

5.30.5 Hadoop Adaptively-Coded Distributed File System

Hadoop Adaptively-Coded Distributed File System - HACFS investigated in Xia et al. (2015) by Mingyuan Xia (McGill U.), Mohit Saxena, Mario Blaum, David Pease at IBM/ARC.

HACFS uses two codes: a fast code with low recovery cost and a compact code with low storage overhead. It exploits the data access skew observed in Hadoop workloads to decide on the initial encoding of data blocks. HACFS uses the fast code to encode a small fraction of the frequently accessed data and provides overall low recovery cost for the system. The compact code to encode the majority of less frequently accessed data blocks and maintain a low and bounded storage overhead.

After an initial encoding, e.g., with the compact code, HACFS dynamically adapts to workload changes by using *upcoding* and *downcoding* to convert data blocks between the fast and compact codes. Upcoding blocks initially encoded with fast code into compact code enabling the HACFS system to reduce the storage overhead. Similarly, downcoding data blocks from compact code to fast code representation lowers the overall recovery cost of the HACFS system. The upcode and downcode operations are efficient and only update the associated parity blocks, while converting blocks between the two codes as shown in Fig. 5.35.

Fewer data accesses are required for rebuild strips with 2×5 encoding, i.e., two strips accesses to rebuild a strip.

5.30.6 HashTag Erasure Code - HTEC

Minimum Storage Regenerating - MSR is a viable alternative to RS codes. The HDFS implementation of HTEC is discussed in (Kralevska et al., 2018). MSR codes allow low sub-packetization levels at the expense of increased repair bandwidth and are designed for efficient repair. MSR codes have emerged as a viable alternative to RS codes as they minimize the repair bandwidth, while they are still optimal in terms of reliability and storage overhead. HTEC codes are simultaneously optimized for storage, reliability, I/O operations, and repair-bandwidth for single and multiple failures of systematic nodes.

HTEC provides the lowest data-read and data-transfer rates, and thus the lowest repair time for an arbitrary sub-packetization level α among all existing MDS codes for distributed storage including MSR codes. The repair process is linear and highly parallel. Additionally, it is shown that HTECs are the first high-rate MDS codes that reduce the repair bandwidth for more than one failure. A summary of notation: $n = k + r$ is the number of nodes.

D	D	D	D	D	P
D	D	D	D	D	P
P	P	P	P	P	P

D	D	D	D	D	P
D	D	D	D	D	P
P	P	P	P	P	P

D	D	D	D	D	P
D	D	D	D	D	P
P	P	P	P	P	P

D	D	D	D	D	P
D	D	D	D	D	P
D	D	D	D	D	P
D	D	D	D	P	P
D	D	D	D	P	P
P	P	P	P	P	P

FIGURE 5.35 Parity encodings to attain reliability in Hadoop. Upcoding (2x5) arrays to 6x5 results in a reduced redundancy from $18/10 = 1.8$ to $42/30 = 1.4$. Horizontal parity codes are simply copied, while the vertical parity bits are the XOR of the parities in the high density code.

k number of systematic nodes.

r number of parity nodes.

d number of nonfailed nodes.

t number of failed nodes $1 \leq t \leq r$.

α the sub-packetization level.

An example of an RS code is $k = 6$ and $r = 3$, so that $n = 9$. If each node stores 9 MB then to reconstruct a failed disk six nodes need to be accessed, so that $6 \times 9 = 54$ MB need to be transferred and XORed to reconstruct a block.

A (9,6) access-optimal HTEC with $\alpha = 9$ requires 3 MB to be transferred from eight nodes, which is $8 \times 3 = 24$ MB. A (9,6) HTEC with $\alpha = 6$ requires the following transfers from $d_2, d_3, d_4, d_5, d_6, P_3$ (3 MB, 3 MB, 4.5 MB, 4.5 MB, 3 MB, 3 MB) with 27 MB total. A hashtag array is shown in Table 5.9.

TABLE 5.9 RS array code with six systematic and 3 parity nodes for $\alpha = 9$. The coefficients are from F_{32} with irreducible polynomial $x^5 + x^3 + 1$. Only the expressions for the first parity are given.

$$\begin{pmatrix} p_{1,1} \\ p_{2,1} \\ p_{3,1} \\ p_{4,1} \\ p_{5,1} \\ p_{6,1} \\ p_{7,1} \\ p_{8,1} \\ p_{9,1} \end{pmatrix} = \begin{pmatrix} 7a_{1,1} + 10a_{1,2} + 18a_{1,3} + 11a_{1,4} + 17a_{1,5} + 6a_{1,6} \\ 26a_{2,1} + 17a_{2,2} + 25a_{2,3} + 27a_{2,4} + 31a_{2,5} + 4a_{2,6} \\ 22a_{3,1} + 12a_{3,2} + 27a_{3,3} + 31a_{3,4} + 31a_{3,5} + 23a_{3,6} \\ 17a_{4,1} + 9a_{4,2} + 14a_{4,3} + 4a_{4,4} + 21a_{4,5} + 15a_{4,6} \\ 10a_{5,1} + 5a_{5,2} + 5a_{5,3} + 13a_{5,4} + 11a_{5,5} + 16a_{5,6} \\ 25a_{6,1} + 16a_{6,2} + 30a_{6,3} + 28a_{6,4} + 10a_{6,5} + 24a_{6,6} \\ 20a_{7,1} + 8a_{7,2} + 21a_{7,3} + 9a_{7,4} + 3a_{7,5} + 25a_{7,6} \\ 23a_{8,1} + 4a_{8,2} + 12a_{8,3} + 16a_{8,4} + 8a_{8,5} + 17a_{8,6} \\ 2a_{9,1} + 21a_{9,2} + 8a_{9,3} + 16a_{9,4} + 7a_{9,5} + 25a_{9,6} \end{pmatrix}$$

The repair process first reads a set of α/r rows from the first parity node and the non-failed systematic nodes and repairs only α/r elements from the failed systematic node. The essence of the algorithm is that the already read set is reused for repair of all subsequent elements.

According to proposition 2 in the paper the bandwidth for repair of a single systematic node is bounded as follows:

$$\frac{n-1}{r} \leq \gamma \leq \frac{n-1}{r} = \frac{r-1}{\alpha} \lceil \alpha r \rceil \lceil kr \rceil$$

5.31 ReGenerating codes

ReGenerating - RG codes were proposed to minimize the volume of data to repair a failed node, while *Locally Repairable* - LR codes minimize the number of helper nodes. The two principal classes of RG codes are *Minimum Bandwidth Regenerating* - MBR and *Minimum Storage Regeneration* - MSR codes.

A large number of MSR codes and their parameters are listed in Table 1 in (Balaji et al., 2018). For example, Zigzag codes are MDS codes with optimal rebuilding (Tamo et al., 2011), (Wang et al., 2011). The permutation code is an optimal exact-repair of a single failed node in distributed storage systems (Cadambe et al., 2011).

The NCCloud storage system is an RG code built on top of a 2-parity *Functional-MSR* - *F-MSR* code, whose performance is evaluated in (Hu et al., 2012), (Chen et al., 2014). Analysis and performance evaluation of lightweight cloud storage systems is reported in (Akintoye and Bagula, 2018).

Considered is a file of size M , which consists of two chunks A and B stored at nodes 1 and 2, respectively. In the case of RAID6 with RS coding nodes 3 and 4 hold A+B and A+2B, respectively. If node 1 fails its contents can be reconstructed by sending B and A+2B from node 2 and 3 to a proxy node, which can reconstruct A and send it to a new node, so that the total traffic is M .

To illustrate *Exact Minimum-Storage Regenerating* - *EMSR* codes consider a file of size M partitioned into four chunks A, B, C, and D. Nodes 1 and 2 hold A,B and C,D, while nodes 3 and 4 hold A+C, 2B+D and 2A+C, B+D. When node 1 fails nodes 2, 3, and 4 send the XOR of their two chunks to the proxy node, which receives C+D, A+2B+C+D, 2A+B+C+D.

These chunks can be used to reproduce A and B, which are then sent to the new node. Total traffic is $0.75M$.

For FMSR codes the file is divided to four chunks, and eight distinct code chunks P1, ..., P8 are formed by different linear combinations of the native chunks. Each code chunk has the same size $M/4$ as a native chunk and any two nodes can be used to recover the original four. If node 1 fails then nodes 2-4 send P3, P5, P7 to a proxy node which reconstructs P1' and P2', which are then sent to a new node. A key property of our FMSR codes is that unlike EMSR the nodes do not perform encoding during repair, while the traffic remains the same.

Consider the double-fault tolerant implementation of F-MSR shown in Fig. 5.36. F-MSR divides the file into four native chunks, and constructs eight distinct code chunks.

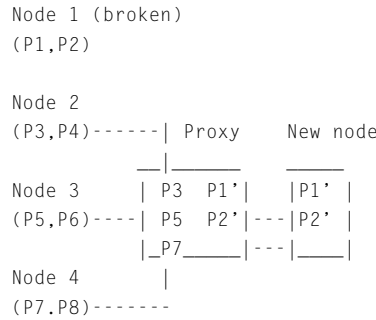


FIGURE 5.36 Functional MSR based on (Hu et al., 2012).

P1, ..., P8 are formed by different linear combinations of the native chunks. Each code chunk has the same size $M/4$ as a native chunk. Any two nodes can be used to recover the original four native chunks. When Node 1 is down, the proxy collects one code chunk from each surviving node, so it downloads three code chunks of size $M/4$ each. Then the proxy regenerates two code chunks P1 and P2 formed by different linear combinations of the three code chunks. Note that P1 and P2 are still linear combinations of the native chunks. The proxy then writes P1 and P2 to the new node. In F-MSR, the storage size is $2M$ (as in RAID-6), but the repair traffic is $0.75M$, which is a 25% saving.

To generalize F-MSR for n storage nodes, divide a file of size M into $2(n-2)$ native chunks and generate $4(n-2)$ code chunks. Then each node will store two code chunks of size $\frac{M}{2(n-2)}$ each. Thus, the total storage size is $\frac{Mn}{n-2}$. To repair a failed node download one chunk from each of $n-1$ nodes, so that the repair traffic is $\frac{M(n-1)}{2(n-2)}$. In contrast, for RAID6, the total storage size is $\frac{Mn}{n-2}$, while the repair traffic is M . When n is large, F-MSR can save the repair traffic by close to 50%. Note that F-MSR keeps only code chunks rather than native chunks. To access a single chunk of a file download and decode the entire file for the particular chunk. F-MSR is acceptable for long-term archival applications with low read frequency.

5.31.1 Pentagon and heptagon codes

The performance of the Pentagon code and Heptagon-code in a Hadoop setting is reported by IIS researchers in (Krishnan et al., 2014). These two codes possess inherent double replication of code symbols, have storage overhead slightly greater than two and their performance is compared against double and triple replication.

Given as an example is a mirrored RAID5 with nine data disks. This scheme adds moderately to the storage overhead of double replication, while simultaneously offering reliability levels similar to that of triple replication as shown in Fig. 5.37.

P is the parity over nine data blocks. The ten blocks are replicated in the following manner:

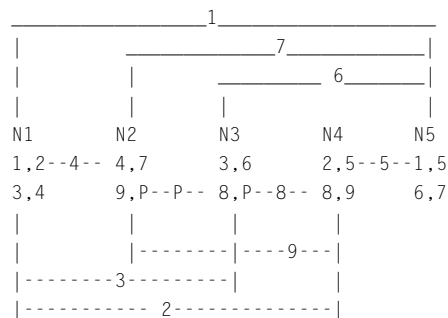


FIGURE 5.37 Pentagon codes.

In the case of two node failures, say N1 and N2, to recover block 4, nodes N3, N4, and N5 internally compute the partial parities

$$P3 = 3 + 6 + P,$$

$$P4 = 2 + 8 + 9,$$

$$P5 = 1 + 5 + 7$$

and transfer it to the replacement node. The replacement node computes $P3 + P4 + P5$ and recovers 4. The repair bandwidth to reconstruct the two failed nodes or repair bandwidth is ten blocks.

The pentagon can be extended to a heptagon with seven nodes and six blocks per node with two parities, hence (21,20). Overhead decreases since $\frac{42}{20} < \frac{20}{9}$, but resiliency is reduced $\frac{2}{7} < \frac{2}{5}$. To extra parities can be added to increase the resiliency of the Heptagon. Different schemes are compared in Table 5.10.

TABLE 5.10 Comparing storage overhead, code length, and MTDL of various coding schemes (for details refer to (Krishnan et al., 2014).

Code	Storage overhead	Code length	MTDL (years)
3-replication	3.00x	3	1.20e+09
pentagon	2.22x	5	1.05e+08
heptagon	2.10x	7	2.68e+07
heptagon-local	2.15x	15	8.34e+09
(10,9)RAID+m	2.22x	20	2.03e+09
(12,11)RAID+m	2.18x	24	6.50e+08

5.31.2 Minimum Storage Regeneration - MSR codes

RS codes provide significant savings in storage space with respect to replication, but impose a huge burden on the I/O and network resources. *Minimum-Storage-Regeneration - MSR* codes minimize network transfers during reconstruction, while also being optimal with respect to storage and reliability. MSR codes however do not address the I/O overhead incurred during reconstructions and are inferior to RS codes in this regard. An optimal-access version of the product-matrix MSR code, is referred to as the PM-RBT code in (Rashmi et al., 2015). Evaluation shows proposed design results in a significant reduction in the number of I/Os during reconstructions (5-fold reduction for typical parameters), while retaining optimality with respect to storage, reliability, and network bandwidth.

Beehive erasure codes are introduced on top of MSR codes in (Li and Li, 2015, 2017). These codes repair multiple failures simultaneously and are implemented using the Product-Matrix MSR in C++ using the *Intel Storage Acceleration Library - ISAL*.

<https://software.intel.com/content/www/us/en/develop/articles/optimizing-storage-solutions-using-the-intel-intelligent->

storage-acceleration-library.html

The authors consider $k = 3$ data nodes (D_1, D_2, D_3) and $r = 2$ redundant nodes (P_1, P_2) with RS coding. If the nodes have 128 MB and single data node can be reconstructed using three nodes, which requires the transfer of $3 \times 128 = 384$ MB. Using regenerating codes specified in (Dimakis et al., 2011) with $K = 3$, $r = 2$ and $D = 4$ helpers. Transfers from four nodes D_1, D_3, P_1, P_2 of 64m MB require $4 \times 64 = 256$ MB to be transferred.

MSR codes will incur even more disk I/O than RS codes, since each helper needs to read all its data to compute a small fraction sent out, so that $4 \times 128 = 512$ MB are read, instead of $3 \times 128 = 384$ MB required with RS codes.

There are erasure codes that save both network transfer and disk I/O during reconstruction. Coalescing reconstructions when multiple disk failures occur can save I/O. Consider the failure of D_1 and D_3 . Reconstruction requires $4 \times 64 = 256$ MB to be transferred.

Using the techniques specified in (Shum, 2011) ten 42.7 MB data transfers are required. 4×42.7 from D_2, P_1, P_2 to D_1 and the same number to D_2 , plus 42.7 MB from D_1 to D_2 and vice-versa, so that 427 MB. These code construction exists for a limited set of parameters.

Zvonimir Bandic et al. at *Western Digital Corp. - WDC* present the evaluation of a high-rate MSR code, known as the Butterfly code in both Ceph and HDFS in (Pamies-Juarez et al., 2016). This code is a simplified version of the MSR codes presented in (Gad et al., 2013) with two parity nodes.

More recently researchers at the IIS have proposed the Clay Code in (Vajha et al., 2018), which has been implemented over the Ceph file system

[https://en.wikipedia.org/wiki/Ceph_\(software\)](https://en.wikipedia.org/wiki/Ceph_(software))

and evaluated over an *Amazon Web Services - AWS* cluster.

5.32 Protection schemes for flash memories

Several schemes for protecting flash memories are discussed in this section.

5.32.1 Flash protection schemes

A high performance flash system resented by DSSD founder is (Bonwick, 2016).

<https://storageconference.us/2016/Slides/JeffBonwick.pdf>

DDSD was acquired for \$1 billion by EMC and subsequently shipped, but the product line was canceled a year later after Dell acquired EMC.

https://www.theregister.co.uk/2017/03/02/dell_cans_standalone_dssd

DSSD stores parities in two dimensions with m rows and n columns. Each chip may be considered as a strip or stripe unit in a RAID array. Given one parity per row and one parity per column all four chip errors can be tolerated as long as they do not constitute a rectangle. DSSD's flash memory layout is shown in Fig. 5.38. Other 2D configurations leading to data loss are discussed in more detail in Chapter 6. The discussion of *Hierarchical RAID - HRAID* coding in Chapter 11 is also pertinent to this discussion.

-	D0	D1	D2	D3	D4	D5	P6	Q7
D0	0,0	0,1	0,2	0,3	0,4	0,5	0,6	0,7
D1	1,0	1,1	1,2	1,3	1,4	1,5	1,6	1,7
D2	2,0	2,1	1,2	(2,3)	2,4	(2,5)	2,6	2,7
P3	3,0	3,1	3,2	3,3	3,4	3,5	3,6	3,7
P4	4,0	4,1	4,2	4,3	4,4	(4,5)	4,6	4,7

FIGURE 5.38 Flash memory layout with two horizontal and two vertical parities. Failed blocks are shown in parentheses. Reconstruct (2,3) using parity 3,3, reconstruct (2,7) using 3,7, reconstruct (2,5) using 2,6, reconstruct (0,5) using 0,6, and reconstruct (4,5) using 4,6.

5.32.2 Differential RAID for flash memories

Diff-RAID is an unconventional RAID array, whose design takes into account the fact that flash memories have very different failure characteristics from HDDs, i.e., the *Bit Error Rate - BER* of an SSD increases with more writes (Balakrishnan et al., 2010). By balancing writes evenly across the array, RAID schemes wear out devices at similar rates, making the RAID susceptible to data loss. Diff-RAID reshuffles the parity distribution on each drive replacement to maintain an age

differential when old devices are replaced by new ones, A simulator to evaluate Diff-RAID's reliability by using BERs from 12 flash chips showed that it is more reliable than RAID5 by orders of magnitude.

For *Multi-Level Cell* - MLC flash technology under consideration the number of erasures was limited to 5000-10000 cycles per block. Diff-RAID differs significantly from RAID5 arrays on HDD in that it attains a higher reliability by distributing parity blocks *unevenly*. Diff-RAID masks higher BERs on aging SSDs while:

- (1) Retaining the low overhead of RAID5.
- (2) Extending the lifetime of commodity SSDs.
- (3) Alleviating the need for expensive error correction hardware.

Load balancing can cause correlated failures, since with balanced writes all SSDs will be susceptible to failure when an SSD fails. Consider $n = 5$ SDDs with 80% of parity allocated on the first SDD and the rest of parities evenly distributed across remaining SDDs, so that we have the following configuration (80, 5, 5, 5, 5).

$$a_{i,j} = \frac{p_i(n-1) + (100 - p_i)}{p_j(n-1) + (100 - p_j)}$$

For example, for $n = 4$ and (70, 10, 10, 10) the aging rate of the first device is twice that of the others. For $n = 5$ and with (80, 5.5, 5, 5), after numerous replacements the ages of remaining devices at replacement time converge to (5750, 4312.5, 2875, 1437.5). The implementation of device replacement can be accomplished via rebuild processing, while maintaining the same parity layout. The reduction in data loss probability, as given by Figures 6 and 7 in the paper, is significant.

5.32.3 Partial MDS - PMDS coding for SSDs

The PMDS method developed at IBM ARC is described in (Blaum et al., 2013) and posted below.

<https://arxiv.org/pdf/1205.0997.pdf>

In SSD flash memories sector failures are more prevalent than whole SSD failures. The sector is the data storage unit protected by a cyclic *Error Correcting Code* - ECC such as *Bose, Chaudhuri, and Hocquenghem* - BCH code.

http://en.wikipedia.org/wiki/BCH_code

Applying RS codes as in RAID6 is not appropriate for two reasons:

- (i) too expensive to dedicate an extra SSD to parity to handle sectors errors;
- (ii) RAID6 cannot handle more than two erased errors.

PMDS similarly to the scheme in (Huang et al., 2012) relies on local and global parities, but is more complicated and does not require special consideration for parities being erased. Given an $m \times n$ array of sectors r erasures can be corrected in a horizontal row. As in the case of the EVENODD code $m = n - 1$ and that n should be prime, although this limitation can be easily circumvented (Blaum et al., 1995). If r columns are erased, an additional s global parities allow correcting s extra erasures. In the case $r = 1$ and $s = 2$ shown in Fig. 5.39, necessary and sufficient conditions for finding codes and tables of codes are given in the paper. The r local parities can efficiently deal with failed sectors in a row and the extra s parities are invoked in rare occasions. Given six check sectors as many failed sectors can be corrected, say four sectors in a column and two other failed sectors.

$a_{0,0}$	$a_{0,1}$	$a_{0,2}$	$a_{0,3}$	$a_{0,4}$
$a_{1,0}$	$a_{1,1}$	$a_{1,2}$	$a_{1,3}$	$a_{1,4}$
$a_{2,0}$	$a_{2,1}$	$a_{2,2}$	$a_{2,3}$	$a_{2,4}$
$a_{3,0}$	$a_{3,1}$	$a_{3,2}$	$a_{3,3}$	$a_{3,4}$

FIGURE 5.39 Layout of the local and global parities for Partial MDS with $r = 1$ and $s = 2$, The four local parities are $a_{i,4} = p_i$, $0 \leq i \leq 3$ and $a_{3,2} = g_1$ and $a_{3,3} = g_2$ are the global parities.

In Fig. 5.39 $a_{0,4}$, $a_{1,4}$, $a_{2,4}$, $a_{3,4}$ are the four local parities, and $a_{3,2}$ and $a_{3,3}$ placed somewhat arbitrarily are the global parities. To simplify the notation we superimpose a one dimensional vector b_i , $0 \leq i \leq 19$, which covers the blocks in one row at a time. The $m \times n$ codeword should satisfy the following six equations, with the first four for local parities and the last two for global parities.

$$S_j = b_{4j+0} \oplus b_{4j+1} \oplus b_{4j+2} \oplus b_{4j+3} = 0, \quad 0 \leq j \leq 3. \quad S_4 = \sum_{i=0}^{19} \alpha^i b_i \quad S_5 = \sum_{i=0}^{19} \alpha^{2i} b_i.$$

Given that disk two and two random blocks have failed, we designate them as $a_{0,2} = u$, $a_{1,2} = y$, $a_{2,2} = v$, $a_{3,2} = w$, and $a_{1,0} = x$, and $a_{1,3} = z$. The unknowns u , v , and w can be determined by using local parities: S_0 , S_2 , and S_3), while the following three equations can be used to determine x , y , and z .

$$x \oplus y \oplus z = S_1, \quad \alpha^5 x \oplus \alpha^7 y \alpha^8 z = S_4, \quad \alpha^{10} \oplus \alpha^{14} y \oplus \alpha^{16} z = S_5.$$

The set of linear equations has a unique solution provided the determinant of the matrix $\mathcal{A}$ defined below is not zero.

$$\mathcal{A} = \begin{pmatrix} 1 & 1 & 1 \\ \alpha^5 & \alpha^7 & \alpha^8 \\ \alpha^{10} & \alpha^{14} & \alpha^{16} \end{pmatrix}$$

$\mathcal{A}$ is a Vandermonde matrix since powers of α in its rows are different, so that $\det(\mathcal{A}) \neq 0$ allowing x , y , and z to be determined.

As a second example, consider two sector errors: $x = a_{1,0}$ and $y = a_{3,1}$ and a failed SSD $u = a_{0,2}$, $z = a_{1,2}$, $v = a_{2,2}$, $w = a_{3,2}$. The values of u and v can be obtained using S_0 and S_2 . We have the additional four equations:

$$x \oplus z = S_1, \quad y \oplus w = S_3, \quad \alpha^5 x + \alpha^7 z \oplus \alpha^{16} y \oplus \alpha^{17} w = S_4, \quad \alpha^{10} x + \alpha^{14} z + \alpha^{32} y + \alpha^{34} w = S_5$$

This system has a unique solution since the determinant is nonzero. A general condition is given by Theorem 4.1 for the necessary and sufficient conditions for the case of RAID5 (single global parity with $r = 1$) plus s global parities). In the special case of $s = 2$ Theorem 5.1 states that the code is PMDS depending on m , n , and the polynomial $f(x)$ defining the underlying field or ring. This polynomial $f(x)$ is not necessarily primitive, but has a certain exponent, which has to exceed $m \times n$. [http://en.wikipedia.org/wiki/Primitive_polynomial_\(field_theory\)](http://en.wikipedia.org/wiki/Primitive_polynomial_(field_theory))

For example, for $f(x) = M_p(x) = 1 + x + x^2 + \dots + x^{p-1}$, the polynomial has a prime exponent p , so that $m \times n < p$. Using Eq. (16) in (Blaum et al., 2013) a search is made for different parameters to obtain Table 1, which is not extensive and does not consider all possible irreducible polynomials, but it gives a good idea.

5.33 Conclusions

We have discussed RAID levels RAID0:7 and some other topics in this chapter:

RAID0 provides load balancing via striping, but no data protection.

RAID1: Mirroring, replication, or duplexing discussed in more detail in Chapter 4.

RAID2: Hamming code can recover from two disk failures. Small strips and synchronized disks accessed concurrently for higher bandwidth.

RAID3: uses small strips one of which is a parity. Disks are synchronized and read concurrently to attain high bandwidths. Single disk failures can be recovered by using parity strips, which are read concurrently from the parity disk.

RAID4 with large stripe units dedicates one disk to parity. This has two disadvantages: (1) the bandwidth of all disks is not available for data access, (2) parity disk may become a bottleneck for high write rates.

RAID5 rotated parity disk array alleviates the two shortcomings of RAID4 by adopting left diagonal parities. Parity striping preserves locality.

Latent Sector Errors - LSEs Dealing with *Latent Sector Errors - LSEs* via *Intra-Disk Redundancy - IDR* and disk scrubbing.

It is more likely that rebuild in RAID5 may fail due to LSEs rather than a second disk failure while rebuild is in progress.

RAID6 uses two check blocks per stripe to deal with rebuild failures due to encountered *Latent Sector Errors - LSEs* or a second disk failure while rebuild. Read-Solomon and parity based coding methods to recover two strips in a stripe such as EVENODD and RDP and Cauchy-RS codes can be used to reduce costs.

RAID7 may recover from up to three disk failures.

Clustered RAID reduces disk overload with one or multiple disks failures. Improves disk response time and reduces rebuild time.

Performance analysis of RAID(4 + k arrays carried using the M/G/1 queueing model.

Rebuild options: Disk-oriented Rebuild - DOR outperforms Stripe-Oriented Rebuild - SOR. Rebuild with Vacationing Server Model - VSM is more efficient than the Permanent Customer Model - PCM in improving disk response times,

since rebuild requests are served at a local priority and rebuild time is reduced since there is a higher probability of consecutive rebuild reads.

5.33.1 Seagate launches self-healing memory

Seagate launches clever new storage system that heals itself on the fly.

<https://www.techradar.com/news/seagate-launches-clever-new-storage-system-that-heals-itself-on-the-fly>

Seagate has announced a new line of set-and-forget storage systems for enterprises that feature a unique self-healing system to increase reliability and minimize the need for human intervention.

Autonomous Drive Regeneration - ADR, the self-correcting system is bundled as part of the Exos CORVAULT high-density storage system, designed to offer SAN-level performance in data center environments.

Seagate claims ADR can renew a drive in-situ, eliminating the need for a manual drive swap. ADR helps return most drives to a “dependable service” state by reconfiguring the drive to bypass any errant components.

In addition to adding to the drive’s longevity, Seagate believes ADR will also help reduce the environmental impact of computer e-waste by keeping the drives in service for longer.

The new drive is housed in a maximum-density 4U chassis that can accommodate 106 drives in only seven inches (18 cm) of rack space. Seagate has further tuned the system to protect the drive against vibrational and acoustic interference, along with heat, and power irregularities.

CORVAULT uses fully redundant, hot swappable dual storage controllers that are powered by the new VelosCT ASIC chips to deliver sequential read/write up to 14 Gb/s and 12 Gb/s, and *Input/Output Operations Per Second - IOPS* of up to 17,680.

All the drives included in the CORVAULT system are equipped with Seagate’s technology and support for *Secure File Transfer Protocol - SFTP*.

https://en.wikipedia.org/wiki/Secure_file_transfer_program

“Self healing memory”, Inventor Michael B. Raynham, USUS6418068B1, HP 2002.

<https://patents.google.com/patent/US6418068B1/en>

Abstract

A memory device responsive to command signals includes multiple banks of memory arrays. Each bank includes a plurality of primary storage cells and a spare unit of spare storage cells. A detector detects an error in a first unit of the primary storage cells in a first one of the banks. A controller responsive to command signals automatically re-maps the first unit of the primary storage cells to the spare unit of storage cells, without interrupting access to memory data.

Chapter 6

Coding for multiple disk failures

Introduction in Section 6.1

Two-dimensional string layouts in Section 6.2

Simple data entanglement layouts with high reliability in Section 6.3

Reed-Solomon - RS codes in Section 6.4

A family of MDS block array codes with two parities in Section 6.5

Codes for correcting of two erasures with independent parities in Section 6.6

Row-Diagonal Parity - RDP codes in Section 6.7

Short write operations in Section 6.8

Addition reading in Section 6.9

Subsections 6.6, 6.7, and 6.8 are based on material written by Mario Blaum for (Thomasian and Blaum, 2009).

6.1 Introduction

Parity based coding methods EVENODD and RDP for tolerating two disk failures are described in detail in this chapter. Given that failed disks are known from a coding point of view the model corresponds to *erasure correction*. This model is similar to the one of multi-track tape recording, which has been studied by different authors: Thomas E. Fuja, (U. Maryland, U. Notre Dame), Chris Heegard (Cornell U.), and Mario Blaum (Fuja et al., 1989) and A.M. Patel (Patel, 1985) at IBM. However, a convolutional approach used in these works, has infinite error propagation. The codes described in this section are of block type, which do not have the problem of infinite error propagation, since an uncorrectable error is always limited by the size of the block. Another advantage of block codes over convolutional codes is that they do not have overhead parity at the end of the block.

We say that a code is binary if its symbols are bits, i.e., elements on the binary finite field or *Galois Field* - $GF(2)$, as opposed to non-binary codes, whose symbols are over larger fields like $GF(256)$ (8 bits or bytes).

https://en.wikipedia.org/wiki/Finite_field

Binary block codes correction of erasures is presented by Garth Gibson (CMU, Vector Institute), Lisa Hellerstein (NYU), Richard Karp (Simons Institute), Randy Katz and David Patterson (Google) at UCB (Gibson et al., 1989), (Hellerstein et al., 1994).

The problem with binary codes is that they are not *Maximum Distance Separable* - *MDS*, do not meet the Singleton bound: in order to correct r erasures, they require r parity disks.

https://en.wikipedia.org/wiki/Singleton_bound

The authors of (Gibson et al., 1989) suggested that research was needed to study non-binary codes that are MDS, and therefore such techniques are reviewed in this chapter.

We first consider the RAID layout described in (Newberg and Wolfe, 1994) which optimizes the layout of disks for some of the two-dimensional schemes.

6.2 2-Dimensional string layouts

Given an $n \times n$ disk array, the 2d parity scheme in (Newberg and Wolfe, 1994) associates n row (horizontal) R parities to protect rows and n columnar (vertical) C parities to protect columns. With Complete 2d (C2d) parity a parity P is added at (n, n) as shown in Fig. 6.1, which protects all data disks and is the XOR of all vertical or diagonal parities.

C2d protects against all three disk failures (regardless of whether they are data or parity), but not against bad squares, i.e., two broken disks in a row and two broken disks in a column. Bad squares may be based on more than four disks, such as shown by Xs in Fig. 6.2.

D	D	D	D	D	R
D	D	D	D	D	R
D	D	D	D	D	R
D	D	D	D	D	R
D	D	D	D	D	R
C	C	C	C	C	P

FIGURE 6.1 Complete 2-dimensional parity protection.

X	X	D	D	D
0	X	X	O	O
X	O	X	O	O
O	O	X	O	X
X	O	O	X	0

D	D	D	D	D
D	D	D	X	X
X	D	X	D	D
D	D	X	D	X
X	D	D	X	D

FIGURE 6.2 Bad k -squares for $k = 4$ on left and the same bad square after the permutation of rows and columns.

According to Corollary 1 in (Newberg and Wolfe, 1994) the number of bad k -squares is the number of ways to choose k rows and k columns divided by $2k$, since bad squares are counted $2k$ times by the expression.

$$s(k) = \frac{(n(n-1) \dots (n-k+1))^2}{2k} = \frac{[n_k]^2}{2k}.$$

According to Claim 6 the *Mean Time to Data Loss - MTDDL* of an $n \times n$ array is given as:

$$\text{MTDDL} = \frac{\text{repair} - \text{interval}}{\sum_{d=0}^n b(d) p^d (1-p)^{n^2-d}}$$

where p is the probability of a disk failure in a repair cycle and $b(d)$ is the number of bad arrangements plus the pivot disk. We have:

$$b(0) = b(1) = b(2) = 0, \quad b(3) = \frac{4}{n^2} s(2)$$

$$b(4) = \frac{5}{n^2} [(n^2 - 4)s(2)], \quad b(5) = \frac{6}{n^2} \left[s(3) + \binom{n^2 - 4}{2} s(2) - 4 \binom{n}{2} \binom{n}{3} \right]$$

$$b(6) = \frac{7}{n^2} \left[s(2) \binom{n^2 - 4}{3} + s(3)(n^2 - 6) - 18s(3) \right], \quad b(d) \leq \binom{n^2 - 1}{d}$$

For more details the reader is referred to the original paper.

6.3 Simple data entanglement layouts with high reliability

The simple data entanglement scheme is presented by Francois Jehan Paris et. in (Estrada-Galiñanes et al., 2016) in the context of two dimensional arrays with n^2 data disks $D_{i,j}$, $1 \leq i \leq n$, $1 \leq j \leq n$. Horizontal and vertical parities are computed as follows.

$$P_i = \oplus_{j=1}^n D_{i,j}, \quad 1 \leq j \leq n, \quad 1 \leq i \leq n,$$

$$Q_j = \oplus_{i=1}^n D_{i,j}, \quad 1 \leq i \leq n, \quad 1 \leq j \leq n,$$

there are n^2 cases when three disk failures $D_{i,j}$, P_i , Q_j will result in data loss.

$$\begin{pmatrix} D_{1,1} & D_{1,2} & D_{1,3} & D_{1,4} & P_1 \\ D_{2,1} & D_{2,2} & D_{2,3} & D_{2,4} & P_2 \\ D_{3,1} & D_{3,2} & D_{3,3} & D_{3,4} & P_3 \\ D_{4,1} & D_{4,2} & D_{4,3} & D_{4,4} & P_4 \\ Q_1 & AQ_2 & Q_3 & Q_4 & . \end{pmatrix}$$

Data entanglement deals with the data loss cases with three disk failures by computing parities differently as follows:

$$P'_i = P_{i-1}D_{i,j}, 2 \leq i \leq n \text{ and } Q'_j = Q_{j-1} \oplus Q_j, 2 \leq j \leq n$$

For example to recover $D_{3,2}$ with failed P_3 and Q_2 can be carried out as follows:

$$P'_3 = P'_4 \oplus_{j=1}^4 D_{4,j} \text{ can be used to recover } D_{3,2}.$$

Four examples of configurations with four disk failures for $n = 4$ which are not recoverable are provided in the papers as follows: $m = n^2 + 2n$ is used to denote the total number of disks to specify the number of configurations leading to disk failure.

Type A quadruple: $D_{3,2}$, $D_{4,4}$, P'_4 , Q'_4 with $m - 3$ cases.

Type B quadruple: $D_{1,2}$, $D_{1,3}$, $D_{3,2}$, $D_{4,3}$ with $\binom{n}{2}^2$ cases.

Type C quadruple: $D_{3,2}$, P'_3 , $D_{4,2}$, P'_4 with $2n$.

Type D quadruple: $D_{3,4}$, P'_3 , P'_4 , Q'_4 with $(m - 3) + \binom{n}{2}^2 + 2n + 2$ out of $\binom{m}{4}$ failures.

The probability is obtained by dividing by $\binom{m}{4}$.

Three examples of five disk failures that lead to data loss are

Types A & B quintuple: $D_{3,4}$, P'_3 , $D_{4,4}$, P'_4 , Q'_4 with $\binom{m-3}{2} + (\binom{m}{2}^2 + 2n)(m - 4)$.

Type E quintuple: $D_{2,4}$, P'_2 , P'_3 , P'_4 , Q'_4 two.

Type F quintuple: $D_{3,3}$, P'_3 , P'_4 , Q'_3 , Q'_4 one.

Probabilities are obtained by dividing the sum by $\binom{n}{5}$.

Fatal quintuple failures are also reported in (Estrada-Galiñanes et al., 2016).

Given that the disk failure rate is λ and α_i is the probability of data loss when a disk failure at state S_i with $i = 1, 4$. Otherwise there is a transition to S_{i+1} . The repair rate per disk is μ .

$$S_0 \rightarrow S_1: \lambda$$

$$S_i \rightarrow S_{i+1}: (1 - \alpha_i)(m - i)\lambda, \quad 1 \leq i \leq 4$$

$$S_{i+1} \leftarrow S_i: (i + 1)\mu, \quad i = 1, 4$$

Given the transition probabilities six linear differential equations can be written:

$$\frac{dp_1(t)}{dt} = -[(m - 1)\lambda + \mu]p_1(t) + m\lambda p_0(t) + 2\mu p_2(t).$$

$$\frac{dp_2(t)}{dt} = -[(m - 2)\lambda + \mu]p_2(t) + (1 - \alpha_1)(m - 1)\lambda p_1(t) + 3\mu p_3(t).$$

$$\frac{dp_3(t)}{dt} = -[(m - 3)\lambda + 3\mu]p_3(t) + (1 - \alpha_2)(m - 2)\lambda p_2(t) + 4\mu p_4(t).$$

$$\frac{dp_4(t)}{dt} = -[(m - 4)\lambda + 4\mu]p_4(t) + (1 - \alpha_3)(m - 3)\lambda p_3(t) + 5\mu p_5(t).$$

$$\frac{dp_5(t)}{dt} = -[(m - 5)\lambda + 5\mu]p_5(t) + (1 - \alpha_4)(m - 4)\lambda p_4(t).$$

$p_i(t)$ is the probability that the system is at S_i with $p_0(0) = 1$.

MTTDL = $\sum_{i=0}^4 p_i^*(s = 0)$ and the reliability after d years is $R(d) \approx e^{-d/MTTDL}$.

6.4 Reed-Solomon codes

Reed Solomon codes (Reed and Solomon, 1960), (MacWilliams and Sloane, 1977) are the most common class of MDS codes. They are described in numerous books in coding theory, we omit their description here, but for a tutorial see the mathematics of RAID6 in (Anvin, 2007). A tutorial from the point of view of the systems programmer can be found in (Plank, 1997a). This paper has some errors and should be treated with caution (for instance, the dispersal matrix A does not have the desired properties). For a correction see (Plank and Ding, 2005) and (Plank, 2005). An early implementation of Reed-Solomon codes was Iceberg from StorageTek, which was a RAID6 2DFT (Friedman, 1995). Reed-Solomon codes have been proposed and used for RAID6 architectures at IBM (Blaum and Ouchi, 1994). Some of the best implementations of Reed-Solomon codes for correction of erasures are based on Cauchy matrices as noted in (Plank and Xu, 2006).

According to an example given in (Gibson, 1992) the two check blocks to protect two disks A and B can be computed as $C_1 = (A + B) \bmod 4$ and $C_2 = (A + 2B) \bmod 4$. If data disk disks A and B fail they can be reconstructed using the inverse of matrix M :

$$M = \begin{pmatrix} 1 & 1 \\ 1 & 2 \end{pmatrix}, \quad M^{(-1)} = \begin{pmatrix} 2 & -1 \\ -1 & 1 \end{pmatrix}, \quad \begin{pmatrix} C_1 \\ C_2 \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 2 \end{pmatrix} \begin{pmatrix} A \\ B \end{pmatrix}$$

$$\begin{pmatrix} A \\ B \end{pmatrix} = \begin{pmatrix} (A + B) \bmod(4) \\ (A + 2B) \bmod(4) \end{pmatrix} \quad \begin{pmatrix} A \\ B \end{pmatrix} = \begin{pmatrix} 2 & -1 \\ -1 & 1 \end{pmatrix} \begin{pmatrix} C_1 \\ C_2 \end{pmatrix} = \begin{pmatrix} (2C_1 - C_2) \bmod(4) \\ (C_2 - C_1) \bmod(4) \end{pmatrix}$$

The problem of finding MDS block array codes with two parities was solved by Mario Blaum in (Blaum, 1987), and Rodney M. Goodman (rip), Robert J. McEliece (rip), and Masahiro Sayano (Aerospace Corp.) at CalTech in (Goodman and Sayano, 1990), (Goodman et al., 1993). There, it is shown that an $(m - 1) \times m$ array code with horizontal and diagonal parity lines (with a toroidal topology), can correct any $(m - 1)$ -bit column in error if and only if m is a prime. Of course the codes can also correct two erased columns, making them very attractive for disk arrays technology. We will study these MDS codes in the next subsection. Although such codes are not optimal in the number of updates or of encoding operations, they are important historically, since they introduce the concept of horizontal and diagonal parity: the rest of the codes to be presented use variations of this concept, and the proofs that they are MDS as well as their recursive decoding algorithms proceed along the same lines.

In Section 6.6 we shall discuss the EVENODD family of block array codes. The EVENODD codes have two parity columns that are independent from each other. This makes updating the array less cumbersome. In Section 6.7 we present the *Row-Diagonal Parity* - RDP family of codes, that is intermediate between those in the previous two sections and optimizes the number of XORs at the encoding. In Section 6.8 we discuss the problem of the optimal number of updates. In particular, we present a distributed parity scheme, the X-code (Xu and Bruck, 1999).

As we can see, we concentrate on the problem of correcting two erasures, since it is the most important problem in practice. However, the correction of multiple erasures is also interesting both from a practical as well as a theoretical point of view. For instance, the codes of Section 6.5 have been extended to multiple parities by describing them as Reed-Solomon codes over particular polynomial rings as noted by Mario Blaum (IBM) and Ron M. Roth (Technion, Israel) (Blaum and Roth, 1993). The generalization to multiple parities of the EVENODD codes of Section 6.6 is treated by Mario Blaum, Jehoshua (Shuki) Bruck (CalTech), and Alexander Vardy (UCSD) in (Blaum et al., 1996). There, it is shown that the extension to three parities still gives an MDS code, but this is not true in general. The extension of the RDP codes presented in Section 6.7 to multiple parities, which remained as an open problem, has been solved Mario Blaum in (Blaum, 2006), Hachiro Fujita (U. Tokyo) (Fujita, 2006) and later by Atul Goel and Peter F. Corbett at NetApp (Goel and Corbett, 2012), (Corbett and Goel, 2012). The problem of multiple erasures is also treated in papers by Gui Liang Feng et al. (Feng et al., 2005a,b), although the codes described there are not MDS in general.

There are several other works that are relevant, although it is beyond the scope of this review to discuss each of them in detail. For instance, James Hafner (rip) at IBM proposed Weaver codes (Hafner, 2005b). Although Weaver codes have several interesting properties from the implementation point of view, they are not MDS. In (Hafner et al., 2005), James Hafner et al. study the problem of failures that exceed the erasure-correcting capability of the code due to hard errors in localized sectors, and they propose matrix methods to recover from these errors. See also (Hafner et al., 2008) by James Hafner et al. for further techniques on the subject.

HoVer codes in (Hafner, 2005a, 2006) as shown in Fig. 6.3, which are based on horizontal and vertical parities. Two examples are Hover(26,29)_{2,1} with efficiency 0.897 and Hover(28,27)_{0,3} with efficiency 0.900. Efficiency is defined as percentage of storage which holds data (Plank, 2005).

X(0,0)	X(0,1)	...	X(0,n-1)	H(0,0)	...	H(0,h-1)
X(1,0)	X(1,1)	...	X(1,n-1)	H(1,0)	...	H(1,h-1)
⋮	⋮	⋮	⋮	⋮	⋮	⋮
X(r-1,0)	X(r-1,1)	...	X(r-1,n-1)	H(r-1,0)	...	H(r-1,h-1)
V(0,0)	V(0,1)	...	V(0,n-1)			
⋮	⋮	⋮	⋮			
V(v-1,0)	V(v-1,1)	...	V(v-1,n-1)			

FIGURE 6.3 Hover Horizontal/Vertical code with h and r check disk per row and column. $(v-1) \times (h-1)$ are left unused and can be used to hold metadata.

For an analysis of *Low Density Parity-Check - LPDC* codes for erasure-correction, see James Plank and Michael G. Thomason (Plank and Thomason, 2007). LDPC codes also known as Gallager codes, in honor of Robert G. Gallager, who developed the concept in his PhD thesis at MIT in 1960 (Gallagher, 2003).

The so called Liberation Codes by James Plank (Plank, 2008a) present good parameters for RAID6 applications, and in most cases approach optimal encoding. A special case of a minimum density with a codeword length of eight is presented by James Plank in (Plank, 2008b). For a systematic code with a minimal number of ones in its parity-check matrix, see (Blum and Roth, 1999).

An experimental study to compare the performance of open source erasure codes for RAID6 is reported in (Plank et al., 2009b). The performance metric is the encoding/decoding speed in MB/second, with RDP demonstrating the best performance in this study.

6.5 A family of MDS block array codes with two parities

Assume that we have m tracks or disks. Let the information in each disk be divided into symbols, where a symbol is a binary vector (bit, byte, sector, etc.). For simplicity, we assume that each symbol is a bit.

The codes we want to construct are defined by means of

$(m-1) \times m$ arrays $(a_{i,j})_{\substack{0 \leq i \leq m-2 \\ 0 \leq j \leq m-1}}$,

where m is prime, such that $a_{i,j}$ represents the i -th symbol in the j -th column (disk). We will assume that the last two columns (disks) carry the parity. However, in order to avoid bottleneck effects when repeated “write” operations are involved, this distribution of the parity may be rotated for different blocks in the disk. We shall further add a fictitious zero-row to the array, i.e., $a_{m-1,j} = 0$ for $0 \leq j \leq m-1$. This is merely done to make the notation easier in the encoding procedure. With this convention, the arrays are now square $m \times m$ arrays.

We define the code $\mathcal{B}_2(m)$ as the set of arrays $(a_{i,j})_{0 \leq i,j \leq m-1}$ with $a_{m-1,j} = 0$ for all j , satisfying the following two equations ($\langle m \rangle_n$ denotes the unique integer l such that $0 \leq l < n$ and $l \equiv m \pmod{n}$); for instance, $\langle 7 \rangle_5 = 2$ and $\langle -2 \rangle_5 = 3$):

$$\bigoplus_{j=0}^{m-1} a_{i,j} = 0, \quad 0 \leq i \leq m-2 \quad (6.1)$$

$$\bigoplus_{l=0}^{m-1} a_{\langle j-l \rangle_m, l} = 0, \quad 0 \leq j \leq m-1. \quad (6.2)$$

Eqs. (6.1) and (6.2) define the encoding. There are two types of parity check equations: horizontal (as given by Eq. (6.1)) and diagonal (as given by Eq. (6.2)). The array has the topology of a torus, so the diagonals are wrapped around. The symbols 0, 1, 2, 3 and 4 in Fig. 6.4 depict the horizontal and diagonal parity lines of the code when $m = 5$. Since a fictitious row has been added, we have a 5×5 array.

Code $\mathcal{B}_2(m)$ can trivially correct a column of erased symbols. For that the horizontal parity check symbols would already have been enough. Before we describe how to decode two erased columns and prove statements about it, we give one typical example.

Example 6.5.1. Consider the array on the left in Fig. 6.5, which is a 5×5 array such that the last row is a fictitious all-zero row. It contains two erased columns: columns 1 and 4.

	0	1	2	3	4
0	0	0	0	0	0
1	1	1	1	1	1
2	2	2	2	2	2
3	3	3	3	3	3
4	4	4	4	4	4

Horizontal parity lines

	0	1	2	3	4
0	0	1	2	3	4
1	1	2	3	4	0
2	2	3	4	0	1
3	3	4	0	1	2
4	4	0	1	2	3

Diagonal parity lines

FIGURE 6.4 The two types of parity check lines of the code $\mathcal{B}_2(5)$.

To decode, start with any of the two columns, say column 1, and consider the diagonal through the known entry $(4, 1)$ in this column at the bottom. All elements on this diagonal are known, except where it meets the other erased column. So this entry, which is $a_{1,4}$, can now be computed. It must be 1. With the horizontal parity also $a_{1,1}$ can now be computed. Its value is 0. This concludes the first cycle of the algorithm. So after row 4 was completely known, now also row 1 is completely known. Starting with the diagonal going through position $(1, 1)$ we find $a_{3,4} = 1$ (notice that $4 - (2)(3) \equiv 3 \pmod{5}$) and subsequently $a_{3,1} = 1$; each time row j is known, then row $j - 3$ modulo 5 will be known. Continuing in this way we get $a_{0,4} = 0$ and $a_{0,1} = 1$, and finally, $a_{2,4} = 0$ and $a_{2,1} = 1$.

	0	1	2	3	4
0	0	?	1	0	?
1	1	?	1	1	?
2	0	?	0	1	?
3	1	?	0	1	?
4	0	0	0	0	0

Before decoding

	0	1	2	3	4
0	0	1	1	0	0
1	1	0	1	1	1
2	0	0	0	1	1
3	1	1	0	1	1
4	0	0	0	0	0

After decoding

FIGURE 6.5 An example of the decoding of two erasures.

We can now state the main theorem of this section.

Theorem 6.5.1. *Code $\mathcal{B}_2(m)$ can correct up to any two erased columns if and only if m is prime.*

The requirement that m is a prime number is not a serious limitation in multitrack magnetic recording and disk arrays applications, since the code can always be shortened to any number of columns by assuming that a certain number of information columns are zero.

Proof of the ‘only if’ part. Suppose that m is not prime and let d be a non-trivial divisor of m . Consider the array with ones on positions $(i, 0)$ and (i, d) , where i runs over all possible multiples of d , $0 \leq d \leq m - 1$. This array can be viewed as the all-zero array in $\mathcal{B}_2(m)$ with two erroneous columns added to it. However this array is itself in $\mathcal{B}_2(m)$, as can be easily checked. So when replacing columns 0 and d by erasures, there is no unique decoding. $\square$

We will prove the ‘if’ part of the theorem by presenting a decoding algorithm for two erased columns.

Algorithm 6.5.1 (2-Erasure Decoding Algorithm for $\mathcal{B}_2(\mathbf{m})$ Codes). Let m be a prime number and let $(r_{i,j})_{\substack{0 \leq i \leq m-2 \\ 0 \leq j \leq m-1}}$ be a received array, which is a word $(a_{i,j})_{\substack{0 \leq i \leq m-2 \\ 0 \leq j \leq m-1}}$ in $\mathcal{B}_2(m)$ with two erased columns, say columns u and v with $0 \leq u < v \leq m - 1$.

First we add a fictitious bottom row of zeros, i.e., $a_{m-1,j} = 0$ for $0 \leq j \leq m - 1$. Then proceed as follows:


```

 $s \leftarrow v - u.$ 
for  $l = 1$  to  $m - 1$  do
  begin    compute  $a_{(-1-ls)_m, v}$  from the diagonal parity check equation through  $((-1 - (l - 1)s)_m, u)$ .
           compute  $a_{(-1-ls)_m, u}$  from the horizontal parity check equation through  $((-1 - ls)_m, v)$ .
  end

```

Observe that the encoding algorithm is the special case of the decoding algorithm in which $u = m - 2$ and $v = m - 1$.

Proof of Algorithm 6.5.1 and the ‘if’ part of Theorem 6.5.1. The proof follows from the following two observations: first, at each stage of the computations of $a_{(-1-ls)_m, v}$ and $a_{(-1-ls)_m, u}$ by means of the parity check equations, all entries involved are known except for the one to be computed.

Secondly, since m is prime, the greatest common divisor between $s = v - u$ and m is one, i.e., $\gcd(s, m) = 1$, so the multiples of $s \pmod{m}$ will cover all indices from 1 to $m - 1$. In other words, all erased values will be computed. $\square$

The above also shows that the minimum (column) distance of $\mathcal{B}_2(m)$ is 3 when m is prime. Since the code can correct any two erased columns, it can also correct a column in error. There is an algorithm for correcting one column in error, but since the RAID model deals with erasures only, we will omit it here (Blaum, 1987), (Blaum et al., 1988).

The codes described above are MDS but very short: their length is roughly equal to the size of the symbol, while Reed-Solomon codes, also being MDS, are much longer. If the symbols have length m , Reed-Solomon codes have length up to 2^m . So, Reed-Solomon codes can always be used instead of $\mathcal{B}_2(m)$ codes, but the converse is not necessarily true. However, $\mathcal{B}_2(m)$ codes have less complexity, since they involve exclusive-OR operations only, avoiding the arithmetic over a Galois field.

RAID architectures proposed by David Patterson, Garth Gibson, and Randy Katz at UCB (Patterson et al., 1988) are an example of an application for which the length of the code has size comparable to or smaller than the size of the symbols. Another application is multi-track magnetic recording by A.M. Patel (Patel, 1985) and P. Prusinkiewicz and Stanislaw Budkowski (Prusinkiewicz and Budkowski, 1976). For more on complexity, see Sections 6.6 and 6.7.

Let us point out that the construction with two parities described in this section can be extended to more parities (Blaum and Roth, 1993): in order to do that, however, the simple description of the code with parity lines of different slopes is not enough. An algebraic description of these codes as Reed Solomon codes over the ring of polynomials modulo $1 + x + \dots + x^{m-1}$, m a prime, is required. For reasons of space, we omit this description here and we refer the reader to (Blaum and Roth, 1993) or to (Blaum et al., 1988) written by Mario Blaum, Patrick C. Farrell, and Henk van Tilborg.

6.6 Codes for correcting two erasures with independent parities

In Section 6.5, we presented a family of array codes that can correct any two erasures using horizontal and diagonal parity lines. The decoding of two erasures involved a simple recursion. The drawback of the construction is that the processes of encoding and of small write operations also involve a recursion. In applications like RAID architectures, the size of each individual symbol may be as big as a whole sector. As a consequence, it is desirable to have a minimal number of parity symbols affected by the update of a single information symbol. The scheme in Section 6.5 forces the updating of most of the parity symbols each time an information symbol is updated.

In this section, we present an efficient encoding procedure, called EVENODD (Blaum et al., 1995), that is based on *eXclusive-OR - XOR* operations and independent parities. Since the parities are independent, there is no recursion during this encoding procedure. The decoding algorithm is slightly more complex than the one presented in the previous section. Since it uses the same principle, we will omit it here. As a result of the simple encoding procedure a limited write action is greatly simplified, since any modified information symbol affects only two symbols in the parity most of the time.

We will assume that the elements of the code are $(m - 1) \times (m + 2)$ arrays $(a_{i,j})_{\substack{0 \leq i \leq m-2 \\ 0 \leq j \leq m+1}}$, m a prime number, with the information stored in columns 0 to $m - 1$ and the parity stored in columns m and $m + 1$. We also assume, as in the previous section, that there is an imaginary 0-row after the last row, i.e., $a_{m-1,j} = 0$, $0 \leq j \leq m + 1$ (with this convention, the array is now an $m \times (m + 2)$ array).

Next we define the code as follows:

Definition 6.6.1. Given m prime, the code $\mathcal{EO}_2(m)$ is the set of arrays $(a_{i,j})_{\substack{0 \leq i \leq m-2 \\ 0 \leq j \leq m+1}}$, such that for each i , $0 \leq i \leq m - 2$,

$$a_{i,m} = \bigoplus_{t=0}^{m-1} a_{i,t} \quad (6.3)$$

$$a_{i,m+1} = S \oplus \left(\bigoplus_{t=0}^{m-1} a_{(i-t)_m, t} \right) \quad (6.4)$$

where S is defined by

$$S = \bigoplus_{t=1}^{m-1} a_{m-1-t, t}, \quad (6.5)$$

Eq. (6.3) and Eq. (6.4) define the encoding. As in the previous section, we have two types of parity: horizontal parity (as given by Eq. (6.3)) and diagonal parity (as given by Eq. (6.4)). Column m is simply the XOR of columns $0, 1, \dots, m-1$ (depicted on the left of Fig. 6.6).

Column $(m+1)$ carries the diagonal parity according to Eq. (6.4). Notice that one diagonal is missing in (6.4). This diagonal consists of the entries $(m-2, 1), (m-3, 2), \dots, (0, m-1)$ (depicted by ∞ in Fig. 6.6 on the right) and in fact it defines S . So it is this special diagonal that decides the parity of the other diagonals. In fact, if the special diagonal has EVEN parity then so do the others. Otherwise, they all have ODD parity. This is why this scheme is called the EVENODD scheme.

It also follows from the above that

$$\bigoplus_{i=0}^{m-2} a_{i,m} = \bigoplus_{i=0}^{m-2} \left(\bigoplus_{j=0}^{m-1} a_{i,j} \right) = S \oplus \left(\bigoplus_{i=0}^{m-2} a_{i,m+1} \right).$$

In other words, the parity S of the special diagonal can also be computed as follows:

$$S = \text{the binary sum of the elements in columns } m \text{ and } m+1. \quad (6.6)$$

0	0	0	0	0	0	
1	1	1	1	1	1	
2	2	2	2	2	2	
3	3	3	3	3	3	

0	1	2	3	∞		0
1	2	3	∞	0		1
2	3	∞	0	1		2
3	∞	0	1	2		3
∞	0	1	2	3		∞

FIGURE 6.6 Parity lines of $\mathcal{EO}_2(5)$.

Code $\mathcal{EO}_2(m)$ defined above can recover the information lost in any two columns, as the $\mathcal{B}_2(m)$ code defined in Section 6.5. Therefore, the minimum distance of the code is 3, in the sense that any non-zero array in the code has at least 3 columns that are non-zero. The proof relies on the fact that m is a prime number and it is based on ideas similar to those in Section 6.5. Without this assumption, the resulting code does not have minimum distance 3, and it cannot retrieve any two erased columns.

The next example illustrates the encoding for $m = 5$.

Example 6.6.1. Let $m = 5$, and let the symbols be denoted by a_{ij} , $0 \leq i \leq 3$, $0 \leq j \leq 6$. The parity symbols are in columns 5 and 6. A practical implementation of this example is to consider 7 disks numbered 0 through 6, each disk has 4 disk sectors, the data sectors are on disks numbered 0, 1, 2, 3 and 4, and the parity disk sectors are on disks numbered 5 and 6. Eq. (6.5) gives

$$S = a_{3,1} \oplus a_{2,2} \oplus a_{1,3} \oplus a_{0,4}.$$

According to Eq. (6.3) and Eq. (6.4) the parity symbols are obtained as follows:

$$a_{l,5} = a_{l,0} \oplus a_{l,1} \oplus a_{l,2} \oplus a_{l,3} \oplus a_{l,4}, \quad 0 \leq l \leq 3$$

$$a_{0,6} = S \oplus a_{0,0} \oplus a_{3,2} \oplus a_{2,3} \oplus a_{1,4}$$

$$\begin{aligned}
a_{1,6} &= S \oplus a_{1,0} \oplus a_{0,1} \oplus a_{3,3} \oplus a_{2,4} \\
a_{2,6} &= S \oplus a_{2,0} \oplus a_{1,1} \oplus a_{0,2} \oplus a_{3,4} \\
a_{3,6} &= S \oplus a_{3,0} \oplus a_{2,1} \oplus a_{1,2} \oplus a_{0,3}
\end{aligned}$$

For instance, assume that we want to encode the five columns (we add the imaginary 0-row) below on the left.

We have to fill up the last two columns with the encoded symbols. Notice that $S = a_{3,1} \oplus a_{2,2} \oplus a_{1,3} \oplus a_{0,4} = 1$. Therefore, the diagonals will have odd parity. The encoding gives the array on the right:

1	0	1	1	0		
0	1	1	0	0		
1	1	0	0	0		
0	1	0	1	1		
0	0	0	0	0	0	0

1	0	1	1	0	1	0
0	1	1	0	0	0	0
1	1	0	0	0	0	1
0	1	0	1	1	1	0
0	0	0	0	0	0	0

Instead of giving the actual decoding algorithm of the EVENODD code $\mathcal{EO}_2(m)$ for correction of two erasures, we give an example that illustrates the idea behind it.

Example 6.6.2. Consider $\mathcal{EO}_2(5)$, as in Example 6.6.1, and assume that we have the array below at the left, in which columns (disks) 0 and 2 have been erased (lost):

?	0	?	1	0	1	1
?	1	?	0	0	0	1
?	1	?	0	0	1	1
?	1	?	1	1	0	0
0	0	0	0	0	0	0

0	0	0	1	0	1	1
1	1	0	0	0	0	1
0	1	0	0	0	1	1
1	1	0	1	1	0	0

This is the main case for the decoding algorithm: two information columns have been erased. The cases in which at least one of the two parity columns has been erased are special cases that are easy to handle.

The first step is finding the parity S of the diagonals. This value follows directly from (6.6). In the array above, we can see that the exclusive-OR of the bits in the two parity columns is 1, therefore the diagonals have odd parity. Once S has been established, the decoding proceeds very much like Algorithm 6.5.1 for $\mathcal{B}_2(m)$ codes does.

Next, the algorithm starts a recursion to retrieve the missing bits $a_{l,0}$ and $a_{l,2}$, $0 \leq l \leq 3$. We first need an entry where we can start. Either entries (4, 0) or (4, 2) would serve this purpose. For instance, taking the diagonal through (4, 0) (i.e., entries (4,0), (3,1), (2,2), (1,3) and (0,4) together with the “fictitious” parity entry (6,0)), we see that only one entry is unknown: $a_{2,2}$. Since the diagonal has parity $S = 1$, we find $a_{2,2} = 0$. Using the horizontal (even) parity, we obtain $a_{2,0} = 0$. Next, we consider the diagonal going through entry (2,0), etc. In this way we find successively: $a_{0,2} = 0$, $a_{0,0} = 0$, $a_{3,2} = 0$, $a_{3,0} = 1$, $a_{1,2} = 0$, $a_{1,0} = 1$. The result is depicted above on the right.

The codes $\mathcal{EO}_2(m)$ can be extended to codes $\mathcal{EO}_r(m)$ that consist of $(m-1) \times (m+r)$ arrays with m information columns and r parity columns, each parity column independently encoded. More precisely, given m , we say that the code $\mathcal{EO}_r(m)$ is the set of arrays $(a_{i,j})_{\substack{0 \leq i \leq m-2 \\ 0 \leq j \leq m+r-1}}$, such that, for each l , $0 \leq l \leq m-2$, and for each s , $1 \leq s \leq r-1$,

$$a_{l,m} = \bigoplus_{t=0}^{m-1} a_{l,t} \quad (6.7)$$

$$a_{l,m+s} = S_s \oplus \left(\bigoplus_{t=0}^{m-1} a_{\langle l-st \rangle_m, t} \right), \quad (6.8)$$

where

$$S_s = \bigoplus_{t=1}^{m-1} a_{\langle -1-st \rangle_m, t}. \quad (6.9)$$

Therefore, $\mathcal{EO}_r(m)$ has even parity over horizontal lines as given by Eq. (6.7), and even or odd parity over the lines of slope s , $1 \leq s \leq r-1$, as given by Eq. (6.8). Bit S_s given by Eq. (6.9), determines the parity of the line of slope s . It can be proven that if m is prime, then code $\mathcal{EO}_3(m)$ is MDS, i.e., it has minimum (column) distance four. However, for $r \geq 4$, this is in general not true any more. For details on the generalization of EVENODD, we refer the reader to (Blaum et al., 1996). For a specific architecture that generalizes EVENODD to three erasures, see (Huang and Xu, 2005).

Regarding complexity, it was shown in (Blaum et al., 1995) that by counting the number of encoding operations, the EVENODD code has less complexity than a corresponding Reed Solomon code. There are several methods to implement Reed Solomon codes that improve complexity, but they are always more complex than array codes based on simple parity. The RDP codes to be presented in the next section minimize the number of encoding operations.

6.7 Row-Diagonal Parity - RDP codes

The RDP codes presented by Peter Corbett et al. in (Corbett et al., 2004) are an intermediate family of codes between the codes of Sections 6.5 and 6.6. Given a prime number m , the codes take $m-1$ information columns of length $m-1$ each to which again two parity columns are added, a horizontal parity column and a diagonal parity column. Thus, the code is an $(m-1) \times (m+1)$ array. The horizontal parity column takes horizontal parity only (without including the diagonal parity column). The diagonal parity column takes the diagonal parity including the horizontal parity column, i.e., the parity lines are not independent, as is the case with the EVENODD code, the horizontal parity has to be computed first. The horizontal and diagonal parity lines of an RDP code with $m=5$ are illustrated in Fig. 6.7. There is a diagonal that can be ignored at the encoding. The diagonal parities are always even, as opposed to EVENODD, which has diagonal parities either even or odd, hence EVENODD. This allows for some saving in the number of XORs at the encoding. Moreover, it was proven in (Corbett et al., 2004) that this number of XORs (for the dimensions of the code) is minimal under the assumption that the code is MDS.

0	0	0	0	0	
1	1	1	1	1	
2	2	2	2	2	
3	3	3	3	3	

0	1	2	3		0
1	2	3		0	1
2	3		0	1	2
3		0	1	2	3

FIGURE 6.7 Parity lines of code $RDP_2(5)$.

Let us write explicitly the encoding equations. Given m prime, the code $RDP_2(m)$ is the set of arrays $(a_{i,j})_{\substack{0 \leq i \leq m-2, \\ 0 \leq j \leq m}}$, such that (assuming, as usual, that $a_{m-1,j} = 0$) for each i , $0 \leq i \leq m-2$,

$$a_{i,m-1} = \bigoplus_{t=0}^{m-2} a_{i,t} \quad (6.10)$$

$$a_{i,m} = \bigoplus_{t=0}^{m-1} a_{\langle i-t \rangle_m, t} \quad (6.11)$$

The decoding of two erased columns proceeds similarly to the decoding of the codes described in Sections 6.5 and 6.6. As stated above, the RDP codes minimize the number of XORs required for the encoding operation.

A generalization of RDP codes was obtained independently by Mario Blaum in (Blaum, 2006) and by Hachiro Fujita in (Fujita, 2006). At a later point Atul Goel and Peter Corbett patented *RAID Triple Parity - RTP* (Corbett and Goel, 2012) and authored (Goel and Corbett, 2012).

This generalization defines a code $RDP_r(m)$, m prime, $r \geq 2$, of length $p+r-1$, where the horizontal parity line (slope 0) is given by (6.10), while the parity line of slope s , $1 \leq s \leq r-1$, is given by

$$a_{i,m+s-1} = \bigoplus_{t=0}^{m-1} a_{\langle i-st \rangle_m, t} \quad (6.12)$$

The parity lines of code $RDP_3(5)$ are illustrated in Fig. 6.8.

0	0	0	0	0		
1	1	1	1	1		
2	2	2	2	2		
3	3	3	3	3		

0	1	2	3		0	
1	2	3		0	1	
2	3		0	1	2	
3		0	1	2	3	

0	2		1	3		0
1	3	0	2			1
2		1	3	0		2
3	0	2		1		3

FIGURE 6.8 Parity lines of code $RDP_3(5)$.

Interestingly, it turns out that the codes $RDP_r(m)$ are MDS when the generalized EVENODD codes $\mathcal{EO}_r(m)$ defined in the previous section are. For instance, when $r = 3$, $RDP_3(m)$ is MDS for any prime number m . However, for $r \geq 4$, this is not true in general, but depends on the prime number m considered. For details of the connection between $\mathcal{EO}_r(m)$ and $RDP_r(m)$ see (Blaum, 2006).

6.8 Short write operations

An important feature of codes for disk arrays is the short write operation. As stated at the beginning of Section 6.6, we assume that a short write is an updating of only one information bit in the information columns. The EVENODD code $\mathcal{EO}_2(m)$ requires updating of two additional bits in the parity columns in most cases, for a total of three updates. However, when the bit to be updated occurs in the special diagonal, then *all* of the bits in the second parity column need to be updated. We can easily verify that on average, $\mathcal{EO}_2(m)$ requires $4 - 2/m$ updates, as shown by Mario Blaum, Jehoshua Bruck, and Alexander Vardy in (Blaum et al., 1996). The RDP code described in Section 6.7 requires also an average of $4 - 2/m$ updates. However, a lower bound on the number of updates for systematic codes is $3 + 1/m$ as shown by Mario Blaum and Ron M. Roth (Blaum and Roth, 1999). Moreover, this last reference shows a construction achieving this lower bound.

Notice that in the codes defined so far, information bits and parity bits are in different columns. In coding theory, such codes are called systematic. If we drop the condition that the codes are systematic, then the conditions change. There are MDS codes meeting the lower bound of three updates if we allow the parity to be distributed among all columns. In other words, each column contains both information and parity bits, there are no columns dedicated exclusively to parity. To the best of our knowledge, the first paper presenting a construction of MDS codes with distributed parity and three updates is (Zaitsev et al., 1983). This result and its generalizations were also studied in (Blaum and Roth, 1999), (Xu and Bruck, 1999), (Xu et al., 1999).

In particular, the X-code by Lihao Xu and Jehoshua Bruck (Xu and Bruck, 1999), provides an easy array code for distributed parity using two diagonal parity lines. An ad-hoc construction is described by Sandra Baylor and Peter F. Corbett and Chan-Ik Park (PosTech U., S. Korea) at IBM in (Baylor et al., 1999) and (Park, 1995).

A low density parity code is described by Lihao Xu (Wayne State U.), Vasken Bohossian (Human-Engine.com), Jehoshua Bruck, and David G. Wagner (U. Waterloo) at CalTech in (Xu et al., 1999). In particular, this latest reference shows that finding optimal distributed parity codes for correction of two columns is equivalent to a well known problem in graph theory, finding graphs that have perfect factorization.

https://en.wikipedia.org/wiki/Graph_factorization

Distributed parity has advantages over dedicated parity. Certainly, there are no bottlenecks in a distributed parity scheme, no disk gets more access than others, so there is no need to rotate the parity columns. However, distributed parity codes cannot be shortened in general. That forces us to choose a code for a certain number of disks. The code will have to be changed if this number of disks changes in the future. In other words, the system is not scalable. For instance, the X-code consists of m columns, m a prime. If we want to implement the X-code in a real system, we have to demand that the system contains a prime number of disks, a requirement that may be too restrictive.

We finish this section by describing the X-code, already described in Chapter 5, which will illustrate the concepts behind distributed parity and is much in the spirit of the codes that have been described so far. Consider the X-code with $m = 5$. The parity-lines are described in Fig. 6.9. Notice that the parities are independent.

2	3	4	0	1
3	4	0	1	2
4	0	1	2	3
0	1	2	3	4

Lines of slope 1

3	4	0	1	2
2	3	4	0	1
1	2	3	4	0
0	1	2	3	4

Lines of slope -1

FIGURE 6.9 The two types of parity check lines of the X-code.

Now, assume that we want to encode the array

0	1	1	0	1
1	0	0	1	0
0	0	1	0	1

As we can see, each column contains 3 bits of information and two bits of parity, so the code is not systematic (on columns, it is systematic on bits) and the parity is distributed. Using the lines of slope 1 following Fig. 6.9, we obtain

0	1	1	0	1
1	0	0	1	0
0	0	1	0	1
0	1	0	1	1

Similarly, using the lines of slope -1 , the second parity row gives

0	1	1	0	1
1	0	0	1	0
0	0	1	0	1
1	0	0	1	1

The final encoded array is then

0	1	1	0	1
1	0	0	1	0
0	0	1	0	1
0	1	0	1	1
1	0	0	1	1

Regarding short write operations, each time a single information bit is modified, then exactly two parity bits need to be modified as well, achieving the lower bound. The decoding of two erased columns is done similarly to the other algorithms presented. For details, see (Xu and Bruck, 1999) and Chapter 5.

6.9 Additional reading

This chapter touches upon some advanced material in a brief manner. The readers are referred to the reading list in the Appendix and the sources for the material for clarification.

Chapter 7

Saving power in disks, flash memories, and servers

Introduction to power consumption in computer systems in Section 7.1
Saving battery power in laptop computers in Section 7.2
Varying spindown threshold based on user behavior in Section 7.3
Exploiting idleness in storage systems in Section 7.4
Making enterprise computers greener by protecting them better in Section 7.5
Policy optimization for dynamic power management in Section 7.6
Managing energy and server resources in hosting centers in Section 7.7
Interplay of energy and performance for RAID running OLTP in Section 7.8
Dynamic speed control for server disk power management in Section 7.9.
Approaches to conserve disk energy in network servers in Section 7.10
Energy efficiency through burstiness in Section 7.11
Dempsey: a tool for modeling hard disk power consumption in Section 7.12
MAID - Massive Arrays of Idle Disks alternative to tape storage in Section 7.13
The Copan MAID storage system in Subsection 7.13.1
Nexsan AutoMAID in Subsection 7.13.2
Self-tuning power aware storage cache replacement algorithm in Section 7.14
Popular Data Concentration - PDC in Section 7.15
Disk layout optimization for reducing energy consumption in Section 7.16
Managing server energy and operational costs in hosting centers in Section 7.17
Improving energy savings while meeting performance goals in RAID in Section 7.17.1
Performance directed energy management for main memory and disks in Section 7.18
Exploiting redundancy to conserve energy in storage systems in Section 7.19
Thermal disk drive design: challenges and possible solutions in Section 7.20
PARAID: the gear-shifting Power-Aware RAID in Section 7.21
DiskGroup: Energy efficient disk layout for RAID1 systems in Section 7.22
Power provisioning for a warehouse-sized computer in Section 7.22.1
Pergamum: Replacing tape with disk-based archival storage in Section 7.23
Energy efficient RAID - ERAID in Section 7.24
Power reduction via write-offloading in Section 7.25
Redundant Arrays of Hybrid Disks - RAHD in section 7.26
Achieving power-efficient, erasure-coded storage in section 7.27
Effect of energy-saving schemes on disk reliability in Section 7.28
Mathematical model of disk reliability versus load and temperature in Section 7.29
Sample-Replicate-Consolidate mapping - SRCMap in Section 7.30
Power Proportional Distributed File Systems - PPDFS in Section 7.31
Dynamic locality improvement to increase effective storage performance in Section 7.32
Disk data reorganization for reducing energy consumption in Section 7.33
File assignment with minimal variance of service time in Section 7.34
Striping-based Energy Aware - SEA placement in Section 7.35
PEARL: Performance, Energy, and ReLiability in disk arrays in Section 7.36
Power proportionality for data center storage in Section 7.37
Economic evaluation of energy saving with reliability constraints in Section 7.38
Dynamic server provisioning for data center power management in Section 7.39

Modeling the energy costs of I/O Workloads in Section 7.40

Energy proportionality is required in addition to energy efficiency in Section 7.41

SDD design tradeoffs from energy perspective in Section 7.42

Green AI in Section 7.43

Conclusions in Section 7.44

7.1 Introduction to power consumption in computer systems

“What matters most to the computer designers at Google is not speed but power – low power, because data centers can consume as much electricity as a city.”

– Eric Schmidt, CEO, Google in New York Times 9/29/2002.

The initial motivation to conserve power consumption in storage systems was to extend the battery lifetimes of mobile notebook/laptop computers equipped with *Hard Disk Drives* - HDDs in 1990s. Early research dealt with adaptive disk spindown to save battery power consumption, but progressed to deal with high cost of electricity in large collections of servers and their disks utilized in cloud computing. High-speed microprocessors may utilize more power than disks and we discuss various techniques for conserving CPU power by turning them off or running them at a lower voltage and frequency.

Hybrid disks are equipped with a nonvolatile cache layer, which holds hot data and allows disk spindown. Spindown has also been applied to RAID, e.g., in the case of mirrored disks power may be saved by spinning down one disk out of a pair of disks. In RAID(4 + k) arrays up to k disks can be spun down, since their contents of the remaining $N - k$ disks can be reconstructed. Given the higher power consumption for spinup the viability of this scheme relies on the frequency of disk accesses.

Publications on energy conservation are introduced in roughly chronological order, which is helpful in showing the influence of earlier works on further developments. Most notably the concept of *Mostly Idle Arrays of Disks* - MAID introduced in 2002 led to the startup Copan Systems in 2004.

7.2 Saving battery power in laptop computers

A proposal for adaptive disk spindown to reduce battery power consumption in laptop computers was proposed by John Wilkes at HP (Wilkes, 1992a). Ordinarily a fixed timeout interval may be used to turn off a notebook when there is no activity. Given that disks are not necessary for certain operations and that they contribute significantly to power consumption and battery depletion disk spin-down alone may be sufficient to save energy. A file that is being edited on a laptop is held in main memory and it is only when it is to be saved to disk that the disk needs to be spun up. A simple algorithm to determine when to spin down disks is as follows:

1. Determine T_A as the weighted average of recent activity duration.
2. Set delay time to $T_d = \alpha T_A$ where $\alpha > 1$.
3. If $T_d > T_{max}$ then long delays following a long activity period may be avoided by setting $T_d = T_{max}$.
4. The disk is spun down T_d seconds after the last activity.

Power up time is predicted similarly.

While there is no cost on spinning down disks, there is a high cost for spinup. Four modes of operation with times and associated power in milliamperes (mA) given for Quantum 160 MB drives are: spinning (T_{spin} , $P_{spin} = 320$ mA), idling (stopped), being started or spun-up (T_{start} , $P_{start} = 1100$ mA), or being accessed (T_{access} , $P_{access} = 400$ mA) (Dougkis and Marsh, 1993). These figures are used in the analysis in (Greenawalt, 1994) to obtain the power consumed per disk access. The increase in power consumption is calculated for small timeout periods.

7.3 Varying spindown threshold based on user behavior

Methods to vary the spindown threshold based on user access patterns and preferences were investigated by Fred Dougkis, P. Krishnan, and Brian N. Bershad at Panasonic Labs at Princeton, NJ in (Dougkis et al., 1995). Reading from a DRAM and writing to an SRAM was used to reduce disk activities assuming that SRAM can survive system crashes. There is only a need to write to disk when the SRAM is full. Data in DRAM can be protected at the extra cost of an *Uninterruptible Power Supply* - UPS.

Most mobile computers spindown after a period of inactivity. At one extreme we have a policy for least energy by spinning down early, but this policy may introduce unacceptable bumps when the user wants to use the computer again. There is also the possibility of increased power consumption due to spinups.

Additive and multiplicative adjustments to the spindown threshold T are investigated (Douglass et al., 1995). Several traces are used to determine the number of bumps versus energy in joules for $T = 2, 5, 10, 30$ seconds and adjustments to them. Disks are modeled as having four modes of operation: (1) spinning, (2) idling (stopped), (3) being started (spun-up), (4) being accessed. The analysis is based on the characteristics of the Quantum Go-Drive 160.

7.4 Exploiting idleness in storage systems

A rather abstract discussion of idleness is presented by John Wilkes et al. at HP in (Golding et al., 1995b). Exploiting idle time in storage systems and a taxonomy of idle-time detection algorithms and metrics to evaluate them are presented. Idleness detection predictors are evaluated as well. Examples of idle time processing at disks are as follows:

Delayed Writeback: This allows the disk to remain idle for longer periods of time and reduces disk utilization, but read requests are processed immediately by an idle disk. There are two advantages for holding dirty blocks in *NonVolatile Storage - NVS* (Baker et al., 1992), (Carson and Setia, 1992).

1. Dirty blocks are overwritten, so that only one copy following a burst of consecutive updates are written to disk.
2. Multiple blocks can be destaged efficiently taking advantage of spatial locality on disk.

Eager LFS Segment Cleaning: This is required in the context of a *Log-Structured File Systems - LFS* proposed in (Rosenblum and Ousterhout, 1992) and has been studied in (Carson and Setia, 1992), (Seltzer, 1993), (McNutt, 1994), (Matthews et al., 1997), (Menon and Stockmeyer, 1998), (Butterworth, 1999). Taking into account the age of blocks ensures that consolidated blocks will not be modified in the near future.

Disk Power Down: This is discussed in the context of disk spindown by John Wilkes et al. (Wilkes, 1992a), Fred Douglass et al. (Douglass et al., 1995), and David Helmbold, Darrell Long, and Bruce Sherrod in (Helmbold et al., 1996).

Disk Scrubbing to detect and correct *Latent Sector Errors - LSEs* was proposed in (Kari, 1997), Scrubbing was investigated in (Schroeder et al., 2010a) and (Iliadis et al., 2011) and compared with the *Intra-Disk Redundancy - IDR* method proposed by Ajay Dholakia et al. (Dholakia et al., 2008).

Database reorganization: A comprehensive survey of this topic written was authored by Bala Iyer (rip) and Gary Sockut at IBM *Silicon Valley Labs - SVL* (Sockut and Iyer, 2009). Modern DBMSs utilize data structures such as B+-trees, which are self-balancing, but were observed to induce “waves of misery” in (Glombiewski et al., 2019). Periodic (manual) reorganization is required for static *Index Sequential Access Method* (Ramakrishnan and Gehrke, 2002).

Rebuild Processing: In the case of mirrored disks the contents of a failed disk can be reconstructed by copying the contents of the surviving disk onto a spare disk. RAID5 arrays require more complex processing involving reading and XORing blocks in a parity group to reconstruct the contents of a failed disk. RAID5 rebuild processing was described in (Muntz and Lui, 1990b), and its performance analyzed in (Holland and Gibson, 1992), (Holland et al., 1994), (Merchant and Yu, 1992, 1996), (Menon, 1994) (based on (Menon and Mattson, 1992a,b)), (Thomasian and Menon, 1994), (Thomasian and Menon, 1997). A detailed discussion of rebuild processing in RAID5 appears in Chapter 5.

Prepositioning arms: This is done to reduce the seek distance for future requests (King, 1990). This work was extended in (Thomasian and Fu, 2006) to disks with *Zoned Bit Recording - ZBR* and hot-spot cylinders.

Idle detectors can be (1) timer-based, (2) rate-based, (3) adaptive-rate-based, (4) rate-change based, and (5) periodic. When the system runs out of work the timer is started for a fixed, variable, or adaptive period. The system is declared idle when no work arrives during the period. A variable period is affected by the time of day. Rate-based detectors declare a system idle when the rate estimate falls below a threshold. The predictor may also predict the idle period duration. The skeptic takes in one or more prediction streams and emits a new one, e.g., taking into account the quality of predictions and discarding those streams which are consistently wrong.

A measure of how good the detector is at finding idle periods is defined by $efficiency = predicted / actual$. A detector’s incompetence evaluates how much of the predicted idle time is not idle: $incompetence = overflow / actual$. A good idleness detector will have a high efficiency and a low incompetence.

7.5 Making enterprise computers greener by protecting them better

It is demonstrated by Nikolai Joukov (ModelizeIT) and Josef Sipek at IBM in (Joukov and Sipek, 2008) that HDDs cause different kinds of problems for different types of computer systems: server, desktop, laptop. Purchase and operating costs of HDDs are given in Table 7.1.

TABLE 7.1 Costs of HDDs. Power includes cooling and *Uninterruptible Power Supply* - UPS losses (x2), Power 20 cents per KWH.

	Unit cost	Power (W)	Electricity cost
Desktop	\$100	10	\$18/year
Server	\$250	15x2	\$53/year

It is observed that solutions developed to date are not capable of solving the power consumption, noise, and data reliability problems without sacrificing hard disk life-time, data reliability, or user convenience. We considered data reliability, recovery, performance, user convenience, and hard disk caused problems together at the enterprise scale. GreenFS is a fan-out stackable system that offers all-time all-data run-time data protection, improves performance under typical user workloads and allows hard disks to be kept off most of the time. As a result, GreenFS improves enterprise data protection, minimizes disk drive-related power consumption and noise and increases the chances of disk drive survivability in case of unexpected external impacts.

Two issues addressed by GreenFS and its operation are summarized as follows:

1. Important data may be left unprotected due to file selection mistakes.
2. The most recent data updates are not protected.

A list of requirements for GreenFS is as follows:

- Provide run-time *Continuous Data Protection* - CDP or at least replication) of HDDs in the enterprise.
- Apply CDP even when a desktop loses connectivity due to temporary network problems or when mobile clients are away from the network infrastructure.
- Avoid significantly increasing the cost of required backup storage.
- Spin the local hard disks up for short periods of time and only several times a day.
- Provide data access latency and bandwidth similar to the operation with the local hard disks at least under typical user workloads.
- Require minimal hardware and software modifications in existing infrastructure.

7.6 Policy optimization for dynamic power management

Dynamic power management policies that trade off performance for power are considered in (Paleologo et al., 1998). The power management policy decides on transition based on system history, workload and performance constraints.

Prior to this work power management policies were formulated heuristically, but formal *Markov Decision Process* - MDP models are utilized in this work to find policies that provide an optimal tradeoff between performance and power. The associated stochastic optimization problem can be solved exactly and efficiently using well-known methods (Ross, 2014).

A disk case study with a disk with five states specified in Table 7.2, with state transition time to active state, and power. LPidle is the Low Power idle state.

TABLE 7.2 Transition times and power during transition.

State #	State	Time	Power
0	active	NA	2.5 W
1	idle	1.0 ms	1.0 W
3	LPidle	40 ms	0.8 W
5	standby	2.2 sec	0.3 W
7	sleep	6.0 sec	0.1 W

Figure 8(a) in the paper is a simplified state transition diagram, where states 2, 4, 6 are intermediate states from the idle state to states 3, 5, 7, respectively. There are transitions between state 0 and 1. Figure 8(b) plots power consumption versus performance in the form of average queue length. The queue length increases linearly as power consumption is reduced below the active state (2.5 W).

7.7 Managing energy and server resources in hosting centers

Server energy consumption is not proportional to server utilization. The reduction of power consumption by matching it to the requests load is considered by Jeff Chase, Amin Vahdat et al. at Duke U. in (Chase et al., 2001). Incoming requests are routed to a subset of active servers, which are run under a target utilization, while excess servers are transitioned to low power states to maintain surplus capacity. There is also the issue of energy savings for the cooling system.

Maximize Service Revenue and Profit - MSRP greedy algorithm is used to control the allocation of resources for successive epochs to determine a vector of resource allocations μ_i for the i^{th} service $1 \leq i \leq N$.

$$\text{profit}(t) = \sum_{i=1}^N U_i(t, \mu_i) - \mu_i \times \text{cost}(t).$$

where the utility for customer i at time t is:

$$U_i(t, \mu_i) = \text{bid}_i(\lambda_i(t, \mu_i) - \text{penalty}_i(t, \mu_i))$$

The solution is constrained by the center's maximum capacity

$$\sum_{i=1}^N \mu_i < \mu_{\max}$$

MSRP uses two functions $\text{grow}(i, \text{target})$ and $\text{shrink}(i, \text{target})$. The price μ_i increases (resp. decreases) with shrinks (resp. grows) until $\text{price}_i(\mu_i) = \text{target}$. The details of the optimization method are beyond the scope of this discussion. The contributions of this study can be summarized as follows:

- Incoming requests are directed to a subset of servers, which leads to energy efficiency since low utilization servers are powered off.
- An adaptive policy is specified to dynamically allocate sufficient resources to each service to attain a selected service quality level. The need for overprovisioning is reduced in this manner.
- An economic framework for server resource allocation allows tradeoffs of service quality during shortages. Utility functions capture the key performance criteria embodied in *Service Level Agreements - SLAs* enabling specification of a continuum of SLAs balancing service quality and cost. The price-setting algorithm determines efficient resource assignments by “selling” the available resources at a profit to the highest bidders. The system does not sell idle capacity when it is economically preferable to step down that capacity to reduce costs. Aside from being adaptable to variations in workload it reduced server energy consumption by 29-78% for Web workloads.

7.8 Interplay of energy and performance for RAID running OLTP

Disks consume a major portion of power in OLTP environments, which is the reason this problem is investigated by Sudhanva Gurusurthi et al. at Penn. State U. - PSU in (Gurusurthi et al., 2003c). Three disk array configurations: RAID4, RAID5, and RAID10 are considered in this study. RAID10 is a striped RAID0 array consisting of pairs of mirrored disks. Thus it has four disks in its smallest configuration. The DiskSim simulator is used with traces from two benchmarks: (1) TPC-C is an OLTP benchmark with short transactions. (2) TPC-H is an OLAP - Online Analytical Processing decision support workload consisting of 22 queries.

<http://www.tpc.org/tpcc>

<http://www.tpc.org/tpch>

Four metrics used in this study are:

- (1) Total energy consumption over all requests E_{tot} .
- (2) Average energy consumption in Joules per I/O request E .
- (3) Response time per I/O request T .
- (4) The product $E \times T$.

The single disk power optimizations for laptops and workstations are shown *not* to be very effective for RAID arrays. Tuning RAID parameters (type, number of disks, stripe size etc.) have more impact on the power and performance behavior of these systems. For example the effect of increasing the stripe size from 2 KB to 256 KB for RAID4, RAID5, and RAID10 on energy consumption is shown in Figure 7 in the paper. It can be observed that larger stripe size result in reduced energy

consumption for positioning for TPC-H benchmark, but less so for TPC-C, which is because TPC-H accesses larger block sizes than TPC-C.

7.9 Dynamic speed control for server disk power management

Energy consumption increases with *Rotations Per Minute - RPM*. Such an investigation is reported in (Gurumurthi et al., 2003a,b) by Sudhanva Gurumurthi, Anand Sivasubramaniam, Mahmut T. Kandemir at *Penn. State U. - PSU*, and Hubertus Franke (IBM Research). The power consumption in idle mode for four disks with different RPMs is as follows:

$$(3600, 1.4W), (5400, 10W), (7200, 11W), (15,000RPM, 17W).$$

A linear least squares fit indicates that power consumption increases roughly linearly with RPM.

$$P_{idle} = 0.0013 \times \text{RPM} + 4.158.$$

A quadratic fit for this purpose is less insightful.

The IBM Ultrastar 36ZX consumes 4.15, 22.3, 39 Watts in Standby, Idle, Active states, respectively. A spinup from Standby to Active takes 38.4 watts and 26.5 seconds. A spindown from Idle to Standby takes 15 seconds.

Sudhanva Gurumurthi et al. propose a *Dynamic RPM - DRPM* scheme to adjust disk RPMs dynamically to save power in (Gurumurthi et al., 2003a,b). The *Traditional Power Mode - TPM* is to stop inactive disks, as in the case of MAID, see Section 7.13. Disks can adjust their RPM without spinning down and heuristics to modify the RPM are presented in these papers.

There are currently no disks which can operate at multiple speeds RPMs, i.e., disks rotated at a lower speed are not operational at those speeds. A Hitachi disk can rotate at two speeds is only operational at the higher speed, since heads are offloaded when spinning slower. Technical reasons why disks can operate at only one speed are given here.

<http://forums.storagereview.com/index.php?/topic/27776-variable-rpm-hdds/>

The graphs for a linear and quadratic model are given in Figure 3 to match IBM projections. *Traditional Power Mode - TPM* is compared with DRPM. Savings in idle energy using TPM_{perf} , DRPM_{perf} , which provide the maximum power savings without performance degradation and combined are given in Figure 6 for varying arrival rates using the quadratic power model.

A significant improvement with DRPM_{perf} with respect to TPM_{perf} is observed for a long-tailed Pareto traffic:

https://en.wikipedia.org/wiki/Pareto_distribution

$$f_X(x) = \alpha\beta^\alpha x^{-(\alpha+1)}, \quad x > \beta^\alpha > 0,$$

$$E[X] = \alpha\beta/(\alpha - 1).$$

Extensive simulations with different workload and hardware parameters are reported showing that DRPM can provide significant energy savings without significantly affecting performance. On the other hand the feasibility of DRPM drives is in doubt.

7.10 Approaches to conserve disk energy in network servers

Four approaches to conserve disk energy in high-performance network servers Enrique V. Carrera, Eduardo Pinheiro, and Ricardo Bianchini (now at *Microsoft Systems Research - MSR*) in (Carrera et al., 2003).

- 1. Exploiting idleness:** Powering disks down during periods of idleness is determined not to be appropriate for the environment under consideration in this study, possibly because of very short idle periods.
- 2. Using Lower Power Disks:** Replacing high-performance SCSI - Small Computer Systems Interface disks with three lower power *Integrated Device Electronics - IDE* disks that can achieve the same performance (throughput) and reliability. Three IDE disks are required to achieve the throughput of a single SCSI disk. According to Table 7.3 in (Carrera et al., 2003) higher power disks consume four times the power of lower power disks, so that little is to be gained with this replacement.
- 3. Mirroring with Lower Power Disks:** Heterogeneous disks: a combination of high-performance and laptop disks such that only one of the two sets of disks is powered on at the same time. Bitmaps are used to keep the disks coherent.
- 4. Multispeed Disks:** DRPM - Dynamic RPM allows disks to be slowed down for lower energy consumption during periods of light load. Two levels of disk RPMs are considered in experimental studies with 15K RPM at higher loads.

TABLE 7.3 Main characteristics of two SCSI and an IDE laptop disk.

Parameter	IBM 36Z15 Ultrastar (high perf)	IBM 73ZX Ultrastar (low perf)	IBM 40GNX Travelstar (laptop)
Standard interface	SCSI	SCSI	IDE
Capacity	18 GB	18 GB	18 GB
Number of platters	4	2	2
Rotations per minute	15,000	10,000	5,400
Disk controller cache	4 MB	4 MB	8MB
Average seek time	3.4 ms	4.9 ms	12 ms
Average rotation latency	2 ms	3 ms	5.5 ms
Internal transfer rate	55 MB/s	53 MB/s	25 MB/s
Power (active)	13.5 W	9.5 W	3.0 W
Power (idle)	10.2 W	6.0 W	3.00 W
Power (standby)	2.5 W	1.4 W	0.25 W
Energy (spin down)	13.0 J	10.0 J	0.4 J
Time (spin down)	1.5 s	1.7 s	0.5 s
Energy (spin up)	135.0 J	97.9 J	8.7 J
Time (spin up)	10.9 s	10.1 s	3.5 s

The fourth approach is shown to provide energy savings of up to 23% for network servers, without performance degradation. It should be noted that disks can be accessed only at the highest RPM and hence a spin-up from the lower speed is required.

In a similar vain a study concerned with storing high volumes of data from high-energy physics experiments used *Enhanced Integrated Drive Electronics - EIDE* disk drives for this purpose was conducted by D. A. Sanders et al. (FermiLab, Chicago, IL) (Sanders et al., 2002), since EIDE drives are three times less expensive than faster SCSI drives. The fairly secure RAID5 array served as a cache to an EMASS robot tape for hard to reproduce experimental data.

7.11 Energy efficiency through burstiness

Redesigning OS policies to increase burstiness to improve storage energy savings is proposed at U. Rochester in (Papathanasiou and Scott, 2003). Burstiness is achieved by bunching disk accesses in time. The deliberately created bursty activity can save up to 78.5% of disk energy, while decreasing disk congestion and spin-up latency.

Four disk power states are considered in this study:

Active: processing disk accesses.

Idle: with disk still spinning, but the electronics partially unpowered and heads withdrawn.

Standby: with disks spun-down.

Sleep: all remaining electronics is powered off.

Synchronous requests from interactive applications are dealt with by prefetching. Bursty operates in two phases: prefetching and idleness. The following tasks are carried out by the system.

1. Flushing dirty buffers.
2. Predicting future accesses, which may use access hints, see e.g. (Patterson et al., 1995).
3. Computing the amount of memory available for prefetching.
4. Freeing memory by flushing dirty pages.
5. Prefetch data with the goal of maximizing the time to next miss.

Disk energy savings increases with total memory size as shown for two experimental workloads. Bursty-MPEG attains 80% and bursty-combined attains $\approx 60\%$ energy savings.

7.12 Dempsey: a tool for modeling hard disk power consumption

Dempsey is a disk simulation tool developed by a team at Princeton U. to model disk power consumption, which is especially important in mobile environments (Zedlewski et al., 2003). Dempsey extracts performance and power consumption parameters from a given disk drive using stimulus-based measurements to extract disk parameters, with no need to manufacturer specifications.

TABLE 7.4 Parameter values for the three alternative models (Zedlewski et al., 2003).

Parameter	Size	Pactive	Pactive-idle	Psleep	T_0
IBM Microdrive	1 GB	624 mW	531 mW	61 mW	2 s
Toshiba HDD	5 GB	1186 mW	891 mW	231 mW	5 s

Dempsey is compared against three alternative models using the parameters specified in Table 7.4. The 2-parameter model is based on two assumptions:

- (1) the disk consumes energy at a constant rate when processing disk requests,
- (2) the disk immediately enters the sleep mode when there are no more requests.

$$E_{total} = E_{active} + E_{idle} = P_{active}T_{active} + P_{sleep}T_{idle}. \quad (7.1)$$

The 3-Parameter model improves upon the 2-Parameter model by further refining idle periods. Eq. (7.1) is used to estimate the total energy consumption and $E_{active} = P_{active}T_{active}$. The behavior during the idle periods assumes that the disk enters an intermediate power-mode, called the “active-idle” mode, as soon as it finishes a disk request and finds no more pending requests in the queue. The transition from the active-idle-mode to the sleep-mode, however, is governed by a fixed waiting threshold T_0 . For an idle-period of length L with $L \leq T_0$, the energy consumption is estimated as $P_{active-idle}L$. For an idle-period of length L with $L > T_0$, the energy consumption is estimated as:

$$P_{active-idle}T_0 + P_{sleep}(L - T_0).$$

Note that the time or energy overheads of the mode transitions are not modeled. $P_{active-idle}$ is measured by observing the disk immediately after a disk request has finished.

The Coarse-Dempsey model is a hybrid between the 3-Parameter model and Dempsey. It uses Eq. (7.1) to estimate total energy consumption. E_{active} is estimated as in the 3-Parameter model, whereas E_{idle} is estimated by Dempsey. Therefore, this model uses the complete Idle-Period Energy Profile of the disk, which also models the time and energy overheads of mode transitions.

Eight traces are used with the Actual, 2 parameter, 3 parameter, course Dempsey, and Dempsey Dempsey can estimate power consumption with high accuracy as demonstrated in tables for several disks with multiple traces.

7.13 MAID - Massive Arrays of Idle Disks alternative to tape storage

MAID was proposed by Dennis Colarelli and Dirk Grunwald at U. Denver in 2002 (Colarelli and Grunwald, 2002), 100 TB tape libraries were preferred over HDD storage at that time. Assuming a 24×7 data center operation and 7.25 cents per KWH (one KWH is in the range 10-20 cents across US at this time), it would annually cost \$9400 to power the tape library and \$91,500 to power HDDs, which is tenfold. However, the disk array would provide a much higher aggregate bandwidth than the tape library, plus tape libraries provide an unacceptably slow access time for many applications.

The cost of commodity disk drives is rapidly dropping, but disks use power even when they are idle, i.e., not being accessed. MAID is a solution to this problem and achieves 1/15th of the power of a RAID system, which is powered up all the time.

A MAID configuration for caching is shown in (Colarelli and Grunwald, 2002). External requests are processed by a virtualization manager, which sends requests to active or passive drives. A subset of the disks controlled by the cache manager are active, while the majority of disks are designated as passive and are spun down. Only RAID0 configuration (with no redundancy) are considered in this study. Power consumption and the mean read response time is reported for different configurations.

IBM 60GXP drives used in the study and subjected to file level traces described in (Ruemmler and Wilkes, 1993). The simulated disk array has 1120 TB of storage: 7,000 disks 160 GB each. The three parameters in the simulation study are:

Power Management Policy: The fixed policy is based on nonactivity set to 512 seconds. The adaptive policy uses a moving-average estimator for the idle period before spindown.

Data Layout: Linear and striped data layout are considered. The strip size is one MB and the stripe width is ten drives.

Cache Use: A cache parameter which is set to 1% determines the number of caching disks.

Nine configurations are considered:

1: Baseline, always on using 100% of power.

2-9: Linear/simple fixed/adaptive, no cache/cache.

The latter configuration uses 15% - 16% of baseline power and their mean response time varies between 650-720 ms.

7.13.1 The Copan MAID storage system

Copan Corp. was a startup to commercialize the MAID concept proposed in (Colarelli and Grunwald, 2002). Its goal was to provide archival storage with the following priorities according to (Guha, 2004):

1. I/O requests to large blocks.
2. Performance measured in data rate: MB/sec rather than I/Os/sec.
3. Time to first byte.

Copan's architecture is different from traditional storage architectures in the following manners.

1. A large shared cache is used for reads, but not for writes (dirty blocks), since the system is not used for OLTP.
2. High-speed non-blocking switches are not required, since the system can tolerate delays.
3. Given that a small fraction of data is accessed, a majority of disks could be taken off-line.
4. No need to limit storage capacity based on interconnection bandwidth, since disks have a very low utilization.
5. High data availability is required for the high volume of data at hand.

Copan's design guidelines are as follows:

- Parity based redundancy (one disk out of N disks dedicated to parity).
- Optimize tradeoffs between power cycling (turning power off/on), storage density, performance, redundancy and cost.
https://en.wikipedia.org/wiki/Power_cycling
- Ensure data protection and performance with drives power-cycling. 30-50% online drives imply ten times more data online than tape systems.
- Ensure I/O rate and parity protection during power cycling.
- Ability to scale the bandwidth.

Disk drive placement in three dimensions is shown in (Guha, 2004), so that the total number of disks is $N = s \times t \times d$, where s is the number of shelves, t is the number of columns of disks, and d is the number of disks in each column.

Copan did not require all the drives in the parity group to be powered on. As shown in Figure 2 in (Guha, 2004) as data writing starts there is on demand powering up of disks holding data and parity blocks. Powering disks off and on lowers their MTTF, so that the number of such transition is monitored so as not to exceed vendor-specified limits.

Copan's financial problems led to its acquisition by *Silicon Graphics Inc.* - SGI in 2010.

<http://www.enterprisestorageforum.com/continuity/news/article.php/3303551/Copan-Does-Disk-for-the-Price-of-Tape.htm>

At that time COPAN 400 was up to three times the density of competitive solutions, such as EMC's Centera, IBM's Information Archive, and Dell's PowerVault DL2100, which cost from 3.5 to 7 times more.

https://www.theregister.co.uk/2010/06/09/sgi_copan_400/

SGI's iteration of the product featured double density: 1.8 PB per cabinet and 23 TB/hour at full bandwidth. Spin-down was a standard feature of EMC storage products: Centera, CLARiiON, and Atmos. SGI was acquired by *HP Enterprise - HPE* in 2016 for \$275 million.

<https://venturebeat.com/2016/08/11/hewlett-packard-enterprise-acquires-sgi-for-275-million/>

7.13.2 Nexsan AutoMAID

Characteristics of Nexsan are given here

<http://www.nexsan.com/wp-content/assets/data-sheets/E-Series--Datasheet.pdf>

There are 5 levels of power savings that AutoMAID supports in the E-Series

<http://www.nexsan.com/wp-content/uploads/whitepapers/automaaid.pdf>

Level 1: Heads parked/unloaded Sub-second recovery time. About 15% to 20% savings

Level 2: Heads unloaded (slows to 4000 RPM) 15 second recovery time. About 35% to 45% savings

Level 3: Stops spinning (sleep mode; powered on) 30 to 45 second recovery time. About 60% to 65% savings

Level 4: Stops drive electronics 30 to 45 second recovery time. About 70% savings

Level 5: Turn off E60X expansion enclosure 30 to 46 second recovery time. About 87% savings, with 15K SATA about 92%

7.14 Self-tuning power aware storage cache replacement algorithm

The effectiveness of disk power management schemes is limited by the fact that the average idle periods for data center server disks are small compared to spindown/spinup times. Several power-aware storage cache management algorithms to save energy are proposed by Qingbo Zhou, Yuanyuan Zhou (UIUC, UCSD), and Pei Cao (Google) at UIUC in (Zhu et al., 2004a). More specifically the *Offline Power-aware Greedy - OPG* algorithm is more energy-efficient than Lazslo Belady's algorithm (Belady, 1966). This algorithm minimizes cache misses by replacing a page to be referenced furthest in the future. Although Belady's algorithm is infeasible it has inspired feasible replacement algorithms. An example showing that Belady's algorithm is not energy optimal is given in Table 7.5 (Jain and Lin, 2016).

TABLE 7.5 Belady's algorithm versus a more energy efficient alternative. The cache size is 4 blocks and disk spin down takes 10 time units.

Time	Req	Belady		Alternative	
		Cache	Hit?	Cache	Hit?
0	A	[A,-,-,-]	N	[A,-,-,-]	N
1	B	[A,B,-,-]	N	[A,B,-,-]	N
2	C	[A,B,C,-]	N	[A,B,C,-]	N
3	D	[A,B,C,D]	N	[A,B,C,D]	N
4	E	[E,B,C,D]	N	[A,E,C,D]	N
5	B	[E,B,C,D]	Y	[A,B,C,D]	N
6	E	[E,B,C,D]	Y	[A,E,C,D]	N
7	C	[E,B,C,D]	Y	[A,E,C,D]	Y
8	D	[E,B,C,D]	Y	[A,E,C,D]	Y
	...	...	...	...	...
16	A	[A,B,C,D]	N	[A,E,C,D]	Y

The number of misses resulting from a storage cache replacement algorithm affects disk energy consumption, but it is not the number of misses, but rather their dispersion that prevents spindown as illustrated below. For the Belady algorithm the disk power state is ON (0-14) and (15-26), while for alternative ON (0,16).

A follow-up study for online *Power-Aware - PA* cache replacement algorithm: PA-LRU keeps blocks from "inactive" disks in the storage cache longer to extend the idle periods for those disks (Zhu et al., 2004a). Energy consumed by such disks is reduced, since they stay in lower power mode longer and spin up and down fewer times. Misses to "active" disks are acceptable since accessing active disks is inexpensive, allowing overall storage energy consumption can be reduced. PA-LRU saves 16% compared to LRU and provides 50% better average response time for OLTP I/O workloads.

Partition-Based - PB-LRU is another power aware cache replacement algorithm proposed in (Zhu et al., 2004b). The cache of size S is subdivided into partitions of size s_i for disk $1 \leq i \leq n$ with $\sum_{i=1}^n s_i = S$. Each partition is managed with by an LRU replacement policy. The partitioning intends to minimize the total energy consumption over all disks based on the observation that the workload is not equally distributed over the disks. The cache misses going to each disk are easier to control if the disk caches are treated separately. In order to find an energy-optimal partitioning, the energy that would be consumed with different partition sizes is estimated. These estimates are then used to find a partitioning that will minimize the total energy consumption of all disks with this constraint.

A robust on-line storage cache replacement algorithm partitions the total system cache amongst the individual disks. Key to this algorithm was the dynamic estimation of the energy that would be consumed by a disk had the partition size assigned to it been different. Once these estimates are available, the optimal partitioning is obtained by reducing the problem to the *Multiple Choice Knapsack Problem - MCKP*, which is NP-hard (Martello and Toth, 1990), but can be solved in pseudo-polynomial time using dynamic programming which was described in a 1952 article by Richard Bellman in *Proc. Nat'l*

Academy of Science, but covered in numerous books (Bellman, 2003), (Dreyfus and Law, 1977) and (Cormen et al., 2009). https://en.wikipedia.org/wiki/Dynamic_programming

The time complexity of the problem is $O(nm^2)$, where n is the number of disks and m is the number of potential partitions. Let $K(i, s)$ ($s \in \{p_1, p_2, \dots, p_m \leq S\}$) be the energy consumed by i disks when a total cache of S is partitioned among those disks.

$$K(0, s) = \begin{cases} 0 & \text{if } s = 0 \\ \infty & \text{otherwise} \end{cases}$$

$$K(i, s) = \begin{cases} \min_{\{j|0 < p_j \leq s\}} \{K(i-1, s-p_j) + E(i, p_j)\} & \text{if } s > 0 \\ \infty & \text{otherwise} \end{cases}$$

$$K^* = \min_{\{s|0 \leq s \leq S\}} \{K(n, s)\}$$

where K^* gives the minimum energy consumption when a storage cache of size S is partitioned among n disks.

A DiskSim simulator extended with a power mode, similarly to (Gurumurthi et al., 2003a) is used to estimate power consumption. Energy consumption with an infinite cache, LRU, PA-LRU, and PB-LRU are also compared in this paper.

7.15 Popular Data Concentration - PDC

The *Popular Data Concentration - PDC* is an energy conservation technique proposed by Eduardo Pinheiro and Ricardo Bianchini at Rutgers U., based on migrating frequently accessed data to a subset of disks, while other disks are allowed to operate in low power mode (Pinheiro and Bianchini, 2004).

When considering disk arrays composed of conventional disks, the *Fixed-Threshold - FT* simply spins down a disk after it has been idle for a fixed period. The idleness threshold is the amount of time for which the energy consumed in idle state is the same as that of powering the disk down and later powering the disk backup.

TABLE 7.6 Summary of simulator validation, EO = energy oblivious, where disk are never powered down, FT = fixed threshold. Energy values are Joules.

Description	Total energy	Files moved	MB moved	Spindown/up	Delayed requests
	Sim/Real/Error	Sim/Real	Sim/Real	Sim/Real	Sim/Real
PDC	152,023/172843/12%	9715/10943	227/283	336/326	1.3%/1.2%
MAID	141,349/156.335/9.6%	8738/8716	204/204	300/304	1.1%/1.0%
FT	190,374/200479/5.0%	n/a	n/a	10/13	0.2%/0.2%
EO	190.346/200833/5.2%	n/a	n/a	n/a	0.0%/0.0%

PDC is compared with other methods in Table 7.6. A comparison of PDC with MAID on a file server leads to the observation that MAID can only conserve energy when the server load is extremely low. With two-speed disks specified in Table 7.7, both PDC and MAID can conserve significant energy, with a small fraction of delayed requests. In spite of the fact that MAID outperforms PDC in preliminary simulation results, more detailed studies reported in the paper lead to the conclusion that PDC is superior to MAID.

7.16 Disk layout optimization for reducing energy consumption

Excessive power consumption is becoming a major barrier to extracting the maximum performance from high-performance computers. Given that disk systems in such computers contribute heavily to power usage they are targeted for optimization by modifying the disk access patterns by profiling disk accesses as discussed in a study conducted at PSU reported in (Son et al., 2005).

This study determines the start disk from which the data is striped, the striping width or the number of disks over which the data is striped, and the stripe unit size. Experimentation is carried out by modifying SPEC96 Floating Point Arithmetic benchmarks to operate on disk resident data. The following schemes are considered.

TABLE 7.7 Main characteristics and simulated power, energy, and time for the two speed disk in (Pineiro and Bianchini, 2004). The high speed of the Cheetah disks were scaled down to produce values for low speed disks. Speed transition costs set to half of cost in the Cheetah disk. Disk power is proportional to the square of its speed.

Description	Value
Storage capacity	9.17 GB
Avg seek time	5.4 msec
High to low speed transition	14.13 joules/5.62 secs
Low to high speed transition	32.96 joules/3.06 secs
High/low rotational speed	10K/3K RPM
Avg rotation high/low speed	3/10 msec
Transfer rate high/low speed	31/9.3 MB/sec
Idle power at high/low speed	5.26/2.17 Watts
Active energy at high/low speed	61/43 mJoules/8 KB

BASE: There is no energy saving strategy and the best execution time across all tested schemes is attained, since it does not incur any performance penalty due to power management. A fixed 64 KB strip size is used for all arrays, and stripe width equals the number of available disks in the system. All the energy and performance results for presented schemes are with respect to this version.

HardWare-HW: This is a pure hardware based conventional power management scheme based on the by Fred Dougliis et al. (Dougliis et al., 1994, 1995), which is discussed in Section 7.3. The same layout as the BASE scheme is followed. A disk is spun down after some idleness to save power and is spun up when a new request arrives. Since the performance cost of spinning up is typically large, this approach can incur significant performance degradation. In order for this scheme to save power, the disk idleness should be long enough to compensate for the spin-up cost.

OPT: This is the profile-driven disk layout detection scheme proposed in the paper, which determines the start disk, stripe unit (strip) size, and stripe factor (width) for each disk-resident array. Also implemented are three more restricted variants of this optimized scheme with the goal to quantify the influence of each of the three components of a disk layout individually.

OPT-1: Each array gets striped from the first disk to determine the stripe size and stripe factor under this constraint.

OPT-2 The stripe unit size is fixed at 64 KB for all arrays, with the goal to determine the start disk and stripe factor.

Opt-3 The stripe factor is set to the total number of disks in the system, which is eight for all arrays, and determine the start disk and the stripe unit.

LOOP: This scheme uses the same fixed layout for each array as in the case of the BASE and HW schemes, but restructures code to increase disk idle periods. The specific transformation used is loop distribution or loop fission (Wolf, 1995), which places the statements in a given loop into separate loops. One can expect this transformation to be useful from a disk energy viewpoint, in particular, in cases where it separates the references to different arrays, thereby minimizing the number of disks that need to be activated for a given loop.

OOP+OPT: The combined scheme first determines the disk layouts using OPT, but later identifies the loop nests that need to be restructured, so that they can be transformed to work well with the determined layouts. This version brings additional benefits over to the OPT scheme.

The experiments show the entire suite of Spec95 floating-point benchmarks that are modified to operate on disk-resident data is very effective in reducing disk energy consumption. The results also show that the performance degradation caused by the approach is very small.

7.17 Managing server energy and operational costs in hosting centers

Energy conservation in cloud servers is considered at *Penn. State U. - PSU* in (Chen et al., 2005). Three solution schemes based on queueing analysis (Kleinrock, 1975), feedback control theory (Doyle et al., 2009), and a hybrid mechanism based on both methods are considered.

Power savings are attained by powering down blade servers, but the downside is that powering up servers incurs additional costs, since:

- (i) servers cannot serve requests while being booted up,
- (ii) increased wear-and-tear resulting in reduced time to failure plus the cost of replacements.

Dynamic Voltage and Frequency Scaling - DVFS is another power management scheme considered in this study. The dynamic power consumed is proportional to the cubic power of the operating clock frequency. Slowing down the clock allows the scaling down of the supply voltages for the circuits, resulting in power savings.

CPUs consume most of the power in a blade server, as much as 75-150 Watts for the Intel Xeon at full speed. DVFS control can provide substantial power savings, since

$$P = P_{fixed} + P_f f^3.$$

While DVFS implementation for a server cluster can be completely decentralized, coordinated/centralized control has been shown by E. N. (Mootaz) Elnozahy to provide substantially higher savings (Elnozahy et al., 2002b).

Application $1 \leq i \leq N$ is assigned m_i servers, so that $\sum_{i=1}^N m_i \leq M$, where M is the total number of servers, some of which may be powered down to save energy. Servers can run between a minimum and maximum frequency. With a voltage V the dynamic power consumption is $V^2 \times f$ and the relationship between V and f can be expressed as $V \propto f$. Given in (Sharma et al., 2003) are the Frequency and Voltage levels for a certain microprocessor as given in Table 7.8.

TABLE 7.8 Frequency and voltage levels available with AMD Athlon. Powernow drivers are used to scale the voltage.

Frequency (MHz)	1529	1197	798	665	532
Voltage (volts)	1.55	1.35	1.2	1.2	1.2

The energy consumption is

$$E = M \int_t P \times dt = M \times (P_{fixed} + P_f f^3)t.$$

M and f can be controlled to reduce power consumption. Let $m_i(z)$ denote the number of servers allocated to application i in time interval z . The total power consumption is given as:

$$\sum_{z=1}^Z \sum_{i=1}^N m_i(z) \times (P_{fixed} + P_f \times f_i(z)^3) \times t.$$

This should be multiplied by the cost of electricity per KW. Added to this is the cost of reboots, which is the objective function to minimize, under the constraint for SLA, frequencies, and the number of servers.

A comparison of most related studies to this work is given in Table 7.9.

TABLE 7.9 Comparing the most related studies to this work. Reboot cost specified as time does not take into account effect on MTBF.

Study	DVS	Multiple servers	Multiple applns	SLA	Reboot cost
Duke (Chang et al., 2008)	No	Yes (5)	Yes (2)	Yes	Yes (time)
Rutgers (Pinheiro et al., 2003)	No	Yes (8)	No	No	Yes (time)
Virginia (Sharma et al., 2003)	Yes	No	No	No	No
IBM (Elnozahy et al., 2002b)	Yes	Yes (10)	No	No	No
Current (Doyle et al., 2009)	Yes	Yes	Yes	Yes	Yes

Queueing Theory Based Approach: The queuing approach is predictive in that it predicts workload behavior in the near future and tunes server allocation and frequency setting appropriately. This approach may not be adaptive enough for fine-grained transient behavior. There are N applications which are served by multiserver queues, m_i is the number of servers and f_i the frequency, which determines the server speed. The arrival rate of requests for application i is $\phi(i)$.

$c_a^2(i)$ and $c_s^2(i)$ are the coefficient of variation of interarrival time and file size, which is related to file access time which equals file size divided by data transfer rate. The bandwidth of server i is βf_i , where β is obtained by measuring HTTP requests of different sizes. The server utilizations are $\lambda(i)\phi(i)/(m_i\beta f_i)$ and $\alpha_{m_i} = \rho_i^{(m_i+1)/2}$. Using an approximation given in (Kleinrock, 1976), (Bolch et al., 2006) for G/G/m queues the mean response time for application i can be expressed:

$$R_i \approx \frac{\phi(i)}{\beta \times f_i} + \frac{\alpha_{m_i}\phi(i)}{\beta \times f_i} \left(\frac{c_a^2(i) + c_s^2(i)}{2m_i(1 - \rho_i)} \right).$$

Control Theoretic Approach used in the study is a simple feedback loop. It is reactive since it bases its decisions on feedback. The difference between the values of target response times and those determined by a simulator is the tracking error. These are used to control of aggregate frequency and server allocation.

Hybrid Approach: The hybrid scheme uses the predictive information of the queuing approach to determine server allocation at granularity (T) and the feedback-based control theoretic approach for frequency setting at granularity of $t < T$.

Results for $N = 30$ applications and $M = 285$ servers are given in Table 7.10 for representative (T, t) granularities. The Queuing approach is more viable at the larger time granularity, but inaccurate at finer granularity. In fact, these inaccuracies at fine granularities result in poor choice of server allocation for the Hybrid scheme at (10, 5). However, at the (60, 5) granularity, the Hybrid scheme is again able to benefit from the proactive prediction of Queuing and the Feedback control for frequency modulation, giving the lowest response times.

TABLE 7.10 Results for workload with 30 applications and 285 servers, based on Table 5 in (Chen et al., 2005). Only costs with a “v” are viable. Cost in cents and RT = response time in ms.

(T,t)	Schemes	Cost	RT	Reboots
(60,60)	Queueing	2625.2v	5.5	275
	Control	2618.7v	6.4	215
	Hybrid	2619.7v	5.6	275
(60,5)	Queueing	2465.1	35.4	275
	Control	2579.6v	6.5	213
	Hybrid	2612.3v	5.4	275
(10,5)	Queueing	2435.5	73.4	342
	Control	2628.7	7.2	323
	Hybrid	2948.4	30.8	342

7.17.1 Improving energy savings while meeting performance goals in RAID

Energy consumption at data centers is heavily contributed by disk arrays. Existing solutions to preserve energy have yielded limited energy savings or significantly degrade data center workload performance.

Hibernator was a study at HP Labs conducted by UIUC students Qingbo Zhu et al. under the direction of Yuanyuan Zhou (UIUC, UCSD), Kimberly Keeton, and John Wilkes (HP, Google) at HP (Zhu et al., 2005). It is a disk array energy management system that improves energy savings, while meeting performance goals. Several techniques are combined to achieve this as follows:

1. Disks can spin at different speeds.
2. Setting the disk speed at a coarse level.
3. Efficient ways to migrate the right data to an appropriate-speed disk automatically.
4. Automatic performance boosts in case performance goals are not being met due to disk energy management.

The Hibernator design was evaluated using trace-driven simulations and a hybrid system comprised of an IBM DB2 server and an emulated storage server with multi-speed disks, since such disks are not available. The Hibernator exhibits an energy saving of 65% while satisfying performance goals, which is six-fold better than previous solutions. In the case of an OLTP workload Hibernator saved 29% of the energy, while matching the txn rate of a RAID5 with no power management.

7.18 Performance directed energy management for main memory and disks

Energy management for main memory and disks was studied by Yuanyuan Zhou and Sarita Adve and their students at UIUC (Li et al., 2005). Control algorithms are used to dynamically transition devices to low power modes after they are idle past a certain length of time. Control algorithms in previous studies are stated to have two major limitations.

- Difficult application-dependent manual tuning of thresholds is required to achieve energy savings without significantly degrading performance.
- There are no performance guarantees.

The following contributions are made in this study to make memory/disk energy-saving schemes practical enough to use in real systems is addressed in this work.

1. Provides performance guarantees for control algorithms and does well for all tested cases.
2. *Performance-Directed Dynamic - PD* method dynamically adjusts its thresholds periodically, based on idleness and workload characteristics. PD consumes the least energy for memory, but does not outperform tuned algorithms for disks.
3. PD is improved by a simpler, optimization-based, threshold-free control algorithm: *Performance-Directed Static - PS* method periodically assigns a static configuration by solving an optimization problem. PS is the best or close to the best across all performance-guaranteed disk algorithms.
4. A hybrid scheme that combines PS and PD algorithms further improves energy savings.

The dynamic algorithm for disks requires the tuning of five parameters:

- (1) checking the period p to examine the disk queue length.
- (2) the upper tolerance (UT) in percentage response time changes to spin up a disk to a higher RPM.
- (3) the lower tolerance (LT) in percentage response time changes to spin down a disk.
- (4) window size W .
- (5) the disk queue length threshold N_{min} .

If the disk queue length is less than a threshold N_{min} , the disk can spin down to a lower RPM, but not lower than a Low Watermark, i.e., a lower bound for rotation rate.

Low watermark is adjusted by monitoring the response time change ΔT_{resp} .

If ΔT_{resp} over the last two W - request windows is:

- larger than UT, then force the disk to the full speed immediately by setting Low watermark to the full RPM;
- between LT and UT, the controller keeps Low watermark;
- less than LT, Low watermark is decreased proportionally based on how much the response time change is lower than LT, i.e., $1 - \Delta T_{resp}/LT$).

7.19 Exploiting redundancy to conserve energy in storage systems

Diverted Accesses - DA paradigm was proposed by researchers at Rutgers U. cooperating with NEC Labs to process reads at the original disk, but divert them to a redundant disk when the original disk is heavily loaded (Pinheiro et al., 2006). Redundant disks are kept in low power mode under light loads to preserve energy. The following energy conservation techniques are considered in this study.

Cezary Dubnicki (U. Rochester) was department head at NEC till 2007, where he led the development of the HYDRAS-tor storage platform (Dubnicki et al., 2009). In 2008 he founded 9livesdata.com in Warsaw, Poland, which continues the effort towards productization.

Fixed Threshold - FT: A disk is transitioned to low power mode after break-even time $T = E_t/(P_h - p_\ell)$, where E_t is energy to power down and up again and P_h and P_ℓ are the power in high and low power mode. The analysis is straightforward and omitted here for the sake of brevity.

Massive Array of Idle Disks - MAID: D_{MAID} extra disks are used to cache recently-accessed files. Upon each block request, if the block is not cached, it is accessed from non-cache disks and also copied to one of the cache disks. To avoid high latencies, requests may bypass the cache disks during periods of high load. Thus, the maximum throughput of a MAID system is that of total number of disks: $N + D_{maid}$ disks times $P_{hit-rate}$. Modeling MAID under light and moderate loads requires knowledge of the temporal locality of accesses to files, at least the miss rate m_{maid} of the cache disks. This is used to estimate the idle times of the cache disks and the non-cache disks. To conserve energy cache disks can be allowed to accumulate the writes to non-cache disk until a miss which will require a powered down

disk to be powered up. At that point, the accumulated writes can be performed on the non-cache disk. This approach promotes energy conservation at the cost of lower reliability. MAID is further discussed in Section 7.13.

Popular Data Concentration - PDC: Like FT - fixed threshold and MAID the original PDC (Pinheiro and Bianchini, 2004) did not consider redundancy explicitly. Reliability will be reduced if it is applied to all fragments arbitrarily, but this problem is avoided if data is migrated in such a way that the n fragments of each block remain on different disks. Over time, the most popular fragments would then be stored on the first set of n disks, the second most popular fragments would be stored on the second set of n disks, and so on. To model PDC, we need to introduce the notions of disk popularity and file system coverage. Previous research has shown the applicability of Zipf's power law, which states probability of a file with rank r being accessed is proportional to $(1/r)^\alpha$, where α is the degree of skewness (Zipf, 1949), (Knuth, 1997a). Popularity is used to take into account disk idleness approximately.

Diverted Accesses (DIV): The original data is on D disks and the redundant data on $R = N - D$ disks. The redundant disks can remain in low-power mode until they are required for reliability or to provide higher bandwidth for higher loads. Idle times are short under high load, leading to an average power consumption of $N P_h$. Modeling DIV under light and moderate loads is provided in the paper.

PDC+DIV: Refer to (Pinheiro et al., 2006).

MAID+DIV: Refer to (Pinheiro et al., 2006).

The latter five techniques use FT as their low-level power-management technique. We compute the energy savings of each technique with respect to an *Energy-Oblivious - EO* system that keeps all disks at P_h (high power mode) at all times.

PDC benefits significantly from smaller coverage and becomes the best technique for very low coverage and high request rates. PDC+DIV also benefits from smaller coverage, which is the percentage of blocks referenced (read or written) assumed to be 70% and the request rate is 64 requests per second, but the improvements over DIV are small compared with the benefits a smaller coverage has on PDC alone. Note also that the savings potential of data migration is small, since the savings of PDC+DIV with 90% coverage are very similar to those of DIV.

7.20 Thermal disk drive design: challenges and possible solutions

Designing disks to operate within the thermal design envelope is critical for reliable operation according to Dave Anderson, John Dykes, Erik Riedel (Seagate, Dell/EMC, ITRenew) at Seagate, since high temperatures can cause various reliability problems such as offtrack writes resulting in corrupted written data or even head crashes (Anderson et al., 2003). Solutions based on a more powerful cooling systems are ruled out, because they are prohibitively expensive.

To avoid thermal emergencies *Dynamic Thermal Management - DTM* was suggested by Sudhanva Gurumurthi, Anand Sivasubramaniam, and Vivek K. Natarajan at Penn. State U. - PSU in (Gurumurthi et al., 2003d). Disks are allowed to serve I/O requests as usual, but if there is an imminent danger of violating the thermal envelope, drive activities are dynamically modified to prevent the occurrence of such a situation. Questions to be answered in this context are:

1. For a disk operating at a given RPM, what is the effect of seeks on temperature and the mean time between seeks to remain within the thermal envelope.
2. Within a seek what is the impact of different phases - acceleration, coasting, deceleration on temperature?
3. Can we modulate the head scheduling or request service schemes for DTM? Given different DTM alternatives, how do we pick one over another for a given set of workload conditions and disk drive parameters?

A detailed understanding of the interaction between workload activities and disk drive parameters, and their impact on temperature, requires detailed toolsets not available when the study reported in (Gurumurthi and Sivasubramaniam, 2006) was conducted. A simulator consisting of a performance and a thermal model was developed based on DiskSim (Ganger et al., 1999) for performance and for the thermal model (Eibeck and Cohen, 1988).

The sources of heat in disk drives are the spindle motor, which rotates the platters and the *Voice-Coil Motor - VCM* which moves the disk arm. Finite difference equations are used to calculate the heat flow, and iteratively calculate the temperatures of these components at each time step, until it converges to a steady state temperature using a time-step simulator.

The first infrastructure for integrated studies of the performance and thermal behavior of storage systems is described by Youngjae Kim (*Texas Instruments - TI*, LG Electronics), Sudhanva Gurumurthi, and Anand Sivasubramaniam at PSU in (Kim et al., 2006). Microbenchmarks followed by real workloads are used to gain insight into how I/O characteristics can affect the temperature of disk drives and to identify simple knobs for temperature optimization. Knobs are identified in running real workloads.

Presented are detailed models capturing the close relationships between capacity, performance, and thermal characteristics over time. The need for continued improvements in *Internal Data Rate - IDR* is motivated by showing that the response

times of real workloads can be improved by 30-60% with a 10K increase in the RPM for disks used in their respective storage systems.

7.21 PARAID: the gear-shifting Power-Aware RAID

The *Power Aware RAID - PARAID* was developed by Charles Weddle and An-I Andy Wang at *Florida State U. - FSU*, Peter Reiher (UCLA) and Geoff Kuenning (Harvey Mudd College) in (Weddle et al., 2007). PARAID takes advantage of variations in system load to save power by spinning down disks. In the higher gear we have a RAID5 array with five disks. At a lower gear the fifth disk in Fig. 7.1 is spun down and the data and parity blocks from that disk are moved to other disks.

+	Disk 1	Disk 2	Disk 3	Disk 4	Disk 5
Gear 1	(1-4)	8	12	((1,4),8,12)	
RAID-5	16	20	(16,20,-)		
	1	2	3	4	(1-4)
Gear 2	5	6	7	(5-8)	8
RAID-5	9	10	(9-12)	11	12
	13	(13-16)	14	15	16
	(17-20)	17	18	19	20

FIGURE 7.1 PARAID disk layout in gear 2 is a RAID5 with five disks. In gear 1 we have a RAID5 with four disks. Before disk 5 is spun down its blocks are copied to the remaining four disks. Parities are given in parentheses.

Several benchmarks were run on the five disk PARAID prototype to measure power consumption. Up to 34% less power consumption is reported with respect to conventional RAID5 with a similar performance and reliability. Spinning down one of five disks is expected to result in 20% reduction in power consumption, so that further spindown is required to achieve this lower power consumption.

7.22 DiskGroup: energy efficient disk layout for RAID1 systems

The potential for energy savings over simple RAID1 mirroring is explored by Lanyue Lu, Peter Varman (Rice U.), and Jun Wang at *U. Central Florida - UCF* in (Lu et al., 2007).

Eight primary and eight secondary disks are considered. In the case secondary disks are organized in one group or $g = 1$ the first eight elements on the first disk: A1:8 are placed across the first row of eight secondary disks. This pattern is repeated for the next seven primary disks by placing B1:8, C1:8, D1:8, E1:8, F1:8, G1:8, H1:8. This placement has the advantage that the failure of a primary disk results in a balanced load increase on secondary disks. The reader should note the similarity to *Group Rotate Declustering - GRD* in (Chen and Towsley, 1996), which was presented in Chapter 4.

eRAID activates secondary disks by placing data on groups of disks, so that fewer disks are activated due to overload or disk failure. The case $n = 8$ disks, $g = 4$ groups, and $s = 2$ disks per group, so that $n = sg$ is given in Figure 6 in (Lu et al., 2007).

M1-M2	M3-M4	M5-M6	M7-M8
A1 A2	C1 C2	E1 E2	G1 G2
A3 A4	C3 C4	E3 E4	G3 G4
A5 A6	C5 C6	E5 E6	G5 G6
A7 A8	C7 C8	E7 E8	G7 G8
B1 B2	D1 D2	F1 F2	H1 H2
B3 B4	D3 D4	F3 F4	H3 H4
B5 B6	D5 D6	F5 F6	H5 H6
B7 B8	D7 D6	F7 F8	H7 H8

FIGURE 7.2 $n = 8$ secondary disks are partitioned into $g = 4$ groups with $s = 2$ disks per group.

Secondary disks are spun up when primary disks are overloaded or fail. It is easy to see that the failure of the first primary disk will result in the activation of only two disks: M1 and M2, which hold its data in Fig. 7.2.

A combinatorial analysis is provided to obtain upper bounds for the number of disks to be spun up for varying parameters. A simulation study is also conducted that varies the number of groups (g), the number of overloaded disks, and the amount of overload for each disk. Simulation is used to obtain the graphs for $n = 8$, $g = 1, 2, 4, 8$.

7.22.1 Power provisioning for a warehouse-sized computer

Large-scale Internet services require a warehouse-sized computing system (Fan et al., 2007). There are strong economic incentives to operate facilities as close as possible to maximum capacity, so that the non-recurring facility costs can be best amortized. This is difficult to achieve in practice because of uncertainties in equipment power ratings and because power consumption varies significantly with the actual computing activity. Presented in this work is the aggregate power usage characteristics of large collections of servers for different applications over a period of approximately six months. These observations allow the evaluation of opportunities for maximizing the use of the deployed power capacity of datacenters.

It is observed that even in well-tuned applications there is a (7 - 16%) gap between achieved and theoretical aggregate peak power usage at the cluster level and this gap grows to almost 40% in whole datacenters. This headroom can be used to deploy additional compute equipment within the same power budget with minimal risk of exceeding it. A modeling framework is used to estimate the potential of power management schemes to reduce peak power and energy usage. The opportunities for power and energy savings are found to be significant, but greater at the cluster-level (thousands of servers) than at the rack-level (tens). It is argued that systems need to be power efficient across the activity range, and not only at peak performance levels.

The inefficient use of the power budget at a given aggregation level is sometimes underutilized in practice large amounts due to the following contributing factors. Components of peak power breakdown for a typical server are given in Table 7.11.

Staged deployment: A facility is rarely fully populated initially, therefore there is a gap between deployed and used power tends to be larger in new facilities.

Fragmentation: Power usage can be stranded simply because the addition of one more unit might exceed that level's limit, e.g., a 2.5 kW circuit may support only four 520 W servers underutilizing that circuit by 17%. If a datacenter is designed such that the *Power Distribution Unit* - PDU-level peak capacity exactly matches the sum of the peak capacities of all of its circuits, such underutilization percolates up the power delivery chain and becomes truly wasted at the datacenter level.

Conservative equipment ratings such as nameplate ratings in computing equipment datasheets often reflect the maximum rating of the power supply instead of the actual peak power draw of the specific equipment. As a result, nameplate values tend to drastically overestimate achievable power draw.

Variable load: Due to variable levels of activity a typical low-end server system consumes less than half its actual peak power when it is idle. Such variability transforms the power provisioning problem into an activity prediction problem.

Statistical effects: It is increasingly unlikely that large groups of systems will be at their peak activity and levels simultaneously as the size of the group increases.

TABLE 7.11 Breakdown of peak power components for a typical server.

Component	Peak power	Count	Total
CPU	40 W	2	80 W
Memory	9 W	3	36 W
Disk	12 W	1	12 W
PCI slots	25 W	2	50 W
Motherboard	25 W	1	25 W
Fan	10 W	1	10 W
System total			213 W

7.23 Pergamum: replacing tape with disk-based archival storage

The Pergamum project was carried out at UCSC by Mark Storer, Kevin Greenan, and Ethan Miller with possible funding by Kaladhar Voruganti at NetApp (Storer et al., 2008). Pergamon was a city in ancient Greece with the Pergamum library

utilizing parches (dried skin). <https://en.wikipedia.org/wiki/Pergamon>

It was second to the Library of Alexandria, Egypt utilizing papyrus (made from a plant with the same name) for writing. https://en.wikipedia.org/wiki/Library_of_Alexandria

Disk-based systems are superior to tape-based systems, because of their random access capability, which makes them better suited for inter-media redundancy and auditing. This point is stated crudely by former Data Domain CEO Frank Sliotman “Tapes Suck” in (Sliotman, 2011).

In the case of tapes belonging to an automated tape library power is consumed after a tape is mounted on a tape drive and is being read or written, while this is not so for disk-based systems, which makes them less suitable for archival storage.

Pergamum is a distributed network of intelligent disk-based storage appliances called tomes, which have an NVRAM at each node to store data signatures, metadata, etc. allowing deferred writes of metadata requests and inter-disk data verification, while disks are powered off. Pergamum uses both intra-disk and inter-disk redundancy to guard against data loss, relying on hash tree-like structures of algebraic signatures to efficiently verify the correctness of stored data. While rebuilding large redundancy stripes, Pergamum uses staggered rebuild to reduce peak energy usage. Pergamum is comparable in both startup and ongoing costs to other archival technologies in Table 7.12, where SUN/STK SL8500 is described.

<https://docs.oracle.com/cd/E19912-01/index.html>

TABLE 7.12 Representative archival technologies (PA = power aware).

System/Ref.	Media	Workload	Redundancy	Consistency	PA
PARAID (Weddle et al., 2007)	disk	server clusters	RAID		yes
Nomad FS (Pinheiro and Bianchini, 2004)	disk	server clusters	none		yes
Google GFS (Ghemawat et al., 2003)	disk	data-intensive	replicas	relaxed	no
Centera (EMC-Data Archive Corp., 2018)	disk	archival	RAID1/4	WORM	no
Venti (Quinlan and Dorward, 2002)	disk	archival	RAID5	content-based	no
Deep store (You et al., 2005)	disk	archival	selective-repl.	same	no
Copan 220A (Guha, 2004)	disk	archival	RAID5	SHA 256	yes
SUN/STK SL8500*	tape	backup	N + 1	WORM media	no
RAIL (Tanabe et al., 1999)	optical	backup/archival	RAID4	write verify	no
Pergamum (Storer et al., 2008)	disk	archival	2-level EC	algebraic sign.	yes

Each Pergamum tome consists of an ARM microprocessor, SATA HDD, and an NVRAM which consume 3.5, 7.5, and less than 0.6 W, respectively. The CPU consumes 0.3 W when inactive. While MAID systems spindown 75% of disks, Pergamum saves energy by having 95% of disks spun down. The flash NVRAM provides low power persistent storage. The rate of writing to this storage is low, because of the nature of the data it holds. Writing to flash can be done $10^4 - 10^5$ times. A cost comparison of archival media is given in Table 7.13.

TABLE 7.13 Cost comparison of archival media.

System	Media	Static Cost	Oper. Cost
SUN STK SL8500	T10,000 tape	\$4,250	\$60
EMC Centera	SATA HD	\$6,600	\$1,800
PARAID	SCSI HD	\$37,800	\$1200
COPAN	SATA HD	\$19,000	\$250
RAIL	UDO2	\$57,000	\$225
Pergamum	SATA HD	\$4700	\$50

Clients access Pergamum data using a file-centric view, while a block centric view is used for inter-tome data transfers. The data on each tome is organized into regions, which consist of multiple segments or stripes protected by check strips. Each stripe consists of multiple blocks as large as 1 MB.

Other wide-area long-term storage systems, such as SafeStore (Kotla et al., 2007), OceanStore (Rhea et al., 2003) and Glacier (Haeberlen et al., 2005), can provide data longevity, but are not energy efficient. For example, SafeStore uses multiple remote storage nodes to ensure data safety, but does not address the issue of reducing power consumption on the remote servers.

7.24 Energy efficient RAID - ERAID

An *Energy-efficient RAID - E-RAID* was proposed by Jun Wang et al. at U. Central Florida - UCF in (Wang et al., 2008b). Energy savings are considered in the context of RAID1 disk arrays without striping and RAID5 disk arrays with striping. The objective is to maximize the energy savings defined as: $S_E = 1 - E_{ERAID}/E_{base}$, given that constraints on response time and throughput are met.

As far as read loads are concerned two queueing models called synchronous and asynchronous in the paper correspond to an open model with Poisson arrivals with rate λ and a closed *Machine Repairman Model - MRM* with M users with think time Z (Kleinrock, 1975). With n requests at the service station and $M - n$ users in think mode requests are generated with rate $(M - k)/Z$.

Requests are cached with probability p_0 in which case the mean access time is $\bar{x}_0 = \mu_0$ and otherwise with probability p_n disk pair $1 \leq n \leq N$ is accessed with mean service time $\bar{x}_n = \mu_n$. Note that $\sum_{n=1}^N p_n = 1 - p_0$.

With Poisson arrivals with rate λ Fraction p_0 of requests have a cache hit rate $\bar{x} = 1/\mu_0$ and each of the disks in the N disk pairs have rate $\bar{x}_n = \mu_n$. The fraction of requests to the mirrored disks is $p_n, 1 \leq n \leq N$

The cache is modeled as an M/M/1 queue with $\rho_0 = \lambda p_0 \bar{x}_0$. When all disk are active we have N M/M/2 queues with server utilization $\rho = \lambda p_n \bar{x}_n / (2)$. We have used the mean response time formula for the mean response time of M/M/2 queues given in (Kleinrock, 1975).

$$T_{D=0} = \frac{\bar{x}_0}{1 - \rho_0} + \sum_{n=1}^N \frac{\bar{x}_n}{1 - \rho_n^2}$$

A control algorithm is used to trade-off energy efficiency and performance degradation by spinning down mirrored disks with the lowest utilizations.

Algorithm 1: Find mirror disks to be spun down.

1: Sort mirrored disks (in pairs) according to load from the lightest to heaviest.

2: for $(1 \leq d \leq N/2, d++)$

3: Predict performance degradation of spinning down first d lowest utilization disks;

4: If any one of the constraints is violated at $d + 1$, then the first d disks are spun down;

That is, without loss of generality the first D disk pairs are assumed to have the lowest load.

The spun-down disks are indexed $1 : D$ and their mirrors are $N/2 + 1 : N/2 + D$. Only read requests are considered to simplify the discussion.

$$T_{D>0} = \frac{\bar{x}_0}{1 - \rho_0} + \sum_{n=1}^D \frac{\bar{x}_n}{1 - \rho_n} + \sum_{n=D+1}^N \frac{\bar{x}_n}{1 - \rho_n^2}$$

Only read requests are considered, which is the worst case, so that the load on mirrors is doubled when D disks with lowest utilizations are spun down.

The closed model is solved using *Mean Value Analysis - MVA* to obtain mean disk response times (Lavenberg, 1983), (Lazowska et al., 1984). Requests are processed similarly to the open model.

An M/M/1/K queueing model is used for write requests with a buffer with capacity K . Given an arrival rate λ and service rate μ , given $a = \lambda/\mu$ the probability that there are k blocks in the system is

$$p_k = p_0 a^k, 0 \leq k \leq K, \text{ where } p_0 = \frac{1 - a}{1 - a^{K+1}}.$$

The throughput of the system is $\lambda^{effective} = \lambda(1 - p_K)$, since requests arriving at a full buffer with K requests are lost. The mean number of writes is:

$$\bar{K} = \sum_{k=1}^K k p_k = \frac{a(1 - (K + 1)a^K) + K a^{K+1}}{(1 - a)(1 - a^{K+1})}.$$

Write response times are of little interest, since they do not affect application response times. The analysis does not take into account the overwriting of dirty blocks and that blocks in batches are written out more efficiently by taking advantage of their proximity.

7.25 Power reduction via write-offloading

Disks consume a significant fraction of the power in enterprise systems. Disk spindown and write offloading are two well known approaches to save energy:

- Disk spindown is considered to be ineffective in enterprise systems, since disk idle periods are too short to allow meaningful energy savings. Analysis of block level traces at Microsoft's Cambridge Research Center in (Narayanan et al., 2008) by Dushyanth Narayanan, Austin Donnelly, and Antony Rowstron shows that this is not so and spindown in idle periods can yield 28-36% power savings.
- Idle periods are extended by write off-loading, i.e., active disks hold dirty blocks to be written to spun-down disks in persistent storage. Write off-loading can be used to reduce power consumption further to up to 45-60%.

Four scenarios are considered:

Baseline: Disks are never spun down, so there are no energy savings and performance is not degraded.

Vanilla: Disks are spun down when idle and spun up again with the next read or write.

Machine-level off-load: Write off-loading is enabled. Rewrites can only be off-loaded to loggers running on the same server.

Rack-level off-load: Writes can be off-loaded to any logger in the rack.

Given as bar charts are the mean and peak power consumption on the least and most idle days (of the year) for the three power saving schemes with respect to the baseline model. The average power consumed at the least idle day is 72%, 64%, and 55% of the baseline.

Median read time is about the same for most and least idle days, about 5 msec.

Percentages of requests incurring spinups are shown to be very low.

7.26 Redundant Arrays of Hybrid Disks - RAHD

Hybrid Disk Drives - HDDs were developed by Samsung by adding flash memory cache to magnetic disks. The effect of clustering frequently accessed data in flash-based cache was evaluated by analyzing disk I/O traces collected over real systems and a 4-fold speedup in performance (Wang et al., 2007).

The *Redundant Array of Hybrid Disks - RAHD* proposed by Yuhui Deng et al. (Jinan U., Guangzhou) in (Wang et al., 2008a) has a provision for efficient energy control. In high speed mode RAHD outplays the purely-magnetic-disk-based RAID5 by a factor of 2.4-4. In the energy-efficient mode a RAHD4/5 can save up to 89% of energy at little performance degradation.

7.27 Achieving power-efficient, erasure-coded storage

Kevin Greenan, Darrell Long, Ethan Miller, Thomas Schwarz (Marquette U.), and Jay Wylie (HP, Google) studied an energy efficient disk array at UCSC (Greenan et al., 2008).

There are eight disks D_i , $0 \leq i \leq 7$, hold the following data symbols: S_i , $0 \leq i \leq 4$ and three parity symbols: $S_5 = S_0 \oplus S_1 \oplus S_2$, $S_6 = S_0 \oplus S_1 \oplus S_3$, $S_7 = S_0 \oplus S_2 \oplus S_3 \oplus S_4$.

In case disks D_0 , D_5 , D_6 , and D_7 are currently active then there is no need to activate disk D_4 to access block s_4 , since it can be reconstructed as

$$s_4 = s_7 \oplus s_3 \oplus s_2 \oplus s_0 = s_0 \oplus s_5 \oplus s_6 \oplus s_7.$$

Three conditions for power awareness erasure-coded arrays are as follows:

1. Processing writes to minimize operational power consumption, while maintaining a sufficient level of reliability.
2. The read policy determines how to reconstruct data on a spun-off disk from active disks.
3. How to efficiently schedule disk activations to service a read request from a spun-off disk.

Write groups are disks that need to be up for a write, for example, updating a data block on D_4 needs only D_7 to be up. Examples of write groups are as follows:

$\{D_0, D_5, D_6, D_7\}$,

$\{D_1, D_5, D_6\}$,

$\{D_2, D_5, D_7\}$,

$\{D_3, D_6, D_7\}$, $\{D_4, D_7\}$.

Matrix methods specified by James Hafner (rip) et al. at ARC in (Hafner et al., 2005) are used to reconstruct data whenever possible without spinning up disks. The method in (Hafner et al., 2005) recovers what is theoretically recoverable,

exceeding the provisions of the code. The solution can be applied statically or incrementally. The method can recover data in lost sectors when these sectors touch more strips in the stripe than the fault tolerance of the erasure code. Algorithm I in (Greenan et al., 2008) is a power aware read algorithm, while Algorithm II is a brute force disk activation algorithm to reconstruct data.

7.28 Effect of energy-saving schemes on disk reliability

The impact of disk utilization on *Annual Failure Rate* - AFR across different disk age groups was investigated at Google in (Pinheiro et al., 2007). AFRs of disks with ages 3 and 6 months and 1, 2, 3, 4, and 5 years under three utilization levels: low, medium, and high are investigated. The measurements carried out at Google were recorded in BigTable (Chang et al., 2008) and analyzed with MapReduce.

<https://en.wikipedia.org/wiki/MapReduce>

AFR decreases after the first three months, which is the usual infancy burnout period (Trivedi, 2002), it increases in the second year and reaches its peak in the third year, after which it decreases slightly in the fourth and fifth years. AFR is higher at low and high load levels at 3 and 6 months, but otherwise increases with disk load level.

7.29 Mathematical model of disk reliability versus load and temperature

A mathematical model for disk reliability called MINT is presented by Shu Yin et al. (U. San Diego) in (Yin et al., 2009). The factors considered in this study are disk utilization, temperature, spindown/spinup transition frequency, and disk age. Their effect on three year old HDD AFR is plotted in the paper, respectively.

As far as load is concerned the minimum AFR is attained at 60% disk utilization, but is higher for 20% than 80% utilization. The AFR increases linearly in the range 35 to 45 centigrades, beyond which it remains flat. The AFR vs transition frequency (from active to standby and vice-versa) per month with up 500 transitions shows a parabolic behavior increasing from 0.01 to 0.4. The authors proceed to build a reliability model of MAID (Colarelli and Grunwald, 2002) and the *Popular Data Concentration* - PDC technique (Pinheiro and Bianchini, 2004). Experimental results are provided comparing the two methods and PDC is shown to outperform MAID at lower access rates.

7.30 Sample-Replicate-Consolidate mapping - SRCMap

Sample-Replicate-Consolidate Mapping - SRCMap is a storage virtualization layer due to Raju Rangaswami et al. at *Florida Int'l U. - FIU* in Miami (Verma et al., 2010), it allocates a subset of disks in proportion to I/O workload intensity.

Instead of migrating data across physical volumes dynamically or replicating entire volumes, both of which are prohibitively expensive, SRCMap samples a subset of blocks from each data volume that constitutes its working set and replicates them on other physical volumes. During a given consolidation interval, SRCMap activates a minimal set of physical volumes to serve the workload and spins down the remaining volumes, redirecting their workload to replicas on active volumes. Both theoretical and experimental evidence is presented to establish the effectiveness of SRCMap in minimizing the power consumption of enterprise storage systems.

TABLE 7.14 Comparison of power management techniques (p = partially addressed) (Verma et al., 2010).

Design goals	Write offloading	Caching systems	Single redundant	Geared RAID
Proportionality	p	n	n	p
Space overhead	y	y	n	n
Reliability	n	n	y	y
Adaptation	n	y	y	y
Heterogeneity	p	p	p	n

Issues raised are summarized in Table 7.14 and as follows:

Fine grained Energy Proportionality - EP: If the peak power of P_{peak} for the maximum workload intensity ρ_{max} then at workload ρ the power intensity should be $P = P_{peak} \frac{\rho}{\rho_{max}}$.

Low space overhead: Replication based strategies achieve energy proportionality by replicating data. These systems spin down replicas in periods of lower activity.

Reliability: The fact that disks are designed to tolerate a maximum number of spinups should be taken into consideration according to *Hitachi Global Storage Technologies - HGST* roadmap in 2007 (Kim and Suk, 2007).

Workload shift adaptation: Power management for storage systems that rely on caching popular data should closely monitor changes in popularity.

Heterogeneity support: Power consumption varies with technology, e.g. it takes less power to access data from a flash memory, but of course *Total Cost of Ownership - TCO* should be taken into account.

7.31 Power Proportional Distributed File Systems - PPDFS

A *Power-Proportional Distributed File System - PPDFS* named Rabbit is described by Greg Ganger (CMU) and Karsten Schwan (Georgia Tech., rip) et al. in (Amur et al., 2010) Rabbit's data layout has three key properties.

- A wide range of power-performance settings, from a low minimum power to a high maximum performance, while maintaining the efficiency of a non-PPDFS design and fine gradation of settings.
- Ideal power-proportionality so that performance-to-power ratio at all performance levels is equivalent to that at the maximum performance level, i.e., all nodes powered-up.
- Near-ideal power proportionality in the face of node failures. Rabbit mitigates contention among multiple workloads using distinct datasets.

The requirements of a PPDFS policy are:

1. With $p \ll N$ nodes active the minimum throughput $tput_p$ is achieved with power pow_p .
2. With all N nodes active the maximum throughput is attained $tput_N$ with power pow_N .
3. Ideal power proportionality implies

$$\frac{tput_i}{pow_i} = \frac{tput_N}{pow_N}, \quad i \in \{p, \dots, N\}.$$

4. Fast, fine-grained scaling with no data movement required.

The equal-work policy for replication factor r is an optimization problem that minimizes p

$$tput_i = \frac{i}{p} tput_p \text{ for } i = p + 1, \dots, N$$

That fair I/O scheduling provides isolation from app_A from app_B is illustrated in Table 7.15. As the number of instances of app_B increases, app_A running on default HDFS does worse, but app_A running with Rabbit based IO isolation is hardly affected.

TABLE 7.15 Throughputs for app_A as number of instances for app_B are varied for Rabbit and default HDFS.

# instances of app_B	Rabbit	Default HDFS
1	180.3	136
2	175.3	101.2
3	173.6	82.5

7.32 Dynamic locality improvement to increase effective storage performance

This discussion has implications on power consumption in disks. but was carried out before power consumption was considered a major concern in disk storage, as discussed in Section 7.33.

Windsor Hsu (IBM/ARC, Data Domain, Datrium, Alibaba) a PhD student of Alan Jay Smith at UCB did his PhD studies at IBM ARC reporting to Honesty Young (Intel).

Only a small fraction of disk data is heavily utilized as noted in Chapter 2 in Hsu's PhD thesis (Hsu, 2003). Prior to the introduction of striping there was a major difference in disks utilizations in computer installations with a large number of

disks. In the case of multiple RAID5 arrays while the disk loads on a single array are balanced, the disk loads across arrays may be unbalanced (Zhang et al., 2018b).

Hsu's studied traces from different platforms: Windows NT, IBM AIX and HP-UX (Hsu et al., 2001). It is noted that the results are affected by the hit rate due to database/file cache. This effect can be taken into account by using an LRU filter and a writeback cache, which keeps dirty blocks for 30 seconds, but unloads/destages up to 512 contiguous blocks together.

Access patterns are represented by graphs whose vertices represent *Reorganization Units - RUs* and the weights of edges connecting vertex i to vertex j represent the desirability of RU_j being placed close to RU_i , e.g., the number of times j is referenced shortly after, i.e., within some number after referencing i . This number denoted by τ is called context size and $\tau = 9$ was used in investigating run clustering.

The second step in run clustering is to analyze the access graph to discover desirable sequences of RUs. This process is similar to the graph-theoretic clustering problem with the difference that we are interested in the sequence of the vertices.

Let G be an access graph built as outlined above and R , the target sequence or run. $|R|$ denotes the number of elements in R and $R[i]$ is its i^{th} element.

1. Find the heaviest edge linking two unmarked vertices.
2. Initialize R to the heaviest edge found and mark the two vertices.
3. Repeat
4. Find an unmarked vertex u such that $\text{headweight} = \sum_{i=1}^{\min(\tau, |R|)} \text{Weight}(u, R[i])$ is maximized.
5. Find an unmarked vertex v such that $\text{tailweight} = \sum_{i=1}^{\min(\tau, |R|)} \text{Weight}(R[|R| - i + 1], v)$ is maximized.
6. if $\text{headweight} > \text{tailweight}$
7. Mark u and add it to the front of R .
8. else
9. Mark v and add it to the back of R .
10. Goto step 3.

As an example, we start with a graph with five RUs: A, R, U, N , and B , where the nodes are placed horizontally in that order from left to right.

Forward links among neighboring nodes:

$[A \rightarrow R] = 2, [R \rightarrow U] = 2, [U \rightarrow N] = 2, [N \rightarrow B] = 2,$

Forward links among nodes two apart:

$[A \rightarrow U] = 2, [R \rightarrow N] = 2, [U \rightarrow B] = 2.$

Backward links $[B \rightarrow R] = 1, [B \rightarrow A] = 1,$

Consider the reference string = $(A, R, U, N, B, A, R, U, N, B)$, Context Size = 2).

Following the discovery of run (R, U) there are four $(A, (R, U), N, B)$ with the following transitions.

$[A \rightarrow (R, U)] = 4, [(R, U) \rightarrow N] = 4, [N \rightarrow B] = 2$

$[B \rightarrow A] = 1, [B \rightarrow (R, U)] = 1, [N \rightarrow A] = 1.$

Following the discovery of run R, U, N we have three nodes: $A, (R, U), (U, N), B$ and the following weights.

$[A \rightarrow (R, U)] = 4, [(R, U) \leftarrow N] = 4, [(U, N) \leftarrow B] = 4, [B \rightarrow A] = 1, [B \rightarrow (R, U)] = 1, [(U, N) \rightarrow A] = 1$

An architecture for *Automatic Locality Improving Storage - ALIS* is described in (Hsu et al., 2005). The workload monitor collects a trace of referenced addresses. When the storage system is relatively idle the workload analyzer examines the trace to determine which blocks should be reorganized. ALIS implements two strategies for clustering data:

Heat Clustering: This is to localize frequently accessed data. Unlike the organ pipe organization (Wong, 1980, 1983), which may place initially contiguous blocks on the two sides of the organ pipe, heat clustering preserves spatial locality.

Run Clustering: Use traces to extract repeated read sequences or runs and placing them sequentially improves prefetching.

The reorganizer running as a background process copies selected blocks into a reorganization area. The copy to access is made by the traffic director, based on which copy provides the shortest access time. Updates are slowed down since all copies of the data block need to be updated.

It should be noted that this work was concerned on improving performance, but also reduces power consumption.

7.33 Disk data reorganization for reducing energy consumption

The concept of *Immediate Successor Relationship Amount - ISRA* is used in (Liao et al., 2011) to represent this relationship among disk data blocks to reduce disk energy consumption, but also improving performance.

The ISRA-based grouping algorithm uses undirected graphs to group data blocks that experience frequent successive accesses. A sort-merge like algorithm is then used to determine the position of every group and its blocks. CMU's DiskSim and an energy model show that both can be reduced by about 50%.

Disk reorganization method described in (Liao et al., 2011) notes that disk energy consumption is mainly based on seeks, so an attempt is made to reduce seek distances by reorganizing disk blocks.

(1) Blocks are tracked, (2) A graph is constructed to (3) Group data, and (4) Sort Data, and (5) Reorganize Disk. The discussion of the algorithm to reorganize data is rather unclear.

The “adaptive disk reorganization” in (Carson and Reynolds, 1989), “disk shuffling” in (Ruemmler and Wilkes, 1991) and “adaptive block rearrangement” in (Akyurek and Salem, 1995) are related studies conducted before saving power was an issue.

7.34 File assignment with minimal variance of service time

Several file allocation policies to minimize mean response time are investigated by Lin-Wen Lee, Peter Scheuermann, Radek Vingralek (Google) at Northwestern U. in (Lee et al., 2000). The issue of minimizing mean response time by minimizing the variance of service time is illustrated by a numerical example in the paper, but a more general example is introduced here to make this point.

A numerical using an M/G/1 queueing model is used to show that reducing variance by not mixing disk requests with different access times can be used to reduce the mean response time, but there is the issue of unequal disk utilizations due to the two streams.

Streams with unequal service times and equal utilizations

Consider two streams with Poisson arrival rates $\lambda_1 = \lambda$ and $\lambda_2 = 2\lambda$. The service times are exponentially distributed with mean $\bar{x}_1 = 2x$ and $\bar{x}_2 = x$, so that the utilization due the streams equals $\rho = 2\lambda x < 1$. The fraction of arrivals in the two streams is $f_1 = 2/3$ and $f_2 = 1/3$. The mean service is:

$$\bar{y} = \frac{1}{3}(2x) + \frac{2}{3}x = \frac{4}{3}x.$$

An M/M/1 queueing formula can be used to determine The mean response time when all request in stream 1 (resp. 2) are processed at server 1 (resp. 2).

$$R_1 = \frac{2x}{1-\rho} \quad R_2 = \frac{x}{1-\rho}, \quad R = \sum_{i=1}^2 f_i R_i = \frac{\bar{y}}{1-\rho}.$$

The mean waiting time is:

$$W_{M/M/1} = R - \bar{y} = \frac{(4/3)\rho x}{1-\rho}.$$

We then consider the uniform routing of request from both streams to two servers. The arrival rate to each servers is $\bar{\lambda} = (\lambda_1 + \lambda_2)/2$ and the second moment of service time is:

$$\overline{y^2} = \frac{1}{3}[2 \times (4x^2)] + \frac{2}{3}[2x^2] = 4x^2.$$

The two servers have equal utilizations and identical hyperexponential service time distributions, so that M/G/1 waiting time formula is applicable. It follows:

$$W_{M/H_2/1} = \frac{\bar{\lambda} \overline{y^2}}{2(1-\rho)} = \frac{1.5\lambda(4x^2)}{2(1-\rho)} = \frac{3\rho x}{1-\rho} > W_{M/M/1}.$$

It follows that not mixing the two streams results in an improved response time, since the coefficient of variation for each stream is one, because of the assumption of exponential distributions, while in the latter case

$$c^2 = \frac{\overline{y^2}}{(\bar{y})^2} - 1 = \frac{4x^2}{(16x^2/9)} - 1 = \frac{5}{4} > 1.$$

Repeating the exercise with fixed service times yields the following again using the P.K. formula for M/G/1 queues.

$$R_1 = \frac{\lambda(2x)^2}{2(1-\rho)} + 2x = \frac{\rho x}{1-\rho} + 2x, \quad R_2 = \frac{2\lambda x^2}{2(1-\rho)} = \frac{0.5\rho x}{1-\rho} + x, \quad R = \sum_{i=1}^2 f_i R_i = \frac{(2/3)x}{1-\rho} + \frac{4}{3}x.$$

When the streams are uniformly distributed over the two servers:

$$R = \frac{1.5\lambda(\frac{1}{3}4x^2 + \frac{2}{3}x^2)}{2(1-\rho)} + \frac{4}{3}x = \frac{1.5\rho(2x^2)}{1-\rho} + \frac{4}{3}x = \frac{1.5\rho x}{1-\rho}.$$

While c^2 for requests processed separately was zero, by mixing requests of different sizes we have:

$$c^2 = \frac{2x^2}{(\frac{4}{3}x)^2} - 1 = \frac{1}{8}.$$

Minimizing the maximum disk utilization across disks is considered to be isomorphic to the multiprocessor scheduling problem investigated by Ronald Graham (Bell Labs, UCSD, rip) (Graham, 1969). Given a set of tasks $\{T_1, \dots, T_r\}$ and m processors the issue at hand is an allocation that will minimize the makespan.

In the context of allocating load on m disks the competitive ratio is the maximal load according to a given heuristic and the optimal load. The offline version of the greedy algorithm, which allocates tasks in decreasing order of their processing times yields the following bound.

$$\frac{\omega_L}{\omega_0} \leq \frac{4}{3} - \frac{1}{m}.$$

Competitive analysis is a method used for analyzing online algorithms, in which the performance of an online algorithm is compared to the performance of an optimal offline algorithm that can view the sequence of requests in advance.

[https://en.wikipedia.org/wiki/Competitive_analysis_\(online_algorithm\)](https://en.wikipedia.org/wiki/Competitive_analysis_(online_algorithm))

The competitive ratio for the online algorithm is $2 - 1/m$.

In the case of file allocation to disks there are n files: f_i , $1 \leq i \leq n$ to be allocated among m disks: d_j , $1 \leq j \leq m$. File assignment can be specified as $\{I_1, I_2, \dots, I_m\}$, where I_i is the set of file indices assigned to disk d_j . The following file allocations are considered.

Greedy: Greedy is an online algorithm where the access rate λ_i and service time s_i for file i is known. It is used to determine the heat of the file $h_i = \lambda_i s_i$, which would be the contribution of the file to disk utilization if it was allocated on that disk. Homogeneous disks provide the same service time. Files are assigned in a manner to balance disk loads assuming an M/G/1 queueing model will minimize the mean response time. Such an analysis was provided by (Piepmeyer, 1975) for an open model with Poisson arrivals.

Sort Partition - SP: All files are ordered in a list $\mathcal{I}$ in descending order of their loads. A *Perfectly Balanced File Assignment - PBFA* is such that $\sum_{i \in I_k} h_i = \rho_0$, such that $\rho_0 = \frac{1}{m} \sum_{i=1}^n \lambda_i s_i$. It can be proven that “Among all PBFAs Sort Partition finds the one with minimal response time”. The best PBFA should minimize the variance of service times. The intuition behind this is that the mean waiting time in M/G/1 queueing systems is given by the Pollaczek-Khintchine - P.K. formula as follows:

$$W = \frac{\lambda \bar{x}^2}{2(1-\rho)} = \frac{\rho \bar{x}(1+c^2)}{2(1-\rho)},$$

where $\overline{x^2}$ is the second moment of service computed over multiple task classes and $\rho = \lambda \bar{x}$ is the utilization factor (Kleinrock, 1975). When all files allocated to a disk are the same size then $c = 0$.

The pseudocode for the Sort-Partition algorithm is as follows:

Algorithm: Sort partition.

Input:

m = number of disks,

n = number of files,

h_i = heat of file i ,

s_i = expected service time of file i .

Output: assignment of files to disks: $\{I_1, I_2, \dots, I_m\}$

Step 1: Compute the average disk utilization $\rho = \frac{1}{m} \sum_{i=1}^n h_i$

Step 2: Sort all files into list I in descending order of their service time.

Step 3: Allocate to each disk d_j the next contiguous segment of I until its load j reaches the maximum allowed level.

$i = 1$

for $j = 1$ to m do

load $_j = 0$, $I_j = \emptyset$.

while load $_j \leq \rho$ or $i \leq n$ do

$I_j = I_j \cup \{i\}$

load $_j = \text{load}_j + h_i$

$i = i + 1$

od

od

if $(i \leq n)$ then

$I_m = I_m \cup \{i, i + 1, \dots, n\}$

fi

Hybrid Partition - HP: Sort Partition is an offline algorithm, which provides superior performance to Greedy in terms of response time. Hybrid partition is an intermediate algorithm that allocates files in batches in distinct allocation intervals. For each allocation intervals ℓ a different disk d_k with the smallest accumulated load is selected. A number of files are allocated to d_k until its load reaches the threshold T_k .

Algorithm: Hybrid partition.

Input:

m = number of disks.

n = number of files.

h_i = heat of file i

b = number of sorted batches of files.

$B_i = i^{th}$ batch of files sorted in descending order of service times.

B = list $(B_1, B_2, \dots, B_b)$

Output: Assignment of files to disks $\{I_1, I_2, \dots, I_m\}$

Step 1: Initialize the load on disk d_i : load $_i = 0$ and set allocation list $I_i = \emptyset$.

Step 2: while (list of batches B has not been exhausted) do

Step 2.1: Select next batch B_{next} not yet assigned

while (B_{next} has not been completely assigned) do

Step 3: Determine the next disk d_k to be used during the next allocation interval ℓ :

disk $_k$ = disk such that load $_k = \min_{s=1, \dots, m} \text{load}_s$

Step 3.1: Determine the threshold θ_k to which the load on d_k can be increased in this interval.

$$\theta_k = 1 - \frac{1 - \text{load}_k}{\text{overflow}}$$

The value of θ_k can be computing using the M/M/1 queueing model for disk k . $E(r_1) = E(s)/(1 - \text{load}_k)$ and $E(r_2) = E(s)/(1 - \theta_k)$. The setting $E(r_2)/E(r_1) = \text{overflow}$ leads to $\theta_k = 1 - (1 - \text{load}_k)/\text{overflow}$, where by experimentation overflow is set to 1.05.

Step 3.2 During the allocation interval ℓ assign to disk d_k a contiguous segment of files from B_{next} , until its load denoted by load_k reaches the threshold θ_k

Update the allocation list I_k

$I_k = I_k \cup \{\text{indices of files allocated in interval } \ell\}$

od

od

Experimentation was carried out by allocating 5000 files over 16 disks. The sizes of files followed a Zipfian distribution with a skew parameter $\theta = \log \frac{X}{100} / \log \frac{Y}{100}$, where $X\%$ of accesses are directed to $Y\%$ of files.

Figures 8 and 9 in (Lee et al., 2000) provide the response time versus the arrival rate for 70/30 and 60/40 skew respectively. It follows from Figures 10 and 11 that as the batch size decreases (78 versus 1250), although hybrid outperforms greedy there is little difference between the two, Note that large batch sizes are tantamount to static scheduling.

7.35 Striping-based Energy Aware - SEA placement

Improving response and energy consumption in disk arrays is addressed in the context of real-world applications, such as *Video-on-Demand - VoD* and Web servers by Xie Tao et al. in (Xie and Sun, 2007), (Xie, 2008). *Striping-based Energy-Aware - SEA* data placements applied to RAID0 and RAID5 arrays are named SEA0 and SEA5.

Experimental results demonstrate that compared with the three data placement algorithms: Greedy, SP - Sort Partition, and HP - Hybrid Partition in (Lee et al., 2000) SEA0 and SEA5 reduce the mean response time 45.8% and 39.3%, while saving energy not less than 9.8% and 7.9%. Compared with a well-known striped file *Parallel Virtual File System - PVFS* (Latham et al., 2004) SEA0 and SEA5 save energy by up to 36.2% and 38.6%, while mean response time increased by 18 and 24 msec.

According to (Breslau et al., 1999) request frequency and file size are inversely correlated. Files are subdivided into two categories: popular and unpopular (Nishikawa et al., 1998). Similarly, disks in the disk array are separated into two zones: hot and cold. Disks in the hot zone hold popular data, whereas disks in the cold zone hold unpopular data. As a result, the overall load balancing between two zones can be achieved, which improves the inter-request parallelism.

The rationale for SEA is that the distribution of web page requests follows a Zipf-like distribution, where the relative probability of a request for the i^{th} most popular file is proportional to $1/i^\alpha$ with $0 \leq \alpha \leq 1$ (Zipf, 1949), (Breslau et al., 1999). The skew parameter for the Zipf distribution is $\theta = \log(X/100)/\log(Y/100)$, where $X\%$ of all accesses are directed to $Y\%$ of files and $\alpha = 1 - \theta$. Given m files the number of popular files is $(1 - \theta)m$ and the number of unpopular files is θm . The ratio between the number of popular and unpopular files is $\eta = 1/\theta - 1$. The ratio of the load due to all popular and unpopular files is denoted by γ .

Disks in the cold zone are set to operate in low speed mode and vice versa, so that the service time of hot and cold files is determined by their size and disk transfer rate. The service time for hot/cold files is $sv_i = s_i/t$ where the disk transfer rate is $t = t^h$ and $t = t^c$ for hot and cold files, respectfully. The heat associated with file i is $h_i = \lambda_i s_i$.

Input: A disk array with n 2-speed disks, a collection of m files in the set $\mathcal{F}$, and skew parameter θ . Hot files are in the set $\mathcal{F}_h$ and cold files in the set $\mathcal{F}_c$, so that $\mathcal{F}_h \cup \mathcal{F}_c = \mathcal{F}$.

Output: A file allocation scheme $X(m, k)$, where the stripe width is computed by dividing the file size s_i by the strip size sp .

$$k = \max_{i=1}^m \left(\frac{s_i}{sp} \right).$$

1. The ratio of popular and unpopular files is $\eta = (1 - \theta)/\theta$. The number of popular (resp. unpopular) files is $|F_h| = (1 - \theta)m$ (resp. $|F_c| = \theta m$).

2. γ is the ratio of the sum of the heats by hot and cold files.

3. The number of disks required to hold hot files is $HD = \gamma n / (\gamma + 1)$ and the remaining disks hold cold files $CD = N - HD$. Initialize the index of strips for hot and cold files $d_h = 1, d_c = 1$.

```

4. Configure HD disks at high speed and CD disks at low speed.
5. for each popular file  $f_p \in F_h$  do
6.  $p = 1$ 
7. for each stripe  $sp_p$  of  $f_p$  do
8.  $X(f_p, p) = d_h$ 
9.  $p = p + 1$ 
10.  $d_h = (d_h + 1) \bmod (HD)$ 
15. end for
17. for each unpopular file  $f_u \in F_c$  do
18.  $u = 1$ 
19. for each stripe  $sp_u$  of  $f_u$  do,
20.  $X(f_u, u) = d_c$ ;
21.  $u = u + 1$ ;
25.  $d_c = (d_c + 1) \bmod (CD)$ 
28. end for

```

Energy savings are computed using hypothetical multi-speed disks. Specifically, disks in the hot zone run at a higher speed mode with more energy consumption, while disks in the cold zone operate at a lower speed to consume less energy. Multi-speed HDDs are not available in the market yet. Two-speed disks from Hitachi operate at one speed, only the higher speed. The main characteristics of the two speed disk according to (Xie, 2008) are as follows:

Seagate Cheetah ST39205LC with SCSI interface.
Storage capacity 9.17 GB, 10K RPM, Rotation time RT = 6 msec.
Transfer rate in high/low mode 31/9.3 MB/s.
Idle power at high/low mode. 5.26/2.17 Joules/s.
Active energy at high/low mode (8 KB read) 61/43 Joules.

Simulation is used to report three performance metrics: the mean response time, energy consumption, and mean response time normalized by mean service time.

7.36 PEARL: Performance, Energy, and ReLIability balanced dynamic data distribution

Hybrid disk storage is considered in (Xie and Sun, 2008). The study is concerned not only with data redistribution to improve performance, but also to conserve energy consumption. The flash memory is much smaller than HDD and acts as a cache and its contents are determined by an appropriate cache replacement algorithm. Obviously data that is frequently updated should be held by the HDD.

PEARL designates zones as flash favorite or hard-disk favorite. Depicted is data striped across two disks with three strips per disk. Each zone has two consecutive stripes. The flash memory can hold two zones. Zone 2 and 3 on disk 1 and disk 2 are considered to be flash favorite at the end of epoch 1 and are moved to the flash memory without being replicated on hard disks. At the end of epoch 2 zones 2 and 4 are moved to flash, so that zone 3 is replaced with zone 4.

PEARL maintains a *Popularity and Location Table - PLT*, which keeps track of the number of reads and writes on a per zone basis. The table also specifies whether the zone is held in a cache or not. The *Data Redistribution Table - DRT* determines which zones should be moved to flash memory.

Han-joon Kim and Sang-goo Lee in (Kim and Lee, 1999) extended the usage of flash memory devices by developing an energy-efficient file placement technique called *Pattern-Based - Popular Data Concentration - PB-PDC* algorithm in (Pinheiro and Bianchini, 2004) by separating read and write I/O requests.

PB-PDC places all read-only data upon a flash drive, while putting the rest of the data on a hard disk. The PB-PDC technique only concentrated on one flash drive with a single hard disk, while PEARL extends to multiple disks.

As the flash memory size is increased per disk from 4 to 12 GB, a considerable improvement in response is attained for the first two workloads. As the number of disks is varied from 6 to 16, PEARL results in a reduced response time for Financial 1 and 2. The improvement in energy is rather limited with PEARL versus PB/PDC.

7.37 Power proportionality for data center storage

Data centers hosting online services show significant variation in load levels over time: hour of the day, day of the week, etc. Significant savings in power consumption can be attained by *Virtual Machine* - VM migration to consolidate computational load and powering down unused servers.

There is significant potential for saving power by powering down excess servers during periods of low load. Techniques like *Virtual Machine* - VM migration can consolidate computational load, but migrating data is not a practical way to consolidate I/O load.

Sierra is a power-proportional distributed storage subsystem for data centers proposed by Eno Thereska (Microsoft, Confluent, AWS), Austin Donnelly, and Dushyanth Narayanan at Microsoft Cambridge in (Thereska et al., 2011), which considers three different configurations with six data chunks: A-F, four servers, and two way replication. We number the four nodes placed in a square one to four, with 1:2 in the top row and 3:4 at the bottom row. The following three configurations are considered.

Naive random: A is replicated at 1,2; B at 3,4; C at 1,3; D at 2,4; E at 2,3; and F at 1,4.

Naive grouping: A, B, C are replicated at nodes 1 and 2 constituting replica group 1. D, E, F are replicated at nodes 3 and 4 constituting replica group 2.

Power aware grouping: A, B at nodes 1 and 2; C at nodes 1 and 4; D at nodes 2 and 3; E, F at nodes 3 and 4; This is a power aware grouping since nodes 1 and 3 constitute Gear Group 0, since it holds a copy of all six chunks. Additional chunks can be activated at gear 2 by powering up nodes 2 and 4.

The gear scheduler is a centralized component aggregates load measurements that schedules servers to power down or up according to predicted system load. It predicts load for each hour based on historical averages from that same hour in previous days. The gear scheduler is simple, because the workloads exhibit predictable patterns.

7.38 Economic evaluation of energy saving with reliability constraint

The volume of data at the archive under consideration grew 60% annually, which made reducing power consumption an important consideration, since 50% of the *Total Cost of Ownership* - TCO is energy (Chen et al., 2015). Of the two solutions to conserve power in storage systems: multispeed disks and disk spin-down, only the latter solution is realistic, since multispeed disks are not available.

Research conducted at Google and *Int'l Disk drive Equipment and Materials Association* - IDEMA idema.org has shown however that according to (Xie and Sun, 2008) the reliability function for the function $R(f)$ for the *Annual Failure Rate* - AFR versus frequency of disk power state transitions:

$$R(f) = 1.51e^{-6} f^2 - 1.08e^{-5} f + 1.39e^{-2}.$$

PREFigure is a robust framework presented at (Yan et al., 2016) that aims at harvesting future idle intervals for power savings while meeting strict quality constraints (Yan et al., 2016). It introduces potential delays in serving IO requests that occur during power savings because spinup time is not negligible. The power saving mechanism is triggered a few times only, such that the disk wear out due to powering up and down does not compromise the disk's lifetime. PREFigure is based on an analytic methodology that uses the histogram of idle times to determine schedules for power saving modes as a function of the above constraints. PREFigure facilitates analysis for the evaluation of the trade-offs between power savings and quality targets for the current workload. Extensive experimentation on a set of enterprise storage traces illustrates PREFigure's effectiveness to consistently achieve high power savings without undermining disk reliability and performance.

The adaptive write cache mechanism called CacheRAID, which reduces energy consumption due to random accesses (Chen et al., 2012). CacheRAID also assists in spindown reducing power consumption of RAID5 system by 65% to 80%.

7.39 Dynamic server provisioning for data center power management

CPU power consumption in data centers increases with the cube of required power, which can be attained by increasing voltage.

<http://www.tomshardware.com/answers/id-3025075/higher-clock-speed-higher-load-power-consumption.html>

Dynamic server provisioning for data center power management was the topic of a PhD thesis by Anshul Gandhi guided by Mor Harchol-Balter at CMU (Gandhi, 2013). Four approaches for reducing power consumption in data centers are proposed.

Power-proportionality: Servers consume power in proportion to their utilization. Processors can operate at various voltage and frequency settings. DVFS is an approach that reduces server power consumption and speed by lowering the operating voltage and frequency. The main challenge with DVFS is choosing the right voltage and frequency settings for a given application. In the case of batch workload the question is whether “race to idle” is the better strategy, i.e., will consume less electricity in completing a certain workload. Servers that have completed their workload can be turned off, hence servers should be run at full speed to complete jobs as soon as possible.

Energy-efficient Servers: The right server architecture for a given workload.

Dynamic Server Provisioning: When to turn servers on and off. Percentiles of response time are used in *Service Level Agreements - SLAs*, which specify an upper bound to customer response times, e.g., Amazon guarantees a response time below 300 ms for 99.9% of requests at the peak load of 500 requests per second.

Consolidation and Virtualization: Amortizing power consumption by resource sharing.

AutoScale proposed in (Gandhi et al., 2012) is a practical dynamic server provisioning solution that reduces the number of servers needed in data centers driven by unpredictable load, while meeting response time SLAs. Two key features of Autoscale are:

- Maintain the right amount of spare capacity to handle short-term fluctuations in request rate.
- Robustness to unpredictable changes in request rate, but also request size.

Server farms consume more than 1.5% of the total electricity in the U.S. at a cost of nearly \$4.5 billion. Given the rising cost of energy, many industries are seeking solutions on how to best make use of their available power. An important question which arises in this context is how to distribute available power among servers in a server farm to maximize performance. By giving more power to a server, one can get higher server frequency (speed). Hence it is commonly believed that for a given power budget performance can be maximized by operating servers at their highest power levels. However, it is also conceivable that one might prefer to run servers at their lowest power levels, which allows more servers to be turned on for a given power budget.

A queueing theoretic model is introduced to predict the optimal power allocation in a variety of scenarios involving IBM BladeCenters. The optimal power allocation varies according to the scenario. It is not always optimal to run servers at their maximum power levels. In fact there are scenarios where it might be optimal to run servers at their lowest power levels or at some intermediate power levels. The optimal power allocation is non-obvious and depends on many factors such as the power-to-frequency relationship in the processors, the arrival rate of jobs, the maximum server frequency, the lowest attainable server frequency and the server farm configuration. The theoretical model allows the exploration of more general settings than can be implemented. It is shown that the optimal power allocation can significantly improve server farm performance, by a factor of typically 1.4 and as much as a factor of 5 in some cases.

7.40 Modeling the energy costs of I/O workloads

A study to model energy costs of I/O workloads by Yan Li and Darrel Long *Storage Systems Research Center - SSRC at U. Calif. Santa Cruz - UCSC* reported in (Li and Long, 2014) uses the following parameters:

S_{pb} storage peak bandwidth requirement.

S_{ab} storage average bandwidth requirement. /newline S_c workload capacity requirement.

D_c device capacity.

D_b device bandwidth.

P_s device standby power consumption.

P_b device busy power consumption.

E the energy power consumption of a workload.

T the run-time of a workload.

N the number of devices needed by the workload.

α Power management efficiency factor.

The number of devices required is:

$$N = \lceil \max(\frac{S_{pb}}{D_b}, \frac{S_c}{D_c}) \rceil.$$

In the case of bandwidth bound workload the number of active devices is: $N_a = \alpha S_{sb}/D_b$ and the energy consumption for a workload taking T time units is:

$$E = T \times P_i(N - N_a) + T \times P_b \times N_a.$$

Random I/O requests are spread over all devices, hence: $E = T \times N \times P_b$.

In the case of capacity bound workloads $N_a = \alpha S_{ab}/D_b$

$$E = T \times p_i \times (N - N_a) + T \times P_b \times N_a$$

In the case of random I/Os all devices need to be kept active

$$E = T \times p_b \frac{S_c}{D_c}$$

where T is the average run time of the workload.

The following three devices were considered in energy use measurements: two latest HDDs and an SSD are included to illustrate the model in practice. HD1: Seagate Desktop T4000M000, 4 TB, HD2 Samsung 2.5 inches Spinpoint 2 TB, SDD Samsung 840 EVO 1 TB.

For sequential write workload measurement results show that $E_{HD1} > E_{SSD1}$ for $T > 8727$ seconds or 2.4 hours, i.e., an SSD would be more efficient in this case.

Contrary to common belief, SSD is not the energy efficient choice for many workloads. Three other workloads considered are Sequential and Random Read Workloads and Random read/write (1:1) workload.

7.41 Energy proportionality is required in addition to energy efficiency

A cluster of wimpy computing nodes called WattDB is proposed by Theo Haerder et al. (Kaiserslautern U.) in (Härder et al., 2011). The individual nodes are dynamically attached and detached to the cluster on demand depending on the current workload needs thereby aiming at energy-proportionality. Some of the findings of this study are as follows:

1. *Demand-based Flash Translation - DFTL* is the most energy-efficient address-mapping scheme, due to its good write amplification and small DRAM footprint.
2. A significant fraction of energy is being consumed by idle flash chips waiting for the completion of NAND operations in the other channels.
3. FTL should be designed to fully exploit the internal parallelism so that energy consumption by idle chips is minimized.
4. Increasing way parallelism (the number of flash chips on a channel) is more effective than increasing channel parallelism in terms of peak energy consumption, performance, and hardware complexity.
5. In designing high-performance and energy-efficient SSDs, channel switching delay, way switching delay, and page write latency need to be incorporated in an integrated manner to determine the optimal configuration of internal parallelism.

7.42 SDD design tradeoffs from energy perspective

In a recent comprehensive study the energy consumption characteristics of various SSD design parameters are investigated by Seokhei Cho in (Cho et al., 2015). A more accurate energy consumption model is used for SSD components: microcontroller, DRAM, NAND flash, and host interface.

Five different FTLs are considered:

- (1) page mapping,
- (2) DFTL (Demand-based FTL) (Gupta et al., 2009),
- (3) block mapping, and
- (4,5) two different hybrid mappings),

and four different channel configurations (two, four, eight, and 16 channels) under seven different workloads (from large-scale enterprise systems to small-scale desktop applications).

Due to the energy consumption/resource utilization characteristics of today's centralized DB servers, the fastest configuration is also the most energy-efficient. Extensive use of SSDs alone cannot enable a fundamental change, because the storage-related energy consumption is typically only a small fraction of the overall energy budget. Even, when this storage-related share is (almost) completely reduced by optimized flash-aware buffer management, the saving effect achieved may be limited by less than 10%.

NVQuery is a *NonVolatile Memory - NVM*-based query accelerator, which performs several basic query functions in memory including aggregation, prediction, bit-wise operations, join operations, as well as exact and nearest distance search queries (Imani et al., 2019). NVQuery is implemented on a content addressable memory and exploits the analog characteristic of NVM in order to enable in-memory processing. To implement nearest distance search in memory, a novel bitline driving scheme is implemented to give weights to the indices of the bits during the search operation. To further improve the energy efficiency, the design supports configurable approximation by adaptively putting memory blocks under voltage overscaling. Experimental evaluation shows that a NVQuery can provide 49.3X performance speedup and 32.9X energy savings as compared to running the same query on traditional processor. Approximation improves the *Energy-Delay Product - EDP* of NVQuery by 7.3X, while providing acceptable accuracy. In addition, NVQuery can achieve 30.1X EDP improvement as compared to the state-of-the-art query accelerators.

7.43 Green AI

Deep Learning - DL is discussed in Chapter 8. In this section we review the discussion of power consumption in DL (Schwartz et al., 2020), which states that the computations required for DL research have been doubling every few months, resulting in an estimated 300,000X increase from 2012 to 2018. DL is inspired by the human brain, which is remarkably energy efficient. The financial cost of the computations can make it difficult for academics, students, and researchers, in poor countries to engage in DL research.

The paper advocates making efficiency an evaluation criterion for research along-side accuracy and related measures. The authors propose reporting the financial cost or “price tag” of developing, training, and running models to provide baselines for the investigation of increasingly efficient methods.

The term Green AI refers to AI research that yields novel results without increasing computational cost, and ideally reducing it. Several measures of efficiency are as follows:

Carbon emission: This depends on how electricity is produced, how about hydro, wind, and nuclear power.

Electricity usage: This measure is hardware dependent and disallows a fair comparison.

Elapsed real-time: This measure is highly influenced by factors such as the underlying hardware, other jobs running on the same machine, and the number of cores used. These factors hinder the comparison between different models.

Number of parameters: Another common measure of efficiency is the number of parameters used by the model. As with run time, this measure is correlated with the amount of work. Unlike the other measures it does not depend on the underlying hardware. Moreover, this measure also highly correlates with the amount of memory consumed by the model.

Number of Floating Point Operations - FPO has appealing properties as follows:

- (1) Directly computes the amount of work done when executing a specific instance of a model and is thus tied to the amount of energy consumed.
- (2) FPO is agnostic to the hardware on which the model is run, facilitating fair comparisons between different approaches, unlike the measures described above.
- (3) FPO is strongly correlated with the running time of the model. FPO also considers the amount of work done at each time step. Several packages exist for computing FPO in various neural network libraries.

Green AI is an emerging focus at the Allen Institute for AI and has the goal to make AI both greener and inclusive - “enabling any inspired undergraduate with a laptop to write high-quality research papers.”

Blog post by Dario Amodei and Danny Hernandez

The blog post running since 2012 has collected the amount of compute used in the largest AI training runs.

<https://openai.com/blog/ai-and-compute/>

It has been increasing exponentially with a 3.4-month doubling time (by comparison, Moore’s Law had a 2-year doubling period). Since 2012, this metric has grown by more than 300,000x. Improvements in compute have been a key component of AI progress, so as long as this trend continues, it’s worth preparing for the implications of systems far outside today’s capabilities.

The three factor advances of AI are: (1) algorithmic innovation, (2) data (which can be either supervised data or interactive environments), (3) the amount of compute available for training. The first two factors are difficult to track,

but compute is quantifiable and provides an opportunity to measure one input to AI progress. Massive compute just exposes the shortcomings of our current algorithms, but within many current domains leads predictably to better performance, and is often complementary to algorithmic advances.

The relevant number is not the speed of a single GPU, nor the capacity of the biggest datacenter, but the amount of compute that is used to train a single model, this is the number most likely to correlate to how powerful our best models are. Compute per model differs greatly from total bulk compute because limits on parallelism (both hardware and algorithmic) have constrained how big a model can be or how much it can be usefully trained. Important breakthroughs are still made with modest amounts of compute - this analysis just covers compute capability.

The trend represents an increase by roughly a factor of 10 each year. It's been partly driven by custom hardware that allows more operations to be performed per second for a given price (GPUs and TPUs), but it's been primarily propelled by researchers repeatedly finding ways to use more chips in parallel and being willing to pay the economic cost of doing so.

Looking at the graph we can roughly see four distinct eras:

Before 2012: It was uncommon to use GPUs for ML, making any of the results in the graph difficult to achieve. The graph at the website gives the total amount of compute versus year using a logarithmic scale (powers of ten), in petaflops-days, used to train selected results that are relatively well known, used a lot of compute for their time, and gave enough information to estimate the compute used.

2012 to 2014: Infrastructure to train on many GPUs was uncommon, so most results used 1-8 GPUs rated at 1-2 TFLOPS for a total of 0.001-0.1 pfs-days.

2014 to 2016: Large-scale results used 10-100 GPUs rated at 5-10 TFLOPS, resulting in 0.1-10 pfs-days. Diminishing returns on data parallelism meant that larger training runs had limited value.

2016 to 2017: Approaches that allow greater algorithmic parallelism such as huge batch sizes, architecture search, and expert iteration, along with specialized hardware such as TPU's and faster interconnects, have greatly increased these limits, at least for some applications.

AlphaGoZero/AlphaZero is the most visible public example of massive algorithmic parallelism, but many other applications at this scale are now algorithmically possible. Leela Zero is an open source program based on the ideas from the AlphaGo papers is available on GPUs.

https://en.wikipedia.org/wiki/AlphaGo_Zero

RED AI refers to AI research that seeks to obtain state-of-the-art results in accuracy through the use of massive computational power. The relationship between model performance and complexity (measured as number of parameters or inference time) has long been understood to be at best logarithmic; for a linear gain in performance, an exponentially larger model is required. Similar trends exist with increasing the quantity of training data and the number of experiments. In each of these cases, diminishing returns come at increased computational cost.

Scientific papers typically include a model trained on a training dataset and evaluated on a test dataset. The process of developing that model often involves multiple experiments to tune its hyperparameters. There are three factors that increase the computational and environmental cost of producing such a result:

The cost of executing the model on a single (**E**)xample (during training or at inference time);

The size of the training (**D**)ataset, which controls the number of times the model is executed during training;

The number of (**H**)yperparameter experiments, which controls how many times the model is trained during model development.

The total cost of producing a (**R**)esult in machine learning increases linearly with each of these quantities. This cost can be estimated as follows:

$$\text{Cost}(R) \propto E \times D \times H. \quad (7.2)$$

Eq. (7.2) simply states that the cost of an AI (**R**)esult grows linearly with the cost of the three parameters.

7.44 Conclusions

We have reviewed publications dealing with methods to save energy in storage systems and servers in general. Novel methods are under development and readers are encouraged to follow emerging research on this topic. Unfortunately, the sixth and last Hotpower workshop was held in 2014,

TABLE 7.16 Advantages of 2PIC over conventional air cooling.

	Increased power density per rack	More computing power in less space	Dramatically less energy used for cooling
Air	4-40 KW	$\leq 10 \text{ KW/m}^2$	1.1-2.0 PUE
2PIC	$\leq 250 \text{ KW}$	$\leq 100 \text{ KW/m}^2$	1.02

<https://www.usenix.org/conference/hotpower14>

Power Usage Effectiveness - *PUE* is a ratio that specifies the efficiency of a computer data center uses energy; i.e., the amount of energy used by the computing equipment in contrast to cooling, lighting, HVAC, etc.

https://en.wikipedia.org/wiki/Power_usage_effectiveness

$$\text{PUE} = \frac{\text{Total Facility Energy}}{\text{IT Equipment Energy}} = 1 + \frac{\text{Non-IT Facility Energy}}{\text{IT Equipment Energy}}.$$

PUE was developed by the Green Grid consortium in 2016 and published as a global standard. Google is a leader in achieving a low PUE of 1.09.

<https://www.google.com/about/datacenters/efficiency/#measuring-efficiency>

A lower PUE of 1.06 was attained by Supermicro at a datacenter with 30K blades.

https://www.supermicro.com/newsroom/pressreleases/2017/press170206_MicroBlade_Key_Win.cfm

3M 2-phase immersion cooling - 2PIC Novec fluid achieves PUE = 1.02.

<https://multimedia.3m.com/mws/media/1127920O/2-phase-immersion-cooling-a-revolution-in-data-center-efficiency.pdf>

Advantages of 2PIC over conventional air cooling are given in Table 7.16.

US Energy Information Administration - EIA provides Table 7.17 for electricity generation by energy source.

<https://www.eia.gov/tools/faqs/faq.php?id=427&t=3>

TABLE 7.17 U.S. utility-scale electricity generation by source, amount, and share of total in 2019.

Energy source	Billion KWH	Share of total
Total - all sources	4,127	
Fossil fuels (total)	2,582	62.6%
Natural Gas	1,586	38.4%
Coal	965	23.4%
Petroleum liquid + coke	18 = 12 + 6	0.4%
Other gases	13	0.3%
Nuclear	809	19.6%
Renewables (total)	728	17.6%
Wind	295	7.1%
Hydropower	288	7.0%
Solar	72	1.7%
Photovoltaic	69	1.7%
Solar thermal	3	0.1%
Biomass (total)	58	1.4%
Wood	39	0.9%
Landfill gas	10	0.3%
Waste	8	0.2%
Geothermal	15	0.4%
Pumped hydropower	−5	−0.1%
Other sources	13	0.3%

The cooling of the Summit computer at the *Oak Ridge National Lab - ORNL* is discussed below.

<https://www.wired.com/story/the-astonishing-engineering-behind-americas-latest-greatest-supercomputer/>

It is not just the water but the electricity generated by hydropower plants that is used for cooling. This is the main reason that supercomputers are placed close to hydropower stations. There are many positive aspects of hydropower, as follows.

<https://www.hydroquebec.com/data-center/about.html>

yet hydropower can be bad for the climate as argued below.

<http://blogs.edf.org/energyexchange/2019/11/15/>

The cooling of the Summit computer at the *Oak Ridge National Lab - ORNL* is discussed below.

<https://www.wired.com/story/the-astonishing-engineering-behind-americas-latest-greatest-supercomputer/>

It is not just the water but the electricity generated by hydropower plants that is used for cooling. This is the main reason that supercomputers are placed close to hydropower stations. There are many positive aspects of hydropower, as mentioned below,

<https://www.hydroquebec.com/data-center/about.html>

yet hydropower can be bad for the climate as argued below.

<http://blogs.edf.org/energyexchange/2019/11/15/>

This future forecast projects that by 2100 $\approx 90\%$ of energy production will be renewable, 9% hydro, and 1% nuclear.

<https://www.futuretimeline.net/data-trends/15.htm>

However, the future is hard to predict and possibly the graphs do not take into account clean power due to fusion.

https://en.wikipedia.org/wiki/Fusion_power

<https://www.theguardian.com/environment/2020/jul/28/worlds-largest-nuclear-fusion-project-under-assembly-in-france>

<https://sputniknews.com/science/202012071081387297-photos-china-turns-on-futuristic-artificial-sun-nuclear-fusion-reactor/>

<https://www.economist.com/science-and-technology/2021/06/24/the-race-to-build-a-commercial-fusion-reactor-hots-up>

Chapter 8

Database parallelism, big data and analytics, deep learning

Stonebraker's classification of computer systems in Section 8.1
Comparison of systems from the viewpoint of CPU performance in Section 8.2
High performance network and channel-based interconnects for storage in Section 8.3
Concurrency and coherency control in datasharing systems in Section 8.4
Combined shared disk and shared nothing systems in Section 8.5
Parallel systems at IBM Research in Section 8.6
Interconnection networks of IBM's BlueGene/L in Section 8.7
Data allocation and transaction routing in multicomputers in Section 8.8
Data allocation in distributed relational databases in Section 8.9
Computers for database applications in Section 8.10
Tandem Corporation in Subsection 8.10.1
Teradata Corporation in Subsection 8.10.2
Technologies in Subsection 8.10.3
Dell Corporation in Subsection 8.10.4
Benchmarking in various forms in Section 8.11
Transaction Processing Council - TPC in Subsection 8.11.1
Storage Performance Council - SPC in Subsection 8.11.2
Standard Performance Evaluation Corporation - SPEC in Subsection 8.11.3
Benchmarking distributed databases in Subsection 8.11.4
Mixed workload benchmarks: OLTP and OLAP in Subsection 8.11.5
Sort benchmarks are discussed in Subsection 8.11.6
Data base machines and backend processors in Section 8.12
Head-per-track disks in Section 8.13
Rotating Associative Processor for Information Dissemination - RAPID in Subsection 8.13.1
Context addressable cellular system for non-numeric processing in Subsection 8.13.2
Rotating Associative Relational Store - RARES in Subsection 8.13.3
The Context-Addressed Segment-Sequential Storage - CASSS in Subsection 8.13.4
Relational Associative Processor - RAP in Subsection 8.13.5
Number of rotations required to read qualifying tuples from RAP in Subsection 8.13.6
Active disk projects are discussed in Section 8.14
Project at CMU in Subsection 8.14.1
Projects at UC Santa Barbara/Maryland 8.14.2
Intelligent disks - IDisk project at UC Berkeley 8.14.3
SmartSTOR project at IBM Almaden Research Center in Subsection 8.14.4
Smart disk cluster project at Northwestern U. in Subsection 8.14.5
Integration of Server, Storage and Database Stack at IBM Almaden in Subsection 8.14.6
Active disk summary in Subsection 8.14.7
Active storage revisited in Subsection 8.14.8
Multidimensional indices on disk, DRAM, and flash in Section 8.15
Survey of multidimensional indices in Subsection 8.15.1
Combining dimensionality reduction with clustering in Subsection 8.15.2
Dimensionality reduction via SVD, PCA, KLT in Subsection 8.15.3
Clustering methods for dimensionality reduction in Subsection 8.15.4

Building an index via CSVD for k -NN queries in Subsection 8.15.5
 Nearest-neighbors query processing with multiple clusters in Subsection 8.15.6
 Exact versus approximate k -NN queries in Subsection 8.15.7
 Indices suited for dimensionality reduced data in Subsection 8.15.8
 Ordered Partition - OP tree index in Subsection 8.15.9
 Stepwise Dimensionality Increasing SDI-tree index in Subsection 8.15.10
 Implementing indices in flash memories in Section 8.16
 Redesign of relational databases by Stonebraker et al. in Section 8.17.
 Parallel and modern databases in Section 8.18
 Almaden Research Backend Relational Engine - ARBRE in Subsection 8.18.1
 IBM's DB2 Parallel Edition - PE in Subsection 8.18.2
 Google's BigTable and Spanner RDBMS in Section 8.19
 Microsoft's Azure in Subsection 8.20
 Amazon's Dynamo DB in Subsection 8.20.1
 Aurora database for processing continual inputs from sensors in Subsection 8.20.2
 IBM and other cloud service providers in Section 8.21
 Distributed databases in cloud computing in Section 8.22
 SpringFS bridging agility and performance in elastic distributed storage in Section 8.23
 Snowflake cloud based data warehousing with SQL support in Section 8.24
 Storage architecture and provisioning in Snowflake in Subsection 8.24.1
 Resource sharing in Snowflake in Subsection 8.24.2
 Review of peer-to-peer computing in Section 8.25
 Fast Array of Wimpy Nodes - FAWN project at CMU in Subsection 8.26
 RAMCloud project at Stanford U. in Section 8.27
 How flash changes the design of database storage engines in Section 8.28
 Hybrid Transaction and Analytic Processing - HTAP in Subsection 8.29
 Intelligent page store for concurrent transaction and query processing in Section 8.30
 Oracle's Exadata DBM in Section 8.31
 Oracle in memory option or Database in Main Memory - DBIM in Section 8.32
 MemSQL/SingleStore in Section 8.33
 Amazon Aurora DBMS in Section 8.34
 Transaction processing in the cloud in Section 8.35
 RAPID and Oracle AutoML: a fast and predictive AutoML pipeline in Section 8.36
 Benchmarking automatic ML frameworks in Section 8.37
 Alibaba X-engine in Section 8.38
 RocksDB with ultrafast data access in Section 8.39
 LightStore project at MIT in Section 8.40
 Pink: High-speed in-storage key-value store with bounded tails in Section 8.41
 BlueDBM appliance for big data analytics in Section 8.42
 WiSer highly available HTAP DBMS for IoT applications in Section 8.43
 Raven RDBMS at Microsoft providing ML in Section 8.44
 Machine Learning data platform - MLdp in Section 8.45
 Databricks in Section 8.46
 Fungible - a new storage architecture for big data in Section 8.47
 Network requirements for resource disaggregation in Section 8.48
 Deep learning and associated hardware in Section 8.49
 GPU accelerated database systems in Section 8.50
 Graphics Processing Unit - GPU solutions in Section 8.51
 Field Programmable Gate Array - FPGA solutions in Section 8.52
 Multichip modules in Section 8.53
 Unified solutions in Section 8.54
 Power consumption in FPGAs and ASICs in Section 8.55
 Hybrid approaches to acceleration in Section 8.56
 Application Specific Integrated Circuits - ASICs in Section 8.57

Tensorflow and Tensor Processing Units - TPUs in Section 8.58
 Increasing computational challenges in Section 8.59
 Quantum Neural Nets - QNNs in Section 8.60
 Data acceleration examples in Section 8.61
 The Q100 data processing Unit - DPU in Subsection 8.61.1
 Many-core architecture for in-memory data processing at Oracle in Subsection 8.61.2
 Cerebras wafer size chips versus GPUS in Section 8.62
 The deep learning workload in Subsection 8.62.1
 Understanding performance in Subsection 8.62.2
 More silicon area means more space for compute cores in Section 8.62.3
 More silicon area means more on chip memory in Subsection 8.62.4
 Fast communication SwarmTM fabric in Subsection 8.62.5
 Co-designed with software for maximum utilization and usability in Subsection 8.62.6
 Conclusions in Section 8.63

8.1 Stonebraker's classification of computer systems

Workload demands especially for database applications tend to increase more rapidly than CPU speeds according to a keynote by Sun Microsystems CTO - Chief Technical Officer Greg Papadopoulos in 1990s (Keeton et al., 1998). The demand for decision support doubles every 9 to 12 months, making its growth faster than both the growth rate of disk capacity and processor performance (2X in 18 months). This suggests a demand for server designs that can scale with beyond the evolutionary path of the servers at the time.

https://en.wikipedia.org/wiki/Greg_Papadopoulos

If database application requirements such as *transaction* - *txn* throughput exceed that of a single mainframe then multiple computer systems are configured to meet load requirements. Michael Stonebraker's categorized multicomputers classified by Michael Flynn as *Multi-Instruction, Multi-Data stream* - *MIMD* computers (Flynn, 1972) into three categories (Stonebraker, 1986).

Shared-everything systems are multiprocessors with shared main memory and disks.

<https://en.wikipedia.org/wiki/Multiprocessing>

There is performance degradation due to a cache coherence and contention for acquisition of latches via spinlocks (Anderson, 1990).

<https://en.wikipedia.org/wiki/Spinlock>

Cache coherence schemes in multiprocessors are discussed in (Culler et al., 1999), (Hennessy and Patterson, 2017), (Nagarajan et al., 2020).

Multiprocessors may have the *Uniform or Nonuniform Memory Access* - *UMA/NUMA* (Culler et al., 1999), (Hwang and Jotwani, 2015).

https://en.wikipedia.org/wiki/Uniform_memory_access

https://en.wikipedia.org/wiki/Non-uniform_memory_access

Modern microprocessors provide multiple cores on one chip, where each core can handle (2-4) threads (Shen and Lipasti, 2013).

[https://en.wikipedia.org/wiki/Multithreading_\(computer_architecture\)](https://en.wikipedia.org/wiki/Multithreading_(computer_architecture))

Three multithreading options are:

- (1) coarse - run another thread if a thread stalls due to a cache miss,
- (2) interleaved - by alternating in issuing instructions from different threads,
- (3) exploit parallelism available across multiple threads to decrease the waste associated with unused issue slots.

Shared disk or data-sharing systems were introduced when the fastest mainframe computers (in spite of multiprocessing) were unable to meet the requirements of the workload, such as *txn* throughput for OLTP. More than one computer could provide a sufficient processing power for higher loads. Disk data may be shared via cached controllers and extended storage, which can be accessed as 4 KB or larger blocks (Rahm, 1992). Scalability in System/390 is offered via the Parallel Sysplex (Nick et al., 1996), which allows up to 32 systems, enabling a workload to be spread transparently across a collection of S/390 systems with shared access to data. The S/390 sysplex offers near-linear scalability and continuous availability.

https://en.wikipedia.org/wiki/IBM_Parallel_Sysplex

Another example of data sharing is the VAX-cluster from the now defunct *Digital Equipment Corporation - DEC* (Kronenberg et al., 1986).

VAXclusters allowed terminals to access any host node via the Ethernet and failovers from one node to another were achievable easily (Kronenberg et al., 1986). The cluster as a whole *never* went down, since upgrades of OpenVMS software allowed shutting down, upgrading, and rebooting individual nodes, while the cluster as a whole continued processing, so that uptimes of sixteen years were measured. Greg Pfister who was the chief architect of IBM's RP3, see Subsection 8.6, later authored a highly readable text on cluster computing (Pfister, 1998).

https://en.wikipedia.org/wiki/Computer_cluster

Shared-nothing means neither main memory, disks, or other form of storage are shared.

<https://en.wikipedia.org/wiki/Ethernet>

Ethernet competes with Infiniband in interconnecting supercomputers.

<https://en.wikipedia.org/wiki/InfiniBand>

Tandem NonStop - TNS and *Teradata DC/1012* in Chapter 1 are two examples of shared nothing systems. Recent books on interconnection networks are as follows (Duato et al., 2003), (Dally and Towles, 2004), (Rojas-Cessa, 2017).

8.2 Comparison of systems from the viewpoint of CPU performance

Txns with Poisson arrivals alternate between processing at the CPU and the disks. This can be represented as an open *Central Server Model - CSM* (Buzen, 1971), which is a product-form *Queueing Network Model - QNM* whose nodes can be analyzed separately (Kleinrock, 1975), (Lazowska et al., 1984). We assume that disks have low utilizations and can be considered as delay servers with no queueing or at least do not constitute bottlenecks.

Let $\bar{y}$ denote the mean service time per visit at the CPU and $\bar{v}$ the mean number of txn visits to the CPU (v), so that the CPU service demand or loading is $\bar{x} = \bar{v}\bar{y}$ and the CPU utilization is $\rho = \lambda\bar{x}$, where λ is the arrival rate of txn arrivals (Kleinrock, 1975). The CPU can be modeled as an M/M/1 queue so that its mean response time is $r_1 = \bar{y}/(1 - \rho)$ for $\rho < 1$ (Kleinrock, 1975) and the mean txn time spent at the CPU over its lifetime is $R_1 = vr_1 = \bar{x}/(1 - \rho)$. For $\rho > 0.5$ the CPU will be overloaded if the txn arrival rate is doubled.

A multiprocessor with two CPUs can be modeled as an M/M/2 queueing system. A doubled arrival rate will maintain the same processor utilization, i.e., $\rho_{2way} = (2\lambda)\bar{x}/2$ as before. In practice the contribution of both CPUs to performance is lowered due to the degradation caused by cache coherence or busy waiting for latches to acquire main memory blocks, e.g., given that the processing power of a single CPU is one, the second processor contributes 1.8 speedup, i.e., a state-dependent server (Lavenberg, 1983). Such coefficients used in capacity planning packages such as BEST/1 by Jeff Buzen et al. (Buzen, 1978) and MAP by Ed Lazowska et al. (Lazowska et al., 1984).

The Argus mainframe capacity planning package developed by Alex Birman at IBM Research, was withdrawn even after its manual was published, since IBM would be required to match Argus' performance predictions. IBM has since introduced a capacity planning tool for z systems in 2018. 5698-AS1 IBM Z Decision Support for Capacity Planning V2.1, https://www-01.ibm.com/common/ssi/ShowDoc.wss?docURL=/common/ssi/rep_sm/1/649/ENUS5698-AS1/index.html&lang=en&request_locale=en

The mean response time for a 2-way multiprocessor modeled as an M/M/2 queue is $R_2 = \bar{x}/(1 - \rho^2)$ (Kleinrock, 1975). If $\rho = 0.9$ in the case of single processor systems $R_1 = 10\bar{x}$, while $R_2 \approx 5\bar{x}$, since not all txns have to wait because one of two servers may be idle.

With double the arrival rate and a CPU twice as fast then $\rho_{fast} = 2\lambda(\bar{x}/2) = \rho$ and $R_{fast} = (\bar{x}/2)/(1 - \rho)$, so that $R_{fast} < R_2$. It is observed that 2-way multiprocessor and a single fast processor provide a lower mean CPU response time than data sharing, which amounts to two computers with half of the load, but a twice as fast processor might not be attainable.

The above conclusion applies to M/M/m but does not to G/G/m m-server systems with general interarrival and service times for which there are no explicit solutions. Several bounds and approximations are given in (Kleinrock, 1976), where it is mentioned that it is shown in (Brumelle, 1971) that when the coefficient of variation $c_p \leq 1$, $m = 1$ is optimum in $G/G/m$, but an example is provided where $G/G/2$ is superior to $G/G/1$ when $c_p > 1$.

8.3 High performance network and channel-based interconnects for storage

We start our discussion with a report written for NASA by Randy Katz, from which the following Abstract and Tables 8.1 and 8.2 are quoted (Katz et al., 1989, 2001).

<https://ntrs.nasa.gov/api/citations/19920010018/downloads/19920010018.pdf>

Abstract

In the traditional mainframe-centered view of a computer system, storage devices are coupled to the system through complex hardware subsystems called I/O channels. With the dramatic shift towards workstation-based computing, and its associated client/server model of computation, storage facilities are now found attached to file servers and distributed throughout the network. In this paper, we discuss the underlying technology trends that are leading to high performance network-based storage, namely advances in networks, storage devices, and I/O controller and server architectures. We review several commercial systems and research prototypes that are leading to a new approach to high performance computing based on network-attached storage.

TABLE 8.1 Comparison of Network, Channel, and Backplane attributes. The comparison is based upon the interconnection distance, transmission bandwidth, transmission latency, inherent reliability, and typical techniques to improve data integrity.

	Network	Channel	Backplane
Distance	>1000 m	10-100 m	1 m
Bandwidth	10-100 Mb/s	40-100 Mb/s	320-1000+ Mb/s
Latency	high (>ms)	medium	low (µsec)
Reliability	low	medium	high
	extensive CRC	Byte parity	Byte parity

SCSI - Small Computer System Interface is a set of standards for physically connecting and transferring data between computers and peripheral devices.

TABLE 8.2 Comparisons among popular backplane and channel interconnects, S/M=Single/Multiple.

Metric	VME	Futurebus	Multibus	SCSI-I
Bus Width	128	96	96	25
Address/Data Multiplexed?	no	Yes	Yes	na
Data Width	16-32	32	32	8
Xfer Size	S/M	S/M	S/M	S/M
# of Bus Masters	Multiple	Multiple	Multiple	Multiple
Split txns	No	Optional	Optional	Optional
Clocking	Async	Async	Suc	Either
Bandwidth, Single Word (0 ns mem)	25	37	20	5.15
Bandwidth, Single Word (150 ns mem)	12.9	15.5	10	5.15
Bandwidth Multiple Word (0 ns mem)	27.9	95.2	40	5.15
Bandwidth Multiple Word (150 ns mem)	13.6	20.8	13.3	5.15
Max # of devices	21	20	21	7
Max Bus Length	0.5 m	.5 m	0.5 m	25 m
Standard	IEEE 1014	IEEE 896	ANSI/IEEE 1296	ANSI X3.131

SCSI - Small Computer System Interface is a set of standards for physically connecting and transferring data between computers and peripheral devices.

Serial Attached SCSI - SAS point to point serial protocol has the following characteristics:

https://en.wikipedia.org/wiki/Serial_Attached_SCSI

1. 2.4 GB/s effective single-lane bandwidth (22.5 Gbaud rate)
2. Unmatched Scalability and Reach
3. 20-bit *Forward Error Correction - FEC* for more robust reliability
4. Enhanced transmitter training algorithm for better signal integrity
5. System-level manageability
6. Fairness enhancements for performance consistency across protocol topologies

High Performance Parallel Interface - HiPPI was originally developed at the Los LANL - Alamos National Laboratory in the mid-1980s as a high speed unidirectional (simplex) point-to-point interface between supercomputers.

InfiniBand - IB is a computer networking communications standard used in high-performance computing that features very high throughput and very low latency. It is used for data interconnect both among and within computers. IB is also used as either a direct or switched interconnect between servers and storage systems, as well as an interconnect between storage systems. It is designed to be scalable and uses a switched fabric network topology. It was developed by the InfiniBand Trade Association (IBTA) to provide the levels of reliability, availability, performance, and scalability necessary for present and future server systems, levels significantly better than can be achieved with bus-oriented I/O structures (Pfister, 2001)

Abstract

In the traditional mainframe-centered view of a computer system, storage devices are coupled to the system through complex hardware subsystems called I/O channels. With the dramatic shift towards workstation-based computing, and its associated client/server model of computation, storage facilities are now found attached to file servers and distributed throughout the network. In this paper, we discuss the underlying technology trends that are leading to high performance network-based storage, namely advances in networks, storage devices, and I/O controller and server architectures. We review several commercial systems and research prototypes that are leading to a new approach to high performance computing based on network-attached storage.

IB since 2014 has been the most commonly used interconnect in supercomputers, but in 2016, Ethernet replaced InfiniBand as the most popular system interconnect of TOP500 supercomputers. Mellanox (now Nvidia Networking) manufactures InfiniBand host bus adapters and network switches, and, in February 2016 it was reported that Oracle Corporation had engineered its own InfiniBand switch units and server adapter chips. Mellanox IB cards are available for Solaris, FreeBSD, RHEL, SLES, Windows, HP-UX, VMware ESX, and AIX.

https://www.mellanox.com/pdf/whitepapers/IB_Intro_WP_190.pdf

IB competes with Ethernet, Fibre Channel, and Intel Omni-Path. The technology is promoted by the InfiniBand Trade Association.

<https://en.wikipedia.org/wiki/InfiniBand>

10 Gb Ethernet (10GE, 10GbE, or 10 GigE) is a group of computer networking technologies for transmitting Ethernet frames at a rate of 10 Gb/s first defined by the IEEE 802.3ae-2002 standard. It defines only full-duplex point-to-point links which are generally connected by network switches; shared-medium *Carrier Sense Multiple Access/Collision Detection - CSMA/CD*.

The 10 Gigabit Ethernet standard encompasses a number of different physical layer (PHY) standards. A networking device, such as a switch or a network interface controller may have different PHY types through pluggable PHY modules. 10GbE can use either copper or fiber cabling. Maximum distance over copper cable is 100 meters.

https://en.wikipedia.org/wiki/10_Gigabit_Ethernet

Intel Omni-Path or *Omni-Path Architecture - OPA* is a high-performance communication architecture owned by Intel, which aims for low communication latency, low power consumption and a high throughput. Intel planned to develop technology based on this architecture for exascale computing. In 2017, Intel was offering at least 7 variations of multi-port Ethernet switches using supporting 100 Gb/s for all ports.

RDMA over Converged Ethernet - RoCE is a network protocol that allows *Remote Direct Memory Access - RDMA* over an Ethernet network. It does this by encapsulating an IB transport packet over Ethernet. RoCE v1 is an Ethernet link layer protocol and hence allows communication between any two hosts in the same Ethernet broadcast domain. RoCE v2 is an internet layer protocol which means that RoCE v2 packets can be routed. Although the RoCE protocol benefits from the characteristics of a converged Ethernet network, the protocol can also be used on a traditional or non-converged Ethernet network.

NVMexpress family of specifications define how host software communicates with *Non-Volatile Memory - NVM* across multiple transports like PCI Express (PCIe), RDMA, TCP and more. It is the industry standard for SSDs in all form factors). NVM Express is the non-profit consortium of tech industry leaders defining, managing and marketing NVMe technology. [NVMexpress.org](https://nvmexpress.org)

The latest NVMe 2.0 library of specifications consists of multiple documents: (1) NVMe Base specification, (2) Command Set specifications (NVM Command Set specification, (3) ZNS Command Set specification, KV Command Set specification), (4) Transport specifications (PCIe Transport specification, (5) Fibre Channel Transport specification, (6) RDMA Transport specification, (7) TCP Transport specification), (8) the NVMe Management Interface specification.

The NVMe specification was designed from the ground up for SSDs. It is a much more efficient interface, providing lower latency, and is more scalable for SSDs than legacy interfaces, like *Serial ATA - SATA*. The first part of the

specification is the host control interface. The NVMe architecture brings a new high performance queuing mechanism that supports 65,535 I/O queues each with 65,535 commands (referred to as queue depth, or number of outstanding commands). Queues are mapped to CPU cores delivering scalable performance. The NVMe interface significantly reduces the number of memory-mapped input/output commands and accommodates operating system device drivers running in interrupt or polling modes for higher performance and lower latency. The NVMe specification also contains host-to-device protocol for SSD commands used by an operating system for: read, write, flush, TRIM, firmware management, temperature, errors and others.

NVM Express over Fabrics - NVMe-oF is the concept of using a transport protocol over a network to connect remote NVMe devices, contrary to regular NVMe where physical NVMe devices are connected to a PCIe bus either directly or over a PCIe switch to a PCIe bus. In August 2017, a standard for using NVMe over Fibre Channel (FC) was submitted by the standards organization *Int'l Committee for Information Technology Standards - ICITS*, and this combination is often referred to as FC-NVMe or sometimes NVMe/FC. As of May 2021, supported NVMe transport protocols are:

FC, FC-NVMe

TCP, NVMe/TCP

Ethernet, RoCE v1/v2 (RDMA over converged Ethernet)

InfiniBand, NVMe over InfiniBand or NVMe/IB

The standard for NVMe over Fabrics was published by NVM Express, Inc. in 2016. The following software implements the NVMe-oF protocol:

(1) Linux NVMe-oF initiator and target.

(2) RoCE transport was supported initially, and with Linux kernel 5.x, native support for TCP was added.

(3) *Storage Performance Development Kit - SPDK* NVMe-oF initiator and target drivers. Both RoCE and TCP transports are supported.

(4) Starwind NVMe-oF initiator and target for Microsoft Windows, supporting both RoCE and TCP transports.

AHCI - Advanced Host Controller Interface has the benefit of wide software compatibility, but has the downside of not delivering optimal performance when used with SSDs connected via the PCI Express bus. As a logical-device interface, AHCI was developed when the purpose of a host bus adapter (HBA) in a system was to connect the CPU/memory subsystem with a much slower storage subsystem based on rotating magnetic media. As a result, AHCI introduces certain inefficiencies when used with SSD devices, which behave much more like RAM than like spinning media.

The NVMe device interface has been designed from the ground up, capitalizing on the low latency and parallelism of PCI Express SSDs, and complementing the parallelism of contemporary CPUs, platforms and applications. At a high level, the basic advantages of NVMe over AHCI relate to its ability to exploit parallelism in host hardware and software, manifested by the differences in command queue depths, efficiency of interrupt processing, the number of uncacheable register accesses, etc., resulting in various performance improvements.

High-end SSDs were using the PCI Express bus before NVMe,

https://en.wikipedia.org/wiki/NVM_Express

The NVM Express - NVMe family of specifications define how host software communicates with non-volatile memory across multiple transports like PCI Express® (PCIe®), RDMA, TCP and more. It is the industry standard for SSDs.

<https://nvmexpress.org/>

Their differences are specified below and Table 8.3.

<https://blogs.cisco.com/datacenter/nvme-for-absolute-beginners>

By standardizing the interface of SSDs, OSs only need one driver to work with all SSDs adhering to the specification.

8.3.1 Intel Compute eXpress Link - CXL

Compute Express Link - CXL from Intel is being developed to supersede the PCIe bus and is envisaged by its developers as making pools of memory (DRAM + *Storage Class Memory - SCM*) sharable between CPUs and also GPUs.

<https://blocksandfiles.com/2021/03/25/cxl-and-the-developing-memory-hierarchy/>

Intel sees CXL as rack-level disaggregator with Optane connectivity, with the following usages.

<https://blocksandfiles.com/2021/08/18/intel-sees-cxl-as-rack-level-disaggregator/>

Caching Devices and Accelerators are accessed via *Network Interface Cards - NICs*.

Accelerators: Include GPU, FPGA with their own memory, such as *HBM - High Bandwidth Memory*.

Memory buffers for memory bandwidth and capacity expansion.

Additional information about CXL is given here.

<https://www.techdesignforums.com/practice/technique/compute-express-link-cxl-protocols/>

TABLE 8.3 High-level comparison of Advanced Host Controller Interface - AHCI and NVMe.

	AHCI	NVMe
Maximum queue depth	One command queue; 32 commands per queue	65535 queues; 65536 commands per queue
Uncacheable register accesses (2000 cycles each)	Six per non-queued command; nine per queued command	Two per command
Interrupt	A single interrupt	Up to 2048 MSI-X interrupts
Parallelism and multiple threads	Requires synchronization lock to issue a command	No locking
Efficiency for 4 KB commands	Command parameters require two serialized host DRAM fetches	Gets command parameters in one 64-byte fetch
Data transmission	Half-duplex	Full-duplex

A look at the key protocols that control the Compute Express Link (CXL) standard for connecting CPUs and accelerators in heterogeneous computing environments.

CXL is an open interconnect standard for enabling efficient, coherent memory accesses between a host, such as a CPU, and a device, such as a hardware accelerator, that is handling an intensive workload.

CXL is expected to be implemented in heterogeneous computing systems that include hardware accelerators addressing topics in artificial intelligence, machine learning, and other specialist tasks.

This article focuses on the CXL protocols.

1. The CXL standard defines three protocols that are dynamically multiplexed together before being transported via a standard PCIe 5.0 PHY at 32 GT/s.
2. The CXL.io protocol is an enhanced version of a PCIe 5.0 protocol that can be used for initialization, link-up, device discovery and enumeration, and register access. It provides a non-coherent load/store interface for I/O devices.
3. The CXL.cache protocol defines interactions between a host and a device, allowing attached CXL devices to efficiently cache host memory with extremely low latency using a request and response approach.
4. The CXL.mem protocol provides a host processor with access to the memory of an attached device using load and store commands, with the host CPU acting as a master and the CXL device acting as a subordinate. This approach can support both volatile and persistent memory architectures.

In data sharing systems cached pages can be transferred from buffer-to-buffer using *Channel to Channel - CTC* data transfers, which are more efficient than data communication protocols such as TCP/IP or IBM's *System Network Architecture - SNA* and take less time than reading the page from disk. Implementations include: *Remote Data Memory Access - RDMA*

https://en.wikipedia.org/wiki/Remote_direct_memory_access

Virtual Interface Architecture - VIA,

https://en.wikipedia.org/wiki/Virtual_Interface_Architecture

RDMA over Converged Ethernet - RoCE,

Intel's Omni-Path high performance communication architecture

<https://en.wikipedia.org/wiki/Omni-Path>

Internet Wide Area RDMA Protocol - iWARP that delivers low latency:

<https://en.wikipedia.org/wiki/iWarp>

<http://www.cs.cmu.edu/afs/cs/project/iwarp/archive/WWW-pages/iwarp.html>

Kernel Bypass: Removes need for context switching from kernel-space to user-space.

Direct Data Placement: Eliminates intermediate buffer copies by reading and writing directly to application memory.

Transport Acceleration: Performs transport processing on the network controller instead of the processor.

RDMA data transfers have the shortcoming that the receiving system is not notified when the data transfer is completed.

https://en.wikipedia.org/wiki/Remote_direct_memory_access

White paper specifying Computer eXpress Link - CXL

https://b373eaf2-67af-4a29-b28c-3aae9e644f30.filesusr.com/ugd/0c1418_14c5283e7f3e40f9b2955c7d0f60bebe.pdf

CXL is an open industry standard interconnect offering high-bandwidth, low-latency connectivity between host processor and devices such as accelerators, memory buffers, and smart I/O devices. It is designed to address the growing high-performance computational workloads by supporting heterogeneous processing and memory systems with applications in Artificial Intelligence, Machine Learning, Analytics, Cloud Infrastructure, Cloudification of the Network and Edge, communication systems, and High Performance Computing. It does this by enabling coherency and memory semantics on top of the PCI Express PCIe 5.0 based I/O semantics for optimized performance in evolving usage models. This is increasingly important as processing data in these emerging applications requires a diverse mix of scalar, vector, matrix and spatial architectures deployed in CPU, GPU, FPGA, smart NICs, and other accelerators.

CXL1.0 debuted in March 2019 supporting dynamic multiplexing between a rich set of protocols that includes I/O (CXL.io, based on PCIe), caching (CXL.cache), and memory (CXL.memory) semantics. CXL maintains a unified, coherent memory space between the CPU (host processor) and any memory on the attached CXL device. This allows both the CPU and device to share resources and operate on the same memory region for higher performance, reduced data-movement, and reduced software stack complexity. Moreover, since the CPU is primarily responsible for coherency management, it can reduce device cost and complexity, as well as overhead traditionally associated with coherency across an I/O link.

The speed of access by CXL is given in Table 8.4.

https://b373eaf2-67af-4a29-b28c-3aae9e644f30.filesusr.com/ugd/0c1418_14c5283e7f3e40f9b2955c7d0f60bebe.pdf

TABLE 8.4 Access times in a memory hierarchy via CLX.

CPU	10^0
DRAM via CLX 1.0/1.1	10
Persistent memory via CLX 2.0	10^2 - 10^3
Performance SSD	10^4
Capacity SSD	10^4
HDD	10^6

8.4 Concurrency and coherency control in data sharing systems

Data sharing systems require inter-system concurrency and coherency control as described by Erhard Rahm (Leipzig U.) in (Rahm, 1993). Coherency control is required, since a database page may be held in the database buffers of more than one computer. As in multiprocessor cache coherence (Culler et al., 1999) when a page is updated the corresponding block at the other system is discarded or it can be updated using the modified data. Asit Dan's PhD thesis dealing performance analysis of data sharing systems was carried out at IBM Research (Dan, 1992).

8.5 Combined shared disk and nothing systems

A hybrid architecture that combines data sharing with data partitioning systems was proposed at IBM Research in (Wolf et al., 1988). The simulated annealing method (Kirkpatrick et al., 1983)

https://en.wikipedia.org/wiki/Simulated_annealing

was used to determine, which databases are to be shared and which are to remain private, the routing of transactions, and how to partition private databases among systems to minimize response time while balancing system loads. The simulated annealing algorithm is specified in Section 10.12 of (Press et al., 2007).

Trace data from large mainframe systems running IBM's IMS DBMS was used to demonstrate the advantage of the hybrid approach.

Microsoft Research investigated Chimera (Greek mythology monster with Lion's head, goat's body, and serpents tail) in (Minhas et al., 2011), which adds data sharing to a partitioned database system. Chimera balances the load of many workloads and scales out for read-mostly workloads achieving almost linear scalability and effective load balancing with less than 2% overhead.

8.6 Parallel systems at IBM Research

The *Research Parallel Processor Prototype - RP3* (Pfister et al., 1985) at IBM Research was based on *New York U. - NYU's* Ultracomputer (Schwartz, 1980). With N processors, N memories and an $N \log N$ message-passing switch connecting them.

The system supported an innovative fetch-and-add process coordination instruction and the custom VLSI network switches could combine references (including fetch-and-adds) from several processors into a single reference, to reduce memory contention, although this was expected to rarely happen but slowed down the network for all accesses (Gottlieb et al., 1986).

<https://en.wikipedia.org/wiki/Ultracomputer>

Request combining in multiprocessors with arbitrary interconnection networks is discussed in (Lebeck and Sohi, 1994).

The RP3, which was built at a cost of \$40 Million had poor performance and was placed in large crates to be mothballed. Only one octant (1/8 of a full system) was completed, since when the RS/6000 was announced it had faster FLP processing power in its one engine than the RP3 had an octant (private communication Greg Pfister). The RP3 nodes were based on *Research OPD Micro Processor - ROMP* processors, where *OPD - Office Product Division*, which had no FP at all; but the FP capability added was nowhere near enough to stand up to the RS/6000. RP3 octants are shown in Figure 2 in the following paper, which has a good description of RP3.

http://faculty.uml.edu/omer_khan/UML/12s.652_files/L4-ibm-rp3-1985.pdf

The GF11 was a parallel computer constructed at IBM Research to carry out Quantum ChromoDynamics - QCD computations (Beetem et al., 1985). It had 576 *Floating Point - FLP* processors arranged in a modified SIMD architecture. Each processor had 1.125 GB memory delivering 20 *Million Floating Point Operations Per Second - MFLOPS*. GF11's peak performance was 11.5 GFLOPS. Processors were interconnected by a Benes network (Benes, 1965), which is a special case of CLOS networks (Clos, 1953), with $m = n = 2$ as discussed below.

https://en.wikipedia.org/wiki/Clos_network]newline https://en.wikipedia.org/wiki/Multistage_interconnection_networks

8.7 Interconnection networks in IBM's BlueGene/L

The BlueGene/L and its successors P and Q are discussed below.

https://en.wikipedia.org/wiki/IBM_Blue_Gene

The BlueGene/L system had several *Interconnection Networks - INs* (Adiga et al., 2005).

1. A 3-dimensional torus for regular data communication between processors.
https://en.wikipedia.org/wiki/Torus_interconnect
2. A fast Ethernet network for accessing the file system and for network management.
3. A collective network providing low-latency global operations such as reduction, e.g., summing up elements of an array, taking their maximum, minimum, etc. The network is organized as a tree, in which each tree node is a node of processors. To sum up elements of an array, the summation starts at the leaves of the tree. A node collects the sum from its children nodes, accumulates the value to its own sum, and propagates it to its parent. Since children nodes can perform the summation in parallel, the total latency for such an operation scales as $O(\log p)$. The collective network also supports broadcast of data starting from the root to all nodes going down the tree. This can be done in parallel with regular data communication across the torus. Each link in the collective tree network has a target bandwidth of 2.8 Gb/s so it takes only about 16 cycles to transmit a 8-byte value.
4. A global barrier network for barriers broadcasting notifications to all processors also organized as a tree. To perform a global barrier, a node collects signals from its children nodes, and propagates the signal up to its parent node, and this is repeated until the root node gets the signal at which point it knows that all nodes have arrived at the barrier. Next, it needs to release the nodes from the barrier. To achieve that, it sends a release signal down the tree to all nodes. The signals propagate very fast since they are processed by special hardware logic at the barrier network controller of each node. The round-trip latency of a barrier over 64K nodes is reported to take only 1.5 microseconds.
5. A *Joint Test Action Group - JTAG* network for verifying designs and testing printed circuit boards after manufacture. JTAG implements standards for on-chip instrumentation in *Electronic Design Automation - EDA* as a complementary tool to digital simulation.

https://en.wikipedia.org/wiki/JTAG#Storing_firmware

The BlueGene/P node architecture implements small, dedicated second-level caches (one per core) and a shared 3rd-level cache, so the hardware must maintain memory consistency between all of the first- and second-level caches. This is done by store-through to the 3rd level cache and invalidating all remote copies of every store in the first and second level caches. BlueGene/P is a successor to BlueGene/L and uses PowerPC processors.

Rather than spinning on a variable, relying on a cache coherence protocol to distribute the block where the variable resides, and using many software instructions, a hardware barrier network works by sending signals along the wires in the network, and a hardware controller in each node that passes the signal along to other nodes. Hence, the speed is very high,

and in most cases the scalability is very good. Details of destination based snoopy caching are discussed in (Salapura et al., 2008).

The name “wormhole” plays on the way packets are sent over the links: the address is so short that it can be translated before the whole message itself arrives. This allows the router to quickly set up the routing of the actual message and then to remove itself from the rest of the conversation. Since a packet is transmitted flit by flit,

[https://en.wikipedia.org/wiki/Flit_\(computer_networking\)](https://en.wikipedia.org/wiki/Flit_(computer_networking))

it may occupy several flit buffers along its path, creating a worm-like image.

https://en.wikipedia.org/wiki/Wormhole_switching

In the wormhole flow control, each packet is broken into small *flow control units - flits*. The header flit holds information about this packet’s route, e.g., the destination address, and sets up the routing behavior for all subsequent flits associated with the packet. The header flit is followed by the data payload and the tail flit performs bookkeeping to close the connection between the two nodes. In wormhole switching each buffer is either idle, or allocated to one packet. A header flit can be forwarded to a buffer if this buffer is idle. This allocates the buffer to the packet. A body or trailer flit can be forwarded to a buffer if this buffer is allocated to its packet and is not full. The last flit frees the buffer. If the header flit is blocked in the network, the buffer fills up, and once full, no more flits can be sent: this effect is called “back-pressure” and can be propagated back to the source.

This behavior is quite similar to *Virtual Cut-Through - VCT* (Kermani and Kleinrock, 1979).

https://en.wikipedia.org/wiki/Virtual_circuit

The major difference being that VCT flow control allocates buffers and channel bandwidth on a packet level, while wormhole flow control does this on the flit level. The main advantage of wormhole switching derives from the pipelined message flow, since transmission latency is insensitive to the distance between the source and destination. Moreover, since the message moves flit by flit across the network, each node needs to store only one flit. Some implementations, however, require storage of multiple flits at each node to improve routing performance. The reduction of buffer requirements at each node has a major effect on the cost and size of multicomputer systems (Mohapatra, 1998).

Since the minimum buffer size of VCT switching is larger than wormhole switching, VCT switching is traditionally regarded as an inefficient *Networks on Chips - NoC* flow control type. As the shrinking transistor size reduces the criticality of buffer amount for NoC designs; the VCT switching may become a promising NoC flow control candidate according to (Wang et al., 2014). A comprehensive comparison between the VCT and wormhole switching based on detailed RTL-level implementations, was used to evaluate the hardware costs with both deterministic and adaptive routing. Compared with the worm-hole switching VCT shortens the critical path by up to 27%, and induces less area and power overheads. Furthermore, the allocator in VCT routers exhibits a better scalability in area overheads.

8.8 Data allocation and transaction routing in multicomputers

A multi-system with a partitioned database system is considered by Philip S. Yu (UIC), Doug Cornell (rip), Dan Dias (rip), and Alex Thomasian at IBM Research in (Yu et al., 1989). Txns in multiple classes which arrive at a frontend are routed to distributed computers with direct access to partitions of data, which have direct access to most of the data required for txn execution. Txns are routed to the computer which has most of the data required for txn execution, while remote data is accessed by *Data Base - DB call shipping* or *I/O request shipping*.

DB call shipping requires *Remote Procedure Calls - RPCs* to be made to the computer holding the data to be processed.

https://en.wikipedia.org/wiki/Remote_procedure_call

RPC request-response protocol is initiated by the client, which sends a request message to a known remote server to execute a specified procedure with supplied parameters. The remote server sends a response to the client and the application continues its execution. While the server is processing the call, the client is blocked, i.e., the client waits until the server has finished processing before resuming execution, unless the client sends an asynchronous request to the server.

In the case of RDBMSs SQL subqueries are processed at the local/remote system holding the data, as in the case of System R* (the distributed version of System R) in (Lohman et al., 1985), the Arbre project (Lorie et al., 1991), and by DB2 *Parallel Edition - PE* (Baru et al., 1995). There is a reduction in the volume of data being transferred to the node coordinating the execution of a query in the case of a highly selective SQL SELECT statements.

The situation is more complicated in the case of non-relational databases such as IMS, which requires specially written programs to be invoked at the remote computer. Given the set of txns and the objects they access an optimization method is proposed in (Yu et al., 1989) for database allocation and txn routing in such a manner so as to minimize intersystem communication, while balancing computer loads.

Let $X(i, j)$ be the fixed routing matrix and $Y(i, j)$ be the database partitioning matrix:

$X(i, j) = 1$ if txn type i is assigned to system j , and $X(i, j) = 0$ otherwise,

$Y(i, j) = 1$ if database i is assigned to system j and $Y(i, j) = 0$ otherwise.

For txn type i and database k the penalty for $X(i, j)Y(k, m) = 1$ is the number of function requests from txn type i to database k if $j \neq m$ and is zero if $j = m$. Thus,

$$\text{penalty} = \sum_i \sum_j \sum_k \sum_{m \neq j} \lambda(i) P(i, k) X(i, j) Y(k, m),$$

where $\lambda(i)$ is the arrival rate of type i txns and the matrix $P(i, k)$ represents the number of database calls by each txn in DB call shipping and the number of requests in I/O request shipping. Both parameters were obtained by trace analysis. The objective is to minimize the penalty subject to the constraints:

$$\sum_j X(i, j) = 1, \quad \forall i \quad \sum_m Y(k, m) = 1, \quad \forall k.$$

Data allocation in multicomputers was formulated by Wesley Chu (UCLA, emeritus) in (Chu, 1969) as a nonlinear integer 0/1 programming problem. The quadratic programming problem is transformed into an integer linear programming problem by introducing a set of new variables as in (Watters, 1967).

Additional constraints to balance loads are required. Let $D(i)$ be the average number of database calls made by type i txns (note that $D(i)$ is different from $\sum_k P(i, k)$ in the case of I/O request shipping). a the average number of instructions for txn initiation and application execution, d the number of instructions to process an I/O request and b and c the average number of instructions to execute a DB call for DB call shipping and I/O request shipping, respectively. Note that in IO request shipping the processing is on the system making requests, while for DB call shipping it is on the system owning the database partition.

Let $U_{CPU}(j)$ denote the CPU load due to txn processing at the j^{th} node constrained as follows:

$$\text{lower limit} \leq U_{CPU}(j) \leq \text{upper limit}.$$

The upper and lower limits are chosen so that the variation of load across systems is within 5% of the average load over all systems. In the case of DB call shipping $U_{CPU}(j)$ can be expressed as

$$U_{CPU}(j) = \sum_i a \lambda(i) X(i, j) + \sum_i \sum_k b \lambda(i) P(i, k) Y(k, k). \quad (8.1)$$

In the case of IO request shipping

$$U_{CPU}(j) = \sum_i \lambda(i) (a + c D(i) X(i, j) + \sum_i \sum_k d \lambda(i) P(i, k) Y(k, j)). \quad (8.2)$$

Communication overhead is included in the above formulation by adding a term for sending/receiving requests and to Eqs. (8.1) and (8.2) as follows:

$$\sum_i \sum_k 2e \lambda(i) P(i, k) [X(i, j)(1 - Y(k, j)) + Y(k, j)(1 - X(i, j))] \quad (8.3)$$

where e is the processing overhead to send/receive a message. $(1 - Y(k, j))$ implies the fact that database k is not allocated on system j , similarly $(1 - X(i, j))$ implies that transaction i was not routed to system j .

The integer linear programming formulation for partitioning the databases and routing txns is the used to obtain the mean response time at the computers using an M/M/1 queueing model (Kleinrock, 1975). This solution is used as a guide in choosing the initial starting values of the search procedure. An iterative search procedure to minimize the response time by finding the optimal probabilistic routing.

Let $G(i, j)$ be the probabilistic routing matrix, i.e., $G(i, j)$ is the fraction of the txn type i routed to the system j . Then the rate of type i txn routed to system j is proportional to $G(i, j) \lambda(i)$. For a given routing $G(i, j)$, the average response time can be calculated using the equation for mean response time for M/M/1 queues.

The simplex reflection method (Nelder and Mead, 1965) is used to minimize overall mean response time via txn routing and data allocation.

http://www.scholarpedia.org/article/Nelder-Mead_algorithm

The method considers the values of the extreme points of a simplex. The worst point is rejected and replaced by a new point along the line joining it to the centroid of the remaining points. This process is repeated until the desired convergence is achieved.

To be specific, let $P_0, P_1, P_2, \dots, P_n$ be the $n + 1^{st}$ points of an n -dimensional space defining the current simplex. The dimension of each $\bar{P}_k, k = 0, \leq n$, is $n = M(L - 1)$. The components of each P_k comprise one choice for the $G(i, j)$. Let Y_k be the computed mean response time for $\bar{P}_k$ and $\bar{P}_h$ be the point whose value Y_h is the highest among all $Y_k, k = 0, \leq n$. At each stage in the search, $\bar{P}_h$ is replaced by a new point:

$$P_x = (1 + q)\bar{P} - q\bar{P}_h,$$

where $q \approx 1$ and $\bar{P}$ is the centroid of all points not including $\bar{P}_h$ given as:

$$\bar{P} = \frac{1}{n-1} \sum_k \bar{P}_k.$$

The value of q is the ratio of the distance between $\bar{P}_h$ and $\bar{P}$ to the distance between $\bar{P}_x$ and $\bar{P}$. The procedure has been modified to take into account the constraints on the variable, i.e., $0 \leq G(i, j) \leq 1$ and $\sum_i G(i, j) = 1, \forall i$.

The search method is very efficient and converges in a few seconds on the IBM 3081 computer system used in the runs. Although a global optimum is not guaranteed experiments using a starting point different from that of the integer programming solution led to the same $G(i, j)$.

Simulated annealing is an alternative method to deal with this problem.

I/O request shipping can take better advantage of fractional routing for improved performance. Let $G(i, j)$ be the probabilistic routing matrix, whose elements specify the fraction of the type i txns routed to system j . The rate of type i txns routed to system j is proportional to $G(i, j)\lambda(i)$. For a given routing $G(i, J)$ the average txn response times can be recalculated.

Let M denote the number of txn types and L the number of systems. The problem at hand is minimizing a function of $n = M(L - 1)$ variables, i.e., the probabilities $G(i, j)$. The search method in (Nelder and Mead, 1965) is to obtain the optimum set of $G(i, j)$ is a modification of the *Simplex Reflection Method - SRM*, which is a standard derivative-free non-linear optimization technique.

The advantage of probabilistic routing is that it offers more flexibility in balancing the requirements with reducing remote requests and keeping the load balanced. At high load, this extra flexibility can be critical if the routing dependent load is dominant. However, if the partition dependent load is dominant, probabilistic routing offers marginal benefits.

Data for allocation algorithms was extracted by analyzing traces from mainframes running IBM's IMS DBMS. The issues addressed include the effectiveness of routing and the limits of partitioning. For systems with lower communications bandwidth spread beyond the machine room the DB call shipping approach is preferable for distributed systems.

8.9 Data allocation with a distributed relational databases

Query optimization and data allocation in distributed relational databases is discussed in (Lohman et al., 1985), (Ceri and Pellagatti, 2011), (Ozsu and Valduriez, 2020).

An early study of data allocation in distributed databases is (Apers, 1988), which differs from file allocation in (Dowdy and Foster, 1982) in the following ways:

- A table might be vertically and horizontally partitioned (Ceri and Pellagatti, 2011), e.g., in a multinational company employee records of each country can be held at that country, but columns with sensitive data such as salaries at company headquarters.
- SQL queries accessing more than one relation have complex processing schedules, which include transmissions of partial results between sites.

Data replication for higher performance and availability is not considered, however.

Given queries and txns with frequencies of usage there is a need to determine where to allocate fragments (possibly redundantly) and the operations on them at the sites of a computer network.

The model can be used in both branch-and-bound and heuristic algorithms. For minimizing total transmission cost the problem of determining a nonredundant allocation is NP-hard (Garey and Johnson, 1979).

<https://en.wikipedia.org/wiki/NP-hardness>

A method for determining the unit of allocation by means of splitting a relation based on the queries and updates is presented. Under restrictive conditions regarding the behavior of query processing algorithms under splitting fragments, the fragments obtained by this method can be regarded as the unit of allocation. Both optimal and heuristic algorithms for minimizing total transmission cost using static and dynamic processing schedules for computing the cost of an allocation have been investigated and compared.

Soft computing methods (Kumar et al., 2019), especially evolutionary computing, can be used to find an optimal solution for the NP hard *Data Allocation Problem - DAP* (Lotfi, 2019). The goal is to minimize data transmission cost, which is affected by the fragment allocation strategy. A hybrid method based on *Differential Evolution - DE* algorithm and *Variable Neighborhood Search - VNS* is proposed in (Hansen and Mladenovic, 2001) for solving the DAP problem, which aims to increase the performance of DE algorithm by applying effective selection and crossover operators. The performance of proposed DEVNS method is experimentally evaluated against nine state-of-the-art methods such as simulated annealing (Kirkpatrick et al., 1983), (Press et al., 2007) using well-known benchmarks reported in literature. DEVNS takes the first position for 13 of 20 test problems.

Data allocation using query optimizers

Following logical relational database design known as normalization (Ramakrishnan and Gehrke, 2002) https://en.wikipedia.org/wiki/Database_normalization

physical database design is undertaken to select indices which may span multiple columns. Table can have more than one index with the clustering index determining the table sort order.

System R was IBM relational database prototype (Astrahan et al., 1976). Query optimization under the name access path selection in System R is described in (Selinger et al., 1979) and (Selinger and Adiba, 1980).

The index selection problem stated succinctly is as follows.

Given a set of tables and a set of SQL statements with expected use frequencies selecting indexes for each table and the ordering rule for records, which determines the clustered index so as to minimize the total processing cost, subject to a limit on total index space.

The fact that System R was not industrial strength led IBM to develop the DB2 DBMS for IBM mainframes with *Multiple Virtual Storage - MVS* OS (Cheng et al., 1984). DB2 EXPLAIN statement can be used to determine the access plan of an SQL query and access cost. The user can add and delete hypothetical indices to evaluate the effect of index choice on processing cost. With each additional index there is the cost of additional disk space and more importantly disk access cost to keep them up-to-date. Less effective indices contributing little to query processing cost are trimmed. A search method is used to determine the final configuration. An early study to automate physical database design is (Finkelstein et al., 1988) Microsoft researchers added a whatif interface to the query optimizer which is described in (Chaudhuri and Narasayya, 1997).

<https://www.microsoft.com/en-us/research/project/autoadmin/>

IBM's design advisor for integrated automatic physical DB2 database design is described in (Zilio et al., 2004).

R* was an adaptation of IBM's System R to a distributed environment. It is a confederation of autonomous, locally-administered databases, which appear as a single database even if geographically dispersed (Lohman et al., 1985). Naming conventions permit R* to access tables at remote sites without resorting to a centralized or replicated catalog. The user need not specify either the current location of or the communication commands required to access remote tables. Tables may be moved physically without affecting existing SQL statements. SQL data manipulation statements are compiled at each site having a table referenced in the statement, coordinated by the site at which the statement originated.

Optimization uses dynamic programming (Bellman, 2003) and pruning to minimize total estimated execution cost at all sites, which is a linear combination of CPU, I/O, and communications costs (Lohman et al., 1985). Query optimizers used in DB2 are described in (Gassner et al., 1993) and are based on query optimizers developed in conjunction with the Starburst project (Lee et al., 1988). An approach similar to (Finkelstein et al., 1988) need to be developed to find improved data allocations for the tables of a distributed relational database.

8.10 Review of multicomputer Data Base Machines - DBMs

A text describing parallel database servers is (Freitas and Lavington, 2000). A text on the design and analysis of fault-tolerant computer systems (Abd-El-Barr, 2007) serves as another source for the material in this section.

8.10.1 Tandem Corporation

Tandem Corp. was formed by former HP engineers in 1974 to design a multicomputer shared-nothing system with duplexed processors and mirrored disks crossconnected to achieve fault-tolerance.

Given that the reliability of a processor is r_p and the reliability of each disk is r_d , then the reliability of a duplexed system of processor-disk pairs is:

$$R_D = 1 - (1 - r_p r_d)^2 = 2r_p r_d - (r_p r_d)^2 \approx 2r_p r_d.$$

In the case of a cross-connected configuration

$$R_C = [1 - (1 - r_p)^2] \times [1 - (1 - r_d)^2] = 4r_p r_d - 2r_p^2 r_d - 2r_d r_p^2 + r_p^2 r_d^2 \approx 4r_p r_d.$$

It follows that the cross connection provides twice the reliability, since $R_C \approx 2R_D$.

Mainframe computers in 1980s had *Mean Time To Failure - MTTF* of a few days. *Tandem NonStop - TNS* computers were designed to have a much higher MTTF, while remaining cost competitive (Tandem Database Group, 1989), e.g., costing twice the price of a single processor system, as opposed to three times the cost as in the case of *Triple Modular Redundancy - TMR* of the *Self-Testing and Repairing - STAR* computer at *Jet Propulsion Labs - JPL* (Avizienis et al., 1971), <https://history.nasa.gov/computers/Ch5-5.html> or four times the price as in the case of Stratus Computers (Harrison and Schmitt, 1987).

https://en.wikipedia.org/wiki/Stratus_Technologies

A long list of Tandem computers is given at the following web site.

http://en.wikipedia.org/wiki/Tandem_Computers

Tandem was subjected to several microprocessor migrations as it was acquired by Compaq it transitioned to the Alpha microprocessor. Later when HP acquired Compaq it transitioned to Itanium, but eventually Intel x86's prevailed.

Nonstop I, NonStop II, NonStop TXP, FOX, NonStop-VLX, -CLX, -Cyclone.

Other product lines are: Rainbow, Dynamite PC, Integrity, Wolfpack.

TNS/R NonStop migration to MIPS: Cyclone/R, NonStop Himalaya K-series, Open System Services, NonStop Himalaya S-Series.

Acquired by Compaq Tandem migrated from its proprietary CISC microprocessor to DEC's RISC Alpha running in emulation mode. It was considered an easy migration which improved performance (Andrews and Sand, 1992). Additional benefits included no reprogramming, no retraining, fast time to market, and debugging of optimized programs as if they were still running on the CISC platform. The paper shares the experience in implementing this migration scheme, with measurements of the resulting performance and codesize. Static rather than dynamic translation was chosen for the following reasons:

1. The first OS released would itself execute via translation.
2. Performance goals were high.
3. The translation algorithms require significant time and memory.
4. Tandem machines were primarily used for months-long execution of a few applications.

Acquisition by Hewlett Packard, led to a migration to Itanium, Itanium migration to Intel X86 was completed in 2014.

I visited Tandem Corp. in Cupertino, CA in 1984 to explore job opportunities, but also discuss my research on concurrency control methods with Jim Gray,

<https://www.microsoft.com/en-us/research/people/gray/>

I picked a Friday because Tandem was known to hold beer bashes on Friday afternoons. Jim demonstrated to me how a single line command could be used to compile a program at a second site and run it at a third site.

<https://www.company-histories.com/TANDEM-COMPUTERS-INC-Company-History.html>

I first met Jim when he gave a talk at USC's CS department after leaving IBM's San Jose Research Center in 1980 (this is before the move to *Almaden Research Center - ARC*) He had written the following letter to IBM management.

<https://jimgray.azurewebsites.net/papers/CritiqueOfIBM%27sCSResearch.pdf>

I presented extensions to his strawman analysis of probability of deadlock, which was a joint work with PhD student In Kyung Ryu (Hanyang U., emeritus). Gray's analysis which did not take into account that the txn holding the requested lock

should be already blocked for a deadlock to occur. It is easy to show given that the mean txn waiting time with respect to an active txn ($W_1 \approx R.3$), i.e., one-third of mean txn response time (refer to Chapter 1). The probability of txns encounter a deadlock is then one third of Gray's estimate. Gray's analysis which first appeared as a short IBM research report but was later reproduced in the book he coauthored with Andreas Reuter (Kaiserslautern U., U. Heidelberg). "Transaction Processing: Concepts and Techniques" (Gray and Reuter, 1992).

Having gained access to massive computing resources (a 4-way and a 2-way mainframe 3081s) at IBM Research allowed me to make simulation runs to validate the accuracy of the analysis in (Thomasian and Ryu, 1991).

Gray at Tandem advocated the debit/credit benchmark exhibiting linear scaleup in Tandem systems. Gray et al. authored "One thousand transactions per second", which had become feasible in 1985 at Tandem, the Bank of America, and Amdahl Corp. (Gray et al., 1984). Gray in 1985 collaborated with 24 authors in academia and industry for the anonymous publication of "A Measure of Transaction Processing Power" in *Datamation* monthly. The Debit/Credit benchmark later led to the *Transaction Processing Council - TPC*, which is discussed in Subsection 8.11.1.

<https://www.tpc.org>

When Tandem was acquired in 1997 by Compaq to become its server division, Gray joined *Digital Equipment Corporation - DEC's System Research Center - SRC*,

https://en.wikipedia.org/wiki/Digital_Equipment_Corporation

but then in 1998 DEC too was acquired by Compaq and Jim joined Microsoft, which allowed him to stay in San Francisco at the *Bay Area Research Center - BARC*. It was said that "Jim brings bad luck to companies", but of course this is not true of Microsoft, which is still thriving.

HP acquired Compaq in 2002 for \$25 billion, making the merged computer company second to IBM.

<https://www.cnet.com/news/hp-to-acquire-compaq-for-25-billion/>

This is also why HP used to produce two lines of personal computers, its own and those by Compaq.

The HP merger was more successful than the Burroughs and UNIVAC merger, which led to UNISYS. HP's former CEO Carly Fiorina who came from Bell Labs is given credit for this merger.

<http://fromsecretarytoceo.com/under-fiorinas-leadership-the-hp-merger-with-compaq-was-one-of-the-most-successful-high-tech-mergers-in-history/>

At Microsoft research Gray co-authored "A \$2k computer can execute about 8K TPS", which is 80x more than one of the largest US bank's traffic in 1970s (Gray and Levine, 2005). Jim edited a book on database benchmarks (Gray, 1993) and his efforts in this area are praised in (DeWitt and Levine, 2008). His web site at Microsoft is as follows.

http://jimgray.azurewebsites.net/default_12_14.htm

Jim went missing in January 2007 while on a boat trip to disperse his mother's ashes. All attempts to find his boat have been unsuccessful so far. Jim was 63 when he died but according to California law his death was proclaimed five years later. A remembrance ceremony was held at *UC Berkeley - UCB* in May 2007.

https://www.theregister.co.uk/2007/04/30/jim_gray_tribute/

Jim's last interview is here.

https://www.theregister.co.uk/2006/05/30/jim_gray/

David DeWitt was who had collaborated with Jim Gray encouraged Microsoft CEO - Bill Gates to establish the *Gray Systems Lab - GSL* in Madison, WI volunteering to lead it.

<https://azuredata.microsoft.com/labs/gsl>

Raghu Ramakrishnan, a former U. Wisconsin professor, who first joined Yahoo! and then Microsoft is GSL's current CTO.

Costly transition from mainframes to shared nothing computers

Two examples of cost overruns in software development when California's *Dept. of Motor Vehicles - DMV* attempted to switch from IBM mainframes to Tandem Cyclone computers. In both cases Tandem computers were less expensive than mainframes, but the cost of conversion turned out to be excessive. This was perhaps a consequence of Fred Brooks' law about software project management according to which "adding human resources to a late software project delay it farther" (Brooks, 1995). Laws related to computer and storage systems are given in Chapter 1, refer to Subsection 1.34.5.

In 1987 the state-wide DMV in California had a project to convert its database from an IBM ES/9000 mainframe to 24 Tandem Cyclone machines running NonStop SQL. Tandem outbid IBM with \$10 Million versus IBM's \$23M

for their system, software implementation on the Tandem ran to major cost overruns. The goal was to update DMVs 20-year-old system and merge the driver's license and vehicle registration databases.

The project was to cost \$27 million and be up and running within 5 years, but in 1993, six years after the project was begun, the DMV project was abandoned completely. Performance-wise Tandem delivered only 70 TPS instead of the projected 208 TPS. It was perhaps because the linear scaleup projected for Tandem computers fails for more complex txns.

According to LA Times the "DMV contracts with Tandem and another vendor, the accounting firm of Ernst & Young, failed to include strict performance standards and penalties.

<https://www.latimes.com/archives/la-xpm-1994-04-28-mn-51501-story.html>

Interestingly, this failure was repeated few years later and \$208M was misspent after six years of development in February 2013.

<https://spectrum.ieee.org/riskfactor/computing/it/dj-vu-all-over-again-californias-dmv-it-project-cancelled>

In conclusion, the cost of software development should be added to the cost of system procurement, since it affects *Total Cost of Ownership - TCO*. A recent article notes that 74% of mainframe modernization efforts fail and that COBOL is the programming language in use.

<https://www.techrepublic.com/article/everyone-wants-to-retire-mainframes-but-74-of-modernization-efforts-fail/>

A report on mainframe use was produced recently.

https://www.oneadvanced.com/siteassets/resources/advanced_mainframe_modernization_report_2020_us.pdf

8.10.2 Teradata Corporation

Teradata is an early DBM company that was the result of collaboration between Phil Neches, who completed his PhD at *California Institute of Technology - CalTech*

<https://www.linkedin.com/in/phil-neches-918b1825/>

and Citibank's *Advanced Technology Group - ATG* in early 1980s.

<https://en.wikipedia.org/wiki/Teradata>

Neches filed his thesis in 1983 and a beta version of the Teradata DBC/1012 computer was shipped to WellsFargo Bank for Christmas 1983.

<https://www.zippia.com/teradata-careers-11345/history/>

<https://www.linkedin.com/pulse/designed-query-everything-history-teradata-keith-prince/>

Neches became the founder, chief scientist, and VP at Teradata.

In 1979 David K Hsiao at *Ohio State U. - OSU* wrote the article "Database Machines are Coming, ..." (Hsiao, 1979), which led to the article "Database machines: An idea whose time has passed" (Boral and DeWitt, 1983). Both authors went on to design and prototype parallel DBMs with similarities to DBC/1012: Bubba by Haran Boral et al. at *MicroElectronics and Computer Technology Corp. - MCC*

https://en.wikipedia.org/wiki/Microelectronics_and_Computer_Technology_Corporation

and Gamma by David DeWitt et al. at U. Wisconsin at Madison.

The Teradata DBC/1012 was a shared-nothing multicomputer with a maximum secondary storage capacity of one terabyte - TB or 10^{12} bytes, which is equivalent to 500 2 GB IBM 0662 IBM Allicat disks.

https://en.wikipedia.org/wiki/History_of_IBM_magnetic_disk_drives#IBM_0662

A DBC/1012 was delivered to Walmart the DBC/1012 computer with a one TB capacity.

Relational tables were spread across the nodes using hashing. The BYNET *Interconnection Network - IN* supported the broadcasting of SQL commands such as SELECTs to all the nodes and could merge sorted runs originating from the nodes. The system's fault-tolerance was based on two copies of BYNET.

<http://lakshmikishore.blogspot.com/2010/09/bynet.html>

Various aspects of Teradata such as Vantage are discussed below.

<https://docs.teradata.com>

Teradata was acquired by NCR in 1991, which itself was acquired by AT&T. NCR after acquiring Teradata developed the NCR 3600 and 3700 product lines. The IN for the 3600 used an enhanced BYNET licensed from Teradata, while the IN for the 3700 was based on a multistage Omega IN (Duato et al., 2003) developed jointly by NCR and Teradata.

https://en.wikipedia.org/wiki/Omega_network

Teradata software was moved to *Suse Linux Enterprise Software - SLES*. While Tetadata has its own database Inspire

Migration and Modernization Toolkit can be used to migrate Sybase DBMS for OLTP on Teradata.

<https://www.ispirer.com/products/sybase-ase-to-teradata-migration>

NCR 3700 evolved to NCR Worldmark (see Section 8.2 in (Freitas and Lavington, 2000)).

<https://www.yumpu.com/en/document/read/24118656/ncr-worldmark-5250-server-pdf>

NCR became an independent public company again in 2007.

Teradata's DBC/1012 used the interleaved declustering method rather than basic mirroring, which allowed a more balanced disk load when a disk failed. Mirrored and hybrid disk arrays are described in Section 4.2 in Chapter 4.

8.10.3 Stratus Technologies

Stratus Technologies was founded in Natick, MA in 1980 and shipped its first computer in 1982.

https://en.wikipedia.org/wiki/Stratus_Technologies

<https://www.stratus.com/>

Stratus computers known for continuous availability were sold by IBM under the name S/88 (Harrison and Schmitt, 1987).

http://pages.cs.wisc.edu/~david/papers/ibmsj1987_stratus.pdf

The main approach for achieving fault-tolerance in Stratus computers are two levels of duplexing: if the output from the two processors on Board A miscompare then the output from Board B takes over. The data is mirrored and disk controllers are duplexed as well. Processors are connected via a network and use *Best Path Routing* - BPR.

http://www.xroadsnetworks.com/ubm/technology/tech_bpr.xrn

8.10.4 Dell Corporation

Dell Corp. was founded in 1984 at a dorm room at the U. Texas at Austin by Michael Dell. It became a private company owned by Dell in 2013, but went public again in 2018. Over the years Dell acquired/merged with many companies including Perot Systems, EMC, and VMware.

<https://en.wikipedia.org/wiki/Dell>

Market shares of computer companies in Q1 of 2021, \$20.9Billion.

No. 1: Dell Technologies - 17.0%

No. 2: HPE's New H3C Group - 15.9%

No. 3-Tie: Inspur - 7.2%

No. 3-Tie: Lenovo 6.9%

No. 5-IBM 5.3.%

Original Design Manufacturers - ODM 31.7%

8.10.5 Inspur Systems

IBM China and Inspur Systems have a joint effort to produce servers with Power architecture.

<https://www.inspursystems.com/>

<https://en.inspur.com/en/2488385/index.html>

https://www.marbridgeconsulting.com/marbridgedaily/archive/article/103975/inspur_ibm_to_form_server_production_and_sales_jv

Relevant to this discussion is the open sourcing of IBM's Power architecture.

<https://www.zdnet.com/article/ibms-power-ful-open-source-gift-big-winner-is-china-losers-include-you-know-who/>

8.11 Benchmarking in various forms

Included in this section is the *Transaction Processing Council* - TPC, *Storage Performance Council* - SPC, *Standard Performance Evaluation Corporation* - SPEC, mixed workload benchmarks: OLTP and OLAP, benchmarking distributed databases, and sort benchmarks.

8.11.1 Transaction Processing Council - TPC

Jim Gray led the effort to establish TPC. TPC's goals according to its founder Omri Serlin are as follows.

<http://www.tpc.org>

1. Determining total system cost including hardware and software to successfully run benchmarks, plus five years maintenance costs.
2. Benchmarks should be specifiable as high-level functional requirements rather than specifying hardware or software platform or code-level requirements to allow any company to run this benchmark.
3. Scaleup rules are to take into account the increasing power of OLTP systems, i.e., the number of users and the size of the database tables is supposed to increase in proportion to system power producing higher txn rates.
4. The maximum txn rate or throughput is constrained by a response time requirement, e.g., 95% of txns completing in less than one second.

A list of active TPC benchmarks is as follows.

<http://tpc.org/information/benchmarks5.asp>

TPC-C models an order-entry environment such as entering and delivering orders, recording payments, checking the status of orders, and monitoring the level of stock at the warehouses. TPC-C involves a mix of five concurrent txns of different types as detailed below:

- (1) The simultaneous execution of multiple txn types that span a breadth of complexity.
- (2) On-line and deferred txn execution modes.
- (3) Multiple on-line terminal sessions.
- (4) Moderate system and application execution time.
- (5) Significant disk input/output.
- (6) Txn integrity (ACID properties).
- (7) Non-uniform distribution of data access through primary and secondary keys.
- (8) Databases consisting of many tables with a wide variety of sizes, attributes, and relationships.
- (9) Contention on data access and update.

The primary metrics are the txn rate per minute (tpmC), the associated price per txn (\$/tpmC), and the availability date of the priced configuration.

TPC-Data Integration - DI, also known as *Extract, Transform, Load - ETL*

https://en.wikipedia.org/wiki/Extract,_transform,_load

is the analysis, combination, and transformation of data from a variety of sources and formats into a unified data model representation. Data Integration is a key element of data warehousing, application integration, and business analytics.

TPC-DS is the industry standard benchmark for measuring the performance of decision support solutions including, but not limited to Big Data systems, including queries and data maintenance.

The underlying business model is a retail product supplier, the database schema, data population, queries, data maintenance model and implementation rules have been designed to be broadly representative of modern decision support systems. This benchmark illustrates decision support systems that:

- (1) Examine large volumes of data.
- (2) Give answers to real-world business questions.
- (3) Execute queries of various operational requirements and complexities, e.g., ad-hoc, reporting, iterative OLAP, data mining.
- (4) Are characterized by high CPU and IO load.
- (5) Are periodically synchronized with source OLTP databases through database maintenance functions.
- (6) Run on “Big Data” solutions, such as RDBMS as well as Hadoop/Spark based systems.

TPC-E models a brokerage firm with customers who generate txns related to trades, account inquiries, and market research. The brokerage firm in turn interacts with financial markets to execute orders on behalf of the customers and updates relevant account information. The benchmark is “scalable” and the number of customers defined for the brokerage firm can be varied to represent the workloads of different-size businesses. The benchmark defines the required mix of txns the benchmark must maintain. The TPC-E metric is given as *Txns Per Second - TPS*.

TPC-H is a decision support benchmark with business oriented ad-hoc queries and concurrent data modifications. The queries and the data populating the database have been chosen to have broad industry-wide relevance. This benchmark illustrates decision support systems that examine large volumes of data, execute queries with a high degree of complexity, and give answers to critical business questions. The performance metric is the TPC-H Composite Query-per-Hour Performance Metric (QphH@Size), and reflects the selected database size against which the queries are executed, the query processing power when queries are submitted by a single stream, and the query throughput when queries are submitted by multiple concurrent users.

TPC-VMS is TPC-Virtual Measurement Single System Specification, which leverages the TPC-C, TPC-E, TPC-H and TPC-DS benchmarks by adding the methodology and requirements for running and reporting performance metrics for

virtualized databases where three database workloads are consolidated onto one server. Test chooses one of the four benchmark workloads (TPC-C, TPC-E, TPC-H, or TPC-DS) and runs one instance of that benchmark workload in each of the 3 virtual machines (VMs) on the *System Under Test - SUT*. The three virtualized databases must have the same attributes, e.g. the same number of TPC-C warehouses, the same number of TPC-E Load Units, or the same TPC-DS or TPC-H scale factors.

The TPC-VMS Primary Performance Metric is the minimum value of the three TPC Benchmark Primary metrics for the TPC Benchmarks run in the Virtualization Environment.

TPCx-BB is an Express Benchmark to measure the performance of Hadoop based Big Data systems. It measures the performance of both hardware and software components by executing thirty frequently performed analytical queries in the context of retailers with physical and online store presence. The queries are expressed in SQL for structured data and in ML algorithms for semi-structured and unstructured data. The SQL queries can use Hive or Spark, while the ML algorithms use ML libraries, user defined functions, and procedural programs.

TPCx-HS or TPC Express Benchmark (TPCx-HS) was developed to provide an objective measure of hardware, OS and commercial Apache *Hadoop File System - HDFS* API compatible software distributions, and to provide the industry with verifiable performance, price-performance and availability metrics. The benchmark models a continuous system availability of 24x7.

The TPCx-HS stresses both hardware and software including Hadoop run-time, Hadoop Filesystem API compatible systems and MapReduce layers. This workload can be used to assess a broad range of system topologies and implementation of Hadoop clusters. The TPCx-HS can be used to assess a broad range of system topologies and implementation methodologies in a technically rigorous and directly comparable, in a vendor-neutral manner.

TPCx-V benchmark measures server performance of running database workloads in virtual machines. The end-to-end benchmarking kit that simulates the multi-tenancy phenomena of elasticity, capacity planning, and load variation in cloud data centers.

TPC-Energy specification contains the rules and methodology for measuring and reporting an energy metric in TPC Benchmarks. This includes the energy consumption of system components associated with typical business information technology environments, which are characterized by energy consumption of servers, disk systems, and other items that consume power and are required by the benchmark specification.

TPC-Pricing subcommittee recommended revisions to the existing pricing methodology so that prices used in published TPC results are verifiable. A decision was made to develop a single pricing specification that is consistent or all TPC benchmarks.

8.11.2 Storage Performance Council - SPC

SPC is a vendor-neutral standards body for the storage industry.

<https://spcresults.org/>

SPC benchmarks can be categorized as follows.

SPC-1 determines the performance of a storage subsystem in *I/Os per Second - IOPS*.

SPC-1/E is an extension to report energy usage.

SPC-1/C is a component level benchmark.

SPC-1/C/E combines the previous two benchmarks.

SPC-2, SPC/2/E, SPC/2/C, SPC/2/C/Energy are benchmarks involving sequential I/O.

8.11.3 Standard Performance Evaluation Corporation - SPEC

SPEC was founded in 1988 by several workstation vendors to standardize performance tests has more than sixty member companies and publishes hundreds of performance results spanning a variety of system performance disciplines in the following areas: Cloud, CPU, Graphics/Workstations, ACCEL/MPI/OpenMP - *Open MultiProcessing Message Passing Interface - MPI* (Snir et al., 1998)

<https://en.wikipedia.org/wiki/OpenMP>

Java Client/Mail Servers, Storage, Power, Virtualization, Web Server.

<https://spec.org>

“The basic SPEC methodology is to provide the benchmarker with a standardized suite of source code based upon existing applications that has already been ported to a wide variety of platforms by its membership. The benchmarker then takes

this source code, compiles it for the system in question and then can tune the system for the best results. The use of already accepted and ported source code greatly reduces the problem of making apples-to-oranges comparisons.”

8.11.4 Benchmarking distributed databases

Yahoo! Cloud Serving Benchmark - YCSB was created by Brian F. Cooper et al. to assist in the evaluation of different cloud systems. The focus was on systems which serve read and write workloads, over batch or analytical system.

<https://en.wikipedia.org/wiki/YCSB>

YCSB workloads data look like the following:

1 table named “usertable”

10 string fields, one ten byte primary key and nine 100 byte strings.

1,000,000 records.

1,000,000 operations with 100 to 1,000 threads for Workload A, B, C, and D.

1,000,000 operations with 10 to 100 threads for Workload TPS-E.

Characteristics of the YCSB benchmark are given in Table 8.5.

TABLE 8.5 Characteristics of the YCSB benchmark.			
Workload	Operation	Distribution	Application
A –Update Heavy	Read: 50%	Zipfian	Store recording recent actions
B –Read Heavy	Read: 95% Update 5%	Zipfian	Photo tag: add 5%, read 95%
C –Read Only	Read: 100%	Zipfian	User profile cache
D –Read Latest	Read: 95% Insert 5%	Latest	User status updates; latest interesting
E –Short Ranges	Scan: 95%	Zipfian uniform	Threaded conversations

The specification of the environment and results of running the benchmark on Yugabyte and Apache Cassandra are given below. Yugabyte is the leading high-performance distributed SQL database.

<https://www.yugabyte.com/>

<https://forum.yugabyte.com/t/ycsb-benchmark-results-for-yugabyte-and-apache-cassandra/59>

Difficulties with benchmarking distributed databases are discussed below.

<https://medium.com/@PingCAP/why-benchmarking-distributed-databases-is-so-hard-2765a26cc2bf>

Overview of existing Benchmarks is presented in (Bog, 2014), which is reproduced as Table 8.6.

8.11.5 Mixed workload benchmarks: OLTP and OLAP

Three alternative ways to create a mixed OLTP and OLAP workload benchmark are as follows:

1. Combining existing benchmarks whose underlying data schemas differ and this would need the schemas to be integrated to create a data schema for both workloads. Enterprise workloads adapting to market situations evolve and change, something that would be required from benchmarks, which lose relevancy because they do not reflect the real workloads any longer.
2. Extension of an existing benchmark with the respective other workload, and the same arguments apply to the second approach.
3. Based on the observation that established benchmarks have aged and do not reflect current enterprise workloads any longer, the creation of a completely new benchmark is recommended.

Combining OLTP with OLAP will inevitable result in a degradation of OLTP response times, but the question is how much degradation in txn response times is acceptable.

The following argument regarding data latency is made in (Bog, 2014).

According to (Bog, 2014) data latency is the delay between the creation of data in the operational system and its availability for analytical processing. In a hybrid OLTP and OLAP system, this latency does not exist, as the actively used data source for both workload parts is the same. One strategy is to extend the txns to include the synchronization of data

TABLE 8.6 Overview of existing benchmarks, based on Table 3.3 in (Bog, 2014). WH = Wholesale supplier, BK = Brokerage form, MC = Mobile carrier, RPS = Retail product supplier, Star = Star schema. Widest/narrowest refers to tables. #tables stands for number updated.

	TPC-C	TPC-E	TATP	TPC-H	TPC-DS	SSB	CH-benCHm.
Domain	OLTP	OLTP	OLTP	OLAP	OLAP	OLAP	Mixed
Standard	Yes	Yes	No	Yes	Yes	No	No
Scenario	WS	BK	eMC	WS	RPS	TPC-H	WS
Schema	3NF	3NF	3NF	3NF	Star	Star	3NF
#Tables	9	33	4	8	24	5	12
Widest	21	24	34	15	34	17	21
Narrowest	3	2	5	3	3	7	3
#Columns	92	191	51	61	425	58	106
#Tables	8	20	3	2	16	1	8
Data	Non-uni	Uni	Uni	Uni	Skewed	Uni	Non-uni
Data set	Synth.	Pseudo-R.	Synth.	Synth.	Real	Synth.	Synth.
#Queries	5	12	7	22	99	13	27
Year	1992	2007	2004	1999	2012	2007	2011

to the analytical system to achieve up-to-date data, but this introduces overhead to txn processing. Another strategy is to synchronize the data immediately upon the commit of a txn. Yet, the interval to transport data and each data preparation step to enhance data for improved reporting performance introduces a delay.

8.11.6 Sort benchmarks

Sorting is a very common operation in batch processing, such as updating sorted sequential files with input data records, but also in evaluating relational operators, such as sort-merge join, duplicate elimination (Ramakrishnan and Gehrke, 2002), (Graefe, 2006). Knuth's rd volume on "Sorting and Searching" is a definite guide on this topic (Knuth, 2005).

Hsiao in his book on early DBMs (Hsiao, 1983) emphasizes the 90%-10% rule, i.e., 90% of execution time is spent executing 10% of the code. As a consequence improving 10% of the code will produce the most significant reduction in execution time.

The July 20, 1981 issue of Computerworld stated that Syncsort's, now Precisely's sort program outperforms IBM's DFSort routines 20% in CPU time, 25% in *EXecute Channel Programs - EXCPs*, and 30% in elapsed time.

<https://en.wikipedia.org/wiki/Syncsort>

The new name for the company is Precisely, hence <https://precisely.com>.

Mainframe sort acceleration by Syncsort is a White Paper, which shows a 60% reduction in total CPU time and elapsed time with respect to for IBM's DFSort.

<https://www.syncsort.com/CMSPages/GetFile.aspx?guid=fc9c4c26-8048-47a4-ac65-5bcbad265bc1>

This was so in spite of the fact that IBM S/360 had hidden instructions, for sorting not disclosed in early releases of IBM *Principles of Operations - PrincOps* Manual. These instructions are disclosed in Appendix A, page A51 in *z-Architecture Principles of Operations* Manual.

<http://www.ibm.com/e-business/linkweb/publications/servlet/pbi.wss?CTY=US&FNC=SRX&PBL=SA22-7832-09>

Sorting of large files benefits from larger memory sizes, reducing the number of passes required for external sort-merge invented by von Neumann (Burks et al., 1963). Merge-sort is illustrated by an example in (Ramakrishnan and Gehrke, 2002).

https://en.wikipedia.org/wiki/Merge_sort

Two methods are proposed to improve performance. Replacement sort uses an Input and Output and $B - 2$ buffers for the current set. The idea is to repeatedly pick the tuple in the current set with the smallest key value that is still greater than the largest key in the output buffer and append it to the output buffer. This allows a new record to be moved from the Input buffer. With a more aggressive implementation, called replacement sort, we can write out runs of approximately $2B$ internally sorted pages on the average. It is meaningful to use the number of I/Os rather than pages, so that a block size b would result in a b -fold reduction in the number of I/Os.

In developing sort algorithms for superscalar processors it is noted by Ramesh Agrawal in (Agrawal, 1996) that the compare and branch sequences required by traditional sort algorithm are not suitable for exploiting the multiple execution units in high performance RISC processors, because of the long latency of the compare instructions and the sequential sorting algorithm. He proposes new sort algorithms that eliminate almost all compares and provide functional parallelism, which can be exploited by multiple execution units. Significantly reduced is the number of passes through keys and improved locality benefits caching.

This new algorithm outperformed the traditional sort algorithms presented in (Nyberg et al., 1995) for running Datamation's disk to disk sort benchmark (one million 100-byte records) on a DEC Alpha processor, which broke several performance records at the time. Following Jim Gray's disappearance in 2007 the sort benchmark is administered by Chris Nyberg of Ordinal Technology Corp.,

<http://www.ordinal.com/>

Mehul Shah of Amazon Web Services, and Naga Govindaraju of Microsoft.

<http://sortbenchmark.org/>

Ordinal Corp founded in 1984 has the following goals:

- (1) Efficient use of large numbers of disks in parallel.
- (2) Efficient use of parallel SMP processors.
- (3) Cache-efficient sort algorithms.

Ordinal provides the Nsort sort/merge program that can quickly sort large amounts of data, using large numbers of processors and disks in parallel. Unique in its CPU efficiency, Nsort is the only single-computer commercial sort program to demonstrate: One TB sorts in 33 minutes and one GB/s file read and write rates.

A list of sort benchmarks and their metrics is as follows.

GraySort Sort rate (TBs/minute) achieved, while sorting a very large amount of data (100 TB minimum).

CloudSort Minimum cost for sorting a very large amount of data on a public cloud (100 TB).

http://sortbenchmark.org/2014_06_CloudSort_v_0_4.pdf

MinuteSort Amount of data that can be sorted in 60 seconds or less (Nyberg et al., 1995).

JouleSort Amount of energy required to sort 10^8 , 10^9 , 10^{10} , 10^{12} records (10 GB, 100 GB, 1 TB, or 100 TB).

PennySort Amount of data that can be sorted for a penny's worth of system time (Nyberg et al., 1995). PennySort has lost approval.

TeraByte Sort Elapsed time to sort 10^{12} bytes of data. This benchmark is now deprecated because it became essentially the same as MinuteSort.

Datamation Sort Amount of time to sort one million records (100 MB). Original sort benchmark defined in "A Measure of Transaction Processing Power". In 1985 winners took 1 hour, now 1 second. So the benchmark is deprecated.

An example of new benchmarks is Tencent Sort which follows GraySort rules. Reported are results of 100 TB in less than 100 sec (60.7 TB/min) for the "Indy" GraySort, 44.8 TB/min for the "Daytona" Graysort, 55.3 TB for the "Indy" MinuteSort, and 36.9 TB for the "Daytona" Minutesort benchmarks using a cluster of 512 OpenPOWER servers optimized for hyperscale data centers.

<http://sortbenchmark.org/TencentSort2016.pdf>

The benchmark is motivated by NVMe solid-state storage devices, 100 Gb Mellanox networking,

https://www.mellanox.com/sites/default/files/doc-2020/mlnx_corporate_deck.pdf

combined with 160 hardware threads across two 10-core OpenPOWER (POWER8) CPUs, allowing a substantial step forward in cluster level performance and an order of magnitude improvement in per-node sort performance.

Disproportionate growth in network and storage bandwidth puts a new degree of pressure on the CPU, and while sort alone still utilizes less than 10%/70% (average/peak) of the CPU, managing the combination of sort, network and storage now demands a high-performance CPU to achieve maximum performance.

8.12 Data Base Machines - DBMs and backend processors

Most DBMs were rather developed to offload database processing tasks from mainframes. The IBM 2938 Array Processor was an early system to increase the processing power of S/360 Models 44, 65, 75, 85 for operations on matrices (Ruggiero and Coryell, 1969). S/360 I/O commands were used to send the data to the AP, which could carry out certain operation such as convolution.

The following categories of DBMs are listed by Stanley Su (U. Florida, Gainesville, emeritus) in (Su, 1988).

Intelligent Secondary Storage Devices were a proposal for logic per disk track devices (Slotnick, 1970). The concept was used to speed up text retrieval as in RAPID (Parhami, 1972) and relational operators as in RAP (Ozkarahan and Sevcik, 1977), which are discussed in more detail in Section 8.13.

Database filters: *Content-Addressable File Store - CAFS* was developed by British *Int'l Computers Limited - ICL* could be used to accelerate relational table scans (Babb, 1979). "The main feature of the hardware is a special store which can rapidly remember or recall data. This data might be pointers from one file to another, in which case the memory helps with queries on joins of files. Alternatively, the memory can help remove redundant data during projection, giving a considerable speed advantage over conventional hardware."

https://en.wikipedia.org/wiki/International_Computers_Limited

A history of ICL which originated in Russia in 1957 and developed computers with an Armenian name: Nairi, but manufactured in Russia: Kazan Manufacturing Enterprise of Computer Systems.

[https://en.wikipedia.org/wiki/Nairi_\(computer\)](https://en.wikipedia.org/wiki/Nairi_(computer))

<http://www.icl.ru/eng/company/history/>

[https://en.wikipedia.org/wiki/Nairi_\(computer\)](https://en.wikipedia.org/wiki/Nairi_(computer))

Associative memories which retrieve data by content could be used for data management as noted by Glen Langdon (IBM, UCSC, rip) in (Langdon, 1978). A collection of papers on associative processing and processors is (Krikelis and Weems, 1997).

The survey of *Associative Processors - APs* in (Yau and Fung, 1977) has the following discussions.

Fully parallel APs: Word-organized APs. Distributed Logic APs. Lee's distributed logic APs and modifications. *Parallel-Element Processing Ensemble - PEPE*.

Bit-Serial APs. The STARAN Processor. Other Bit-Serial APs. Linear Array Processor. *Extended Content Addressed Memory - ECAM*. Other Developments: Byte-Serial - Word-Parallel APs, Data Manipulators.

Word-Serial APs.

Block-Oriented APs.

DIALOG is based on the fact that most database applications perform a small number of simple operations on large amounts of data and that a small fraction of the data accessed is required by the application (Wah and Yao, 1980). It is more cost effective to perform database operations directly on the data in the secondary storage in order to avoid the transfer of unnecessary data across different levels of the memory hierarchy. DIALOG DBM brings processing power closer to the devices on which data are stored and offloads database processing functions from the main computer.

Graphics Processing Unit - GPUs have been developed to enhance the computing power of microprocessors.

https://en.wikipedia.org/wiki/Graphics_processing_unit

Microprocessors are being extended with vector instructions (Asanovic, 1998). Such extensions are criticized as being suited only to certain benchmarks.

<https://www.zdnet.com/article/linus-torvalds-i-hope-intels-avx-512-dies-a-painful-death/>

Backend Data Base Machines An early backend computer for database management was developed at Bell Labs by Canaday et al. (Canaday et al., 1974).

8.12.1 List of advantages and disadvantages

1. **Economy through specialization.** The backend hardware and especially the software can be specialized to handle just the DBMS function. This means that an OS tailored to the DBMS function rather than a general purpose OS is required allowing a greatly simplified interface between the OS and the DBMS. The expected advantages are: (a) smaller memory requirement; (b) less processing time for simpler programs; (c) smaller development costs; (d) shorter development cycle.
2. **Datasharing.** A backend system may serve two or more host machines. Physical separation of host and backend machines to be located either remotely or locally and still share the database.
3. **Data base protection.** Provide additional reliability and security.
4. **DBMS for a new host machine.** This can be provided using the backend for another host computer by simply writing the interface for a new host.
5. **Cost of a second machine.** This cost is counter-balanced by the reduced cost of the host due to reduced processing.
6. **Unbalanced resources.** The CPU of the back-end machine may turn out to be highly overloaded, while the host is mostly idle, or vice versa. This may be a permanent or transient mismatch.

7. Response time overhead. This is due to intersystem communication and especially transferring data from the backend to the host.

VLSI Search Engine was proposed by Siang Sun Song as part of a 1981 PhD thesis at CMU with H.T. Kung (Harvard U.), who was carrying out research on Systolic Arrays at the time (Song, 1981). Song's array could handle basic relational operators: select, join, project, union, and intersection, etc. Overall performance was determined by the I/O between the two sites.

Non-Von was a paper design of massively parallel DBMs by David Shaw (Columbia U., Morgan-Stanley, hedge fund) as part of his PhD thesis at Stanford U. (Shaw, 1985).

https://en.wikipedia.org/wiki/David_E._Shaw

The intent was high performance execution of data manipulation tasks such as relational algebra operators. The Non-VON architecture was based on a tree-structured primary processing subsystem, with a secondary processing subsystem based on intelligent disk drives. A high-bandwidth parallel interface was provided for rapid data transfer between the two subsystems.

https://en.wikipedia.org/wiki/David_E._Shaw

Multi-Backend Database System - MDBS was developed by David Hsiao et al. (Demurjian et al., 1986). The system is configured with two or more minicomputers, where one minicomputer functions as a host and the others as backends with disk access to provide a centralized database. Backend minicomputers were configured in a parallel fashion and accessible via a broadcast bus. The database was distributed across all of the disks of the backends. Txns were broadcast by the controller to all the backends. Each backend maintained an independent txn queue in order to maximize its access operations and to minimize its idle time. Early DBM research at *Ohio State U.* - *OSU* is discussed (Baum and Hsiao, 1976).

DIRECT I was an MIMD DBM developed by David DeWitt et al. at U. Wisconsin (DeWitt, 1979), (Boral et al., 1982). The original DIRECT design consisted of a set of query processors executing relational algebra operations on the database held in a shared-disk cache based on *Charge Coupled Devices* - *CCDs*. A crossbar switch was used to connect query processors with CCD memories.

https://en.wikipedia.org/wiki/Charge-coupled_device

DIRECT II consisted of eight LSI 11/23 processors, which were connected to a multiport memory addressable at 512 byte boundaries. Such pages could be sent to a single or multiple processors.

Gamma I was another DBM developed at U. Wisconsin, which was based on eleven VAX/780 processors (DeWitt et al., 1988), (DeWitt et al., 1990). A performance evaluation study was conducted to determine the effect of relation size and indices on response time for selection, join, and aggregation queries, and single-tuple updates from the viewpoint of the number of processors employed. A Teradata DBC/1012 DBM of similar size was used for interpreting results.

Gamma II was implemented on an Intel iSPC/2 hypercube with 32 processors and disks (DeWitt et al., 1990).

https://en.wikipedia.org/wiki/Intel_iPSC

Gamma II was scalable to hundreds of processors and had the following features.

1. Horizontally partitioned relations across multiple disk drives enabling relations to be accessed in parallel.
2. Novel parallel algorithms based on hashing were used to implement relational operators such as join and aggregate functions.
3. Dataflow scheduling to coordinate multi-operator queries.

Measurement results exhibited almost linear increase in throughput for hash joins (Ramakrishnan and Gehrke, 2002) as the number of processor/disk pairs was increased.

The *Chained Declustering* - *CD* data layout for replication was proposed in conjunction with the Gamma DBM (Hsiao and DeWitt, 1990), which is described in Section 4.2 in Chapter 4.

Bubba was a DBM developed by Haran Boral (rip) a former student of David DeWitt at U. Wisconsin at the *MicroElectronics and Computer Technology Corporation* - *MCC* at Austin, TX (Boral et al., 1990).

https://en.wikipedia.org/wiki/MicroElectronics_and_Computer_Technology_Corporation

It had forty nodes in its final configuration. Data was declustered across the nodes via hashing or range partitioning (Ramakrishnan and Gehrke, 2002). None of the companies involved with MCC tried to commercialize Bubba, possibly because Teradata's DBC/1012 was already a successful commercial product.

MCC was founded by several US computer companies motivated by Japan's *Ministry of Int'l Trade and Industry* - *MITI Fifth Generation Computer Systems* - *FGCS* project in 1982.

https://en.wikipedia.org/wiki/Fifth_generation_computer

Interest in MCC diminished with the failure of the FGCS project and it ceased operation in 2004.

It is stated in (DeWitt and Gray, 1990) that parallel DBMs with exotic hardware will be replaced by conventional shared-nothing computers, which provide speedup and scaleup in processing relational database queries. A review of techniques used by such systems and a survey of commercial and research systems is provided in this paper.

Super Database Computer - SDC developed at U. Tokyo consists of shared memory multiprocessors with a sort engine (Kitsuregawa and Ogawa, 1990) connected via an Omega interconnection network (Duato et al., 2003), (Dally and Towles, 2004), (Rojas-Cessa, 2017), which contradicts the exotic hardware criterion mentioned in (DeWitt and Gray, 1990).

https://en.wikipedia.org/wiki/Omega_network

Dynamic routing was used to minimize skewed data distribution for hash joins.

A comprehensive survey of parallel architectures for database systems has appeared in (Hurson et al., 1989b).

Processor Per Track - PPT designs for selective data retrieval from rotating disks were based on logic-per-track concept was proposed by Daniel Slotnick in (Slotnick, 1970). A logic chip attached to each read/write head disk was described in (Parker, 1971), where each head could search for a fixed key. The PPT solution is not applicable to modern disks with highly packed tracks, but the idea is applicable to other storage technologies.

Processor Per Head - PPH would be desirable especially if the disk heads could be aligned to read the tracks on the same cylinder in parallel (Gurumurthi et al., 2009).

Processor Per Disk - PPD was proposed by Tomas Lang (UCLA, UCI, rip), Ez Nahourai (IBM, retired), Hirotaka (Ken) Kasuga, and Eduardo B. Fernandez (IBM, FAU) at UCLA in (Lang et al., 1977a). This idea was pursued in the Teradata DBC/1012 computer, which was realized later (Staff, 1985). Teradata was a commercial system utilizing this concept.

Most DBMs are not standalone computers, but were rather developed to offload database processing tasks from mainframes. The IBM 2938 Array Processor was an early system to increase the processing power of S/360 Models 44, 65, 75, 85 for operations on vectors and matrices (Ruggiero and Coryell, 1969). S/360 I/O commands were used to send the data to the AP, which could carry out certain operation such as convolution.

8.13 Head-per-track disks

8.13.1 Rotating Associative Processor for Information Dissemination - RAPID

RAPID was a proposal by Behrooz Parhami (Sharif U., UCSB) at UCLA (Parhami, 1972) motivated by (Slotnick, 1970) is an SIMD computer in that the same command is issued to processors associated with per track R/W heads, which have enough logic to carry out searches by comparing an argument against an appropriate field of disk records as they go past the disk R/W head. Records satisfying the search criterion are read from disk for further processing.

8.13.2 Context Addressable Cellular System for Non-Numeric Processing - CASSM

CASSM was developed by Stanley Su, George Copeland, and Jack Lipovski at U. Florida at Gainesville allows data to be represented in a structure very close to user perception and allows the search operations of high level queries to be implemented directly (Copeland et al., 1973).

8.13.3 Rotating Associative Relational Store - RARES

RARES was developed C.S. Lin and Diane and John Smith at U. Utah supported relational operations and also provided a mechanism for efficient sorting (Lin et al., 1975) Data is stored bit serial byte parallel allowing a reduction in the capacity of storage local to the search logic.

8.13.4 The Context-Addressed Segment-Sequential Storage - CASSS

CASSS is a hardware solution to retrieve data from a large not preorganized file. There is no need for special data structuring to reduce search time (Healy, 1976). Search instructions chosen to provide a wide variety of search operations useful in information retrieval. Of particular importance in this application is the method of quasi-parallel instruction execution, which allows a full string search of the entire data base in a single cycle of the sequential storage device used.

8.13.5 Relational Associative Processor - RAP

RAP is a back-end database processor proposed by Esen Ozkaran (rip) with advisor Kenneth Sevcik (rip) at U. Toronto, which was intended to take over relational database management functions in a computer system (Ozkaran and Sevcik, 1977). RAP's design includes multiprogramming and virtual memory for enhanced performance.

The execution times of several of the operations are evaluated in (Ozkaran et al., 1977). Simulation is used to determine the number of rotations required to read all selected tuples based on a search criterion, since they may be located in the same angular position, referred to hereafter as columns. Of interest is the expected value of the maximum number of selected records in a column. Note that zoned disks with a variable number of sectors per track were not available at the time.

8.13.6 Number of rotations required to read qualifying tuples from RAP

There are two stages in applying relational algebra operators to records or tuples in a table. To process the relational algebra query

$$(\text{age} > 25) \wedge (\text{salary} > 100K)$$

applied to records in an Employee table which includes columns employee#, age, and salary. The first pass marks records with age > 25 and a second pass marks the subset of marked tuples with salary > \$100K.

All selected records can then be read in one disk rotation provided they are all different angular disk positions or columns.

To determine the number of rotations required to read selected records consider an M -track disk with N records per track, so the disk holds $L = N \times M$ records. With a selectivity factor s the number of selected records is $K = \lceil sL \rceil$. For large N it is likely that $K \leq N$ and that the records are in different columns and can be read in one rotation. Otherwise multiple selected records might be in the same column.

Let k_n denote the number of selected records in column n $1 \leq n \leq N$ so that $0 \leq k_n \leq \min(K, M)$ and $\sum_{n=1}^N k_n = K$. The number of rotations is determined by k_{max} .

In the first analysis we simplify the discussion by assuming that $K \leq M$ so that all qualified tuples may be in one column. The number of times the configuration $\underline{k} = \{k_1, k_2, \dots, k_N\}$ occurs is the number of all possible partitions of K among N columns (Kleinrock, 1975):

$$G(K, N) = \binom{K+N-1}{N-1} = \frac{(3+4-1)!}{3! \times 3!} = 20.$$

The mean number of rotations with K selected tuples is given as:

$$\bar{k}_{max} = \sum_{\forall Config.} k_{max} \frac{\# \text{ of configurations with } k_{max}}{G(K, N)}.$$

A program can be written to generate all possible configurations to count the number of rotations if $K > M$.

What is discussed so far is based on the Bose-Einstein - BE statistic.

https://en.wikipedia.org/wiki/Bose%E2%80%93Einstein_statistics

Mean disk rotations with records placed according to Bose-Einstein - BE statistic

Given that a station can hold $M \geq K$ jobs the number of configurations in a queueing network with $N = 4$ stations and $K = 3$ jobs is:

$$G(K, N) = \binom{K+N-1}{N-1} = \frac{3+4-1}{3! \times 3!} = 20.$$

There are four configurations of the form (3,0,0,0) with $k_{max} = 3$.

There are $\binom{6}{2} = 12$ configurations of the form (2,1,0,0) with $k_{max} = 2$.

There are four configurations of the form (1,1,1,0) with $k_{max} = 1$.

The mean of the maximum number of records per column is:

$$\bar{k}_{max}^{BE} = \frac{1}{20}(3 \times 4 + 2 \times 12 + 1 \times 4) = \frac{40}{20} = 2.$$

The flaw of this analysis is that all configurations according to the BE statistic are assumed to be equally likely, which is not true, e.g., the third configuration with three balls in two boxes, is twice as likely as the first two: $|**|$, $||**|$, $|*|*$.

Mean disk rotations with records placed according to Maxwell-Boltzmann - MB statistic

The analysis of static locking or lock preclaiming in (Potier and Leblanc, 1980) is based on the BE statistic, but this problem is addressed in (Thomasian and Ryu, 1983) using the MB statistic, which is more suitable to the problem at hand as noted in (Thomasian, 1998).

https://en.wikipedia.org/wiki/Maxwell%E2%80%93Boltzmann_statistics

We use probability tree diagrams for the analysis.

<https://www.mathsisfun.com/data/probability-tree-diagrams.html>

Records are denoted with asterisks.

$$P1 = P[* , - , - , -] = 1$$

$$P2 = P[* , * , - , -] = P1 \times (N - 1)M / (L - 1) = 9/11 = 9/11$$

$$P3 = P[** , - , - , -] = P1 \times (M - 1) / (L - 1) = 2/11 = 2/11$$

$$P4 = P[* , * , * , -] = P2 \times (N - 2)M / (L - 2) = 9/11 \times 6/10 = 9/11 \times 3/5 = 27/55$$

$$P5 = P[** , * , - , -] = P2 \times 2(M - 1) / (L - 2) = 9/11 \times 4/10 = 18/55$$

$$P6 = P[** , * , * , -] = P3 \times (N - 1)M / (L - 2) = 2/11 \times 9/10 = 9/55$$

$$P7 = P[*** , - , - , -] = P3 \times 1 / (L - 2) = 2/11 \times 1/10 = 1/55$$

$$P8 = P[* , * , * , *] = P4 \times M / (L - 3) = 27/55 \times 3/9 = 9/55$$

$$P9 = P[** , * , * , -] = P4 \times 2M / (L - 3) = 27/55 \times 2/3 = 18/55$$

$$P10 = P[*** , * , - , -] = P5 \times 2M / (L - 3) = 18/55 \times 2/3 = 12/55$$

$$P11 = P[*** , * , * , -] = (P5 + P6) \times 1 / (L - 3) = 27/55 \times 1/3 = 9/55$$

$$P12 = P[*** , * , * , *] = P7 \times 3M / (L - 3) = 1/55$$

$$\bar{k}_{max}^{MB} = \frac{9}{55} \times 1 + \frac{36}{55} \times 2 + \frac{10}{55} \times 3 = \frac{111}{55} \approx 2.02.$$

The MB statistic yields a slightly higher value for the mean number of rotations, which is consistent with intuition.

The analysis in (Seeger, 1996) can be applied with some modifications to the problem at hand.

8.14 Active disks projects

Active disks combine the data storage and processing capability of HDDs, but this concept has been extended to other technologies, such as flash and DRAM. Various applications such as data mining access disks at a lower priority, while the disk is being accessed by an OLTP workload at a higher priority. Active disks utilize excess disk bandwidth and processing power of microprocessors serving as disk controllers.

Disk controllers tend to be less expensive, but less powerful than microprocessors associated with servers. Given the large number of disks the raw cumulative computing power of onboard processors, i.e., the sum of their MIPS ratings, was expected to exceed the computing power of servers.

Given a table of employee salaries distributed over N disks, the i^{th} disk computes the sum of salaries (S_i) and the number of employees (n_i), which can be transmitted to a central node to compute the average salary

$$\bar{S} = \frac{\sum_{i=1}^N S_i}{\sum_{i=1}^N n_i}.$$

Note that there is a major reduction in the volume of transmitted data and I/O bandwidth requirements.

A concise description of active disks is given by Erik Riedel at PDL/CMU (Riedel, 1999).

A number of important I/O intensive applications can take advantage of computational power available directly at storage devices to improve their overall performance more effectively, balance their consumption of system wide resources, and provide functionality that would not be otherwise available.

Active Disks are helpful from the following viewpoint. The size of the databases and the processing requirements for *Decision Support Systems - DSS* are growing rapidly. This is attributable to the length of accumulated history, the level of detail of collected data, and the increased number of users and queries as noted in (Keeton et al., 1998). Influential active disk projects are as follows.

8.14.1 Active disk project at CMU/PDL

This project was led by Erik Riedel (HP/Seagate/EMC/Dell/ITRenew) with Garth Gibson (Vector Institute) and Christos Faloutsos at CMU (Riedel et al., 1998), (Riedel, 1999).

A synthetic dataset from IBM's *Almaden Research Center - ARC Quest* project led by Rakesh Agrawal et al. (Agrawal et al., 1996) for loan applications by evaluating *k-Nearest Neighbor - NN* queries was used in experiments. The relational table contained the following attributes:

(age, education, salary, commission, zip_code, car_make, house_value, outstanding_mortgage).

In addition to analytical models, measurements from a system prototype lead to the conclusion that the active disk system scales beyond the point where the server saturates (Riedel, 1999).

Table scan is a popular operation in relational databases, which is applied when an argument is not sargable, i.e., cannot be accessed via an index. Table scans of large tables can be carried out in parallel if the table is partitioned across several nodes. The processing at the disk acts as a filter reducing the volume of data being transferred. A "GROUP BY" SQL statement computes the min, max, mean based on some categorization, e.g., year in college. To compute the average each disk has to send its mean or sum and the number of selected records.

Freeblock scheduling developed by Christopher R. Lumb et al. at PDL/CMU enhances Active Disks by maximizing disk arm utilization for data transfers (Lumb et al., 2000b). It utilizes opportunistic reading of unread low-priority data blocks, while the arm having accessed a high priority disk block A is moving to process another high priority disk block B.

Consider a disk being accessed by an OLTP application and a background data mining application, such as a *k*-NN query. Disk accesses due to OLTP txns to specific records are processed at a higher priority, since mean txn response time is an important performance metric, while data mining requests are processed as freeblock requests in any order.

The problem with freeblock scheduling is that opportunities for freeblock scheduling of disk blocks at inner and outer disk tracks are smaller. Such blocks may need to be accessed as non-free block accesses to complete the table scan.

In the experimental study reported in (Riedel et al., 2000), OLTP requests were generated by running txns accessing 8 KB disk blocks and the R:W ratio similarly to the TPC-C benchmark was 2:1. The background process accessed 4 KB blocks. The bandwidth due to freeblock requests varies with MPL. The last 5% of requests took 30% of the time of a full scan. Initiating low priority requests when the disk is idle, would result in an increase in the response time of OLTP requests.

Freeblock scheduling outside the disk firmware is described in (Lumb et al., 2000a). The implementation is carried out as a user application in Linux or inside the FreeBSD kernel.

8.14.2 Active disk project at UCSB/Maryland

Anurag Acharya (Google), Mustafa Uysal (HP, VMWare, Google/LinkedIn), and Joel Saltz (SUNY Stony Brook) at *U. Calif. at Santa Barbara - UCSB* and *U. Maryland at College Park* reported on a joint study on active disks in (Acharya et al., 1998).

A host computer runs the main computation and acts as a coordinator, scheduler, and combiner of results, while the bulk of processing is carried out at disks. The datacube operator (Gray et al., 1996a) and three other applications: external sorting, image convolution, and generating composite satellite images were considered.

8.14.3 Intelligent disks - IDisks

This project was carried out by Kimberly Keeton (HP), David Patterson (Google), and Joe Hellerstein at UCB (Keeton et al., 1998). The goal of the project is to move data intensive processing closer to data. Four options were considered for each disk and its processor.

1. **Complete shared-nothing database server and OS:** A well-understood model, but requires up to 64 MB of costly DRAM.
2. **Functions specified by the disk manufacturer:** The database would be executed entirely on a centralized frontend node. Record accesses would entail disk accesses with possible optimization such as *Shortest Access Time First - SATF* (Jacobson and Wilkes, 1991). An advantage of this approach is that it would entail minimal changes to database software.
3. **Run DB storage/data manager and a reduced OS** with the remaining DB functionality executed on a centralized frontend node. This approach would reduce the memory costs, simplifying both the database code and the OS to be run on IDISKS. The advantage of this approach was that the interface between the storage/data manager and the rest of the database software was reasonably clearly defined. The tuple-based interface was too low-level to allow considerable functionality to be implemented on-disk.
4. **Run reduced OS and relational operators,** e.g., scan, sort, and join, on the disk processor. This approach would keep memory costs low, yet allow considerable data-intensive functionality to be implemented on disk. The challenge was to define a general-purpose interface to allow primitive operations to be downloaded to and executed on the IDISK processor. This option is considered the most desirable for IDISK.

An earlier related study Robert Hagmann and Domenico Ferrari at UCB considered the off-loading of database functions to a back-end computer with the goal of maximizing the benefits of distributed processing (Hagmann and Ferrari, 1986). Six variants of the database software were constructed and benchmarked to determine the most interesting functional subdivisions using two different databases and query streams.

8.14.4 SmartSTOR project at IBM Almaden

This project was conducted by Windsor Hsu (IBM, Data-Domain/EMC, Datrium, Alibaba), a PhD student of Alan Jay Smith (UCB, emeritus), while reporting to Honesty Young (Intel, China) at IBM/ARC (Hsu et al., 2000). SmartSTOR is significantly different from the other active disk projects by noting:

Enhancing the computing power of individual disks is not economical due to stringent constraints on the power budget and cost of individual disks. The low market for such high cost, high performance, and high functionality disks would further increase their cost.

Given the three choices: (1) No offloading, (2/3) Single/multiple relation offloading, the third approach is implemented in SmartSTOR, which then provides the necessary coordination among several disks, while the host computer running the application program offloads tasks to one or multiple SmartSTORs. The TPC-D benchmark was used to evaluate SmartStor.

8.14.5 Smart disk cluster project at Northwestern U.

This project was run by Gokhan Memik, Mahmut Kandemir, and Alok Chaudhary at Northwestern U. (Memik et al., 2001). It has many similarities to the aforementioned projects. A novel aspect is the bundling of frequently occurring operations together. A simulator is developed to show that the Smart Disk Cluster outperforms a single host and even the faster cluster architecture considered in this study.

8.14.6 Integrating server, storage and database stack at IBM Almaden

This was a joint study conducted by Lin Qiao at *Silicon Valley Labs - SVL*, Inderpal Narang (retired), and David Chambliss at IBM/ARC (Qiao et al., 2008) noting that IBM storage servers are equipped with 4-8 Power processors and a large memory allows analyzing data locally at the storage server with the following advantages:

(1) Reduction of data flowing to the host, i.e., less data traffic through I/O channel to the host, which results in a better utilization of the host memory bandwidth.

(2) Reduction of host's CPU consumption and hence enabling more workload on the host.

A crucial task is identifying which database operations are beneficial to be offloaded, e.g., offloading starjoin queries (Ramakrishnan and Gehrke, 2002) achieves the above goals and reduces query elapsed time.

8.14.7 Active disks summary

The largest benefit arising from active disks comes from the parallel processing provided by a large number of disks. Although the processing power of each disk controller is much smaller than the server CPUs, the cumulative CPU power at disks may exceed the computing power of servers. Transmitting filtered data reduces bus bandwidth requirements allowing the placement of more disks on I/O buses. This also reduces main memory space requirements. It is argued correctly in (Hsu et al., 2000) that it is not cost effective to increase disk costs by equipping them with more powerful microprocessors, since not all disks will be used as active disks.

8.14.8 Active storage revisited

It is noted in (Smullen et al., 2008) that the proliferation of digital data has resulted in a rapid growth in data-intensive applications, especially in the area of unstructured data processing. The growing popularity of unstructured data processing applications is necessitating rethinking system architectures to efficiently run these applications, from both the performance and power viewpoints. The offloading of computations to disk drive processors is accompanied with the evaluation of the microarchitectural aspects of active storage. A set of unstructured data processing benchmarks are used to examine two choices along the I/O path where the computation can be offloaded in existing system architectures – a disk drive processor and a disk array controller. The evaluation demonstrates that there are interesting tradeoffs in the choice of each location and that microarchitectural enhancements to these processors can provide significant performance boosts. It is shown that active storage architectures can provide large power savings along the I/O path, while exploiting the data-level parallelism on the storage side of the system.

8.15 Multidimensional indices on disk, DRAM, and flash

Survey of multidimensional indices based on (Zhang, 2015) in Subsection 8.15.1. Subsection 8.15.2 discusses the *Singular Value Decomposition* - SVD dimensionality reduction technique combined with data clustering. Multidimensional indices implemented on flash memories in Section 8.16.

8.15.1 Survey of multi-dimensional indices

High dimensional indices pose challenging problems in storing them on disks and DRAM for efficient processing various queries, e.g., *Nearest Neighbor* - NN queries used in similarity search applied to feature vectors (Faloutsos, 1996), (Gaede and Günther, 1998), (Castelli, 2002) in (Castelli and Bergman, 2002), (Yu, 2002), (Samet, 2006). The number of pages accessed from disk and CPU processing time are the major performance metrics, but the major cost difference between sequential and random disk accesses should be noted.

We are concerned with *Point Access Methods* - PAMs rather than *Spatial Access Methods* - SAMs, which in two or three dimensions are of interest in *Computer Aided Design* - CAD. PAMs index high dimensional data points representing feature vectors, which are used for similarity search via nearest neighbor queries.

SAMs represent objects in 2-d with *Minimum Bounding Rectangles* - MBRs, i.e., the coordinates of left lower corner and right upper corner (Faloutsos, 1996). Points are best represented by PAMs, since with 2-d and 3-d SAMs the space required to represent points is doubled and tripled, since corner coordinates are the same.

Quad trees (Finkel and Bentley, 1974) and k-d trees (Bentley, 1975) are two main memory resident indexing structures. k-d-b trees combine k-d trees and one-dimensional B-trees (Ramakrishnan and Gehrke, 2002) to build a disk based multidimensional index (Robinson, 1981). R-trees were designed to handle spatial data on secondary storage for CAD applications (Guttman, 1984). R⁺-trees (Sellis et al., 1987) and R*-trees (Beckmann et al., 1990) are improved version of R-trees.

SS-trees (White and Jain, 1996) use hyperspheres to partition the space, while SR-trees (Katayama and Satoh, 1997) use the intersection of hyperspheres and hyperrectangles to represent regions. SR-trees outperform both R*-trees and SS-trees. X-trees (Berchtold et al., 1996) combine linear and hierarchical structures using supernodes to avoid overlaps to attain improved performance for NN search in higher dimensions.

The Pyramid tree (Berchtold et al., 1998) is not affected by the curse of dimensionality, a term coined by Richard Bellman best known for dynamic programming (Bellman, 2003).

https://en.wikipedia.org/wiki/Curse_of_dimensionality

The *i*MinMax(θ) (Ooi et al., 2000) method which maps points from high-dimensional to single-dimensional space has been shown in experiments to outperform the Pyramid scheme for range queries.

i-Distance (Yu et al., 2001) is an efficient method for k -NN search in a high-dimensional space. Data is partitioned into several clusters and each partition has a reference point. The data in each cluster are transformed into a single dimensional space according to the similarity with respect to a reference point. The one-dimensional values of different clusters are disjoint. A B+tree can be used to index the one-dimensional space and k -NN search are implemented using range searches. The search starts with a small radius, which is gradually increased to form a bigger query sphere to capture k points. The i -Distance method is potentially lossy since multiple data points in the high-dimensional space may be mapped to the same value in the single dimensional space.

One technique to cope with high dimensional data is to reduce dimensionality by pruning less important dimensions after an appropriate rotation of coordinates (Faloutsos, 1996). Dimensionality reduction using *Singular Value Decomposition* - SVD was used in (Korn et al., 1997) with outliers represented with their original dimensions.

Clustered SVD - CSVD results in a higher dimensionality reduction than applying SVD while incurring the same *Normalized Mean Square Error* - NMSE as experimentally demonstrated in (Thomasian et al., 1998), (Castelli et al., 2000, 2003). Consider points surrounding three non-intersecting lines in three dimensional space. Once the line or cluster is determined each points can be represented by a single rather than three coordinates, a three-fold reduction in required memory space and k -NN search. Two points on each coordinate need be specified.

The *Local Dimensionality Reduction* - LDR method (Chakrabarti and Mehrotra, 2000) finds local correlations and performs dimensionality reduction on the locally correlated data. It is shown in (Castelli et al., 2003) that CSVD which uses the k -means clustering method outperforms LDR. *Multilevel Mahalanobis-based Dimensionality Reduction* - MMDR technique (Faloutsos, 1996)

is used in (Jin et al., 2003) to reduce the dimensionality of the original dataset before constructing the index (Jin et al., 2003).

https://en.wikipedia.org/wiki/Mahalanobis_distance

Δ -tree is a main memory index structure, which represents each level with a different number of dimensions (Cui et al., 2005). The number of dimensions increases towards the leaf level, which contains full dimensions of the data.

Vector Approximation - VA-File (Weber et al., 1998) represents each data object using the cell into which it falls. Due to the sparsity of high-dimensional space, it is very unlikely that several points can share a cell. Nearest neighbor search sequentially scans the VA-File to determine the upper and lower bound distance from the query to each cell. During the filter step, if the approximate lower bound is greater than the current upper bound, it is not considered further and otherwise it is a candidate. During the refine step, all the candidates are sorted according to their lower bound distances. The actual objects are retrieved and the distance to the query is computed. The nearest points are returned. The drawback is that the performance is highly dependent on the number of bits per dimension.

Metric trees transform the feature vector space into metric space, and then index the metric space (Uhlmann, 1991). A metric space is a pair $\mathcal{M} = (\mathcal{F}, d)$, where $\mathcal{F}$ is a domain of feature values, and d is a distance function with the following properties:

- (i) **Symmetry** $d(F_x, F_y) = d(F_y, F_x)$,
- (ii) **Non-negativity** $d(F_x, F_y) > 0, F_x \neq F_y$,
- (iii) **Triangle inequality** $d(F_x, F_y) \leq d(F_x, F_z) + d(F_z, F_y)$.

The search space is organized based on relative distances of objects, rather than their absolute positions in a multi-dimensional space.

Vantage Point - VP-tree (Yianilos, 1993) partitions a dataset according to distances between the objects and a vantage or reference point, which is a corner point. The median value of the distances is chosen as separating radius to partition dataset into two balanced subsets. The same procedure is applied recursively on each subset.

MVP-tree (Bozkaya and Özsoyoglu, 1997) uses multiple vantage points and exploits pre-computed distances in the leaf nodes to provide further filtering during search operations. Both trees are built in a top-down manner and balance can not be guaranteed during insertion and deletion. Costly reorganization is required to prevent performance degradation.

M-tree (Ciaccia et al., 1997) is a paged metric-tree index. It is balanced and able to deal with dynamic data. Leaf nodes of an M-tree store the feature vectors of the indexed objects O_j and distances to their parents, whereas internal nodes store routing objects O_r , distances to their parents O_p , covering radii $r(O_r)$ and corresponding covering tree pointers. The M-tree reduces the number of distance computations by storing distances.

OMNI-family (Filho et al., 2001) is a set of indexing methods based on the same underlying theory that all the points S_i located between ℓ and u are candidate results for a spherical query with radius r and given point Q for a specific focus F_i , where $\ell = d(Q, F_i) - r, u = d(Q, F_i) + r$. For multiple foci, the candidates are the intersections of S_i . Given a dataset, a set of foci is found. For each point in the dataset the distance to each of the foci is calculated. The search process can be applied to sequential scan, B+-trees and R-trees. For B+ trees, the distances for each focus F_i are indexed, a range query

is performed on each index, finally the intersection is obtained. For R-trees, the distances for all the foci, which forms a lower dimensional data, are indexed, and single range query is performed.

8.15.2 Combining dimensionality reduction with clustering

Query by Image Content - QBIC project at IBM/ARC (Flickner et al., 1995) utilized *Content-Based Image Retrieval - CBIR* method with two steps:

- (i) characterize images by their features and define similarity measures.
- (ii) extract features such as color, texture, and shape.

The similarity of images represented as a point in high-dimensional space is determined by *k-nearest-neighbor - k-NN* queries.

Representing images by their feature vectors and searching for similarity according to the distances of the points representing them in high dimensional space via *k-NN* queries with respect to a target image is a popular paradigm. Given satellite images feature vectors representing texture were first computed and used for representing images.

Dimensionality reduction via SVD to individual clusters of a dataset results in higher dimensionality reduction for the same NMSE than applying SVD to the whole dataset. The cost of processing *k-NN* queries is further reduced by suitable indexing structures such as the *Ordered-Partition - OP-tree* index (Kim and Park, 1986) and the *Stepwise Dimensionality Increasing - SDI-tree* (Thomasian and Zhang, 2006).

A survey of data reduction methods is given in (Barbará et al., 1997). SVD described in (Faloutsos, 1996) was used in (Korn et al., 1997) to compress data so that it can be fitted into main memory for *k-NN* search. The code for SVD is provided in “Numerical Recipes” (Press et al., 2007), but the IBM *Engineering and Scientific Subroutine Library - ESSL* (IBM Corp., 2000) was used in (Castelli et al., 2003).

k-NN queries are based on the Euclidean distance corresponding for $p = 2$ in the following formula.

$$D^2(\mathbf{u}, \mathbf{v}) = \left[\sum_{i=1}^n |u_i - v_i|^p \right]^{1/p}. \quad (8.4)$$

Given precomputed norms only requires the computation of the inner product of the vectors.

$$D^2(\mathbf{u}, \mathbf{v}) = ||\mathbf{u}||^2 + ||\mathbf{v}||^2 - 2\mathbf{u} \times \mathbf{v}.$$

k-NN queries can be implemented by a sequential scan of the dataset holding the feature vectors of all images. This has a running time of $O(MN)$, where M is the dataset cardinality (number of images) and N is the dimensionality (number of features per image). Interestingly sequential scans outperform some multidimensional indexing methods for higher dimensions.

Combining clustering and SVD to attain higher efficiency for CBIR was considered in (Thomasian et al., 1998), (Chakrabarti and Mehrotra, 2000), (Castelli et al., 2000), (Castelli et al., 2003), (Böhm et al., 2004), since higher dimensionality reduction is attained by taking advantage of local correlations in clustered data.

Clustering reduces the cost of *k-NN* search roughly by the number of clusters (H) assuming that there are approximately the same number of points per cluster. Clusters can be represented compactly as hyperspheres with a centroid and a radius, which is the distance of the farthest point from the centroid.

Since a query point may reside in the volume shared by two or more hyperspheres, multiple clusters may need to be searched to locate the k nearest-neighbor points. *k-NN* search has to be trimmed as much as possible to reduce query processing cost.

Indexing is a form of clustering, however, most indexing structures, such as those in the R-tree family perform poorly for higher dimensions (Faloutsos, 1996), since multiple overlapping MBRs need to be searched.

<https://en.wikipedia.org/wiki/R-tree>

This is the incentive for dimensionality reduction before indexing.

8.15.3 Dimensionality reduction via SVD, PCA, KLT

Principal Component Analysis - PCA, *Singular Value Decomposition - SVD*, and *Karhunen-Loève Transform - KLT* are related methods to reduce the number of dimensions after rotating coordinates to attain the minimal information loss (lost variance) for the desired level of data compression (Faloutsos, 1996).

Consider matrix $\mathbf{X}$, whose M rows are the feature vectors for images. Each image has N features, which constitute the columns of $\mathbf{X}$, whose SVD decomposition is given as (Faloutsos, 1996):

$$\mathbf{X} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^T, \quad (8.5)$$

where $\mathbf{U}$ is an $M \times N$ column-orthonormal matrix, $\mathbf{V}$ is an $N \times N$ unitary matrix of *eigenvectors*, and $\mathbf{\Sigma}$ is a diagonal matrix of singular values.

Without loss of generality: $\sigma_i \geq \sigma_{i+1}$, $1 \leq i$, so that the rank of $\mathbf{X}$ equals the number of nonzero singular values (Faloutsos, 1996). The cost of computing eigenvalues using SVD is $O(MN^2)$.

PCA is based on eigenvalue decomposition of the *covariance matrix* $\mathbf{C}$.

$$\mathbf{C} = \frac{1}{M} \mathbf{X}^T \mathbf{X} = \mathbf{V} \mathbf{\Lambda} \mathbf{V}^T, \quad (8.6)$$

where $\mathbf{V}$ is the matrix of *eigenvectors* as in Eq. (8.5) and the diagonal matrix $\mathbf{\Lambda}$ holds the *eigenvalues*. Since $\mathbf{C}$ is positive-semidefinite its N eigenvectors are orthonormal and its eigenvalues are nonnegative. In addition to the cost of computing $\mathbf{C}$, which is $O(MN^2)$ there is the cost of computing the eigenvalues is $O(N^3)$.

It follows from Eq. (8.7) $\lambda_i = \sigma_i^2/M$ and conversely $\sigma_i = \sqrt{M\lambda_i}$ for $1 \leq i \leq N$.

$$\mathbf{C} = \frac{1}{M} \mathbf{X}^T \mathbf{X} = \frac{1}{M} (\mathbf{V} \mathbf{\Sigma} \mathbf{U}^T) (\mathbf{U} \mathbf{\Sigma} \mathbf{V}^T) = \frac{1}{M} \mathbf{V} \mathbf{\Sigma}^2 \mathbf{V}^T. \quad (8.7)$$

The eigenvectors constitute the principal components of $\mathbf{X}$, hence the following transformation yields uncorrelated features:

$$\mathbf{Y} = \mathbf{X} \mathbf{V}. \quad (8.8)$$

Retaining the first n dimensions of $\mathbf{Y}$ maximizes the fraction of preserved variance and minimizes the NMSE:

$$\text{NMSE} = \frac{\sum_{i=1}^M \sum_{j=n+1}^N y_{i,j}^2}{\sum_{i=1}^M \sum_{j=1}^N y_{i,j}^2} = \frac{\sum_{j=n+1}^N \lambda_j}{\sum_{j=1}^N \lambda_j} \quad (8.9)$$

Dimensionally reduced data via SVD allows approximate processing of k -NN queries.

Recall and *Precision* are two metrics, which are used to quantify retrieval accuracy and efficiency (Faloutsos, 1996).

https://en.wikipedia.org/wiki/Precision_and_recall

Precision is the fraction of retrieved documents which are relevant to the query, while *recall* is the fraction of the relevant documents that are retrieved. Exact processing of k -NN queries on dimensionality reduced data is discussed in (Korn et al., 1998) and this method is extended to exact processing over multiple clusters in (Thomasian et al., 2008).

A more efficient method to compute Euclidean distance with dimensionally reduced data than the one in (Korn et al., 1997) is used in (Thomasian et al., 1998) and specified in (Thomasian et al., 2008).

8.15.4 Clustering methods for dimensionality reduction

Cluster analysis partitions a set of objects specified by multi-dimensional feature vectors into clusters, so that the objects in a cluster are similar, but objects in different clusters are dissimilar. Clustering is discussed in Chapter 9 and only certain aspects of clustering are discussed here.

Clustering algorithms tend to fail in high dimensional spaces, because of the inherent sparsity of the points. Clustering of high dimensional data is classified as: subspace, pattern-based, and correlation clustering in (Kriegel et al., 2009).

Clustering makes it possible to take advantage of local correlations and this results in a smaller NMSE versus applying SVD to the whole dataset.

Subspace Clustering finds clusters in different lower-dimensional subspaces of a dataset taking advantage of the fact that many dimensions in high dimensional datasets are redundant and hide clusters in noisy data (Kriegel et al., 2009).

CLustering In QUES - CLIQUE is a subspace clustering method which combines density and grid based clustering (Agrawal et al., 1993a). An a priori style search method for *Association Rule Mining - ARM* (Agrawal et al., 1993b) is used to find dense subspaces. Once the dense subspaces are found they are sorted by coverage and only subspaces with the greatest coverage are kept and the rest are pruned. The algorithm then looks for adjacent dense grid units in each of the selected subspaces using a depth first search. Clusters are formed by combining these units using a greedy growth scheme.

The algorithm starts with an arbitrary dense unit and greedily grows a maximal region in each dimension, until the union of all the regions covers the entire cluster. The weakness of this method is that the subspaces are aligned with the original dimensions.

Projected Clustering - ProClus uses an algorithmic framework to determine the number of dimensions in each such cluster-specific subspace (Aggarwal et al., 1999). **Oriented projected CLUSter - ORCLUS** in (Aggarwal and Yu, 2000) differs from ProClus in that similarly to CSVD (Thomasian et al., 1998), (Castelli et al., 2003) it looks for non-axes parallel subspaces. The algorithm is divided into three steps:

- (1) **Assign Phase:** Data points are assigned to the nearest cluster centers.
- (2) **Subspace Determination:** Redefines the subspace associated with each cluster by calculating the covariance matrix.
- (3) **Merge Phase:** Merge near clusters.

The number of clusters and the subspace dimensionality must be specified and a general scheme for selecting suitable values is provided. A statistical measure called the cluster sparsity coefficient is provided for inspecting after clustering to evaluate the choice of subspace dimensionality.

In an attempt to attain a higher dimensionality reduction the *Local Dimensionality Reduction - LDR* method in (Chakrabarti and Mehrotra, 2000) generates SVD-friendly clusters. It takes advantage of *Lower Bounding Property - LBP* (Korn et al., 1998) for exact query processing. It produces fewer false positives by incorporating the reconstruction distance (the squared distance of eliminated dimensions) as an additional dimension with each point. A succinct description of the LDR method is given in the Appendix in (Castelli et al., 2003). CSVD is preferable to LDR on two accounts.

- CSVD produces indexes with better precision-recall characteristics than SVD for the same index size, or, equivalently, smaller indexes having similar precision-recall characteristics.
- For Recall = 94%, where queries return on average 19 out of the 20 correct results, CSVD is roughly 150 times faster than linear scan, 5.7 times faster than LDR best, and eight times faster than LDR on the average.

Computing Correlation Connected Clusters - C4 in (Böhm et al., 2004) is based on a combination of PCA and density-based clustering paradigm (DBSCAN).

<https://en.wikipedia.org/wiki/DBSCAN>

The method has a determinate result and is robust against noise and has superior performance over competing methods such as DBSCAN, CLIQUE, and ORCLUS.

8.15.5 CSVD to build an index for k -NN queries

The following steps are followed for combining SVD and clustering in (Castelli et al., 2003):

- Select an objective function:** (1) *Index Space Compression - ISC*, defined as the ratio of the original size ($N \cdot M$), to the size attained after applying SVD to H clusters as given by Eq. (8.10).
- (2) NMSE given by Eq. (8.9),
 - (3) Desired recall,

$$V = N \cdot H + \sum_{h=1}^H (N \cdot p_h + m_h \cdot p_h), \quad (8.10)$$

where m_h is the number of points and p_h the number of dimensions retained in the h^{th} cluster after rotating its coordinates, denoted by $\mathbf{Y}^{(h)}$. The first term accounts for the space required by the centroids, while $N \cdot p_h$ is the space occupied by the projection matrix and $m_h \cdot p_h$ is the space required by the projected points.

Selecting the number of clusters: H can be specified by the user or determined by database size to maintain a reasonable size for clusters which can be fitted into main memory. Some clustering algorithms determine the desired number of clusters.

Partitioning: This step divides the row of the database table $\mathbf{X}$ into H clusters: $\mathbf{X}^{(h)}$, $h = 1, \dots, H$ with $m_1, \dots, m_H$ points. CSVD supports a wide range of classical clustering methods such as the *Linde-Buzo-Gray - LBG* algorithm (Linde et al., 1980), since the k -means method does not scale well to high dimensions.

https://en.wikipedia.org/wiki/Linde%E2%80%93Buzo%E2%80%93Gray_algorithm

For each cluster, this step determines the centroid $\mu^{(h)}$, and the *cluster radius* $R^{(h)}$, i.e., the distance of the farthest point from the centroid. The coordinates of the centroid $\mu^{(h)}$ are subtracted from each group of vectors $\mathbf{X}^{(h)}$.

Rotating partitions: For $\mathbf{X}^{(h)}$, $h = 1, \dots, H$ the eigenvectors are computed according to Eq. (8.5) or Eq. (8.6) for the partition. The coordinates of points in partitions are rotated into uncorrelated frames of reference separately for each cluster according to Eq. (8.8).

Dimensionality reduction: This is a global procedure, which depends on the selected objective function. The products of $\lambda_i^{(h)} m_h, \forall i, \forall h$ are sorted in ascending order to produce an ordered list $\mathcal{L}$ of $H \cdot N$ elements. Each element j of the list contains the label κ_j of the corresponding cluster, and the dimension ∂_j associated with the eigenvalue. The list $\mathcal{L}$ is scanned starting from its head. During step j , the ∂_j^{th} dimension of the κ_j^{th} cluster is discarded. The process ends when the target value of the objective function is reached:

Index space compression. The index volume is computed using Eq. (8.10).
The NMSE is recomputed using Eq. (8.11).

$$NMSE = \frac{\sum_{h=1}^H \sum_{i=1}^{m_h} \sum_{j=n_h+1}^N (y_{i,j}^{(h)})^2}{\sum_{h=1}^H \sum_{i=1}^{m_h} \sum_{j=1}^N (y_{i,j}^{(h)})^2} = \frac{\sum_{h=1}^H m_h \sum_{j=n_h+1}^N \lambda_j^{(h)}}{\sum_{h=1}^H m_h \sum_{j=1}^N \lambda_j^{(h)}}. \quad (8.11)$$

As dimensions are omitted an experiment is run to determine the recall with a sufficiently large number of sample queries (Castelli et al., 2003).

Constructing the within-cluster index. Build index for cluster $\mathbf{Y}^{(h)}, 1 \leq h \leq H$.

8.15.6 Nearest-neighbors query processing for multiple clusters

k -NN queries follow a branch-and-bound algorithm. At first the feasible region of the search problem is the entire space, and the partition is given by the clustering. During the search the target function is upper-bounded by the distance of the current k^{th} neighbor in k -NN search, and the feasible region is pruned by discarding clusters having distance larger than the running upper-bound. The following steps are used in carrying out k -NN queries:

Preprocessing and primary cluster identification.

Distance computation from clusters: The distance between the preprocessed query point $\mathbf{q}$ and cluster i is defined as

$$\max \left\{ \left[D(\mathbf{q}, \mu^{(i)} - R^{(i)}), 0 \right] \right\}.$$

The clusters are sorted in increasing order of distance with the primary cluster in first position.

Searching the primary cluster: This step produces a list of k -NN points ordered in increasing order of distance. Let d_{max} be the distance of the farthest point in the list.

Searching the other clusters: If the distance to the next cluster does not exceed d_{max} , then the cluster is searched, otherwise, the search is terminated. While searching a cluster, if points closer to the query than d_{max} are found, they are added to the list of k current best results, and d_{max} is updated.

8.15.7 Exact versus approximate k -NN queries

Post-processing to attain exact k -NN processing takes advantage of the *Lower-Bounding Property* - LBP (Faloutsos, 1996) (also see Lemma 1 in (Korn et al., 1998)):

Given that the distance of points in the subspace with reduced dimensions is less than the original distance, a range query guarantees no false dismissals.

False alarms are discarded by referring to the original dataset. Noting the relationship between range and k -NN queries, the latter can be processed as follows:

- (1) Find the k nearest neighbors of the query point Q in the subspace.
- (2) Determine the farthest actual distance to Q among these k -NN points, denoting it by d_{max} .
- (3) Issue a range query centered on Q with radius d_{max} .
- (4) For all points obtained in this manner find their original distances to Q , by referring to the original dataset and rank the points, i.e., select the k -NNs.

An offline experiment was used to determine k^* , which yields a recall $\mathcal{R} \approx 1$. The CPU time required by the exact method for a sequential scan is lower than the approximate method, even for $\mathcal{R} = 0.8$, which is attributable to the fact that the exact method issues a k -NN query only once and this is followed by less costly range queries.

Optimal data reduction is studied in the context of the exact query processing in (Thomasian et al., 2008). At one extreme retaining all dimensions does not require post processing, while at the other extreme the cost of the initial K -NN

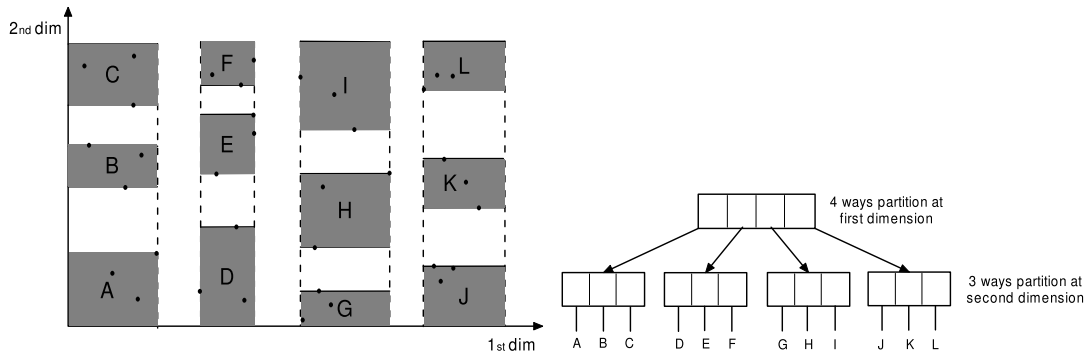


FIGURE 8.1 The partition for an OP-tree with 12 points and the corresponding linked-list hierarchical structure.

processing step is reduced with few dimensions, but there is an increase in postprocessing cost, so that a minimum CPU cost is observed as the NMSE is varied.

8.15.8 Indices suited for dimensionality reduced data

PAM indexing structures can be classified as *Data Partitioning - DP*, such as R-tree, SS-tree, SR-trees and *Space Partitioning - SP*, k-d-b-tree, hB-trees (Samet, 2006). DP partitions space in a manner that regions have an approximately equal number of points. SP divides space into non-overlapping regions, so that any point in the space can then be identified to lie in exactly one region. Another categorization is disk-resident versus main memory-resident. In the former (resp. latter) case the number of disk pages accessed (resp. CPU time) are the major performance measures.

With the increasing dimensionality of feature vectors, most multi-dimensional indices lose their effectiveness. The so-called curse of dimensionality results in an increased overlap among the nodes of the index and a low fanout and an increased index height (Faloutsos, 1996).

https://en.wikipedia.org/wiki/Curse_of_dimensionality

David L. Donoho at Stanford has talked about the curses and blessings of dimensionality.

<https://statistics.stanford.edu/events/high-dimensional-data-analysis-curses-and-blessings-dimensionality>

8.15.9 Ordered Partition - OP tree index

The OP-tree index for efficient processing of k -NN queries, recursively equipartitions data points one dimension at a time in a round-robin manner until the data points can fit into a leaf node in (Kim and Park, 1986). The two properties of OP-trees are ordering and partitioning. Ordering partitions the search space and partitioning rejects unwanted space without actual distance computation. A fast k -NN search algorithm is also described in (Kim and Park, 1986). The original OP-tree index had one point per partition, but was extended to multiple points per partition in (Castelli et al., 2003) as shown in Fig. 8.1. Consider the following sample data set with twelve points.

1:(1, 2, 5), **2**:(3, 8, 7), **3**:(9, 10, 8), **4**:(12, 9.2), **5**:(8, 7, 20), **6**:(6.6, 23),

7:(0, 3, 27), **8**:(2, 13.9), **9**:(11, 11, 15), **10**:(14, 17, 13), **11**:(7, 14, 12), **12**:(10, 12, 3)

As far as the first partition is concerned we partition points into three regions, so that the number of points in each region is about the same.

$R_1(-\infty, 3)$, $R_2(3, 6)$, $R_3(6, +\infty)$

R_1 is partitioned into subregions $R_{1,1}(-\infty, 2)$, $R_{1,2}(2, 8)$, $R_{1,3}(8, +\infty)$

R_2 is partitioned into subregions $R_{2,1}(-\infty, 4)$, $R_{2,2}(4, 7)$, $R_{2,3}(7, +\infty)$

R_3 is partitioned into subregions $R_{3,1}(-\infty, 9)$, $R_{3,2}(9, 12)$, $R_{3,3}(12, +\infty)$

Partitioning using the second dimension into three partitions yields nine subregions. The points held by the partitions are as follows

$R_{1,2}$ (**1**), $R_{2,1}$ (**7**), $R_{3,1}$ (**8**),

$R_{2,1}$ (**11**), $R_{2,2}$ (**5**, **6**), $R_{2,3}$ (**3**),

$R_{3,1}$ (**4**), $R_{3,2}$ (**12**, **9**), $R_{3,3}$ (**10**).

The building of a persistent semi-dynamic OP-tree index is presented in (Thomasian and Zhang, 2008). Serialization is used to compact the dynamically allocated nodes of the OP-tree in main memory, which form linked lists, into contiguous

memory locations. Alternatively, a compact index can be built directly to fulfill this condition. The index can then be saved onto disk as a single file and loaded into main memory with a single data transfer. This takes less time than loading individual index pages one at a time, because of the high positioning time, which is improving at a very slow rate, while the improvement in disk transfer rates is significant. However the touching of a single page results in the prefetching of the remaining pages of the index was initiated by Windows OS.

The original OP-tree was static and the tree had to be rebuilt in main memory as new points are added. Several methods which support inserting points dynamically are described in (Thomasian and Zhang, 2006). The SVD transform-based dimensionality reduction to ensure high query precision has the drawback that SVD is expensive to compute and not readily applicable to dynamic data.

Performing SVD-based dimensionality reduction in dynamic databases is discussed in (Kanth et al., 1998). “When the data distribution changes considerably so as to degrade query precision, the recomputation of the SVD transform and incorporation in the existing index structure is recommended. For recomputing the SVD-transform, we propose a novel technique that uses aggregate data from the existing index rather than the entire data. This technique reduces the SVD-computation time without compromising query precision. We then explore efficient ways to incorporate the recomputed SVD transform in the existing index structure without degrading subsequent query response times. These techniques reduce the computation time by a factor of 20 in experiments on color and texture image vectors. The error due to approximate computation of SVD is less than 10%”.

8.15.10 Stepwise Dimensionality Increasing SDI-tree index

The SDI-tree is aimed to reduce the number of disk accesses and CPU processing cost (Thomasian and Zhang, 2006). It is built using feature vectors transformed via PCA. Dimensions are retained in non increasing order of their variance using the parameter p , specified below. The SDI-tree shown in Fig. 8.2 is a disk-resident index with each node corresponding to a disk page. Starting with a few features at the highest level, the number of retained feature vector elements is increased to include all of the dimensions.

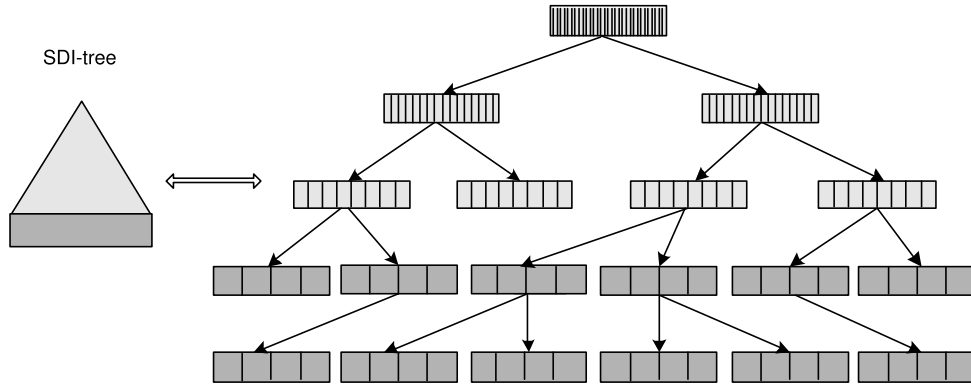


FIGURE 8.2 The SDI-tree representation.

A node of the index is an array of entries $EntrySize$, which is a function of the dimensionality. Given pagesize S and $n_\ell \leq N$ as the number of dimensions at level ℓ the fanout at level ℓ is:

$$F_\ell \approx S/EntrySize(n_\ell).$$

The nodes of the tree are organized as hyperspheres, with both the centroid and the radius calculated based on n_ℓ dimensions.

Parameter p which determines the number of dimensions n_ℓ and the fanout F_ℓ at level ℓ , is the fraction of increased variance at successive levels of the index, starting with the highest level, until 100% variance is achieved. The number of dimensions selected at level $\ell \geq 1$ satisfies;

$$\sum_{k=1}^{n_\ell} \lambda_k / \sum_{k=1}^N \lambda_k \geq \min(\ell \times p, 1).$$

Fig. 8.3(a) shows the cumulative normalized variance versus the number of dimensions for dataset COLH64 (in (Castelli et al., 2003)). With $p = 0.20$ the number of dimensions at level one through five is 2, 4, 8, 16 and 64. Fig. 8.3(b) shows the case for dataset TXT55 (in (Castelli et al., 2003)) with $p = 0.30$. The number of dimensions at level one through four in this case is 2, 8, 21, and 55. The SDI index differs from the Δ -tree in that it is a disk resident index structure with fixed node sizes, while the Δ -tree is a main memory resident index with variable node sizes and fixed fanouts. The SDI-tree differs from the *Telescopic Vector - TV-tree* (Lin et al., 1994) in that it uses a single parameter, specifying the fraction of variance to be added to each level, without the risk of having a large number of active dimensions.

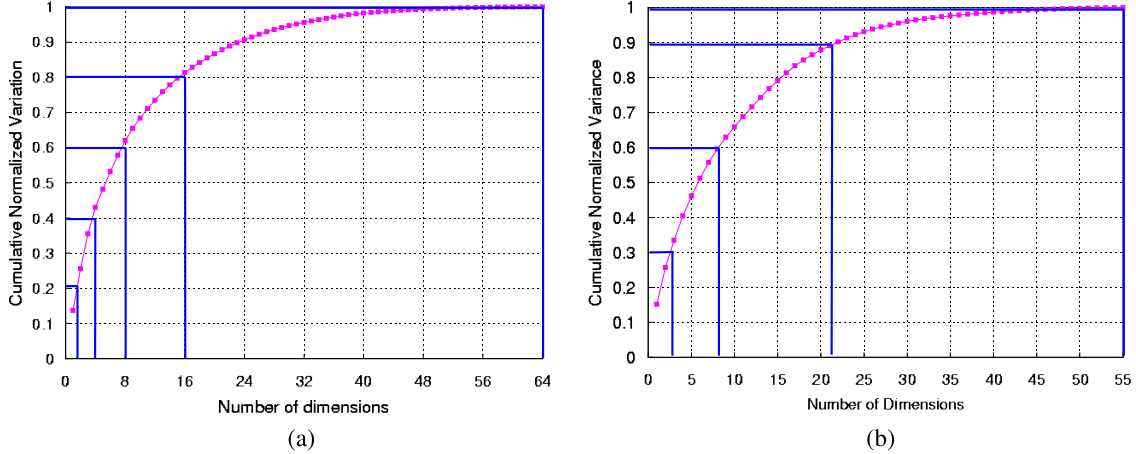


FIGURE 8.3 Cumulative variance v.s. number of dimensions. (a) COLH64. (b) TXT55.

SDI trees are compared to VAMSR-trees and the VA-file indexes in (Thomasian and Zhang, 2006). The VAMSR-tree uses the same split algorithm as VAMSplit R-tree (White and Jain, 1996), but it is based on an SR-tree structure. Experiments have shown that SDI-trees access fewer disk pages and incur less CPU time than SR-trees, VAMSR-trees, Vector Approximation - VA-Files, and the *iDistance* method. In CPU time SDI outperforms the sequential scan and OMNI methods.

8.16 Implementing indices in flash memories

The following discussion is based on (Fevgas et al., 2020) which discuss placing indices in flash SSDs, rather than magnetic disks. Treating SSDs as simply another category of block devices ignores their idiosyncrasies, like erase-before-write, wear-out and asymmetric read/write, and may lead to poor performance. These peculiarities of SSDs require reinvention of the indexing techniques that have been designed primarily for HDDs. In this work, we present a concise overview of the SSD technology and the challenges it poses. The article surveys 62 flash-aware indexes for various data types, analyze the main techniques they employ, and comment on their main advantages and disadvantages, aiming to provide a systematic and valuable resource for researchers working on algorithm design and index development for SSDs. Additionally, we discuss future trends and new lines of research related to this field.

Flash memories were introduced in Chapter 2. Byte addressable NAND cells could use only a single voltage threshold storing it. *Single Level Cell - SLC*, *MultiLevel Cell - MLC*, *Triple Level Cells - TLC*, *Quadruple Level Cell - QLC* storing single, two, three, four bits per cell, exploiting multiple voltage thresholds. QLC flash provides higher capacities for read-intensive applications, but is slower and less durable than MLC and TLC. Another method to increase flash storage density is to stack flash cells in layers, as in 3D NAND or *Vertical NAND - V-NAND* and up to 64-layer chips are available, while 96-layer TLC NAND has been introduced. Three important features of NAND memories are as follows (Michelsoni et al., 2010).

1. Asymmetric read/write latencies: reads are faster than writes.
2. Writes are slow because an erase-before-write, which means that in place updates unlike HDDs triggers a number of reads and writes in flash devices.
3. Flash memories have a limited life cycle: every flash cell exhibits a high bit error rate (wear-out) after a certain number of program/erase cycles have been performed. Uneven writing of the pages may render whole areas completely unreliable or unavailable.

To hide or even alleviate some of these peculiarities, SSDs provide the *Flash Translation Layer - FTL*, which runs on the device controller and employs an out-of-place policy, using logical address mapping, and takes care of wear leveling, space reclamation, and bad block management. Thus, SSDs operate as block devices through FTL.

Traditional indices will perform poorly if they are directly written to SSDs, e.g., B+-trees during updates generate small random writes, which may lead to long write times.

SSD's Host Interface Controller is connected to a CPU with an SRAM cache, which holds the firmware and DRAM controller and multiple flash controllers connected via dedicated channels to flash packages.

NVM express - NVMe is an optimized, high-performance scalable host controller interface designed to utilize *PCI express - PCIe* based solid-state storage. NVMe provides efficient access to storage devices built with NVM. The performance metrics addressed by NVMe are bandwidth, *I/Os per second - IOPs*, and latency. The maximum IOPs possible for Serial ATA was 200 KIOPS whereas NVMe devices can exceed 1,000 KIOPS. By supporting PCIe and Fabrics such as *Remote Direct Memory Access - RDMA* and Fibre Channel.

https://en.wikipedia.org/wiki/Remote_direct_memory_access

NVMe can support much higher bandwidths than SATA or *Serial Attached SCSI - SAS*

https://en.wikipedia.org/wiki/Serial_ATA

https://en.wikipedia.org/wiki/Serial_Attached_SCSI

Finally, next generation memory technologies may have read access latency under a microsecond, requiring a streamlined protocol that enables an end-to-end latency of under 10 microseconds, including the software stack.

NVMe is a completely new architecture for storage, from the software stack to the hardware devices and systems. Unlike SAS and SATA protocols NVMe protocol supports multiple deep queues. Typical SAS devices support up to 256 commands and SATA devices support up to 32 commands in a single queue. These were adequate for hard disk drive technologies, but cannot take full advantage of current and next-generation NVM technologies. In contrast, NVMe supports 64K commands per queue and up to 64K queues. These queues are designed such that I/O commands and responses to those commands operate on the same processor core and can take advantage of the parallel processing capabilities of multi-core processors. Each application or thread can have its own independent queue, so no I/O locking is required.

FTL obscures flash idiosyncrasies by providing a standard interface with the host system. The main functions of FTL are address mapping, wear leveling, garbage collection, and bad block management. FTL implements an out-of-place update policy, since flash cells cannot be updated in-place. It marks old pages as invalid and writes the updated data into clean locations. It hides the complexity of programming the flash cells, maintaining a logical to physical address mapping mechanism. Three different mapping methods have been proposed, which differ from the viewpoint of mapping granularity: page-level, block-level, and hybrid (Kwon et al., 2011).

Out-of-place updates leave flash pages invalidated; thus, FTL initiates a garbage collection procedure to reclaim free space for incoming data (Cornwell, 2012). Garbage collection is invoked when the drive is idle or runs out of free pages. It exploits the pages utilization index and invalid total pages to select the proper blocks (Kwon et al., 2011) Next, it copies any valid pages of the candidate blocks into new positions, and performs the erases. Continuous erase operations wear flash memory cells and FTL provides a wear leveling mechanism to protect them from premature aging. It distributes the erasures uniformly across all medium, utilizing an erasure counter for each erase block. SSDs contain bad blocks, which occurred during operation, or even preexisted as a result of the manufacturing process. FTL handles defective blocks through address mapping. When a block is recognized as faulty, its data is copied to another functional block and it is marked as bad for future use.

Large batches of parallel I/Os are to be utilized to ensure that SSDs internal parallelization and NVMe protocol are fully exploited. In-memory buffering is a prerequisite for batched I/O and it also accelerates the performance of indices, since it reduces the number of accesses to secondary storage. Proposed methods exploit in memory buffers for logging, overflow nodes and use batch write or/and read operations.

Early works used batch writes to alleviate the difference between reads and writes, whereas more recent ones target internal parallelization. Batched I/O exploits the internal parallelization and the potentials of the NVMe protocol in a more efficient way, thus better performance is achieved. They also eliminate small random write operations that may cause frequent garbage collection operations and fragmentation of the FTL mapping table; this is highly dependent on the proprietary mapping algorithm of FTL.

The large batches of random reads also exploit the internal parallelization of SSDs and the efficiency of the NVMe protocol. Additionally, they do not differ much, in terms of performance, from the sequential ones. Using large I/O batches, besides accelerating the performance of reads and writes, results also in the separation of reads and writes, limiting the interference between them.

Open-Channel architecture proposal does not have a firmware FTL implemented on the device, but instead leaves the management of the physical solid-state storage to the computer's OS.

https://en.wikipedia.org/wiki/Open-channel_SSD

Elevating commodity storage with the SALSA host translation layer is relevant to this discussion (Ioannou et al., 2018).

Flash-based indices can be categorized as follows:

- (1 and 2) FTL and raw based B-trees.
- (3) Skip lists.
- (4 and 5) FTL and raw based hash indices.
- (6 and 7) FTL and raw based PAM indices.
- (8) FTL based SAM indices.
- (9) Generic frameworks.
- (10 and 11) FTL and raw based inverted indices.

Examples of indexes in various categories are provided in the paper.

In a recent study Bianca Schroeder et al. studied the reliability of modern file systems in the face of SSD errors (Jaffer et al., 2020). File systems considered are ext4, Btrfs, and F2FS. Different SSD error modes are considered and fault injection is used to explore error cases. The results indicate that in 16% of cases a file system cannot be mounted or repaired. Key file metadata structures that can cause such failures and design guidelines for file systems deployed in SSDs are given in Table 8.7.

TABLE 8.7 Table (10) in (Fevgas et al., 2020) lists different types of flash errors and their manifestation in the file system: (a/b) Read/Write I/O Error, (c) Corruption, (d) Shorn write, (e) Lost write.

SSD Flash Errors	(a)	(b)	(c)	(d)	(e)
Uncorrectable bit corruption	x				
Silent bit corruption			x		
FTL metadata corruption	x	x	x		
Misdirected writes	x		x		
Shorn writes				x	
Dropped write	x	x	x		x
Incomplete program operation				x	x
Incomplete erase operation			x		

8.17 Redesign of relational databases by Stonebraker et al.

The last twenty-five years of commercial DBMS development can be summed up as *One Size Fits All - OSFA*, i.e., traditional DBMSs designed and optimized for business data processing have been used to support many other applications with widely varying characteristics and requirements.

Popular relational databases trace their roots to the 1970s System R at IBM (Astrahan et al., 1976). DB2 is a direct descendent of System R (Cheng et al., 1984). Oracle adopted System R's user interface. Microsoft SQL Server is a direct descendent of Sybase System 5, which borrowed heavily from System R.

<https://en.wikipedia.org/wiki/Sybase>

Current RDBMSs were architected for the business data processing market in a time of different user interfaces and different hardware characteristics and include the following System R architectural features:

1. Disk-oriented storage and indexing structures.
2. Multithreading/multiprogramming to hide latency.
3. Locking-based concurrency control mechanisms.
4. Log-based recovery.

Extensions include: compression, shared-disk architectures, bitmap indexes, user-defined data types and operators, etc.

It is argued in (Stonebraker and Çetintemel, 2005) that the OSFA concept is no longer applicable to the database market and it is predicted that the commercial world will fracture into a collection of independent database engines, e.g., for

stream-processing and data-warehouse. The end of an architectural era and the need for a complete redesign of existing database architectures is declared in (Stonebraker et al., 2007b).

Presented in (Stonebraker et al., 2007a) benchmarks that major RDBMSs could be beaten by an order of magnitude by specialized architectures in several application areas, including:

- Text (specialized engines from Google, Yahoo, etc.).
- Data Warehouses (column stores such as MonetDB) (Zukowski et al., 2005).
- Vertica cofounded by Stonebraker and Andrew Palmer in 2005 was acquired by HP in 2011 is based on columnar C-store (Stonebraker et al., 2005).

<https://en.wikipedia.org/wiki/Vertica>

Vertica features

Column-oriented storage: This increases performance of sequential record access at the expense of common transactional operations such as single record retrieval, updates, and deletes.

Massively parallel processing - MPP: This architecture distributes queries on independent nodes and scale performance linearly.

Analytics: Standard SQL interface with many analytics capabilities built-in, including time series gap filling/interpolation, event-based windowing and sessionization, pattern matching, event series joins, statistical computation such as regression analysis, and geospatial analysis.

Machine Learning - ML: In-database ML including categorization, fitting and prediction to enhance processing speed by eliminating the need for down-sampling and data movement.

Algorithms: In-database algorithms including linear regression, logistic regression, k-means clustering, Naive Bayes classification, random forest decision trees, and support vector machine regression and classification. Allows deployment of ML models to multiple clusters.

High Compression: High Compression possible because columns of homogeneous datatype are stored together and because updates to the main store are batched.

Shared-nothing architecture: This reduces system contention for shared resources and allows gradual degradation of performance in the face of hardware failure.

Novel Features: Automated workload management, data replication, server recovery, query optimization, and storage optimization.

Native Integration: Native integration with open source big data technologies like Apache Kafka and Apache Spark.

Standard Interfaces: Support for standard programming interfaces, including ODBC, JDBC, ADO.NET, and OLEDB.

High performance: High-performance and parallel data transfer to statistical tools such as built-in ML algorithms.

Vertica's aims to significantly increase query performance in data warehouses, while reducing the *Total Cost of Ownership - TCO* by reducing the hardware footprint.

In late 2011, the Vertica Analytics Platform Community Edition was made available for free with certain limitations: one terabyte of raw data, three-node servers per cluster, community-based support.

Stream processing engines such as TIBCO StreamBase and Coral8 for *Complex Event Processing - CEP* and real-time intelligence. Stream Processing engines are discussed in Chapter 9.

<https://complexevents.com/coral8-inc/>

Scientific and intelligence databases array storage engines such as MATLAB® and *Asynchronous Array of Simple Processors - ASAP*.

https://en.wikipedia.org/wiki/Asynchronous_array_of_simple_processors

The business data processing market can be beaten handily in almost any other market of significant enough size to warrant the investment in a specialized engine. Presented in (Stonebraker et al., 2007a) is evidence that current RDBMSs are not even appropriate for business data processing. A new DBMS engine for OLTP applications is presented in (Stonebraker et al., 2007b). The H-Store research group shut down in 2016, since by 2009, the VoltDB discussed in Chapter 9 developed a commercial version.

Five major issues that H-Store can leverage over current DBMS

Main Memory Size Increase: There has been a rapid increase in main memory sizes since 1970s, from MegaBytes - MB to GigaBytes - GB and even TeraBytes - TB. A shared nothing grid of twenty nodes, each with 32 GB of main memory, but 100 GB in near future, costs less than \$50,000. As such, a one TB database may be held in main memory. Most OLTP databases are less than 1 TB in size and growing in size quite slowly, e.g., TPC-C requires about 100 MB per physical distribution center (warehouse). A very large retail enterprise might have 1000 warehouses, requiring around 100 GB. Thus OLTP should be considered a main memory market, while disk-oriented solutions were used for this problem. TimesTen (Lahiri et al., 2013) and SolidDB are main memory products inheriting the baggage of System R, such as a disk-based recovery log and dynamic locking.

SolidDB is an in-memory and disk-based relational DBMS developed by UNICOM Global, which delivers extreme speed as a persistent, relational in-memory database that meets performance and reliability demands of real-time applications (Lindström et al., 2013).

<https://en.wikipedia.org/wiki/SolidDB>

Multi-threading and Resource Control: OLTP txns are very lightweight such that the heaviest TPC-C txn reads about 200 records. In a main memory environment such a txn takes less than one millisecond. Most OLTP environments do not have “user stalls”, e.g., when an Amazon user clicks “buy it”, he activates an OLTP txn which will only report back to the user when it finishes. Because of an absence of disk operations and user stalls the elapsed time of an OLTP txn is minimal. It makes sense to run each SQL command in a txn to completion with a single-threaded execution model, rather than paying for the overheads of isolation between concurrently executing statements. A single thread will yield 1 KTPS.

Current RDBMSs have elaborate multi-threading systems to try to fully utilize CPU and disk resources. This allows several-to-many queries to be running in parallel. Moreover, they also have resource governors to limit the multiprogramming load, so that other resources (IP connections, file handles, main memory for sorting, etc. are not exhausted. These features are irrelevant in a single threaded execution model. No resource governor is required in a single threaded system. In a single-threaded execution model, there is also no reason to have multi-threaded data structures. Hence the elaborate code required to support, e.g., concurrent B-trees can be completely removed. This results in a more reliable system, and one with higher performance.

But “What about long running commands?”, but in real-world OLTP systems, there aren’t any for two reasons: (1) Operations that appear to involve long-running txns, such as a user inputting data for a purchase on a web store, are usually split into several transactions to keep transaction time short. Good application design keeps OLTP queries small. (2) Longer-running ad-hoc queries are not processed by the OLTP system; instead such queries are directed to a data warehouse system, optimized for this activity. There is no reason for an OLTP system to solve a non-OLTP problem. Such thinking only applies in a “one size fits all” world.

Grid Computing and Fork-lift Upgrades Current RDBMSs were written for 1970s multiprocessor mainframes. In 1980’s shared disk architectures were spearheaded by Sun and HP, and most DBMSs were expanded to include capabilities for this architecture. DBMSs should be optimized for Grid computing or blade computing, so that can be horizontally partition data over the nodes of a grid, a tactic first investigated in Teradata, followed by Gamma (DeWitt et al., 1990) and Bubba (Boral et al., 1990).

Users do not want to perform “fork-lift” or major upgrades, hence new system should be architected for incremental expansion. If N grid nodes do not provide enough horsepower, then one should be able to add another K nodes to producing a system with $N + K$ nodes. This upgrade should be performed without taking the DBMS down. To achieve incremental upgrade without going down requires significant capabilities, not found in existing systems.

High Availability - HA Relational and other DBMSs in 1970s ran on a single machine. Companies lost money when the machine went down and that led to fault-tolerant computers by Tandem and Stratus. Disasters such as fire, earthquakes, flooding were dealt with by sending log tapes off site. Running the log tapes then brought the system back to the state when disaster happened. In the 1980s organizations made contracts with disaster recovery services, such as Comdisco, for backup machine resources, so the log tapes could be installed quickly on remote backup hardware. This strategy minimized the time that an enterprise was down as a result of a disaster.

<http://www.encyclopedia.chicagohistory.org/pages/2620.html>

Organizations run a hot standby within the enterprise, so that real-time failover can be accomplished. Alternately, some companies run multiple primary sites, so failover is even quicker. The point to be made is that businesses are much more willing to pay for multiple systems in order to avoid the crushing financial consequences of down time. HA and built-in disaster recovery are becoming essential features in the OLTP. Obvious conclusions are:

1. Every OLTP DBMS will need to keep multiple replicas consistent, requiring the ability to run seamlessly on a grid of geographically dispersed systems.
2. Most existing RDBMS vendors have glued multi-machine support onto the top of their original *Shared Multi-Processor* - *SMP* architectures. In contrast, it is clearly more efficient to start with shared-nothing support.
3. The best way to support shared nothing is to use multiple machines in a peer-to-peer configuration, so that OLTP load can be dispersed across multiple machines, and inter-machine replication can be utilized for fault tolerance. In this manner all machine resources are available during normal operation and failures only cause degraded operation with fewer resources.

Some systems implement a “hot standby” (Polyzois and Garcia-Molina, 1994), (Ji et al., 2003), whereby a second machine sits effectively idle waiting to take over if the first one fails, but normally only half of the resources available. If the load is shared among the two sites there is a performance degradation if either site fails, as in the case of mirrored disks.

These points argue for a complete redesign of RDBMS engines so they can implement peer-to-peer HA. In an HA system, regardless of whether it is hot-standby or peer-to-peer, logging can be simplified by maintaining an undo log, in case a txn fails and needs a roll back, however, the undo log does not have to persist beyond the completion of the txn. As such, it can be a main memory data structure that is discarded on txn commit. There is never a need for redo, because that will be accomplished via network recovery from a remote site. When the dead site resumes activity, it can be refreshed from the data on an operational site.

It is argued in (Lau and Madden, 2006) that failover/rebuild is as efficient as redo log processing. Recovery logic is simplified, since there is no need for a persistent redo log, just a transient undo one. There is no need for ARIES-style logging (Mohan et al., 1992) and with this functionality failed sites can be brought up to date from operational sites when they resume operation. Again, a large amount of complex code has been made obsolete, and a different capability is required.

Emerging *Non-Volatile Memory* - *NVM* technologies are slightly slower than DRAM, but all writes are persistent, even after power loss. Explored in (DeBrabant et al., 2014) are two possible use cases of NVM for OLTP DBMSs. The first is where NVM completely replaces DRAM and the other is where NVM and DRAM coexist in the system. For each case the performance of a disk-oriented DBMS with a memory-oriented DBMS is compared using two OLTP benchmarks. The performance of different recovery algorithms on these NVM devices and the evaluation shows that in both storage hierarchies, memory-oriented systems outperform their disk-oriented counterparts. As skew decreases the performance of the two architectures converge, showing that neither architecture is ideally suited for an NVM-based storage hierarchy.

8.18 Parallel Data Base Machines - DBMs

Parallel DBMs are listed in approximate chronological order.

8.18.1 Almaden Research Backend Relational Engine - ARBRE

The *Almaden Research Backend Relational Engine* - *ARBRE* was a project led by Raymond Lorie at IBM/ARC. to exploit database parallelism by extending IBM’s System R relational database prototype (Astrahan et al., 1976) to a message-passing multicomputer system (Lorie et al., 1989), (Lorie et al., 1991).

It is argued that parallel processing may be the only means of satisfying the long-term performance requirements to increase txn throughput and decrease query response times. Parallelism is attained by partitioning the database among processors. Solutions to some challenging problems such as parallel sorting and relational joins are presented.

Txn commit in the distributed system was accomplished by the coordinator log txn execution protocol proposed by James Stamos and Flaviu Cristian (UCSD, rip) (Stamos and Cristian, 1993). The protocol eliminates two rounds of messages from the presumed commit protocol and reduces the number of log forces needed for distributed atomic commit. The new protocol is compared to existing protocols, it is explained when it is desirable and how it affects the *Write Ahead Log* - *WAL* protocol and the database crash recovery algorithm (Gray and Reuter, 1992).

8.18.2 IBM's DB2 Parallel Edition

The DB2 *Parallel Edition - PE* project was started at IBM Research at Hawthorne, but completed in collaboration with IBM's Austin Research and the Toronto Labs (Baru et al., 1995). The effort involved researchers associated with U. Wisconsin's Gamma and MCC's Bubba projects. Non-blocking 3-phase commit (Skeen, 1981) is mentioned, but not followed because of its high overhead and unsuitability for reliable environments (Skeen, 1981).

https://en.wikipedia.org/wiki/Three-phase_commit_protocol

Coordinator log txn execution protocol utilized in Arbre would have been more appropriate in this environment. DB2 PE was based on DB2, while ARBRE described in (Lorie et al., 1991) was based on System R.

DB2 PE was implemented on IBM's shared nothing *Scalable Parallel SP2* computer (Agerwala et al., 1995), which was discontinued in 2003.

https://en.wikipedia.org/wiki/IBM_Scalable_POWERparallel

Decision support and data mining were potential DB2 PE applications. In developing DB2 PE it was argued that shared everything systems lack scalability, since state of the art memory systems of the time could support only ten 500 MIPS RISC processors. Similarly, datasharing systems have limitations from the viewpoint of cache coherence, although extended storage allows scalable systems up to 32. Function shipping described earlier has the advantage of filtering data and only sending qualifying tuples for further processing. DB2 PE has two major components *DB2 Data Management - DM* and *Data Communication - DC*. The latter was the main part requiring modification.

DB2's *Data Definition Language - DDL* had to be extended to specify data placement, but no changes were made to the *Data Manipulation Language - DML*. The data in a table may be fully or partially declustered. The data in two joinable tables may be non-overlapped or fully overlapped. Three techniques to assign records in a table are (1) round-robin, (2) hash, and (3) range (Ramakrishnan and Gehrke, 2002). Broadcast and direct data communication was supported. Optimized query execution is illustrated by several examples in the paper. Attributes associated with files can be broadcast versus directed or read-ahead versus stepping. The query optimizer takes into account the cost of data communication in addition to CPU and disk I/O processing cost. Table queues are provided as a means of communication between senders and receivers.

8.19 Google File System, Bigtable, and Spanner

The *Google File System - GFS* is built to run on top of inexpensive commodity components that often fail and uses replication to tolerate failures rather than utilize distributed RAID as in Swift/RAID (Cabrera and Long, 1991). GFS constantly monitor itself and detect, tolerate, and recover promptly from component failures.

The system stores a modest number of large files. A few million files, each typically 100 MB or larger are expected. Multi-GB files are the common case and should be managed efficiently. GFS is not optimized for small files, although they are supported.

The workloads primarily consist of two kinds of reads: large streaming reads and small random reads. Large streaming reads typically read hundreds of KBs. Successive operations from the same client often read through a contiguous region of a file. A small random read typically reads a few KBs. Performance-conscious applications often batch and sort their small reads to advance steadily through the file rather than go back and forth.

The workloads also have many large, sequential writes that append data to files. Typical operation sizes are similar to those for reads. Once written, files are seldom modified again. Small writes at arbitrary positions in a file are supported but do not have to be efficient.

The system must efficiently implement well-defined semantics for multiple clients that concurrently append to the same file. Files are often used as producer-consumer queues or for many-way merging. Hundreds of producers, one per machine, will concurrently append to a file. Atomicity with minimal synchronization overhead is essential. The file may be read later, or a consumer may be reading through the file simultaneously.

High sustained bandwidth is more important than low latency. Most target applications place a premium on processing data in bulk at a high rate, while few have stringent response time requirements for an individual read or write.

Brief description of bigtable

Bigtable is a typical wide column store, other examples of which are discussed in Chapter 9. Two arbitrary string values (row key and column key) and timestamp (hence three-dimensional mapping) are mapped into an associated

arbitrary byte array. It is not a relational database and can be better defined as a sparse, distributed multi-dimensional sorted map.

Bigtable is designed to scale into the petabyte range across hundreds or thousands of machines, and to make it easy to add more machines to the system and automatically start taking advantage of those resources without any reconfiguration.

Google's copy of the web can be stored in a bigtable where the row key is a domain-reversed *Universal Resource Locator - URL*, and columns describe various properties of a web page, with one particular column holding the page itself. The page column can have several timestamped versions describing different copies of the web page timestamped by when they were fetched. Each cell of a bigtable can have zero or more timestamped versions of the data. Another function of the timestamp is to allow for both versioning and garbage collection of expired data.

Tables are split into multiple tablets – segments of the table are split at certain row keys to limit their size. When table size grows beyond a specified limit, the tablets may be compressed using the algorithm BMDiff and the Zippy/Snappy compression algorithm, which is a less space-optimal variation of LZ77 but more efficient in terms of computing time. [https://en.wikipedia.org/wiki/Snappy_\(compression\)](https://en.wikipedia.org/wiki/Snappy_(compression))

The Spanner is layered on an implementation of Google's BigTable (Corbett et al., 2013), which is a compressed, high performance, proprietary data storage system built on *Google File System - GFS*, Chubby Lock Service, SSTable (log-structured storage like LevelDB), etc. (Chang et al., 2008). A public version of Bigtable was made available as a service. Bigtable also underlies Google Cloud Datastore available as a part of the Google Cloud Platform.

<https://en.wikipedia.org/wiki/Bigtable>

Spanner extends BigTable with Paxos protocols to solve consensus problems, which is the process of agreeing on one result among a group of participants in networks of unreliable processors which becomes difficult when the participants or the network experiences failures (Gray and Lamport, 2006).

[https://en.wikipedia.org/wiki/Paxos_\(computer_science\)](https://en.wikipedia.org/wiki/Paxos_(computer_science))

Cloud Spanner is compared with relational and non-relational databases in Table 8.8 and its pricing is given in Table 8.9.

TABLE 8.8 Cloud spanner versus relational and nonrelational databases.

	Spanner	Relational	Non-Relational
Schema	Yes	Yes	No
SQL	Yes	Yes	No
Consistency	Strong	Strong	Eventual
Availability	High	Failover	High
Scalability	Horizontal	Vertical	Horizontal
Replication	Automatic	Configurable	Configurable

TABLE 8.9 Cloud spanner pricing: Network ingress is FREE, cross-region and internet egress rates apply.

Feature	Regional Price	Multi-Region Price
Nodes	\$0.90 per node per hr	\$3.00 per node per hr
Storage	\$0.30 per GB per month	\$0.50 per GB per month

Google F1 was developed to support the Adwords business using Spanner, which replaced a MySQL implementation. F1 is a hybrid database that combines high availability, the scalability of NoSQL systems like BigTable, and the consistency and usability of traditional SQL databases. F1 provides synchronous cross-datacenter replication and strong consistency.

Synchronous replication results in a higher commit latency, which is mitigated by using a hierarchical schema model with structured data types and via smart application design. F1 also includes a fully functional distributed SQL query engine and automatic change tracking and publishing.

8.20 Microsoft Azure

Microsoft Azure was released in February 2010 is a cloud computing service with many functionalities: building, testing, deploying, and managing applications and services. It provides *Software as a Service - SaaS*, *Platform as a Service - PaaS*, and *Infrastructure as a Service - IaaS*. It supports many different programming languages, tools and frameworks, both from Microsoft and third-parties, listed below.

https://en.wikipedia.org/wiki/Microsoft_Azure

(1) Compute, (2) Mobile services, (3) Storage services, (4) Data management, (5) Messaging, (6) Media services, (7) *Content Delivery Network - CDN*,

https://en.wikipedia.org/wiki/Content_delivery_network

(8) Developer, (9) Management, (10) *Machine Learning - ML*,

https://en.wikipedia.org/wiki/Machine_learning

(11) Functions, (12) *Internet of Things - IOT*.

https://en.wikipedia.org/wiki/Internet_of_things

According to (Chen et al., 2017) Microsoft Azure Storage is a global cloud storage in 38 geographic regions. To protect customer data against catastrophic data center failures, it optionally replicates data to secondary *Data Centers - DCs* hundreds of miles away. Using Microsoft OneDrive as an example, this paper illustrates the characteristics of typical cloud storage workloads and the opportunity to lower storage cost for georedundancy with erasure coding. Presented is the design, implementation and evaluation of Giza – a strongly consistent, versioned object store that applies erasure coding across global data centers. The key technical challenge is achieving single cross-Data Center round trip latency for the common contention-free workload, while also maintaining strong consistency when there are conflicting access. Giza addresses the challenge with a novel implementation of well-known distributed consensus algorithms tailored for restricted cloud storage APIs. Giza is deployed to 11 *Data Centers - DCs* across 3 continents and experimental results demonstrate that it achieves our design goals.

Azure uses *Local Reconstruction Codes - LRC* coding that reduces the bandwidth and I/Os required for repair reads over prior codes, while still allowing a significant reduction in storage overhead (Huang et al., 2012). Giza's configuration and design tradeoffs are given in Table 8.10 and Table 8.11.

TABLE 8.10 Giza configuration. US(7/3) represents 7 DCs for data and 3 DCS for metadata.

	Coding	Data & Metadata DCs	ping (max)
US-2-1	2 + 1	US(3/3)	46 ms
US-6-1	6 + 1U	S(7/3)	71 ms
World-2-1	2 + 1	US(1/1), EU(1/1), JP(1/1)	240 ms
World-6-1	6 + 1	US(3/1), EU(2/1), JP(2/1)	241 ms

TABLE 8.11 Giza trade-offs.

	Geo-Rep.	Giza		
# of DCs	2	3	5	7
Erasur coding	-	2 + 1	4 + 1	6 + 1
Storage overhead	2.6	1.9	1.6	1.5
Cost savings	-	27%	38%	42%
Cross-DC traffic (put)	1x	1x	1x	1x
Cross-DC traffic (get)	0	0.5x	0.75x	0.83
DC rebuild	1x	2x	3x	6x

To summarize:

- Presented is the design and implementations of Giza, a strongly consistent, versioned object store that erasure codes objects across globally distributed data centers.
- Giza achieves minimum latency in the common case when there are no concurrent conflicting access, and ensures strong consistency in rare cases when there is contention.

- Giza applies well-known distributed protocols: Paxos (Lamport, 2002) and Fast Paxos (Lamport, 2005) in a novel way on top of restricted cloud storage APIs.
- Giza is deployed in 11 DCs across 3 continents and experimental results demonstrate that it achieves our design goals.

8.20.1 Amazon DynamoDB - ADDB

Amazon's DynamoDB - ADDB was announced by its CTO Werner Vogels in January 2012. It is a fast and scalable proprietary NoSQL database service, where NoSQL stands for non-relational, but also Not only SQL.

<https://en.wikipedia.org/wiki/NoSQL>

ADDB supports key-value and document data structures as part of *Amazon Web Services* - . Unlike relational DBMSs most NoSQL DBMSs do not provide ACID guarantees, but rather eventual consistency. Amazon's Dynamo has a multi-master design requiring the client to resolve version conflicts. ADDB uses synchronous replication across multiple data-centers for high durability and availability.

According to Vogels ADDB has the following features.

<https://www.allthingsdistributed.com/2012/01/amazon-dynamodb.html>

Manageability: This is a critical requirement for alleviating the need for provisioning hardware and software, setting up and configuring a distributed database cluster, and managing its operations. The complexity of scaling is resolved by repartitioning data over more machine resources to meet I/O performance requirements. ADDB automatically replicates data across multiple Availability Zones to meet stringent availability and durability requirements.

Scalability: ADDB scales the resources dedicated to a table to hundreds or thousands of servers spread over multiple Availability Zones to meet storage and throughput requirements. There are no pre-defined limits to the amount of data each table can store. Developers can store and retrieve any amount of data and DynamoDB will spread the data across more servers as the amount of data grows.

Fast: By using flash SSDs ADDB achieves high throughput at very low latency. By not indexing all attributes the cost of read and write operations is low, as write operations involve updating only the primary key index, thereby reducing the latency of both read and write operations.

Durable and Highly Available: ADDB replicates its data over at least three different data centers, so that the system can continue to operate and serve data even under complex failure scenarios.

Flexible: ADDB does not force its users into a particular data or consistency model, since ADDB tables do not have a fixed schema, but instead allow each data item to have any number of attributes. Developers can use stronger consistency models when accessing the database and take advantage of the atomic increment/decrement functionality of DynamoDB for counters.

Low Cost: ADDB's pricing is simple and predictable:

(1) storage is \$1 per GB per month.

(2) requests are priced based on how much capacity is reserved:

\$0.01 per hour for every 10/50 units of Write/Read Capacity.

A unit of Read (or Write) Capacity equals one read (or write) per second of capacity for items up to 1 KB in size. With eventually consistent reads achieve twice as many reads per second for a given amount of Read Capacity.

8.20.2 Aurora database for processing continual inputs from sensors

Amazon Aurora is a relational database service developed and offered by *Amazon Web Services - AWS* beginning Oct. 2014. Aurora is available as part of the Amazon.

https://en.wikipedia.org/wiki/Amazon_Aurora

The basic processing model and architecture of Aurora is a system to manage data streams for monitoring applications, which differ from conventional business data processing, that the software system must process and react to continual inputs from sensors rather than from humans and this requires rethinking the fundamental architecture of a DBMS for this application area Abadi et al. (2003).

Traditional DBMSs have been oriented toward business data processing, and consequently are designed to address the needs of these applications (Carney et al., 2002).

1. It is assumed that the DBMS is a passive repository and that humans initiate queries and transactions on this repository, which is called a *Human-Active, DBMS-Passive - HADP* model.
2. Current values of data elements are easy to obtain in traditional databases, while previous values can only be found by referring to the DBMS log. Monitoring applications require data management that extends over some history of values reported in a stream. Consider a monitoring application that tracks the location of items of interest. Ceiling-mounted sensors inside a building and the GPS system generate large volumes of location data.

3. Most monitoring applications are trigger-oriented. If one is monitoring a chemical plant, then one wants to alert an operator if a sensor value gets too high or if another sensor value has recorded a value out of range more than twice in the last 24 hours. Every application could potentially monitor multiple streams of data, requesting alerts if complicated conditions are met. Thus, the scale of trigger processing required in this environment far exceeds that found in traditional DBMS applications. Triggers and alerters have been added as an after-thought and are hence second-class citizens. These implementations do not scale up to a large number of triggers (Widom and Ceri, 1996).
4. DBMSs assume that data elements are synchronized and that queries have exact answers. In many stream-oriented applications, data arrives asynchronously and answers must be computed with incomplete information. Stream data is often lost, stale, or intentionally omitted for processing reasons. An object being monitored may move out of range of a sensor system, thereby resulting in lost data. The most recent report on the location of the object becomes more and more inaccurate over time. Moreover, in managing data streams with high input rates, it might be necessary to shed load by dropping less recent input data. By necessity this leads to approximate answers.
5. Monitoring applications have real-time requirements. Applications that monitor mobile sensors often have a low tolerance for stale data, making these applications effectively real time. The added stress on a DBMS that must serve real-time applications makes it imperative that the DBMS employ intelligent scheduling) and graceful degradation strategies, i.e., load shedding) during periods of high load. QoS specifications are required for running such systems.

8.21 IBM and other cloud service providers

IBM cloud services was based on the acquisition of Softlayer in June 2013, although prior to that IBM had announced public and hybrid cloud services in 2011.

https://en.wikipedia.org/wiki/IBM_cloud_computing

In July 2019 IBM acquired Red-Hat Linux to enhance its hybrid cloud service for \$34 Billion.

Former Red-Hat CEO is current IBM president, but is to become the next CEO after current CEO steps down.

<https://seekingalpha.com/article/4333975-ibms-main-asset-is-can-sell-red-hats-offerings>

Jim Whitehurst stepped down on 7/2/21 less than two years after he joined through Red Hat acquisition.

<https://www.cnbc.com/2021/07/02/ibm-president-jim-whitehurst-steps-down-after-red-hat-deal.html>

IBM launched on 6/28/21 AI-powered new Cloud Pak for Network Automation software to help adoption of 5G.

<https://seekingalpha.com/news/3710354-ibm-launches-ai-powered-new-cloud-pak-for-network-automationn-software-to-help-adoption-of-5g>

Top ten service providers in 2021, led by Amazon AWS, Microsoft Azure, and IBM are given here.

<https://www.c-sharpcorner.com/article/top-10-cloud-service-providers/>

A list of companies is provided in the following web site.

<https://www.zdnet.com/article/the-top-cloud-providers-of-2021-aws-microsoft-azure-google-cloud-hybrid-saas/>

The Cloud Wars ranking of the 10 most-influential cloud vendors in the world based on *Point-OF-View - POV* business customers.

<https://cloudwars.co/cloud-wars-top-10-vendors-world/>

1. **Microsoft Azure:** Cloud Revenue for Q ending 3/31: \$17.7B, Cloud Services: IaaS, PaaS, SaaS, CEO: Satya Nadella.
2. **Amazon AWS:** Cloud Revenue for Q ending 3/31: \$13.5B, Cloud Services: IaaS, PaaS, CEO: Andy Jassy.
3. **Google cloud:** Cloud Revenue for Q ending 3/31: \$4B, Cloud Services: IaaS, PaaS, SaaS, CEO: Thomas Kurian.
4. **Salesforce:** Q4 Cloud Revenue: \$5.82B, Cloud Services: SaaS, PaaS CEO: Marc Benioff.
5. **SAP:** Cloud Revenue for Q ending 3/31: \$2.57B, Cloud Services: SaaS, PaaS, CEO: Christian Klein.
6. **Oracle:** Fiscal Q3 Cloud Revenue (est.): \$1.6B, Cloud Services: SaaS, PaaS, IaaS, CEO: Safra Catz.
7. **ServiceNow** Cloud Revenue for Q ending 3/31: \$1.29B, Cloud Services: SaaS, CEO: Bill McDermott.
8. **Workday:** Fiscal Q4 Cloud Revenue: \$1.01B, Cloud Services: SaaS, PaaS, CEO: Aneel Bhusri.
9. **IBM** Cloud Revenue for Q ending 3/31: \$6.5B, Cloud Services: SaaS, PaaS, IaaS, CEO: Arvind Krishna.
10. **Snowflake:** Fiscal Q4 Cloud Revenue: \$178M, Cloud Services: IaaS, PaaS.

The following is a prediction of increase in revenue in following years.

<https://www.gartner.com/en/newsroom/press-releases/2020-07-23-gartner-forecasts-worldwide-public-cloud-revenue-to-grow-6point3-percent-in-2020>

The worldwide public cloud services market is forecast to grow 6.3% in 2020 to total \$257.9 billion, up from \$242.7 billion in 2019, according to Gartner, Inc. Reported is growth in different categories: *BPaaS: Business Process as a Service, PaaS: Platform as a Service, SaaS: Software as a Service, SMSS: Cloud Management and Security a Service, IaaS: Infrastructure as a Service.*

A recent ranking of IaaS by revenue is as follows:

<https://www.crn.com/news/cloud/aws-microsoft-alibaba-led-iaas-cloud-services-in-2020-gartner>

While AWS' IaaS cloud services revenue grew 28.7% to \$26.2B as the overall size of the market increased, its percentage share of the IaaS market fell to 40.8 percent from 44.6% in 2019, according to Gartner.

Microsoft held onto the No. 2 spot with IaaS revenue that climbed 59.12 percent to \$12.7B, accounting for 19.7% of the market, up from 17.4% in 2019. The global coronavirus pandemic and move to remote work increased demand from existing Microsoft Azure cloud customers to move their mission-critical workloads to the cloud.

Third-place Alibaba, the largest IaaS cloud services provider in Asia, saw its IaaS revenue jump 52.8% to \$6.1B billion. The overall market share moved to 9.5% from 8.8%. Its highest growth rate - 105% - came in the education vertical, its enterprise communication and collaboration platform, by employees and students working and studying from home.

Google Cloud finished fourth. Its IaaS revenue climbed 66.1% to \$3.9B, n, while its market share moved to 6.1% from 5.2% in 2019. Google Cloud's focus on industry verticals is paying off, with retail, government and healthcare sector spending helping drive its growth in IaaS, along with its focus on a hybrid and multi-cloud strategy.

China's Huawei was fifth with a 202.8% increase in IaaS revenue to \$2.7B, as its market share surged to 4.2% from 1.9% in 2019 with 200% IaaS market growth. More than 90% of Huawei's revenue comes from Greater China, as the region continues to see rapid cloud market growth, according to Gartner.

For the second quarter of 2021 the following cloud earnings were reported:

Microsoft \$19.5 billion, up 36%

AWS \$14.81 billion, up 37%

Google Cloud \$4.63 billion, up 54%.

<https://cloudwars.co/microsoft/microsoft-q2-cloud-revenue-still-bigger-than-amazon-google-cloud/>

8.22 Distributed databases in cloud computing

DBMS for update intensive application workloads as well as decision support systems for data analytics play an important role in ensuring the smooth transition of applications from the traditional enterprise to cloud infrastructures. Scalable data management has been a goal in traditional enterprise setting. Cloud computing brings its own set of novel challenges. The discussions that follow are relevant to developing and deploying internet scale applications. Considered is support for update heavy applications, and ad-hoc analytics. Discussed are design choices made by some large scale databases.

PNUTS is a massively parallel and geographically distributed database for Yahoo!'s web applications (Cooper et al., 2008). Storage is organized as hashed or ordered tables providing low latency for large numbers of concurrent requests including updates and queries, and novel per-record consistency guarantees. It is a hosted, centrally managed, and geographically distributed service and utilizes automated load-balancing and failover.

PNUTS design was influenced by Google's BigTable (Chang et al., 2008) which introduced and popularized concepts such as very large-scale structured data storage, i.e., the "NoSQL" movement, see e.g., Mike Lukides, blog in 2012.

<http://radar.oreilly.com/2012/02/nosql-non-relational-database.html>

Many of the architectural principles of BigTable, such as the idea of tablets of data that can be moved between servers for load balancing were adopted in PNUTS. The stronger notion of timeline consistency, rather than Amazon's eventual consistency was adopted in PNUTS (DeCandia et al., 2007), although eventual consistency is also supported.

Distributed Hash Table - DHT, like Chord (Stoica et al., 2001) provides an alternative architecture for scalable distributed databases. Unlike PNUTS, which allows arbitrary assignment of tablets to servers, Chord uses a consistent hashing scheme that deterministically map data to servers. In the design phase of PNUTS a DHT-like model was considered, but was not adopted due to the extra flexibility provided by arbitrary tablet assignment for load balancing and latency, since it avoids searching the Chord ring for data.

Sherpa: Lessons from Yahoo!'s Cloud Database PNUTS are described in (Cooper et al., 2019). PNUTS uses *Optimistic Concurrency Control - OCC* protocol for *Transaction Management - TM*. In PNUTS we have test-and-set write instead of read-modify-write, where a call performs the requested write to the record if and only if the present version of the record is the same. This call can be used to implement txns that first read a record, and then do a write to the record based on the read. The test-and-set write ensures that two such concurrent increment transactions are properly serialized.

The API of PNUTS with comparison to that of SQL reveals too many implementation details such as sequence numbers, which allow the application to indicate cases where it can do with some relaxed consistency for higher performance. Similarly, a test-and-set write allows the implementation of single-row txns without any locks, a highly desirable property in distributed systems. According to the need, the API can be packaged into the traditional begin txn and commit for single-row transactions at the cost of losing expressiveness. PNUTS model can provide serializability on a per-record basis, but

makes no guarantees for multi-record txns. If an application reads or writes the same record multiple times in the same txn, the application must use record versions to validate its own reads and writes to ensure txn serializability.

ElasTraS: is an Elastic transactional data store in the cloud, which has emerged as a successful paradigm for internet scale computing (Das et al., 2010). A contributing factor is the elasticity of resources. In spite of the elasticity provided by the infrastructure and the scalable design of the applications, the underlying database is not very elastic and scalable, and hence limits scalability. ElasTraS addresses the issue of scalability and elasticity of the data store in a cloud computing environment to leverage from the elastic nature of the underlying infrastructure, while providing scalable transactional data access. Described is the design of a system in progress, highlights the major design choices, analyzes the different guarantees provided by the system, and identifies several important challenges.

ecStore is an elastic transactional cloud storage (Vo et al., 2010), which offers PaaS - Platforms as a Service, where storage and compute power are adjusted dynamically and therefore it is important to build a highly scalable and reliable storage that can elastically scale on-demand with minimal startup cost. ecStore supports automated data partitioning and replication, load balancing, efficient range query, and transactional access. Txns can span multiple nodes using multi-version *Optimistic Concurrency Control - OCC*.

G-Store is a scalable data store for transactional multi key access in the cloud, which has emerged as a preferred platform for deploying scalable web-applications (Das et al., 2010). With the growing scale of these applications and the data associated with them, scalable data management systems form a crucial part of the cloud infrastructure. *Key-Value - KV* stores such as BigTable, PNUTS, Dynamo, and their open source analogues, have been the preferred data stores for applications in the cloud. In these systems, data is represented as KV pairs and atomic access is provided only at the granularity of single keys. While these properties work well for current applications, they are insufficient for the next generation web applications. Key Grouping protocol based on a Key-Group abstraction provides efficient, scalable, and transactional multi key access. An implementation using a standard key-value store and experiments using a cluster of commodity machines show that G-Store preserves the desired properties of key-value stores, while providing multi key access functionality at a low overhead.

Adapting Microsoft SQL Server for Cloud Computing Cloud SQL Server is a RDBMS based on Microsoft SQL server designed to scale-out to cloud computing workloads (Bernstein et al., 2011). Scale out uses a partitioned database on a shared-nothing system architecture. Txns are constrained to execute on one partition, to avoid the need for *two-phase commit - 2PC*. The database is replicated for high availability using a custom primary-copy replication scheme (Ozsu and Valduriez, 2020). It currently serves as the storage engine for Microsoft's Exchange Hosted Archive and SQL Azure. Load balancing on primary and two secondary replicas is shown in Table 8.12.

TABLE 8.12 Load balancing primary and secondary replicas.

PA_p	PB_p	PC_p	PD_p
PB_{s1}	PA_{s1}	PB_{s2}	PA_{s2}
PC_{s1}	PD_{s1}	PD_{s2}	PC_{s2}

Cloud SQL Server is currently used as the database system for *Exchange Hosted Archive - EHA* and SQL Azure. EHA is used for archiving messages and ensuring compliance, e.g., by implementing retention policies.

Megastore developed at Google is a storage system developed to meet the requirements of today's interactive online services (Baker et al., 2011). Megastore blends the scalability of a NoSQL with the conventional RDBMSs providing both strong consistency guarantees and high availability. It provides fully serializable ACID semantics within fine-grained partitions of data, which allows synchronous replication of each write across a *Wide Area Network - WAN* with reasonable latency and supports seamless failover between datacenters. Megastore's semantics and replication algorithm and experiences supporting a wide range of applications at Google are described in the paper.

CloudTPS provides scalable txns for web applications in the NoSQL cloud data stores provide scalability and high availability properties for web applications, but sacrifice data consistency, which cannot be afforded by many applications. CloudTPS is a scalable Txn Manager, which guarantees full ACID properties for multi-item txns issued by web applications, even in the presence of server failures and network partitions. It is implemented on top of the two main families of scalable data layers: BigTable and SimpleDB. Performance evaluation on top of HBase (open-source BigTable) in our local cluster and Amazon SimpleDB in the Amazon cloud shows that our system scales linearly at least up to 40 and 80 nodes in the two cases. Globule is the research group lead by Guillaume Pierre at IRISA's Myriads team. <http://www.irisa.fr/en>
http://www.globule.org/?page_id=25

Windows Azure Storage - WAS is a highly available cloud storage service with strong consistency (Calder et al., 2011). WAS provides customers the ability to store seemingly limitless amounts of data for any duration of time. Data from anywhere at any time and only pay for what they use and store. Data is stored durably using both local and geographic replication to facilitate disaster recovery. Currently, WAS storage comes in the form of Blobs or files, Tables (structured storage), and Queues (message delivery). In this paper, we describe the WAS architecture, global namespace, and data model, as well as its resource provisioning, load balancing, and replication systems.

Strong Consistency – Many especially enterprise customers want strong consistency with the ability to perform conditional reads, writes, and deletes for optimistic concurrency control (Kung and Robinson, 1981) on the strongly consistent data. There are now many locking-based storage services on Azure, such as Azure SQL and PostgreSQL (private communication Phil Bernstein). For this, WAS provides three properties that the CAP theorem (Brewer, 2010) claims are difficult to achieve at the same time: strong consistency, high availability, and partition tolerance.

In (Mohan, 1992) lists the shortcomings of the *Optimistic Concurrency Control - OCC* approach in supporting all the features expected in a full-function DBMS. Surprisingly, in spite of OCC having been around for a long time and its performance having been studied in various contexts, no complete system design, let alone a full-blown implementation, exists. The problems with OCC relate to support for access paths (indexes, hash-based storage), partial rollbacks, nested transactions, fine-granularity (e.g., record-level) conflict checking, different isolation levels, distributed txns, and so on. This paper increases awareness of the implementation aspects of OCC.

8.23 SpringFS bridging agility and performance in elastic distributed storage

SpringFS is an elastic storage system that can quickly change its number of active servers, while retaining elasticity and performance goals was led by a team at CMU/PDL (Xu et al., 2014a,b). SpringFS uses a novel technique, termed bounded write offloading, that restricts the set of servers where writes to overloaded servers are redirected. This technique, combined with the read offloading and passive migration policies used in SpringFS, minimizes the work needed before deactivation or activation of servers. Analysis of real-world traces from Hadoop deployments at Facebook and Cloudera customers and experiments with the SpringFS prototype confirm SpringFS's agility, show that it reduces the amount of data migrated for elastic resizing by up to two orders of magnitude, and show that it cuts the percentage of active servers required by 67–82%, outdoing state-of-the-art designs by 6–120%. A comparison of cloud scalability and elasticity is provided here <https://theappsolutions.com/blog/cloud/cloud-computing-elasticity-vs-scalability/>

1. ability to resize dynamically as workload varies,
2. More difficult than elastic computing,
3. Benefits,
4. Re-use for other purposes or reduce energy usage,
5. Save machine hours (operating cost),
6. Most distributed storage is not elastic,
7. Designed for load balancing, not elasticity,
8. Examples, GFS and HDFS,
9. Deactivating servers may make data unavailable.

8.24 Snowflake cloud based data warehousing with SQL support

Snowflake is a cloud-based data warehousing system with SQL support, which is motivated by three goals:

1. compute and storage elasticity;
2. support for multitenancy;
3. high performance.

Presented in (Vuppapapati et al., 2020) is the design and implementation of Snowflake, along with a discussion on how recent changes in cloud infrastructure such as emerging hardware, fine-grained billing, etc. Snowflake query planning, optimization, concurrency control mechanisms are discussed in (Dageville et al., 2016).

Snowflake customers are given access to computational resources through *Virtual Warehouse - VWs*, where each VW is essentially a set of AWS EC2 instances on top which customer queries execute in a distributed fashion.

<https://aws.amazon.com/ec2/instance-types/>

Customers pay for compute-time based on the VW size and each VW can be elastically scaled on an on-demand basis upon customer request. To support elasticity at fine-grained timescales, Snowflake maintains a pool of pre-warmed EC2 instances; upon receiving a request, Snowflake simply adds/removes EC2 instances to/from that VW pre-warmed instances are used to avoid startup time. Each VW may run multiple concurrent queries and a customer may run multiple VWs.

Shared-nothing architectures have been the foundation of traditional query execution engines and data warehouses. In such architectures, persistent data is partitioned across a set of compute nodes, each of which is responsible only for its local data. Such shared-nothing architectures have enabled query execution engines that scale well, provide cross-job isolation and good data locality resulting in high performance for a variety of workloads. Major disadvantages are:

Hardware-workload mismatch: Shared-nothing architectures make it hard to balance CPU, memory, storage and bandwidth resources provided by compute nodes, and the workloads. For instance, a node configuration that is ideal for bandwidth-intensive compute-light bulk loading may be a poor fit for compute-extensive bandwidth-light complex queries. Many customers, however, want to run a mix of queries without setting up a separate cluster for each query type. Thus, to meet performance goals, resources usually have to be over-provisioned; this results in resource under-utilization on an average and in higher operational costs.

Lack of Elasticity: Even if one could match the hardware resources at compute nodes with workload demands, static parallelism and data partitioning inherent to (inelastic) shared-nothing architectures constrain adaptation to data skew and time-varying workloads. Queries run by our customers have extremely skewed intermediate data sizes that vary over five orders of magnitude and have CPU requirements that change by as much as an order of magnitude within the same hour. Moreover, shared-nothing architectures do not admit efficient elasticity. Adding/removing nodes to elastically scale resources requires large amounts of data to be reshuffled. This increases network bandwidth requirements, but also results in significant performance degradation, since the set of nodes participating in data reshuffling are also responsible for query processing.

Traditional data warehousing systems were designed to operate on queries with predictable volume and rate, e.g., data coming from within the organization: transactional systems, ERP - enterprise resource planning and *Customer Relationship Management* applications, etc. The situation has changed significantly.

An increasingly large fraction of data currently comes from less controllable, external sources, e.g., application logs, social media, web applications, mobile systems, etc., resulting in ad-hoc, time-varying, and unpredictable query workloads. Shared-nothing architectures are inflexible, incur high cost, provide poor performance, and are inefficient. Snowflake was designed to overcome these limitations by being an elastic, transactional query execution engine with SQL support comparable to state-of-the-art databases.

The key insight in Snowflake design is that the limitations of shared-nothing architectures are due to the tight coupling of compute and storage. Snowflake's solution is to decouple the two by disaggregating compute from persistent storage, which holds customer data similarly to Amazon S3 and Azure Blob Storage. Compute elasticity is achieved using a pool of pre-warmed nodes, that can be assigned to customers on an on-demand basis. The two key Snowflake ideas are:

Custom-designed storage system for management and exchange of ephemeral/intermediate data that is exchanged between compute nodes during query execution. Ephemeral storage system was necessary because of the limitations of persistent data stores providing:

- (1) necessary latency and throughput performance for exchanging of intermediate data;
- (2) stronger availability and durability than what is needed for intermediate data.

Write-through cache for persistent data rather than just ephemeral storage for intermediate data.

Combined with a custom-designed query scheduling mechanism for disaggregated storage, Snowflake reduces the additional network load caused by compute-storage disaggregation as well as alleviates the performance overheads of reduced data locality.

The key findings of the paper are as follows:

- (1) Query types are: read-only queries (28%), write-only queries (13%) and read-write queries (59%).
- (2) Intermediate data sizes vary over multiple orders of magnitude across queries, with some queries exchanging hundreds of GBs or even TBs of intermediate data. The amount of such data has little or no correlation with the amount of persistent data read by the query or the execution time of the query.
- (3) Even with a small amount of local storage capacity, skewed access distributions and temporal access patterns common in data warehouses enable reasonably high average cache hit rates (60-80%) for persistent data accesses.
- (4) Some customers exploit support for elasticity (for 20% of the clusters) and the number of compute nodes in their cluster can change by two orders of magnitude during the lifetime of the cluster.

(5) While the peak resource utilization can be high, the average resource utilization is usually low. We observe average CPU (51%), Memory (19%), Network Tx-transmit (11%) and Network Rx-receive (32%).

Topics for future research are as follows:

Decoupling of compute and ephemeral storage: Currently, compute and ephemeral storage are still tightly coupled and the ratio of compute capacity and ephemeral storage capacity varies by several orders of magnitude, leading to either underutilization of CPU or thrashing of ephemeral storage for ad-hoc query processing workloads. More work is needed in ephemeral storage system design, especially in terms of providing fine-grained elasticity, multi-tenancy, and cross-query isolation. Multi-tenancy is supported through the *Virtual Warehouse* - VW abstraction, where each VW operates on an isolated set of nodes, with its own ephemeral storage system.

Deep storage hierarchy: Snowflake ephemeral storage system caches frequently read persistent data to both reduce the network traffic and to improve data locality. However, existing mechanisms for improving caching and data locality were designed for two-tier storage systems: memory and HDD/SSD. New mechanisms are needed to efficiently exploit the emerging deep storage hierarchy.

Pricing at sub-second timescales: Snowflake achieves this by using a pool of pre-warmed nodes. Most cloud providers have recently transitioned to subsecond pricing, leading to new technical challenges in efficiently achieving resource elasticity and resource sharing across multiple tenants. Resolving these challenges may require design decisions and tradeoffs different from those in Snowflake's current design.

8.24.1 Storage architecture and provisioning in Snowflake

Two design decisions in ephemeral storage system.

(1) Rather than designing a pure in-memory storage system use both memory and local SSDs — tasks write as much intermediate data as possible to their local memory; when memory is full, intermediate results spilled to local SSDs. The rationale is that while in-memory systems can achieve superior performance when entire data fits in memory, they are too restrictive to handle the variety of our target workloads. There are queries that exchange hundreds of gigabytes/terabytes of intermediate data; which cannot be fit in main memory.

(2) Allow intermediate data to spill into remote persistent data store in case the local SSD capacity is exhausted. Spilling intermediate data to AWS S3, instead of other compute nodes, is preferable since it does not require keeping track of intermediate data location.

A future direction for performance-critical queries is to have intermediate data to entirely fit in memory or SSD.

Resource Elasticity is attained in Snowflake via BlowFish design which achieves one of its core goals: resource elasticity, that is, scaling of compute and storage resources on an on-demand basis. Disaggregating compute from persistent storage enables Snowflake to independently scale compute and persistent storage resources. Storage elasticity is offloaded to persistent datastores; compute elasticity, on the other hand, is achieved using a pre-warmed pool of nodes that can be added/removed to/from customer *Virtual Warehouses* - VWs on an on-demand basis. By keeping a pre-warmed pool of nodes, Snowflake is able to provide compute elasticity at the granularity of tens of seconds.

Persistent Data Caching for intermediate data is short-lived, so while storing intermediate data requires large memory and storage capacity at peak, the demand on average is low. The statistical multiplexing of ephemeral storage system capacity between intermediate data and frequently accessed persistent data improves performance for two reasons:

- (1) queries in data warehouse systems exhibit highly skewed access patterns over persistent data;
- (2) ephemeral storage system performance is significantly better than remote persistent data stores.

Snowflake uses hashing over persistent data file names. Each node uses a simple LRU policy to decide caching and eviction of persistent data files. Given the performance gap between our ephemeral storage system and remote persistent data store, such opportunistic caching of persistent data files Snowflake's execution time.

Maintaining the right system semantics during opportunistic caching of persistent data files requires:

- (1) Ensuring data consistency, the “view” of persistent files in ephemeral storage system must be consistent with those in remote persistent data store. This is achieved by forcing the ephemeral storage system to act as a write-through cache for persistent data files.
- (2) Consistent hashing of persistent data files on nodes in a naive way requires reshuffling of cached data when VWs are elastically scaled. The lazy consistent hashing optimization of ephemeral storage system in Snowflake avoids such data reshuffling altogether.

Even though our ephemeral storage capacity is much lower (around 0.1% on an average), than that of a customer's persistent data skewed file access distributions and temporal file access patterns enable reasonably high cache hit rates of 80%/60% for read-only/read-write queries.

8.24.2 Resource sharing in Snowflake

The variability of resource usage over time across VW, indicates that customer workloads are bursty in nature and shared architecture enables Snowflake to achieve better resource utilization via fine-grained statistical multiplexing. VW sizes are exposed to customers as small, large, XL, etc. T-shirts. Customers are not aware of how these VWs are implemented, e.g., number of nodes used. It would be best to maintain the same abstract VW interface to customers and change the underlying implementation to use shared resources instead of isolated nodes. The challenge, however, is to achieve isolation properties close to our current architecture. The key metric of interest is query performance, which is end-to-end query completion time. While a purely shared architecture is likely to provide good average-case performance, maintaining good performance at tail is challenging. The two key resources that need to be isolated in VWs are compute and ephemeral storage.

Performance isolation is a key challenge in cloud computing, but Linux has few defenses against performance interference in shared resources such as processor caches and memory buses, so applications in a cloud can experience unpredictable performance caused by behavior of other programs. CPI² tool uses cycles-per-instruction (CPI) data obtained by hardware performance counters to identify problems, select the perpetrators and throttle them (Zhang et al., 2013b). It automatically learns normal and anomalous behaviors by aggregating data from multiple tasks in the same job. CPI values of three latency sensitive jobs are given in Table 8.13.

TABLE 8.13 CPI values (mean and standard deviation) of a few latency-sensitive jobs, and the number of tasks they contain.

Job	CPI	Tasks
Job A	0.88 ± 0.09	312
Job B	1.36 ± 1.26	1040
Job C	2.03 ± 0.20	1250

8.25 Review of peer-to-peer computing

Peer-to-peer - P2P computing/networking is a distributed application architecture that partitions tasks or workloads between peers. Peers make a portion of their resources such as processing power, disk storage or network bandwidth, directly available to other network participants without the need for central coordination by servers or stable hosts. Peers are both suppliers and consumers of resources, in contrast to the traditional client-server model in which the consumption and supply of resources is divided. P2P was popularized by the file sharing system Napster in 1999.

<https://en.wikipedia.org/wiki/Peer-to-peer>

Emerging collaborative P2P systems are going beyond the era of homogeneous peers doing similar things while sharing resources and are looking for diverse peers that can bring in unique resources and capabilities to a virtual community thereby empowering it to engage in greater tasks beyond those that can be accomplished by individual peers, yet that are beneficial to all the peers.

Chord scalable peer-to-peer lookup service

A fundamental problem confronting peer-to-peer applications is to efficiently locate the node that stores a particular data item (Stoica et al., 2001). Chord is a distributed lookup protocol addresses this problem with the operator:

“given a key map it onto a node”.

Data location can be easily implemented on top of Chord by associating a key with each data item, and storing the key/data item pair at the node to which the key maps. Chord adapts efficiently as nodes join and leave the system, and can answer queries even if the system is continuously changing. In an N -node system each node maintains information about $O(\log N)$ nodes and resolves all lookups with as many messages. Analysis, simulations, and experiments show that Chord is scalable, with communication cost and the state maintained by each node scaling logarithmically with the number of Chord nodes.

A survey and comparison of peer-to-peer overlay network scheme using the following abbreviations are as follows (Lua et al., 2004).

F Freenet: Freenet is a peer-to-peer platform for censorship-resistant communication. It uses a decentralized distributed data store to keep and deliver information, and has a suite of free software for publishing and communicating on the Web without fear of censorship (Clarke et al., 2000).

<https://en.wikipedia.org/wiki/Freenet>

G Gnutella is a large peer-to-peer network. It was the first decentralized P2P network of its kind which led to later networks.

<https://en.wikipedia.org/wiki/Gnutella>

K FastTrack/Kazaa were originally created and developed by Estonian programmers who sold it to the creators of Skype (based on P2P backend) and later Joost and Rdio.

<https://en.wikipedia.org/wiki/Kazaa>

B Bit-Torrent - BT is a communication protocol for P2P that enables users to distribute files over the Internet in a decentralized manner.

<https://en.wikipedia.org/wiki/BitTorrent>

D Overnet/Donkey 2000: Compared to earlier Napster, eDonkey2000 featured “swarming” downloads, meaning that clients could download different pieces of a single file from different peers, effectively utilizing the combined bandwidth of all of the peers instead of being limited to the bandwidth of a single peer. At first as in Napster the servers were isolated from one another, but later versions of the eD2K enabled servers to form a search network. This allowed servers to forward search queries from their locally connected clients to other servers, allowing clients to effectively find peers connected to any server on the server network, thereby increasing download swarm size. It also allowed clients to find and download files not available from clients connected to the same server.

An improvement compared to Napster was the use of file hashes instead of simple filenames in search results. File searches initiated by the user were keyword-based and matched against the filename list stored on the eDonkey2000 server, but the server returned a list of filenames paired with the hash values of those files to the client. When selecting a file from the list presented to the user, the client would actually initiate a download by hash value. This meant that a file could have many different filenames across different peers, but would be considered identical for purposes of downloading if its hash was the same. The two-level (client and server) peer-to-peer network architecture offered a balance between centralized systems like Napster, and decentralized systems like Gnutella. Where Napster ultimately proved to be vulnerable was its centralized server cluster, which was a stable target for legal action. Gnutella’s original design, featuring total elimination of the server network in favor of purely peer-to-peer searching, quickly proved to be infeasible due to massive search traffic overhead between peers.

<https://en.wikipedia.org/wiki/EDonkey2000>

A comparison of aforementioned P2P systems is as follows:

Decentralization **F:** Loose DHT functionality. **G:** Topology is flat with equal peers. **K:** No explicit central server. Peers are connected to their Super-Peers. **B:** Centralized model with a Tracker keeping track of peers. **D:** Hybrid two-layer network composed of clients and servers.

Architecture: **F:** Keywords and descriptive text strings to identify data objects. **G:** Flat and Ad-Hoc network of peers. Flooding request and peers download directly. **K:** Two-level hierarchical network of Super-Peers and peers. **B:** Peers request information from a central Tracker. **D:** Servers provide the locations of files to requesting clients for download directly.

Lookup protocol: **F:** Keys based on Descriptive Text String search from peer to peer. **G:** Query flooding. **K:** Super-peers. **B:** Tracker. **D:** Client-server peers.

System Parameters: **F & G & K & D:** None. **B:** .torrent file.

Routing Performance: **F:** Guarantee to locate data using Keysearch until the requests exceed the *Hops-To-Live - HTL* limit. **G:** No guarantee to locate data; improvements made in adapting Ultrapeer-client topologies; good performance for popular content. **K:** Some degree of guarantee to locate data, since queries are routed to the Super-Peers which has a better scaling; Good performance for popular content. **B % D:** Guarantee to locate data and guarantee performance for popular content.

Routing State: **F & G & K & D:** Constant. **B:** Constant but chocking (temporary refusal to upload) may occur.

Peers Join/Leave: **F & G & K & B:** Constant. **D:** Constant with bootstrap-ping from other peer and connect to server to register files being shared.

Security: **F:** Low; Suffers from man-in-middle and Trojan attacks.

G: Low; Threats: flooding, malicious content, virus spreading, attack on queries, and denial of service attacks.

K: Low; Threats: flooding, malicious or fake content, viruses, etc. Spywares monitor the activities of peers in the

background.

B: Moderate; Centralized Tracker manages file transfer and allows more control which makes it much harder faking IP addresses, port numbers, etc.

D: Moderate; Similar threats as the FastTrack and Bit-Torrent.

Routing State: F: No hierarchy or central point of failure exists.

G: Degradation of the performance; Peer receives multiple copies of replies from peers that have the data; Requester peers can retry.

K: The ordinary peers are re-assigned to other Super-Peers.

B: The Tracker keeps track of the peers and availability of the pieces of files; Avoid Choking by fibrillation by changing the peer that is choked once every ten seconds.

D: Reconnecting to another server; Will not receive multiple replies from peers with available data.

A comparison of P2P and grid computing is given succinctly by Ken Crowcroft et al.

<https://www.cl.cam.ac.uk/teaching/2003/AdvSysTop/grid-p2p-paper.pdf>

Books on P2P systems are listed in the Appendix.

8.26 Fast Array of Wimpy Nodes - FAWN

FAWN is a fast, scalable, and energy-efficient cluster architecture for data-intensive computing, developed by David Anderson, Michael Kaminsky (Intel) et al. at CMU/PDL to take advantage of flash storage (Andersen et al., 2009).

<http://www.pdl.cmu.edu/FAWN/>

A FAWN cluster links together a large number of “wimpy” nodes built using energy-efficient processors and small amounts of flash memory into an ensemble cluster that can perform the same amount of work at a fraction of power of a traditional cluster. This discussion is therefore relevant to Chapter 7.

The clustered key-value store - FAWN-KV - runs atop these nodes, using a specialized log-like back-end hash-based datastore (FAWN-DS) to ensure that the system can absorb the large volume imposed write workload.

https://en.wikipedia.org/wiki/Key-value_database

FAWN-KV uses a two-level cache hierarchy to ensure that unbalanced workloads cannot create hot-spots on wimpy nodes thus impairing system’s ability to service queries at a guaranteed rate.

The evaluation of a small-scale FAWN cluster suggests that it can be a practical approach to building large-scale storage for seek-intensive workloads and further analysis indicates that a FAWN cluster is cost-competitive with other approaches to provide high query rates with significantly less power.

Primary-key, read-mostly workloads with small objects are gaining importance for Internet applications, such as e-commerce with high rates of requests to objects with a unique ID. Such workloads are not suited to conventional disk-based architectures mainly because of the high disk positioning time. Using DRAM is more costly and power intensive, but RAMClouds prototype for DRAM based distributed key-value storage provides higher throughput (Ousterhout et al., 2011).

Of interest in FAWN is the number of queries per joule. Note that power in watts (W) is equal to the energy E in joules (J) per second: $W = J/sec$.

<https://en.wikipedia.org/wiki/Joule>

FAWN nodes consume 40 W versus 200 W in conventional systems with disks. According to (Andersen et al., 2009) FAWN uses 800 MHz XScale chips with 256 MB DRAM coupled with 2-16 GB of Flash memory. Comparison of an Intel server and a FAWN node is given Table 8.14.

Shown in slide 10 of SOSP’09 are MIPS/Watt versus MIPS with a logarithmic scale.

<http://vijay.vasu.org/static/talks/fawn-sosp2009-slides.pdf>

A custom ARM Mote core is less than 10 MIPS, Xscale is 800 MHz followed by Atom Z500, which is over a 1000 MIPS, and the Xeon 3750 is about 50,000 MIPS. The consumption per watt versus MIPS increases linearly initially, but Xeon 3750 constitutes a soft spot with ≈ 500 MIPS/Watt.

It is assumed that the cost per KWH is ten cents and it also minimizes the *Total Cost of Ownership - TCO*. The cost of electricity in KWH varies 2-fold across US states approximately in the range 10-20 cents.

<https://www.electricchoice.com/electricity-prices-by-state/>

Low cost electricity was offered to 60% of consumers in Texas from retail companies. Texas with abundant access to natural gas and wind power had not winterized its equipment and was disconnected from the national grid until a freezing storm hit Texas in 2/21 and power was lost.

<https://www.nytimes.com/2021/02/21/us/texas-electricity-ercot-blackouts.html>

TABLE 8.14 Comparison of a high-end machine and a FAWN node, TCO - total cost of ownership.

Metric	Server-class node	Wimpy flash-based node
CPU	x86 Quad-core 2.3 GHz	800 MHz ARM
Memory (DRAM)	4-32 GB	128 MB
Memory cost/GB (in USF)	50	50
Storage capacity GB	40 GB-1 TB (per disk)	512 MB-16 GB
Storage cost per GB in USD	0.30	10
Disk speed (read latency, ms)	4	1
Queries/sec. (data in memory)	80,000	3000
Queries/sec. (data on disk)	170	150-300
Power consumption (Watts)	300	0.75
TCO over 3 years in USD	9000	453

Further analysis indicates that a FAWN cluster is cost-competitive with other approaches, e.g., DRAM, HDDs, SSDs, to providing high query rates, while consuming significantly less power.

On page 28 of SOSP'09 slides the dataset size is varied from 0.1 TB to up to 10k TB given with a logarithmic scale on the y-axis and the query rate in millions per second from 0.1 to 1000 (also logarithmically). FAWN plus DRAM is the best solution for smaller datasets and highest request rates, but at very high dataset sizes and low request rates flash plus disk is the best.

It is argued in (Andersen et al., 2009) that dynamic power scaling on traditional systems is surprisingly inefficient. A primary energy-saving benefit of *Dynamic Voltage and Frequency Scaling* - DVFS is the ability to reduce voltage as it reduces frequency, but modern CPUs already operate near minimum voltage at the highest frequencies. Even if processor energy was completely proportional to load, non-CPU components such as memory, motherboards, and power supplies dominate energy consumption, requiring that all components be scaled back with demand. As a result, running a modern, DVFS-enabled system at 20% of its capacity may still consume over 50% of its peak power. Despite improved power scaling technology, systems remain most energy-efficient when operating at peak utilization.

A promising path to energy proportionality is turning machines off entirely, but this technique does not apply to FAWN-KV's target workloads, since key-value systems often have *Service-Level Agreements* - SLAs for query response latency in hundreds of milliseconds; the inter-arrival time and latency bounds of the requests prevents shutting machines down and many seconds to be up again during low load.

Energy proportionality alone is not a panacea: systems ideally should be both proportional and efficient at 100% load. The FAWN approach provides a simple way to improve energy efficiency.

Mark Weiser (rip), Brent B. Welch (Panasas, Google), Alan J. Demers (Cornell U.) and Scott Shenker (*Int'l Computer Science Inst. - ICSI, UCB*) at Xerox's *Palo Alto Research Center - PARC* as early as 1994 had proposed millions of instructions per joule as a metric in discussing scheduling for reduced CPU energy in (Weiser et al., 1994). Energy consumption is reduced by lowering the clock-speed by reducing the voltage.

8.27 RAMCloud project at Stanford

The RAMCloud project was led by John Ousterhout at Stanford U. (2009-2017) (Ousterhout et al., 2011, 2015).

<https://web.stanford.edu/~ouster/cgi-bin/projects.php>

The goal of the project was to create a large-scale datacenter storage system with the lowest possible access latency by keeping all data in DRAM at all times. As an extension of Gray's 5-minute rule a 1 KB record is accessed at least once every 30 hours, it is not only faster to store it in memory than on disk, but also cheaper according to Jim Gray's five minute rule (Gray and Putzolu, 1987).

https://en.wikipedia.org/wiki/Five-minute_rule

The disadvantages of RAMCloud are high cost and energy usage per bit, which makes RAMCloud storage 50-100X worse than a disk-based system and 5-10 times worse than a flash memory (Andersen et al., 2011). RAMCloud requires more floor space in a datacenter than disk or flash-based storage. It remains attractive for applications with very high throughput requirements. The three attributes of RAMClouds are as follows:

TABLE 8.15 A comparison of disk technology in 2009 versus 25 years ago based on Table 2 in (Ousterhout et al., 2015).

Time period	Mid 1980s	2009	Improvement
Disk capacity	30 MB	500 GB	16667x
Maximum transfer rate	2 MB/s	100 MB/s	50x
Latency (seek+rotate)	20 ms	10 ms	2x
Capacity/bandwidth (large blocks)	15 s	5000 s	333x worse
Capacity/bandwidth (1 KB blocks)	600 s	58 days	8333x worse
Gray's rule (1 KB blocks)	5 min	30 hours	360x worse

Low Latency: In an 80 node prototype it takes less than 5/13.5 microseconds to read/write a 100 B object. In a large data center with 100,000 nodes reads will take less than 10 microseconds, which is a 1000-fold improvement.

Large Scale: RAMCloud allows clusters to grow to at least 10,000 servers with storage capacity of 1 PB or more. These memories are aggregated into a single coherent key-value store.

Durability: Although RAMCloud keeps all data in DRAM, it maintains backup copies of data on secondary storage, ensuring a high level of durability and availability.

The lack of improvement in disk technology from mid-19980s to 2009 which served as a justification of RAMcloud are given in Table 8.15. A few of the interesting aspects of RAMCloud are as follows (Ousterhout et al., 2015):

Fast crash recovery: To minimize DRAM cost RAMCloud keeps only a single copy of data in DRAM, but maintains multiple copies on secondary storage.

Log-structured memory: RAMCloud manages in-memory storage using an approach similar to *Log-Structured File System - LFS* (Rosenblum and Ousterhout, 1992). This allows RAMCloud to use DRAM twice as efficiently as traditional storage allocators such as malloc (memory allocation) (Rumble et al., 2014).

https://en.wikipedia.org/wiki/C_dynamic_memory_allocation

Rules-based programming decomposes algorithms into a collection of rules, which can make incremental progress towards a goal. RAMCloud contains several modules that must manage distributed resources in a concurrent and fault-tolerant fashion.

Implementing linearizability provides exactly-once semantics for *Remote Procedure Calls - RPCs* even in the face of crashes and reconfiguration, which uses this to implement a high performance txn mechanism in RAMCloud (Lee et al., 2015).

Scalable indexes: The original RAMCloud data model was a key-value store, but was extended to provide scalable and high-performance secondary indexes (Kejriwal et al., 2016).

Tables 8.16, 8.17, and 8.18 are based on Tables I, II, and III in (Ousterhout et al., 2015), where selected performance metrics are given in Table I, hardware configuration in Table II, and components of network latency in Table III.

The Configuration for RAMcloud experiments is given in Table 8.17. All nodes ran Linux 2.6.32 and were connected to a two-level Infiniband fabric with full bisection bandwidth. The Infiniband fabric supports 32 Gb/sec bandwidth, but PCI Express limits the nodes to about 24 Gb/sec.

TABLE 8.16 Selected performance metrics for RAMCloud, measured using the hardware described in Table II (4X QDR Infiniband Networking).

Read latency (100-byte objects, cds one client, unloaded server)	4.7 μ s
Read bandwidth (1 MB objects, one client, unloaded server)	2.7 GB/secW
Write latency (100-byte objects, one client, unloaded server)	13.4 μ s
Write bandwidth (1 MB objects, one client, unloaded server)	430 MB/sec
Read throughput (100-byte objects, many clients, single server)	950K object/sec
Multiread throughput (100-byte objects, many clients, one server)	6M object/sec
Multiwrite throughput (100-byte objects, many clients, one server)	450K object/sec
Crash recovery throughput (per server, unloaded)	800 MB/sec or 2.3M object/sec
Crash recovery time (40 GB data, 80 servers)	1.9 sec

TABLE 8.17 Hardware configuration of the 80-node cluster used for benchmarking.

CPU	Xeon X3470 (4 × 2.93 GHz cores, 3.6 GHz Turbo)
RAM	24 GB DDR3 at 800 MHz
Flash disks	2 crucial M4 SSDs CT128M4SSD2 (128 GB)
NIC	Mellanox ConnectX-2 Infiniband HCA
Switches	Mellanox MSX6036 (4X FDR) and Infiniscale IV (4X QDR)

TABLE 8.18 The components of network latency for Round-Trip RPCs in large datacenters (":" is used to specify a range of values).

Component	Traversals	2009 microsec	Possible 2014 microsec	Limit microsec
Network switches	10	100-300	3-5	0.2
Operating system	4	40-60	0	0
Network interface controller (NIC)	4	8:120	2:4	0.2
Application/server software	3	1:2	1:2	1
Propagation delay	2	1	1	1
Total round-trip latency		150:400	7:12	2.4

“Traversals” indicates the number of times a packet passes through each component in a round trip (e.g., five network switches must be traversed in each direction for a three-level datacenter network). “2009” estimates total round-trip latency for each component in a typical large datacenter in 2009 using 1 Gb/sec Ethernet technology. “Possible 2014” estimates best-case latencies achievable at reasonable cost in 2014 using Infiniband or 10 Gb/sec Ethernet technology. “Limit” estimates the best latencies that can be achieved in the next 5 to 10 years, assuming new network architectures and a radical integration of the NIC with the CPU. All estimates assume no contention.

8.28 How flash changes the design of database storage engines

We summarize answers to questions by database experts at Aerospike, Cassandra, FoundationDB, RethinkDB, and Tokutek using the following as a source.

<https://www.oreilly.com/ideas/how-flash-changes-the-design-of-database-storage-engines>

Tokutek’s storage engine runs on MySQL, ReThinkDB, MariaDB, Percona Server, and MongoDB; while the others are in the NoSQL category.

Random Reads: Random reads are fast, but one has to read a whole block at a time. Applications may still benefit from locality of reference, so that a further read is satisfied by cached data in main memory.

Raw throughput: Flash is two orders of magnitude better than disks.

Read latency: This is usually around 50 to 100 microseconds in flash and 5-50 milliseconds per read from disk. Flash is not improving since it has reached its limit in latency.

Parallelism: Flash drives offer multiple (about ten) controllers, which benefit database designs that can use multiple threads and cores.

Aerospike can simply run with DRAM backed by HDDs, but otherwise stores indexes in DRAM and the rest of the data in Flash. After index look up in DRAM data can be retrieved from multiple Flash drives in parallel. Writes to Flash are greatly reduced, because the indexes are updated in DRAM.

Cassandra sorts data to achieve locality of reference. It carries out defragmentation operations in order to maintain its *Log-Structured Merge - LSM* trees, which keeps data sorted into ordered sets, and combines sets to make them as long as possible. LSM-trees maintain key-value pairs with performance characteristics that make them attractive for indexed access to files with high update rate.

https://en.wikipedia.org/wiki/Log-structured_merge-tree

The LSM tree was developed by Edward Cheng, Dieter Gawlick (IBM, Amdahl, DEC/HP, Oracle), assisted by Patrick (rip)

and Elizabeth O’Neil (UMass, Boston) to minimize extra disk seeks (O’Neil et al., 1996). LSM-trees have proven useful with Flash in dramatically reducing writes.

FoundationDB used Flash controllers to solve the problem of fragmented writes. Just as sophisticated disk drives buffer and reorder writes to minimize seeks, Flash controllers follow LSM trees at the database engine level.

Tokutek offers a clustered database, keeping data with the index (Ramakrishnan and Gehrke, 2002). Clustering is determined to be ideal for retrieving ranges of data, especially for repeated values, as in denormalized NoSQL document stores like the MongoDB database that Tokutek supports. Compression also reduces read and write overhead, saving storage space and transfer time. Tokutek generally gets a compression ratio of 5:1-7:1 for MySQL and MariaDB. They do even better on MongoDB, up to 10:1 compression, because its document stores are denormalized. Denormalization obviates the need for joins, but incurs extra processing for updating replicated data (Schkolnick and Sorenson, 1982).

In MySQL the InnoDB storage engine provides *Multi-Version Concurrency Control - MVCC* row-level locking, https://en.wikipedia.org/wiki/Multiversion_concurrency_control

ACID compliance as well as other features. Aerospike, FoundationDB, RethinkDB, and Tokutek use MVCC or a similar approach to write new versions of data continuously, while cleaning up old versions, instead of directly replacing stored data with new values.

In general, a single write request by a database can turn into multiple writes, because of the need to update the data as well as multiple indexes. There is also the issue of updating the parity and check blocks. By storing indexes in memory, Aerospike achieves a predictable write amplification of two, whereas other applications suffer from a factor of ten.

Write Amplification - WA is an undesirable phenomenon associated with flash memory and SSDs, where the actual amount of information physically-written to the storage media is a multiple of the logical amount intended to be written.

https://en.wikipedia.org/wiki/Write_amplification

This is because flash memory must be erased before it can be rewritten, with much coarser granularity of the Erase operation when compared to the Write operation, the process to perform these operations results in moving (or rewriting) user data and metadata more than once. Thus, rewriting some data requires an already-used-portion of flash to be read, updated, and written to a new location, together with initially erasing the new location if it was previously used at some point in time. Due to the way flash works, much larger portions of flash must be erased and rewritten than actually required by the amount of new data. This multiplying effect increases the number of writes required over the life of the SSD, which shortens the time it can operate reliably. The increased writes also consume bandwidth to the flash memory, which reduces random write performance to the SSD. Many factors will affect the WA of an SSD; some can be controlled by the user and some are a direct result of the data written to and usage of the SSD.

Intel and SiliconSystems (acquired by Western Digital in 2009) used the term *Write Amplification - WA* by the ratio of Writes committed to the flash memory to the Writes coming from the host system. Without compression WA cannot drop below one, but with compression, SandForce has claimed to achieve a write amplification of 0.5, with best-case values as low as 0.14 in the SF-2281 controller.

https://en.wikipedia.org/wiki/Write_amplification

Write amplification - WA analysis with respect to SSDs

Write-Amplification - WA is a critical factor limiting the random write performance and write endurance in storage devices based on SSD NAND-flash memories. The impact of garbage collection on write amplification is influenced by the level of over-provisioning. Presented in (Hu et al., 2009) is a novel probabilistic model of write amplification for log-structured flash-based SSDs. Specifically, the impact of over-provisioning on write amplification analytically and by simulation assuming workloads of uniformly-distributed random short writes. Moreover, they propose modified versions of the greedy garbage-collection reclaiming policy and compare their performance. Finally, they analytically evaluate the benefits of separating static and dynamic data in reducing write amplification, and how to address endurance with proper wear leveling.

Suppose that I pages have been rewritten and hence have been invalidated in a block before it is selected for garbage collection. The block still has V valid pages, where $V + I = n_p$, that have to be relocated to another block before the block can be erased and reclaimed. In other words, in order to rewrite I user pages, the number of physical pages that have to be written is $V + I$. Therefore $A = (V + I)/I = 1 + V/I$ and WA is defined as $A_f = V/I$.

Consider an SSD with raw storage capacity of t blocks of which the user can use $u < t$ blocks. The Over-provisioning factor is $O_f = t/u$ and the Spare factor $S_f = 1 - u/t$.

Simulation and analysis shows that other reasonable assumptions A_f drop from about 3.5 to zero as S_f is varied from 0.1 to 0.9.

Write amplification analysis in Flash -based SSDs is presented in (Hu et al., 2009).

Flash memories are expected to last as long as the system, e.g., Intel S3700 SSD can last five years with 10 writes per day.

Both Aerospike and FoundationDB offer strict durability guarantees.

Flash offers 100K operations per second compared to 300 for typical disks. Flash creates the most challenge to database design, because the traditional single-threaded design of relational databases worked fine when disks were the bottleneck, but now the threads become the bottleneck. FoundationDB internally uses its own light-weight processes, like Erlang does. Stagnation in latency improvement makes parallelism even more important.

http://erlang.org/doc/efficiency_guide/processes.html

Because of extensive parallelism deep queues work better on Flash than disks. Aerospike is used by diverse applications such as e-commerce, web auctions, airline pricing, and telecom routing. These applications require many short txns and involve high volumes of data. The concurrency provided by Aerospike allows the consistent completion of database operations 99% of the time, within a few milliseconds, and scales cost effectively on small clusters.

Flash technology offers database designers with new challenges. A NetApp blog emphasizes the issue of simplified infrastructure as follows:

<https://blog.netapp.com/blogs/how-can-all-flash-storage-save-you-money-its-simple/>

- Replacing disks with flash memory frees up space and cuts power and cooling costs.
- Makes infrastructure choices with the future in mind, such as cloud computing, possible.
- Standardizes infrastructure leads to adaptable data centers.

While Flash storage may be more expensive initially it reduces the TCO in six months.

An SSD controller consists of a processor, a buffer manager, and a Flash controller connected to multiple channels, whose operation with respect to multiple flash memory packages can be interleaved.

In SSD firmware, FTL is implemented to emulate a hard disk. FTL exposes an array of logical blocks to the host and implements a set of complex mechanisms to manage flash memory blocks and optimize SSD performance. A detailed survey on the FTL algorithms is provided in (Gal and Toledo, 2005) and here we briefly introduce three key roles of FTL as follows:

- (1) logical block mapping, which maps logical block addresses (LBAs) to physical block addresses (PBAs);
- (2) garbage collection, which handles the not-in-place-write issue, and asynchronously recycles dirty blocks in a way similar to LFS (Rosenblum and Ousterhout, 1992);
- (3) wear leveling shuffles cold blocks (read intensive) with hot blocks (write intensive) to even out writes over flash memory blocks.

Exploiting the internal parallelism of SSDs, more specifically how to optimize scan, join, aggregation and sort operators, is discussed in (Fan et al., 2014). Another study of internal parallelism of Flash memories is by Feng Chen and Binbing Hou (Louisiana State U. - LSU), Rubao Lee (OSU) is (Chen et al., 2016). Various forms of parallelism in Flash are as follows:

Channel-Level Parallelism: The data bus is the main performance bottleneck, so SSDs usually incorporate multiple channels to connect the controller to the flash memory packages. Each channel can operate independently and simultaneously. Resource redundancy is provided, e.g., some SSDs adopt an independent ECC engine for each channel to further speedup I/Os. which improves performance but also increases production cost.

Package-Level Parallelism: In order to optimize resource utilization, a channel is usually shared among multiple flash memory packages. Since each flash memory package can be operated independently, operations on flash memory packages attached to the same channel can be interleaved. As so, the bus utilization can be optimized.

Die-Level Parallelism: A flash memory package often includes two or more dies (chips), e.g., a Samsung K9LBG08U0M flash memory package is composed of two K9GAG08U0M dies. Each die can be selected individually and execute a command independent of the others, which increases the throughput.

Plane-Level Parallelism: A flash memory chip is typically composed of two or more planes. Flash memories from Samsung and MIC support performing the same operation (e.g., read/write/erase) on multiple planes simultaneously. Flash memories from MIC provide cache mode to further parallelize medium access and bus transfer.

Given in Fan et al. (2014) is the time versus the number of threads (1:30) for parallel scan in Figure 3 for HDD, SSD-SLC and SSD-MLC. The time versus the number of threads (1:20) is given for parallel aggregation in Figure 4 for HDD,

SSD-SLC and SSD-MLC. MLC and SLC perform similarly outperforming HDD, which shows the highest time for ten threads. The join times for traditional and parallel hash joins are given in Figure 4 versus memory size, which is varied from 2 to 256 MB, with parallel hash join exhibiting good performance beyond 16 MB.

Making effective use of flash in three contexts is discussed in (Athanasoulis et al., 2010):

- (i) as a log device for txn processing on memory-resident data,
- (ii) as the main data store for transaction processing, and
- (iii) as an update cache for HDD-resident data warehouses.

According to Table 8.19 reproduced from (Athanasoulis et al., 2010) HDD random access latency and bandwidth have improved by only 3.5X since 1980, their sequential bandwidth lags far behind their capacity growth, and the ratio of sequential to random access throughput has increased 19 fold.

TABLE 8.19 Comparison of Hard Disk Drives (1980, 2010) and Flash Drives (2010), BW = Bandwidth.

Device & Year	Capacity (GB)	Cost (\$)	Cost/MB (\$)	Random Access Latency (ms)	Random Access BW (MB/s)	Seq'l Access BW (MB/s)	Seq'l BW Random BW	Device Scan (s)
HDD 1980	0.1	20000	200	28	0.28	1.2	4.3	83
HDD 2010	1000	300	0.0003	8	0.98	80	81.6	12500
SSD 2010	100	2000	0.02	0.026	300	700	2.3	143

8.29 Hybrid Transaction Analytic Processing - HTAP

The list of companies providing *Hybrid Transactional Analytic Processing - HTAP*

https://en.wikipedia.org/wiki/Hybrid_transactional_analytical_processing

is as follow: Alibaba DRDS, TiDB, Hubble, ArangoDB, Aerospike, Apache Ignite/GridGain In-Memory Data Fabric, IBM Db2 IDAA, InterSystems, Kdb+, Microsoft SQL Server, Neo4j, Oracle 12c In-Memory, SAP HANA, memSQL, MongoDB, VoltDB, NuoDB, OrientDB, DataStax, eXtremeDB, Splice Machine, EsgynDB, Cloud Spanner, HarperDB, Amazon Aurora (Parallel Query) and BlobCity.

8.30 Intelligent page store for concurrent txn and query processing

The intelligent page store (Dias et al., 1992) was a paper design proposed by Dan Dias (rip), Ambuj Goyal (retired), and Francis Parr at IBM Research in early 1990s.

The page store provides two paths for accessing data. Update and txn traffic are presented the current version of the data by the page store. Complex queries against the same data are presented a recent txn consistent snapshot of the data. Periodically, new snapshots are created for access by new queries. By using an implicit versioning scheme, the page store maintains one copy of most of the data, while providing consistent access. This scheme allows the txn and query systems to be independently optimized. It is shown that txn performance is largely unaffected by concurrent queries, and disks are used efficiently since a query copy of the database is obviated. The versioning scheme utilized in this study is based on (Bober and Carey, 1992).

8.31 Oracle Exadata database machine

Oracle Exadata DBM where Exa= 10^{18} was introduced in 2008.

<https://www.oracle.com/engineered-systems/exadata/database-machine-x8>

What follows is partially based on a presentation by Oracle's Juan Loaiza about "Oracle's Autonomous Database".

<https://www.youtube.com/watch?v=2HsVShulgx8>

Oracle claims Exadata is run by 77% of the Fortune Global 100 companies delivering high performance, cost effectiveness, and availability for Oracle databases.

<https://fortune.com/fortune500/>

It features a cloud-based architecture with scale-out high-performance database servers and intelligent storage servers with *Peripheral Component Interconnection - PCI* Flash.

Ultra-fast InfiniBand is a computer networking communications standard used in high-performance computing that is used to connect all servers and storage. It features very high throughput and very low latency interconnect among and within

computers as either a direct or switched interconnect between servers and storage systems. It is designed to be scalable and uses a switched fabric network topology.

<https://en.wikipedia.org/wiki/InfiniBand>

Infiniband promoted by its Trade Association as of 2014 has been the most commonly used supercomputer interconnect. Mellanox and Intel manufacture InfiniBand host bus adapters and network switches. Oracle in 2/16 engineered its own InfiniBand switches and server adapter chips for its product lines. Mellanox Infiniband cards are available for Solaris, FreeBSD, *Red Hat Enterprise Linux - RHEL*, Windows, HP-UX, VMware ESX, and AIX.

https://en.wikipedia.org/wiki/Red_Hat_Enterprise_Linux

Suse Linux Enterprise Server - SLES,

https://en.wikipedia.org/wiki/SUSE_Linux_Enterprise_Server

Infiniband competes with Ethernet, Fibre Channel, and Intel Omni-Path.

<https://www.intel.com/content/www/us/en/high-performance-computing-fabrics/omni-path-driving-exascale-computing.html>

Exadata runs all types of database workloads including OLTP, Data Warehousing, In-Memory Analytics, as well as mixed workloads.

MVCC is used to alleviate lock contention between txn updating the database and queries requiring a consistent view of data.

Oracle Exadata has the following hardware components according to:

<https://www.oracle.com/technetwork/database/exadata/overview/index.html>

Up to 912 CPU cores and 28.5 TB memory per rack for database processing.

Up to 360 CPU cores per rack dedicated to SQL processing in storage.

From 2 to 19 Database Servers per rack.

From 3 to 18 Storage Servers per rack.

Up to 920 TB of raw flash capacity per rack.

Up to 2.1 PB of raw disk capacity per rack.

Hybrid Columnar Compression often delivers 10X-15X compression ratios.

40 Gb/second (QDR) InfiniBand Network.

Complete redundancy for high availability.

Two other sources of information on Exadata are as follows.

https://en.wikipedia.org/wiki/Oracle_Exadata

<https://www.oracle.com/engineered-systems/exadata/>

Another source of information about Teradata is the Sept. 2019 White Paper by Mark Peters: “Exadata X8M Boasts In-memory Performance, RDMA Fabric, and Shared Storage for an Unmatched Oracle Database Platform”.

<https://www.oracle.com/a/ocom/docs/applications/performance-management/esg-exadata-x8m.pdf?elqTrackId=ab63a87882af4c72844597e1e3ad8097&elqaid=87236&elqat=2>

(1) Exadata X8M provides fast in-memory performance and ultra-low latency in a cost-effective way through the use of RDMA, persistent memory, and new virtualization software.

(2) X8M compared with the X8 and its InfiniBand fabric, offers 100 Gb *RDMA over Converged Ethernet - RoCE* as the internal fabric to deliver latency of under 19 microseconds (10X improvement over the X8).

https://en.wikipedia.org/wiki/RDMA_over_Converged_Ethernet

RoCE is a protocol that runs InfiniBand RDMA software on top of Ethernet. RoCE v1 is an Ethernet link layer protocol allowing communication between any two hosts in the same Ethernet broadcast domain, while RoCE v2 is an internet layer protocol which allows packet routing.

(3) With 1.5 TB of *Persistent MEMORY - PMEM* per storage server and up to 21.5 TB of PMEM per standard full rack, organizations can achieve up to 16 million OLTP 8K read IOPS, 2.5X the X8.

(4) Log writes gain a significant improvement to reliably complete more txns faster and at scale.

(5) By utilizing *Linux Kernel Virtual Machine - KVM* organizations gain a cost-effective virtualization layer that enables better consolidation without sacrificing performance.

https://en.wikipedia.org/wiki/Kernel-based_Virtual_Machine

The following are highlights of the three main new technology areas.

RoCE3's speed and performance are combined with risk reduction and reliability. RDMA provides extreme performance, enabling access to data on a remote computer without involving the OS and hence CPU. The network card directly reads/writes memory with no extra copying or buffering and at very low latency. RoCE on Exadata X8M supports all Exadata RDMA optimizations and the benefits are significant.

Additional benefits to performance can be gained by leveraging RoCE automated “class of service” and “messaging prioritization”, which ensures that the higher priority messages that require low latency are not impacted by high-throughput traffic. If the cluster is running a batch request or doing a backup (both requiring high-throughputs), committing a txn or ensuring the cluster is up and running via heartbeat checks are not impacted.

[https://en.wikipedia.org/wiki/Heartbeat_\(computing\)](https://en.wikipedia.org/wiki/Heartbeat_(computing))

Exadata RoCE helps mitigate risks, e.g., a feature called Priority-based Flow Control can detect the speed of incoming packets. If the packets are coming too fast or unable to be serviced, the feature tells the sender to pause until the packet buffer has empty space. With *Explicit Congestion Notification - ECN*, fast packet senders can be detected and forced to slow down to ensure no packets are dropped. An early authoritative comparative survey of flow-control methods is (Gerla and Kleinrock, 1980).

Exadata uses RoCE for instant failure detection; since otherwise failures are determined by long timeouts, making it hard to detect a slow response versus a server failure, but now Exadata uses RDMA and frequent heartbeat messages between nodes to detect possible failures at sub-second speeds. RDMA uses hardware, so remote ports respond even if the software is slow.

A list of additional Exadata X8M features is as follows:

1. High throughput and low-CPU usage for large data transfers.
2. Unique Exafusion *Direct-to-Wire Protocol - DoWP* to deliver 3x faster inter-node OLTP cluster messaging. This allows processes to read and send Oracle *Real Applications Cluster - RAC* messages directly over the Infiniband network bypassing the overhead of entering the OS kernel, and running the normal networking software stack.
3. Unique *Smart Fusion Block Transfer - SFBT* eliminates log write on inter-node block move. Without this feature, a hot block can be transferred from a sending node to a receiver node only after the sending node has made changes in its redo log buffer durable in its redo log, but with this feature this latency of redo log write at the sending node is eliminated. This is important for OLTP workloads.
4. Unique RDMA protocol to coordinate txns between nodes.
5. Ultra low-latency I/Os to persistent memory in storage servers.
6. Ultra low-latency writes of database logs to persistent memory in storage servers.
7. Ability to unify cluster switching and client switching operations and management to save costs.

Persistent Memory - PMEM. Oracle has incorporated Intel Optane DC PMEM, which is discussed in Chapter 2, in the Exadata X8M. Unlike traditional DRAM, the persistent memory enables data to survive power failures, can be read like DRAM, and persist writes through the utilization of intelligent software.

The PMEM accelerator sits in front of the flash, adding another tier of capacity and performance. With the Oracle Database using RDMA instead of regular I/O protocols to read/write to PMEM, it can achieve 2.5x higher IOPS throughput compared with the recently introduced X8 Exadata, and 10x better latency.

PMEM automatically tiers and shares data across databases, helping to increase the effective capacity by 10x; for log writes, the PMEM accelerator can address the criticality of low latency to improve txn commit times; and, by bypassing the software and network stack, a one-way RDMA writes to PMEM across multiple storage servers, which can show log write performance improvements of up to 8x.

Virtualization with KVM Exadata X8M: KVM delivers 2x more guest VM memory (1.5 TB/server), 50% more guest VMs per server, and faster network latency. Some key functionalities of *Oracle Linux Virtualization Manager - OLVM* are as follows.

<https://docs.oracle.com/en/virtualization/oracle-linux-virtualization-manager/>

(1) Live Migration. (2) VM High Availability. (3) Policies (DRS, DPM, Anti-affinity). (4) REST API. (5) Virtual Appliances and Templates. (6) Enterprise Manager Integration.

New features will only be added to OLVM:

- (1) Snapshotting of existing VMs to improve backup and restore functionality.
- (2) Role based access.

8.32 Oracle in memory option or Database in Main Memory - DBIM

Oracle has introduced a dual-format in-memory database called the Oracle in Memory Option (Lahiri et al., 2015) or *Database in Memory - DBIM* (Mishra et al., 2016). Row format data enhanced with indexing is accessed by OLTP workloads, while column formats are better suited for analytic operations, since typically a small number of columns is inspected from a large number of rows. Given that no single data format is ideal for all types of workloads, this approach simultaneously maintains both formats with transactional consistency between them.

The columnar data built by a process called populate is held in main memory, but not on disk. The In-Memory declaration can be performed for multiple classes of objects: tablespaces, tables, partitions of a table. The In-Memory advisor analyses historical database workloads to recommend such data. In-Memory columns which can be compressed, are amenable to SIMD vector processing, and indexing.

The in-memory column format allows per CPU core scan speeds exceeding billions of rows per second. This allows a variety of query optimizations Group By and cube aggregations. Column store scale-out is possible by the RAC - Real Application Cluster feature.

<https://logicalread.com/oracle-12c-rac-mc05/#.X0afQFB7nCI>

Data can be distributed by partition, subpartition, row-id range, and by default. Columnar data can be replicated to attain fault-tolerance.

Dropping analytic indexes may provide a substantial speedup for OLTP DML - Data Manipulation Language operations by eliminating costly index maintenance.

Dynamic In-Memory Expressions - DIMEs are presented in (Mishra et al., 2016). This mechanism automatically detects frequently occurring expressions in queries, creating columnar representations of these expression results, e.g., $(\text{salary} + \text{bonus}) \times (1 - \text{tax_percent})$.

At runtime queries can directly access DIMEs and this results in significant speedups.

This is an instance of materialized views.

https://en.wikipedia.org/wiki/Materialized_view

A comprehensive collection of papers on this topic is provided in (Gupta and Mumick, 1999).

<https://mitpress.mit.edu/books/materialized-views>

“Materialized views are views that have been computed and stored in databases, because they reduce the need to recompute the view and/or data being queried, they speed up the querying of large amounts of data. Further, because they provide a systematic way to describe how to recompute the data, maintenance and evolution can be automated. Materialized views are especially useful in data warehousing, query optimization, integrity constraint maintenance, OLAP, and applications such as billing, banking, and retailing”.

8.33 MemSQL/SingleStore

MemSQL renamed SingleStore was cofounded by Nikita Shamgunov (Microsoft, Facebook) in 2011,

<https://www.linkedin.com/in/nikitashamgunov/>

<https://en.wikipedia.org/wiki/MemSQL>

Eric Hanson who coauthored a paper on stored procedures with Michael Stonebraker (Stonebraker et al., 1987)

https://en.wikipedia.org/wiki/Stored_procedure

joined MemSQL in November 2016 and presented the following talk:

“A trillion rows per second as a foundation for interactive analytics”.

<https://www.youtube.com/watch?v=5-kuzcBfTjw>

<https://www.linkedin.com/in/erichanson/>

An interview with Shamgunov and Hansen is posted below.

<http://highscalability.com/blog/2012/8/14/memsql-architecture-the-fast-mvcc-inmem-lockfree-codegen-and.html>

SingleStore which uses MVCC is a fast, distributed, in-memory, SQL RDBMS, which provides scale-out by hash partitioning across leaf nodes. SingleStore can store database tables either as rowstores or columnstores as determined by the user at data definition time. Rowstore tables are stored in-memory with snapshots and txn logs on disk. Data for all columnstore tables is stored on-disk, using a rowstore-like structure to handle incoming inserts into the columnstore. Rowstore and columnstore tables differ in more than just the storage medium used.

- Rowstores store information in row format are optimized for singleton or small insert, update or delete queries associated with OLTP.
- Columnstores are optimized for complex queries typically associated with OLAP analytics. Disk-based columnstores store one million row segments.

MemSQL can process more than 3 billion rows/sec/core. A synthetic benchmark with 57.8 billion rows was used in the study. Average query time was 52.5 milliseconds and 1.1 billion rows were processed per second.

MemSQL claims to run Oracle’s own demo ten times as fast, and sixty times cheaper.

<https://www.memsql.com/blog/memsql-does-oracles-own-demo-ten-times-as-fast-sixty-times-cheaper/>

MemSQL is faster because of its support of sorted columnar tables, while Oracle In-Memory Option is a columnar compressed dataset with no ordering information. MemSQL's compression format allows it to perform operations on sort-ordered compressed data.

Sort order preserving data compression is discussed in (Zandi et al., 1993) It is demonstrated in (Antoshenkov, 1997) that the fast dictionary-based methods can be applied to order-preserving compression almost with the same freedom as in the general case. The proposed method can be applied at the same speed yielding a compression rate only marginally lower than the traditional order-indifferent dictionary encoding. Procedures for encoding and generating the encode tables are described covering such order-related features as ordered data set restrictions, sensitivity and insensitivity to a character position, and one-symbol encoding of each frequent trailing character sequence. The experimental results presented demonstrate five-fold compression on real-life data sets and twelve-fold compression on Wisconsin benchmark text fields. A more recent work "dictionary-based order-preserving string compression for main memory column stores is (Binnig et al., 2009). More details appear in a PhD thesis of Stefan Hildenbrand supervised by Carsten Binnig at ETH, in the context of SAP HANA (Hildenbrand, 2012).

https://en.wikipedia.org/wiki/SAP_HANA

8.34 Amazon Aurora

Amazon Aurora is a MySQL and PostgreSQL compatible DBMS for the cloud, which combines the simplicity of open source databases and performance and availability of enterprise databases.

https://en.wikipedia.org/wiki/Amazon_Aurora

The effort was led by Alexandre Verbitski (Microsoft, Amazon), Anurag Gupta (Oracle, AWS), Debanjan Saha (IBM/Softlayer, AWS, Google). Aurora became available in conjunction with Amazon AWS in October 2014.

Amazon *Relational Database Service - RDS* makes it easy to set up operate and scale relational databases in the cloud for MySQL, PostgreSQL, MariaDB, Oracle, SQL server. The three contributions of Aurora according to (Verbitski et al., 2017) are as follows:

1. Can tolerate correlated failures by using a quorum system for maintaining replicated data developed in (Gifford, 1979). Each copy of V copies of a replicated file is assigned a vote. To read a file V_r and to write file V_w votes are necessary. Each read must be aware of the most recent write, formulated as $V_r + V_w > V$, which ensures the set of nodes used for a read intersects with the set of nodes used for a write and the read quorum contains at least one location with the newest copy. Each write must be aware of the most recent write to avoid conflicting write formulated as $V_w > V/2$. A common approach to tolerate the loss of a single node is to replicate data to $V = 3$ nodes and set $V_r = V_w = 2$. A quorum model with $V = 6$ votes was used in (Verbitski et al., 2017) with $V_w = 4$ and $V_r = 3$.
2. The lower quarter of database operations are offloaded to smart storage. Redo records when applied to before images generate after images. Only redo log records in Aurora cross the network boundary reducing traffic. According to the paper a mirrored MySQL and Aurora with replicas do 7.4 and 0.95 I/Os per txn, respectively and over a 30 minute period Aurora processed 35 times more txns than MySQL.
3. Multiphase synchronization, crash recovery, and checkpointing in distributed storage are eliminated.

It is stated in (Verbitski et al., 2018) states that pushing redo processing to a storage service reduces networking traffic, avoids checkpoints and crash recovery, allows failovers to replicas without loss of data, and enables fault-tolerant storage that heals without database involvement. Traditional implementations which use distributed consensus algorithms for commits, reads, replication, and membership changes amplify the cost of underlying storage. Aurora establishes invariants to avoid distributed consensus under most circumstances and this results in an improved performance, reduced variability, and lower costs.

[https://en.wikipedia.org/wiki/Invariant_\(mathematics\)#Invariants_in_computer_science](https://en.wikipedia.org/wiki/Invariant_(mathematics)#Invariants_in_computer_science)

8.35 Transaction processing in the cloud

There has been recent interest in cloud databases and txn processing in a cloud environment. Deuteronomy developed by Justin Levandoski (Amazon), David Lomet et al. at Microsoft (Levandowski et al., 2015a), (Levandowski et al., 2015b) separates txn from data storage functionality via a *Txn Component - TC* and *Data Component - DC*. Deuteronomy redesign has resulted in a high-performance key value store, which is a latch-free B+tree like data structure held in flash memory, but cached in DRAM. LLAMA is latch-free, log structured cache and storage engine (Levandowski et al., 2013).

The redesign of TC was driven by two goals:

- (1) high performance regardless of whether the TC and DC are co-located or distributed.

(2) exploiting multi-core and multi-socket machines, large main memories, Flash storage.

This paper summarizes the design of the new TC which addresses these two goals.

The TC consists of three main components:

- (a) an MVCC component to manage the concurrency control logic;
- (b) a version manager to manage the redo log and cached records read from the DC;
- (c) a TC Proxy at the DC which submits committed operations to the DC, which maintains database state.

The *Time-Stamp Ordering - TSO* concurrency control method assigns timestamps to txns. All of its writes are associated with this timestamp and all of its reads only see versions that are visible as of its timestamp (Bernstein et al., 1987). Enforcing TSO aborts txns when timestamp ordering cannot be maintained.

https://en.wikipedia.org/wiki/Timestamp-based_concurrency_control

While TSO never took off in practice, see e.g., (Bernstein, 2017), it is claimed that TSO performs well when all concurrency control metadata is kept in memory.

TSO has the advantage that it enables serializable isolation without needing validation required by *Optimistic Concurrency Control - OCC*. In the case of MVCC TSO the TC tracks txns via a txn table, which holds the status: active, committed, or aborted, txn id and the timestamp issued when the txn starts. Version information for a record is managed by a latch-free MVCC hash table. The entry for each version in the table is marked with the txn id that created the version. This permits an easy check of the txn information for each version, including its timestamp.

https://en.wikipedia.org/wiki/Multiversion_concurrency_control

8.36 RAPID and Oracle AutoML: a fast and predictive AutoML pipeline

RAPID is a simple, modern low power data processor that is at least an order of magnitude performance/power improvement in SQL processing than can be achieved over a modern system running on today's complex processors (Balkesen et al., 2018).

RAPID is designed from the ground up with hardware/software co-design in mind to provide architecture-conscious extreme performance while consuming less power in comparison to the modern database systems. The detailed design and implementation of RAPID, a relational, columnar, in-memory query processing engine supporting analytical query workloads is presented in (Yakovlev et al., 2020).

https://labs.oracle.com/pls/apex/f?p=labs:49:::::P49_PROJECT_ID:14

Oracle's AutoML is an iteration-free pipeline designed to provide accurate models and shorter runtime. This is achieved by eliminating the need to continuously iterate over various pipeline configurations. The approach builds and tunes only the best candidate pipeline and achieves better scores at a fraction of the time compared to open source AutoML tools, such as H2O and Auto-sklearn.

The Oracle *Accelerated Data Science - ADS*

https://en.wikipedia.org/wiki/Software_development_kit is a Python library that is included as part of the Oracle Cloud Infrastructure Data Science Service.

https://docs.cloud.oracle.com/en-us/iaas/tools/ads-sdk/latest/user_guide/automl/overview.html

ADS offers a friendly user interface, with objects and methods that cover all the steps of ML models, from data acquisition to model evaluation and interpretation. The Oracle AutoML consists of the following steps:

A. Preprocessing.

B. Algorithm Selection. Given a dataset and a prediction task, the goal of this step is to identify the algorithm that maximizes the model score. The ADS algorithm selection stage is designed to rank algorithms based on their estimated predictive performance on the given dataset. For a given dataset, the algorithm selection process is as follows:

1. Extract relevant dataset characteristics, such as dataset shape, feature correlations, and appropriate meta-features.
2. Invoke specialized score-prediction metamodels that were learned to predict algorithm performance across a wide variety of datasets and domains.
3. Rank algorithms based on their predicted performance.
4. Select the optimal algorithm.

C. Adaptive Data Reduction. Adaptive Sampling iteratively subsamples the dataset and evaluates each sample to obtain a score for a specific algorithm. The goal of is to find the smallest sample size that adequately represents the full dataset. It is used in subsequent pipeline stages without sacrificing the quality of the model. The adaptive sampling process is as follows:

- (1) For a given algorithm and dataset, identify a representative sample.

- (2) Leverage meta-learning to predict algorithm performance on the given sample.
- (3) Iterate until the score converges.
- (4) The identified sample is then used for subsequent stages of the AutoML Pipeline.

D. Hyperparameter optimization: Feature Selection stage aims to select a subset of features that are highly predictive of the target. This speeds up model training without loss of predictive performance. The ADS feature selection approach leverages meta-learning to identify the optimal feature subset for a given algorithm and dataset. For a given dataset, the feature selection process is as follows:

- (1) Obtain the dataset meta-features, similar to those obtained in the algorithm selection stage.
- (2) Rank all features using multiple ranking algorithms. Feature rankings are ordered lists of features from most to least important.
- (3) For each feature ranking, the optimal feature subset is identified.
- (4) Algorithm performance is predicted by leveraging meta-learning on a given feature subset.
- (5) Iterating over multiple feature subsets, the optimal subset is determined.

E. Hyperparameter tuning: This stage determines the optimal values for the model. Tuning tends to be the most time-consuming stage of an AutoML pipeline. Therefore, first-order requirements for efficiency and scalability of hyperparameter tuning process.

The research contributions of this study are:

Iteration free optimizer: Iteration-free sequence of ML pipeline stages, consisting of algorithm selection, adaptive data reduction, and hyperparameter optimization, is first of its kind. Each pipeline stage optimization outcome is final and only affects the downstream stages.

Proxy Models are predictive of relative performance of algorithms and data subsets.

Adaptive Data Reduction - ADR selects a representative sample of a dataset, along both the row and feature dimensions, optimized for a selected algorithm. ADR speeds up hyperparameter optimization with minimal impact on predictive performance of models.

8.37 Benchmarking automatic ML frameworks

AutoML is a framework used in production systems allowing companies to quickly pass extracted value from their data on to their customers. AutoML automates many parts of a ML pipeline. First, it automates the feature engineering process through *term frequency-inverse document frequency - tf-idf* processing,

<https://en.wikipedia.org/wiki/Tf%E2%80%93idf>

date processing, categorical encoding and numeric feature scaling. Its date preprocessing includes converting timestamps into binary features like weekend or weekday and splitting up components such as day, month and year. AutoML also performs feature reduction when more than 100,000 columns exist using reduction methods such as PCA. This library requires the type of each feature as input in order to preprocess correctly. In addition, AutoML automates the model construction, tuning, selection, and ensembling process.

AutoML utilizes highly optimized libraries such as Scikit-Learn, XGBoost, TensorFlow, Keras, and LightGBM for its algorithm implementations. It also contains prebuilt model infrastructures for each classification and regression method which have a less than one millisecond prediction time.

Despite its features, it has poor extensibility. It also tends to perform poorly with multi-class classification problems. It also does not support a time limiting feature and thus each algorithm must be run to completion in an unbounded amount of time. This weighs against the usage of this framework in time constrained scenarios such as frequently retrained production systems.

Auto-sklearn uses the sklearn framework to automatically create an ML pipeline (Feurer et al., 2015).

<https://en.wikipedia.org/wiki/Scikit-learn>

It includes feature engineering methods such as one-hot encoding, numeric feature standardization, PCA and more.

<https://en.wikipedia.org/wiki/One-hot>

The models use sklearn estimators for classification and regression problems. Auto-sklearn creates a pipeline and optimizes it using Bayesian search. The default hyperparameter values are warm started using 140 pre-trained datasets from OpenML using a meta-learning process. Computed are 38 statistics for a dataset and initializes the hyperparameters to the optimized parameters of a dataset with statistics closest to the training set (determined by L1 or Manhattan or taxicab distance). Auto-sklearn tries all the relevant data manipulators and estimators on a dataset but can be manually restricted. It also has multi-threading support. One of the greatest advantages of this platform is its easy integration into the existing sklearn

ecosystem of tools which provides an avenue for extension. Auto-sklearn uses the optimization framework SMAC3 which implements Bayesian search along with a racing mechanism to quickly assess model performance. This package lacks the ability to process natural language inputs and the ability to automatically discern between categorical and numerical inputs. The package also does not handle string inputs and requires manual integer encoding to accept categorical strings.

Tree-Based Pipeline Optimization Tool - TPOT is a generic programming based optimizer that generalizes ML pipelines, extending the scikit learning framework with its own base regressor and classifier methods. Like auto-sklearn, TPOT sources its data manipulators and estimators from sklearn and its search space can be limited through a configuration file. Time restrictions are applied to TPOT by changing the maximum execution time or the population size. The optimization process also supports pausing and resuming. The most important feature of this framework is the ability to export a model to code to be further modified by hand. TPOT cannot automatically process natural language inputs and also is not able to process categorical strings which must be integer encoded before passing in data. Also note that version 0.9 was used when testing this framework.

H2O is an ML framework similar to scikit-learn containing a collection of ML algorithms that execute on a server cluster accessible by a variety of interfaces and programming languages. H2O includes an automatic ML module that uses its own algorithms to build a pipeline. Configuration is limited to algorithm choice, stopping time, and degree of k-fold validation. It performs an exhaustive search over its feature engineering methods and model hyperparameters to optimize its pipelines. It currently supports imputation, one-hot encoding, and standardization for feature engineering and supports generalized linear models, basic deep learning models, gradient boosting machines, and dense random forests for its machine learning models. It supports two methods of hyperparameter optimization; Cartesian grid search and random grid search. The end result is an ensemble model that can be saved and reloaded into the H2O framework to be used in production systems. The primary drawback of H2O is its massive resource usage. During testing, this framework suffered multiple failures during long-running processes due to inadequate garbage collection.

Benchmarking automatic ML benchmarks is the topic of (Balaji and Allen, 2018),

8.38 Alibaba's X-engine

Alibaba runs worlds largest e-commerce platform for more than 600 million customers in China, with a *Gross Merchandise Value - GMV* or total sales dollar value exceeding \$768 billion in FY2018. Alibaba is aiming for a trillion dollars in 2020. In fact GMV transacted in the Alibaba Ecosystem was US \$1,239 billion for fiscal year 2021, which was mainly in China.

The following discussion is based in (Huang et al., 2019), (Li, 2019). Online e-commerce txns have three notable characteristics:

1. drastic (122 fold) increase of *Txns per Second - TPS* for major promotion events, such as single's day in China, which is equivalent to cyber Monday in US.
https://en.wikipedia.org/wiki/Singles%27_Day
<https://www.scmp.com/tech/big-tech/article/3110075/singles-day-annual-shopping-event-helps-raise-performance-chinas>
2. too many hot records that cannot fit in system buffers.
3. quick shift of the "temperature" from hot to warm to cold of different records due to the availability of promotions on different categories over different short time periods.
<http://www.cs.utah.edu/~lifeifei/papers/vldb-cloud-native.pdf>

The shared storage architecture is represented by AWS Aurora (Verbitski et al., 2017) and Alibaba's "Ultra-low latency and failure resilient distributed file system for shared storage cloud database (Cao et al., 2018).

Decoupling or separating storage from compute allows the exploitation of shared storage capabilities:

- (1) Compute and storage nodes can use different types of hardware and can be customized separately. Compute nodes do not need to consider the ratio of memory size to disk capacity, which is highly dependent on the application scenario and hard to predict.
- (2) Placing disks in a single storage pool reduce the risk of fragmentation, imbalance of disk usage among nodes, and space wastage. The capacity and throughput of a storage cluster can be easily scaled out transparently.
- (3) Since data are all stored on the storage cluster, there is no local persistent state on compute nodes, making it easier and faster to perform database migration. Data reliability can also be improved because of the data replication and other high availability features of the underlying distributed storage system.

Cloud database services benefit from this architecture as follows:

- (1) databases can build on a more secure and easily scalable environment based on virtualization techniques.
- (2) some key features of databases, such as multiple read-only instances and checkpoints could be enhanced with the support of back-end storage clusters, which provide fast I/O, data sharing, and snapshot. With rapid changes in data storage

technology current cloud platforms have trouble taking full advantage of the emerging hardware standards such as RDMA and NVMe SSD. Widely used open-source distributed file systems such as HDFS and Ceph are found to have much higher latency than local disks. With the latest PCIe SSDs the performance gap could even reach orders of magnitude. The performance of relational databases like MySQL running directly on these storage systems is significantly worse than that on local PCIe SSDs with the same CPU and memory configurations. To tackle this issue, cloud computing vendors, like AWS.

Google Cloud Platform and Alibaba Cloud, provide instance store, which uses a local SSD and high I/O VM instance to fulfill customers' needs of high performance databases.

<https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/InstanceStorage.html>

Running cloud databases on an instance store have several drawbacks.

- (1) Instance store has limited capacity, not suitable for a large database service.
- (2) It cannot survive underlying disk drive failure. Databases have to manage data replication by themselves for data reliability.
- (3) An instance store uses a general-purpose file system, such as ext4 or XFS. When using low I/O latency hardware like RDMA or PCIe SSD, the message-passing cost between kernel space and user space compromises the I/O throughput. Even worse, an instance store cannot support a shared-everything database cluster architecture, which is a key feature for an advanced cloud database service.

Described in (Cao et al., 2018) is the design and implementation of the PolarFS distributed file system, which provides ultra-low latency, high throughput and high availability by adopting the following mechanisms.

- (1) The POLARDB architecture takes advantage of emerging hardware such as RDMA and NVMe SSD, and implements a lightweight network stack and I/O stack in user space to avoid trapping into kernel and dealing with kernel locks.
- (2) PolarFS provides a POSIX-like file system API, which is compiled into the database process and replaces the file system interfaces provided by the OS, so that the whole I/O path can be kept in user space.
- (3) The I/O model of the PolarFS's data plane is also designed to eliminate locks and avoid context switches on the critical data path: unnecessary memory copies are eliminated, meanwhile DMA is heavily utilized to transfer data between main memory and RDMA NIC/NVMe disks. With all these features, the end-to-end latency of PolarFS has been reduced drastically, being quite close to that of local file system on SSD.

Distributed file systems deployed in the cloud production environment typically have thousands of machines. Within such a scale, failures caused by hardware or software bugs are common. Therefore, a consensus protocol is needed to ensure that all committed modifications will not get lost in corner cases, and replicas can always reach agreement and become bitwise identical.

The Paxos family of protocols is widely recognized for solving consensus. Raft, a variant of Paxos is much easier to understand and implement. Many distributed systems are developed based on Raft, but Raft applied to PolarFS was found to seriously impede the I/O scalability when using extra low latency hardware, e.g., NVMe SSD and RDMA whose latency is on the order of tens of microseconds. An enhanced parallel Raft consensus protocol was developed, which allows out-of-order log acknowledging, committing and applying, while letting PolarFS comply with traditional I/O semantics. With this protocol, parallel I/O concurrency of PolarFS has been improved significantly.

X-Engine relies on a write-optimized storage engine PolarDB, which utilizes a tiered storage architecture with the LSM tree to leverage hardware acceleration such as FPGA-accelerated compactions,

https://en.wikipedia.org/wiki/Field-programmable_gate_array

and a suite of optimizations including asynchronous writes in txns, multi-staged pipelines and incremental cache replacement during compactions. PolarDB relies on PolarFS, a very low latency and failure resilient distributed file system for shared storage cloud database.

1. PolarFS takes full advantage of emerging hardware such as RDMA - Remote Direct Memory Access

https://en.wikipedia.org/wiki/Remote_direct_memory_access

and NVMe - Non Volatile Memory express,

https://en.wikipedia.org/wiki/NVM_Express

a host controller interface and storage protocol created to accelerate data transfer between enterprise and client systems and SSDs over a computer's high-speed PCIe - Peripheral Component Interconnect express bus. These are implemented in a lightweight network and I/O stack in user space to avoid trapping into kernel and dealing with kernel locks.

2. PolarFS provides a *Portable Operating System Interface* - *POSIX*-like file system API, which is intended to be compiled into the database process and replace the file system interfaces provided by the OS, so that the whole I/O path can be kept in user space. POSIX is a family of standards specified by the IEEE Computer Society for maintaining compatibility

between operating systems.

<https://en.wikipedia.org/wiki/POSIX>

3. The I/O model of the PolarFS's data plane is designed to eliminate locks and avoid context switches on the critical data path: all unnecessary memory copies are also eliminated, meanwhile DMA is heavily utilized to transfer data between main memory and RDMA *Network-Interface Controller* - NIC/NVMe disks. With all these features, the end-to-end latency of PolarFS has been reduced drastically, being quite close to that of local file system on SSD.

Distributed file systems deployed in cloud production have thousands of machines. Within such a scale, failures caused by hardware or software bugs are common. A consensus protocol is used to ensure that all committed modifications will not get lost and replicas can always reach agreement and become bitwise identical.

Paxos and Raft consensus protocols

Consensus is the process of agreeing on one result among a group of participants, which becomes difficult when the participants or their communication medium may experience failures (Lamport, 2002), (Gray and Lamport, 2006). Raft described in (Ongaro and Ousterhout, 2014) is an efficient replacement for Paxos which produces results. Raft enhances understandability by separating the key elements of consensus, such as leader election, log replication, and safety, and it enforces a stronger degree of coherency to reduce the number of states that must be considered. Raft was observed to impede the I/O scalability of PolarFS on low latency hardware, such as NVMe SSD and RDMA with latency of tens of microseconds. Parallel Raft enhanced consensus protocol allowed out-of-order log acknowledging, committing and applying.

PolarDB on top of PolarFS is an RDBMS modified from AliSQL, which is recently available as a database service on Alibaba cloud computing platform. PolarDB follows the shared storage architecture, and supports multiple read-only instances, so that database nodes of PolarDB are divided into two types: a primary node and read only nodes. The primary node can handle both read and write queries, while the ReadOnly node only provides service to read queries. Both primary and ReadOnly nodes share redo log files and data files under the same database directory in PolarFS. Preliminary evaluation results compare PolarDB performance with performance at a local Ext4 file system (about the same) and MySQL on RDS cloud service, much better.

8.39 RocksDB with ultrafast data access

RocksDB is a high performance embedded DB for *Key-Value* - KV data. It is a fork of Google's LevelDB optimized to exploit many CPU cores and make efficient use of SSD using LSM trees for I/O bound workloads. RocksDB is used by web-scale enterprises including Facebook, Yahoo!, and LinkedIn. RocksDB is not distributed, no failover, not highly available.

<https://en.wikipedia.org/wiki/RocksDB>

<https://en.wikipedia.org/wiki/LevelDB>

Keys and values are arbitrary byte arrays. The keys are ordered within the key value store according to a user-specified comparator function. The following summary is based on a presentation by Dhruba Borthakur at Facebook.

https://www.youtube.com/watch?v=V_C-T5S-w8g

Application servers → network roundtrip (50 microsec) → DB server → disk access (10 millisecond)
replacing disk with SSD (100 microsec) or RAM (100 nanosecond), so latency dominated by network. Attaching storage to application servers eliminates network delay. List of pre-existing open source KV stores: Berkeley DB, SQLite, Kyoto TreeDB, Level DB

KV store: high performance, no txn log, fixed size keys

Random Reads/writes: LevelDB 129K/164K; Kyoto TreeDB 151K/88.5K; SQLite₃ 134K/9.86K

Changes made by RocksDB: 10X higher write rate, fewer stalls, 7X lower write amplification. Bloom filters for range scans, ability to avoid read-modify-write, optimization for flash or RAM (Bloom, 1970).

https://en.wikipedia.org/wiki/Bloom_filter

8.40 LightStore project at MIT

LightStore is a joint effort led by MIT and Daegu Gyeongbuk Institute of Science and Technology in S. Korea.

<http://news.mit.edu/2019/solid-state-drives-flash-storage-data-center-efficiency-0403>

<https://www.dgist.ac.kr/en/>

LightStore is a Flash store with low-power embedded processors, Gigabytes of DRAM, TBs of NAND flash, directly connectable to a datacenter network port (Chung et al., 2019). A large-scale distributed storage cluster can be formed simply by adding more LightStore nodes to the network.

Applications in a datacenter can take multiple software-defined views of LightStore stores via adapter layers translating KV. *Yahoo! Cloud Serving Benchmark*. - YCSB is used to generate block and file accesses to LightStore, which is estimated to be 2.0x power-efficient and 2.3x space-efficient than an x86-based all-flash array system of the same capacity. A summary of the YCSB benchmark is given in Table 8.20.

TABLE 8.20 A summary of *Yahoo! Cloud Serving Benchmark* - YCSB, P = Point, R = Range, U = Uniform, L = Latest (Cooper et al., 2010), Z = Zipfian.

	Load	A	B	C	D	E	F
R:W	0:100	50:50	95:5	100:0	95:5	95:5	50:50 (RMW)
Query type	P	P	P	P	P	R	P
Request distr.	U	Z	Z	Z	L	Z	Z

Experimental results show that:

- (1) the LightStore node performance is comparable to an x86 server with a single SSD;
- (2) a four-node LightStore cluster is 7.4X better per Joule than an x86 server with four SSDs.

8.41 PinK: high-speed in-storage key-value store with bounded tails

This effort similarly to Lightstore was led by Samsung's Datalab and MIT (Im et al., 2020).

<https://datalab.dgist.ac.kr/>

PinK high-speed in-storage KV store with bounded tails using an LSM-tree, because LSM-trees support a wider variety of operations and show better performance, especially for writes. The LSM-tree is difficult to implement in a KV-SSD, which typically use hash-based schemes. PinK is an implementation of an LSM-tree-based KV-SSD, which compared to a hash-based KV-SSD, reduces 99th percentile tail latency by 73%, improves average read latency by 42% and shows 37% higher throughput. The key idea is to avoid the use of Bloom filters and instead use a small amount of DRAM to keep/pin the top levels of the LSM-tree.

Observed problems can be summarized as follows:

- (1) LSM-tree exhibits higher average-latency because of multi-level search, and also exhibit unpredictable tail latency because of Bloom filters;
- (2) Bloom filters require lots of computational power to reconstruct after each compaction.
- (3) Level compaction (excluding Bloom filter reconstruction) requires a lot of computation and I/O bandwidth;
- (4) Compaction I/Os may trigger GC which in turn generates more I/Os, resulting in high write amplification.

Table 8.21 lists average, 99th, 99.9th, and 99.99th percentile of read latency of Hash, LSM-tree, and PinK. As expected, PinK showed better average latency with shorter tails compared to the others. Thanks to Bloom filters, LSM-tree performed fairly well compared to hash-based one, but had long tails. Hash suffered from long tails due to multiple flash I/Os caused by hash misses and collisions. YCSB-E showed longer latency than the others because it issued range queries that carry multiple GET() commands.

8.42 BlueDBM: an appliance for big data analytics

The BlueDBM project at MIT is a system architecture accelerating Big Data analytics, which incorporates a distributed Flash-based storage with in-store processing capability and a low-latency high-throughput networks (Jun et al., 2015).

BlueDBM is a joint effort with *Quanta Research Cambridge* - QRC, which is the North American R&D subsidiary of Taiwan's Quanta Computer Inc., world's largest manufacturer of laptop computers.

<https://qrclab.wordpress.com/about/>

TABLE 8.21 Comparison of average and tail latency (unit: μ s).

Org.	Percentile	A	B	C	D	E	F
Hash	Average	410	573	592	501	5,628	370
	99 th	2,180	2,550	2,900	3,030	17,550	1,850
	99.9 th	4,180	4,600	5,710	5,090	25,360	3,260
	99.99 th	9,430	9,340	9,830	7,530	34,420	5,180
LSM-tree	Average	302	395	722	294	3,142	329
	99 th	640	960	1,870	890	5,790	680
	99.9 th	1,700	1,630	2,680	1,370	8,800	1,890
	99.99 th	5,250	3,140	3,450	3,210	10,740	3,750
PinK	Average	236	290	732	161	3,027	248
	99 th	490	700	1,820	490	5,550	540
	99.9 th	670	1,040	2,180	720	6,640	800
	99.99 th	1,300	1,800	2,370	1,060	7,590	1,540

BlueDBM has demonstrated performance and power efficiency exceeding existing cluster architectures. When processing massive amounts of data, performance is often bound by the capacity of fast local DRAM. In cluster systems with more DRAM, the network software stack often becomes a bottleneck. BlueDBM mitigates these issues by providing an extremely fast access to a scalable network of flash-based storage devices and provides a platform for application-specific hardware accelerators on the datapath on each of the storage devices.

BlueDBM performance has been demonstrated using a wide array of applications including terasort, graph analytics and KV stores. A current goal is providing a simple programming interface to achieve maximum performance of the BlueDBM architecture.

8.43 WiSer highly available HTAP DBMS for IoT applications

In a distributed database txns invariably synchronize with a coordinator before commit. This model while enforcing serializability does not satisfy application's availability requirements. In the era of IoT this problem has become more severe due to the requirement of HTAP, where aggregation constraints need to be enforced as part of txns. Current systems bypass this problem by creating escrows, allowing occasional overshoots of constraints, which are handled via compensating application logic (O'Neil, 1986).

Wildfire-Serializable - WiSer database project at IBM/ARC sought to provide both *High Availability* - HA and consistency for HTAP within the same distributed DBMS without sharding, thus extending atomicity and durability to handle consistency (Barber et al., 2019). WiSer does no sharding and all nodes can run txns that modify the entire database, and yet enforces aggregation constraints. Both read-write conflicts and aggregation constraint violations are resolved lazily in the serialized data. WiSer also covers node joins and departures as database tables, thus simplifying correctness and failure handling.

WiSer splits database commit into two steps.

(1) PROMISE stage, after which the deltas for state changes of a txn X are persisted durably and the txn is symbolically committed. There are three "symbolic" entities in the delta resolved lazily:

- (a) the position of X's delta in the serializable order;
- (b) X's conflict status – whether the queries in the txn saw the latest state in serializable order;
- (c) X's constraint status, whether the database state up to X satisfies aggregation constraints.

These symbols capture the contingency semantics: if X confronts access conflicts or violates constrain will roll back and the application can specify one or more compensatory actions.

(2) SERIALIZE stage picks the position of X's delta in serializable order, which is done via consensus. This stage inherits the HA and liveness properties of that consensus protocol. The design of WiSer is presented and experiments suggest the approach has promise.

8.44 Raven RDBMS at Microsoft provides ML

Describes in (Karanasos et al., 2020) is an RDBMS undertaken at *Gray Systems Lab - GSL* in Madison, WI, which also does ML. The RDBMS provides a natural starting point. The RDBMS provides a natural starting point given its mature infrastructure for fast data access and processing, along with support for enterprise features, such as encryption, auditing, and high-availability. To take advantage of all of the above, we need to address a key concern: Can in-RDBMS scoring of ML models match (outperform?) the performance of dedicated frameworks? Raven is a system that leverages native integration of ML runtimes (such as ONNX Runtime) deep within SQL Server and a unified intermediate representation (IR) to enable advanced cross-optimizations between ML and database operators.

Open Neural Network eXchange - ONNX defines a common set of operators - the building blocks of ML and deep learning models - and a common file format to enable AI developers to use models with a variety of frameworks, tools, runtimes, and compilers.

<https://azure.microsoft.com/en-us/blog/onnx-runtime-is-now-open-source/>

Initial evaluation on real data demonstrates performance gains of up to $5.5 \times$ from the native integration of ML in SQL Server and up to $24 \times$ from cross-optimizations. An early preview of the ONNX Runtime integration is currently available with Azure's SQL Database Edge.

8.45 Machine Learning data platform - MLdp

A brief description of MLdp at Apple (Agrawal et al., 2019) is as follows. "MLdp offers a minimalist and flexible data model for all varieties of data, strong version management to guarantee reproducibility of ML experiments, and integration with major ML frameworks. MLdp also maintains the data provenance to help users track lineage and dependencies among data versions and models in their ML pipelines. In addition to table-stake features, such as security, availability and scalability, MLdp's internal design choices are strongly influenced by the goal to support rapid ML experiment iterations, which cycle through data discovery, data exploration, feature engineering, model training, model evaluation, and back to data discovery."

The ML platform supports the following:

1. a conceptual data model to describe both slowly changing and volatile data,
2. simple mechanisms for continuous data injection,
3. a domain specific language for effective data and feature engineering,
4. a hybrid data store suited for large data volumes of stable raw assets and high concurrent updates on volatile data, with physical data layout designs that support data versioning, and are optimized for incremental updates, delta tracking among versions, and ML training access patterns,
5. a data access interface integrated with ML frameworks, allowing ubiquitous data access both within data centers as well as on the edge,
6. explicit version management to ensure reproducible ML experiments,
7. tracking for data lineage and provenance,
8. a toolset and a programming interface for data exploration and discovery.

MLdp is compared with other related systems in Table 8.22.

TensorFlow was developed by the Google Brain team for internal use, but was released under an Apache License. It has a particular focus on training and inference of *Deep Neural Networks - DNNs*.

<https://en.wikipedia.org/wiki/TensorFlow>

Keras is an open-source library that provides a Python interface for AI NNs acting as an interface for the TensorFlow library.

<https://en.wikipedia.org/wiki/Keras>

PyTorch is an open source ML library based on the Torch library, used for computer vision and *Natural Language Processing - NLP* at Facebook's AI Research lab.

<https://en.wikipedia.org/wiki/PyTorch>

Michelangelo is Uber's ML platform.

<https://eng.uber.com/michelangelo/>

FBLeaRner Flow is Facebook's AI backbone.

<https://code.fb.com/core-data/introducing-fblearner-flow-facebook-s-ai-backbone>

ProveDB is described in (Miao et al., 2017).

Colaboratory or Colab is a product from Google Research, which allows anybody to write and execute arbitrary Python code through the browser, and is especially well suited to ML, data analysis and education. Colab is a hosted Jupyter

TABLE 8.22 ML platform eco-system (Agrawal et al., 2019), *1 Integrated with major ML frameworks, *2 Integrated with in-house execution clusters and elastic compute cloud, *3 integrated with an in-house solution.

Solution	Integrated data system	Model training	Experimentation	Evaluation	Deployment
SageMaker	No, e.g. S3, EBS, EFS	Yes	Yes	Yes	Yes
Azure ML	No, bring own data store	Yes	Yes	Yes	Yes
Cloud AI	No, e.g., Google Cloud Store	Yes	Yes	Yes	Yes
TensorFlow	N/A	Yes	No	No	Yes
Keras	N/A	Yes	No	No	No
PyTorch	N/A	Yes	No	No	No
Michelangelo	No (HDSS data lake, Cassandra)	Yes	Yes	Yes	Yes
FBLeamer Flow	N/A	No	Yes	No	Yes
Colaboratory	N/A	No	Yes	Yes	No
Databricks	N/A	No	Yes	Yes	No
MLdp	Yes	*1	*2	*3	*3

notebook service that requires no setup to use, while providing free access to computing resources including GPUs.

<https://research.google.com/colaboratory/faq.html>

8.46 Databricks

Databricks is a company founded by original creators of Apache Spark, which grew out of the AMPLab project at UCB that was involved in making Apache Spark, an open-source distributed computing framework built atop Scala. Scala is a strong statically typed general-purpose programming language which supports both object-oriented programming and functional programming. Designed to be concise, many of Scala's design decisions are aimed to address criticisms of Java.

[https://en.wikipedia.org/wiki/Scala_\(programming_language\)](https://en.wikipedia.org/wiki/Scala_(programming_language))

<https://en.wikipedia.org/wiki/Databricks>

The source of the following discussion is based on this.

<https://databricks.com/product/delta-lake-on-databricks>

Delta Lake is an open format storage layer that delivers reliability, security and performance on your data lake – for both streaming and batch operations. By replacing data silos with a single home for structured, semi-structured and unstructured data, Delta Lake is the foundation of a cost-effective, highly scalable lakehouse.

High-quality, reliable data is delivered from a reliable single source of truth for all of your data, including real-time streams, so your data teams are always working with the most current data. With support for ACID txns and schema enforcement, Delta Lake provides the reliability that traditional data lakes lack. This enables you to scale reliable data insights throughout the organization and run analytics and other data projects directly on your data lake – for up to 50x faster time-to-insight.

Lightning-fast performance: Delta Lake delivers massive scale and speed. And because it's optimized with performance features like indexing, Delta Lake customers have seen ETL workloads execute up to 48% faster.

Open and agile: All data in Delta Lake is stored in open Apache Parquet format, allowing data to be read by any compatible reader. APIs are open and compatible with Apache Spark. With Delta Lake on Databricks, you have access to a vast open source ecosystem and avoid data lock-in from proprietary formats.

Automated and trusted data engineering: Simplify data engineering with Delta Live Tables - an easy way to build and manage data pipelines for fresh, high-quality data on Delta Lake. It helps data engineering teams by simplifying ETL development and management with declarative pipeline development, improved data reliability and cloud-scale production operations to help build the lakehouse foundation.

Security and governance at scale: Delta Lake reduces risk by enabling fine-grained access controls for data governance, functionality typically not possible with data lakes. You can quickly and accurately update data in your data lake to comply with regulations like GDPR and maintain better data governance through audit logging. These capabilities are natively integrated and enhanced on Databricks as part of the Unity Catalog, the first multi-cloud data catalog for the Lakehouse.

8.47 Fungible - a new storage architecture for big data

Fungible founded by Pradeep Sidhu and Bertrand Serlet (Apple) is a pioneer in data-centric computing:

https://en.wikipedia.org/wiki/Fungible_Inc.

Sidhu was a founder of Juniper Networks in 1996, which ranks 4th after Cisco, Huawei, Arista.

<https://www.networkworld.com/article/3211410/the-10-most-powerful-companies-in-enterprise-networking.html>

Ranking of networking companies

1-**Cisco Systems, Inc.** founded by Len Bosack and Sandy Lerner in 1984 in San Francisco develops, manufactures and sells networking hardware, software, telecommunications equipment and other high-technology services and products. Cisco's revenue in 2019 was \$51.70 billion.

https://en.wikipedia.org/wiki/Cisco_Systems

In 1991 Bosack founded XKL which does optical networking.

<https://en.wikipedia.org/wiki/XKL>

2-**Huawei Technologies Co., Ltd.** was founded in 1987 by Ren Zhengfei, a high-ranking officer in *People's Liberation Army - PLA* in Shenzhen China. It provides telecommunications equipment and consumer electronics like smartphones. Huawei revenue in 2019 was \$123 billion.

Huawei *Chief Financial Officer - CFO* Meng Wanzhou is being held in Canada for breaking US export sanctions to Iran at this time (7/1/21).

<https://en.wikipedia.org/wiki/Huawei>

3-**Arista Networks** was founded in 2004 in Santa Clara, CA by Andy Bechtolsheim (SUN cofounder), David Cheriton (Stanford U.), and Kenneth Duda. The company designs and sells multilayer network switches to deliver *Software Defined Networking - SDN* solutions for large datacenter, cloud computing, high-performance computing, and high-frequency trading environments. These products include an array of 10/25/40/50/100 GB Ethernet low-latency cut-through switches, including the 7124SX, which remained the fastest switch using *Small Form-factor Pluggable - SFP+* optics through September 2012, with its sub-500 ns latency, and the 7500 series, Arista's modular 10G/40G/100Gbit/s switch. Arista's has its own Linux-based network *Extensible Operating System - EOS*. Its revenue in 2019 was \$2.27 B.

https://en.wikipedia.org/wiki/Arista_Networks

4-**Juniper Networks, Inc.** headquartered in Sunnyvale, CA, develops and markets networking products, including routers, switches, network management software, network security products, and software-defined networking technology. Its revenue in 2019 was \$4.445 Billion.

https://en.wikipedia.org/wiki/Juniper_Networks

<https://www.juniper.net>

The following goals are set in the following Fungible white paper (Siman-Tov, 2019).

1. Reexamine the limitations of existing storage architectures.
2. Discuss the benefits of disaggregated storage versus *Direct Attached Storage - DAS*, connecting aggregates of computer and storage servers by a multilayer CLOS switch due to Charles Clos in 1938, but such networks are prone to congestion (Duato et al., 2003), (Dally and Towles, 2004), (Rojas-Cessa, 2017).
https://en.wikipedia.org/wiki/Clos_network
Benes networks are a subtype of CLOS networks.
Remote Data memory Access - RDMA networking has also limited scalability.
https://en.wikipedia.org/wiki/Remote_direct_memory_access
3. Describe *Data Processing Units - DPUs* invented by Fungible. which serve as high performance controllers for SSDs offloading storage functions from the CPU.
<https://www.fungible.com/blog/the-cpu-has-served-us-extremely-well-for-decades/>

Replacing HDDs with Flash SSDs does not result in the expected performance gain when SATA/SAS protocols developed for HDDs are utilized for connectivity. Higher performance is achieved with the introduction of NVMe, which results in faster, low latency, parallel paths to media. Due to inefficiencies in the storage stack flash drives rated 750 KIOPS max out at 100 KIOPS.

CPUs are overloaded running user applications such as ML and very large datasets for MapReduce processing over numerous storage servers, plus datacenter services such as compression, encryption, erasure coding, etc. The problem is exacerbated by the plateauing of the Moore's law.

<https://www.technologyreview.com/s/615226/were-not-prepared-for-the-end-of-moores-law/>

<https://www.cnet.com/news/how-intel-will-keep-moores-law-cranking-for-years-to-come/>

This has led to general purpose microprocessors giving way to scale-out of heterogeneous programmable and fixed function compute elements (Sidhu, 2019).

Microservices based application architectures decompose each application into a set of services, which are written, maintained, and operated independent of others (Surianarayanan et al., 2002).

<https://en.wikipedia.org/wiki/Microservices>

Each microservice is implemented by a set of identical servers running over multiple different compute nodes, while interacting over common storage space accessing high demand contents simultaneously.

An important consequence of microservices architecture is a dramatic increase in network bandwidth between servers in a given data center.

With the advent of big-data workloads are becoming data-centric. It is beneficial to run simple algorithms on large datasets rather than complex algorithms on small amounts of data. A data-centric world is justified by the fact that moving data and performing lightweight computations are as important as traditional arithmetic and logic operations. A data-centric workload can be specified as follows:

- Work arrives in the form of packets to a switched network interface such as Ethernet or PCIe.
- The packets on a given interface belong to a large number of flows, which allows multiplexing.
- The computation performed by data centric workloads is stateful in that there is a need to read/write the large state held in external memory associated with the packet.
- The IO to arithmetic ratio is medium to high.

Today's data centers do not cope well with data-centric workloads.

1. The multiplexing caused by a data-centric workload severely impacts the performance of a compute-centric workload.
2. Network elements cannot do stateful computations at high speeds.

Stateful computations are functions in the form $S \rightarrow (T, S)$, i.e., they take some state and return a value as well as an updated state. Stateful computations can be composed monadically (with a single argument), to reduce the syntactic burden of passing the state from one computation to the next.

Fungible's DPUs execute data-centric workloads efficiently as follows:

1. The DPU performs data-centric computations an order of magnitude more efficiently than general purpose CPUs using three techniques.
 - a. an architecture for handling heavily multiplexed workloads.
 - b. specialized accelerators for most frequently executed operations.
 - c. efficiently handling dataflow while applying lightweight computations.
 - d. The DPU provides IP over Ethernet (IPoE) on the network side and PCIe interface on the local bus connecting compute and storage.
2. The DPU serves as the end-point of stateful network connections.

The Fungible Data Processor and its placement is shown below.

<https://www.servethehome.com/fungible-f1-dpu-for-distributed-compute/>

<https://www.servethehome.com/wp-content/uploads/2020/08/Hot-Chips-32-Fungible-DPU-Sit.jpg>

A graph given in this web page shows that the speed of the Ethernet is overtaking CPU speeds.

<https://www.forbes.com/sites/forbestechcouncil/2019/11/12/data-centers-need-to-become-data-centric/#1487f3246a3d>

Storage pooling is achieved via *Software Defined Storage - SDS*.

https://en.wikipedia.org/wiki/Software-defined_storage

<https://www.computerweekly.com/feature/Software-defined-datacentres-demystified>

Decoupling storage from compute allows more uniform latencies and simplifies data placement. Traditionally compute servers and storage arrays were separate appliances connected via network switches. Hyperconverged infrastructure brought compute and storage into a single server Storage is locally attached to compute elements as in DAS. Composable disaggregated infrastructure used software concepts to enable composability of disaggregated architecture. The resource sharing principle is enunciated in Section 5.1 in (Kleinrock, 1976) and is based on (Kleinrock, 1974).

Summary of Forbes 20/2/12 article by Jai Menon

Menon (IBM, Dell/EMC, Cloudistics) who recently joined Fungible as its chief scientist wrote the following.

<https://www.forbes.com/sites/forbestechcouncil/2020/02/12/new-scale-out-storage-for-the-cloud-world/#4254dfc8768d>

- Workload diversity: Cloud data centers run many diverse workloads from many tenants simultaneously. Therefore, the storage system needs to be able to support a broad range of performance and durability requirements.
- *Stock Keeping Unit - SKU* or distinct item for sale proliferation results from workload diversity and each workload needs a server with a possibly different combination of core type and count, memory size, local and remote storage, GPU count, and network connectivity. For example, AWS supports more than 200 instance types.
- Unpredictable performance: Many cloud apps are unusable if latency is unpredictable. They expect consistent latency even when networks are highly utilized.
- Expensive redundancy: Redundancy, especially of high-performance data, is typically provided using replication, which increases cost. Other approaches, such as RC - Erasure Coding across the network, have inadequate performance.
- Stranded resources: The use of local storage can lead to stranded resources when one server does not have enough local storage and another server has more than it needs.
- Inefficient I/O processing: Traditional x86 servers are inefficient at handling I/O. Half the cores of a typical server are consumed doing I/O; only the rest are available for customer workloads.

That storage system must be able to scale out has several implications:

- (1) Unlike enterprise storage systems, individual storage nodes are not designed to be highly available. Instead, the system is designed to handle the failure of one or more nodes that make up the storage system. Recovery from Node and SSD failures should be very fast.
- (2) The data protection coding scheme must operate across storage nodes with high performance. Few enterprise storage systems have this capability. Fewer still have acceptable performance when EC is turned on. Most enterprise and cloud storage systems turn off EC, or do EC in the background, in order to avoid performance penalties.
- (3) The storage must scale to thousands of nodes. This requires the storage system to separate the data plane and the control plane. This is a concept known in networking as *Software Defined Network - SDN*, but it is new to storage. We are not aware of any enterprise storage system that separates the data and control planes.

Given the above trends and challenges, a new class of storage is needed for cloud data centers.

This storage system supports NVMeoF, specifically using IP over Ethernet. Unlike the current generation of enterprise or cloud storage, this can make remote block storage as fast as local storage (file services can be built on top of fast block storage). Using remote storage to emulate local storage alleviates the need to support server SKUs with different local storage options. It also eliminates the stranded resource problem, as all resources are pooled remotely and available to be assigned to any server.

For consistent performance, the network and the storage system must support low tail latency, even when the network scales to hundreds of racks and is highly utilized. This requires the use of a new class of transport protocols that can support low tail latency and also the use of emerging features in SSDs such as predictable latency mode. Current enterprise and cloud storage systems do not maintain low tail latency for large or highly congested networks.

The storage system needs to support a diversity of workloads and multitenancy. For every volume exposed by the storage system, it should be possible to separately specify the level of durability needed. Similarly, a minimum performance guarantee must be supported for each volume. Finally, it should be possible for each volume to be encrypted with a different key.

In contrast, most current enterprise and cloud storage systems support one level of durability for all volumes.

The storage system must be able to support line-rate encryption and compression. With the rising level of security threats in a multitenant cloud, both encryption on the wire and at rest must be supported. Typical enterprise systems cannot support line-rate encryption at hundreds of Gbps. Furthermore, neither enterprise nor cloud storage systems typically support in-line compression at high speed.

Due to x86 limitations in handling data-centric workloads, newer I/O processors may need to be deployed to build these storage systems.

“The term *Software-Defined Data Center - SDDC* rose to prominence during VMworld 2012. A SDDC an IT facility where the elements of the infrastructure - networking, storage, CPU and security - are virtualised and delivered as a service. The provisioning and operation of the entire infrastructure is entirely automated by software.”

Some of the words mentioned above are explained below.

<https://searchconvergedinfrastructure.techtarget.com/feature/Words-to-go-Composable-disaggregated-infrastructure>

8.48 Network requirements for resource disaggregation

According to (Gao et al., 2016) traditional datacenters are designed as a collection of servers, each of which tightly couples the resources required for computing tasks. A recent paradigm shift to *Disaggregated DataCenter - DDC* architecture is to build as a standalone resource blade and interconnected using a network fabric. A key enabling (or blocking) factor for disaggregation will be the network – to support good application-level performance it becomes critical that the network fabric provides low latency communication even under the increased traffic load that disaggregation introduces. A workload-driven approach is used to derive the minimum latency and bandwidth requirements that the network in disaggregated datacenters must provide to avoid degrading application-level performance and explore the feasibility of meeting these requirements with existing system designs and commodity networking technology. Typical latency and peak bandwidth requirements within a traditional server are given in Table 8.23.

TABLE 8.23 Typical latency and peak bandwidth requirements within a traditional server.

Communication	Latency (ns)	Bandwidth (GBps)
CPU-CPU	10	500
CPU-memory	20	500
CPU-SSD	10^4	5
CPU-HDD	10^6	1

TABLE 8.24 Applications, workloads, systems and datasets used in the study are given based on references in brackets.

Application Domain	Application	System	Dataset
Off-disk Batch	WordCount	Hadoop	Wikipedia edit history [27]
Off-disk Batch	Sort	Hadoop	Sort benchmark generator
Graph Processing	Collaborative Filtering	GraphLab	Netflix movie ratings [22]
Point Queries	Key-value store	Memcached	YCSB
Streaming Queries	Stream WordCount	Spark Streaming	Wikipedia edit history [27]
In-memory Batch	WordCount	Spark	Wikipedia edit history [27]
In-memory Batch	Sort	Spark	Sort benchmark generator
Parallel Dataflow	Pagerank	Timely Dataflow	Friendster Social Network [9]
In-memory Batch	SQL	Spark SQL	Big Data Benchmark [6]
Point Queries	Key-value store	HERD	YCSB

Table 8.24 divides applications into two categories (the top and bottom five) based on the network latency and bandwidth needed to achieve a low performance penalty. Some of the applications in the table are as follows:

[6] Berkeley Big Data Benchmark. <https://amplab.cs.berkeley.edu/benchmark/>

[9] Friendster Social Network. <https://snap.stanford.edu/data/com-Friendster.html>

[22] Netflix Rating Trace. <http://www.select.cs.cmu.edu/code/graphlab/datasets/>

[27] Wikipedia Dump. <https://dumps.wikimedia.org/>

Class A applications require a network with an end-to-end latency of 5 μ s and bandwidth of 40 Gbps to maintain an average performance penalty under 5%.

Class B applications require network latencies of 3 μ s and 40-100 Gbps bandwidth to maintain an average performance penalty under 8%.

The feasibility of meeting the requirements identified above.

Local memory: The initial estimate is between 20-30% local memory, which corresponds to between 1.50-2.25 GB/core. We look to existing hardware prototypes for validation of this requirement. The FireBox prototype targets 128 GB of local memory shared by 100 cores leading to 1.28 GB/core, while the analysis in (Lim et al., 2009) uses 1.5 GB/core. Further, (Lim et al., 2012) at HP Labs also indicates 25% local memory as a desirable setting, and HP's "The Machine" uses an even larger fraction of local memory: 87%. Thus we conclude that our requirement on local memory is compatible with demonstrated hardware prototypes.

Network bandwidth: The bandwidth requirements are easily met: 40 Gbps is available today in commodity datacenter switches and server NICs; in fact, even 100 Gbps switches and NICs are available. Thus, ignoring the potential effects of congestion, providing the network bandwidth needed for disaggregation should pose no problem. Moreover, this should continue to be the case in the future because the trend in link bandwidths currently exceeds that in number of cores.

Network latency: The picture is less clear with respect to latency. The various components of network latency and whether they can be accommodated in target budget of 3 μ s (for Class B apps) to 5 μ s (for Class A apps). Listed in the table are the six components of the end-to-end latency. Estimates are based on the following common assumptions about existing datacenter networks:

- (1) the one-way path between servers in different racks crosses three switches (two ToR and one fabric switch) while that between servers in the same rack crosses a single ToR switch,
- (2) inter-rack distances of 40 m and intra-rack distances of 4 m with a propagation speed of 5 ns/m,
- (3) cut-through switches.

The round-trip latency includes the software overheads associated with moving the page to/from the NIC at both the sending and receiving endpoints (hence 2x the OS and data copy overheads), 6 switch traversals, 4 link traversals in each direction including two intra-rack and two cross-rack, and the transmission time for a 4 KB page.

The network introduces three unavoidable latency overheads: (i) data transmission time, (ii) propagation delay; and (iii) switching delay. These components contribute to roughly 3.14 μ s across racks and 1.38 μ s within a rack. In contrast, the network software at the endpoints is a significant contributor to the end-to-end latency! Recent work reports a round-trip kernel processing time of 950 ns measured on a 2.93 GHz Intel CPU running FreeBSD. With these estimates, the network software contributes roughly 3.9 μ s latency — this represents 55% of the end-to-end latency in our baseline inter-rack scenario and 73% in our baseline intra-rack scenario. The end-to-end latencies we estimated in our baseline scenarios (whether inter- or intra-rack) fail to meet our target latencies for either Class B or Class A applications. Hence, we consider potential optimizations and technologies that can reduce these latencies. Two technologies show promise: RDMA and integrated NICs.

Using RDMA: allows bypassing the packet processing in the kernel, thus eliminating the OS overheads. Using RDMA: Infiniband or Omnipath.

http://www.infinibanda.org/content/pages.php?pg=about_us_infiniband

<http://www.intel.com/content/www/us/en/high-performance-computing-fabrics/omni-path-architecture-fabric-overview.html>

a reduced end-to-end latency of 5.14 μ s across racks 3.38 μ s within a rack.

Using NIC integration: Integration of NIC functions closer to the CPU would reduce the overheads associated with copying data to/from the NIC, e.g., to half a microsecond. The use of RDMA together with NIC integration reduces the end-to-end latency to 4.14 μ s across racks; within a rack, this further reduces down to 2.38 μ s.

Takeaways from the analysis:

- With current networking technologies the overhead of network software is the key barrier. Technologies such as RDMA and integrated NICs that eliminate some of these overheads offer promise: reducing end-to-end latencies to 4.14 μ s between racks and 2.38 μ s within a rack. Demonstrating such latencies in a working prototype remains an important topic for future exploration.
- Even assuming RDMA and NIC integration, the end-to-end latency across racks (4.14 μ s) meets target latency only for Class A, but not Class B, applications. The target latency for Class B apps is only met by the end-to-end latency within a rack. Thus, Class B jobs will have to be scheduled within a single or nearby racks. That is, while Class A jobs can be scheduled at blades distributed across the datacenter, Class B jobs will need to be scheduled within a rack. The design and evaluation of such schedulers remains an open topic for future research.
- While new network hardware such as high-bandwidth links (e.g., 100 Gbps or 1 Tbps as in and high-radix switches (e.g., 1000 radix switch [31]) are certainly useful, they optimize a relatively small piece of the overall latency in our baseline scenario technologies. All-optical switches also provide both potentially negligible switching delay and high bandwidth. Once the benefits of RDMA and NIC integration, then the contribution of new links and switches could bring

even the cross-rack latency to within our 3 μ s target for Class B applications, enabling true datacenter-scale disaggregation; e.g., using 100 Gbps links reduces the end-to-end latency to 3.59 μ s between racks, close to our 3 μ s.

- Managing network congestion to achieve zero or close-to-zero queuing within the network will be essential; e.g., a packet that is delayed such that it is queued behind (say) 4 packets will accumulate an additional delay of 4-0.82 μ s! Indeed, reducing such transmission delays may be the reason to adopt high-speed links.

Simulation Setup. Simulated is a topology of 9 racks, 144 total endpoints and a full bisection bandwidth Clos topology with 36 KB buffers per port; our two changes from prior work are to use 40 Gbps or 100 Gbps access links, and setting propagation and switching delays as discussed in Table 4 with RDMA and NIC integration. We map the 5 EC2-node cluster into a disaggregated cluster with 15 blades: 5 each of compute, memory and disk. Then, we extract the flowsize and inter-arrival time distribution for each endpoint pair in the 15 blades disaggregated cluster, and generate traffic using the distributions. Finally, we embed the multiple disaggregated clusters into the 144-endpoint datacenter with both rack-scale and datacenter-scale disaggregation, where communicating nodes are constrained to be within a rack and unconstrained, respectively. We evaluate five protocols; in each case, we set protocol-specific parameters following the default settings but adapted to our bandwidth-delay product as recommended.

TCP with an initial congestion window of 2.

DCTCP leverages ECN for enhanced performance in datacenter contexts.

pFabric approximates *Shortest-Job-First* - *SJF* scheduling in a network context using switch support to prioritize flows with a smaller remaining flow size. The initial congestion window is set to 12 packets and a retransmission timeout of 45 μ s.

pHost emulates pFabric's behavior but using only scheduling at the end hosts and hence allows the use of commodity switches. We set pHost to have a free token limit of 8 packets and a retransmission timeout of 9.5 μ s.

Fastpass is a datacenter network framework that aims for high utilization with zero queueing (Perry et al., 2014). It provides low median and tail latencies for packets, high data rates between machines, and flexible network resource allocation policies. The key idea in Fastpass is fine-grained control over packet transmission times and network paths.

<http://fastpass.mit.edu/>

8.49 Deep learning and associated hardware

Deep learning - *DL* is a subarea of *Machine Learning* - *ML*, itself a subarea of *Artificial Intelligence* - *AI*.

https://en.wikipedia.org/wiki/Deep_learning

There are numerous books on deep learning which are based on *Artificial Neural Networks* - *ANNs*, see e.g., (Goodfellow et al., 2016).

https://en.wikipedia.org/wiki/Artificial_neural_network

<https://en.wikipedia.org/wiki/Backpropagation> A recent publication is: "The Principles of Deep Learning Theory: An Effective Theory Approach to Understanding Neural Networks" by Daniel A. Roberts and Sho Yaida, 2021.

<https://deeplearningtheory.com/PDLT.pdf>

Deep learning allows computational models that are composed of multiple processing layers to learn representations of data with multiple levels of abstraction. These methods have dramatically improved the state-of-the-art in speech recognition, visual object recognition, object detection and many other domains such as drug discovery and genomics. Deep learning discovers intricate structure in large data sets by using the backpropagation algorithm to indicate how a machine should change its internal parameters that are used to compute the representation in each layer from the representation in the previous layer. Deep convolutional nets have brought about breakthroughs in processing images, video, speech and audio, whereas recurrent nets have shone light on sequential data such as text and speech (LeCun et al., 2015).

ANNs mimic brain-like functions based on simple artificial neuron, which process a weighted sum of the inputs from other neurons. Learning involves adjusting the weights and thresholds of the NN to improve the accuracy of the result. The weights are nonlinear functions such as *Rectified Linear Unit* - *ReLU* $\max(0, z)$. Examples of transfer functions are:

S-shaped sigmoid $\Phi(z) = 1/(1 + e^{-z})$,

hyperbolic tangent $\tanh(x) = (e^{2x} - 1)/(e^{2x} + 1)$.

<https://towardsdatascience.com/activation-functions-neural-networks-1cbd9f8d91d6>

These artificial neurons are collected into layers, with the outputs of one layer becoming the inputs of the next layer.

The two phases of NN are training or learning and inference or prediction and they refer to development versus production. The developer chooses the number of layers and the type of NN, and training determines the weights. Training uses FLP numbers, which is one reason GPUs are popular. Quantization transforms FLP numbers into 8 bits integers, which

are usually good enough for inference. 8-bit integer multipliers use 6X less energy and also 6X less area than IEEE 754 16-bit FLP multipliers. The advantage for integer addition is 13X in energy and 38X in area according to William Dally's presentation.

“High-Performance Hardware for Machine Learning”

https://berkeley-deep-learning.github.io/cs294-dl-f16/slides/DL_HW_Berkeley_0916.pdf

NNs consist of interconnected nodes with an input level, hidden (multiple levels), and output level.

<https://www.mathworks.com/discovery/deep-learning.html>

Three kinds of NNs are popular:

Multi-Layer Perceptrons - MLP: Each new layer is a set of nonlinear functions of weighted sum of all outputs *Fully Connected - FC* from a prior one, which reuses the weights.

Convolutional Neural Networks - CNN: Each ensuing layer is a set of nonlinear functions of weighted sums of spatially nearby subsets of outputs from the prior layer, which also reuses the weights.

Brief review of CNNs

CNNs convolve learned features with input data, and uses 2D convolutional layers suited to processing 2D data such as images. CNNs eliminate the need for manual feature extraction by extracting features directly from images. The relevant features are not pretrained; they are learned while the network trains on a collection of images. This automated feature extraction makes DL models highly accurate for computer vision tasks such as object classification.

CNNs learn to detect different features of an image using tens or hundreds of hidden layers, which increase the complexity of the learned image features, e.g., first layer detects edges and the last learns how to detect more complex shapes.

With a DL workflow, relevant features are automatically extracted from images. In addition, DL performs “end-to-end learning” – where a network is given raw data and a task to perform, such as classification, and it learns how to do this automatically.

Another key difference is DL algorithms scale with data. Shallow learning refers to ML methods that plateau at a certain level of performance when you add more examples and training data to the network.

A key advantage of DL networks is that they often continue to improve as the data size increases. It takes a few thousand images to get reliable results, while a high-performance GPU takes less time to analyze all those images.

Training a DL model can take a long time from days to weeks. Using MATLAB with a GPU reduces the time required to train a network and can cut the training time for an image classification problem from days down to hours.

Recurrent Neural Networks - RNN: Each subsequent layer is a collection of nonlinear functions of weighted sums of outputs and the previous state. The most popular RNN is *Long Short-Term Memory - LSTM*,

https://en.wikipedia.org/wiki/Long_short-term_memory

where the challenge is what to forget and what to pass on as state to the next layer. The weights are reused across time steps.

Two examples of DNNs:

Image processing at Microsoft

2012: 8 layers, 1.4 GFLOP, $\approx 16\%$ error.

2015: 16X model, 152 layers, 22.6 GFLOP, $\approx 3.5\%$ error.

Speech recognition at Baidu

2014: 80 GFLOP, 7000 hours of data, $\approx 8\%$.

2015: 465 GFLOP, 12,000 hours, $\approx 5\%$.

The following sections discuss GPUs, FPGAs, and ASICs accelerated databases.

8.50 GPU accelerated database systems

The following discussing is based on (Breß et al., 2014b), whose contents are summarized in Table 8.25.

TABLE 8.25 GPU systems in academia in chronological order.

System	Institute	Year	Open Source	Ref.
CoGaDB	U. Magdeburg	2013	yes	(Breß et al., 2014b), (Breß et al., 2013)
GPUDB	Ohio State U.	2013	yes	(Yuan et al., 2013)
GPUQP	HKUST	2007	yes	(Rabl et al., 2013)
GPUDx	Nanyang Tech. U.	2011	no	(He and Yu, 2011)
MapD	MIT	2013	no	(Mostak, 2013)
Ocelot	Tech. U. Berlin	2013	yes	(Heimel et al., 2013)
OmniDB	Nanyang Tech. U.	2013	yes	(Zhang et al., 2013a)
Virginian	xNEC Labs USA	2012	yes	(Bakkum and Skadron, 2010)

CoGaDB

CoGaDB is a column-oriented GPU-accelerated DBMS which focuses on GPU-aware query optimization to achieve efficient co-processor utilization during query processing.

Storage System: CoGaDB persists data on disk, but loads the complete database into main memory on startup. If the database is larger than the main memory, CoGaDB relies on the OS's virtual memory management to swap the least recently used memory pages on disk.

Storage Model: CoGaDB stores data in data structures optimized for in-memory databases, hence, it stores the data column-wise and compresses VARCHAR columns using dictionary encoding. Furthermore, the data has the same format when stored in the CPU's or the GPU's memory.

Processing Model: CoGaDB uses the operator-at-a-time bulk processing model to make efficient use of the memory hierarchy. This is the basis for efficient query processing using all processing resources.

Query Placement & Optimization: CoGaDB uses the *Hybrid Query Processing Engine - HyPE* as physical optimizer to optimize physical query plans to increase inter-device parallelism by keeping track of the load condition on all (co-)processors, CPU vs GPU).

GPUDB

GPUDB was developed at OSU in order to study the performance behavior of OLAP queries on GPUs, GPUDB keeps the database in the CPU's main memory to avoid the HDD bottleneck. A crucial optimization for main-memory DBMS with respect to GPU accelerated execution: In case data is stored in pinned host memory, query execution times can significantly improve, and speedups up to 6.5x for certain queries of the *Star Schema Benchmark - SSB* (Rabl et al., 2013).

Storage Model: GPUDB stores the data column-wise because GPUDB is optimized for warehousing workloads. Additionally, GPUDB supports common compression techniques such as *Run Length Encoding - RLE*, bit encoding, and dictionary encoding) to decrease the impact of the PCIe bottleneck and to accelerate data processing.

Processing Model: GPUDB uses a block-oriented or vectorized processing model: Blocks are kept in GPU RAM until they are completely processed. Thus, the PCIe bottleneck can be further reduced by overlapping data transfers with computation. For certain queries speedups up to 2.5x compared to no overlapping of processing and data transfers were observed.

GPUDB compiles queries to driver programs, which execute a query by calling preimplemented GPU operators. Hence, GPUDB executes all queries on the GPU and the CPU performs only dispatcher and post processing tasks (i.e., the CPU is used less than 10% of the time during processing SSB queries).

Query Placement & Optimization: GPUDB has no support for executing queries on the CPU and GPU in parallel.

GPUQP

GPUQP is a relational query processing system, which stores data in-memory and uses the GPU to accelerate query processing (He et al., 2009). Each relational operator can be executed on the CPU or the GPU.

Storage System: GPUQP supports in-memory and disk-based processing. There is an attempt to keep data cached in GPU memory, but no detail is provided about the data placement strategy.

Storage Model: Use is made of columnar storage and query processing, which fits the hardware capabilities of modern CPUs and GPUs.

Processing Model: Basic processing strategy is operator-at-a-time bulk processing. GPUQP is also capable of partitioning data for one operator and execute the operator on the CPU and the GPU concurrently, but the impact on the overall performance is small.

Query Placement & Optimization: A System R-style optimizer with an analytical cost model to select the cheapest query plan. For each operator, GPUQP allocates either the CPU, the GPU, or both processors (partitioned execution). The query optimizer splits a query plan to multiple sub-plans containing at most ten operators. For each sub-query, all possible plans are created and the cheapest sub-plan is selected. Finally, GPUQP combines the sub-plans to a final physical query plan. The focus is on optimizing single queries and do not discuss multi-query optimization. Load-aware query scheduling and multiple GPUs are not considered.

GPUDx

GPUDx is the only system in this survey that supports txn processing on a GPU. 2PL, partition-based execution and k-set-based execution, The major finding of GPUDx is that locking-based protocols do not work well on GPUs. Instead, lock-free protocols such as partition-based execution or k-set should be used.

MapD

MapD is a data processing and visualization engine, combining traditional query processing capabilities of DBMSs with advanced analytic and visualization functionality (Mostak, 2013). One application scenario is the visualization of twitter messages on a map in which the geographical position of tweets is shown and visualized as heat map.

<https://www.omnisci.com/demos/tweetmap>

Storage System: The data processing component of MapD is a relational DBMS, which can handle data volumes that do not fit the main memory. MapD also tries to keep as much data in-memory as possible to avoid disk accesses.

Storage Model: MapD stores data in a columnar layout, and further partitions columns into chunks. A chunk is the basic unit of MapD's memory manager. The basic processing model of MapD is processing one operator-at-a-time. Due to the partitioning of data into chunks, it is also possible to process on a per-chunk basis. Hence, MapD is capable of applying block-oriented processing.

Model: MapD processes queries by compiling a query to executable code for the CPU and GPU.

Query Placement & Optimization: The optimizer tries to split a query plan in parts, and processes each part on the most suitable processing device, e.g., text search using an index on the CPU and table scans on the GPU. MapD does not assume that an input data set fits in GPU RAM, and it applies a streaming mechanism for data processing.

Ocelot

Ocelot is an OpenCL extension of MonetDB, enabling operator execution on any OpenCL capable device, including CPUs and GPUs (Breß et al., 2014a).

<http://goo.gl/GHeUv>

<https://en.wikipedia.org/wiki/OpenCL>

Storage System is built on top of the in-memory model of MonetDB. Input data is automatically transferred from MonetDB to the GPU when needed by an operator. In order to avoid expensive transfers, operator results are typically kept on the GPU. They are only returned at the end of a query, or if the device memory is too filled to fulfill requests. Additionally, Ocelot implements a device cache to keep relevant input data available on the GPU.

Storage Model: Data is kept column-wise in *Binary Association Tables* - BATs. Each BAT consists of two columns: One (optional) head storing object identifiers, and one (mandatory) tail storing the actual values.

Processing Model: Ocelot inherits the operator-at-a-time bulk processing model of MonetDB, but extends it by introducing lazy evaluation and making heavy use of the OpenCL event model to forward operator dependency information to the GPU. This allows the OpenCL driver to automatically interleave and reorder operations, e.g., to hide transfer latencies by overlapping the transfer with the execution of a previous operator.

Query Placement & Optimization: In MonetDB, each query plan is represented in the *MonetDB Assembly Language* - MAL. Ocelot reuses this infrastructure and adds a new query optimizer, which rewrites MAL plans by replacing data processing MAL instructions of vanilla MonetDB with the highly parallel OpenCL MAL instructions of Ocelot. Ocelot does not support cross-device processing, meaning it executes the complete workload either on the CPU or on the GPU.

OmniDB

OmniDB aims for good code maintainability while exploiting all hardware resources for query processing (Zhang et al., 2013a).

<https://code.google.com/p/omnidb-paralleldbonapu/>

The basic idea is to create a hardware oblivious database kernel, which accesses the hardware via adapters implementing a common set of operators decoupling the hardware from the database kernel.

Storage System & Model: OmniDB is based on GPUQP with similar architectural properties. OmniDB keeps data in-memory in a column-oriented data layout.

Processing Model: OmniDB schedules and processes work units, which can vary in granularity. It also supports block-oriented processing.

Query Placement & Optimization: OmniDB chooses the processing device with highest throughput for a work unit. To avoid overloading a single device, OmniDB's scheduler ensures that the workload on one processing device may not exceed a certain percentage of the average workload on all processing devices. The cost model relies on the adapters to provide cost functions for the underlying processing devices.

Virginian

Virginian is a GPU-accelerated DBMS keeping data in main memory and supporting filter and aggregation operations on all processing devices (Bakkum and Skadron, 2010).

Storage System: Virginian uses no traditional caching of operators, but *Uniform Virtual Addressing - UVA*, which allows a GPU kernel to directly access data stored in pinned host memory. The accessed data is transferred over the bus transparently to the device and efficiently overlaps computation and data transfers.

Storage Model: Virginian implements tablets, which stores fixed size values column oriented. Additionally, tables can handle variable sized data types such as strings, which are stored in a dedicated section inside the tablet. Thus, Virginian supports strings on the GPU. This differs from other GDBMSs, which apply dictionary compression on strings first and work only on compressed values in the GPU RAM.

Model: Virginian uses operator-at-a-time processing as basic query-processing model. It implements an alternative processing scheme. While most systems call a sequence of highly parallel primitives requiring one new kernel invocation per primitive, Virginian uses the opcode model, which combines all primitives in a single kernel. This avoids writing data back to global memory and reading it again in the next kernel ultimately resulting in block-wise processing on the GPU.

Query Placement & Optimization: Virginian can either process queries on the CPU or on the GPU. Thus, there is no mechanism splitting up the workload between CPU and GPU processing devices and hence, no hybrid query optimizer is available.

8.51 Graphics Processing Unit - GPU solutions

High performance CPUs have large memory and storage capacity and robust network connectivity. CPUs can be programmed with numerous programming languages and are more general purpose than GPUs or *Field Programmable Gate Arrays - FPGAs*, but fail to match them in raw compute capabilities. They are the best solution for ML applications involving large scale data analytics, where large memory capacity and network capabilities are required.

GPUs have been used since 2008 to speed up DL training to an acceptable level, but it is not unusual for training to take days or weeks.

https://en.wikipedia.org/wiki/Graphics_processing_unit

While many optimization strategies are being developed to reduce the long learning time. DNNs continue to increase in size and complexity since the computational requirements are growing. In addition to the computational load, the memory capacity and bandwidth are a heavy influence on the overall performance of training, and inference to a lesser degree. The computation for many network layers has a high compute-to-bandwidth ratio; there is a large amount of data reuse, and the parameters are shared across each layer of input data. Conversely, the more fully connected network layers have very low compute-to-bandwidth ratios; the data reuse is very low and there are no shared parameters.

As GPU processing has improved to address the compute challenges for NNs many models have become bandwidth bound and the peak performance of the GPU (or CPU) is not realized. In worst-cases the efficiency of the GPU is in the range of 15-20% of peak theoretical performance. This need for increased memory bandwidth being addressed with the use of *High Bandwidth Memory - HBM* discussed later.

https://en.wikipedia.org/wiki/High_Bandwidth_Memory

GPUs are efficient for DL primitives involving highly parallel compute operations: convolutions, matrix multiply, and activation functions. With the addition of HBM stacked memory to the latest GPU models, the bandwidth is improved to the point where many of the primitives can effectively exploit all available GPU compute resources. For dense linear algebra operations, the performance improvement of GPUs over CPUs is in the 10-20:1 range.

The primary programming model for GPUs is based on maximizing parallel processing. A GPU design with 64 compute units, may have four SIMD engines, each with 16 FLP units. When the utilization approaches 100%, the peak performance is over 10 TFLOPS (tera = 10^{12}) (single precision fp32) and 25 TFLOPS (fp16). Additional GPU performance is realized when inner product instructions combine multiply and addition functions for vectors to match primitives for matrix operations. GPUs are typically optimal designs for DNN training, but also provide substantial performance for inference operations

NVIDIA's Jensen Huang at 2018 GPU Technology Conf. stated that the AlexNet training process which took six days five years ago on two of Nvidia's GTX 580s now takes 18 minutes on the DGX-2 (a factor of 500).

<https://spectrum.ieee.org/view-from-the-valley/computing/>

<https://www.hpcwire.com/2018/03/27/nvidia-riding-high-as-gpu-workloads-and-capabilities-soar/hardware/move-over-moores-law-make-way-for-huangs-law>

The authors of (Krizhevsky et al., 2017) trained a large, deep CNN called ALEXnet to classify the 1.2 million high-resolution images in the ImageNet LSVRC-2010 contest into the 1000 different classes. ALEXnet achieved top 1 and 5 error rates of 37.5% and 17.0%, respectively, which is considerably better than the previous state-of-the-art. The NN has 60 million parameters and 650,000 neurons and five convolutional layers, some of which are followed by max-pooling layers, and three fully connected layers with a final 1000-way softmax. To make training faster, use was made of non-saturating neurons and a very efficient GPU implementation of the convolution operation.

ALEXnet showed its capabilities in identifying images in Imagenet: A Large-Scale Hierarchical Image Database (Deng et al., 2009):

"The explosion of image data on the Internet has the potential to foster more sophisticated and robust models and algorithms to index, retrieve, organize and interact with images and multimedia data. But exactly how such data can be harnessed and organized remains a critical problem. We introduce here a new database called "ImageNet", a large-scale ontology of images built upon the backbone of the WordNet structure. ImageNet aims to populate the majority of the 80,000 synsets of WordNet with an average of 500-1000 clean and full resolution images. This will result in tens of millions of annotated images organized by the semantic hierarchy of WordNet. This paper offers a detailed analysis of ImageNet in its current state: 12 subtrees with 5247 sunsets and 3.2 million images in total. We show that ImageNet is much larger in scale and diversity and much more accurate than the current image datasets. Constructing such a large-scale database is a challenging task. We describe the data collection scheme with Amazon Mechanical Turk. Lastly, we illustrate the usefulness of ImageNet through three simple applications in object recognition, image classification and automatic object clustering. We hope that the scale, accuracy, diversity and hierarchical structure of ImageNet can offer unparalleled opportunities to re-searchers in the computer vision community and beyond."

"WordNet is a large lexical database of English. Nouns, verbs, adjectives and adverbs are grouped into sets of cognitive synonyms (synsets), each expressing a distinct concept. Synsets are interlinked by means of conceptual-semantic and lexical relations. The resulting network of meaningfully related words and concepts can be navigated with the browser (link is external). WordNet is also freely and publicly available for download. WordNet's structure makes it a useful tool for computational linguistics and natural language processing.

<https://wordnet.princeton.edu/>

In 2006 Nvidia's GPU had a 4X performance advantage over other GPUs. In 2018 the Nvidia GPU was 20X faster than comparable GPUs: the GPUs were 1.7X faster each year. Moores law would predict a doubling every two years, however Nvidia's GPU performance was more than tripled every two years fulfilling Huang's law. Huang's law was coined by Christopher Mims from *Wall Street J.* - *WSJ* on 9/19/2020. that GPUs get 1.7x faster each year.

https://en.wikipedia.org/wiki/Huang%27s_law

AMD GPUs are overall inferior compared with those of NVIDIA.

<https://www.tomshardware.com/features/amd-vs-nvidia-gpus>

It was announced on July 2018 that NVIDIA is acquiring ARM for \$40 Billion.

<https://finance.yahoo.com/news/nvidias-acquisition-arm-proves-boon-063208683.html>

<https://www.arm.com/products/silicon-ip-cpu>

On 2/12/21 Nvidia's acquisition of Arm Ltd. from SoftBank run into opposition from Qualcomm, Google and Microsoft

who have filed a complaint with U.S. antitrust regulators.

<https://www.cnn.com/2021/02/12/qualcomm-objects-to-nvidias-40-billion-arm-acquisition.html>

ARM's primary business is selling *Intellectual Property - IP* to create *MicroController Units - MCUs*, CPUs, and *Systems-on-Chips - SoC* based on those cores. The *Original Design Manufacturer - ODM* combines the ARM core with other parts to produce a complete device, typically one that can be built in existing fabs at low cost and still deliver substantial performance. The most successful implementation has been the ARM7TDMI with hundreds of millions sold.

https://en.wikipedia.org/wiki/ARM_architecture

MxNet allows scalable parallelization of training tasks using multiple CPUs and GPUs.

https://en.wikipedia.org/wiki/Apache_MXNet

Middleware libraries provide operations to the frameworks that are tuned to specific hardware for maximum performance. MIOpen from AMD support the Radeon Instinct line of AMD GPUs, while cuDNN from NVIDIA supports similar functions on NVIDIA GPUs. BLAS, FFT, and *Random Number Generator - RNG* are a set of standard math library routines. *NVIDIA Collective Communications Library - NCCL* implements multi-GPU and multi-node collective communication primitives.

<https://developer.nvidia.com/cudnn>

AMD's ROCm platform for GPUs includes system runtime functions and drivers, compilers, and porting software for *Compute Unified Device Architecture - CUDA* programs for its Radeon GPUs (Sandkers and Kandrot, 2010).

<https://en.wikipedia.org/wiki/CUDA>

The processing proceeds as follows:

1. Copy data from CPU's main memory to GPU memory,
2. CPU initiates the GPU compute kernel,
3. GPU's CUDA cores execute the kernel in parallel,
4. Copy the resulting data from GPU memory to main memory.

Versions of CUDA run on successive NVIDIA GPUs: Tesla (2007), Fermi (2011), Kepler (2012), Maxwell (2015), Pascal (2016), Volta (2017), Turing (2018), Ampere (2020).

https://en.wikipedia.org/wiki/List_of_Nvidia_graphics_processing_units

The Pascal GPU had the following characteristics: 10 TFLOPS FP32, 20 TFLOPSFP16, 16 GB HBM – 750 GB/s, 300 W TDP, 67 GFLOPS/W (FP16), 16 nm process, 160 GB/s NV Link.

NVIDIA DGX-1 DL Supercomputer. 170 TFLOPS, 8x Tesla P100 16 GB. NVLink Hybrid Cube Mesh Optimized DL Software, Dual Xeon7 TB SSD Deep Learning Cache, Dual 10GbE, Quad IB 100 Gb, 3RU – 3200 W.

NVIDIA Parker, 1.5 Teraflop FP16, 4 GB of LPDDR4 @ 25.6 GB/s, 15 W TDP (1 W idle, <10 W typical), 100 GFLOPS/W (FP16), 16 nm process.

<https://hexus.net/tech/news/cpu/96070-nvidia-parker-soc-features-256-core-pascal-gpu/>

ROCm supports the common ML frameworks such as Caffe/Caffe-2, TensorFlow, Torch, and MXNet to provide a “full-stack” solution. ROCm is also compliant with the *Heterogeneous Systems Architecture - HSA* open standard, and scales to multi-GPU configurations.

Composable Disaggregated Infrastructure - CDI is proposed to increase GPU efficiency and hence power efficiency. With this approach, accelerators like GPUs are pooled into enclosures and not tied to specific servers and linked to multiple servers using a PCI-Express switch fabric, allowing for the devices to be disaggregated from the servers and then composed on the fly by a PCI-Express fabric controller. It is commonplace today that companies will want to have eight GPUs per server. Whenever possible, applications that are CPU-only will not be scheduled on hybrid CPU-GPU nodes. The model also assumes that only a single job runs on any server that is not disaggregated and composed.

<https://www.nextplatform.com/2020/11/17/counting-the-cost-of-under-utilized-gpus-and-doing-something-about-it/>

A forthcoming book on multicore and GPU programming is (Barlas, 2021).

8.52 Field Programmable Gate Array - FPGA solutions

FPGAs are often used to implement inference accelerators and can be efficiently configured to minimize the overhead or unnecessary functions that are included in GPU devices.

https://en.wikipedia.org/wiki/Field-programmable_gate_array

They are limited to either integer inference or lower precision FLPs compared to GPUs. The key attribute of FPGAs is the ability to configure the array with efficient design with little or no overhead, and the ability to dynamically at run time reconfigure the array characteristics. FPGAs can provide both latency and performance per watt advantage over CPUs

and GPUs in DL inference operations. FPGAs achieve this efficiency by enabling implementation of DL algorithms using reduced arithmetic precision, pruned networks, and custom high-performance hardware implementation.

Xilinx has demonstrated that FPGAs can be used to implement CNN engines with a peak performance of over fifteen *Tera Operations per Second - TOPS* at 8-bit accuracy and 80% efficiency for some CNNs.

<https://www.kdnuggets.com/2020/05/tops-just-hype-dark-ai-silicon-disguise.html>

Pruning techniques used mainly in the context of *Long Short Term Memory - LSTM*, which is an RNN. *Multi-Layer Perceptron - MLP* neural processing.

https://en.wikipedia.org/wiki/Multilayer_perceptron

has demonstrated that the model sizes can be effectively reduced by up to 20X providing a significant advantage in implementing an optimized solution.

The discussion that follows is based on the following review article (István, 2019).

Programmable hardware devices in the datacenter can be categorized based on their location with respect to the data source and the CPU: (1) on the side, (2) in data-path, (3) coprocessor.

- 1. On the side:** The most traditional way is on the side attachment to the processor via an interconnect such as PCIe. The CPU owns the data and sends it to the accelerator, resulting typically in significant additional latency per operation (communication latency and data transformation overhead). This encourages offloading operations at large granularity and without requiring repeated communication between the CPU and the accelerator. There have also been proposals to deploy FPGAs in this way for data filtering and decompression at IBM (Sukhwani et al., 2015).
- 2. In data path:** Another way of placing acceleration functionality is in data-path. The goal of the accelerator is to filter or transform data at the speed that it is received from the data source. Much of the research effort in this space has been centered around in-SSD processing, but more recently, there has been efforts in using *RDMA Network Interface Cards - NICs* to accelerate distributed databases. NICs are limited to data manipulation acceleration, but there are efforts to make NICs and networking hardware in general, more programmable allowing offloading complex, application-specific, operations.
- 3. Coprocessor:** The third option with coprocessors is in the form of CPUs that integrate domain-specific or general-purpose programmable coprocessors. The Oracle DAX (Aingaran et al., 2016) implements DB-specific operations, such as data decompression, scan acceleration, comparison-based filtering. It occupies negligible chip space and does not increase the cost of the CPU. As opposed to the DAX, the Intel Xeon+FPGA platform offers an FPGA beside the CPU cores for general-purpose acceleration. The FPGA has high bandwidth cache coherent access to the main memory of the CPU and can be reprogrammed differently creating acceleration opportunities without the usual overhead of the on the side accelerators.

FPGA chips can be programmed to implement arbitrary circuits and have been used to validate designs before they are turned into ASICs. They have recently target for implementing data processing accelerators in datacenters due to their flexibility, since their role can change over time, as opposed to an ASIC and order of magnitude better energy efficiency than that of traditional CPUs (Teubner and Woods, 2013).

FPGAs are composed of *Look-Up Tables - LUTs*, on-chip memory *Block RAM - BRAM* and *Digital Signal Processing - DSP* units.

<https://www.nandland.com/articles/block-ram-in-fpga.html>

https://en.wikipedia.org/wiki/List_of_Xilinx_FPGAs

The components are configured and interconnected flexibly allowing the programmer to implement any hardware logic on top. Small ARM cores can be integrated inside the programmable fabric, e.g., in Xilinx' Zynq product line. FPGAs offer two types of parallelism:

Pipeline parallelism allows complex functionality can be executed in steps without reducing throughput. The benefit of FPGAs in this context is that the communication between pipeline stages is very efficient due to the physical proximity and availability of on-chip memory to construct FIFO buffers.

Data Parallel Execution is like SIMD at a higher level, referred to as *Single Program Multiple Data - SPMD* paradigm (Auguin and Larbay, 1983), credit for which was claimed later in (Darema, 2001).

<https://en.wikipedia.org/wiki/SPMD>

In MIMD a parallel computer consists of multiple processors running their own program and operating on their own data. In SPMD a parallel computer consists of multiple processor each running their own copy of the same program. According to (Brorsson, 1991) an SPMD program typically consists of a number of parallel executable segments (e.g., parallel do-loops) separated by serial segments that are executed by a single processor. Between the parallel and the serial segments there are barriers to synchronize the threads. The barrier synchronization primitive uses a shared

variable on which the threads spin until all threads have reached the barrier (Culler et al., 1999).

[https://en.wikipedia.org/wiki/Barrier_\(computer_science\)](https://en.wikipedia.org/wiki/Barrier_(computer_science))

FPGAs are programmed by synthesizing circuits using hardware definition languages such as Verilog or VHDL, and creating a “bitstream” for a specific device type that defines the behavior of every logic resource on the chip.

<https://en.wikipedia.org/wiki/Verilog>

The tool lays out the circuit on the “chip surface” and define connections and their routing between circuit elements. Since FPGAs have flexible clocking options and the programmer can define a target frequency (e.g., 300 MHz), the tools have to set up routing such that signals are propagated within the clock periods.

It is also possible to perform *Partial Reconfiguration* - PR so that only a portion of the FPGAs resources are reprogrammed. A hardware-accelerated operator can be replaced with another one without having to bring the device offline. A limitation of PR is that regions can only be defined at coarse granularity, their size can’t be redefined at runtime and their reprogramming requires milliseconds.

A limitation of FPGAs is that all application logic occupies chip space, i.e., it is not possible to “page” code in or out dynamically. This means that the complexity of the operator that is being offloaded is limited, by the available logic resources (area) on the FPGA. This also applies to the “state” of an algorithm that stored as data in the on-chip BRAM - Block RAM memories. These can be accessed in a single clock cycle, but if the data does not fit in the available BRAM, high latency off-chip DRAM has to be used.

<https://www.nandland.com/articles/block-ram-in-fpga.html>

Many early projects of FPGA-based DB acceleration propose deploying them as on the side accelerators for row stores. FPGAs successfully accelerate selection, projection, group-by aggregation, joins and even sorting by an order of magnitude when compared to MySQL and Postgres. The benefits are significantly reduced by the cost of communication over PCIe and the software overhead of preparing the data for the FPGA to work on (parsing and copying pages). In traditional on the side deployments the high latency communication (microseconds over PCIe) often forces designs to move entire operators onto the FPGA, even if only parts of the operator were a good match for the hardware. This leads to complications, because even though FPGAs excel at parallel and pipelined execution, they behave poorly when an algorithm requires iterative “loops” or has widely branching “if-then-else” logic. CPUs deliver higher performance due to their higher clock rates.

In FPGAs the branching logic needs to be mapped to logic gates that encode all outcomes, resulting in very large circuits. Since the space on the FPGA is limited, the larger circuits result in reduced parallelism leading to lower throughput. This means that even though FPGAs could be successful in accelerating the common case of an algorithm, they might not be able to handle corner or pathological cases, and in practice this leads to uncertainty in the query optimizer or wasted work if a corner case is encountered during execution.

In parallel with accelerator-based efforts, there have been numerous advances in the space of analytical databases. Column-oriented databases, such as MonetDB (Boncz et al., 2008), are widely deployed and typically outperform row-oriented ones by at least an order of magnitude and can take advantage of many-core CPUs efficiently. As a result, the speedups that FPGAs offer when targeting core SQL operators have shrunk and often are not enough to motivate the additional effort of integrating specialized hardware in the server architecture. For the above reasons, FPGA-based acceleration ideas are often received with pessimism. However, changes in the hardware available in datacenters and the cloud, as well as the changes in database architecture and user workloads, create novel opportunities for FPGA-based acceleration.

Regarding In-memory database acceleration on FPGAs interest has declined in both industry and academia for the following three reasons (Fang et al., 2020).

1. FPGAs integrated with conventional I/O provide insufficient bandwidth, limiting performance.
2. GPUs providing higher throughput, easier programming, have emerged as a strong accelerator alternative.
3. Programming FPGAs required developers to have full-stack skills, from high-level algorithm design to low-level circuit implementations.

However these challenges are being addressed. New interface technologies connect FPGAs into the system at main-memory bandwidth and the latest FPGAs provide local memory competitive in capacity and bandwidth with GPUs. Ease of programming is improving through support of shared coherent virtual memory between the host and the accelerator, support for higher-level languages, and domain-specific tools to generate FPGA designs automatically.

Surveyed in (Fang et al., 2020) are FPGA designs that can operate at the speed of main memory.

Predictions about FPGAs

FPGA predictions made in (Mencer et al., 2020) are as follows:

1. There will be successful CPU+FPGA server chips, or FPGAs with direct access to the CPU's cache hierarchy, but there is no certainty.
2. *System on a Chip* - SOC FPGA chips will grow and expand, driving the medical, next-generation telecom, and automotive industries, among others.
3. Developers successfully using FPGAs will have to hide the fact that there is an FPGA inside.
4. The FPGA name will remain, and chips called FPGAs will be built, but everything inside will be completely different.
5. With dataflow optimization in order to make FPGAs easier to program, the performance of FPGAs will be reduced so they are no longer competitive with CPUs, which will always be easier to program.
6. There will be FPGAs with dynamic routing, evolving interconnect, and runtime-flexible data movement.
7. The complete software stack on top of FPGAs, will be open source. There are already initial efforts with Yosys and Lattice FPGAs.
<http://www.clifford.at/yosys/>
8. All semiconductor architectures will be combined into single chips with combinations of *Tensor Processing Units* - TPUs,
https://en.wikipedia.org/wiki/Tensor_Processing_Unit
GPUs, CPUs, ASICs, and FPGAs. Some may be combinations of the whole of each. Others will be combinations of parts of each.
9. More chips will be focused on limited application spaces, and fewer on general-purpose chips. In a way, everything is becoming an SoC.

8.53 Multichip modules

The following discussion is extracted from the following web site.

<https://semiengineering.com/getting-serious-about-chiplets/>

A recent article on this topic is (Frazelle, 2020b).

In the traditional single die *Integrated Circuit* - IC manufacturing process, most of the defective die are eliminated during wafer test, but wafer testing is inherently limited by the number of probes that can be used to apply stimuli and measure responses of the die on a wafer. Limitations in access to power and ground rails can also limit the ability to apply tests at the full target speed of the device. Because of the cost of overcoming these inherent wafer test limitations, the historic approach has been to allow some test escapes at the wafer level and to catch these during package test.

In the chiplet package flow, the escape rate at wafer test has a much larger impact on final packaged (board) yield. This is because a defective die, when packaged with good die, will result in a failure of the entire package (board). This combinational effect means the impact of wafer escapes is magnified exponentially in the final product, adversely affecting package yield.

In addition, the cost of defects after packaging is higher because there is more value bound up in a chiplet package. Because of these two reinforcing effects there is a higher *Known Good Die* - KGD test quality requirement at wafer test for chiplets.

Pros and cons for SoCs, chiplets and traditional system design based on [semico.com](https://www.semico.com) are given in Table 8.26.

	Monolithic SoC	Chiplet on substrate	Discrete IC on PCB
Design cost	Highest, 7 nm > \$200M	Less costly, than monolithic	Least expensive
Design time	Longest, 118+ months	Shorter, 12 months faster for derivative design	Shorter design time, six months
Design risk	Highest risk, including missing future features having to do a redesign	Lower risk, change out chiplets to +/- features, simple redesign	Lowest risk
Performance	Highest, but some functions do not scale, inefficient resource use	Good performance can use most appr. fn needed by the chiplet	Highest power consumption
Time to market	Longest	Quicker	Shortest
Product size	Smallest	Small	largest

The fastest growing technology to manufacture high quality *Known Good Die* - *KGD* is the usage of test methods at wafer probe level (Arnold et al., 1998). This may be achieved by test patterns with higher fault coverage and possibly by the introduction of additional fault models to test for defects that are normally ignored at wafer test.

Testing the interconnect between the chiplets on the substrate can be done in two ways: traditional boundary scan techniques used for testing between packaged chips on boards or modules can be used if the overhead of adding boundary scan chains to the chiplets is possible. If adding boundary scan chains to all the chiplets is not physically or economically possible, the chiplets can be treated essentially as IP cores, and hierarchical test techniques traditionally used for cores within large SoCs can be used to test the interconnect instead. What's more, hierarchical *Automatic Test Pattern Generator* - *ATPG* allows the test patterns used to test the chiplets during wafer sort to be re-used almost as is at the package level for testing any internal chiplet defects that may have been introduced during the packaging process.

https://en.wikipedia.org/wiki/Automatic_test_pattern_generation

On the verification side, chiplets can be viewed as the next element along the axis of reuse in the design flow. At the most abstract are behavioral models that have to go through high-level synthesis, then *Register Transfer Level* - *RTL* models that go through logic synthesis and layout to hard IP that has been fully laid out and now chiplets, prefabricated pieces of IP.

https://en.wikipedia.org/wiki/Register-transfer_level

The verification side of this has not moved in the same way, which is why the time and cost of verification has been increasing.

There is a growing awareness of Portable Stimulus playing a role here

<https://www.electronicdesign.com/technologies/test-measurement/article/21806605/11-myths-about-portable-stimulus>

because the models can be assembled in the same way as IP blocks, unlike *Universal Verification Methodology* - *UVM* models that cannot.

https://en.wikipedia.org/wiki/Universal_Verification_Methodology

A *Product Service System* - *PSS* model is a complete specification of what a block is supposed to be able to do and results it should provide, and can also carry information about the testing performed on it.

When assembling these at the system level, it thus becomes easy to see if the design will activate paths through the chiplet that have never been tested before. Test cases also will be created that verify the interactions between the chiplets. Portable stimulus testcases can either be run on abstract models of the chiplets or on the actual devices themselves, perhaps integrated through some type of package breadboarding.

Chiplets are expected to solve many problems in design, but are hardly a panacea for all applications. The user has to decide up front how much memory to pair with the CPU, which is difficult to change later on. At the *Printed Circuit Board Level* - *PCB*, this is modular, so the user can change out one memory for another. With chiplets, the user cannot create an inventory of package parts with memory and cannot add a memory stack in at the time of the packaging. And inventory management here is not trivial. This is one of the issues DARPA is facing with chiplets. At the time of packaging, they have to decide what goes in there and make sure all of the products will be available for the entire product lifetime. If there is an end of life for a chiplet, can they fall back on something else. What if a chiplet company gets acquired.

Complicating all of these issues are smaller process geometries. The complexity of lower process geometries intensifies the complexity of testing chiplets. We expect to see a similar paradigm shift in test strategies as we currently see with devices moving to lower nodes. Often test strategies that worked at the higher nodes have to be supplemented with new test methodologies. Adding a system-level test strategy can help get closer to a 100% test coverage as it supplements the testing done with structural and functional test strategies.

Chiplets continue to be an enticing possibility for companies. Marvell's *MODular CHips* - *iMoChi* architecture, which is based on internally developed components, is a first step toward this kind of plug-and-play approach. But expanding this to include an entire ecosystem of IP providers adds some new challenges to this scheme.

According to (Frazelle, 2020a) there are a few different ways of interconnecting chiplets to form MCMs, most of which are proprietary to each vendor. AMD calls its die-to-die interconnect *Infinity Fabric* - *IF* which is not just a die-to-die interconnect but a processor-to-processor interconnect.

https://en.wikichip.org/wiki/amd/infinity_fabric

IF is made up of two parts: *Infinity Scalable Data/Control Fabric* - *SDF/SCF*. SCF handles communication for controls such as power management, thermal controls, security, and tests. Because most die-to-die interconnects are proprietary, the benefits of building with chiplets as if they are Lego blocks have not entirely come to fruition. Adding functionality such as an FPGA requires adopting the proprietary die-to-die interconnect. As a response and solution to this problem, Intel open sourced its *Advanced Interface Bus* - *AIB*, which is a die-to-die interconnect standard that enables system design in a modular way with libraries containing chiplet IP (intellectual property) blocks.

AMD at the *Int'l Electron Devices Meeting - iEDM* in 2017 discussed splitting up its 32-core EPYC server-class chip onto four 8-core chiplets. This resulted in a 10% overhead for the added I/O to the four chiplets. Since it could fit 9% more of the smaller dies onto a wafer than larger dies, however, it was able to reduce the overall cost because of the improvements in die yield. Schor, D. 2017. IEDM 2017: AMD's grand vision for the future of HPC.

8.54 Unified solutions

The use of DL applications is expanding with the combination of very large data sets, robust algorithms, and high-performance compute platforms that enables a large variety of applications for commercial deployment. The compute requirements are substantial and growing for both training and inference. Unified DL configurations combining CPU, GPU, and FPGA products are emerging solutions that enable the compute capabilities and flexibility that are needed to address these challenges and assures the best framework for future proofing.

Two appropriate building blocks for all three compute elements are the ROCm and MIopen software stacks described below:

https://www.phoronix.com/scan.php?page=news_item&px=ROCm-Compute-Stack-Overview
<https://arxiv.org/abs/1910.00078>

ROCm is exascale-class programming language independent platform for accelerated computing, which provides choice, minimalism and modular software development to GPU computing as outlined in Table 8.27. ROCm supports multi-GPU computing and has a rich system run time with the critical features that large-scale application, compiler and language-run-time development requires.

<https://www.amd.com/en/graphics/servers-solutions-rocm>

TABLE 8.27 Fully open source ROCm platform.

Applications	HPC& ML Apps			
Tools	Debugger	Performance Analysis	System Validation	System Management
Frameworks	Tensorflow	PyTorch	Kokkos	RAJA
Math Libraries	MIOpen	FFT, RNG	BLAS, Sparse	Eigen
Communication Libraries	RCCL	UCX, libfabric	MPICH	OpenMPI
ROCm Programming Models	OpenMP	HIP	OpenCL	Python

Three open source software libraries to implement common training and inference operations for deep learning are (1) Caffe/Caffe-2 (Facebook),

<https://research.fb.com/downloads/caffe2/>

TensorFlow (Google) (Abadi et al., 2016), Torch, PyTorch,

<https://en.wikipedia.org/wiki/PyTorch>

and MXNet (Amazon).

https://en.wikipedia.org/wiki/Apache_MXNet

Such frameworks are supported across both GPUs and FPGAs, effectively hide most heterogeneity from application developers and enable portability across different systems.

Google's Tensorflow is an open-source software library applicable to DNNs for ML.

<https://en.wikipedia.org/wiki/TensorFlow>

TPUs based on FPGAs are 15 to 30 times faster with 30 to 80 times better energy efficiency than CPUs and GPUs in similar technologies (Jouppi et al., 2018).

<https://dl.acm.org/doi/fullHtml/10.1145/3154484#body-6>

The high accuracy of DNNs has led to the development of NN accelerators that improve performance by two orders of magnitude. Scaling accelerators for higher performance with increasingly larger NNs exacerbates the cost and energy overhead of the on-chip SRAM buffers and the off-chip DRAM. The hardware architecture and software scheduling and partitioning techniques for TETRIS, a scalable NN accelerator using 3D memory is presented in (Gao et al., 2017). The following points are made in this study:

(1) The high throughput and low energy characteristics of 3D memory allow us to rebalance the NN accelerator design,

using more area for processing elements and less area for SRAM buffers.

(2) NN computations are moved closer to the DRAM banks to decrease bandwidth pressure and increase performance and energy efficiency.

(3) Despite the use of Small SRAM buffers the presence of 3D memory simplifies dataflow scheduling for NN computations. Overall TETRIS improves the performance by 4.1x and reduces the energy by 1.5X over NN accelerators with conventional, low-power DRAM memory systems.

AlexNet is a CNN with eight layers; the first five are convolutional layers, some of them followed by max-pooling layers, and the last three are fully connected layers. It used the non-saturating ReLU activation function, which has shown improved training performance over tanh and sigmoid functions.

<https://en.wikipedia.org/wiki/AlexNet>

ZFnet and its code are given below.

<https://towardsdatascience.com/zfnet-an-explanation-of-paper-with-code-f1bd6752121d>

ResNet is a residual NN utilizing skip connections, or shortcuts with double- or triple-layer skips. Typical ResNet models contain nonlinearities (ReLU) and batch normalization in between.

https://en.wikipedia.org/wiki/Residual_neural_network

Some details about neural nets used in the evaluation are given in Table 8.28.

TABLE 8.28 The NNs used in our evaluation. We show the *feature map - fmap* and filter sizes in MB for the largest layer and the whole NN (largest/total). We use 16-bit fixed-point data.

	AlexNet	ZFnet	VGG16	VGG19	ResNet
#Convs	5	5	13	16	151
#FCs	3	3	3	3	1
Fmap size	0.58/1.32	2.4/3.4	6.4/17.0	6.4/28.0	1.6/40
Filter size	74/124	150/214	204/276	204/286	4/6/110
Year	2012	2013	2014	2014	2015

With the increasing adoption of distributed architectures for analytical databases, as well as the disaggregation efforts in the datacenter, there are numerous opportunities for moving computation closer to the data source to reduce the data movement bottlenecks. These bottlenecks arise from the fact that the access bandwidths are higher and closer to the data source than over the network/interconnect and they can be eliminated by pushing filtering or similar data reduction operations closer to source. Thus, the main goal of having the accelerator in the data-path is to maintain the data access bandwidth high while reducing the amount of data sent to the processor. The data source is often a network-attached flash storage and recent projects, for instance, BlueDBM (Jun et al., 2015, 2016) which is described below.

The need for random accesses has proven to be slow unless all the data can be accommodated in DRAM. For many domains the dataset size is 5-20 TB. A cluster with 100 servers with 128-256 GB would be required to accommodate all the data in DRAM. On the other hand, such datasets could be stored easily in the flash memory of a rack-sized cluster. Flash storage has much better random access performance than HDDs, which makes it desirable for analytics workloads. However, currently available off-the-shelf SSD flash storage does not make effective use of flash storage, because it incurs additional overhead during flash device management and network access.

BlueDBM provides in-store processing capability and a low-latency high-throughput intercontroller network between storage devices. BlueDBM outperforms a flash-based system without these features by a factor of 10 for some important applications. While the performance of a DRAM-centric system falls sharply even if only 5% to 10% of the references are to secondary storage, this sharp performance degradation is not an issue in BlueDBM. BlueDBM presents an attractive point in the cost/performance tradeoff for Big Data analytics.

Modern DB engines that make use of the multi-core CPUs and SIMD units are rarely compute bound once the data is loaded into main memory. Unless reading data from storage, offloading core SQL operators is unlikely to bring orders of magnitudes performance improvements. This is different for training and inference in ML. A significant portion of ML pipelines operate on relational data and the case has been made that there is a benefit in integrating these directly in the database. Furthermore, there is also interest in including such components in the internal databases modules, to perform optimizations depending on the workload characteristics and a model. Since this could require on-line retraining that, without hardware acceleration, could hurt user throughput significantly. Acceleration of training as part of user workloads is being explored, e.g., by Prof. Hadi Esmaeilzadeh et al. in DANa (Mahajan et al., 2018).

“In-RDBMS hardware acceleration of advanced analytics”, abstract (Mahajan et al., 2018)

“The data revolution is fueled by advances in ML, databases, and hardware design. Programmable accelerators are making their way into each of these areas independently. As such, there is a void of solutions that enables hardware acceleration at the intersection of these disjoint fields. This paper sets out to be the initial step towards a unifying solution for in *Database Acceleration of Advanced Analytics - DAnA*. Deploying specialized hardware, such as FPGAs, for in-database analytics currently requires hand-designing the hardware and manually routing the data. Instead, DAnA automatically maps a high-level specification of advanced analytics queries to an FPGA accelerator. The accelerator implementation is generated for a *User Defined Function - UDF*, expressed as a part of an SQL query using a Python-embedded *Domain-Specific Language - DSL*. To realize an efficient in-database integration, DAnA accelerators contain a novel hardware structure, Striders, that directly interface with the buffer pool of the database. Striders extract, cleanse, and process the training data tuples that are consumed by a multi-threaded FPGA engine that executes the analytics algorithm. We integrate DAnA with PostgreSQL to generate hardware accelerators for a range of real-world and synthetic datasets running diverse ML algorithms. Results show that DAnA-enhanced PostgreSQL provides, on average, 8.3x end-to-end speedup for real datasets, with a maximum of 28.2x. Moreover, DAnA-enhanced PostgreSQL is, on average, 4.0x faster than the multi-threaded Apache MADLib running on Greenplum. DAnA provides these benefits while hiding the complexity of hardware design from data scientists and allowing them to express the algorithm in ≈ 30 -60 lines of Python.”

Greenplum is discussed here.

<https://en.wikipedia.org/wiki/Greenplum>

8.55 Power consumption in FPGAs and ASICs

It is stated in “Data Processing in FPGAs” (Teubner and Woods, 2013) that power consumption and heat dissipation concerns forced the semiconductor industry to radically change its course, shifting from sequential to parallel computing. Improving performance of applications has now become much more difficult than when frequency scaling was an option. This is also affecting databases and data processing applications in general and has led to the popularity of so-called data appliances, i.e., specialized data processing engines, where software and hardware are sold together in a closed box. FPGAs which increasingly play an important role in such systems are attractive because the performance gains of specialized hardware can be significant, while power consumption is much less than that of commodity processors. FPGAs which are more flexible than ASICs and can be integrated into complex systems in many different ways, e.g., directly in the network for a high-frequency trading application.

In what follows we discuss the concept of dark silicon considered by Esmaeilzadeh et al. (Esmaeilzadeh et al., 2011).

https://en.wikipedia.org/wiki/Dark_silicon

In the electronics industry, dark silicon is the amount of circuitry of an *Integrated Circuit - IC* that cannot be powered-on at the nominal operating voltage for a given *Thermal Design Power - TDP* constraint. It is the maximum amount of heat generated by a computer chip or component that the cooling system in a computer is designed to dissipate under any workload.

https://en.wikipedia.org/wiki/Thermal_design_power

This is a challenge in the era of nanometer semiconductor nodes, where transistor scaling and voltage scaling are no longer in line with each other, resulting in the failure of Dennard scaling. This discontinuation of Dennard scaling has led to sharp increases in power densities that hamper powering-on all the transistors simultaneously at the nominal voltage, while keeping the chip temperature in the safe operating range. According to recent studies, researchers from different groups have projected that, at 8 nm technology nodes, the amount of Dark Silicon may reach up to 50–80% depending upon the processor architecture, cooling technology, and application workloads. Dark Silicon may be unavoidable even in server workloads with abundance of inherent client request-level parallelism. The emergence of Dark Silicon introduces several challenges on the architecture, *Electronic Design Automation - EDA*, and Hardware-Software co-design communities. For instance, how best to utilize the plethora of transistors (with potentially many dark ones) for designing and managing energy-efficient on-chip many-core systems under peak power and thermal constraints. Architects have initiated several efforts in leveraging the Dark Silicon to design application-specific and accelerator-rich architectures. Recently, researchers have explored how Dark Silicon exposes new challenges and opportunities for the EDA community. In particular, they have

demonstrated the thermal, reliability (soft error and aging), and process variation concerns for Dark Silicon many-core processors. The abstract for (Esmaeilzadeh et al., 2011) follows.

“Since 2005, processor designers have increased core counts to exploit Moore’s Law scaling, rather than focusing on single-core performance. The failure of Dennard scaling, to which the shift to multicore parts is partially a response, may soon limit multicore scaling just as single-core scaling has been curtailed. This paper models multicore scaling limits by combining device scaling, single-core scaling, and multicore scaling to measure the speedup potential for a set of parallel workloads for the next five technology generations. For device scaling, we use both the *Int’l Technology Roadmap for Semiconductors - ITRS* projections

<https://www.itrs2.net>

and a set of more conservative device scaling parameters.

To model single-core scaling, we combine measurements from over 150 processors to derive Pareto-optimal frontiers for area/performance and power/performance.

<https://web.stanford.edu/group/sisl/k12/optimization/MO-unit5-pdfs/5.8Pareto.pdf>

Finally, to model multicore scaling, we build a detailed performance model of upper-bound performance and lower-bound core power. The multicore designs we study include single-threaded CPU-like and massively threaded GPU-like multicore chip organizations with symmetric, asymmetric, dynamic, and composed topologies. The study shows that regardless of chip organization and topology, multicore scaling is power limited to a degree not widely appreciated by the computing community. Even at 22 nm (just one year from now), 21% of a fixed-size chip must be powered off, and at 8 nm, this number grows to more than 50%. Through 2024, only 7.9×average speedup is possible across commonly used parallel workloads, leaving a nearly 24-fold gap from a target of doubled performance per generation.”

Amazon is already offering FPGAs running Xilinx’s OpenCL-based compute framework as PCIe-attached accelerators. In the “ML-backed” database scenario it will also be paramount to be able to take decisions with low latency using learned models – this further motivates the use of FPGAs. Even though GPUs are a de facto standard for ML acceleration, when it comes to low latency inference, FPGAs can offer benefits since they do not require batching in their processing modules:

Voodoo is a Vector Algebra for Portable Database Performance on Modern Hardware (Pirk et al., 2016). In-memory databases require careful tuning and engineering tricks to achieve good performance. Such database performance engineering is hard: a plethora of data and hardware-dependent optimization techniques form a design space that is difficult to navigate for a skilled engineer – even more so for a query compiler. To facilitate performance-oriented design exploration and query plan compilation Voodoo’s declarative intermediate algebra that abstracts the detailed architectural properties of the hardware, such as multi- or many-core architectures, caches and SIMD registers, without losing the ability to generate highly tuned code. Voodoo is easier to reason about and tune than low-level C and related hardware-focused extensions enabling the Voodoo compiler to produce OpenCL code that rivals and even outperforms the fastest state-of-the-art in memory DBs for both GPUs and CPUs. In addition, Voodoo makes it possible to express techniques as diverse as cache-conscious processing, predication and vectorization (again on both GPUs and CPUs) with just a few lines of code. Control vectors allow a code generating frontend to expose parallelism to the Voodoo compiler in an abstract manner, enabling portable performance across hardware platforms.

Voodoo can be used as an alternative backend for MonetDB, a popular open-source in-memory database. The backend allows MonetDB to perform at the same level as highly tuned in-memory databases, including HyPeR and Ocelot. Techniques used in existing in-memory databases are given in Table 8.29.

8.56 Hybrid approaches to acceleration

Since all functionality, regardless whether used or not, occupies chip space on the FPGA, corner (pathological) cases often cannot be efficiently handled in hardware. It is important to design accelerators such that they behave predictably even if the particular instance of the problem cannot be fully handled. State of the art solutions overcame such cases by splitting functionality between FPGA and software, such that the part on the FPGA remains beneficial to execution time regardless of the input data contents or distribution.

The tradeoffs in determining the best way to configure a system using either CPU, GPU, or FPGA devices, a comparison chart for various attributes for the devices. An important observation is that there are complementary strengths and weaknesses and there are no obvious “one size fits all” solutions as specified in Table 8.30.

TABLE 8.29 Techniques used in existing in-memory databases.

System	Bandwidth efficiency technique	CPU efficiency	Hardware target
HyPer (Neumann, 2011)	Pipelining	Compilation	CPU-only
MonetDB (Boncz et al., 2008)		Bulk-processing	CPU-only
Vectorwise (Zukowski et al., 2005)	Cache-friendly	Partitioning	CPU-only
Tupeware (Crotty et al., 2015)	Pipelining	Compilation	CPU-only
Ocelot (Heimel et al., 2013)		Bulk-processing	GPU-optimized
Voodoo (Pirk et al., 2016)	Tunable	Tunable	Tunable

TABLE 8.30 Comparison of CPUs, GPUs, FPGAs.

Feature	Analysis	Winner
DNN Training	GPU FLP capabilities are greater	GPU
DNN inference	FPGA can be customized, and has lower latency	FPGA
Large data analysis	CPUs support largest memory and storage FPGAs are good for inline processing	CPU/FPGA
Timing latency	Algorithms implemented on FPGAs provide deterministic timing and can be an order of magnitude faster than GPUs	FPGA
Processing/Watt	Customized designs can be optimal	FPGA
Processing/\$	GPUs win because of large processing capabilities. FGPA	GPU/FGPA
Interfaces	FPGAs can implement many different interface	
Backward compatibility	CPUS have more stable architecture than GPUs Migrating RTL to new FPGAs requires some work	CPU
Ease of change	CPUs and GPUs provide an easier path to changes to application functionality	GU/CPU
Customization	FPGAs provide broader flexibility	FPGA
Size	CPU and FPGA's lower power consumption leads to smaller volume solution	CPU/FPGA
Development	CPUs easier to program than GPU, both easier than FPGA	CPU

8.57 Application Specific Integrated Circuit - ASIC

It is generally possible to take custom designs originally developed for an FPGA and implement them in an ASIC chip for improved performance and power efficiency.

https://en.wikipedia.org/wiki/Application-specific_integrated_circuit

The design steps are:

(1) Requirements engineering, (2) *Register-Transfer-Level - RTL*, (3) Functional verification, (4) Logic synthesis transforms RTL designs into library constructs called standard cells, (5) Placement via a tool that attempts to find an optimized placement subject to a variety of constraints, (6) Routing which uses a tool that makes electrical connections among to cells, (7) Sign-off. Given the final layout, circuit extraction computes the parasitic resistances and capacitances. This is then further mapped into delay information from which the circuit performance can be estimated, usually by static timing analysis. Other final tests such as design rule checking and power analysis collectively called signoff ensure that the device will function correctly over all extremes of the process, voltage and temperature. The photomask information is released for chip fabrication when testing is complete.

One of the key challenges for increasing performance is to build large network configurations. There are two types of parallelism:

Model parallelism is implemented by coordinating the work of a DNN model spread across multiple hardware nodes, jointly operating on the same batch of data. A high degree of communication and synchronization among nodes is necessary, especially during training, to keep the information in each node consistent.

Data parallelism is implemented by assigning a batch of data to each node, where a node implements that entire DNN model, end-to-end.

TABLE 8.31 Different number representations used by DNNs. FP = Floating Point, Int= Integer, S = Sign, E = Exponent, M = Mantissa.

Numbers	Format	Range	Accuracy
FP32	S = 1, E = 8, M = 23	$10^{-38} - 10^{38}$	.0000006%
FP16	S = 1, E = 5, M = 10	$6 \times 10^{-5} - 6 \times 10^4$	0.5%
Int32/16/8	S = 1, M = 31/15/7	$0 - 2 \times 10^9/6 \times 10^4/127$	1/2

8.58 Tensorflow and Tensor Processing Units - TPUs

Major improvements in cost-energy-performance are expected to come from domain-specific hardware. Custom ASIC TPUs have been deployed in Google's datacenters, since 2015 accelerates the inference phase of NNs (Jouppi et al., 2017).

Google's *Tensor Processing Unit - TPU* is an example of an ASIC custom designed for deep learning. DNNs are applicable to a wide range of problems, so a DNN-specific ASIC can be reused for speech, vision, language, translation, search ranking, etc. DNNs using extra and larger layers to capture higher levels of patterns or concepts, and GPUs provided enough computing to develop them.

TensorFlow is an interface for expressing ML and an implementation for executing such algorithms (Abadi et al., 2016) <https://en.wikipedia.org/wiki/TensorFlow>

Tensorflow uses the dataflow paradigm advocated by MIT's Jack Dennis.

<http://csg.csail.mit.edu/Users/arvind/ISCAfinal.pdf>

Petri Nets (Peterson, 1977) can also be used to formally specify the dataflow paradigm (Kavi et al., 1986).

https://en.wikipedia.org/wiki/Petri_net

This flexible system can be used to express a wide variety of algorithms, including training and inference algorithms for DNN models, and it has been used for conducting research and for deploying ML systems into production across more than a dozen areas of computer science and other fields, including speech recognition, computer vision, robotics, information retrieval, *Natural Language Processing - NLP*, geographic information extraction, and computational drug discovery.

FLP is used for DNN implementation and optimization training, but for inference operations many network weights can be represented with considerably less precision than their original training output. Training is typically done on a combination of GPUs and CPUs using FP32, although there is a trend towards FP16 or even FP8 precision. Different number representations used by DNNs are given in Table 8.31.

FLP functions used for DNN training include *Basic Linear Algebra Subprograms - BLAS*,

https://en.wikipedia.org/wiki/Basic_Linear_Algebra_Subprograms

General Matrix Vector Multiplication - GEMV and GEMM routines, Conv2D operations, batch normalization, SGD, SVD, element wise matrix operations, and non-linear functions such as Softmax and ReLU. Efficient GPU and CPU data transfer is essential for the efficient implementation of these algorithms.

The softmax function or normalized exponential function is a generalization of the logistic function to multiple dimensions. It is used in multinomial logistic regression and is often used as the last activation function of a NN to normalize the output of a network to a probability distribution over predicted output classes, based on Luce's choice axiom that the probability of selecting one item over another from a pool of many items is not affected by the presence or absence of other items in the pool, also known as *Independence from Irrelevant Alternatives - IIA*.

https://en.wikipedia.org/wiki/Luce%27s_choice_axiom

In the following formula σ is softmax, $\vec{z}$ is the input vector, e^{z_i} is standard exponential function for input vector, K is the number of classes.

$$(\vec{\sigma})_i = e^{z_i} / \sum_{j=1}^K e^{z_j}, \quad 1 \leq i \leq K.$$

Inference implementations on FPGAs typically use fused mode of operations, where most of the network is implemented using large and flexible on-chip memory and flexible arithmetic capabilities unique to Xilinx FPGA. Xilinx has implemented overlay engines for implementing CNN and RNN engines. xfdnn from Xilinx provides a software middleware which runtime compiler, and a quantizer to map and run DNN models trained on GPUs and CPUs onto optimized overlay engines for CNN and RNN processing. Xilinx also provides Reconfigurable Acceleration Stack, which consists of the prebuilt hardware platform on FPGA, a runtime and design tools to enable development of a broad set of customized DL solutions.

The following discussion on the design and evaluation of the TPUs draws the following conclusions.

- User facing inference apps usually emphasize response-time.
- The NVIDIA Tesla K80 GPU is underutilized for inference due to latency limits, and is just a little faster than a Haswell CPU.
- Despite having a much smaller and lower power chip the TPU has 25 times as many *Multiply Accumulators - MACs* and 3.5 times as much on-chip memory as the Tesla K80 GPU.
<https://www.nvidia.com/en-gb/data-center/tesla-k80/>
- The TPU is about 15x - 30x faster at inference than the K80 GPU and the Haswell CPU.
- Four of the six NN apps are memory-bandwidth limited on the TPU; if the TPU were revised to have the same memory system as the K80 GPU, it would be about 30X - 50X faster than the GPU and CPU.
- The performance per Watt of the TPU is 30X - 80X that of contemporary products; the revised TPU with K80 memory would be 70x - 200x better.
- While most architects have been accelerating CNNs, they represent just 5% of our datacenter workload.

TABLE 8.32 Six NN applications that represent 95% of the TPU's workload. The columns are the NN name; the number of *Lines Of Code - LOC*; the types and number of layers in the NN (FC - fully connected, Conv - convolution, Vector, Pool - pooling).

Name	Loc	Layers				NL Fn	Weights	TPUOps/ Weight B.	TPU Batch size	% deployed TPUs 7/16
		FC	Conv	Vector	Pool					
MLP0	100	5				R	20M	200	200	61%
MLP1	1000	4				R	5M	168	168	
LSTM0	1000	24		34		S,T	52M	64	64	29%
LSTM1	1500	37		19		S,T	34M	96	96	
CNN0	1000		16			R	8M	2888	8	5%
CNN1	1000	4	72		13	R	100M	1759	32	

Table 8.32 shows two examples of each of the three types of NNs, which represent 95% of NN inference workload used at Google datacenters as benchmarks. Benchmarks are small pieces of larger applications that run on the host server, so that TensorFlow codes are short: 100 to 1500 lines. The applications are typically user-facing, which leads to rigid response-time limits. Each model needs between 5M and 100M weights (9th column of the table), which can take a lot of time and energy to access. To amortize the access costs, the same weights are reused across a batch of independent examples during inference or training, which improves performance.

The heart of the TPU is a 65,536 8-bit MAC matrix multiply unit that offers a peak throughput of 92 TeraOps/second (TOPS) and a large (28 MiB) software-managed on-chip memory. MAC stands for multiply accumulate operation:

$$a \leftarrow a + (b \times c).$$

https://en.wikipedia.org/wiki/Multiply%E2%80%93accumulate_operation

The TPU's deterministic execution model is a better match to the 99th-percentile response-time requirement of our NN applications than time-varying optimizations of CPUs and GPUs that help average throughput more than guaranteed latency. The lack of such features helps explain why, despite having myriad MACs and a big memory, the TPU is relatively small and low power.

The TPU is compared to a server-class Intel Haswell CPU and an Nvidia K80 GPU in Table 8.33. The workload written in the high-level TensorFlow framework uses production NN applications (MLPs, CNNs, and LSTMs) that represent 95% of our datacenters' NN inference demand. Despite low utilization for some applications, the TPU is on average about 15X-30X faster than its contemporary GPU or CPU, with TeraOPS/Watt about 30X-80X higher. Moreover, using the CPU's GDDR5 memory in the TPU would triple achieved TOPS and raise TOPS/Watt to nearly 70X the GPU and 200X the CPU.

A recent review of FPGA hardware is reported by Zsolt Istvan (István, 2019)

<https://zistvan.github.io/teaching/fpgas19/debull-draft.pdf>

at IBM (Sukhwani et al., 2015) and Oracle AutoML (Agrawal et al., 2017), (Yakovlev et al., 2020).

A long-term vision for application developers is a full and seamless programming environment that works across CPUs, GPUs, and FPGAs. This could initially focus on support for a common language and runtime, such as OpenCL, and later be extended to additional languages.

<https://en.wikipedia.org/wiki/OpenCL>

TABLE 8.33 Benchmarked servers use Haswell CPUs, K80 GPUs, and TPUs. Haswell has 18 cores, and the K80 has 13 SMX processors.

Model	mm ²	nm	MHz	TDP	Measured Idle/Busy	TOPS/s 8b/FP	GB/s	On-Chip memory	Dies	DRAM	TDP	Measured Idle/Busy
Haswell*	662	22	2300	145	41/45 W	2.6/1.3	51	51 MiB	2	256 GiB	504 W	159/455 W
NVIDIA+	561	28	560	150	25/98 W	-/2.8	160	8 MiB	8	256 MiB + 12 GiBx82	1838 W	357/991 W
TPU	N/A	28	700	75/28 W	40 W	92/-	34	28 MiB	4	256 MiB + 8 GiBx4	861 W	290 W/384 W

TABLE 8.34 Increase of training layers over time.

Network	Year	Application	Layers
LeNet-5 for MNIST*	1998	Handwritten Digital Recognition	7
AlexNet	2012	ImageNet	8
DeepFace	2014	Face recognition	7
VGG-19	2014	ImageNet	19
GoogLeNet	2015	ImageNet	22
ResNet	2015	ImageNet	152
Inception-ResNet	2016	ImageNet	246

The language support could hide any internal differences in compilers and runtimes between the CPU, GPU, and FPGA implementations. Moreover, this seamless programming environment facilitates the end-to-end optimization of resource allocation to take maximum advantage of the collection of available compute and memory capability of the system.

Zero or near zero network weights can be pruned from the graph. Static reduction of weights can account for a modest reduction in weights, while dynamic pruning can achieve considerably higher levels of pruning. Such pruning can considerably reduce the computation requirements for inference operations. Once a model for DL is trained, deploying it for inference in DL applications has additional requirements. While the compute demand for inference is less in terms of complexity, training data, and long training times, there are several key requirements for inference.

In cloud and server systems, inference requirements fall into two categories: low latency single batch, and very large batch high throughput. Inference solutions focus on efficiency of implementation in terms of these requirements. Tasks implemented on CPUs are increasingly being implemented on GPUs and more recently FPGAs, which can implement lower precision arithmetic, pruned networks, local SRAM, and a wide variety of DRAM. Inference solutions implemented on FPGAs can be deployed in a datacenters and embedded environments like surveillance, self-driving cars, IoT edge servers, etc.

8.59 Increasing computational challenges

There are a number of standard DNN configurations, each differing by the number of layers and interconnect patterns between layers. It follows from Table 8.34 that between 2012-16 the number of layers in common DNNs has grown by over 2.3X per year. There have been recent discussions of extending the number of layers in future versions of ResNet to 1,000. DNN training is achieved by fitting the parameters using the stochastic gradient descent and other optimization techniques to arrive at parameter updates. Iterative updates are required to improve the model accuracy with a small incremental improvement rate. Training with large sample sizes such as ImageNet (with 14 million images) and a model such as Resnet can take ≈ 30 -40K iterations to converge to a stable solution. As the DNN complexity and the size of the training set both increase, the total computational load, as a high-level estimation, can exceed 10^{20} FLOPS.

LeNet was trained in handwriting recognition on MNIST data.

<https://resources.wolframcloud.com/NeuralNetRepository/resources/LeNet-Trained-on-MNIST-Data>

Very Deep Convolutional Networks for large-scale Image Recognition - VGG (Simonyan and Zisserman, 2015). is also based on CNNs, and was applied to the ImageNet Challenge in 2014 achieving 92.7% top-5 test accuracy on the ImageNet dataset. Major improvements of VGG over AlexNet included using large kernel-sized filters (sizes 11 and 5 in the first and second convolutional layers) with multiple (3x3) kernel-sized filters, one after another. Three DNNs are described here.

<https://blog.paperspace.com/popular-deep-learning-architectures-alexnet-vgg-googlenet/>

8.60 Quantum Neural Nets - QNNs

The following discussion is based on a recent blog compares *Quantum Computing - QC* and deep learning.

<https://www.datasciencecentral.com/profiles/blogs/quantum-computing-deep-learning-and-artificial-intelligence>

QNNs combine ANNs with the advantages of quantum information processing in order to develop more efficient algorithms. One investigation is motivated by the difficulty to train classical NNs for big data applications and the hope that features of quantum computing such as quantum parallelism can be used for this purpose. Since the technological implementation of a quantum computer is in a premature stage, such QNNs are mostly theoretical proposals that await their full implementation in physical experiments.

The blog states that QCs can work as a DNN. Of the several barriers to DL is simply the time and compute resources necessary to train a CNN or RNN, which can take weeks of compute time on hundreds of GPUs to train complex deep nets. Renting 800 GPUs from Amazon's cloud computing service for a week costs ≈\$120,000. The hope is that QC will be able to reduce that to minutes or seconds using only a single device.

The D-Wave approach is based on the concept of quantum annealing or Adiabatic QC.

<https://www.dwavesys.com/>

It has been shown that this can be modeled by a multi-layer Restricted Boltzmann Machine, which is a deep learning ANN. There are important architectural differences between the way QC works and current deep nets, particularly in the way back propagation is handled by adjusting the model after each iteration to get closer to the desired answer and it literally means applying values from the results end of the ANN back to layers at the front of the ANN before starting over again. Researchers have already designed an accommodation by adding additional functional hidden layers that provide the same result as back prop.

Quantum Enhanced Learning is the application of Quantum to speed up or enhance traditional machine learning methods. Yet to come is pure Quantum Learning which will use quantum effects to yield predictive results using methods that look nothing like our current techniques.

According to a presentation at the IEEE Workshop on Benchmarking Quantum Computational Devices and Systems at the *Int'l Conf. on Rebooting Computing - ICRC 2019* in San Mateo, CA.

<https://icrc.ieee.org/>

benchmarking of quantum computers in the near and medium term will deal quantifying all sources of error and understanding how these error sources impact applications. Processor speed will be important someday, but not yet.

<https://quantum.ieee.org/education/quantum-supremacy-and-quantum-computer-performance>

8.61 Data acceleration examples

This section discusses two designs and the use of AI in accelerator design at Google (Yazdanbakhsh et al., 2021). The design of the Q100 *Data Processing Unit - DPU* at Columbia U. in Subsection 8.61.1. Many-core architecture for in-memory data processing at Oracle in Section 8.61.2.

8.61.1 The Q100 data processing unit

The ISA, the micro-architecture, and hardware implementation of Q100 DPU developed by Kenneth Ross et al. at Columbia U. is described in (Wu et al., 2014), (Wu et al., 2015). Q100 contains a heterogeneous collection of fixed-function ASIC tiles, which can process relational tables and columns quickly and energy-efficiently as specified in Table 8.35. Q100 uses coarse grained instructions that manipulate streams of data, thereby maximizing pipeline and data parallelism, and minimizing the need to time multiplex the accelerator tiles and spill intermediate results to memory. This work explores a Q100 design space of 150 configurations, selecting three for further analysis: a small, power-conscious implementation, a high-performance implementation, and a balanced design that maximizes performance per Watt. The Q100 handles the TPC-H queries with three orders of magnitude less energy than a state of the art software DBMS, while the performance-oriented design out-performs the same DBMS by 70X.

Intel E5620 Xeon server with 2 chips. Each chip contains 4 cores 8 threads running at 2.4 GHz with 12 MB LLC, 3 channels of DDR3, providing 24 GB RAM. Comparisons are done using estimated single core area and power consumption derived from published specification.

TABLE 8.35 Design characteristics of Q100 tiles post place and route, and compared to a Xeon core. *The slowest tile, the partitioner, determines the frequency of Q100 at 315 MHz.

Tile	Area		Power		Crit. Path ns	Width (bits)		
	mm ²	%Xeon	mW	%Xeon		Record	Column	Comparator
Aggregator	0.029	0.07%	7.1	0.14%	1.95		256	256
ALU	0.091	0.21%	12.0	0.24	%0.29		64	64
BoolGen	0.003	0.01%	0.2	< 0.01%	0.41		256	256
ColFilter	0.001	< 0.01%	0.1	< 0.01%	0.23		256	
Joiner	0.016	0.04%	2.6	0.05%	0.51	1024	256	64
Partitioner	0.942	2.20%	28.8	0.58%	*3.17	1024	256	64
Sorter	0.188	0.44%	39.4	0.79%	2.48	1024	256	64
Append	0.011	0.03%	5.4	0.11%	0.37	1024	256	
ColSelect	0.049	0.11%	8.0	0.16%	0.35	1024	256	
Concat	0.003	0.01%	1.2	0.02%	0.28		256	
Stitch	0.011	0.03%	5.4	0.11%	0.37		256	

8.61.2 Many-core architecture for in-memory data processing at Oracle

The problem with multicore as discussed by David Patterson translates to the issue of ease of parallel programming.

<https://spectrum.ieee.org/computing/software/the-trouble-with-multicore>

The article lists languages which allow parallel programming, but it does not mean languages make parallel programming easy.

https://en.wikipedia.org/wiki/List_of_concurrent_and_parallel_programming_languages

Few universities offer good courses in parallel programming.

<http://cs149.stanford.edu/fall20>

Highest energy cost is expended in processing is data movement rather than computation. That energy is the limiting factor in processor design is addressed in (Agrawal et al., 2017). As the application data grows to exabytes there is an opportunity to design a bandwidth-optimized architecture for big data computation by specializing hardware for data movement. The presented *Data Processing Unit - DPU* is a shared memory many-core that is specifically designed for high bandwidth analytics workloads.

The DPU contains a unique *Data Movement System - DMS*, which provides hardware acceleration for data movement and partitioning operations at the memory controller that is sufficient to keep up with *Double Data Rate - DDR* bandwidth (Jacob et al., 2008).

https://en.wikipedia.org/wiki/DDR_SDRAM

The DPU also provides acceleration for core to core communication via a unique hardware RPC mechanism called the Atomic Transaction Engine. Comparison of a DPU chip fabricated in 40 nm with a Xeon processor on a variety of data processing applications shows a 3x-15x performance per watt advantage.

The primary goal of the DPU is to optimize for analytic workloads characterized by (i) large dataset sizes (tens of TB), (ii) data parallel computation, (iii) complex data access patterns that can be made memory bandwidth-bound using software techniques.

Analytic workflow involves partitioning the working set into several data chunks to be independently analyzed, followed by a final result aggregation stage, hence an ideal architecture for such a memory-bound workload would aim to compute at memory-bandwidth. This requires keeping the workload memory-resident, with DRAM memory channels feeding data into the processing units.

With a practical memory channel bandwidth of 10 GBps to scan a nominal workload size of 10 TB in under a second requires ≈ 1000 channels per rack. This results in 3 KW (at 3 W per channel) budgeted for main memory and given 20 KW per rack this leaves only 17 W for the rest of the system (per channel) including networking between the DPUs. A standard PCIe controller consumes a minimum of 10 W, leaving a power budget of < 7 W for the processor. Two principles therefore guide the DPU's architecture: (1) specialize hardware to optimize for data movement, and (2) replace power-hungry hardware features which are not performance critical for data parallel applications with low power hardware assists which allow software to implement the feature.

HyperLogLog is an algorithm for the count-distinct problem, approximating the number of distinct elements in a multiset. Calculating the exact cardinality requires an amount of memory proportional to the cardinality, impractical for very large data sets. Probabilistic cardinality estimators, such as the HyperLogLog algorithm, use significantly less memory at the cost of obtaining an approximation of the cardinality. The HyperLogLog algorithm is able to estimate cardinalities of $> 10^9$ with standard error of 2%, using 1.5 kB of memory. HyperLogLog is an extension of the earlier LogLog algorithm derived from the 1984 Flajolet–Martin algorithm.

<https://en.wikipedia.org/wiki/HyperLogLog>

8.62 Cerebras wafer size chips vs GPUS

The following discussion is mainly based on Cerebra white paper.

<https://www.cerebras.net/wp-content/uploads/2019/08/Cerebras-Wafer-Scale-Engine-An-Introduction.pdf>

Cerebras founded by Andrew Feldman announced the CS-1 chip on September 2019,

<https://www.linkedin.com/in/andrewfeldman/>

which is the world's largest computer 1.2 trillion transistor *Wafer Scale Engine* - WSE. CS-1 is more than 56x larger than the largest GPU containing 3,000x more on chip memory and more than 10,000x the memory bandwidth.

Wafers are sliced from a 12-inch-diameter ingot of silicon into hundreds of separate chips. Pieces of the chip dubbed core are interconnected to other cores. The interconnections allow all the cores functioning at high speeds together. CS-1 is 200 times faster than the Joule Supercomputer which uses 16,384 cores and is No. 82 on a list of the top 500 supercomputers in the world. Cerebras is 10,000 faster than a GPU, it costs several million dollars and uses 20 kilowatts of power.

<https://venturebeat.com/2020/11/17/cerebras-wafer-size-chip-is-10000-times-faster-than-a-gpu/>

An interesting aspect of CS-1 is that it can finish calculations faster than real time, meaning it can start the simulation of a power plant's reaction core when the reaction starts and finishes the simulation before the reaction ends.

Google, Facebook, and Baidu, among others, have noted that the cost for training (thousands of dollars) is the fundamental impediment to AI progress.

8.62.1 The deep learning workload

A *Neural Network* - *NN* is a sequence of layers, arranged from the input **I** where data flows in to the output **O**, where classification predictions are output, The function of each layer is parameterized by the model parameters of that layer, dominated by simple, highly parallel operation, such as multiplying a vector (the input data) by a matrix (the model parameters).

In inference, the parameters are fixed and data flows through in one direction from **I** to **O**. At **O**, we get a result, but during training, data flows both ways. A labeled set of input data, called the training set, flows from **I** to **O**. At the output, rather than providing an answer, the output is compared to the correct answer for that particular input. The network improves the accuracy of I-to-O calculation by reducing the errors it made. The process is as follows: with each new training sample, the network changes how it makes its output predictions such that it is continuously reducing the error; error being the difference between the predicted output and the correct output.

This information in the form of a gradient begins to work its way backwards, in the **O**-to-**I** direction. As this data flows through each layer, it interacts with the **I**-to-**O** data that produced it, determining how to change the layer's parameters to most effectively reduce the error. The parameters are then adjusted, and this process continues for numerous passes over the set of training examples, i.e., a multi-stage training loop is created.

The time required to train a network depends on the rate at which inputs can be processed through this feedback loop. The faster the inputs can move through the loop, the more inputs are sent through the loop per unit time and the faster the network trains.

8.62.2 Understanding performance

Training time can be reduced by reducing the time it takes for the inputs to travel through the feedback loop. The only things happening in the feedback loop are calculations and communications.

Accelerating calculation is achieved by increasing the number of compute cores, specifically more FLP *Multiply Accumulate* - *MAC* units do more calculations in less time. Placing high-speed memory close to cores ensures that the cores are always occupied doing calculations. Placing memory off chip means the cores are frequently waiting for the data on which to do their calculations.

High-bandwidth, low-latency communication ensures that compute cores can be gathered together into communicating groups that quickly solve problems that would take a single core too long to do. In addition, high-bandwidth, low-latency communication ensures that the results of each stage in the loop are quickly delivered to the subsequent stage.

For training, GPUs with thousands of cores have dominated CPUs with dozens of cores, since the big = core CPUs were not designed for the type of calculations in a multistage training loop, neither the little cores in GPUs, but that effect is offset by their number. Nvidia's GPUs have grown from 551 mm² to 815 mm², since more area provides more room for cores and more computing power and more memory close to cores.

However, as AI compute became more demanding, the number of cores needed in the training loop exceeds the number of cores on a single chip. More cores can be achieved by adding chips to build a cluster, but off-chip communication is four orders of magnitude slower than on chip communication.

NVIDIA designed NVLink as a way to improve communication among chips by paying \$6.8 billion to acquire Mellanox, which is the company that pioneered InfiniBand.

<https://www.mellanox.com/>

Moving the memory onto the chip increases the calculation performance of nearby cores. Cores tuned for AI work get more calculations per unit time or that they avoid doing useless work like multiplying by zero. Optimizations that provide more calculations and avoid wasting time on non-useful work will also improve performance. All of these strategies are trying to drive up calculation and accelerate communication through one or more of three strategies: (1) more and better cores, (2) more memory close to cores, and (3) more low-latency bandwidth between cores.

A large wafer with sufficient silicon area allows numerous cores and would distribute memory across the chip, giving each core its own local memory. A communication fabric across all the cores would be efficient since it would be on-die, and many thousands of times faster than off-chip technologies like InfiniBand.

This been done before because it is very difficult. Nobody has ever built and brought to market a chip larger than 840 mm². The design, manufacturing, power, cooling, communication and coordination challenges are immense. The resulting chip would be 50x larger than the largest chip on the market today, several 100,000s of AI optimized cores and GBs of on chip memory distributed across the cores and petabytes/s of core to core bandwidth.

Before Cerebras there were other attempts to build wafer-scale chips. These included Trilogy in US and Anamartic in UK.

https://en.wikipedia.org/wiki/Wafer-scale_integration

<https://www.nytimes.com/1984/07/08/business/can-troubled-trilogy-fulfill-its-dream.html?pagewanted=all>

<http://www.computinghistory.org.uk/det/4619/Anamartic-Wafer-Stack/>

8.62.3 More silicon area means more space for compute cores

The Cerebras *Wafer Scale Engine* - WSE delivers 400,000 programmable compute cores. The *Sparse Linear Algebra* - SLA cores, are optimized for operations fundamental to neural network calculation. They are small and fast, have no caches and other features in general purpose cores.

The programmability of SLA cores ensures that they can run the NN algorithms of the constantly changing field of DL. Each core performs both data processing and control processing, which is required for parallel processing coordination. Control processing is achieved with a full set of general-purpose instructions. These instructions provide programmable primitives such as arithmetic, logical, and branching operations. These primitives provide the foundation on which any parallel algorithm can be mapped.

Tensor operations are the core of the neural network workload. To achieve high performance, the SLA cores have a specialized tensor processing engine where full tensors are first-class operands in architecture. The tensor operations are programmable, so the same engine can be programmed to perform a variety of tensor operations such as convolution or matrix multiply. The hardware internally optimizes the tensor processing to achieve datapath utilization three of four times greater than graphics processing units.

Cerebras invented sparsity harvesting technology, which allows the SLA Cores to improve performance by harvesting the plentiful sparsity in NN workloads, such as multiplying by zero is a waste of silicon, power, and time.

Half to nearly all the elements in the vectors and matrices that are to be multiplied together are zeros. The source of the zeros are fundamental DL operations, such as the *Rectified Linear Unit* - ReLU nonlinearity and dropout, both of which introduce zeros into neural network tensors. Newer methods are emerging around weight sparsity that add even more zeros. Put simply, there is a growing interest in making tensors sparser still.

GPUs and TPUs are dense execution engines; they perform the same computation task on an entire vector or matrix of data. This is a wise approach when the vector or matrix is dense. In the dense environment, efficiencies are gained by

bunching the data together and providing a single instruction to be applied to all of the data. But when the data is 50-98% zeros, as it often is in NNs, then such multiplications are wasted.

Because the Cerebras SLA core was designed specifically for NNs, it never multiplies by zero. To take advantage of this sparsity, the core has built-in, fine-grained dataflow scheduling, so that computing is triggered by non-zero data. All zeros are filtered out and can be skipped in the hardware. An SLA core saves the power and energy by skipping useless computations, but it also gains performance advantage by using that same time to do useful work instead while other architectures are stuck multiplying by zero.

Redundant cores and interconnections are provided to deal with defects.

https://www.hotchips.org/hc31/HC31_1.13_Cerebras.SeanLie.v02.pdf

An article on Cerebras appeared in the Jan. 2000 issue of IEEE Spectrum.

<https://spectrum.ieee.org/semiconductors/processors/cerebras-giant-chip-will-smash-deep-learning-speed-barrier>

8.62.4 More silicon area means more on chip memory

The closer memory is to a compute core, the faster the calculation, the lower the latency, and the less power is used for moving data. High-performance DL requires each core to operate at maximum levels and this requires close collaboration and proximity between the core and memory. In DL memory is used to hold the model's parameters, the activations, the model configuration, and more. In state-of-the-art networks, the model parameters already run into the GBs, and this memory requirement is expected to grow. But large models, needing a lot of memory and a lot of compute, create fundamental challenges for traditional architectures.

For maximum performance, the entire model should fit in the fastest memory, which is the memory closest to the computation cores. This is not the case in CPUs, TPUs, and GPUs, where main memory is not integrated with compute. Instead, the vast majority of memory is based off-chip, far away on separate DRAM chips or a stack of these chips in a *High Bandwidth Memory - HBM* device. As a result, main memory is excruciatingly slow.

While computer architects have tried for years to address the memory bottleneck, the primary solution has been memory hierarchies, based on many levels of on-chip and near-chip caches. These are costly, small, difficult to use, and provide sometimes unpredictable benefits. The simple and unavoidable truth is that once the on-chip/off-chip boundary is crossed, the latency to memory explodes and the bandwidth plummets. The extreme latency penalty in accessing off-chip memory forces a downward spiral of performance. This is one of the fundamental reasons graphics processors are slow when doing artificial intelligence work.

Cerebras solved this problem, since Cerebras WSE has 18 GBs of on chip memory and 9.6 PB/s of memory bandwidth. Respectively this is 3,000 and 10,000 times more than the leading graphics processing unit. As a result the WSE keeps the entire NN model, that is all the parameters to be learned, on the same silicon as the compute cores, where they can be accessed at full speed. This is possible because memory on the WSE is widely distributed alongside the computational elements, allowing the system to achieve extremely high memory bandwidth at single-cycle latency, with all model parameters in on-chip memory, all of the time. The Cerebras WSE delivers more compute cores, more local memory, and more memory bandwidth than any chip in history. This enables fast computation, reduces the time it takes to train a model, and uses less energy as well.

8.62.5 Fast communication Swarm fabric

Performance improvements also come from accelerating communication, since AI is a communication intensive workload—layers, and therefore cores, communicate constantly—so the fabric that links cores together is fundamental to performance. Maximum performance is achieved when cores can communicate at high bandwidth and low latency since the cores can be clustered together to do in a group what would take an individual core too long to do.

Off-chip communication is tens of thousands of times slower than on-chip communication. Small chips that wish to communicate must be clustered together via Ethernet, InfiniBand, PCI-E, or other off-chip technologies, which all suffer an enormous performance penalty compared to staying on-silicon. On-chip communication is faster, and it consumes less than a thousandth the power per bit moved. The Cerebras Swarm communication fabric creates a massive on chip network that delivers breakthrough bandwidth and low latency at a fraction of the power draw of the traditional communication techniques used to cluster graphics processing units into clusters. The 400,000 cores on the Cerebras WSE are connected via the Swarm communication fabric in a 2D mesh with 100 Petabits per second of bandwidth. Swarm provides a hardware routing engine to each of the compute cores and connects them with short wires optimized for latency and bandwidth. The resulting fabric supports single-word active messages that can be handled by the receiving cores without any software overhead. The fabric provides flexible, all-hardware communication.

The Cerebras software configures all the cores on the WSE to support the precise communication required for training the user-specified model. For each neural network, Swarm provides a unique and optimized communication path, which is different than the approach taken by CPUs and GPUs that have one hard-coded on-chip communication path into which all NNs are forced to be included. Swarm messages traverse one hardware link with nanosecond latency. The aggregate bandwidth of the system is measured in tens of petabytes per second. Communication software such as TCP/IP and MPI is not needed, avoiding associated performance penalties. The energy cost of communication in this architecture is well below one picojoule per bit, which is nearly two orders of magnitude lower than central processing units or graphics processing units. As a result of the Swarm communication fabric, the WSE trains models faster and uses less power.

8.62.6 Co-designed with software for maximum utilization and usability

The Cerebras software stack has been tightly co-developed with the WSE to take full advantage of its unique capabilities. Cerebras graph compiler enables automatic translation of an ML NN into an optimized executable for the massive computational resources of the WSE. To make it easy and intuitive for ML practitioners to use the WSE, the Cerebras software stack provides an interface to existing high-level ML frameworks, such as TensorFlow and PyTorch. The graph compiler begins by extracting a dataflow graph representation of the NN from the user-provided framework representation.

Portions of the network are matched to optimized microcode programs, and these programs are mapped to specific resource regions of the immense computational array. The software then configures the Swarm interconnect fabric, creating a tailored data path through these regions of compute to maximize locality and minimize communication cost. The result is a highly optimized placement of the neural network onto the WSE, ready for the user to perform training or inference. In addition to traditional execution modes, the scale of the WSE enables novel methods of model-parallel execution. The WSE is capable of running an entire NN on the fabric at once – mapping each layer of the network to a single stage in a multi-stage pipeline for full, layer-parallel, pipelined execution. The user can then rapidly stream data through the pipeline, running all stages and thus, all layers in the neural network, simultaneously. This approach is unique to the WSE and is only possible because of its immense scale.

The Cerebras Software Stack

1. allows users to program the WSE using familiar, high-level ML frameworks;
2. automatically compiles ML framework-defined models into WSE executables;
3. maximizes utilization of all 400,000 cores and takes advantage of the WSE's unique scale.

A recent article dealing with Cerebras is as follows.

<https://www.newyorker.com/tech/annals-of-technology/the-worlds-largest-computer-chip>

Google's use of AI in chip design

Google Brain director Jeff Dean spoke at *Int'l Solid State Circuits Conf. - ISSC'20* about how the company is using AI to advance its internal development of custom chips. AI in some cases make better decisions than humans about how to lay out circuitry in a chip.

<https://ai.googleblog.com/2021/02/introducing-model-search-open-source.html>

“Apollo: Transferable Architecture Exploration” (Yazdanbakhsh et al., 2021) represents an intriguing development that moves beyond Dean's hint that ML could be used for some low-level design decisions, known as “place and route”, which determines the layout of the circuits that form the chip's operations.

<https://www.zdnet.com/article/googles-deep-learning-finds-a-critical-path-in-ai-chips/>

Optimizing accelerator configurations of a diverse set of neural architectures with alternative design constraints is explored in (Yazdanbakhsh et al., 2021). The framework finds high reward design configurations (up to 24.6% speedup) more sample-efficiently than a baseline black-box optimization approach. It is further shown that by transferring knowledge between target architectures with different design constraints, Apollo is able to find optimal configurations faster and often with better objective value (up to 25% improvements). This encouraging outcome portrays a promising path forward to facilitate generating higher quality accelerators.

8.63 Conclusions

We have described several techniques to improve the performance of database systems, most notably relational databases which are amenable to parallel processing. Relational databases were initially implemented on general purpose “main-frame” computers, but more efficient shared nothing computers were later introduced to carry out data mining, business intelligence, and decision support applications.

Teradata DBC/1012 was the first commercial shared nothing system using a large number of microprocessors for database applications such as data warehousing. Gamma and Bubba prototypes led to new algorithms for processing relational operators on DBMs.

HDDs are constrained by disk bandwidth in carrying out random accesses, but not disk capacity. Active disks were programmed to process low priority disk requests via freeblock scheduling, while the disk arm is moving to satisfy high priority requests. The processor associated with a disk used for OLTP could be programmed to carry out system activities, such as disk scrubbing or data mining.

Some early DBMs were based on enhancing the Read/Write heads in disk drives for parallel search such as RAPID, and relational algebra operators, as in the case of RAP - Relational Associative Processor. Given the very high radial track density of current disks such heads per track disks are not feasible anymore.

Note that disks are no longer the dominant technology for processing data as high performance storage, but this concept has been applied by associating processors with DRAM and flash memories as in the case of FAWN, Lighthouse, RAMCloud, and BlueBDM.

Hybrid OLTP and analytic processing has been extended to combining ML with DBMSs. A more recent development is DL which is accomplished with neural networks. GPUs, FPGAs, and ASICs are used to enhance CPUs, which is the motivation for NVIDIA with ARM and AMD with Xilinx mergers.

“The Future of Deep Learning Is Photonic Computing with light could slash the energy needs of neural networks” according to Ryan Hamerly.

<https://spectrum.ieee.org/computing/hardware/the-future-of-deep-learning-is-photonic>

Back in 2004 Adam Bosworth, wrote the following article, which is summarized later.

<https://adambosworth.net/2004/12/29/where-have-all-the-good-databases-gone/>

Bosworth argues that databases don’t scale as they should. People are spoiled by Google’s indexing and millisecond search capabilities, something that today’s database products don’t provide. Databases need more flexibility to change; needed is dynamic schema so that as the business model/description of goods or services changes and evolves, this evolution can be handled seamlessly.

There is also the issue of *Roll Your Own - RYO* hardware as noted below.

<https://redmonk.com/sograzy/2013/03/21/roll-your-own-hardware/>

It turns out hardware vendors are vulnerable to the same trend, which is surprising intuitive given the perceived difference between authoring code and manufacturing hardware, which requires an upfront capital and expertise gained in several iterations.

It is known that Google’s servers are not products of computer vendors but rather machines it designed itself. However Google’s RYO is considered to be the exception, not the rule.

And for the most part, industry analysts tracking the hardware market implicitly validated these claims, because their numbers reflected minimal competition for traditional hardware suppliers. But as discussed in a 2011 Wired article by Bob McMillan, it was clear that the picture painted by the analyst reports was at best incomplete, because there is significant under-reporting from *Original Device Manufacturers - ODMs* and as noted by Andy Bechtolsheim (SUN co-founder in 1982):

“It’s hard to get those numbers because they’re not reported to IDC or any of the market firms that count servers.”

Intel has crowned Google as the world’s fifth largest server Rackspace, which was historically been 60% Dell and 40% HP wants to use its own designs.

<https://www.rackspace.com/>

Chapter 9

Structured, unstructured, and diverse databases

Categories of file systems in Section 9.1
Mainframe Count-Key-Data Disk - CKD organizations in Section 9.2
Hierarchical and network *Data Base Management Systems* - *DBMSs* in Section 9.3
Information Management System - IMS in Subsection 9.3.1
Transaction Processing Facility - TPF in Subsection 9.3.2
Network databases in Subsection 9.3.3
Relational data model in Section 9.4
Methods to reduce the level of lock contention in OLTP in Subsection 9.4.1
Informix in Subsection 9.4.2
Berkeley's INGRES relational database in Subsection 9.4.3
Oracle's relational databases in Subsection 9.4.4
Ranking methodology for database engines in Section 9.5
Overall ranking of all database types in Section 9.6
Relational database management systems in Section 9.7
Object relational databases in Section 9.8
Data mining in Section 9.9
Data warehousing and OLAP in Section 9.10
Distinct schools of thought in data warehouse design in Section 9.11
The Inmon approach in Subsection 9.11.1
The Kimball approach in Subsection 9.11.2
Deciding factors for Kimball vs Inmon approaches in Subsection 9.11.3
Conclusions on data warehousing in Subsection 9.11.4
Data lakes in Section 9.12
Open source big data projects in Section 9.13
Semi-structured data and its model in Section 9.14
Big data technology and the five Vs in Section 9.15
Hadoop technology ecosystem in Section 9.16
Distributed batch vs inline processing in Section 9.17
NoSQL/non-relational databases in Section 9.18
Key-value stores in Section 9.19
Document stores in Section 9.20
Time-series databases in Section 9.21
Kubernetes and other containers in Section 9.22
Graph databases in Section 9.23
Object-oriented databases in Section 9.24
Search engines for text in Section 9.25
Web search engines in Section 9.26
Resource Description Framework - RDF in Section 9.27
Wide column stores in Section 9.28
Multivalued databases in Section 9.29
Native XML databases in Section 9.30
Realtime processing in Section 9.31
Event stores in Section 9.32

Streaming analytics in Section 9.33

Aurora database to manage data streams in Subsection 9.33.1

Trill: a high-performance incremental query processor for diverse analytics in Section 9.34

Summary of Forrester WaveTM streaming analytics, Q3, 2109 in Section 9.35

Content stores in Section 9.36

Multimodel databases in Section 9.37

Main memory databases in Section 9.38

Distributed file systems and object storage in Section 9.39

Enterprise backup and recovery software solutions in Section 9.40

Analytics and Business Intelligence - ABI platforms in Section 9.41

Blockchain, bitcoin, Ethereum in Section 9.42

9.1 Categories of file systems

Early file systems were held on punched cards and later on magnetic tapes, which lent themselves to sequential processing. Magnetic disks which became popular later allowed random access to data. A similar capability is available with databases held in DRAM and flash memories.

COBOL which used to be the most common programming language for commercial applications supports three types of file systems: sequential, index sequential, and relative file organization.

https://www.tutorialspoint.com/cobol/cobol_file_organization.htm

PL/I developed in conjunction with S/360 computers supported three types of file access methods: sequential, *Index Sequential Access Method - ISAM*, *REGIONAL(1)*, but *Virtual Sequential Access Method - VSAM* was introduced later. VSAM, which is IBM's version of a B+-tree is internally supported by DB2's query optimizer (Keehn and Lacy, 1974), (IBM Corp., 1974), (Bohl, 1981), (Wiederhold, 1987), (IBM Corp., 2013).

Sequential files are usually sorted according to a key. Updates can be applied efficiently from one sequential file, e.g., based on updates, to another (a master/file - M/F), by sorting it according to the same key. Updates are not in place and the modified M/F is written to another tape. To insert/delete a single record in a sorted file the remaining records in the file have to be moved back/forward one position. For deletions a field in each record is used to designate a record as deleted without removing it. Sorting has the advantage that a missing key can be determined by scanning half of the file on the average, rather than the whole file when it is not sorted.

ISAM files are sorted according to a key which is used for indexing (IBM Corp., 1973), (IBM Corp., 1974), (Bohl, 1981), (Wiederhold, 1987), (Ramakrishnan and Gehrke, 2002). The structure of ISAM files is static and is determined at the time the index is first built using a sorted input file, since this results in improved file structure and less disk arm positioning time as the file is being built.

<https://en.wikipedia.org/wiki/ISAM>

ISAM files require multiple empty disk cylinders for construction. As part of a class project at UCLA I had to try several disks attached to UCLA's IBM 360/91 computer to find a disk with multiple empty disk cylinders. The first cylinder in ISAM holds cylinder indexes specifying the largest index on cylinders that follow. The cylinder determines the cylinder number on which a target record resides. The COREINDEX option allows the cylinder index to be held in main memory to save a disk access. In the usual case of blocking multiple logical records per block, the key is that of the last block record. The first track on each cylinder determines the track on that cylinder on which a record is located. ISAM files can be dedicated a few tracks on each cylinder as overflow areas, but an independent overflow area with several cylinders may be provided. Performance degrades as overflow areas are filled.

B+-trees or IBM's *Virtual Sequential Access Method - VSAM* (Lovelace et al., 2013)

https://en.wikipedia.org/wiki/Virtual_Storage_Access_Method

is a dynamic version of static ISAM files. I B+-trees reconfigure dynamically into a balanced tree as records are added and deleted. We skip details since a good description is provided in (Ramakrishnan and Gehrke, 2002).

An interesting observation on B+-trees is reported in (Glombiewski et al., 2019). A newly created B+-tree index may speed up database processing after its creation, but with database updates which cause node splits results in a "wave of misery" due to effort required for frequent node splits. Free space left during index creation may merely delay the onset of the first wave. A theoretically sound way to avoid these waves of misery as well as some simple and practical means to reduce their amplitude is proposed in this work.

A fractal tree index is a data structure that keeps data sorted and allows searches and sequential access in the same time as a B+-tree, but with insertions and deletions it is asymptotically faster than a B+-tree. Like a B+-tree a node can have

more than two children, but unlike a B+-tree a fractal tree index has buffers at each node, which allow insertions, deletions and other changes to be stored in intermediate locations. The goal of the buffers is to schedule disk writes so that each write performs a large amount of useful work, thereby avoiding the worst-case performance of B+-trees, in which case each disk write may change a small amount of data on disk. Like a B+-tree, fractal tree indexes are optimized for systems that read and write large blocks of data is commercialized in TokuDB by Tokutek.

https://en.wikipedia.org/wiki/Fractal_tree_index

<https://en.wikipedia.org/wiki/TokuDB>

The rate at which two million 200 byte files in a shallow tree could be created with ext4, xfs, btrfs, azfs and tokufs for one, two, four threads are reported in this study and tokufs achieves the best performance is shown in Table 9.1 based on (Esmet et al., 2012).

TABLE 9.1 Microfile creation and scan performance in a directory hierarchy. The first column names the file system, the next three columns show write performance in files per second with varying number of threads, and the last column shows the scan rate, that is how many files per second can be traversed in a recursive directory walk.

# threads	1	4	8	scan (thread)
ext4	217	365	458	10,629
XFS	196	154	143	731
Btrfs	928	671	560	204
ZFS	44	194	219	303
Tokufs	17,088	16,960	16,092	143,006

The 4th extended files - *ext4* introduced in 2008 is a journaling file system for Linux.

<https://en.wikipedia.org/wiki/Ext4>

XFS is a high-performance 64-bit journaling file system created at *Silicon Graphics, Inc. - SGI* in 1993.

<https://en.wikipedia.org/wiki/XFS>

Btrfs - B-tree File System is based on the *Copy-On-Write - COW* principle, initially designed at Oracle Corp. for use in Linux.

<https://en.wikipedia.org/wiki/Btrfs>

The experiment deals with five million 200-byte files in a balanced directory hierarchy in which each member contains at most 128 entries. Tokufs is faster than the other file systems by one to two orders of magnitude for both reads and writes.

The ext4 file system performs better than the other traditional file systems on the scan, probably because its hashed directory scheme preserves locality on scans.

Btrfs does well on writes, compared to the other traditional file systems. Btrfs gets the advantage of a log-structured file system for creating files, but suffers on reads since the resulting directory structure loses spatial locality.

ZFS performs poorly, although it does better on a higher thread write workload. This is perhaps because ZFS requires a disk sync to create a file.

<https://en.wikipedia.org/wiki/ZFS>

XFS performs poorly on file creation, but relatively well on scans.

COBOL's relative file consists of records ordered by their relative address. The key attributes of relative file organization are as follows:

1. Records can be read in sequential order just like sequential and ISAM files.
2. Records can be accessed using the relative key.
3. Records can be inserted using relative key with the address being calculated using relative key.
4. Relative file provides the fastest access to the records.
5. The main disadvantage of this file system is that if some intermediate records are missing, they will also occupy space.

PL/I's REGIONAL(1) datasets do not have recorded keys and the region number serves as the sole identification of a particular record. The character value of the source key should be an unsigned decimal integer not exceeding 16,777,215. If the region number exceeds this figure, it is treated modulo $2^{24} = 16777216$. Regional(1) files accessed sequentially will return records in a random order, which will need to be sorted.

Several techniques to calculate the address of a record with direct organization are given in (IBM Corp., 1974) and a detailed flowchart of ISAM and *Direct Access Method - DAM* are provided in (IBM Corp., 1973).

Relative track address: This is specified by 3 bytes as TTR, where TT is the relative track number and R the relative block number in a track.

Relative track and key: The user specifies a 3-byte binary number, which the system converts to a relative track address, which is searched using the key.

Relative block address: The user specifies a 3-byte binary number, which is converted to the relative block number.

Actual address: The user specifies 8 bytes MBCCCHHR.

Example: Consider a 2314 disk drive whose tracks can hold twenty 200 byte records.

1. With 80% packing 10,000 locations are required.
2. With twenty records per track 500 tracks are required.
3. A prime number close to 500 is 499.
4. Divide the key by 499 and the remainder determines relative tracks address.

Extensible and linear hashing are two methods to deal with additions and deletions in hashing (Ramakrishnan and Gehrke, 2002). A simulation study comparing the two methods is reported in (Rathi et al., 1990).

9.2 Mainframe count-key-data disk organizations

IBM mainframes utilized large form factor disks referred to as in (Patterson et al., 1988) *Single Large Expensive Disks - SLEDs* (Patterson et al., 1988). Such disks have physical block sizes, which are multiples of application program's logical record sizes according to *Extended Count Key Data - ECKD* format (IBM Corp., 1974), (Bohl, 1981), (Wiederhold, 1987). The RAID paradigm was a proposal to reduce the cost of storage hardware by emulating expensive IBM *Direct Access Storage Devices - DASD*, such as 3390s with 3.5" disk drives with fixed sized blocks - 512 byte sectors.

https://www-03.ibm.com/ibm/history/exhibits/storage/storage_3390.html

https://www.ibm.com/ibm/history/exhibits/storage/storage_chrono20.html

IBM discontinued production of DASD/SLEDs and switched to small form factor 3.5" disk drives in 1992 which is the year it sold its disk business to Hitachi for \$2.05 B and *Hitachi Global Storage Technologies - HGST* was formed.

Originally CKD records had a one-to-one correspondence to a physical track of a DASD device; however over time the records have become virtualized such that in a modern IBM mainframe there is no longer a direct correspondence between the CKD record ID and a physical layout of a track. An IBM mainframe constructs ECKD track images in memory and executes the channel programs against the image. To bridge between the native fixed block sized disks and the variable length ECKD/CKD record format, The ECKD track in memory is compacted and mapped onto a series of fixed blocks of an FBA disk subsystem. Of the 83 CKD CCWs implemented for System/360 and System/370 channels 56 are emulated on System/390 and later systems.

Specifying the default device geometry: z/OS DFSMSdfp Storage Administration, SC23-6860-01.

https://www.ibm.com/support/knowledgecenter/en/SSLTBW_2.1.0/com.ibm.zos.v2r1.idas200/s2057.htm

When allocating space for a new data set on DASD, *Storage Management System - SMS* converts all space requests in tracks (TRK) or cylinders (CYL) into requests for space in megabytes - MB. If a generic device type such as the 3380 is specified, SMS uses the device geometry for that generic device to convert tracks or cylinders into MB. Given that 3390 are virtual and are emulated on disks with blocks sufficient virtual disk space can be provided.

After SMS converts space requests to MB, the space values are passed to the *Automatic Class Selection - ACS* routines. The values are later used to determine the number of tracks or cylinders to allocate for the data set.

There is only one default device geometry for the entire SMS complex. Default device geometry is an installation's definition of how much space is represented by a TRK or a CYL. The device geometry is the track size and number of tracks per cylinder for the device.

The device geometry for 3380 is 47476 bytes per track, 15 tracks per cylinder.

The device geometry for 3390 is 56664 bytes per track, 15 tracks per cylinder.

It is up to each installation to decide what values to use.

9.3 Hierarchical and network Data Base Management Systems - DBMSs

Advantages of DBMSs over file systems are discussed in most textbooks on DBMSs such as (Ramakrishnan and Gehrke, 2002), but a quick summary is as follows.

<https://www.geeksforgeeks.org/advantages-of-dbms-over-file-system/>

An early categorization of DBMSs was hierarchical versus network. DBMS records are connected to each other via physical links in both cases, e.g., a parent record points to a linked list of his children records. These and relational DBMSs were covered in the early editions of Chris Date's "An Introduction to Database Systems", starting in 1975, while the

8th edition (Date, 2003) only covers relational databases. The older DBMSs are not being utilized for developing new applications, because of the unavailability of skilled programmers and high development costs, but their object code is run on descendants of IBM S/360 mainframes, referred to as z-systems.

For many decades general purpose computers, especially IBM S/360 mainframes and its plug-compatibles and descendants introduced in Chapter 1 supported batch processing on files and databases, *OnLine Transaction Processing - OLTP*, query processing in relational databases, data mining, data warehousing, *OnLine Analytic Processing - OLAP* (Ramakrishnan and Gehrke, 2002). Books on novel paradigms such as Machine Learning, Business Intelligence, Deep Learning, Data Science are listed in the Appendix.

9.3.1 Information Management System - IMS

IMS has a hierarchical organization, which is a subset of network databases. IMS has two components: *Data Communications - DC* and *Data Base - DB*.

https://en.wikipedia.org/wiki/IBM_Information_Management_System

Customer Information Control System - CICS can be used instead of IMS/DC for data communication.

<https://en.wikipedia.org/wiki/CICS>

IMS *Fast-Path - FP* is a main-memory resident database providing higher throughputs and low response times by holding a subset of data required for OLTP in main memory, e.g., credit card number, available credit, expiration date, status: OK versus lost or stolen, instead of a full record with customer name, address, etc. held on disk. Logging in disk-based systems was carried out in a particularly efficient manner by writing on the first available space on a disk track, to minimize rotational latency. In addition to validation the remaining balance is updated and a log record is written as part of txn commit to record the purchase if it is approved. The log is used to update disk contents every night and the main memory database is refreshed.

Dieter Gawlick (IBM/Amdahl, DEC/Compaq/HP, Oracle) has commented on forms of *Concurrency Control - CC* in IMS FP (Gawlick and Kinkade, 1985) based on escrow protocol formalized in (O'Neil, 1986).

9.3.2 Transaction Processing Facility - TPF

TPF was intended for high volume processing of relatively simple txns was developed in 1960s by IBM and major American and European airlines of the time. One such user is Sabre, which is a travel technology company providing air bookings in North America. American Airlines founded Sabre in 1960, and it was spun off in 2000.

https://en.wikipedia.org/wiki/Sabre_Corporation

At this time TPF is used by VISA and American Express credit card companies for authorization.

https://en.wikipedia.org/wiki/Transaction_Processing_Facility

There are tightly and loosely coupled versions of TPF. In tightly coupled systems locking is accomplished via a Record Hold Table. The latter is a data sharing system according to Stonebraker's classification (Stonebraker, 1986). Serialized access to data records in a loosely coupled system is achieved by record level locking. Locks are held offboard in the *Direct Access Storage Device - DASD* control units, using the *Limited Lock Facility - LLF*, which was later replaced by the *Multipathing Lock Facility - MPLF*. To run clustered loosely coupled z/TPF requires either MPLF in all disk control units or an alternative locking device called a Coupling Facility.

https://en.wikipedia.org/wiki/Coupling_Facility

TPF data is held in 381, 1055 and 4K byte records. Physical memory size is partitioned accordingly and this results in a major reduction in data allocations overhead in main memory and at disks. Such a scheme referred to as subpooling was proposed in the context of IBM's VM operating system for main memory data allocation (Bozman et al., 1984). It outperforms algorithms such as first-fit, modified first-fit, and best-fit which are CPU-inefficient. Buddy systems are fast but suffer from internal fragmentation (Cormen et al., 2009). With advances in DASD technology, i.e., higher data transfer rates due to higher linear recording densities the read/write of small blocks is dominated by positioning time while transfer time is negligible.

Early TPF systems allocated data on a few tracks achieving short-stroking with reduced seek times. Spreading the data over more disks reduce their utilization (ρ), since for random arrivals due to queueing effects the mean disk response time is inversely proportional to the complement of disk utilization, i.e., $1 - \rho$ (Kleinrock, 1975). Of interest to the discussion is the fast general purpose synchronization mechanism described in (Robinson, 1985).

9.3.3 Network databases

Committee on Data Systems Languages - CODASYL

<https://en.wikipedia.org/wiki/CODASYL>

defined *COmmon Business Oriented Language - COBOL*

<https://en.wikipedia.org/wiki/COBOL>

but also its *Data Base Task Group - DBTG* led by Charles Bachman defined CODASYL databases with a network organization, where CODASYL stands for “Conference on Data Systems Languages”.

Cobol is still being used at this time and there was a call for COBOL programmers to handle unemployment benefit applications associated with Coronavirus pandemic in April 2020.

<https://www.techrepublic.com/article/cobol-programmers-are-in-demand-to-fight-the-coronavirus-pandemic/>

Bachman received ACM’s Turing award for his work on databases authoring “programmer as navigator”, since early database programmers navigated “linked lists on disk” (Bachman, 1973).

IDMS roots go back to *Integrated Data Store - IDS*, developed at *General Electric - GE* by Charles Bachman in 1964. IDMS is now marketed by Computer Associates, which has been acquired by Broadcom - a global infrastructure technology leader.

https://en.wikipedia.org/wiki/CA_Technologies

<https://www.broadcom.com>

In the early 1960s IDS was taken from its original form, by the computer group of the B.F. Goodrich Chemical Division, and re-written in a language called *Intermediate System Language - ISL*. The Chemical Division computer struck a deal with John Cullinane, whose company became known as Cullinet.

<https://en.wikipedia.org/wiki/Cullinet>

<https://en.wikipedia.org/wiki/IDMS>

Cincom’s TOTAL network database

https://en.wikipedia.org/wiki/Cincom_Systems was available on minicomputers from Harris Corp.

https://en.wikipedia.org/wiki/Harris_Corporation

There is an XML version of Total DBMS

<https://www.cincom.com/pdf/TX020220-3.pdf>

Case Western Reserve U. - CWRU and used in database courses in 1970s-80s using the first (1975) edition of Date (2003). Harris has since merged with L3.

https://en.wikipedia.org/wiki/L3_Technologies Using TOTAL required sufficient knowledge of COBOL programming to make database calls.

Cincom later produced the Supra relational database for SQL applications.

Criticisms of navigational APIs fall into two categories:

Usability: application code quickly becomes unreadable and difficult to debug.

Data independence: application code needs to change whenever the data structure changes.

For many years the primary defense of navigational databases was performance, which use internal storage structures that contain physical links or pointers from one record to another, which allow very efficient navigation. This led Turing Award winner Bachman to write the article “programmer as navigator” (Bachman, 1973).

https://en.wikipedia.org/wiki/Navigational_database

At this point there is the issue of virtualization which counteracts optimizing allocations on DASD. IMS is compared with IDMS in Table 9.2. The ranking methodology used in this table is described in Section 9.5.

TABLE 9.2 Navigational DBMS. First three columns are the ranking by category, while last three grading at three dates.

6/20	5/20	6/19	Company	Data model	6/20	5/20	6/19
1.	1.	1.	IMS	Navigational	2.51	+0.41	+0.35
2.	2.	2.	IDMS	Navigational	1.04	+0.07	+0.12

9.4 Relational data model

The relational database model was the brainchild of Edgar (Ted) Codd (rip) at IBM’s San Jose Research Center (Codd, 1970), which was later moved uphill to *Almaden Research Center - ARC*.

<http://www.almaden.ibm.com/almaden/welcome.html>

The *Santa Teresa Lab - STL* (McCue, 1978), renamed *Silicon Valley Lab - SVL* was built as a center for database software development. Some of its activity was later partially moved to IBM's Toronto Lab. under the direction of Janet Perna.

https://www.ibm.com/ibm/history/witexhibit/wit_hall_perna.html

This was perhaps IBM's way of spreading employment across countries to compensate for the revenue they generate and benefit from local talent. Note that DB2 provided IBM with a 34.6% marketshare in 2001, but at this time 20 years later IBM ranks sixth in db-engines category.

<https://db-engines.com/en/ranking>

Relational databases are a collection of tables, where each table has multiple columns with attributes belonging to possibly different domains. A domain is the original sets of atomic values used to model data. By atomic value, we mean that each value in the domain is indivisible as far as the relational model is concerned. Rows in a table are distinguishable by their unique key based on one or more columns. Records in tables are linked to each other via foreign keys acting as logical pointers. Costly joins are required to answer queries and the cost is especially pronounced in the case of disk based *relational DBMS - RDBMS*.

A major advantage of relational databases is increased productivity, which was noted by Codd in his Turing award article (Codd, 1982). This is important since labor costs are increasing much more rapidly than hardware costs. There is also the following consideration.

Growth in demands from end users for new applications is outstripping the capability of data processing departments to implement the corresponding application programs.

There is an incentive to reduce the cost of software development due to the shortage of skilled programmers and increasing salaries, while the cost of hardware is dropping. Skilled SQL programmers would be required to formulate SQL queries such as those in "Joe Celkoe's SQL for Smarties" (Celko, 2014).

Relational algebra is a set-oriented language allowing relational queries to be specified succinctly.

https://en.wikipedia.org/wiki/Relational_algebra

SQL is first translated to relational algebra operators to which query optimization is applied to determine the least cost plan in CPU and I/O processing to evaluate a query. The two costs should be appropriately weighted.

Given a SELECT statement on the EMPLOYEE table with a given qualification, e.g., names of employees under 30 whose salary is greater than \$100K. While scanning the EMPLOYEE table to find the names of qualifying employees is an option, the query optimizer could use associated histograms to find the least costly method to process the query.

The query optimizer described in (Selinger et al., 1979) used dynamic programming to minimize storage and CPU costs. The query optimization algorithm has been extended to a distributed RDBMS to take into account communication costs. The effect of new memory technologies should be taken into account in assessing query processing costs.

Relational databases were initially considered to be too inefficient as a commercial product, because of their high resource consumption and poor performance. On the other hand increased productivity is a more important consideration, since most decision support questions could be answered quickly with minimal programming effort leading to increased profitability.

In early 1970s *Device Independent Accessing Model - DIAM* by Michael Senko et al. (Senko et al., 1973) was a proposal competing with the relational data model at IBM research. The ensuing debates may have resulted in the slowing down the adoption of the relational data model at IBM.

The relational database model led to several early relational DBMS prototypes at IBM (McGee, 1981). The project to build a relational database at IBM San Jose called System R was started in 1973 (Astrahan et al., 1976), (Blasgen et al., 1981), (Chamberlin et al., 1981).

https://en.wikipedia.org/wiki/IBM_System_R

The *Structured English Query Language - SEQUEL* published in (Chamberlin et al., 1976), renamed SQL, became the standard relational programming language, getting Oracle founder Larry Ellison interested adopting it as an interface.

https://en.wikipedia.org/wiki/Oracle_Corporation

The *Entity-Relationship - ER* logical database design methodology due to Peter Chen was first described in (Chen, 1976) and is covered in most database textbooks such as (Ramakrishnan and Gehrke, 2002). Given an ER diagram, the database can be implemented as network or relational database with physical links or foreign keys.

https://en.wikipedia.org/wiki/Entity%E2%80%93relationship_model

Logical relational database design involves normalization to eliminate redundancy by transforming wider tables with redundancy into multiple narrower tables with no redundancy, which remain related to each other via foreign keys. Normal-

ization reduces the processing cost for updates, but adds cost to query processing since joins are required (Ramakrishnan and Gehrke, 2002).

As an example for the need for normalization consider an Employee table which has fields with information about employees, but also the branches they work at. Since many employees work at each branch this information is repeated many times. Normalization replaces branch information in Employee records with a branch#, which will acts as a logical pointer/foreign_key to records in Branch table. This results in a reduction in storage space, but has little effect on update cost since branch info is rarely updated.

The Employee table has $E = 10,000$ employees, whose records are $e = 500$ bytes each and include employee and branch information including the branch number specified by $k = 5$ bytes, so that $E \times e = 5$ MB. There are $B = 10$ branches with $b = 200$ bytes each, including the branch#. Given that k is the length of the key to specify branch#, the size of the Employee table after normalization is $E(e - b + k) \approx 3$ MB and the Branches table is $Bb = 2000$ bytes large, i.e., a 40% reduction in storage requirements. A more important advantage is that only a single branch record need to be updated, although such updates tend to be rare.

The commonly used Equijoin involves bringing records with equal primary and foreign keys together. It is a costly operation when the tables are stored on disk. Hashing and sort-merge methods have been developed for Equijoins are explained in (Ramakrishnan and Gehrke, 2002). In fact with a sorted Branch table can be held in main memory a single pass over the larger Employee table would be required to create a joined table.

The performance of relational databases can be improved by denormalization, which is simply not applying normalization.

<https://en.wikipedia.org/wiki/Denormalization>

The effect of denormalization on database performance was reported in (Schkolnick and Sorenson, 1982). Denormalization is important in the context of star schemas associated with data warehousing (Ramakrishnan and Gehrke, 2002).

https://en.wikipedia.org/wiki/Star_schema

Snowflake queries normalize dimensions tables saving space and increasing query processing cost.

<https://www.xplenty.com/blog/snowflake-schemas-vs-star-schemas-what-are-they-and-how-are-they-different/#whatsstar>

The performance of System R prototype was considered inadequate for IBM's top of the line mainframes computers in early 1980s, which ran the *Multiple Virtual Storage - MVS* operating system.

https://en.wikipedia.org/wiki/History_of_IBM_mainframe_operating_systems

System R was released with for lower end mainframes running DOS/360, VSE, and VM operating systems.

IBM developed industrial strength DB2 RDBMS for MVS (Cheng et al., 1984), which has evolved to Z/OS.

https://en.wikipedia.org/wiki/OS/360_and_successors

Db2 runs with *Linux, UNIX and Windows - LUW*. Early implementations of DB2 were inefficient for OLTP in that they used page-level locking for data and quarter page locks for indices. This resulted in unacceptably high levels of lock/data contention for OLTP. Developments in the Db2 family of databases are discussed here.

https://en.wikipedia.org/wiki/IBM_Db2

9.4.1 Methods to reduce the level of lock contention in OLTP

Peter Franaszek and the author visited IBM Fellow Rick Baum at Poughkeepsie, NY in 1990 to get his feedback on their research on *Concurrency Control - CC* methods for high data contention OLTP environments. To reduce data contention they were considered executing txns in two or more phases, e.g., the optimistic die option in the first phase and *two-phase locking - 2PL* in latter phases if validation failed (Franaszek et al., 1992).

The optimistic die option runs a txn to the end before checking for data conflicts to validate the txn. Validated txns, i.e. those with no data conflicts can commit, but otherwise are restarted. Data conflicts occur when a data item accessed by a txn is modified by another txn before validation. Running txns to the end prefetches all database blocks required for txn reexecution into the database buffer. Buffer retention combined with access invariance allowed very efficient txn rerun requiring no I/O The optimistic kill option checks for data conflicts following each data access, so that a txn that has encountered a data conflict is aborted and restarted right away.

Analysis for static and dynamic accesses to data with the optimistic die and kill option is provided in (Ryu and Thomasian, 1987). These analyses can be extended to two phase processing methods with optimistic die in the first phase and kill in latter phases.

Txns are run with access invariance with database buffer retention in the simulation study in (Franaszek et al., 1992), but not in (Agrawal et al., 1987) which favored locking with blocking policy. Mike Carey's PhD thesis with Michael Stonebraker at UCB postulated a slow processor, which also favored locking with blocking given that optimistic CC wastes CPU processing due to txn aborts and restarts to deal with data contention level at the cost of extra CPU processing. In addition this study does not take into account the fact that all or least many items accessed in the second phase of txn execution are already in the buffer.

DB2's coarse granularity page level locking results in high levels of lock contention environments with the standard 2PL and this according to Baum limited the throughput potentially available by the IBM mainframes in 1990. Extreme lock contention may result in a major performance degradation or thrashing due to the snowball effect when a large fraction of txns get blocked (Thomasian, 1993). A simplified analysis based on (Thomasian, 1993) is provided in Chapter 1.

The *Wait Depth Limited* - WDL method was devised in response to Baum's feedback that a more efficient locking method is required, since that is what DB2 uses. WDL was an improvement over the *Running Priority* - RP locking method (Franaszek and Robinson, 1985), which aborts and restarts a blocked txn holding a lock requested by a running txn, e.g., if txn *C* is conflicted by a blocked txn *B* then *B*, which is blocked by txn *A* is aborted: $C \rightarrow B \rightarrow A$. WDL additionally takes into account txn progress based on locks already held by a txn. Simulation results reported in (Franaszek et al., 1992) show a substantial improvement in the maximum achievable throughput with respect to standard locking.

High lock contention in DB2 was later alleviated in its later release by implementing fine-granularity locking in DB2 via *ARIES* - *Algorithms for Recovery and Isolation Exploiting Semantics* for DB2, which was based on similar ideas developed at IBM for IMS and documented by C. Mohan (retired 6/30/20), Don Haderle et al. at IBM/ARC in (Mohan et al., 1992). A good description of ARIES appears in Ramakrishnan and Gehrke (2002).

https://en.wikipedia.org/wiki/Algorithms_for_Recovery_and_Isolation_Exploiting_Semantics

A discussion of four DB2 codebases by former IBMer James Hamilton appears below.

<https://perspectives.mvdirona.com/2017/12/1187/>

9.4.2 Informix

Informix under the name Relational Database Systems was founded in 1980 and renamed Informix in 1986. Informix databases run on Unix, Microsoft Windows, and Apple Macintosh operating systems.

https://en.wikipedia.org/wiki/Informix_Corporation

IBM's acquisition of Informix led by Janet Perna in Toronto in 2001 for \$1 billion provided IBM with 120,000 new accounts and Informix's RDBMS supporting the Object Relational Data Model (Stonebraker and Moore, 1996).

https://en.wikipedia.org/wiki/Object-relational_database

IBM and *Hindustan Computers Limited* - HCL

https://en.m.wikipedia.org/wiki/HCL_Technologies

signed a \$1.8 Billion acquisition strategic partnership to jointly develop and market IBM Informix effective May 2017.

https://en.wikipedia.org/wiki/IBM_Informix

HCL ranks fifth among Indian IT companies.

Tata Consulting Services - TCS <https://www.tcs.com/>

Infosys <https://www.infosys.com/>

Tech Mahindra https://en.wikipedia.org/wiki/Tech_Mahindra

Wipro <https://www.wipro.com/en-US/>

HCL Technologies <https://www.hcltech.com/>

IBM provides the data integration platform InfoSphere Information Server, which includes a family of products to understand, cleanse, monitor, transform and deliver data, and to collaborate to bridge the gap between business and IT. InfoSphere also provides *Massively Parallel Processing* - MPP capabilities to deliver a highly scalable and flexible integration platform that handles all data volumes.

<https://www.ibm.com/analytics/information-server>

9.4.3 Berkeley's INGRES relational database

Michael Stonebraker and Eugene Wong at UCB founded the *Interactive Graphics REtrieval System - INGRES* project to produce a relational DBMS in 1970, which is the same year as (Codd, 1970) was published, preceded by the IBM research report RJ599 under the title “Derivability, Redundancy and Consistency of Relations Stored in Large Data Bank” in August 1969.

<https://dominoweb.draco.res.ibm.com/reports/RJ599.pdf>

[https://en.wikipedia.org/wiki/Ingres_\(database\)](https://en.wikipedia.org/wiki/Ingres_(database))

INGRES source code was initially available on magnetic tape for a modest fee to universities.

Relational Technologies Inc. - RTI founded by Stonebraker, Wong, and Lawrence Rowe in 1976 was taken over by *Computer Associates - CA*, which was acquired by Broadcom in 2018, which is also provider of mainframe software.

https://en.wikipedia.org/wiki/CA_Technologies

<https://www.broadcom.com/products/mainframe>

Stonebraker received the ACM Turing award in 2014 for his accomplishments at Berkeley and post UCB retirement at MIT, which are discussed in (Brodie, 2019).

https://amturing.acm.org/vp/stonebraker_1172121.cfm

With Joseph Hellerstein (UCB) he has edited three volumes of “Readings in Database Systems”.

Caro Lucas who received his MS in *Electrical Engineering - EE* at U. Tehran (1968-1973) did his PhD with Eugene Wong (1973-76). He returned to U. Tehran where he became known the father of robotics in Iran.

https://en.wikipedia.org/wiki/Caro_Lucas

PhD thesis to build histograms for Ingres

At *Case Western Reserve U. - CWRU* I supervised the PhD thesis of Robert (Bob) Kooi, who had access to a PDP/11 computer running Ingres on top of UNIX. With access to Ingres source code Kooi proposed to add histograms to improve the performance of INGRES query optimizer, which otherwise relied on crude default values.

I visited Bob Epstein at Berkeley who was the manager of the Ingres project in December 1978 to ask questions about INGRES. Epstein is responsible for four IT startups including Sybase, which was later acquired by SAP.

http://www.computerhistory.org/events/bio/Bob_Epstein

<https://en.wikipedia.org/wiki/Sybase>

I was interested in comparing INGRES performance, especially the number of disk accesses, CPU time, and elapsed time with and without histograms and with respect to the TOTAL network database. That network databases outperform relational databases is verified in the following study, which unfortunately does not provide sufficient details.

<https://raima.com/network-model-vs-relational-model/>

There was also the issue that the cost of building histograms could be reduced by sampling (Olken and Rotem, 1986), but in a small database the histogram could be built by simply scanning the whole table.

I received a draft of Bob's thesis in Fall 1979, a month after I had left CWRU to join the EE-systems department at *U. Southern California - USC*. I read the thesis right away and sent my comments for mostly minor revisions, but was dropped from the PhD committee to expedite graduation. (Kooi, 1980).

In 1994 I found out about Bob's untimely death resulting from a brain aneurysm by reading his obituary written by Michael Stonebraker (Stonebraker, 1994). Bob had been the manager of the query optimization group at Oracle. Mike's note led many to think that Bob was his PhD student.

9.4.4 Oracle's relational databases

Oracle's Lawrence Ellison became interested in building a relational database by reading Ted Codd's original article on this topic (Codd, 1970).

https://en.wikipedia.org/wiki/Larry_Ellison

In 1977 he founded Software Development Labs, renamed Oracle Corp. in 1983, by investing \$1200 of his own money.

https://en.wikipedia.org/wiki/Oracle_Corporation

Sun Microsystems, whose ascent is described in (Hall and Barry, 1991), was acquired by Oracle in 2009 providing it with a hardware platform for Oracle DBMS on Sun servers with the Solaris OS.

https://en.wikipedia.org/wiki/Sun_Microsystems

https://en.wikipedia.org/wiki/Sun_acquisition_by_Oracle

Sun in 2005 had acquired StorageTek - STK which was founded in 1969. STK had developed automated tape libraries, but also Iceberg disk array (Friedman, 1995), which is described in Chapter 5.

https://en.wikipedia.org/wiki/Storage_Technology_Corporation

In 2018 Oracle became the second largest software company by revenue after Microsoft.

https://en.wikipedia.org/wiki/List_of_the_largest_software_companies

Oracle has recently developed the Exadata *Data Base Machine - DBM*, which is discussed in more detail in Chapter 8.

Oracle innovations are presented in a September 2019 speech by Oracle CEO Larry Ellison.

<https://www.forbes.com/sites/oracle/2019/09/17/larry-ellison-details-4-database-innovations-including-a-new-free-version/#2ab9f56254ef>

Sun which had developed the Java programming language allowed its use at Google, but things changed after Oracle bought Sun and this led to a never-ending lawsuit.

https://en.wikipedia.org/wiki/Oracle_America,_Inc._v._Google,_Inc.

<https://www.wired.com/story/the-case-that-never-ends-oracle-wins-latest-round-vs-google/>

<https://www.forbes.com/sites/danwoods/2019/01/31/oracles-newest-audit-tactic-focusing-on-java/#2a220bb54960>

It should be noted that IBM did not sue Oracle for copying IBM's SEQUEL.

<https://arstechnica.com/tech-policy/2020/03/before-it-sued-google-for-copying-from-java-oracle-got-rich-copying-ibms-sql/>

Regarding the questing "Did Oracle buy TikTok?" the following is said below.

<https://arstechnica.com/tech-policy/2020/09/everything-we-know-so-far-about-oracle-not-actually-buying-tiktok/>

Oracle did not buy and is not buying TikTok, but it is investing in the company. The purchase of Tik-Tok has ByteDance spinning off TikTok operations in the US and elsewhere to a new company, TikTok Global.

<https://bytedance.com/en/>

Oracle will hold a 12.5% stake in TikTok Global at its inception and will also serve as the cloud hosting provider for the new entity. Walmart will hold another 7.5% stake. The remaining 80 percent will be held by ByteDance. The new company also plans to launch an initial public offering of stock before the end of 2021.

Where have all the good databases gone?

This 2004 presentation by Google's Adam Bosworth foresaw major changes in DBMS anticipating by years the rise of the MapReduce/NoSQL movements, it succinctly identified the central problem: "The products that the database vendors were building had less and less to do with what the customers wanted". The three simple things being asked were:

<https://adambosworth.net/2004/12/29/where-have-all-the-good-databases-gone/>

1. **Dynamic schema** so that as the business model changes and evolves, this evolution can be handled seamlessly in a system running all the time.
2. **Dynamic partitioning** of data across a large numbers of machines.
3. **Modern indexing** similarly to Google.

Bosworth's message to the Open Source community was to build LAMP (Linux, Apache and Tomcat and MySQL and PHP and PERL and Python) to do for databases what they did for web servers, i.e., dynamism and robustness. Systems that scale linearly, are flexible and dynamically reconfigurable and load balanced and easy to use.

9.5 Ranking methodology for database engines

Rankings at <https://db-engines.com> are determined as follows.

- Number of mentions of the system on websites when searching for system name and database. Google, Bing and Yandex were used for this measurement.
<https://www.google.com>, www.bing.com, <https://yandex.com>, <https://en.wikipedia.org/wiki/Yandex>
- General interest in the system based on the frequency of searches in Google Trends, which is a website by Google that analyzes the popularity of top search queries in Google Search across various regions and languages. The website uses

graphs to compare the search volume of different queries over time.

https://en.wikipedia.org/wiki/Google_Trends

- Frequency of technical discussions about the system in Q&A sites:
https://en.wikipedia.org/wiki/Stack_Overflow, <https://stackoverflow.com/>, <https://dba.stackexchange.com/>
- Number of job offers, in which the system is mentioned by web sites such as:
www.indeed.com, www.simplyhired.com.
- Number of profiles in professional networks in which systems are mentioned.
www.linkedin.com, www.upwork.com
- Relevance in social networks in which the system is mentioned.
twitter.com, parler.com

The popularity value of a system by standardizing and averaging of the individual parameters. Product revenue would have been a better indicator of product popularity, but in a company with multiple products it may be difficult to determine product revenue which may be a company secret.

The tables appearing in the book reflect the ranking when the book was being written, but the most up-to-date ranking from various database categories can be obtained from here.

<https://db-engin.com/en/ranking/>

9.6 Overall ranking of all database types

While relational databases are still the most popular, *Multi-Model - MM* databases, key-value and wide-columns databases are gaining popularity. Most DBMSs are organized around a single data model that determines how data is organized, stored, and manipulated, versus MM databases in Section 9.37. DB engine rankings are varying year to years and the reader is encouraged to refer to the web for the latest ranking. The ranking at this time is given in Table 9.3.

<https://db-engines.com/en/ranking>

TABLE 9.3 Complete ranking based on scores, *Key-Value - KV, Multi-Model - MM*.

6/20	5/20	6/19	Company	Data model	6/20	5/20	6/19
1.	1.	1.	Oracle	Rel, MM	1343.59	−1.85	+44.37
2.	2.	2.	MySQL	Rel, MM	1277.89	−4.75	+54.26
3.	3.	3.	M'soft SQL Server	Rel, MM	1067.31	−10.99	−20.45
4.	4.	4.	PostgreSQL	Rel, MM	522.99	+8.19	+46.36
5.	5.	5.	MongoDB	Doc. MM	437.08	−1.92	+33.17
6.	6.	6.	IBM Db2	Rel, MM	161.81	−0.83	−10.39
7.	7.	7.	Elasticsearch	SE, MM	149.69	+0.56	+0.86
8.	8.	8.	Redis	KV, MM	145.64	+2.17	−0.48
9.	9.	11.	SQLite	Rel	124.82	+1.78	−0.07
10.	11.	10.	Cassandra	Wide Col.	119.01	−0.15	−6.17

A newly announced Oracle Database 21c is discussed in 1/22/21 article and has the following highlights.

<https://cloudwars.co/oracle/oracle-database-21c-takes-on-aws-microsoft-in-clash-of-the-titans/>

9.7 Relational database management systems

Given that *Relational Data Base Management Systems - RDBMSs* are the most popular we discuss their advantages and disadvantages.

RDBMS Advantages:

1. RDBMS is well-documented and mature technology available from established companies.
2. There is a well-defined SQL standard.
3. Relational data model has solid mathematical foundations.
4. There are plenty of SQL programmers.
5. RDBMSs are ACID-compliant (Ramakrishnan and Gehrke, 2002).

RDBMS Disadvantages:

1. RDBMSs do not work well with unstructured or semi-structured data, which makes them ill-suited for large analytics or IoT applications.
2. The tables in relational databases do not necessarily map one-to-one with an object or class representing the same data.
3. Schemas and types must be identical between source and destination tables for migration to work. This is known as schema constraint.

According to the following web site the fraction of RDBMSs has dropped from 90% to 75% from 2/2013 to 2/2018.

<https://www.alooma.com/blog/types-of-modern-databases>

That relational databases are doomed is discussed in this web page.

<https://readwrite.com/2009/02/12/is-the-relational-database-doomed/>

A long alphabetic list comparing relational databases from different viewpoints is given below.

https://en.wikipedia.org/wiki/Comparison_of_relational_database_management_systems

General information: maintainer, first public release date, latest stable version, latest release date, public issues list.

Operating system: Windows, macOS, Linux, BSD, UNIX, AmigaOS, z/OS, IOS, Android, OpenVMS.

Fundamental Features: ACID, referential integrity, txns, fine-grained locking, *Multi-Version Concurrency Control* - MVCC, unicode, inference, type inference.

<https://home.unicode.org/>

Limits: Max DB size, Max table size, Max row size, Max columns per row, Max blob/clob size, Max char size, Max NUMBER size, Min DATE value. Max columns name size).

Tables and Views Temporary table, materialized view (Ramakrishnan and Gehrke, 2002).

Indexes R/R+ tree (Guttman, 1984), (Faloutsos, 1996), Hash, expression, partial, reverse, bitmap, *Generalized Search Tree* - *GiST* index (Hellerstein et al., 1995),

<https://en.wikipedia.org/wiki/GiST>

Generalized Inverted Index - *GIN* (Zobel and Moffat, 2006)), Full-text, Spatial, *Forest of Trees* - *FOT*, duplicate index prevention.

Database Capabilities Union, Intersect, Except, Inner/Outer joins, Inner selects, Merge joins, Blobs/Clobs, Common table expressions, Windowing functions, Parallel query, System-versioned tables.

Data types: Types system (static/dynamic), Integer, Floating point, Decimal, String, Binary, date/time, Boolean, other.

Partitioning Range, Composite (Range+Hash), List, Expression, Round-robin.

Access control Native network encryption, brute force protection, enterprise directory compatibility, password complexity rules, patch access, run unprivileged, audit, resource limit, separation duties, *Rule-Based Access Control* - *RBAC*

https://en.wikipedia.org/wiki/Role-based_access_control

security classification, *attribute based access control* - *abac*,

https://en.wikipedia.org/wiki/Attribute-based_access_control

also known as Policy Based Access Control for *Identity Management* - *IDM*.

https://en.wikipedia.org/wiki/Identity_management

Ranking of relational databases is given in Table 9.4.

Oracle Database is an RDBMS that implements object-oriented features such as user-defined types, inheritance, and polymorphism is called an *Object-Relational DBMS* - *ORDBMS*.

<https://docs.oracle.com/database/121/CNCPT/intro.htm#CNCPT941>

MySQL is an open-source web-based business application, which falls under the Oracle umbrella, especially for e-commerce, used by Facebook, Google and Adobe. It runs on virtually all platforms: Linux, UNIX, and Windows.

<https://www.mysql.com/>

Adobe is best known for its Adobe Flash web software ecosystem, Photoshop image editing software, Adobe Illustrator vector graphics editor, Acrobat Reader, the *Portable Document Format* - *PDF*, and Adobe Creative Suite, as well as its successor Adobe Creative Cloud. Adobe was founded by John Warnock and Charles Geschke, who left XEROX *Palo Alto Research Center* - *PARC* in 1982.

https://en.wikipedia.org/wiki/Adobe_Inc.

SQL Server from Microsoft is a popular RDBMS, which was released in 2008, 2012, 2014, and 2016. SQL server provides BI - Business Intelligence tools and links to Microsoft offerings such as Excel, Word and SharePoint.

https://en.wikipedia.org/wiki/Microsoft_SQL_Server

PostgreSQL is a successor to the Ingres project led by Michael Stonebraker (Stonebraker et al., 1976, 2014). It is an open-source ORDBMS with an emphasis on extensibility and standards compliance (Stonebraker and Rowe, 1986).

TABLE 9.4 Ranking of relational databases.

6/20	5/20	6/19	Company	Data model	6/20	5/20	6/19
1.	1.	1.	Oracle	Rel, MM	1343.59	−1.85	+44.37
2.	2.	2.	MySQL	Rel, MM	1277.89	−4.75	+54.26
3.	3.	3.	SQL Server	Rel, MM	1067.31	−10.99	−20.45
4.	4.	4.	PostgreSQL	Rel, MM	522.99	+ 8.19	+46.36
5.	5.	5.	IBM Db2	Rel, MM	161.81	−0.83	−10.39
6.	6.	7.	SQLite	Rel,	124.82	+1.78	−0.07
7.	7.	6.	M'soft Access	Rel,	117.18	−2.72	−23.83
8.	8.	8.	MariaDB	Rel, MM	89.79	−0.30	+4.59
9.	9.	9.	Hive	Rel,	78.65	−2.89	−0.40
10.	10.	10.	Teradata	Rel, MM	73.28	−0.60	−3.36
11.	11.	13.	SAP Server	Rel,	53.09	−0.90	−2.03
12.	13.	12.	SAP HANA	Rel, MM	50.82	+0.29	−5.56
13.	12.	11.	FileMaker	Rel,	50.16	−0.80	−7.64
14.	14.	14.	Azure SQL DB	Rel, MM	47.78	+5.03	+18.77
15.	15.	16.	Google BigQuery	Rel,	28.29	+0.70	+5.16

PostgreSQL runs on Windows, Linux, Solaris and macOS X and can be used for online gaming apps, database automation tools, and domain registries. It provides better support for complex objects, user extendibility for data types, operators and access methods, facilities for active databases, i.e., triggers (Ramakrishnan and Gehrke, 2002), and inferencing including forward- and backward-chaining, simplify the DBMS code for crash recovery, can take advantage of optical disks, multiple tightly-coupled processors, and custom designed VLSI chips, and make as few changes as possible to the relational model.

<https://en.wikipedia.org/wiki/PostgreSQL>

IBM's Db2 is a family of data management products, including database servers. They initially supported the relational model, but were extended to support object-relational features and non-relational structures like JSON and XML. DB2 was initially released in 1983 on IBM mainframes with OS/MVS (Cheng et al., 1984), but now runs with: z/OS Linux, Unix-like, Windows Platforms: z/Architecture, x86-64, x86-32, SPARC, IBM POWER microprocessors. Db2 is written in: C, C++, assembly language, PL/X (IBM internal systems programming language).

https://en.wikipedia.org/wiki/IBM_Db2_Family

SQLite is embedded into the end program as an RDBMS in a C library. It is ACID-compliant and implements most of the SQL standard with PostgreSQL. However, SQLite uses a dynamically and weakly typed SQL syntax that does not guarantee the domain integrity. SQLite is a popular choice as embedded database software for local/client storage in application software such as web browsers. It is arguably the most widely deployed DB engine, as it is used today by several browsers, operating systems, and embedded systems.

<https://en.wikipedia.org/wiki/SQLite>

Microsoft Access is a DBMS that combines the relational Microsoft *Joint Engineering Technology - Jet* database engine with a *Graphical User Interface - GUI* and software-development tools.

https://en.wikipedia.org/wiki/Microsoft_Jet_Database_Engine

It is part of Microsoft Office suite with a professional and educational versions.

https://en.wikipedia.org/wiki/Microsoft_Access

MariaDB is a community-developed, commercially supported fork of the MySQL RDBMS, intended to remain free and open-source software under the *GNU's Not Unix - GNU OS General Public License - GPL*.

<https://en.wikipedia.org/wiki/GNU>

<https://en.wikipedia.org/wiki/MariaDB>

Hive is data warehouse software built on top of Apache Hadoop for providing data query and analysis. Hive gives an SQL-like interface to query data stored in various databases and file systems that integrate with Hadoop.

https://en.wikipedia.org/wiki/Apache_Hive

Teradata Corp. is a provider of database and analytics-related software, products, and services, which is discussed in Chapter 8.

<https://en.wikipedia.org/wiki/Teradata>

<https://docs.teradata.com/>

SAP DBMS allows thousands of tables. Some products like *ERP Central Component - ECC/Enterprise Resource Planning - ERP* and *R/3* have more than 30,000 tables, whereas other products like CRM might have fewer than 10,000. Regardless of the number, these tables are tied to each other through established relationships.

<https://www.sap.com/products/database-data-management/hana-database-management-system.html>

HANA SAP based on Sybase is an in-memory, column-oriented, RDBMS developed and marketed by *Systems Applications and Products - SAP*, which is also discussed in Section 9.38. Its primary function as a database server is to store and retrieve data as requested by the applications. In addition, it performs advanced analytics (predictive analytics, spatial data processing, text analytics, text search, streaming analytics, graph data processing) and includes *Extract, Transform, Load - ETL* capabilities as well as an application server.

https://en.wikipedia.org/wiki/SAP_HANA

FileMaker is a cross-platform RDBMS application from Claris Int'l, a subsidiary of Apple Inc. It integrates a database engine with a GUI and security features, allowing users to modify the database by dragging new elements into layouts, screens, or forms.

<https://en.wikipedia.org/wiki/FileMaker>

Azure SQL DB from Microsoft is a managed cloud database provided as part of Microsoft Azure. A cloud database is run on a cloud computing platform, and access to it is provided as a service. Managed database services take care of scalability, backup, and high availability of the database.

https://en.wikipedia.org/wiki/Microsoft_Azure_SQL_Database

BigQuery from Google is a fully-managed, serverless data warehouse that enables scalable, cost-effective and fast analysis over petabytes of data. It is *Software as a Service - SaaS* that supports querying using *American National Standards Institute - ANSI SQL*. It also has built-in machine learning capabilities.

<https://en.wikipedia.org/wiki/BigQuery>

Software as a Service - SaaS licensing and delivery model in which software is licensed on a subscription basis and is centrally hosted. It is sometimes referred to as “on-demand software” and was formerly referred to as “software plus services” by Microsoft. Salesforce is a provider of SaaS. The two main varieties of SaaS are as follows,

https://en.wikipedia.org/wiki/Software_as_a_service

Vertical SaaS. Software which answers the needs of a specific industry, e.g., healthcare, agriculture, real estate, finance industries.

Horizontal SaaS. Focuses on a software category (marketing, sales, developer tools, HR) but are industry neutral.

9.8 Object relational databases

Object-relational databases are a hybrid of both approaches.

https://en.wikipedia.org/wiki/Object-relational_database

Object relational databases are compared in Table 9.5.

https://en.wikipedia.org/wiki/Comparison_of_object-relational_database_management_systems

Adaptive Server Enterprise - ASE was developed as Sybase by Robert Epstein in 1984. SAP acquired Sybase in 2010 and stopped using the name Sybase in 2014.

<https://en.wikipedia.org/wiki/Sybase>

ASE is predominantly used on the Unix platform, but also Microsoft Windows.

https://en.wikipedia.org/wiki/Adaptive_Server_Enterprise

Ispirer automatically migrates Sybase ASE to Teradata by converting the entire database schema (tables and SQL objects), as well as transfers data. Business logic conversion is greatly facilitated due to the power and flexibility of the Ispirer tool.

<https://www.ispirer.com/products/sybase-ase-to-teradata-migration>

CUBRID combines cube standing for a space for data and bridge standing for data bridge. It is an open-source SQL-based RDBMS object extensions developed by CUBRID Corp. for OLTP.

<https://en.wikipedia.org/wiki/CUBRID>

<https://www.cubrid.org>

DB2 from IBM is discussed in Section 9.7.

TABLE 9.5 Listing of Object Relational databases, *Version Managed Data Store - VMDS*, *Geographical Information System - GIS*.

Name	Vendor	License	OS/Notes
Adaptive Server Enterp.	SAP	Proprietary	Cross-platform
CUBRID	NHN Corp.	GPL/BSD	Linux, Windows
DB2	IBM	Proprietary	Cross-platform
Greenplum Database	Greenplum/EMC	Proprietary	?(Uses PostgreSQL codebase)
Informix	IBM	Proprietary	Cross-platform
Caché	InterSystems	Proprietary	Cross-platform
SQL Server	Microsoft Corp.	Proprietary	Windows, Linux (.NET)
Oracle Database	Oracle Corp.	Proprietary	Linux, Windows, Unix
PostgreSQL	PostgreSQL	PostgreSQL Lic.	Cross-platform
OpenEdge Adv. Bus. Lan.	Progress Softw.	Proprietary	Cross-platform
Virtuoso Univ. Server	OpenLink Softw.	GPLv2/prop.	Cross-platform
VMDS	GE Energy	Proprietary	?(GIS, Oracle Database)
WakandaDB	4th Dimension	AGPLv3/prop.	Windows, Linux, macOS(REST) server-side JavaScript
XDB Enterprise Server	Micro Focus	Proprietary	DOS, Windows NT, OS/2
Zope Object DB	Zope Corp.	Zope Pub. Lic.	Cross-platform For Python

Greenplum is a massively parallel Postgres with scale-out capability which was acquired by EMC in 2010. It allows fast data ingestion and refresh required for analytics models. Greenplum has done better than Netezza and successfully converted from Teradata.

<https://greenplum.org/gpdb-sandbox-tutorials/>

<https://greenplum.org/>

<https://en.wikipedia.org/wiki/Greenplum>

Netezza is a specialized analytics database acquired by IBM in 2010. IBM pulled the plug on Netezza in July 2019 partially, since it uses not cloud compatible.

<https://en.wikipedia.org/wiki/Netezza>

https://www.theregister.co.uk/2019/07/03/rip_netezza_ibms_fpgapowered_data_warehousing_dream/

The retirement of Netezza left IBM customers facing either moving to IBM's *Integrated Analytics System - ISA* or its *Db2 Warehouse on Cloud - Db2WoC*. Alternatively, customers could look at data warehousing products beyond IBM's. On June 2020 IBM announced that Netezza Performance Server on the Cloud would return on both IBM Cloud and AWS. One of the reasons was that although it was supposed to run the same SQL engine as DB2, some customers were finding difficulties switching.

https://www.theregister.com/2020/06/05/db2_migration_netezza/

Informix is a product family within IBM's Information Management division that is centered on several RDBMS offerings. Informix Software subsidiary was acquired by IBM in 2001, but was part of a deal in 2017 with HCL.

https://en.wikipedia.org/wiki/IBM_Informix

Intersystem Cache' is a highly optimized database engine for performance, concurrency, scalability, and reliability. There is a high degree of platform-specific optimization to attain maximum performance. Cache' is a full-featured database system, which includes all the features needed for running mission-critical applications such as journaling,

https://en.wikipedia.org/wiki/Journaling_file_system

backup and recovery, and system administration tools. Cache' is designed to require significantly less administrative effort than other database products and in fact the majority of deployed Caché systems have no database.

<https://download.intersystems.com/download/register.csp>

SQL server from Microsoft discussed in Section 9.7.

Oracle database is discussed in Section 9.7.

PostgreSQL is discussed in Section 9.7.

OpenEdge Advanced Business Language - ABL is maintained by Progress Software Corporation, which was classified as a 4th-generation programming language because of its use of an English-like syntax to simplify software devel-

opment. ABL helps developers to develop applications using its own integrated relational database and programming tool.

https://en.wikipedia.org/wiki/OpenEdge_Advanced_Business_Language

Virtuoso Universal Server is a middleware and DB engine hybrid combining the functionality of a traditional RDBMS, ORDBMS, virtual database, RDF, XML, free-text, web application server and file-server in a single system. Rather than have dedicated servers for different functionality realms, Virtuoso is a “universal server” that enables a single multithreaded server process that implements multiple protocols.

https://en.wikipedia.org/wiki/Virtuoso_Universal_Server

Version Managed Data Store - VMDS provided by *General Electric - GE* Energy as part of its Smallworld technology platform and was designed from the outset to store and analyze the highly complex spatial and topological networks typically used by enterprise utilities such as power distribution and telecommunications.

<https://en.wikipedia.org/wiki/VMDS>

Wakanda DB NoSQL, Object database, Web framework, Server-Side JavaScript, Mobile application development It uses a JavaScript platform to develop and run web or mobile apps.

<https://wakanda.github.io/about-wakanda>

Wakanda is a fictional country in Sub-Saharan Africa appearing in Marvel comic books superhero Black Panther.

9.9 Data mining

Given a table whose rows list items purchased by a customer per visit to a supermarket, known as market basket or shopping cart data *Association Rules Mining - ARM* algorithm was developed at IBM/ARC by Rakesh Agrawal, Tomasz Imielinski (Rutgers U.), and Arun Swami (IBM, SGI, LinkedIn)

<http://rakesh.agrawal-family.com/>

<https://www.cs.rutgers.edu/people/professors/details/tomasz-imielinski>

<https://www.linkedin.com/in/arunswami/>

to generate all significant association rules among purchased items (Agrawal et al., 1993b). ARM is covered in numerous books (Ramakrishnan and Gehrke, 2002), (Han and Kamber, 2011), (Zaki and Jr., 2014), (Aggarwal, 2015). ARM determines the support and confidence as follows: an $s\%$ support for two items **A** and **B**, such as bread and butter implies that they appear together in $s\%$ of txns. The confidence is then the ratio of the support for $\mathbf{A} \cap \mathbf{B}$ and **A** alone. Multiple passes over the dataset are required to find all association rules (with a lower bound for the support) when the number of objects is large. Algorithms to reduce the cost of ARM include sampling, partitioning and parallel processing (Zaki, 1999).

In the early days of data mining due to the uncertainty associated with the value of information gained from data mining it was considered important to keep costs down and hence data mining applications were not run on mainframe computers. Decline in processing cost made previously costly applications affordable.

Parallel processing is appropriate for large-scale data mining and shared nothing computer systems are the most suitable for this purpose. More recently this is exemplified by big data processing frameworks, e.g., Hadoop and Spark.

<https://www.kdnuggets.com/2016/03/top-big-data-processing-frameworks.html>

Regression analysis is primarily used for two distinct purposes:

1. widely used for prediction and forecasting.
2. regression analysis can be used to infer causal relationships between the independent and dependent variables.

https://en.wikipedia.org/wiki/Regression_analysis

In *Machine Learning - ML Support Vector Machines - SVMs* are supervised learning models with associated learning algorithms that analyze data used for classification and regression analysis.

https://en.wikipedia.org/wiki/Support_vector_machine

Classification assigns items to the appropriate classes using its attributes, where each item is represented by a feature vector, which may be based on color, texture, etc.

https://en.wikipedia.org/wiki/Statistical_classification

Decision tree or classification tree based methods split items according to attributes to form classification. Software packages have been developed by the ML community, which have been scaled to database applications. In addition there are *Neural Networks - NNs*, *Rule Based Algorithm - RBA* and combinations of methods.

In clustering unlike classification the groups are not predefined. A distance measure such as the Euclidean distance between the feature vectors of the items is used to form the clusters. The *agglomerative* clustering algorithm is a *hierarchical* algorithm, which starts with as many clusters is a very expensive method.

k-nearest-neighbor - *k-NN* search is used for similarity search in the case of *Content Based Image Retrieval* - *CBIR*. Similarity is determined by the Euclidean distance of the feature vector of the query image and the feature vectors of all the images. The calculation required is to find the *k* smallest Euclidean distances.

Given the number of clusters *nonhierarchical clustering* or *partitional clustering* algorithms create the clusters in one step. *k*-means method is in the family of squared error clustering algorithms.

Outline of *k*-means clustering

Step 1: Select *k* random points, which are far from each other and designate them as the centroids of the clusters.

Step 2: Assign remaining points to *k* clusters, whose centroid is closest to it based on Euclidean distance.

Step 3: Recompute the centroids for all clusters based on the items assigned to them and repeat **Step 2** with the new centroids, until there is no change in membership.

One measure of the quality of clustering is the *Sum of Squared Distances* - *SSDs* of points in each cluster, computed across all clusters. The algorithm is applied several times and the results from the iteration with the smallest SSD selected.

Various aspects of data clustering are covered in (Aggarwal and Reddy, 2014). Clustering of large datasets not fitting into the main memory is a challenging problem and multiple passes over the dataset are required to implement the *k*-means method. One such method Birch was proposed by Tian Zhang, Raghu Ramakrishnan (Yahoo, Microsoft/Gray Systems Lab - GSL) and Miron Livny at U. Wisconsin (Zhang et al., 1996).

<https://en.wikipedia.org/wiki/BIRCH>

An introduction to clustering big and high dimensional data is (Kogan, 2006). Its author edits a journal on Grouping Multidimensional Data (Recent Advances in Clustering),

<https://www.springer.com/gp/book/9783540283485>

which has published a survey of clustering data mining techniques is provided in (Berkhin, 2006). Author's research in combining clustering and dimensionality reduction for indexing high dimensional data which appeared in (Castelli et al., 2000, 2003) is presented Chapter 8.

9.10 Data warehousing and OLAP

Online Analytic Processing - *OLAP* provides a swift method to answer queries on multidimensional data.

https://en.wikipedia.org/wiki/Online_analytical_processing

A list of OLAP software vendors is given in Table 9.6 and below.

https://en.wikipedia.org/wiki/Comparison_of_OLAP_servers

TABLE 9.6 List of top OLAP vendors in 2006 in millions of USD.

Vendor	Global Revenue	Company
Microsoft Corp.	\$1,806M	Microsoft
Hyperion Sol. Corp.	\$1,077M	Oracle
Cognos	\$735M	IBM
Business Objects	\$416M	SAP
MicroStrategy	\$416M	Micro.
SAP AG	\$330M	SAP
Cartesis	\$210M	SAP
Applix	\$205M	IBM
Infor	\$199M	Infor
Oracle Corporation	\$159M	Oracle
Others	\$152M	Others
Total	\$5700M	

An up-to-date list of OLAP vendors in 7/21 is as follows.

<https://www.softwaretestinghelp.com/best-olap-tools/>

Variations of OLAP are discussed in the following sections.

9.10.1 Multi-dimensional OLAP - MOLAP

MOLAP with proprietary formats is not a relational database. It stores data as a multidimensional array compatible with Excel, which makes it easy to learn.

<https://www.guru99.com/multidimensional-online-analytical-processing.html>

https://en.wikipedia.org/wiki/Microsoft_Excel

Essbase: Essbase began as a product of Arbor Software, which merged with Hyperion Software in 1998. Hyperion was acquired by Oracle in 2007.

<https://www.oracle.com/database/technologies/olap.html>

Express Server: Web-based environment that runs on Oracle database.

<https://www.oracle.com/database/technologies/xe-downloads.html>

Yellowfin: Business analytics tools for creating reports and dashboards.

<https://www.yellowfinbi.com/>

Clear Analytics: Clear analytics is an Excel-based business solution.

<https://www.clearanalyticsbi.com/>

SAP Business Intelligence: <https://support.sap.com/en/my-support/software-downloads.html>

MOLAP is classical OLAP that facilitates data analysis by using a multidimensional data cube. MOLAP tools process information with the same amount of response time irrespective of the level of summarizing. MOLAP server implements two level of storage to manage dense and sparse data sets. MOLAP can manage, analyze, and store considerable amounts of multidimensional data. It helps to automate computation of higher level of aggregates data. It is less scalable than *Relational OLAP - ROLAP* as it handles only a limited amount of data.

MOLAP advantage:

1. Performance, fast data retrieval and data slicing and dicing operations.
2. Complex calculations can be programmed by the user.

MOLAP Disadvantages:

1. A limited amount of data can be handled.
2. Typically proprietary systems.
3. Transferring data from relational tables is cumbersome and redundant.

The following example with three dimensions (timeid, pid, locid) is quoted from (Ramakrishnan and Gehrke, 2002):

The sales multidimensional dataset is three-dimensional array, i.e., a cube with three dimensions (locid, pid, and tid), which hold sales values. The dataset can be represented as a star schema with the Sales relation in the center with its three columns serving as foreign keys to Locations, Products, and Times relations.

Locations: (locid, state, country)

Products: (pid, pname, category, price)

Times: (timeid, date, week, month, quarter, year, holiday_flag)

Sales: (pid, timeid, locid, sales)

A cross tabulation of Sales by Year and State is given via pivoting is given in Table 9.7.

TABLE 9.7 A cross tabulation of Sales by Year and State is given via pivoting based in (Ramakrishnan and Gehrke, 2002).

	WI	CA	Total
1995	63	81	144
1996	38	107	145
1997	65	35	110
Total	176	223	399

Computing multiple related GROUP BYs and aggregates is one of the core operations of OLAP applications, which led to the proposal of the Cube Operator by Jim Gray et al. (Gray et al., 1996a), (Ramakrishnan and Gehrke, 2002). The following lattice of GROUP BY queries as a result of the following Cube query:

CUBE pid, locid, timeid BY SUM Sales produces the following lattice of Group By queries.

```
[pid,locid,timeid]
[pid,locid]      [pid,timeid]    [locid,timeid]
[pid]           [locid]         [timeid]
[null]
```

9.10.2 Relational OLAP - ROLAP

ROLAP accesses a relational database by using SQL has the following advantages.

1. No data limitation and can handle large amounts of data.
2. Can use functionality of relational databases.
3. Saves space when dealing with sparse data.

ROLAP has the following disadvantages:

1. Slow performance because of large datasets.
2. Inflexible since limited to SQL functions.
3. Data may need to be reformatted for end-users.

It is shown in (Zhao et al., 1997) that instead of cubing a relational table directly it is faster to first convert the table to an array, and apply cubing to MOLAP rather the original ROLAP database.

9.10.3 Hybrid OLAP - HOLAP

HOLAP is based on the merger of the best features of MOLAP and ROLAP allowing for fast calculations from RDBMS by using pre-calculated cubes.

Advantages of HOLAP are as follows:

- Scalability, flexibility, and speed.
- RDBMS SQL functionality.
- Ability to “drill-down” from a cube to a relational table.
- Fast because of pre-calculated cubes.

Disadvantages of HOLAP are as follows:

1. Change in existing architectures disruptive.
2. Need to learn new technologies and skills.

Micro-architectural analysis of OLTP and OLAP

“Where does time go when a database system is executed on a modern computer platform?” is a question asked and answered in (Ailamaki et al., 1999). The study introduces a framework for analyzing query execution time on a DBMS running on a server with a modern processor and memory architecture in 1999. Examined were four commercial memory-resident DBMSs running on an Intel Xeon and NT 4.0. OS.

The study makes the following points:

1. Half the execution time is spent in stalls.
2. 90% of the memory stalls are due to:
 - Second-level cache data misses, while first-level data stalls are not important (implying dataplacement should focus on the second-level cache),
 - First-level instruction cache misses, while second-level instruction stalls are not important.

About 20% of the stalls are caused by implementation details such as branch misprediction.

3. Using simple queries rather than full TPC-D workloads provides a methodological advantage.

The abstract and conclusions of a recent study on OLAP performance are discussed here (Sirin and Ailamaki, 2020):

“Understanding micro-architectural behavior is important for efficiently using hardware resources. Recent work has shown that in-memory OLTP systems severely underutilize their core microarchitecture resources. Whereas, OLAP workloads exhibit a completely different computing pattern. OLAP workloads are read-only, bandwidth-intensive, and include various data access patterns. With the rise of column-stores, they run on high-performance engines that are tightly optimized for modern hardware. Consequently, micro-architectural behavior of modern OLAP systems remains unclear. This work presents a micro-architectural analysis of a set of OLAP systems. The results show that traditional commercial OLAP systems suffer from their long instruction footprint, which results in high response times. High-performance columnstores execute tight instruction streams; however, they spend 25 to 82% of their CPU cycles on stalls both for sequential- and random-access-heavy workloads. Concurrent query execution can improve the utilization, but it creates interference in the shared resources, which results in suboptimal performance.”

OLAP systems are characterized as follows:

1. Traditional OLAP systems are unlike traditional commercial OLTP systems, but they still carry high instruction overheads and deliver performance orders of magnitude lower than state-of-the-art columnstores.
2. Such columnstores use numerous optimization techniques such as SIMD, predication and bloomfilters. However, they spend 25 to 82% of their CPU cycles to stalls. Scan-intensive queries suffer from bandwidth-bounded data cache stalls, while join-intensive queries suffer from latency-bounded data cache stalls.
3. Scan-intensive queries on the state-of-the-art column-stores saturate the memory bandwidth before saturating the cores. Join-intensive queries saturate the cores before saturating the memory bandwidth. Concurrently executing scan- and join-intensive queries enables the saturation of both the cores and bandwidth. The interference in the shared memory bandwidth results in suboptimal performance.

9.11 Distinct schools of thought in data warehouse design

Barry Devlin and Paul Murphy at IBM coined the term information warehouse in (Devlin and Murphy, 1988). W.H. (Bill) Inmon made data warehouses practical by publishing a how-to book in 1992, which is now in its 4th edition (Inmon, 2005). https://en.wikipedia.org/wiki/Bill_Inmon

The following discussion is based on *The Data Administrator Newsletter - TDAN* (tdan.com).

The Inmon method and the Kimball method are two distinctive schools of thought for architecting data warehouses: They both view the data warehouse as the central data repository for the enterprise to serve enterprise reporting needs, and they both use ETL - Extract, Transform, Load to load the data warehouse. The key distinction is how the data structures are modeled, loaded, and stored in the data warehouse. This difference in the architecture impacts the initial delivery time of the data warehouse and the ability to accommodate future changes in the ETL design. When a data architect is asked to design and implement a data warehouse from the ground up, there is the issue of what architecture style should be used to build the data warehouse and what criteria can help an architect choose between the Inmon or the Kimball architecture.

9.11.1 The Inmon method

The building of the data warehouse begins with the corporate data model, which identifies the key subject areas. From this model, a detailed logical model is created for each major entity. The entity structure is built in normalized form and data redundancy is avoided as much as possible. The next step is building the physical model where the physical implementation of the data warehouse is also normalized. This is what Inmon calls “data warehouse” where the single version of truth for the enterprise is managed.

This normalized model makes loading the data less complex, but using this structure for querying is hard as it involves many tables and joins. Inmon suggests building department specific data marts, which can have denormalized data to help with reporting. Any data that comes into the data warehouse is integrated, and the data warehouse is the only source of data for the different data marts. This ensures that the integrity and consistency of data is kept intact across the organization.

<http://web.stanford.edu/dept/itss/docs/oracle/10gR2/server.102/b14223/concept.htm>

Advantages of the Inmon approach are as follows:

1. The data warehouse truly serves as the single source of truth for the enterprise, as it is the only source for the data marts and all the data in the data warehouse is integrated.
2. Data update anomalies are avoided because of very low redundancy. This makes ETL process easier and less prone to failure.
3. The business processes can be understood easily, as the logical model represents the detailed business entities.
4. Very flexible - as the business requirements change or source data changes, it is easy to update the data warehouse as one thing is in only one place.
5. Can handle varied reporting needs across the enterprise.

The disadvantages of Inmon method:

1. The model and implementation can become complex over time as it involves more tables and joins.
2. Experts in data modeling and of the business itself are hard to find and are often expensive.
3. The initial set-up and delivery will take more time, and management needs to be aware of this.
4. More ETL work is needed as the data marts are built from the data warehouse.
5. A fairly large team of specialists need to be around to successfully manage the environment.

9.11.2 The Kimball method

The key difference with Inmon's approach is that the dimensional model proposed by Ralph Kimball for data warehousing model is not normalized. The fundamental concept of dimensional modeling is the star schema: a fact table surrounded by many dimensions. The fact table has all the measures that are relevant to the subject area, and it also has the foreign keys from the different dimensions that surround the fact. The dimensions are denormalized completely so that the user can drill up and drill down without joining to another table. Multiple star schemas may be built to satisfy different reporting requirements. Integration is achieved in the dimensional model using the concept of conformed dimensions, which are the key dimensions that are shared across the different facts will be built once and be used by all the facts. This ensures that one thing or concept is used the same way across the facts.

Another key artifact of the Kimball model is the "enterprise bus matrix", where facts are listed vertically and the conformed dimensions are listed horizontally. Where ever the dimensions play a foreign key role in the fact, it is marked in the document. This serves as an anchoring document showing how the star schemas are built and what is left to build in the data warehouse.

<https://www.kimballgroup.com/data-warehouse-business-intelligence-resources/kimball-techniques/kimball-data-warehouse-bus-architecture/>

Some of the advantages of the Kimball method are as follows:

1. Quick to set-up and build, so the first phase of the data warehousing project will be delivered quickly.
2. The star schema can be easily understood by the business users and is easy to use for reporting. BI tools work well with star schema.
3. The footprint of the data warehousing environment is small, so it occupies less space in the database and it makes the management of the system fairly easier.
4. The performance of the star schema model is very good. The database engine will perform a star join where a Cartesian product will be created using all of the dimension values and the fact table will be queried finally for the selective rows.
5. A few developers and architects is enough to keep the data warehouse performing effectively.
6. Works really well for department-wise metrics and *Key Performance Indicators* - KPI tracking, as the data marts are geared towards department-wise or business process-wise reporting.
7. Drill-across, where a BI tool goes across multiple star schemas to generate a report can be successfully accomplished using conformed dimensions.

Here are some of the disadvantages of the Kimball approach:

1. The essence of the "one source of truth" is lost, as data is not fully integrated before serving reporting needs.
2. Redundant data can cause data update anomalies over time.
3. Adding columns to the fact table can cause performance issues, since the fact tables are designed to be very deep. If new columns are to be added, the size of the fact table becomes much larger and will not perform well. This makes the dimensional model hard to change as the business requirements change.
4. Cannot handle all the enterprise reporting needs because the model is oriented towards business processes rather than the enterprise as a whole.
5. Integration of legacy data into the data warehouse can be a complex process.

9.11.3 Deciding factors for Kimball vs Inmon approaches

1. **Reporting Requirements:** Inmon approach works best if the reporting requirements are strategic and enterprise-wide and integrated reporting is needed. If the reporting requirements are tactical and business process/team oriented, then Kimball works best.
2. **Project Urgency:** If the organization has enough time to wait for the first delivery of the data warehouse then Inmon approach can be followed and the Kimball approach otherwise.
3. **Future Staffing Plan:** If the company can afford a large team of specialists for the data warehouse, then the Inmon method is better, otherwise Kimball is more suited.
4. **Frequency of Changes:** If the reporting requirements are expected to change more rapidly and the source systems are known to be volatile, then the Inmon approach is more flexible and works better. If the requirements and source systems are relatively stable, the Kimball method can be used.
5. **Organization Culture:** The Inmon approach is better if the sponsors of the data warehouse and the managers of the firm understand the value proposition of the data warehouse. The Kimball approach is better otherwise.

9.11.4 Conclusions on data warehousing

Both the Inmon and Kimball approaches have successfully delivered data warehouses. There are even hybrid models. It cannot be said that one approach is better than the other; since both have their advantages and disadvantages and work fine in different scenarios. The architect has to select an approach for the data warehouse depending on the different factors; a few key ones were identified in this paper. Finally, for any approach to be successful, it needs to be carefully thought out and designed to satisfy the organization's BI reporting needs.

9.12 Data lakes

Data lake differs from data warehousing in that it keeps original data fidelity and offers greater scalability than the ETL used by data warehousing. Organizations collected data in data lakes, but it has been difficult to extract valuable information from them. According to Bill Inmon applying the following ingredients to data lakes “will turn them from a garbage dump to a gold mine”.

https://en.wikipedia.org/wiki/Data_lake

1. **Metadata** is a roadmap to the raw data that resides in a data lake and is used to decipher its data. As an example searches on *data base & logic programming - dblp* web site yields the affiliations of individuals with the same name.
2. **Integration Mapping** shows how the data in the lake can be integrated.
3. **Context** is what the data is about. “Turkey” is the name of a country and a bird. It is obviously the former if mentioned in history and geography books and the latter in the context of food or types of birds.
4. **Metaprocess** provides information about the following:
 1. How the data was processed when it was first generated.
 2. Source of data.
 3. Data selection process for inclusion in the data lake.
 4. Further processing of data in the data lake.
 5. Lineage or provenance of data.

https://en.wikipedia.org/wiki/Data_lineage

James Dixon's blog retrieved on 11/7/15 states:

<https://jamesdixon.wordpress.com/>

“If you think of a datamart as a store of bottled water – cleansed and packaged and structured for easy consumption – the data lake is a large body of water in a more natural state. The contents of the data lake stream in from a source to fill the lake, various users of the lake can come to examine, dive in, or take samples.”

A list of key challenges and recommendations for data lakes based on Gartner report are given in following lists:

<https://www.gartner.com/doc/reprints?id=1-1M5EDSWC&ct=190712&st=sb>

- Data needs to be available to the business rapidly, even if that means the data will not be fully curated, governed or integrated.

- Although loosely governed data may meet the rapid access needs of some portions of the customer community, the data will still need refinement to be included in the data warehouse when appropriate.
- IT will need to prioritize the use of its resources when choosing data to curate into the data warehouse.

Recommendations for data and analytics leaders in creating data management strategies are as follows:

1. Collect metadata on the data in the data lake in order to understand how that data is being used and how to start defining the refinement of the datasets.
2. Prioritize datasets for refinement and movement into a data warehouse by using data lake metadata.
3. Use data preparation tools to perform portions of an overall refinement process, by integrating with other types of curation and preparation as part of an iterative approach to data refinement.

9.12.1 Major data lake types

Data in data lakes has been categorized as the following data ponds: 1. Raw. 2. Analog. 3. Application. 4. Textual. 5. Archival, but only three categories are relevant to our discussion.

(1) Analog data. (2) Application data. (3) Textual data.

Analog Data: Analog data is based on measurements of heat, weight, composition, size, etc. Measurements should be accompanied with time of day, location, recording instrument, measurement unit (pound vs kilogram, centimeter vs inch, Fahrenheit vs centigrade, etc.).

Not knowing the unit e.g. kg/cm^2 versus *pounds per square inch - psi* has led to disasters,

<https://www.simscale.com/blog/2017/12/nasa-mars-climate-orbiter-metric/>

The seven base units are: Meter (Distance), Kilogram (Weight), Seconds (Time), Ampere (Electric Current), Kelvin (Temperature), and Candela (Luminosity).

Kx systems and its Kdb+ software

Kx systems is a data analysis software developer and a vendor.

https://en.wikipedia.org/wiki/Kx_Systems

Kdb+ is a language optimized for ingesting, analyzing and storing massive amounts of structured data in real-time and has set multiple records in *Securities Technology Analysis Center - STAC* benchmarks.

<https://www.stacresearch.com/>

Kdb+ is used in financial modeling and data analysis by Wall Street investment banks, and energy, telecommunications, government, and other industries.

Wai-Mee Ching who developed an APL compiler at IBM Research later worked at Kx Systems. He has since developed ELI, a language with *A Programming Language - APL*-like features, but does not require a specialized keyboard with Greek letters.

[https://en.wikipedia.org/wiki/APL_\(programming_language\)](https://en.wikipedia.org/wiki/APL_(programming_language))

<http://fastarray.appspot.com/index.html>

Data reduction techniques for analog data

Data Deduplication discussed in Chapter 2 entails the removal of large chunks of redundant data.

Data excision is the removal of data unneeded for analysis, e.g., data recorded during periods of low activity: night, weekends, holidays.

Data compression has different forms as discussed in Chapter 2.

Data smoothing removes or modifies outliers due to an incorrect reading or a transmission error. Before computing the electric bill at *Tehran Regional Electric Company - TREC* a high-low test was applied to detect incorrect meter readings by comparing consumption against the reading for the previous month and the same period reading a year ago, since electric consumption in KWH is expected to have a cyclic behavior due to *Heating, Ventilation, and Air Conditioning - HVAC*.

Sampling can be used for query processing, as in the case of online aggregation proposed in (Hellerstein et al., 1997).

Rounding is used to remove unnecessary digits, e.g., a measurement device producing digits beyond its accuracy.

Encoding replaces long strings with shorter ones, as in the case of urls.

<https://en.wikipedia.org/wiki/TinyURL>

Twitter used TinyURL until 2009, before switching to Bitly.

<https://en.wikipedia.org/wiki/TinyURL>

<https://en.wikipedia.org/wiki/Bitly>

Tokenization applied to data security, is the process of substituting a sensitive data element with a non-sensitive equivalent, referred to as a token, that has no extrinsic or exploitable meaning or value. The original value is returned via detokenization.

[https://en.wikipedia.org/wiki/Tokenization_\(data_security\)](https://en.wikipedia.org/wiki/Tokenization_(data_security))

It may just be used as succinctly way of referencing data, e.g., A stands for grades above 90.

Thresholding is a form of excision or exclusion for values below and above the threshold.

Clustering groups objects whose feature vectors are close to each other. Multiple points can be represented a single point, which is the centroid of the cluster.

Application Data: The applications may be txns specifying various forms of data, sales, payment, checking, contract, inventory management, and billing and bill payment data. The data may be in the form of records with keys, which can be included in a relational table. Such data can be processed using *Association Rule Mining - ARM* discussed in Section 9.9.

Textual Data is unstructured. This includes phone conversations, corporate contracts, email, insurance claims, sales pitches, court orders, tweets, invitations.

Text data pond

Textual data may contain valuable documents such as *Human Resource - HR* records, insurance policies, loan applications, corporate memos, wills, etc. Text is inherently complex and there are ninety different functions that define the inner workings of textual disambiguation.

Comments/blobs were early starts to define data.

Sound-ex and stemming are to make sense of blob contents. Sound-ex groups words according to a common sound made when word is pronounced. Stemming related to sound-ex is the practice of reducing words to their common Greek or Latin stems. In many cases reducing words to a common foundation helps in understanding syntax. Remove “ed” or “ing” if the word/web ends with them.

<https://en.wikipedia.org/wiki/Stemming>

Tagging is done for many purposes, such as: keywords, action words, red flag (warning) words. The primary drawback is that in order to do tagging you have to know what words need to be tagged.

Natural Language Processing - NLP uses taxonomies to gather sentiment. Taxonomies are much more powerful and general than tagging. With NLP it is possible to gather enough information to start to do real textual analytics. In many ways textual analytics began with taxonomies and NLP.

https://en.wikipedia.org/wiki/Natural_language_processing

Textual Disambiguation builds on taxonomies and NLP. It provides all the capabilities of sentiment analysis, but textual disambiguation provides two important advances over NLP. Those advances are the focus on context and the ability to recognize word and sentence structures, e.g., sentiment analysis tells us the author does not like scrambled eggs, when he says “I do not like scrambled eggs”, but textual disambiguation goes one step further than NLP, giving us a different level of information than NLP, why the author does not like scramble eggs, because they are too expensive, they are runny, they are cold, they have too much cholesterol, he is vegetarian.

Other aspects of textual data are as follows.

Inline contextualization only works on text that has predictable occurrences of data.

Proximity of words yields different meanings, e.g., bruin means brown bear but also a nickname for UCLA students.

Alternate spelling. Words in England are spelled slightly differently from US, e.g., colour instead of color, Elisabeth instead of Elizabeth. A document written in US may be turned into British spelling by running a spell-checker in UK and vice-versa.

Homographic resolution is that the interpretation of a word or abbreviation depends of who wrote it,

(1) “ha” written by a cardiologist means “heart attack”,

(2) “ha” written by an endocrinologist interprets means “hepatitis A”,

<https://www.webmd.com/diabetes/what-is-endocrinologist>

(3) “ha” written by a general practitioner means “headache”.

Acronym resolution, e.g., *AWOL - Absent WithOut Leave*. There are usually several interpretations for an acronym based on the context. Commonly occurring abbreviations in this book are given in the Preface.

Taxonomy resolution, e.g., Volkswagen, Ford, Toyota can be classified as cars. An ontology is a grouping related taxonomies, e.g., countries and states inside countries.

Date standardization that July 5, 1999 and 7/5/1999 are the same as 1999/07/05.

In contrast to structured data, which conforms to well-defined schemas, text has little explicit structure, requiring computer processing toward understanding of the content encoded in text. The current NLP technology has not yet reached a point to precisely understand natural language text, but a wide range of statistical and heuristic approaches to management and analysis of text data have been developed over the past few decades. They are usually very robust and can be applied to analyze and manage text data in any natural language and any topic.

According to “Text Data Management and Analysis” (Zhai and Massung, 2016), there are two kinds of text information systems depending on collaboration with humans:

- Recommender systems find from the most relevant text data needed for solving a specific application problem, thus effectively turning big raw text data into much smaller relevant text data that can be more easily processed by humans.
- Provide support by analyzing patterns in text data to extract and discover useful actionable knowledge directly useful for task completion or decision making.

Information Retrieval - IR which predates database search.

https://en.wikipedia.org/wiki/Information_retrieval

In information retrieval *term frequency - inverse document frequency* *tf - idf*, is a numerical statistic that is intended to reflect how important a word is to a document in a collection or corpus.

<https://en.wikipedia.org/wiki/Tf%E2%80%93idf>

Representative books on IR are: (Manning et al., 2008), (Buttcher et al., 2010), (Baeza-Yates and Ribeiro-Neto, 2011).

The Lambda architecture

The Lambda architecture consists of batch and speed view layers, while server layer provides a merger for querying (John and Misra, 2017). Kappa is Lambda with the batch layer removed.

https://en.wikipedia.org/wiki/Lambda_architecture

Batch layer stores immutable data, constantly growing in size and recomputes views all the time.

Speed or real-time layer is a constant stream of data, which is discarded after a certain interval.

Serving layer indexing to improve performance of exposed batch views, exposes real-time views created incrementally by the speed layer, merges result from both batch and speed views in a consistent fashion.

9.13 Open source big data projects

Five open-source projects are identified in (Marz and Warren, 2015).

Batch computation systems: *Hadoop Distributed File System - HDFS* is a fault-tolerant storage system that can scale to petabytes of data. Hadoop MapReduce produced is a horizontally scalable computation framework that integrates with HDFS (Dean and Ghemawat, 2004).

https://en.wikipedia.org/wiki/Apache_Hadoop

Hadoop is available from Cloudera and Hortonworks, which are merging and MapR acquired by HP.

<https://mapr.com/>

Serialization provides tools and libraries for transferring objects between languages: serialize an object into a byte array from any language, and then deserialize that byte array into an object in another language. A *Schema Definition Language - SDL* defines objects and their fields, and provides mechanisms to safely version objects so that a schema can be evolved without invalidating existing objects.

Three notable serialization frameworks are:

1. **Apache Thrift** <http://thrift.apache.org/>

2. **Protocol Buffers** developed at Google. https://en.wikipedia.org/wiki/Protocol_Buffers

3. **Apache Avro** is a row-oriented remote procedure call and data serialization framework, which uses JSON for defining data types and protocols and serializes data in a compact binary format.

https://en.wikipedia.org/wiki/Apache_Avro

Random-access NoSQL databases include Cassandra, HBase, MongoDB, Voldemort, Riak, CouchDB, and others. NoSQL databases sacrifice the full expressiveness of SQL and instead specialize in certain kinds of operations. They all have different semantics and are meant to be used for specific purposes. They are not meant to be used for arbitrary data warehousing. The choice of a NoSQL database is like choosing between a hash map, sorted map, linked list, or vector when choosing a data structure to use in a program.

Messaging/queuing systems such as Apache Kafka provide a way to send and consume messages between processes in a fault-tolerant and asynchronous manner. A message queue is a key component for doing realtime processing.

https://en.wikipedia.org/wiki/Message_queue

Realtime computation system such as Storm are high throughput, low latency, stream-processing systems. They cannot carry out the range of computations a batch-processing system can, but they process messages extremely quickly.

<https://storm.apache.org/>

9.14 Semi-structured data and its model

The advantages of the semi-structured data model are as follows:

https://en.wikipedia.org/wiki/Semi-structured_data

1. Represents the information of data sources that cannot be constrained by a schema.
2. Provides a flexible format for data exchange between different types of DBMSs.
3. Helpful for browsing structured data as semi-structured.
4. Schema can easily be changed.
5. Data transfer format may be portable.

The main disadvantage of the semi-structured or self-describing DB model is that queries are not as efficient as in the case of structured data model. The records in a semi-structured database are stored with unique IDs, which are referenced with pointers to their location on disk. This allows navigational or path-based queries, but is not efficient because it has to seek around the disk following pointers, as in the case with navigational databases (Bachman, 1973). This is not the case for data held in main memory.

The popularity of semi-structured data has increased with the advent of the Internet, where full-text documents and structured databases are not the only forms of data anymore, and different applications need a medium for exchanging information. Semi-structured data is also often found in *Object-Oriented - OO* databases.

eXtensible Markup Language - XML and *Electronic Data Interchange - EDI* are all forms of semi-structured data.

<https://developers.google.com/gmail/markup/>

https://en.wikipedia.org/wiki/GS1_EDI

<https://en.wikipedia.org/wiki/XML>

Object Exchange Model - OEM was created prior to XML as a means of self-describing a data structure.

https://en.wikipedia.org/wiki/Object_Exchange_Model

Lore was a DBMS for XML developed at Stanford U., which was popularized by web services that are developed utilizing SOAP principles.

<http://infolab.stanford.edu/lore/home/index.html>

Simple Object Access Protocol - SOAP is a message exchange protocol specification for structured information in the implementation of web services in computer networks.

<https://en.wikipedia.org/wiki/SOAP>

The purpose of SOAP is to provide extensibility, neutrality and independence. It uses XML Information Set for its message format and relies on application layer protocols such as *HyperText Transfer Protocol - HTTP*.

https://en.wikipedia.org/wiki/Hypertext_Transfer_Protocol

or *Simple Mail Transfer Protocol - SMTP*, for message negotiation and transmission.

https://en.wikipedia.org/wiki/Simple_Mail_Transfer_Protocol

SOAP has three major characteristics:

Extensibility: Security and WS-Addressing are among the extensions under development.

Web-Services - WS-Addressing is a standardized way of including message routing data within SOAP headers. Instead of relying on network-level transport to convey routing information, a message utilizing WS-Addressing may contain its own dispatch metadata in a standardized SOAP header.

<https://en.wikipedia.org/wiki/WS-Addressing>

Neutrality: SOAP can operate over any protocol such as HTTP, SMTP, TCP, UDP.

https://en.wikipedia.org/wiki/User_Datagram_Protocol

Independence: SOAP allows for any programming model.

XQuery 3.1 is a versatile query and application development language for processing the information content of structured and semi-structured documents, relational databases and tree-based databases (Section 27.7 in (Ramakrishnan and Gehrke, 2002)). XQuery supports powerful optimizations and pre-compilation leading to very efficient searches over large amounts of data, including XML-native databases with an efficient internal storage (Chamberlin et al., 2001).

<https://en.wikipedia.org/wiki/XQuery>

JavaScript Object Notation - JSON is an open standard format to transmit data objects consisting of attribute-value pairs. <https://www.json.org/>

It is used primarily to transmit data between a server and web application as an alternative to XML. JSON has been popularized by web services developed utilizing *Representational State Transfer - REST* principles.

https://en.wikipedia.org/wiki/Representational_state_transfer

MongoDB and **Couchbase** are two DBMSs that store data natively in JSON format, leveraging the pros of semi-structured data architecture.

<https://en.wikipedia.org/wiki/MongoDB>

https://en.wikipedia.org/wiki/Couchbase,_Inc

Oracle Autonomous JSON Database is a cloud document database service that makes it simple to develop JSON-centric applications. It features simple document APIs, serverless scaling, high performance ACID txns, comprehensive security, and low pay-per-use pricing. Autonomous JSON Database automates provisioning, configuring, tuning, scaling, patching, encrypting, and repairing of databases, eliminating database management and delivering 99.995% availability.

<https://www.oracle.com/autonomous-database/autonomous-json-database/>

9.14.1 Pros and cons of semi-structured data format

Advantages

Programmers trying to make objects persistent from their application to a database do not need to worry about object-relational impedance mismatch, but can serialize objects via a light-weight library.

1. Support for nested or hierarchical data of ten simplifies data models representing complex relationships between entities.
2. Support for lists of objects simplifies data models by avoiding messy translations of lists into a relational data model.

Disadvantages

1. The traditional relational data model has a popular and ready-made query language, SQL.
2. Prone to “garbage in, garbage out”; by removing restraints from the data model, there is less fore-thought that is necessary to operate a data application.

There are many ways to make data persistent in Java: *Java Data Base Connectivity - JDBC* (Ramakrishnan and Gehrke, 2002), serialization, file IO, *Java Cryptography Architecture - JCA*, object databases, and XML databases. The majority of data is made persistent in databases, specially relational databases, which are the standard mode of persistent storage for most industries.

https://en.wikipedia.org/wiki/Java_Database_Connectivity

9.15 Big data technology and the five Vs

Big data deals with five Vs. (1) Volume, (2) Velocity, (3) Variety, (4) Veracity, (5) Value.

Volume determines the amount of data in various forms: mail, Twitter, Facebook, other social media, videos, pictures, SMS - *Short Message Service*, call records, or data from devices and sensors. Powers of ten used in specifying volume are listed in the Preface.

Velocity is the rate at which generated big data is accumulated. Some generated data is ephemeral and need not be accumulated. Credit card swipes are generated at millions of purchase sources and each swipe takes considerable amount of processing to detect overdraft, fraud, recording the amount, issuing alerts, i.e., an email to the holder of credit card is generated when it is used.

Variety specifies unstructured data, which is 80% of data: photos, video clips, voice recordings, social media updates, data from sensors.

Veracity which is the validity, trustworthiness, correctness and accuracy of data.

Value is the most important dimension of big data.

Data analytics application should provide the following capabilities: (1) Storing data. (2) Enriching or processing data. (3) Data analysis and visualization.

The various segments and verticals within the Big Data according to Forbes are as follows:

<https://www.forbes.com/sites/davefeinleib/2012/06/19/the-big-data-landscape/#feb42035e6dd>

(1) Big Data infrastructure. (2) Components of its ecosystem. (3) Analytics architecture. (4) Distributed batch processing. (5) Distributed databases and NoSQL. (6) Real-time and stream processing.

Major big data technologies are as follows:

Hadoop is a distributed framework for data storage and computation on commodity hardware in a reliable and scalable manner. Hadoop via HDFS distributes data in chunks over all the nodes and then processing the data concurrently on all the nodes using mappers and reducers via MapReduce.

NoSQL is basically a tool to process huge volumes of multi-structured data, such as HBase and Cassandra. Unlike traditional database systems, they generally have no *SPOF* - *Single Point of Failure* and are scalable.

Massively Parallel Processing - MPP databases: These are computational platforms that are able to process data at a very fast rate. Data is segmented into chunks across different nodes in the cluster and then processed in parallel. There is similarity to Hadoop in terms of data segmentation and concurrent processing at each node. Unlike Hadoop MPP operates at disk level, MPP databases load the data into memory and operate upon the collective memory of all nodes in the cluster.

Exascale scientific data management has become an increasingly difficult challenge, since modern experiments and instruments are generating unprecedented volumes of data and their accompanying dataflows are becoming more complex (Lassnig et al., 2020). Straightforward approaches are no longer applicable at the required scales and there are few reports on long-term operational experiences.

There are challenges in operating the exascale dataflows of the high energy physics experiment ATLAS, detailing the concepts and architecture of the data management system Rucio (Barisits et al., 2019) that was built to take up these challenges. Also described are how other experiments evaluated, modified, and adopted the Rucio system for their own needs.

<https://rucio.cern.ch/>

9.16 Hadoop technology ecosystem

Apache Hadoop is a collection of open-source software utilities that facilitate using a network of many computers to solve problems involving massive amounts of data and computation. It provides a software framework for distributed storage and processing of big data using the MapReduce programming model. All the modules in Hadoop are designed with a fundamental assumption that hardware failures are common occurrences and should be automatically handled by the framework.

The storage part of Apache Hadoop is HDFS and a processing part is the MapReduce programming model.

https://en.wikipedia.org/wiki/Apache_Hadoop

Hadoop splits files into large blocks and distributes them across nodes in a cluster. It then transfers packaged code into nodes to process the data in parallel, which is an instance of data-centric computing.

https://en.wikipedia.org/wiki/Data-centric_computing

This approach takes advantage of data locality, where nodes manipulate data accessible to them. This allows the dataset to be processed faster and more efficiently than it would be in a more conventional supercomputer architecture that relies on a parallel file system, where computation and data are distributed via high-speed networking.

The base Apache Hadoop framework is composed of the following modules:

Hadoop Common – contains libraries and utilities needed by other Hadoop modules.

HDFS stores data on commodity machines, providing high aggregate bandwidth across the cluster.

YARN - Yet Another Resource Negotiator for Hadoop was introduced in 2012 is responsible for managing computing resources in clusters and using them for scheduling user applications.

MapReduce is a programming model and an associated implementation for processing and generating big data sets with a parallel, distributed algorithm on a cluster. A MapReduce program is composed of a map procedure, which performs filtering and sorting, and a reduce method, which performs a summary operation.

<https://en.wikipedia.org/wiki/MapReduce>

Apache Hadoop's MapReduce and HDFS components were inspired by Google's papers on MapReduce and Google File System described in (Dean and Ghemawat, 2004). The Hadoop framework itself is mostly written in Java. MapReduce

is explained in (Parsian, 2015), (Ozsu and Valduriez, 2020).

<https://www.oreilly.com/library/view/data-algorithms/9781491906170/preface01.html>

Apache Pig is a platform on top of Hadoop, which can be used in conjunction with or as a substitute for MapReduce. It is a high-level language and is widely used for creating components to analyze large datasets. Pig has a compiler that translates Pig scripts to MapReduce jobs which can be processed in parallel.

1. Programming in Pig Latin is easy.
2. Optimizing the jobs is efficient and easy.

It is extendible.

Application logic or interaction in Hadoop can be accomplished with one of the following:

Hive is a data warehousing layer that is built on top of the Hadoop platform providing a facility to interact with, process, and analyze HDFS data with SQL-like Hive queries. This makes the transition from the RDBMS to Hadoop easier.

Cascading is a framework that exposes a set of data processing APIs and other components that define, share, and execute the data processing over the Hadoop/Big Data stack. It is an abstracted API layer over Hadoop, which is widely used for application development, because of its ease of development, creation of jobs, and job scheduling.

9.16.1 Four steps on the workflow of an analytical application

a. Business dataset selection where architects and designers identify and decide upon the data sources that will be providing the input data to the application for analytics.

b. Dataset processing for data ingestion and processing. The ETL step applies filtering, which reduces the volume to limit the analysis to meaningful and valued data. Filtered data is next transformed by converting to an entity-driven form, e.g., Hive table, JSON, *Plain Old Java Object - POJO*, etc.

https://en.wikipedia.org/wiki/Plain_old_Java_object

This marks the completion of the ETL step and makes the data ingested into the system available for actual processing. A dataset is loaded into the analytical data mart depending upon the use case and the duration for which a given dataset is to be analyzed. Only one day's worth of data may be required for analytics, although a data mart may have a year's worth of credit card txns. This segregation is important because it helps in figuring out where real-time compute capability is needed and which data should be operated upon a deep learning application.

c. Automated solution has the following choices:

1. **Analytical Engine** executes various batch jobs, statistical queries, or data cubes on the landing data mart.
2. **Dashboard/Workbench** components generally operate on low latency, close to real-time, analytical data marts.
3. **Auto learning synchronization mechanism** is perfect for advanced analytical application and captures patterns and evolves the data management methodologies.
4. **Measured analysis and optimization:**

d. Presentation depends upon the target audience: Data visualization can be achieved using a custom-built *User Interface - UI* presentation layer, business insight reports, dashboards, charts, graphs, and so on.

9.17 Distributed batch vs inline processing

The two categories of processing that can be applied to data:

- (1) Batch processing processes events in batches;
- (2) Sequential or inline processing processes events as they are generated.

A key aspect of batch processing is batch size, which can be determined by time ranges such as hourly or daily batches, or dynamic and data-driven, based on the occurrence of a particular pattern to start and end.

Once a batch boundary is demarcated by adding a header/trailer, in certain cases, the order of records or the sequence needs to be maintained. Once nodes complete the processing of their allocated batches, the results are collated and summarized for the final results.

Computation or storage capability can be added by two means:

Vertical scaling adds more compute capability in the form of CPUs or RAMs to an existing node to produce a more powerful machine. Vertical scaling is limited and failure prone and the higher-end machines are expensive. Grosch's law which states performance increases with the square of the cost is not applicable in all cases.

https://en.wikipedia.org/wiki/Grosch%27s_law

It has been asserted however that this law was applicable to the way computers were priced by IBM at the time the law was enunciated.

Horizontal scaling adds more than one node to perform the task is the preferred solution to the problem.

In clustering the computational capability is derived from a collection of nodes. At its core, Hadoop is a distributed, batch-processing compute framework operating upon MapReduce principles. HDFS has a master/slave architecture, which consists of a single NameNode, a master server that manages the file system namespace and regulates access to files by clients. In addition, there are a number of DataNodes, usually one per node in the cluster, which manage storage attached to the nodes that they run on. HDFS exposes a file system namespace and allows user data to be stored in files. Internally, a file is split into one or more blocks and these blocks are stored in a set of DataNodes.

The NameNode executes file system namespace operations like opening, closing, and renaming files and directories. It also determines the mapping of blocks to DataNodes. The DataNodes serve read and write requests from clients and also perform block creation, deletion, and replication upon instruction from the NameNode.

https://hadoop.apache.org/docs/r1.2.1/hdfs_design.html

The NameNode in Hadoop used to be a single point of failure, but YARN has removed this limitation by decoupling MapReduce and Hadoop and providing the scope of integration with other real-time, parallel processing compute engines like Spark, *Message Processing Interface - MPI*, etc.

Moving data to compute makes sense for low volume data, but this is not viable for Big Data (data-centric computing). So Hadoop has introduced a shift in paradigm by creating batches of input data called blocks and distributing them to each node in the cluster. At the initialization stage, the Big Data file is pushed into HDFS. Then, the file is split into chunks by the Hadoop NameNode (master node) and is placed onto individual DataNodes (slave nodes) in the cluster for concurrent processing.

Job Tracker moves execution code or processing to the data, which is an instance of data-centric computing. The compute component includes a Mapper and a Reduce class, where the Mapper class does the job of data filtering, transformation, and splitting. Only local blocks are processed by mapper. Once the Mappers are executed, their outputs are shuffled through to the appropriate Reduce nodes. A Reduce class, by its functionality, is an aggregator for compiling all the results from the mappers.

9.18 NoSQL/non-relational databases

NoSQL databases can take a variety of forms and are hence better suited for complex web applications. The main difference between NoSQL and relational databases is that the latter rigidly define the data types to be inserted into the database, whereas NoSQL databases can be schema agnostic, allowing unstructured and semi-structured data to be stored and manipulated. There are several types of NoSQL databases and some products fall into more than one category, e.g., Couchbase is both a document database and a *Key-Value - KV* store.

<https://en.wikipedia.org/wiki/NoSQL>

In distributed NoSQL databases the data and the control for execution is distributed across all nodes, so that there is no single point of control. The following processes add complexity to ensure correctness.

Replication is tracked by a special component of distributed databases. Special software is provided to propagate updates/additions/deletions to all copies of data once it is logged. A monograph on data replication is (Kemmer et al., 2010).

[https://en.wikipedia.org/wiki/Replication_\(computing\)](https://en.wikipedia.org/wiki/Replication_(computing))

Duplication is to ensure that there is not a single point of failure and if a node fails other nodes provide duplicated data.

A clear advantage NoSQL database environment is a non-relational and predominately distributed database system, which facilitates the rapid analysis of extremely high-volume, disparate data types. With the advent of Big Data, NoSQL databases have become a cheap and scalable alternative to the traditional RDBMS. The *User Service Platform - USP* offers availability and fault-tolerance.

<https://www.friendly-tech.com/blog/what-is-tr-369-usp-and-what-value-does-it-bring-to-service-and-application-providers>

NoSQL offers a flexible and extensible schema model with added advantages of endless scalability, distributed setup, and the liberty of interfacing with non-SQL interfaces.

NoSQL databases can be classified as follows:

9.19 Key-value stores

A **key-value - KV** store is a data storage paradigm designed for storing, retrieving, and managing associative arrays, and a data structure known as a dictionary or hash table. Dictionaries contain a collection of objects, or records, which in turn have many different fields within them. These records are stored and retrieved using a key that uniquely identifies the record. An alphabetic list of KV stores and a listing by revenue is provided in Table 9.8.

https://en.wikipedia.org/wiki/Key%E2%80%93value_database

<https://db-engines.com/en/ranking/key-value+store>

TABLE 9.8 Ranking of key-value stores, Multi-Model - MM.

6/20	5/20	6/19	Company	Data model	6/20	5/20	6/19
1.	1.	1.	Redis	Key-value, MM	145.64	+2.17	−0.48
2.	2.	2.	Amazon DynamoDB	MM	64.87	+0.15	+9.61
3.	3.	3.	Azure Cosmos DB	MM	30.80 +	0.13	+2.56
4.	4.	4.	Memcached	Key-value	24.81	+0.88	−3.20
5.	5.	5.	Hazelcast	Key-value, MM	8.41	−0.27	+0.55

Descriptions of KV stores ordered by ranking are as follows:

Redis is an open source NoSQL KV data store and more generally data structure server used as a database, cache and message broker. It is unlike MongoDB disk-based document store, which could be used as a KV store. The closest analog is Memcached, but with built-in persistence (snapshotting or journaling to disk) and more datatypes. The data will not disappear when restarted similarly to memcached, which is cached in DRAM. Those two additions make Redis valuable.

Redis supports data structures such as strings, hashes, lists, sets, sorted sets with range queries, bitmaps, hyperlog logs, geospatial indexes with radius queries and streams. Redis has built-in replication, Lua scripting, LRU eviction, txns and different levels of on-disk persistence, and provides high availability via Redis Sentinel and automatic partitioning with Redis Cluster. It runs atomic operations on the following types, like appending to a string; incrementing the value in a hash; pushing an element to a list; computing set intersection, union and difference; or getting the member with highest ranking in a sorted set. In order to achieve its outstanding Redis works with an in-memory dataset. Depending on use case it can be persisted either by dumping the dataset to disk periodically, or by appending each command to a log. Persistence can be optionally disabled, if you just need a feature-rich, networked, in-memory cache. Redis also supports trivial-to-setup master-slave asynchronous replication, with very fast non-blocking synchronization, auto-reconnection with partial resynchronization on net split. Other features are: Txns, Pub/Sub, Lua scripting, Keys with a limited time-to-live, LRU eviction of keys.

<https://redis.io>

Amazon DynamoDB is a fast, fully managed NoSQL database service that makes it simple and cost-effective to store and retrieve any amount of data, and serve any level of requested traffic. Its under ten millisecond latency makes it suitable for gaming, advertising technology, mobile, etc.

https://en.wikipedia.org/wiki/Amazon_DynamoDB

Azure Cosmos DB is Microsoft’s globally-distributed NoSQL multi-mode database service, “for managing data at planet-scale” launched in 2010. Cosmos DB stores “items” in “containers” or “documents” in “collections” when using the MongoDB-compatible API. Containers are grouped in “databases” analogous to namespaces above schema-agnostic containers, which means that a schema is enforced when adding items.

https://en.wikipedia.org/wiki/Cosmos_DB

By default, every field in each item is automatically indexed, generally providing good performance without tuning to specific query patterns. These defaults can be modified by setting an indexing policy which can specify, for each field the index type and precision desired. Cosmos DB offers two types of indexes:

range supporting range and ORDER BY queries,

spatial supporting spatial queries from points, polygons and line strings encoded in standard GeoJSON fragments.

A comprehensive discussion of indexing structures appears in (Samet, 2006).

Containers can also enforce unique key constraints to ensure data integrity. Each Cosmos DB container exposes a change feed, which clients can subscribe to in order to get notified of new items being added or updated in the container.

Item deletions are currently not exposed by the change feed. Changes are persisted by Cosmos DB, which makes it possible to request changes from any point in time since the creation of the container.

Time to Live - TTL can be specified at the container level to let Cosmos DB automatically delete items after a certain amount of time (in seconds). This countdown starts after the last update of the item.

Memcached is a general-purpose distributed memory-caching system, which is free and open-source software. It is often used to speed up dynamic database-driven websites by caching data and objects in RAM to reduce the number of times an external data source must be read.

<https://en.wikipedia.org/wiki/Memcached>

Hazelcast can lead to the *In Memory Data Grids - IMDG*, which result in slow or unscalable RDBMS. Scaling a non-efficient RDBMS at best involves knowledge of complex configuration techniques and at worst could require the addition of expensive non-commodity hardware. Hazelcast can be easily added into the workflow to solve this issue, it can also be used to expedite reading by caching data in memory and if slow updates are an issue for user applications.

<https://hazelcast.com/>

<https://en.wikipedia.org/wiki/Hazelcast>

TABLE 9.9 Comparing in-memory datagrid and in-memory database.

	IMDG	IMDB
Benefits	Flexible, support any type of application and cache any object Scale-out is automated Integration with various programming languages	Highly automated Full support for SQL No changes to application Minimal to no administration
Use cases	Web applications that require extreme performance and scale Real-time big data applications that need fast data access	Increasing scale and performance of existing ER, CRM, and other package applications Building database applications that require low latency Improving performance for data warehouse and analytic workloads

IMDG and *In-Memory Data Bases - IMDB* are compared in Table 9.9.

<https://www.electronicdesign.com/technologies/embedded-revolution/article/21805454/whats-the-difference-between-an-inmemory-data-grid-and-inmemory-database>

9.20 Document stores

A document-oriented database, or document store, is a computer program designed for storing, retrieving and managing document-oriented information, which is a form of semi-structured data. Document databases are listed alphabetically below and their ranking is given in Table 9.10.

https://en.wikipedia.org/wiki/Document-oriented_database

<https://db-engines.com/en/ranking/document+store>

MongoDB is an open-source database used for a wide variety of applications, allowing its schemas to change quickly as applications evolve, while still providing the functionality developers expect from traditional databases, such as secondary indexes, a full query language, and strict consistency. MongoDB is built for scalability, performance and high availability, scaling from single server deployments to large, complex multi-site architectures. MongoDB leverages in memory computing providing high performance reads and writes. MongoDB's native replication and automated failover enable enterprise-grade reliability and operational flexibility. MongoDB was in the news 6/12/20 as it was adopted by *Hongkong and Shanghai Banking Corp. - HSBC*.

<https://diginomica.com/hsbc-moves-65-relational-databases-one-global-mongodb-database>

<https://www.mongodb.com/>

DynamoDB is covered in Section 9.19.

Azure Cosmos DB is covered in Section 9.19.

TABLE 9.10 Ranking of document stores, *Multi-Model - MM, Google Cloud - GC.*

6/20	5/20	6/19	Company	Data model	6/20	5/20	6/19
1.	1.	1.	MongoDB	Document, MM	437.08	-1.92	+33.17
2.	2.	2.	Amazon DynamoDB	MM	64.87	+0.15	+9.61
3.	3.	4.	Azure Cosmos DB	MM	30.80	+0.13	+2.56
4.	4.	3.	Couchbase	Document, MM	29.14	+0.56	-4.22
5.	5.	5.	CouchDB	Document	16.07	-0.85	-3.15
6.	6.	7.	Firebase Realtime	Document	13.99	+0.87	+3.14
7.	7.	6.	MarkLogic	MM	11.24	+0.28	-2.26
8.	8.	8.	Realm	Document	8.36	-0.02	+0.69
9.	9.	10.	GC Firestore	Document	7.52	+1.18	+2.64
10.	10.	11.	ArangoDB	Multi-model	5.38	+0.70	+0.81
11.	11.	12.	GC Datastore	Document	5.15	+0.49	+0.65
12.	12.	9.	OrientDB	Multi-model	4.82	+0.68	-0.77

Couchbase server originally known as Membase is an open source, distributed NoSQL document-oriented database that is optimized for interactive applications serving many concurrent users; creating, storing, retrieving, aggregating, manipulating and presenting data. Couchbase provides easy-to-scale key-value or document access with low latency and high sustained throughput. It is designed to be clustered from a single machine to very large scale deployments.

<https://www.couchbase.com>

<https://www.infoworld.com/article/3430716/couchbase-review-a-smart-nosql-database.html>

CouchDB from Apache is an open-source document-oriented NoSQL database developed by Ericsson Communications in Erlang.

[https://en.wikipedia.org/wiki/Erlang_\(programming_language\)#Data_types](https://en.wikipedia.org/wiki/Erlang_(programming_language)#Data_types)

<https://en.wikipedia.org/wiki/Ericsson>

CouchDB uses multiple formats and protocols to store, transfer, and process its data. It uses JSON to store data, JavaScript as its query language using MapReduce and HTTP for an API. CouchDB can even serve web apps directly and data or apps can be efficiently distributed using CouchDB's incremental replication. CouchDB supports master-master setups with automatic conflict detection.

https://en.wikipedia.org/wiki/Apache_CouchDB

Firebase is a mobile and web application development platform developed by Firebase, Inc. in 2011. It was acquired by Google in 2014 and as of March 2020, this platform has 19 products and 1.5 million apps.

<https://en.wikipedia.org/wiki/Firebase>

MarkLogic server is an Enterprise NoSQL database, which fuses together database internals, search-style indexing, and application server behaviors into a unified system. It uses XML documents as its data model and stores the documents within a txn repository. It indexes the words and values from each of the loaded documents, as well as the document structure. Because of its unique Universal Index, MarkLogic does not require advance knowledge of the document schema. Through its application server capabilities, it is programmable and extensible. MarkLogic Server clusters on commodity hardware using a shared-nothing architecture and differentiates itself in the market by supporting massive scale and high performance customer deployments have scaled to hundreds of terabyte of source data, while maintaining sub-second query response time.

<https://developer.marklogic.com/free-developer/>

Realm is an open source ODBMS, initially for mobile OS, but also available for platforms such as Xamarin, React Native, and others, including desktop applications, and is Apache licensed.

[https://en.wikipedia.org/wiki/Realm_\(database\)](https://en.wikipedia.org/wiki/Realm_(database))

Google Cloud Firestore is a fast, fully managed, serverless, cloud-native NoSQL document database that simplifies storing, synchronizing, and querying data for your mobile, web, and IoT apps at global scale. Its client libraries provide live synchronization and offline support, while its security features and integrations with Firebase and GCP accelerate building truly serverless apps.

<https://cloud.google.com/firestore>

ArangoDB is a fast growing native multi-model NoSQL database, combining the power of graphs, with JSON documents and a key-value store.

<https://en.wikipedia.org/wiki/JSON>

ArangoDB makes all of data-models accessible with a single elegant declarative query language. ArangoDB is the simple, versatile and efficient answer to many challenges facing developers. Simplifying complexity and increasing productivity is the mission of ArangoDB GmbH (company with limited liability).

www.arangodb.com

<https://wiki2.org/en/ArangoDB>

Google Cloud Datastore is a highly scalable, fully managed NoSQL database service on the Google Cloud Platform. Cloud storage allows the user to save data and files in an off-site, but accessible location either through the public internet or a dedicated private network connection.

https://en.wikipedia.org/wiki/Google_Cloud_Datastore

Firestore is the next major version and rebranding of Datastore, which takes the best of Datastore and the Firebase Realtime Database. Firestore is a NoSQL document database built for automatic scaling, high performance, and ease of application development.

https://en.wikipedia.org/wiki/Google_Cloud_Datastore

OrientDB is an open source NoSQL database management system written in Java. It is a Multi-model database, supporting graph, document, key-value, and object models, but the relationships are managed as in graph databases with direct connections between records. OrientDB since 2018 is supported by SAP.

<https://en.wikipedia.org/wiki/OrientDB>

9.21 Time-series databases

“Time series forecasting is a key ingredient in the automation and optimization of business processes: in retail, deciding which products to order and where to store them depends on the forecasts of future demand in different regions; in cloud computing, the estimated future usage of services and infrastructure components guides capacity planning; and workforce scheduling in warehouses, call centers, factories requires forecasts of the future workload. Recent years have witnessed a paradigm shift in forecasting techniques and applications, from computer-assisted model- and assumption-based to data-driven and fully-automated. This shift can be attributed to the availability of large, rich, and diverse timeseries data sources, posing unprecedented challenges to traditional time series forecasting methods. As such, how can we build statistical models to efficiently and effectively learn to forecast from large and diverse data sources? How can we leverage the statistical power of “similar” time series to improve forecasts in the case of limited observations? What are the implications for building forecasting systems that can handle large data volumes?” (Faloutsos et al., 2020)

Time Series DataBase - TSDB is optimized for storing and serving time series through associated pairs of time(s) and value(s). Although it is possible to store time-series data with many different database formats, the design of these systems with time as a key index is distinctly different from relational databases. Early time series databases are associated with industrial applications, which could efficiently store measured values from sensory equipment, but are now used in support of a much wider range of applications. Data compression is used to reduce the volume of data. Data storage and access in time-series databases is discussed in (Dunning and Friedman, 2014) and alphabetically and by revenue here.

https://en.wikipedia.org/wiki/Time_series_database

The ranking of time-series databases is given in Table 9.11.

<https://db-engines.com/en/ranking/time+series+dbms>

TABLE 9.11 Ranking of time-series databases and their revenue, *Multi-Model - MM*.

6/20	5/20	6/19	Company	Data model	6/20	5/20	6/19
1.	1.	1.	InfluxDB	Time Series	21.18	+0.26	+3.19
2.	2.	2.	Kdb+	Time Series, MM	5.86	+0.49	+0.05
3.	3.	4.	Prometheus	Time Series	4.59	+0.28	+1.27
4.	4.	3.	Graphite	Time Series	3.63	+0.17	+0.30
5.	5.	5.	RRDtool	Time Series	2.90	+0.38	+0.23

InfluxDB is an open-source time series database, written in Go and optimized for fast, high-availability storage and retrieval of time series data in fields such as operations monitoring (Turnbull, 2016), application metrics, IoT sensor data, and real-time analytics.

<https://en.wikipedia.org/wiki/InfluxDB>

Go is a statically typed, compiled programming language designed at Google by Robert Griesemer, Rob Pike, and Ken Thompson (invented B a predecessor to C at Bell Labs). Go is syntactically similar to C, but with memory safety, garbage collection, structural typing, and *Communicating Sequential Process - CSP* capability.

[https://en.wikipedia.org/wiki/Go_\(programming_language\)](https://en.wikipedia.org/wiki/Go_(programming_language))

CSP is a formal language for describing patterns of interaction in concurrent systems.

https://en.wikipedia.org/wiki/Communicating_sequential_processes

kdb+ is a column-based relational time series database with in-memory abilities, developed and marketed by Kx Systems. The database is commonly used in high-frequency trading to store, analyze, process, and retrieve large data sets at high speed.

<https://en.wikipedia.org/wiki/Kdb%2B>

Kdb+ is supposed to be world's fastest time-series database.

<https://kx.com/why-kx/>

What makes kdb+ fast?

This discussion is based on the following blog.

<https://kx.com/blog/what-makes-time-series-database-kdb-so-fast/>

The relational and columnar database kdb+ is well known for exceptionally fast analytics on large scale datasets in motion and at rest. This has made kdb+ the technology of choice for capital markets applications and industrial IoT applications involving large amounts of time-series data. This is due to the way data is optimally stored for manipulation and querying of time-series data and relational data in the programming platform. Although there are other columnar databases in the market, there are no databases that combine all of these aspects together.

This optimization enables kdb+ to deliver orders of magnitude better performance when working with sensor and other types of time-series data compared to alternative technologies. Some performance snippets from various customer implementations of kdb+ running on a single server are listed below.

1. Ingest and store 30 million sensor readings/second and over 10 TB per day.
2. Aggregate over 40 billion sensor readings in under 2 minutes.
3. Retrieve over 100 million sensor readings per second.
4. Search in-memory tables at 4 billion records/second.
5. Sub-millisecond latency for streaming event processing.

What makes kdb+ unique is that as an in-memory, time-series database it enables data to be ingested and made immediately available for queries. This makes it ideal for industrial IoT applications for ingesting, storing, processing, and analyzing time-series data – including IoT sensor data used in manufacturing and financial market data.

To achieve this level of performance, data is first placed in in-memory tables using a prescribed schema and protected via an on-disk log. By going to memory first, and making data available immediately for query, it enables kdb+ to support much higher ingestion rates of many millions of readings per second, hundreds of MBs/s on a single server than other technologies.

According to the following report written by Hugh Hyndman.

<https://kx.com/media/2018/06/KdbTransitive-Comparisons.pdf>

kdb+ (with Raspberry Pi, Macbook, servers with 1, 4, 8 cores vs InfluxDB vs Elasticsearch in a processing Query 4 yields the following queries per second:

1,333, 24,266, 12,455, 53,682, 3,600, 79.

Raspberry Pis are small single-board computers developed in UK to promote computer science in schools and developing countries.

https://en.wikipedia.org/wiki/Raspberry_Pi

Relevant to the discussion is Oracle Database in main memory, also see Oracle patent (Lahiri et al., 2015).

<https://www.oracle.com/a/tech/docs/database-in-memory-ds-19c.pdf>

<https://patents.justia.com/patent/10311154>

Prometheus is a free software application used for event monitoring and alerting. It records real-time metrics in a time series database built using a HTTP pull model, with flexible queries and real-time alerting. It is written in Go, licensed under the Apache 2 License, with the source code available on GitHub, and is a graduated project of the Cloud Native Computing Foundation, along with Kubernetes and Envoy.

[https://en.wikipedia.org/wiki/Prometheus_\(software\)](https://en.wikipedia.org/wiki/Prometheus_(software))

<https://prometheus.io/>

Graphite is a *Free Open-Source Software - FOSS* tool that monitors and graphs numeric time-series data such as the performance of computer systems and released as open-source software in 2008. Graphite was developed by Orbitz Worldwide, Inc. <https://orbitz.com> a subsidiary of Expedia Group. Orbitz is a travel fare aggregator website and travel metasearch engine.

<https://en.wikipedia.org/wiki/Orbitz>

Graphite collects, stores, and displays time-series data in real time (Dixon, 2015). The tool has three main components:

1. Carbon - a Twisted daemon that listens for time-series data.
2. Whisper - a simple database library for storing time-series data.

Graphite web app - A Django web app that renders graphs on-demand using Cairo library.

[https://en.wikipedia.org/wiki/Graphite_\(software\)](https://en.wikipedia.org/wiki/Graphite_(software))

RRDtool stores data in a circular buffer based database, thus the system storage footprint remains constant over time. RRDtool assumes time-variable data in intervals of a certain length, called steps. When data is not available on time, RRDtool will automatically interpolate submitted data to fit its internal time-steps. The value for a specific step, that has been interpolated, is named a *Primary Data Point - PDP*. Multiple PDPs may be consolidated according to a consolidation function (CF) to form a *Consolidated Data Point - CDP*. Typical consolidation functions are average, minimum, maximum. After the data has been consolidated, the resulting CDP is stored in a *Round-Robin Archive - RRA*, which stores a fixed number of CDPs and specifies how many PDPs should be consolidated into one CDP and which CF to use. The total time covered by an RRA can be calculated as:

time covered = (#CDPs stored) × (#PDPs per CDP) × (step time length).

<https://en.wikipedia.org/wiki/RRDtool>

9.22 Kubernetes and other containers

The problem of an application failing to run correctly when moved from one environment to another typically arise due to differences in configuration underlying library requirements and other dependencies. This problem is addressed by containers which provide a lightweight, immutable infrastructure for application packaging and deployment. An application or service, its dependencies, and its configuration are packaged together as a container image. The containerized application can be tested as a unit and deployed as a container image instance to the host OS. This way, containers enable developers and IT professionals to deploy applications across environments with little or no modification.

Virtual Machines - VMs virtualize the underlying hardware so that multiple OS instances can run on the hardware. VM benefits include the ability to run different operating systems on the same server, more efficient and cost-effective utilization of physical resources, and faster server provisioning. Each VM contains an OS image, libraries, applications, and more, and therefore can become quite large.

A container virtualizes the underlying OS and causes the containerized app to perceive that it has the OS, including CPU, memory, file storage, and network connections, all to itself. Because the differences in underlying OS and infrastructure are abstracted, as long as the base image is consistent. The container can be deployed and run anywhere. Something that is incredibly attractive for developers.

Containers share the host OS and they do not need to boot an OS or load libraries. This enables containers to be much more efficient and lightweight. Containerized applications can start in seconds and many more instances of the application can fit onto the machine as compared to a VM scenario. The shared OS approach has the added benefit of reduced overhead when it comes to patching and updates.

Kubernetes is an open-source container-orchestration system for automating computer application deployment, scaling, and management originally designed by Google and is now maintained by the Cloud Native Computing Foundation.

<https://en.wikipedia.org/wiki/Kubernetes>

Kubernetes vs Docker is discussed below.

<https://containerjournal.com/topics/container-ecosystems/kubernetes-vs-docker-a-primer/>

Though containers are portable, they are constrained by the OS, e.g., a container for Linux can't run on Windows, and vice versa. As noted by Gerald J. Popek (UCLA, rip) at Robert O. Goldberg (rip) in (Popek and Goldberg, 1974) *Virtual Machines* - VM were implemented in a limited number of third generation computer systems such as aforementioned CP-67 on the IBM S/360 Model 67. A book on this topic is (Smith and Nair, 2005).

This extensibility is provided in large part by the Kubernetes API, which is used by internal components as well as extensions and containers that run on Kubernetes. The platform exerts its control over compute and storage resources by defining resources as Objects, which can then be managed as such.

<https://en.wikipedia.org/wiki/Kubernetes>

“An Introduction to Kubernetes” was written by Justin Ellingwood in 2018.

<https://www.digitalocean.com/community/tutorials/an-introduction-to-kubernetes>

Alternatives to Kubernetes in alphabetic order are as follows:

<https://analyticsindiamag.com/9-best-kubernetes-alternatives-for-devops-engineers/>

AWS Fargate is a compute engine that one can use with Amazon *Elastic Container Service* - ECS to run containers without having to manage servers or clusters of Amazon EC2 instances. Fargate removes the need to choose server types, decide when to scale your clusters or optimize cluster packing. It allows a developer to focus on design and build applications instead of managing the infrastructure which runs them.

<https://aws.amazon.com/fargate/>

Apache Mesos is a cluster manager that provides efficient resource isolation and sharing across distributed applications or frameworks. The Mesos kernel runs on every machine and provides applications such as Hadoop, Jenkins, Spark, Aurora, and other frameworks on a dynamically shared pool of nodes.

<http://mesos.apache.org/>

Cloudify is an open-source cloud orchestration framework, which enables a developer to model applications and services and automate the entire life cycle including deployment on any cloud or data center environment, monitoring all aspects of a deployed application, detecting issues and failure, manually or automatically remediating such issues, and performing ongoing maintenance tasks.

cloudify.co

Docker first released by Docker Inc. in 2013 is an open platform for developing, shipping, and running applications, which enables you to separate your applications from your infrastructure so you can deliver software quickly. This platform provides the ability to package and run an application in a loosely isolated environment known as a container. By taking advantage of Docker's methodologies for shipping, testing, and deploying code quickly, one can significantly reduce the delay between writing code and running it in production.

<https://www.docker.com/>

[https://en.wikipedia.org/wiki/Docker_\(software\)](https://en.wikipedia.org/wiki/Docker_(software))

<https://containerjournal.com/topics/container-ecosystems/kubernetes-vs-docker-a-primer/>

IronWorker is a solution for handling your container-based workloads. It is an enterprise-grade background job processing system based on Docker. It enables application developers the ability to asynchronously process workloads without having to worry about scaling infrastructure. It runs on public clouds, private clouds, and on-premise and since the code is executed in Docker containers, tasks can be written in any language.

www.iron.io

Kontena Classic is a developer-friendly, open-source platform for orchestrating applications that are run on Docker containers. It simplifies deploying and running containerized applications on any infrastructure. By leveraging technologies such as Docker, Container Linux, and Weave Net, it provides complete solutions for organizations of any size.

<https://stackshare.io/kontena>

Nomad is a flexible workload orchestrator that enables an organization to easily deploy and manage any containerized or legacy application using a single, unified workflow. It enables developers to use declarative infrastructure-as-code for deploying applications and can run a diverse workload of Docker, non-containerized, microservice, and batch applications. Its key features include deploying containers and legacy applications, device plug-ins and GPU support, federation for multi-region and multi-cloud, among others.

<https://www.nomadproject.io/use-cases/simple-container-orchestration>

OpenShift Container Platform from Red Hat offers a consistent hybrid cloud foundation for building and scaling containerized applications. It is based on Docker-formatted Linux containers, Google Kubernetes orchestration, and *Red Hat Enterprise Linux* - RHEL. It provides developers the ability to accelerate application delivery and includes features such as support for standardized containers through the Red Hat APIs for Docker, container orchestration, scheduling, and management at scale with Kubernetes, distributed application platform with container networking, streamlined

deployment as well as administration, etc.

<https://www.openshift.com/products/container-platform>

Rancher is an open-source platform which implements a purpose-built infrastructure for running containers in production. Rancher takes in raw computing resources from any public or private cloud in the form of Linux hosts. The key product features of Rancher include cross-host networking, container load balancing, persistent storage services, multi-tenancy & user management, resource management, multi-orchestration engines and much more.

<https://rancher.com/>

9.23 Graph databases

A graph is a collection of points (vertices) and connecting lines (edges). Graph databases maintain relationships as a graph (Robinson et al., 2015).

https://en.wikipedia.org/wiki/Graph_database

Table 9.12 provides the key attributes and certain dimensions that can be used for selecting the appropriate NoSQL databases:

1. Storage structure for the data model.
2. Performance of the distributed database: low, medium, and high.
3. Scalability in terms of capacity and processing by adding more nodes to the cluster on three scales.
4. The scale of flexibility of use and the ability to cater to diverse structured or unstructured data and use cases.
5. Complexity of development and modeling, the complexity of operation and maintainability.

TABLE 9.12 Comparison of NoSQL Databases.

Data model (1)	Performance (2)	Scalability (3)	Flexibility (4)	Complexity (5)
Key-value	High	High	High	None
Column-oriented	High	High	Moderate	Low
Document store	High	Variable	High	Low
Graph database	Variable	Variable	High	High

Rankings of graph databases and their revenue are given in Table 9.13.

<https://db-engines.com/en/ranking/graph+dbms>

TABLE 9.13 Ranking of graph databases and their revenue, *Multi-Model - MM*.

6/20	5/20	6/19	Company	Data model	6/20	5/20	6/19
1.	1.	1.	Neo4j	Graph	48.27	−1.49	−1.28
2.	2.	2.	Azure Cosmos DB	MM	30.80	+0.13	+2.56
3.	3.	4.	ArangoDB	MM	5.38	+0.70	+0.81
4.	4.	3.	OrientDB	MM	4.82	+0.68	−0.77
5.	5.	5.	Virtuoso	MM	2.28	−0.07	−0.83

Neo4J is a Java-based open source NoSQL transactional graph database released in 2010. It can be used to search social network data and explore connections between data (Vukotic et al., 2014), (Robinson et al., 2015).

<https://en.wikipedia.org/wiki/Neo4j>

<https://neo4j.com/>

Connections between data are stored – not computed at query time. Cypher is Neo4j’s powerful, graph-optimized query language that understands, and takes advantage of these stored connections.

<https://neo4j.com/developer/cypher/>

When trying to find patterns or insights within data, Cypher queries are often much simpler and easier to write than massive SQL Joins, which are not required since Neo4j does not have tables. For comparison with SQL, here’s a simple Cypher query matching all products in a category hierarchy: while the same command in SQL takes eight lines, with six joins.

```
MATCH (p:Product)-[:CATEGORY]->(l:ProductCategory)-
[:PARENT*0..]->(:ProductCategory name:"Dairy Products")
Return(p.name)
```

CosmosDB from Microsoft, **ArangoDB**, **OrientDB** are discussed in Section 9.20.

Virtuoso Universal Server is discussed in Section 9.8.

Oracle Spatial and Graph Database Melli Annamalai and Jean Ihm have authored “Graph Databases and Analytics: How to Use Them.”

<https://www.oracle.com/a/tech/docs/sg-ooow2019-graph-databases-and-analytics.pdf>

Given three characters Melli, Jean, and John the following relations can exist. Both Melli and John collaborate with Jean, but Melli feuds with John.

https://en.wikipedia.org/wiki/Oracle_Spatial_and_Graph

Graph analytics deal with the following.

1. Community detection and influencer analysis.
2. Churn risk analysis/targeted marketing, HR Turnover analysis.
3. Clustering - Product recommendation.
4. Anomaly detection - Identify fraud.
5. Path analysis and reachability. Manage Bill of Materials, Outage analysis in utilities networks, vulnerability analysis in IP networks.
6. Pattern matching - Tax fraud detection, data extraction.

9.23.1 Categories of graph models

Graph models can be categorized into Property Graph Models and RDF graphs.

Property Graph Model - PGM is used for path and analytics and to detect patterns and anomalies. *Product Overview:*

- **PGQL** is a powerful SQL-like language for specifying graph additions to the SQL standard.

<https://www.dataversity.net/what-is-a-property-graph/>

- Java API for analytics.
- Rich user interface.
- Notebook.
- Shell UI.
- Graph Visualization.
- Enterprise capabilities. - Built on Oracle infrastructure.

PGQL example

```
PATH any_edge as ()-[]- ()
SELECT n, m MATCH(n) -/: any_edge/ -> m
WHERE n.name='Lucy' and m.name='Jean'
```

Property Graph Product Overview

- Store, manage, query and analyze graphs.
 - Highly scalable in-memory analytics.
 - Tens of billions of edges and vertices.
 - Fifty+ pre-built graph analysis algorithms.
- Detecting components and communities - Tarjan's, Kosaraju's, Weakly connected components, label propagation, etc.
 Ranking and walking - Pagerank, personalized pagerank, betweenness centrality, etc. Evaluating community structures - Conductance, modularity, triangle counting, Adamic-Adar, etc. Path-finding Path distance, Dijkstra's, Bellman-Ford's, etc.
 A good discussion of graph algorithms is provided here.

<https://www.geeksforgeeks.org/graph-data-structure-and-algorithms/>

RDF Graph Model is used for data federation, knowledge representation, and the semantic web. RDF databases are described in more detail in Section 9.27.

https://en.wikipedia.org/wiki/Resource_Description_Framework

Using Graphs on a Big Data Platform, as in the case of Oracle Big Data Spatial and Graph Runs on Hadoop Platform

- In-memory parallel graph analytics server (PGX)
- Load graph into memory for analysis
- Automate graph refresh

TABLE 9.14 Recommended memory Size and # of processors vs graph size.

Graph size	Physical memory	Recommended # of CPU Processors
10-100M edges	<=14 GB RAM	2-4/16 processors
100M-1B edges	14-100 GB RAM	4-12/16-32 processors
>1B edges	100 GB RAM	12-32 or more processors

- Client libraries
- Java API to develop applications
- Command-line submission of graph queries
- Graph visualization tool
- APIs to update graph store

The Gartner group suggests the five categories of graph models:

Social graphs deal with connections between people; as in Facebook, Twitter. There is the idea of six degrees of separation.

https://en.wikipedia.org/wiki/Six_degrees_of_separation

Intent graphs deal with reasoning and motivation.

Consumption or payment graphs used in the retail industry and E-commerce companies such as Amazon, eBay and Walmart to track the consumption of individual customers.

Interest graph maps a person's interests and is often complemented by a social graph. It has the potential to follow the previous revolution of web organization by mapping the web by interest rather than indexing webpages.

Mobile graph: this is built from mobile data including data from the web, applications, digital wallets, GPS, and Iot devices.

Recommended physical memory size and number of Processors versus graph size is given in Table 9.14.

Oracle Spatial and Graph Database is described in (Godfrind et al., 2015).

https://en.wikipedia.org/wiki/Oracle_Spatial_and_Graph

AWS Neptune is a graph database with many applications, which is utilized by many companies. A more detailed description is provided in Section 9.27.

As an example of application of graph models-find nodes that are in most shortest paths between other nodes and hence identify nodes that can cause vulnerabilities in a grid.

A recent study on up to 128 AWS machines reported extensive experiments on scale-out performance of eight distributed graph processing systems (Hadoop, HaLoop, Vertica, Giraph, GraphLab, GraphX, Flink Gelly, and Blogel) across four workloads (PageRank, WCC, SPSS, K-hop) over four very large datasets (Twitter, World RoadNetwork, UK 200705, and ClueWeb) (Ammar and Özsu, 2018). Evaluated are a wider set of systems that follow more varied computation models. Results indicate that the best system varies according to workload and particular data graph that is used. Some previous results are confirmed and when reported results are different the reasons for the divergence are explained.

9.24 Object-oriented databases

In object databases information is represented in the form of objects in *Object-Oriented - OO* programming.

https://en.wikipedia.org/wiki/Object_database

OO databases became popular three decades ago and have been the topic of specialized conferences. Early books on this topic are (Kim, 1990), (Bancilhon et al., 1992), (Bertino and Martino, 1993). Object-relational databases are a hybrid of both approaches. A comparison of OO DBMSs is given here:

https://en.wikipedia.org/wiki/Comparison_of_object_database_management_systems

and they are ranked in Table 9.15.

<https://db-engines.com/en/ranking/object+oriented+dbms>

Intersystem Cache' is discussed In Section 9.8.

Versant Object Database - VOD allowed developers using Object-Oriented languages to transactionally store their information by allowing the respective language to act as the DB DDL - Data Definition Language. The memory model is the database schema model. Persistence in VOD is implemented by declaring a list of classes.

https://en.wikipedia.org/wiki/Versant_Object_Database

In 2012 Versant was acquired by Actian, which had formerly acquired Ingres.

TABLE 9.15 OO database rankings and their revenues.

6/20	5/20	6/19	Company	Data model	6/20	5/20	6/19
1.	1.	1.	InterSystems Caché	MM	3.46	+0.36	+0.39
2.	2.	2.	Versant Object DB-VOD	OO	2.01	+0.10	+0.12
3.	4.	4.	ObjectStore	OO	1.59	+0.39	+0.27
4.	3.	3.	Db4o	OO	1.44	+0.08	+0.02
5.	6.	7.	InterSystems IRIS	MM	1.23	+0.18	+0.57
6.	5.	5.	Matisse	OO	1.22	+0.07	+0.04

<https://en.wikipedia.org/wiki/Actian>

Actian was acquired by *Hindustan Computers Limited - HCL* and Sumeru Equity Partners in 2018.

Relational Technology Incorporated - RTI was founded in 1980 by Michael Stonebraker, Eugene Wong, and Lawrence A. Rowe at UCB to commercialize Ingres. RTI was renamed Ingres in 1989 and was later acquired by Actian.

ObjectStore can handle data created by applications that use OO programming techniques, avoiding the Object to Relational Mapping overhead required when using OO data with a Relational Database (Lamb et al., 1991).

<https://en.wikipedia.org/wiki/ObjectStore>

Db4o was an embeddable open-source object database for Java and .NET developers first shipped in 2001 and acquired by Versant Corp. in 2008. It was developed, commercially licensed and supported by Actian until October 2014.

<https://en.wikipedia.org/wiki/Db4o>

InterSystems IRIS based in Cambridge, MA is a vendor of software systems and technology for high-performance DBMS, rapid application development, integration, and healthcare information systems. The vendor's products include IRIS Data Platform, Caché DBMS, the InterSystems Ensemble integration platform, the HealthShare and TrakCare healthcare information system.

<https://en.wikipedia.org/wiki/InterSystems>

Matisse object developers database is described below.

http://www.matisse.com/pdf/product_information/collateral/the_emergence_of_the_object-sql_database.pdf

GT.M is a database engine with scalability proven in the largest real-time core processing systems in production at financial institutions worldwide, as well as in large, well known healthcare institutions, but with a small footprint that scales down to use in small clinics, virtual machines and software appliances. The GT.M data model is a hierarchical associative memory that imposes no restrictions on the data types of the indexes and the content – the application logic can impose any schema, dictionary or data organization suited to its problem domain.

GT.M's compiler for *Massachusetts Univ. Hospital Utility Multiprogramming System - MUMPS* scripting language

<https://en.wikipedia.org/wiki/MUMPS>

implements full support for ACID txns using optimistic concurrency control (Ramakrishnan and Gehrke, 2002) and *Software Transactional Memory - STM* that resolves the common mismatch between databases and programming languages.

<https://sourceforge.net/projects/fis-gtm/>

9.25 Search engines for text

Search engines for text rankings are listed in Table 9.16.

<https://db-engines.com/en/ranking/search+engine>

Elasticsearch is developed in Java an open-source search server based on Apache Lucene,

https://en.wikipedia.org/wiki/Apache_Lucene

which provides a distributed, multitenant-capable full-text search engine with a RESTful web interface and schema-free JSON documents.

<https://www.elastic.com/>

Splunk starting as a log analysis tool has expanded to include data analytics for monitoring txns. Splunk's Big Data solutions include Splunk Analytics for Hadoop to do the analytics, the Splunk ODBC Driver for connecting to enterprise applications like Tableau, and Splunk DB Connect for connecting to a variety of data sources. Splunk is considered to be cloud monitoring leader.

<https://www.splunk.com/>

TABLE 9.16 Search engine - SE, Multi-Model-MM, Resource Description Framework.

6/20	5/20	6/19	Company	Data model	6/20	5/20	6/19
1.	1.	1.	Elasticsearch	SE,MM	149.69	+0.56	+0.86
2.	2.	2.	Splunk	SE	88.08	+0.33	+3.46
3.	3.	3.	Solr	SE	51.26	-1.32	-9.22
4.	4.	4.	MarkLogic	MM/RDF	11.24	+0.28	-2.26
5.	5.	5.	Sphinx	SE	6.35	+0.33	+0.03
6.	5.	6.	Azure Search	SE	6.30	+0.18	+0.44
7.	7.	7.	ArangoDB	MM	5.38	+0.70	+0.81
8.	8.	8.	Algolia	SE	4.78	+0.32	+0.40
9.	10.	11.	Amazon Cloud	SE	2.54	-0.05	+0.00
10.	11.	9.	Virtuoso	MM	2.28	-0.07	-0.83

Tableau products query relational databases, online analytical processing cubes, cloud databases, and spreadsheets to generate graph-type data visualizations. The products can also extract, store, and retrieve data from an in-memory data engine. Salesforce acquired Tableau in 2019.

https://en.wikipedia.org/wiki/Tableau_Software

Salesforce was cofounded in 1999 by former Oracle executive Marc Benioff as a *Software as a Service* - *SaaS* cloud-based company in San Francisco.

https://en.wikipedia.org/wiki/Software_as_a_service

It provides *Customer Relationship Management* - *CRM* service and also sells a complementary suite of enterprise applications focused on customer service, marketing automation, analytics, and application development.

<https://en.wikipedia.org/wiki/Salesforce>

[salesforce.com](https://www.salesforce.com)

SaaS has been criticized by Richard Stallman of the *Free Software Foundation* - *FSF* who considers *SaaS* a violation of the principles of free software and that the users do not have even the executable file that does their computing and that computing is done at someone else's server not accessible by the user, so that it is impossible for them to ascertain what it really does, and impossible to change it.

https://en.wikipedia.org/wiki/Richard_Stallman

Solr is an open-source enterprise-search platform, written in Java, from the Apache Lucene project. Its major features include full-text search, hit highlighting, faceted search, real-time indexing, dynamic clustering, database integration, NoSQL features and rich document handling.

https://en.wikipedia.org/wiki/Apache_Solr

Sphinx can be used either as a stand-alone server or as a *Search Engine* - *SE* SphinxSE for the MySQL family of databases. When run as a standalone server Sphinx operates similar to a DBMS and can communicate with MySQL, MariaDB and PostgreSQL through their native protocols or with any ODBC-compliant DBMS via ODBC. MariaDB, a fork of MySQL, is distributed with SphinxSE.

[https://en.wikipedia.org/wiki/Sphinx_\(search_engine\)](https://en.wikipedia.org/wiki/Sphinx_(search_engine))

Azure Cognitive Search is the only cloud search service with built-in AI capabilities that enrich all types of information to easily identify and explore relevant content at scale. It uses Microsoft's natural language stack that Bing and Office and AI services across vision, language, and speech. Allows user to spend more time innovating and less time maintaining a complex cloud search solution.

https://en.wikipedia.org/wiki/Azure_Search

ArangoDB is discussed in Section 9.23.

Algolia's model provides search as a service, offering web search across a client's website using an externally hosted search engine. Although in-site search has long been available from general web search providers such as Google, this is typically done as a subset of general web searching. The search engine crawls or spiders the web at large, including the client site, and then offers search features restricted to only that target site. This is a large and complex task, available only to large organizations at the scale of Google or Microsoft. Algolia's product only indexes their clients' sites and so the search task is far simpler. Data for the client site is pushed from the client to Algolia via a RESTful JSON API (Leonard Richardson, Mike Amundsen (2013), *RESTful Web APIs*, O'Reilly), then the search box is added

simply to the client's web pages.

This search model is intended to give the performance and sophistication advantages of a full in-house search engine operating on the native web site back-end database, but with the simplicity of setup of using a site-restricted Google search.

<https://en.wikipedia.org/wiki/Algolia>

<https://www.algolia.com/>

A blog has been written by Algolia CTO Julien Lemoine.

<https://stories.algolia.com/algolia-s-fury-road-to-a-worldwide-api-c1536c46f3a5>

<http://highscalability.com/blog/2015/3/9/the-architecture-of-algolia-s-distributed-search-network.html>

Amazon CloudSearch is a scalable cloud-based search service that forms part of **Amazon Web Services - AWS**. CloudSearch is typically used to integrate customized search capabilities into other applications supporting faceted searching, free text searching, text processing, Boolean expressions, customized relevance ranking, field-based sorting, and other advanced functions. The service is made available as a *pay-as-you go* - *payg* pricing plan.

<https://searchaws.techtarget.com/definition/Amazon-CloudSearch>

Virtuoso DB was covered in Section 9.8.

9.26 Web search engines

A list of web search engines which are defined below is as follows.

https://en.wikipedia.org/wiki/Web_search_engine

Google has 75% of the search market share and charges most for advertising. Created as a research project at Stanford U. in 1996 by students Sergey Brin and Larry Page.

<https://en.wikipedia.org/wiki/Google>

Google is very profitable with \$124M per day profit.

https://www.theregister.com/2020/10/30/alphabet_quarterly_results/

YouTube was founded in 2005 by veterans of PayPal and was purchased over a year later by Google. YouTube receives more than 1.5 billion logged in users per month and feeds over 1 billion hours of video each day to users.

<https://en.wikipedia.org/wiki/YouTube>

Amazon was launched in 1995 by Jeff Bezos who retired on 7/4/21. Bezos set up a web page to sell books from his garage. It is now the largest company selling all kinds of goods online. Last year more than half of all online shopping searches began not at Google, but at Amazon, which has acquired Whole Foods. *Amazon Web Services - AWS* is a very profitable operation.

AWS revenue \$46B (2020), while Operating income is \$13.5B.

Amazon revenue \$386B (2020), while Operating income is \$22.9B, and Net income is \$21.33 B.

IBM revenue is \$73.6B (2020), Operating income \$13.21B, Net income \$5.53B.

https://en.wikipedia.org/wiki/Amazon_Web_Services

[https://en.wikipedia.org/wiki/Amazon_\(company\)](https://en.wikipedia.org/wiki/Amazon_(company))

<https://en.wikipedia.org/wiki/IBM>

Facebook was founded by Mark Zuckerberg and Harvard U. students. It became available to students in 2004 and to the world in 2006. With more than 1.5 billion logged in visitors per month, Facebook also gives businesses and advertisers incredible market access, and tends to be where people are when they are not working.

<https://en.wikipedia.org/wiki/Facebook>

Microsoft Bing replaced MSN Search as Microsoft's answer to Google in 2009. It provides power for Yahoo search and AOL and now handles 33% of all searches.

https://en.wikipedia.org/wiki/Microsoft_Bing

Baidu founded in 2000 is the dominant search engine in China with over 82% market share where Google comes in at 0.61% and Bing at 0.37%. Baidu powers 3.3 billion searches per day. [Baidu.com](https://www.baidu.com) is in Chinese.

<https://en.wikipedia.org/wiki/Baidu>

Yandex has its roots in a Russian project to aid in the classification of patents in 1990 by Arkadia. The term Yandex was adopted in 1993 standing for *Yet Another iNDEXer* - Yandex was adopted in 1993 leading to [Yandex.ru](https://yandex.ru) in 1997. In 2011 Yandex went public on the *New York Stock Exchange - NYSE* with an *Initial Program Offering - IPO* of \$1.3 billion making it the second largest at the time. Yandex currently powers more than half of all searches in Russia. By comparison the largest IPOs in the US have been as follow. newline <https://www.statista.com/statistics/226152/largest-ipos-in-the-united-states-by-issue-volume/>

9.27 Resource Description Framework - RDF

The RDF data model is similar to classical conceptual modeling approaches such as *Entity-Relationship - ER*. It makes statements about web resources in expressions of the form of triples: *subject-predicate-object*. The subject denotes the resource, and the predicate denotes aspects of the resource, and expresses a relationship between the subject and the object.
https://en.wikipedia.org/wiki/Resource_Description_Framework

RDF has several serialization/file formats, so the particular encoding for resources or triples varies from format to format. This mechanism for describing resources is a major component in the *World Wide Web Consortium - W3C*'s Semantic Web activity: an evolutionary stage of the *World Wide Web - WWW* in which automated software can store, exchange, and use machine-readable information distributed throughout the Web, in turn enabling users to deal with the information with greater efficiency and certainty. RDF's simple data model and ability to model disparate, abstract concepts have also led to its increasing use in knowledge management applications unrelated to Semantic Web activity.

The Semantic Web is an extension of the WWW through standards set by the W3C. The goal of the Semantic Web is to make Internet data machine-readable (Pollock, 2009).

https://en.wikipedia.org/wiki/Semantic_Web

A collection of RDF statements intrinsically represents a labeled, directed multi-graph, which in theory makes an RDF data model better suited to certain kinds of knowledge representation than are other relational or ontological models. However, in practice, RDF data is often stored in relational database or native representations (Triple- or Quad-stores).

The predominant query language for RDF graphs is SPARQL.

<https://en.wikipedia.org/wiki/SPARQL>

An example of a SPARQL query to show country capitals in Africa is as follows:

```
PREFIX ex: <http://example.com/exampleOntology#>
SELECT ?capital ?country
WHERE {
  ?x ex:city ?capital ;
      ex:isCapitalOf ?y .
  ?y ex:country ?country ;
      ex:isInContinent ex:Africa .
}
```

RDF stores by ranking are listed in Table 9.17.

<https://db-engines.com/en/ranking/rdf+store>

TABLE 9.17 Ranking of Resource Description Framework - RDF stores.

6/20	5/20	6/19	Company	Data model	6/20	5/20	6/19
1.	1.	1.	MarkLogic	MM	11.24	+0.28	-2.26
2.	2.	3.	Apache Jena	RDF	2.69	-0.10	+0.42
3.	3.	2.	Virtuoso	MM	2.28	-0.07	-0.83
4.	4.	4.	Amazon Neptune	MM	2.17	+0.41	+0.93
5.	5.	5.	GraphDB	MM	1.25	+0.06	+0.16
6.	6.	7.	Stardog	MM	1.15	+0.08	+0.44
7.	7.	6.	AllegroGraph	MM	0.88	+0.04	-0.04
8.	8.	8.	Blazegraph	MM	0.67	+0.03	+0.11

A list of semantic web frameworks is as follow:

Marklogic is discussed in Section 9.20.

Apache Jena is an open source Semantic Web framework for Java, which provides an API to extract data from and write to RDF graphs represented as an abstract model, which can be sourced with data from files, databases, URLs or a combination of these. Jena is similar to RDF4J but unlike it Jena provides support for OWL (Web Ontology Language).

https://en.wikipedia.org/wiki/Apache_Jena

https://en.wikipedia.org/wiki/Web_Ontology_Language

The framework has various internal reasoners and the Pellet reasoner that can be set up to work in Jena.

<https://archive.org/details/probabilisticlog00goer>

Virtuoso Universal Server discussed in Section 9.8.

Neptune from Amazon is a managed graph database product, which used as a web service and is part of AWS. Neptune supports popular graph models property graph, W3C's RDF, and query languages Apache TinkerPop Gremlin and SPARQL, including other AWS products. Neptune is *Health Insurance Portability and Accountability Act - HIPAA* eligible and compatible for Payment Card Industry Data Security Standard, and *International Organization for Standardization - ISO*. The data is stored in a cluster volume, a virtual volume utilizing SSD drives. The sizes of these volumes are dynamic, they increase depending on how much data is stored in the database, with a maximum of 64 TB. The Amazon Neptune *Service Level Agreement - SLA* policy is designed to offer a monthly uptime 99.9%, increasing database performance and availability by integrating the engine with a virtual storage based on SSD drives, that are specially made for database workloads. Neptune maintains copies of the user's data in multiple availability zones. In case of failures, Neptune automatically detects any failed segments in a disk volume and repairs them.

https://en.wikipedia.org/wiki/Amazon_Neptune

<https://aws.amazon.com/neptune/>

GraphDB was developed by a German company was last released in 2011. Its last version was released in May 2011 and the company went bankrupt on 1/1/2012.

Stardog turns users data into machine-understandable knowledge. Data separates from knowledge in the following categories:

situational: meaning alters depending on circumstances.

layered: associations between concepts allow for nuanced understanding.

changing: new discoveries instantly change meanings.

These facets of knowledge represent the context that is often missing from data. When traditional data management systems attempt to capture context they fail. These failures generate gridlock over mastering data, delay timelines when adding new data sources or properties, and cause missing data from analyses that lead to mistrust. Knowledge Graphs are then built for the fluctuating nature of knowledge. They offer a more flexible foundation for digital operations by easily accepting new data, new definitions and requirements. Knowledge graphs are there because being data-driven is not enough.

<https://www.stardog.com/>

Allegrograph not only stores RDF triples, a standard format for Linked Data, but it also operates as a document store designed for storing, retrieving and managing document-oriented information, in JSON-LD format.

<https://en.wikipedia.org/wiki/AllegroGraph>

Blazegraph is a triplestore and graph database, which is used in the Wikidata SPARQL endpoint.

<https://en.wikipedia.org/wiki/Blazegraph>

Wikidata is a free and open knowledge base that can be read and edited by both humans and machines. Wikidata acts as central storage for the structured data of its Wikimedia sister projects including Wikipedia, Wikivoyage, Wiktionary, Wikisource, and others.

<https://en.wikipedia.org/wiki/Wikidata>

<https://www.wikidata.org>

9.28 Wide column stores

A wide-column store (or extensible record store) is a type of NoSQL database. It uses tables, rows, and columns, but unlike a relational database, the names and format of the columns can vary from row to row in the same table. A wide-column store can be interpreted as a two-dimensional key-value store.

[https://en.wikipedia.org/wiki/Column_\(data_store\)](https://en.wikipedia.org/wiki/Column_(data_store))

Column stores are the exact opposite of row-based databases and are highly scalable offering very high performance. Examples of column stores are HBase and Hypertable.

https://en.wikipedia.org/wiki/Apache_HBase

<https://en.wikipedia.org/wiki/Hypertable>

Given a rowkey in Hbase one may have multiple versions of columns with associated data values.

<http://www.informit.com/articles/article.aspx?p=2253412>

<https://www.oreilly.com/library/view/hbase-the-definitive/9781449314682/ch01.html>

Cassandra, Scylla, and HBase modeled after Google's BigTable are examples of a wide-column DBMSs, which are schema agnostic and enable users to store data in column families or tables, a single row of which can be thought of as a

record – a multi-dimensional KV store.

https://en.wikipedia.org/wiki/Apache_Cassandra

[https://en.wikipedia.org/wiki/Scylla_\(database\)](https://en.wikipedia.org/wiki/Scylla_(database))

Column stores are described by Stonebraker et al. in (Stonebraker and Çetintemel, 2005) and benchmarked in (Stonebraker et al., 2007a). Vertica founded in 2005 was the direct result of commercializing the C-Store research system (Stonebraker et al., 2005) by its researchers and was acquired in 2011 by HP.

<https://www.vertica.com/>

Presented is the design of a read-optimized relational DBMS that contrasts sharply with most current systems, which are write-optimized. Among the many differences in its design are: storage of data by column rather than by row, careful coding and packing of objects into storage including main memory during query processing, storing an overlapping collection of column-oriented projections, rather than the current fare of tables and indexes, a non-traditional implementation of txns, which includes high availability and snapshot isolation for read-only txns, and the extensive use of bitmap indexes to complement B+-tree structures. Performance data on a subset of TPC-H shows that C-Store is substantially faster than popular commercial products.

Wide-column stores are designed with the goal of scaling well enough to manage petabytes of data across as many as thousands of commodity servers in a massive, distributed system. Although schema-free, wide column stores like Scylla and Cassandra use an SQL variant called CQL for data definition and manipulation, making it easier to those already familiar with RDBMS.

Wide-column stores rankings are given in Table 9.18.

<https://db-engines.com/en/ranking/wide+column+store>

TABLE 9.18 *Wide Column - WC store rankings.*

6/20	5/20	6/19	Company	Data model	6/20	5/20	6/19
1.	1.	1.	Cassandra	WC	119.01	−0.15	−6.17
2.	2.	2.	HBase	WC	48.73	−0.99	−9.30
3.	3.	3.	Azure Cosmos DB	MM	30.80	+0.13	+2.56
4.	4.	4.	Datastax Enterprise	WC, MM	8.36	+0.74	−0.54
5.	5.	5.	Azure Table Storage	WC	5.27	+0.59	+0.73
6.	6.	6.	Accumulo	WC	3.85	+0.18	−0.26
7.	7.	7.	Google Cloud Bigtable	WC	2.93	+0.75	+1.06
8.	8.	8.	ScyllaDB	MM	2.71	+0.81	+1.29
9.	9.	9.	MapR-DB	MM	0.63	+0.03	−0.06
10.	10.	11.	Alibaba Table Store	WC	0.36	+0.04	+0.14

Cassandra from Apache is a free and open-source, distributed, wide column store, NoSQL DBMS designed to handle large amounts of data across many commodity servers, providing high availability with no single point of failure. Cassandra offers robust support for clusters spanning multiple datacenters, with asynchronous masterless replication allowing low latency operations for all clients. Cassandra offers the distribution design of Amazon Dynamo with the data model of Google’s Bigtable (Chang et al., 2008).

https://en.wikipedia.org/wiki/Apache_Cassandra

HBase from Apache is an open-source, distributed, versioned, non-relational database modeled after Google’s Bigtable. Hbase is suited for random, real-time read/write access to Big Data with the goal of hosting very large tables with billions of rows and millions of columns on top of clusters of commodity hardware. Just as Bigtable leverages the distributed data storage provided by the *Google File System - GFS*, Apache HBase provides Bigtable-like capabilities on top of Hadoop and HDFS.

<https://hbase.apache.org/>

https://en.wikipedia.org/wiki/Apache_HBase

Azure Cosmos DB is discussed in Section 9.19.

Datastax Enterprise is a storage application which uses Apache Cassandra. The company also produced a proprietary version of Cassandra named *DataStax Enterprise - DSE*. In December 2018, DataStax released DSE 6.7 with five key new feature upgrades, including: improved analytics, geospatial search, improved data protection in the cloud,

enhanced performance insights and new developer integration tools with Apache Kafka Connector and certified production Docker images.

<https://en.wikipedia.org/wiki/DataStax>

Azure Table Storage stores structured NoSQL data in the cloud, providing a key/attribute store with a schemaless design and because of this it is easy to adapt your data as the needs of your application evolve. Azure tables are ideal for storing structured, non-relational data. Common uses of Azure Table storage include:

<https://docs.microsoft.com/en-us/azure/storage/tables/table-storage-overview>

- Storing terabytes of structured data capable of serving web scale applications.
- Storing datasets that do not require complex joins, foreign keys, or stored procedures and can be denormalized for fast access.
- Quickly querying data using a clustered index.
- Accessing data using the Data protocol and LINQ queries with *Windows Communication Function - WCF Data Service .NET Libraries*.

<https://docs.microsoft.com/en-us/dotnet/csharp/programming-guide/concepts/linq/introduction-to-linq-queries>

WCF is a framework for building service-oriented applications. The user can use WCF to send data as asynchronous messages from one service endpoint to another. A service endpoint can be part of a continuously available service hosted by *Internet Information Service - IIS*,

https://en.wikipedia.org/wiki/Internet_Information_Services

or it can be a service hosted in an application. IIS is a file and Web application server that can be used on a *Local Area Network - LAN*, a wide area network (WAN), or the Internet. WCF Data Services is Microsoft's platform which combines runtime and web service through which the services such as ASP.NET are exposed.

https://en.wikipedia.org/wiki/WCF_Data_Services

Accumulo was developed by *National Security Agency - NSA* and contributed to Apache. It is a highly scalable sorted, distributed KV store based on Google's Bigtable. It is a system built on top of Apache Hadoop, ZooKeeper, and Thrift. Written in Java, Accumulo has cell-level access labels and server-side programming mechanisms.

https://en.wikipedia.org/wiki/Apache_Accumulo

Accumulo has the following features.

Column Visibility Apache Accumulo extends the Bigtable data model with new element to the key called Column Visibility. This element stores a logical combination of security labels that must be satisfied at query time in order for the key and value to be returned as part of a user request. This allows data of varying security requirements to be stored in the same table and allows users to see only those keys and values for which they are authorized.

Server-side Programing mechanism called Iterators allows users to perform additional processing at the Tablet Server. The range of operations that can be applied is equivalent to those that can be implemented within a MapReduce Combiner function, which produces an aggregate value for several key-value pairs.

User Key Ordering Apache Accumulo orders entries in order of user keys, and exposes an iterator over a key range. This allows locality of reference not available from some other distributed stores (Cassandra and Voldemort order by hash of the user key).

Google Cloud BigTable is a compressed, high performance, proprietary data storage system built on *Google File System - GFS*, Chubby Lock Service (Burrows, 2006), SSTable (log-structured storage like LevelDB), etc. A public version of Bigtable was made available as a service in 2015 (Chang et al., 2008).

<https://en.wikipedia.org/wiki/Bigtable>

Scylla is an open-source distributed NoSQL column-oriented data store, which was designed to be compatible with Apache Cassandra, while achieving significantly higher throughputs and lower latencies.

[https://en.wikipedia.org/wiki/Scylla_\(database\)](https://en.wikipedia.org/wiki/Scylla_(database))

ScyllaDB and Samsung NVMe SSDs accelerate NoSQL database performance, according to a 2017 Samsung report. Scylla outperformed Cassandra on a cluster of 24-core machines by a margin of 10–37X depending on the *Yahoo! Cloud Serving Benchmark - YCSB*.

<https://en.wikipedia.org/wiki/YCSB>

MapR DB is a high-performance NoSQL DBMS system built into the MapR Data Platform. It is a highly scalable multi-model database that brings together operations and analytics as well as real-time streaming and database workloads to enable a broader set of next-generation data-intensive applications in organizations.

<https://mapr.com/products/mapr-database/>

An in-depth look at MapR database compared to Cassandra, HBase is as follows.

<https://mapr.com/blog/database-comparison-an-in-depth-look-at-mapr-db/>

Alibaba Tablestore is a distributed NoSQL data storage service built on Alibaba Cloud's Apsara distributed computing system, which comes with 99.99% high availability and 99.999,999,999% data reliability.

https://en.wikipedia.org/wiki/Reliability,_availability_and_serviceability

It is an example how large corporations have used the previously disfavored *Roll Your Own* - RYO paradigm. Tablestore enables seamless expansion of data size and access concurrency through data sharding and server load balancer technologies, providing storage of and real-time access to massive structured data.

<https://www.alibabacloud.com/product/table-store>

9.29 Multivalue databases

MultiValue databases are a type of NoSQL and multidimensional databases, which differs from a relational database in that they have features that support and encourage the use of attributes, which can take a list of values, rather than all attributes being single valued.

<https://en.wikipedia.org/wiki/MultiValue>

Rankings of multivalue databases are given in Table 9.19.

<https://db-engines.com/en/ranking/multivalue+dbms>

TABLE 9.19 Multivalue DBMS rankings.							
6/20	5/20	6/19	Company	Data model	6/20	5/20	6/19
1.	1.	1.	Adabas	Multivalue	4.65	+0.52	+0.40
2.	2.	2.	UniData, UniVerse	Multivalue	3.85	+0.48	+0.71
3.	3.	3.	jBASE	Multivalue	2.22	+0.38	+0.61
4.	4.	3.	D3	Multivalue	1.43	0.04	+0.49
5.	5.	4.	Model 204	Multivalue	1.08	+0.10	+0.11

Adabas is a contraction of “Adaptable Database System”. It is an inverted index database developed by Software AG, Darmstadt, Germany to run on IBM mainframes, but as of 2019 it is marketed for use on a wider range of platforms.

<https://en.wikipedia.org/wiki/ADABAS>

IBM *z Integrated Information Processor* - zIIP is a dedicated processor designed to operate asynchronously with general purpose processors to assist with several types of analytics, systems monitoring and other applications. The DB2 *Universal DB* - UDB for z/OS was the first product exploiting zIIP processors, not limited to just DB2 or IBM products.

https://en.wikipedia.org/wiki/Db2_Database

IBM Software offerings that can exploit IBM zIIPs are listed here.

<https://www.ibm.com/downloads/cas/6VYXGE34>

UniData/UniVerse are part of Rocket U2 suite of DBMS software. Both of these products are operating environments, which run on current Unix, Linux and Windows operating systems.

Rocket Software is a privately held software development firm founded in 1990, it offers solutions on the IBM Z, IBM Power, and embedded database platforms. Rocket allow users to use predictive analytics with deep data, develop AI and machine learning capabilities, and design mobile and browser applications. Rocket serves markets including the financial, banking, health care, government, insurance, aerospace, auto manufacturing, and retail industries. Rocket has a business partnership with IBM that began in 1994 with a licensing agreement for Rocket QMF tools. The company is headquartered in Waltham, MA.

https://en.wikipedia.org/wiki/Rocket_Software

Rocket Software solutions for IBM z-systems are listed here

https://www.rocketsoftware.com/sites/default/files/resource_files/pc_ibm_zsystems_letter_march2018_final.pdf

jBASE is a DBMS comprising development tools, middleware and a multi-dimensional database. jBASE takes the best points of the relational database model and adds to it several significant benefits including ease of use, superb performance, small footprint and all the rich MultiValue features, making it ideally suited to all business uses from the Internet to OLAP to transactional applications. The architecture is uniquely designed to allow all and any application

development tools and backend databases to form part of a jBASE solution. jBC compiler generates C-code for platforms, excellent support for external databases like Oracle, SQL Server, DB2 etc.

<https://www.pickwiki.com/index.php/JBase>

Rocket D3 DBMS is part of a powerful MultiValue development environment suited for today's business-critical transactional and analytical database applications. It provides scalability, flexibility, seamless interoperability, connectivity, and low system management cost.

<https://www.rocketsoftware.com/products/rocket-d3>

In August, 2007 Rocket software expanded zIIP eligibility to include Adabas, CICS, IMS, IDMS and VSAM.

https://en.wikipedia.org/wiki/Rocket_Software

<https://www.rocketsoftware.com/> Jnewline Other *Independent Software Vendors - ISVs* have introduced support for execution of their products on zIIPs. IBM's zIIP processors permit direct, real-time SQL access to Adabas even though the data may still be stored in a non-relational form.

https://www.rocketsoftware.com/sites/default/files/resource_files/WP_Data_Adabas-Data-Virtualization.pdf

Model 204 - M204 DBMS developed and commercialized by *Computer Corp. of America - CCA* in 1972 runs on IBM and compatible mainframe. It incorporates a programming language and an environment for application development. It is Implemented in assembly language for IBM System/360 and its successors, M204 can deal with very large databases https://en.wikipedia.org/wiki/Computer_Corporation_of_America and txn loads of 1000 TPS. CCA was acquired in 2010 by Rocket software.

9.30 Native XML databases

An XML database allows data to be specified and sometimes stored in XML format (Ramakrishnan and Gehrke, 2002). This data can be queried, transformed, exported and returned to a calling system. XML databases are a form of document-oriented databases, which are in turn a category of NoSQL database.

https://en.wikipedia.org/wiki/XML_database

List of native XML databases according to ranking is given in Table 9.20.

<https://db-engines.com/en/ranking/native+xml+dbms>

TABLE 9.20 Native XML databases.

6/20	5/20	6/19	Company	Data model	6/20	5/20	6/19
1.	1.	1.	MarkLogic	Multi-model	11.24	+0.28	-2.26
2.	2.	3.	Oracle-Berkeley DB	Multi-model	3.20	+0.47	+0.23
3.	3.	2.	Virtuoso	Multi-model	2.28	-0.07	-0.83
4.	4.	4.	BaseX	Native XML	1.68	+0.52	+0.28
5.	5.	5.	Sedna	Native XML	0.98	+0.10	+0.08

Marklogic is discussed in Section 9.20.

Berkeley DB is a software library intended to provide a high-performance embedded database for key/value data. Berkeley DB is written in C with API bindings for C++, C#, Java, Perl, PHP, Python, Ruby, Smalltalk, and Tcl.

<https://en.wikipedia.org/wiki/Smalltalk>

Tcl is a multi-paradigm: event-driven, functional, imperative, object-oriented language designed by John Ousterhout (Stanford) who was earlier at *UC Berkeley - UCB*.

<https://en.wikipedia.org/wiki/Tcl>

Berkeley DB originated as part of *Berkeley Software Distribution - BSD*, which was Berkeley's version of AT&T's Unix OS.

https://en.wikipedia.org/wiki/Berkeley_Software_Distribution

After 4.3BSD (1986), the BSD developers attempted to remove or replace all code originating in the original AT&T Unix from which BSD was derived, which required rewriting the Unix database package. Berkeley DB was first released in 1991 and later included with 4.4 BSD. In 1996 Netscape requested that the authors of Berkeley DB improve and extend the library to suit Netscape's requirements for a *Lightweight Directory Access Protocol - LDAP* server.

https://en.wikipedia.org/wiki/Lightweight_Directory_Access_Protocol

and for use in the Netscape browser. That request led to the creation of Sleepycat Software led by Margo Seltzer (Harvard U., British Columbia U.), which was acquired by Oracle Corporation in February 2006.

Virtuoso is discussed in Section 9.8.

BaseX is a native and light-weight XML DBMS and XQuery processor, developed as a GitHub community project. It is specialized in storing, querying, and visualizing large XML documents and collections. BaseX uses a tabular representation of XML tree structures to store XML documents. The XPath Accelerator encoding scheme and Staircase Join Operator have been taken as inspiration for speeding up XPath location steps. Additionally, BaseX provides several types of indices to improve the performance of path operations, attribute lookups, text comparisons and full-text searches.

<https://en.wikipedia.org/wiki/BaseX>

Sedna is an open-source DBMS that provides native storage for XML data. The distinctive design decisions employed in for XML data and memory management based on layered address space.

[https://en.wikipedia.org/wiki/Sedna_\(database\)](https://en.wikipedia.org/wiki/Sedna_(database))

Sedna is implemented from scratch at Russian Academy of Sciences in Moscow for storing large amounts of XML data. The key contribution of this system is an improved schema-based clustering storage strategy efficient for both XML querying and updating, and powered by a novel memory management technique (Taranov et al., 2010).

[https://en.wikipedia.org/wiki/Sedna_\(database\)](https://en.wikipedia.org/wiki/Sedna_(database))

9.31 Realtime stream processing

Real-time stream processing is executed on the most current data. The two operatives here are velocity and latency, where Hadoop and related distributed batch processing systems fall short. In many cases there is a need for accurate results in a fraction of seconds, such as credit card fraud, monitoring business activity, and so on, *Complex Event Processing - CEP* is discussed by David Luckham in (Luckham, 2002, 2011) and (Etzion and Niblett, 2011).

https://en.wikipedia.org/wiki/Complex_event_processing

Apache Storm was a brainchild of Nathan Marz at Twitter, which was adopted by Apache and then *Cloudera Distributed Hadoop - CDH*, *Hortonworks Data Platform - HDP*, and so on. It is a highly scalable, distributed, fast, reliable real-time computing system designed to process high-velocity data. Cassandra complements the compute capability by providing fast reads and writes, and this is the best combination available for a data store with Storm. It helps developers to create a data flow model in which tuples flow continuously through a collection of processing components referred to as a topology. Data can be ingested to Storm using distributed messaging queues such as Kafka, RabbitMQ, and so on. Trident is another layer of abstraction API over Storm that brings microbatching capabilities into it.

<https://storm.apache.org/>

Cell-phones have evolved from phones to smartphones, the data generated is not just call data, but also *Call Data Record - CDR*, which captures voice, data, and *Short Message Service - SMS*. Voice and SMS are predominantly structured as they are because of telecom protocols worldwide:

Cellular Inter-carrier Exchange Roamer - CIBER,

http://www.cdg.org/news/events/CDMASeminar/04_LatinAm/040526Joe%20Kammerer.pdf

Short Message Peer-to-Peer - SMPP,

https://en.wikipedia.org/wiki/Short_Message_Peer-to-Peer

Short Message Service Center - SMSC,

https://en.wikipedia.org/wiki/Short_Message_service_center

The data or IP traffic flowing in/out of these smart devices is unstructured and high volume. It could be a music track, a picture, a tweet, or just about anything. CDR processing and billing is generally a batch job, but other things are real-time:

1. Geo-tracking of the device, e.g., users get an SMS as soon as a state border is crossed.
2. Usage and alerts that inform about the broadband consumption limit.
3. Prepaid mobile cards provide an efficient charge-tracking system.

9.32 Event stores

Event stores are optimized for storage of events. Conceptually, in an event store, only the events of a dossier or policy are stored and the instantiation of all other objects in memory can be derived from these events. In an event store objects are instantiated “on the fly” based on the events.

https://en.wikipedia.org/wiki/Event_store

That each event has a double timeline enables event stores to correct errors of events that have been entered into the event store database before.

1. Valid date is the date at which the event occurred.
2. Txn date is the date at which the event is entered into the database.

Once stored events cannot be changed anymore, even if they are erroneous and the only way to change/correct these events is to instantiate a new event with the new values and using the double timeline. A correcting event would have the new values of the original event, with an event data of that corrected event, but a different txn date. This mechanism ensures reproducibility at each moment in the time, even in the time period before the correction has taken place. One advantage of the event store concept is that handling the effects of back dated events is much easier.

An event store will simplify the code in that rolling back erroneous situations and rolling up the new correct situations is not needed anymore. Disadvantage may be that the code needs to reinstantiate all objects in memory based on the events each time a service call is received for a specific dossier. Rankings of event-stores are given in Table 9.21.

<https://db-engines.com/en/ranking/event+store>

TABLE 9.21 Event stores.

6/20	5/20	6/19	Company	Data model	6/20	5/20	6/19
1.	1.	1.	Event Store	Event	0.68	+0.03	+0.06
2.	2.	2.	IBM Db2 ES	Multi-model	0.28	−0.01	+0.04
3.	3.	3.	NEventStore	Event	0.09	+0.01	−0.08

EventStoreDB is an industrial-strength Event Sourcing database that stores critical data in streams of immutable events. It was built from the ground up making it the best solution for building event-sourced systems.

<https://eventstore.com/>

Advantages of EventStore are as follows.

Audit: EventStoreDB stores your data as a series of immutable events over time, providing one of the strongest audit log options available.

Time travel: All state changes are kept, so it is possible to move systems backward and forward in time, which is valuable for debugging and “what if” analysis.

Root cause analysis: Business events can be tied back to originating events providing traceability and visibility of entire workflows from start to finish.

Event-driven architecture - realtime streams: Unlike traditional databases which optimize pooling data into opaque silos, event streams can be listened to, allowing businesses to react in real-time to events.

Service autonomy: If a service goes down dependent services can “catch up” when the source comes back up, because events are stored in a sequence in the stream, knowing the current position allows synchronization.

Replay and reshape: The series of changes (events) in a stream can be replayed and transformed as new business requirements inevitably emerge, e.g., the event stream can be replayed to a point in time and “what if” analysis can be used to project potential future outcomes.

Legacy migration: to modern distributed architectures can be carried out in an incremental manner, gradually replacing specific pieces of functionality with event-sourced services (strangler pattern) existing read paths of the legacy system can remain in place, while writes are directed to the services.

<https://docs.microsoft.com/en-us/azure/architecture/patterns/strangler>

Asynchronous first: Event sourced systems strive for the minimum amount of synchronous interaction - consistency boundaries are consciously chosen so that business requirements are met and everything else is eventually consistent - this results in responsive, high performance, scalable systems.

Occasionally connected: Since there is a log of all state changes of an application, it can be used in occasionally connected system scenarios - when a device is disconnected it can continue to work on its own data locally and synchronize upon connection.

Observability: In event-sourced systems, events flow through queues and streams increasing observability - what is uniquely powerful is that the events can contain the business context, which allows real-time observations and analytics.

One way data flow: Data in a *Command Query Responsibility Segregation* - CQRS event-sourced system flows one way (through independent models to update or read information). This brings an improved ability to reason about data and debug as each component in the data flow has a single responsibility CQRS is the notion that you can use a different model to update information than the model you use to read information.

<https://martinfowler.com/bliki/CQRS.html>

Fault tolerance: Event streams are fundamentally just logs with strong backup and recovery characteristics - writing just the core “source of record” data to the event stream enables the rebuilding of the downstream projections - EventStoreDB itself is also a distributed database technology with failover if a leader fails.

IBM Db2 Event Store is a memory-optimized database designed to rapidly ingest and analyze streamed data for event-driven applications. IBM Watson Studio is built into IBM Db2 Event Store for an integrated, collaborative development experience with support for creating ML models.

<https://www.ibm.com/downloads/cas/7POKYZJD>

Enterprise look to leverage ML to derive more value from data at greater speeds.

77% report using ML learning for stored data.

73% use it for streamed data.

88% say they need to perform analytics in near-real time.

Organizations report their current solutions have room for improvement:

1. 49% data analysis.
2. 43% real-time analytics.
3. 42% ability to view results in real-time.
4. 31% data storage on the same platform enterprises look to leverage ML.

IBM Db2 Event Store is now available on IBM Cloud Pak for Data, an extensible data and AI platform - built on the Red Hat OpenShift Container Platform - that modernizes how businesses collect, organize and analyze data and infuse AI throughout their organizations.

NEventStore is a persistence agnostic Event Store for .NET It is used to abstract different storage implementations when using event sourcing as storage mechanism. It is targeted at *Domain Driven Design - DDD* and CQRS based applications. indexCQRS patterns

<https://docs.microsoft.com/en-us/azure/architecture/patterns/cqrs>

It is not intended to be a general stream store.

<http://neventstore.org/>

Temporal databases store data relating to time instances (Snodgrass, 1999). It offers temporal data types and stores information relating to past, present and future time. Temporal databases could be uni-temporal, bi-temporal or tri-temporal: Temporal aspects usually include the following:

1. Valid time is the time period during which a fact is true in the real world.
2. Txn time is the time at which a fact is recorded in the database.
3. Decision time is the time at which the decision was made about the fact.

https://en.wikipedia.org/wiki/Temporal_database

9.33 Streaming analytics

Various aspects of stream mining are discussed in (Garofalakis et al., 2016). Aurora was a joint prototyping effort between Brandeis U., Brown U., and MIT (Abadi et al., 2003), which led to Borealis. Both as well as efforts at Stanford and UCLA are covered in this book.

9.33.1 Aurora database to manage data streams

The basic processing model and architecture of Aurora is a system to manage data streams for monitoring applications, which differ from conventional business data processing, that the software system must process and react to continual inputs from sensors rather than from humans and this requires rethinking the fundamental architecture of a DBMS for this application area Abadi et al. (2003).

Traditional DBMSs have been oriented toward business data processing, and consequently are designed to address the needs of these applications (Carney et al., 2002).

1. It is assumed that the DBMS is a passive repository and that humans initiate queries and transactions on this repository, which is called a *Human-Active, DBMS-Passive - HADP* model. .

2. Current values of data elements are easy to obtain in traditional databases, while previous values can only be found by referring to the DBMS log. Monitoring applications require data management that extends over some history of values reported in a stream. Consider a monitoring application that tracks the location of items of interest. Ceiling-mounted sensors inside a building and the GPS system generate large volumes of location data.
3. Most monitoring applications are trigger-oriented. If one is monitoring a chemical plant, then one wants to alert an operator if a sensor value gets too high or if another sensor value has recorded a value out of range more than twice in the last 24 hours. Every application could potentially monitor multiple streams of data, requesting alerts if complicated conditions are met. Thus, the scale of trigger processing required in this environment far exceeds that found in traditional DBMS applications. Triggers and alerters have been added as an after-thought and are hence second-class citizens. These implementations do not scale up to a large number of triggers (Widom and Ceri, 1996).
4. DBMSs assume that data elements are synchronized and that queries have exact answers. In many stream-oriented applications, data arrives asynchronously and answers must be computed with incomplete information. Stream data is often lost, stale, or intentionally omitted for processing reasons. An object being monitored may move out of range of a sensor system, thereby resulting in lost data. The most recent report on the location of the object becomes more and more inaccurate over time. Moreover, in managing data streams with high input rates, it might be necessary to shed load by dropping less recent input data. By necessity this leads to approximate answers.
5. Monitoring applications have real-time requirements. Applications that monitor mobile sensors often have a low tolerance for stale data, making these applications effectively real time. The added stress on a DBMS that must serve real-time applications makes it imperative that the DBMS employ intelligent scheduling) and graceful degradation strategies, i.e., load shedding) during periods of high load. QoS specifications are required for running such systems.

Streaming analytics software makes it possible to analyze data in real time, which is very different from analyzing data at rest. Stream mining arose in the telecommunications area in late 1990s following the advent of data mining in databases. Programs constantly calculate statistical analytics while moving with the stream of data. The value of this real-time data is transient and insights must be acted on immediately to gain real value. Streaming analytics technology has been adopted by the finance and capital markets industry. In capital markets, real-time data is a crucial tool to understand what is happening right now to take appropriate action. For example, a streaming analytics model might watch market data streams with instructions to take specific action if certain conditions are met, e.g., trade the stocks if the spread between two stocks deviates by more than a certain percentage in any five second period.

High Frequency Trading - HFT

HFT is a special case of algorithmic trading, which involves computers placing a large amount of trades at sub-second speeds with a view of profiting at the expense of slower traders (Acharya and Sidnal, 2016).

HFT is steadily taking over the equity trading world and involves very high speed systems placing trades at sub-millisecond speeds across multiple stock exchanges. It is a good example for Big Data analytics - especially the velocity aspect of big data. For HFT strategies to be profitable, real time processing of big data is essential, but HFT systems designed using real time stream processing have certain drawbacks. It is argued in Acharya and Sidnal (2016) that a theoretical framework for building HFT systems using the complex event processing paradigm could overcome the drawbacks of stream processing. Complex event processing enables detecting patterns of events from disparate events streams and responds to the detected pattern.

US *Securities and Exchange Commission - SEC* defines HFT using the following characteristics:

1. Very fast and complex systems are used for placing and executing orders on the stock exchanges.
2. The HFT firms make use of sophisticated tools like individual feeds and co-location services of the exchanges to minimize latency.
<https://www.marketsmedia.com/co-location-how-close-can-you-get/>
3. The positions are maintained for a very short duration.
4. A lot of orders are submitted and canceled subsequently.
5. At the end of the trading day, none or minimal positions are carried over to the next trading.

Several textbooks dealing with high-frequency trading are listed in the Appendix.
 HFT simulation with Stream Analytics with stream analytics is described below.
<https://docs.microsoft.com/en-us/azure/stream-analytics/stream-analytics-high-frequency-trading>

Two web pages discussing streaming analytics are as follows.

<https://www.trustradius.com/streaming-analytics>

<https://www.trustradius.com/buyer-blog/understanding-streaming-analytics>

The streaming analytics category is an evolution of *Complex Event Processing - CEP*. CEP engines were first created for capital markets such as stock trading algorithms where a response within milliseconds is required. Streaming analytics applications, by contrast, tend to have higher latency but are designed for the very high throughput of the big data era. CEP and real-time processing are discussed in Section 9.31.

There are three main technologies underlying streaming analytics capabilities.

SQL-Based: Many CEP tools and some streaming analytics vendors use SQL as the underlying analytics engine. Various flavors of streaming SQL emerging in 2016 made SQL a very viable option, e.g., TIBCO Streambase uses a version of SQL called StreamSQL.

(Inference) Rule-Based: This is more commonly seen in CEP products and works by using if-then-else logic. Each incoming record is compared to the rules and then and if appropriate some action is triggered. This approach works best with highly structured data.

Programmatic: Code is written in a standard or a proprietary language like IBM *Streams Processing Language - SPL*.

<https://www.ibm.com/docs/en/streams/4.2.0?topic=reference-streams-processing-language-spl>

System S IBM's stream processing platform which provides a programming model and an execution platform for user-developed applications that ingest, filter, analyze, and correlate potentially massive volumes of continuous data streams. Details about this project are provided here.

https://researcher.watson.ibm.com/researcher/view_group_subpage.php?id=2534

A paper describing the challenges of prototyping a specific application on System S is (Wu et al., 2007).

Open source vs proprietary tools

The advantage of open-source tools is that they are usually free and innovative technologies often debut as open-source. Open-source tools usually dominate discussion at the infrastructure layer and being incorporated as components of products from large proprietary vendors.

Open-source technologies in the streaming analytics arena include Apache Flink, Apache Spark Streaming, Apache Samza, Apache Kafka Streaming, Apache Storm, Google Dataflow, Twitter Heron, and AirBnB StreamAlert. All of these tools originally used the programmatic logic approach, but now support streaming SQL too.

<https://en.wikipedia.org/wiki/StreamSQL>

Trends in stream processing

Platforms for streaming analytics include: IBM Streaming Analytics,

<https://www.ibm.com/cloud/analytics-engine>

Vitria Operational Intelligence,

<https://www.vitria.com/company/leadership/>

<http://teknecroncorp.com/about-us.html>

https://en.wikipedia.org/wiki/Dale_Skeen

Microsoft Azure Stream Analytics, and

https://en.wikipedia.org/wiki/Azure_Stream_Analytics

Statistical Analysis System - SAS Event Stream Process

[https://en.wikipedia.org/wiki/SAS_\(software\)](https://en.wikipedia.org/wiki/SAS_(software))

and TIBCO Streambase.

https://en.wikipedia.org/wiki/TIBCO_Software

Regarding the issue of using proprietary tools versus open-source tools it has been said that open-source streaming analytics tools are focused on smaller projects, but their time to market is quite expensive and the learning curve can

be long. These tools require the use of a programming language and they don't provide visual tooling to help develop applications faster. Open-source tools lack stability and tend to change as the code base is being constantly modified.

Open source tools differentiate themselves from others by becoming increasingly specialized.

1. doing correlation abstraction, aggregation, and filtering;
2. stateful computing platforms like Apache Spark Streaming, and Apache Flink and Samza;
3. set of tools focused on ingestion and processing of data flows and ETL like Apache Kafka and NiFi, which is a software project from the Apache Software Foundation designed to automate the flow of data between software systems by leveraging the concept of ETL, it is based on the "NiagaraFiles" software previously developed by the *National Security Agency - NSA*.

<https://www.nsa.gov/News-Features/News-Stories/Article-View/Article/1830203/niagarafiles-keep-data-flowing/#pop3363321>

Specialization adds complexity since several different tools are required for an end-to-end solution. Proprietary platforms like IBM Streaming Analytics, TIBCO StreamBase and Apama Streaming Analytics usually provide capabilities across the entire ecosystem. Large vendors may provide tools in conjunction with the streaming platform like visual analytics, advanced numerical analysis, collaboration capabilities, and other functionality. Proprietary vendors say that this portfolio approach can reduce project complexity in the long run.

As in many technology domains where significant expertise is required to derive value, streaming analytics technology is in the process of being simplified to extend its user base beyond the IT department. The goal is for business users to create real-time models without any help from a developer by using visual tooling and clicks. It is best to develop applications using IBM's proprietary SPL.

Another trend is the expansion of the BI - Business Intelligence domain to include real-time streaming data. BI tools are designed to look back at historical data, but there is increasing interest to build real-time data stream analysis into their products. Zoomdata sells a data visualization and analytics platform designed for people to explore and analyze vast quantities of data and near-real-time data. It has had patents in this area since 2014.

<https://en.wikipedia.org/wiki/Zoomdata>

9.34 Trill: a high-performance incremental query processor for diverse analytics

Trill query processor fulfills a combination of three requirements for big data analytics. Described in (Chandramouli et al., 2014) are Trill's new design and architecture, and reported are experimental results that demonstrate its high performance cross diverse analytics scenarios.

Tempo-relational Model: Trill handles streaming and relational queries with early results, across the latency spectrum from real-time to offline.

Fabric and Language Integration: Trill is architected as a high-level language library that supports rich data-types and user libraries, and integrates well with existing distribution fabrics and applications.

Performance: Trill's throughput is high across the latency spectrum. For streaming data, Trill's throughput is 2-4 orders of magnitude higher than comparable streaming engines. For offline relational queries, Trill's throughput is comparable to major commercial columnar DBMS.

Big data analytics requires a diverse range of types of analytics, with a variety of latency settings over which the analytics is applied:

1. **Real-time streaming queries** including queries on real-time data, which may reference slow-changing data such as social network graphs or data from data markets, e.g., a smartphone user is notified if any friends are nearby.
2. **Temporal queries on historical logs:** For example, the average click-through-rate of ads in a 10-minute window, on a 30-day log.
3. **Progressive relational queries on collected data** for exploratory queries over logs to better understand the data. Computing progressively provides early results on partial data and refining as more data is streamed in, allows productive and cost-effective exploration.

Desirable features in existing systems and Trill are given in Table 9.22.

Re Fabric & Language Integration Trill is written as a library in C# and thus benefits from arbitrary HLL data-types, a rich library ecosystem, integration with arbitrary program logic, ingesting data without "handing off" to a server or copying to native memory, and easily embedding within scale-out fabrics and as part of a Cloud application.

TABLE 9.22 Desirable features in existing systems and Trill.

	Stream insight STREAM	Storm	Naiad Spark stream	Vertica, Shark, SQL sever	Trill
Temporal	Y	Y	Y	N	Y
Incremental	Y	Y	Y	N	Y
HLL	Some	N	Y	N	Y
Fabrics	B	N	N	N	Y
Throughput	Low	Low	Mid	High	High
Batched ops	No	N	Y	Y	Y
Latency	Y	N	N	N	Y
Columnar	N	N	N	Y	Y

9.35 Summary of Forrester Wave™ streaming analytics, Q3, 2109

The Forrester report classifies providers of streaming analytics in three categories:

<https://www.ibm.com/downloads/cas/8M6D9KGO>

Contenders: Alibaba (Realtime compute), EsperTech (Esper, Esper Enterprise Ed., EsperHA).

Strong performers: SAS (Event Stream Processing - ESP), Cloudera (DataFlow - CDF), Impetus (Stream Analytics), AWS (Amazon Kinesis Data Analytics).

Leaders: Software AG (Apama Streaming Analytics Platform), IBM (Streams), TIBCO, Microsoft (Azure Stream Analytics), Google (Cloud Dataflow).

Streaming analytics companies are listed below:

Software AG Apama is a full-featured streaming analytics platform well-suited for real-time applications, such as IoT and supply-chain applications. Apama is included in Software AG's cumulocity IoT cloud that simplifies device connectivity management and now offers world-class real-time analytics.

<https://cumulocity.com/guides/concepts/introduction/>

Software AG was founded in W. Germany in 1969 and is now Germany's second largest company after SAP. Its products include ADABAS high performance transactional DBMS.

https://en.wikipedia.org/wiki/Software_AG

IBM's Streaming Analytics was developed at IBM research. It has strengths in architecture, low-latency performance, management, development, deployment, and integration. Streams is ideal for enterprises with complex, high-scale, real-time applications that need maximum architectural flexibility. Streams is also available in IBM data Pak for cloud, making it an embedded, easy-to-use element of IBM's cloud data management platform.

Microsoft Streaming Analytics is available to Azure developers.

<https://docs.microsoft.com/en-us/azure/stream-analytics/>

It is based on Trill at Microsoft research, which is a high-performance, in-memory streaming analytics engine which uses Stream analytics Query Language to define streaming analytics and can extend the functionality with custom code written in JavaScript or C#. Developers can also easily build and deploy Azure Stream analytics applications in the cloud as well as on devices running Azure IoT edge runtime using the same language (Chandramouli et al., 2014).

<https://badrish.net/papers/trill-vldb2015-slides.pdf>

Microsoft Azure Stream analytics has strengths in scalability, high availability, deployment, and applications. Azure Stream analytics is an easy on-ramp for SQL developers. Zero-code integration with over fifteen other Azure services makes it easy to try and therefore adopt, making the product the real-time backbone for enterprises needing real-time streaming applications. Additionally, through integration with IoT Hub and Azure functions, it offers seamless interoperability with thousands of devices and business applications.

Google unifies streaming analytics and batch processing. Google's cloud dataflow is that solution for *Google cloud Platform - GcP* developers and has strengths in scalability, management, and security and is a managed service that is only available on GcP.

TIBCO Software augments human intelligence with real-time intelligence, by combining TiBco Streaming for real-time streaming analytics with TIBCO Spotfire for visual analytics with its add-on. Developers and business intelligence professionals alike can get up and running quickly with rich, no-code visual development tools. Developers can always

drop to code when needed. TIBCO is also behind Flogo, an open source edge computing framework to extend streaming analytics applications to IoT devices.

https://en.wikipedia.org/wiki/Edge_computing

Integration with TIBCO data Science injects machine learning models into autonomous, real-time intelligent applications. Its integration with TIBCO Spotfire visual analytics also makes it an ideal core technology to power real-time command centers. While enterprises can choose on-premises and/or cloud deployments TIBCO has made significant investments to make all platforms cloud-capable.

Cloudera DataFlow - CDF brings data management discipline to streaming analytics. In addition to streaming analytics it brings features such as data provenance and other data management capabilities of batch-oriented big data and data warehouse platforms. The solution also includes edge management and two-way messaging technology needed to build and deploy IoT applications. CDF is composed completely of open source components:

Apache NiFi <https://nifi.apache.org/>,

Apache Storm <https://storm.apache.org/>,

Apache Kafka https://en.wikipedia.org/wiki/Apache_Kafka,

and many others.

Cloudera offers strengths in streaming data platforms, management, security, development, extensibility, and deployment. CDF offers a no-code visual development and comprehensive platform management tool that abstracts away all the complexity of the many open source projects working to create real-time applications, including IoT. CDF is a natural fit for enterprises that also use Cloudera enterprise data Hub, data Warehouse, and/or data Science Workbench, because data management capabilities are integrated and consistent. CDF is deployable on-premises and in the cloud, as well as within **containers**.

<https://azure.microsoft.com/en-us/overview/what-is-a-container/>

SAS Event Stream Processing - ESP makes the event-driven world smarter. Enterprises already have process-driven apps like ERP, *Human Resource Management - HRM*, accounting, etc, but to succeed enterprises must also detect and respond to real-time events portending threats or unveiling opportunities. SAS ESP is designed not only to capture and respond to those events but also to learn from them.

<https://documentation.sas.com/doc/en/helpcenterwlcml/1.0/home.htm>

SAS is the vendor with the most built-in analytics for ML and other advanced analytics. SAS ESP has strengths in low-latency performance, advanced analytics, deployment, and integration and can handle a broad range of use cases, from fraud detection to IoT. Developers can create streaming analytics applications using a coding interface. SAS ESP is available standalone or in the cloud.

AWS Kinesis offers a real-time system for cloud applications allowing the use of SQL to develop streaming analytics queries. The solution also includes open source libraries based on Apache Flink that enable developers to use the Java programming language to create streaming analytics queries. Kinesis is tightly integrated with other services, including Kinesis data firehouse, video Streams, and data Streams. Together these services provide end-to-end capabilities to move and analyze real-time data.

Apache Flink is an open-source, unified stream/batch processing framework. The core of Apache Flink is a distributed streaming data-flow engine written in Java and Scala. Flink executes arbitrary dataflow programs in a data-parallel and pipelined (hence task parallel) manner. [4] Flink's pipelined runtime system enables the execution of bulk/batch and stream processing programs. [5][6] Furthermore, Flink's runtime supports the execution of iterative algorithms natively.

Flink provides a high-throughput, low-latency streaming engine[8] as well as support for event-time processing and state management. Flink applications are fault-tolerant in the event of machine failure and support exactly-once semantics. Programs can be written in Java, Scala, Python, and SQL and are automatically compiled and optimized into dataflow programs that are executed in a cluster or cloud environment.

Flink does not provide its own data-storage system, but provides data-source and sink connectors to systems such as AWS Kinesis, Apache Kafka, HDFS, Apache Cassandra, and Elasticsearch.

https://en.wikipedia.org/wiki/Apache_Flink

Impetus Stream Analytix makes Apache Spark work for enterprises in real-time. Impetus' Stream Analytix provided tools are used by enterprises. It is architecturally positioned to support other open source streaming analytics software such as Apache Flink. To add additional streaming analytics and CEP capabilities, the company also embeds Esper from EsperTech for enterprises wishing to adopt Apache Spark as their streaming engine.

EsperTech is a rich, performant, and embeddable streaming analytics solution, which offers a free, downloadable version of Esper, its open source streaming analytics solution. The company also offers commercial, closed source versions.

There is Esper *High availability - HA*, an enterprise solution, and an embeddable version that both enterprises and ISVs can use directly in other software. EsperTech offers strengths in core streaming analytics capabilities, low-latency performance, development tools, and deployment. Developers can use either visual tools or *Esper Programming Language - EPL* that extends the SQL standard and enables rich expressions over events and time. EPL can be compiled into byte code for quick execution and easy distribution to the streaming analytics engine. EsperTech does not provide a managed cloud service.

Alibaba Realtime Compute leverages open source Apache Flink to create an engine that supports both streaming analytics and batch processing. In 2019 Alibaba acquired Berlin-based Data Artisans a key contributor to Apache Flink, which is now known as Ververica and contributes to focus on Apache Flink and its own Ververica platform. Apache Flink has an Apache Beam runner, which means applications written using the apache Beam SDK can run on other Apache Beam engines, such as Google Cloud Dataflow and IBM Streams, and vice versa. That makes streaming applications portable to streaming engines that support Apache Beam.

<https://beam.apache.org/>

Modern *Stream Processing Engines - SPEs* process large data volumes under tight latency constraints are discussed in (Zeuch et al., 2019). Many SPEs execute processing pipelines using message passing on shared-nothing architectures and apply a partition-based scaleout strategy to handle high-velocity input streams. SPEs rely on a *Java Virtual Machine - JVM* to achieve platform independence and speed up system development by abstracting from the underlying hardware.

https://en.wikipedia.org/wiki/Java_virtual_machine

it is shown that taking the underlying hardware into account is essential to exploit modern hardware efficiently. An extensive experimental analysis of current SPEs and SPE design alternatives optimized for modern hardware was undertaken to identify potential bottlenecks. It was revealed that state-of-the-art SPEs are not capable of fully exploiting current and emerging hardware trends, such as multi-core processors and high-speed networks. Based on our analysis a set of design changes to the common architecture of SPEs to scale-upon modern hardware. It is shown that the single-node throughput can be increased by up to two orders of magnitude compared to state-of-the-art SPEs by applying specialized code generation, fusing operators, batch-style parallelization strategies, and optimized windowing. This speedup allows for deploying typical streaming applications on a single or a few nodes instead of a large cluster.

9.36 Content stores

A content repository or store is a database of digital content with an associated set of data management, search and access methods allowing application-independent access to the content, with the ability to store and modify content in addition to searching and retrieving. The content repository acts as the storage engine for a larger application such as a content or document management system, which adds a user interface on top of the repository's application programming interface.

Advantages provided by repositories:

1. Common rules for data access allow many applications to work with the same content without interrupting the data.
<https://tech.gc.com/data-interruption-process/>
2. Signal changes happen, letting other applications using the repository know that something has been modified.
3. Developers can deal with data using programs that are more compatible with the desktop programming environment.
4. The data model is scriptable when users use a content repository.

A content repository may provide functionality such as: (1) Add / edit / delete content, (2) Hierarchy and sort order management, (3) Query / search, (4) Versioning, (5) Access control, (6) Import / export, (7) Locking, (8) Life-cycle management, (9) Retention and holding / records management.

https://en.wikipedia.org/wiki/Content_repository

Ranking of content stores is given in Table 9.23.

<https://db-engines.com/en/ranking/content+store>

TABLE 9.23 Content stores.

10/21	9/21	10/21	Company	Data model	10/21	9/21	10/21
1	1	1	Jackrabbit	Content	5.91	+0.11	+0.88
2	2	2	Modeshape	Content	0.42	−0.03	+0.02

Jackrabbit is an open source content repository from Apache for the Java platform. The project was in 2004 to build the *Java Content Repository - JCR* API.

https://en.wikipedia.org/wiki/Apache_Jackrabbit

ModeShape is a hierarchical, transactional, and consistent data store with support for queries, full-text search, events, versioning, references, and flexible and dynamic schemas. It is very fast, highly available, extremely scalable, and it is open source and written in Java. Clients can query content through JDBC and SQL. ModeShape is organized in a tree-like hierarchical structure where related data is stored close together, where navigation to related content is just as common and important as fast key-based lookups or queries. The hierarchical organization is similar to a file system, making ModeShape a natural for storing files annotated with metadata. ModeShape can even automatically extract the structured information within the files so that clients can navigate or use typed queries to find files satisfying complex, structurally-oriented criteria. ModeShape is an excellent store for data with a complex schema, since the schema can vary over the database and evolve over time. ModeShape is the perfect data store for all kinds of applications, including repositories, content management systems, historical data services, provisioning and governance systems, and metadata management systems.

<https://modeshape.jboss.org/>

9.37 Multimodel databases

Most database management systems are organized around a single data model that determines how data can be organized, stored, and manipulated. A multimodel database is designed to support multiple data models against a single, integrated backend. The main difference between the available multimodel databases is related to their architectures. Multimodel databases can support different models either within the engine or via different layers on top of the engine. Multimodel databases are compared in Table 9.24.

https://en.wikipedia.org/wiki/Multi-model_database

TABLE 9.24 Comparison of Multimodel databases in alphabetic order.

Database	SQL	Doc.	Graph	Object	License	Txns
ArangoDB	No	Yes	Yes	No	Apache 2	Full ACID, locking
Azure Cosmos DB	Yes	Yes	Yes	Yes	Proprietary	Full ACID within partition
CrateIO	Yes	Yes	No	Yes	Apache 2	Eventual consistency, OCC
EnterpriseDB	Yes	Yes	No	Yes	Proprietary	Full ACID
MarkLogic	Yes	Yes	Yes	No	Proprietary	Full ACID
OrientDB	Yes	Yes	Yes	Yes	Apache 2	Full ACID
SAP HANA	Yes	Yes	Yes	No	Proprietary	Full ACID
Virtuoso	Yes	Yes	Yes	Yes	Prop./GNU GPLv2	Full ACID

ArangoDB is discussed in Section 9.20.

Azure Cosmos DB is discussed in Section 9.19.

CrateDB is a distributed SQL DBMS that integrates a fully searchable document-oriented data store. It is open-source, written in Java, based on a shared-nothing architecture, and is designed for high scalability and includes components from Presto, Lucene, Elasticsearch and Netty. CrateDB is a product of crate.io.

<https://en.wikipedia.org/wiki/CrateDB>

EnterpriseDB/EDB is a Massachusetts based private company, which provides software and services based on the open-source database PostgreSQL.

<https://en.wikipedia.org/wiki/EnterpriseDB>

Marklogic is discussed in Section 9.20.

OrientDB is discussed in Section 9.20.

SAP HANA is discussed in Section 9.7.

Virtuoso DB is discussed in Section 9.7.

Chao Zhang et al. at U. Helsinki proposed the UniBench benchmark for *Multi-Model DBs - MMDBs* (Zhang et al., 2018a). Previous benchmarks are inadequate because they lack a comprehensive consideration for multiple models of data. UniBench consists of a mixed data model, a synthetic multi-model data generator, and a set of core workloads. Specifically, the data model simulates an emerging application: social Commerce, a Web-based application combining E-commerce and social media. The data generator provides diverse data format including JSON, XML, key-value, tabular, and graph. The workloads are comprised of a set of multi-model queries and txns aiming to cover essential aspects of MMDB. All workloads were implemented on ArangoDB and OrientDB to illustrate the feasibility of the proposed benchmarking system and show the learned lessons through the evaluation of these two MMDBs.

9.38 Main memory databases

An in-memory database is a DBMS that primarily relies on main memory for data storage. It is contrasted with DBMSs that employ a disk storage mechanism.

https://en.wikipedia.org/wiki/In-memory_database

IBM's hierarchical *Information Management Systems - IMS* has an in main memory version IMS Fastpath - FP as discussed in Section 9.3.1.

Considered in (DeWitt et al., 1984) is the changes necessary to allow a relational DBMS to take advantage of large amounts of main memory. The authors evaluate AVL (Adelson-Velskii and Landis, 1962) vs B+-tree access methods for main memory databases, hash-based query processing strategies vs sort-merge, and study recovery issues when most or all of the database fits in main memory. As expected, B+-trees are the preferred storage mechanism unless more than 80–90% of the database fits in main memory. A major conclusion is that hash based query processing strategies are advantageous for large memory situations.

An early publication on this topic is (Grund et al., 2010), which discusses HYRISE main memory hybrid storage engine at Hasso-Plattner-Institute and MIT. An emerging class of hybrid OLTP and OLAP database systems allows to process analytical queries directly on transactional data. By offering arbitrarily current snapshots of the transactional data for OLAP, these systems enable real-time business intelligence (Funke et al., 2012).

SAP HANA is an in-memory, column-oriented, RDBMS and marketed by SAP SE.

https://en.wikipedia.org/wiki/SAP_HANA

Its primary function as a database server is to store and retrieve data as requested by applications. In addition, it performs advanced analytics (predictive analytics, spatial data processing, text analytics, text search, streaming analytics, graph data processing) and includes ETL capabilities as well as an application server.

While HANA requires the entire database to be in memory and while memory prices have declined dramatically, memory still commands a huge premium over disk. Plus, a single server can accommodate only so much memory in the first place, so petabyte-scale data work on HANA can require huge hardware expenditures.

Teradata has determined that after a certain percentage of data is stored in memory instead of disk, the returns quickly diminish. In fact 43% of all I/O hits only 1% of all disk cylinders, and that 94% percent of IO hits only 20% of all cylinders, which is an instance of the 80-20 rule.

With that in mind, the company announced the addition of “Intelligent Memory” functionality to its database appliances, which ensures that the most frequently used or hot database data in a database stays resident in a special extended memory region and infrequently queried (“cold”) data stays on disk. The determination of which data is hot and cold is updated dynamically, with Teradata consequently moving certain data in and out of RAM, at opportune times, in terms of processing inactivity on the cluster.

This is not a caching scheme which typically moves recent data into RAM, rather than frequently used data, which is a simpler approach and is used by Teradata File Segment cache.

<https://www.zdnet.com/article/teradata-enters-the-in-memory-fray-intelligently>

TimesTen is a main memory database developed by Marie-Anne Neimat et al. at HP Labs (Lahiri et al., 2013).

<https://www.fastcompany.com/3019296/44marie-anne-neimat>

Neimat was Alan Jay Smith's first PhD student at UCB with a PhD thesis on “Search Mechanisms for Large Files” in 1979 (named but not uploaded).

<https://www2.eecs.berkeley.edu/Pubs/Dissertations/Faculty/smith-a.html>

TimesTen was spun off in 1996 by HP into a separate venture capital funded startup and its product became popular for telecommunications, where response times in milliseconds/microseconds are required for packet switching. The company with ninety employees was profitable when it was acquired by Oracle Corp. in 2005. Neimat as director of TimesTen development at Oracle added many Oracle database features to the product such as support for PL/SQL and integration

with Oracle SQL Developer and Enterprise Manager.

<https://en.wikipedia.org/wiki/TimesTen>

VoltDB is a NewSQL relational database that supports SQL access from within pre-compiled Java stored procedures. The unit of txn is the stored procedure, written in Java interspersed with SQL. VoltDB relies on horizontal partitioning down to the individual hardware thread to scale, k-safety (synchronous replication) to provide high availability, and a combination of continuous snapshots and command logging for durability (crash recovery). VoltDB is based on H-Store (Kallman et al., 2008). Data and the associated processing are distributed across the CPU cores within the servers composing a single VoltDB cluster. By extending its shared-nothing foundation to the per-core level, VoltDB scales with the increasing core-per-CPU counts on multi-core servers.

<https://en.wikipedia.org/wiki/VoltDB>

<https://www.voltodb.org>

Microsoft Hekaton is a database engine for memory resident data and OLTP workloads fully integrated into SQL Server (Diaconu et al., 2013). Users utilize Hekaton by declaring tables as memory optimized. Hekaton tables are accessed using Transact-SQL (T-SQL) in the same way as regular SQL Server tables are: <https://en.wikipedia.org/wiki/Transact-SQL>. A query can reference both Hekaton tables and regular tables and a txn can update data in both types of tables. The engine achieves high concurrency by using latch-free data structures and a new optimistic MVCC technique, which is described by Per-Ake Larson et al. in (Larson et al., 2011).

[https://en.wikipedia.org/wiki/Hekaton_\(database\)](https://en.wikipedia.org/wiki/Hekaton_(database))

A text on main memory databases (Faerber et al., 2017) reviews the following databases: SQL Server Hekaton, H-Store and VoltDB, memSQL, HyPer, SAP HANA, and other systems.

<https://justinlevandoski.org/papers/fnt-mmdb.pdf>

HyPer is a hybrid OLTP & OLAP main memory database system based on virtual memory snapshots (Kemper and Neumann, 2011). OLTP and OLAP present different challenges for database architectures. A current solution is to split their data into two separate systems, one database for OLTP and one data warehouse for OLAP. While this separation allows decent txn rates its disadvantages include excessive resource consumption and data freshness due to the delay of only periodically initiating the *Extract Transform Load* - ETL-data staging. HyPer can handle both OLTP and OLAP simultaneously by using hardware-assisted replication mechanisms to maintain consistent snapshots of the transactional data.

9.39 Distributed file systems and object storage

Members of Gartner's Magic Quadrant for Distributed File Systems and Object Storage denoting the four quadrants as NE, NW, SE, and SW. The October 2020 MQ is provided here and its is noted that IBM appears twice, as itself and RedHat.

<https://blocksandfiles.com/2020/10/21/gartners-annual-mqubulation-of-distributed-files-systems-and-objects/>

In what follows companies are listed into four categories:

Leader - L, which is the *NorthEast* - NE quadrant.

Challenger - C, which is the *NorthWest* - NW quadrant.

Visionary - V, which is the *SouthEast* - SE quadrant.

Niche player - N, which is the *SouthWest* - SW quadrant.

The x-axis is in increasing order of vision and the y-axis increasing ability to execute.

NE - Dell/EMC, IBM, Qumulo, Scality.

NW - Cloudian, Hitachi Vantara, Quantum, Huawei.

SE - Pure Storage, NetApp, Red Hat.

SW - Inspur, DDN, Caringo.

A brief description of members is given in the following alphabetic list:

Caringo: founded in 2005 in Austin, TX delivers intelligent data management for content access, delivery and archiving. Its object storage product is Swarm.

<https://www.caringo.com/products/swarm/partial-file-restore>

<https://www.caringo.com/>

Cloudian is a privately held company founded in 2011 In San Mateo, CA. Its object storage product Hyperstore costs 0.5 cents per GB per month.

<https://cloudian.com/>

Data Direct Networks - DDN is a privately held company, founded in 1998 in Chatsworth, CA. DDN is a storage supplier in *High Performance Computing - HPC*. Its product is *Web Object Scalar - WOS*.

www.ddn.com

DDN acquired Lustre from Intel, an open-source, distributed parallel file system software platform designed for scalability, high-performance, and high-availability. Lustre provides a coherent, global POSIX-compliant namespace for very large scale computer infrastructure, including the world's largest supercomputer platforms. It supports hundreds of petabytes of data storage and hundreds of gigabytes per second in simultaneous, aggregate throughput. Some of the largest current installations have individual file systems in excess of fifty petabytes of usable capacity, and have reported throughput speeds exceeding one terabyte/sec.

https://wiki.lustre.org/Main_Page

Dell/EMC Isilon is a distributed file system, which can scale to 252 nodes and *Elastic Cloud Storage - ECS* object storage. In January 2019, support for the new Isilon F8100 all-flash was added, providing up to 3:1 hardware compression. In June 2019, support for the Isilon H5600 was added, providing 800 TB capacity. ECS enhancements include the capability to start at 60 TB and scale up to 8.6 TB per rack in ECS 3.2.1; support for *Write-Once-Read-Many - WORM* for data ingested via NFS; and faster S3 LIST execution in ECS 3.3. Dell EMC also released ClarityNow, which provides data discovery, classification and mobility capabilities in a single pane of glass for Isilon, ECS and third party systems.

Hitachi Vantara Content Platform is a Santa Clara, CA based subsidiary of Hitachi, which entered the object storage market in 2006.

Huawei Oceanstor 900 V5 is deployed as a scale-out *Network-Attached Storage - NAS* product, while FusionStorage 8.0 is positioned as a block and object cloud storage product. Huawei was founded in Shenzhen, China in 1987.

www.huawei.com

<https://en.wikipedia.org/wiki/Huawei>

IBM Spectrum Scale & Cloud Object Storage - COS: IBM Spectrum Scale is a scale-out file system built on IBM's *General Parallel File System - GPFS* (Schmuck and Haskin, 2002). Recent enhancements include support for containerized workloads and erasure coding, which allows Spectrum Scale to run on commodity servers without requiring SAN - Storage Area Networks as the underlying storage. IBM COS is a highly scalable and resilient object storage product, added to IBM's product portfolio through its acquisition of Cleversafe.

Inspur Active Storage was launched in 2006 and the infrastructure *Software Defined Storage - SDS* platform in 2014. Inspur is a server vendor which was founded in China in 1940 and its history is summarized below.

https://en.inspur.com/en/about_inspur/index.html

Its server business using IBM Power chips is doing well.

<https://www.itjungle.com/2019/09/09/ibm-and-inspur-power-systems-buck-the-server-decline-trends/>

Inspur diversified into the storage market in 2001. AS13000 is unified distributed storage sold as a distributed file system and object storage product designed to scale from 3 to 5,120 nodes, from 36 TB to 5160 PB capacity. AS13000 has added enhancements for *Access Control Lists - ACLs*, IPv6, object fuzzy query, file remote copy and doubling of all-flash performance to 5 GB/sec. 36 TB to 5160 PB capacity.

In computer security, an ACL is a list of permissions associated with a system resource (object). An ACL specifies which users or system processes are granted access to objects, as well as what operations are allowed on given objects. A typical ACL entry specifies a subject and an operation, e.g., if a file object has an ACL that contains (Alice: read, write; Bob: read), this would give Alice permission to read and write the file and only give Bob permission to read it.

https://en.wikipedia.org/wiki/Access-control_list

NetApp StorageGRID is a publicly held company founded in 1992 in Sunnyvale, CA, which is discussed in Chapter 5. It primarily sells on-premises storage infrastructure, but has expanded to hybrid cloud data services. <https://www.netapp.com/data-storage/>

StorageGRID is an object storage product for which the primary access method is the Amazon S3 API.

<https://aws.amazon.com/glacier/>

Notable enhancements include tiering of data to Amazon Glacier and multifactor authentication, in addition to performance and user-interface improvements. Glacier is part of the AWS suite of cloud computing services, which is designed for long-term storage of data that is infrequently accessed and for which retrieval latency times of 3 to 5 hours are acceptable. Storage costs are a consistent \$0.004 per GB per month, which is substantially cheaper than Amazon's *Simple Storage Service - S3*.

https://en.wikipedia.org/wiki/Amazon_Glacier

Pure Storage Flashblade is from a company founded in Mountain View, CA in 2009, which focuses on delivering flash-based SSD arrays for the enterprise market. Pure Storage is discussed in more detail in Chapter 2. The FlashBlade distributed file system and object storage product is based on a scale-out blade design, where each blade contains a storage controller and an SSD. The blades provide different capacities, low-latency, high-throughput through parallel access. FlashBlade leverages distributed processing of many concurrent I/O requests across many nodes/cores to allow parallel processing. A single 15-blade FlashBlade has 120 Intel Xeon cores, so that a full 75-blade system supports 600 cores.

<https://www.purestorage.com>

Qumulo cluster is a product of the company founded in Seattle, WA in 2012, which offers a distributed file system product that was designed for large-scale, high-throughput file workloads that require granular performance analytics and capacity management. The Qumulo cluster can scale up from four nodes and 96 TB capacity (or 20 TB on AWS) to 1,000 nodes or 360 PB capacity. Qumulo has added the following features: S3 interface to Qumulo clusters (through a partnership with MinIO),

<https://docs.aws.amazon.com/AmazonS3/latest/userguide/Welcome.html>

cross-protocol permissions between a communication protocol for providing shared access to files, printers, and serial ports between nodes on a network

https://en.wikipedia.org/wiki/Server_Message_Block

such as *Network File System - NFS* to improve throughput and latency for all-flash and hybrid clusters, I/O assurance during drive rebuilds, multinode failure support, and four-drive failure protection.

<https://qumulo.com/>

Red Hat Ceph Storage. IBM acquired Red Hat based in Raleigh, NC in July 2019 for \$34 billion. Red Hat has a strong background in open-source products across OSs, middleware, cloud management, automation, containers, *Platform as a Service - PaaS*, and storage. Red Hat Ceph Storage is an open-source software product supporting block, object storage access and file access, and the underlying storage for Red Hat's data analytics infrastructure solution and Red Hat Hyperconverged Infrastructure for Cloud. The BlueStore backend, which promises a two-times performance boost over the traditional FileStore back end. Red Hat Gluster Storage is an open-source product primarily sold as a scale-out NAS product, and the underlying storage for Red Hat Hyperconverged Infrastructure for Virtualization. The Red Hat Gluster Storage 3.4 release included macOS client support for SMBs,

https://en.wikipedia.org/wiki/Server_Message_Block

read-only volume mounts and support for upgrading the *Red Hat Enterprise Linux - RHEL* version. Recently, Red Hat shifted its OpenShift Container Storage standard from Gluster to Ceph.

[https://en.wikipedia.org/wiki/Ceph_\(software\)](https://en.wikipedia.org/wiki/Ceph_(software))

<https://www.redhat.com/en>

Scality RING is delivered since 2010 by Scality a venture-backed, privately held company founded in 2009 in San Francisco, CA. It has been delivering its RING scale-out, peer-to-peer distributed shared-nothing software that can be deployed on x86 commodity hardware. It features an integrated virtual file system that provides file storage services without the need for external file gateways, unlike the majority of other object storage solutions. Scality RING improved security with role-based access and encryption, enhanced multitenancy for service providers, added support for NFSv4 and Kerberos network authentication protocol,

<https://web.mit.edu/kerberos/>

[https://en.wikipedia.org/wiki/Kerberos_\(protocol\)](https://en.wikipedia.org/wiki/Kerberos_(protocol))

and delivered support for Ceph over S3 as a back-end store. Scality has expanded its data management and mobility across multiple edge and core RING offerings and public clouds, with life cycle tiering and Zenko data orchestrator integration.

<https://www.scality.com/>

SwiftStack Swift is an object storage platform based on OpenStack Swift in addition to a well-integrated file system gateway layered on top of the object storage core by the San Francisco, CA company. SwiftStack's Ispace supports creating a single namespace across multiple cloud and on-premises environments, in addition to enhanced support for Splunk and Veeam.

<https://en.wikipedia.org/wiki/Splunk>

<https://www.swiftstack.com/>

<https://www.veeam.com/>

SUSE Enterprise Storage - SES is based on Ceph and provides unified access for block, file and object protocols. *Software- und System-Entwicklung - SUSE* (Software and Systems Development) was founded in Nuremberg, Germany

in 1992. It was a founding organization of Ceph Advisory Board and number two contributor to the Ceph open-source community.

<https://www.suse.com/>

Western Digital ActiveScale is a large form factor object storage system that starts at 720 TB capacity. ActiveScale introduced NFS access, performance improvements for small files, quotas and a self-service provisioning portal, an object storage solution based on Amplidata, which was acquired in 2015. That same year, the vendor launched its product as Active Archive, a high-density system designed for multipetabyte object storage deployments and high-throughput operations. Western Digital announced its intention to exit the Storage Systems business, inclusive of ActiveScale and IntelliFlash. It was announced on 2/3/2020 that Queorum is to acquire Western Digital's IntelliFlash.

<https://www.Western Digital.com/>

9.40 Enterprise Backup and recovery software solutions

Gartner's 2021 Magic Quadrant method described in Section 9.39 is used to classify Enterprise Backup and Recovery Software Solutions. In what follows only strengths are discussed.

<https://www.crn.com/slide-shows/storage/gartner-s-top-13-backup-and-recovery-software-leaders/1>

Veeam-L offers instant recovery of Microsoft Hyper-V, VMware VMs, *Network Attach Storage - NAS* environments, and automated instant recovery of Microsoft SQL Server and Oracle Databases. Also provided is transparent public cloud backup costs.

<https://en.wikipedia.org/wiki/Veeam>

Rubrik-L offers an intuitive *User Interface - UI* that is easy to operate and scale. Additionally, *Rubrik Cloud Data management - RCDM* provides comprehensive support for all major relational databases and NoSQL databases.

<https://en.wikipedia.org/wiki/Rubrik>

Commvault-L allows customers can select one or a combination of software-only, appliance or Software data protection as a service SaaS solutions for hybrid cloud world. It has its origins in Bell Labs but was spun off by Lucent.

<https://en.wikipedia.org/wiki/Commvault>

Veritas-L supports a broad range of operating systems, hypervisors and public clouds, and integrates with all major primary storage array players. NetBackup 9.0 provides over 500 APIs that help automate deployment and policy management.

<https://www.veritas.com/protection/system-recovery>

Dell - L 's PowerProtect Cyber Recovery solution offers comprehensive ransomware detection and recovery capabilities both on-premises and in public clouds. Its Data Protection Suite provides deep integration with VMware.

<https://www.delltechnologies.com/en-us/data-protection/cyber-recovery-solution.htm#tab0=0&tab1=0>

Cohesity-L enables *Internet Service Vendors* to integrate with the Cohesity platform and offers a broad range of additional data reuse capabilities.

<https://www.cohesity.com/>

IBM-C 's Spectrum Protect Plus components are all packaged into a single *Open Virtual Appliance - OVA*

<https://www.lifewire.com/ova-file-4144357>

that can be used to deploy the backup software as a virtual appliance in VMware and Hyper-V environments, while IBM Blueprints simplifies deployments with pre-validated reference architectures and sizing guidelines.

<https://www.ibm.com/products/ibm-spectrum-protect-plus>

Druva-V metadata pipeline and repository offers differentiated search and analytics capabilities that enable use cases such as e-discovery, ransomware detection and recovery, and storage optimization.

<https://en.wikipedia.org/wiki/Druva>

Acronis-V has differentiated edge computing and ransomware security capabilities including the ability to actively scan for security threats and verify the authenticity and recoverability of backup copies.

<https://en.wikipedia.org/wiki/Acronis>

Zerto-N provides a single platform that combines backup and DR functionality to simplify data protection. It offers an intuitive UI that facilitates ease of use.

<https://en.wikipedia.org/wiki/Zerto>

Unitrends-n offers unlimited retention of backup data with no egress charges, and optional disaster recovery through its Unitrends Cloud data centers, at significantly lower costs when compared to public cloud providers.

<https://www.unitrends.com/>

Micro focus-N 's Data Protector Express edition for virtual environments offers one of the lowest price-per-socket licensing among all the vendors in the market, while supporting a broad range of purpose-built deduplication appliance vendors.

https://en.wikipedia.org/wiki/Micro_Focus

9.41 Analytics and Business Intelligence - ABI platforms

Our discussion is based on the following Gartner web page.

<https://www.gartner.com/doc/reprints?id=1-1YBTIWVR&ct=200211&st=sb>

BI platforms are no longer differentiated by their data visualization capabilities, which are commoditized, instead, differentiation is shifting to the following aspects:

1. Support for enterprise reporting capabilities, i.e., platforms with agile data visualization capabilities can modernize their enterprise reporting needs. These are commonly met by older BI products from vendors like SAP (Business Objects), Oracle (Business Intelligence Suite) and IBM (Cognos).
2. Augmented analytics. Machine learning (ML) and AI-assisted data preparation, insight generation and explanation – to augment how business people and analysts explore and analyze data – are fast becoming key sources of competitive differentiation, and therefore core investments.

ABI platform functionality includes the following fifteen critical capability areas:

Security: Capabilities that enable platform security, administering of users, auditing of platform access and authentication.

Manageability: Capabilities to track usage, manage how information is shared and by whom, perform impact analysis and work with third-party applications.

Cloud: The ability to support building, deploying and managing analytics and its applications in the cloud, based on data in the cloud, on-premises, and across multicloud deployments.

Data source connectivity: Capabilities that enable users to connect to and ingest structured and unstructured data held by on-premises and cloud storage platforms.

Data preparation: Support for drag-and-drop, user-driven combination of data from different sources, and the creation of analytic models such as user-defined measures, sets, groups and hierarchies.

Model complexity: Support for complex data models, including the ability to handle multiple fact tables, interoperate with other analytic platforms and support knowledge graph deployments.

Catalog: The ability to automatically generate and curate a searchable catalog of the artifacts created and used by the platform and their dependencies.

Automated insights: A core attribute of augmented analytics is the ability to apply ML techniques to automatically generate insights for end users, e.g., by identifying the most important attributes in a dataset).

Advanced analytics: Advanced analytical capabilities that are easily accessed by users, being either contained within the ABI platform itself or usable through the import and integration of externally developed models.

Data visualization: Support for highly interactive dashboards and the exploration of data through the manipulation of chart images. Visualization options that go beyond those of pie, bar and line charts, such as heat and tree maps, geographic maps, scatter plots and other special-purpose visuals.

Natural language query: This enables users to query data using business terms that are either typed into a search box or spoken.

Data storytelling: The ability to combine interactive data visualization with narrative techniques in order to package and deliver insights in a compelling, easily understood form for presentation to decision makers.

Embedded analytics: Capabilities include a *Software Development Kit - SDK* with APIs and support for open standards in order to embed analytic content into a business process, an application or a portal.

Natural Language Generation - NLG: The automatic creation of linguistically rich descriptions of insights found in data. Within the analytics context, as the user interacts with data, the narrative changes dynamically to explain key findings or the meaning of charts or dashboards.

Reporting: The ability to create and distribute to consumers grid-layout, multipage reports on a scheduled basis.

The aforementioned Gartner Quadrant yields the following.

NE: Microsoft, Tableau, Qlik, Thoughtspot.

NW: Microstrategy, TIBCO Software, Looker.

SE: Salesforce, Oracle, SAS, Sisense, SAP, Yellowfin.

SW: IBM, Domo, Information Builders, Board Analytics, Logi Analytics, Alibaba Cloud, Dinda, Birst.

Looker was founded in Santa Cruz, CA in 2012 by Lloyd Tabb and Ben Porterfield,

[https://en.wikipedia.org/wiki/Looker_\(company\)](https://en.wikipedia.org/wiki/Looker_(company))

grew out of Tabb's experience developing software at companies like Netscape. Looker makes use of a simple modeling language called LookML that lets data teams define the relationships in their database so business users can explore, save, and download data without needing to know SQL. Looker was the first commercially available BI platform built for and aimed at scalable or massively parallel relational database management systems like Amazon Redshift, Google BigQuery, HP Vertica, Netezza, and Teradata. In June 2019 Google acquired Looker for \$2.6B.

Qlik was founded in Lund, Sweden in 1993, by Bjorn Berg (rip) and Staffan Gestrelus. The company's first product QlikView was built to enable users to retrieve detailed data analysis from various systems.

<https://en.wikipedia.org/wiki/Qlik>

Qlik's Associative Engine lets users do big data analytics, combining a number of data sources so that associations and connections can be formed across the data. The two main products QlikView and Qlik Sense serve different purposes running on the same engine. In QlikView, the user is pursuing their day-to-day tasks, analyzing data with a slightly configurable dashboard, most of the data is static. Qlik Sense allows concatenation of different data sources and fully configuring the visualizations, allowing drill-down on an individual data record. Qlik Data Catalyst is an enterprise data management product and Qlik Core is an analytics development platform built around Qlik's Associative engine and company-authored open source libraries. Qlik GeoAnalytics provides mapping capabilities and built-in support for geoanalytic use cases. Qlik Data-Market provides access to a cloud-based service with a variety of external data content that can be used to cross-reference and augment users' data, and Qlik Connectors are tools that allow QlikView and Qlik Sense to interact with external data sources not accessible through normal *Open Database Connectivity* - ODBC or OLE-DB data connections. ODBC is a standard application programming interface for accessing DBMSs. The designers of ODBC aimed to make it independent of database systems and operating systems.

https://en.wikipedia.org/wiki/Open_Database_Connectivity

Object Linking and Embedding - OLE DB designed by Microsoft, allows accessing data from a variety of sources in a uniform manner https://en.wikipedia.org/wiki/OLE_DB

ThoughtSpot was founded in 2012. CEO and Co-founder, Ajeet Singh, previously co-founded Nutanix. The company produces BI analytics search software.

<https://en.wikipedia.org/wiki/ThoughtSpot>

In February 2016 ThoughtSpot announced it had increased its revenue by 810% over the previous year and in May 2016 it raised \$50 million in Series C funding led by General Catalyst Partners.

<https://www.investopedia.com/articles/personal-finance/102015/series-b-c-funding-what-it-all-means-and-how-it-works.asp>

The company is now valued over \$1 billion and by 2020, the company had \$100 million in annual recurring revenue.

Sisense in February 2013, as the first company in CrunchBase's "Developer Spotlight Program." Sisense analyzed 10 years of CrunchBase data in a public interactive investment dashboard dubbed CrunchAnalytics. By analyzing a decade's worth of data about 100,000 companies, the dashboard provides the public with a research tool that enables users to query the data by different categories to see trends in investment over the past ten years.

9.41.1 Crunchbase finds business information about companies

Crunchbase is a platform for finding business information about private and public companies. Its information includes investments and funding information, founding members and individuals in leadership positions, mergers and acquisitions, news, and industry trends.

<https://www.crunchbase.com/>

Originally built to track startups, the Crunchbase website contains information on public and private companies on a global scale. Crunchbase sources its data in four ways: (1) the venture program, (2) ML, (3) an in-house data team, and (4) the Crunchbase community. Members of the public can submit information to the Crunchbase database. These submissions are subject to registration, social validation, and are often reviewed by a moderator before being accepted for publication. Crunchbase products are listed below.

<https://en.wikipedia.org/wiki/Crunchbase>

Crunchbase Pro is designed for advanced users offering in-depth search capabilities, market trend analytics, and alerts to track industries, people, companies, and investors.

Crunchbase marketplace allows users to search and analyze companies with integrated third-party data sources. Users can also access additional data like web traffic, tech stack, and IT budgets.

Crunchbase enterprise lets users perform typical business intelligence and analytics queries, such as setting up and following investment trends, identifying and tracking growth industries, and identifying companies that are close to their own.

Crunchbase for applications allows users to integrate Crunchbase data into their product, providing their customers the ability to automatically contextualize prospects to prioritize their sales pipeline and identify key buy signals.

In April 2013 Sisense introduced an alternative to in-memory technology called In-Chip analytics, designed to maximize the disk, memory and CPU with resulting low latency. In May 2013, Sisense unveiled Prism 10X, which allows users to analyze 100 times more data at 10 times the speed of in-memory solutions. The company's In-Chip Analytics technology couples a columnar database with smart algorithms that use the in-chip cache, RAM or disk as needed instead of relying solely on RAM or disk. In October 2013, Sisense introduced Crowd Accelerated BI, a technology that allows companies to scale users by providing faster query results as load increases and scale to thousands of simultaneous users on a single box. Each query is broken down to its basic components, which then helps the software decide which data to store on disk and which data to bring to memory. As more queries are cataloged, the software is able to more quickly recognize future similar queries and call up relevant data. In April 2017, Sisense introduced Sisense Pulse which provides alert when it detects results outside of normal parameters for a particular metric.

<https://en.wikipedia.org/wiki/Sisense>

Sisense CTO Farkash proposes taking huge chunks of data from relational databases and vectorize it, and then run analytics on the slices. SiSense takes analytics software and makes it run in parallel on multicore processors, SiSense has run queries against as much as 20 TB of data running on the Intel and AMD chips. The technology works on Intel and AMD architectures. This represents a divergence from the high-performance computing Hadoop Hive Mind of big data. In the Hadoop Hive world, you go to IT and beg for an hour of query time, then try to stuff as much data as possible into that time.

9.42 Blockchain, Bitcoin, Ethereum

The blockchain discussion is based on the following Wiki page:

<https://en.wikipedia.org/wiki/Blockchain>

Cryptographer David Chaum first proposed a blockchain-like protocol in his 1982 dissertation. Further work on a cryptographically secured chain of blocks was described in 1991 by Stuart Haber and W. Scott Stornetta, who wanted to implement a system where document timestamps could not be tampered with. In 1992, Haber, Stornetta, and Dave Bayer incorporated Merkle trees to the design, which improved its efficiency by allowing several document certificates to be collected into one block.

In cryptography a hash tree or Merkle tree is a tree in which every leaf node is labeled with the cryptographic hash of a data block, and every non-leaf node is labeled with the cryptographic hash of the labels of its child nodes. Hash trees allow efficient and secure verification of the contents of large data structures. Hash trees are a generalization of hash lists and hash chains.

https://en.wikipedia.org/wiki/Merkle_tree

The first blockchain was conceptualized by Satoshi Nakamoto (a person or a group) in 2008, which improved the design in an important way using a Hashcash-like method to timestamp blocks without requiring them to be signed by a trusted party and introducing a difficulty parameter to stabilize rate with which blocks are added to the chain. The design was implemented the following year by Nakamoto as a core component of the cryptocurrency bitcoin, where it serves as the public ledger for all txns in the network.

Blockchain is a growing list of records, called blocks, that are linked using cryptography. Each block contains a cryptographic hash of the previous block, a timestamp, and txn data. A Merkle tree is a tree in which every leaf node is labeled with the cryptographic hash of a data block, and every non-leaf node is labeled with the cryptographic hash of the labels of its child nodes. Hash trees allow efficient and secure verification of the contents of large data structures. Demonstrating that a leaf node is a part of a given binary hash tree requires computing a number of hashes proportional to the logarithm of the number of leaf nodes of the tree.

https://en.wikipedia.org/wiki/Blockchain#/media/File:Bitcoin_Block_Data.svg

Blockchain is a decentralized, distributed, and oftentimes public, digital ledger consisting of records called blocks that is used to record txns across many computers so that any involved block cannot be altered retroactively, without the alteration of all subsequent blocks. This allows the participants to verify and audit txns independently and relatively inexpensively. A blockchain database is managed autonomously using a peer-to-peer network and a distributed timestamping server. They are authenticated by mass collaboration powered by collective self-interests. Such a design facilitates robust workflow where participants' uncertainty regarding data security is marginal. The use of a blockchain removes the characteristic of infinite reproducibility from a digital asset. It confirms that each unit of value was transferred only once, solving the long-standing problem of double spending. A blockchain has been described as a value-exchange protocol that can maintain title rights because, when properly set up to detail the exchange agreement, it provides a record that compels offer and acceptance.

Blocks hold batches of valid txns that are hashed and encoded into a Merkle tree.

https://en.wikipedia.org/wiki/Merkle_tree

Each block includes the cryptographic hash of the prior block in the blockchain, linking the two. The linked blocks form a chain. This iterative process confirms the integrity of the previous block, all the way back to the initial block, which is known as the genesis block. In addition to a secure hash-based history, any blockchain has a specified algorithm for scoring different versions of the history so that one with a higher score can be selected over others.

Peers supporting the database have different versions of the history from time to time. They keep only the highest-scoring version of the database known to them. Whenever a peer receives a higher-scoring version (usually the old version with a single new block added) they extend or overwrite their own database and retransmit the improvement to their peers. There is never an absolute guarantee that any particular entry will remain in the best version of the history forever.

Blockchains are typically built to add the score of new blocks onto old blocks and are given incentives to extend with new blocks rather than overwrite old blocks. Therefore, the probability of an entry becoming superseded decreases exponentially as more blocks are built on top of it, eventually becoming very low.

In August 2014, the bitcoin blockchain file size, containing records of all txns reached 20 GB. In January 2015, the size had grown to almost 30 GB, and from January 2016 to January 2017, the bitcoin blockchain grew from 50 GB to 100 GB in size. The ledger size has exceeded 200 GiB by early 2020. AS this time (6/24/21) the file size exceeds 343.35 GB.

<https://www.statista.com/statistics/647523/worldwide-bitcoin-blockchain-size/>

According to Accenture, an application of the diffusion of innovations theory suggests that blockchains attained a 13.5% adoption rate within financial services in 2016, therefore reaching the early adopters phase.

https://en.wikipedia.org/wiki/Technology_adoption_life_cycle

Industry trade groups joined to create the Global Blockchain Forum in 2016, an initiative of the Chamber of Digital Commerce. In May 2018, Gartner found that only 1% of CIOs indicated any kind of blockchain adoption and only 8% of CIOs were in the short-term planning to experiment with blockchain.

Bitcoin uses a *Proof-of-Work* - PoW system, where the chain with the most cumulative PoW is considered the valid one by the network. There are a number of methods that can be used to demonstrate a sufficient level of computation. Within a blockchain the computation is carried out redundantly rather than in the traditional segregated and parallel manner. A list of proof of work is provided here:

https://en.wikipedia.org/wiki/Proof_of_work

The *block time* is the average time it takes for the network to generate one extra block in the blockchain. Some blockchains create a new block as frequently as every five seconds. By the time of block completion, the included data becomes verifiable. In cryptocurrency, this is practically when the txn takes place, so a shorter block time means faster txns. The block time for Ethereum is set to between 14 and 15 seconds, while for bitcoin it is on average 10 minutes.

<https://en.wikipedia.org/wiki/Ethereum>

Ethash algorithm reduce the advantage of specialized ASICs in mining

Ethereum is a decentralized, open-source blockchain with smart contract functionality. Ether (ETH) is the native cryptocurrency of the platform and after Bitcoin, it is the second-largest cryptocurrency by market capitalization, while Ethereum is the most actively used blockchain.

<https://en.wikipedia.org/wiki/Ethereum>

Ethereum was proposed in 2013 by programmer Vitalik Buterin at Univ. Waterloo. He dropped out after getting a major development fund. In 2014, development was crowdfunded, and the network went live with an initial supply of 72 million coins on 7/30/2015.

https://en.wikipedia.org/wiki/Vitalik_Buterin

Non-Fungible Tokens - NFTs are units of data stored on digital ledgers, called blockchains, including Ethereum blockchains, that certify a digital asset to be unique and therefore not interchangeable. NFTs represent photos, videos, audio, and other types of digital files. Access to the original file is not restricted to the buyer, while copies are available for anyone to obtain, NFTs are tracked on blockchains to provide the owner with a proof of ownership that is separate from copyright. The NFT market value tripled in 2020, reaching more than \$250M. During the first quarter of 2021, NFT sales exceeded \$2B.

https://en.wikipedia.org/wiki/Non-fungible_token#Ethereum

The ten most prominent cryptocurrencies other than bitcoin are as follows

<https://www.investopedia.com/tech/most-important-cryptocurrencies-other-than-bitcoin/>

The most prominent design goal of the ETHash algorithm created for *Ethereum - ETH* is to be ASIC resistant, preferring to accommodate GPU-based mining.

<https://www.cudominer.com/kb/how-does-the-ethash-algorithm-work/>

The algorithm uses a large dataset that is periodically regenerated, and slowly grows over time. It fits into the VRAM of a modern GPU, where VRAM is a special type of RAM that is responsible for holding on to information that the GPU needs. It enhances performance by allowing the GPU to fetch info and beam it to your monitor. This element of the operation is derived from the Dagger algorithm and the large varying dataset contributes to ASIC resistance.

<http://www.hashcash.org/papers/dagger.html>

The other part of ASIC resistance is achieved through a derivative of the “Hashimoto” algorithm, which exerts a lot of memory intensity in terms of random-access reads from the dataset. The inputs from the dataset are run through a derivative of the “Keccak” algorithm to output potential block solutions. This is well suited to the large core counts in modern GPUs.

<https://medium.com/verifyas/why-dagger-hashimoto-for-ethereum-773f0792a689>

<https://coinguides.org/keccak-algorithm-miner-coins/>

Performance versus time in [bitcoinwiki.org](https://en.bitcoin.org/wiki/Performance) ranks CPUs, GPUS, FPGAs, ASIC in increasing order in terms of Throughput in Megahash/sec and Energy Efficiency in Megahash/joule in Table 9.25.

<https://medium.com/fpga-guide/cryptocurrency-mining-why-use-fpga-for-mining-fpga-vs-gpu-vs-asic-explained-5aaa400082b9>

Miners commonly make significant memory clock frequency increases on their GPUs to achieve considerable improvements in Ethash hash rates. Power or temperature limits, along with manually-set high fan speeds are sometimes seen, to offset the associated heat output.

It's common to see custom-built PCs specified with 6 or more high-end GPUs for mining Ethereum, with minimal emphasis on the other components, to get the most cost-effective mining farms.

Mining for cryptocurrency accounts for about 0.2% of global electricity consumption.

<https://www.technologyreview.com/2019/06/12/873/bitcoin-mining-may-be-pumping-out-as-much-as-kansas-city/>

Researchers at MIT and the Technical U. of Munich suggest that Bitcoin mining was consuming 45.8 terawatt-hours of electricity per year as of November 2018. In turn it produced estimated annual emissions of 22-23 megatons of carbon dioxide.

The difficulty of bitcoin mining is discussed below.

<https://en.bitcoin.it/wiki/Difficulty>

Mining difficulty is a measure how difficult it is to find a hash value below the largest value (a 256 bit number).

<https://www.bitcoinmining.com/what-is-bitcoin-mining-difficulty/>

A bit coin profitability calculator is given at this web site.

<http://www.bitcoinx.com/profit/>

Ether(eum) versus bitcoin mining is discussed here.

<https://coincentral.com/ethereum-mining-vs-bitcoin-mining-which-is-more-profitable/>

Performance versus time in [bitcoinwiki.org](https://en.bitcoin.org/wiki/Performance) ranks CPUs, GPUS, FPGAs, ASIC in increasing order in terms of Throughput in Megahash/sec and Energy Efficiency in Megahash/joule in Table 9.25.

<https://medium.com/fpga-guide/cryptocurrency-mining-why-use-fpga-for-mining-fpga-vs-gpu-vs-asic-explained-5aaa400082b9>

TABLE 9.25 CPUs, GPUs, FPGAs, and ASIC efficiency.

Hardware	Adaptability	Availability	Price	User-friendly	Power-efficiency
ASIC		Yes		Yes	#1
CPU	Yes	Yes	Yes	Yes	#3
FPGA	Yes	Yes			#2

According to Iran's Tasmin News Agency, 45,000 mostly ASIC machines were confiscated in Tehran, since these machines were consuming 95 Gigawatts per hour in a country with subsidized energy costs.

<https://www.coindesk.com/bitcoin-miners-seized-electricity-iran>

Chapter 10

Heterogeneous Disk Arrays - HDAs

Introduction to RAID in Section 10.1
Data allocation background in Section 10.2
Balancing disk allocations in Section 10.2.1
Allocation methods considered in this study in Subsection 10.2.2
Analytic justification for HDA in Section 10.3
Improved response time due to HDA in Subsection 10.3.1
The effect of disk failures on response time in Subsection 10.3.2
Shortcut reliability analysis of HDA in Subsection 10.3.3
HDA data allocation experiment setup in Section 10.4
Virtual array allocation requests in Subsection 10.4.1
Estimating Virtual Array - VA widths in normal mode in Subsection 10.4.2
Dealing with load increase in degraded mode in Subsection 10.4.3
Data allocation experiments in Section 10.5
Description of the experiment in Subsection 10.5.1
Assumptions and parameter settings of Subsection 10.5.2
Comparison of allocation methods in Subsection 10.5.3
Sensitivity analysis in Subsection 10.5.4
Clustered RAID5 in Section 10.5.5
Rebuild processing in HDA in Section 10.6
RAID+ data layout based on Latin squares in Section 10.7
Related work in Section 10.8
Utility functions to provision storage systems in Section 10.9
Conclusions in Section 10.10

10.1 Introduction to RAID

The RAID paradigm ensures high availability as a solution to the high cost of downtime due to disk failures (Chen et al., 1994b). In addition the loss of certain data is unacceptable, because it is irreproducible or too costly to reproduce, e.g., *Large Hadron Collider - LHC* experiments are very expensive to set up.

<https://home.cern/science/experiments>

A solution to the latter problem is data backup and data replication across multiple possibly distant sites. Logging of modifications and frequent backups are required to safeguard dynamic data.

Replication and *Erasur e Coding* - EC are the two main methods for dealing with disk failures in RAID (Chen et al., 1994b). Striping partitions large files into strips which are allocated in round-robin manner across the disks of the array with the intention of balancing disk loads. Strips in a row are known as a stripe. RAID0 implements striping, but provides no redundancy.

EC is the more efficient method for large disk arrays due to higher reliability levels at lower redundancy, but requires more disk accesses. In RAID5 (with single rotated parities) four disk accesses are required to update data and parity blocks, which is known as the small write penalty, while two disk accesses are required in mirrored disks. Writing of large files in RAID5/6/7 can be carried out efficiently as full stripe writes across all disks as check strips to be computed are written. Mirroring requires the data to be written twice. Given N disks RAID5 adds $100/N\%$ to disk space requirements of a file, while RAID1 doubles it.

RAID arrays have also been classified as *k-Disk Failure Tolerant - kDFT* in (Thomasian and Blaum, 2009). RAID($4 + k$) arrays are *kDFT* and with *Maximum Distance Separable - MDS* EC incur the minimum level of redundancy: k check disks

to tolerate k disk failures. The redundancy level with N disks is k/N (Thomasian and Blaum, 2009), which is much lower than $100 \times (k - 1)\%$ redundancy for k -way replication.

Blocks on failed disks in RAID5 are reconstructed on demand, but this results in the doubling of read loads on nonfailed disks. Rebuild in RAID5 is the systematic reconstruction of the blocks of a failed disk by XORing the blocks of surviving disks and writing them on a spare disk. Rebuild in RAID5 fails when a second disk fails while rebuild is in progress or an unreadable disk sector known as *Latent Sector Errors - LSEs* is encountered on surviving disks.

RAID5 was extended to RAID6 which is a 2DFT with two check strips per stripe to deal with two disk failures but more importantly with LSEs. RAID6 can be implemented using Reed-Solomon codes and its variations and EVENODD and RDP parity coding (Thomasian and Blaum, 2009). Performance evaluation and examination of open-source erasure coding libraries for storage is carried out in (Plank et al., 2009b). The disk access cost, which is the main consideration in this study, is the same in all three cases and generally $(k + 1)$ block reads and $k + 1$ block writes are required for updating data and check blocks in RAID($4 + k$) arrays.

The *Mean Time to Data Loss - MTDDL* is a single measure that can be used to quantify RAID resilience to failure, as determined by disk failure rate and the frequency of LSEs (Chen et al., 1994b). Another measure of interest is the reliability at the end of intended lifetime, but is difficult to estimate since it requires the distribution of time to failure. That MTDDL is a desirable reliability metric is defended in (Iliadis and Venkatesan, 2015). *Expected Annual Fraction of Data Loss - EAFDL* has been introduced as another metric for data storage reliability (Iliadis and Venkatesan, 2014). *Shortcut Reliability Analysis - SRA* (Thomasian, 2006a) described in Chapter 4, and is used in Section 10.3.3 to compare the reliability of two HDA configurations.

A data center rather than procuring multiple disk arrays with different RAID levels, whose capacity may not be fully utilized, can consolidate data in a fewer HDAs. Fewer RAID arrays entail a lower cost, footprint, and power consumption. HDA suits the cloud storage paradigm which stores data in virtualized pools of storage.

The study reported in (Thomasian and Xu, 2011a, 2016) as well this chapter is concerned with database applications such as OLTP, which entails random disk accesses to small data blocks some of which are updated.

Virtual Arrays - VAs with RAID levels best suited for the respective application data are allocated in the HDA. VAs hold collection of objects such as relational tables and their indices, which have similar performance and availability characteristics. A VA is allocated as multiple *Virtual Disk - VDs* based on VA size and access bandwidth requirements. Fig. 10.1 shows the sharing of disk space in an HDA with three RAID levels. index VDs Virtual Disks

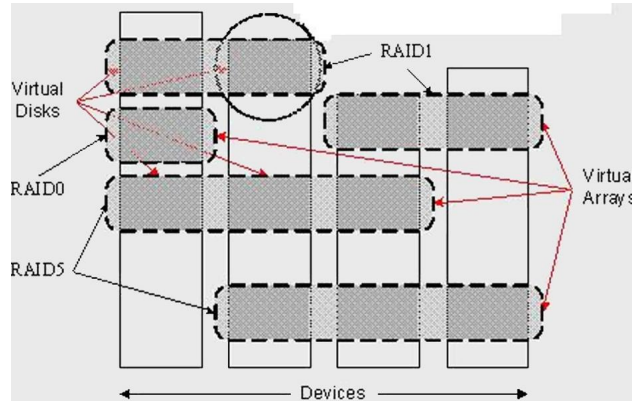


FIGURE 10.1 HDA with four disks and five VAs with different RAID levels: two RAID1 VAs with 2 disks each, two RAID5 VAs with 3 disks each, and a single disk referred to as RAID0, since it has no redundancy.

Only RAID1 and RAID5 are considered in this study, but we specify the analysis to estimate the load to allocate RAID0, RAID0/1, RAID($4 + k$), $k > 1$, and clustered RAID arrays. With a single failed disk in RAID($k + 4$) the read load on surviving disks is increased $k + 1$ -fold and this may result in overload.

Clustered RAID sets the parity group size (G) over which the parity is computed to a smaller value than the number of disks ($G < N$) (Muntz and Lui, 1990b). This has the advantage of reduced read load increase on surviving disks as determined by the declustering ratio $\alpha = (G - 1)/(N - 1) < 1$. A detailed discussion of clustered RAID is given in Chapter 5.

We consider data storage on high performance HDDs, which is a popular storage medium for databases and file systems. While disk capacities and transfer rates are increasing rapidly, the disk access time to randomly placed disk blocks, which

is typical for OLTP, has shown slow improvements due to its mechanical nature. The disk transfer rate for sequential data transfers is quite high due to increased disk recording densities and high RPMs.

HDAs accommodate multiple RAID levels in a single disk array based on application requirements. A RAID level accommodating the most stringent availability requirements for a small subset of datasets to be allocated would require extra disk space and result in an unnecessarily high overhead for updating check blocks for EC and or data replicas for all datasets, since disk bandwidths constitute a bottleneck for random disk accesses. Reliable storage is not necessary for files, which can be reproduced by rerunning the associated application. It is assumed that the HDA's *Disk Array Controller* - DAC can emulate multiple RAID levels on shared disks. This has been demonstrated as the *RAID Engine and Optimizer* - REO in (Kenchammana-Hosekote et al., 2007), which is discussed in Section 10.8.

Reduction in disk bandwidth requirements

As an example of sharing the bandwidth let's assume that $F_6 = 30\%$ of VAs require RAID6, $F_5 = 40\%$ RAID5, and $F_0 = 30\%$ RAID0. Let's assume $\lambda_6 = \lambda_5 = \lambda_0 = \lambda$ for the three VA types and that the disk service time is $\bar{x}$ per disk access. RAID6 arrays incur six, RAID5 four, and RAID0 one disk access for updating disk blocks. Reads incur one access in all three cases. The fraction of read and write accesses is $f_r = 2/3$ and $f_w = 1/3$.

$$\begin{aligned} & F_6(f_r + 6f_w) + F_5(f_r + 4f_w) + F_0(f_r + f_w) = \\ & 0.3\left(\frac{2}{3} + \frac{1}{3} \times 6\right) + 0.4\left(\frac{2}{3} + \frac{1}{3} \times 4\right) + 0.3\left(\frac{2}{3} + \frac{1}{3} \times 1\right) = \\ & 0.3 \times 83 + 0.4 \times 2 + 0.3 = 1.9. \end{aligned}$$

Using the most stringent RAID6 level for all VDs would result in the following disk load $8/3 = 2.67$, so that there is a 30% reduction in disk access bandwidth using HDA, which can be translated into an increased number of allocations.

DRAM and flash memories are more expensive per GB than disks, so that they were initially utilized mostly as disk caches, rather than HDD replacements. *Storage Class Memories* - SCMs are expected to eventually replace HDDs, due to their lower latency, high bandwidth, lower power consumption and footprint according to (Freitas and Wilcke, 2008). DRAM and flash-based databases are discussed in Chapter 8.

The purchase cost of storage hardware, which constitutes a significant fraction of the cost of computer installations, but is exceeded by *Total Cost of Ownership* - TCO especially due to power consumption. In Section 10.8 we discuss several studies that address the issue of optimal design of storage systems.

The allocation of VAs in a disk array requires knowledge of their load requirements. This load can be obtained with various tools, such as IBM's *Resource Measurement Facility* - RMF.

<http://www-03.ibm.com/systems/z/os/zos/features/rmf/>

and *Systems Management Facility* - SMF allows a breakdown by workload, e.g., OLTP versus batch.

http://en.wikipedia.org/wiki/IBM_System_Management_Facilities

Crystal was a tool developed at BGS to extract data for input parameters for its BEST/1 capacity planning tool (Buzen, 1978).

It is difficult to specify VA loads without access to such data. As part of a preliminary study we have used an alternative method of generating a synthetic stream of allocation requests for VAs to compare the effectiveness of data allocation methods for HDA. Each VA allocation request has a given RAID level, storage space requirement, and its load in periods of high activity.

VAs with a high load in one period may have a light load in another. Given such data we carry out VA allocations based on the load in the peak period, while keeping track of disk loads in other periods with a relatively heavy load. This is to ensure that disk utilizations are not exceeded in any period (with a single disk failure).

VA parameters (load and size) are used to determine their width (W), which is the number of *Virtual Disks* - VDs required to materialize them.

The primary goal of the allocation study is to maximize the number of allocated VAs, with the secondary goal of balancing disk bandwidths and capacity utilizations. The former will improve disk response times. Clustered RAID5 can be used to take advantage of the tradeoff between disk capacity and bandwidth utilization in degraded mode. This results in an increased number of allocations when the system is disk bandwidth bound, which is usually the case. Data compression

can be used to save disk space and a later point we discuss clustering as a means of reducing system load with a single disk failure.

The usual optimization methods cannot be applied directly to the problem at hand for two reasons:

1. This is not a straightforward 2-dimensional bin-packing problem, because allocation requests are malleable, e.g., clustered RAID5 provides a tradeoff between VA size and bandwidth or disk utilization requirements (in degraded mode with a single disk failure).
https://en.wikipedia.org/wiki/Bin_packing_problem
2. Allocation requests become available one at a time and are processed in strict FCFS order.

Availability of requests in batches would allow the application of certain optimizations such as those discussed in (Miller, 1974), (Kou and Markowsky, 1977), (Leinberger et al., 1999), and (Pardalos and Resende, 2002).

The heuristic allocation method in this study formulates the equivalent to 2-dimensional bin-packing. Rectangles representing VD bandwidth and space requirement are allocated into a larger rectangle representing disk bandwidth and capacity. Obviously, the smaller the bandwidth and space requirement per VD, the smaller the possibility of wasted bandwidth or capacity. This is at the cost of a small increase in the space requirement for metadata. The allocation of VDs of a VA is treated as a vector-packing problem (Hill, 1994).

We consider single pass data allocation methods based on disk bandwidth requirements and two other methods, which take into account both bandwidth and capacity requirements. The latter methods are more robust than the former when disk capacity can constitute a constraint. In carrying out allocations we take into account the load increase due to disk failures. Since the number of allocations is expected to be constrained by disk bandwidth, rather than disk capacity, we study the effect of adopting the clustered RAID5 paradigm, since it offers a tradeoff between disk bandwidth (in degraded mode) and disk space requirements.

10.2 Data allocation in a Heterogeneous Disk Array - HDA

This section is organized as follows. Section 10.2.1 provides the background for HDA data allocation. Section 10.2.2 specifies the methods used for allocating VDs. Section 10.3 presents simplified analyses of HDA performance and reliability to justify its viability.

10.2.1 Balancing disk allocations

Disk allocation requests can be modeled as vectors in two-dimensions, where the x-axis stands for disk access bandwidth or utilization and the y-axis stands for disk capacity utilization (Hill, 1994). We consider higher performance, smaller capacity SCSI disks, which are more appropriate for OLTP and database applications, rather than high capacity low performance SATA disks, which are used for archival storage (Anderson et al., 2003). This method can be extended to flash memories by taking into account the complexities associated with writing data (Li et al., 2016).

We simplify the discussion by only considering accesses to randomly placed disk blocks. A disk is then represented by the Disk Vector: $\vec{D} = (X, C)$, where X denotes the maximum disk bandwidth for accessing small disk blocks or equivalently disk utilization in the case of sequential accesses and C the disk capacity as shown in Fig. 10.2.

Addressing would be simplified if files are allocated at the same position at all disks, provided identical disks are used. This would benefit large files since data transfer rate and time of same-sized data blocks varies significantly from outer to inner tracks. We do not attempt allocations on the same position at all disks and simplify the data allocation study by keeping track of the remaining unutilized disk capacity, rather the range of disk sectors over which VDs have been allocated. We do not consider the possible inefficiency due to disk space fragmentation, which would result from deallocation of VAs. Disk compaction may be helpful in eliminating unused disk space, but there is the issue of having equal-sized empty spaces at the same location at multiple disks.

Data allocation in two dimensions is modeled as vector addition. The disk bandwidth requirement is determined by assuming FCFS scheduling, i.e., mean seek distance based on uniform accesses to disk blocks, mean latency one half of disk rotation time, plus transfer time which is negligible for accesses to small blocks, but otherwise block size divided by average disk transfer rate. In the case of non-zoned disks with uniform accesses the mean seek distance is one third of the maximum. The mean seek time is determined by the seek time characteristic and is computed for disks with zoning assuming all disk blocks are accessed uniformly (Thomasian et al., 2007a).

We consider a worst case scenario, although disk access times can be reduced using the following methods:

Caching & prefetching eliminate disk access time and are discussed in Chapter 1.

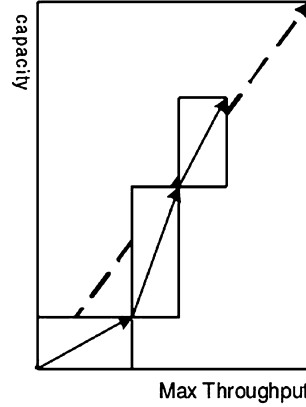


FIGURE 10.2 Allocation vectors for three VDs on a disk drive. The x- and y-axis represent the disk bandwidth in processing accesses to small randomly placed blocks and disk capacity utilization.

Data placement: Placing frequently accessed files according to the organ pipe organization (Wong, 1980) and or utilizing the *Automatic Locality Improving Storage - ALIS* paradigm (Hsu et al., 2005).

Access optimization: *Shortest Access Time First - SATF* disk scheduling policy significantly reduces disk positioning time with respect to FCFS under heavier disk loads (Jacobson and Wilkes, 1991).

The sum of all allocation vectors should not exceed the disk vector $\vec{D} = (X, C)$ in Fig. 10.2. Full disk bandwidth and capacity utilization is attained when the end-point of allocation vectors reaches the end point of $\vec{D}$, but this is unlikely and with increasing disk capacities almost all workloads are expected to be bandwidth-bound. With priority queueing the mean response time of high priority requests is acceptable even when the disk utilization is high.

Heterogeneous disk drives $\vec{D}_n = (X_n, C_n)$, $1 \leq n \leq N$, where N is the number of disks was considered in earlier studies (Thomasian and Han, 2005b), (Thomasian et al., 2005). This will lead to unbalanced disk utilizations and higher disk access times on slower disks.

The resource requirements of the VDs for VA_i are specified as $\vec{d}_i = (x_i, c_i)$, where x_i is the access rate and c_i is the disk capacity requirement per VD. With the initialization $X_n^r = X_n$, $C_n^r = C_n$, $1 \leq n \leq N$, as additional VDs are allocated the residual disk bandwidth and capacity is updated as:

$$X_n^r = X_n^r - x_i \text{ and } C_n^r = C_n^r - c_i.$$

A VD allocation at the n^{th} disk is successful if $X_n^r \geq 0$ and $C_n^r \geq 0$, i.e., both constraints are satisfied.

Disk utilization is a more robust metric than disk bandwidth, since the disk may be involved in processing disk accesses with varying sizes. Let U_n^x and U_n^c denote the current utilization of the bandwidth and capacity of the n^{th} disk in the array. They are initialized as $U_n^x = 0$, $U_n^c = 0$, $1 \leq n \leq N$ and the allocations are taken into account as

$$U_n^x = U_n^x + u_i^x \text{ and } U_n^c = U_n^c + u_i^c,$$

where u_i^x and u_i^c are the utilization of disk access bandwidth and capacity by VA_i . Given that $\mathcal{J}_n$ denotes the set of VD allocations at the n^{th} disk we should have:

$$U_n^x = \sum_{j \in \mathcal{J}_n} u_j^x < 1, \quad U_n^c = \sum_{j \in \mathcal{J}_n} u_j^c < 1.$$

The workload is considered disk bandwidth/utilization or capacity bound depending on which limit is reached first.

10.2.2 Allocation methods considered in this study

Five bin-pack methods which also take into account disk capacities are considered for allocating VDs on N disks in this study. VDs belonging to the same VA need to be allocated on different disks to allow recovery according to the RAID paradigm.

Round-Robin: Allocate VDs beyond the last allocated VD with wrap-around, i.e., modulo the number of disks N . This method equalizes the allocated disk space, provided different VAs utilize the same sizes for VDs, but does not balance bandwidth or disk utilizations.

Random: Allocate VDs on disk drives randomly, but ensure that VDs belonging to the same VA are allocated at different disks. Stop when an unsuccessful allocation is encountered. Experiments showed that retrying does not result in a significant increase in VA allocations. This method does not ensure balanced capacity and bandwidth utilizations.

Best-Fit: Select the disk with the minimum remaining bandwidth or maximum disk utilization, which can fit this request.

First-Fit: Disks are considered in increasing order of their indices, i.e., starting with the first disk. A VD is allocated on the first disk that can hold it.

Worst-Fit: Allocate requests on disks with minimum bandwidth utilization, provided that both constraints are satisfied.

Given the bandwidth and capacity utilizations for the n^{th} disk U_n^x and U_n^c given by equations above, two more sophisticated allocation methods minimize the following objective functions:

Min-F1: Allocate VDs on disk drives such that the maximum disk bandwidth or capacity utilization at the disks is minimized. Note that without the latter constraint this is equivalent to Worst-Fit.

$$F_1 = \max_{1 \leq n \leq N} \{U_n^x, \beta U_n^c\}, \quad (10.1)$$

Min-F2: Minimize the weighted sum of the variances of disk access bandwidths and capacity utilizations.

$$F_2 = \text{Var}_{1 \leq n \leq N} (U_n^x) + \beta \text{Var}_{1 \leq n \leq N} (U_n^c). \quad (10.2)$$

$\text{Var}(\cdot)$ is the variance computed over N disks.

$\beta \geq 0$ is an emphasis factor for disk capacity utilization.

10.3 Analytic justification for HDA

In a single array allocating $n < N$ disks to RAID1 and $N - n$ disks to RAID5 in a single disk array. This approach provides less flexibility than HDA, because the mixture and resource demands for different RAID levels are not known a priori, so that this may result in an unequal disk utilization. There is potential for improving disk access time by sharing disk space, as demonstrated in Section 10.3.1. There is a small reduction in reliability, however, as shown in Section sec:reliability.

10.3.1 Improved response time due to HDA

The discussion is simplified by assuming all disk requests are reads to randomly placed disk blocks with exponential service times and Poisson arrivals, so that the processing of disk requests can be modeled as an M/M/1 queueing system (Kleinrock, 1975). Given an arrival rate λ and mean disk service time $\bar{x}_d$, the mean disk response time is $R = \bar{x}_d / (1 - \rho)$, where $\rho = \lambda \bar{x}_d$ is the disk utilization factor.

The arrival rate of read requests to RAID1 and RAID5 arrays is Λ_{R1} and Λ_{R5} , respectively. Two disk array configurations are considered with $N = 8$ disks:

C₁: Vertical Disk Partitioning: We assume $n = 2$ disks are dedicated to RAID1 and $N - n = 6$ disks to RAID5, which is 75% of disk array capacity.

C₂: Horizontal Disk Partitioning: This is the HDA paradigm with RAID1 allocated over $N = 8$ disks occupying $2/N$ or 25% of the space on each disk. RAID5 in this case requires $(5/8) \times (1 + \frac{1}{8}) \approx 0.7$ of disk array capacity.

The mean response time for RAID1 with C₁ allocation is: $R_{R1} = \bar{x}_d / (1 - \rho)$, where $\rho = (\Lambda_{R1}/2) \bar{x}_d$. The mean response time for RAID5 with C₁ is $R_{R5} = \bar{x}_d / (1 - \rho)$, where $\rho = (\Lambda_{R5}/6) \bar{x}_{disk}$.

For C₂ there are two components to disk utilization, since RAID1 and RAID5 disk arrays share disk space: $\rho' = [(\Lambda_{R1} + \Lambda_{R5})/N] \bar{x}_d$, so that $R'_{R1} = R'_{R5} = \bar{x}_d / (1 - \rho')$. If $\rho' < \rho$ or $\Lambda_{R5} < (N/2 - 1) \Lambda_{R1}$ then $R'_{R1} < R_{R1}$. For $N = 8$ C₂ will improve the RAID1 response time with respect to C₁ for $\Lambda_{R1} > \Lambda_{R5}/3$.

RAID1 response times in C₂ can be improved by processing its accesses at a higher nonpreemptive priority than RAID5 accesses. $R''_{R1} = \bar{x}_d / (1 - \rho'')$, where $\rho'' = (\Lambda_{R1}/N) \bar{x}_d$, since only the disk utilization due to RAID1 accesses affects accesses to mirrored disks (Kleinrock, 1976). For C₁ with $\rho = (\Lambda_{R1}/2) \bar{x}_{disk} = 0.8$, $R_{R1} = \bar{x}_d / (1 - 0.8) = 5 \bar{x}_d$ and for C₂ $\rho'' = n \times 0.8 / N = 0.2$ and $R''_{R1} = \bar{x}_d / (1 - 0.2) = 1.25 \bar{x}_d$, i.e., a 4-fold improvement.

10.3.2 The effect of disk failures on response time

We use the HDA configuration in Fig. 10.3 to illustrate the increases in mean disk response times as a result of a single disk failure. The HDA has 12 RAID1 arrays specified by the first letters of the alphabet. All disk requests are reads with mean disk service time $\bar{x}_d$ and an arrival rate to each VA is 2λ , so that the disk utilization per VD is $\rho_{VD} = \lambda \overline{vex}_d$. Disk utilizations in normal mode are then $\rho = 3\rho_{VD}$ and the mean response times $R = \bar{x}_d / (1 - \rho)$.

1	2	3	4	5	6	7	8
A_1	A_2	B_1	B_2	C_1	C_2	D_1	D_2
H_2	E_1	E_2	F_1	F_2	G_1	G_2	H_1
L_1	L_2	I_1	I_2	J_1	J_2	K_1	K_2

FIGURE 10.3 An HDA with $N = 8$ disks and twelve VAs with mirrored disk configurations.

We assume that read requests are routed uniformly to the two VDs of RAID1 VAs even when the load of the two disks on which they reside are not balanced. When the third disk fails then the mean response time at VAs unaffected by this failure obviously does not change. Due to the unavailability of E_2 , the load on E_1 will be doubled and hence $\rho_2 = 4\rho_{VD}$. This increased load affects the mean response time of VA arrays (A_1, A_2) and (L_1, L_2) . Due to the unavailability of B_1 and I_1 the load on B_2 and I_2 will be doubled, so that $\rho_4 = 5\rho_{VD}$. This increase affects the mean response time for VA (F_1, F_2) .

The mean response times at the disk level are $R_1 = R_5 = R_6 = R_7 = R_8 = R$, $R_2 = \bar{x}_d / (1 - \rho_2)$, $R_4 = \bar{x}_d / (1 - \rho_4)$, so that the mean response time in degraded mode is

$$R_{degraded} = [R_1 + (4/3)R_2 + (5/3)R_4 + R_5 + R_6 + R_7 + R_8]/7.$$

Given $\rho_{VD} = 0.1$ the mean response time in normal mode is:

$$R_{norm} = (10/7)\bar{x}_d \approx 1.43\bar{x}_d.$$

In degraded mode

$$R_{degraded} = [1 + (4/3)(10/6) + (5/3)(10/5) + 4]/7 \approx 1.5\bar{x}_d,$$

which is a small increase in mean disk response time.

The mean response times for VAs in degraded mode are: $R_{A_1, A_2} = R_{L_1, L_2} = (R + R_2)/2$, $R_{E_1} = R_2$, $R_{B_2} = R_{I_2} = R_4$, $R_{F_1, F_2} = (R_4 + R)/2$.

10.3.3 Shortcut reliability analysis of HDA

We next utilize the *Shortcut Reliability Analysis - SRA* in (Thomasian, 2006a) to compare the reliability of C_1 and C_2 configurations. The reliability of RAID1/0 with p disk pairs is $R_{R1}(p) = [1 - (1 - r)^2]^p$, while the reliability of a RAID5 with w disks is $R_{R5}(w) = r^w + w(1 - r)r^{w-1}$. We consider highly reliable disks, so that $r = 1 - \varepsilon$ with $\varepsilon \ll 1$.

$$R_{C_1} = R_{R1}(1)R_{R5}(6) = [1 - (1 - r)^2] \times [r^6 + 6r^5(1 - r)] \approx 1 - 16\varepsilon^2.$$

$$R_{C_2} = R_{R1}(4)R_{R5}(8) = \{[1 - (1 - r)^2]^4\} \times [r^8 + 8r^7(1 - r)] \approx 1 - 32\varepsilon^2.$$

It follows that the C_1 is more reliable than C_2 , since it tolerates two disk failures, as long as they do not affect the same array: RAID1 or RAID5.

10.4 HDA data allocation experiment setup

This section is organized as follows. In Subsection 10.4.1 we specify the characteristics of the VA allocation requests. In Subsections 10.4.2 and 10.4.3 we provide analytic expressions for system load per VA in normal and degraded modes. The former load estimates are used to obtain the widths of the VAs and the latter to estimate disk bandwidth requirements.

10.4.1 Virtual array allocation requests

We are concerned with allocating VAs in an HDA with N identical disks. Requests to allocate VAs become available one at a time and are indexed consecutively. VAs are allocated in the order they become available, e.g., VA_{i+1} is allocated after VA_i . Each VA is associated with a certain RAID level, which cannot be reassigned. A VA allocation fails when not all of its VD's can be allocated due to bandwidth or capacity constraints. The goal is to maximize the number of VA allocations, while striving for balanced disk loads.

The VA load is determined by the anticipated arrival rate of disk requests and their mean service time. Disk I/O trace analysis generated by OLTP workloads has shown that disk accesses are to small randomly placed disk blocks, see e.g. (Ramakrishnan et al., 1992), while query processing and data mining applications generate sequential accesses to data files. In the following discussion VAs are specified as follows:

RAID1 versus RAID5 ratio: The RAID level of a VA is specified as $\ell = 1$ for RAID1 and $\ell = 5$ for RAID5. A fraction f_1 of VA allocation requests are for RAID1 and a fraction $f_5 = 1 - f_1$ are for RAID5. These fractions may follow an observed pattern or are based on RAID level assignments based on workload characteristics as in (Anderson et al., 2002), (Thomasian and Xu, 2011a).

Virtual Array - VA Sizes: The size V_i of VA_i is generated according to a RAID level dependent distribution, as if the data was to be allocated with no redundancy as a RAID0 array. The effective sizes of VA_i , denoted by V'_i are determined below and utilized in the allocation process. For a discussion of metadata required for HDA (with heterogeneous disks) the reader is referred to (Thomasian and Han, 2005b). Given the critical nature of metadata, multiple copies of it are stored on disk for the sake of higher reliability as in the case of the IRON file system described in (Prabhakaran et al., 2005). The space requirements for metadata are small with respect to the VA sizes and are ignored in this study.

For RAID1 VAs the effective size is $V'_i = 2V_i$, since the disk space requirement is doubled. For RAID0/5/6/7 VAs which are k DFT arrays with $0 \leq k \leq 3$, the effective size of the VAs is $V'_i = V_i(1 + \frac{k}{n-k})$, where n is the VA width. The VA width W_i is determined based on the size of the VA and its load in normal mode (see Eq. (10.8) in Section 10.4.2). V'_i in clustered RAID is discussed in Section 10.5.5. The same strip size may be adopted in all allocations to reduce external disk space fragmentation, while increasing internal fragmentation, but this rule was not followed in this study.

VA loads: The service time of disk accesses to small randomly placed disk blocks is dominated by positioning time, while sequential accesses are dominated by disk transfer time. The arrival rate of requests to VA_i is $\Lambda_i = V_i \kappa_\ell$, where κ_ℓ is the I/O intensity per GB for RAID level ℓ . The per GB rate for RAID1 is set to be higher than RAID5 ($\kappa_1 > \kappa_5$), since RAID1 arrays tend to be used for high performance OLTP applications. These parameters are also varied in the experimental study to emulate I/O-bound, balanced, and capacity-bound workloads.

VA loads may vary significantly in different periods. The load in the heaviest period is used in data allocation, while the loads in different periods are taken into account to ensure that the disk bandwidth in any period is not exceeded. Allocations are made in degraded mode with one disk failure to ensure system is not overloaded.

Read:Write Ratio: The fraction of read and write requests to VA_i is denoted by f_r and $f_w = 1 - f_r$, respectively. Read (resp. write) requests to the disks are misses from a read cache (resp. destages from an NVRAM cache, which allows destaging to be deferred (Treiber and Menon, 1995)). We ignore the possibility of repeated updates of dirty blocks and the potential locality for destages. In effect we postulate a scenario where writes are carried out immediately, but with a lower priority than reads.

Disk access time is determined by HDD characteristics. An analytic method to compute the mean disk service time for random disk accesses with FCFS scheduling in zoned HDDs is reported in (Thomasian et al., 2007a). The *Shortest Access Time First - SATF* disk scheduling policy currently implemented in most disk drives significantly improves disk access time and increases attainable bandwidth. FCFS scheduling is considered in this study, since it is a pessimistic upper bound to disk service times and can be estimated simply. Disk service times can be adjusted according to disk utilization. The effect of number of outstanding disk requests on disk service time is considered in Chapter 3.

10.4.2 Estimating virtual array widths in normal mode

Logical read and write requests are processed as *Single Read - SR* and *Single Write - SW* accesses. The mean service times for SR, SW, and also *Read-Modify-Write - RMW* disk accesses are denoted by $\bar{x}_{SR}$, $\bar{x}_{SW}$, and $\bar{x}_{RMW}$, respectively. These are derived in Section 10.5.2.

The load across the W_i VDs of RAID0 is the arrival rate of requests to the array multiplied by the mean disk service time divided by the number of VDs (W_i), which yields the contribution of each VD to disk utilization.

$$\rho_i = \frac{\Lambda_i}{W_i} [f_r \bar{x}_{SR} + f_w \bar{x}_{SW}]. \quad (10.3)$$

The width of RAID1/0 VAs is set to $W_i = 2$ in this study, since VA sizes for such arrays are selected to be relatively small. In a more realistic study fixed size VD's for all allocations could be used at the cost of internal fragmentation.

The read load per VD assumes that read requests are distributed evenly over the two disks. The per VD utilization is the sum of the loads for read and write requests.

$$\rho_i = \frac{\Lambda_i}{2} \left[\frac{f_r}{2} \bar{x}_{SR} + f_w \bar{x}_{SW} \right]. \quad (10.4)$$

The cost equations for $(k + 1)$ -way replication are simple extensions of RAID1 costs. In the case of RAID5/6/7 the updating of data and check blocks is accomplished via RMW requests, as opposed to the *ReConstruct Write - RCW* method (Thomasian, 2005a). Alternatives for updating small data blocks with the RMW method are:

Method A Given a modified data block (d_{new}), the DAC - Disk Array Controller first reads the old data block (d_{old}), if it is not already cached, and computes $d_{diff} = d_{old} \oplus d_{new}$. The check blocks for the i^{th} block of a code can be computed for the i^{th} strip as follows (Plank and Huang, 2013):

$$p_{diff} = d_{diff}, q_{diff} = \alpha^i d_{diff}, r_{diff} = \alpha^{2i} d_{diff}.$$

The old check blocks $p_{old}, q_{old}, r_{old}$ are also read from disk to compute:

$$p_{new} = p_{old} \oplus p_{diff}, q_{new} = q_{old} \oplus q_{diff}, r_{new} = r_{old} \oplus r_{diff},$$

which in addition to d_{new} are written to appropriate disks as SW requests. A logical write request requires $k + 1 = 2/3/4$ SRs to read the old blocks and as many SWs to write them. The resulting VA load is:

$$\rho'_i = \Lambda_i [f_r \bar{x}_{SR} + (k + 1) f_w (\bar{x}_{SR} + \bar{x}_{SW})]. \quad (10.5)$$

Method B This method postulates an XOR capability at the disks to compute d_{diff} and modified check blocks. The DAC sends the modified data block d_{new} to the data disk to compute d_{diff} , which is sent via the DAC to the check disks, specifying the identity of the check block (P, Q, and R). The check disks read p_{old}, q_{old} , and r_{old} , compute p_{new}, q_{new} , and r_{new} , by applying appropriate coefficients and write them after one disk rotation, so that $\bar{x}_{RMW} = \bar{x}_{SR} + T_{rot}$, where T_{rot} is the disk rotation time. The VA load using this method is:

$$\rho'_i = \Lambda_i [f_r \bar{x}_{SR} + (k + 1) f_w \bar{x}_{RMW}]. \quad (10.6)$$

Method C The DAC after reading d_{diff} from the data disk computes p_{diff}, q_{diff} , and r_{diff} , which are then sent to appropriate RAID5/6/7 disks to be XORed with p_{old}, q_{old} , and r_{old} check blocks. Similarly to Method B an XOR capability is required at the disks.

Method D *Disk Architecture for Composite Operations - DACO* reduces the cost of processing RMW accesses via a specialized read/write head proposed in (Li and Shu, 2010), where the write head is at a very short distance after the read head on a single disk actuator (Jacob et al., 2008) allowing sufficient delay for a just read data block to be XORed with the difference block, before it is overwritten. DACO result in a great improvement in processing RMW requests, since $\bar{x}_{RMW} \approx \bar{x}_{SW}$, but is not considered further because of difficulties associated with its implementation.

Method A is less susceptible to data loss, termed write holes, than Method B and C, since the DAC logs its operations to recover from failures, such as power loss. Methods B and C incur the same disk load. Method A substitutes the disk rotation time associated with Methods B and C with an SW request (seek and latency). By issuing of SW requests after SR requests are completed and if there are no intervening disk accesses then an extra seek is not required at check disks, but almost a full disk rotation is incurred, so that the first three methods incur approximately the same cost:

$$\bar{x}_{SR} + \bar{x}_{SW} \approx \bar{x}_{RMW}.$$

The VA width to implement RAID0/5/6/7 arrays is determined by the maximum disk utilization allowed per VD (on each disk) (ρ_{max}) and a maximum capacity constraint per VD (v_{max}), which is expressed as a fraction of all disk capacities. W_i is the maximum of these widths, less than N :

$$W_i^{bandwidth} = \lceil \rho'_i / \rho_{max} \rceil, \quad W_i^{capacity} = \lceil V_i / v_{max} \rceil. \quad (10.7)$$

$$W_i = \min \left[\max \left(W_i^{\text{bandwidth}}, W_i^{\text{capacity}} \right), N \right]. \quad (10.8)$$

Limiting the per VD bandwidth utilization to a smaller fraction of disk bandwidth spreads the load across more disks. This also reduces the possibility of disk overload when VA loads are underestimated. Allocating a RAID5 VA across all N disks maximizes parallelism for read accesses and minimizes the space dedicated to check blocks, but has the disadvantage that if a single disk fails then the read load at remaining $N - 1$ disks is doubled. While for $W < N$ the per VD load increase is higher than the case $W = N$, but fewer disk drives are affected by a disk failure. HDA is effectively a clustered RAID since the VA widths $W < N$. VA widths are reported in conjunction with allocation studies of clustered RAID in Section 10.5.5.

Sequential accesses can be handled similarly to accesses to small blocks with the difference that the seek time for sequential requests is negligible compared to transfer time and given *Zero Latency Reads - ZLR* no latency is incurred. The preemptive resume policy is applied to large sequential accesses to ensure an acceptable response time for small accesses. Preemptions should be applied sparingly since they incur extra positioning time.

Large sequential updates can be processed efficiently as full-stripe writes, if all the strips in a stripe are updated. Otherwise if the majority of the strips in a stripe are updated, then the *ReConstruct Write - RCW* method discussed in (Thomasian, 2005a) is more appropriate. For the sake of brevity we do not discuss accesses to large variable sized block.

10.4.3 Dealing with load increase in degraded mode

Data allocations is carried out at the peak load in degraded mode with a single failed disk among the disks of the current VA, but also all previous VAs allocated before it. If there are several periods with very high load allocation the effect of allocating the VAS to all such periods should be checked to ensure a successful allocation.

With uniform routing of read requests on the surviving disk in *Basic Mirroring - BM* is doubled when one of two disks fails, while the write load remains the same. Both disks for VA_i are allocated with increased loads. We use RAID1/F1 to specify RAID1 arrays with a single failure.

$$\rho^{RAID1/F1} = \Lambda_i(f_r \bar{x}_{SR} + f_w \bar{x}_{SW}). \quad (10.9)$$

Alternative RAID1 data organizations where the data on each disk is distributed over multiple disks resulting in a more balanced load are described in (Thomasian and Xu, 2008), (Thomasian and Tang, 2012) and Chapter 4.

Disk utilizations for read and write requests in clustered RAID5 with a single failure (RAID5/F1) and a parity group size $G \leq W$ are given below based on the discussion in (Thomasian, 2005b). The parity group size is the number of strips protected by a single parity strip in RAID5. Using $W = W_i$ to simplify the notation, the arrival rate per disk for VA_i is $\lambda_i = \Lambda_i / W$.

$$\rho_{read}^{CRAID5/F1} = \lambda_i r_i (1 + \alpha) \bar{x}_{SR}, \quad \alpha = \frac{G - 1}{W - 1} \leq 1. \quad (10.10)$$

$$\rho_{write}^{CRAID5/F1} = \frac{\lambda_i w_i}{W - 1} [2(W - 2) \bar{x}_{RMW} + 2 \bar{x}_{SW} + (G - 2) \bar{x}_{SR}]. \quad (10.11)$$

10.5 Data allocation experiments

Section 10.5.1 describes the experimental method used to evaluate the data allocation methods considered in this study. Section 10.5.2 describes the assumptions made in the experimental study. Section 10.5.3 summarizes the results of the experimental study to compare the allocation methods. Sensitivity of results with respect to parameter settings to determine the width of VAs and the parameter β for the Min-F1 and Min-F2 method are evaluated in Section 10.5.4. We then assess the effect of clustered RAID5 on the number of allocations in Section 10.5.5.

10.5.1 Description of the experiment

Experiments to study the efficiency of VA allocations in HDA are reported in this section. VA attributes are generated one at a time from given distributions using pseudo-random numbers. Allocation requests are processed in *First-In, First-Out - FIFO* order by different VA allocation policies. The goal of the study is to determine the allocation policy that maximizes

the number of allocated VAs, while resulting in a balanced load. The allocation requests are repeated multiple times and the average over the runs is reported. The same sequence of pseudo-random numbers is used to generate the same allocation requests for different allocation policies.

In addition to the number HDA disks and their characteristics, the input parameters for the experiment are:

- (i) The fraction of RAID1 (f_1) versus RAID5 ($f_5 = 1 - f_1$) allocation requests in the input stream.
- (ii) The size and bandwidth requirements for VAs, where the former are sampled from different distributions for RAID1 and RAID5.
- (iii) The fraction of read (f_r) and write requests ($f_w = 1 - f_r$). The requests are to small randomly placed disk blocks whose transfer time is negligible, so that the disk service time is determined by positioning time, which is the sum of mean seek time and mean latency, set to one half of rotation time.

The experiment to estimate the efficiency of VA allocation methods can be specified as follows.

1. Initialization: $I_{R1} = I_{R5} = 0$ and the VA index $i = 1$.
2. Determine the RAID level ℓ for VA_i according to the fraction of RAID1 ($\ell = 1$) versus RAID5 ($\ell = 5$) VAs by generating a uniformly distributed random number $u \in (0, 1)$. If $u \leq f_1$ then VA_i is a RAID1 and otherwise it is a RAID5.
3. Generate the size of VA_i , which is denoted by V_i , based on the size distribution for RAID level ℓ .
4. Compute the access rate to VA_i which is proportional to VA size: $\Lambda_i = \kappa_\ell V_i$, where κ_ℓ depends on ℓ and the workload category: bandwidth-bound, capacity-bound, or balanced.
5. For RAID5 allocation requests determine the VA load according to Section 10.4.2. Use thresholds for maximum disk bandwidth or rather load (ρ_{max}) and capacity (v_{max}) using Eq. (10.8) to determine width W_i . The discussion is shortened by setting $W_i = 2$ for RAID1 VAs, but the method used for RAID5 can be used to determine the width of RAID1/0 arrays.
6. Calculate VA loads in degraded mode using the expressions in Section 10.4.3.
7. Determine if all VDs of VA_i can be allocated successfully by satisfying disk space and bandwidth constraints, the latter in *degraded mode*. Ensure that the load in no other high load period is exceeded. If so set $I_{R1\&R5} = I_{R1} + I_{R5}$ and exit.
8. Set $I_{R\ell} = I_{R\ell} + 1$ based on the RAID level ℓ of the allocated VA_i . Increment the disk bandwidth and capacity utilization or decrement the residual bandwidth and capacity for all allocation periods being considered, set $i = i + 1$, and return to Step 2.

This experiment was repeated for normal mode operation using the disk utilizations in Section 10.4.2 for comparison purposes. Allocation methods are ranked by the number of allocated VAs, which is specified as $I_{R1\&R5} = I_{R1} + I_{R5}$, where I_{R1} and I_{R5} are counters for allocated RAID1 and RAID5 VAs.

VA loads may vary significantly over time, so that we have VAs sharing disk space, which reach their peak load in different periods. The allocation should be carried out using the load in a period, but ensuring that the disk load is not exceeded in other periods.

10.5.2 Assumptions and parameter settings

We use the following parameter settings to run experiments. A disk array with $N = 12$ IBM-Hitachi Global Storage Technologies - HGST-Western Digital Corp. - WDC Ultrastar 18ES disk drives.

<https://www.cnet.com/products/ibm-ultrastar-18es-hard-drive-9-1-gb-ultra-wide-scsi-series/>

Such disks are characterized using the DIXTRAC package at CMU's Parallel Data Laboratory - PDL,

<http://www.pdl.cmu.edu/DiskSim/diskspecs.shtml>

and are summarized in Table 10.1.

The size of the nonredundant data stored in each VA is assumed to be exponentially distributed with a mean $\bar{V}_1 = 256$ MB for RAID1 and $\bar{V}_5 = 768$ MB for RAID5 VAs. The RAID sizes obtained by sampling are rounded to multiples of 256 KB, which can be placed in strips of this size.

HDA introduces an additional layer in accessing data, since there is a mapping for virtual to physical disk addresses using the metadata specified in (Thomasian and Han, 2005b). At a lower level there are directories specifying the tables and indexes of relational DBMSs. Caching the metadata will alleviate extra disk accesses.

We set $v_{max} = 1/50$ of the capacity of all disks and $\rho_{max} = 1/20$ the bandwidth of each disk. We set $\beta = 1$ based on the sensitivity analysis in Section 10.5.4 for the Min-F1 and Min-F2 allocation methods. The access rate for RAID1 VAs is set to be ten times higher than the rate for RAID5 VAs, i.e., $\kappa_1 = 10\kappa_5$. While RAID1 arrays are three times smaller on the average, the rate of accesses to RAID1 arrays is set to be 3.3 times higher than RAID5 on the average. We consider three VA workloads with different access rates for RAID5 VAs in normal mode: (1) Bandwidth-Bound: $\kappa_5 = 8.5$, (2) Balanced: $\kappa_5 = 3.3$, (3) Capacity-Bound: $\kappa_5 = 2.1$ accesses/second per GB.

TABLE 10.1 Characteristics of IBM Ultrastar 18ES disk drives. The mean seek and transfer time are computed assuming random accesses to disk blocks taking into account disk zoning.

Model	IBM DNES-309170W
Disk Capacity	$C = 9.17$ GB
Number of cylinders/zones	11,474/11
Tracks per cylinder	5
Rotations per minute	7200 RPM
Disk rotation time	$T_{rot} = 8.33$ ms
Latency	$\bar{x}_{lat} \approx T_{rot}/2$
Head settling time	$T_h = 0.14$ ms
Mean seek time	$\bar{x}_{seek} \approx 7.16$ ms
Mean transfer time per blocks	$\bar{x}_{xfer} \approx 0.16$ ms
Mean read time ($\bar{x}_{SR}$)	$\bar{x}_{seek} + \bar{x}_{lat} + \bar{x}_{xfer}$
Read time for RAID5	$\bar{x}_{SR} \approx 11.49$ ms
Disk write time	$\bar{x}_{SW} = \bar{x}_{SR} + T_h$
RMW time for RAID5	$\bar{x}_{RMW} = \bar{x}_{SR} + T_{rot}$

We consider three cases for the composition of read and write requests: (i) all requests are reads ($f_r = 1$), (ii) 75% of requests are reads ($f_r = 0.75$), (iii) 50% of requests are reads ($f_r = 0.5$). Parameters used in the experimental study are summarized in Table 10.2.

TABLE 10.2 Parameters used in the experimental study.

Number of disks	$N = 12$
RAID1/5 allocations fraction	$f_1, f_5 = 1 - f_1$
Per GB rate to RAID5/RAID1	$\kappa_5, \kappa_1 = 10\kappa_5$
Mean RAID1 size	$\bar{V}_1 = 256$ MB
Mean RAID5 size	$\bar{V}_5 = 768$ MB
Load for VA_i	ρ'_i
Fraction of reads/writes	$f_r/F_w = 1 - f_r$
Maximum bandwidth per VD	$\rho_{max} = 1/20 = 0.005$
Maximum capacity per VD	$v_{max} = 1/50 = 4\%$ (all disks)
RAID5 VA width	W
Parity group size	G
Declustering ratio	$\alpha = \frac{G-1}{W-1}$
Capacity emphasis factor	$\beta \geq 0$

10.5.3 Comparison of allocation methods

The number of allocated RAID1 and RAID5 VAs is the key performance metric in evaluating data allocation methods. The number of allocated VAs in the two categories follows the fractions of requests in the input stream, since the allocations are carried out in FCFS order.

Tables 10.3, 10.4, and 10.5 are based on 100%, 75%, and 50% read requests in normal mode, while Tables 10.6, 10.7, and 10.8 are based on the same fractions of read requests in degraded mode. The three cases: bandwidth-bound, balanced, and capacity-bound workloads are considered. The same pseudo-random sequence was used in experiments to generate the same synthetic sequence of allocation requests for a fair comparison.

The allocation experiments were repeated one hundred times, so as to obtain the average number of allocated VAs over these iterations. Increasing the number of iterations yielded comparable results. The key metric in the comparison is the

TABLE 10.3 Comparison of the allocation methods with RAID5:RAID1 = 3:1, and $r = 1$ in normal mode.

Method	Bandwidth-Bound				Balanced				Capacity-Bound			
	Best	Allocations			Best	Allocations			Best	Allocations		
		R1	R5	R1&R5		R1	R5	R1&R5		R1	R5	R1 & R5
Min-F1	80	47.1	142.4	189.6	90	57.2	173.2	230.4	90	69.2	206.7	275.9
Min-F2	78	46.4	142.3	188.6	76	56.2	172.4	228.6	86	68.2	205.6	273.8
Worst-Fit	66	45.2	137.6	182.8	15	48.1	150.4	198.5	9	61.3	183.6	244.9
Best-Fit	63	41.1	123.1	164.2	12	44.3	135.3	179.6	8	57.4	178.3	235.7
Round-Robin	17	33.3	102.3	135.6	10	44.4	133.4	177.8	8	63.2	190.1	253.3
First-Fit	13	33.1	100.4	133.5	0	35.1	103.0	138.1	0	38.1	115.3	153.5
Random	13	29.1	90.3	119.4	6	33.0	107.3	140.3	2	57.2	172.2	229.3

TABLE 10.4 Comparison of the allocation methods with RAID5:RAID1 = 3:1, and $r = 0.75$ in normal mode.

Method	Bandwidth-Bound				Balanced				Capacity-Bound			
	Best	Allocations			Best	Allocations			Best	Allocations		
		R1	R5	R1&R5		R1	R5	R1&R5		R1	R5	R1 & R5
Min-F1	78	41.4	125.1	166.5	87	50.2	152.1	202.3	89	68.5	204.7	273.2
Min-F2	75	40.7	124.9	165.7	73	49.3	151.4	200.7	85	67.5	203.6	271.1
Worst-Fit	63	39.7	120.8	160.5	14	42.3	132.1	174.4	9	60.7	181.7	242.4
Best-Fit	61	36.1	108.1	144.2	12	38.9	118.8	157.8	8	56.8	176.6	233.4
Round-Robin	16	29.2	89.8	119.1	10	39.0	117.2	156.2	8	62.6	188.2	250.8
First-Fit	13	29.1	88.2	117.2	0	30.8	90.5	121.3	0	37.7	114.2	151.9
Random	12	25.5	79.3	104.8	6	29.0	94.3	123.2	2	56.6	170.5	227.1

TABLE 10.5 Comparison of the allocation methods with RAID5:RAID1 = 3:1, and $r = 0.5$ in normal mode.

Method	Bandwidth-Bound				Balanced				Capacity-Bound			
	Best	Allocations			Best	Allocations			Best	Allocations		
		R1	R5	R1&R5		R1	R5	R1&R5		R1	R5	R1 & R5
Min-F1	75	32.4	97.9	130.3	84	39.3	119.1	158.4	89	68.2	203.7	271.9
Min-F2	73	31.9	97.8	129.7	70	38.6	118.5	157.2	85	67.2	202.6	269.8
Worst-Fit	61	31.1	94.6	125.7	14	33.1	103.4	136.5	9	60.4	180.8	241.2
Best-Fit	59	28.3	84.6	112.9	12	30.5	93.0	123.5	8	56.6	175.7	232.3
Round-Robin	16	22.9	70.3	93.2	10	30.5	91.7	122.3	8	62.3	187.3	249.5
First-Fit	12	22.8	69.0	91.8	0	24.1	70.8	95.0	0	37.6	113.6	151.2
Random	12	20.0	62.1	82.1	6	22.7	73.8	96.5	2	56.3	169.6	225.9

number of times an allocation method performed best by allocating the most VAs. It follows from the tables that there were many ties, with more than one method providing the best allocation. The following conclusions can be drawn from the tables.

1. The number of VAs allocated in normal mode is almost double the number of those in degraded mode for $r = 1$. This is because the load on surviving disks in degraded mode almost doubles the load in normal mode for both RAID1 and RAID5 arrays.
2. Min-F1 and Min-F2 are consistently the best methods in terms of the number of allocations in all configurations, although Min-F1 outperforms Min-F2. For given workloads and disk characteristics both disk bandwidth and capacity need to be considered for robust resource allocation.

TABLE 10.6 Comparison of the allocation methods with RAID5:RAID1 = 3:1, and $r = 1$ in degraded mode.

Method	Bandwidth-Bound				Balanced				Capacity-Bound			
	Allocations				Allocations				Allocations			
	Best	R1	R5	R1&R5	Best	R1	R5	R1&R5	Best	R1	R5	R1 & R5
Min-F1	71	23.2	72.0	95.2	87	28.2	84.6	112.8	87	34.2	105.3	139.5
Min-F2	71	23.2	72.0	95.2	73	27.3	84.6	111.9	83	33.4	102.3	135.7
Worst-Fit	56	21.8	71.6	93.4	14	23.7	75.2	98.9	9	29.7	93.4	123.1
Best-Fit	54	20.3	68.7	89.0	12	22.0	70.1	92.1	8	27.6	90.2	117.8
Round-Robin	19	16.4	50.6	67.0	10	21.1	66.4	87.5	8	30.8	95.1	125.9
First-Fit	10	16.0	48.5	64.5	0	18.7	56.8	75.5	0	19.3	59.7	79.0
Random	10	13.7	47.2	60.9	6	20.2	61.7	81.9	2	28.0	86.9	114.9

TABLE 10.7 Comparison of the allocation methods with RAID5:RAID1 = 3:1 and $r = 0.75$ in degraded mode.

Method	Bandwidth-Bound				Balanced				Capacity-Bound			
	Allocations				Allocations				Allocations			
	Best	R1	R5	R1&R5	Best	R1	R5	R1&R5	Best	R1	R5	R1 & R5
Min-F1	82	17.3	64.0	81.3	98	23.2	72.6	95.8	80	33.0	104.0	137.0
Min-F2	75	17.3	64.0	81.3	69	23.1	73.5	96.6	83	32.1	101.0	133.1
Worst-Fit	66	16.5	59.6	76.1	10	21.3	66.9	88.2	7	29.0	92.0	121.0
Best-Fit	63	15.0	58.8	73.8	9	19.6	62.4	82.0	6	27.1	89.0	116.1
Round-Robin	12	12.8	45.6	58.4	11	16.8	56.6	73.4	6	30.0	92.1	122.1
First-Fit	16	13.7	44.7	58.5	0	17.9	53.8	71.7	0	18.1	58.0	76.1
Random	12	14.0	38.7	52.7	7	17.8	53.6	71.4	2	27.0	85.0	112.1

TABLE 10.8 Comparison of the allocation methods with RAID5:RAID1 = 3:1 and $r = 0.5$ in degraded mode.

Method	Bandwidth-Bound				Balanced				Capacity-Bound			
	Allocations				Allocations				Allocations			
	Best	R1	R5	R1&R5	Best	R1	R5	R1&R5	Best	R1	R5	R1 & R5
Min-F1	78	15.3	57.6	72.9	98	21.7	66.6	88.3	84	33.0	104.0	137.0
Min-F2	75	15.3	55.7	71.1	71	20.9	67.4	88.3	90	32.1	101.0	133.1
Worst-Fit	63	14.7	51.9	66.6	6	19.9	63.8	83.7	3	29.0	92.0	121.0
Best-Fit	61	13.3	51.2	64.5	5	18.2	59.5	77.7	3	27.1	89.0	116.1
Round-Robin	13	10.7	39.4	50.1	10	14.0	47.5	68.7	6	30.0	92.1	122.1
First-Fit	16	12.0	39.6	51.6	0	16.9	51.8	61.5	0	18.1	58.0	76.1
Random	12	11.2	32.9	44.1	3	15.2	47.2	62.4	2	26.1	83.2	109.3

3. Worst-Fit and Best-Fit methods provide good allocations for bandwidth-bound workloads in normal and degraded modes. The number of allocations is lower than Min-F1 and Min-F2, however.
4. First-Fit, Random, and Round-Robin are the worst among all allocation methods considered, since they do not optimize bandwidth or capacity allocation.

Table 10.9 provides the comparison of average bandwidth and capacity utilization for all disks and number of RAID1 and RAID5 VAs allocated with RAID5:RAID1 = 3:1, $r = 1$ in degraded mode for Min-F1, Min-F2, and the less efficient Round-Robin allocation methods, for bandwidth-bound workload. The following observations can be made:

1. Min-F1 and Min-F2 attain high disk bandwidth utilization for bandwidth-bound and balanced workloads, while disk capacity utilizations are low for bandwidth-bound workloads, since as expected the bandwidth bound is reached first.
2. For capacity-bound allocations Min-F1 and Min-F2 minimize the variation of disk utilizations and this results in an increase in the number of VA allocations.

3. Poor allocation methods have high standard deviations for disk bandwidth and capacity utilization.

TABLE 10.9 Disk utilizations after allocations with RAID5:RAID1 = 3:1, $r = 1$, and bandwidth-bound workload in degraded mode.

Method	Bandwidth		Capacity		No. of VAs		
	Avg	Std	Avg	Std	R1	R5	Total
Min-F1	90.7	1.6	51.3	2.6	23	72	95
Min-F2	90.4	2.3	51.3	2.2	23	72	95
RR	63.8	20.1	36.8	8.4	16	51	67

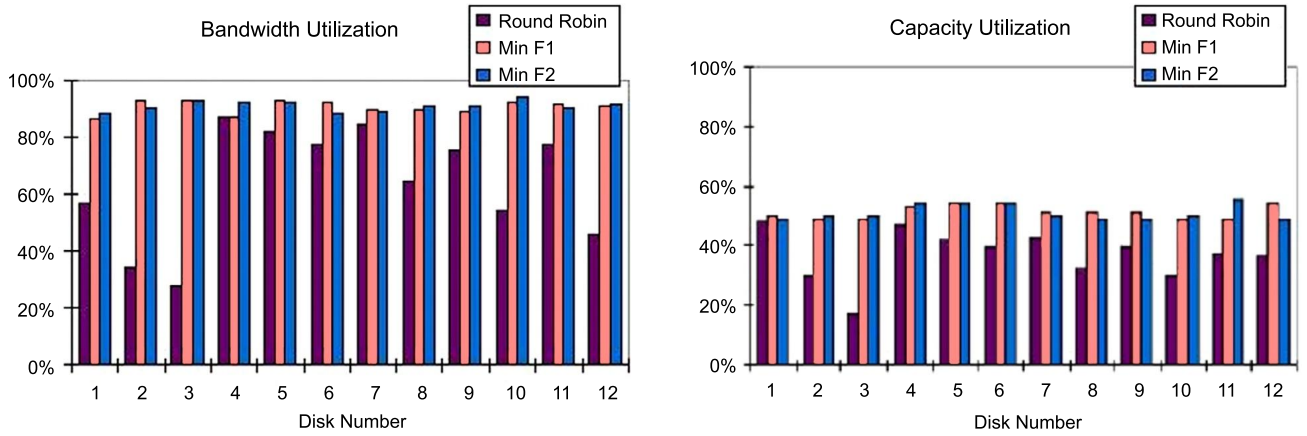


FIGURE 10.4 Disk bandwidth and capacity utilizations with RAID5:RAID1 = 3:1, $r = 1$, and bandwidth-bound workload in degraded mode.

Disk bandwidth and capacity utilizations with bandwidth-bound workloads for Min-F1, Min-F2, and Round-Robin allocation methods are shown in Fig. 10.4. It follows that for Min-F1 and Min-F2 all disks are saturated roughly equally as far as disk bandwidth or capacity or both are concerned for bandwidth-bound, capacity-bound, and balanced workloads, respectively. Little or no improvement in allocations is therefore expected with more sophisticated allocation methods for the postulated workload.

10.5.4 Sensitivity analysis

In Section 10.5.4.1 we study the sensitivity of the number of allocations to β , which is used in conjunction with Min-F1 and Min-F2 allocation methods. In Section 10.5.4.2 we study the sensitivity of allocation results with respect to the granularity of allocations: ρ_{max} and v_{max} .

10.5.4.1 Sensitivity to β in Min-F1 and Min-F2

The parameter β in Eqs. (10.1) and (10.2) emphasizes capacity versus bandwidth utilization for Min-F1 and Min-F2 allocation methods. Higher values of β assign more weight to disk capacity utilization, so that more balanced disk capacity utilizations are achieved.

The following parameters are used in experiments: $N = 12$ disks, RAID5:RAID1 = 3:1, the fraction of read requests is $r = 1$. The VA sizes are exponentially distributed with $\bar{V}_1 = 256$ MB for RAID1 and $\bar{V}_5 = 768$ MB for RAID5. The access rates for RAID1 are ten times the access rates of RAID5: ($\kappa_1 = 10\kappa_5$). v_{max} is set to $1/50^{th}$ of the capacity of all disks and ρ_{max} is set to $1/20$ of the bandwidth of a single disk. We consider bandwidth-bound, balanced, and capacity-bound workloads.

The following conclusions can be drawn from Table 10.10.

1. For balanced and capacity-bound workloads the number of VAs allocated increases more than 10% when β is increased from zero to one, but there is little improvement for $\beta > 1$.
2. The effect on the bandwidth-bound workload is also less significant. Note that a large value for β would result in the effect of bandwidth utilizations being ignored. We have used $\beta = 1.0$ in the experimental results reported in this study.

TABLE 10.10 Sensitivity of β in Min-F1/Min-F2 with RAID5:RAID1 = 3:1, and $r = 1$ in degraded mode.

β	Bandwidth-Bound	Balanced	Capacity-Bound
≥ 1	116/116	127/129	139/139
0.5	108/116	123/129	133/139
0	108/108	108/108	123/123

10.5.4.2 The effect of ρ_{max} and v_{max} on allocations

The parameters ρ_{max} and v_{max} determine the maximum bandwidth (or disk bandwidth utilization) and the maximum capacity allocated per VD on a disk drive, respectively. ρ_{max} is the more critical parameter, since it restricts the bandwidth utilization of a VA on a disk. Because VA loads are estimates, restricting the load per VD reduces the possibility of a disk becoming overloaded when the VA load was underestimated in the allocation phase. The smaller the values of these two parameters, the more VDs are required for the allocation and vice-versa.

The following parameters are used in the experiment: $N = 12$, RAID5:RAID1 = 1:0, i.e., only RAID5 requests, and $r = 1$, since the load increase in degraded mode is highest for read requests. The VA sizes are exponentially distributed with $\bar{V}_1 = 256$ MB for RAID1 and $\bar{V}_2 = 768$ MB for RAID5. The allocation method Min-F1 is used in this experiment, because it is one of the two best methods as shown in Section 10.2 with β set to one. We consider the bandwidth-bound, balanced, and capacity-bound workloads.

TABLE 10.11 The effects of ρ_{max} (given in the first column) and v_{max} (the percentages in the second row) on the number of VAs allocations in degraded mode.

ρ_{max}	Bandwidth-bound			Balanced			Capacity-bound		
	4%	2%	1%	4%	2%	1%	4%	2%	1%
1/10	46	46	46	74	93	123	88	104	135
1/20	49	49	49	77	96	125	88	104	135
1/40	50	50	50	82	99	128	88	104	135

The following conclusions can be drawn from Table 10.11. ρ_{max} affects the number of VDs for both balanced and bandwidth-bound workloads, but has no effect on capacity-bound workloads, since the width is determined by v_{max} rather than ρ_{max} in this case. If we consider $\rho_{max} = 1/20$ and a capacity-bound workload, the increase in the number of allocations can be explained by the fact that for $v_{max} = 1/25$, $1/50$, and $1/100$ the mean disk capacity utilization was determined to be 96.5%, 98.5%, and 99.2%, respectively, while the standard deviation of capacity utilization was quite small in all cases. The improvement in allocation efficiency is due to the reduction in the sizes of allocation requests. Incidentally, the average RAID5 width was determined to be 4.7, 5.6, 7.3 for $\rho_{max} = 1/10$, $\rho_{max} = 1/20$, $\rho_{max} = 1/40$, respectively.

10.5.5 Clustered RAID5

Clustered RAID5 is a method to attain a reduction in disk load increase when operating in degraded mode. In the case of HDA it can be used to attain more allocations for bandwidth-bound workloads when a sufficient disk capacity is available.

Given the width of VAs $W \leq N$ and parity group size $G \leq W$, we study the effect of the declustering ratio: $\alpha = (G - 1)/(W - 1)$ on the number of allocated VAs with the clustered RAID5 organization. Note that given that $W < N$ HDA is already a clustered RAID, so that $G < W$ makes it “doubly” clustered. This experiment could have been carried out with read and write requests, since disk loads for RAID5 with and without clustering are given in (Thomasian, 2005b), but this discussion is simplified by postulating only read accesses, especially since the increase in disk loads in degraded mode is highest in this case and can be simply expressed as $1 + \alpha$, where $\alpha = (G - 1)/(W - 1)$. Note that a clustered RAID5 with $G = 2$ corresponds to RAID1 and one disk access is required to reconstruct a block on the failed disk.

For a disk drive with capacity C with a workload consisting of accesses to randomly placed small data blocks, the *Capacity over Bandwidth Ratio* - CBR is given as: $\gamma_d = C\bar{x}_{SR}$, where $[\bar{x}_{SR}]^{-1}$ is the maximum disk bandwidth for SR requests. The *Bandwidth over Space Ratio* - BSR is the inverse of CBR and has been defined in the context of storage for multimedia applications (Sitaram and Dan, 2000). The capacity of IBM 18ES disk drives considered in this study is $C = 9.17$ GB and their maximum bandwidth is $1000/\bar{x}_{SR} \approx 87$ accesses per second, so that $\gamma_d = 9.17/87 = 0.105$

GB/second. This corresponds to the slope of the diagonal in Fig. 10.2. It is desirable for $\gamma_c(\alpha)$ for RAID5 to be close to the disk CBR: $\gamma_c(\alpha) \approx \gamma_d$, such that successive allocation vectors follow the diagonal in Fig. 10.2.

The RAID5 access rate per GB with a bandwidth-bound workload is $\kappa_5 = 8.5$ accesses per second. The capacity bandwidth ratio for clustered RAID5 is given in Table 10.12. For RAID5 without clustering $\gamma_c(1) < \gamma_d$, but $\gamma_c(0.25) \approx \gamma_d$. To clarify note that the bandwidth 15.22 for $\alpha = 1$ is twice the bandwidth in normal mode, i.e., 7.61. The load increase for $\alpha = 0.125$ is $7.61 \times 1.125 = 8.56$ GB. Without clustering the size of the data portion of the VA is $V_D = 0.87 \times 11/12 \approx 0.8$ GBs. For $\alpha = 0.125$ and $G = 1 + \alpha(W - 1) \approx 2.375$. The size of the clustered VA is then $V(0.125) = 0.8 \times (1 + 1/2.375) \approx 1.14$ GB.

TABLE 10.12 Change of capacity/bandwidth ratio (γ_c) for VAs with 8.5 accesses per second per GB versus the declustering ratio α in clustered RAID5 with all read requests ($r = 1$).

α	G	Capacity	Bandwidth	$\gamma_c(\alpha)$
0.125	2	1.14	8.56	0.133
0.25	4	1.01	9.51	0.106
0.375	5	0.96	10.46	0.091
0.5	7	0.92	11.42	0.081
0.625	8	0.90	12.37	0.073
0.75	9	0.89	13.32	0.067
0.875	11	0.88	14.27	0.061
1	12	0.87	15.22	0.057

TABLE 10.13 Number of RAID5 allocations with bandwidth-bound workload (8.5 access per second per GB) in degraded mode with $N = 12$ disks. The (c)/(b) next to the number of allocations indicates the capacity/bandwidth reaches the limit first.

α	$f_r = 1.0$	$f_r = 0.75$	$f_r = 0.5$
0.25	108.6 (c)	69.4 (b)	50.6 (b)
0.5	92.0 (b)	61.7 (b)	46.4 (b)
0.75	78.8 (b)	55.5 (b)	42.8 (b)

TABLE 10.14 Number of RAID5 allocations with bandwidth-bound workload with $\kappa_5 = 8.5$ accesses per sec/GB in degraded mode with $N = 12$ disks. The (c)/(b) next to the number of allocations means the capacity/bandwidth reaches the limit first.

α	$f_r = 1.0$	$f_r = 0.75$	$f_r = 0.5$
0.25	94.6 (c)	74.8 (b)	53.4 (b)
0.5	106.3 (c)	70.7 (b)	51.3 (b)
0.75	103.9 (b)	67.0 (b)	49.4 (b)

We make allocation experiments with $\alpha = 0.25, 0.5, 0.75$. The following conclusions can be drawn from Tables 10.13 and 10.14,

1. If the disk bandwidth is the bottleneck resource then more allocations can be made with smaller values of α , since the bandwidth requirement per VA is lower in degraded mode.
2. As α decreases the system reverts from bandwidth to capacity bound. This is because as $G = \alpha(N - 1) + 1$ gets smaller, the disk capacity overhead for RAID5 ($1/G$) gets larger. The number of allocations increases, until the disk capacity becomes the bottleneck resource.
3. If disk capacity is the bottleneck resource then more allocations can be made with larger values of α , because the capacity overhead per VA is lowest for $G = N$.

TABLE 10.15 Comparison of the relative number of allocations with and without clustering in degraded mode with respect to setting $W = N$.

	RAID5 (width = W)		Clustering+ W		
	Rel #	Avg Width	Rel #	Avg Width	Avg γ
$f_r = 1.0$	1.45	5.8	1.57	3.9	0.106
$f_r = 0.75$	1.30	5.7	1.49	2.03	0.086
$f_r = 0.50$	1.22	5.6	1.34	2.8	0.06

Finally, we use the general configuration for all experiments and a bandwidth-bound workload with the Min-F1 method and $\beta = 1$. For each allocated VA α is set such that VAs CBR γ_c is close to γ_d . Once the value of α is determined, the new width will be calculated accordingly. It can be concluded from Table 10.15 that clustered RAID5 arrays increase the number of VA allocations significantly when all requests are reads ($r = 1$), but clustering has less effect for higher fractions of write requests.

10.6 Rebuild processing in HDA

Rebuild processing is a systematic reconstruction of the contents of a failed disk, which is accomplished by reading the contents of surviving disks to reconstruct missing data on the failed disk. Rebuild processing in HDA can be carried out using one of the following methods:

1. For dedicated sparing a spare VD (of the same size as other VDs) can be provided or allocated on demand, as long as the new VD is on a separate disk.
2. Spare areas can be allocated as in distributed sparing (Thomasian and Menon, 1997).
3. When utilizing higher redundancy levels the overwriting of check strips via restriping can be used as described in (Rao et al., 2011). RAID7 is converted to RAID6, RAID6 to RAID5, RAID5 to RAID0 as disks fail and check strips are overwritten.
4. Parity sparing by combining same-sized RAID5 arrays, but in this case the VDs of the two VAs need to have been allocated on disjoint disks.

A meaningful option to start rebuild processing for a VA using the *Disk-Oriented Rebuild - DOR* method when there are no external disk requests for that disk (Holland et al., 1994). Prioritized accessing of disks was presented in Section 10.3.1, where RAID1/0 VAs were accessed at a higher priority than RAID5 arrays. A similar paradigm is applicable in rebuild processing.

Given adequate bus bandwidth and processing power the rebuilding of multiple VAs can proceed concurrently. VA rebuilding can be prioritized according to various criteria:

1. VAs in more critical states first, e.g., RAID5 before RAID6 both with single disk failures.
2. The importance of application accessing VA, more specifically the cost of downtime as discussed in (Patterson, 2002).
3. Reducing the load on heavily loaded disks after a VA is rebuild.

The rebuild process in an HDA with 48 strips, 23 of which are not utilized, so that disk array is almost half empty, is shown in Fig. 10.5. There are four VAs which can be specified by their RAID level and coordinates of their strips. For example, **A** is a RAID5 with five VDs, on disks $D_2 : 6$ and a spare disk. Each VD consists of two strips, which are placed on the same disk. The position of the parity strips follows the default left symmetric layout.

Consider the failure of D_2 in Fig. 10.3, which holds two VDs: (a_1, a_6) and c_2 belonging to **A** and **C**, respectively. The rebuilding of a_1 on $s_a 0$, which can be accomplished by accessing and XORing appropriate steps:

$$s_1 \leftarrow a_2 \oplus a_3 \oplus a_4 \oplus a_5 \text{ and}$$

$$s_3 \leftarrow a_7 \oplus a_8 \oplus a_9 \oplus a_1$$

c_2 can be reconstructed as $s_4 \leftarrow c_1$. Rebuild can be prioritized based in the criticality of the situation, e.g., two disk failures in RAID6 are as critical as one disk failure in RAID5.

The circumstances under which it is beneficial to repair **A** before **C** depends on the arrival rate of requests to the two arrays. To simplify the discussion we assume that all requests are reads to same size blocks. The excess load on a_1, a_3 , and a_4 is $3 \times \Lambda_A/4)\bar{x}$, while the load on c_1 is $\Lambda_C\bar{x}$. Rebuilding **A** should be prioritized with respect to **C** if $0.75\Lambda_A > \Lambda_C$.

In Fig. 10.5 if both **A** and **D** have a single disk failure, rebuilding **A** should be expedited, since **D** can tolerate an additional disk failure, while **A** cannot.

D_1	D_2	D_3	D_4	D_5	D_6	D_7	D_8	D_9	D_{10}	D_{11}	D_{12}
$x_{1,1}$	$x_{1,2}$	$x_{1,3}$	$x_{1,4}$	$x_{1,5}$	$x_{1,6}$	$x_{1,7}$	$x_{1,8}$	$x_{1,9}$	$x_{1,10}$	$x_{1,11}$	$x_{1,12}$
$x_{2,1}$	a_1	a_2	a_3	a_4	a_5	s_1	$x_{2,8}$	$x_{2,9}$	b_1	b_2	s_2
$x_{2,1}$	a_6	a_7	a_8	a_9	a_{10}	s_3	$x_{2,8}$	$x_{2,9}$	$x_{2,10}$	x_{11}	$x_{2,12}$
c_1	c_2	s_4	$x_{4,4}$	$x_{4,5}$	d_1	d_2	d_3	d_4	d_5	d_6	s_5

FIGURE 10.5 HDA disk array with $N = 12$ disks and equal strip sizes. **A** is a RAID5 VA with five VDs, two strips each, **D** is a RAID6 VA with six VDs. s_1 and s_3 are spare strips dedicated to **A**, s_2 is a spare strip dedicated to **B**, s_4 is a spare strip dedicated to **C** and s_5 is a check strip dedicated to **D**. There are two RAID1 VAs **B** and **C** with s_3 and s_4 as their spares. Empty space is specified with $x_{i,j}$, where i is the row and j column or disk number.

10.7 RAID+ data layout based on Latin squares

Existing RAID solutions partition large disk enclosures so that each RAID group uses its own disks exclusively. This achieves good performance isolation across underlying disk groups, at the cost of disk under-utilization and slow RAID reconstruction from disk failures.

RAID+ proposed by Weimin Zheng et al. at Tsinghua U. (Zhang et al., 2018b, 2020) spreads both normal I/O and reconstruction workloads to a larger disk pool in a balanced manner.

Unlike systems conducting randomized placement, RAID+ employs deterministic addressing enabled by mutually orthogonal Latin squares, based on which it constructs 3-D data templates mapping a logical data volume to uniformly distributed disk blocks across all disks.

https://en.wikipedia.org/wiki/Latin_square

While the total read/write volume remains unchanged, with or without disk failures, many more disk drives participate in data service and disk reconstruction. The evaluation of a 60-drive disk enclosure using both synthetic and real-world workloads shows that RAID+ significantly speeds up data recovery while delivering better normal I/O performance and higher multi-tenant system throughput.

Deterministic mapping of k -block stripes to n -disk pool, where k decoupled from n RAID+ has the following properties:

- (1) Fault tolerance of RAID, no blocks in stripe placed on same disk.
- (2) Guaranteed load balance, uniform distribution of data blocks, for both application I/O and recovery.
- (3) Ease of maintenance: deterministic, computable addressing.

RAID+ is achieved by using *Mutually Orthogonal Latin Squares - MOLS* (Laywine and Mullen, 1998), (Keedwell and Denes, 2015).

A Latin square of order n is an $n \times n$ array filled with n different symbols, so that *each symbol occurs exactly once in each row and column*.

Two Latin squares orthogonal, if when superimposed, each of n^2 ordered pairs appears exactly once as shown in Fig. 10.6.

1	2	3	4	5
2	3	4	5	1
3	4	5	1	2
4	5	1	2	3
5	1	2	3	4

+

1	2	3	4	5
3	4	5	1	2
5	1	2	3	4
2	3	4	5	1
4	5	1	2	3

⇒

1, 1	2, 2	3, 4	4, 4	5, 5
2, 3	3, 4	4, 5	5, 1	1, 2
3, 5	4, 1	5, 2	1, 3	2, 4
4, 2	5, 3	1, 4	2, 5	3, 1
5, 4	1, 5	2, 1	3, 2	4, 3

FIGURE 10.6 Two orthogonal squares superimposed yielding a Latin square with ordered pairs.

Latin squares are discussed in most books on combinatorics. RAID+: Guaranteed Uniform Distribution using 3D templates has the following advantages:

- (1) deterministic addressing,
- (2) uniform data distribution,
- (3) flexible adaptive setup,
- (4) improved spatial locality.

The description of the data layout based on Latin squares is quite lengthy and is omitted here.

For comparison, the authors evaluate two RAID organizations utilizing large disk pools, both of which build eight 6+1 RAID-5 array with 64 KB strip unit sizes, consuming 56 disks with the last 3 reserved as hot spares. RAID-5C divides each array into multiple 1 GB extents and concatenates them in around-robin manner, while RAID-50 stripes across these

8 arrays at block size of 12 MB ($2\text{MB} \times 6$). Also evaluated are two randomized data placement schemes, RAIDR and RAIDH, where the latter uses CRUSH style hashing based random mapping (Weil et al., 2006b).
<https://en.google-info.org/22263235/1/jenkins-hash-function.html#article>

Both place blocks within each stripe to different disks while aiming for balanced data distribution to all n disks.

To assign a block to a disk, RAIDR utilizes the system random number generator (with system time as seed) and is therefore nondeterministic. RAIDH, instead, uses the Jenkins hash function adopted by systems such as Ceph (Weil et al., 2006a).

RAID+ is also a flat organization with n disks, where two data templates are used to carve space uniformly from all disks. Each template is designed by “stacking” a sequence of $k \times n$ MOLs. Cells at the same location through the k layers then form a k -width data stripe. Given such a set of k Latin-squares, one can easily compute the locations of any stripe’s blocks, on a k -subset of the n drives. The mathematical properties of mutually orthogonal Latin squares guarantee the uniform data distribution on all working drives for normal or single-failure recovery accesses. Since data distribution by each template is always uniform, user scan hosts different RAID organizations within the same n -drive disk pool, such as RAID5 or RAID6.

When a disk failure occurs, both random placement and RAID+ allow all surviving disks to equally participate in reconstruction, cutting theoretical RAID rebuild time to $k/(n-1)$ of that of conventional RAID. Also, hot spares are optional with these organizations. RAID+’s deterministic and uniform data placement enables it to achieve perfect load balance within smaller address extents and retain spatial locality, both significant advantages (crucial to sequential access and RAID rebuild) over random schemes.

Metrics used in the study are:

- (1) Recovery speed (online and offline), interference to application performance.
- (2) Data re-distribution after failures.
- (3) Normal performance: synthetic and application, single- and multi-workload.

Our evaluation with a 60-drive disk enclosure using both synthetic/real work-loads shows that RAID+ significantly speeds up data recovery while delivering better normal I/O performance and higher multi-tenant system throughput.

10.8 Related work

HDA related studies are presented in approximate chronological order.

That implementation of multiple RAID levels in one disk array was perhaps first exemplified by HP’s AutoRAID described in (Wilkes et al., 1996). It is argued in (Voigt, 1998) that AutoRAID was developed in response to the following dilemma: “Selection of RAID levels and stripe characteristics based on application characteristics was and is a black art”.

AutoRAID consisted of a RAID1 array at the higher level, which is accessed first, and a RAID5 with *Log-Structured Array - LSA* organization (Menon, 1995) at the lower level. LSA is an extension of the concept of *Log-structured File System - LFS* (Rosenblum and Ousterhout, 1992) to RAID5 disk arrays (Menon, 1995). The inclusion property for multilevel CPU caches (Hennessy and Patterson, 2017) is not followed in AutoRAID, i.e., RAID1 data is not stored at the RAID5 level, since this would result in wasted disk space and incur the overhead of updating the same data at both RAID levels. Stated differently data written to the higher RAID1 level is not written through to RAID5.

AutoRAID is initially populated with data formatted as RAID1/0, but as disk space is exhausted, mirrored data is demoted to RAID5/LSA format, via full stripes so that check strips are computed on the fly (Chen et al., 1994b), (Thomasian and Blaum, 2009). When the size of the active working set exceeds the space available for RAID1, a potential problem is excessive disk activity which may result in thrashing.

A succession of tools to create self-configuring and self-managing storage systems at HP Research: *Forum* (Borowsky et al., 2007), *Minerva* is a suite of tools for automated design of large storage systems described in (Alvarez et al., 2001), which culminated in *Disk Array Designer - DAD* in (Anderson et al., 2005). Ergastulum renamed Minerva selects a RAID level for the set of stores or allocation requests, and utilizes a complex solver to assign stores onto disk arrays for the attribute mapping problem (Golding et al., 1995a), (Shriver, 1999).

According to (Borowsky et al., 2007) specification of storage systems by means of user-oriented QoS attributes is essential to ease of use and efficient resource utilization. Attribute-managed storage systems hide details of the underlying storage systems through virtual store abstractions — units of storage with quality of service guarantees. The mapping of virtual stores onto physical storage devices can be optimized to achieve high level goals such as balancing system performance against total system cost. The feasibility of this approach with a prototype matching engine called Forum. The following advantages of the attribute-based approach are listed.

- Automated mappings can be made in minutes rather than the weeks needed for manual configuration; because the storage system knows the goals of its clients, it can adapt to changes in physical configurations automatically;
- the optimization engine can be directed to meet different goals, such as how to balance system performance against total system cost;
- the same optimization technology that can generate initial assignments of workload to devices can be used to ask “what if?” questions to explore the effects of future changes in configuration or load;
- once the system is running, the QoS goals for the virtual stores can be monitored, and adjustments made in the assignment.

The main Forum solver components are as follows:

- device performance models for utilization, throughput, and response time
- QoS constraints that express things like “you cannot put more data on a device than it can hold”,
- the device utilization cannot be greater than 1;
- sets of device parameters that describe the capabilities of available devices;
- workload model which captures variability in requirements of an application;
- a search engine, which explores the potential space of assignments.

A RAID level selection tool was developed in the context of DAD (Anderson et al., 2002). Given the descriptions of the workload and the capabilities of storage devices it specifies an assignment. It is assumed that disk arrays provide *Logical UNits - LUNs*, which may be RAID5 or RAID1 arrays.

An SCSI LUN can be addressed with a combination of the controller ID, the target ID, a disk ID, and occasionally the slice ID. The identifications (IDs) in a UNIX OS are generally joined as one word. A typical example is the address c1t2d3s4. This refers to controller 1, target 2, disk 3 and slice 4. Full device addresses are as follows: c-part: controller ID of host bus adapter t-part: target ID classifying SCSI target on the bus d-part: disk ID classifying LUN on the target s-part: slice ID classifying exact slice on the disk.

<https://www.techopedia.com/definition/321/logical-unit-number-lun>

The workload description is in the form of stores (chunks of data) and *streams*, which are accesses to the stores. Given the RAID level Minerva carries out the following steps: (i) array allocation, (ii) array configuration, and (iii) store assignment. DAD uses best-fit bin-packing with randomization and backtracking. It can be utilized to improve the current storage configuration as part of an automated storage management system. The data allocation experiments in (Anderson et al., 2005) are based on workload characterization in several studies at HP.

Oracle’s *Stripe and Mirror Everywhere - SAME* paradigm is obviously not the best storage scheme in all cases as noted in (Anderson et al., 2005). RAID level selection: RAID1/0 versus RAID5 is addressed in (Anderson et al., 2002). *Tagging-based* and *solver-based* approaches are considered. Tagging may be based on rule’s of thumb, i.e., *rule-based* or *model based*, which selects the RAID level minimizing the I/O rate. The solver-based approach has two variants: partially adaptive and fully adaptive. The former does not allow RAID levels to be reassigned and the latter does.

RAID1 is the appropriate RAID level for small write requests, since it incurs less overhead than RAID arrays with erasure coding. RAID1/0 is a generalization of BM - Basic Mirroring with multiple pairs of mirrored disks. When a disk fails, the read load of its pair is doubled, but this is not so for data layouts, where data on one disk is replicated on more than one disk (Thomasian and Xu, 2008). Clustered RAID5 reduces the increase in disk load upon a disk failure.

The reading / writing of large files can be done in parallel in RAID5, while this is not so in RAID1, unless the data is striped as in RAID1/0, which is described in Chapter 4. An analytic approach to select RAID levels based on access characteristics of allocation requests is proposed in (Thomasian and Xu, 2011a). RAID5 postulated in this study allows in-place updating of small blocks and full stripe updating of large blocks as in the case of RAID5/LSA.

The RAID level yielding the lower load: RAID1 or RAID5, is selected. The discussion of HDA in (Thomasian and Xu, 2011c), (Thomasian and Xu, 2016) does not rely on (Thomasian and Xu, 2011a) and assumes that the RAID level for VAs is specified a priori.

In the case of EMC’s Centera file system for archival storage small files are stored as RAID1 and large files as RAID5. <http://www.emc.com/products/family/emc-centera-family.htm>

Overload control, especially in degraded mode, can be applied to throttle access to VAs for less critical applications to ensure satisfactory performance for more critical ones. There is the more general issue of attaining *Quality of Service - QoS* for different applications sharing disk space according to (Borowsky et al., 2007).

RAID5 disk arrays may utilize heterogeneous disks, as in the case of a cluster of workstations. A block-distribution algorithm that can be used to build disk arrays from heterogeneous disks is provided in (Cortes and Labarta, 2001). It

is also shown that arrays using this algorithm are able to achieve a higher performance than when blocks are distributed assuming that all disks have the lowest common speed.

Heterogeneity at both the RAID and disk level was investigated in (Thomasian and Han, 2005b). A two level allocation scheme is used with space allocated to RAID1 and RAID5 “containers” with prespecified widths. More space in each category is allocated when it is exhausted. Ds of VAs tagged as RAID1 or RAID5 with prespecified loads are allocated in these containers taking into account disk utilizations, as well as container capacity. The number of VDs associated with each VA and their capacity and bandwidth requirements are prespecified. This study in (Thomasian and Han, 2005b) was extended in (Thomasian et al., 2005), while this chapter is based on (Thomasian and Xu, 2011c), (Thomasian and Xu, 2016).

As an example of storage heterogeneity a RAID1 array consisting of an ordinary disk and *MicroElectroMechanical System - MEMS* storage was considered at HP (Uysal et al., 2003). According to (Carley et al., 2000) authored at CMU’s *Data Storage Systems Center - DSSC* MEMS storage technology is about ten times more expensive than HDDs, but much faster. MEMS storage is discussed in some detail in Chapter 2. Simulators replaying I/O traces of real applications are used to show that replacing disks with MEMS-based storage improves performance dramatically, but with a cost performance ratio several times better than conventional arrays.

The URSA minor and self* storage projects at PDL/CMU

Given that no single encoding scheme or fault model is optimal for all data, URSA Minor project at PDL/CMU is a versatile storage system that allows matching of storage devices to access patterns, reliability requirements, and cost goals on a per-data item basis (Abd-El-Malek et al., 2005). Different data types can share a scalable storage infrastructure and but still enjoy specialized choices, rather than suffering from “one size fits all”. Experiments with Ursa Minor showed 2-3-fold performance benefits versus a single general configuration.

The origin of the URSA minor project can be traced back to self*-storage project, which is based on storage bricks (Ganger et al., 2003). An administrator specifies goals and responds to complaints by instructing Supervisors which gather performance information from Workers (storage bricks) which receive requests from Routers which interface with Clients.

Storage virtualization is defined as the creation of a set of logical storage devices from a single physical storage resource, e.g., the coexistence of multiple VAs in a single disk array. Five attributes for multidimensional storage virtualization according to (Peng and Chiueh, 2008) are: (1) Availability, (2) Bandwidth, (3) Capacity, (4) Delay (worst case), (5) Elasticity (probabilistic bandwidth and delay guarantees). This work concentrates on disk scheduling policies and admission control for multiple virtual disks placed on a single disk drive.

The *RAID Engine and Optimizer - REO* described by Deepak R. Kenchammana-Hosekote (IBM, NetApp, Salesforce), Dingshan He, and James Lee Hafner (rip) at IBM/ARC in (Kenchammana-Hosekote et al., 2007) works for any XOR-based erasure code and any combination of sector or disk failures. The emphasis of this study is on firmware reuse and reducing complexity. The following XOR-based k DFT, $1 \leq k \leq 3$ codes are considered: RAID5 (1DFT), EVENODD2, RDP, X-code (2DFT), EVENODD3, and WEAVER3 (3DFT). Benchmarking results with various workloads indicate that REO may attain modest improvements over existing RAID implementations.

The *Redundant Array of Independent Filesystems - RAID*, which provides RAID support with different levels at the per file level was proposed by Nikolai Joukov (IBM Watson, modelizeIT.com) and Erez Zadok et al. at (SUNY Stony Brook) in (Joukov et al., 2007). There is heavy reliance on stackable file systems, which provide flexibility in composition and development (Heidemann and Popek, 1994).

Panasas was founded by Garth Gibson and William Courtright in 1999 and shipped its first product in 2004.

<https://en.wikipedia.org/wiki/Panasas>

Parallel File System - PFS stores striped files as RAID1 or RAID5, depending on their size as described in (Welch et al., 2008). The four PFS advantages are as follows:

- (i) Scalable computing of parities for the files at the client site.
- (ii) Capability for end-to-end data integrity checking.
- (iii) Parallel rebuild capability for files.
- (iv) Limiting unrecoverable faults to individual files.

The initial data placement is uniform random. *Passive balancing* is used to place new data into empty nodes, while *active balancing* moves existing objects from one node to another to eliminate hotspots.

Methods to reallocate VAs whose load was incorrectly estimated or varied significantly over time are required for system maintenance. The disk cooling algorithm proposed by Peter Scheuermann, Gerhard Weikum, and Peter Zabback in (Scheuermann et al., 1998), which balances disk loads by reallocating datasets according to their “heat”, i.e., access rate, is relevant to this discussion. Similar capabilities have been developed for storage migration for VMware’s virtual machines (vMotion).

<https://www.vmware.com/products/vsphere/vmotion.html>

An early study of utilizing flash-based SSDs in enterprise storage systems is reported at Microsoft Research Center at Cambridge, UK in (Narayanan et al., 2009). The study utilized workload traces from large and small data centers, was concerned with optimizing the performance, capacity, power and reliability needs of the expected workload, while minimizing financial costs. SSDs can be used to completely replaced disks, but also as an intermediate tier between disks and DRAM. An automated tool was developed that given device models and a workload trace could determine the least-cost storage configuration to support performance, capacity, and fault-tolerance requirements. It was determined that replacing disks by SSDs is not a cost-effective option for any of the considered workloads due to the low capacity per dollar of SSDs, but this conclusion needs to be reassessed with reduced SSD flash prices.

Disk latencies are six orders of magnitude higher than NVRAM caches, which cost much more per byte. While SSD flash costs have fallen they are seven times more expensive than HDDs in March 2020.

<https://www.computerweekly.com/feature/Flash-storage-tiers-Performance-cost-and-use-cases>

Configuring multi-tier systems combining SSDs with disks to reduce cost, but to benefit from the superior I/O performance and low power consumption is a challenging problem, which was addressed in a joint *Florida Int’l U. - FIU* and *IBM/ARC* effort in (Guerra et al., 2011). The proposed *Extent-based Dynamic Tiering - EDT* solution consists of *Configuration Adviser - CA* to determine the mix of storage devices required to satisfy a given workload at minimum cost and a *Dynamic Tier Manager - DTM* which performs dynamic extent placement once the system is running to satisfy performance requirements while minimizing dynamic power consumption. EDT-DTM is able to consolidate load within tiers when feasible, rapidly respond to unexpected changes in the workload, and carefully control the overhead due to extent migration. EDT incurs lower capital and operating cost, consumes less power, but delivers similar or better performance relative to SAS-only storage systems.

A project to optimize vertical and horizontal data placement in multi-tiered storage was carried out as part of the ExaPlan project at *IBM Zurich Lab - ZRL* led by Ilias Iliadis (Iliadis et al., 2017). Access characteristics of the data are aligned to the characteristics of the storage devices, to achieve a higher performance for a given cost. ExaPlan not only determines data-to-tier assignment, but also the number of devices in each tier to minimize the system’s mean response time for a given budget and workload using a queueing model. Minimizing the mean response time is typically intractable, since the optimization problem is both nonconvex and combinatorial in nature, but ExaPlan circumvents this intractability by introducing a parameterized data placement approach. ExaPlan is shown to directly minimize the system’s mean response time estimated by a queueing model.

10.9 Using utility functions to provision storage systems

The use of utility functions instead of fixed requirements is a way for the system administrator to communicate underlying costs and benefits to an automated provisioning tool (Strunk et al., 2008). Utility functions provide a way to specify storage requirements in terms of the organization’s cost/benefit structure, allowing an automated tool to provide the level of service that produces the most cost-effective storage solution. A utility function expresses the desirability of a given configuration the benefits of the provided service, less the associated costs. The objective of the tool then becomes maximizing utility rather than just minimizing system cost.

Table 10.16 shows the general effects on various system metrics caused by increasing the data encoding parameters, m , n , or ℓ . With m -of- n erasure codes, a data block is divided into n fragments, any m of which can be used to reconstruct the original data and ℓ is the number of storage nodes holding a dataset.

The magnitudes of these effects vary considerably and are difficult to quantify without detailed models. It is difficult to make proper encoding choices manually because changing a single parameter affects nearly all the metrics. A system that is able to choose these parameters automatically must be able to make trade-offs across metrics.

Assuming that each I/O generates \$0.001 per IOPS for an OLTP workload per year is

$$Revenue = \$0.001 \times IOPS_{WL} \times AV_{DS} \times \left(\frac{3.2 \times 10^7 \text{ s}}{1 \text{ yr}} \right)$$

where AV_{DS} is the dataset availability and $IOPS_{WL}$ is the workload IOPS.

TABLE 10.16 General effects of encoding parameters on system metrics.

Metric	$m \uparrow$	$n \uparrow$	$\ell \uparrow$
Availability	↓	↑	↓
Capacity consumed	↓	↓	-
Read bandwidth	↓	↑	↑
Read latency	↑	-	↓
Write bandwidth	↑	↓	↑
Write latency	-	↑	↓

Given the cost of downtime is \$10,000/hour

$$Cost_{downtime} = \frac{\$10,000}{\text{hr}} \times (1 - AV_{DS}) \times \left(\frac{8766 \text{ hr}}{1 \text{ y}} \right)$$

The cost of losing the dataset (e.g., \$100 M) would be scaled by the annual failure rate:

$$Cost_{dataloss} = \$100M \times AFR_{DS}$$

where *Annual Failure Rate - AFR*

$$Utility = Revenue - Cost_{downtime} - Cost_{dataloss} \quad (10.12)$$

Attributes of Components

Client: CPU time for data encode/decode (s), Network streaming bandwidth (MB/s), Network latency (s).

Dataset: Size (MB).

Storage node: AFR (%), Fractional node availability (%), Disk capacity (MB), Purchase cost (\$), Max streaming bandwidth (MB/s), Initial positioning time (s), Network streaming bandwidth (MB/s), Network latency (s), Power consumption (W).

Performance model estimates for disk and network resources as well as throughput and latency estimates for the workloads. This information is derived from a closed-loop queueing model that is constructed and solved based on the system description and dataset assignments in the candidate configuration. The model is similar to that described by Thereska et al. in (Thereska et al., 2006), with queueing centers for the CPU and network as well as the storage nodes network and disk. The service demand placed on each queueing node is based on the encoding parameters for each dataset as well as the read/write ratio, I/O size, and sequentiality of the workloads. Workload intensity is specified as a multiprogramming level and a think time. These correspond to a maximum level of parallelism and an application's processing delay between I/Os, respectively. The open source PDQ queueing model solver is used to analyze the model via *Mean Value Analysis - MVA*.

<http://www.perfdynamics.com/Tools/PDQ.html>

An intuitive discussion of MVA appears in (Lazowska et al., 1984) and solution methods for *Queueing Network Models - QNMs* are discussed in Chapter 1.

Availability model estimates the availability of each dataset based on the availability of the storage nodes that the dataset is spread across and the encoding parameters used. The formula sums the probability for “available” states.

$$AV = \sum_{f=0}^{n-m} \binom{\ell, \ell-f}{A}_{SN}^{\ell-f} (1 - A_{SN})^f$$

where n and m are the encoding parameters for the dataset, ℓ is the number of storage nodes that the data spans, and A_{SN} is the minimum individual availability of the set of ℓ storage nodes. The dataset is considered to be available as long as no more than $n - m$ of the ℓ storage nodes are down.

Reliability model represents the likelihood of losing data from the primary storage system A *Markov Chain - MC* model is used to estimate the *Annual Failure Rate - AFR* for a dataset. The chain has $n - m + 2$ states, representing the number of device failures. The final state of the chain represents a data loss event. The MC a 1-of-3 encoding is shown

below has four states $\mathcal{S}_f$, $0 \leq f \leq 3$, where f is the number of failures with $\mathcal{S}_3$ denoting the failed state. With ℓ disks with failure rate λ we have the following transitions.

$$\mathcal{S}_f \xrightarrow{\ell\lambda} \mathcal{S}_{f+1}, 1 \leq f \leq 2$$

Failures are fixed with rate μ so that $\mathcal{S}_f \xrightarrow{\mu} \mathcal{S}_0$, $1 \leq f \leq 2$. Repair operations are handled by re-encoding the dataset which repairs all failures in the same operation, causing all repair transitions to lead to the failure-free state. The repair rate is based on the size of the dataset and a fixed fraction of the streaming bandwidth from the slowest storage node in the data distribution. The MC is solved to determine the expected time until the absorbing state.

Capacity This model calculates the storage increase of each dataset due to the redundancy of the encoding scheme, which is given as n/m . It also calculates the capacity usage of each storage node based on the datasets they store.

Cost and Model The purchase cost and power models are very similar, producing system-wide metrics based on the quantity and type of storage nodes that are used. Each storage node type has fixed cost and power attributes. When a storage node is used in a configuration, it is assumed to cost a fixed amount to purchase and consume a fixed amount of power. These cost and power attributes are then summed to produce the system-wide metrics for cost and power consumption.

Storage metrics provided by system models are listed below and are calculated for candidate configurations by the current set of system models. The table is organized by the component to which the metric refers. With the exception of the system-wide metrics, these metrics are calculated for each instance of a component, e.g., each storage node has a disk utilization that is calculated based on the accesses that it receives)

Client: CPU utilization (%), Network utilization (%).

Dataset: Annual failure rate (%), Fractional availability (%), Capacity increase from encoding, Mean time to failure (hr), “Nines” of availability.

Storage node: Raw capacity consumed (MB), Capacity utilization (%), Disk utilization (%), Network utilization (%), Power consumed (W).

System-wide: Total capacity consumed (MB), Capacity utilization (%), System power consumed (W), Total system cost (\$).

Workload: Bandwidth (MB/s), Throughput (IO/s), Request latency (s).

While many optimization techniques generate candidate configurations are available a *Genetic Algorithm* - GA was used in this study, see e.g., Chapter 9 in (Mitchell, 1997).

https://en.wikipedia.org/wiki/Genetic_algorithm

GA refines a population of candidate solutions over a number of generations, where each generation contains a hundred candidates. The utility value is used to generate a fitness value for each candidate, which are used to create a new population of candidates. The process is repeated and the new population is based on selection, crossover, and mutation operations to introduce randomness into the search and avoid local maxima and maintain diversity within the population. The solver continues producing configurations with higher utility. The stopping condition was no improvement upon the best configuration for 40 generations.

The fitness value determines how likely a candidate is to be selected for reproduction into the next generation. Due to capacity constraints, not all configurations are feasible. To bias the solution toward feasible, high-utility solutions, fitness is calculated as:

$$fitness = \begin{cases} utility & \text{if } (utility \geq 2 \quad OCC = 0) \\ \frac{1}{1+OCC} & \text{if } (OCC > 0) \\ \frac{1}{3-utility} + 1 & \text{otherwise} \end{cases}$$

OCC is the sum of the *Over-Committed Capacity* - OCC from the individual storage nodes (in MB). For those infeasible solutions, the fitness value will be between zero and one. When the solution is feasible, $OCC = 0$ (no storage nodes are over-committed). Utility values less than two are compressed into the range between two and one, eliminating negative utility to be compatible with the Roulette selection algorithm and ensuring that all feasible solutions have a fitness value greater than the infeasible ones.

Selection function probabilistically chooses candidates to use as a basis for the next generation of solutions using the fitness value for guidance. The solver can use the default Tournament selection algorithms (Mitchell, 1997), rather than Roulette.

Candidate representation for use in the GA, the storage configuration space is encoded as a matrix (Strunk, 2008). This matrix has one row for each dataset and one column for each storage node. Non-zero values at a particular location are

used to indicate that the dataset (represented by the row) is stored on the storage node. The values of the m and n encoding parameters for a dataset are determined by the number of entries in that dataset's row with values greater than two and one, respectively, e.g., a row of [03212] denotes an encoding of 1-of-3, with fragments stored on nodes 2, 3, 4, and 5 ($\ell = 4$). This matrix representation makes it easy to maintain the invariant: $1 \leq m \leq n \leq \ell$.

When the GA is initialized the matrix is generated randomly, with values from zero to three in each cell, but these values are then changed via the crossover and mutation operations as the optimization progresses.

Tool effectiveness is determined by how reliably it finds configurations that have high utility. Experiments show that the solutions produced by the solver approach the optimum and that the solver is able to quickly find good solutions for difficult problems.

TABLE 10.17 Effect of workload importance on provisioning decisions: A workload that generates more revenue per I/O should not necessarily have a higher level of data protection. This table compares two scenarios that differ only in the average revenue generated per completed I/O. The “more valuable” dataset's optimal data distribution is less available than that of the “less valuable” dataset because the cost of the additional downtime is more than offset by the additional performance of a less available data distribution. The data distributions in the table are written as: $m/n/\ell$.

			\$0.001 per IO	\$0.001 per IO	\$ 0.0001 per IO	\$0.00001 per IO
Distribution	1/2/6	1/3/7	1/2/6 (opt)	1/3/7	1/2/6	1/3/7 (opt)
Performance	1250 IO/s	1041 IO/s	\$76.3M	\$65.4M	\$7.6M	\$6.5M
Availability	1.5 nines	2.4 nines	−\$2.8M	−\$329K	−\$2.8M	−\$329K
Reliability(AFR)	2.0×10^{-6}	1.1×10^{-10}	−\$407	−0.02	−\$407	−\$0.02
Power	600 W	700 W	−\$631	−\$756	−\$631	−\$756
Purchase cost	\$60 K	\$70K	−\$2 K	−\$23K	−\$20 K	−\$23 K
Total utility			\$73.4 M	\$65.1M	\$4.7M	\$6.2 M

Components used in case study:

The client is based on measurements from a 2.66 GHz Pentium 4. The storage node data is based on the data sheet specifications for a single disk drive, combined with a 1 Gb/s network connection, processing and cache.

Client: CPU 0.2 ms, Net latency 125 μ s, Net BW MB/s.

Dataset: Size 100 GB.

Storage node: Annual Failure Rate - AFR 0.015, Availability 0.95, Capacity 500 GB, Cost \$5000, Disk BW 70 MB/s, Disk latency 5.5 ms, Net bw 119 MB/s, Net latency 125 μ s Power 50 W.

Workload (#1, #2) I/O size 8 kB *Multiprogramming Level - MPL* 5, Think time 1 ms, Rand frac 0.5.

Workload (#3) I/O size 32 kB, MPL 10, Think time 1 ms, Rand frac 0.0, Read frac 1.

Using the previous \$0.0001 scenario as an example, the optimal solution uses 14 storage nodes (seven for each dataset) and costs \$70 K, which may exceed the available budget and is hence unworkable. Table 10.16 compares this optimal solution to two alternatives: total hardware budget \$30 K and another \$20 k. Notice that these two alter-natives use six and four storage nodes respectively (at \$5000 each) to stay within their budget. The amortized “AP cost” in the table reflects this cost amortized over the system's expected three year lifetime.

The results of provisioning 2 systems are shown in Table 10.17, which shows both the metrics and costs for each part of the utility function. The table shows the optimal configuration for each scenario: 1-of-2 declustered across 6 (1/2/6) for the 0.1 cent scenario and 1-of-3 across 7 (1/3/7) for the 0.01 cent scenario.

Examining the various contributions to the total utility, it can be seen from Table 10.18 that the main trade-off between these two scenarios is in performance versus availability. For the scenario with the higher revenue per I/O, it is advantageous to choose the data distribution with higher performance at the cost of lower availability (1/2/6), because the revenue generated by the extra 209 I/Os per second per workload more than offsets the cost incurred by the extra downtime of this configuration. For the lower revenue scenario, the extra throughput cannot counteract the availability difference, causing the lower performing, more available data distribution (1/3/7) to be preferred.

10.10 Conclusions

We have described HDA, argued about its advantages, and used a synthetic workload to experimentally compare several single-pass data allocation methods for HDA, which are compared by the number of allocated VAs, disk bandwidth and capacity utilizations, and balancedness of allocations.

TABLE 10.18 Utility can be used to design for limited storage budgets. The optimal system configuration costs a total of \$70K, giving an amortized purchase (AP) cost of \$23K, but the utility function can be used to choose the best configuration that fits within other (arbitrary) budgets as well. Limiting the budget constrains the total hardware available, and the utility function guides the solution to make cost effective trade-offs as the system capabilities are scaled down to meet the limited budget.

Total budget	\$70 k	\$30 k	\$20 K
Distribution	1/3/7	1/2/3	1/2/2
Performance	\$6.5M	\$6.7M	\$5.8M
Availability	-\$329 k	-\$636 k	-\$219 k
Reliability	-\$0.02	-\$163	-\$81
Power	-\$736	-\$316 6	-\$210
AP cost	-\$23K	-\$10K	-\$6.6k
Total utility	\$6.2M	\$6.1M	\$5.6M

While only RAID1 and RAID5 are considered in the experimental study, we have provided the formulas for calculating the load for other RAID levels. These analytic results provided in this paper can be used in further studies of HDA.

The two allocation methods: Min-F1 and Min-F2, which take into account both disk access bandwidth and capacity, outperform methods such as Worst-Fit and Best-Fit, which only take into account disk bandwidth. Methods in the latter category outperform methods such as Round-Robin and Random, which do not take into account either disk bandwidth or capacity.

Experimental results show that the better data allocation methods result in disks with fully utilized bandwidth, capacity, or both bandwidth and capacity. More complex data allocation methods would be required to deal with larger allocation requests with respect to disk bandwidth and capacity.

Clustered RAID5 is a means of decreasing the increased load on surviving disks when disk failures occur. An experimental approach was used to quantify the increase in number of allocations and the optimal value for the declustering ratio (α).

Intermixed bandwidth- and capacity-bound requests can benefit from an adaptive allocation method, which monitors the end point of allocations over time with respect to the disk vector in Fig. 10.2 and applies declustering to bandwidth-bound and data compression/deduplication to capacity-bound allocation requests.

An alternate formulation as a bin-packing problem is to start with a fixed number of VAs to be allocated with the goal to minimize the number of utilized HDAs. Processing allocations in decreasing order of their load is a well known bin-packing paradigm will yield improved results.

Simple instances of HDA performance analysis are given in Section 10.3. Performance evaluation of HDA via analysis, simulation, or measurements and benchmarking of a prototype is an area of further investigation. Analytic methods for performance evaluation methods for RAID5 (Thomasian et al., 2007a) and RAID1 (Thomasian and Xu, 2008) can be extended to HDA operating in normal, degraded, and rebuild modes.

Several studies such as AutoRAID at HP with multiple RAID levels are presented. We conclude the discussion with two studies to generate optimized configurations.

Chapter 11

Hierarchical RAID - HRAID

Introduction to HRAID in Section 11.1
Intra- & internode coding in HRAID in Section 11.2
Concurrency Control in HRAID in Section 11.3
RAID IOPS with no disk failures in Section 11.4
RAID IOPS with disk failures in Section 11.5
HRAID response times in Section 11.6
HRAID2/2 performance in Section 11.7
RAID and HRAID reliability in Section 11.8
Shortcut reliability analysis of HRAID in Section 11.9
Simulation to estimate the MTDL in Section 11.10
Multistep recovery in HRAID in Section 11.11
Related work in Section 11.12
Collective Intelligent Bricks or Icecube project at IBM in Section 11.13
Reliability analysis of Icecube in Subsection 11.13.1
Conclusions in Section 11.14

11.1 Introduction to HRAID

The *Hierarchical RAID - HRAID* under the name multi-level RAID was first described in (Thomasian, 2006d). Further research on HRAID was conducted at *Shenzhen Institute of Advanced Technologies - SIAT* which is part of the *Chinese Academy of Sciences - CAS* siat.cas.cn, and was reported in (Thomasian and Tang, 2011a,b), (Thomasian et al., 2012).

We consider an HRAID with N Storage Nodes - SNs with M disks per SN. Each SN is a RAID0 or RAID($4 + \ell$), $1 \leq \ell \leq 3$ storage system with ℓ check strips per stripe. Each SN is an ℓ DFT $0 \leq \ell \leq 3$, which can mask the failure of ℓ out of M disks per SN. HRAID(k/ℓ) extends the RAID paradigm to mask the failures k out of N SNs. Internode recovery is attained with $0 \leq k \leq 3$ strips per stripe at each SN. *Maximum Distance Separable - MDS* erasure codes are used at both levels, but the overall code is not MDS as will be shown below.

We consider three HRAID options:

Option I utilizes intranode disk recovery only serving as a baseline for reliability and performance evaluation. There are N RAID0 or RAID($4 + \ell$), ℓ DFT, $0 \leq \ell \leq 3$ disk arrays operating independently, so this is not an HRAID after all. As disks fail rather than continuing operation in degraded mode, rebuild via restriping is used to attain a progression to lower redundancy levels to attain improved performance (Rao et al., 2011).

A node is considered failed when more than ℓ of its disks fail or its controller fails. There is no need to update internode check blocks and therefore this option attains a higher maximum *I/Os Per Second - IOPS* than Options II and III which update inter-node check strips.

Option II provides internode check blocks, which allow internode recovery if intranode recovery fails. This option leads demonstrates a significant improvement in MTDL with respect to Option I and this serves as a justification for the HRAID paradigm. There is a performance degradation with respect to Option I due to the overhead for updating internode check blocks, but this is the cost paid for higher reliability.

While HRAID(k/ℓ) with k with $k \geq 1$ can continue its operation in degraded mode after less than k SN failures, rebuild processing can be used to improve performance. No spare nodes are provided and data strips from the failed SN are written onto inter-SN check strips. There is the issue of the very high bandwidth required for SN rebuild, since the interconnect is designed for normal mode operation.

Option III behaves similarly to Option II, but does not undertake SN rebuild processing at the internode level which would require a high bandwidth interconnect across nodes. On demand reconstruction of blocks using intranode and

internode coding may incur a high delay in accessing blocks since recovery takes multiple steps, but a very high bandwidth interconnect as in Option II is not required.

Rebuild is the systematic reconstruction of the contents of a failed disk on one of the following disk areas: Various rebuild options are as follows:

Dedicated sparing wastes the bandwidth of a spare disk. A powered hot spare disk utilizes power and has a failure rate equaling to other active disks. Ordinarily spare disks can be shared among several arrays (Ng, 1994b), such a scheme is only suitable for HRAID for arrays in the same cluster (there are N arrays per cluster). Extra disks on SNs and even an extra SN with spare disks can be used for this purpose.

Distributed sparing tolerates single disk failure by allocating $1/N^{th}$ of the capacity of each disk to spare areas (Menon and Mattson, 1992a), (Thomasian and Menon, 1997), as discussed in Chapter 5.

Restriping overwrites higher level check strips treating them as spare areas (Rao et al., 2011). Starting with RAID(4 + k) the disk array transitions to RAID5 (with $k = 1$) and then RAID0.

Parity sparing combines two RAID5 arrays with the same number of parity strips of same size into one array (Menon and Mattson, 1992b). A similar scheme is applicable to combining SNs, where the check strips of one SN (a RAID5) protect the strips of another SN which is a RAID0.

11.2 Intranode & internode coding in HRAID

There are N Storage Nodes - SNs and M disks per SN. Internode (resp. intranode) data protection is achieved via internode (resp. intranode) check strips. Intranode check strips are computed over all strips at an SN, including internode check strips. Internode check strips are computed over data strips at the same position at the remaining SNs, but not intranode check strips. In HRAID k/ℓ the coding across the N SNs is $kNFT$, while the coding at each SN with M disks is ℓDFT .

The HRAID k/ℓ data layout with $N = M$ dedicates $k + \ell$ check strips per M strips on an SN. Check strips follow a left symmetric layout as in RAID5 (Chen et al., 1994b). Strips are shifted from SN to SN and on a per row basis at each SN. The storage utilization (S_u) is one minus the fraction of redundant strips, which determines storage redundancy S_r :

$$S_e = \frac{M - k - \ell}{M} = 1 - \frac{k + \ell}{M} = 1 - S_r. \quad (11.1)$$

In effect, the redundancy level is $N(k + \ell)$ disks in each row divided by the total number of disks $D = N \times M$.

These points are reinforced by the following examples.

Example 1 (HRAID1/1 with $N = M = 4$). In Fig. 11.1 data strips are denoted by $D_{i,j}^n$, and intranode and internode parity/check strips are denoted by $P_{i,j}^n$ and $Q_{i,j}^n$, respectively. The superscript $1 \leq n \leq N$ denotes the SN number, i is the row or stripe number applicable to the rows of all disks, which extends over nodes and $1 \leq j \leq M$ is the disk number at each SN.

Node 1				Node 2				Node 3				Node 4			
$D_{1,1}^1$	$D_{1,2}^1$	$P_{1,3}^1$	$Q_{1,4}^1$	$D_{2,1}^2$	$P_{2,2}^2$	$Q_{2,3}^2$	$D_{2,4}^2$	$P_{3,1}^3$	$Q_{3,2}^3$	$D_{3,3}^3$	$D_{3,4}^3$	$Q_{4,1}^4$	$D_{4,2}^4$	$D_{4,3}^4$	$P_{4,4}^4$
$D_{2,1}^2$	$P_{2,2}^2$	$Q_{2,3}^2$	$D_{2,4}^2$	$P_{3,1}^3$	$Q_{3,2}^3$	$D_{3,3}^3$	$D_{3,4}^3$	$Q_{4,1}^4$	$D_{4,2}^4$	$D_{4,3}^4$	$P_{4,4}^4$	$D_{1,1}^1$	$D_{1,2}^1$	$P_{1,3}^1$	$Q_{1,4}^1$
$P_{3,1}^3$	$Q_{3,2}^3$	$D_{3,3}^3$	$D_{3,4}^3$	$Q_{4,1}^4$	$D_{4,2}^4$	$D_{4,3}^4$	$P_{4,4}^4$	$D_{1,1}^1$	$D_{1,2}^1$	$P_{1,3}^1$	$Q_{1,4}^1$	$D_{2,1}^2$	$P_{2,2}^2$	$Q_{2,3}^2$	$D_{2,4}^2$
$Q_{4,1}^4$	$D_{4,2}^4$	$D_{4,3}^4$	$P_{4,4}^4$	$D_{1,1}^1$	$D_{1,2}^1$	$P_{1,3}^1$	$Q_{1,4}^1$	$D_{2,1}^2$	$P_{2,2}^2$	$Q_{2,3}^2$	$D_{2,4}^2$	$P_{3,1}^3$	$Q_{3,2}^3$	$D_{3,3}^3$	$D_{3,4}^3$

FIGURE 11.1 A 4×4 HRAID1/1 with $N = 4$ nodes and $M = 4$ disks per node. Only the first $M = 4$ stripes are shown in the figure.

Due to repetition only the data layout for the first M rows is shown. To balance the disk update load, the P and Q parity strips are rotated from row-to-row at each node, resulting in repeating right-to-left diagonals as noted above.

To update $d_{4,1}^{2new}$, a disk block in strip $D_{4,1}^2$, with $d_{4,1}^{2new}$, we have the following steps:

- (1) Read $d_{4,1}^{2old}$, $p_{4,3}^{2old}$, $q_{4,1}^{1old}$;
- (2) Compute difference block: $d_{4,1}^{2diff} = d_{4,1}^{2new} \oplus d_{4,1}^{2old}$;
- (3) Update P parity: $p_{4,3}^{2new} = p_{4,3}^{2old} \oplus d_{4,1}^{2diff}$;
- (4) The difference block is sent to SN₁ to update the internode and intranode parities:

$$q_{4,1}^{1new} = q_{4,1}^{1old} \oplus d_{4,1}^{2diff},$$

$$p_{4,4}^{1new} = p_{4,4}^{1old} \oplus q_{4,1}^{1diff}.$$

Failed disks are firstly dealt with via intranode coding, but if the number of failed disks at the SN exceeds ℓ via internode coding. If Disk 1 at SN₁ fails then restriping is applied at the node level and there is a transition from RAID5 to RAID0 at that node. $D_{1,1}^1 \leftarrow D_{1,2}^1 \oplus P_{1,3}^1, D_{1,1}^1 \rightarrow P_{1,3}^1$

$$D_{2,1}^1 \leftarrow D_{2,4}^1 \oplus P_{2,2}^1, D_{2,1}^1 \rightarrow P_{2,2}^1.$$

Although the Q strip has to be updated with the data strips substituting the parity strips as follows. $Q_{1,3}^{2new} \leftarrow Q_{1,3}^{2old} \oplus D_{1,1}^1$

$$Q_{2,2}^{2old} \leftarrow Q_{2,2}^{2new} \oplus D_{2,1}^1$$

If SN₁ fails then the steps required to reconstruct the stripe in the first row are as follows.

$$D_{1,1}^1 = D_{1,1}^2 \oplus Q_{1,1}^4, D_{1,1}^1 \rightarrow Q_{1,1}^4$$

$$D_{1,2}^2 = Q_{1,2}^3 \oplus D_{4,2}^1, Q_{1,2}^3 \rightarrow D_{1,2}^2.$$

Example 2 (HRAID2/1 with $N = M = 5$). We use P to denote the intranode RAID5 parity strip and Q and R for the internode check strips. Only the first row is shown for brevity.

$$(D_{1,1}^1, D_{1,2}^1, P_{1,3}^1, Q_{1,4}^1, R_{1,5}^1),$$

$$(D_{1,1}^2, P_{1,2}^2, Q_{1,3}^2, R_{1,4}^2, D_{1,5}^2),$$

$$(P_{1,1}^3, Q_{1,2}^3, R_{1,3}^3, D_{1,4}^3, D_{1,5}^3),$$

$$(Q_{1,1}^4, R_{1,2}^4, D_{1,3}^4, D_{1,4}^4, P_{1,5}^4),$$

$$(R_{1,1}^5, D_{1,2}^5, D_{1,3}^5, P_{1,4}^5, Q_{1,5}^5).$$

If SN₅ fails reconstruction of data strips starting with SN₁ at R strips yields:

$$(D_{1,1}^1, D_{1,2}^1, P_{1,3}^1, Q_{1,4}^1, D_{1,5}^2),$$

$$(D_{1,1}^2, P_{1,2}^2, Q_{1,3}^2, D_{1,3}^5, D_{1,5}^2),$$

$$(P_{1,1}^3, Q_{1,2}^3, -, D_{1,4}^3, D_{1,5}^3),$$

$$(Q_{1,1}^4, -, D_{1,3}^4, D_{1,4}^4, P_{1,5}^5).$$

$D_{1,2}^5$ is reconstructed at SN₁:

$$R_{1,5}^1 \leftarrow D_{1,2}^5 (= D_{1,2}^1 \oplus Q_{1,2}^3) \text{ and } D_{1,3}^5 \text{ at SN}_2:$$

$$R_{1,4}^2 \leftarrow D_{1,3}^5 (= Q_{1,3}^2 \oplus D_{1,3}^4).$$

The P strips at SN₁ and SN₂ are updated as

$$P_{1,3}^{1new} = P_{1,3}^{1old} \oplus R_{1,5}^{1old} \oplus D_{1,2}^5 \text{ and } P_{1,2}^{2new} = P_{1,2}^{2old} \oplus R_{1,4}^{2old} \oplus D_{1,3}^5.$$

The P strips at SN₃ and SN₄ need to be updated to remove the R strips, although this step can be deferred.

If SN₄ fails following SN₅, then restriping utilizes the Q strips.

$$D_{1,3}^4 \text{ is reconstructed at SN}_1: Q_{1,4}^1 \leftarrow D_{1,3}^4 \text{ (reconstructed via } Q_{1,3}^2 \text{ and } R_{1,3}^3, 2 \text{ equations in 2 unknowns).}$$

$$D_{1,4}^4 \text{ is reconstructed at SN}_2: Q_{1,3}^2 \leftarrow Q_{1,4}^1 \oplus D_{1,3}^5 \oplus D_{1,4}^3 \text{ The outcome is:}$$

$$(D_{1,1}^1, D_{1,2}^1, P_{1,3}^1, D_{1,3}^4, D_{1,5}^2),$$

$$(D_{1,1}^2, P_{1,2}^2, D_{1,4}^4, D_{1,3}^5, D_{1,5}^2),$$

$$(P_{1,1}^3, -, D_{1,3}^5, D_{1,4}^3, D_{1,5}^3).$$

A different data layout will result if SN₄ fails before SN₅, since the first node to fail will utilize the R strips. Each SN keeps track of the identities of reconstructed nodes, as discussed below. SN₁ and SN₂ have four data strips in its first row, while SN₃ has three data strips, because of rotation of strips when all stripes are reconstructed.

Example 3 (HRAID1/1 with $N > M$ ($N = 5$ and $M = 4$)). Data is at first allocated over the first M out of N SNs, following the data layout for HRAID1/1 with $N = M$. This ensures that all disks are protected via internode check codes. The disks at the remaining $d = N - M$ SNs are protected by ℓ intranode check codes only. This first row at the $N = 5$ nodes is as follows:

$$(D_{1,1}^1, D_{1,2}^1, P_{1,3}^1, Q_{1,4}^1), (D_{1,1}^2, P_{1,2}^2, Q_{1,3}^2, D_{1,4}^2), (P_{1,1}^3, Q_{1,2}^3, D_{1,3}^3, D_{1,4}^3),$$

$$(Q_{1,1}^4, D_{1,2}^4, D_{1,3}^4, P_{1,4}^4), (D_{1,1}^5, D_{1,2}^5, P_{1,4}^5, Q_{1,4}^5)$$

The Q strip in the first $M = 4$ SNs is computed over all SNs, including the fifth. For load balancing purposes the SN stripes are rotated. Strip $Q'_{1,5}$ on SN₅ is optionally provided to ensure an equal number of data strips at all nodes. When SN₁ fails we have:

$$Q_{1,3}^2 \leftarrow D_{1,1}^1 = D_{1,1}^2 \oplus Q_{1,1}^4 \oplus D_{1,1}^5,$$

$$Q_{1,3}^2 \leftarrow D_{1,2}^1 = Q_{1,2}^3 \oplus Q_{1,2}^4 \oplus D_{1,2}^5.$$

The storage efficiency in this case, similarly to Eq. (11.1) is $u = 1 - (k + \ell)/M$.

Example 4 (HRAID1/1 with $M > N$ ($M = 2N = 8$)). Additional internode check strips are allocated per SN to make internode recovery possible. In the case $M = 2N$ two internode check strips Q and R are shifted by two from node to node to cover odd and even numbered disks, respectively.

$$\begin{aligned} &(D_{1,1}^1, D_{1,2}^1, D_{1,3}^1, D_{1,4}^1, D_{1,5}^1, P_{1,6}^1, Q_{1,7}^1, R_{1,8}^1), \\ &(D_{1,1}^2, D_{1,2}^2, D_{1,3}^2, P_{1,4}^2, Q_{1,6}^2, R_{1,6}^2, D_{1,7}^2, D_{1,8}^2), \\ &(D_{1,1}^3, P_{1,2}^3, Q_{1,3}^3, R_{1,4}^3, D_{1,5}^3, D_{1,6}^3, D_{1,7}^3, D_{1,8}^3), \\ &(Q_{1,1}^4, R_{1,2}^4, D_{1,3}^4, D_{1,4}^4, D_{1,5}^4, D_{1,6}^4, D_{1,7}^4, P_{1,8}^4). \end{aligned}$$

If SN₁ fails then its five data strips can be recovered using the Q and R strips at the other SNs. The five data strips will overwrite Q and R strips at the three surviving SNs, leaving one empty strip. The P parities at the other three SNs need to be updated accordingly.

For a given N , one intranode check strip, the number of internode checks strips x satisfies the following formula determining the maximum of data disks per SN to make recovery from an SN failure possible. $M - (x + 1) = x(N - 1)$, e.g., for $N = 4$, $x = 2$ then $M = 9$.

A simple rule for restriping at the SN level for $k > 1$ is that when there are several candidate strips before higher order strips are overwritten, e.g., in the case of HRAID2/1 in Example 2 with internode check strips Q and R , the R strips are overwritten before Q check strips. There is the issue of whether the Q and R , in addition to P strips, can be used to deal with local (intranode) disk failures. Consider the failure of two disks at the same SN. The first disk is reconstructed using P strips, but when a second disk fails there are two options:

- (1) The system can continue its operation in degraded mode.
- (2) One of the internode check strips can be used for restriping.

In the case of two SN failures, which require the Q and R strips for restriping, the R strips which were utilized for intranode recovery can be overwritten. In this study we use internode check strips for restriping to deal with SN failures and intranode check strips otherwise.

Remapped strips maintain their original functionality and are addressed with their original coordinates, so that virtual to physical strip translation tables are required. A higher level directory is not required to determine the table and the node provided simple remapping rules are followed. In Example 2 given that SN₅ and SN₄ their data strips will be recreated at the R and Q strips at the other SNs.

11.3 Concurrency control in HRAID

Multiple check blocks in addition to an updated data block need to be updated atomically. Txns can be used to ensure that the parity blocks associated with a data block are updated. We consider concurrency control based on locking in this study (Thomasian, 1998), (Ramakrishnan and Gehrke, 2002).

Given the identity of a block to be updated, the identities of all associated check blocks can be determined, so that locks on all such blocks can be requested concurrently at all nodes. The processing of lock requests at SNs can be serialized by using a dedicated link.

Following the writing of modified data blocks onto NVRAM caches, SNs send acknowledgments (ACKs) to the client which originated an update and acts as coordinator. Once the coordinator receives ACKs from all SNs it issues a commit to allow modified data to be updated and locks to be released. Thus the more complex 2-phase-commit - 2PC is not required (Ramakrishnan and Gehrke, 2002).

Let $R(\cdot)$, $W(\cdot)$, and $S(\cdot)$ denote the reading, writing, and sending of a block, which may be a multicast to several SNs specified by superscripts. To update $d_{4,1}^2$, a disk block in strip $D_{4,1}^2$, with $d_{4,1}^{2new}$ we have the following steps:

1. $R(d_{4,1}^{2old})$, compute difference block, and write new data block:

$$d_{4,1}^{2diff} = d_{4,1}^{2new} \oplus d_{4,1}^{2old}, W(d_{4,1}^{2diff}); S_{(1)}d_{4,1}^{2diff}.$$

2. Compute new check blocks and update them:

$$\begin{aligned} p_{4,3}^{2new} &= p_{4,3}^{2old} \oplus d_{4,1}^{2diff}, W(p_{4,3}^{2new}); \\ q_{4,1}^{1new} &= q_{4,1}^{1old} \oplus d_{4,1}^{2diff}, W(q_{4,1}^{1new}); \\ p_{4,4}^{1new} &= p_{4,4}^{1old} \oplus q_{4,1}^{1diff}, W(p_{4,4}^{1new}). \end{aligned}$$

To illustrate how CC is applied in HRAID consider the updating of block $d_{1,1}^1$ in Example 2 in Section 11.2. In this case we provide more detail by specifying the txn to update $d_{1,1}^1$ at SN₁, which then invokes subtxns to update check strips at SN₄ and SN₅ by sending $d_{1,1}^{1diff}$ as shown in Table 11.1.

TABLE 11.1 Subtxns required for updating a data block at D_1 at SN₁ for the HRAID2/1 in Example 2.

Subtxn at SN ₁	Subtxn at SN ₄	Subtxn at SN ₅
$R(d_{1,1}^{1old}), R(p_{1,3}^{1old})$ $d_{1,1}^{1diff} = d_{1,1}^{1old} \oplus d_{1,1}^{1new}$ $W(d_{1,1}^{1new}), p_{1,3}^{1new} = d_{1,1}^{1diff} \oplus p_{1,3}^{1old}$ $W(p_{1,3}^{1new}), S_{4,5}(d_{1,1}^{1diff})$		
	$R(q_{1,1}^{4old}), R(p_{1,5}^{4old})$ $q_{1,1}^{4new} = q_{1,1}^{4old} \oplus d_{1,1}^{1diff}$ $p_{1,5}^{4new} = p_{1,5}^{4old} \oplus d_{1,1}^{1diff}$ $W(q_{1,1}^{4new}), W(p_{1,5}^{4new})$	$R(r_{1,1}^{5old}), R(p_{1,4}^{5old})$ $q_{1,1}^{4new} = q_{1,1}^{4old} \oplus d_{1,1}^{1diff}$ $p_{1,4}^{5new} = p_{1,4}^{5old} \oplus d_{1,1}^{1diff}$ $W(q_{1,1}^{5new}), W(p_{1,4}^{5new})$

11.4 RAID IOPS with no disk failures

We consider the cost of disk accesses in processing an OLTP workload issuing accesses to small randomly placed disk blocks (Ramakrishnan et al., 1992). We utilize the performance analysis method in (Thomasian, 2005a), (Thomasian et al., 2007a), (Thomasian et al., 2012) for RAID5 and RAID6 and extend it to RAID7. The mean and second moment of disk accesses classified as *Single Reads* - SR, *Single Write* - SW, and *Read-Modify Writes* - RMW are used in an M/G/1 queueing model assuming disk requests arrive according to a Poisson process. We ignore internode communication delays, because they are expected to be orders of magnitude smaller than disk access times.

SRs require a single disk access to read the data block, while updates are costly due to the *Small Write Penalty* - SWP, because the data and multiple check blocks need to be updated. The updating of a block d_{old} requires updating $p_{new} = p_{old} \oplus d_{diff}$, where $d_{diff} = d_{old} \oplus d_{new}$ and writing the new parity block (Thomasian et al., 2007a). To cope with “write holes” the XOR is preferably carried out at the RAID controller, which logs its operations to allow recovery in the case of a system crash. This requires an SR followed by an SW.

The mean service time for SR and SW accesses is denoted by $\bar{x}_{SR}$ and $\bar{x}_{SW}$, respectively. The fraction of read (resp. write) requests is f_R (resp. $f_W = 1 - f_R$). The fraction of reads versus writes is given as R:W, e.g., 4:1, so that $f_R = R/(R + W)$. Update requests in RAID(4 + ℓ), $\ell \geq 1$ may be processed as Provided the disk arm does not move after the SR request, this is tantamount to a *Read-Modify-Write* - RMW access, which can be processed as a data read followed by a write after one disk rotation, i.e., $\bar{x}_{RMW} = \bar{x}_{SR} + T_{rot}$, where $T_{rot} = 60000/RPM$ is the disk rotation time in milliseconds, but the load incurred by RMW is modeled as the sum of independent SR and SW requests in this study.

For RAID(4 + ℓ), $\ell \geq 1$ the updating of a data block requires $(\ell + 1)$ SR accesses to read data and check blocks and $(\ell + 1)$ SW accesses to write them.

To update a data block in HRAID(k/ℓ) $(k + 1)(\ell + 1)$ SRs are required to read data blocks and internode and intranode check blocks and $(k + 1)(\ell + 1)$ SWs to write them.

$$\lambda_0^{max} = [f_R \bar{x}_{SR} + f_W \bar{x}_{SW}]^{-1},$$

$$\lambda_\ell^{max} = [f_R \bar{x}_{SR} + f_W (\ell + 1)(\bar{x}_{SR} + \bar{x}_{SW})]^{-1}.$$

$$\lambda_{k/\ell}^{max} = \{f_R \bar{x}_{SR} + f_W [(k + 1)(\ell + 1)(\bar{x}_{SR} + \bar{x}_{SW})]\}^{-1}.$$

Given that the head settling time is short: $\bar{x}_{SW} \approx \bar{x}_{SR}$, and the relative maximum IOPS for RAID0, RAID ℓ and HRAID k/ℓ per disk are:

$$1 : [1 + f_W (1 + 2\ell)]^{-1} : [1 + f_W + 2f_W (k\ell + k + \ell)]^{-1}.$$

Studies of disk I/O traces for OLTP workloads indicated that R:W = 4:1, but with the advent of very large read caches a smaller ratio for read requests, such as R:W = 1:1, is plausible. The maximum normalized throughput drops to 0.42 for

RAID6 and to 0.31 for HRAID1/2 for $f_W = 0.2$. The drop is more drastic for $f_R = 0.5$: 0.40 for RAID6 and 0.15 for HRAID1/2. This is an inevitable drop in performance to attain a higher reliability.

11.5 RAID IOPS with disk failures

We consider a RAID($4 + \ell$) with N disks of which $0 \leq n \leq \ell$ disks have failed.¹ Given uniform disk accesses the probability of a successful (resp. unsuccessful) read request is $1 - n/N$ (resp. n/N). The number of disk accesses required in the two cases is D_{SR} and $(N - \ell)D_{SR}$, so that the read cost with n disk failures is:

$$C_\ell^{(n)} = \left[\frac{N - n}{N} + \frac{n}{N}(N - \ell) \right] D_{SR} = \left[1 + n - \frac{n(\ell + 1)}{N} \right] D_{SR}.$$

In the case of RAID5 with a single disk failure with ($\ell = n = 1$), the read load on the $N - 1$ surviving disks is doubled.

The decision tree method utilized in (Thomasian, 2005a) to estimate the cost of processing updates in degraded mode in RAID5/6 arrays is applied to RAID7 in Fig. 11.2. D , P , Q and R denote the availability of a data block and the three corresponding check blocks, while $\bar{D}$, $\bar{P}$, $\bar{Q}$, $\bar{R}$ represent unavailable blocks.

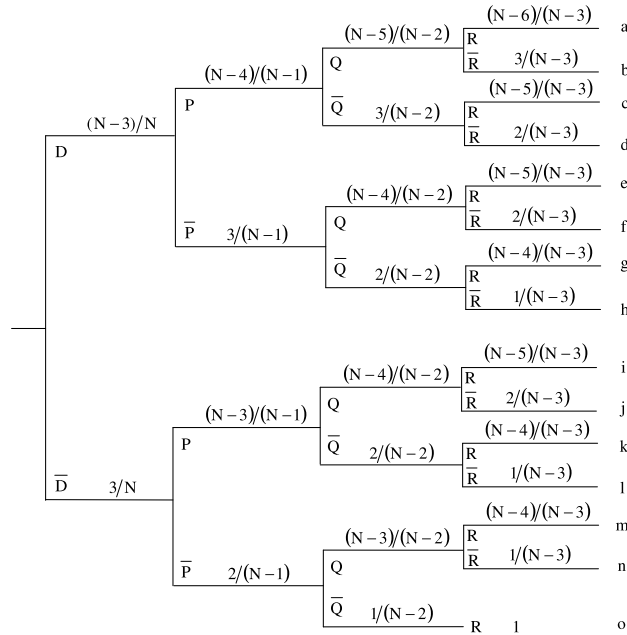


FIGURE 11.2 Decision tree to obtain disk access costs in RAID7 with three disk failures.

- (a) No disk failures: $4D_{RMW}$;
- (b,c,e) One unavailable check block: $3D_{RMW}$;
- (d,f,g) Two unavailable check blocks: $2D_{RMW}$;
- (h) Three unavailable check blocks: D_{RMW} ;
- (i) Data block unavailable: $C + 3D_{SW}$;
- (j,k,m)/(l,n,o) Data block and one/two check blocks unavailable: $C + 2D_{RMW}$.

The cost of read and write accesses with $n = 3$ disk failures is given in Table 11.2.

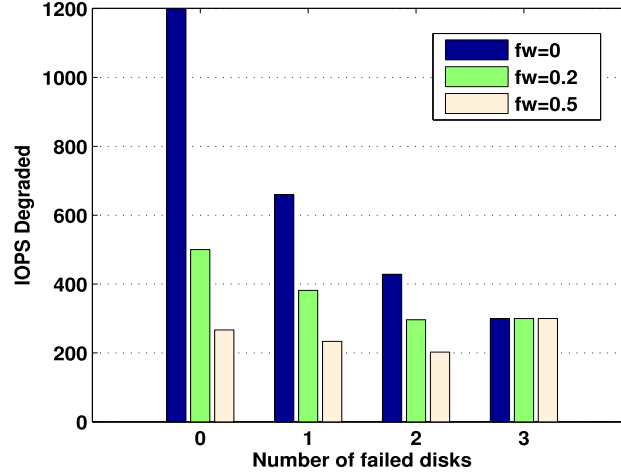
Fig. 11.3 shows the variation of IOPS for $f_W = 0, 0.2, 0.5$ with $N = 12$ disks and $0 \leq n \leq 3$ disk failures. When all requests are reads ($f_W = 0$) the maximum IOPS is determined by the number of available disks, so that for $n = 3$ failed disks out of $N = 12$ disks as expected the maximum IOPS drops by 25%. The costs in Table 11.2 are used to compute the maximum IOPS for $f_W > 0$.

Of the three rebuild options described in Section 10.1, rebuild with restriping is considered in this study. The failure of successive disks in RAID7 with P , Q , and R check strips leads to a RAID6 with P and Q check blocks, a RAID5 with P

1. In this section we use N instead of M for the number of disks, to be consistent with our previous papers on RAID analysis.

TABLE 11.2 Cost of processing updates in RAID7 with N disks.

Mode	Read	Write
Normal	D_{SR}	$4D_{RMW}$
One failure	$\frac{2(N-2)}{N}D_{SR}$	$\frac{N-4}{N}D_{SR} + \frac{3}{N}D_{SW} + \frac{4N-7}{N}D_{RMW}$
Two failures	$\frac{3N-8}{N}D_{SR}$	$\frac{2(N-4)}{N}D_{SR} + \frac{6(N-2)}{N(N-1)}D_{SW} + \frac{2(N-2)(2N-5)}{N(N-1)}D_{RMW}$
Three failures	$\frac{4(N-3)}{N}D_{SR}$	$\frac{3(N-4)}{N}D_{SR} + \frac{9(N-3)}{N(N-1)}D_{SW} + \frac{(N-3)(4N-13)}{N(N-1)}D_{RMW}$

**FIGURE 11.3** Maximum IOPS in degraded mode of operation for varying number of disk failures starting with a fault-tree RAID7 with $N = 12$ disks.

parity strips, and finally RAID0. The maximum IOPS for varying number of disk failures is given in Fig. 11.4 for operation after restriping. Interestingly, there is an increase in maximum IOPS with restriping for larger values of $f_W > 0$, in spite of the fact that fewer disks are available. This is because of the reduced load, since fewer check blocks need to be updated. The IOPS of RAID0 resulting from three disk failures in a RAID7 is not affected by f_W .

Consider a single SN with M disks and ℓ check strips per stripe. As disk fails and check strips are overwritten via restriping the maximum IOPS may increase at the cost of a reduced reliability.

Given that the fraction of reads and writes is f_R and $f_W = 1 - F_R$, and the disk access time for read and write requests is $\bar{x}$ then due to Small Write Penalty - SWP the maximum throughput of RAID($4 + k$) with M disks is:

$$T_{max}(M) = M / [(f_R + 2f_W(\ell + 1))\bar{x}].$$

If one disk fails and one of its check strips is used for restriping we have RAID($4 + (\ell - 1)$) with $M - 1$ disks:

$$T_{max}(M - 1) = (M - 1) / [f_R + 2f_W\ell]\bar{x}$$

For $F_R = 2/3$ there is an increase in throughput after a single disk failure for $M > \ell + 2$. A more detailed discussion of this topic appears in Section 11.4.

11.6 HRAID response times

The effect of fault-tolerance on the performance degradation of RAID5 and RAID6 disk arrays with respect to RAID0 was investigated in (Thomasian et al., 2007a). This paper extends this methodology to HRAID(k/ℓ), but while results with node failures are presented for HRAID2/2, the associated analysis is given in Section 11.7. The postulated workload can be summarized as follows:

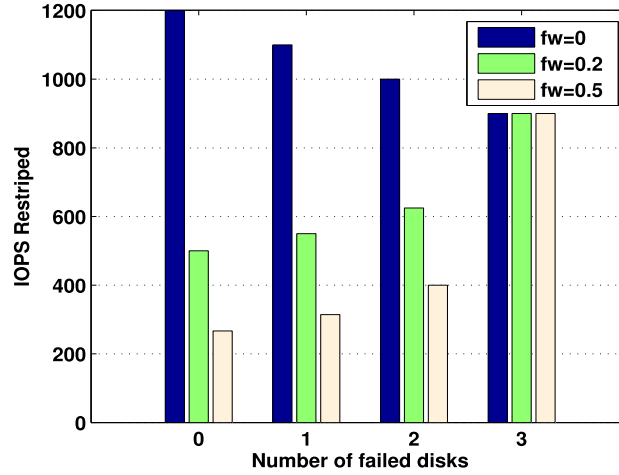


FIGURE 11.4 Maximum IOPS after restriping for varying number of disk failures starting with a fault-tree RAID7 with $N=12$ disks.

- The arrival process for read requests to disks is Poisson with rate Λ . Read accesses to disk are a result of cache misses at several levels of the memory hierarchy.
- The Poisson arrival process is better justified for read requests than write requests, since update writes are a result of periodic destaging of dirty blocks from the NVS cache when it is full or is about to be filled (Thomasian et al., 2007a). Dirty blocks in the NVS cache may be overwritten several times before they are destaged, which obviates several disk writes. Destaging is done in batches so that disk scheduling can be used to minimize destage time. These effects as quantified by trace-driven simulation in (Treiber and Menon, 1995) were taken into account in (Thomasian and Menon, 1997). Worst case assumption is made in this study that dirty blocks are not overwritten and that updates similarly to reads are uniformly distributed.
- It is assumed that disk requests are distributed uniformly over all nodes, the disks at each node, and the blocks at each disk, so that the rate of read and update accesses to each disk is $\lambda = \Lambda/D$, where Λ is the arrival rate of requests to HRAID and $D = N \times M$ is the total number of disks.
- The fraction of logical read and writes is f_R and $f_W = 1 - f_R$. In normal operating mode each read requires an SR access and each write requires updating the logical block and the associated check blocks. Given that disk accesses are uniformly distributed across all disks and that check strips are placed uniformly over all disks, updates result in balanced disk loads.
- Update requests are processed as RMW requests comprising SRs followed by SWs for data and intranode and internode check blocks.
- Disk access times are the sum of the delay at the *Disk Array Controller - DAC*, seek time, rotational latency, and transfer time. Instead of a detailed model to compute disk service times as in (Thomasian et al., 2007a), we assume that the first two moments of disk access time for read and update accesses: $\bar{x}_{SR}^i$ and $\bar{x}_{SW}^i$ ($1 \leq i \leq 2$) are known.
- A FCFS disk scheduling policy is implicitly assumed, so that unlike *Shortest Access Time First - SATF* scheduling the mean disk access time $\bar{x}_d$ is not reduced as the disk access rate is increased and the queue-length increases. Modern disk drives utilize the *Shortest Access Time First Scheduling - SATF* policy, so that the mean disk access time decreases with the queue-length as noted in (Thomasian, 2011), (Thomasian, 2013).
- Read requests to disk affect transaction response times for OLTP applications, so that it is beneficial to process them at a higher priority than update requests, since they do not affect performance.

The above assumptions lead to an M/G/1 single-server queueing model per disk, with Poisson arrivals with rate λ and general service times with i^{th} moment $\bar{x}_d^i$. The mean waiting time of requests is given by the *Pollaczek-Khintchine - PK* formula (Kleinrock, 1975), (Lavenberg, 1983).

$$W = \frac{\lambda \bar{x}_d^2}{2(1 - \rho)} = \frac{\rho \bar{x}(1 + c^2)}{2(1 - \rho)},$$

where $\rho = \lambda \bar{x}_d$ is the server utilization and $\bar{x}^2 = (1 + c^2)(\bar{x}_d)^2$. where $c^2 = \bar{x}^2/(\bar{x})^2 - 1$ is the squared coefficient of variation of service time, e.g., $c = 0$ for fixed service times and $c = 1$ for exponential service times.

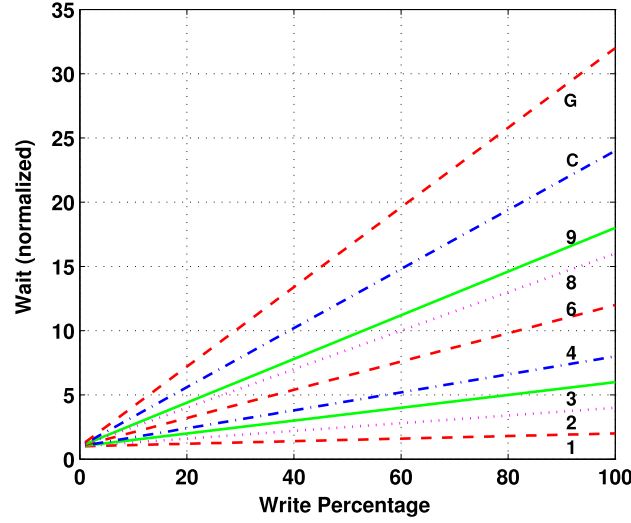


FIGURE 11.5 Normalized mean waiting for prioritized read requests for different RAID and HRAID configurations, differentiated by the values of $(k + 1)(\ell + 1)$ for $0 \leq \ell, k \leq 3$ given above (we use the letters $C = 12$ for $k = 3, 2$ and $\ell = 2, 3$ and $G = 16$ for $k = \ell = 3$).

For the M/M/1 queueing system: $W = \rho\bar{x}/(1 - \rho)$ and mean response time is: $R = W + \bar{x} = \bar{x}/(1 - \rho)$. The normalized waiting time $W/\bar{x}$ increases rapidly with the utilization factor $\rho = 0.125, 0.25, 0.5, 0.9$ as $1/7, 1/3, 1, 9$. This effect should be taken into account in interpreting results pertaining to waiting time.

We first consider RAID0 disk arrays with no redundancy. Disk utilizations for SR and SW requests in this case are: $\rho_{SR} = f_R \lambda \bar{x}_{SR}$ and $\rho_{SW} = f_W \lambda \bar{x}_{SW}$. If read accesses are given a higher priority than update accesses, then their mean waiting time is affected by ρ_R rather than the total disk utilization: $\rho = \rho_R + \rho_W$. The mean waiting time for higher priority read requests is (Kleinrock, 1976):

$$W_R^{(0)} = \frac{W_0}{1 - \rho_R} \approx \frac{\rho \bar{x}}{1 - \rho_R}, \quad (11.2)$$

where $W_0 = \rho_R \bar{x}_{SR}^2 / (2\bar{x}_{SR}) + \rho_W \bar{x}_{SW}^2 / (2\bar{x}_{SW})$ is a weighted sum of the residual service times of disk requests in service. Given that $\bar{x}_{SR}^2 \approx \bar{x}_{SW}^2 = 2(\bar{x}_d)^2$ it follows $W_0 \approx \rho \bar{x}_d / 2$.

In the case of RAID($4 + \ell$) with $1 \leq \ell \leq 3$ the load due to read and update requests is $\rho_R = f_R \lambda \bar{x}_{SR}$ and $\rho_W = 2f_W \lambda (1 + \ell) \bar{x}$. If read requests are processed at higher priority than updates, then their mean waiting time is:

$$W_R^{(\ell)} = \frac{\lambda (\bar{x}_d)^2 [f_R + 2f_W(1 + \ell)]}{1 - \rho_R}, \quad \ell \geq 1. \quad (11.3)$$

The relative values of waiting time $W_R^{(\ell)} / W_R^{(0)}$ for $\ell = 1, 2, 3$ are:

$$f_W = 0.2: \quad W_R^{(1)} / W_R^{(0)} = 1.6, \quad W_R^{(2)} / W_R^{(0)} = 2, \quad W_R^{(3)} / W_R^{(0)} = 2.4;$$

$$f_W = 0.5: \quad W_R^{(1)} / W_R^{(0)} = 2.5, \quad W_R^{(2)} / W_R^{(0)} = 3.5, \quad W_R^{(3)} / W_R^{(0)} = 4.5.$$

The analysis extended to HRAID k/ℓ yields:

$$W_R^{(k/\ell)} = \frac{\lambda \bar{x}^2 [f_R + 2f_W(1 + k)(1 + \ell)]}{1 - \rho_R}, \quad k \geq 1, \ell \geq 1. \quad (11.4)$$

Fig. 11.5 is a plot of $W_R^{(k/\ell)}$ normalized with respect to $W_R^{(0)}$, which shows that it increases linearly with f_W . Different slopes are associated with the different multiples of $(k + 1)(\ell + 1)$.

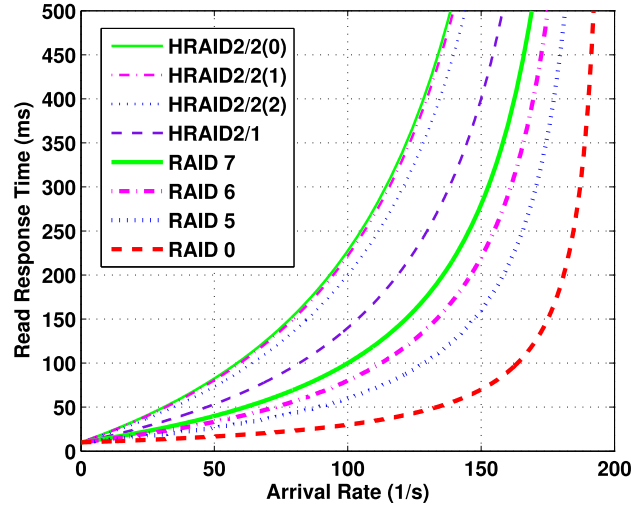


FIGURE 11.6 Prioritized read response time with 0, 1, 2 node failures (in parentheses) for HRAID2/2 and no failures for HRAID2/1, RAID($4 + \ell$), $1 \leq \ell \leq 3$ and RAID0 for R:W = 1:1.

Fig. 11.6 plots the mean read response for nonfailed disks with $\bar{x} = 10$ ms for the sake of concreteness. We utilize Eq. (11.3) for RAID($\ell + 4$), $1 \leq \ell \leq 3$ and Eq. (11.4) for HRAID k/ℓ in Section 11.6. For HRAID2/2 in degraded mode we have utilized the cost equations derived in Section 11.7. The mean read response time remains below 100 ms for 60% disk utilization for HRAID2/2 and 80% disk utilization in all other cases. It should be noted that similarly to Figure 3.3 in (Kleinrock, 1976) the lower priority classes may be saturated while higher priority class has a reasonable mean waiting time. The mean response time of read requests to a failed disk equals the response time of fork-join requests, to access corresponding blocks on surviving disks $R_n^{F/J}$, where $n = M - 1$ is the number of disks involved.

M/M/1 queues have an exponential response time distribution, so that the maximum of n response times with mean R is $R_n^{F/J} \approx R_n^{max} = R \sum_{i=1}^n 1/i$. The distribution of read response time with priority queueing has a coefficient of variation $c < 1$, so that the approximate methods in (Thomasian and Menon, 1997), (Thomasian et al., 2007a), (Thomasian, 2014a) may be used to compute R_n^{max} . The overall mean response time is a weighted sum according to the fraction of failed and nonfailed disks (Thomasian and Menon, 1994), (Thomasian and Menon, 1997), (Thomasian et al., 2007a).

11.7 HRAID2/2 performance

We consider HRAID2/2 with two intranode check blocks: P and Q and two internode check blocks: R and S . We use $\bar{D}$ to indicate that a disk is broken. The fraction and cost for read requests with $n = 1, 2$ broken disks is:

$$D: f = (N - n)/N \Rightarrow D_{SR};$$

$$\bar{D}: f = n/N \Rightarrow (N - 4)D_{SR}.$$

The fraction and cost of write requests with $n = 1$ is:

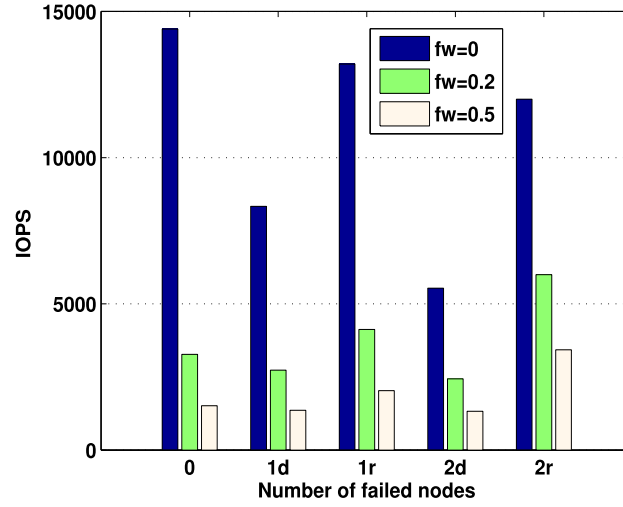
1. $D \wedge R \wedge S: f = (N - 3)/N \Rightarrow 9D_{RMW};$
2. $(D \wedge R \wedge \bar{S}) \vee (D \wedge \bar{R} \wedge S): f = 1/N \Rightarrow 6D_{RMW};$
3. $\bar{D} \wedge R \wedge S: f = 2/N \Rightarrow (N - 4)D_{SR} + 6D_{RMW}.$

The fraction and cost of write requests with $n = 2$ is:

1. $D \wedge R \wedge S: f = (N - 3)(N - 4)/[N(N - 1)] \Rightarrow 9D_{RMW};$
2. $D \wedge R \wedge \bar{S} \vee (D \wedge \bar{R} \wedge S): f = 4(N - 3)/[N(N - 1)] \Rightarrow 6D_{RMW};$
3. $D \wedge \bar{R} \wedge \bar{S}: f = 2/[N(N - 1)] \Rightarrow 3D_{RMW};$
4. $\bar{D} \wedge R \wedge S: f = 2(N - 3)/[N(N - 1)] \Rightarrow (N - 5)D_{SR} + 6D_{RMW};$
5. $(\bar{D} \wedge \bar{R} \wedge S) \vee (\bar{D} \wedge R \wedge \bar{S}):$
 $f = 4/[N(N - 1)] \Rightarrow (N - 5)D_{SR} + 3D_{RMW}.$

TABLE 11.3 Cost of operation for HRAID2/2 with N nodes. Mode settings 0F, 1F, and 2F indicate number of failed nodes.

Mode	Read	Write
0F	D_{SR}	$9D_{RMW}$
1F	$\frac{2N-5}{N}D_{SR}$	$\frac{2(N-4)}{N}D_{SR} + \frac{9(N-1)}{N}D_{RMW}$
2F	$\frac{3N-10}{N}D_{SR}$	$\frac{2(N-3)(N-5)}{N(N-1)}D_{SR} + \frac{9(N-2)}{N}D_{RMW}$


FIGURE 11.7 IOPS for HRAID2/2 with zero, one, and two disk failures in degraded mode (d) and after restriping (r).

The maximum throughput of HRAID2/2 with zero, one, and two node failures are given by the cost functions given in Table 11.3. In what follows $\bar{x}_{RMW}$ is shorthand for $\bar{x}_{SR} + \bar{x}_{SW}$.

$$T_{*/}^{0F} = \frac{NM}{f_R \bar{x}_{SR} + (k+1)(\ell+1)f_W \bar{x}_{RMW}}, \quad k = \ell = 2;$$

$$T_{2/2}^{1F} = \frac{(N-1)M}{f_R \frac{2N-5}{N} \bar{x}_{SR} + f_W \left[\frac{2(N-4)}{N} \bar{x}_{SR} + \frac{9(N-1)}{N} \bar{x}_{RMW} \right]};$$

$$T_{2/2}^{2F} = \frac{(N-2)M}{f_R \frac{3N-10}{N} \bar{x}_{SR} + f_W \left[\frac{2(N-3)(N-5)}{N(N-1)} \bar{x}_{SR} + \frac{9(N-2)}{N} \bar{x}_{RMW} \right]}.$$

The maximum IOPS for HRAID2/2 in normal mode and with one and two disk failures are shown in Fig. 11.7 in degraded mode (d) and after restriping (r). There is a significant drop in HRAID2/2 IOPS in degraded mode, following the failure of one and two nodes, but the IOPS recovers after restriping. The IOPS is determined by the number of surviving nodes for $f_W = 0$, but as observed earlier the IOPS increases after restriping for $f_W > 0$, which is due to the reduced load for updating internode check blocks.

11.8 RAID and HRAID reliability

Controller and interconnection network failures are not considered. We first consider an HRAID1/1 with N SNs depicted as columns and M disks depicted as rows in a column. Consider a column with two disk failures at positions/rows i and j , which cannot be recovered by internode recovery. Internode recovery is possible unless there are two failed disks at another SN column which are in the same positions i and j . The probability that HRAID1/1 fails due to four disk failures

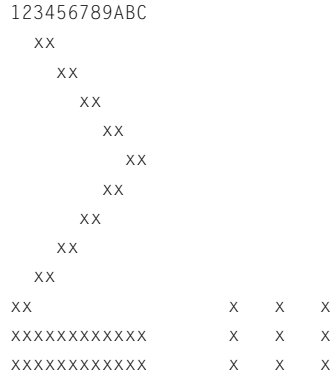


FIGURE 11.8 HRAID2/2 with $N = M = 12$ and $k = \ell = 2$ and with $d_{max} = 12(2 + 2) - 2 \times 2 = 44$ and $min + 1 = (k + 1)(\ell + 1) = 9$ as shown on the right (twelve disks or columns A = 10, B = 11, C = 12).

for $N = M = 12$ is:

$$Prob[Data_loss] = \frac{\binom{N}{2}\binom{M}{2}}{\binom{D}{4}} = 0.00025.$$

The number of disk failures leading to data loss in HRAID k/ℓ regardless of N and M is $(k + 1)(\ell + 1)$, so that all disk failure up to and including d_{min} can be tolerated.

$$d_{min} = (k + 1)(\ell + 1) - 1.$$

For $k = \ell = 1$ the number of rectangles of four failed disks, that can be placed anywhere in the plane of $N \times M$ disks is $N(N - 1)/2 \times M(M - 1)/2$. This is because of failed disks that can be moved vertically and horizontally, as long as there are $k + 1$ failed disks in a row and $\ell + 1$ disks in a column. The probability of data loss for HRAID k/ℓ with $N \times M$ disks is given as:

$$P_{(N \times M)}^{(k, \ell)} = \frac{\binom{N}{k+1}\binom{M}{\ell+1}}{\binom{N \times M}{(k+1)(\ell+1)}} \quad (11.5)$$

Given that HRAID k/ℓ tolerates k node failures and ℓ disk failures per node on surviving nodes, at most d_{max} disk failures can be tolerated.

$$d_{max} = k \times M + (N - k) \times \ell = N(k + \ell) - k\ell,$$

where we have set $M = N$. Given that there are $N(k + \ell)$ check disks in the best case fewer disk failures can be tolerated, so that the overall code is not MDS. It should be emphasized that MDS provides protection for all disk failure configurations, while this is a best case scenario.

Example 5 (with $N = M = 12$ and $\ell = k = 2$ to illustrate d_{max}). Consider HRAID2/2 with $K = \ell = 2$ and $N = M = 12$. Each one of the $N = 12$ nodes constituting columns of $M = 12$ disks (or rows) in the following figure. Each node can tolerate up to $\ell = 2$ failed disks via intranode recovery. Additionally the remaining $M - \ell$ rows can tolerate $k = 2$ disk failures via internode recovery. Intranode recoveries should be carried out first, before attempting intranode recovery.

There is a large difference between the maximum and minimum of the number of disk failures that can lead to data loss. For $N = M = 12$ (d_{min}, d_{max}) are given as follows:

(4 : 23) for HRAID1/1, (9 : 44) for HRAID2/2, refer to Fig. 11.8, and (16, 63) for HRAID3/3.

Monte-Carlo simulation can be used to obtain the mean number of disk failures (d_{mean}) leading to data loss (Thomasian, 2013).

https://en.wikipedia.org/wiki/Monte_Carlo_method

The number of failed disks to data loss increases with k and ℓ .

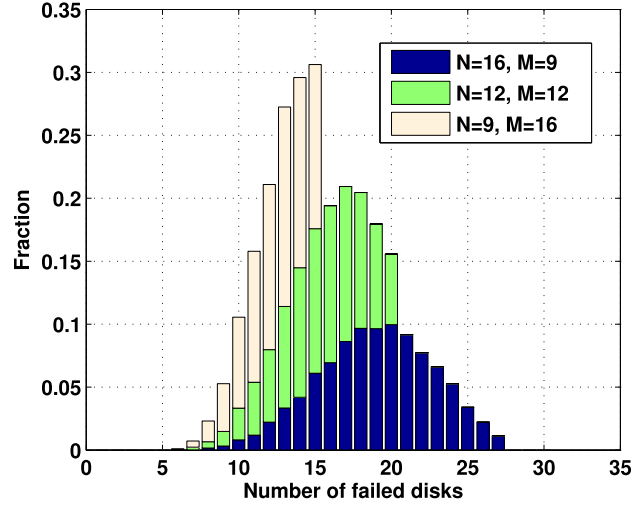


FIGURE 11.9 Number of failed disks leading to data loss with intranode recovery only for three HRAID1/2 configurations with 9×16 , 12×12 , and 16×9 disks. Different colors indicate frequencies in each case.

The simulation to determine d_{mean} also determines the *Mean Time to Data Loss - MTDDL*. The following relation should hold. Note $D = N \times M$ is the total number of disks.

$$MTDDL = \sum_{i=1}^{d_{mean}} \frac{1}{(D-i)\delta}. \quad (11.6)$$

Simulation results in Fig. 11.9 show the distribution of failed disks leading to data loss for HRAID1/2 with intranode recovery only. There are no controller failures and given $D = N \times M = 144$ disks we set $N = 9$ and $M = 16$; $N = M = 12$, and $N = 16$ and $M = 9$. The number of failed disks leading to data loss increases for higher values of N , since failed disks are spread over a larger number of nodes, although the number of disks per node is smaller. This effect does not arise when internode recovery is in effect.

11.9 Shortcut reliability analysis of HRAID

We apply the method in (Thomasian, 2006a), which was presented in Chapters 4 and 10 to obtain approximate reliability expression for RAID($4 + \ell$), $\ell \geq 1$ with N disks. Let $r = 1 - \epsilon$ denote the reliability of each disk, where $\epsilon \ll 1$ is the disk unreliability. Let R_ℓ denote the approximate reliability of RAID($4 + \ell$) with R_0 for RAID0, e.g., in the case of RAID5 which can tolerate one disk failure:

$$R_1 = r^N + N(1-r)r^{N-1} = (1-\epsilon)^N + N\epsilon(1-\epsilon)^{N-1} \approx 1 - \binom{N}{2}\epsilon^2 + 2\binom{N}{3}\epsilon^3 - \dots$$

It is shown in (Thomasian, 2006a) that the smallest power n in ϵ^n determines the minimum number of disk failures leading to data loss, which is two in this case. In other words RAID5 can tolerate a single disk failure. The approximate reliability equation for RAID($\ell + 4$), $\ell \geq 1$ obtained by induction is:

$$R_\ell \approx 1 - \binom{N}{\ell+1}\epsilon^{\ell+1} + (\ell+1)\binom{N}{\ell+2}\epsilon^{\ell+2} - \dots$$

Note that the first term indicating the probability of data loss due to $\ell + 1$ disk failures is subtracted from one.

Given that $R_{k/\ell}(N, M)$ denote the reliability of HRAID k/ℓ with N nodes and M disks as affected by disk failures only, the reliability of HRAID1/0(N, M) and HRAID0/1(N, M) can be expressed by first noting that $R_1 = r^M + M(1-r)r^{M-1}$ and $R_0 = r^M$:

$$R_{0/1}(N, M) = (R_1)^N \approx 1 - \frac{NM(M-1)}{2}\epsilon^2;$$

$$R_{1/0}(N, M) = (R_0)^N + N(1 - R_0)(R_0)^{N-1} \approx 1 - \frac{N(N-1)M^2}{2}\epsilon^2.$$

It follows that $d_{min} = (k+1)(\ell+1) = 2$ disk failures may lead to data loss in both cases and that $R_{0/1} > R_{1/0}$.

It follows from the expressions for approximate reliability for HRAID1/2 and HRAID2/1 that both can fail with $(k+1)(\ell+1) = 6$ disk failures.

$$R_{1/2}(N, M) = R_2^N + N(1 - R_2)R_2^{N-1} \approx 1 - \frac{N(N-1)M^2(M-1)^2(M-2)^2}{72}\epsilon^6 + \dots$$

$$R_{2/1}(N, M) = R_1^N + N(1 - R_1)R_1^{N-1} + \binom{N}{2}(1 - R_1)^2R_1^{N-2} \approx 1 - \frac{N(N-1)(N-2)M^3(M-1)^3}{24}\epsilon^6 + \dots$$

HRAID1/2 is more reliable than HRAID2/1, since $R_{1/2}(N, M) > R_{2/1}(N, M)$ implies $N > 2 + (M-2)^2/(3M(M-1))$, which is true for reasonable values of N . To determine the probability that HRAID1/2 encounters data loss with the sixth disk failure, consider the configuration with one failed node due to three failed disks and another node with two failed disks. The probability of a disk failure at this node is $p_{1/2} = (M-2)/D_S$ with $D_S = (N-2)M + M - 2$. In the case of HRAID2/1 array consider two nodes with two failed disks each, so that their data can only be reconstructed via the internode check code. Data loss occurs when a third node, which already has a failed disk, encounters a second disk failure. The probability of this event is $p_{2/1} = (M-1)/D_S$. The inequality $p_{1,2} < p_{2,1}$ leads to $N + M > 4$, which is always true.

11.10 Simulation to estimate the MTDL

The simulation study in (Thomasian et al., 2012) considers both disk and controller, but not interconnection network failures. Controller failures are tantamount to the failure of all of its disks, since they will become inaccessible if the controller fails.

The time to disk failures is assumed to be exponentially distributed, because of its mathematical tractability in that it can be incorporated into Markovian reliability models (Trivedi, 2002; Trivedi and Bobbio, 2017), but it also simplifies the programming of the simulation as specified below.

A node is designated to be failed when the controller fails or the number of disk failures at the node exceeds ℓ , since failed disks at the SN will not be accessible. With Option I this leads to data loss and for Option II the SN is considered to have failed. With a functioning controller nonfailed disks can be used for recovery with Option III.

In the case of Option I with no internode redundancy data loss occurs as soon as one of the SNs fails. In the case of Option II data loss occurs if the number of failed SNs exceeds k . In the case of Option III data loss occurs when a disk block cannot be recovered via multistep recovery. We ignore other failures such as the failures in the interconnection network, which may be duplexed for higher reliability.

We simplify the simulation study for reliability analysis by assuming that controller failure similarly to disk failure are exponentially distributed. The discussion in (Thomasian et al., 2012) deals with a closed self-repairing system in which no external resources are introduced when failures occur. Disk capacity for repair is provided in the form of intranode and internode check strips, which are utilized via restriping, so that the number of node and disk repairs is limited by k and ℓ , which determine the number of restripings possible.

Repairs in the form of restriping at the intra-node level are assumed to be instantaneous, so we ignore the effect of operation in degraded mode while restriping is in progress. This is because disk rebuild time or *Mean Time to Repair - MTTR* is much smaller than the MTTF. This effect is also ignored when rebuild due to SN failures is in effect, although this would require the rebuild of data on M disks.

We set the disk to a million hours: $MTTF = 1/\delta = 10^6$ and vary the controller failure rate (γ) as a multiple of δ . The fact that all failures are exponentially distributed we use the superposition property of Poisson processes to simplify the simulation (Trivedi, 2002), (Kleinrock, 1975), i.e., the overall failure rate Ω is set to the sum of the failure rates of the current state to generate the time to next event or failure, since only disk and controller failures are modeled in the simulation study and repairs (restripings) at the intranode and internode level are assumed to be instantaneous. A probabilistic argument is used to identify the failed component based on the ratios of failure rates of components and Ω .

Hierarchical modeling discussed in Chapter 16 in (Trivedi and Bobbio, 2017) was used in the intelligent storage brick system at IBM (Fleiner et al., 2006). This remains an area of further investigation in the context of HRAID.

Procedure: Mean Time to Data Loss

Inputs: N : The number of storage nodes - SNs.

M : The number of disks per node.

$D = N \times M$: Total number of disks.

k : Internode redundancy level ($k = 0$ for Option I).

ℓ : Intranode redundancy level, $\ell = 0$ for RAID0, RAID(4 + ℓ), $1 \leq \ell \leq 3$.

δ : Disk failure rate, $\delta = 10^{-6}$, so that MTTF = $1/\delta = 10^6$ hours.

γ : Controller failure rate, multiple of δ .

Outputs (with initializations):

$F_{cd} = 0/1$ data loss occurred due to controller/disk failure.

$N_c = 0$: Number of failed controllers.

$N_{df} = 0$: Number of disks that actually fail,
since disks attached to a failed controller are inaccessible and are considered failed.

Simulation Variables: $Clock = 0$: Simulation time initialized.

Two states: 0 for failed and 1 for operational.

$C[0 : N - 1]$: The state of controllers, initially $C[n] = 1, \forall n$.

$Disks[0 : N - 1, 0 : M - 1]$: The state of $D = N \times M$ disks.

($Disks[n, m] = 1, \forall n, \forall m$): Initial disk states.

$N_n = 0$: Number of failed nodes initially.

$F[0 : N - 1]$: Number of failed disks at node n (initialize $F[n] = 0, \forall n$).

Ω : Failure rate for a given configuration.

T_{NF} : Time to next failure.

$N_c = 0$: Number of failed controllers.

$N_d = 0$: Number of failed disks.

$u_i : i^{th}$ uniformly distributed pseudo-random variable in (0, 1).

1: Set total failure rate:

$$\Omega = (N - N_c)\gamma + (D - N_d)\delta.$$

Compute time to next failure and increment $Clock$:

$$T_{NF} = (-1/\Omega)\ln(u_1),$$

$$Clock += T_{NF}.$$

2: Probability of a controller failure: $p = (N - N_c)\gamma / \Omega$.

If $u_2 \leq p$ then controller failure: goto Step 4.

Else disk failure: goto Step 4.

3: Determine failed controller: $n = \lfloor N \times u_3 \rfloor$.

If $C[n] == 0$ then regenerate u_3 and recompute n ,

else { $C[n] = 0, N_c ++, N_n ++, D[n, m] = 0, 0 \leq m \leq M - 1$,

$t = M - F[n]$ disk inaccessible, so $N_d = N_d + t$ }.

If $N_n > k$ then { $F_{cd} = 0$, goto Step 6 }, else goto Step 1.

4: Determine the index of the failed disk:

$$t = \lfloor D \times u_4 \rfloor.$$

Node#: $n = \lfloor t/M \rfloor$; Disk#: $m = \text{mod}(t, M)$.

If $Disks[n, m] == 0$ (disk already failed) regenerate u_4 and recompute n and m ,

else { $Disks[n, m] = 0, N_d ++, N_{df} ++, F[n] ++$ }.

If $F[n] \leq \ell$ goto Step 1.

else { $C[n] = 0, t = M - F[n], N_d = N_d + t, N_n ++$ }.

If $N_n \leq k$ then go to Step 1, else $F_{cd} = 1$.

5: Return($Clock, F_{cd}, N_c, N_{df}$).

TABLE 11.4 MTTDL, $\overline{N}_c$, $\overline{N}_{df}$, F_{cd} , $\gamma = 0$.

$\ell \backslash k$	0	1	2	3
0	6.9 ± 0.241 0, 1.0 0%	14.5 ± 0.215 0, 2.0 0%	23.0 ± 0.234 0, 3.0 0%	31.9 ± 0.4469 0, 4.0 0%
1	36.9 ± 0.4038 0, 5.2 0%	58.9 ± 0.742 0, 8.0 0%	78.4 ± 0.631 0, 10.3 0%	97.7 ± 0.6056 0, 12.3 0%
2	81.7 ± 0.823 0, 11.3 0%	118.8 ± 0.830 0, 15.8 0%	148.7 ± 1.0108 0, 19.2 0%	176.8 ± 1.0300 0, 21.8 0%
3	139.6 ± 0.878 0, 18.8 0%	191.5 ± 1.351 0, 24.7 0%	231.8 ± 0.845 0, 28.8 0%	268.1 ± 1.088 0, 32.1 0%

TABLE 11.5 MTTDL, $\overline{N}_c$, $\overline{N}_{df}$, F_{cd} , $\gamma = 1$.

$\ell \backslash k$	0	1	2	3
0	6.5 ± 0.1915 0.08, 0.9 7.9%	13.3 ± 0.2106 0.16, 1.8 8.2%	20.8 ± 0.3052 0.25, 2.8 8.9%	29.4 ± 0.4673 0.34, 3.7 10.2%
1	27.6 ± 0.5234 0.33, 4.0 33.4%	47.9 ± 0.5379 0.57, 6.5 23.1%	65.6 ± 0.5179 0.76, 8.6 19.8%	82.9 ± 0.5670 0.96, 10.4 19.6%
2	49.8 ± 0.5761 0.59, 6.9 58.6%	83.8 ± 0.9875 0.99, 11.2 39.6%	113.0 ± 1.0503 1.31, 14.4 32.3%	138.3 ± 0.9103 1.59, 16.9 27.7%
3	65.4 ± 0.8160 0.78, 8.9 78.0%	116.6 ± 1.2231 1.34, 15.1 56.5%	157.2 ± 0.9471 1.80, 19.5 44.2%	193.4 ± 1.2399 2.16, 23.0 37.2%

The MTTDL is obtained for $I_1 = 1000$ system failures, i.e., a run that starts with a fault-free system until data loss occurs to measure time to data loss. This repeated $I_2 = 10$ times to obtain the MTTDL using the batch means method with a confidence interval within 5% of the mean with a 95% confidence level (Lavenberg, 1983).

The MTTDLs are reported in thousands of hours. We also obtain the percentage of cases data loss is due to disk or controller failures and the mean number of failed disks and controllers at the point, where data loss occurs over $I = I_1 \times I_2 = 10,000$ runs. We vary the controller failure rate in millions of hours.

Tables 11.4, 11.5, 11.6, and 11.7 were obtained using Option II that the failure of more than k nodes leads to data loss. Option I corresponds to $k = 0$ which is the first column in each table, which implies that the failure of more than ℓ disks at a node or controller failures lead to data loss. The number following forward slash in the following list are simulation results based on Option II, which stands for single step internode recovery.

The first row in every box stands for the MTTDL, the second row represents the mean number of failed controllers and disks, and the third row is the percentage of cases data loss occurred due to controller failures.

The following conclusions can be drawn.

- For Option I (column I in the tables for $k = 0$), the MTTDL increases for higher RAID levels: RAID($4 + \ell$), $1 \leq \ell \leq 3$. The MTTDL in this case could be simply obtained as the minimum of the MTTDLs of N SNs.

TABLE 11.6 MTTDL, $\overline{N}_c$, $\overline{N}_{df}$, F_{cd} , $\gamma = 2$.

$\ell \backslash k$	0	1	2	3
0	6.0 ± 0.1069 0.14, 0.9 14.1%	12.5 ± 0.1433 0.29, 1.7 14.7%	19.3 ± 0.1624 0.45, 2.5 16.0%	27.0 ± 0.3590 0.63, 3.4 17.7%
1	22.2 ± 0.3821 0.53, 3.2 52.8%	39.5 ± 0.5462 0.94, 5.5 40.8%	56.0 ± 0.3895 1.29, 7.2 34.8%	71.2 ± 0.4451 1.62, 8.9 33.0%
2	33.6 ± 0.5991 0.80, 4.9 79.6%	63.0 ± 0.6417 1.46, 8.3 65.0%	87.8 ± 0.8221 1.98, 11.2 53.9%	112.0 ± 0.6069 2.46, 13.6 47.8%
3	38.5 ± 1.1412 0.94, 5.3 93.6%	77.0 ± 1.1448 1.75, 10.2 82.4%	111.1 ± 1.4974 2.47, 14.0 71.2%	145.1 ± 0.6855 3.11, 17.0 62.8%

TABLE 11.7 MTTDL, $\overline{N}_c$, $\overline{N}_{df}$, F_{cd} , $\gamma = 4$.

$\ell \backslash k$	0	1	2	3
0	5.2 ± 0.0676 0.25, 0.7 25.1%	10.7 ± 0.1379 0.51, 1.5 25.9%	16.6 ± 0.2160 0.79, 2.2 27.9%	23.2 ± 0.3010 1.06, 2.9 28.3%
1	15.3 ± 0.2858 0.74, 2.2 73.9%	29.4 ± 0.5501 1.35, 4.0 61.1%	42.4 ± 0.3605 1.91, 5.5 55.2%	55.5 ± 0.4007 2.41, 6.9 52.4%
2	19.6 ± 0.3353 0.94, 2.8 93.9%	39.1 ± 0.5872 1.80, 5.3 86.0%	59.4 ± 0.8077 2.59, 7.5 78.3%	77.8 ± 0.5353 3.30, 9.4 72.1%
3	20.6 ± 0.4492 0.99, 2.9 98.9%	42.2 ± 0.4286 1.95, 5.7 96.2%	66.2 ± 0.8631 2.88, 8.4 92.3%	90.2 ± 0.9244 3.74, 10.8 86.7%

Expected value of the MTTDL for Option I

Let's assume SN's are RAID7 arrays which upon failure restripe transitioning to RAID6, RAID5 and then RAID0. A further failure will lead to data loss and we are interested in the MTTDL given M disks per SN and a disk failure rate δ . we have the transition rates $M\delta$, $(M - 1)\delta$, $(M - 2)\delta$, and $(M - 3)\delta$, which leads to data loss. The corresponding means and variances are: $m_i = 1/((M - i)\delta)$ and $v_i^2 = 1/((M - i)\delta)^2$, $0 \leq i \leq 3$. The mean and variance of the time to data loss at each SN is the sum of the means and the variances, which we denote as M and V .

The mean and variance of a Gamma distribution with a probability density function

$$f(x) = \frac{\beta(\beta x)^{(\alpha-1)}}{\Gamma(\alpha)} e^{-\beta x}$$

has a mean α/β and variance α/β^2 .

https://en.wikipedia.org/wiki/Gamma_distribution

It follows $\alpha = \beta M$, so that $\beta = V/M$ and $\alpha = V$. Alternatively:

$$f(x) = \frac{1}{\Gamma(k)\theta^k} x^{k-1} e^{-\frac{x}{\theta}}.$$

The lower incomplete Gamma function is $\gamma(s, x) = \int_x^\infty t^{s-1} e^{-t} dt$.

https://en.wikipedia.org/wiki/Incomplete_gamma_function

Given N SNs data loss occurs as the first SN fails. The expected value of the minimum of $n = N$ random variables with the Gamma distribution is:

$$E(\min(x_1, \dots, x_n)) = \int_x^\infty P[\min(x_1, \dots, x_n) \geq t] dt = \int_0^\infty (X_1 > t)^n dt = \int_0^\infty \left(\frac{\Gamma(k, \frac{t}{\theta})}{\Gamma(k)} \right)^n dt = \frac{\theta}{[\Gamma(k)]^n} \int_0^\infty \Gamma(k, u)^n du.$$

For large N the Fisher-Tippett-Gnedenko theorem states that the minimum can be approximated by a Gumbel distribution so that the minimum should be close to:

<https://math.stackexchange.com/questions/1511899/expected-value-of-minimum-of-n-gamma-functions>

$$E(\min(X_1, \dots, X_n)) = \frac{1}{n(n+1)} \sum_{k=1}^n E[X_k] \approx \frac{[X_k]}{n+1}.$$

Approximating the time to failure can be approximated with a normal distribution $\mathcal{N}(\mu, \sigma^2)$ with pdf and *Cumulative Distribution Function - CDF* followed by the standard CDF.

$$\phi(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}.$$

$$\Phi(x) = \frac{1}{2} \left[1 + \operatorname{erf}\left(\frac{x-\mu}{\sigma\sqrt{2}}\right) \right].$$

$$\operatorname{erf}(x) = \frac{2}{\sqrt{\pi}} \int_0^x e^{-t^2} dt.$$

Given the standard normal case $Z_1, Z_2 \dots Z_n$ iid $\mathcal{N}(0, 1)$ then the *Probability Distribution Function - PDF* of the minimum is:

$$P(Z_{(1)} \leq z) = 1 - P(Z_{(1)} > z) = 1 - P(Z_1 > z, \dots, Z_n > z) = 1 - [1 - \Phi(z)]^n$$

Differentiation yields the pdf

$$f_1(z) = \frac{df}{dz} [1 - \Phi(z)]^n = n[1 - \Phi(z)]^{n-1} \phi(z).$$

Numerical methods can be used to obtain the mean based on the standard distribution (Trivedi, 2002).

- With Option II increasing ℓ has the most effect when the controllers are highly reliable or do not fail at all ($\gamma = 0$).
- Increasing the value of k results in a significant improvement in the MTDDL.
- With Option I for $0 \leq \gamma \leq 2$ HRAID k/ℓ provides a higher MTDDL for $\ell > k$, but there is a crossover point for $\gamma = 4$.
- Higher values of k and ℓ attain a dramatic increase in MTDDL. Comparing the MTDDL for $k = \ell = 0$ with $k = \ell = 3$ there is a 39-fold increase for $\gamma = 0$, 30-fold for $\gamma = 1$, 24-fold for $\gamma = 2$ and 17.3-fold for $\gamma = 4$.
- For the most reliable configuration considered in this study ($k = \ell = 3$) the MTDDL for $\gamma = 0, 1, 2, 4$ with Option II is: 268.1, 194.1, 145.1, 90.2 kilohours. When the controller failure rate is high, little advantage is to be gained by internode recovery.
- The fraction of cases that data loss occurs due to controller failures decreases with k and increases with γ as would be expected. This fraction is larger with increasing ℓ .
- Referring forward to the elapsed time given as intervals in the barcharts in Section 11.11, it can be seen that Option III does result in a significant increase in MTDDL with respect to Option II. (numerical results are provided in Section 11.11. As long as the controller is not broken nonfailed disks can be used in recovery.

11.11 Multistep recovery in HRAID

With an interconnection network designed for HRAID operation in normal mode, rebuilding SNs via restriping may take an excessive amount of time. HRAID operating with Option III substitutes rebuild processing with on demand reconstruction of requested data blocks. This option may be appropriate for archival storage systems with low access rates. There are opportunities to power down some disks and nodes to save energy, an issue elaborated in Chapter 7.

Internode on demand reconstruction is not possible with more than ℓ disks failures at a node with Option II, but is possible with Option III given less than k SNs have failed.

In what follows we set controller failure rates to zero, so that only disk failures are considered. In what follows HRAID is represented as a square, where each column represents one of the N SNs, and there are M rows for the disks at each SN. The occurrence of data loss can be determined by considering all columns left to right and attempting to recover all failed disk with intranode recovery first followed by internode recovery.

Example 6 (*HRAID1/1 with six failed disks*). Determine the recovery cost to access Disk X, which is the last disk to be have failed, given that disks A-E failed before.

D		C
	X	B
E	A	

FIGURE 11.10 Failed disks are denoted by capital letters. Disk is a column are at the same SN which is a RAID5 and cannot recover from two disk failures.

In Fig. 11.10 as disk X fails, disk A in the same column (node) and disk B in the same row as X are already broken, so that one step recovery at the intranode or the internode level is not possible, since $k = \ell = 1$.

A cannot be recovered via internode recovery since E is broken,

E cannot be recovered via intranode recovery since D is broken,

D cannot be recovered via internode recovery since C is broken,

C cannot be recovered via intranode recovery since B is broken,

B cannot be recovered via internode recovery since X is broken, hence data loss.

As noted earlier the minimum number of disk failures leading to data loss $d_{min} = 4$ with the four disks constituting a rectangle (with an appropriate displacement), but here six disks are involved leading to data loss.

Also note that disks are allowed to participate in recovery, while due to failure of more than ℓ disks at a node, not all of its disks are recoverable. An SN with greater than ℓ disk failures is assumed to be broken with Option II.

Example 7 (*HRAID1/1, $N = M = 12$, $d_{max} = 23$ failed disks*). We consider multistep recovery in HRAID1/1 with $N = M = 12$ with $d_{max} = N(k + \ell) - k\ell = 23$ failed disks as shown in Fig. 11.11. The number of recovery steps is maximized when there are two disk failures per row with the exception of disk C, which lends itself to internode recovery. To recover a block on disk Z which is the last one to fail, we start the recovery with disk C and work our way backwards.

Z	1	1	1	1	1	1	1	1	1	1	Y
1	V	1	1	1	1	1	1	1	1	U	1
1	1	R	1	1	1	1	1	1	Q	1	1
1	1	1	N	1	1	1	1	M	1	1	1
1	1	1	1	J	1	1	I	1	1	1	1
1	1	1	1	1	F	D	1	1	1	1	1
1	1	1	1	1	1	C	1	1	1	1	1
1	1	1	1	1	G	1	H	1	1	1	1
1	1	1	1	K	1	1	1	L	1	1	1
1	1	1	0	1	1	1	1	1	P	1	1
1	1	S	1	1	1	1	1	1	T	1	1
1	W	1	1	1	1	1	1	1	1	1	X

FIGURE 11.11 HRAID1/1 with $N = 12$ SNs and $M = 12$ disks per SN. Capital letters specify failed disks and 1 stands for healthy disks.

A Greedy Heuristic for recovery was used for enumerating the number of recovery steps. In Table 11.8 we provide counts for the number of times up to ten recovery steps were required per failed disk over 10,000 iterations. These counts are less accurate for higher numbers of recovery steps, because such cases are infrequent. The number of single step recoveries increases rapidly for higher k and ℓ .

From Figs. 11.12, 11.13, 11.14 it is observed that the mean number of recovery steps increases with the progression of simulation time, but remains smaller than 1.5 in all cases.

The simulator used in this section differs from the simulator in Section 11.10 in the following ways:

- (i) There are no controller failures and hence no disks are considered to be failed due to this reason.
- (ii) Internode recovery attempts the recovery of failed nodes required for the recovery of the last disk to have failed, since it is postulated that this disk is being accessed. We are interested in the recovery cost of this node only.

TABLE 11.8 Number of recovery steps after data loss over 10000 iterations.										
Steps	1	2	3	4	5	6	7	8	9	10
HRAID1/1	64687	5458	1770	401	77	10	1	2	0	0
HRAID1/2	130193	16101	4111	627	232	78	32	6	0	0
HRAID1/3	204894	16735	15161	3444	608	187	77	29	9	2
HRAID2/1	93458	3829	1600	382	66	3	0	0	0	0
HRAID2/2	163767	20171	4141	1194	324	66	16	5	0	0
HRAID2/3	230993	39750	12942	2760	850	243	55	27	4	2
HRAID3/1	121253	1747	887	335	51	7	0	0	0	0
HRAID3/2	202704	13176	6471	1168	257	44	3	3	0	0
HRAID3/3	269170	44596	11069	2895	738	170	41	11	0	0

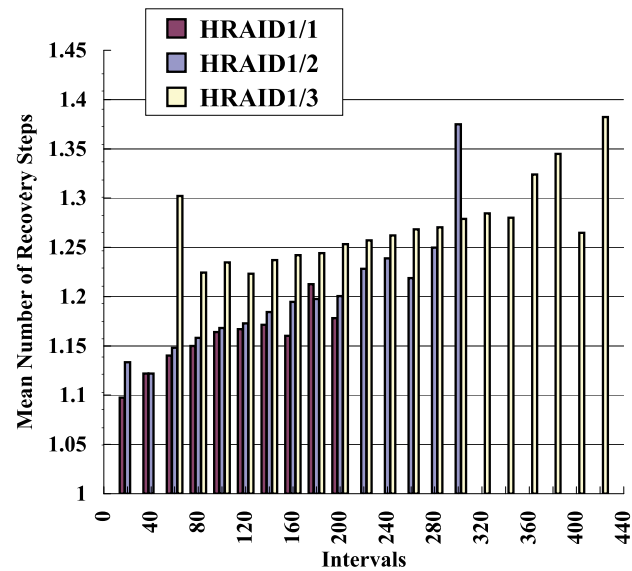


FIGURE 11.12 The mean number of recovery steps for successive intervals for HRAID1/* under Greedy Heuristic.

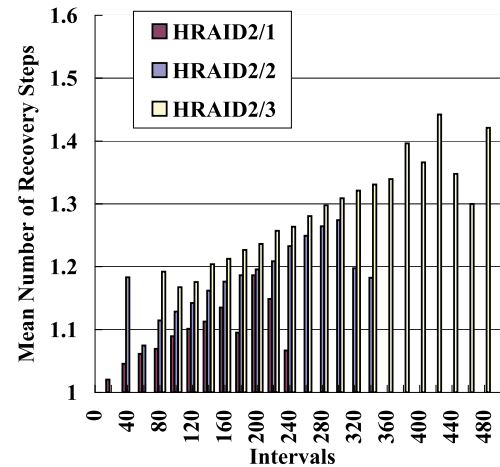


FIGURE 11.13 The mean number of recovery steps for successive intervals for HRAID2/* under Greedy Heuristic.

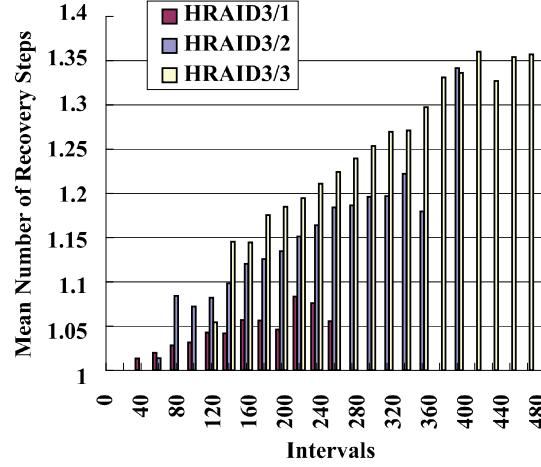


FIGURE 11.14 The mean number of recovery steps for successive intervals for HRAID3/* under Greedy Heuristic.

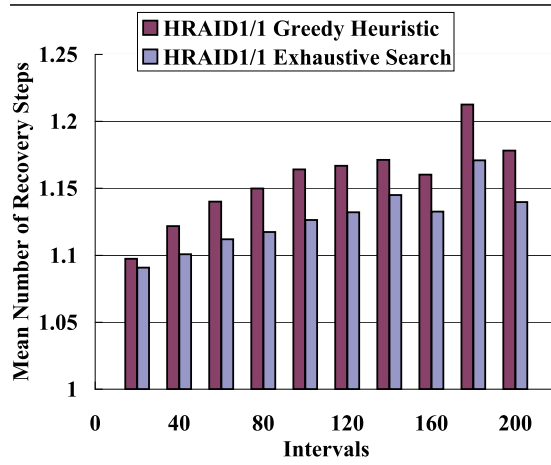


FIGURE 11.15 The mean number of steps under Greedy Heuristic vs Exhaustive Search for HRAID1/1.

Let $F[n]$ denote the number of failed disks at the n^{th} node ($1 \leq n \leq N$) and $G[m]$ the number of disks failed at m^{th} row, where there are as many rows as rows disk per SN ($1 \leq m \leq M$).

Both are incremented when a disk with coordinates (n, m) fails.

If $F[n] \leq \ell$ then recover using intranode coding and exit.

If $G[m] \leq k$ then recover using internode coding and exit.

If $F[n] - \ell \geq G[m] - k$ then insert the ids of failed disks in the n^{th} column into a stack, followed by the ids of failed disks in the m^{th} row. Reverse the order otherwise, placing the disk ids which are least costly to recover on top of the stack.

If successful count the number of steps to recovery. Otherwise insert the ids of additional disks and try to recover them recursively. If a disk cannot be recovered pop the disk id from the stack. Continue recovery procedure until stack empty.

Example 8 (HRAID1/1, Disk A fails). To determine the number of recovery steps to recover disk A, which is the last one to have failed in Fig. 11.16, we start the recovery with the less costly coordinate. In this case there are two failed disks in the y coordinate, versus three in the x coordinate. The two disks in the y coordinate (E and F) are placed at the top of the stack, hence {B, C, D, E, and F}. Disk A can be recovered after recovering disks E and F, a total of three recovery steps, but recovering E and F requires 6 XORs (3 per row), while the recovery of B, C, D also requires 6 XORs. Note that intranode recovery is preferable even if it involves more disks, since it does not require internode communication.

Example 9 (HRAID1/1, Disk A fails). Same as Example 3, but with two additional failed disks G and H in Fig. 11.17.

In recovering F we need to recover H, which should be added to the stack, but then both H and F can be popped from the stack (7 XORs). Similarly two steps are required for G and E (6 XORs). The total number of recovery steps including A is five and 13 XORs. Otherwise it takes four steps if B, C, and D are recovered before A (6 XORs).

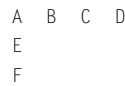


FIGURE 11.16 HRAID(1,1) with $N = 4$ SNs and $M = 3$ disks per SN.

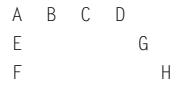


FIGURE 11.17 HRAID(1,1) with $M = 3$ and eight failed disks.

An exhaustive search method to determine the minimum number of recovery steps. It follows from Fig. 11.15 that there is little difference in the number of recovery steps of the two methods. The exhaustive search method is not worthy of further consideration given the small number of recovery steps and the small improvement with respect to the greedy heuristic.

From Fig. 11.18, 11.19, 11.20 we report the average number of accesses required per failed disk for the intervals given in the figures for HRAID1/ k/ℓ with $1 \leq k \leq 3$, $1\ell \leq 3$. The mean number of accesses to surviving disks increases as more failed disks are encountered and it exceeds four as the system approaches data loss.

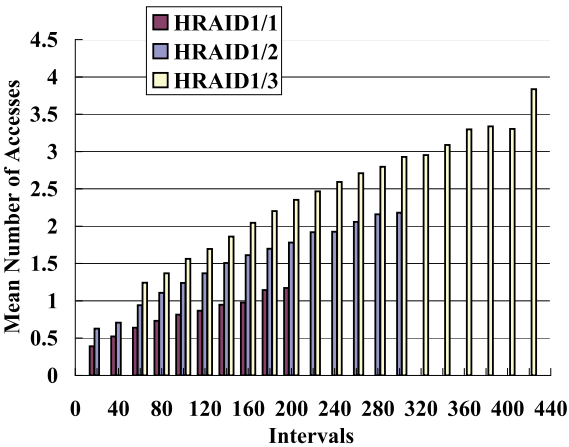


FIGURE 11.18 Average number of extra accesses per surviving disk for HRAID1/*.

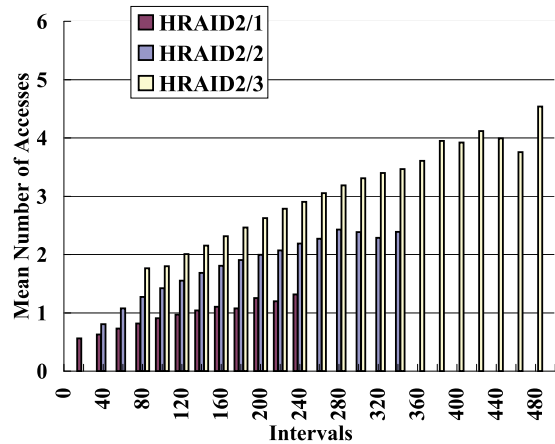


FIGURE 11.19 Average number of extra accesses per surviving disk for HRAID2/*.

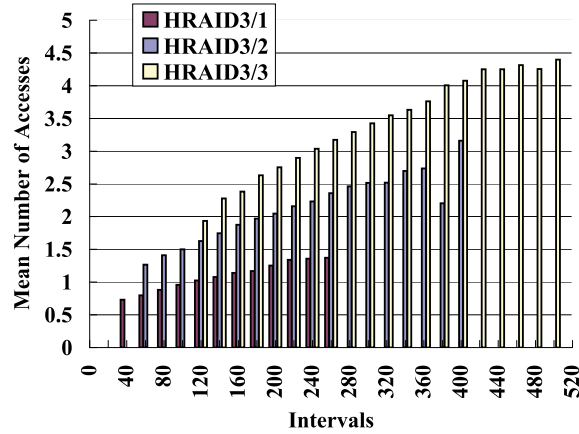


FIGURE 11.20 Average number of extra accesses per surviving disk for HRAID3/*.

These figures also reveal the time t_i when the i^{th} simulation ends. While n_i can be obtained directly, given t_i , the number of failed disks n_i in simulation run i can be obtained from the following equation.

$$t_i = \sum_{j=1}^{n_i} [(D - j)\delta]^{-1}.$$

The values of n_i provide the distribution of the failed disks at the time data loss occurs.

In what follows we list the MTDLs in thousands of hours for Options I, II, and III for HRAID k/ℓ disk arrays with no controller failures ($\gamma = 0$).

HRAID1/{1/2/3} **II** = (59/119/191.5) **III** = (200/280/440)

HRAID2/{1/2/3} **II** = (78/149/232) **III** = (240/340/480)

HRAID3/{1/2/3} **II** = (98/177/268) **III** = (260/400/500)

There is a significant improvement in MTDL proceeding from Option II to III, which is only true for no controller failures. A simple explanation is that Option III utilizes nonfailed disks at nodes, even when the number of failed disks has exceeded ℓ and the whole node would have been considered failed.

11.12 Related work

The following discussion is more or less chronological.

Pei Cao et al. under the direction of John Wilkes at HP Labs undertook the TickerTAIP project (Cao et al., 1994c). In traditional RAID the centralized DAC - Disk Array Controller is a single point of failure and limits the maximum number of disks to which the DAC can scale. TickerTAIP is a parallel architecture that distributes controller functions across several loosely coupled processors. The result is better scalability, fault tolerance, and flexibility. TickerTAIP's controllers coordinated their activities to implement the RAID5 paradigm in updating data blocks.

Data could be written according to RAID0, RAID4, and RAID5 formats, as shown in Fig. 11.21.

	node 0	node 1	node 2	node 3
Stripe 0	(-, -, -, u)	(-, -, -, u)	(2, 0, 3, d)	(-, 0, -, p)
Stripe 1	(4, 1, 2, d)	(5, 1, 2, d)	(-, 1, -, p)	(3, 1, 2, d)
Stripe 2	(-, -, -, u)	(-, 2, -, p)	(6, 2, 1, d)	(7, 2, 1, d)

FIGURE 11.21 A write request spanning logical blocks 2:7, each node contains logical and physical block number (row), node to send parity to, and a block type: d = data, p = parity, u = unused). Parities are placed diagonally. Only stripe 1 is a full stripe.

Given a modified data block d_{new} TickerTAIP considered three options to compute and update the parity.

At originator. d_{new} is sent to the data disk to overwrite d_{old} . In addition d_{new} is XORed with d_{old} , which is read from the disk before it is overwritten to compute $d_{diff} = d_{old} \oplus d_{new}$. d_{diff} is sent to the parity disk, where it is to be XORed with p_{old} to compute $p_{new} = p_{old} \oplus d_{diff}$, which overwrites p_{old}

Solely parity. d_{new} is sent to both the data and parity disks, d_{old} is read from the data disk and sent to the parity disk and is overwritten by d_{new} . p_{old} is read from the parity disk and XORed with d_{new} and d_{old} to compute p_{new} which overwrites p_{old} . Uses more messages than other schemes.

At parity. d_{old} is read from the data disk and XORed with d_{new} to compute $d_{diff} = d_{old} \oplus d_{new}$. d_{new} overwrites d_{old} and d_{diff} is sent to the parity disk. The parity disk reads p_{old} , computes $p_{new} = p_{old} \oplus d_{diff}$, and writes it on the parity disk. Simulation results show that (3) yields the highest throughput.

The *two-phase commit - 2PC* protocol is used to ensure write atomicity (Ramakrishnan and Gehrke, 2002). There is also the issue of how dependencies among requests are handled, e.g., metadata has to be committed before the corresponding data, but strict serialization will degrade performance.

Concurrency Control in storage systems was reported by Khalil Amiri, Garth A. Gibson and Richard A. Golding at CMU's PDL in (Amiri et al., 2000). This study defines base storage txns for atomic updating of data blocks and associated parities. It is shown by simulation that *TimeStamping Ordering - TSO* has a superior performance with respect to locking, although TSO exhibited inferior performance in (Bernstein, 2017).

Network RAID protocols were investigated at IBM/ARC in (Kenchammana-Hosekote et al., 2004). The choice of protocol depends on the relative performance of client systems to storage bricks, the semantic guarantees desired, and the offered workload. The protocols were classified according to the data path between clients and storage, the method used for serializing operations, and the method used to ensure atomicity and failure tolerance. It was determined that under normal conditions pessimistic and optimistic serialization methods provide similar performance.

Hierarchical RAID was a hardware implementation by Sung Hoon Baek et al. at Jungwon U., S. Korea in (Baek et al., 2001). Lower level RAID controllers, which are treated as virtual disks by higher level RAID controllers. The RAID5 paradigm is applied across virtual disks, while each virtual disk follows the RAID5 paradigm internally. It takes four disk failures placed in a rectangle for data loss to occur in HRAID5/5. The storage efficiency for HRAID5/5 is given as $S_e = (N - 1)(M - 1)/D$, e.g., $S_e = 81\%$ for $N = M = 10$, which is used to show that S_e is maximized at $N = M$. A *Unified Modeling Language - UML* diagram (Booch, 2017) was used to calculate the delay for updating data blocks.

Grid Codes were proposed by Mingqiang Li (IBM China, *Chinese U. HK - CUHK*, Lenovo, Huawei, Kanzhun Ltd.) and Jiwu Shu at Tsinghua U. Grid codes use coding in two dimensions to protect strips arranged in two dimensions to attain a higher reliability (Li et al., 2009). Horizontal (H) and Vertical (V) *Parity Code - PC* or HVPC code is the simplest Grid code consisting of a *Single Parity Code - SPC* over k_1 rows and k_2 columns, so that the horizontal and vertical parity strips can be computed as follows:

$$P_{i,*} = D_{i,0} \oplus D_{i,1} \oplus \dots \oplus D_{i,k_2-1}, \quad i = 0, 1, \dots, k_1 - 1;$$

$$P_{*,j} = D_{0,j} \oplus D_{1,j} \oplus \dots \oplus D_{k_1-1,j}, \quad j = 0, 1, \dots, k_2 - 1.$$

The diagonal parity is

$$P_{*,*} = \bigoplus_{i=0}^{k_1-1} P_{i,*} = \bigoplus_{i=0}^{k_2-1} P_{*,i}$$

Grid codes are based on parity codes, which allow XOR processing and consider up to three disk failures. X-code and SPC are two codes which can act vertically to protect against one and two disk failures, while SPC, EVENODD, and STAR are codes that can act horizontally and vertically to protect against one to three failures. X-code has the limitation that it requires a prime number of disks, while the other codes are flexible that an arbitrary number of disks filled with zeroes can be postulated. The overall code is not MDS, which is also the case for HRAID.

Given that H and V codes can protect against K and L strip failures, all $(K + 1)(L + 1) - 1$ strip failures can be tolerated. In other words $t = (K + 1)(L + 1)$ is the lowest number of strip failures that will lead to data loss. For $K = L = 1$ data loss occurs if the four strips constitute a rectangle.

The 2-dimensional GRID codes listed in (Li et al., 2009) are summarized in Table 11.9. Since $k_1 = k_2$ needs to be a prime number for the X-code, we have modified the example in (Li et al., 2009), so that instead of $k_1 = k_2 = 10$ we use $k_1 = k_2 = 11$ with the total number of strips: $D = k_1 \times k_2 = 121$. Then the redundancy ratio is the number of check strips divided by D .

Federated Array of Bricks - FAB project at HP Labs reported in (Saito et al., 2004) dealt with a collection of bricks, where brick was a storage appliance containing disks, CPU, NVRAM, and *Network Interface Cards - NICs*. The goal was to provide the reliability of traditional arrays with lower cost and better scalability. A majority-voting algorithm was used to replicate or erasure-code logical blocks across bricks and a reconfiguration algorithm to move data in the background when bricks are added or removed. It is argued that voting is practical and necessary for reliable, high-throughput storage

TABLE 11.9 The maximum number of failures (t) and the redundancy percent ($r\%$) associated with miscellaneous Grid codes.

Codes	t	Redundancy ratio ($r\%$)
(SPC,SPC)	3	$(k_1 + k_2 - 1)/D = 17.35\%$
(SPC,EO)	5	$(2k_1 + k_2 - 2)/D = 25.6\%$
(SPC, X-Code)	5	$(k_1 + 2k_2 - 2)/D = 25.6\%$
(SPC,STAR)	7	$(3k_1 + k_2 - 3)/D = 33.8\%$
(EO,EO)	8	$(2k_1 + 2k_2 - 4)/D = 33\%$
(STAR,X-Code)	11	$(3k_1 + 2k_2 - 6)/D = 40.5\%$
(STAR,STAR)	15	$(3k_1 + 3k_2 - 9)/D = 47.1\%$

systems. Brick failures and recoveries are handled smoothly without disturbing client requests. A 22-node FAB Linux cluster was prototyped.

11.13 Collective Intelligent Bricks - CIB or Icecube project at IBM

The CIB or Icecube project at IBM/ARC was led by Winfred Wilcke with the goal to build a multiterabyte maintenance-free storage system. An early prototype was built (Wilcke et al., 2006), but the plan to productize was abandoned.

<https://www.computerworld.com/article/2557569/ibm-almaden-research-center-s-intelligent-bricks-and-kybos-software-supersma.html>

The goals of the Icecube project were stated as follows:

- (1) Easier system management and reduced *Total Cost of Ownership - TCO*.
- (2) Better scalability at low cost.
- (3) Better environments - floor space, power, cooling, and noise.
- (4) Order of magnitude improvements.

Some features of the brick architecture are as follows:

- (1) Hardware organized as bricks, where each brick does processing, storage, and networking.
- (2) Small number of brick types, preferably one with $20 \times 20 \times 20$ centimeter dimensions.
- (3) Network protocol for inter-brick communication.
- (4) Bricks are low cost commodity components and failures are dealt with at the higher system level.
- (5) Bricks are allowed to fail in place without impacting system operation noticeably as long as most of the bricks survive.

Bricks have couplers on each of their six sides, which are connected to an 8-port switch chip. There is a PCI bus with an x86 CPU and 12 MB memory. Connected to the bus is a switch connecting twelve 2.5 inch 80 GB disks, which is 960 GB or approximately one TB. A prototype with 27 bricks provides 26 TB, i.e., ≈ 1 TB per brick.

Cooling is accomplished using a coolant liquid or air.

An array of storage bricks can be conceptualized in three dimensions. It consists of live, dead, recently dead, backup bricks, and 2nd generation backup data, usually on bricks close to recently dead bricks.

From Richard Golding's private communication on 9/20: "Data on failed bricks was to be rebuilt across all blocks and given the Icecube hardware had a 3d mesh interconnect. The hope was that this would give a lot of bandwidth between pairs of nodes. In practice it did not work that way, maybe because multipath packet routes did not work correctly, because of the limitations of the Ethernet switches that connected each node to its neighbors. The result was that rebuilds were slower than hoped."

Simulations to estimate the reliability of Icecube systems were carried out by Claudio Fleiner, Robert Garner, James Hafner et al. at IBM ARC in (Fleiner et al., 2006). Plotting the percentage of live bricks in a 3-D mesh system it was observed that up to 40% of bricks may die before the system is non-linearly degraded. 2-D meshes were shown to be even less resilient to failures.

1DFT (RAID5) at the node level and 2NFT at the node level provides 40-75% storage efficiency.

0DFT in bricks and 3NFT at brick level provides 50-75% storage efficiency.

Bit serial data is transferred via capacitive couplers on bit surfaces.

The 8b/10b signal encoding not allowing more than five consecutive 1's or 0's in transmitting by Alfred Widmer and Peter Franaszek at IBM was used to ensure synchronization over Ethernet, Fibrechannel, and Infiniband (Widmer and Franaszek, 1983).

Position server maps brick addresses onto (x,y,z) coordinates.

Cube shortest path routing rather than a spanning tree protocol is used.

https://en.wikipedia.org/wiki/Spanning_Tree_Protocol

General parallel File Systems - GPFS now known as Spectrum Scale

https://en.wikipedia.org/wiki/IBM_Spectrum_Scale

and Kybos (Wong et al., 2005) were two file systems that ran on Icecube.

GPFS provides the fundamental I/O functionality (Schmuck and Haskin, 2002),

- Reads/writes data from/to disks, replicates data as required, provides parallelism and interface to external world, etc.
- Each brick contains one instance of GPFS's *Network Storage Device - NSD* service.

Kybos introduced by Theodore Wong, Richard Golding, Joseph Glider et al. at IBM/ARC was a distributed storage system configuration management tool that prevents performance specification violations and system shutdown (Wong et al., 2005). Kybos provides self-management as follows:

- Translates “resource pools” into GPGS actions and resource configuration.
- Provides system monitoring services for its own and human use.
- Adapts to failures and new resources by controlling GPFS configuration and actions.

Kybos can translate simple, high-level specifications into brick-level enforcement targets, It invokes centralized algorithms only when an action requires global state.

K. K. Rao, James Hafner (rip), and Richard Golding at IBM/ARC analyzed the reliability of the Icecube system in (Rao et al., 2006, 2011). The system is complex in that each node participates in multiple redundancy sets. Hierarchical reliability modeling is used to estimate the MTDDL while taking into account LSEs, but not the effect of IDR and disk scrubbing. Self-repair is carried out via restriping, where the number of restripings is constrained by the redundancy level. Intranode and internode redundancies are varied to meet reliability goals. It is postulated that failures are limited to node hardware and disk drives, but not switches and links. The reliability target is expressed in terms of data loss events per petabyte-year, which can be deduced from the MTDDL. The fact that failed bricks are not removed or replaced may result in some healthy nodes becoming inaccessible.

11.13.1 Reliability analysis of Icecube

The following notation is used:

N = number of nodes (higher model)

d = number of drives in the array

λ_N = node failure rate (higher model)

λ_d = drive failure rate = $1/MTTR_d$

μ_d = drive repair rate = $1/MTTF_d$

μ_N = node rebuild rate (higher model)

$h_d = h = (R - 1) \times C \times HER$ = probability of an uncorrectable error during rebuild

$h_N = d \times h$ probability of an uncorrectable error during node rebuild

C = disk capacity R = redundancy set size

HER = disk hard error rate expressed per number of bytes read

The Markovian model of a RAID5 array with undetectable errors can be represented with three states S_i with the following transition rates, where i in state S_i represents the number of failed disks.

$$S_0 \xrightarrow{d(1-h)\lambda_d} S_1, \quad S_0 \xrightarrow{dh\lambda_d} S_2$$

$$S_1 \xrightarrow{(d-1)\lambda_d} S_2, \quad S_1 \xrightarrow{\mu_d} S_0$$

Using the analysis in Chapter 4 Given that $\mu_d \gg \lambda_d$ the RAID5 MTDDL is give as follows.

$$MTDDL = \frac{(2d - 1 - dh)\lambda_d + \mu_d}{d(d-1)\lambda_d^2 + d(d-1)\lambda_d\mu_d h} \approx \frac{\mu_d}{d(d-1)\lambda_d^2 + d(d-1)\lambda_d\mu_d C \times HER} \quad (11.7)$$

Data “objects” are stored across multiple nodes in such a system to meet performance and reliability requirements. Each data object constitutes exactly one stripe of data, so that the parity can be computed entirely from this data. For a given data object, the set of nodes that contain the data and its corresponding redundancy (parity) elements constitutes a redundancy set. The node set is the set of all the nodes in the storage system.

Data is evenly distributed across all the nodes in the storage system, thus, each node has one or more redundancy set relationships with every other node in the node set. The total number of redundancy sets of size R in a node set of size N is given by $\binom{N}{R}$. The even distribution of data implies that the failure domain is the entire node set and not just individual redundancy sets.

For a system in which the nodes have internal RAID, We represent the RAID array internal to a node in a Markov model and obtain array failure rates from it. We then use these failure rates in a higher level Markov model representing the redundancy arrangement between nodes. Nodes are not amenable to service actions and that on a drive failure, the array is restriped removing the failed drive from the array.

We define array failure as the failure of disk drives beyond the fault tolerance provided by the RAID scheme. λ_D , the rate of array failure and λ_S the rate of a sector error during a restripe can be obtained as follows:

$$\lambda_D(\text{RAID5}) \approx \frac{d(d-1)\lambda^2}{\mu_d}$$

$$\lambda_S \approx d(d-1)\lambda_d \times C \times \text{HER}$$

We will use these rates in the higher level model for the erasure codes between nodes. Markov models with nodes with internal RAID are as follows.

$$\begin{aligned} S_0 &\xrightarrow{N\lambda_N + \lambda_D} S_1 \\ S_1 &\xrightarrow{\mu_N} S_0 \quad S_1 \xrightarrow{(N-1)(\lambda_N + \lambda_D + \lambda_S)} S_2 \end{aligned}$$

Here N is the number of nodes in the node set, λ_N is the node failure rate and μ_N is the node rebuild rate. The array failure rate, λ_D , and the rate of sector error during a restripe, λ_S , correspond to the internal RAID in the nodes.

S_0 is when the system is fully operational. The system transitions to S_1 when either a node fails or a node experiences an array failure. In this state, the data of this node is rebuilt on the remaining nodes in the node set. S_2 represents the data loss state caused by a second node or array failure or a sector error during an internal RAID re-stripe while the node rebuild is in progress.

The MTDDL for the scheme with internal RAID and node FT = 1, FT = 2, and FT = 3 is given by:

$$\begin{aligned} MTDDL_{1R, NFT1} &\approx \frac{\mu_n}{N(N-1)(\lambda_N + \lambda_D)(\lambda_N + \lambda_D + \lambda_S)} \\ MTDDL_{1R, NFT2} &\approx \frac{\mu_N^2}{N(N-1)(N-2)(\lambda_N + \lambda_D)^2(\lambda_N + \lambda_D + k_2\lambda_S)} \\ MTDDL_{1R, NFT3} &\approx \frac{\mu_N^3}{N(N-1)(N-2)(N-3)(\lambda_N + \lambda_D)^2(\lambda_N + \lambda_D + k_3\lambda_S)} \end{aligned}$$

In configurations for nodes without internal RAID, individual drives within each node are used to realize the erasure code between nodes. We assume that no more than one drive per node is used in each redundancy set, that is, each block of a data stripe is on a different node; thus, each node failure causes only a single erasure on each redundancy set. The Markov model for nodes without internal RAID and an erasure code FT = 1 between nodes is:

$$\begin{aligned} S_0 &\xrightarrow{N} \lambda_N(1-h_N)S_1, S_0 \xrightarrow{N} d\lambda_d(1-h_d)S_2, S_0 \xrightarrow{N} (d\lambda_d h_d + \lambda_N h_N)S_3 \\ S_1 &\xrightarrow{(N-1)(d\lambda_d + \lambda_N)} S_3, \quad S_2 \xrightarrow{(N-1)(d\lambda_d + \lambda_N)} S_3 \\ S_1 &\xrightarrow{\mu_N} S_0, \quad S_2 \xrightarrow{\mu_d} S_0 \end{aligned}$$

In a rebuild the destination node receives all the required redundancy data and performs the necessary XOR. For a node set size of N , a redundancy set size of R and a fault tolerance of t , we express the amounts of data below in units of a node's worth of data. Note that this means that R nodes are involved in the rebuild of one lost data object.

Amount old data rebuilt by each node = $1/(N - 1)$.

Amount of data received at each node from other nodes for rebuild = $(R - t)/(N - 1)$.

Total data received by all $(N-1)$ nodes = $R - t$.

Total data sourced by each node = $(R - t)/(N - 1)$.

The effective rebuild time will be the maximum time required to move data in and out of nodes, to and from disks, and through the interconnecting network, depending on where the bottleneck lies.

Total data in and out of a node = $2 \frac{R-t}{N-1}$. The fraction of redundancy sets that are critical and hence, can contribute to a sector loss are represented in the k_2 and k_3 terms in the MTDDL expressions for internal RAID, $FT = 2$ and $FT = 3$ respectively. Each node is a part of $\binom{N-1}{R-1}$ redundancy set.

$$k_2 = \frac{\binom{N-2}{R-2}}{\binom{N-1}{R-1}} = \frac{R-1}{N-1} \text{ and } k_3 = \frac{\binom{N-3}{R-3}}{\binom{N-1}{R-1}} = \frac{(R-1)(R-2)}{(N-1)(N-2)}$$

The analysis of nodes without Internal RAID is omitted for the sake of brevity,

$MTTF_N$ = node MTTF = 400,000 hours

$MTTF_d$ = drive MTTF = 300,000 hours

HER = drive hard error rate = 1 sector in 10^{14} bits read

C = drive capacity = 300 GB

Maximum drive throughput = 150 I/O operations/sec.

Drive sustained transfer rate (average) = 40 MB/sec.

N = node set size = 64

R = redundancy set size = 8

d = 12 drives per node

Re-stripe command size = 1 MB

Rebuild command size = 128 KB

Link speed = 10 Gbps (800 MB/sec. sustained)

Capacity utilization = 75%

Bandwidth utilization for rebuild, re-stripe = 10%

For the specific target we have chosen in this paper, either the [FT2, Internal RAID 5] or the [FT3, No Internal RAID] configurations meet the reliability requirement with the condition that the rebuild block size is at least 64 KB.

Zygaria was developed at IBM/ARC by Theodore Wang, Richard Golding et al. to ensure adequate application performance by sharing the resources of the large-scale storage systems (Wong et al., 2006). A hierarchical I/O scheduling algorithm to manage performance resources on an underlying storage devices ensured that each pool and session receives at least a reserve rate of throughput and caps usage at a limit rate using hierarchical token buckets and *Earliest Deadline First - EDF* I/O scheduling. Once it has fulfilled the reserves of all active sessions and pools, unused throughput is shared fairly among active sessions and pools. Devices are assumed to perform their own low-level head scheduling and are not modeled in detail.

Bianca Schroeder (U. Toronto) and Garth Gibson (CMU, Vector Inst.) at CMU in addition to assessing the reliability of HDDs also tracked the relative frequency of hardware component replacements for the most frequently replaced components given in Table III in (Schroeder and Gibson, 2007a). For 3-out-of-10 systems considered, HDD replacements are 30.6%, 34.8%, and 49.1% for HPC1, COM1, and COM2 systems, respectively. Power supply and memory failures dominate in COM1 at 34.8%, and 20.1%, respectively. Table II in the paper summarizes the causes for failure as: CPU 44%, memory 29%, HDD 16%, PCI motherboard 9%, and power supply 2%, so that only one-sixth of system failure are due to HDDs.

11.14 Conclusions

The chapter dealt with HRAID design, performance, reliability, and recovery. Reported are simulation results for HRAID k/ℓ with three options.

Only intranode recovery is possible with Option I. This will lead to the lowest MTDDL, but has the lowest overhead for updating inter-node check blocks and does not require an interconnection network among SNs.

Option II allows both on demand intra- and internode recovery on demand for rebuild. Rebuild may take excessive time unless a high bandwidth interconnection network is provided.

Option III does not provide inter-node rebuild, so that a lower bandwidth interconnection network will suffice. It is more suitable for low performance environments, such as accessing archival storage.

Shortcut reliability analysis and simulation results indicate that $\text{HRAID}_{k/\ell}$ with $\ell > k$ provides a higher reliability when there are no controller failures or when data loss occurs due to disk rather controller failures.

Some topics discussed in (Thomasian and Tang, 2011a,b), but not repeated here are as follows:

- Performability combines performance with reliability. It was defined in (Thomasian and Xu, 2008) as the expected number of I/Os processed at full disk utilization until data loss occurs for a given workload. A higher MTDDL is achievable by increasing the degree of redundancy, but then a higher fraction of system resources are used to update check blocks lowering the maximum IOPS. There is a tradeoff between higher reliability and performance, e.g., RAID0 attains four times the RAID5 throughput in processing write requests but then its time to disk failure is lower than RAID5. Note that this happens in two phases for RAID5 with the failure of the first and second disk, so that unlike RAID0, RAID5 continues its operation until the second disk failure occurs.
- A design study to balance disk and controller reliabilities is reported in (Thomasian and Tang, 2011b). A more interesting problem is minimizing the overall cost, while meeting performance and reliability goals.

Some further areas of investigation are as follows.

- A more rigorous specification of HRAID data layouts when $N \neq M$, especially $N < M$.
- Data moves required for handling SN additions and removals to minimize data transfer cost.
- A more detailed specification of HRAID from the viewpoint of interaction with its clients, concurrency control, and commit processing for updating data blocks.
- Extending simulation to determine the MTDDL to take into account repair times.
- Application of a hierarchical reliability modeling to HRAID is an area of further investigation.
- Further demonstration that for higher reliability it is better to dedicate more redundancy at lower than higher levels.
- Extending HRAID to multiple levels to provide further scalability.

Chapter 12

Conclusions

Storage and processing of data has aspects touching many areas of computer science and engineering. Major developments are occurring in the following areas:

1. Processing big data through explosive growth of the volume of data.
2. Novel methods of storing and manipulating data, e.g., NoSQL.
3. Data deduplication, compression, and encryption and combinations of the latter.
4. Data allocation in memory hierarchies enhanced with flash memories, and generally SCMs - Storage Class Memories.
5. Novel coding methods for higher reliability are being developed for distributed storage but also SSDs.
6. Data-centric processing since the energy consumed by data movement exceeds computation.
7. Dealing with the blurring of memory versus storage distinction.
8. Disaggregated storage.

The need for long term data preservation is an extension of RAID reliability discussion, which is considered in (Baker et al., 2006).

Abstract

Many emerging Web services, such as email, photosharing, and web site archives, need to preserve large amounts of quickly-accessible data indefinitely into the future. In this paper, we make the case that these applications' demands on large scale storage systems over longtime horizons require us to re-evaluate traditional storage system designs. We examine threats to long-lived data from an end-to-end perspective, taking into account not just hardware and software faults but also faults due to humans and organizations. We present a simple model of long-term storage failures that helps us reason about the various strategies for addressing these threats in a cost-effective manner. Using this model we show that the most important strategies for increasing the reliability of long-term storage are detecting latent faults quickly, automating fault repair to make it faster and cheaper, and increasing the independence of data replicas.

Data preservation addressed by researchers at IBM ARC in (Gladney and Lorie, 2005) ensures data will be remained intelligible in the distant future. A virtual machine is used to represent and to render data. Simple data can be preserved as representations using well-known standards. Enough of a complete solution is presented to enable practical aggressive preservation programs.

Digital permanence investigated at Google in (Blum and Beyer, 2019) deals with techniques to anticipate and then meet the expected lifetime of data stored in digital media.

Provenance or lineage of data is specifiable as a graph, which describe "the relationships among all sources, processing steps, contextual information dependencies that contributed to the existence of a piece of data" (Carata et al., 2014). A rather comprehensive primer on provenance with references is provided.

Data cleaning was addressed by AT&T researchers in (Dasu and Johnson, 2003). "Data analysts at information-intensive businesses are frequently asked to analyze new data sets that are often dirty-composed of numerous tables possessing unknown properties. Prior to analysis, this data must be cleaned and explored, which is often a long and arduous task. Ensuring data quality is a notoriously messy problem that can only be addressed by drawing on methods from many disciplines, including statistics, exploratory data mining, database management, and metadata coding." Data cleaning is also known as data cleansing.

https://en.wikipedia.org/wiki/Data_cleansing

The authors of a recent book on data cleaning state that: "Dirty data across businesses and governments costs trillions every year" (Ilyas and Chu, 2019). Other books on this topic are (Olson, 2013), (Batini and Scannapieco, 2006), (Sadiq, 2013).

Appendix

Table of contents for the Appendix

- Books on topics related to computer storage in Section [A.1](#).
- ACM, IEEE, USENIX and their publications in Section [A.2](#).
- Journals, conferences, and workshops dealing with storage systems in Section [A.3](#).
- Workshops and conferences related to data base machines in Subsection [A.3.1](#).
- Web sites for trade publications in Section [A.4](#).
- Storage research in industry in Section [A.5](#).
- Storage research at universities in Section [A.6](#).
- Funding agencies, national labs, and research institutes in Section [A.7](#).

A.1 Books on topics related to storage

1. Computer organization, architecture, and fault-tolerance.
2. Operating systems.
3. Storage systems.
4. Coding for storage systems.
5. Database systems, machine learning.
6. High frequency trading.
7. Performance and reliability analysis.
8. Computer arithmetic.
9. Algorithms and data structures.
10. Peer to peer computing.
11. Computer history via interviews.

A.1.1 Computer organization, architecture and fault-tolerance

1. Jose' Duato and Sudhakar Yalamanchili and Lionel Ni, Interconnection Networks: An Engineering Approach (Duato et al., [2003](#)).
2. William James Dally (Stanford/Nvidia) and Brian Towles (D.E. Shaw Research). Principles and Practices of Interconnection Networks (Dally and Towles, [2004](#)).
3. Carl Hamacher (U. Toronto, Queen's U.), Zvonko Vranesic, Safwat Zaky, and Naraig Manjikian (Queen's U.). Computer Organization and Embedded Systems, 6th ed. (Hamacher et al., [2012](#)).
4. Andrew S. Tanenbaum (Vrije U.) and Todd Austin. Structured Computer Organization, 6th ed. (Tanenbaum and Austin, [2012](#)).
5. David Patterson (UCB, Google) and John Hennessy (Stanford U., emeritus). Computer Organization and Design - The Hardware/Software Interface, 5th ed. (Patterson and Hennessy, [2017](#)).
6. John Hennessey and David Patterson. Computer Architecture - A Quantitative Approach, 6th ed. (Hennessy and Patterson, [2017](#)).
7. Roberto Rojas-Cessa (NJIT). Interconnections for Computer Communications and Packet Networks Rojas-Cessa ([2017](#)). item William Stallings (independent). Computer Architecture and Organization, 11th ed. (Stallings, [2019](#)).
8. Israel Koren (U. Mass. emeritus) and C. Mani Krishna (U. Mass). Fault-Tolerant Systems, 2nd ed. (Koren and Krishna, [2020](#)).

9. Gerassimos Barlas. *Multicore and GPU Programming: An Integrated Approach*, 2nd ed., 2021.
10. Behrooz Parhami (UCSB). *Dependable Computing: A Multilevel Approach*.
https://web.ece.ucsb.edu/~parhami/text_dep_comp.htm

A.1.2 Operating systems

Most operating systems books discuss I/O systems, file organizations, memory hierarchies, and virtual memories:

1. James L. Peterson (U. Texas at Austin, IBM, retired) and Abraham Silberschatz (U. Texas, Bell Labs, Yale U.). *Operating System Concepts*, 1st ed., 1985.
2. Andrew Tanenbaum (Vrije U.) and Albert Woodhull (Hampshire College, Amherst, MA). *Operating Systems Design and Implementation*, 3rd edition (Tanenbaum and Woodhull, 2006).
3. Thomas Anderson (U. Washington) and Michael Dahlin (U. Texas, Austin). *Operating Systems Principles and Practice*, 2nd ed. (4 volumes) (Anderson and Dahlin, 2014).
4. Remzi and Andrea Arpaci-Dusseau (U. Wisc., Madison). *Operating Systems: Three Easy Pieces* (Arpaci-Dusseau and Arpaci-Dusseau, 2015).
5. Andrew Tanenbaum and Herbert Bos (Vrije U.). *Modern Operating Systems*, 3rd edition (Tanenbaum and Bos, 2017).
6. William Stallings (independent), *Operating Systems: Internals and Design Principles*, 9th ed. (Stallings, 2017b).
7. Abraham Silberschatz (Yale U.), Peter Baer Galvin (Starfish Storage), and Greg Gagne (Westminster College, Salt Lake City, Utah), *Operating System Concepts*, 10th ed. (Silberschatz et al., 2018).

A.1.3 Storage systems

1. Richard Matick (IBM, rip). *Storage Systems and Technology* (Matick, 1977).
2. Hugh Sierra (IBM, rip). *An Introduction to Direct Access storage Devices* (Sierra, 2012) (first published 1990).
3. Garth Gibson (UCB, CMU, Vector Inst.). *Redundant Disk Arrays - Reliable, Parallel Secondary Storage* (Gibson, 1992).
4. C. Denis Mee (IBM) and Eric D. Daniel, eds. *Magnetic Storage Handbook* (Mee et al., 1996).
5. Hai Jin (Huazhong U.), Toni Cortes (Poly. U. Catalunya), and Rajkumar Buyya (U. Melbourne, AU), eds. *High Performance Mass Storage and Parallel I/O: Technologies and Application* (Jin et al., 2001).
6. John Wilkes (HP, Google) et al. *Storage, Data, and Information Systems* (Wilkes et al., 2008).
7. Bruce Jacob (U. Maryland), Spencer W. Ng (IBM, HGST, SanDisk, retired), and David Wang (MetaRAM). *Memory Systems: Cache, DRAM, Disk* (Jacob et al., 2008).
 Rino Micheloni (Microsemi Corp.), Luca Crippa, and Alessia Marelli. *Inside NAND Flash Memories* (Micheloni et al., 2010).
8. Hai Li (NYU) and Yiran Chen (U. Pittsburgh). *Nonvolatile Memory Design: Magnetic, Resistive, and Phase Change* (Li and Chen, 2011).
9. Moinuddin K. Qureshi (GaTech), Sudhanva Gurumurthi (AMD), Bipin Rajendran (King's College London). *Phase Change Memory: From Devices to Systems* (Qureshi et al., 2011).
10. Santosh K. Kurinec (Rochester Inst. Tech.) and Krzysztof Iniewski (Redian Tech., Vancouver, Canada). *Nanoscale Semiconductor Memories: Technology and Applications* (Kurinec and Iniewski, 2013).
11. Bernard Dieny (Spintec), Ronald B. Goldfarb (NIST), and Kyung-Jin Lee (KAIST), eds. *Introduction to Magnetic Random Access Memory* (Dieny et al., 2017).

A.1.4 Coding for storage systems

1. W. Wesley Peterson (IBM, U. Hawaii, rip) and E. J. Weldon (U. Hawaii, emeritus). "Error-Correcting Codes" (Peterson and Weldon, 1972).
2. Florence J. MacWilliams (Bell Labs, emeritus) and Neil J. A. Sloane (AT&T, emeritus). *The Theory of Error-Correcting Codes* (MacWilliams and Sloane, 1977).
3. Bane Vasic (U. Arizona) and Erozan M. Kurtas (Seagate Tech.), eds. "Coding and Signal Processing for Magnetic Recording Systems" (Vasic and Kurtas, 2005).
4. Erozan M. Kurtas and Bane Vasic, eds. *Advanced Error Control Techniques for Data Storage Systems* (Kurtas and Vasic, 2006).

5. Eiji Fujiwara (Tokyo Inst. of Tech.). Code Design for Dependable Systems: Theory and Practical Applications (Fujita, 2006).
6. Mario Blaum (IBM Research). A Short Course on Error-Correcting Codes (Blaum, 2019).

A.1.5 Database systems and machine learning

1. Stefano Ceri and Giuseppe Pellagatti (Politecnico di Milano). Distributed Databases: Principles and Systems, first published in 1984 (Ceri and Pellagatti, 2011).
2. Jim Gray (IBM, Tandem, DEC, Microsoft, rip) and Andreas Reuter (Kaiserslautern U., U. Heidelberg). Transaction Processing: Concepts and Techniques (Gray and Reuter, 1992).
3. Raghu Ramakrishnan (U. Wisc., Yahoo!, Microsoft) and Johannes Gehrke (Cornell, Microsoft). Database Management Systems, 3rd edition (Ramakrishnan and Gehrke, 2002).
<https://pages.cs.wisc.edu/~dbbook/>
4. Shai Shalev-Shwartz (Hebrew U., Jerusalem) and Shai Ben-David (U. Waterloo). Understanding Machine Learning: From Theory to Algorithms (Shalev-Shwartz and Ben-David, 2014).
5. Nathan Marz (nathanmarz.com) and James Warren (LinkedIn). Big Data: Principles and Best Practices of Scalable Realtime Data Systems (Marz and Warren, 2015).
6. Ian Goodfellow (Google, Apple), Yoshua Bengio (U. Montreal), and Aaron Courville (U. Montreal). Deep Learning: Adaptive Computation and Machine Learning (Goodfellow et al., 2016).
7. Minos Garofalakis (Tech. U. Crete), Johannes Gehrke (Microsoft), and Rajeev Rastogi (Bell Labs, Amazon, India). Data Stream Management: Processing High Speed Data Streams (Garofalakis et al., 2016).
8. Ramesh Sharda (U. Oklahoma), Dyrson Delen (same), and Efraim Urban (emeritus). Business Intelligence, Analytics, and Data Science (Sharda et al., 2018) (book4you).
9. M. Tamer Ozsu (U. Waterloo) and Patrick Valduriez (INRIA). Principles of Distributed Database Systems, 4th ed. (Ozsu and Valduriez, 2020).
10. Andy Pavlo (CMU) and Joy Arulraj (GaTech). Non-Volatile Memory Database Management Systems (Arulraj and Pavlo, 2019).
11. Abraham Silberschatz (U. Texas at Austin, Bell Labs, Yale), Henry F. Korth (Bell Labs, LeHigh U.), and S. Sudarshan (Bell Labs, IIT Bombay). Database Systems Concepts, 7th ed. (Silberschatz et al., 2019).
12. Avrim Blum (CMU, ttic.edu/), John Hopcroft (Stanford U., emeritus), and Ravindran Kannan (Microsoft Research, India). Foundations of Data Science (Blum et al., 2020).

A.1.6 High frequency trading

1. Roy S. Freedman (NYU). Introduction to Financial Technology. Academic Press, 2006 (book4you).
2. Irene Aldridge (Cornell). High-Frequency Trading: A Practical Guide to Algorithmic Strategies and Trading Systems. John Wiley & Sons, 2009 (book4you).
3. Irene Aldridge. High-Frequency Trading: A Practical Guide to Algorithmic Strategies and Trading Systems. Wiley, 2013.
4. Irene Aldridge, Steven Krawciw. Real-Time Risk: What Investors Should Know About FinTech, High-Frequency Trading, and Flash Crashes. John Wiley & Sons, 2017 (book4you).
5. Irene Aldridge, Marco Avellaneda (NYU). Big Data Science in Finance. Wiley, 2021 (book4you).

A.1.7 Performance and reliability analysis

1. Leonard Kleinrock (UCLA, emeritus). Queueing Systems, Vol. I/II: Theory/Computer Applications (Kleinrock, 1975, 1976).
2. Arnold O. Allen (IBM Ed. Center, HP, rip). Probability Statistics and Queueing Theory with Computer Science Applications (Allen, 1978, 1990).
3. Hisashi Kobayashi (IBM Research, Princeton, emeritus). Modeling and Analysis: An Introduction to System Performance Evaluation Methodology (Kobayashi, 1978).
4. Kishor S. Trivedi (Duke U.). Probability and Statistics with Reliability, Queueing, and Computer Science Applications (Trivedi, 1982, 2002).
5. Hisashi Kobayashi and Brian L. Mark (George-Mason U.). System Modeling and Analysis: Foundations of System Performance Evaluation (Kobayashi and Mark, 2008).

6. William J. Stewart (NC State, emeritus). Probability, Markov Chains, Queues, and Simulation: The Mathematical Basis of Performance Modeling (Stewart, 1989).
7. Hisashi Kobayashi, Brian L. Mark, and William Turin (AT&T Research, consultant). Probability, Random Processes, and Statistical Analysis: Applications to Communications, Signal Processing, Queueing Theory and Mathematical Finance (Kobayashi et al., 2014).
8. Dror Feitelson (Hebrew U., Jerusalem). Workload Modeling for Computer Systems Performance Evaluation (Feitelson, 2015). .
<https://www.cs.huji.ac.il/~feit/wlmod/wlmod.pdf>
<https://www.cs.huji.ac.il/~feit/perf.html>
10. William J. Stewart (NC State, emeritus). Introduction to the Numerical Solution of Markov Chains (Stewart, 2021).

While there are discrete-event simulation languages, at no extra cost high level languages can be used instead, e.g. SimPY based on Python.

<https://simpy.readthedocs.io/en/latest/>

A.1.8 Computer arithmetic and logic design

1. Israel Koren (U. Mass, retired). Computer Arithmetic Algorithms (Koren, 2002).
2. Zvi Kohavi (Technion, Emeritus) and Niraj Jha (Princeton U.). Switching and Finite Automata Theory (Kohavi and Jha, 2009).
3. Milos Ercegovac (UCLA, emeritus) and Tomas Lang (UCI, rip). Digital Arithmetic (Ercegovac and Lang, 2003).
4. Behrooz Parhami (UCSB). Computer Arithmetic: Algorithms and Hardware Designs, 2nd ed. (Parhami, 2010).
5. Jean-Pierre Deschamps (U. Autònoma de Barcelona), Gustavo Sutter, and Enrique Cantó. Guide to FPGA Implementation of Arithmetic Functions, 2012 (Deschamps et al., 2012).

A.1.9 Algorithms and data structures

1. Thomas H. Cormen (Dartmouth U.), Charles E. Leiserson (MIT), Ronald L. Rivest (MIT), and Clifford Stein (Columbia U.). Introduction to Algorithms, 3rd edition, 2009 (Cormen et al., 2009).
2. Donald E. Knuth (Stanford U., emeritus). The Art of Computer Programming, Volume I: Fundamental Algorithms, 3rd ed. (Knuth, 1997b).
3. Donald E. Knuth. The Art of Computer Programming, Vol. II: Seminumerical Algorithms, 2nd ed. (Knuth, 1997a).
4. Donald E. Knuth. The Art of Computer Programming, Vol. III: Sorting and Searching, 2nd ed. (Knuth, 2005).
<https://www-cs-faculty.stanford.edu/~knuth/taocp.html>

A.1.10 Peer to peer computing

A comparison of P2P and grid computing is given succinctly here.

1. <https://www.cl.cam.ac.uk/teaching/2003/AdvSysTop/grid-p2p-paper.pdf>
2. Ian T. Foster (anl.gov) and Carl Kesselman (isi.edu). The Grid 2 - Blueprint for a New Computing Infrastructure, 2nd ed. (Foster and Kesselman, 2004).
3. Klaus Wehrle and Ralf Steinmetz (darmstadt.de). Peer-to-Peer Systems and Applications (Wehrle and Steinmetz, 2005).
4. Ian J. Taylor (Cardiff U.). From P2P to Web Services and Grids: Peers in a Client/Server World, (Taylor, 2015).
5. Ramesh Subramanian qu.edu, Brian D. Goodman (eds). Peer-to-peer computing: The evolution of a disruptive technology" (Subramanian and Goodman, 2005).
6. Xuemin Shen, Heather Yu, John Buford, and Mursalin Akon (eds). Handbook of Peer-to-Peer Networking (Shen et al., 2010).
7. Quang Hieu Vu, Mihai Lupu, and Beng Chin Ooi www.comp.nus.edu.sg. Peer-to-Peer Computing: Principles and Applications (Vu et al., 2010).
8. Yu-Kwong Ricky Kwok (U. Hong Kong). Peer-to-Peer Computing: Applications, Architecture, Protocols, and Challenges (Kwok, 2011).
9. Smitry Korzun [https://petsu.ru](http://petsu.ru) and Andrei Gurtov. Structured Peer-to-Peer Systems: Fundamentals of Hierarchical Organization, Routing, Scaling, and Security (Korzun and Gurtov, 2012).

A.1.11 Computer history via interviews

1. *Oral History* - OH section of *Charles Babbage Institute* - CBI of U. Minnesota.
<http://www.cbi.umn.edu/oh/index.html>
2. Libraries Digital Conservancy Home at UMN.
<https://conservancy.umn.edu>
3. CalTech Oral Histories.
<http://oralhistories.library.caltech.edu/>
4. Engineering Technology History Wiki of transcripts of interviews.
https://ethw.org/Main_Page
5. Aspects of computer history at *Computer History Museum* - CHM.
<https://computerhistory.org/>
6. Interviews at ACM Ubiquity magazine since 2000.
<https://ubiquity.acm.org/>
<https://www.edge.org/people> interviews with important people not just technology
7. Marianne Winslett interviews at ACM SIGMOD Record, quarterly since 1986.
<https://sigmodrecord.org>
8. IEEE Annals of the History of Computing, quarterly since 1992.
<https://www.computer.org/csdl/magazine/ah>
9. IEEE Computer History web.
<https://history.computer.org/>

A.2 ACM, IEEE, USENIX, and their publications

Association for Computing Machinery - ACM was founded in 1947. Its publications appear in its *Digital Library* - DL, with old publications scanned and digitized.

<https://dl.acm.org> require a paid subscription.

ACM has *Special Interest Groups* - SIGs which organize conferences and workshops in their field.

SIGARCH - Architecture, <https://www.sigarch.org/>.

SIGOPS - Operating Systems, <https://www.sigops.org/>.

SIGMOD - Management of Data, <https://sigmod.org/>.

SIGMETRICS - Performance Measurement, Modeling and Evaluation, <https://www.sigmetrics.org/>.

SIGWEB - Hypertext and the Web, <https://www.sigweb.org/>.

American Institute of Electrical Engineers - AIEE was founded in 1884.

[https://ethw.org/History_of_Institute_of_Electrical_and_Electronic_Engineers_\(IEEE\)_Standards](https://ethw.org/History_of_Institute_of_Electrical_and_Electronic_Engineers_(IEEE)_Standards)

Institute of Radio Engineers - IRE was founded in 1912. The *Institute of Electrical and Electronics Engineers* - IEEE resulted from the merger of AIEE and IRE on 1/1/1983. IEEE Spectrum is a monthly for all IEEE members and free for life members with combined age plus membership years exceeding one hundred.

<https://spectrum.ieee.org>

IEEE Transactions publishing storage related papers are:

<https://www.ieee.org/publications/index.html>

Computers, Information Theory, Parallel and Distributed Systems - TPDS, Dependable and Secure Computing - DSC, Knowledge and Data Eng. - TKDE, Big Data, Cloud Computing - CC, Services Computing, Magnetics, Reliability, Solid State Circuits.

IEEE has numerous societies including the Computer Society founded in 1946:

<https://www.ieee.org/communities/societies/index.html>

https://en.wikipedia.org/wiki/IEEE_Computer_Society

The IEEE Computer Society has a number of Technical Committees - TCs.

<https://www.computer.org/communities/technical-committees>

Computer Communications, <http://tab.computer.org/tcdp/>

Internet <http://tab.computer.org/tci/TCI-index.html>

Data Engineering, <http://tab.computer.org/tcde/>

Real Times Systems, <http://tab.computer.org/tcrts/>

Computer Architecture, <http://ieeetcca.org/>

Distributed Processing, <http://tab.computer.org/tcdp/>

High Performance Computing, <https://tc.computer.org/tchpc/>
 Data Engineering Bulletins published quarterly available at:
http://sites.computer.org/debull/bull_issues.html

USENIX is an association that was founded as “UNIX Users Group” in 1975, but has diversified immensely since its inception. It provides free access to conference papers and video, slides, and audio of its presentations.
<https://www.usenix.org>

The *Computer Measurement Group* - CMG was founded in 1974, It has replaced its annual CMG conference (1975-2013) with the annual Impact conference.
<https://cmgimpact.com/>

Institution of Engineering and Technology - IET was started in UK 150 years ago, but is now an international institution.
https://en.wikipedia.org/wiki/Institution_of_Engineering_and_Technology
<https://www.theiet.org/>

IEEE, ACM, and USENIX accept papers for publication, presentation or both. Many researchers post their articles on the web site of their institution or personal web sites for early dissemination. A web site available for early publication is arxiv.org.

Springer publishes books and periodicals in numerous fields. Springer Nature since January 2021 has committed to transition the vast majority of its not already Open Access journals to become *Transformative Journals* - TJ on the path to fully open access.
<https://www.springernature.com>

Creative Commons (CC) license is one of several public copyright licenses that enable the free distribution of an otherwise copyrighted “work”.
https://en.wikipedia.org/wiki/Creative_Commons_license

Other publishers of technical books are as follows. *Chemical Rubber Company* - CRC Press founded in 1903, which is now part of Routledge/Francis&Taylor and Informa in London.

<https://www.routledge.com/>
https://en.wikipedia.org/wiki/CRC_Press
Cambridge U. Press - CUP <https://www.cambridge.org/us/academic>
Oxford U. Press - OUP <https://global.oup.com/?cc:us>
<https://www.elsevier.com>

Morgan-Kaufman Publishers - MKP is now part of Elsevier.
<https://www.elsevier.com/books-and-journals/morgan-kaufmann>
 Academic Press is also part of Elsevier.
<https://www.elsevier.com/books-and-journals/academic-press>

A Resource Portal for AI, Big Data, New Data Management Technologies and Data Science.
<http://www.odbms.org/>

A bibliography of *database and logic programming* - dblp publications was started by Michael Ley at trier.de and was the main source of this book’s bibliography.

<https://dblp.uni-trier.de/dblp.org>
 These two web sites provide access counts and some downloads.
<https://scholar.google.com>
<https://www.researchgate.net/>
<https://semanticscholar.org>
<https://citeseerx.ist.psu.edu/index>

A.3 Journals, conferences, and workshops dealing with storage systems

Bell System Technical J. - BSTJ, quarterly, 1922-1996.

<https://www.bell-labs.com/our-research/technical-journal/>

Bell Labs Technical J. - BLTJ is published by Alcatel Lucent, since 1996 with varying number of issues till 2014.

<https://onlinelibrary.wiley.com/journal/15387305/>

IEEE Trans. on Computers, since 1952, monthly since 1968.

<https://www.computer.org/csdl/journal/tc>

IBM J. of Research & Development - R&D, quarterly since 1957, later bimonthly.

<https://ieeexplore.ieee.org/xpl/RecentIssue.jsp?punumber=5288520>

- Communications of the ACM* - CACM, monthly since 1958.
<https://cacm.acm.org>
- J. of the ACM* - JACM, quarterly since 1959, bimonthly since 1963.
<https://dl.acm.org/journal/jacm>
- IBM Systems J., quarterly 1962-2008, merged with IBM J. R.&D. in 2009.
<https://ieeexplore.ieee.org/xpl/RecentIssue.jsp?punumber=5288519>
- Proc. of the IEEE, monthly since 1963.
<https://www.proceedingsoftheieee.ieee.org>
- Int'l Conf. on *Massive Storage Systems and Technology* - MSST, annual since 1974. Sponsored by IEEE Computer Society (1988-2016), merged with 6th NASA Goddard Conf. in 1998, sponsored by U. Santa Clara, School of Engineering.
<https://www.storageconference.us>
- Information Systems, quarterly since 1975, 8 issues per year since 2013.
<https://www.journals.elsevier.com/information-systems>
- Int'l Conf. on *Very Large Data Bases* - VLDB annual since 1975.
<https://www.vldb.org>
- IEEE Data Engineering Bulletin, quarterly since 1978.
<https://tc.computer.org/tcde/data-engineering-bulletin/>
- Int'l Conf. on *Architectural Support for Programming Languages and Operating Systems* - ASPLOS, cosponsored by ACM SIGARCH and SIGOPS, biennial since 1982.
<https://dblp.uni-trier.de/db/conf/asplos/>
- USENIX *Annual Technical Conf.* - ATC, biannual 1983-94, annual since 1995.
<https://dblp.uni-trier.de/db/conf/usenix/>
- ACM *Trans. on Databases* - TODS, quarterly since 1976.
<https://tods.acm.org>
- ACM *Trans. on Computer Systems* - TOCS, quarterly since 1983.
<https://tocs.acm.org>
- J. Parallel and Distributed Computing* - JPDC, since 1984, varies.
<https://www.journals.elsevier.com/journal-of-parallel-and-distributed-computing>
- IEEE Int'l Conf. on *Data Engineering* - ICDE, annual since 1984, except 1985.
<https://ieeexplore.ieee.org/xpl/conhome/9093725/proceeding>
- J. Computer Science & Technology* - JCST, quarterly since 1986, bimonthly since 1995.
<https://www.springer.com/journal/11390>
- IEEE *Trans. on Knowledge and Data Engineering* - TKDE, quarterly since 1989, bimonthly since 1992, monthly since 2004.
<https://www.computer.org/csdl/journal/tk/>
- Int'l Workshop on High-Performance Transaction Systems - HPTS, biennial since 1985.
<http://www.hpts.ws/>
- Int'l Conf. for High Performance Computing, Networking, Storage and Analysis, IEEE/ACM SuperComputing - SC Conf., annual since 1988.
<https://sc21.supercomputing.org/>
- IEEE *Trans. on Parallel and Distributed Systems* - TPDS, bimonthly since 1990, monthly after 1993.
<https://dblp.uni-trier.de/db/journals/tpds/>
- Int'l Conf. on *Information and Knowledge Management* - CIKM, since 1992.
<https://dl.acm.org/doi/proceedings/10.1145/3340531>
- Very Large Data Base* - VLDB J., quarterly since 1992, bimonthly since 2008.
<https://www.springer.com/journal/778>
- USENIX Symp. on *Operating System Design and Implementation* - OSDI, biennial since 1994.
<https://dblp.uni-trier.de/db/conf/osdi/>
- USENIX *File and Storage Technologies* - FAST Conf., annual since 2002, except 2006.
<https://dblp.uni-trier.de/db/conf/fast/>
- Conf. on *Innovative Data Systems Research* - CIDR, biennial since 2003.
<http://cidrdb.org/cidr2020/>
- IEEE *Trans. on Dependable and Secure Computing* - TDSC, quarterly 2004-2011, bimonthly since 2011.
<https://www.computer.org/csdl/journal/tq>

- ACM *Trans. on Storage Systems - TOS*, since 2005, quarterly.
<https://tos.acm.org>
- Int'l Workshop on *Data Management on New Hardware - DaMoN*, annual since 2005.
<https://sites.google.com/view/damon2020>
- European Conf. on Computer Systems - Eurosys, annual since 2006 in Europe.
<https://www.eurosys.org>
- eXtremely Large Data Base - XLDB* Conf., annual since 2007.
<https://www.xldb.org/>
<https://en.wikipedia.org/wiki/XLDB>
- IEEE *Networking, Architecture, and Storage - NAS* Conf., annual since 2006, in China.
<https://dblp.uni-trier.de/db/conf/nas/>
- USENIX Workshop Power-Aware Computing and Systems - HotPower, biannual 2008-2014.
<https://www.usenix.org/conferences/byname/157>
- IEEE *Trans. Services Computing*, quarterly since 2008, bimonthly since 2015.
<https://www.computer.org/csdl/journal/sc>
- Trans. Large-Scale Data- and Knowledge-Centered Systems - TLSDKSC*, published as *Lecture Notes in Computer Science - LNCS* by Springer since 2009, Springer.
<https://dblp.uni-trier.de/db/journals/tlsdkcs/index.html>
- Proc. VLDB - PVLDB, monthly since 2008.
<https://www.vldb.org/pvldb>
- TPCTC - Technology Conference on Performance Evaluation and Benchmarking, annual since 2009, published by Springer.
<http://www.tpc.org/tpctc/default5.asp>
- USENIX *Workshop on Hot Topics in Storage and File Systems*, annual since 2009.
<https://www.usenix.org/conference/hotstorage2020>
- ACM SYSTOR: Int'l Systems and Storage Conf./The Israeli Experimental Systems Conf., annual since 2009 in Haifa, Israel.
<https://dblp.uni-trier.de/db/conf/systor/>
- Foundations and Trends in Databases, Joseph Hellerstein and Surajit Chaudhuri (editors), quarterly since 2009.
<https://www.nowpublishers.com/DBS>
- International Workshop on *Accelerating Data Management Systems Using Modern Processor and Storage Architectures - ADMS*, annual since 2010, in conjunction with VLDB.
<http://www.adms-conf.org/>
- Center for Memory and Recording Research - CMRR* at UCSD Non-volatile Memories Workshop - NVMW, annual since 2010.
<https://cmrr.ucsd.edu/events/nvmw.html>
- IEEE *Trans. on Cloud Computing*, since 2013, quarterly.
<https://www.computer.org/csdl/journal/cc>
- IEEE *Trans. on Big Data*, since 2015, quarterly.
<https://www.computer.org/csdl/journal/bd>

A.3.1 Workshops and conferences related to data base machines

DBMs dating back to 1960s were designed for efficient data processing, but required considerable CPU and I/O processing. An early step in offloading mainframes was IBM's auxiliary unit for array calculations (Ruggiero and Coryell, 1969). *Data Base Machines - DBMs* followed this trend. The first and second *Int'l Workshop on Database Machines - IWDM* <https://dblp.uni-trier.de/db/conf/iwdm/> held in Florence, Italy in 1981 and San Diego, CA in 1982. IWDM was held four more times: Munich (Leilich and Missikoff, 1983), U. Wisconsin, Madison in 1985 (DeWitt and Boral, 1985), Tokyo (Kitsuregawa and Tanaka, 1988), Dauville, France (Boral and Faudemay, 1989).
 Proc. of the *North Atlantic Treaty Organization - NATO Advanced Study Institute - ASI* Subseries F held in France, "Database Machines: Modern Trends and Applications" (Sood and Qureshi, 1986).

The impact of technology on the design of special-purpose DBMs is surveyed in “Parallel Architectures for Database Systems” (Hurson et al., 1989a).

Int’l Symposia on *Databases in Parallel and Distributed Systems - DPDS* were held twice: (Jajodia et al., 1988) and (Agrawal and Bell, 1990).

Parbase 1990 was the First Int’l Conference on Databases, Parallel Architectures, and Their Applications (Rishe et al., 1990a), which was followed by a postconference on Parallel Architectures (Rishe et al., 1990b).

A.4 Web sites for trade publications

Readers are encouraged to follow developments in the field by reading online trade publications, some of which are mentioned here. A subscription is required for some publications:

<https://www.anandtech.com>,
<https://arstechnica.com>,
<https://blocksandfiles.com>,
 Bloomberg <https://www.bloomberg.com>,
<https://www.businessinsider.com>,
cleantechnica.com,
<https://cloudwars.co>,
<https://www.computerweekly.com>,
<https://www.computerworld.com>,
<https://www.crn.com>,
 Database Trends and Application <https://www.dbta.com>,
 Datamation <https://www.datamation.com>,
 EE Times <https://www.eetimes.com>,
 Enterprise AI - Advanced Computing in the age of AI <https://www.enterpriseai.news>,
 Enterprise Storage Forum <https://www.enterprisestorageforum.com>,
<https://www.extremetech.com>,
 Forbes <https://www.forbes.com>,
<https://www.forbes.com/sites/tomcoughlin/#2a4a2cca4498>,
 Forrester <https://go.forrester.com/>,
 Gartner Group <https://www.gartner.com>,
 Geeksforgeeks <https://www.geeksforgeeks.org/>,
 High Performance Computing - HPC <https://www.hpcwire.com>,
<https://www.itjungle.com> (survive, adapt, thrive),
<https://www.networkworld.com>,
 Next Platform <https://www.nextplatform.com>,
 PC Magazine <https://www.pcmag.com/>,
 Popular Mechanics <https://www.popularmechanics.com/>,
 Real World Technology <https://www.realworldtech.com/>,
<https://searchstorage.techtarget.com>,
<https://www.seekingalpha.com>,
<https://semiengineering.com>,
<https://www.techradar.com>,
<https://www.techrepublic.com/>,
<https://www.techtarget.com>,
<https://www.theinquirer.net/> (mothballed Dec. 2019),
<https://www.theregister.co.uk>,
<https://www.tomshardware.com>,
<https://www.trustradius.com>,
<https://venturebeat.com/>,
<https://wikibon.com>,
<https://www.wired.com>,
<https://www.zdnet.com>.

A.4.1 Few interesting blogs

Jim Gray's web site at Microsoft.

<https://jimgray.azurewebsites.net/>

James Hamilton, former IBM architect at *Amazon Web services* - AWS.

<https://perspectives.mvdirona.com/>

Vincent Grancille has published the Data science and Machine Learning Repository.

<https://www.datasciencecentral.com/profiles/blogs/comprehensive-repository-of-data-science-and-ml-resources>

<https://www.kdnuggets.com/news/index.html>

<https://www.oreilly.com/radar/>

<https://storagemojo.com/> - on hiatus since 2019.

A.5 Storage research in industry

There have been major changes in computer and storage industry since (Thomasian, 2010a) was published. US companies with sizable research are/were are listed alphabetically.

Amazon Research: <https://www.aboutamazon.com/research>

Apple Research concentrates on *Machine Learning* - ML judging from its name.

<https://machinelearning.apple.com/>

Patent activities at Google and Apple are compared below.

https://getpocket.com/explore/item/the-real-difference-between-google-and-apple?utm_source=pocket-newtab

AT&T was formed in late 19th century becoming a telephone monopoly known as Ma Bell. AT&T was broken up in 1984 after a ten year anti-monopoly lawsuit, which resulted in seven Baby Bells. *Southwestern Bell Corp.* - SBC acquired AT&T Corp. in 2005 taking its name.

Bell Labs was established by Alexander Graham Bell in 1925.

en.wikipedia.org/wiki/Bell_Labs

Initially Bell Labs ownership was split between AT&T and Western Electric. In 1996 AT&T spun-off Bell Labs with most of its equipment manufacturing business to form Lucent. A small number of researchers were retained to create AT&T Labs, which was headed by Behzad Nadji till January 2006 after AT&T was acquired by SBC. In 2006 Lucent signed a merger with Alcatel and stopped doing basic research in 2008. In 2015 Nokia acquired Alcatel Lucent for \$16.6 Billion. Narain Gehani, a former employee, who later joined *New Jersey Institute of Technology* - NJIT in Newark, NJ wrote a book in praise of Bell Labs (Gerhani, 2003), but received poor reviews for his work.

<https://www.amazon.com/gp/customer-reviews/R1FIW5QJOGNV46>

<https://ubiquity.acm.org/article.cfm?id=764016>

<https://www2.dmst.aueb.gr/dds/pubs/Breview/2004-CR-Bell/html>

Cisco Research <https://research.cisco.com/>

Data Domain specializing in data deduplication for backup was founded in 2001. It was acquired in 2009 by EMC, which merged with Dell in 2016, hence Dell/EMC.

[https://en.wikipedia.org/wiki/Data_Domain_\(corporation\)](https://en.wikipedia.org/wiki/Data_Domain_(corporation))

<https://en.wikipedia.org/wiki/Dell EMC>

Facebook Research: <https://research.fb.com>

Google Research has branches in US and World-Wide.

<https://ai.google/research/>

<https://research.google/pubs/>

Deepmind founded in 2010 was acquired by Google in 2014 and Alphabet was made Google's parent company in 2015.

<https://en.wikipedia.org/wiki/DeepMind>

Hitachi Global Storage Technologies - HGST was a merger of Hitachi and IBM's storage businesses in 2003, which resulted in the transfer of half of IBM storage researchers to HGST. HGST was acquired by *Western Digital Corp.* - WDC in 2012, which also acquired SanDisk, the 3rd largest flash manufacturer in 2015.

Western Digital intends to exit dedicated storage business.

<https://www.anandtech.com/show/14890/western-digital-to-exit-storage-systems-sells-off-intelliflash-division>

HP was split into *HP Enterprise* - HPE and HP in 2014. the latter specializes in printers and personal computers and HPE in servers and storage.

<https://www.hpe.com/us/en/home.html>

HP labs conducts research in 3D printing, AI and emerging computing technologies such as *Quantum Computing* - QC.

<https://www8.hp.com/us/en/hp-labs/research/overview.html>

<https://www.labs.hp.com/hplabs/index>

Compaq which acquired Tandem Corp, and *Digital Equipment Corp.* - DEC was later acquired by HP. Tandem reports are available here.

Activities at HP and HPE research centers are discussed in Chapter 1.

<https://www.hpl.hp.com/hplabs/index/Tandem>

IBM research founded in 1945 has 3000 researchers in nineteen labs worldwide and leads in the number of patents, but not necessarily in highly ranked AI patents.

<https://www.research.ibm.com/labs/>

IBM nonconfidential research reports are posted here.

<https://dominoweb.draco.res.ibm.com/index.html>

Research reports at IBM's *Zurich Research Lab.* - ZRL on reliable and secure data availability are posted here.

<https://www.zurich.ibm.com/cloudstorage/>

Information about IBM products is available here.

<https://www.ibm.com/support/knowledgecenter/en/>

Intel Research

<https://www.intel.com/content/www/us/en/research/overview.html>

https://en.wikipedia.org/wiki/Intel_Research_Lablets

Intel is responsible for many generations of microprocessors starting in 1971.

<https://www.intel.com/content/www/us/en/homepage.html>

It recently introduced the Optane Memory technology.

<https://www.intel.com/content/www/us/en/architecture-and-technology/optane-memory.html>

Intel sold its flash business in October 2020 to SK Hynix for \$9 billion. In 2021 Pat Gelsinger rejoined Intel as its CEO.

https://en.wikipedia.org/wiki/Pat_Gelsinger He plans to catch up with *Taiwan Semiconductor Manufacturing Company* - TSMC in manufacturing capacity. Areas of Intel Research are discussed here.

<https://www.intel.com/content/www/us/en/research/overview.html>

Microsoft Research has a thousand researchers spread over multiple labs worldwide with the main lab in Redmond, WA. There are labs worldwide: Beijing China, Bangalore India, Montreal, Canada.

<https://www.microsoft.com/en-us/research/>

https://en.wikipedia.org/wiki/Microsoft_Research

Between 2010 and 2018, 154,000 AI patents were filed worldwide, with Microsoft having by far the largest percentage of those patents, at 20%

Cheng Huang a former student of Lihao Xu at Wayne State U. leads research on distributed storage systems.

<https://www.microsoft.com/en-us/research/people/chengh/>

Microsoft's Silicon Valley Lab, which included former IBM Fellow Rakesh Agrawal, opened in 2001, but was closed in 2014.

<http://rakesh.agrawal-family.com/>

Nippon Electric Company - NEC labs are located at Princeton, NJ and San Jose, CA.

<https://www.nec-labs.com/>

Qualcomm Research: <https://www.qualcomm.com/invention>

Seagate closed its research facility in Pittsburgh in 2008 after only six years, laying off Eric Riedel (EMC, ITrenew.com) and Alma Riska (NetApp).

<https://www.bizjournals.com/pittsburgh/stories/2008/09/15/daily35.html>

<https://www.seagate.com/>

A list of companies with highly rated AI patents > 3.5 is listed below.

<https://blogs-images.forbes.com/louisclumbus/files/2019/01/Leading-AI-Companies.jpg>

Several storage related organizations and their functions are introduced in the book.

Storage Networking Industry Association - SNIA <https://www.snia.org>

Transaction Processing Council - TPC tpc.org

Storage Performance Council - SPC <https://spcresults.org>

Standard Performance Evaluation Corporation - SPEC <https://www.spec.org/>

A.6 Storage research at universities

Electrical Eng. and Computer Science - EECS Dept. at Univ. of Calif. at Berkeley - UCB led by David Patterson, Garth Gibson, and Randy Katz, who published the seminal RAID paper (Patterson et al., 1988). Katz later authored “A personal recollection of how storage became a system” (Katz, 2010). The led to several PhD theses at UCB.

<https://www2.eecs.berkeley.edu/Pubs/Dissertations/>

Parallel Data Laboratory - PDL at the Computer Science Dept. at *Carnegie-Mellon Univ. - CMU* is the top storage systems academic research center, which expanded storage research that was started at UCB.

<https://www.pdl.cmu.edu> It is now headed by Greg Ganger who replaced Garth Gibson, who joined the [VectorInstitute.ai](https://www.vectorinstitute.ai) in Toronto, Canada.

Storage Systems Research Center - SSRC at *Univ. of Calif. at Santa Cruz's - UCSC* was founded by Darrell Long, who did his PhD with Francios Jehan Paris (U. Houston) at UCSD. Ethan Miller who did his PhD with Randy Katz is another SSRC faculty who consults at Pure Storage

<https://www.ssrc.ucsc.edu>

Other storage researchers at UCSB are Carlos Maltzahn and Scott Brandt.

Tzi-cker Chiueh who did his PhD with Randy Katz at UCB left *State U. NY - SUNY* at Stony Brook, NY in 2014 to join the *Industrial Technology Research Institute - ITRI* in Taiwan. He has led a project on *Reconfigurable Array of Inexpensive Batteries - RAIBA*.

<https://chiueh.jimdofree.com/>

Erez Zadok who did his PhD with Jason Nieh at Columbia U. now runs the *File Systems and Storage Lab - FSL* at *State Univ. of NY - SUNY* at Stony Brook, NY.

<https://www.fsl.cs.sunysb.edu/>

<https://www.filesystems.org/>

Peter Desnoyers at Northeastern U. has led research on *Shingled Magnetic Recording - SMR* and flash memories.

<http://www.ccs.neu.edu/home/pjd/pubs.html>

Gemini Storage Systems Lab at *Univ. of Calif. at San Diego - UCSD* was led by Walter Burkhard (emeritus).

<http://cseweb.ucsd.edu/groups/gemini/burkhard/>

Non-Volatile Systems Lab at UCSD is led by Steven Swanson.

<https://cmrr.ucsd.edu/research/faculty-profiles/swanson.html>

David Hung-Chang Du heads the *Center of Research in Intelligent Storage - CRIS* at U. Minnesota.

<https://cse.umn.edu/cris>

Patrick P. C. Lee at *Chinese U. of Hong Kong - CUHK* did his PhD with Dan Rubenstein and Vishal Misra at Columbia U. He has collaborated with John C.S. Lui at the same university, who is a student of Richard Muntz at UCLA.

<https://www.cse.cuhk.edu.hk/~pclee/www/index.html>

A.7 Funding agencies, national labs, and research institutes

Aerospace Corporation's early history is given below, which lists the *Global Positioning System - GPS* launched in 1978 and operated by US Space Force is listed as an accomplishment.

<https://aerospace.org>

https://en.wikipedia.org/wiki/Global_Positioning_System

<https://aerospace.org/article/history-aerospace>

https://en.wikipedia.org/wiki/The_Aerospace_Corporation

Aerospace Corp. operates the only *Federally Funded Research and Development Center - FFRDC* committed to space-related science and engineering.

Allen Institute for AI is a research institute founded by late Microsoft co-founder Paul Allen in 2015.

<https://alleninstitute.org/>

Allen Institute led by Oren Etzioni (U. Washington at Seattle) seeks to achieve scientific breakthroughs by constructing AI systems with reasoning, learning, and reading capabilities.

https://en.wikipedia.org/wiki/Allen_Institute_for_AI

American National Standards Institute - ANSI was formed in 1918 by five engineering societies and three government agencies.

<https://www.ansi.org/>

https://en.wikipedia.org/wiki/American_National_Standards_Institute

Argonne National Laboratory - ANL is under the supervision of . It has a data management and storage systems group with researchers at U. Chicago.

<https://www.anl.gov/mcs/data-management-and-storage-systems>

Brookhaven National Lab - BNL with the motto “Passion for discovery” was founded in 1947 and is affiliated with *State Univ. of New York - SUNY* at Stony Brook.

https://en.wikipedia.org/wiki/Brookhaven_National_Laboratory

With funding from DoE these two organizations have established a three node quantum network, which is the key device needed for the Quantum internet.

<https://www.bnl.gov/newsroom/news.php?a=117346>

Computing Research Association - CRA is a non-profit association of North American academic departments of computer science, computer engineering, and related fields; laboratories and centers in industry, government, and academia engaging in basic computing research; and affiliated professional societies. CRA formed in 1972 is based in Washington D.C.

<https://cra.org>

Defense Advanced Projects Research Agency - DARPA has the motto “Creating Breakthrough Technologies and Capabilities For National Security”. DARPA is best known for the ARPANET packet-switching network.

<https://en.wikipedia.org/wiki/ARPANET>

<https://www.darpa.mil>

Dept. of Energy - DoE was founded by Jimmy Carter in 1977 in response to the gas shortage precipitated by the oil embargo by Arab states in 1973 following the Yom Kippurs war.

https://en.wikipedia.org/wiki/1973_oil_crisis

<https://www.energy.gov/>

https://en.wikipedia.org/wiki/United_States_Department_of_Energy

U.S. officials and scientists have unveiled a plan to construct a potentially hackproof quantum Internet to operate parallel to the world’s existing networks with DoE and its labs such as Argonne as project’s main pillar.

<https://www.energy.gov/articles/quantum-internet-future-here>

<https://www.technologyreview.com/2018/10/25/139304/us-unhackable-quantum-networks/>

Daegu Gyeongbuk Institute of Science and Technology - DGIST is a public science and engineering university located in that city in S. Korea. Under a Special Act on Support of Scientists and Engineers for Strengthening National Science and Technology Competitiveness, the Korean government founded DGIST in 2004 as a research institute. In 2008, the act was amended to extend the role of the institute to a university.

<https://www.dgist.ac.kr/en/>

Fermilab fnal.gov works in several areas of computing, such as advanced networking and Open Science Grid Consortium. <https://en.wikipedia.org/wiki/Fermilab>

Indian Institute of Science - IISc is located in Bangalore, India. Research is conducted in many areas such as databases, machine learning, and coding in distributed systems. <https://www.iisc.ac.in/>

Information Science Institute - ISI <https://www.isi.edu> which is located in Marina del Rey, CA was founded by Rand Corporation’s Keith Uncapher in 1972. <https://www.isi.edu/about/history/>

To qualify for federal grants he applied to join ISI with UCLA but was told that it will take 15 months. This task was accomplished in five days with *U. Southern Calif. - USC* under the leadership of Sr VP Zohrab Kaprielian (rip) (Bekey and Bekey, 2015). ISI which is currently affiliated with USC’s Vitterbi School of Engineering conducts research in networking, computational systems and technology, etc.

<https://www.viterbischool.usc.edu>

https://en.wikipedia.org/wiki/Information_Sciences_Institute

Institute for Advanced Study - IAS was founded in Princeton, NJ in 1930.

https://en.wikipedia.org/wiki/Institute_for_Advanced_Study

It is an independent postdoctoral research center for theoretical research. It served as the academic home of internationally preeminent scholars, including Albert Einstein, J. Robert Oppenheimer, John von Neumann, and Kurt Gödel.

<https://www.ias.edu/>

INRIA is France’s National Institute for Research in Digital Science and Technology.

<https://www.inria.fr/en>

It has the following publications. <https://hal.inria.fr/inria-00384970/document>

Int'l DOI Foundation - IDF, where *DOI - Digital Information Identifier - IDF*, is a not-for-profit membership organization that is the governance and management body for the federation of Registration Agencies providing DOI services and registration, according to ISO standard (ISO 26324) for the DOI system. <https://www.doi.org/>

Int'l Computer Science Institute - ICSI founded in 1988 at *U. Calif. at Berkeley - UCB* hosts visiting researchers, but most members hold joint positions with UCB. <https://www.icsi.berkeley.edu/icsi/>

Korean Advanced Institute of Science and Technology - KAIST is a university founded 50 years ago. <https://www.kaist.ac.kr/site/historyen/>

Lawrence Berkeley National Lab - LBL <https://www.lbl.gov> does research in *High Performance Computing - HPC* and *Quantum Computing - QC* at its *Computational Research Dept. - CRD*. <https://crd.lbl.gov/>
LBL's data management group is led by Arie Shoshani, and two former members were Doron Rotem (emeritus) and Frank Olken (rip). <https://sdm.lbl.gov/~arie>

Lawrence Livermore National Lab - LLNL has a computing mission to unify file system that deliver fast I/O performance over distributed storage. <https://computing.llnl.gov/newsroom/new-unify-file-systems-deliver-fast-io-performance-over-distributed-storage>

Los Alamos National Lab - LANL was established in 1943 as part of the Manhattan project to develop atomic bombs later dropped on Hiroshima and Nagasaki in August 1945. <https://www.lanl.gov>
LANL publishes the Los Alamos Science and Technology Magazine. <https://www.lanl.gov/discover/publications/1663/index.php>

National Academy of Engineering - NAE is a private, independent, nonprofit institution founded in 1964 that provides engineering leadership for the nation. https://en.wikipedia.org/wiki/National_Academy_of_Engineering
David Patterson is a member of *National Academy of Engineering - NAE*. <https://www.nae.edu>

National Academy of Sciences is a nonprofit, non-governmental organization, which is part of the National Academies of Sciences, Engineering, and Medicine. <https://www.nationalacademies.org/>
National Research Council - NRC is part of National Academies. <https://tethys.pnnl.gov/organization/national-research-council-national-academies-nrc>
It conducted a study on the future of computing via *Computer Science and Telecommunications Board - CSTB*. <http://www.cstb.org>
<https://science.osti.gov/-/media/ascr/ascac/pdf/meetings/mar11/Yelick.pdf>

National Aeronautics and Space Administration - NASA is an independent agency of the U.S. federal government responsible for the civilian space program, as well as aeronautics and space research. It operates Skylab space station, which was followed by the Int'l space station and *Jet Propulsion La - JPL* in Pasadena, CA. <https://www.nasa.gov/>

National Center for Atmospheric Research - NCAR <https://ncar.ucar.edu/>, where *U. Corporation for Atmospheric Research - UCAR*. NCAR invites scientists to help further develop two of its premiere geoscience data visualization and analysis software programs: *Geoscience Community Analysis Toolkit - GeoCAT* and *Visualization and Analysis Platform for Ocean, Atmosphere, and Solar Researchers - VAPOR*.

National Institute of Science and Technology - NIST <https://www.nist.gov>
renamed from *National Bureau of Standards - NBS* publishes reports on various topics such as big data analytics and has several resource centers such as *Computer Security Resource Center - CSRC*, which is part of its Information Technology Lab. <https://csrc.nist.gov/>

National Science Foundation - NSF ([NSF.gov](https://www.nsf.gov)) has been a major source of research funding. NSF's history is described by Peter Freeman (UCI, GMU, GaTech, emeritus) et al. in (Freeman et al., 2019). NSF's *Computer and Information Science and Engineering - CISE* funds computer architecture research. An NSF-Sponsored Community Visioning Workshop held at IBM/ARC is (Amvrosiadis et al., 2018). <https://sites.google.com/vt.edu/data-storage-research/>

National Security Agency - NSA, where NSA stands for No Such Agency is the world leader in cryptology. <https://www.nsa.gov>

The Next Wave Journal publishes a few issues per year with volume 23 published in 2021. <https://www.nsa.gov/News-Features/the-next-wave/>

Nippon Telephone & Telegraph - NTT has now a Silicon Valley Startup. https://en.wikipedia.org/wiki/Nippon_Telegraph_and_Telephone

Oak Ridge National Laboratory - ORNL researches advanced high-performance computing for scientific discovery at *Computing and Computational Sciences Directorate - CCSD*.
<https://www.ornl.gov/directorate/ccsd>

Optoelectronics Information Storage Research

http://english.wnlo.hust.edu.cn/Faculty/Optoelectronics_Information_Storage1.htm

at *Huazhong Univ. of Science and Technology - HUST*. It is led by Dan Feng who collaborates with Hong Juang at U. Texas at Arlington.

<http://faculty.hust.edu.cn/dfeng/en/index.htm>

Pacific Northwest National Laboratory - PNNL is a leading center for scientific discovery in chemistry, data analytics, and earth science, and for technological innovation in energy resiliency and national security.

<https://www.pnnl.gov/>

Quanta Research Cambridge - QRC Lab. is the North American R&D subsidiary of Taiwan-based Quanta Computer, Inc., the world's largest manufacturer of laptop computers.

<https://qrclab.wordpress.com/about/>

Rand Corp. <https://www.rand.org>

is a nonprofit global policy think tank created in 1948 by Douglas Aircraft Company to offer research and analysis to the US Armed Forces. It is financed by the U.S. government and private endowment, corporations, universities and private individuals. It states its role as developer of solutions to public policy challenges to help make communities throughout the world safer and more secure, healthier and more prosperous. Not all RAND plans are peaceful as exemplified by the following.

<https://greekcitytimes.com/2020/09/30/us-air-force-affiliated-think-tank-calls-for-washington-to-exploit-tensions-in-the-south-caucasus-armenia/>

Samsung's DataLab <https://datalab.dgist.ac.kr/>

Samsung has research centers all over the world, such as in Poland, There are eight locations in US and two in Canada for *Samsung Research in America - SRA* is:

<https://www.sra.samsung.com/>

Sandia Lab <https://www.sandia.gov>

https://en.wikipedia.org/wiki/Sandia_National_Laboratories

has the primary mission to develop, engineer, and test the non-nuclear components of nuclear weapons and high technology. Sandia was recently in the news because its white man reeducation camp.

<https://www.lawenforcementtoday.com/white-men-who-work-at-government-funded-nuclear-laboratory-forced-to-attend-reeducation-camp/>

Santa Fe Institute <https://www.santafe.edu/>

is an independent, nonprofit theoretical research institute located in Santa Fe, NM and dedicated to the multidisciplinary study of the fundamental principles of complex adaptive systems, including physical, computational, biological, and social systems.

Sarnoff Corp. started out as **David Sarnoff Research Center** which was established in 1941 at Princeton, NJ. It was integrated in 1988 into SRI International. https://en.wikipedia.org/wiki/Sarnoff_Corporation

Shenzhen Institutes of Advanced Technology - SIAT is part of *Chinese Academy of Sciences - CAS*.

<http://english.siat.cas.cn/>

Simons Research Institute located at the Berkeley campus hosts researchers in various theoretical computer science programs. <https://simons.berkeley.edu/programs/past>

Southwest Research Institute - SwRI in San Antonio, TX since 1947.

https://en.wikipedia.org/wiki/Southwest_Research_Institute

Stanford Linear Accelerator Center - SLAC, which was established in 1962 was later renamed *SLAC National Accelerator Laboratory*. <https://www6.slac.stanford.edu/>

https://en.wikipedia.org/wiki/SLAC_National_Accelerator_Laboratory

SRI International renamed from *Stanford Research Institute - SRI* in Palo Alto, CA (1946-1970).

https://en.wikipedia.org/wiki/SRI_International

Texas Advanced Computing Center - TACC with motto "Powering Discoveries That Change The World" is located at Univ. of Texas at Austin. <https://www.tacc.utexas.edu/use-tacc/getting-started>

Bibliography

- Abadi, D.J., Carney, D., Çetintemel, U., Cherniack, M., Conway, C., Lee, S., Stonebraker, M., Tatbul, N., Zdonik, S.B., 2003. Aurora: a new model and architecture for data stream management. *VLDB J.* 12 (2), 120–139. <https://doi.org/10.1007/s00778-003-0095-z>.
- Abadi, M., Barham, P., Chen, J., et al., 2016. Tensorflow: a system for large-scale machine learning. In: Roscoe, K.K.T. (Ed.), 12th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2016. Savannah, GA, USA, November 2–4, 2016. USENIX Association, pp. 265–283. <https://www.usenix.org/conference/osdi16/technical-sessions/presentation/abadi>.
- Abali, B., Franke, H., Poff, D.E., Jr., R.A.S., Schulz, C.O., Herger, L., Smith, T.B., 2001. Memory eXpansion Technology (MXT): Software support and performance. *IBM J. Res. Dev.* 45 (2), 287–302. <https://doi.org/10.1147/rd.452.0287>.
- Abate, J., Dubner, H., Weinberg, S.B., 1968. Queuing analysis of the IBM 2314 disk storage facility. *J. ACM* 15 (4), 577–589. <https://doi.org/10.1145/321479.321484>.
- Abbott, M.L., Fisher, M.T., 2016. Scalability Rules: Principles for Scaling Web Sites, 2. Addison-Wesley. ISBN 978-0-13-443160-4.
- Abbott, R.K., Garcia-Molina, H., 1992. Scheduling real-time transactions: a performance evaluation. *ACM Trans. Database Syst.* 17 (3), 513–560. <https://doi.org/10.1145/132271.132276>.
- Abd-El-Barr, M., 2007. Design and Analysis of Reliable and Fault-Tolerant Computer Systems. Imperial College Press. ISBN 1-86094-668-2.
- Abd-El-Malek, M., Courtright II, W.V., Cranor, C., Ganger, G.R., et al., 2005. Ursa minor: versatile cluster-based storage. In: Gibson, G. (Ed.), Proceedings of the FAST '05 Conference on File and Storage Technologies. December 13–16, 2005, San Francisco, California, USA. USENIX. <http://www.usenix.org/events/fast05/tech/abd-el-malek.html>.
- Acharya, A., Sidnal, N.S., 2016. High frequency trading with complex event processing. In: 23rd IEEE International Conference on High Performance Computing Workshops, HiPC 2016 Workshops. Hyderabad, India, December 19–22, 2016. IEEE Computer Society, pp. 39–42.
- Acharya, A., Uysal, M., Saltz, J.H., 1998. Active disks: programming model, algorithms and evaluation. In: Bhandarkar, D., Agarwal, A. (Eds.), ASPLOS-VIII Proceedings of the 8th International Conference on Architectural Support for Programming Languages and Operating Systems. San Jose, California, USA, October 3–7, 1998. ACM Press, pp. 81–91.
- Adelson-Velskii, G.M., Landis, E.M., 1962. An algorithm for the organization of information. *Sov. Math. Dokl.* 3, 1259–1263.
- Adiga, N.R., et al., 2005. Blue gene/l torus interconnection network. *IBM J. Res. Dev.* 49 (2–3), 265–276. <https://doi.org/10.1147/rd.492.0265>.
- Agarwal, B., Akella, A., Anand, A., Balachandran, A., Chitnis, P., Muthukrishnan, C., Ramjee, R., Varghese, G., 2010. Endre: an end-system redundancy elimination service for enterprises. In: Proceedings of the 7th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2010. April 28–30, 2010, San Jose, CA, USA. USENIX Association, pp. 419–432. http://www.usenix.org/events/nsdi10/tech/full_papers/aggarwal.pdf.
- Agarwal, R.C., 1996. A super scalar sort algorithm for RISC processors. In: Jagadish, H.V., Mumick, I.S. (Eds.), Proceedings of the 1996 ACM SIGMOD International Conference on Management of Data. Montreal, Quebec, Canada, June 4–6, 1996. ACM Press, pp. 240–246.
- Agerwala, T., Martin, J.L., Mirza, J.H., Sadler, D.C., Dias, D.M., Snir, M., 1995. SP2 system architecture. *IBM Syst. J.* 34 (2), 152–184. <https://doi.org/10.1147/sj.342.0152>.
- Aggarwal, A., Vitter, J.S., 1988. The input/output complexity of sorting and related problems. *Commun. ACM* 31 (9), 1116–1127. <https://doi.org/10.1145/48529.48535>.
- Aggarwal, C.C. (Ed.), 2015. Data Mining: The Textbook. Springer. ISBN 978-3-3191-4141-1.
- Aggarwal, C.C., Reddy, C.K. (Eds.), 2014. Data Clustering: Algorithms and Applications. CRC Data Mining and Knowledge Discovery Series. Chapman & Hall. ISBN 978-1-4665-5821-2.
- Aggarwal, C.C., Yu, P.S., 2000. Finding generalized projected clusters in high dimensional spaces. In: Chen, W., Naughton, J.F., Bernstein, P.A. (Eds.), Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data. May 16–18, 2000, Dallas, Texas, USA. ACM, pp. 70–81.
- Aggarwal, C.C., Procopiuc, C.M., Wolf, J.L., Yu, P.S., Park, J.S., 1999. Fast algorithms for projected clustering. In: Delis, A., Faloutsos, C., Ghandeharizadeh, S. (Eds.), SIGMOD 1999, Proceedings ACM SIGMOD International Conference on Management of Data. Philadelphia, Pennsylvania, USA, June 1–3, 1999. ACM Press, pp. 61–72.
- Aghayev, A., Ts'o, T.Y., Gibson, G., Desnoyers, P., 2017. Evolving ext4 for shingled disks. In: Kuenning, G., Waldspurger, C.A. (Eds.), 15th USENIX Conference on File and Storage Technologies, FAST 2017. Santa Clara, CA, USA, February 27–March 2, 2017. USENIX Association, pp. 105–120. <https://www.usenix.org/conference/fast17/technical-sessions/presentation/aghayev>, 2017.
- Agrawal, P., Arya, R., Bindal, A., Bhatia, S., et al., 2019. Data platform for machine learning. In: Boncz, P.A., Manegold, S., Ailamaki, A., Deshpande, A., Kraska, T. (Eds.), Proceedings of the 2019 International Conference on Management of Data, SIGMOD Conference 2019. Amsterdam, The Netherlands, June 30–July 5, 2019. ACM, pp. 1803–1816.

- Agrawal, R., Bell, D.A. (Eds.), 1990. Proceedings of the Second International Symposium on Databases in Parallel and Distributed Systems. Trinity College, Dublin, Ireland, July 2–4, 1990. IEEE Computer Society. ISBN 0-8186-2052-8. <http://ieeexplore.ieee.org/xpl/mostRecentIssue.jsp?punumber=293>.
- Agrawal, R., Carey, M.J., Livny, M., 1987. Concurrency control performance modeling: alternatives and implications. *ACM Trans. Database Syst.* 12 (4), 609–654. <https://doi.org/10.1145/32204.32220>.
- Agrawal, R., Imielinski, T., Swami, A.N., 1993a. Mining association rules between sets of items in large databases. In: Buneman, P., Jajodia, S. (Eds.), Proceedings of the 1993 ACM SIGMOD International Conference on Management of Data. Washington, DC, USA, May 26–28, 1993. ACM Press, pp. 207–216.
- Agrawal, R., Imielinski, T., Swami, A.N., 1993b. Mining association rules between sets of items in large databases. In: Buneman, P., Jajodia, S. (Eds.), Proceedings of the 1993 ACM SIGMOD International Conference on Management of Data. Washington, DC, USA, May 26–28, 1993. ACM Press, pp. 207–216.
- Agrawal, R., Mehta, M., Shafer, J.C., Srikant, R., Arning, A., Bollinger, T., 1996. The quest data mining system. In: Simoudis, E., Han, J., Fayyad, U.M. (Eds.), Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD-96). Portland, Oregon, USA. AAAI Press, pp. 244–249. <http://www.aaai.org/Library/KDD/1996/kdd96-040.php>.
- Agrawal, S.R., Idicula, S., Raghavan, A., Sedlar, E., 2017. A many-core architecture for in-memory data processing. In: Hunter, H.C., Moreno, J., Emer, J.S., Sánchez, D. (Eds.), Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture, MICRO 2017. Cambridge, MA, USA, October 14–18, 2017. ACM, pp. 245–258.
- Agrawala, A.K., Rauscher, T.G., 2014. Foundation of Microprogramming: Architecture, Software, and Applications. ACM Monograph Series. Academic Press. ISBN 9781483215877.
- Ailamaki, A., DeWitt, D.J., Hill, M.D., Wood, D.A., 1999. Dbmss on a modern processor: where does time go? In: Atkinson, M.P., Orlowska, M.E., Valduriez, P., Zdonik, S.B., Brodie, M.L. (Eds.), VLDB'99, Proceedings of 25th International Conference on Very Large Data Bases. September 7–10, 1999, Edinburgh, Scotland, UK. Morgan Kaufmann, pp. 266–277. <http://www.vldb.org/conf/1999/P28.pdf>.
- Aingaran, K., Jairath, S., Lutz, D., 2016. Software in silicon in the oracle SPARC M7 processor. In: 2016 IEEE Hot Chips 28 Symposium (HCS). Cupertino, CA, USA, August 21–23, 2016. IEEE, pp. 1–31.
- Akintoye, S., Bagula, A., 2018. Lightweight cloud storage systems: analysis and performance evaluation. *Int. J. Sci. Eng. Res.* 9, 1447–1454.
- Akyürek, S., Salem, K., 1995. Adaptive block rearrangement. *ACM Trans. Comput. Syst.* 13 (2), 89–121. <http://doi.acm.org/10.1145/201045.201046>.
- Alba, A., Alatorre, G., Bolik, C., et al., 2014. Efficient and agile storage management in software defined environments. *IBM J. Res. Dev.* 58 (2/3). <https://doi.org/10.1147/JRD.2014.2302381>.
- Allen, A.O., 1978. Probability Statistics and Queueing Theory with Computer Science Applications. Academic Press. ISBN 978-120510504.
- Allen, A.O., 1990. Probability, Statistics, and Queueing Theory. Computer Science and Scientific Computing. Academic Press. ISBN 978-0120510511.
- Alvarez, G.A., Burkhard, W.A., Cristian, F., 1997. Tolerating multiple failures in RAID architectures with optimal storage and uniform declustering. In: Pleszkun, A.R., Mudge, T.N. (Eds.), Proceedings of the 24th International Symposium on Computer Architecture. Denver, Colorado, USA, June 2–4, 1997. ACM, pp. 62–72.
- Alvarez, G.A., Burkhard, W.A., Stockmeyer, L.J., Cristian, F., 1998. Declustered disk array architectures with optimal and near-optimal parallelism. In: Valero, M., Sohi, G.S., DeGroot, D. (Eds.), Proceedings of the 25th Annual International Symposium on Computer Architecture, ISCA 1998. Barcelona, Spain, June 27–July 1, 1998. IEEE Computer Society, pp. 109–120.
- Alvarez, G.A., Borowsky, E., Go, S., Romer, T.H., Becker-Szendy, R.A., Golding, R.A., Merchant, A., Spasojevic, M., Veitch, A.C., Wilkes, J., 2001. Minervat: an automated resource provisioning tool for large-scale storage systems. *ACM Trans. Comput. Syst.* 19 (4), 483–518. <http://doi.acm.org/10.1145/502912.502915>.
- Amdahl, G.M., 1967. Validity of the single processor approach to achieving large scale computing capabilities. In: Proc. AFIPS Spring Joint Computing Conference FJCC'67, pp. 483–485.
- Amdahl, G.M., 2013. Computer architecture and Amdahl's law. *IEEE Computer* 46 (12), 38–46. <https://doi.org/10.1109/MC.2013.418>.
- Amdahl, G.M., Blaauw, G.A., Jr., F.P.B., 1964. Architecture of the IBM system/360. *IBM J. Res. Dev.* 8 (2), 87–101. <https://doi.org/10.1147/rd.82.0087>.
- Amer, A., Long, D.D.E., Miller, E.L., Pâris, J., Schwarz, T.J.E., 2010. Design issues for a shingled write disk system. In: Khatib, M.G., He, X., Factor, M. (Eds.), IEEE 26th Symposium on Mass Storage Systems and Technologies, MSST 2012. Lake Tahoe, Nevada, USA, May 3–7, 2010. IEEE Computer Society, pp. 1–12.
- Amiri, K., Gibson, G.A., Golding, R.A., 2000. Highly concurrent shared storage. In: Proceedings of the 20th International Conference on Distributed Computing Systems. Taipei, Taiwan, April 10–13, 2000. IEEE Computer Society, pp. 298–307.
- Ammar, K., Özsü, M.T., 2018. Experimental analysis of distributed graph systems. *Proc. VLDB Endow.* 11 (10), 1151–1164. <http://www.vldb.org/pvldb/vol11/p1151-ammar.pdf>.
- Amur, H., Cipar, J., Gupta, V., Ganger, G.R., Kozuch, M.A., Schwan, K., 2010. Robust and flexible power-proportional storage. In: Hellerstein, J.M., Chaudhuri, S., Rosenblum, M. (Eds.), Proceedings of the 1st ACM Symposium on Cloud Computing, SoCC 2010. Indianapolis, Indiana, USA, June 10–11, 2010. ACM, pp. 217–228.
- Amvrosiadis, G., Butt, A.R., Tarasov, V., Zadok, E., Zhao, M., Ahmad, I., Arpacı-Dusseau, R., Chen, F., Chen, Y., Che, Y., Cheng, Y., Chidambaram, V., Silva, D.D., Demke-Brown, A., Desnoyers, P., Flinn, J., He, X., Jiang, S., Kuenning, G., Li, M., Carlos Maltzahn, C., Miller, E.L., Mohror, K., Rangaswami, R., Reddy, N., Rosenthal, D., Tosun, A.S., Talagala, N., Varman, P., Vazhkudai, S., Zhang, A.W.X., Zhang, Y., Zheng, M., 2018. Data storage research vision 2025: report on NSF visioning workshop held May 30–June 1, 2018. Technical report. National Science Foundation, USA.
- Andersen, D.G., Franklin, J., Kaminsky, M., Phanishayee, A., Tan, L., Vasudevan, V., 2009. FAWN: a fast array of wimpy nodes. In: Matthews, J.N., Anderson, T.E. (Eds.), Proceedings of the 22nd ACM Symposium on Operating Systems Principles 2009, SOSP 2009. Big Sky, Montana, USA, October 11–14, 2009. ACM, pp. 1–14.

- Andersen, D.G., Franklin, J., Kaminsky, M., Phanishayee, A., Tan, L., Vasudevan, V., 2011. FAWN: a fast array of wimpy nodes. *Commun. ACM* 54 (7), 101–109. <http://doi.acm.org/10.1145/1965724.1965747>.
- Anderson, D., Dykes, J., Riedel, E., 2003. More than an interface – SCSI vs. ATA. In: Chase, J. (Ed.), *Proceedings of the FAST '03 Conference on File and Storage Technologies*. March 31–April 2, 2003, Cathedral Hill Hotel, San Francisco, California, USA. USENIX. <http://www.usenix.org/events/fast03/tech/anderson.html>.
- Anderson, E., Swaminathan, R., Veitch, A.C., Alvarez, G.A., Wilkes, J., 2002. Selecting RAID levels for disk arrays. In: Long, D.D.E. (Ed.), *Proceedings of the FAST '02 Conference on File and Storage Technologies*. January 28–30, 2002, Monterey, California, USA. USENIX, pp. 189–201. <http://www.usenix.org/publications/library/proceedings/fast02/andersonRAID.html>.
- Anderson, E., Spence, S., Swaminathan, R., Kallahalla, M., Wang, Q., 2005. Quickly finding near-optimal storage designs. *ACM Trans. Comput. Syst.* 23 (4), 337–374. <http://doi.acm.org/10.1145/1113574.1113575>.
- Anderson, P., Black, R., Cerkuskaite, A., et al., 2018. Glass: a new media for a new era? In: Goel, A., Talagala, N. (Eds.), *10th USENIX Workshop on Hot Topics in Storage and File Systems, HotStorage 2018*. Boston, MA, USA, July 9–10, 2018. USENIX Association. <https://www.usenix.org/conference/hotstorage18/presentation/anderson>.
- Anderson, S.F., Earle, J.G., Goldschmidt, R.E., Powers, D.M., 1967. The IBM system/360 model 91: floating point execution unit. *IBM J. Res. Dev.* 11 (1), 34–53. <https://doi.org/10.1147/rd.152.0153>.
- Anderson, T., Dahlin, M., 2014. *Operating Systems: Principles and Practice*, 2. Recursive Books. ISBN 978-0985673529.
- Anderson, T.E., 1990. The performance of spin lock alternatives for shared-memory multiprocessors. *IEEE Trans. Parallel Distrib. Syst.* 1 (1), 6–16. <https://doi.org/10.1109/71.80120>.
- Andrews, K., Sand, D., 1992. Migrating a CISC computer family onto RISC via object code translation. In: Flahive, B., Wexelblat, R.L. (Eds.), *ASPLOS-V Proceedings – Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*. Boston, Massachusetts, USA, October 12–15, 1992. ACM Press, pp. 213–222.
- Andrews, M., Bender, M.A., Zhang, L., 1996. New algorithms for the disk scheduling problem. In: *37th Annual Symposium on Foundations of Computer Science, FOCS '96*. Burlington, Vermont, USA, 14–16 October, 1996. IEEE Computer Society, pp. 550–559.
- Angus, J.E., 1998. On computing mtbf for a k-out-of-n:g repairable systems. *IEEE Trans. Reliab.* 37 (2). <https://doi.org/10.1109/24.488912>.
- Antoshenkov, G., 1997. Dictionary-based order-preserving string compression. *VLDB J.* 6 (1), 26–39. <https://doi.org/10.1007/s007780050031>.
- Anvin, H.P., 2007. The mathematics of RAID6. <https://mirrors.edge.kernel.org/pub/linux/kernel/people/hpa/raid6.pdf>.
- Apers, P.M.G., 1988. Data allocation in distributed database systems. *ACM Trans. Database Syst.* 13 (3), 263–304. <https://doi.org/10.1145/44498.45063>.
- Appuswamy, R., Borovica-Gajic, R., Graefe, G., Ailamaki, A., 2017. The five-minute rule thirty years later and its impact on the storage hierarchy. In: Bordawekar, R., Lahiri, T. (Eds.), *International Workshop on Accelerating Analytics and Data Management Systems Using Modern Processor and Storage Architectures. ADMS@VLDB 2017*, Munich, Germany, September 1, 2017, pp. 1–8. <http://www.adms-conf.org/2017/camera-ready/5minute-rule.pdf>.
- Appuswamy, R., Graefe, G., Borovica-Gajic, R., Ailamaki, A., 2019. The five-minute rule 30 years later and its impact on the storage hierarchy. *Commun. ACM* 62 (11), 114–120. <https://doi.org/10.1145/3318163>.
- Argumedo, A.J., et al., 2008. Scaling tape-recording areal densities to 100 gb/in². *IBM J. Res. Dev.* 52 (4–5), 513–528. <https://doi.org/10.1147/rd.524.0513>.
- Arnold, R., Menon, S.M., Brackett, B., Richmond, R., 1998. Test methods used to produce highly reliable known good die (kgd). In: *Proceedings 1998 Int'l Conf. on Multichip Modules and High Density Packaging*, pp. 374–382.
- Arpaci-Dusseau, R.H., Arpaci-Dusseau, A.C., 2015. *Operating Systems: Three Easy Pieces*, 1. Amazon Digital Services. ISBN 978-1985086593.
- Arulraj, J., Pavlo, A., 2019. *Non-Volatile Memory Database Management Systems. Synthesis Lectures on Data Management*. Morgan & Claypool Publishers.
- Asanovic, K., 1998. *Vector Microprocessors*. PhD thesis. EECS Department University of California at Berkeley. <https://www2.eecs.berkeley.edu/Pubs/techRpts/1998/1604.html>.
- Ashley, J.J., Bernal, M., Burr, G.W., et al., 2000. Holographic data storage technology. *IBM J. Res. Dev.* 44 (3), 341–368. <https://doi.org/10.1147/rd.443.0341>.
- Astrahan, M.M., Blasgen, M.W., Chamberlin, D.D., Eswaran, K.P., Gray, J., Griffiths, P.P., King III, W.F., Lorie, R.A., McJones, P.R., Mehl, J.W., Putzolu, G.R., Traiger, I.L., Wade, B.W., Watson, V., 1976. System R: relational approach to database management. *ACM Trans. Database Syst.* 1 (2), 97–137. <https://doi.org/10.1145/320455.320457>.
- Athanassoulis, M., Ailamaki, A., Chen, S., Gibbons, P.B., Stoica, R., 2010. Flash in a DBMS: where and how? *IEEE Data Eng. Bull.* 33 (4), 28–34. <http://sites.computer.org/debull/A10dec/athanassoulis.pdf>.
- Auguin, M., Larbay, F., 1983. Opsila: an advanced simd for numerical analysis and signal processing. In: *Microcomputers: Developments in Industry, Business, and Education/Ninth EUROMICRO Symposium on Microprocessing and Microprogramming*. Madrid, Spain, pp. 311–318.
- Avizienis, A., Gilley, G.C., Mathur, F.P., Rennels, D.A., Rohr, J.A., Rubin, D.K., 1971. The STAR (self-testing and repairing) computer: an investigation of the theory and practice of fault-tolerant computer design. *IEEE Trans. Comput.* 20 (11), 1312–1321. <https://doi.org/10.1109/T-C.1971.223133>.
- Babb, E., 1979. Implementing a relational database by means of specialized hardware. *ACM Trans. Database Syst.* 4 (1), 1–29. <https://doi.org/10.1145/320064.320065>.
- Bach, M.J., 1986. *The Design of the UNIX Operating System*. Prentice-Hall. ISBN 0-13-201799-7.
- Bachman, C.W., 1973. The programmer as navigator. *Commun. ACM* 16 (11), 635–658. <http://doi.acm.org/10.1145/355611.362534>.
- Bachmat, E., Lam, T., 2005. On the effect of a configuration choice on the performance of a mirrored storage system. *J. Parallel Distrib. Comput.* 65 (3), 382–395. <https://doi.org/10.1016/j.jpdc.2004.11.003>.

- Bachmat, E., Schindler, J., 2002. Analysis of methods for scheduling low priority disk drive tasks. In: Muntz, R.R., Martonosi, M., de Souza e Silva, E. (Eds.), *Proceedings of the International Conference on Measurements and Modeling of Computer Systems, SIGMETRICS 2002*. June 15–19, 2002, Marina Del Rey, California, USA. ACM, pp. 55–65.
- Backus, J.W., 1978. Can programming be liberated from the von Neumann style? A functional style and its algebra of programs. *Commun. ACM* 21 (8), 613–641. <https://doi.org/10.1145/359576.359579>.
- Baek, S.H., Kim, B.W., Joung, E.J., Park, C., 2001. Reliability and performance of hierarchical RAID with multiple controllers. In: Kshemkalyani, A.D., Shavit, N. (Eds.), *Proceedings of the Twentieth Annual ACM Symposium on Principles of Distributed Computing, PODC 2001*. Newport, Rhode Island, USA, August 26–29, 2001. ACM, pp. 246–254.
- Baeza-Yates, R., Ribeiro-Neto, B., 2011. *Modern Information Retrieval: The Concepts and Technology Behind Search*, 2. Addison-Wesley, UK. ISBN 9780321416919.
- Bairavasundaram, L.N., Goodson, G.R., Pasupathy, S., Schindler, J., 2007. An analysis of latent sector errors in disk drives. In: Golubchik, L., Ammar, M.H., Harchol-Balter, M. (Eds.), *Proceedings of the 2007 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems, SIGMETRICS 2007*. San Diego, California, USA, June 12–16, 2007. ACM, pp. 289–300.
- Bairavasundaram, L.N., Arpaci-Dusseau, A.C., Arpaci-Dusseau, R.H., Goodson, G.R., Schroeder, B., 2008. An analysis of data corruption in the storage stack. *ACM Trans. Storage* 4 (3), 8:1–8:28. <http://doi.acm.org/10.1145/1416944.1416947>.
- Baker, J., Bond, C., Corbett, J.C., Furman, J.J., Khorlin, A., Larson, J., Leon, J., Li, Y., Lloyd, A., Yushprakh, V., 2011. Megastore: providing scalable, highly available storage for interactive services. In: Franklin, M. (Ed.), *CIDR 2011, Fifth Biennial Conference on Innovative Data Systems Research*, Online Proceedings. Asilomar, CA, USA, January 9–12, 2011, pp. 223–234. [www.cidrdb.org/cidr2011/Talks/CIDR11_Larson.pdf](http://cidrdb.org/cidr2011/Talks/CIDR11_Larson.pdf).
- Baker, M., Asami, S., Deprit, E., Ousterhout, J.K., Seltzer, M.I., 1992. Non-volatile memory for fast, reliable file systems. In: Flahive, B., Wexelblat, R.L. (Eds.), *ASPLOS-V Proceedings – Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*. Boston, Massachusetts, USA, October 12–15, 1992. ACM Press, pp. 10–22.
- Baker, Mary, Shah, Mehul A., Rosenthal, David S.H., Roussopoulos, Mema, Maniatis, Petros, Giuli, Thomas J., Bungale, Prashanth P., 2006. A fresh look at the reliability of long-term digital storage. In: Berbers, Yolande, Zwaenepoel, Willy (Eds.), *Proceedings of the 2006 EuroSys Conference*. Leuven, Belgium, April 18–21, 2006. ACM, pp. 221–234.
- Bakkum, P., Skadron, K., 2010. Accelerating SQL database operations on a GPU with CUDA. In: Kaeli, D.R., Leeser, M. (Eds.), *Proceedings of 3rd Workshop on General Purpose Processing on Graphics Processing Units, GPGPU 2010*. Pittsburgh, Pennsylvania, USA, March 14, 2010. In: *ACM International Conference Proceeding Series*, vol. 425. ACM, pp. 94–103.
- Balafoutis, E., Paterakis, M., Triantafyllou, P., Nerjes, G., Muth, P., Weikum, G., 2003. Clustered scheduling algorithms for mixed-media disk workloads in a multimedia server. *Clust. Comput.* 6 (1), 75–86. <https://doi.org/10.1023/A:1020923202104>.
- Balaji, A., Allen, A., 2018. Benchmarking automatic machine learning frameworks. *Corr*, arXiv:1808.06492.
- Balaji, S.B., Krishnan, M.N., Vajha, M., Ramkumar, V., Sasidharan, B., Kumar, P.V., 2018. Erasure coding for distributed storage: an overview. *CoRR*, arXiv:1806.04437.
- Balakrishnan, M., Kadav, A., Prabhakaran, V., Malkhi, D., 2010. Differential RAID: rethinking RAID for SSD reliability. *ACM Trans. Storage* 6 (2), 4:1–4:22. <http://doi.acm.org/10.1145/1807060.1807061>.
- Balkesen, C., Kunal, N., Giannakis, G., et al., 2018. RAPID: in-memory analytical query processing engine with extreme performance per watt. In: Das, G., Jermaine, C.M., Bernstein, P.A. (Eds.), *Proceedings of the 2018 International Conference on Management of Data, SIGMOD Conference 2018*. Houston, TX, USA, June 10–15, 2018. ACM, pp. 1407–1419.
- Bancilhon, F., Delobel, C., Kanellakis, P., 1992. *Building an Object-Oriented Database System: The Story of O2*. Morgan Kaufmann.
- Bansal, S., Modha, D.S., 2004. CAR: clock with adaptive replacement. In: Thekkath, C. (Ed.), *Proceedings of the FAST '04 Conference on File and Storage Technologies*. March 31–April 2, 2004, Grand Hyatt Hotel, San Francisco, California, USA. USENIX, pp. 187–200. <http://www.usenix.org/events/fast04/tech/bansal.html>.
- Barbará, D., DuMouchel, W., Faloutsos, C., Haas, P.J., Hellerstein, J.M., Ioannidis, Y.E., Jagadish, H.V., Johnson, T., Ng, R.T., Poosala, V., Ross, K.A., Sevcik, K.C., 1997. The new jersey data reduction report. *IEEE Data Eng. Bull.* 20 (4), 3–45. <http://sites.computer.org/debull/97DEC-CD.pdf>.
- Barber, R., Storm, A.J., Tian, Y., et al., 2019. Wiser: a highly available HTAP DBMS for iot applications. In: *2019 IEEE International Conference on Big Data (Big Data)*. Los Angeles, CA, USA, December 9–12, 2019. IEEE, pp. 268–277.
- Bard, Y., 1980. A model of shared DASD and multipathing. *Commun. ACM* 23 (10), 564–572. <https://doi.org/10.1145/359015.359019>.
- Bard, Y., 1982. Modeling I/O systems with dynamic path selection, and general transmission networks. *ACM SIGMETRICS Perform. Eval. Rev.* 11 (4), 118–129. <https://doi.org/10.1145/1035332.1035312>.
- Barisits, M., Beermann, T., Berghaus, F., et al., 2019. Rucio – scientific data management. *CoRR*, arXiv:1902.09857.
- Barlas, Gerassimos, 2021. *Multicore and GPU Programming: An Integrated Approach*, 2nd edition. Morgan-Kaufmann. ISBN 978-0-12-814120-5.
- Barnsley, M., Sloan, A., 1989. Method and apparatus for compression and decompression of digital data. <https://patents.google.com/patent/CA2062965A1/en>. Patent CA2062965A1.
- Barton, R.S., 1987. A new approach to the functional design of a digital computer. *IEEE Ann. Hist. Comput.* 9 (1), 11–15. <https://doi.org/10.1109/MAHC.1987.10002>.
- Baru, C.K., Fecteau, G., Goyal, A., Hsiao, H., Jhingran, A., Padmanabhan, S., Copeland, G.P., Wilson, W.G., 1995. DB2 parallel edition. *IBM Syst. J.* 34 (2), 292–322. <https://doi.org/10.1147/sj.342.0292>.
- Bashe, C.J., Buchholz, W., Hawkins, G.V., Ingram, J.J., Rochester, N., 1981. The architecture of IBM's early computers. *IBM J. Res. Dev.* 25 (5), 363–376. <https://doi.org/10.1147/rd.255.0363>.

- Baskett, F., Chandy, K.M., Muntz, R.R., Palacios, F.G., 1975. Open, closed, and mixed networks of queues with different classes of customers. *J. ACM* 22 (2), 248–260. <https://doi.org/10.1145/321879.321887>.
- Batcher, K.E., 1974. STARAN parallel processor system hardware. In: American Federation of Information Processing Societies: 1974 National Computer Conference. 6–10 May 1974, Chicago, Illinois, USA. In: AFIPS Conference Proceedings, vol. 43. AFIPS Press, pp. 405–410.
- Batini, C., Scannapieco, M., 2006. *Data Quality: Concepts, Methodologies and Techniques*. Springer. ISBN 978-3540331728.
- Baum, R.I., Hsiao, D.K., 1976. Database computers – a step towards data utilities. *IEEE Trans. Comput.* 25 (12), 1254–1259. <https://doi.org/10.1109/TC.1976.1674592>.
- Baylor, S., Corbett, P.F., Park, C.-I., 1999. Efficient method for providing fault tolerance against double device failures in multiple device systems. <https://patents.google.com/patent/US5862158A/en>. US Patent 5,862,158.
- Beach, B., 2015. Backblaze Open Source Reed-Solomon Erasure Coding Source Code. Technical report. backblaze.com.
- Beckmann, N., Kriegel, H., Schneider, R., Seeger, B., 1990. The r*-tree: an efficient and robust access method for points and rectangles. In: Garcia-Molina, H., Jagadish, H.V. (Eds.), *Proceedings of the 1990 ACM SIGMOD International Conference on Management of Data*. Atlantic City, NJ, USA, May 23–25, 1990. ACM Press, pp. 322–331.
- Beetem, J.F., Denneau, M., Weingarten, D., 1985. The GF11 supercomputer. In: Gannon, T.F., Agerwala, T., Freiman, C. (Eds.), *Proceedings of the 12th Annual Symposium on Computer Architecture*. Boston, MA, USA, June 1985. IEEE Computer Society, pp. 108–115. <https://dl.acm.org/citation.cfm?id=327139>.
- Beigi, H., 2011. *Fundamentals of Speaker Recognition*. Springer, New York, NY. ISBN 978-0-387-77592-0.
- Bekey, G.A., Bekey, M., 2015. *A Remarkable Trajectory: From Humble Beginnings to Global Prominence*. Createspace Independent Publishing Platform. ISBN 978-1511619264.
- Belady, L.A., 1966. A study of replacement algorithms for virtual-storage computer. *IBM Syst. J.* 5 (2), 78–101. <https://doi.org/10.1147/sj.52.0078>.
- Bell, G., 2001. A personal digital store. *Commun. ACM* 44 (1), 86–91. <https://doi.org/10.1145/357489.357513>.
- Bell, G., Newell, A.C., 1971. *Computer Structures: Principles and Examples*. McGraw-Hill, New York, NY. ISBN 0-13-201799-7. <http://www.worldcat.org/oclc/59141337>.
- Bellman, R., 2003. *Dynamic Programming*. Computer Science. Dover. ISBN 978-0486428093.
- Benes, V.E., 1965. *Mathematical Theory of Connecting Networks and Telephone Traffic*. Academic Press. ISBN 0-12-087550-0.
- Bentley, J.L., 1975. Multidimensional binary search trees used for associative searching. *Commun. ACM* 18 (9), 509–517. <http://doi.acm.org/10.1145/361002.361007>.
- Berchtold, S., Keim, D.A., Kriegel, H., 1996. The x-tree: an index structure for high-dimensional data. In: Vijayaraman, T.M., Buchmann, A.P., Mohan, C., Sarda, N.L. (Eds.), *VLDB'96, Proceedings of 22th International Conference on Very Large Data Bases*. September 3–6, 1996, Mumbai (Bombay), India. Morgan Kaufmann, pp. 28–39. <http://www.vldb.org/conf/1996/P028.PDF>.
- Berchtold, S., Böhm, C., Kriegel, H., 1998. The pyramid-technique: towards breaking the curse of dimensionality. In: Haas, L.M., Tiwary, A. (Eds.), *SIGMOD 1998, Proceedings ACM SIGMOD International Conference on Management of Data*. Seattle, Washington, USA, June 2–4, 1998. ACM Press, pp. 142–153.
- Berkhin, P., 2006. A survey of clustering data mining techniques. In: Kogan, J., Nicholas, C.K., Teboulle, M. (Eds.), *Grouping Multidimensional Data – Recent Advances in Clustering*. Springer, pp. 25–71.
- Bernstein, P., 2017. Timestamp-based concurrency control in sdd-1. In: *Failed Aspiration in Database Systems – FADS@VLDB'17 Workshop*. Presentation only.
- Bernstein, P.A., Hadzilacos, V., Goodman, N., 1987. *Concurrency Control and Recovery in Database Systems*. Addison-Wesley. ISBN 0-201-10715-5. <http://research.microsoft.com/en-us/people/philbe/ccontrol.aspx>, 1987.
- Bernstein, P.A., Cseri, I., Dani, N., et al., 2011. Adapting Microsoft SQL server for cloud computing. In: Abiteboul, S., Böhm, K., Koch, C., Tan, K. (Eds.), *Proceedings of the 27th International Conference on Data Engineering, ICDE 2011*. April 11–16, 2011, Hannover, Germany. IEEE Computer Society, pp. 1255–1263.
- Bertino, E., Martino, L., 1993. *Object-Oriented Database Systems: Concepts and Architectures*. Addison-Wesley. ISBN 978-0201624397.
- Bestavros, A., 1990. SETH: a VLSI chip for the real-time information dispersal and retrieval for security and fault-tolerance. In: Wah, B.W. (Ed.), *Proceedings of the 1990 International Conference on Parallel Processing, Volume 1: Architecture*. Urbana-Champaign, IL, USA, August 1990. Pennsylvania State University Press, pp. 457–464.
- Bestavros, A., 1991. IDA-based disk arrays. Technical Memorandum 45312-890707-01TM, July 1989. Bell Labs, Holmdel, NJ. <http://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.56.4765>.
- Bhadkamkar, M., Guerra, J., Useche, L., Burnett, S., Liptak, J., Rangaswami, R., Hristidis, V., 2009. BORG: block-reorganization for self-optimizing storage systems. In: Seltzer, M.I., Wheeler, R. (Eds.), *7th USENIX Conference on File and Storage Technologies, Proceedings*. February 24–27, 2009, San Francisco, CA, USA. USENIX, pp. 183–196. http://www.usenix.org/events/fast09/tech/full_papers/bhadkamkar/bhadkamkar.pdf.
- Bhandarkar, D., 1996. *Alpha Implementations and Architecture – Complete Reference and Guide*. Digital Press. ISBN 978-1-55558-130-5.
- Bhandarkar, D., Clark, D.W., 1991. Performance from architecture: comparing a RISC and CISC with similar hardware organization. In: Patterson, D.A., Rau, B. (Eds.), *ASPLOS-IV Proceedings – Forth International Conference on Architectural Support for Programming Languages and Operating Systems*. Santa Clara, California, USA, April 8–11, 1991. ACM Press, pp. 310–319.
- Bhattacharjee, A., Lustig, D., 2018. *Architectural and Operating System Support for Virtual Memory*. Synthesis Lectures on Computer Architecture. Morgan-Claypool.
- Binnig, C., Hildenbrand, S., Färber, F., 2009. Dictionary-based order-preserving string compression for main memory column stores. In: Çetintemel, U., Zdonik, S.B., Kossmann, D., Tatbul, N. (Eds.), *Proceedings of the ACM SIGMOD International Conference on Management of Data, SIGMOD 2009*. Providence, Rhode Island, USA, June 29–July 2, 2009. ACM, pp. 283–296.

- Bitton, D., Gray, J., 1988. Disk shadowing. In: Bancilhon, F., DeWitt, D.J. (Eds.), Fourteenth International Conference on Very Large Data Bases, Proceedings. August 29–September 1, 1988, Los Angeles, California, USA. Morgan Kaufmann, pp. 331–338. <http://www.vldb.org/conf/1988/P331.PDF>, 1988.
- Blaauw, G.A., Jr., F.P.B. Computer Architecture: Concepts and Evolution. Addison-Wesley. ISBN 978-0201105575.
- Blaner, B., Abali, B., Bass, B.M., Chari, S., Kalla, R.N., Kunkel, S.R., Lauricella, K., Leavens, R., Reilly, J.J., Sandon, P.A., 2013. IBM POWER7+ processor on-chip accelerators for cryptography and active memory expansion. *IBM J. Res. Dev.* 57 (6). <https://doi.org/10.1147/JRD.2013.2280090>.
- Blasgen, M.W., Astrahan, M.M., Chamberlin, D.D., Gray, J., Blasgen III, W.F., Lindsay, B.G., Lorie, R.A., Mehl, J.W., Price, T.G., Putzolu, G.R., Schkolnick, M., Selinger, P.G., Slutz, D.R., Strong, H.R., Traiger, I.L., Wade, B.W., Yost, R.A., 1981. System R: an architectural overview. *IBM Syst. J.* 20 (1), 41–62. <https://doi.org/10.1147/sj.201.0041>.
- Blaum, M., 1987. A class of byte-correcting array codes. Technical report. IBM Almaden Research Center, San Jose, CA.
- Blaum, M., 2006. A family of MDS array codes with minimal number of encoding operations. In: Proceedings 2006 IEEE International Symposium on Information Theory, ISIT 2006. Seattle, Washington, USA, July 9–14, 2006. IEEE, pp. 2784–2788.
- Blaum, M., 2019. A short course on error-correcting codes. CoRR, arXiv:1908.09903.
- Blaum, M., Ouchi, N.K., 1994. Method and means for b-adjacent coding and rebuilding data from up to two unavailable DASDs in a DASD array. <https://patents.justia.com/patent/5333143>. US Patent 5,333,143.
- Blaum, M., Roth, R.M., 1993. New array codes for multiple phased burst correction. *IEEE Trans. Inf. Theory* 39 (1), 66–77. <https://doi.org/10.1109/18.179343>.
- Blaum, M., Roth, R.M., 1999. On lowest density MDS codes. *IEEE Trans. Inf. Theory* 45 (1), 46–59. <https://doi.org/10.1109/18.746771>.
- Blaum, M., Farrell, P.C., van Tilborg, H.C., 1988. Array codes. In: Pless, V.S., Huffman, W. (Eds.), *Handbook of Coding Theory*. Elsevier, Amsterdam, The Netherlands, pp. 1855–1909. Chapter 22.
- Blaum, M., Brady, J., Bruck, J., Menon, J., 1994. EVENODD: an optimal scheme for tolerating double disk failures in RAID architectures. In: Patterson, D.A. (Ed.), *Proceedings of the 21st Annual International Symposium on Computer Architecture*. Chicago, IL, USA, April 1994. IEEE Computer Society, pp. 245–254.
- Blaum, M., Brady, J., Bruck, J., Menon, J., 1995. EVENODD: an efficient scheme for tolerating double disk failures in RAID architectures. *IEEE Trans. Comput.* 44 (2), 192–202. <https://doi.org/10.1109/12.364531>.
- Blaum, M., Bruck, J., Vardy, A., 1996. MDS array codes with independent parity symbols. *IEEE Trans. Inf. Theory* 42 (2), 529–542. <https://doi.org/10.1109/18.485722>.
- Blaum, M., Brady, J., Bruck, J., Menon, J., Vardy, A., 2001. The EVENODD code and its generalizations. In: Jin, H., Cortes, T., Buyya, R. (Eds.), *High Performance Mass Storage and Parallel I/O: Technologies and Applications*. Wiley, New York, NY. ISBN 0-471-20809-4, pp. 187–208.
- Blaum, M., Hafner, J.L., Hetzler, S., 2013. Partial-mds codes and their application to RAID type of architectures. *IEEE Trans. Inf. Theory* 59 (7), 4510–4519. <https://doi.org/10.1109/TIT.2013.2252395>.
- Blomer, J., Kalfane, M., Karpinski, M., Karp, R., Luby, M., Zuckerman, D., 1995. An xor-based erasure-resilient coding scheme. Technical Report TR-95-048. International Computer Science Institute.
- Bloom, B.H., 1970. Space/time trade-offs in hash coding with allowable errors. *Commun. ACM* 13 (7), 422–426. <http://doi.acm.org/10.1145/362686.362692>.
- Blum, A., Hopcroft, J., Kannan, R., 2020. *Foundations of Data Science*. Cambridge University Press. ISBN 978-1108485067.
- Blum, A.M., Goyal, A., Heidelberg, P., Lavenberg, S.S., Nakayama, M.K., Shahabuddin, P., 1994. Modeling and analysis of system dependability using the system availability estimator. In: *Digest of Papers: FTCS/24, The Twenty-Fourth Annual International Symposium on Fault-Tolerant Computing*. Austin, Texas, USA, June 15–17, 1994. IEEE Computer Society, pp. 137–141.
- Blum, R., Beyer, B., 2019. Achieving digital permanence. *Commun. ACM* 62 (5), 7.
- Bober, P.M., Carey, M.J., 1992. Multiversion query locking. In: Yuan, L. (Ed.), *18th International Conference on Very Large Data Bases, Proceedings*. August 23–27, 1992, Vancouver, Canada. Morgan Kaufmann, pp. 497–510. <http://www.vldb.org/conf/1992/P497.PDF>.
- Bobrow, D.G., Burchfiel, J.D., Murphy, D.L., Tomlinson, R.S., 1972. Tenex, a paged time sharing system for the PDP-10. *Commun. ACM* 15 (3), 135–143. <https://doi.org/10.1145/361268.361271>.
- Bog, A., 2014. *Benchmarking Transaction and Analytic Processing – The Creation of a Mixed Workload Benchmark and Its Application*. Springer-Verlag. ISBN 978-3-642-38070-9.
- Bohl, M., 1981. *Introduction to IBM Direct Access Storage Devices*. Science Research Associates, Chicago, IL. ISBN 0-574-21140-3.
- Böhm, C., Kailing, K., Kröger, P., Zimek, A., 2004. Computing clusters of correlation connected objects. In: Weikum, G., König, A.C., Deßloch, S. (Eds.), *Proceedings of the ACM SIGMOD International Conference on Management of Data*. Paris, France, June 13–18, 2004. ACM, pp. 455–466.
- Bohossian, V., Fan, C.C., LeMahieu, P.S., Riedel, M.D., Xu, L., Bruck, J., 2001. Computing in the RAIN: a reliable array of independent nodes. *IEEE Trans. Parallel Distrib. Syst.* 12 (2), 99–114. <https://doi.org/10.1109/71.910866>.
- Bolch, G., Greiner, S., de Meer, H., Trivedi, K.S., 2006. *Queueing Networks and Markov Chains – Modeling and Performance Evaluation with Computer Science Applications*, 2nd edition. Wiley. ISBN 978-0-471-56525-3. <http://eu.wiley.com/WileyCDA/WileyTitle/productCd-0471565253.html>.
- Bolosky, W.J., Corbin, S., Goebel, D., Douceur, J.R., 2000. Single instance storage in windows 2000. In: *USENIX Windows System Symp. (WSS)*. USENIX, pp. 1–12.
- Boncz, P.A., Kersten, M.L., Manegold, S., 2008. Breaking the memory wall in monetdb. *Commun. ACM* 51 (12), 77–85. <https://doi.org/10.1145/1409360.1409380>.
- Bonetti, A., Golman, R., Giterman, R., Teman, A., Burg, A., 2020. Gain-cell embedded drams: modeling and design space. *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.* 28 (3), 646–659.

- Bonwick, J., 2016. Scalable high performance flash systems. In: 32nd Symposium on Mass Storage Systems and Technologies. MSST 2016, Santa Clara, CA, USA, May 2–6, 2016.
- Booch, G., 2017. The Unified Modeling Language User Guide, 2. Object Technology Series. Addison-Wesley. ISBN 978-0134852157.
- Boral, H., DeWitt, D.J., 1983. Database machines: an idea whose time passed? A critique of the future of database machines. In: Leilich, H., Missikoff, M. (Eds.), Database Machines, International Workshop. Munich, Germany, September 1983. Springer, pp. 166–187.
- Boral, H., Faudemay, P. (Eds.), 1989. Database Machines, Sixth International Workshop, IWDM '89, Proceedings. Deauville, France, June 19–21, 1989. Lecture Notes in Computer Science, vol. 368. Springer. ISBN 3-540-51324-8.
- Boral, H., DeWitt, D.J., Friedland, D., Jarrell, N.F., Wilkinson, W.K., 1982. Implementation of the database machine DIRECT. IEEE Trans. Softw. Eng. 8 (6), 533–543. <https://doi.org/10.1109/TSE.1982.235882>.
- Boral, H., Alexander, W., Clay, L., Copeland, G.P., Danforth, S., Franklin, M.J., Hart, B.E., Smith, M.G., Valduriez, P., 1990. Prototyping bubba, a highly parallel database system. IEEE Trans. Knowl. Data Eng. 2 (1), 4–24. <https://doi.org/10.1109/69.50903>.
- Bornholt, J., Lopez, R., Carmean, D.M., Ceze, L., Seelig, G., Strauss, K., 2016. A dna-based archival storage system. In: Conte, T., Zhou, Y. (Eds.), Proceedings of the Twenty-First International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS '16. Atlanta, GA, USA, April 2–6, 2016. ACM, pp. 637–649.
- Borowsky, E., Golding, R.A., Merchant, A., Shriver, E., Spasojevic, M., Wilkes, J., 2007. Using attribute-managed storage to achieve qos. In: Proc. 5th International Workshop on Quality of Service. Columbia University, New York, NY, pp. 203–206.
- Bose, N.K., 1939. On the construction of balanced incomplete block designs. Ann. Eugenics 9, 353–399. <https://doi.org/10.1007/s10479-018-2973-1>.
- Bouknight, W.J., Denenberg, S.A., McIntyre, D.E., Randall, J.M., Sameh, A.H., Slotnick, D.L., 1972. The illiac iv system. Proc. IEEE 60 (4).
- Bouricius, W.G., Carter, W.C., Jessep, D.C., Schneider, P.R., Wadia, A.B., 1971. Reliability modeling for fault-tolerant computers. IEEE Trans. Comput. 20 (11), 1306–1311. <https://doi.org/10.1109/T-C.1971.223132>.
- Boxma, O.J., Cohen, J.W., 1991. The M/G/1 queue with permanent customers. IEEE J. Sel. Areas Commun. 9 (2), 179–184. <https://doi.org/10.1109/49.68445>.
- Bozkaya, T., Özsoyoglu, Z.M., 1997. Distance-based indexing for high-dimensional metric spaces. In: Peckham, J. (Ed.), SIGMOD 1997, Proceedings ACM SIGMOD International Conference on Management of Data. Tucson, Arizona, USA, May 13–15, 1997. ACM Press, pp. 357–368.
- Bozman, G.P., Bucu, W.M., Daly, T.P., Tetzlaff, W.H., 1984. Analysis of free-storage algorithms – revisited. IBM Syst. J. 23 (1), 44–64. <https://doi.org/10.1147/sj.231.0044>.
- Brady, J., 2018. Computer Systems Architect: From Systems Engineer to IBM Fellow, Experiencing IBM from the Inside. Amazon Digital Services.
- Brandwajn, A., 1981. Models of DASD subsystems: basic model of reconnection. Perform. Eval. 1 (3), 263–281. [https://doi.org/10.1016/0166-5316\(81\)90005-5](https://doi.org/10.1016/0166-5316(81)90005-5).
- Brandwajn, A., 1983a. Models of DASD subsystems with multiple access paths: a throughput-driven approach. IEEE Trans. Comput. 32 (5), 451–463. <https://doi.org/10.1109/TC.1983.1676256>.
- Brandwajn, A., 1983b. A study of dynamic reconnection. In: Schwetman, H.D., Bruell, S.C., Dowdy, L.W. (Eds.), Proceedings of the International Conference on Measurements and Modeling of Computer Systems, SIGMETRICS 1983. August 29–31, 1983, Minneapolis, Minnesota, USA. ACM, pp. 1–11.
- Breslau, L., Cao, P., Fan, L., Phillips, G., Shenker, S., 1999. Web caching and zipf-like distributions: evidence and implications. In: Proceedings IEEE INFOCOM '99, The Conference on Computer Communications, Eighteenth Annual Joint Conference of the IEEE Computer and Communications Societies, The Future Is Now. New York, NY, USA, March 21–25, 1999. IEEE Computer Society, pp. 126–134.
- Breß, S., Siegmund, N., Bellatreche, L., Saake, G., 2013. An operator-stream-based scheduling engine for effective GPU coprocessing. In: Catania, B., Guerrini, G., Pokorný, J. (Eds.), Advances in Databases and Information Systems – 17th East European Conference, ADBIS 2013, Proceedings. Genoa, Italy, September 1–4, 2013. In: Lecture Notes in Computer Science, vol. 8133. Springer, pp. 288–301.
- Breß, S., Heime, M., Saecker, M., Köcher, B., Markl, V., Saake, G., 2014a. Ocelot/hype: optimized data processing on heterogeneous hardware. Proc. VLDB Endow. 7 (13), 1609–1612. <http://www.vldb.org/pvldb/vol7/p1609-bress.pdf>.
- Breß, S., Heime, M., Siegmund, N., Bellatreche, L., Saake, G., 2014b. GPU-accelerated database systems: survey and open challenges. Trans. Large Scale Data Knowl. Centered Syst. 15, 1–35. https://doi.org/10.1007/978-3-662-45761-0_1.
- Brewer, E., Ying, L., Greenfield, L., Cypher, R., T'so, T., 2016. Disks for datacenters. Technical Report. Google, 111 8th Ave, New York, NY. <https://research.google.com/pubs/pub44830.html>. 16 pages.
- Brewer, E.A., 2010. A certain freedom: thoughts on the CAP theorem. In: Richa, A.W., Guerraoui, R. (Eds.), Proceedings of the 29th Annual ACM Symposium on Principles of Distributed Computing, PODC 2010. Zurich, Switzerland, July 25–28, 2010. ACM, p. 335.
- Brewer, E.A., 2012. Pushing the CAP: strategies for consistency and availability. IEEE Computer 45 (2), 23–29. <https://doi.org/10.1109/MC.2012.37>.
- Broder, A.Z., Glassman, S.C., Manasse, M.S., Zweig, G., 1997. Syntactic clustering of the web. Comput. Netw. 29 (8–13), 1157–1166. [https://doi.org/10.1016/S0169-7552\(97\)00031-7](https://doi.org/10.1016/S0169-7552(97)00031-7).
- Brodie, M.L. (Ed.), 2019. Making Databases Work: The Pragmatic Wisdom of Michael Stonebraker. ACM Books Series, vol. 22. Morgan and Claypool, San Rafael, CA. ISBN 978-1-94748-719-2.
- Brooks, F., 1995. The Mythical Man-Month – Essays on Software Engineering, 2. Addison-Wesley. ISBN 978-0-201-83595-3. First edition 1975.
- Brorsson, M., 1991. Local vs. global memory in the IBM RP3: experiments and performance modelling. In: Proceedings of the Third IEEE Symposium on Parallel and Distributed Processing, SPDP 1991. 2–5 December 1991, Dallas, Texas, USA. IEEE Computer Society, pp. 496–503.
- Brown, A.B., Patterson, D.A., 2000. Towards availability benchmarks: a case study of software RAID systems. In: Proceedings of the General Track: 2000 USENIX Annual Technical Conference. San Diego, CA, USA, June 18–23, 2000. USENIX, pp. 263–276. <http://www.usenix.org/publications/library/proceedings/usenix2000/general/brown.html>.

- Brumelle, S.L., 1971. Some inequalities for parallel-server queues. *Oper. Res.* 19 (2).
- Bruno, J.L., Brustoloni, J.C., Gabber, E., Özden, B., Silberschatz, A., 1999. Disk scheduling with quality of service guarantees. In: *IEEE International Conference on Multimedia Computing and Systems, ICMCS 1999, Volume 2*. Florence, Italy, June 7–11, 1999. IEEE Computer Society, pp. 400–405.
- Bucy, J.S., Schindler, J., Schlosser, S.W., Ganger, G., et al., 2008. The disksim simulation environment version 4.0 reference manual. Technical Report CMU-PDL-08-101. Parallel Data laboratory, CMU, Pittsburgh, PA.
- Burks, A.W., Goldstine, H.H., von Neumann, J., 1963. Preliminary discussion of the logical design of an electronic computing instrument-1946. Technical report. U.S. Army Ordnance Department.
- Burr, G.W., Kurdi, B.N., Scott, J.C., Lam, C.H., Gopalakrishnan, K., Shenoy, R.S., 2008. Overview of candidate device technologies for storage-class memory. *IBM J. Res. Dev.* 52 (4–5), 449–464. <https://doi.org/10.1147/rd.524.0449>.
- Burrows, M., 2006. The chubby lock service for loosely-coupled distributed systems. In: Bershad, B.N., Mogul, J.C. (Eds.), *7th Symposium on Operating Systems Design and Implementation (OSDI '06)*. November 6–8, Seattle, WA, USA. USENIX Association, pp. 335–350. <http://www.usenix.org/events/osdi06/tech/burrows.html>.
- Burrows, M., Wheeler, D.J., 1994. A block-sorting lossless data compression algorithm. Technical Report 124. DEC SRC Research, Palo Alto, CA. <https://www.hpl.hp.com/techreports/Compaq-DEC/SRC-RR-124.html>.
- Burrows, M., Jerian, C., Lampson, B.W., Mann, T.P., 1992. On-line data compression in a log-structured file system. In: Flahive, B., Wexelblat, R.L. (Eds.), *ASPLOS-V Proceedings – Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*. Boston, Massachusetts, USA, October 12–15, 1992. ACM Press, pp. 2–9.
- Buttcher, S., Clarke, C.L.A., Cormack, G.V., 2010. *Information Retrieval: Implementing and Evaluating Search Engines*. The MIT Press, Cambridge, Mass. ISBN 978-0262026512.
- Butterworth, H.E., 1999. The design of segment filling and selection algorithms for efficient free-space collection in a log-structured array. Technical report. IBM Hursley. Unpublished report.
- Buzen, J.P., 1971. *Queueing Network Models of Multiprogramming*. Outstanding Dissertations in the Computer Sciences. Garland Publishing, New York. ISBN 0-8240-4401-0.
- Buzen, J.P., 1973. Computational algorithms for closed queueing networks with exponential servers. *Commun. ACM* 16 (9), 527–531. <https://doi.org/10.1145/362342.362345>.
- Buzen, J.P., 1978. A queueing network model of MVS. *ACM Comput. Surv.* 10 (3), 319–331. <http://doi.acm.org/10.1145/356733.356738>.
- Buzen, J.P., Chen, P.P., 1974. Optimal load balancing in memory hierarchies. In: *IFIP Congress*, pp. 271–275.
- Buzen, Jeffrey P., Goldberg, P.S., 1974. Guidelines for the use of infinite source queueing models in the analysis of computer system performance. In: *American Federation of Information Processing Societies: 1974 National Computer Conference*. 6–10 May 1974, Chicago, Illinois, USA. In: *AFIPS Conference Proceedings*, vol. 43. AFIPS Press, pp. 371–374.
- Buzen, J.P., Shum, A.W., 1987. A unified operational treatment of RPS reconnect delays. In: Bunt, R.B. (Ed.), *Proceedings of the 1987 ACM SIGMETRICS Conference on Measurement and Modeling of Computer Systems*. Banff, Alberta, Canada, May 11–14, 1987. ACM, pp. 78–92.
- Buzen, J.P., Shum, A.W., 1990. I/O performance trade-offs and MVS/ESA considerations. In: *16th International Computer Measurement Group Conference, Proceedings*. December 10–14, 1990, Orlando, FL, USA. Computer Measurement Group, pp. 695–702. http://www.cmg.org/?s2member_file_download=proceedings/1990/90INT079.pdf.
- Cabrera, L., Long, D.D.E., 1991. Swift: using distributed disk striping to provide high I/O data rates. *Comput. Syst.* 4 (4), 405–436. http://www.usenix.org/publications/compsystems/1991/fall_cabrera.pdf.
- Cadambe, V.R., Huang, C., Li, J., 2011. Permutation code: optimal exact-repair of a single failed node in MDS code based distributed storage systems. In: Kuleshov, A., Blinovsky, V.M., Ephremides, A. (Eds.), *IEEE International Symposium on Information Theory Proceedings, ISIT 2011*. St. Petersburg, Russia, July 31–August 5, 2011. IEEE, pp. 1225–1229.
- Cady, F., Zhuang, Y., Harchol-Balter, M., 2011. Stochastic analysis of hard disk drives. *Int. J. Stoch. Anal.* 2011, 390548.
- Calder, B., Wang, J., Ogun, A., et al., 2011. Windows azure storage: a highly available cloud storage service with strong consistency. In: Druschel, P. (Ed.), *Proceedings of the 23rd ACM Symposium on Operating Systems Principles 2011, SOSP 2011*. Cascais, Portugal, October 23–26, 2011. ACM, pp. 143–157.
- Calderbank, A.R., Coffman Jr., Edward G., Flatto, L., 1984. Optimum head separation in a disk system with two read/write heads. *J. ACM* 31 (4), 826–838. <http://doi.acm.org/10.1145/1634.1638>.
- Calderbank, A.R., Coffman Jr., Edward G., Flatto, L., 1985. Sequencing problems in two-server systems. *Math. Oper. Res.* 10 (4), 585–598. <https://doi.org/10.1287/moor.10.4.585>.
- Canaday, R.H., Harrison, R.D., Ivie, E.L., Ryder, J.L., Wehr, L.A., 1974. A back-end computer for data base management. *Commun. ACM* 17 (10), 575–582. <https://doi.org/10.1145/355620.361172>.
- Canyon, R., 2013. *Open: How Compaq Ended IBM's PC Domination and Helped Invent Modern Computing*. BenBilla Books. ISBN 978-1937856991.
- Cao, P., Felten, E.W., Li, K., 1994a. Implementation and performance of application-controlled file caching. In: *Proceedings of the First USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. Monterey, California, USA, November 14–17, 1994. USENIX Association, pp. 165–177. <http://dl.acm.org/citation.cfm?id=1267651>.
- Cao, P., Lim, S.B., Venkataraman, S., Wilkes, J., 1994b. The TickerTAIP parallel RAID architecture. *ACM Trans. Comput. Syst.* 12 (3), 236–269. <http://doi.acm.org/10.1145/185514.185517>.
- Cao, P., Lim, S.B., Venkataraman, S., Wilkes, J., 1994c. The TickerTaip parallel RAID architecture. *ACM Trans. Comput. Syst.* 12 (3), 236–269. <https://doi.org/10.1145/185514.185517>.

- Cao, P., Felten, E.W., Karlin, A.R., Li, K., 1995. A study of integrated prefetching and caching strategies. In: Gaither, B.D. (Ed.), *Proceedings of the 1995 ACM SIGMETRICS Joint International Conference on Measurement and Modeling of Computer Systems*. Ottawa, Canada, May 15–19, 1995. ACM, pp. 188–197.
- Cao, P., Felten, E.W., Karlin, A.R., Li, K., 1996. Implementation and performance of integrated application-controlled file caching, prefetching, and disk scheduling. *ACM Trans. Comput. Syst.* 14 (4), 311–343. <http://doi.acm.org/10.1145/235543.235544>.
- Cao, W., Liu, Z., Wang, P., Chen, S., Zhu, C., Zheng, S., Wang, Y., Ma, G., 2018. Polarfs: an ultra-low latency and failure resilient distributed file system for shared storage cloud database. *Proc. VLDB Endow.* 11 (12), 1849–1862. <http://www.vldb.org/pvldb/vol11/p1849-cao.pdf>.
- Carata, L., Akoush, S., Balakrishnan, N., Bytheway, T., Sohan, R., Seltzer, M., Hopper, A., 2014. A primer on provenance. *Commun. ACM* 57 (5), 52–60. <https://doi.org/10.1145/2596628>.
- Carey, M.J., Jauhari, R., Livny, M., 1989. Priority in DBMS resource scheduling. In: Apers, P.M.G., Wiederhold, G. (Eds.), *Proceedings of the Fifteenth International Conference on Very Large Data Bases*. August 22–25, 1989, Amsterdam, The Netherlands. Morgan Kaufmann, pp. 397–410. <http://www.vldb.org/conf/1989/P397.PDF>.
- Carley, L.R., Ganger, G.R., Nagle, D., 2000. Mems-based integrated-circuit mass-storage systems. *Commun. ACM* 43 (11), 72–80. <http://doi.acm.org/10.1145/353360.353367>.
- Carney, D., Çetintemel, U., Cherniack, M., Convey, C., Lee, S., Seidman, G., Stonebraker, M., Tatbul, N., Zdonik, S.B., 2002. Monitoring streams – a new class of data management applications. In: Bernstein, P.A. (Ed.), *Proceedings of 28th International Conference on Very Large Data Bases, VLDB 2002*. Hong Kong, August 20–23, 2002. Morgan Kaufmann, pp. 215–226. <http://www.vldb.org/conf/2002/S07P02.pdf>.
- Carrera, E.V., Pinheiro, E., Bianchini, R., 2003. Conserving disk energy in network servers. In: Banerjee, U., Gallivan, K.A., González, A. (Eds.), *Proceedings of the 17th Annual International Conference on Supercomputing, ICS 2003*. San Francisco, CA, USA, June 23–26, 2003. ACM, pp. 86–97.
- Carson, S.D., Nirkhe, V., Vongsathorn, P., 1992. A discrete-state model of the two-headed disk. *Inf. Process. Lett.* 41 (6), 341–345. [https://doi.org/10.1016/0020-0190\(92\)90162-O](https://doi.org/10.1016/0020-0190(92)90162-O).
- Carson, S.D., Reynolds, P.F., 1989. Adaptive disk reorganization. Technical Report UMIACS TR-89-4. University of Maryland, College Park, MD.
- Carson, S.D., Setia, S., 1992. Analysis of the periodic update write policy for disk cache. *IEEE Trans. Softw. Eng.* 18 (1), 44–54. <https://doi.org/10.1109/32.120315>.
- Carson, S.D., Setia, S., 1994. Optimal write batch size in log-structured file systems. *Comput. Syst.* 7 (2), 263–281. http://www.usenix.org/publications/compsystems/1994/spr_carson.pdf.
- Carson, S.D., Vongsathorn, P., 1989. Error bounds on disk arrangement using frequency information. *Inf. Process. Lett.* 31 (4), 209–213. [https://doi.org/10.1016/0020-0190\(89\)90125-7](https://doi.org/10.1016/0020-0190(89)90125-7).
- Castelli, V., 2002. Multidimensional indexing structures for content-based retrieval. In: Caatelli, V., Bergman, L.D. (Eds.), *Image Databases: Search and Retrieval of Digital Imagery*. Wiley-Interscience, New York, NY, pp. 373–433. Chapter 14.
- Castelli, V., Bergman, L.D. (Eds.), 2002. *Image Databases: Search and Retrieval of Digital Imagery*. Wiley Interscience, New York, NY. ISBN 978-0471321163.
- Castelli, V., Thomasian, A., Li, C., 1997a. Multidimensional data clustering and dimension reduction for indexing and searching.
- Castelli, V., Thomasian, A., Li, C., 1997b. Searching multidimensional indexes using associated clustering and dimension reduction information.
- Castelli, V., Thomasian, A., Li, C., 2000. CSVD: clustering and singular value decomposition for approximate similarity search in high-dimensional spaces. Technical report. IBM T.J. Watson Research Center, Yorktown Heights, NY.
- Castelli, V., Thomasian, A., Li, C., 2003. CSVD: clustering and singular value decomposition for approximate similarity search in high-dimensional spaces. *IEEE Trans. Knowl. Data Eng.* 15 (3), 671–685. <https://doi.org/10.1109/TKDE.2003.1198398>.
- Castro, P.C., Ishakian, V., Muthusamy, V., Slominski, A., 2019. The rise of serverless computing. *Commun. ACM* 62 (12), 44–54. <https://doi.org/10.1145/3368454>.
- Celko, J., 2014. *Joe Celko's SQL for Smarties: Advanced SQL Programming*. Morgan Kaufmann Series in Data Management Systems. Morgan Kaufmann. ISBN 978-0128007617.
- Ceri, S., Pellagatti, G. *Distributed Databases: Principles and Systems*. McGraw-Hill-India. ISBN 978-0070265110.
- Chakrabarti, K., Mehrotra, S., 2000. Local dimensionality reduction: a new approach to indexing high dimensional spaces. In: Abbadi, A.E., Brodie, M.L., Chakravarthy, S., et al. (Eds.), *VLDB 2000, Proceedings of 26th International Conference on Very Large Data Bases*. September 10–14, 2000, Cairo, Egypt. Morgan Kaufmann, pp. 89–100. <http://www.vldb.org/conf/2000/P089.pdf>.
- Chamberlin, D.D., Astrahan, M.M., Eswaran, K.P., Griffiths, P.P., Lorie, R.A., Mehl, J.W., Reisner, P., Wade, B.W., 1976. SEQUEL 2: a unified approach to data definition, manipulation, and control. *IBM J. Res. Dev.* 20 (6), 560–575. <https://doi.org/10.1147/rd.206.0560>.
- Chamberlin, D.D., Astrahan, M.M., Blasgen, M.W., Gray, J., King III, W.F., Lindsay, B.G., Lorie, R.A., Mehl, J.W., Price, T.G., Putzolu, G.R., Selinger, P.G., Schkolnick, M., Slutz, D.R., Traiger, I.L., Wade, B.W., Yost, R.A., 1981. A history and evaluation of system R. *Commun. ACM* 24 (10), 632–646. <https://doi.org/10.1145/358769.358784>.
- Chamberlin, D.D., Florescu, D., Robie, J., Simeon, J., Stefanescu, M., 2001. Xquery: a query language for XML. Technical report. World Wide Web Consortium. <http://www.w3.org/TR/xquery>.
- Chandramouli, B., Goldstein, J., Barnett, M., DeLine, R., Platt, J.C., Terwilliger, J.F., Wernsing, J., 2014. Trill: a high-performance incremental query processor for diverse analytics. *Proc. VLDB Endow.* 8 (4), 401–412. <http://www.vldb.org/pvldb/vol8/p401-chandramouli.pdf>.
- Chandy, J.A., 2007. Dual actuator logging disk architecture and modeling. *J. Syst. Archit.* 53 (12), 913–926. <https://doi.org/10.1016/j.sysarc.2007.03.001>.
- Chandy, K.M., Lamport, L., 1985. Distributed snapshots: determining global states of distributed systems. *ACM Trans. Comput. Syst.* 3 (1), 63–75. <https://doi.org/10.1145/214451.214456>.
- Chang, E.Y., Garcia-Molina, H., 1997. Reducing initial latency in media servers. *IEEE Multimed.* 4 (3), 50–61. <https://doi.org/10.1109/93.621582>.

- Chang, F., Dean, J., Ghemawat, S., Hsieh, W.C., Wallach, D.A., Burrows, M., Chandra, T., Fikes, A., Gruber, R.E., 2008. Bigtable: a distributed storage system for structured data. *ACM Trans. Comput. Syst.* 26 (2), 4:1–4:26. <http://doi.acm.org/10.1145/1365815.1365816>.
- Chang, F.W., Gibson, G.A., 1999. Automatic I/O hint generation through speculative execution. In: Seltzer, M.I., Leach, P.J. (Eds.), *Proceedings of the Third USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. New Orleans, Louisiana, USA, February 22–25, 1999. USENIX Association, pp. 1–14. <https://dl.acm.org/citation.cfm?id=296807>.
- Chang, L., Kuo, T., 2005. Efficient management for large-scale flash-memory storage systems with resource conservation. *ACM Trans. Storage* 1 (4), 381–418. <http://doi.acm.org/10.1145/1111609.1111610>.
- Chase, J.S., Anderson, D.C., Thakar, P.N., Vahdat, A., Doyle, R.P., 2001. Managing energy and server resources in hosting centres. In: Marzullo, K., Satyanarayanan, M. (Eds.), *Proceedings of the 18th ACM Symposium on Operating System Principles, SOSP 2001*. Chateau Lake Louise, Banff, Alberta, Canada, October 21–24, 2001. ACM, pp. 103–116.
- Chaudhuri, S., Narasayya, V.R., 1997. An efficient cost-driven index selection tool for Microsoft SQL server. In: Jarke, M., Carey, M.J., Dittrich, K.R., et al. (Eds.), *VLDB'97, Proceedings of 23rd International Conference on Very Large Data Bases*. August 25–29, 1997, Athens, Greece. Morgan Kaufman, pp. 146–155. <http://www.vldb.org/conf/1997/P146.PDF>.
- Chee, C., Lu, H., Tang, H., Ramamoorthy, C.V., 1998. Adaptive prefetching and storage reorganization in A log-structured storage system. *IEEE Trans. Knowl. Data Eng.* 10 (5), 824–838. <https://doi.org/10.1109/69.729739>.
- Chen, F., Hou, B., Lee, R., 2016. Internal parallelism of flash memory-based solid-state drives. *ACM Trans. Storage* 12 (3), 13:1–13:39. <https://doi.org/10.1145/2818376>.
- Chen, Henry C.H., Hu, Y., Lee, P.P.C., Tang, Y., 2014. NCCloud: a network-coding-based storage system in a cloud-of-clouds. *IEEE Trans. Comput.* 63 (1), 31–44. <https://doi.org/10.1109/TC.2013.167>.
- Chen, M., Kandlur, D.D., Yu, P.S., 1993. Optimization of the grouped sweeping scheduling (GSS) with heterogeneous multimedia streams. In: Garcia-Luna-Aceves, J.J., Rangan, P.V. (Eds.), *Proceedings of the First ACM International Conference on Multimedia '93*. Anaheim, CA, USA, August 1–6, 1993. ACM Press, pp. 235–242.
- Chen, P.M., Lee, E.K., 1995. Striping in a RAID level 5 disk array. In: Gaither, B.D. (Ed.), *Proceedings of the 1995 ACM SIGMETRICS Joint International Conference on Measurement and Modeling of Computer Systems*. Ottawa, Canada, May 15–19, 1995. ACM, pp. 136–145.
- Chen, P.M., Patterson, D.A., 1990. Maximizing performance in a striped disk array. In: Baer, J., Snyder, L., Goodman, J.R. (Eds.), *Proceedings of the 17th Annual International Symposium on Computer Architecture*. Seattle, WA, USA, June 1990. ACM, pp. 322–331.
- Chen, P.M., Gibson, G.A., Katz, R.H., Patterson, D.A., 1990. An evaluation of redundant arrays of disks using an Amdahl 5890. In: Nutt, G.J. (Ed.), *Proceedings of the 1990 ACM SIGMETRICS Conference on Measurement and Modeling of Computer Systems*. University of Colorado, Boulder, Colorado, USA, May 22–25, 1990. ACM, pp. 74–85.
- Chen, P.M., Lee, E.K., Drapeau, A.L., et al., 1994a. Performance and design evaluation of the RAID-II storage server. *Distrib. Parallel Databases* 2 (3), 243–260. <https://doi.org/10.1007/BF01266330>.
- Chen, P.M., Lee, E.K., Gibson, G.A., Katz, R.H., Patterson, D.A., 1994b. RAID: high-performance, reliable secondary storage. *ACM Comput. Surv.* 26 (2), 145–185. <http://doi.acm.org/10.1145/176979.176981>.
- Chen, P.P., 1976. The entity-relationship model – toward a unified view of data. *ACM Trans. Database Syst.* 1 (1), 9–36. <https://doi.org/10.1145/320434.320440>.
- Chen, R., Xu, L., 2019. Practical performance study of erasure codes for high performance data storage systems. Technical report. Computer Science Dept, Wayne State University, Detroit, MI.
- Chen, R., Xu, L., 2020. Practical performance evaluation of space optimal erasure codes for high-speed data storage systems. *Springer Nat. Comput. Sci.* 1 (1), 54. <https://doi.org/10.1007/s42979-019-0057-1>.
- Chen, S., Towsley, D.F., 1993. The design and evaluation of RAID 5 and parity striping disk array architectures. *J. Parallel Distrib. Comput.* 17 (1–2), 58–74. <https://doi.org/10.1006/jpdc.1993.1005>.
- Chen, S., Towsley, D.F., 1996. A performance evaluation of RAID architectures. *IEEE Trans. Comput.* 45 (10), 1116–1130. <https://doi.org/10.1109/12.543706>.
- Chen, S., Stankovic, J.A., Kurose, J.F., Towsley, D.F., 1991. Performance evaluation of two new disk scheduling algorithms for real-time systems. *Real-Time Syst.* 3 (3), 307–336. <https://doi.org/10.1007/BF00364960>.
- Chen, T., Yeh, T.T., Wei, H., Fang, Y., Shih, W., Hsu, T., 2012. Cacheraid: an efficient adaptive write cache policy to conserve RAID disk array energy. In: *IEEE Fifth International Conference on Utility and Cloud Computing, UCC 2012*. Chicago, IL, USA, November 5–8, 2012. IEEE Computer Society, pp. 117–124.
- Chen, T., Wei, H., Yeh, T.T., Hsu, T., Shih, W., 2015. An energy-efficient and reliable storage mechanism for data-intensive academic archive systems. *ACM Trans. Storage* 11 (2), 10:1–10:21. <http://doi.acm.org/10.1145/2720021>.
- Chen, T.C., 1978. Computer technology and the database user. In: Yao, S.B. (Ed.), *Fourth International Conference on Very Large Data Bases*. West Berlin, Germany, September 13–15, 1978. IEEE Computer Society, pp. 72–86.
- Chen, T.C., Chang, H., 1978. Magnetic bubble memory and logic. *Adv. Comput.* 17, 223–282. [https://doi.org/10.1016/S0065-2458\(08\)60393-9](https://doi.org/10.1016/S0065-2458(08)60393-9).
- Chen, Y., Das, A., Qin, W., Sivasubramaniam, A., Wang, Q., Gautam, N., 2005. Managing server energy and operational costs in hosting centers. In: Eager, D.L., Williamson, C.L., Borst, S.C., Lui, J.C.S. (Eds.), *Proceedings of the International Conference on Measurements and Modeling of Computer Systems, SIGMETRICS 2005*. June 6–10, 2005, Banff, Alberta, Canada. ACM, pp. 303–314.
- Chen, Y.L., Mu, S., Li, J., Huang, C., Li, J., Ogun, A., Phillips, D., 2017. Giza: erasure coding objects across global data centers. In: Silva, D.D., Ford, B. (Eds.), *USENIX Annual Technical Conference, USENIX ATC 2017*. Santa Clara, CA, USA, July 12–14, 2017. USENIX Association, pp. 539–551. <https://www.usenix.org/conference/atc17/technical-sessions/presentation/chen-yu-lin>.

- Cheng, J.M., Looseley, C.R., Shibamiya, A., Worthington, P.S., 1984. IBM database 2 performance: design, implementation, and tuning. *IBM Syst. J.* 23 (2), 189–210. <https://doi.org/10.1147/sj.232.0189>.
- Cheng, R.M.K., Gillies, D.W., 1996. Disk management for a hard real-time file system. In: *Proceedings of the Eighth Euromicro Workshop on Real-Time Systems*, RTS 1996. L'Aquila, Italy, June 12–14, 1996. IEEE Computer Society, pp. 255–261.
- Chervenak, A.L., Katz, R.H., 1991. Performance of a RAID prototype. In: Keller, T.W. (Ed.), *Proceedings of the 1991 ACM SIGMETRICS Conference on Measurement and Modeling of Computer Systems*. San Diego, California, USA, May 21–24, 1991. ACM, pp. 188–197.
- Chien, C., 1993. Seek distances in disks with dual arms and mirrored disks. *Perform. Eval.* 18 (3), 175–188. [https://doi.org/10.1016/0166-5316\(93\)90015-M](https://doi.org/10.1016/0166-5316(93)90015-M).
- Ching, W., Ju, D., 1991. Execution of automatically parallelized APL programs on RP3. *IBM J. Res. Dev.* 35 (5), 767–778. <https://doi.org/10.1147/rd.355.0767>.
- Cho, S., Park, C., Won, Y., Kang, S., Cha, J., Yoon, S., Choi, J., 2015. Design tradeoffs of ssds: from energy consumption's perspective. *ACM Trans. Storage* 11 (2), 8:1–8:24. <http://doi.acm.org/10.1145/2644818>.
- Chou, H., DeWitt, D.J., 1985. An evaluation of buffer management strategies for relational database systems. In: Pirotte, A., Vassiliou, Y. (Eds.), *VLDB'85, Proceedings of 11th International Conference on Very Large Data Bases*. August 21–23, 1985, Stockholm, Sweden. Morgan Kaufmann, pp. 127–141. <http://www.vldb.org/conf/1985/P127.PDF>.
- Chou, H., DeWitt, D.J., Katz, R.H., Klug, A.C., 1985. Design and implementation of the Wisconsin storage system. *Softw. Pract. Exp.* 15 (10), 943–962. <https://doi.org/10.1002/spe.4380151003>.
- Choukse, E., Erez, M., Alameldeen, A.R., 2018. Compresso: pragmatic main memory compression. In: *51st Annual IEEE/ACM International Symposium on Microarchitecture, MICRO 2018*. Fukuoka, Japan, October 20–24, 2018. IEEE Computer Society, pp. 546–558.
- Chu, W.W., 1978. Statistical multiplexing systems for computer communications. <https://patents.google.com/patent/US4093823A/en>.
- Chu, W.W., 1969. Optimal file allocation in a multiple computer system. *IEEE Trans. Comput.* 18 (10), 885–889. <https://doi.org/10.1109/T-C.1969.222542>.
- Chu, W.W., Opderbeck, H., 1972. The page fault frequency replacement algorithm. In: *American Federation of Information Processing Societies: Proceedings of the AFIPS '72 Fall Joint Computer Conference, Part I*. December 5–7, 1972, Anaheim, California, USA. In: *AFIPS Conference Proceedings*, vol. 41. AFIPS/ACM/Thomson Book Company, Washington D.C., pp. 597–609.
- Chu, Y., 1975. *High-Level Language Computer Architecture*. Academic Press. ISBN 978-0-12-174150-1.
- Chun, B., Dabek, F., Haeberlen, A., Sit, E., Weatherspoon, H., Kaashoek, M.F., Kubiatowicz, J., Morris, R.T., 2006. Efficient replica maintenance for distributed storage systems. In: Peterson, L.L., Roscoe, T. (Eds.), *3rd Symposium on Networked Systems Design and Implementation (NSDI 2006)*, Proceedings. May 8–10, 2007, San Jose, California, USA. USENIX. <http://www.usenix.org/events/nsdi06/tech/chun.html>.
- Chung, C., Koo, J., Im Arvind, J., Lee, S., 2019. Lightstore: software-defined network-attached key-value drives. In: Bahar, I., Herlihy, M., Witchel, E., Lebeck, A.R. (Eds.), *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS 2019*. Providence, RI, USA, April 13–17, 2019. ACM, pp. 939–953.
- Ciaccia, P., Patella, M., Zezula, P., 1997. M-tree: an efficient access method for similarity search in metric spaces. In: Jarke, M., Carey, M.J., Dittrich, K.R., Lochovsky, F.H., Loucopoulos, P., Jeusfeld, M.A. (Eds.), *VLDB'97, Proceedings of 23rd International Conference on Very Large Data Bases*. August 25–29, 1997, Athens, Greece. Morgan Kaufmann, pp. 426–435. <http://www.vldb.org/conf/1997/P426.PDF>.
- Clark, Brian Eldridge, Timms Jr., George David, Lawlor, Francis Daniel, Schmidt-Stumpf, Werner Eric, Stewart, Terrence James, 1988. Parity Spreading to Enhance Storage Access. US Patent US4761785A. IBM. <https://patents.google.com/patent/CA1270333A>.
- Clarke, I., Sandberg, O., Wiley, B., Hong, T.W., 2000. Freenet: a distributed anonymous information storage and retrieval system. In: Federrath, H. (Ed.), *Designing Privacy Enhancing Technologies, International Workshop on Design Issues in Anonymity and Unobservability*, Proceedings. Berkeley, CA, USA, July 25–26, 2000. In: *Lecture Notes in Computer Science*, vol. 2009. Springer, pp. 46–66.
- Clos, C., 1953. A study of non-blocking switching networks. *Bell Syst. Tech. J.* 32 (2), 406–424.
- Codd, E.F., 1970. A relational model of data for large shared data banks. *Commun. ACM* 13 (6), 377–387. <http://doi.acm.org/10.1145/362384.362685>.
- Codd, E.F., 1982. Relational database: a practical foundation for productivity. *Commun. ACM* 25 (2), 109–117. <http://doi.acm.org/10.1145/358396.358400>.
- Coffman, E.G., Denning, P.J., 1973. *Operating Systems Theory*. Prentice-Hall.
- Coffman, E.G., Hofri, M., 1978. A class of FIFO queues arising in computer systems. *Oper. Res.* 26 (5), 864–880. <https://doi.org/10.1287/opre.26.5.864>.
- Coffman, E.G., Hofri, M., 1982. On the expected performance of scanning disks. *SIAM J. Comput.* 11 (1), 60–70. <https://doi.org/10.1137/0211005>.
- Coffman, E.G., Hofri, M., 1986. Queueing models of secondary storage devices. *Queueing Syst.* 1 (2), 129–168. <https://doi.org/10.1007/BF01536186>.
- Coffman, E.G., Hofri, M., 1990. Queueing models of secondary storage systems. In: Takagi, H. (Ed.), *Stochastic Analysis of Computer and Communication Systems*. North-Holland, Amsterdam, The Netherlands, pp. 549–588. Chapter 16.
- Colarelli, D., Grunwald, D., 2002. Massive arrays of idle disks for storage archives. In: Giles, R.C., Reed, D.A., Kelley, K. (Eds.), *Proceedings of the 2002 ACM/IEEE Conference on Supercomputing*, CD-ROM. Baltimore, Maryland, USA, November 16–22, 2002. IEEE Computer Society, pp. 56:1–56:11.
- Colbourn, C.J., van Oorschot, P.C., 1989. Applications of combinatorial designs in computer science. *ACM Comput. Surv.* 21 (2), 223–250. <http://doi.acm.org/10.1145/66443.66446>.
- Colgrove, J., Davis, J.D., Hayes, J., Miller, E.L., et al., 2015. Purity: building fast, highly-available enterprise flash storage from commodity components. In: Sellis, T.K., Davidson, S.B., Ives, Z.G. (Eds.), *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*. Melbourne, Victoria, Australia, May 31–June 4, 2015. ACM, pp. 1683–1694.
- Conway, A.E., Goyal, A., 1986. Monte Carlo simulation of computer system availability/reliability models. Technical Report RC 12459. IBM Research.

- Cooper, B.F., Ramakrishnan, R., Srivastava, U., et al., 2008. PNUTS: yahoo!'s hosted data serving platform. *Proc. VLDB Endow.* 1 (2), 1277–1288. <http://www.vldb.org/pvldb/vol1/1454167.pdf>.
- Cooper, B.F., Silberstein, A., Tam, E., Ramakrishnan, R., Sears, R., 2010. Benchmarking cloud serving systems with YCSB. In: Hellerstein, J.M., Chaudhuri, S., Rosenblum, M. (Eds.), *Proceedings of the 1st ACM Symposium on Cloud Computing, SoCC 2010*. Indianapolis, Indiana, USA, June 10–11, 2010. ACM, pp. 143–154.
- Cooper, B.F., Narayan, P.P.S., Ramakrishnan, R., Srivastava, U., et al., 2019. PNUTS to Sherpa: lessons from yahoo!'s cloud database. *Proc. VLDB Endow.* 12 (12), 2300–2307. <http://www.vldb.org/pvldb/vol12/p2300-cooper.pdf>.
- Copeland, G.P., Lipovski, G.J., Su, S.Y.W., 1973. The architecture of CASSM: a cellular system for non-numeric processing. In: Lipovski, G.J., Szygenda, S.A. (Eds.), *Proceedings of the 1st Annual Symposium on Computer Architecture*. Gainesville, FL, USA, December 1973. ACM, pp. 121–128.
- Corbett, J.C., Dean, J., Epstein, M., Fikes, A., et al., 2013. Spanner: Google's globally distributed database. *ACM Trans. Comput. Syst.* 31 (3), 8:1–8:22. <https://dl.acm.org/citation.cfm?id=2491245>.
- Corbett, P., Kleiman, S., English, R., 2006. Row-diagonal parity technique for enabling efficient recovery from double failures in a storage array. <https://patents.google.com/patent/US20030126523>.
- Corbett, P.F., Feitelson, D.G., 1996. The Vesta parallel file system. *ACM Trans. Comput. Syst.* 14 (3), 225–264. <http://doi.acm.org/10.1145/233557.233558>.
- Corbett, P.F., Goel, A., 2012. Triple parity technique for enabling efficient recovery from triple failures in a storage array. <https://patents.google.com/patent/US7640484B2/en>.
- Corbett, P.F., English, R., Goel, A., et al., 2004. Row-diagonal parity for double disk failure correction. In: Thekkath, C. (Ed.), *Proceedings of the FAST '04 Conference on File and Storage Technologies*. March 31–April 2, 2004, Grand Hyatt Hotel, San Francisco, California, USA. USENIX, pp. 1–14. <http://www.usenix.org/events/fast04/tech/corbett.html>.
- Cormen, T.H., Leiserson, C.E., Rivest, R.L., Stein, C., 2009. *Introduction to Algorithms*, 3. The MIT Press. ISBN 978-0262033848.
- Cornwell, M., 2012. Anatomy of a solid-state drive. *Commun. ACM* 55 (12), 59–63. <https://doi.org/10.1145/2380656.2380672>.
- Cortada, J.W., 2019a. IBM: The Rise and Fall and Reinvention of a Global Icon. The MIT Press. ISBN 978-0262039444.
- Cortada, J.W., 2019b. Building the system/360 mainframe nearly destroyed IBM. *IEEE Spectr.* <https://spectrum.ieee.org/tech-history/silicon-revolution/building-the-system360-mainframe-nearly-destroyed-ibm>.
- Cortes, T., Labarta, J., 2001. Extending heterogeneity to RAID level 5. In: Park, Y. (Ed.), *Proceedings of the General Track: 2001 USENIX Annual Technical Conference*. Boston, Massachusetts, USA, June 25–30, 2001. USENIX, pp. 119–132. <http://www.usenix.org/publications/library/proceedings/usenix01/cortes.html>.
- Courtright II, W.V., Gibson, G.A., Holland, M., Zelenka, J., 1996. Raidframe: rapid prototyping for disk arrays. In: Reed, D.A., Gaither, B.D. (Eds.), *Proceedings of the 1996 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer System*. Philadelphia, Pennsylvania, USA, May 23–26, 1996. ACM, pp. 268–269.
- Courtright II, W.V., Gibson, G., Holland, M., Reilly, L.N., Zelenka, J., 1997. Raidframe: a rapid prototyping tool for rraid systems. Technical Report CMU-CS-97-142. Parallel Data Laboratory, Carnegie-Melon University.
- Cragon, H.G., 1991. *Branch Strategy Taxonomy and Performance Models*. Monograph. IEEE Computer Society Press. ISBN 978-0818691119.
- Cragon, H.G., 1996. *Memory Systems and Pipelined Processors*. Computer Science. Jones and Bartlett. ISBN 978-0867204742.
- Cray Research, 1976. *Cray 1 Computer System*. Cray Research Corp., Bloomington, MI. 1, No. 2240004.
- Cringeley, R.X., 2014. *The Decline and Fall of IBM: End of an American Icon?* NeRD TV LLC. ISBN 978-0-9904444-2-8.
- Crotty, A., Galakatos, A., Dursun, K., Kraska, T., Çetintemel, U., Zdonik, S.B., 2015. Tupeware: “big” data, big analytics, small clusters. In: Kossmann, D., Ailamaki, A., Stonebraker, M. (Eds.), *CIDR 2015, Seventh Biennial Conference on Innovative Data Systems Research*, Online Proceedings. Asilomar, CA, USA, January 4–7, 2015. [www.cidrdb.org](http://cidrdb.org/cidr2015/Papers/CIDR15_Paper23u.pdf). http://cidrdb.org/cidr2015/Papers/CIDR15_Paper23u.pdf.
- Cui, B., Ooi, B.C., Su, J., Tan, K., 2005. Indexing high-dimensional data for efficient in-memory similarity search. *IEEE Trans. Knowl. Data Eng.* 17 (3), 339–353. <https://doi.org/10.1109/TKDE.2005.46>.
- Culler, D.E., Singh, J.P., Gupta, A., 1999. *Parallel Computer Architecture – A Hardware/Software Approach*. Morgan Kaufmann. ISBN 978-1-55860-343-1.
- Dageville, B., Cruanes, T., Zukowski, M., Antonov, V., Avanes, A., et al., 2016. The snowflake elastic data warehouse. In: Özcan, F., Koutrika, G., Madden, S. (Eds.), *Proceedings of the 2016 International Conference on Management of Data, SIGMOD Conference 2016*. San Francisco, CA, USA, June 26–July 01, 2016. ACM, pp. 215–226.
- Dally, W.J., Towles, B., 2004. *Principles and Practices of Interconnection Networks*. Morgan Kaufmann Series in Computer Architecture and Design Series. Elsevier. ISBN 978-0122007514.
- Dan, A., 1992. *Performance Analysis of Data Sharing Environments*. ACM distinguished dissertations. MIT Press. ISBN 978-0-262-04132-4.
- Dan, A., Towsley, D.F., 1990. An approximate analysis of the LRU and FIFO buffer replacement schemes. In: Nutt, G.J. (Ed.), *Proceedings of the 1990 ACM SIGMETRICS Conference on Measurement and Modeling of Computer Systems*. Boulder, Colorado, USA, May 22–25, 1990. ACM, pp. 143–152.
- Dan, A., Yu, P.S., Chung, J., 1993. Database access characterization for buffer hit prediction. In: *Proceedings of the Ninth International Conference on Data Engineering*. Vienna, Austria, April 19–23, 1993. IEEE Computer Society, pp. 134–143.
- Danowitz, A., Kelley, K., Mao, J., Stevenson, J.P., Horowitz, M., 2012a. CPU DB: recording microprocessor history. *ACM Queue* 10 (4), 10. <https://doi.org/10.1145/2181796.2181798>.
- Danowitz, A., Kelley, K., Mao, J., Stevenson, J.P., Horowitz, M., 2012b. CPU DB: recording microprocessor history. *Commun. ACM* 55 (4), 55–63. <https://doi.org/10.1145/2133806.2133822>.

- Darema, F., 2001. The SPMD model: past, present and future. In: Cotronis, Y., Dongarra, J.J. (Eds.), Recent Advances in Parallel Virtual Machine and Message Passing Interface, 8th European PVM/MPI Users' Group Meeting, Proceedings. Santorini/Thera, Greece, September 23–26, 2001. In: Lecture Notes in Computer Science, vol. 2131. Springer, p. 1.
- Das, S., Agrawal, D., Abbadi, A.E., 2010. G-store: a scalable data store for transactional multi key access in the cloud. In: Hellerstein, J.M., Chaudhuri, S., Rosenblum, M. (Eds.), Proceedings of the 1st ACM Symposium on Cloud Computing, SoCC 2010. Indianapolis, Indiana, USA, June 10–11, 2010. ACM, pp. 163–174.
- Dashti, A., Kim, S.H., Shahabi, C., Zimmermann, R., 2003. Streaming Media Server Design. Prentice-Hall, Upper Saddle River Road. ISBN 978-0130670380.
- Dasu, T., Johnson, T., 2003. Exploratory Data Mining and Data Cleaning, 1. Wiley-Interscience, New York, NY. ISBN 978-0471268512.
- Date, C.J., 2003. An Introduction to Database Systems, 8. Addison-Wesley-Longman. ISBN 0321197844.
- David, H.A., Nagaraja, H.N., 2003. Order Statistics, 3. Wiley-Interscience, New York, NY. ISBN 978-0471389262.
- Dayal, U., Blaustein, B.T., Buchmann, A.P., Chakravarthy, U.S., Hsu, M., Ledin, R., McCarthy, D.R., Rosenthal, A., Sarin, S.K., Carey, M.J., Livny, M., Jauhari, R., 1988. The hipac project: combining active databases and timing constraints. SIGMOD Rec. 17 (1), 51–70. <https://doi.org/10.1145/44203.44208>.
- de S. Matos, R., Dantas, J., Araujo, J., Trivedi, K.S., Maciel, P.R.M., 2017. Redundant eucalyptus private clouds: availability modeling and sensitivity analysis. J. Grid Comput. 15 (1), 1–22. <https://doi.org/10.1007/s10723-016-9381-z>.
- de Souza e Silva, E., Gail, H.R., 2000. Transient solutions for Markov chains. Comput. Probab. 24.
- Dean, J., Barroso, L.A., 2013. The tail at scale. Commun. ACM 56 (2), 74–80. <http://doi.acm.org/10.1145/2408776.2408794>.
- Dean, J., Ghemawat, S., 2004. Mapreduce: simplified data processing on large clusters. In: Brewer, E.A., Chen, P. (Eds.), 6th Symposium on Operating System Design and Implementation (OSDI 2004). San Francisco, California, USA, December 6–8, 2004. USENIX Association, pp. 137–150. <http://www.usenix.org/events/osdi04/tech/dean.html>.
- DeBrabant, J., Arulraj, J., Pavlo, A., Stonebraker, M., Zdonik, S.B., Dullloor, S., 2014. A prolegomenon on OLTP database systems for non-volatile memory. In: Bordawekar, R., Lahiri, T., Gedik, B., Lang, C.A. (Eds.), International Workshop on Accelerating Data Management Systems Using Modern Processor and Storage Architectures – ADMS 2014. Hangzhou, China, September 1, 2014, pp. 57–63. http://www.adms-conf.org/2014/adms14_debrabant.pdf.
- DeCandia, G., Hastorun, D., Jampani, M., Kakulapati, G., Lakshman, A., Pilchin, A., Sivasubramanian, S., Vosshall, P., Vogels, W., 2007. Dynamo: Amazon's highly available key-value store. In: Bressoud, T.C., Kaashoek, M.F. (Eds.), Proceedings of the 21st ACM Symposium on Operating Systems Principles 2007, SOSP 2007. Stevenson, Washington, USA, October 14–17, 2007. ACM, pp. 205–220.
- Demurjian, S.A., Hsiao, D.K., Menon, J., 1986. A multi-backend database system for performance gains, capacity growth and hardware upgrade. In: Proceedings of the Second International Conference on Data Engineering. February 5–7, 1986, Los Angeles, California, USA. IEEE Computer Society, pp. 542–554.
- Deng, J., Dong, W., Socher, R., Li, L., Li, K., Li, F., 2009. Imagenet: a large-scale hierarchical image database. In: 2009 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR 2009). 20–25 June 2009, Miami, Florida, USA. IEEE Computer Society, pp. 248–255.
- Deng, Y., 2011. What is the future of disk drives, death or rebirth? ACM Comput. Surv. 43 (3), 23:1–23:27. <http://doi.acm.org/10.1145/1922649.1922660>.
- Denning, P.J., 1967. Effects of scheduling on file memory operations. In: American Federation of Information Processing Societies: Proceedings of the AFIPS '67 Spring Joint Computer Conference. Atlantic City, New Jersey, USA, April 18–20, 1967. In: AFIPS Conference Proceedings, vol. 30. AFIPS/ACM/Thomson Book Company, Washington D.C., pp. 9–21.
- Denning, P.J., 1968a. Thrashing: its causes and prevention. In: American Federation of Information Processing Societies: Proceedings of the AFIPS '68 Fall Joint Computer Conference, Part I. December 9–11, 1968, San Francisco, California, USA. In: AFIPS Conference Proceedings, vol. 33. AFIPS/ACM/Thomson Book Company, Washington D.C., pp. 915–922.
- Denning, P.J., 1968b. The working set model for program behaviour. Commun. ACM 11 (5), 323–333. <https://doi.org/10.1145/363095.363141>.
- Denning, P.J., 1970. Virtual memory. ACM Comput. Surv. 2 (3), 153–189. <http://doi.acm.org/10.1145/356571.356573>.
- Deschamps, J., Sutter, G., Cantó, E., 2012. Guide to FPGA Implementation of Arithmetic Functions. Lecture Notes in Electrical Engineering, vol. 149. Springer. ISBN 978-94-007-2986-5.
- Desnoyers, P., 2014. Analytic models of SSD write performance. ACM Trans. Storage 10 (2), 8:1–8:25. <https://doi.org/10.1145/2577384>.
- Detlefs, D., Dosser, A., Zorn, B.G., 1994. Memory allocation costs in large C and C++ programs. Softw. Pract. Exp. 24 (6), 527–542. <https://doi.org/10.1002/spe.4380240602>.
- Devlin, B.A., Murphy, P.T., 1988. An architecture for a business and information system. IBM Syst. J. 27 (1), 60–80. <https://doi.org/10.1147/sj.271.0060>.
- DeWitt, D.J., 1979. DIRECT – a multiprocessor organization for supporting relational database management systems. IEEE Trans. Comput. 28 (6), 395–406. <https://doi.org/10.1109/TC.1979.1675379>.
- DeWitt, David J., Katz, Randy H., Olken, Frank, Shapiro, Leonard D., Stonebraker, Michael, Wood, David A., 1984. Implementation techniques for main memory database systems. In: Yormark, Beatrice (Ed.), SIGMOD'84, Proceedings of Annual Meeting. Boston, Massachusetts, USA, June 18–21, 1984. ACM Press, pp. 1–8.
- DeWitt, D.J., Boral, H. (Eds.), 1985. Database Machines, Fourth International Workshop. Grand Bahama Island, March 1985. Springer. ISBN 3-540-96200-X.
- DeWitt, D.J., Gray, J., 1990. Parallel database systems: the future of database processing or a passing fad? SIGMOD Rec. 19 (4), 104–112. <http://doi.acm.org/10.1145/122058.122071>.
- DeWitt, D.J., Levine, C., 2008. Not just correct, but correct and fast: a look at one of Jim Gray's contributions to database system performance. SIGMOD Rec. 37 (2), 45–49. <https://doi.org/10.1145/1379387.1379403>.

- DeWitt, D.J., Ghandeharizadeh, S., Schneider, D.A., 1988. A performance analysis of the gamma database machine. In: Boral, H., Larson, P. (Eds.), *Proceedings of the 1988 ACM SIGMOD International Conference on Management of Data*. Chicago, Illinois, USA, June 1–3, 1988. ACM Press, pp. 350–360.
- DeWitt, D.J., Ghandeharizadeh, S., Schneider, D.A., Bricker, A., Hsiao, H., Rasmussen, R., 1990. The Gamma database machine project. *IEEE Trans. Knowl. Data Eng.* 2 (1), 44–62. <https://doi.org/10.1109/69.50905>.
- Dholakia, A., Eleftheriou, E., Hu, X., Iliadis, I., Menon, J., Rao, K.K., 2008. A new intra-disk redundancy scheme for high-reliability RAID storage systems in the presence of unrecoverable errors. *ACM Trans. Storage* 4 (1), 1:1–1:42. <http://doi.acm.org/10.1145/1353452.1353453>.
- Diaconu, C., Freedman, C., Ismert, E., Larson, P., Mittal, P., Stonecipher, R., Verma, N., Zwilling, M., 2013. Hekaton: SQL server's memory-optimized OLTP engine. In: Ross, K.A., Srivastava, D., Papadias, D. (Eds.), *Proceedings of the ACM SIGMOD International Conference on Management of Data, SIGMOD 2013*. New York, NY, USA, June 22–27, 2013. ACM, pp. 1243–1254.
- Dias, D.M., Goyal, A., Parr, F.N., 1992. An intelligent page store for concurrent transaction and query processing. In: Yu, P.S. (Ed.), *RIDE-TQP '92, Second International Workshop on Research Issues on Data Engineering: Transaction and Query Processing*. Tempe, Arizona, USA, February 2–3, 1992. IEEE Computer Society, pp. 12–19.
- Dieny, B., Goldfarb, R.B., Lee, K.-J. (Eds.), 2017. *Introduction to Magnetic Random Access Memory*. Wiley, IEEE. ISBN 9781439807453.
- Dimakis, A.G., Godfrey, B., Wainwright, M.J., Ramchandran, K., 2007. Network coding for distributed storage systems. In: *INFOCOM 2007. 26th IEEE International Conference on Computer Communications, Joint Conference of the IEEE Computer and Communications Societies*. 6–12 May 2007, Anchorage, Alaska, USA. IEEE, pp. 2000–2008.
- Dimakis, A.G., Godfrey, B., Wu, Y., Wainwright, M.J., Ramchandran, K., 2010. Network coding for distributed storage systems. *IEEE Trans. Inf. Theory* 56 (9), 4539–4551. <https://doi.org/10.1109/TIT.2010.2054295>.
- Dimakis, A.G., Ramchandran, K., Wu, Y., Suh, C., 2011. A survey on network codes for distributed storage. *Proc. IEEE* 99 (3), 476–489. <https://doi.org/10.1109/JPROC.2010.2096170>.
- Dimitrijevic, Z., Rangaswami, R., Chang, E.Y., 2005. Systems support for preemptive disk scheduling. *IEEE Trans. Comput.* 54 (10), 1314–1326. <https://doi.org/10.1109/TC.2005.170>.
- Dimopoulou, M., Antonini, M., Barbry, P., Appuswamy, R., 2020. Storing digital data into DNA: a comparative study of quaternary code construction. In: *2020 IEEE International Conference on Acoustics, Speech and Signal Processing, ICASSP 2020, Barcelona, Spain, May 4–8, 2020*. IEEE, pp. 4332–4336.
- Ding, X., Jiang, S., Chen, F., Davis, K., Zhang, X., 2007. Diskseen: exploiting disk layout and access history to enhance I/O prefetch. In: Chase, J., Seshan, S. (Eds.), *Proceedings of the 2007 USENIX Annual Technical Conference*. Santa Clara, CA, USA, June 17–22, 2007. USENIX, pp. 261–274. <http://www.usenix.org/events/usenix07/tech/ding.html>.
- Ditzel, D.R., Patterson, D.A., 1980. Retrospective on high-level language computer architecture. In: Lenfant, J., Borgerson, B.R., Atkins, D.E., et al. (Eds.), *Proceedings of the 7th Annual Symposium on Computer Architecture*. La Baule, France, May 6–8, 1980. ACM, pp. 97–104.
- Dixon, J., 2015. *Monitoring with Graphite*. O'Reilly Media. ISBN 978-1491916438.
- Doshi, B.T., 1986. Queueing systems with vacations – a survey. *Queueing Syst.* 1 (1), 29–66. <https://doi.org/10.1007/BF01149327>.
- Douglis, F., 1993. The compression cache: using on-line compression to extend physical memory. In: *Proceedings of the Usenix Winter 1993 Technical Conference*. San Diego, California, USA, January 1993. USENIX Association, pp. 519–529.
- Douglis, F., Marsh, B., 1993. Low-power disk management for mobile computers. Technical Report MITL-TR-53-93. Matsushita Information Technology Laboratory (MITL), Princeton, NJ.
- Douglis, F., Krishnan, P., Marsh, B., 1994. Thwarting the power-hungry disk. In: *USENIX Winter 1994 Technical Conference, Conference Proceedings*. San Francisco, California, USA, January 17–21, 1994. USENIX Association, pp. 292–306. <https://www.usenix.org/conference/usenix-winter-1994-technical-conference/thwarting-power-hungry-disk>.
- Douglis, F., Krishnan, P., Bershad, B.N., 1995. Adaptive disk spin-down policies for mobile computers. In: *Proceedings of the 2nd Symposium on Mobile and Location-Independent Computing, MLICS'95*. Ann Arbor, MI, USA, 10–11 April 1995. USENIX, pp. 121–137. <http://www.usenix.org/publications/library/proceedings/mob95/douglis.html>.
- Dowdy, L.W., Foster, D.V., 1982. Comparative models of the file assignment problem. *ACM Comput. Surv.* 14 (2), 287–313. <http://doi.acm.org/10.1145/356876.356883>.
- Dowdy, L.W., Gordon, K.D., 1987. Performance sensitivity with respect to the multiprogramming level. *Comput. Syst. Sci. Eng.* 2 (2), 92–98.
- Dowdy, L.W., Eager, D.L., Gordon, K.D., Saxton, L.V., 1984. Throughput concavity and response time convexity. *Inf. Process. Lett.* 19 (4), 209–212. [https://doi.org/10.1016/0020-0190\(84\)90087-5](https://doi.org/10.1016/0020-0190(84)90087-5).
- Doyle, J.C., Francis, B.A., Tannenbaum, A.R., 2009. *Feedback Control Theory*. Books on Electrical Engineering. Dover. ISBN 978-0486469331.
- Dreyfus, S.E., Law, A.M., 1977. *The Art and Theory of Dynamic Programming*. Mathematical Science and Engineering. Academic Press. ISBN 978-0122218606.
- Duato, J., Yalamanchili, S., Ni, L., 2003. *Interconnection Networks: An Engineering Approach*. Morgan Kaufman Publishers, San Francisco, CA. ISBN 1-55860-852-4. Revised printing.
- Dubnicki, C., Gryz, L., Heldt, L., et al., 2009. Hydrastor: a scalable secondary storage. In: Seltzer, M.I., Wheeler, R. (Eds.), *7th USENIX Conference on File and Storage Technologies, Proceedings*. February 24–27, 2009, San Francisco, CA, USA. USENIX, pp. 197–210. http://www.usenix.org/events/fast09/tech/full_papers/dubnicki/dubnicki.pdf.
- Dulong, C., Krishnaiyer, R., Kulkarni, D., et al., 1999. An overview of the Intel ia-64 compiler. *Intel Technol. J.*
- Dulong, C., Shrivastav, P., Refah, A., 2001. The making of a compiler for the Intel® Itanium processor. *Intel Technol. J.*

- Dunn, M., Feldman, T., 2014. Shingled magnetic recording: models, standardization, and applications. Technical report. Storage Networking Industry Association. https://www.snia.org/sites/default/files/Dunn-Feldman_SNIA_Tutorial_Shingled_Magnetic_Recording-r7_Final.pdf.
- Dunning, T., Friedman, E., 2014. Time Series Databases: New Ways to Store and Access Data. O'Reilly Media. ISBN 978-1-49191-472-4.
- Durstenfeld, R., 1964. Algorithm 235: random permutation. *Commun. ACM* 7 (7), 420. <http://doi.acm.org/10.1145/364520.364540>.
- Ebbers, M., Kettner, J., O'Brian, W., Ogden, B., 2011. Introduction to the new mainframe z/os basics. Technical report. IBM Corp. <https://www.redbooks.ibm.com/redbooks/pdfs/sg246366.pdf>.
- Edward, J., Coffman, G., Trumball, C.J.M., 1973. A comparison of two disk scanning policies. *Inf. Process. Lett.* 2, 15–17. <https://doi.org/10.1016/j.ipl.2018.05.005>.
- Coffman Jr., Edward G., Klimko, L.A., Ryan, B., 1972. Analysis of scanning policies for reducing disk seek times. *SIAM J. Comput.* 1 (3), 269–279. <https://doi.org/10.1137/0201018>.
- Effelsberg, W., Härder, T., 1984. Principles of database buffer management. *ACM Trans. Database Syst.* 9 (4), 560–595. <https://doi.org/10.1145/1994.2022>.
- Eibeck, P.A., Cohen, D.J., 1988. Modeling thermal characteristics of a fixed disk drive. *IEEE Trans. Components Hybrids Manuf. Technol.* 11 (4), 566–5707. <https://doi.org/10.1109/TC.1980.1675475>.
- Ekman, M., Stenström, P., 2005. A cost-effective main memory organization for future servers. In: 19th International Parallel and Distributed Processing Symposium (IPDPS 2005), CD-ROM/Abstracts Proceedings. 4–8 April 2005, Denver, CO, USA. IEEE Computer Society.
- El-Shimi, A., Kalach, R., Kumar, A., Ottean, A., Li, J., Sengupta, S., 2012. Primary data deduplication – large scale study and system design. In: Heiser, G., Hsieh, W.C. (Eds.), *USENIX Annual Technical Conference*. Boston, MA, USA, June 13–15, 2012. USENIX Association, pp. 285–296. [https://www.usenix.org/conference/atc12/technical-sessions/presentation/el-shimi](http://www.usenix.org/conference/atc12/technical-sessions/presentation/el-shimi).
- Elerath, J.G., Pecht, M.G., 2007. Enhanced reliability modeling of RAID storage systems. In: The 37th Annual IEEE/IFIP International Conference on Dependable Systems and Networks, DSN 2007, Proceedings. 25–28 June 2007, Edinburgh, UK. IEEE Computer Society, pp. 175–184.
- Elnozahy, E.N., Plank, J.S., 2004. Checkpointing for peta-scale systems: a look into the future of practical rollback-recovery. *IEEE Trans. Dependable Secure Comput.* 1 (2), 97–108. <https://doi.org/10.1109/TDSC.2004.15>.
- Elnozahy, E.N., Alvisi, L., Wang, Y., Johnson, D.B., 2002a. A survey of rollback-recovery protocols in message-passing systems. *ACM Comput. Surv.* 34 (3), 375–408. <https://doi.org/10.1145/568522.568525>.
- Elnozahy, E.N., Kistler, M., Rajamony, R., 2002b. Energy-efficient server clusters. In: Falsafi, B., Vijaykumar, T.N. (Eds.), *Power-Aware Computer Systems, Second International Workshop, PACS 2002, Revised Papers*. Cambridge, MA, USA, February 2, 2002. In: *Lecture Notes in Computer Science*, vol. 2325. Springer, pp. 179–196.
- EMC-Data Archive Corp., 2018. What is EMC Centera? https://course.ccs.neu.edu/csg389/2005/readings/centera_guide.pdf.
- Endo, P.T., et al., 2016. High availability in clouds: systematic review and research challenges. *J. Cloud Comput.* 5, 16. <https://doi.org/10.1186/s13677-016-0066-8>.
- English, R., Stepanov, A., 1992. Loge: a self-organizing disk controller. In: *Winter USENIX*.
- Ercegovac, M.D., Lang, T., 2003. *Digital Arithmetic*. Computer Architecture and Design. Morgan-Kaufman, Burlington, MA.
- Eriksson, H., 1994. Mbone: the multicast backbone. *Commun. ACM* 37 (8), 54–60. <https://doi.org/10.1145/179606.179627>.
- Eshghi, K., Tang, H.K., 2005. A framework for analyzing and improving content-based chunking algorithms. Technical Report HPL-2005-30(R.1). Hewlett Packard Lab, Palo Alto, CA.
- Esmailzadeh, H., Blem, E.R., Amant, R.S., Sankaralingam, K., Burger, D., 2011. Dark silicon and the end of multicore scaling. In: Iyer, R., Yang, Q., González, A. (Eds.), *38th International Symposium on Computer Architecture (ISCA 2011)*. San Jose, CA, USA, June 4–8, 2011. ACM, pp. 365–376.
- Esmet, J., Bender, M.A., Farach-Colton, M., Kuszmaul, B.C., 2012. The tokufs streaming file system. In: Rangaswami, R. (Ed.), *4th USENIX Workshop on Hot Topics in Storage and File Systems, HotStorage'12*. Boston, MA, USA, June 13–14, 2012. USENIX Association. <https://www.usenix.org/conference/hotstorage12/workshop-program/presentation/esmet>.
- Estrada-Galiñanes, V., Pâris, J., Felber, P., 2016. Simple data entanglement layouts with high reliability. In: *35th IEEE International Performance Computing and Communications Conference, IPCCC 2016*. Las Vegas, NV, USA, December 9–11, 2016. IEEE Computer Society, pp. 1–8.
- Etzion, O., Niblett, P., 2011. *Event Processing in Action*. Manning. ISBN 978-1935182214.
- Faerber, F., Kemper, A., Larson, P., Levandoski, J.J., Neumann, T., Pavlo, A., 2017. Main memory database systems. *Found. Trends Databases* 8 (1–2), 1–130. <https://doi.org/10.1561/19000000058>.
- Falkoff, A.D., Iverson, K.E., Jr., E.H.S., 1964. A formal description of SYSTEM/360. *IBM Syst. J.* 3 (2), 198–262. <https://doi.org/10.1147/sj.32.0198>.
- Faloutsos, C., 1996. *Searching Multimedia Databases by Content*. Advances in Database Systems. Kluwer Academic Publishers/Springer-Verlag. ISBN 978-0792397779.
- Faloutsos, C., Flunkert, V., Gasthaus, J., Januschowski, T., Wang, Y., 2020. Forecasting big time series: theory and practice. In: Seghrouchni, A.E.F., Sukthankar, G., Liu, T., van Steen, M. (Eds.), *Companion of The 2020 Web Conference 2020*. Taipei, Taiwan, April 20–24, 2020. ACM/IW3C2, pp. 320–321.
- Fan, B., Tantisiroj, W., Xiao, L., Gibson, G., 2009. Diskreduce: raid for data-intensive scalable computing. Technical Report CMU-PDL-09-112. Parallel Data Laboratory, Carnegie-Mellon University, Pittsburgh, PA. Proc. 4th Petascale Data Storage Workshop, in conjunction with Supercomputing '09, November 2009.
- Fan, X., Weber, W., Barroso, L.A., 2007. Power provisioning for a warehouse-sized computer. In: Tullsen, D.M., Calder, B. (Eds.), *34th International Symposium on Computer Architecture (ISCA 2007)*. June 9–13, 2007, San Diego, California, USA. ACM, pp. 13–23.
- Fan, Y., Lai, W., Meng, X., 2014. Optimizing database operators by exploiting internal parallelism of solid state drives. *IEEE Data Eng. Bull.* 37 (2), 12–18. <http://sites.computer.org/debull/A14june/p12.pdf>.

- Fang, J., Mulder, Y.T.B., Hidders, J., Lee, J., Hofstee, H.P., 2020. In-memory database acceleration on FPGAs: a survey. *VLDB J.* 29 (1), 33–59. <https://doi.org/10.1007/s00778-019-00581-w>.
- Feitelson, D.G., 2015. *Workload Modeling for Computer Systems Performance Evaluation*, 1. Cambridge University Press. ISBN 978-1107078239.
- Feng, G.L., Deng, R.H., Bao, F., Shen, J., 2005a. New efficient MDS array codes for RAID part I: Reed-Solomon-like codes for tolerating three disk failures. *IEEE Trans. Comput.* 54 (9), 1071–1080. <https://doi.org/10.1109/TC.2005.150>.
- Feng, G.L., Deng, R.H., Bao, F., Shen, J., 2005b. New efficient MDS array codes for RAID part II: Rabin-like codes for tolerating multiple (greater than or equal to 4) disk failures. *IEEE Trans. Comput.* 54 (12), 1473–1483. <https://doi.org/10.1109/TC.2005.200>.
- Fernández, E.B., Lang, T., Wood, C., 1978. Effect of replacement algorithms on a paged buffer database system. *IBM J. Res. Dev.* 22 (2), 185–196. <https://doi.org/10.1147/rd.222.0185>.
- Ferrari, D., 1978. *Computer System Performance Evaluation*. Prentice-Hall, Englewood Cliffs, NJ. ISBN 978-0131651265.
- Feurer, M., Klein, A., Eggenberger, K., Springenberg, J.T., Blum, M., Hutter, F., 2015. Efficient and robust automated machine learning. In: Cortes, C., Lawrence, N.D., Lee, D.D., Sugiyama, M., Garnett, R. (Eds.), *Advances in Neural Information Processing Systems 28: Annual Conference on Neural Information Processing Systems 2015*. December 7–12, 2015, Montreal, Quebec, Canada, pp. 2962–2970. <http://papers.nips.cc/paper/5872-efficient-and-robust-automated-machine-learning>.
- Fevgas, A., Akritidis, L., Bozanis, P., Manolopoulos, Y., 2020. Indexing in flash storage devices: a survey on challenges, current approaches, and future trends. *VLDB J.* 29 (1), 273–311. <https://doi.org/10.1007/s00778-019-00559-8>.
- Filho, R.F.S., Traina, A.J.M., Jr., C.T., Faloutsos, C., 2001. Similarity search without tears: the OMNI family of all-purpose access methods. In: Georgakopoulos, D., Buchmann, A. (Eds.), *Proceedings of the 17th International Conference on Data Engineering*. April 2–6, 2001, Heidelberg, Germany. IEEE Computer Society, pp. 623–630.
- Finkel, R.A., Bentley, J.L., 1974. Quad trees: a data structure for retrieval on composite keys. *Acta Inform.* 4, 1–9. <https://doi.org/10.1007/BF00288933>.
- Finkelstein, S.J., Schkolnick, M., Tiberio, P., 1988. Physical database design for relational databases. *ACM Trans. Database Syst.* 13 (1), 91–128. <http://doi.acm.org/10.1145/42201.42205>.
- Flatto, L., Hahn, S., 1984. Two parallel queues created by arrivals with two demands: I. *SIAM J. Appl. Math.*, 1041–1053.
- Fleiner, C., Garner, R.B., Hafner, J.L., Rao, K.K., Kenchammana-Hosekote, D.R., Wilcke, W.W., Glider, J.S., 2006. Reliability of modular mesh-connected intelligent storage brick systems. *IBM J. Res. Dev.* 50 (2–3), 199–208. <https://doi.org/10.1147/rd.502.0199>.
- Flickner, M., Sawhney, H.S., Ashley, J., et al., 1995. Query by image and video content: the QBIC system. *IEEE Computer* 28 (9), 23–32. <https://doi.org/10.1109/2.410146>.
- Flynn, M.J., 1972. Some computer organizations and their effectiveness. *IEEE Trans. Comput.* 21 (9), 948–960. <https://doi.org/10.1109/TC.1972.5009071>.
- Flynn, M.J., 1998. Computer engineering 30 years after the IBM model 91. *IEEE Computer* 31 (4), 27–31. <https://doi.org/10.1109/2.666839>.
- Fontana, R.E., 2018. Ten year storage technology landscape for hdd, nand, and tape. In: *IEEE 34th Symposium on Mass Storage Systems and Technologies, MSST 2018*. May 14–16, 2018, Santa Clara University, CA, USA. IEEE, pp. 1–8.
- Ford, D., Labelle, F., Popovici, F.I., Stokely, M., Truong, V., Barroso, L., Grimes, C., Quinlan, S., 2010. Availability in globally distributed storage systems. In: Arpaci-Dusseau, R.H., Chen, B. (Eds.), *9th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2010, Proceedings*. October 4–6, 2010, Vancouver, BC, Canada. USENIX Association, pp. 61–74. http://www.usenix.org/events/osdi10/tech/full_papers/Ford.pdf.
- Ford, D.A., Morris, R.J.T., Bell, A.E., 1998. Redundant arrays of independent libraries (RAIL): the starfish tertiary storage system. *Parallel Comput.* 24 (1), 45–64. [https://doi.org/10.1016/S0167-8191\(97\)00116-6](https://doi.org/10.1016/S0167-8191(97)00116-6).
- Foster, I.T., Kesselman, C. (Eds.), 2004. *The Grid 2 – Blueprint for a New Computing Infrastructure*, 2. Computer Architecture and Design. Morgan Kaufmann. ISBN 978-1558609334.
- Franaszek, P.A., Considine, J.P., 1979. Reduction of storage fragmentation on direct access devices. *IBM J. Res. Dev.* 23 (2), 140–148. <https://doi.org/10.1147/rd.232.0140>.
- Franaszek, P.A., Robinson, J.T., 1985. Limitations of concurrency in transaction processing. *ACM Trans. Database Syst.* 10 (1), 1–28. <https://doi.org/10.1145/3148.3160>.
- Franaszek, P.A., Robinson, J.T., 1997. On variable scope of parity protection in disk arrays. *IEEE Trans. Comput.* 46 (2), 234–240. <https://doi.org/10.1109/12.565608>.
- Franaszek, P.A., Robinson, J.T., 1998. Design and analysis of internal organizations for compressed random access memories. Technical Report RC 21146. IBM T.J. Watson Research Center.
- Franaszek, P.A., Robinson, J.T., 2001. On internal organization in compressed random-access memories. *IBM J. Res. Dev.* 45 (2), 259–270. <https://doi.org/10.1147/rd.452.0259>.
- Franaszek, P.A., Robinson, J.T., Thomasian, A., 1992. Concurrency control for high contention environments. *ACM Trans. Database Syst.* 17 (2), 304–345. <http://doi.acm.org/10.1145/128903.128906>.
- Franaszek, P.A., Robinson, J.T., Thomasian, A., 1993. Centralized and distributed wait depth limited concurrency control methods and apparatus.
- Franaszek, P.A., Robinson, J.T., Thomasian, A., 1996. Raid level 5 with free blocks/parity cache. <https://patents.google.com/patent/US5522032/en>. US Patent 5522032.
- Frazelle, J., 2020a. The life of a data byte. *Commun. ACM* 63 (12), 38–45. <https://doi.org/10.1145/3410625>.
- Frazelle, J., 2020b. Chipping away at Moore’s law. *ACM Queue* 18 (1), 5–15. <https://doi.org/10.1145/3387945.3388515>.
- Freeman, P.A., Adrion, W.R., Aspray, W., 2019. *Computing and the National Science Foundation 1950–2016: Building a Foundation for Modern Computing*. ACM Books. ISBN 978-1450372763.
- Freitas, A., Lavington, S.H., 2000. *Mining Very Large Databases with Parallel Processing*. Kluwer Academic Publishers/Springer, New York, NY. ISBN 978-1-4613-7523-4. Kluwer Academic Publishers was original publisher.

- Freitas, R.F., Wilcke, W.W., 2008. Storage-class memory: the next storage system technology. *IBM J. Res. Dev.* 52 (4–5), 439–448. <https://doi.org/10.1147/rd.524.0439>.
- Friedman, M.B., 1995. The performance and tuning of a storagetek iceberg raid6 disk subsystem. *Trans. Comput. Meas. Group*, 77–88.
- Fu, G., Thomasian, A., Han, C., Ng, S.W., 2004a. Rebuild strategies for redundant disk arrays. In: Kobler, B., Hariharan, P.C. (Eds.), 21st IEEE Conference on Mass Storage Systems and Technologies/12th NASA Goddard Conference on Mass Storage Systems and Technologies. Greenbelt, Maryland, USA, April 13–16, 2004. IEEE, pp. 223–226. <http://storageconference.org/nasa/conf2004/Papers/MSST2004-25-Fu-a.pdf>.
- Fu, G., Thomasian, A., Han, C., Ng, S.W., 2004b. Rebuild strategies for clustered raid. In: *Proc. Int'l Symp. Performance Evaluation Computer and Telecommunication Systems – SPECTS'04*, pp. 598–607.
- Fu, Y., Shu, J., Shen, Z., Zhang, G., 2016. Reconsidering single disk failure recovery for erasure coded storage systems: optimizing load balancing in stack-level. *IEEE Trans. Parallel Distrib. Syst.* 27 (5), 1457–1469. <https://doi.org/10.1109/TPDS.2015.2442979>.
- Fuja, T.E., Heegard, C., Blaum, M., 1989. Cross parity check convolutional codes. *IEEE Trans. Inf. Theory* 35 (6), 1264–1276. <https://doi.org/10.1109/18.45283>.
- Fujita, H., 2006. Modified low-density MDS array codes. In: *Proceedings 2006 IEEE International Symposium on Information Theory, ISIT 2006*. The Westin Seattle, Seattle, Washington, USA, July 9–14, 2006. IEEE, pp. 2789–2793.
- Fujita, H., Sakaniwa, K., 2007. Modified low-density MDS array codes for tolerating double disk failures in disk arrays. *IEEE Trans. Comput.* 56 (4), 563–566. <https://doi.org/10.1109/TC.2007.1000>.
- Fuller, S.H., 1975. Analysis of Drum and Disk Storage Units. *Lecture Notes in Computer Science*, vol. 31. Springer, New York, NY. ISBN 978-0387071862.
- Fuller, S.H., Baskett, F., 1975. An analysis of drum storage units. *J. ACM* 22 (1), 83–105. <https://doi.org/10.1145/321864.321871>.
- Fuller, S.H., Shaman, P., Lamb, D., Burr, W.E., 1977. Evaluation of computer architectures via test programs. In: *American Federation of Information Processing Societies: 1977 National Computer Conference*. Dallas, Texas, USA, June 13–16, 1977. In: *AFIPS Conference Proceedings*, vol. 46. AFIPS Press, pp. 147–160.
- Funke, F., Kemper, A., Neumann, T., 2012. Compacting transactional data in hybrid OLTP & OLAP databases. *Proc. VLDB Endow.* 5 (11), 1424–1435. <https://doi.org/10.14778/2350229.2350258>. http://vldb.org/pvldb/vol5/p1424_florianfunke_vldb2012.pdf.
- Furino, S., Maio, Y., Yin, J., 1996. *Frames and Resolvable Designs: Uses, Constructions, and Existence*. CRC Press, Boca Raton. ISBN 978-0849339844.
- Gad, E.E., Mateescu, R., Blagojevic, F., Guyot, C., Bandic, Z., 2013. Repair-optimal MDS array codes over GF(2). In: *Proceedings of the 2013 IEEE International Symposium on Information Theory*. Istanbul, Turkey, July 7–12, 2013. IEEE, pp. 887–891.
- Gaede, V., Günther, O., 1998. Multidimensional access methods. *ACM Comput. Surv.* 30 (2), 170–231. <https://doi.org/10.1145/280277.280279>.
- Gafsi, J., Biersack, E., 2000. Modeling and performance comparison of reliability strategies for distributed video servers. *IEEE Trans. Parallel Distrib. Syst.* 11 (4), 412–430. <https://doi.org/10.1109/71.850836>.
- Gal, E., Toledo, S., 2005. Algorithms and data structures for flash memories. *ACM Comput. Surv.* 37 (2), 138–163. <https://doi.org/10.1145/1089733.1089735>.
- Gallagher, R.G., 2003. *Low-Density Parity-Check Codes*. MIT Press, Cambridge, MA. ISBN 978-0262571777. First published 1960.
- Gallo, M.L., Sebastian, A., Eleftheriou, E., 2019. Phase-change memory enables energy-efficient brain-inspired computing. In: *Device Research Conference. DRC 2019*, Ann Arbor, MI, USA, June 23–26, 2019. IEEE, pp. 197–198.
- Gandhi, A., 2013. *Dynamic Server Provisioning for Data Center Power Management*. PhD thesis. Computer Science Dept., Carnegie Mellon University, Pittsburgh, PA. Technical Report CMU-CS-13-110.
- Gandhi, A., Harchol-Balter, M., Raghunathan, R., Kozuch, M.A., 2012. Autoscale: dynamic, robust capacity management for multi-tier data centers. *ACM Trans. Comput. Syst.* 30 (4), 14:1–14:26. <http://doi.acm.org/10.1145/2382553.2382556>.
- Ganger, G., Worthington, B., Patt, Y., 1999. The disksim simulation environment version 2.0, reference manual. <http://www.pdl.cmu.edu/DiskSim/>.
- Ganger, G.R., 1995. Generating representative synthetic workloads: an unsolved problem. In: *21st International Computer Measurement Group Conference, Proceedings*. December 4–8, 1995, Nashville, TN, USA. Computer Measurement Group, pp. 1263–1269.
- Ganger, G.R., Kaashoek, M.F., 1997. Embedded inodes and explicit grouping: exploiting disk bandwidth for small files. In: *USENIX Annual Technical Conference*, pp. 1–17.
- Ganger, G.R., Patt, Y.N., 1993. The process-flow model: examining I/O performance from the system's point of view. In: Owicki, S.S., Muntz, R.R. (Eds.), *Proceedings of the 1993 ACM SIGMETRICS Conference on Measurement and Modeling of Computer Systems*. Santa Clara, California, USA, May 10–14, 1993. ACM, pp. 86–97.
- Ganger, G.R., Worthington, B.L., Hou, R.Y., Patt, Y.N., 1993. Disk subsystem load balancing: disk striping vs. conventional data placement. In: *27th Annual Hawaii International Conference on System Sciences (HICSS-27)*. January 1993, Honolulu, Hawaii. IEEE Computer Society. ISBN 0-8186-5090-7, pp. 40–49.
- Ganger, G.R., Strunk, J.D., Klosterman, A.J., 2003. Self-* storage: brick-based storage with automated administration. Technical Report CMU-CS-03-178. Parallel Data Laboratory, Carnegie-Mellon University, Pittsburgh, PA.
- Gao, M., Pu, J., Yang, X., Horowitz, M., Kozyrakis, C., 2017. TETRIS: scalable and efficient neural network acceleration with 3d memory. In: Chen, Y., Temam, O., Carter, J. (Eds.), *Proceedings of the Twenty-Second International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS 2017*. Xi'an, China, April 8–12, 2017. ACM, pp. 751–764.
- Gao, P.X., Narayan, A., Karandikar, S., Carreira, J., Han, S., Agarwal, R., Ratnasamy, S., Shenker, S., 2016. Network requirements for resource disaggregation. In: Keeton, K., Roscoe, T. (Eds.), *12th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2016*. Savannah, GA, USA, November 2–4, 2016. USENIX Association, pp. 249–264. <https://www.usenix.org/conference/osdi16/technical-sessions/presentation/gao>.

- Gao, R.X., Wang, L., Helu, M., Teti, R., 2020. Big data analytics for smart factories of the future. *CIRP Ann.* 69 (2), 668–692. <http://www.sciencedirect.com/science/article/pii/S0007850620301359>.
- Garcia-Molina, H., 1982. Reliability issues for fully replicated databases. *IEEE Computer* 15 (9), 34–42. <https://doi.org/10.1109/MC.1982.1654135>.
- Garey, M.R., Johnson, D.S., 1979. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W.H. Freeman. ISBN 0-7167-1045-5.
- Garner, H.L., 1959. The residue number system. *IRE Trans. Electron. Comput.* 8 (2), 140–147. <https://doi.org/10.1109/TEC.1959.5219515>.
- Garofalakis, M.N., Gehrke, J., Rastogi, R. (Eds.), 2016. *Data Stream Management – Processing High-Speed Data Streams*. Data-Centric Systems and Applications. Springer. ISBN 978-3-540-28607-3.
- Gass, S.I., 2010. *Linear Programming: Methods and Applications*, 5. Computer Science. Dover Books. ISBN 978-0486432847.
- Gassner, P., Lohman, G.M., Schiefer, K.B., Wang, Y., 1993. Query optimization in the IBM DB2 family. *IEEE Data Eng. Bull.* 16 (4), 4–18. <http://sites.computer.org/debull/93DEC-CD.pdf>.
- Gawlick, D., Kinkade, D., 1985. Varieties of concurrency control in IMS/VS fast path. *IEEE Database Eng. Bull.* 8 (2), 3–10. <http://sites.computer.org/debull/85JUN-CD.pdf>.
- Geist, Robert, Daniel, Stephen, 1987. A continuum of disk scheduling algorithms. *ACM Trans. Comput. Syst.* 5 (1), 77–92. <https://doi.org/10.1145/7351.8929>.
- Geist, R., Reynolds, R.G., Suggs, D., 1994. Minimizing mean seek distance in mirrored disk systems by cylinder remapping. *Perform. Eval.* 20 (1–3), 97–114. [https://doi.org/10.1016/0166-5316\(94\)90008-6](https://doi.org/10.1016/0166-5316(94)90008-6).
- Gemmell, J., Bell, G., Lueder, R., 2006. Mylifebits: a personal database for everything. *Commun. ACM* 49 (1), 88–95. <https://doi.org/10.1145/1107458.1107460>.
- Georgiadis, L., Nikolaou, C., Thomasian, A., 2004. A fair workload allocation policy for heterogeneous systems. *J. Parallel Distrib. Comput.* 64 (4), 507–519. <https://doi.org/10.1016/j.jpdc.2004.03.005>.
- Gerhani, N., 2003. *Bell Labs: Life in the Crown Jewel*. Silicon Press. ISBN 978-0929306278.
- Gerla, M., Kleinrock, L., 1980. Flow control: a comparative survey. *IEEE Trans. Commun.* 28 (4), 553–574. <https://doi.org/10.1109/TCOM.1980.1094691>.
- Ghandeharizadeh, S., Huang, L., Kamel, I., 2003. A cost driven disk scheduling algorithm for multimedia object retrieval. *IEEE Trans. Multimed.* 5 (2), 186–196. <https://doi.org/10.1109/TMM.2003.811623>.
- Ghemawat, S., Gobioff, H., Leung, S., 2003. The Google file system. In: Scott, M.L., Peterson, L.L. (Eds.), *Proceedings of the 19th ACM Symposium on Operating Systems Principles 2003, SOSP 2003*. Bolton Landing, NY, USA, October 19–22, 2003. ACM, pp. 29–43.
- Ghose, S., Hsieh, K., Boroumand, A., Ausavarungrin, R., Mutlu, O., 2018. Enabling the adoption of processing-in-memory: challenges, mechanisms, future research directions. *CoRR*, arXiv:1802.00320.
- Ghose, S., Boroumand, A., Kim, J.S., Gómez-Luna, J., Mutlu, O., 2019. Processing-in-memory: a workload-driven perspective. *IBM J. Res. Dev.* 63 (6), 3:1–3:19. <https://doi.org/10.1147/JRD.2019.2934048>.
- Gibson, G.A., 1992. *Redundant Disk Arrays – Reliable, Parallel Secondary Storage*. ACM distinguished dissertations. MIT Press. ISBN 978-0-262-07142-0.
- Gibson, G.A., Hellerstein, L., Karp, R.M., Katz, R.H., Patterson, D.A., 1989. Failure correction techniques for large disk arrays. In: Emer, J.S., Hennessy, J.L. (Eds.), *ASPLOS-III Proceedings – Third International Conference on Architectural Support for Programming Languages and Operating Systems*. Boston, Massachusetts, USA, April 3–6, 1989. ACM Press, pp. 123–132.
- Gibson, G.A., Stodolsky, D., Chang, F.W., Courtright II, W.V., et al., 1995. The Scotch parallel storage systems. In: *COMPCON '95: Technologies for the Information Superhighway, Digest of Papers*. San Francisco, California, USA, March 5–9, 1995. IEEE Computer Society, pp. 403–410.
- Gifford, D.K., 1979. Weighted voting for replicated data. In: Schroeder, M.D., Jones, A.K. (Eds.), *Proceedings of the Seventh Symposium on Operating System Principles, SOSP 1979*, Asilomar Conference Grounds. Pacific Grove, California, USA, 10–12 December 1979. ACM, pp. 150–162.
- Gladney, H.M., Lorie, R.A., 2005. Trustworthy 100-year digital objects: durable encoding for when it's too late to ask. *ACM Trans. Inf. Syst.* 23 (3), 299–324. <https://doi.org/10.1145/1080343.1080346>.
- Glombiewski, N., Seeger, B., Graefe, G., 2019. Waves of misery after index creation. In: Grust, T., Naumann, F., Böhm, A., Lehner, W., Härder, T., Rahm, E., Heuer, A., Klettke, M., Meyer, H. (Eds.), *Datenbanksysteme für Business, Technologie und Web (BTW 2019)*, 18. Fachtagung des GI-Fachbereichs “Datenbanken und Informationssysteme” (DBIS), Proceedings. 4.-8. März 2019, Rostock, Germany. In: *LNI*, vol. P-289. Gesellschaft für Informatik, Bonn, pp. 77–96.
- Goda, K., Kitsuregawa, M., 2012. The history of storage systems. *Proc. IEEE* 100 (Centennial-Issue), 1433–1440. <https://doi.org/10.1109/JPROC.2012.2189787>.
- Godfrind, A., Pitts, R., Viehmann, H., Kothuri, R., 2015. *Pro Oracle Spatial for Oracle Database 12c*. Apress. ISBN 978-1430263135.
- Goel, A., Corbett, P.F., 2012. RAID triple parity. *Oper. Syst. Rev.* 46 (3), 41–49. <http://doi.acm.org/10.1145/2421648.2421655>.
- Golding, R., Shriver, E., Sullivan, T., Wilkes, J., 1995a. Attribute managed storage. In: *Workshop on Modeling and Specification of I/O*. San Antonio, TX, October 1995.
- Golding, R.A., Bosch II, P., Staelin, C., Sullivan, T., Wilkes, J., 1995b. Idleness is not sloth. In: *USENIX 1995 Technical Conference on UNIX and Advanced Computing Systems, Conference Proceedings*. New Orleans, Louisiana, USA, January 16–20, 1995. USENIX Association, pp. 201–212. <https://www.usenix.org/conference/usenix-1995-technical-conference/idleness-not-sloth>.
- Goldman, N., et al., 2013. Towards practical, high-capacity, low-maintenance information storage in synthesized dna. *Nature* 494 (7935), 77–80.
- Goodfellow, I., Bengio, Y., Courville, A., 2016. *Deep Learning: Adaptive Computation and Machine Learning*. The MIT Press. ISBN 978-0262035613.
- Goodman, R., Sayano, M., 1990. Size limits on phased burst error correcting array codes. *Electron. Lett.* 26.

- Goodman, R.M., McEliece, R.J., Sayano, M., 1993. Phased burst error-correcting array codes. *IEEE Trans. Inf. Theory* 39 (2), 684–693. <https://doi.org/10.1109/18.212304>.
- Goodson, G.R., Wylie, J.J., Ganger, G.R., Reiter, M.K., 2004. Efficient byzantine-tolerant erasure-coded storage. In: 2004 International Conference on Dependable Systems and Networks (DSN 2004), Proceedings. 28 June–1 July 2004, Florence, Italy. IEEE Computer Society, pp. 135–144.
- Gopalan, K., Huang, L., Peng, G., Chiueh, T., Lin, Y., 2006. Statistical admission control using delay distribution measurements. *TOMCCAP* 2 (4), 258–281. <https://doi.org/10.1145/1201730.1201732>.
- Gottlieb, A., Edler, J., Lipkis, J., 1986. Considerations for massively parallel UNIX systems on the NYU ultracomputer and IBM rp3.
- Goyal, A., Shahabuddin, P., Heidelberger, P., Nicola, V.F., Glynn, P.W., 1992. A unified framework for simulating Markovian models of highly dependable systems. *IEEE Trans. Comput.* 41 (1), 36–51. <https://doi.org/10.1109/12.123381>.
- Gracin, S., Jr., J.S., Wilkins, S., von Wolfersdorf, J., Wong, F., 2007. Fibre Channel Protocol for Linux and z/VM on IBM System Z. Technical report. IBM Almaden Research Center, San Jose. www.redbooks.ibm.com.
- Grad, B., 2002. A personal recollection: IBM's unbundling of software and services. *IEEE Ann. Hist. Comput.* 24 (1), 64–71. <https://doi.org/10.1109/85.988583>.
- Graefe, G., 2006. Implementing sorting in database systems. *ACM Comput. Surv.* 38 (3), 10. <https://doi.org/10.1145/1132960.1132964>.
- Graefe, G., 2007. The five-minute rule twenty years later, and how flash memory changes the rules. In: Ailamaki, A., Luo, Q. (Eds.), *Workshop on Data Management on New Hardware, DaMoN 2007*. Beijing, China, June 15, 2007. ACM, p. 6.
- Graefe, G., 2008. The five-minute rule 20 years later: and how flash memory changes the rules. *ACM Queue* 6 (4), 40–52. <https://doi.org/10.1145/1413254.1413264>.
- Graham, R.L., 1969. Bounds on multiprocessing timing anomalies. *SIAM J. Appl. Math.* 17 (2), 416–429.
- Grassmann, W.K., 1991. Finding transient solutions in Markovian event systems through randomization. In: Stewart, W.J. (Ed.), *Numerical Solution of Markov Chains (Probability Pure and applied)*. Marcel Dekker. ISBN 978-0824784058, pp. 357–371.
- Gray, G.T., Smith, R.Q., 2001. Sperry rand's third-generation computers 1964–1980. *IEEE Ann. Hist. Comput.* 23 (1), 3–16. <https://doi.org/10.1109/85.910845>.
- Gray, G.T., Smith, R.Q., 2004. Sperry rand's first generation computers, 1955–1960: hardware and software. *IEEE Ann. Hist. Comput.* 26 (4), 20–34. <https://doi.org/10.1109/MAHC.2004.34>.
- Gray, J. (Ed.), 1993. *The Benchmark Handbook for Database and Transaction Systems*, 2. Morgan Kaufmann Publishers. ISBN 1-55860-292-5.
- Gray, J., 2002. Storage bricks have arrived (presentation). In: Long, D.D.E. (Ed.), *Proceedings of the FAST '02 Conference on File and Storage Technologies*. January 28–30, 2002, Monterey, California, USA. <http://www.usenix.org/publications/library/proceedings/fast02/miller.html>.
- Gray, J., Graefe, G., 1997. The five-minute rule ten years later, and other computer storage rules of thumb. *SIGMOD Rec.* 26 (4), 63–68. <http://doi.acm.org/10.1145/271074.271094>.
- Gray, J., Lampert, L., 2006. Consensus on transaction commit. *ACM Trans. Database Syst.* 31 (1), 133–160. <https://doi.org/10.1145/1132863.1132867>.
- Gray, J., Levine, C., 2005. Thousands of debitcredit transactions-per-second: easy and inexpensive. Technical report. Microsoft.
- Gray, J., Putzolu, G.R., 1987. The 5 minute rule for trading memory for disk accesses and the 10 byte rule for trading memory for CPU time. In: Dayal, U., Traiger, I.L. (Eds.), *Proceedings of the Association for Computing Machinery Special Interest Group on Management of Data 1987 Annual Conference*. San Francisco, CA, USA, May 27–29, 1987. ACM Press, pp. 395–398.
- Gray, J., Reuter, A., 1992. *Transaction Processing: Concepts and Techniques*. Data Management Systems. Morgan Kaufmann Publishers. ISBN 978-1558601901.
- Gray, J., Shenoy, P.J., 2000. Rules of thumb in data engineering. In: Lomet, D.B., Weikum, G. (Eds.), *Proceedings of the 16th International Conference on Data Engineering*. San Diego, California, USA, February 28–March 3, 2000. IEEE Computer Society, pp. 3–10.
- Gray, J., Good, B., Gawlick, D., Homan, P., Sammer, H., 1984. One thousand transactions per second. Technical Report 85.1. Tandem Computers. https://jimgray.azurewebsites.net/papers/TandemTR85.1_1kTPS.pdf. PN87608.
- Gray, J., Horst, B., Walker, M., 1990. Parity striping of disk arrays: low-cost reliable storage with acceptable throughput. In: McLeod, D., Sacks-Davis, R., Schek, H. (Eds.), *16th International Conference on Very Large Data Bases, Proceedings*. August 13–16, 1990, Brisbane, Queensland, Australia. Morgan Kaufmann, pp. 148–161. <http://www.vldb.org/conf/1990/P148.PDF>.
- Gray, J., Bosworth, A., Layman, A., Pirahesh, H., 1996a. Data cube: a relational aggregation operator generalizing group-by, cross-tab, and sub-total. In: Su, S.Y.W. (Ed.), *Proceedings of the Twelfth International Conference on Data Engineering*. February 26–March 1, 1996, New Orleans, Louisiana, USA. IEEE Computer Society, pp. 152–159.
- Gray, J., Helland, P., O'Neil, P.E., Shasha, D.E., 1996b. The dangers of replication and a solution. In: Jagadish, H.V., Mumick, I.S. (Eds.), *Proceedings of the 1996 ACM SIGMOD International Conference on Management of Data*. Montreal, Quebec, Canada, June 4–6, 1996. ACM Press, pp. 173–182.
- Green Jr., Paul E., 1992. *Optical Switching Networks*, 1. Prentice-Hall. ISBN 978-0133194920.
- Greenan, K.M., Long, D.D.E., Miller, E.L., Schwarz, T.J.E., Wylie, J.J., 2008. A spin-up saved is energy earned: achieving power-efficient, erasure-coded storage. In: Alvisi, L., Maniatis, P. (Eds.), *Proceedings of the Fourth Workshop on Hot Topics in System Dependability, HotDep 2008*. San Diego, CA, USA, December 7, 2008. USENIX Association. http://www.usenix.org/events/hotdep08/tech/full_papers/greenan/greenan.pdf.
- Greenan, K.M., Plank, J.S., Wylie, J.J., 2010. Mean time to meaningless: MTDDL, Markov models, and storage system reliability. In: Schindler, J. (Ed.), *2nd USENIX Workshop on Hot Topics in Storage and File Systems, HotStorage'10*. Boston, MA, USA, June 22, 2010. USENIX Association. <https://www.usenix.org/conference/hotstorage-10/mean-time-meaningless-mttddl-markov-models-and-storage-system-reliability>.
- Greenawalt, P.M., 1994. Modeling power management for hard disks. In: Madiseti, V.K., Gelenbe, E., Walrand, J.C. (Eds.), *MASCOTS '94, Proceedings of the Second International Workshop on Modeling, Analysis, and Simulation On Computer and Telecommunication Systems*. January 31–February 2, 1994, Durham, North Carolina, USA. IEEE Computer Society, pp. 62–66.

- Griffioen, J., Appleton, R., 1994. Reducing file system latency using a predictive approach. In: USENIX Summer 1994 Technical Conference, Conference Proceeding. Boston, Massachusetts, USA, June 6–10, 1994. USENIX Association, pp. 197–207. <https://www.usenix.org/conference/usenix-summer-1994-technical-conference/reducing-file-system-latency-using-predictive>.
- Grund, M., Krüger, J., Plattner, H., Zeier, A., Cudré-Mauroux, P., Madden, S., 2010. HYRISE – a main memory hybrid storage engine. *Proc. VLDB Endow.* 4 (2), 105–116. <http://www.vldb.org/pvldb/vol4/p105-grund.pdf>.
- Guerra, J., Pucha, H., Glider, J.S., Belluomini, W., Rangaswami, R., 2011. Cost effective storage using extent based dynamic tiering. In: Ganger, G.R., Wilkes, J. (Eds.), 9th USENIX Conference on File and Storage Technologies. San Jose, CA, USA, February 15–17, 2011. USENIX, pp. 273–286. <http://www.usenix.org/events/fast11/tech/techAbstracts.html#Guerra>.
- Guha, A., 2004. A new approach to disk-based scalable mass storage system. In: Kobler, B., Hariharan, P.C. (Eds.), 21st IEEE Conference on Mass Storage Systems and Technologies/12th NASA Goddard Conference on Mass Storage Systems and Technologies. Greenbelt, Maryland, USA, April 13–16, 2004. IEEE, pp. 421–425. <http://storageconference.org/nasa/conf2004/Papers/MSST2004-47-Guha-a.pdf>.
- Guha, A., 2006. Power-managed storage: longer data life and lower energy consumption. In: LeFebvre, W. (Ed.), Proceedings of the 20th Conference on Systems Administration (LISA 2006). Washington, DC, USA, December 3–8, 2006. USENIX. <http://www.usenix.org/events/lisa06/tech/slides/guha.pdf>.
- Gunawi, H.S., Hao, M., Leesatapornwongsa, T., et al., 2014. What bugs live in the cloud? A study of 3000+ issues in cloud systems. In: Lazowska, E., Terry, D., Arpaci-Dusseau, R.H., Gehrke, J. (Eds.), Proceedings of the ACM Symposium on Cloud Computing. Seattle, WA, USA, November 3–5, 2014. ACM, pp. 7:1–7:14.
- Guo, F., Efstathopoulos, P., 2011. Building a high-performance deduplication system. In: Nieh, J., Waldspurger, C.A. (Eds.), 2011 USENIX Annual Technical Conference. Portland, OR, USA, June 15–17, 2011. USENIX Association. <https://www.usenix.org/conference/usenixatc11/building-high-performance-deduplication-system>.
- Gupta, A., Mumick, I.S. (Eds.), 1999. *Materialized Views: Techniques, Implementations, and Applications*. MIT Press, Cambridge, MA. ISBN 978-0262571227.
- Gupta, A., Kim, Y., Ugaonkar, B., 2009. DFTL: a flash translation layer employing demand-based selective caching of page-level address mappings. In: Soffa, M.L., Irwin, M.J. (Eds.), Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS 2009. Washington, DC, USA, March 7–11, 2009. ACM, pp. 229–240.
- Gupta, R.K., Franklin, M.A., 1978. Working set and page fault frequency paging algorithms: a performance comparison. *IEEE Trans. Comput.* 27 (8), 706–712. <https://doi.org/10.1109/TC.1978.1675177>.
- Gurumurthi, S., Sivasubramaniam, A., 2006. Thermal issues in disk drive design: challenges and possible solutions. *ACM Trans. Storage* 2 (1), 41–73. <http://doi.acm.org/10.1145/1138041.1138044>.
- Gurumurthi, S., Sivasubramaniam, A., Kandemir, M.T., Franke, H., 2003a. DRPM: dynamic speed control for power management in server class disks. In: Gottlieb, A., Li, K. (Eds.), 30th International Symposium on Computer Architecture (ISCA 2003). 9–11 June 2003, San Diego, California, USA. IEEE Computer Society, pp. 169–179.
- Gurumurthi, S., Sivasubramaniam, A., Kandemir, M.T., Franke, H., 2003b. Reducing disk power consumption in servers with DRPM. *IEEE Computer* 36 (12), 59–66. <https://doi.org/10.1109/MC.2003.1250884>.
- Gurumurthi, S., Zhang, J., Sivasubramaniam, A., Kandemir, M.T., Franke, H., Vijaykrishnan, N., Irwin, M.J., 2003c. Interplay of energy and performance for disk arrays running transaction processing workloads. In: 2003 IEEE International Symposium on Performance Analysis of Systems and Software, Proceedings. March 6–8, 2003, Austin, Texas, USA. IEEE Computer Society, pp. 123–132.
- Gurumurthi, S., Zhang, J., Sivasubramaniam, A., Kandemir, M.T., Franke, H., Vijaykrishnan, N., Irwin, M.J., 2003d. Interplay of energy and performance for disk arrays running transaction processing workloads. In: 2003 IEEE International Symposium on Performance Analysis of Systems and Software, Proceedings. March 6–8, 2003, Austin, Texas, USA. IEEE Computer Society, pp. 123–132.
- Gurumurthi, S., Sankar, S., Stan, M.R., 2009. Using intradisk parallelism to build energy-efficient storage systems. *IEEE MICRO* 29 (1), 50–61. <https://doi.org/10.1109/MM.2009.21>.
- Gustafson, J.L., 1988. Reevaluating Amdahl's law. *Commun. ACM* 31 (5), 532–533. <https://doi.org/10.1145/42411.42415>.
- Gustafson, J.L., 2015. *The End of Error: Unum Computing. 1*. CRC Computational Science. Chapman & Hall, Boca Raton, FL. ISBN 978-1482239867.
- Guttman, A., 1984. R-trees: a dynamic index structure for spatial searching. In: Yonmark, B. (Ed.), SIGMOD'84, Proceedings of Annual Meeting. Boston, Massachusetts, USA, June 18–21, 1984. ACM Press, pp. 47–57.
- Güvenilir, Faruk A., 2019. Scalable Virtual Memory via Tailored and Larger Page Sizes. PhD thesis. University of Texas at Austin, Austin, TX. <https://eecs.utexas.edu/pub/TR-HPS-2019-001.pdf>. TR-HPS-2019-001.
- Habib, S., 1988. *Microprogramming and Firmware Engineering*. Van Nostrand Reinhold. ISBN 978-044235543.
- Haeblerlen, A., Mislove, A., Druschel, P., 2005. Glacier: highly durable, decentralized storage despite massive correlated failures. In: Vahdat, A., Wetherall, D. (Eds.), 2nd Symposium on Networked Systems Design and Implementation (NSDI 2005), Proceedings. May 2–4, 2005, Boston, Massachusetts, USA. USENIX. <http://www.usenix.org/events/nsdi05/tech/haeblerlen.html>.
- Hafner, J., 2005a. Hover erasure codes for disk arrays. Technical Report RJ 10352. IBM Research at Almaden. <https://dominoweb.draco.res.ibm.com/reports/rj10352.pdf>.
- Hafner, J.L., 2005b. WEAVER codes: highly fault tolerant erasure codes for storage systems. In: Gibson, G. (Ed.), Proceedings of the FAST '05 Conference on File and Storage Technologies. December 13–16, 2005, San Francisco, California, USA. USENIX. http://www.usenix.org/events/fast05/tech/hafner_weaver.html.
- Hafner, J.L., 2006. Hover erasure codes for disk arrays. In: 2006 International Conference on Dependable Systems and Networks (DSN 2006), Proceedings. 25–28 June 2006, Philadelphia, Pennsylvania, USA. IEEE Computer Society, pp. 217–226.

- Hafner, J.L., Rao, K.K., 2006. Notes on reliability models for non-mds erasure codes. Technical Report RJ 10391. IBM Almaden Research Center, San Jose, CA. <https://dominoweb.draco.res.ibm.com/reports/rj10391.pdf>.
- Hafner, J.L., Deenadhayalan, V., Kanungo, T., Rao, K., 2004. Performance metrics for erasure codes in storage systems. Technical Report RJ 10321. IBM Almaden Research Center, San Jose, CA.
- Hafner, J.L., Deenadhayalan, V., Rao, K.K., Tomlin, J.A., 2005. Matrix methods for lost data reconstruction in erasure codes. In: Gibson, G. (Ed.), Proceedings of the FAST '05 Conference on File and Storage Technologies. December 13–16, 2005, San Francisco, California, USA. USENIX. http://www.usenix.org/events/fast05/tech/hafner_matrix.html.
- Hafner, J.L., Deenadhayalan, V., Belluomini, W., Rao, K., 2008. Undetected disk errors in RAID arrays. IBM J. Res. Dev. 52 (4–5), 413–426. <https://doi.org/10.1147/rd.524.0413>.
- Hagmann, R.B., Ferrari, D., 1986. Performance analysis of several back-end database architectures. ACM Trans. Database Syst. 11 (1), 1–26. <https://doi.org/10.1145/5236.5242>.
- Hall, M., 1986. Combinatorial Theory, 2. Wiley-Interscience, New York, NY.
- Hall, M., Barry, J., 1991. Sunburst – The Ascent of Sun Microsystems, 1. Contemporary Books. ISBN 978-0809239894.
- Hall, R.J., 2016. Tools for predicting the reliability of large-scale storage systems. ACM Trans. Storage 12 (4), 24:1–24:30. <https://doi.org/10.1145/2911987>.
- Hamacher, C., Vranesic, Z., Zaky, S., Manjikian, N., 2012. Computer Organization and Embedded Systems, 6. McGraw-Hill, New York, NY. ISBN 978-0073380650.
- Hamming, R.W., 1950. Error detecting and error correcting codes. Bell Syst. Tech. J. 29 (2), 147–160.
- Hansoth, N., 2016. A look inside china's chart-topping new supercomputer. The Next Platform, The Stackhouse Publishing. <https://www.nextplatform.com/2016/06/20/look-inside-chinas-chart-topping-new-supercomputer/>.
- Han, J., Kamber, M., 2011. Data Mining Concepts and Techniques, 3. The Morgan Kaufmann Series in Data Management Systems. Elsevier. ISBN 978-9380931913.
- Hanani, H., 1975. Balanced incomplete block designs and related designs. Discrete Math. 11 (3), 255–369. [https://doi.org/10.1016/0012-365X\(75\)90040-0](https://doi.org/10.1016/0012-365X(75)90040-0).
- Hansen, P., Mladenovic, N., 2001. Variable neighborhood search: principles and applications. Eur. J. Oper. Res. 130 (3), 449–467. [https://doi.org/10.1016/S0377-2217\(00\)00100-4](https://doi.org/10.1016/S0377-2217(00)00100-4).
- Hanson, W.A., 2020. The CORAL supercomputer systems. IBM J. Res. Dev. 64 (3/4), 1:1–1:10. <https://doi.org/10.1147/JRD.2019.2960220>.
- Hao, M., Soundararajan, G., Kenchammana-Hosekote, D.R., Chien, A.A., Gunawi, H.S., 2016. The tail at store: a revelation from millions of hours of disk and SSD deployments. In: Brown, A.D., Popovici, F.I. (Eds.), 14th USENIX Conference on File and Storage Technologies, FAST 2016. Santa Clara, CA, USA, February 22–25, 2016. USENIX Association, pp. 263–276. <https://www.usenix.org/conference/fast16/technical-sessions/presentation/hao>.
- Harchol-Balter, M., 2013. Performance Modeling and Design of Computer Systems: Queueing Theory in Action, 1. Cambridge University Press. ISBN 978-1107027503.
- Härder, T., Hudlet, V., Ou, Y., Schall, D., 2011. Energy efficiency is not enough, energy proportionality is needed! In: Xu, J., Yu, G., Zhou, S., Unland, R. (Eds.), Database Systems for Advanced Applications – 16th International Conference, DASFAA 2011, International Workshops: GDB, SIM3, FlashDB, SNSMW, DaMEN, DQIS, Proceedings. Hong Kong, China, April 22–25, 2011. In: Lecture Notes in Computer Science, vol. 6637. Springer, pp. 226–239.
- Haritsa, J.R., Carey, M.J., Livny, M., 1993. Value-based scheduling in real-time database systems. VLDB J. 2 (2), 117–152. <http://www.vldb.org/journal/VLDBJ2/P117.pdf>.
- Harris, D.M., Harris, S.L., 2012. Digital Design and Computer Architecture, 2. Morgan Kaufmann. ISBN 978-0123944245.
- Harrison, E.S., Schmitt, E.J., 1987. The structure of system/88, a fault-tolerant computer. IBM Syst. J. 26 (3), 293–318. <https://doi.org/10.1147/sj.263.0293>.
- Harrison, T.J., Landeck, B.W., Clair, H.K.S., 1981. Evolution of small real-time IBM computer systems. IBM J. Res. Dev. 25 (5), 441–452. <https://doi.org/10.1147/rd.255.0441>.
- Hartman, J.H., Ousterhout, J.K., 1995. The zebra striped network file system. ACM Trans. Comput. Syst. 13 (3), 274–310. <https://doi.org/10.1145/210126.210131>.
- Haskin, R.L., 1998. Tiger shark – a scalable file system for multimedia. IBM J. Res. Dev. 42 (2), 185–198. <https://doi.org/10.1147/rd.422.0185>.
- Hatfield, D.J., Gerald, J., 1971. Program restructuring for virtual memory. IBM Syst. J. 10 (3), 168–192. <https://doi.org/10.1147/sj.103.0168>.
- Haverkort, B.R., Marie, R., Rubino, G., Trivedi, K.S., 2001. Performability Modelling: Techniques and Tools. Wiley, New York, NY.
- He, B., Yu, J.X., 2011. High-throughput transaction executions on graphics processors. Proc. VLDB Endow. 4 (5), 314–325. <https://doi.org/10.14778/1952376.1952381>.
- He, B., Lu, M., Yang, K., Fang, R., Govindaraju, N.K., Luo, Q., Sander, P.V., 2009. Relational query coprocessing on graphics processors. ACM Trans. Database Syst. 34 (4), 21:1–21:39. <https://doi.org/10.1145/1620585.1620588>.
- Healy, L.D., 1976. A character-oriented context-addressed segment sequential storage. In: Flynn, M.J., Garcia, O.N., Siewiorek, D.P. (Eds.), Proceedings of the 3rd Annual Symposium on Computer Architecture. Clearwater, FL, USA, January 1976. ACM, pp. 172–177.
- Heidemann, J.S., Popek, G.J., 1994. File-system development with stackable layers. ACM Trans. Comput. Syst. 12 (1), 58–89. <https://doi.org/10.1145/174613.174616>.
- Heimel, M., Saecker, M., Pirk, H., Manegold, S., Markl, V., 2013. Hardware-oblivious parallelism for in-memory column-stores. Proc. VLDB Endow. 6 (9), 709–720. <http://www.vldb.org/pvldb/vol6/p709-heimel.pdf>.

- Hellerstein, J.M., Naughton, J.F., Pfeffer, A., 1995. Generalized search trees for database systems. In: Dayal, U., Gray, P.M.D., Nishio, S. (Eds.), VLDB'95, Proceedings of 21th International Conference on Very Large Data Bases. September 11–15, 1995, Zurich, Switzerland. Morgan Kaufmann, pp. 562–573. <http://www.vldb.org/conf/1995/P562.PDF>.
- Hellerstein, J.M., Haas, P.J., Wang, H.J., 1997. Online aggregation. In: Peckham, J. (Ed.), SIGMOD 1997, Proceedings ACM SIGMOD International Conference on Management of Data. Tucson, Arizona, USA, May 13–15, 1997. ACM Press, pp. 171–182.
- Hellerstein, L., Gibson, G.A., Karp, R.M., Katz, R.H., Patterson, D.A., 1994. Coding techniques for handling failures in large disk arrays. *Algorithmica* 12 (2/3), 182–208. <https://doi.org/10.1007/BF01185210>.
- Helmbold, D.P., Long, D.D.E., Sherrod, B., 1996. A dynamic disk spin-down technique for mobile computing. In: Ahmadi, H., Katz, R.H., Akyildiz, I.F., Haas, Z.J. (Eds.), MOBICOM '96, Proceedings of the Second Annual International Conference on Mobile Computing and Networking. Rye, New York, USA, November 10–12, 1996. ACM, pp. 130–142.
- Hennessy, J., Patterson, D., 2018. A new golden age for computer architecture: domain-specific hardware/software co-design, enhanced security, open instruction sets, and agile chip development. <https://www.youtube.com/watch?v=3LVEjsn8Ts>.
- Hennessy, J.L., Patterson, D.A., 2017. Computer Architecture – A Quantitative Approach, 6th edition. Morgan Kaufmann. ISBN 978-0-12-811905-1.
- Hennessy, J.L., Patterson, D.A., 2019. The golden age for computer architecture. *Commun. ACM* 62, 48–60.
- Herbst, N.R., Kounev, S., Reussner, R.H., 2013. Elasticity in cloud computing: what it is, and what it is not. In: Kephart, J.O., Pu, C., Zhu, X. (Eds.), 10th International Conference on Autonomic Computing, ICAC'13. San Jose, CA, USA, June 26–28, 2013. UNIX Association, pp. 23–27. <https://www.usenix.org/conference/icac13/technical-sessions/presentation/herbst>.
- Hetzler, S.R., Kabelac, W.J., 1993. Sector architecture for fixed block disk drive patent.
- Hildenbrand, S., 2012. Scaling Out Column Stores: Data, Queries, and Transactions. PhD thesis. ETH, Zurich, Switzerland. Diss. ETH No. 20314.
- Hill, M.D., Marty, M.R., 2008. Amdahl's law in the multicore era. *Computer* 41 (7), 33–38. <https://doi.org/10.1109/MC.2008.209>.
- Hill, R.A., 1994. System for managing data storage based on vector-summed size-frequency vectors for data sets, devices, and residual storage on devices.
- Hitz, D., Walsh, P., 2009. How to Castrate a Bull: Unexpected Lessons on Risk, Growth, and Success in Business. Wiley, New York, NY. ISBN 978-0470345238.
- Hofri, M., 1980. Disk scheduling: FCFS vs. SSTF revisited. *Commun. ACM* 23 (11), 645–653. <http://doi.acm.org/10.1145/359024.359034>.
- Hofri, M., 1983. Should the two-headed disk be greedy? – yes, it should. *Inf. Process. Lett.* 16 (2), 83–85. [https://doi.org/10.1016/0020-0190\(83\)90030-3](https://doi.org/10.1016/0020-0190(83)90030-3).
- Holland, M., 1994. On-line Data Reconstruction in Redundant Disk Arrays. PhD thesis. Electrical and Computer Engineering Dept., Carnegie-Mellon University, Pittsburgh, PA.
- Holland, M., Gibson, G.A., 1992. Parity declustering for continuous operation in redundant disk arrays. In: Flahive, B., Wexelblat, R.L. (Eds.), ASPLOS-V Proceedings – Fifth International Conference on Architectural Support for Programming Languages and Operating Systems. Boston, Massachusetts, USA, October 12–15, 1992. ACM Press, pp. 23–35.
- Holland, M., Gibson, G.A., Siewiorek, D.P., 1994. Architectures and algorithms for on-line failure recovery in redundant disk arrays. *Distrib. Parallel Databases* 2 (3), 295–335. <https://doi.org/10.1007/BF01266332>.
- Hord, D.M., 1990. Parallel Supercomputing in SIMD Architectures. CRC Press, Boca Raton, FL. ISBN 0-8493-4271-6.
- Hou, H., Lee, P.P.C., 2020. Binary MDS array codes with optimal repair. *IEEE Trans. Inf. Theory* 66 (3), 1405–1422. <https://doi.org/10.1109/TIT.2019.2939111>.
- Hou, R.Y., Menon, J., Patt, Y.N., 1993. Balancing i/o response time and disk rebuild time in a raid5 disk array. In: 26th Annual Hawaii International Conference on System Sciences (HICSS-26). Maui, Hawaii, USA, January 4–7, 1994, pp. 70–79.
- Hsiao, D.K., 1979. Database machines are coming, database machines are coming! *IEEE Computer* 12 (3), 7–9. <https://doi.org/10.1109/MC.1979.1658647>.
- Hsiao, D.K., 1983. Advanced Database Machine Architecture, 1. Prentice-Hall. ISBN 978-0130112620.
- Hsiao, H., DeWitt, D.J., 1990. Chained declustering: a new availability strategy for multiprocessor database machines. In: Proceedings of the Sixth International Conference on Data Engineering. February 5–9, 1990, Los Angeles, California, USA, pp. 456–465.
- Hsu, W.W., 2003. Dynamic Locality Improvement Techniques for Increasing Effective Storage Performance. PhD thesis. University of California, Berkeley, CA. <https://www2.eecs.berkeley.edu/Pubs/TechRpts/2003/5440>.
- Hsu, W.W., Smith, A.J., 2003. Characteristics of I/O traffic in personal computer and server workloads. *IBM Syst. J.* 42 (2), 347–372. <https://doi.org/10.1147/sj.422.0347>.
- Hsu, W.W., Smith, A.J., 2004. The performance impact of I/O optimizations and disk improvements. *IBM J. Res. Dev.* 48 (2), 255–289. <https://doi.org/10.1147/rd.482.0255>.
- Hsu, W.W., Smith, A.J., Young, H.C., 2000. Projecting the performance of decision support workloads on systems with smart storage (smartstor). In: Seventh International Conference on Parallel and Distributed Systems, ICPADS 2000. Iwate, Japan, July 4–7, 2000. IEEE Computer Society, pp. 417–425.
- Hsu, W.W., Smith, A.J., Young, H.C., 2001. Characteristics of production database workloads and the TPC benchmarks. *IBM Syst. J.* 40 (3), 781–802. <https://doi.org/10.1147/sj.403.0781>.
- Hsu, W.W., Smith, A.J., Young, H.C., 2005. The automatic improvement of locality in storage systems. *ACM Trans. Comput. Syst.* 23 (4), 424–473. <http://doi.acm.org/10.1145/1113574.1113577>.
- Hu, X., Eleftheriou, E., Haas, R., Iliadis, I., Pletka, R., 2009. Write amplification analysis in flash-based solid state drives. In: Allalouf, M., Factor, M., Feitelson, D.G. (Eds.), Proceedings of SYSTOR 2009: The Israeli Experimental Systems Conference. 2009, Haifa, Israel, May 4–6, 2009. In: ACM International Conference Proceeding Series. ACM, p. 10.
- Hu, Y., Chen, H.C.H., Lee, P.P.C., Tang, Y., 2012. Nccloud: applying network coding for the storage repair in a cloud-of-clouds. In: Bolosky, W.J., Flinn, J. (Eds.), Proceedings of the 10th USENIX Conference on File and Storage Technologies, FAST 2012. San Jose, CA, USA, February 14–17, 2012. USENIX Association, p. 21. <https://www.usenix.org/conference/fast12/nccloud-applying-network-coding-storage-repair-cloud-clouds>.

- Huang, C., Xu, L., 2005. STAR: an efficient coding scheme for correcting triple storage node failures. In: Gibson, G. (Ed.), *Proceedings of the FAST '05 Conference on File and Storage Technologies*. December 13–16, 2005, San Francisco, California, USA. USENIX. <http://www.usenix.org/events/fast05/tech/huang.html>.
- Huang, C., Chen, M., Li, J., 2007. Pyramid codes: flexible schemes to trade space for access efficiency in reliable data storage systems. In: *Sixth IEEE International Symposium on Network Computing and Applications (NCA 2007)*. 12–14 July 2007, Cambridge, MA, USA. IEEE Computer Society, pp. 79–86.
- Huang, C., Simitci, H., Xu, Y., Ogun, A., Calder, B., Gopalan, P., Li, J., Yekhanin, S., 2012. Erasure coding in windows azure storage. In: Heiser, G., Hsieh, W.C. (Eds.), *212 USENIX Annual Technical Conference*. Boston, MA, USA, June 13–15, 2012. USENIX Association, pp. 15–26. <https://www.usenix.org/conference/atc12/technical-sessions/presentation/huang>.
- Huang, G., Cheng, X., Wang, J., Wang, Y., He, D., Zhang, T., Li, F., Wang, S., Cao, W., Li, Q., 2019. X-engine: an optimized storage engine for large-scale e-commerce transaction processing. In: Boncz, P.A., Manegold, S., Ailamaki, A., Deshpande, A., Kraska, T. (Eds.), *Proceedings of the 2019 International Conference on Management of Data, SIGMOD Conference 2019*. Amsterdam, The Netherlands, June 30–July 5, 2019. ACM, pp. 651–665.
- Huang, H., Hung, W., Shin, K.G., 2005. FS2: dynamic data replication in free disk space for improving disk performance and energy consumption. In: Herbert, A., Birman, K.P. (Eds.), *Proceedings of the 20th ACM Symposium on Operating Systems Principles 2005, SOSP 2005*. Brighton, UK, October 23–26, 2005. ACM, pp. 263–276.
- Huawei Technologies, 2014. Huawei OceanStor enterprise unified storage system, RAID2.0+. Technical report. Huawei Corp., Shenzhen, China. <http://support.huawei.com/enterprise/thread-298411.html>.
- Huck, J.C., Flynn, M.J., 1989. *Analyzing Computer Architectures*. IEEE Computer Society Press. ISBN 978-0-8186-8857-7.
- Huffman, D.A., 1952. A method for the construction of minimum-redundancy codes. *Proc. IRE* 40 (9), 1098–1101.
- Huggahalli, R., Iyer, R.R., Tetrack, S., 2005. Direct cache access for high bandwidth network I/O. In: *32st International Symposium on Computer Architecture (ISCA 2005)*. 4–8 June 2005, Madison, Wisconsin, USA. IEEE Computer Society, pp. 50–59.
- Hughes, G.F., Murray, J.F., 2005. Reliability and security of RAID storage systems and D2D archives using SATA disk drives. *ACM Trans. Storage* 1 (1), 95–107. <http://doi.acm.org/10.1145/1044956.1044961>.
- Hughes, G.F., Murray, J.F., Kreutz-Delgado, K., Elkan, C., 2002. Improved disk-drive failure warnings. *IEEE Trans. Reliab.* 51 (3), 350–357. <https://doi.org/10.1109/TR.2002.802886>.
- Hunter, D., Jacobs, P., 1982. Modeling real dasd configurations. In: Disney, R.L., Ott, T.S. (Eds.), *Applied Probability – Computer Science: The Interface Vol. I*. Birkhauser, Basel, Switzerland, pp. 451–470.
- Hurson, A.R., Miller, L.L., Pakzad, S.H., Eich, M.H., Shirazi, B.A., 1989a. Parallel architectures for database systems. *Adv. Comput.* 28, 107–151. [https://doi.org/10.1016/S0065-2458\(08\)60047-9](https://doi.org/10.1016/S0065-2458(08)60047-9).
- Hurson, A.R., Miller, L.L., Pakzad, S.H., Eich, M.H., Shirazi, B.A., 1989b. Parallel architectures for database systems. *Adv. Comput.* 28, 107–151. [https://doi.org/10.1016/S0065-2458\(08\)60047-9](https://doi.org/10.1016/S0065-2458(08)60047-9).
- Hurwitz, J., Kirsch, D., Dalrymple, F., 2019. *LinuxOne® for Dummies – IBM Limited Edition*. Wiley. ISBN 978-1-119-56831-5.
- Hwang, K., Jotwani, N., 2015. *Advanced Computer Architecture: Parallelism, Scalability, Programmability*, 3. McGraw-Hill India. ISBN 978-0070702103.
- Hwang, K., Ni, L.M., 1980. Resource optimization of a parallel computer for multiple vector processing. *IEEE Trans. Comput.* 29 (9), 831–836. <https://doi.org/10.1109/TC.1980.1675683>.
- Hwang, K., Su, S., Ni, L.M., 1981. Vector computer architecture and processing techniques. *Adv. Comput.* 20, 115–197. [https://doi.org/10.1016/S0065-2458\(08\)60497-0](https://doi.org/10.1016/S0065-2458(08)60497-0).
- Hwang, K., Jin, H., Ho, R.S.C., 2002. Orthogonal striping and mirroring in distributed RAID for i/o-centric cluster computing. *IEEE Trans. Parallel Distrib. Syst.* 13 (1), 26–44. <https://doi.org/10.1109/71.980025>.
- Ibaraki, T., Katoh, N., 1988. *Resource Allocation Problems: Algorithmic Approaches*. Foundations of Computing. The MIT Press. ISBN 978-0262090278.
- IBM Corp., 1967. *IBM System/360 Principles of Operation*. Technical report. IBM.
- IBM Corp., 1973. *DOS/VS LIOCS Volume 3 – DAM and ISAM Logic*. Technical report. IBM Corp. https://archive.org/details/bitsavers_ibm370DOSVIOCSVolume3DAMandISAMLogicRel28Jun73_36886118/mode/2up.
- IBM Corp., 1974. *Introduction to IBM Direct Access Storage Devices*. Technical report. IBM Corp. http://www.bitsavers.org/pdf/ibm/dasd/GC20-1649-9_DASDintro_Dec75.pdf.
- IBM Corp., 2000. *Engineering and Scientific Subroutine Library for AIX, Guide and Reference*. Technical report. IBM.
- IBM Corp., 2013. *Enterprise PL/I for z/OS – Programming Guide Version 4 Release 4*. Technical report. IBM Corp., Armonk, NY.
- IBM Corp., 2016. *IBM TS4500 Tape Library: Specifications*. Technical report. IBM. <http://www-03.ibm.com/systems/storage/tape/ts4500/specifications.html>.
- IBM Corp. Int'l Tech. Support Org., 1997. *IBM RAMAC Virtual Array*. Technical report. IBM Corp. Int'l Tech Support Org., San Jose, CA. [www.redbooks.ibm.com](http://redbooks.ibm.com).
- IBM Corp. Int'l Tech. Support Org., 2016. *Implementing IBM Storage Data Deduplication Solutions*. Technical report. IBM Corp., Armonk, NY. [www.redbooks.ibm.com](http://redbooks.ibm.com).
- Iliadis, I., Venkatesan, V., 2014. Expected annual fraction of data loss as a metric for data storage reliability. In: *IEEE 22nd International Symposium on Modelling, Analysis & Simulation of Computer and Telecommunication Systems, MASCOTS 2014*. Paris, France, September 9–11, 2014. IEEE Computer Society, pp. 375–384.

- Iliadis, I., Venkatesan, V., 2015. Rebuttal to “Beyond MTDDL: a closed-form RAID-6 reliability equation”. *ACM Trans. Storage* 11 (2), 9:1–9:10. <http://doi.acm.org/10.1145/2700311>.
- Iliadis, I., Haas, R., Hu, X., Eleftheriou, E., 2008. Disk scrubbing versus intra-disk redundancy for high-reliability raid storage systems. In: Liu, Z., Misra, V., Shenoy, P.J. (Eds.), *Proceedings of the 2008 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems, SIGMETRICS 2008*. Annapolis, MD, USA, June 2–6, 2008. ACM, pp. 241–252.
- Iliadis, I., Haas, R., Hu, X., Eleftheriou, E., 2011. Disk scrubbing versus intradisk redundancy for RAID storage systems. *ACM Trans. Storage* 7 (2), 5:1–5:42. <http://doi.acm.org/10.1145/1970348.1970350>.
- Iliadis, I., Kim, Y., Sarafijanovic, S., Venkatesan, V., 2016. Performance evaluation of a tape library system. In: *24th IEEE International Symposium on Modeling, Analysis and Simulation of Computer and Telecommunication Systems, MASCOTS 2016*. London, United Kingdom, September 19–21, 2016. IEEE Computer Society, pp. 59–68.
- Iliadis, I., Jelitto, J., Kim, Y., Sarafijanovic, S., Venkatesan, V., 2017. Exaplan: efficient queueing-based data placement, provisioning, and load balancing for large tiered storage systems. *ACM Trans. Storage* 13 (2), 17:1–17:41. <http://doi.acm.org/10.1145/3078839>.
- Ilyas, I.F., Chu, X., 2019. *Data Cleaning*. ACM Books. ISBN 978-1450371537.
- Im, J., Bae, J., Chung Arvind, C., Lee, S., 2020. Pink: high-speed in-storage key-value store with bounded tails. In: Gavrilovska, A., Zadok, E. (Eds.), *USENIX Annual Technical Conference, USENIX ATC 2020*. July 15–17, 2020. USENIX Association, pp. 173–187. <https://www.usenix.org/conference/atc20/presentation/im>.
- Imani, M., Gupta, S., Sharma, S., Rosing, T.S., 2019. Nvquery: efficient query processing in nonvolatile memory. *IEEE Trans. CAD Integr. Circuits Syst.* 38 (4), 628–639. <https://doi.org/10.1109/TCAD.2018.2819080>.
- Inmon, W., 2005. *Building the Data Warehouse*, 4. John Wiley and Sons. ISBN 978-81-265-0645-3.
- Ioannou, N., Kourtis, K., Koltsidas, I., 2018. Elevating commodity storage with the SALSA host translation layer. In: *26th IEEE International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems, MASCOTS 2018*. Milwaukee, WI, USA, September 25–28, 2018. IEEE Computer Society, pp. 277–292.
- István, Z., 2019. The glass half full: using programmable hardware accelerators in analytics. *IEEE Data Eng. Bull.* 42 (1), 49–60. <http://sites.computer.org/debull/A19mar/p49.pdf>.
- Smullen IV, C.W., Tarapore, S.R., Gurumurthi, S., Ranganathan, P., Uysal, M., 2008. Active storage revisited: the case for power and performance benefits for unstructured data processing applications. In: Ramírez, A., Bilardi, G., Gschwind, M. (Eds.), *Proceedings of the 5th Conference on Computing Frontiers*, 2008. Ischia, Italy, May 5–7, 2008. ACM, pp. 293–304.
- Iyer, B.R., Wilhite, D., 1994. Data compression support in databases. In: Bocca, J.B., Jarke, M., Zaniolo, C. (Eds.), *Vldb’94, Proceedings of 20th International Conference on Very Large Data Bases*. September 12–15, 1994, Santiago de Chile, Chile. Morgan Kaufmann, pp. 695–704. <http://www.vldb.org/conf/1994/P695.PDF>.
- Iyer, S., Druschel, P., 2001. Anticipatory scheduling: a disk scheduling framework to overcome deceptive idleness in synchronous I/O. In: Marzullo, K., Satyanarayanan, M. (Eds.), *Proceedings of the 18th ACM Symposium on Operating System Principles, SOSP 2001*. Chateau Lake Louise, Banff, Alberta, Canada, October 21–24, 2001. ACM, pp. 117–130.
- Jacob, B.L., Ng, S.W., Wang, D.T., 2008. *Memory Systems: Cache, DRAM, Disk*. Morgan Kaufmann. ISBN 978-0-12-379751-3.
- Jacobson, D.M., Wilkes, J., 1991. *Disk scheduling algorithms based on rotational position*. Technical Report HPL-CSP-91-7rev1. HP Laboratories, Palo Alto, CA.
- Jaffer, S., Maneas, S., Hwang, A.A., Schroeder, B., 2019. Evaluating file system reliability on solid state drives. In: Malkhi, D., Tsafir, D. (Eds.), *USENIX Annual Technical Conference, USENIX ATC 2019*. Renton, WA, USA, July 10–12, 2019. USENIX Association, pp. 783–798. <https://www.usenix.org/conference/atc19/presentation/jaffer>.
- Jaffer, S., Maneas, S., Hwang, A., Schroeder, B., 2020. The reliability of modern file systems in the face of ssd errors. *ACM Trans. Storage* 16 (1), 2. <https://dl.acm.org/doi/10.1145/3375553>.
- Jain, A., Lin, C., 2016. Back to the future: leveraging Belady’s algorithm for improved cache replacement. In: *43rd ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2016*. Seoul, South Korea, June 18–22, 2016, pp. 78–89.
- Jajodia, S., Kim, W., Silberschatz, A. (Eds.), 1988. *Proceedings of the International Symposium on Databases in Parallel and Distributed Systems*. Austin, Texas, USA, December 5–7, 1988. IEEE Computer Society. ISBN 0-8186-0893-5. <http://ieeexplore.ieee.org/xpl/mostRecentIssue.jsp?punumber=5535>.
- Ji, M., Veitch, A.C., Wilkes, J., 2003. Seneca: remote mirroring done write. In: *Proceedings of the General Track: 2003 USENIX Annual Technical Conference*. San Antonio, Texas, USA, June 9–14, 2003, pp. 253–268. <http://www.usenix.org/events/usenix03/tech/ji.html>.
- Jiang, S., Ding, X., Xu, Y., Davis, K., 2013. A prefetching scheme exploiting both data layout and access history on disk. *ACM Trans. Storage* 9 (3), 10:1–10:23. <http://doi.acm.org/10.1145/2508010>.
- Jiang, W., Hu, C., Zhou, Y., Kanevsky, A., 2008a. Are disks the dominant contributor for storage failures – a comprehensive study of storage subsystem failure characteristics. *ACM Trans. Storage* 4 (3), 7:1–7:25. <http://doi.acm.org/10.1145/1416944.1416946>.
- Jiang, W., Hu, C., Zhou, Y., Kanevsky, A., 2008b. Are disks the dominant contributor for storage failures? A comprehensive study of storage subsystem failure characteristics. In: Baker, M., Riedel, E. (Eds.), *6th USENIX Conference on File and Storage Technologies, FAST 2008*. February 26–29, 2008, San Jose, CA, USA, pp. 111–125. <http://www.usenix.org/events/fast08/tech/jiang.html>.
- Jin, H., Hwang, K., 2000. Stripped mirroring RAID architecture. *J. Syst. Archit.* 46 (6), 543–550. [https://doi.org/10.1016/S1383-7621\(99\)00027-2](https://doi.org/10.1016/S1383-7621(99)00027-2).
- Jin, H., Cortes, T., Buyya, R. (Eds.), 2001. *High Performance Mass Storage and Parallel I/O: Technologies and Applications*. Wiley-IEEE Press. ISBN 978-0471208099.

- Jin, H., Ooi, B.C., Shen, H.T., Yu, C., Zhou, A., 2003. An adaptive and efficient dimensionality reduction algorithm for high-dimensional indexing. In: Dayal, U., Ramamritham, K., Vijayaraman, T.M. (Eds.), *Proceedings of the 19th International Conference on Data Engineering*. Bangalore, India, March 5–8, 2003, pp. 87–98.
- John, T., Misra, P., 2017. *Data Lake for Enterprises: Leveraging Lambda Architecture for building Enterprise Data Lake*. Packt. ISBN 978-1-78728-134-9.
- Johnson, Mark, Ishwar, Prakash, Prabhakaran, Vinod M., Schonberg, Daniel, Ramchandran, Kannan, 2004. On compressing encrypted data. *IEEE Trans. Signal Process.* 52 (10), 2992–3006. <https://doi.org/10.1109/TSP.2004.833860>.
- Jones, R., Hosking, A., Moss, E., 1996. *The Garbage Collection Handbook: The Art of Automatic Memory Management*. CRC Applied Algorithms and Data Structures Series. CRC Press, Taylor & Francis Group, Boca Raton, London, New York. ISBN 978-1420082791.
- Jones, R.E., Lins, R.D., 1996. *Garbage Collection – Algorithms for Automatic Dynamic Memory Management*. Wiley. ISBN 978-0-471-94148-4.
- Joshi, A.D., Ramasubramanian, N., 2015. Comparison of significant issues in multicore cache coherence. In: *Proc. 2015 International Conference on Green Computing and Internet of Things (ICGCIoT)*. Greater Noida, India, pp. 108–112.
- Joshi, G., Soljanin, E., Wornell, G.W., 2017. Efficient redundancy techniques for latency reduction in cloud systems. *TOMPECS 2* (2), 12:1–12:30. <http://doi.acm.org/10.1145/3055281>.
- Joukov, N., Sipek, J., 2008. Greenfs: making enterprise computers greener by protecting them better. In: Sventek, J.S., Hand, S. (Eds.), *Proceedings of the 2008 EuroSys Conference*. Glasgow, Scotland, UK, April 1–4, 2008, pp. 69–80. <http://doi.acm.org/10.1145/1352592.1352600>.
- Joukov, N., Krishnakumar, A.M., Patti, C., Rai, A., Satnur, S., Traeger, A., Zadok, E., 2007. RAIF: redundant array of independent filesystems. In: *24th IEEE Conference on Mass Storage Systems and Technologies (MSST 2007)*. San Diego, California, USA, 24–27 September 2007, pp. 199–214. <http://doi.ieeecomputersociety.org/10.1109/MSST.2007.30>.
- Jouppi, N.P., Young, C., Patil, N., Patterson, D.A., et al., 2017. In-datacenter performance analysis of a tensor processing unit. In: *Proceedings of the 44th Annual International Symposium on Computer Architecture, ISCA 2017*. Toronto, ON, Canada, June 24–28, 2017. ACM, pp. 1–12.
- Jouppi, N.P., Young, C., Patil, N., Patterson, D.A., 2018. A domain-specific architecture for deep neural networks. *Commun. ACM* 61 (9), 50–59. <https://doi.org/10.1145/3154484>.
- Jr., A.M.J., Malek, M., 1988. Survey of software tools for evaluating reliability, availability, and serviceability. *ACM Comput. Surv.* 20 (4), 227–269. <https://doi.org/10.1145/50020.50062>.
- Jr., G.G.L., 1984. An introduction to arithmetic coding. *IBM J. Res. Dev.* 28 (2), 135–149. <https://doi.org/10.1147/rd.282.0135>.
- Jun, S.W., Liu, M., Lee, S., Hicks, J., Ankcorn, J., King, M., Xu, S., Arvind, 2015. Bluedbm: an appliance for big data analytics. In: Marr, D.T., Albonesi, D.H. (Eds.), *Proceedings of the 42nd Annual International Symposium on Computer Architecture*. Portland, OR, USA, June 13–17, 2015. ACM, pp. 1–13.
- Jun, S.W., Liu, M., Lee, S., Hicks, J., Ankcorn, J., King, M., Xu, S., Arvind, 2016. Bluedbm: distributed flash storage for big data analytics. *ACM Trans. Comput. Syst.* 34 (3), 7:1–7:31. <https://doi.org/10.1145/2898996>.
- Kallman, R., Kimura, H., Natkins, J., Pavlo, A., et al., 2008. H-store: a high-performance, distributed main memory transaction processing system. *Proc. VLDB Endow.* 1 (2), 1496–1499. <http://www.vldb.org/pvldb/vol1/1454211.pdf>.
- Kamel, I., Niranjan, T., Ghandeharizadeh, S., 2000. A novel deadline driven disk scheduling algorithm for multi-priority multimedia objects. In: Lomet, D.B., Weikum, G. (Eds.), *Proceedings of the 16th International Conference on Data Engineering*. San Diego, California, USA, February 28–March 3, 2000. IEEE Computer Society, pp. 349–361.
- Kanth, K.V.R., Agrawal, D., Singh, A.K., 1998. Dimensionality reduction for similarity searching in dynamic databases. In: Haas, L.M., Tiwary, A. (Eds.), *SIGMOD 1998, Proceedings ACM SIGMOD International Conference on Management of Data*. Seattle, Washington, USA, June 2–4, 1998. ACM Press, pp. 166–176.
- Kao, H., Páris, J., Schwarz, T.J.E., Long, D.D.E., 2013. A flexible simulation tool for estimating data loss risks in storage arrays. In: *IEEE 29th Symposium on Mass Storage Systems and Technologies, MSST 2013*. May 6–10, 2013, Long Beach, CA, USA, pp. 1–5.
- Karanasos, K., Interlandi, M., Psallidas, F., Sen, R., Park, K., Popivanov, I., Xin, D., Nakandala, S., Krishnan, S., Weimer, M., Yu, Y., Ramakrishnan, R., Curino, C., 2020. Extending relational query processing with ML inference. In: *CIDR 2020, 10th Conference on Innovative Data Systems Research, Online Proceedings*. Amsterdam, The Netherlands, January 12–15, 2020. [www.cidrdb.org](http://cidrdb.org/cidr2020/papers/p24-karanasos-cidr20.pdf). <http://cidrdb.org/cidr2020/papers/p24-karanasos-cidr20.pdf>.
- Kari, H., Saikkonen, H., Lombardi, F., 1994. Detecting latent sector faults in modern SCSI disks. In: Madiseti, V.K., Gelenbe, E., Walrand, J.C. (Eds.), *MASCOTS '94, Proceedings of the Second International Workshop on Modeling, Analysis, and Simulation On Computer and Telecommunication Systems*. January 31–February 2, 1994, Durham, North Carolina, USA, pp. 403–404.
- Kari, H.H., 1997. *Latent Sector Faults and Reliability of Disk Arrays*. PhD thesis. University of Technology, Espoo, Finland. <http://www.tcs.hut.fi/~hhk/>.
- Karlsson, P., 2006. Solaris 10 features in-depth: containers and zfs. http://uk.sun.com/sunnews/events/2006/mar/javauk06/presentations/Peter_Karlsson_Containers_ZFS.pdf.
- Katayama, N., Satoh, S., 1997. The sr-tree: an index structure for high-dimensional nearest neighbor queries. In: Peckham, J. (Ed.), *SIGMOD 1997, Proceedings ACM SIGMOD International Conference on Management of Data*. Tucson, Arizona, USA, May 13–15, 1997, pp. 369–380.
- Katz, R.H., 2010. RAID: a personal recollection of how storage became a system. *IEEE Ann. Hist. Comput.* 32 (4), 82–87. <https://doi.org/10.1109/MAHC.2010.66>.
- Katz, Randy H., Gibson, Garth A., Patterson, David A., 1989. Trends in high performance disk systems. *Proc. IEEE, Special Issue on Supercomputers*, 1842–1858.
- Katz, Randy H., Gibson, Garth A., Patterson, David A., 2001. Trends in high performance disk systems. In: *High Performance Mass Storage and Parallel I/O: Technologies and Applications*. IEEE-Wiley, New York, NY. ISBN 0-471-20809-4, pp. 15–34.
- Kavi, K.M., Buckles, B.P., Bhat, U.N., 1986. A formal definition of data flow graph models. *IEEE Trans. Comput.* 35 (11), 940–948. <https://doi.org/10.1109/TC.1986.1676696>.

- Keedwell, A.D., Denes, J., 2015. *Latin Squares and Their Applications*, 2. North-Holland. ISBN 978-0444635556.
- Keehn, D.G., Lacy, J.O., 1974. VSAM data set design parameters. *IBM Syst. J.* 13 (3), 186–212. <https://doi.org/10.1147/sj.133.0186>.
- Keeton, K., 2017. Memory-driven computing - Keynote slides & audio. In: Kuenning, G., Waldspurger, C.A. (Eds.), 15th USENIX Conference on File and Storage Technologies, FAST 2017. Santa Clara, CA, USA, February 27–March 2, 2017. USENIX Association.
- Keeton, K., Patterson, D.A., Hellerstein, J.M., 1998. A case for intelligent disks (idisks). *SIGMOD Rec.* 27 (3), 42–52. <http://doi.acm.org/10.1145/290593.290602>.
- Kejriwal, A., Gopalan, A., Gupta, A., Jia, Z., Yang, S., Ousterhout, J.K., 2016. SLIK: scalable low-latency indexes for a key-value store. In: Gulati, A., Weatherspoon, H. (Eds.), 2016 USENIX Annual Technical Conference, USENIX ATC 2016. Denver, CO, USA, June 22–24, 2016. USENIX Association, pp. 57–70. <https://www.usenix.org/conference/atc16/technical-sessions/presentation/kejriwal>.
- Kemme, B., Peris, R.J., Patio-Martínez, M., 2010. *Database Replication*. ISBN 9781608453818. www.morganclaypool.com.
- Kemper, A., Neumann, T., 2011. Hyper: a hybrid oltp&olap main memory database system based on virtual memory snapshots. In: Abiteboul, S., Böhm, K., Koch, C., Tan, K. (Eds.), *Proceedings of the 27th International Conference on Data Engineering, ICDE 2011*. April 11–16, 2011, Hannover, Germany. IEEE Computer Society, pp. 195–206.
- Kenchammana-Hosekote, D., Golding, R.A., Fleiner, C., Zaki, O., 2004. The design and evaluation of network RAID protocols. Technical Report RJ 10316. IBM Almaden Research Center.
- Kenchammana-Hosekote, D.R., He, D., Hafner, J.L., 2007. REO: a generic RAID engine and optimizer. In: Arpaci-Dusseau, A.C., Arpaci-Dusseau, R.H. (Eds.), 5th USENIX Conference on File and Storage Technologies, FAST 2007. February 13–16, 2007, San Jose, CA, USA. USENIX, pp. 261–276. <http://www.usenix.org/events/fast07/tech/kenchammana.html>.
- Kermani, P., Kleinrock, L., 1979. Virtual cut-through: a new computer communication switching technique. *Comput. Netw.* 3, 267–286. [https://doi.org/10.1016/0376-5075\(79\)90032-1](https://doi.org/10.1016/0376-5075(79)90032-1).
- Khan, O., Burns, R.C., Plank, J.S., Pierce, W., Huang, C., 2012. Rethinking erasure codes for cloud file systems: minimizing I/O for recovery and degraded reads. In: Bolosky, W.J., Flinn, J. (Eds.), *Proceedings of the 10th USENIX Conference on File and Storage Technologies, FAST 2012*. San Jose, CA, USA, February 14–17, 2012. USENIX Association, p. 20. <https://www.usenix.org/conference/fast12/rethinking-erasure-codes-cloud-file-systems-minimizing-io-recovery-and-degraded>.
- Kidder, T., 2000. *The Soul of a New Machine*. Back Bay Books. ISBN 978-0316491976.
- Kilburn, T., Edwards, D.B.G., Lanigan, M.J., Sumner, F.H., 1962. One-level storage system. *IRE Trans. Electron. Comput.* 11 (2), 223–235. <https://doi.org/10.1109/TEC.1962.5219356>.
- Kim, B.S., Park, S.B., 1986. A fast k nearest neighbor finding algorithm based on the ordered partition. *IEEE Trans. Pattern Anal. Mach. Intell.* 8 (6), 761–766. <https://doi.org/10.1109/TPAMI.1986.4767859>.
- Kim, D.W., Lee, S., Park, C., 2004. Balanced RM2: an improved data placement scheme for tolerating double disk failures in disk arrays. In: Bubak, M., van Albada, G.D., Sloot, P.M.A., Dongarra, J.J. (Eds.), *Computational Science – ICCS 2004*, 4th International Conference, Proceedings, Part III. Kraków, Poland, June 6–9, 2004. In: *Lecture Notes in Computer Science*, vol. 3038. Springer, pp. 371–378.
- Kim, H., Lee, S., 1999. A new flash memory management for flash storage system. In: 23rd International Computer Software and Applications Conference (COMPSAC '99). 27–19 October 1999, Phoenix, AZ, USA. IEEE Computer Society, pp. 284–289.
- Kim, H., Seshadri, S., Dickey, C.L., Chiu, L., 2014. Evaluating phase change memory for enterprise storage systems: a study of caching and tiering approaches. *ACM Trans. Storage* 10 (4), 15:1–15:21. <https://doi.org/10.1145/2668128>.
- Kim, M.Y., 1985. Parallel operation of magnetic disk storage devices: synchronized disk interleaving. In: DeWitt, D.J., Boral, H. (Eds.), *Database Machines, Fourth International Workshop*. Grand Bahama Island, March 1985. Springer, pp. 300–330.
- Kim, M.Y., 1986. Synchronized disk interleaving. *IEEE Trans. Comput.* 35 (11), 978–988. <https://doi.org/10.1109/TC.1986.1676699>.
- Kim, M.Y., Tantawi, A.N., 1991. Asynchronous disk interleaving: approximating access delays. *IEEE Trans. Comput.* 40 (7), 801–810. <https://doi.org/10.1109/12.83618>.
- Kim, P., Suk, M., 2007. Ramp load/unload technology in hard disk drives.
- Kim, S., Lee, S., Kim, T., Huh, J., 2017. Transparent dual memory compression architecture. In: 26th International Conference on Parallel Architectures and Compilation Techniques, PACT 2017. Portland, OR, USA, September 9–13, 2017. IEEE Computer Society, pp. 206–218.
- Kim, W., 1990. *Introduction to Object-Oriented Databases*. The MIT Press. ISBN 0-262-11124-1.
- Kim, Y., Gurumurthi, S., Sivasubramaniam, A., 2006. Understanding the performance-temperature interactions in disk I/O of server workloads. In: 12th International Symposium on High-Performance Computer Architecture, HPCA-12 2006. Austin, Texas, USA, February 11–15, 2006. IEEE Computer Society, pp. 176–186.
- King, R.P., 1990. Disk arm movement in anticipation of future requests. *ACM Trans. Comput. Syst.* 8 (3), 214–229. <http://doi.acm.org/10.1145/99926.99930>.
- King III, W. Frank, 1971. Analysis of demand paging algorithms. In: *IFIP Congress* (1), pp. 485–490.
- Kinzer, S., 2004. *All the Shah's Men: An American Coup and the Roots of Middle East Terror*. Wiley. ISBN 978-0471678786.
- Kirkpatrick, S., Jr., D.G., Vecchi, M.P., 1983. Optimization by simulated annealing. *Science* 220 (4598), 671–680.
- Kitsuregawa, M., Ogawa, Y., 1990. Bucket spreading parallel hash: a new, robust, parallel hash join method for data skew in the super database computer (SDC). In: McLeod, D., Sacks-Davis, R., Schek, H. (Eds.), 16th International Conference on Very Large Data Bases, Proceedings. August 13–16, 1990, Brisbane, Queensland, Australia. Morgan Kaufmann, pp. 210–221. <http://www.vldb.org/conf/1990/P210.PDF>.
- Kitsuregawa, M., Tanaka, H. (Eds.), 1988. *Database Machines and Knowledge Base Machines*, 5th International Workshop on Database Machines, Proceedings. Tokyo, Japan, 1987. Engineering and Computer Science, vol. 43. Kluwer/Springer. ISBN 978-1-4612-8948-7.

- Kleinrock, L., 1974. Resource allocation in computer systems and computer-communication networks. In: Rosenfeld, J.L. (Ed.), *Information Processing, Proceedings of the 6th IFIP Congress 1974*. Stockholm, Sweden, August 5–10, 1974. North-Holland, pp. 11–18.
- Kleinrock, L., 1975. *Queueing Systems, Vol. I: Theory*, 1. Wiley-Interscience, New York, NY. ISBN 978-0471491101.
- Kleinrock, L., 1976. *Queueing Systems, Vol. II: Computer Applications*. Wiley-Interscience, New York, NY. ISBN 978-0471491118.
- Kline, Demijan, Hazay, Carmit, Jagmohan, Ashish, Krawczyk, Hugo, Rabin, Tal, 2012. On compression of data encrypted with block ciphers. *IEEE Trans. Inf. Theory* 58 (11), 6989–7001. <https://doi.org/10.1109/TIT.2012.2210752>.
- Knuth, D.E., 1997a. *The Art of Computer Programming-Seminumerical Algorithms*, 3, Volume 2. Addison-Wesley, Boston, MA. ISBN 0201896842.
- Knuth, D.E., 1997b. *The Art of Computer Programming, Volume I: Fundamental Algorithms*, 3rd edition. Addison-Wesley. ISBN 0201896834.
- Knuth, D.E., 2005. *The Art of Computer Programming-Sorting and Searching*, 2, Volume 3. Addison-Wesley, Boston, MA. ISBN 0201853949.
- Kobayashi, H., 1978. *Modeling and Analysis: An Introduction to System Performance Evaluation Methodology*. Systems Programming Series. Addison-Wesley. ISBN 978-0201144574.
- Kobayashi, H., Mark, B.L., 2008. *System Modeling and Analysis: Foundations of System Performance Evaluation*. Pearson. ISBN 978-0130348357.
- Kobayashi, H., Mark, B.L., Turin, W., 2014. *Probability, Random Processes, and Statistical Analysis: Applications to Communications, Signal Processing, Queueing Theory and Mathematical Finance*. Cambridge University Press. ISBN 978-0521895446.
- Kogan, J., 2006. *Introduction to Clustering Large and High Dimensional Data*. Cambridge University Press. ISBN 978-0521617932.
- Kogge, P.M., 1981. *The Architecture of Pipelined Computers*, 1. McGraw-Hill, New York, NY. ISBN 978-0-07-035237-7.
- Kohavi, Z., Jha, N.K., 2009. *Switching and Finite Automata Theory*, 3. Cambridge University Press. ISBN 978-0521857482.
- Konheim, A.G., 2010. *Hashing in Computer Science: Fifty Years of Slicing and Dicing*. John Wiley and Sons. ISBN 978-0470344736.
- Kooi, R., 1980. *The Optimization of Queries in Relational Databases*. PhD thesis. Case Western Reserve University, Cleveland, OH. Advised by A. Thomasian.
- Koren, I., 2002. *Computer Arithmetic Algorithms*, 2. A. K. Peters, Natick, MA. ISBN 1568811608.
- Koren, I., Krishna, C.M., 2020. *Fault-Tolerant Systems*, 2. Morgan-Kaufmann. ISBN 978-0128181058.
- Korn, F., Jagadish, H.V., Faloutsos, C., 1997. Efficiently supporting ad hoc queries in large datasets of time sequences. In: Peckham, J. (Ed.), *SIGMOD 1997, Proceedings ACM SIGMOD International Conference on Management of Data*. Tucson, Arizona, USA, May 13–15, 1997. ACM Press, pp. 289–300.
- Korn, F., Sidiropoulos, N.D., Faloutsos, C., Siegel, E.L., Protopapas, Z., 1998. Fast and effective retrieval of medical tumor shapes. *IEEE Trans. Knowl. Data Eng.* 10 (6), 889–904. <https://doi.org/10.1109/69.738356>.
- Korzun, D., Gurtov, A., 2012. *Structured Peer-to-Peer Systems: Fundamentals of Hierarchical Organization, Routing, Scaling, and Security*. Springer. ISBN 978-1461454823.
- Kotla, R., Alvisi, L., Dahlin, M., 2007. Safestore: a durable and practical storage system. In: Chase, J., Seshan, S. (Eds.), *Proceedings of the 2007 USENIX Annual Technical Conference*. Santa Clara, CA, USA, June 17–22, 2007. USENIX, pp. 129–142. <http://www.usenix.org/events/usenix07/tech/kotla.html>.
- Kou, L.T., Markowsky, G., 1977. Multidimensional bin packing algorithms. *IBM J. Res. Dev.* 21 (5), 443–448. <https://doi.org/10.1147/rd.215.0443>.
- Kralevska, K., Gligoroski, D., Jensen, R.E., Øverby, H., 2018. Hashtag erasure codes: from theory to practice. *IEEE Trans. Big Data* 4 (4), 516–529. <https://doi.org/10.1109/TBDDATA.2017.2749255>. <https://arxiv.org/abs/1609.02450>.
- Krawczyk, H., 1993. Secret sharing made short. In: Stinson, D.R. (Ed.), *Advances in Cryptology – CRYPTO ’93, 13th Annual International Cryptology Conference, Proceedings*. Santa Barbara, California, USA, August 22–26, 1993. In: *Lecture Notes in Computer Science*, vol. 773. Springer, pp. 136–146.
- Kriegel, H., Kröger, P., Zimek, A., 2009. Clustering high-dimensional data: a survey on subspace clustering, pattern-based clustering, and correlation clustering. *ACM Trans. Knowl. Discov. Data* 3 (1), 1:1–1:58. <https://doi.org/10.1145/1497577.1497578>.
- Krikelis, A., Weems, C.C. (Eds.), 1997. *Associative Processing and Processors*. IEEE Computer Society. ISBN 978-0818676611.
- Krishnan, M.N., Prakash, N., Lalitha, V., Sasidharan, B., Kumar, P.V., Narayanamurthy, S., Kumar, R., Nandi, S., 2014. Evaluation of codes with inherent double replication for hadoop. In: Swanson, S. (Ed.), *6th USENIX Workshop on Hot Topics in Storage and File Systems, HotStorage ’14*. Philadelphia, PA, USA, June 17–18, 2014. USENIX Association. <https://www.usenix.org/conference/hotstorage14/workshop-program/presentation/krishnan>.
- Krizhevsky, A., Sutskever, I., Hinton, G.E., 2017. Imagenet classification with deep convolutional neural networks. *Commun. ACM* 60 (6), 84–90. <http://doi.acm.org/10.1145/3065386>.
- Kronenberg, N.P., Levy, H.M., Strecker, W.D., 1986. Vaxclusters: a closely-coupled distributed system. *ACM Trans. Comput. Syst.* 4 (2), 130–146. <https://doi.org/10.1145/214419.214421>.
- Kruus, E., Ungureanu, C., Dubnicki, C., 2010. Bimodal content defined chunking for backup streams. In: Burns, R.C., Keeton, K. (Eds.), *8th USENIX Conference on File and Storage Technologies*. San Jose, CA, USA, February 23–26, 2010. USENIX, pp. 239–252. http://www.usenix.org/events/fast10/tech/full_papers/kruus.pdf.
- Kryder, M.H., Kim, C.S., 2009. After hard disk drives – what comes next. *IEEE Trans. Magn.* 45 (10), 3406–3413. <https://doi.org/10.1109/TIP.2017.2751960>.
- Kuck, D.J., 1979. *Structure of Computers and Computations*. Structure of Computers & Computations, vol. 1. John Wiley & Sons. ISBN 978-0471027164.
- Kuck, D.J., Stokes, R.A., 1982. The Burroughs scientific processor (bsp). *IEEE Trans. Comput.* 31 (5), 363–376.
- Kumar, K., Roy, S., PauloDavim, J., 2019. *SoftComputing Techniques for Engineering Optimization*. CRC Press. ISBN 978-0429053641.
- Kung, H.T., 1986. Memory requirements for balanced computer architectures. In: Aiso, H. (Ed.), *Proceedings of the 13th Annual Symposium on Computer Architecture*. Tokyo, Japan, June 1986. IEEE Computer Society, pp. 49–54. <https://dl.acm.org/citation.cfm?id=17362>.

- Kung, H.T., Robinson, J.T., 1981. On optimistic methods for concurrency control. *ACM Trans. Database Syst.* 6 (2), 213–226. <https://doi.org/10.1145/319566.319567>.
- Kuratti, A., Sanders, W.H., 1995. Performance analysis of the raid5 disk array. In: *Proc. IEEE Int'l Computer Performance and Dependability Symp.* IEEE, Springer, New York, NY, pp. 236–245.
- Kurinec, S.K., Iniewski, K. (Eds.), 2013. *Nanoscale Semiconductor Memories: Technology and Applications*, 1. Devices, Circuits, and Systems. CRC Press, Boca Raton, FL. ISBN 978-1466560604.
- Kurtas, E.M., Vasic, B., 2006. *Advanced Error Control Techniques for Data Storage Systems*. CRC Press. ISBN 0-9493-9457-X.
- Kurzweil Jr., Fred, 1988. Small disk arrays - the emerging approach to high performance. In: *COMPCON-88 The 34th IEEE Computer Society Int'l Conf.* San Francisco, CA, March 1988. IEEE Computer Society.
- Kwok, Y.-K.R., 2011. *Peer-to-Peer Computing: Applications, Architecture, Protocols, and Challenges*. CRC Computational Science. Chapman & Hall. ISBN 978-1439809358.
- Kwon, S.J., Ranjekar, A., Ko, Y., Chung, T., 2011. FTL algorithms for nand-type flash memories. *Des. Autom. Embed. Syst.* 15 (3–4), 191–224. <https://doi.org/10.1007/s10617-011-9071-9>.
- Laden, G., Ta-Shma, P., Yaffe, E., Factor, M., Fienblit, S., 2007. Architectures for controller based CDP. In: *Arpaci-Dusseau, A.C., Arpaci-Dusseau, R.H. (Eds.), 5th USENIX Conference on File and Storage Technologies, FAST 2007*. February 13–16, 2007, San Jose, CA, USA. USENIX, pp. 107–121. <http://www.usenix.org/events/fast07/tech/laden.html>.
- Lahiri, T., Neimat, M., Folkman, S., 2013. Oracle timesten: an in-memory database for enterprise applications. *IEEE Data Eng. Bull.* 36 (2), 6–13. <http://sites.computer.org/debull/A13june/TimesTen1.pdf>.
- Lahiri, T., Chavan, S., Colgan, M., Das, D., et al., 2015. Oracle database in-memory: a dual format in-memory database. In: *Gehrke, J., Lehner, W., Shim, K., et al. (Eds.), 31st IEEE International Conference on Data Engineering, ICDE 2015*. Seoul, South Korea, April 13–17, 2015. IEEE Computer Society, pp. 1253–1258.
- Lamb, C., Landis, G., Orenstein, J.A., Weinreb, D., 1991. The objectstore database system. *Commun. ACM* 34 (10), 50–63. <https://doi.org/10.1145/125223.125244>.
- Lamport, L., 2002. Paxos made simple, fast, and byzantine. In: *Bui, A., Fouchal, H. (Eds.), Proceedings of the 6th International Conference on Principles of Distributed Systems, OPODIS 2002*. Reims, France, December 11–13, 2002. In: *Studia Informatica Universalis*, vol. 3. Suger, Saint-Denis, rue Catulienne, France, pp. 7–9.
- Lamport, L., 2005. Fast paxos. Technical Report MSR-TR-2005-112. Microsoft Research.
- Lang, T., Nahouraii, E., Kasuga, K., Fernández, E.B., 1977a. An architectural extension for a large database system incorporating a processor for disk search. In: *Proceedings of the Third International Conference on Very Large Data Bases*. Tokyo, Japan, October 6–8, 1977. IEEE Computer Society, pp. 204–210.
- Lang, T., Wood, C., Fernández, E.B., 1977b. Database buffer paging in virtual storage systems. *ACM Trans. Database Syst.* 2 (4), 339–351. <https://doi.org/10.1145/320576.320585>.
- Langdon, G., 1978. A note on associative processors for data management. *ACM Trans. Database Syst.* 3 (2), 148–158. <https://doi.org/10.1145/320251.320254>.
- Lantz, M., 2018. Why the future of data storage is (still) magnetic tape. *IEEE Spectr.* <https://spectrum.ieee.org/computing/hardware/why-the-future-of-data-storage-is-still-magnetic-tape>.
- Larson, D.D., Young, J.R., Studebaker, T.J., Craybill, C.L., 1987. Storagetek 4400 automated cartridge system. In: *Eight IEEE Symposium on Mass Storage Systems, MSS 1987*. Tucson, AZ, USA, May 11–14, 1987, pp. 61–67.
- Larson, P., Blanas, S., Diaconu, C., Freedman, C., Patel, J.M., Zwilling, M., 2011. High-performance concurrency control mechanisms for main-memory databases. *Proc. VLDB Endow.* 5 (4), 298–309. <https://doi.org/10.14778/2095686.2095689>. http://vldb.org/pvldb/vol5/p298_per-akelarsen_vldb2012.pdf.
- Lascu, O., White, B., Troy, J., et al., 2020. IBM z15 (8561) technical guide. Technical Report SG24-8851-00. IBM. <https://www.redbooks.ibm.com/redbooks/pdfs/sg248851.pdf>.
- Lassnig, M., Barisits, M., Christidis, D., 2020. Experiences in exascale scientific data management. *IEEE Data Eng. Bull.* 43 (1), 9–22. <http://sites.computer.org/debull/A20mar/p9.pdf>.
- Lastras-Montañó, Luis Alfonso, Meaney, Patrick J., Stephens, Eldee, Trager, Barry M., O'Connor, James A., Alves, Luiz C., 2011. A new class of array codes for memory storage. In: *Information Theory and Applications Workshop, ITA 2011*. San Diego. IEEE, pp. 223–232. <https://doi.org/10.1109/ITA.2011.5743586>.
- Latham, R., Miller, N., Ross, R., Carns, P., 2004. A next-generation parallel file system for Linux clusters: an introduction to the second parallel virtual file system. *Linux World Mag.*, 56–59.
- Lau, E., Madden, S., 2006. An integrated approach to recovery and high availability in an updatable, distributed data warehouse. In: *Dayal, U., Whang, K., Lomet, D.B., et al. (Eds.), Proceedings of the 32nd International Conference on Very Large Data Bases*. Seoul, Korea, September 12–15, 2006. ACM, pp. 703–714. <http://dl.acm.org/citation.cfm?id=1164188>.
- Lau, E.J., Ferrari, D., 1983. Program restructuring in a multilevel virtual memory. *IEEE Trans. Softw. Eng.* 9 (1), 69–79. <https://doi.org/10.1109/TSE.1983.236296>.
- Lavenberg, S.S. (Ed.), 1983. *Computer Performance Modeling Handbook*. Notes and Reports in Computer Science and Applied Mathematics. Academic Press, New York, NY. ISBN 978-0124387201.
- Lawlor, F.D., 1981. Efficient Mass Storage Parity Recovery Mechanism. Technical Disclosure Bulletin 23.2. IBM, Yorktown Heights, NY, pp. 986–987.
- Laywine, C.F., Mullen, G.L., 1998. *Discrete Mathematics Using Latin Squares*. Discrete Mathematics and Optimization. Wiley. ISBN 9780471240648.

- Lazowska, E.D., Zahorjan, J., Graham, G.S., Sevcik, K.C., 1984. Quantitative System Performance – Computer System Analysis Using Queueing Network Models. Prentice Hall, Upper Saddle River, NJ. ISBN 978-0-13-746975-8. <http://dl.acm.org/citation.cfm?id=2971>.
- Le, T., 2014. Optimizing storage solutions using the Intel intelligent storage acceleration library. Technical report. Intel.
- Lebeck, A.R., Soh, G.S., 1994. Request combining in multiprocessors with arbitrary interconnection networks. *IEEE Trans. Parallel Distrib. Syst.* 5 (11), 1140–1155. <https://doi.org/10.1109/71.329673>.
- LeCun, Y., Bengio, Y., Hinton, G.E., 2015. Deep learning. *Nature* 521 (7553), 436–444. <https://doi.org/10.1038/nature14539>.
- Lee, C., Park, S.J., Kejriwal, A., Matsushita, S., Ousterhout, J.K., 2015. Implementing linearizability at large scale and low latency. In: Miller, E.L., Hand, S. (Eds.), *Proceedings of the 25th Symposium on Operating Systems Principles, SOSP 2015*. Monterey, CA, USA, October 4–7, 2015. ACM, pp. 71–86.
- Lee, E.K., Katz, R.H., 1993a. The performance of parity placements in disk arrays. *IEEE Trans. Comput.* 42 (6), 651–664. <https://doi.org/10.1109/12.277289>.
- Lee, E.K., Katz, R.H., 1993b. An analytic performance model of disk arrays. In: Owicki, S.S., Muntz, R.R. (Eds.), *Proceedings of the 1993 ACM SIGMETRICS Conference on Measurement and Modeling of Computer Systems*. Santa Clara, California, USA, May 10–14, 1993. ACM, pp. 98–109.
- Lee, E.K., Thekkath, C.A., 1996. Petal: distributed virtual disks. In: Dally, B., Eggers, S.J. (Eds.), *ASPLOS-VII Proceedings – Seventh International Conference on Architectural Support for Programming Languages and Operating Systems*. Cambridge, Massachusetts, USA, October 1–5, 1996. ACM Press, pp. 84–92.
- Lee, L., Scheuermann, P., Vingralek, R., 2000. File assignment in parallel I/O systems with minimal variance of service time. *IEEE Trans. Comput.* 49 (2), 127–140. <https://doi.org/10.1109/12.833109>.
- Lee, M.K., Freytag, J.C., Lohman, G.M., 1988. Implementing an interpreter for functional rules in a query optimizer. In: Bancilhon, F., DeWitt, D.J. (Eds.), *Fourteenth International Conference on Very Large Data Bases, Proceedings*. August 29–September 1, 1988, Los Angeles, California, USA. Morgan Kaufmann, pp. 218–229. <http://www.vldb.org/conf/1988/P218.PDF>.
- Leilich, H., Missikoff, M. (Eds.), 1983. *Database Machines, International Workshop*. Munich, Germany, September 1983. Springer. ISBN 3-540-12959-6.
- Leinberger, W., Karypis, G., Kumar, V., 1999. Multi-capacity bin packing algorithms with applications to job scheduling under multiple constraints. In: *Proceedings of the International Conference on Parallel Processing 1999, ICPP 1999*. Wakamatsu, Japan, September 21–24, 1999. IEEE Computer Society, pp. 404–412.
- Levandoski, J.J., Lomet, D.B., Sengupta, S., 2013. LLAMA: a cache/storage subsystem for modern hardware. *Proc. VLDB Endow.* 6 (10), 877–888. <http://www.vldb.org/pvldb/vol6/p877-levandoski.pdf>.
- Levandoski, J.J., Lomet, D.B., Sengupta, S., Stutsman, R., Wang, R., 2015a. High performance transactions in deuteronomy. In: *CIDR 2015, Seventh Biennial Conference on Innovative Data Systems Research, Online Proceedings*. Asilomar, CA, USA, January 4–7, 2015. [www.cidrdb.org](http://cidrdb.org/cidr2015/Papers/CIDR15_Paper15.pdf). http://cidrdb.org/cidr2015/Papers/CIDR15_Paper15.pdf.
- Levandoski, J.J., Sengupta, S., Stutsman, R., Wang, R., 2015b. Transaction processing techniques for modern hardware and the cloud. *IEEE Data Eng. Bull.* 38 (1), 50–57. <http://sites.computer.org/debull/A15mar/p50.pdf>.
- Leventhal, A., 2009. Triple-parity RAID and beyond. *ACM Queue* 7 (11), 30. <http://doi.acm.org/10.1145/1661785.1670144>.
- Lewis, N., 2017. Purchasing power: rivalry, dissent, and computing strategy in supercomputer selection at Los Alamos. *IEEE Ann. Hist. Comput.* 39 (3), 25–40. <https://doi.org/10.1109/MAHC.2017.3481344>.
- Li, F., 2019. Cloud native database systems at Alibaba: opportunities and challenges. *Proc. VLDB Endow.* 12 (12), 2263–2272. <http://www.vldb.org/pvldb/vol12/p2263-li.pdf>.
- Li, H., Chen, Y., 2011. *Nonvolatile Memory Design: Magnetic, Resistive, and Phase Change*. CRC Press, Boca Raton, FL. ISBN 978-1439807453.
- Li, J., Li, B., 2015. Beehive: erasure codes for fixing multiple failures in distributed storage systems. In: Salem, K., Strunk, J.D. (Eds.), *7th USENIX Workshop on Hot Topics in Storage and File Systems, HotStorage 2015*. Santa Clara, CA, USA, July 6–7, 2015. USENIX Association. <https://www.usenix.org/conference/hotstorage15/workshop-program/presentation/li>.
- Li, J., Li, B., 2017. Beehive: erasure codes for fixing multiple failures in distributed storage systems. *IEEE Trans. Parallel Distrib. Syst.* 28 (5), 1257–1270. <https://doi.org/10.1109/TPDS.2016.2623309>.
- Li, M., Lee, P.P.C., 2014. Toward i/o-efficient protection against silent data corruptions in RAID arrays. In: *IEEE 30th Symposium on Mass Storage Systems and Technologies, MSST 2014*. Santa Clara, CA, USA, June 2–6, 2014. IEEE Computer Society, pp. 1–12.
- Li, M., Shu, J., 2010. DACO: a high-performance disk architecture designed specially for large-scale erasure-coded storage systems. *IEEE Trans. Comput.* 59 (10), 1350–1362. <https://doi.org/10.1109/TC.2010.22>.
- Li, M., Shu, J., Zheng, W., 2009. GRID codes: strip-based erasure codes with high fault tolerance for storage systems. *ACM Trans. Storage* 4 (4), 15:1–15:22. <https://doi.org/10.1145/1480439.1480444>.
- Li, Q., Shi, L., Xue, C.J., Wu, K., Ji, C., Zhuge, Q., Sha, E.H.-M., 2016. Access characteristic guided read and write cost regulation for performance improvement on flash memory. In: *14th USENIX Conference on File and Storage Technologies (FAST 16)*. USENIX, Santa Clara, CA. ISBN 978-1-931971-28-7, pp. 125–132. <https://www.usenix.org/conference/fast16/technical-sessions/presentation/li-qiao>.
- Li, X., Li, Z., Zhou, Y., Adve, S.V., 2005. Performance directed energy management for main memory and disks. *ACM Trans. Storage* 1 (3), 346–380. <http://doi.acm.org/10.1145/1084779.1084782>.
- Li, Y., Long, D.D.E., 2014. Which storage device is the greenest? modeling the energy cost of I/O workloads. In: *IEEE 22nd International Symposium on Modelling, Analysis & Simulation of Computer and Telecommunication Systems, MASCOTS 2014*. Paris, France, September 9–11, 2014. IEEE Computer Society, pp. 100–105.
- Li, Y., Dhotre, N.S., Ohara, Y., Kroeger, T.M., Miller, E.L., Long, D.D.E., 2013. Horus: fine-grained encryption-based security for large-scale storage. In: Smith, K.A., Zhou, Y. (Eds.), *Proceedings of the 11th USENIX Conference on File and Storage Technologies, FAST 2013*. San Jose, CA, USA, February 12–15, 2013. USENIX, pp. 147–160. https://www.usenix.org/conference/fast13/technical-sessions/presentation/li_yan.

- Liao, X., Bai, S., Wang, Y., Hu, S., 2011. Isra-based grouping: a disk reorganization approach for disk energy conservation and disk performance enhancement. *IEEE Trans. Comput.* 60 (2), 292–304. <https://doi.org/10.1109/TC.2010.153>.
- Lillibridge, M., Eshghi, K., Bhagwat, D., Deolalikar, V., Trezis, G., Camble, P., 2009. Sparse indexing: large scale, inline deduplication using sampling and locality. In: Seltzer, M.I., Wheeler, R. (Eds.), 7th USENIX Conference on File and Storage Technologies, Proceedings. February 24–27, 2009, San Francisco, CA, USA. USENIX, pp. 111–123. http://www.usenix.org/events/fast09/tech/full_papers/lillibridge/lillibridge.pdf.
- Lim, K.T., Chang, J., Mudge, T.N., Ranganathan, P., Reinhardt, S.K., Wenisch, T.F., 2009. Disaggregated memory for expansion and sharing in blade servers. In: Keckler, S.W., Barroso, L.A. (Eds.), 36th International Symposium on Computer Architecture (ISCA 2009). Austin, TX, USA, June 20–24, 2009. ACM, pp. 267–278.
- Lim, K.T., Turner, Y., Santos, J.R., AuYoung, A., Chang, J., Ranganathan, P., Wenisch, T.F., 2012. System-level implications of disaggregated memory. In: 18th IEEE International Symposium on High Performance Computer Architecture, HPCA 2012. New Orleans, LA, USA, 25–29 February. IEEE Computer Society, 2012, pp. 189–200.
- Lin, C.S., Smith, D.C.P., Smith, J.M., 1975. The design of a rotating associative array memory for a relational database management application. In: Kerr, D.S. (Ed.), Proceedings of the International Conference on Very Large Data Bases. Framingham, Massachusetts, USA, September 22–24, 1975. ACM, pp. 453–455.
- Lin, K., Jagadish, H.V., Faloutsos, C., 1994. The TV-tree: an index structure for high-dimensional data. *VLDB J.* 3 (4), 517–542. <http://www.vldb.org/journal/VLDBJ3/P517.pdf>.
- Lin, T., Tarn, W., 1991. Scheduling periodic and aperiodic tasks in hard real-time computing systems. In: Keller, T.W. (Ed.), Proceedings of the 1991 ACM SIGMETRICS Conference on Measurement and Modeling of Computer Systems. San Diego, California, USA, May 21–24, 1991. ACM, pp. 31–38.
- Linde, Y., Buzo, A., Gray, R., 1980. An algorithm for vector quantizer design. *IEEE Trans. Commun.* 28 (1), 84–95. <https://doi.org/10.1109/TCOM.1984.1095956>.
- Lindström, J., Raatikka, V., Ruuth, J., Soini, P., Vakkila, K., 2013. IBM soliddb: in-memory database optimized for extreme speed and availability. *IEEE Data Eng. Bull.* 36 (2), 14–20. <http://sites.computer.org/debull/A13june/soliddb1.pdf>.
- Lindstrom, P., Isenburt, M., 2006. Fast and efficient compression of floating-point data. *IEEE Trans. Vis. Comput. Graph.* 12 (5), 1245–1250. <https://doi.org/10.1109/TVCG.2006.143>.
- Liptay, J.S., 1968. Structural aspects of the system/360 model 85 II: the cache. *IBM Syst. J.* 7 (1), 15–21. <https://doi.org/10.1147/sj.71.0015>.
- Liu, C.L., Layland, J.W., 1973. Scheduling algorithms for multiprogramming in a hard-real-time environment. *J. ACM* 20 (1), 46–61. <http://doi.acm.org/10.1145/321738.321743>.
- Liu, L., Özsu, M.T. (Eds.), 2018. Encyclopedia of Database Systems, Second Edition. Springer. ISBN 978-1-4614-8266-6.
- Livny, M., Khoshafian, S., Boral, H., 1986. Multi-disk management algorithms. *IEEE Database Eng. Bull.* 9 (1), 24–36. <http://sites.computer.org/debull/86MAR-CD.pdf>.
- Lo, R.W., Matloff, N.S., 1992. A probabilistic limit on the virtual size of replicated disk systems. *IEEE Trans. Knowl. Data Eng.* 4 (1), 99–102. <https://doi.org/10.1109/69.124901>.
- Lohman, G.M., Mohan, C., Haas, L.M., Daniels, D., Lindsay, B.G., Selinger, P.G., Wilms, P.F., 1985. Query processing in r*. In: Kim, W., Reiner, D.S., Batory, D.S. (Eds.), Query Processing in Database Systems. Springer, pp. 31–47.
- Lorie, R.A., Daudenarde, J., Hallmark, G., Stamos, J.W., Young, H.C., 1989. Adding intra-transaction parallelism to an existing DBMS: early experience. *IEEE Data Eng. Bull.* 12 (1), 2–8. <http://sites.computer.org/debull/89MAR-CD.pdf>.
- Lorie, R.A., Daudenarde, J., Stamos, J.W., Young, H.C., 1991. Exploiting database parallelism in a message-passing multiprocessor. *IBM J. Res. Dev.* 35 (5), 681–696. <https://doi.org/10.1147/rd.355.0681>.
- Lotfi, N., 2019. Data allocation in distributed database systems: a novel hybrid method based on differential evolution and variable neighborhood search. *Springer Nat. Appl. Sci.* 1 (12). <https://link.springer.com/content/pdf/10.1007%2Fs42452-019-1787-3.pdf>.
- Lotfi-Kamran, Pejman, Sarbazi-Azad, Hamid, 2022. Data Prefetching Techniques in Computer Systems. *Advances in Computers*, vol. 125. Academic Press. ISBN 9780323851190.
- Lovelace, M., Davidauska, J., Satia, A., Sokal, V., 2013. VSAM demystified. Technical report. Technical Support Organization, Poughkeepsie Center. <https://books.google.com/books?vid=ISBN:9780738437439>. www.redbooks.ibm.com.
- Lowe, S.D., 2015. A Gorilla Guide to Hyperconverged Infrastructure for Data Center Architects. Actual Tech Media, Bluffton, SC. ISBN 978-1-943952-02-1.
- Lu, G., Jin, Y., Du, D.H.C., 2010. Frequency based chunking for data de-duplication. In: MASCOTS 2010, 18th Annual IEEE/ACM International Symposium on Modeling, Analysis and Simulation of Computer and Telecommunication Systems. Miami, Florida, USA, August 17–19, 2010. IEEE Computer Society, pp. 287–296.
- Lu, L., Varman, P.J., Wang, J., 2007. Diskgroup: energy efficient disk layout for RAID1 systems. In: International Conference on Networking, Architecture, and Storage, NAS 2007. 29–31 July 2007, Guilin, China. IEEE Computer Society, pp. 233–242.
- Lu, S., Luo, B., Patel, T., Yao, Y., Tiwari, D., Shi, W., 2020. Making disk failure predictions smarter! In: Noh, S.H., Welch, B. (Eds.), 18th USENIX Conference on File and Storage Technologies, FAST 2020. Santa Clara, CA, USA, February 24–27, 2020. USENIX Association, pp. 151–167. <https://www.usenix.org/conference/fast20/presentation/lu>.
- Lua, E.K., Crowcroft, J., Pias, M., Sharma, R., Lim, S., 2004. A survey and comparison of peer-to-peer overlay network schemes. *IEEE Commun. Surv. Tutor.*
- Luckham, D.C., 2002. The Power of Events: An Introduction to Complex Event Processing. Addison-Wesley Professional. ISBN 978-0201727890.
- Luckham, D.C., 2011. Event Processing for Business: Organizing the Real-Time Enterprise. Wiley. ISBN B005Y04XUU.

- Lumb, C.R., Golding, R.A., 2004. D-SPTF: decentralized request distribution in brick-based storage systems. In: Mukherjee, S., McKinley, K.S. (Eds.), *Proceedings of the 11th International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS 2004*. Boston, MA, USA, October 7–13, 2004. ACM, pp. 37–47.
- Lumb, C.R., Schindler, J., Ganger, G.R., Nagle, D., Riedel, E., 2000a. Towards higher disk head utilization: extracting “free” bandwidth from busy disk drives. In: Jones, M.B., Kaashoek, M.F. (Eds.), *4th Symposium on Operating System Design and Implementation (OSDI 2000)*. San Diego, California, USA, October 23–25, 2000. USENIX Association, pp. 87–102. <http://dl.acm.org/citation.cfm?id=1251236>.
- Lumb, C.R., Schindler, J., Ganger, G.R., Nagle, D., Riedel, E., 2000b. Towards higher disk head utilization: extracting “free” bandwidth from busy disk drives. In: Jones, M.B., Kaashoek, M.F. (Eds.), *4th Symposium on Operating System Design and Implementation (OSDI 2000)*. San Diego, California, USA, October 23–25, 2000. USENIX Association, pp. 87–102. <http://dl.acm.org/citation.cfm?id=1251236>.
- Lunt, B.M., 2018. A view to the cloud. *IEEE Spectr.* 55 (8), 40–43.
- Luo, J., Huang, C., Xu, L., 2010. Decoding STAR code for tolerating simultaneous disk failure and silent errors. In: *Proceedings of the 2010 IEEE/IFIP International Conference on Dependable Systems and Networks, DSN 2010*. Chicago, IL, USA, June 28–July 1 2010. IEEE Computer Society, pp. 61–70.
- Luo, X., Shu, J., 2013. Load-balanced recovery schemes for single-disk failure in storage systems with any erasure code. In: *42nd International Conference on Parallel Processing, ICPP 2013*. Lyon, France, October 1–4, 2013. IEEE Computer Society, pp. 552–561.
- Lynch, W.C., 1972. Do disk arms move? *ACM SIGMETRICS Perform. Eval. Rev.* 1 (4), 3–16.
- Ma, A., Traylor, R., Douglass, F., Chamness, M., Lu, G., Sawyer, D., Chandra, S., Hsu, W., 2015. Raidshield: characterizing, monitoring, and proactively protecting against disk failures. *ACM Trans. Storage* 11 (4), 17:1–17:28. <http://doi.acm.org/10.1145/2820615>.
- MacWilliams, F.J., Sloane, N.J.A., 1977. *The Theory of Error-Correcting Codes*. North Holland, Amsterdam, The Netherlands. ISBN 0444851933.
- Madnick, S.E., Donovan, J.J., 1974. *Operating Systems*. McGraw-Hill Computer Science Series. McGraw-Hill. ISBN 978-0070394551.
- Madnick, S.E., Moulton, A.G., 1968. Script, an online manuscript processing system. *IEEE Trans. Eng. Writ. Speech*, 92–108.
- Mahajan, D., Kim, J.K., Sacks, J., Ardan, A., Kumar, A., Esmailzadeh, H., 2018. In-rdbms hardware acceleration of advanced analytics. *Proc. VLDB Endow.* 11 (11), 1317–1331. <http://www.vldb.org/pvldb/vol11/p1317-mahajan.pdf>.
- Manber, U., 1994. Finding similar files in a large file system. In: *USENIX Winter 1994 Technical Conference, Conference Proceedings*. San Francisco, California, USA, January 17–21, 1994. USENIX Association, pp. 1–10. <https://www.usenix.org/conference/usenix-winter-1994-technical-conference/finding-similar-files-large-file-system>.
- Maneas, S., Mahdavian, K., Emami, T., Schroeder, B., 2020. A study of SSD reliability in large scale enterprise storage deployments. In: Noh, S.H., Welch, B. (Eds.), *18th USENIX Conference on File and Storage Technologies, FAST 2020*. Santa Clara, CA, USA, February 24–27, 2020. USENIX Association, pp. 137–149. <https://www.usenix.org/conference/fast20/presentation/maneas>.
- Manning, C.D., Raghavan, P., Schütze, H., 2008. *Introduction to Information Retrieval*. Cambridge University Press. ISBN 978-0521865715.
- Manolopoulos, Y., Kollias, J.G., 1989. Performance of a two-headed disk system when serving database queries under the scan policy. *ACM Trans. Database Syst.* 14 (3), 425–442. <http://doi.acm.org/10.1145/68012.68017>.
- Manolopoulos, Y., Kollias, J.G., 1990. Optimal data placement in two-headed disk systems. *BIT* 30 (2), 216–219.
- Manolopoulos, Y., Vakali, A., 1991. Seek distances in disks with two independent heads per surface. *Inf. Process. Lett.* 37 (1), 37–42. [https://doi.org/10.1016/0020-0190\(91\)90247-F](https://doi.org/10.1016/0020-0190(91)90247-F).
- Markatos, A.T., Watson, R.N.M., Moore, S., Sewell, P., Neumann, P.G., 2019. Inside risks: through computer architecture, darkly. *Commun. ACM* 62 (6).
- Martello, S., Toth, P., 1990. *Knapsack Problems: Algorithms and Computer Implementations*. John Wiley & Sons. ISBN 978-0471924203.
- Marz, N., Warren, J., 2015. *Big Data: Principles and Best Practices of Scalable Realtime Data Systems*. Manning Publications, Shelter Island. ISBN 9781617290343.
- Mattick, R., 1975. Memory and storage. In: Stone, H.S. (Ed.), *An Introduction to Computer Architecture*. Science Research Associates, Chicago, IL, pp. 176–247. Chapter 5.
- Mattick, R., 1977. *Storage Systems and Technology*, 1. John Wiley & Sons, New York, NY. ISBN 978-0471576297.
- Matthews, J.N., Roselli, D.S., Costello, A.M., Wang, R.Y., Anderson, T.E., 1997. Improving the performance of log-structured file systems with adaptive methods. In: Banâtre, M., Levy, H.M., Waite, W.M. (Eds.), *Proceedings of the Sixteenth ACM Symposium on Operating System Principles, SOSP 1997*. St. Malo, France, October 5–8, 1997. ACM, pp. 238–251.
- Mattson, R.L., Ng, S.W., 1993. Method and means for managing dasd array accesses when operating in degraded mode. <https://patents.justia.com/patent/5265098>. US Patent 5,265,098.
- Mattson, R.L., Gecsei, J., Slutz, D.R., Traiger, I.L., 1970. Evaluation techniques for storage hierarchies. *IBM Syst. J.* 9 (2), 78–117. <https://doi.org/10.1147/sj.92.0078>.
- May, J.M., 2001. *Parallel I/O for High Performance Computing*. Morgan-Kaufmann, Burlington, MA. ISBN 1558606645.
- Mazonka, O., 2016. Easy as π : the importance sampling method. <http://jrxv.net/x/16/ism.pdf>.
- McCree, A., Barnwall III, T.P., 1995. A mixed excitation LPC vocoder model for low bit rate speech coding. *IEEE Trans. Speech Audio Process.* 3 (4), 242–250. <https://doi.org/10.1109/89.397089>.
- McCue, G.M., 1978. IBM’s Santa Teresa laboratory – architectural design for program development. *IBM Syst. J.* 17 (1), 4–25. <https://doi.org/10.1147/sj.171.0004>.
- McDysan, D.E., Spohn, D.L., 1995. *ATM: Theory and Application*. McGraw-Hill Series on Computer Communications. McGraw-Hill, New York, NY. ISBN 0070603626.
- McGee, W.C., 1981. Data base technology. *IBM J. Res. Dev.* 25 (5), 505–519. <https://doi.org/10.1147/rd.255.0505>.
- McKusick, M.K., 2012. Disks from the perspective of a file system. *ACM Queue* 10 (9), 10. <https://doi.org/10.1145/2367376.2367378>.

- McKusick, M.K., Joy, W.N., Leffler, S.J., Fabry, R.S., 1984. A fast file system for UNIX. *ACM Trans. Comput. Syst.* 2 (3), 181–197. <https://doi.org/10.1145/989.990>.
- McNutt, B., 1986. An empirical study of variations in DASD volume activity. In: Morley, D., et al. (Eds.), Twelfth International Computer Measurement Group Conference, Proceedings. Las Vegas, NV, USA, December 8–12, 1986. Computer Measurement Group, pp. 274–283. http://www.cmg.org/?s2member_file_download=/proceedings/1986/86INT035.pdf.
- McNutt, B., 1994. Background data movement in a log-structured disk subsystem. *IBM J. Res. Dev.* 38 (1), 47–58. <https://doi.org/10.1147/rd.381.0047>.
- McNutt, B., 2002. The Fractal Structure of Data Reference – Applications to the Memory Hierarchy. *Advances in Database Systems*, vol. 22. Kluwer. ISBN 0-792-37945-4.
- Mead, C., Conway, L., 1980. *Introduction to VLSI Systems*. Addison-Wesley. ISBN 978-0201043587.
- Meaney, Patrick J., Lastras-Montañó, Luis Alfonso, Papazova, Vesselina K., Stephens, Eldee, Johnson, J.S., Alves, Luiz C., O'Connor, James A., Clarke, William J., 2012. IBM zEnterprise redundant array of independent memory subsystem. *IBM J. Res. Dev.* 56 (1), 1–4. <https://doi.org/10.1147/JRD.2011.2177106>.
- Mee, C.D., Daniel, E.D., Mark, e., Clark, H., 1996. *Magnetic Storage Handbook*, 2. McGraw-Hill, New York, NY. ISBN 978-0070412750.
- Megiddo, N., Modha, D.S., 2003. ARC: a self-tuning, low overhead replacement cache. In: Chase, J. (Ed.), Proceedings of the FAST '03 Conference on File and Storage Technologies. March 31–April 2, 2003, Cathedral Hill Hotel, San Francisco, California, USA. USENIX. <http://www.usenix.org/events/fast03/tech/megiddo.html>.
- Megiddo, N., Modha, D.S., 2004. Outperforming LRU with an adaptive replacement cache algorithm. *IEEE Computer* 37 (4), 58–65. <https://doi.org/10.1109/MC.2004.1297303>.
- Meister, D., Kaiser, J., Brinkmann, A., Cortes, T., Kuhn, M., Kunkel, J.M., 2012. A study on data deduplication in HPC storage systems. In: Hollingsworth, J.K. (Ed.), SC Conference on High Performance Computing Networking, Storage and Analysis, SC '12. Salt Lake City, UT, USA, November 11–15, 2012. IEEE/ACM, p. 7.
- Memik, G., Kandemir, M.T., Choudhary, A.N., 2001. Design and evaluation of a smart disk cluster for DSS commercial workloads. *J. Parallel Distrib. Comput.* 61 (11), 1633–1664. <https://doi.org/10.1006/jpdc.2001.1743>.
- Mencer, O., Allison, D., Blatt, E., et al., 2020. The history, status, and future of fpgas. *Commun. ACM* 63 (10), 36–39. <https://doi.org/10.1145/3410669>.
- Menon, J., 1994. Performance of RAID5 disk arrays with read and write caching. *Distrib. Parallel Databases* 2 (3), 261–293. <https://doi.org/10.1007/BF01266331>.
- Menon, J., 1995. A performance comparison of RAID-5 and log-structured arrays. In: Proceedings of the 4th International Symposium on High Performance Distributed Computing (HPDC '95). Washington, DC, USA, August 2–4, 1995. IEEE Computer Society, pp. 167–178.
- Menon, J., Cortney, J., 1993. The architecture of a fault-tolerant cached RAID controller. In: Smith, A.J. (Ed.), Proceedings of the 20th Annual International Symposium on Computer Architecture. San Diego, CA, USA, May 1993. ACM, pp. 76–86.
- Menon, J., Mattson, D., 1992a. Distributed sparing in disk arrays. In: 37th Annual IEEE Computer Society Conf. (COMPCON'92), Digest of Papers. San Francisco, California, USA, February 1992. IEEE, pp. 410–421.
- Menon, J., Mattson, D., 1992b. Comparison of sparing alternatives for disk arrays. In: Gottlieb, A. (Ed.), Proceedings of the 19th Annual International Symposium on Computer Architecture. Gold Coast, Australia, May 1992. ACM, pp. 318–329.
- Menon, J., Stockmeyer, L.J., 1998. An age threshold algorithm for garbage collection in log-structured arrays and file systems. Technical Report RJ 10120. IBM Almaden Research Center, San Jose, CA.
- Menon, J., Roche, J., Kasson, J.M., 1993. Floating parity and data disk arrays. *J. Parallel Distrib. Comput.* 17 (1–2), 129–139. <https://doi.org/10.1006/jpdc.1993.1011>.
- Merchant, A., Yu, P.S., 1992. Design and modeling of clustered RAID. In: Digest of Papers: FTCS-22, The Twenty-Second Annual International Symposium on Fault-Tolerant Computing. Boston, Massachusetts, USA, July 8–10, 1992. IEEE Computer Society, pp. 140–149.
- Merchant, A., Yu, P.S., 1994. An analytical model of reconstruction time in mirrored disks. *Perform. Eval.* 20 (1–3), 115–129. [https://doi.org/10.1016/0166-5316\(94\)90009-4](https://doi.org/10.1016/0166-5316(94)90009-4).
- Merchant, A., Yu, P.S., 1995. Analytic modeling and comparisons of striping strategies for replicated disk arrays. *IEEE Trans. Comput.* 44 (3), 419–433. <https://doi.org/10.1109/12.372034>.
- Merchant, A., Yu, P.S., 1996. Analytic modeling of clustered RAID with mapping based on nearly random permutation. *IEEE Trans. Comput.* 45 (3), 367–373. <https://doi.org/10.1109/12.485575>.
- Merten, A.G., 1970. Some Quantitative Techniques for File Organizations. PhD thesis. University of Wisconsin, Madison, WI. Technical Report No. 15.
- Meyer, D.T., Bolosky, W.J., 2011. A study of practical deduplication. In: Ganger, G.R., Wilkes, J. (Eds.), 9th USENIX Conference on File and Storage Technologies. San Jose, CA, USA, February 15–17, 2011. USENIX, pp. 1–13. <http://www.usenix.org/events/fast11/tech/techAbstracts.html#Meyer>.
- Meza, J., Wu, Q., Kumar, S., Mutlu, O., 2015. A large-scale study of flash memory failures in the field. In: Lin, B., Xu, J.J., Sengupta, S., Shah, D. (Eds.), Proceedings of the 2015 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems. Portland, OR, USA, June 15–19, 2015. ACM, pp. 177–190.
- Miao, H., Chavan, A., Deshpande, A., 2017. ProvdB: lifecycle management of collaborative analysis workflows. In: Binnig, C., Hellerstein, J.M., Parameswaran, A.G. (Eds.), Proceedings of the 2nd Workshop on Human-In-the-Loop Data Analytics, HILDA@SIGMOD 2017. Chicago, IL, USA, May 14, 2017. ACM, pp. 7:1–7:6.
- Micheloni, R., Crippa, L., Marelli, A., 2010. *Inside NAND Flash Memories*. Springer, New York, NY. ISBN 978-9048194308.
- Miller, E.L., 2014. Inside the pure storage flash array: building a high performance, data reducing storage system from commodity ssds-presentation. In: IEEE 30th Symposium on Mass Storage Systems and Technologies, MSST 2014. Santa Clara, CA, USA, June 2–6, 2014.

- Miller, L.W., 1974. Branch and bound and heuristic approaches to a sequencing problem with team size requirements. *Trans. Am. Inst. Electr. Eng.* 6 (3), 206–215.
- Minhas, U.F., Lomet, D.B., Thekkath, C.A., 2011. Chimera: data sharing flexibility, shared nothing simplicity. In: Desai, B.C., Cruz, I.F., Bernardino, J. (Eds.), 15th International Database Engineering and Applications Symposium (IDEAS 2011). Lisbon, Portugal, September 21–27, 2011. ACM, pp. 152–161.
- Mirhoseini, A., Potkonjak, M., Koushanfar, F., 2012. Coding-based energy minimization for phase change memory. In: Groeneveld, P., Sciuto, D., Has-soun, S. (Eds.), The 49th Annual Design Automation Conference 2012, DAC '12. San Francisco, CA, USA, June 3–7, 2012. ACM, pp. 68–76.
- Mishra, A., Chavan, S., Holloway, A., Lahiri, T., et al., 2016. Accelerating analytics with dynamic in-memory expressions. *Proc. VLDB Endow.* 9 (13), 1437–1448. <http://www.vldb.org/pvldb/vol9/p1437-mishra.pdf>.
- Mitchell, T., 1997. *Machine Learning*. McGraw-Hill. ISBN 978-0-07-042807-2.
- Mittal, S., 2017. A survey of techniques for architecting tlbs. *Concurr. Comput. Pract. Exp.* 29 (10). <https://doi.org/10.1002/cpe.4061>.
- Mitzenmacher, M., 2001. The power of two choices in randomized load balancing. *IEEE Trans. Parallel Distrib. Syst.* 12 (10), 1094–1104. <https://doi.org/10.1109/71.963420>.
- Mogi, K., Kitsuregawa, M., 1996. Hot mirroring: a study to hide parity upgrade penalty and degradations during rebuilds for RAID5. In: Jagadish, H.V., Mumick, I.S. (Eds.), *Proceedings of the 1996 ACM SIGMOD International Conference on Management of Data*. Montreal, Quebec, Canada, June 4–6, 1996. ACM Press, pp. 183–194.
- Mohan, C., 1992. Less optimism about optimistic concurrency control. In: Yu, P.S. (Ed.), *RIDE-TQP '92, Second International Workshop on Research Issues on Data Engineering: Transaction and Query Processing*. Tempe, Arizona, USA, February 2–3, 1992. IEEE Computer Society, pp. 199–204.
- Mohan, C., Haderle, D.J., Lindsay, B.G., Pirahesh, H., Schwarz, P.M., 1992. ARIES: a transaction recovery method supporting fine-granularity locking and partial rollbacks using write-ahead logging. *ACM Trans. Database Syst.* 17 (1), 94–162. <http://doi.acm.org/10.1145/128765.128770>.
- Mohapatra, P., 1998. Wormhole routing techniques for directly connected multicomputer systems. *ACM Comput. Surv.* 30 (3), 374–410. <https://doi.org/10.1145/292469.292472>.
- Mostak, T., 2013. An overview of mapd (massively parallel database). Technical report. MIT. http://geops.csail.mit.edu/docs/mapd_overview.pdf.
- Mumolo, E., 1999. Prediction of disk arm movements in anticipation of future requests. In: *MASCOTS 1999, Proceedings of the 7th International Symposium on Modeling, Analysis and Simulation of Computer and Telecommunication Systems*. 24–28 October 1999, College Park, Maryland, USA. IEEE Computer Society, pp. 305–312.
- Muntz, R.R., Lui, J.C.S., 1990a. Performance analysis of disk arrays under failure. In: McLeod, D., Sacks-Davis, R., Schek, H. (Eds.), *16th International Conference on Very Large Data Bases, Proceedings*. August 13–16, 1990, Brisbane, Queensland, Australia. Morgan Kaufmann, pp. 162–173. <http://www.vldb.org/conf/1990/P162.PDF>.
- Muntz, R.R., Lui, J.C.S., 1990b. Performance analysis of disk arrays under failure. In: McLeod, D., Sacks-Davis, R., Schek, H. (Eds.), *16th International Conference on Very Large Data Bases, Proceedings*. August 13–16, 1990, Brisbane, Queensland, Australia. Morgan Kaufmann, pp. 162–173. <http://www.vldb.org/conf/1990/P162.PDF>.
- Muntz, R.R., Wong, J.W., 1974. Asymptotic properties of closed queueing network models. In: *Proc. 8th Princeton Conference on Information Sciences and Systems*.
- Muthitacharoen, A., Chen, B., Mazieres, D., 2001. A low-bandwidth network file system. In: *Proc. ACM Symp. Oper. Syst. Principles*, pp. 1–14.
- Nadji, B., Thomasian, A., 1984. Throughput estimation in queueing networks. *Comput. Perform.* 5 (4), 197–206. <https://doi.org/10.1007/s11042-007-0179-7>.
- Nagarajan, V., Sorin, D.J., Hill, M.D., 2020. *A Primer on Memory Consistency and Cache Coherence, 2. Synthesis Lectures on Computer Architecture*. Morgan & Claypool. ISBN 978-1681737096.
- Nagy, B., Farmer, J.D., Bui, Q.M., Trancik, J., 2012. Statistical basis for predicting technological progress. Technical Report 12-07-008. Santa Fe Institute, Santa Fe, NM.
- Narayanan, D., Donnelly, A., Rowstron, A.I.T., 2008. Write off-loading: practical power management for enterprise storage. *ACM Trans. Storage* 4 (3), 10:1–10:23. <http://doi.acm.org/10.1145/1416944.1416949>.
- Narayanan, D., Thereska, E., Donnelly, A., Elnikety, S., Rowstron, A.I.T., 2009. Migrating server storage to ssds: analysis of tradeoffs. In: Schröder-Preikschat, W., Wilkes, J., Isaacs, R. (Eds.), *Proceedings of the 2009 EuroSys Conference*. Nuremberg, Germany, April 1–3, 2009. ACM, pp. 145–158.
- Nelder, J.A., Mead, R., 1965. A simplex method for function minimization. *Comput. J.* 7, 308–314.
- Nelson, M., Kitts, D.L., Merrill, J.H., Harano, G., 1987. The near mass storage system. In: *Eight IEEE Symposium on Mass Storage Systems, MSS 1987: Storage Systems: Perspectives*. Tucson, AZ, USA, May 1987. IEEE Computer Society.
- Nelson, R.D., Iyer, B.R., 1985. Analysis of a replicated data base. *Perform. Eval.* 5 (3), 133–148. [https://doi.org/10.1016/0166-5316\(85\)90008-2](https://doi.org/10.1016/0166-5316(85)90008-2).
- Nelson, R.D., Tantawi, A.N., 1988. Approximate analysis of fork/join synchronization in parallel queues. *IEEE Trans. Comput.* 37 (6), 739–743. <https://doi.org/10.1109/12.2213>.
- Nerjes, G., Muth, P., Weikum, G., 1997. Stochastic service guarantees for continuous data on multi-zone disks. In: Mendelson, A.O., Özsoyoglu, Z.M. (Eds.), *Proceedings of the Sixteenth ACM SIGACT-SIGMOD-SIGART Symposium on Principles of Database Systems*. Tucson, Arizona, USA, May 12–14, 1997. ACM Press, pp. 154–160.
- Neumann, T., 2011. Efficiently compiling efficient query plans for modern hardware. *Proc. VLDB Endow.* 4 (9), 539–550. <http://www.vldb.org/pvldb/vol4/p539-neumann.pdf>.
- Neuts, M., 1981. *Matrix Geometric Solutions in Stochastic Model – An Algorithmic Approach*, revised edition. John Hopkins University Press, Baltimore, MD. ISBN 3257227892. Dover Publications, 1995.

- Newberg, L.A., Wolfe, D., 1994. String layouts for a redundant array of inexpensive disks. *Algorithmica* 12 (2/3), 209–224. <https://doi.org/10.1007/BF01185211>.
- Ng, S.W., 1987. Reliability, availability, and performance analysis of duplex disk systems. In: Hamza, M.H. (Ed.), *Reliability and Quality Control*. Acta Press, Calgary, Canada, pp. 5–9.
- Ng, S.W., 1991. Improving disk performance via latency reduction. *IEEE Trans. Comput.* 40 (1), 22–30. <https://doi.org/10.1109/12.67317>.
- Ng, S.W., 1994a. Crosshatch disk array for improved reliability and performance. In: Patterson, D.A. (Ed.), *Proceedings of the 21st Annual International Symposium on Computer Architecture*. Chicago, IL, USA, April 1994. IEEE Computer Society, pp. 255–264.
- Ng, S.W., 1994b. Sparing for redundant disk arrays. *Distrib. Parallel Databases* 2 (2), 133–149. <https://doi.org/10.1007/BF01267324>.
- Ng, S.W., 1998. Advances in disk technology: performance issues. *IEEE Computer* 31 (5), 75–81. <https://doi.org/10.1109/2.675641>.
- Ng, S.W., 2001. Handling seek time variabilities in shortest access time first disk scheduling. Technical Report RJ 10206. IBM Almaden Research Center, San Jose, CA.
- Ng, S.W., Mattson, R.L., 1992. Maintaining good performance in disk arrays during failure via uniform parity group distribution. In: *Proceedings of the First International Symposium on High Performance Distributed Computing, HPDC '92*. Syracuse, NY, USA, September 9–11, 1992. IEEE, pp. 260–269.
- Ng, S.W., Mattson, R.L., 1994. Uniform parity group distribution in disk arrays with multiple failures. *IEEE Trans. Comput.* 43 (4), 501–506. <https://doi.org/10.1109/12.278490>.
- Ng, Y.-W., 1976. Reliability Modeling and Analysis for Fault-Tolerant Computers. PhD thesis. University of California at Los Angeles, Los Angeles, CA.
- Ng, Y.W., Avizienis, A., 1980. A unified reliability model for fault-tolerant computers. *IEEE Trans. Comput.* 29 (11), 1002–1011. <https://doi.org/10.1109/TC.1980.1675495>.
- Nick, J.M., Chung, J., Bowen, N.S., 1996. IBM system/390 division: overview of IBM system/390 parallel sysplex – a commercial parallel processing system. In: *Proceedings of IPPS '96, The 10th International Parallel Processing Symposium*. Honolulu, Hawaii, USA, April 15–19, 1996. IEEE Computer Society, pp. 488–495.
- Nicola, V.F., Dan, A., Dias, D.M., 1992. Analysis of the generalized clock buffer replacement scheme for database transaction processing. In: Molloy, M.K., Gaither, B.D. (Eds.), *Proceedings of the 1992 ACM SIGMETRICS Joint International Conference on Measurement and Modeling of Computer Systems*. Newport, Rhode Island, USA, June 1–5, 1992. ACM, pp. 35–46.
- Nicola, V.F., Shahabuddin, P., Heidelberger, P., Glynn, P., 1993. Fast simulation of steady-state availability in non-Markovian highly dependable systems. In: *FTCS-23 The Twenty-Third International Symposium on Fault-Tolerant Computing*. IEEE Press, Toulouse, France, pp. 38–47.
- Nicola, V.F., Shahabuddin, P., Nakayama, M.K., 2001. Techniques for fast simulation of models of highly dependable systems. *IEEE Trans. Reliab.* 50 (3), 246–264.
- Nishikawa, N., Hosokawa, T., Mori, Y., Yoshida, K., Tsuji, H., 1998. Memory-based architecture for distributed WWW caching proxy. *Comput. Netw.* 30 (1–7), 205–214. [https://doi.org/10.1016/S0169-7552\(98\)00117-2](https://doi.org/10.1016/S0169-7552(98)00117-2).
- Nuyens, M., Wierman, A., 2008. The foreground-background queue: a survey. *Perform. Eval.* 65 (3–4), 286–307. <https://doi.org/10.1016/j.peva.2007.06.028>.
- Nyberg, C., Barclay, T., Cvetanovic, Z., Gray, J., Lomet, D.B., 1995. Alphasort: a cache-sensitive parallel external sort. *VLDB J.* 4 (4), 603–627. <http://www.vldb.org/journal/VLDBJ4/P603.pdf>.
- O'Connor, P., Kleyner, A., 2012. *Practical Reliability Engineering*, 5. Wiley, New York, NY. ISBN 978-0470979815.
- Ohanyan, V.K., Thomasian, A., 2016. Expected value of the maximum and the Laplace transform of i.i.d exponentials. *Model. Artif. Intell.* 11 (3). <http://ejournal11.com/en/index.html>.
- Olken, F., Rotem, D., 1986. Simple random sampling from relational databases. In: Chu, W.W., Gardarin, G., Ohsuga, S., Kambayashi, Y. (Eds.), *VLDB'86 Twelfth International Conference on Very Large Data Bases, Proceedings*. August 25–28, 1986, Kyoto, Japan. Morgan Kaufmann, pp. 160–169. <http://www.vldb.org/conf/1986/P160.PDF>.
- Olson, J.E., 2013. *Data Quality: The Accuracy Dimension*. Data Management Systems. Morgan Kaufmann. ISBN 978-1558608917.
- O'Neil, E.J., O'Neil, P.E., Weikum, G., 1993. The LRU-K page replacement algorithm for database disk buffering. In: Buneman, P., Jajodia, S. (Eds.), *Proceedings of the 1993 ACM SIGMOD International Conference on Management of Data*. Washington, DC, USA, May 26–28, 1993. ACM Press, pp. 297–306.
- O'Neil, E.J., O'Neil, P.E., Weikum, G., 1999. An optimality proof of the LRU-K page replacement algorithm. *J. ACM* 46 (1), 92–112. <https://doi.org/10.1145/300515.300518>.
- O'Neil, P.E., 1986. The Escrow Transactional Method. *ACM Trans. Database Syst.* 11 (4), 405–430. <https://doi.org/10.1145/7239.7265>.
- O'Neil, Patrick E., Cheng, Edward, Gawlick, Dieter, O'Neil, Elizabeth J., 1996. The log-structured merge-tree (LSM-tree). *Acta Inf.* 33 (4), 351–385. <https://doi.org/10.1007/s002360050048>.
- Oney, W.C., 1975. Queueing analysis of the scan policy for moving-head disks. *J. ACM* 22 (3), 397–412. <https://doi.org/10.1145/321892.321902>.
- Ongaro, D., Ousterhout, J.K., 2014. In search of an understandable consensus algorithm. In: Gibson, G., Zeldovich, N. (Eds.), *USENIX Annual Technical Conference, USENIX ATC '14*. Philadelphia, PA, USA, June 19–20, 2014. USENIX Association, pp. 305–319. <https://www.usenix.org/conference/atc14/technical-sessions/presentation/ongaro>.
- Ooi, B.C., Tan, K., Yu, C., Bressan, S., 2000. Indexing the edges – a simple and yet efficient approach to high-dimensional indexing. In: Vianu, V., Gottlob, G. (Eds.), *Proceedings of the Nineteenth ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems*. May 15–17, 2000, Dallas, Texas, USA. ACM, pp. 166–174.
- Organick, E.I., 1972. *The Multics System: An Examination of Its Structure*. The MIT Press, Cambridge, MA. ISBN 978-0262150125.
- Organick, E.I., 1973. *Computer System Organization: The B5700/B6700 Series*. Monograph Series. Academic Press. ISBN 978-0125282505.

- Ousterhout, J.K., Douglass, F., 1989. Beating the I/O bottleneck: a case for log-structured file systems. *Oper. Syst. Rev.* 23 (1), 11–28. <http://doi.acm.org/10.1145/65762.65765>.
- Ousterhout, J.K., Agrawal, P., Erickson, D., Kozyrakis, C., et al., 2011. The case for RAMCloud. *Commun. ACM* 54 (7), 121–130. <http://doi.acm.org/10.1145/1965724.1965751>.
- Ousterhout, J.K., Gopalan, A., Gupta, A., Kejriwal, A., Lee, C., Montazeri, B., Ongaro, D., Park, S.J., Qin, H., Rosenblum, M., Rumble, S.M., Stutsman, R., Yang, S., 2015. The RAMCloud storage system. *ACM Trans. Comput. Syst.* 33 (3), 7:1–7:55. <https://doi.org/10.1145/2806887>.
- Ozkarahan, E.A., Sevcik, K.C., 1977. Analysis of architectural features for enhancing the performance of a database machine. *ACM Trans. Database Syst.* 2 (4), 297–316. <http://doi.acm.org/10.1145/320576.320577>.
- Ozkarahan, E.A., Schuster, S.A., Sevcik, K.C., 1977. Performance evaluation of a relational associative processor. *ACM Trans. Database Syst.* 2 (2), 175–195. <http://doi.acm.org/10.1145/320544.320553>.
- Ozsu, M.T., Valduriez, P., 2020. *Principles of Distributed Database Systems*, 4. Springer. ISBN 978-3030262525. <https://cs.uwaterloo.ca/~ddbook/index.html>.
- O’Whielacronx, Z., 2012. Zfec 1.4.24. Open source code distribution. Technical report, pypi.org. <https://pypi.python.org/pypi/zfec/>.
- Page, I.P., Wood, R.T., 1981. Empirical analysis of a moving head disc model with two heads separated by a fixed number of tracks. *Comput. J.* 24 (4), 339–342. <https://doi.org/10.1093/comjnl/24.4.339>.
- Paleologo, G.A., Benini, L., Bogliolo, A., Micheli, G.D., 1998. Policy optimization for dynamic power management. In: Chawla, B.R., Bryant, R.E., Rabaey, J.M. (Eds.), *Proceedings of the 35th Conference on Design Automation*. Moscone center, San Francisco, California, USA, June 15–19, 1998. ACM Press, pp. 182–187.
- Pamies-Juarez, L., Blagojevic, F., Mateescu, R., Guyot, C., Gad, E.E., Bandic, Z., 2016. Opening the chrysalis: on the real repair performance of MSR codes. In: Brown, A.D., Popovici, F.I. (Eds.), *14th USENIX Conference on File and Storage Technologies, FAST 2016*. Santa Clara, CA, USA, February 22–25, 2016. USENIX Association, pp. 81–94. <https://www.usenix.org/conference/fast16/technical-sessions/presentation/pamies-juarez>.
- Papathanasiou, A.E., Scott, M.L., 2003. Energy efficiency through burstiness. In: *5th IEEE Workshop on Mobile Computing Systems and Applications (WMCSA 2003)*. 9–10 October 2003, Monterey, CA, USA. IEEE Computer Society, pp. 44–53.
- Pardalos, P.M., Resende, M.G.C. (Eds.), 2002. *Handbook of Applied Optimization*. Oxford University Press, New York, NY. ISBN 978-0195125948.
- Parhami, B., 1972. A highly parallel computing system for information retrieval. In: *AFIPS Fall Joint Computing Conference* (2), pp. 681–690. <http://doi.acm.org/10.1145/1480083.1480090>.
- Parhami, B., 2010. *Computer Arithmetic: Algorithms and Hardware Designs*, 2. Oxford University Press, New York, NY. ISBN 0195328485.
- Pâris, J., Schwarz, T.J.E., Long, D.D.E., 2006. Self-adaptive disk arrays. In: Datta, A.K., Gradinariu, M. (Eds.), *Stabilization, Safety, and Security of Distributed Systems*, 8th International Symposium, SSS 2006, Proceedings. Dallas, TX, USA, November 17–19, 2006. In: *Lecture Notes in Computer Science*, vol. 4280. Springer, pp. 469–483.
- Paris, L.G., Mackay, M.P., 2012. Singlepass data compression and encryption. <https://patents.google.com/patent/US8886926B2/en>.
- Park, A., Balasubramanian, K., 1986. Providing fault tolerance in parallel secondary storage systems. Technical Report CS-TR-057-86. Department of Computer Science, Princeton University, Princeton, NJ.
- Park, C., 1995. Efficient placement of parity and data to tolerate two disk failures in disk array systems. *IEEE Trans. Parallel Distrib. Syst.* 6 (11), 1177–1184. <https://doi.org/10.1109/71.476189>.
- Parker, J.L., 1971. A logic-per-track retrieval system. In: Freiman, C.V., Griffith, J.E., Rosenfeld, J.L. (Eds.), *Information Processing, Proceedings of IFIP Congress 1971, Volume 1 – Foundations and Systems*. Ljubljana, Yugoslavia, August 23–28, 1971. North-Holland, pp. 711–716.
- Parmelee, R.P., Peterson, T.I., Jr., C.C.T., Hatfield, D.J., 1972. Virtual storage and virtual machine concepts. *IBM Syst. J.* 11 (2), 99–130. <https://doi.org/10.1147/sj.112.0099>.
- Parsian, M., 2015. *Data Algorithms*. O’Reilly Media. ISBN 9781491906170.
- Patel, A.M., 1985. Adaptive cross parity code for a high density magnetic tape subsystem. *IBM J. Res. Dev.* 29 (5), 546–562. <https://doi.org/10.1147/rd.294.0356>.
- Patterson, David A., Ditzel, David R., 1980. The case for the reduced instruction set computer. *SIGARCH Comput. Archit. News* 8 (6), 25–33. <https://doi.org/10.1145/641914.641917>.
- Patterson, D., Gibson, G.A., Katz, R.H., 1987. A case for redundant arrays of inexpensive disks (raid). Technical Report UCB/CSD-87-391. EECS Department, University of California, Berkeley. <http://www2.eecs.berkeley.edu/Pubs/TechRpts/1987/5853.html>.
- Patterson, D.A., Gibson, G.A., Katz, R.H., 1988. A case for redundant arrays of inexpensive disks (RAID). In: Boral, H., Larson, P. (Eds.), *Proceedings of the 1988 ACM SIGMOD International Conference on Management of Data*. Chicago, Illinois, USA, June 1–3, 1988. ACM Press, pp. 109–116.
- Patterson, D.A., 2002. A simple way to estimate the cost of downtime. In: Couch, A.L. (Ed.), *Proceedings of the 16th Conference on Systems Administration (LISA 2002)*. Philadelphia, PA, USA, November 3–8, 2002. USENIX, pp. 185–188. <http://www.usenix.org/publications/library/proceedings/lisa02/tech/patterson.html>.
- Patterson, D.A., Hennessy, J.L., 2017. *Computer Organization and Design RISC V Edition – The Hardware/Software Interface*, 1. Series in Computer Architecture and Design. Morgan Kaufmann. ISBN 978-0128122754.
- Patterson, D., Waterman, A., 2017. *The RISC-V Reader: An Open Architecture Atlas*, 1. Strawberry Canyon. ISBN 978-0999249116.
- Patterson, R.H., Gibson, G.A., Satyanarayanan, M., 1993. A status report on research in transparent informed prefetching. *Oper. Syst. Rev.* 27 (2), 21–34. <https://doi.org/10.1145/155848.155855>.
- Patterson, R.H., Gibson, G.A., Ginting, E., Stodolsky, D., Zelenka, J., 1995. Informed prefetching and caching. In: Jones, M.B. (Ed.), *Proceedings of the Fifteenth ACM Symposium on Operating System Principles, SOSP 1995*. Copper Mountain Resort, Colorado, USA, December 3–6, 1995. ACM, pp. 79–95.

- Paulo, J., Pereira, J., 2014. A survey and classification of storage deduplication systems. *ACM Comput. Surv.* 47 (1), 11:1–11:30. <http://doi.acm.org/10.1145/2611778>.
- Pekhimenko, G., Seshadri, V., Kim, Y., Xin, H., Mutlu, O., Gibbons, P.B., Kozuch, M.A., Mowry, T.C., 2013. Linearly compressed pages: a low-complexity, low-latency main memory compression framework. In: Farrens, M.K., Kozyrakis, C. (Eds.), *The 46th Annual IEEE/ACM International Symposium on Microarchitecture, MICRO-46*. Davis, CA, USA, December 7–11, 2013. ACM, pp. 172–184.
- Pekhimenko, G.G., 2016. Practical Data Compression for Modern Memory Hierarchies. PhD thesis. Dept. of Computer Science, Carnegie-Mellon University, Pittsburgh, PA.
- Pelkey, J., 2019. Wesley Chu and the statistical multiplexer 1966–1975 (section 5.4). In: Pelkey, J. (Ed.), *A History of Computer Communications 1968–1988*. <http://www.historyofcomputercommunications.info/Book/5/5.4WesleyChuStatistical%20Multiplexer66-75.html>.
- Peng, G., Chiueh, T., 2008. Availability and fairness support for storage qos guarantee. In: *28th IEEE International Conference on Distributed Computing Systems (ICDCS 2008)*. 17–20 June 2008, Beijing, China. IEEE Computer Society, pp. 589–596.
- Perl, Y., Maram, V., Kadakuntla, N., 1991. The cascading of the {LZW} compression algorithm with arithmetic coding. In: Storer, J.A., Reif, J.H. (Eds.), *Proceedings of the IEEE Data Compression Conference, DCC 1991*. Snowbird, Utah, USA, April 8–11, 1991. IEEE Computer Society, pp. 277–286.
- Perry, J., Ousterhout, A., Balakrishnan, H., Shah, D., Fugal, H., 2014. Fastpass: A Centralized Zero-Queue Datacenter Network. In: *ACM SIGCOMM 2014*. Chicago, IL.
- Peters, T.J., Waterman Jr., Robert H., 1982. In: *Search of Excellence: Lessons from America's Best-Run Companies*. Harper and Row, New York, NY. ISBN 978-0060451530.
- Peterson, J.L., 1977. Petri nets. *ACM Comput. Surv.* 9 (3), 223–252. <https://doi.org/10.1145/356698.356702>.
- Peterson, W.W., Weldon, E.J., 1972. *Error Correcting Codes, 2*. The MIT Press. ISBN 978-0262527316.
- Pfister, G.F., Brantley, W.C., George, D.A., Harvey, S.L., Kleinfelder, W.J., McAuliffe, K.P., Melton, E.S., Norton, V.A., Weiss, J., 1985. The IBM research parallel processor prototype (RP3): introduction and architecture. In: *International Conference on Parallel Processing, ICPP'85*, University Park, PA, USA, August 1985. IEEE Computer Society Press, pp. 764–771.
- Pfsister, G., 1998. In: *Search of Clusters, 2*. Prentice-Hall, Upper Saddle River, NJ. ISBN 978-0138997090.
- Pfister, Greg F., 2001. An introduction the InfiniBand architecture. In: Jin, Hai, Cortes, Toni, Buyya, Rajkumar (Eds.), *High Performance Mass Storage and Parallel I/O: Technologies and Applications*. IEEE-Wiley, New York, NY. ISBN 0-471-20809-4, pp. 616–642.
- Piepmeyer, W.F., 1975. Optimal balancing of I/O requests to disks. *Commun. ACM* 18 (9), 524–527. <http://doi.acm.org/10.1145/361002.361013>.
- Pinheiro, E., Bianchini, R., 2004. Energy conservation techniques for disk array-based servers. In: Feautrier, P., Goodman, J.R., Sez nec, A. (Eds.), *Proceedings of the 18th Annual International Conference on Supercomputing, ICS 2004*. Saint Malo, France, June 26–July 01, 2004. ACM, pp. 68–78.
- Pinheiro, E., Bianchini, R., Carrera, E.V., Heath, T., 2003. Dynamic cluster reconfiguration for power and performance. In: Benini, L., Kandemir, M., Ramanujam, J. (Eds.), *Compilers and Operating Systems for Low Power*. Kluwer Academic Publishers, Norwell, MA. Chapter 1.
- Pinheiro, E., Bianchini, R., Dubnicki, C., 2006. Exploiting redundancy to conserve energy in storage systems. In: Marie, R.A., Key, P.B., Smirni, E. (Eds.), *Proceedings of the Joint International Conference on Measurement and Modeling of Computer Systems, SIGMETRICS/Performance 2006*. Saint Malo, France, June 26–30, 2006. ACM, pp. 15–26.
- Pinheiro, E., Weber, W., Barroso, L.A., 2007. Failure trends in a large disk drive population. In: Arpaci-Dusseau, A.C., Arpaci-Dusseau, R.H. (Eds.), *5th USENIX Conference on File and Storage Technologies, FAST 2007*. February 13–16, 2007, San Jose, CA, USA. USENIX, pp. 17–28. <http://www.usenix.org/events/fast07/tech/pinheiro.html>.
- Pirk, H., Moll, O.R., Zaharia, M., Madden, S., 2016. Voodoo – a vector algebra for portable database performance on modern hardware. *Proc. VLDB Endow.* 9 (14), 1707–1718. <http://www.vldb.org/pvldb/vol9/p1707-pirk.pdf>.
- Plambeck, K.E., 1989. Concepts of enterprise systems architecture/370. *IBM Syst. J.* 28 (1), 39–61. <https://doi.org/10.1147/sj.281.0039>.
- Plambeck, K.E., Eckert, W., Rogers, R.R., Webb, C.F., 2002. Development and attributes of z/architecture. *IBM J. Res. Dev.* 46 (4–5), 367–380. <https://doi.org/10.1147/rd.464.0367>.
- Plank, J.S., 1997a. A tutorial on Reed-Solomon coding for fault-tolerance in raid-like systems. *Softw. Pract. Exp.* 27 (9), 995–1012. [https://doi.org/10.1002/\(SICI\)1097-024X](https://doi.org/10.1002/(SICI)1097-024X).
- Plank, J.S., 1997b. An overview of checkpointing in uniprocessor and distributed systems, focusing on implementation and performance. Technical Report CS-97-372. University of Tennessee. Also published as “Program diagnostics” in: Webster, John G. (Ed.), *Encyclopedia of Electrical and Electronics Engineering*. John Wiley & Sons, Inc.
- Plank, J.S., 2005. Erasure codes for storage applications (tutorial). In: Gibson, G. (Ed.), *4th USENIX Conference on File and Storage Technologies, FAST 2005*. February 26–29, 2005, San Jose, CA, USA. USENIX. <https://www.usenix.org/legacy/events/fast05/tutorials/tutonefile.html#t1>.
- Plank, J.S., 2008a. The RAID-6 liberation codes. In: Baker, M., Riedel, E. (Eds.), *6th USENIX Conference on File and Storage Technologies, FAST 2008*. February 26–29, 2008, San Jose, CA, USA. USENIX, pp. 97–110. <http://www.usenix.org/events/fast08/tech/plank.html>.
- Plank, J.S., 2008b. A new minimum density RAID-6 code with a word size of eight. In: *Proceedings of The Seventh IEEE International Symposium on Networking Computing and Applications, NCA 2008*. July 10–12, 2008, Cambridge, Massachusetts, USA. IEEE Computer Society, pp. 85–92.
- Plank, J.S., Ding, Y., 2005. Note: Correction to the 1997 tutorial on Reed-Solomon coding. *Softw. Pract. Exp.* 35 (2), 189–194. <https://doi.org/10.1002/spe.631>.
- Plank, J.S., Huang, C., 2013. Tutorial: Erasure coding for storage applications. <https://www.usenix.org/conference/fast13>.
- Plank, J.S., Thomason, M.G., 2007. An exploration of non-asymptotic low-density, parity check erasure codes for wide-area storage applications. *Parallel Process. Lett.* 17 (1), 103–123. <https://doi.org/10.1142/S0129626407002909>.
- Plank, J.S., Xu, L., 2006. Optimizing Cauchy Reed-Solomon codes for fault-tolerant network storage applications. In: *Fifth IEEE International Symposium on Network Computing and Applications, NCA 2006*. 24–26 July 2006, Cambridge, Massachusetts, USA. IEEE Computer Society, pp. 173–180.

- Plank, J.S., Luo, J., Schuman, C.D., Xu, L., Wilcox-O'Hearn, Z., 2009a. A performance evaluation and examination of open-source erasure coding libraries for storage. In: Seltzer, M.I., Wheeler, R. (Eds.), 7th USENIX Conference on File and Storage Technologies, Proceedings. February 24–27, 2009, San Francisco, CA, USA. USENIX, pp. 253–265. http://www.usenix.org/events/fast09/tech/full_papers/plank/plank.pdf.
- Plank, J.S., Luo, J., Schuman, C.D., Xu, L., Wilcox-O'Hearn, Z., 2009b. A performance evaluation and examination of open-source erasure coding libraries for storage. In: Seltzer, M.I., Wheeler, R. (Eds.), 7th USENIX Conference on File and Storage Technologies, Proceedings. February 24–27, 2009, San Francisco, CA, USA. USENIX, pp. 253–265. http://www.usenix.org/events/fast09/tech/full_papers/plank/plank.pdf.
- Plank, J.S., Simmerman, S., Schuman, C.D., 2014. Jerasure: a library in c facilitating erasure coding for storage applications – version 1.2. Technical Report UT-CS-14-721. University of Tennessee, Knoxville, TN. <http://web.eecs.utk.edu/~jplank/plank/papers/CS-08-627.pdf>.
- Pollock, J.T., 2009. Semantic Web for Dummies. Dummies. Wiley. ISBN 978-0470498163.
- Polyzois, C.A., Garcia-Molina, H., 1994. Evaluation of remote backup algorithms for transaction-processing systems. *ACM Trans. Database Syst.* 19 (3), 423–449. <https://doi.org/10.1145/185827.185836>.
- Polyzois, C.A., Bhide, A., Dias, D.M., 1993. Disk mirroring with alternating deferred updates. In: Agrawal, R., Baker, S., Bell, D.A. (Eds.), 19th International Conference on Very Large Data Bases, Proceedings. August 24–27, 1993, Dublin, Ireland. Morgan Kaufmann, pp. 604–617. <http://www.vldb.org/conf/1993/P604.PDF>.
- Popa, R.A., Zeldovich, N., Balakrishnan, H., 2011. Cryptdb: a practical encrypted relational dbms. Technical Report MIT-CSAIL-TR-2011-005. MIT Computer Science and Artificial Intelligence Laboratory, Cambridge, MA.
- Popa, R.A., Redfield, C.M.S., Zeldovich, N., Balakrishnan, H., 2012. Cryptdb: processing queries on an encrypted database. *Commun. ACM* 55 (9), 103–111. <https://doi.org/10.1145/2330667.2330691>.
- Popek, G.J., Goldberg, R.P., 1973. Formal requirements for virtualizable third generation architectures. In: Schorr, H., Perlis, A.J., Weiner, P., Frazer, W.D. (Eds.), Proceedings of the Fourth Symposium on Operating System Principles, SOSP 1973. Thomas J. Watson Research Center, Yorktown Heights, New York, USA, October 15–17, 1973. ACM, p. 121.
- Popek, G.J., Goldberg, R.P., 1974. Formal requirements for virtualizable third generation architectures. *Commun. ACM* 17 (7), 412–421. <http://doi.acm.org/10.1145/361011.361073>.
- Potier, D., Leblanc, P., 1980. Analysis of locking policies in database management systems. *Commun. ACM* 23 (10), 584–593. <https://doi.org/10.1145/359015.359021>.
- Prabhakaran, V., Bairavasundaram, L.N., Agrawal, N., Gunawi, H.S., Arpaci-Dusseau, A.C., Arpaci-Dusseau, R.H., 2005. IRON file systems. In: Herbert, A., Birman, K.P. (Eds.), Proceedings of the 20th ACM Symposium on Operating Systems Principles 2005, SOSP 2005. Brighton, UK, October 23–26, 2005. ACM, pp. 206–220.
- Press, W.H., Teukolsky, S.A., Vetterling, W.T., Flannery, B.P., 2007. Section 10.12. Simulated annealing methods. In: Numerical Recipes: The Art of Scientific Computing. Cambridge University Press. ISBN 0521880688.
- Prieve, B.G., Fabry, R.S., 1976. Vmin-an optimal variable-space page replacement algorithm. *Commun. ACM* 19 (5), 295–297. <https://doi.org/10.1145/360051.360231>.
- Prusinkiewicz, P., Budkowski, S., 1976. A double track error-correction code for magnetic tape. *IEEE Trans. Comput.* 25 (6), 642–645. <https://doi.org/10.1109/TC.1976.1674665>.
- Pugh, E.W., 2002. Origins of software bundling. *IEEE Ann. Hist. Comput.* 24 (1), 57–58. <https://doi.org/10.1109/85.988580>.
- Pugh, E.W., Johnson, L.R., Palmer, J.H., 2003. IBM's 360 and Early 370 System. History of Computing. MIT Press, Cambridge, MA. ISBN 9780262517201.
- Qiao, L., Raman, V., Narang, I., Pandey, P., Chambliss, D.D., Fuh, G., Ruddy, J.A., Chen, Y., Yang, K., Ling, F., 2008. Integration of server, storage and database stack: moving processing towards data. In: Alonso, G., Blakeley, J.A., Chen, A.L.P. (Eds.), Proceedings of the 24th International Conference on Data Engineering, ICDE 2008. April 7–12, 2008, Cancún, Mexico. IEEE Computer Society, pp. 1200–1208.
- Quinlan, S., Dorward, S., 2002. Venti: a new approach to archival storage. In: Long, D.D.E. (Ed.), Proceedings of the FAST '02 Conference on File and Storage Technologies. January 28–30, 2002, Monterey, California, USA. USENIX, pp. 89–101. <http://www.usenix.org/publications/library/proceedings/fast02/quinlan.html>.
- Qureshi, M.K., Gurumurthi, S., Rajendran, B., 2011. Phase Change Memory: From Devices to Systems. Synthesis Lectures on Computer Architecture. Morgan & Claypool.
- Rabin, M.O., 1989. Efficient dispersal of information for security, load balancing, and fault tolerance. *J. ACM* 36 (2), 335–348. <http://doi.acm.org/10.1145/62044.62050>.
- Rabl, T., Poess, M., Jacobsen, H., O'Neil, P.E., O'Neil, E.J., 2013. Variations of the star schema benchmark to test the effects of data skew on query performance. In: Seelam, S., Tuma, P., Casale, G., Field, T., Amaral, J.N. (Eds.), ACM/SPEC International Conference on Performance Engineering, ICPE'13. Prague, Czech Republic, April 21–24, 2013. ACM, pp. 361–372.
- Rahm, E., 1992. Performance evaluation of extended storage architectures for transaction processing. In: Stonebraker, M. (Ed.), Proceedings of the 1992 ACM SIGMOD International Conference on Management of Data. San Diego, California, USA, June 2–5, 1992. ACM Press, pp. 308–317.
- Rahm, E., 1993. Empirical performance evaluation of concurrency and coherency control protocols for database sharing systems. *ACM Trans. Database Syst.* 18 (2), 333–377. <http://doi.acm.org/10.1145/151634.151639>.
- Ramakrishnan, K.K., Biswas, P., Karedla, R., 1992. Analysis of file I/O traces in commercial computing environments. In: Molloy, M.K., Gaither, B.D. (Eds.), Proceedings of the 1992 ACM SIGMETRICS Joint International Conference on Measurement and Modeling of Computer Systems. Newport, Rhode Island, USA, June 1–5, 1992. ACM, pp. 78–90.
- Ramakrishnan, R., Gehrke, J., 2002. Database Management Systems, 3. McGraw-Hill. ISBN 978-0-07-115110-8.
- Ramamritham, K., Stankovic, J.A., 1994. Scheduling algorithms and operating systems support for real-time systems. *Proc. IEEE* 82 (1), 55–67.

- Rao, K.K., Hafner, J.L., Golding, R.A., 2006. Reliability for networked storage nodes. In: 2006 International Conference on Dependable Systems and Networks (DSN 2006), Proceedings. 25–28 June 2006, Philadelphia, Pennsylvania, USA. IEEE Computer Society, pp. 237–248.
- Rao, K.K., Hafner, J.L., Golding, R.A., 2011. Reliability for networked storage nodes. *IEEE Trans. Dependable Secure Comput.* 8 (3), 404–418. <https://doi.org/10.1109/TDSC.2010.21>.
- Rashmi, K.V., Shah, N.B., Gu, D., Kuang, H., Borthakur, D., Ramchandran, K., 2014. A “hitchhiker’s” guide to fast and efficient data reconstruction in erasure-coded data centers. In: Bustamante, F.E., Hu, Y.C., Krishnamurthy, A., Ratnasamy, S. (Eds.), *ACM SIGCOMM 2014 Conference, SIGCOMM’14*. Chicago, IL, USA, August 17–22, 2014. ACM, pp. 331–342.
- Rashmi, K.V., Nakkiran, P., Wang, J., Shah, N.B., Ramchandran, K., 2015. Having your cake and eating it too: jointly optimal erasure codes for i/o, storage, and network-bandwidth. In: Schindler, J., Zadok, E. (Eds.), *Proceedings of the 13th USENIX Conference on File and Storage Technologies, FAST 2015*. Santa Clara, CA, USA, February 16–19, 2015. USENIX Association, pp. 81–94. <https://www.usenix.org/conference/fast15/technical-sessions/presentation/rashmi>.
- Rashmi, K.V., Shah, N.B., Ramchandran, K., 2017. A piggybacking design framework for read-and download-efficient distributed storage codes. *IEEE Trans. Inf. Theory* 63 (9), 5802–5820. <https://doi.org/10.1109/TIT.2017.2715043>.
- Rathi, A., Lu, H., Hedrick, G.E., 1990. Performance comparison of extendible hashing and linear hashing techniques. In: *SIGSMALL/PC Symposium*, pp. 178–185.
- Reddy, A.L.N., Banerjee, P., 1991. Gracefully degradable disk arrays. In: *Proceedings of the 1991 International Symposium on Fault-Tolerant Computing*. Montreal, Canada. IEEE Computer Society, pp. 401–409.
- Reddy, A.L.N., Wyllie, J.C., 1993. Disk scheduling in a multimedia I/O system. In: Garcia-Luna-Aceves, J.J., Rangan, P.V. (Eds.), *Proceedings of the First ACM International Conference on Multimedia ’93*. Anaheim, CA, USA, August 1–6, 1993. ACM Press, pp. 225–233.
- Reddy, A.L.N., Wyllie, J.C., 1994. I/O issues in a multimedia system. *Computer* 27 (3), 69–74. <https://doi.org/10.1109/2.268888>.
- Reddy, A.L.N., Wyllie, J.C., Wijayarathne, R., 2005. Disk scheduling in a multimedia I/O system. *TOMCCAP* 1 (1), 37–59. <http://doi.acm.org/10.1145/1047936.1047941>.
- Reed, I.S., Solomon, G., 1960. Polynomial codes over certain finite fields. *J. Soc. Ind. Appl. Math.* 8 (2), 300–304.
- Reibman, A.L., Trivedi, K.S., 1988. Numerical transient analysis of Markov models. *Comput. Oper. Res.* 15 (1), 19–36. [https://doi.org/10.1016/0305-0548\(88\)90026-3](https://doi.org/10.1016/0305-0548(88)90026-3).
- Reibman, A.L., Trivedi, K.S., Kumar, S., Ciardo, G., 1989. Analysis of stiff Markov chains. *INFORMS J. Comput.* 1 (2), 126–133. <https://doi.org/10.1287/ijoc.1.2.126>.
- Reilly, E.D., 2003. *Milestones in Computer Science and Information Technology*. The Greenwood Press. ISBN 978-1573565219.
- Reiser, M., 2000. Mean value analysis: a personal account. In: Haring, G., Lindemann, C., Reiser, M. (Eds.), *Performance Evaluation: Origins and Directions*. In: *Lecture Notes in Computer Science*, vol. 1769. Springer, pp. 491–504.
- Reiser, M., Kobayashi, H., 1975. Queuing networks with multiple closed chains: theory and computational algorithms. *IBM J. Res. Dev.* 19 (3), 283–294. <https://doi.org/10.1147/rd.193.0283>.
- Reiser, M., Lavenberg, S.S., 1980. Mean-Value Analysis of Closed Multichain Queuing Networks. *J. ACM* 27 (2), 313–322. <https://doi.org/10.1145/322186.322195>.
- Resch, J.K., Plank, J.S., 2011. AONT-RS: blending security and performance in dispersed storage systems. In: Ganger, G.R., Wilkes, J. (Eds.), 9th *USENIX Conference on File and Storage Technologies*. San Jose, CA, USA, February 15–17, 2011. USENIX, pp. 191–202. <http://www.usenix.org/events/fast11/tech/techAbstracts.html#Resch>.
- Rhea, S.C., Wells, C., Eaton, P.R., Geels, D., Zhao, B.Y., Weatherspoon, H., Kubitowicz, J., 2001. Maintenance-free global data storage. *IEEE Internet Comput.* 5 (5), 40–49. <https://doi.org/10.1109/4236.957894>.
- Rhea, S.C., Eaton, P.R., Geels, D., Weatherspoon, H., Zhao, B.Y., Kubitowicz, J., 2003. Pond: the oceanstore prototype. In: Chase, J. (Ed.), *Proceedings of the FAST ’03 Conference on File and Storage Technologies*. March 31–April 2, 2003, Cathedral Hill Hotel, San Francisco, California, USA. USENIX. <http://www.usenix.org/events/fast03/tech/rhea.html>.
- Riedel, E., 1999. *Active Disk – Remote Execution for Network Attached Storage*. PhD thesis. Carnegie Mellon U., Pittsburgh, PA. Technical Report CMU-CS-99-177.
- Riedel, E., Gibson, G.A., Faloutsos, C., 1998. Active storage for large-scale data mining and multimedia. In: Gupta, A., Shmueli, O., Widom, J. (Eds.), *VLDB’98, Proceedings of 24th International Conference on Very Large Data Bases*. August 24–27, 1998, New York City, New York, USA. Morgan Kaufmann, pp. 62–73. <http://www.vldb.org/conf/1998/p062.pdf>.
- Riedel, E., Faloutsos, C., Ganger, G.R., Nagle, D., 2000. Data mining on an OLTP system (nearly) for free. In: Chen, W., Naughton, J.F., Bernstein, P.A. (Eds.), *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*. May 16–18, 2000, Dallas, Texas, USA. ACM, pp. 13–21.
- Rishe, N., Navathe, S.B., Tal, D. (Eds.), 1990a. *Proceedings of the First International Conference on Databases, Parallel Architectures, and Their Applications, PARBASE-90*. Miami Beach, Florida, USA, March 7–9, 1990. IEEE Computer Society. ISBN 0-8186-2035-8.
- Rishe, N., Navathe, S.B., Tal, D. (Eds.), 1990b. *Parallel Architectures (Postconference PARBASE-90)*. IEEE Computer Society. ISBN 0-8186-9166-2.
- Riska, A., Riedel, E., 2003. It’s not fair – evaluating efficient disk scheduling. In: 11th *International Workshop on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS 2003)*. 12–15 October 2003, Orlando, FL, USA. IEEE Computer Society, pp. 288–295.
- Riska, A., Riedel, E., 2006a. Disk drive level workload characterization. In: Adya, A., Nahum, E.M. (Eds.), *Proceedings of the 2006 USENIX Annual Technical Conference*. Boston, MA, USA, May 30–June 3, 2006. USENIX, pp. 97–102. <http://www.usenix.org/events/usenix06/tech/riska.html>.
- Riska, A., Riedel, E., 2006b. Long-range dependence at the disk drive level. In: *Third International Conference on the Quantitative Evaluation of Systems (QEST 2006)*. 11–14 September 2006, Riverside, California, USA. IEEE Computer Society, pp. 41–50.

- Riska, A., Riedel, E., 2008. Idle read after write – IRAW. In: Isaacs, R., Zhou, Y. (Eds.), 2008 USENIX Annual Technical Conference, Proceedings. Boston, MA, USA, June 22–27, 2008. USENIX Association, pp. 43–56. <http://www.usenix.org/events/usenix08/tech/fullpapers/riska/riska.pdf>.
- Riska, A., Riedel, E., Iren, S., 2004. Adaptive disk scheduling for overload management. In: 1st International Conference on Quantitative Evaluation of Systems (QEST 2004). 27–30 September 2004, Enschede, The Netherlands. IEEE Computer Society, pp. 176–185.
- Rissanen, J., Jr., G.G.L., 1979. Arithmetic coding. IBM J. Res. Dev. 23 (2), 149–162. <https://doi.org/10.1147/rd.232.0149>.
- Rissanen, J., Mohiuddin, K.M., 1989. A multiplication-free multialphabet arithmetic code. IEEE Trans. Commun. 37 (2), 93–98. <https://doi.org/10.1109/26.20074>.
- Rivest, R.L., 1997. All-or-nothing encryption and the package transform. In: Biham, E. (Ed.), Fast Software Encryption, 4th International Workshop, FSE '97, Proceedings. Haifa, Israel, January 20–22, 1997. In: Lecture Notes in Computer Science, vol. 1267. Springer.
- Robinson, I., Webber, J., Eifrem, E., 2015. Graph Databases, 2. O'Reilly. ISBN 978-1491932001.
- Robinson, J.T., 1981. The k-d-b-tree: a search structure for large multidimensional dynamic indexes. In: Lien, Y.E. (Ed.), Proceedings of the 1981 ACM SIGMOD International Conference on Management of Data. Ann Arbor, Michigan, USA, April 29–May 1, 1981. ACM Press, pp. 10–18.
- Robinson, J.T., 1985. A fast general-purpose hardware synchronisation mechanism. In: Navathe, S.B. (Ed.), Proceedings of the 1985 ACM SIGMOD International Conference on Management of Data. Austin, Texas, USA, May 28–31, 1985. ACM Press, pp. 122–130.
- Robinson, J.T., Devarakonda, M.V., 1990. Data cache management using frequency-based replacement. In: Nutt, G.J. (Ed.), Proceedings of the 1990 ACM SIGMETRICS Conference on Measurement and Modeling of Computer Systems. Boulder, Colorado, USA, May 22–25, 1990. ACM, pp. 134–142.
- Rojas-Cessa, R., 2017. Interconnections for Computer Communications and Packet Networks. CRC Press, Orlando, FL. ISBN 978-1482226966.
- Romanski, B., Heldt, L., Kilian, W., Lichota, K., Dubnicki, C., 2011. Anchor-driven subchunk deduplication. In: Ta-Shma, P., Moreira, J., Shriram, L. (Eds.), Proceedings of SYSTOR 2011: The 4th Annual Haifa Experimental Systems Conference. Haifa, Israel, May 30–June 1, 2011. ACM, p. 16.
- Rosenblum, M., 1995. The Design and Implementation of a Log-structured File System. Series in Engineering and Computer Science. Springer. ISBN 978-0792395416.
- Rosenblum, M., Ousterhout, J.K., 1992. The design and implementation of a log-structured file system. ACM Trans. Comput. Syst. 10 (1), 26–52. <http://doi.acm.org/10.1145/146941.146943>.
- Rosenfeld, E., Zuck, A., Amit, N., Factor, M., Tsafir, D., 2020. RAIDP: replication with intra-disk parity. In: Bilas, A., Magoutis, K., Markatos, E.P., Kostic, D., Seltzer, M.I. (Eds.), EuroSys '20: Fifteenth EuroSys Conference 2020. Heraklion, Greece, April 27–30, 2020. ACM, pp. 26:1–26:17.
- Rosenthal, D.S.H., 2008. Bit preservation: a solved problem? In: Proceedings of the 5th International Conference on Digital Preservation, iPRES 2008. London, UK, September 29–30, 2008. <http://hdl.handle.net/11353/10.294179>.
- Rosenthal, D.S.H., 2010a. Keeping bits safe: how hard can it be? Commun. ACM 53 (11), 47–55. <https://doi.org/10.1145/1839676.1839692>.
- Rosenthal, D.S.H., 2010b. Format obsolescence: assessing the threat and the defenses. Libr. Hi Tech 28 (2), 195–210. <https://doi.org/10.1108/07378831011047613>.
- Ross, S., 2014. Introduction to Probability Models, 11. Academic Press, New York, NY. ISBN 0124079482.
- Rothman, J.B., Smith, A.J., 1999. Sector cache design and performance. Technical report. Computer Science Division, U. Calif. at Berkeley. <https://www2.eecs.berkeley.edu/Pubs/TechRpts/1999/CSD-99-1034.pdf>.
- Ruemmler, C., Wilkes, J., 1991. Disk shuffling. Technical Report HPL-91-156. HP Laboratories, Palo Alto, CA.
- Ruemmler, C., Wilkes, J., 1993. UNIX disk access patterns. In: Proceedings of the Usenix Winter 1993 Technical Conference. San Diego, California, USA, January 1993. USENIX Association, pp. 405–420.
- Ruemmler, C., Wilkes, J., 1994. An introduction to disk drive modeling. IEEE Computer 27 (3), 17–28. <https://doi.org/10.1109/2.268881>.
- Ruggiero, J.F., Coryell, D.A., 1969. An auxiliary processing system for array calculations. IBM Syst. J. 8 (2), 118–135. <https://doi.org/10.1147/sj.82.0118>.
- Ruiz, H., 2013. Slingshot: AMD's Fight to Free an Industry from the Ruthless Grip of Intel. Greenleaf Press. ISBN 978-160832444.
- Rumble, S.M., Kejriwal, A., Ousterhout, J.K., 2014. Log-structured memory for dram-based storage. In: Schroeder, B., Thereska, E. (Eds.), Proceedings of the 12th USENIX Conference on File and Storage Technologies, FAST 2014. Santa Clara, CA, USA, February 17–20, 2014. USENIX, pp. 1–16. <https://www.usenix.org/conference/fast14/technical-sessions/presentation/rumble>.
- Ruparelia, N.B., 2016. Cloud Computing. The MIT Press, Cambridge, MA. ISBN 978-0262529099.
- Russell, R.M., 1978. The cray-1 computer system. Commun. ACM 21 (1), 63–72. <https://doi.org/10.1145/359327.359336>.
- Ryu, I.K., Thomasian, A., 1987. Performance analysis of centralized databases with optimistic concurrency control. Perform. Eval. 7 (3), 195–211. [https://doi.org/10.1016/0166-5316\(87\)90040-X](https://doi.org/10.1016/0166-5316(87)90040-X).
- Saavedra, R.H., Smith, A.J., 1995. Measuring cache and TLB performance and their effect on benchmark runtimes. IEEE Trans. Comput. 44 (10), 1223–1235. <https://doi.org/10.1109/12.467697>.
- Sacco, G.M., Schkolnick, M., 1982. A mechanism for managing the buffer pool in a relational database system using the hot set model. In: Eighth International Conference on Very Large Data Bases, Proceedings. September 8–10, 1982, Mexico City, Mexico. Morgan Kaufmann, pp. 257–262. <http://www.vldb.org/conf/1982/P257.PDF>.
- Sacco, G.M., Schkolnick, M., 1986. Buffer management in relational database systems. ACM Trans. Database Syst. 11 (4), 473–498. <https://doi.org/10.1145/7239.7336>.
- Sadiq, S. (Ed.), 2013. Handbook of Data Quality: Research and Practice. Springer-Verlag. ISBN 978-3-642-36257-6.
- Sahner, R.A., Trivedi, K.S., Puliafito, A., 1996. Performance and Reliability Analysis of Computer Systems. Kluwer Academic Publishers, Norwell, MA. ISBN 0792396502.
- Saito, Y., Frölund, S., Veitch, A.C., Merchant, A., Spence, S., 2004. FAB: building distributed enterprise disk arrays from commodity components. In: Mukherjee, S., McKinley, K.S. (Eds.), Proceedings of the 11th International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS 2004. Boston, MA, USA, October 7–13, 2004. ACM, pp. 48–58.

- Salapura, V., Blumrich, M.A., Gara, A., 2008. Design and implementation of the blue gene/p snoop filter. In: 14th International Conference on High-Performance Computer Architecture (HPCA-14 2008). 16–20 February 2008, Salt Lake City, UT, USA. IEEE Computer Society, pp. 5–14.
- Salem, K., Garcia-Molina, H., 1986. Disk striping. In: Proceedings of the Second International Conference on Data Engineering. February 5–7, 1986, Los Angeles, California, USA. IEEE Computer Society, pp. 336–342.
- Salla, A., Pughton, P., 2018. ABCs of z/OS Ayttem Programing Volume 10. Technical Report SG24-6990-05. IBM. <http://www.redbooks.ibm.com/redbooks/pdfs/sg246990.pdf>.
- Samet, H., 2006. Foundations of Multidimensional and Metric Data Structures. Morgan-Kaufmann Publishers, Burlington, MA. ISBN 3257227892.
- Sanders, D.A., Cremaldi, L.M., Eschenburg, V., Lawrence, D.N., Riel, C., Summers, D.J., Petravick, D.L., 2002. Redundant arrays of IDE drives. IEEE Trans. Nucl. Sci. 49 (4), 1834–1840. <https://doi.org/10.1109/TNN.2010.2099171>.
- Sandkerson, J., Kandrot, E., 2010. CUDA by Example: An Introduction to General-Purpose GPU Programming. Addison-Wesley. ISBN 978-0-13-138768-3.
- Sankar, S., Gurumurthi, S., Stan, M.R., 2008. Intra-disk parallelism: an idea whose time has come. In: 35th International Symposium on Computer Architecture (ISCA 2008). Beijing, China, June 21–25, 2008. IEEE Computer Society, pp. 303–314.
- Sathiamoorthy, M., Asteris, M., Papailiopoulos, D.S., Dimakis, A.G., Vadali, R., Chen, S., Borthakur, D., 2013. XORing Elephants: Novel Erasure Codes for Big Data. Proc. VLDB Endow. 6 (5), 325–336. <http://www.vldb.org/pvldb/vol6/p325-sathiamoorthy.pdf>.
- Sauer, C., Härder, T., Graefe, G., 2019. Instant restore after a media failure (extended version). Inf. Syst. 82, 90–101. <https://doi.org/10.1016/j.is.2018.11.001>.
- Sayood, K., 2017. Introduction to Data Compression, 5. Multimedia Information and Systems. Morgan Kaufmann Publishers, Burlingame, MA. ISBN 978-0128094747.
- Scherr, A.L., 1967. An Analysis of Time-Shared Computer Systems. The MIT Press, Cambridge, MA. ISBN 978-0262190343.
- Scheuermann, P., Weikum, G., Zaback, P., 1998. Data partitioning and load balancing in parallel disk systems. VLDB J. 7 (1), 48–66. <https://doi.org/10.1007/s007780050053>.
- Schindler, J., Griffin, J.L., Lumb, C.R., Ganger, G.R., 2002. Track-aligned extents: matching access patterns to disk drive characteristics. In: Long, D.D.E. (Ed.), Proceedings of the FAST '02 Conference on File and Storage Technologies. January 28–30, 2002, Monterey, California, USA. USENIX, pp. 259–274. <http://www.usenix.org/publications/library/proceedings/fast02/schindler.html>.
- Schindler, J., Shete, S., Smith, K.A., 2011. Improving throughput for small disk requests with proximal I/O. In: Ganger, G.R., Wilkes, J. (Eds.), 9th USENIX Conference on File and Storage Technologies. San Jose, CA, USA, February 15–17, 2011. USENIX, pp. 133–147. <http://www.usenix.org/events/fast11/tech/techAbstracts.html#Schindler>.
- Schkolnick, M., Sorenson, P.G., 1982. The effects of denormalisation on database performance. Austral. Comput. J. 14 (1), 12–18.
- Schlosser, S.W., Schindler, J., Papadomanolakis, S., Shao, M., Ailamaki, A., Faloutsos, C., Ganger, G.R., 2005. On multidimensional data and modern disks. In: Gibson, G. (Ed.), Proceedings of the FAST '05 Conference on File and Storage Technologies. December 13–16, 2005, San Francisco, California, USA. USENIX. <http://www.usenix.org/events/fast05/tech/schlosser.html>.
- Schmuck, F.B., Haskin, R.L., 2002. GPFS: a shared-disk file system for large computing clusters. In: Long, D.D.E. (Ed.), Proceedings of the FAST '02 Conference on File and Storage Technologies. January 28–30, 2002, Monterey, California, USA. USENIX, pp. 231–244. <http://www.usenix.org/publications/library/proceedings/fast02/schmuck.html>.
- Schroeder, B., Gibson, G.A., 2007a. Understanding disk failure rates: what does an MTTF of 1, 000, 000 hours mean to you? ACM Trans. Storage 3 (3), 8:1–8:31. <http://doi.acm.org/10.1145/1288783.1288785>.
- Schroeder, B., Gibson, G.A., 2007b. Disk failures in the real world: what does an MTTF of 1, 000, 000 hours mean to you? In: Arpaci-Dusseau, A.C., Arpaci-Dusseau, R.H. (Eds.), 5th USENIX Conference on File and Storage Technologies, FAST 2007. February 13–16, 2007, San Jose, CA, USA. USENIX, pp. 1–16. <http://www.usenix.org/events/fast07/tech/schroeder.html>.
- Schroeder, B., Pinheiro, E., Weber, W., 2009. DRAM errors in the wild: a large-scale field study. In: Douceur, J.R., Greenberg, A.G., Bonald, T., Nieh, J. (Eds.), Proceedings of the Eleventh International Joint Conference on Measurement and Modeling of Computer Systems, SIGMETRICS/Performance 2009. Seattle, WA, USA, June 15–19, 2009. ACM, pp. 193–204.
- Schroeder, B., Damouras, S., Gill, P., 2010a. Understanding latent sector errors and how to protect against them. ACM Trans. Storage 6 (3), 9:1–9:23. <http://doi.acm.org/10.1145/1837915.1837917>.
- Schroeder, B., Damouras, S., Gill, P., 2010b. Understanding latent sector errors and how to protect against them. In: Burns, R.C., Keeton, K. (Eds.), 8th USENIX Conference on File and Storage Technologies. San Jose, CA, USA, February 23–26, 2010. USENIX, pp. 71–84. http://www.usenix.org/events/fast10/tech/full_papers/schroeder.pdf.
- Schroeder, B., Pinheiro, E., Weber, W., 2011. DRAM errors in the wild: a large-scale field study. Commun. ACM 54 (2), 100–107. <https://doi.org/10.1145/1897816.1897844>.
- Schroeder, B., Lagisetty, R., Merchant, A., 2016. Flash reliability in production: the expected and the unexpected. In: Brown, A.D., Popovici, F.I. (Eds.), 14th USENIX Conference on File and Storage Technologies, FAST 2016. Santa Clara, CA, USA, February 22–25, 2016. USENIX Association, pp. 67–80. <https://www.usenix.org/conference/fast16/technical-sessions/presentation/schroeder>.
- Schulze, Martin, Gibson, Garth, Katz, Randy H., Patterson, David A., 1989. How reliable is a RAID? In: Digest of Papers, Thirty-Fourth IEEE Computer Society International Conference: Intellectual Leverage, COMPCON Spring 89. San Francisco, CA, USA, February 27–March 3, 1989. IEEE Computer Society, pp. 118–123.
- Schwabe, E.J., Sutherland, I.M., 1994. Improved parity-declustered layouts for disk arrays. In: SPAA, pp. 76–84. <http://doi.acm.org/10.1145/181014.181045>.
- Schwabe, E.J., Sutherland, I.M., 1996. Improved parity-declustered layouts for disk arrays. J. Comput. Syst. Sci. 53 (3), 328–343. <https://doi.org/10.1006/jcss.1996.0073>.

- Schwartz, J.T., 1980. Ultracomputers. *ACM Trans. Program. Lang. Syst.* 2 (4), 484–521. <https://doi.org/10.1145/357114.357116>.
- Schwartz, R., Dodge, J., Smith, N.A., Etzioni, O., 2020. Green AI. *Commun. ACM* 63 (12), 54–63. <https://doi.org/10.1145/3381831>.
- Schwarz, T.J.E., Burkhard, W.A., 1996. Almost complete address translation (ACATS) disk array declustering. In: *Proceedings of the Eighth IEEE Symposium on Parallel and Distributed Processing, SPDP 1996*. New Orleans, Louisiana, USA, October 23–26, 1996. IEEE Computer Society, pp. 324–331.
- Schwarz, T.J.E., Steinberg, J., Burkhard, W.A., 1999. Permutation development data layout (PDDL). In: *Proceedings of the Fifth International Symposium on High-Performance Computer Architecture*. Orlando, FL, USA, January 9–12, 1999. IEEE Computer Society, pp. 214–217.
- Schwarz, T.J.E., Pâris, J., Long, D.D.E., Amer, A., 2008. Increased reliability with spiral data layouts. In: Miller, E.L., Williamson, C.L. (Eds.), *16th International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS 2008)*. Baltimore, Maryland, USA, September 8–10, 2008. IEEE Computer Society, pp. 189–198.
- Schwarz, T.J.E., Amer, A., Kroeger, T., Miller, E.L., Long, D.D.E., Pâris, J., 2016. RESAR: reliable storage at exabyte scale. In: *24th IEEE International Symposium on Modeling, Analysis and Simulation of Computer and Telecommunication Systems, MASCOTS 2016*. London, United Kingdom, September 19–21, 2016. IEEE Computer Society, pp. 211–220.
- Seeger, B., 1996. An analysis of schedules for performing multi-page requests. *Inf. Syst.* 21 (5), 387–407. [https://doi.org/10.1016/0306-4379\(96\)84498-X](https://doi.org/10.1016/0306-4379(96)84498-X).
- Selinger, P.G., Adiba, M.E., 1980. Access path selection in distributed database management systems. In: Deen, S.M., Hammersley, P. (Eds.), *Proceedings International Conference on Data Bases, ICOD 1980*. University of Aberdeen, UK, July 1980. Heyden & Son, 1980, pp. 204–215.
- Selinger, P.G., Astrahan, M.M., Chamberlin, D.D., Lorie, R.A., Price, T.G., 1979. Access path selection in a relational database management system. In: Bernstein, P.A. (Ed.), *Proceedings of the 1979 ACM SIGMOD International Conference on Management of Data*. Boston, Massachusetts, USA, May 30–June 1. ACM, pp. 23–34.
- Sellis, T.K., Roussopoulos, N., Faloutsos, C., 1987. The r+-tree: a dynamic index for multi-dimensional objects. In: Stocker, P.M., Kent, W., Hammersley, P. (Eds.), *Vldb'87, Proceedings of 13th International Conference on Very Large Data Bases*. September 1–4, 1987, Brighton, England. Morgan Kaufmann, pp. 507–518. <http://www.vldb.org/conf/1987/P507.PDF>.
- Seltzer, M.I., 1993. Transaction support in a log-structured file system. In: *Proceedings of the Ninth International Conference on Data Engineering*. Vienna, Austria, April 19–23, 1993. IEEE Computer Society, pp. 503–510.
- Seltzer, M.I., Chen, P., Ousterhout, J.K., 1990. Disk scheduling revisited. In: *Proceedings of the Usenix Winter 1990 Technical Conference*. Washington, D.C. USA, 1990, pp. 1–10.
- Senko, M.E., Altman, E.B., Astrahan, M.M., Fehder, P.L., 1973. Data structures and accessing in data-base systems. I: evolution of information systems. *IBM Syst. J.* 12 (1), 30–44. <https://doi.org/10.1147/sj.121.0030>.
- Shafaei, M., Hajkazemi, M.H., Desnoyers, P., Aghayev, A., 2017. Modeling drive-managed SMR performance. *ACM Trans. Storage* 13 (4), 38:1–38:22. <https://doi.org/10.1145/3139242>.
- Shah, J., Saxena, V., 2011. Video encryption: a survey. *Int. J. Comput. Sci. Issues* 8 (2). www.IJCSI.org.
- Shalev-Shwartz, S., Ben-David, S., 2014. *Understanding Machine Learning: From Theory to Algorithms*. Cambridge University Press. ISBN 978-1-107-05713-5.
- Shamir, A., 1979. How to share a secret. *Commun. ACM* 22 (11), 612–613. <http://doi.acm.org/10.1145/359168.359176>.
- Shang, P., Wang, J., Zhu, H., Gu, P., 2011. A new placement-ideal layout for multiway replication storage system. *IEEE Trans. Comput.* 60 (8), 1142–1156. <https://doi.org/10.1109/TC.2010.175>.
- Shannon, C.E., 1949. A mathematical theory of communication. *Bell Syst. Tech. J.* 27, 379–423.
- Shao, M., Schlosser, S.W., Papadomanolakis, S., Schindler, J., Ailamaki, A., Ganger, G.R., 2007. Multimaps: preserving disk locality for multidimensional datasets. In: Chirkova, R., Dogac, A., Özsu, M.T., Sellis, T.K. (Eds.), *Proceedings of the 23rd International Conference on Data Engineering, ICDE 2007*. The Marmara Hotel, Istanbul, Turkey, April 15–20, 2007. IEEE Computer Society, pp. 926–935.
- Sharda, R., Delen, D., Turban, E., 2018. *Business Intelligence, Analytics, and Data Science*, 4. Pearson Education. ISBN 978-0134633282.
- Sharma, V., Thomas, A., Abdelzaher, T.F., Skadron, K., Lu, Z., 2003. Power-aware QoS management in Web servers. In: *Proceedings of the 24th IEEE Real-Time Systems Symposium (RTSS 2003)*. 3–5 December 2003, Cancun, Mexico. IEEE Computer Society, pp. 63–72.
- Sharnghani, H., Arora, K., 2000. Itanium processor architecture. *IEEE MICRO* (September–October), 24–43.
- Shaw, D.E., 1985. Relational query processing on the non-von supercomputer. In: Kim, W., Reiner, D.S., Batory, D. (Eds.), *Query Processing in Database Systems*. Springer. ISBN 978-3-642-82377-0, pp. 248–258.
- Shen, J.P., Lipasti, M., 2013. *Modern Processor Design: Fundamentals of Superscalar Processors*. Waveland Press. ISBN 978-1478607830.
- Shen, X., Yu, H., Buford, J., Akon, M. (Eds.), 2010. *Handbook of Peer-to-Peer Networking*. Springer. ISBN 978-0387097510.
- Shenoy, P.J., Vin, H.M., 1998. Cello: a disk scheduling framework for next generation operating systems. In: Vernon, M.K., Gibson, G., Latouche, G., Leutenegger, S.T. (Eds.), *Proceedings of the 1998 ACM SIGMETRICS Joint International Conference on Measurement and Modeling of Computer Systems, SIGMETRICS '98/PERFORMANCE '98*. Madison, Wisconsin, USA, June 22–26, 1998. ACM, pp. 44–55.
- Shilane, P., Huang, M., Wallace, G., Hsu, W., 2012. Wan-optimized replication of backup datasets using stream-informed delta compression. *ACM Trans. Storage* 8 (4), 13:1–13:26. <https://doi.org/10.1145/2385603.2385606>.
- Shirriff, K., Ousterhout, J.K., 1994. Sawmill: a high-bandwidth logging file system. In: *USENIX Summer 1994 Technical Conference, Conference Proceedings*. Boston, Massachusetts, USA, June 6–10, 1994. USENIX Association, pp. 125–136. <https://www.usenix.org/conference/usenix-summer-1994-technical-conference/sawmill-high-bandwidth-logging-file-system>.
- Shriver, E., 1999. A formalization of the attribute mapping problem. Technical Report HPL-SSP-1999-127. HP Labs. <https://www.hpl.hp.com/techreports/1999/HPL-1999-127.html>.

- Shriver, E.A.M., Merchant, A., Wilkes, J., 1998. An analytic behavior model for disk drives with readahead caches and request reordering. In: Vernon, M.K., Gibson, G., Latouche, G., Leutenegger, S.T. (Eds.), *Proceedings of the 1998 ACM SIGMETRICS Joint International Conference on Measurement and Modeling of Computer Systems, SIGMETRICS '98/PERFORMANCE '98*. Madison, Wisconsin, USA, June 22–26, 1998. ACM, pp. 182–191.
- Shum, K.W., 2011. Cooperative regenerating codes for distributed storage systems. In: *Proceedings of IEEE International Conference on Communications, ICC 2011*. Kyoto, Japan, 5–9 June, 2011. IEEE, pp. 1–5.
- Sidhu, P., 2019. Fungible's vision for the data-centric era. Technical report. Fungible. https://go.fungible.com/clients/fungible/uploads/pdf/Fungible_Data-Centric-Era_WhitePaper.pdf.
- Sierra, H.M., 2012. *An Introduction to Direct Access Storage Devices*. Computer Architecture and Design. Morgan Kaufmann, San Diego, CA. ISBN 978-0126425802. Academic Press, 1990.
- Siewiorek, D.P., Bell, G., Newell, A.C., 1981. *Computer Structures: Principles and Examples*, 2. McGraw-Hill, New York, NY. ISBN 0070573026.
- Silberschatz, A., Galvin, P.B., Gagne, G., 2018. *Operating System Concepts*, 10. Wiley. ISBN 978-1119456339.
- Silberschatz, A., Korth, H.F., Sudarshan, S., 2019. *Database Systems Concepts*, 7. McGraw-Hill, New York, NY. ISBN 978-0078022159.
- Silvester, J., Thomasian, A., 1981. Performance modeling of a large-scale multiprogrammed computer using best/1. In: Buzen, J. (Ed.), *Proc. 3rd Int'l. Conf. on Computer Capacity Management (ICCCM)*. Chicago, IL, pp. 197–211.
- Siman-Tov, B., 2019. The case for disaggregated storage. Technical report. Fungible. <https://go.fungible.com/clients/fungible/uploads/pdf/the-case-for-disaggregating-storage.pdf>.
- Simonyan, K., Zisserman, A., 2015. Very deep convolutional networks for large-scale image recognition. In: Bengio, Y., LeCun, Y. (Eds.), *3rd International Conference on Learning Representations, ICLR 2015, Conference Track Proceedings*. San Diego, CA, USA, May 7–9, 2015. <http://arxiv.org/abs/1409.1556>.
- Sirin, U., Ailamaki, A., 2020. Micro-architectural analysis of OLAP: limitations and opportunities. *Proc. VLDB Endow.* 13 (6), 840–853. <http://www.vldb.org/pvldb/vol13/p840-sirin.pdf>.
- Sitaram, D., Dan, A., 2000. *Multimedia Servers: Applications, Environments, and Designs*. Morgan Kaufmann Publishers, San Francisco, CA. ISBN 1558604308.
- Sites, R.L., 1978. An analysis of the cray-1 computer. In: McCluskey, E.J., et al. (Eds.), *Proceedings of the 5th Annual Symposium on Computer Architecture*. Palo Alto, CA, USA, April 1978. ACM, pp. 101–106.
- Sites, R.L., 1993. Alpha AXP architecture. *Commun. ACM* 36 (2), 33–44. <https://doi.org/10.1145/151220.151226>.
- Sivathanu, M., Prabhakaran, V., Arpaci-Dusseau, A.C., Arpaci-Dusseau, R.H., 2005. Improving storage system availability with D-GRAID. *ACM Trans. Storage* 1 (2), 133–170. <https://doi.org/10.1145/1063786.1063787>.
- Skeen, D., 1981. Nonblocking commit protocols. In: Lien, Y.E. (Ed.), *Proceedings of the 1981 ACM SIGMOD International Conference on Management of Data*. Ann Arbor, Michigan, USA, April 29–May 1, 1981. ACM Press, pp. 133–142.
- Slootman, F., 2011. *Tape Sucks: Inside Data Domain, a Silicon Valley Growth Story*. Together Editing. ISBN 0615484069.
- Slotnick, D.L., 1970. Logic per track devices. *Adv. Comput.* 10, 291–296. [https://doi.org/10.1016/S0065-2458\(08\)60440-4](https://doi.org/10.1016/S0065-2458(08)60440-4).
- Smaragdakis, Y., Kaplan, S.F., Wilson, P.R., 1999. EELRU: simple and effective adaptive page replacement. In: Menascé, D.A., Williamson, C. (Eds.), *Proceedings of the 1999 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems*. Atlanta, Georgia, USA, May 1–4, 1999. ACM, pp. 122–133.
- Smith, A.J., 1976a. A queueing network model for the effect of data compression on system efficiency. In: *American Federation of Information Processing Societies: 1976 National Computer Conference*. 7–10 June 1976, New York, NY, USA. In: *AFIPS Conference Proceedings*, vol. 45. AFIPS Press, pp. 457–465.
- Smith, A.J., 1976b. Analysis of the optimal, look-ahead demand paging algorithms. *SIAM J. Comput.* 5 (4), 743–757. <https://doi.org/10.1137/0205052>.
- Smith, A.J., 1978a. On the effectiveness of buffered and multiple arm disks. In: McCluskey, E.J., et al. (Eds.), *Proceedings of the 5th Annual Symposium on Computer Architecture*. Palo Alto, CA, USA, April 1978. ACM, pp. 242–248.
- Smith, A.J., 1978b. Sequentiality and prefetching in database systems. *ACM Trans. Database Syst.* 3 (3), 223–247. <http://doi.acm.org/10.1145/320263.320276>.
- Smith, A.J., 1978c. Bibliography on paging and related topics. *Oper. Syst. Rev.* 12 (4), 39–56. <https://doi.org/10.1145/775406.775409>.
- Smith, A.J., 1981a. Input/output optimization and disk architectures: a survey. *Perform. Eval.* 1 (2), 104–117. [https://doi.org/10.1016/0166-5316\(81\)90013-4](https://doi.org/10.1016/0166-5316(81)90013-4).
- Smith, A.J., 1981b. Analysis of long term file reference patterns for application to file migration algorithms. *IEEE Trans. Softw. Eng.* 7 (4), 403–417. <https://doi.org/10.1109/TSE.1981.230843>.
- Smith, A.J., 1981c. Bibliography on file and I/O system optimization and related topics. *Oper. Syst. Rev.* 15 (4), 39–54.
- Smith, A.J., 1982. Cache memories. *ACM Comput. Surv.* 14 (3), 473–530. <https://doi.org/10.1145/356887.356892>.
- Smith, A.J., 1985. Disk cache-miss ratio analysis and design considerations. *ACM Trans. Comput. Syst.* 3 (3), 161–203. <http://doi.acm.org/10.1145/3959.3961>.
- Smith, J.E., Nair, R., 2005. *Virtual Machines: Versatile Platforms for Systems and Processes*. Computer Architecture and Design. Morgan Kaufmann. ISBN 978-158609105.
- Smith, K.A., Seltzer, M.I., 1996. A comparison of FFS disk allocation policies. In: *Proceedings of the USENIX Annual Technical Conference*. San Diego, California, USA, January 22–26, 1996. USENIX Association, pp. 15–26.
- Smootherman, M.K., Jr., E.H.S., Robelen, R.J., 2016. The IBM ACS project. *IEEE Ann. Hist. Comput.* 38 (1), 60–74. <https://doi.org/10.1109/MAHC.2015.50>.

- Snir, M., Dongarra, J., Kowalik, J.S., Huss-Lederman, S., Otto, S.W., Walke, D.W., 1998. MPI: The Complete Reference (2-Volume Set), 2. Scientific and Engineering Computation. MIT Press. ISBN 978-0262692168.
- Snodgrass, R.T., 1999. Developing Time-Oriented Database Applications in SQL. Data Management Systems. Morgan Kaufmann. ISBN 1-55860-436-7.
- Sockut, G.H., Iyer, B.R., 2009. Online reorganization of databases. *ACM Comput. Surv.* 41 (3), 14:1–14:136. <http://doi.acm.org/10.1145/1541880.1541881>.
- Solomon, D., 2014. Data Compression: The Complete Reference Paperback. Springer. ISBN 978-8184898002.
- Solworth, J.A., Orji, C.U., 1991. Distorted mirrors. In: Proceedings of the First International Conference on Parallel and Distributed Information Systems (PDIS 1991). Fontainebleu Hilton Resort, Miami Beach, Florida, USA, December 4–6, 1991. IEEE Computer Society, pp. 10–17.
- Solworth, J.A., Orji, C.U., 1993. Distorted mapping techniques to achieve high performance in mirrored disk systems. *Distrib. Parallel Databases* 1 (1), 81–102. <https://doi.org/10.1007/BF01277521>.
- Son, S.W., Chen, G., Kandemir, M.T., 2005. Disk layout optimization for reducing energy consumption. In: Arvind, Rudolph, L. (Eds.), Proceedings of the 19th Annual International Conference on Supercomputing, ICS 2005. Cambridge, Massachusetts, USA, June 20–22, 2005. ACM, pp. 274–283.
- Song, S.S., 1981. On a High-Performance VLSI Solution to Database Problems. PhD thesis. Computer Science Dept. at CMU, Pittsburgh, PA.
- Sood, A.K., Qureshi, A.H., 1986. Database Machines: Modern Trends and Applications. NATO ASI Subseries F. Springer, New York, NY. ISBN 978-3642829390. Softcover reprint 2012.
- Spirn, J.R., 1977. Programme Behaviour: Models and Measurements. Operating & Programming Systems Series. Elsevier. ISBN 978-0444002198.
- Squires, J.P., Bagnell, G.N., Sander, C.M., Anderson, K.M., 1994. Multiple actuator disk drive. <https://patents.google.com/patent/EP0505363A1/en>. US patent 9209077.
- Staff, T., 1985. DBC/1012 Database Computer System Manual. Teradata Corp., 10000 Innovation Drive, Dayton, OH. 2.0, Document No. C10-0001-02.
- Stallings, W., 2017a. Cryptography and Network Security: Principles and Practice, 7. Pearson, India. ISBN 9789332585225.
- Stallings, W., 2017b. Operating Systems: Internals and Design Principles, 9. Pearson. ISBN 978-0134670959.
- Stallings, W., 2019. Computer Architecture and Organization, 11. Pearson. ISBN 978-0134997193.
- Stamos, J.W., Cristian, F., 1993. Coordinator log transaction execution protocol. *Distrib. Parallel Databases* 1 (4), 383–408. <https://doi.org/10.1007/BF01264014>.
- Stevens, W.R. Heuristics for disk drive positioning in 4.3 BSD. *Comput. Syst.* 2 (3), 251–274. http://www.usenix.org/publications/compsystems/1989/sum_stevens.pdf.
- Stewart, W.J., 1989. Probability, Markov Chains, Queues, and Simulation: The Mathematical Basis of Performance Modeling. Princeton University Press. ISBN 978-0-691-14062-9.
- Stewart, W.J., 2021. Introduction to the Numerica: Solution of Markov Chains. Princeton University Press. ISBN 978-0691036991.
- Stockmeyer, L., 1994. Parallelism in parity-declustered layouts for disk arrays. Technical Report RJ 9915 (87295). IBM Almaden Research Center.
- Stodolsky, D., Holland, M., Courtright II, W.V., Gibson, G.A., 1994. Parity-logging disk arrays. *ACM Trans. Comput. Syst.* 12 (3), 206–235. <http://doi.acm.org/10.1145/185514.185516>.
- Stoica, I., Morris, R.T., Karger, D.R., Kaashoek, M.F., Balakrishnan, H., 2001. Chord: a scalable peer-to-peer lookup service for internet applications. In: Cruz, R.L., Varghese, G. (Eds.), Proceedings of the ACM SIGCOMM 2001 Conference on Applications, Technologies, Architectures, and Protocols for Computer Communication. August 27–31, 2001, San Diego, CA, USA. ACM, pp. 149–160.
- Stoica, Ion, Shenker, Scott, 2021. From cloud computing to sky computing. In: Angel, Sebastian, Kasikci, Baris, Kohler, Eddie (Eds.), HotOS '21: Workshop on Hot Topics in Operating Systems. Ann Arbor, Michigan, USA, June, 1–3, 2021. ACM, pp. 26–32.
- Stoica, R., Athanassoulis, M., Johnson, R., Ailamaki, A., 2009. Evaluating and repairing write performance on flash devices. In: Boncz, P.A., Ross, K.A. (Eds.), Proceedings of the Fifth International Workshop on Data Management on New Hardware, DaMoN 2009. Providence, Rhode Island, USA, June 28, 2009. ACM, pp. 9–14.
- Stonebraker, M., 1986. The case for shared nothing. *IEEE Database Eng. Bull.* 9 (1), 4–9. <http://sites.computer.org/debull/86MAR-CD.pdf>.
- Stonebraker, M., 1994. In memory of Bob Kooi (1951–1993). *SIGMOD Rec.* 23 (1), 3. <http://doi.acm.org/10.1145/181550.181551>.
- Stonebraker, M., Çetintemel, U., 2005. “One size fits all”: an idea whose time has come and gone (abstract). In: Aberer, K., Franklin, M.J., Nishio, S. (Eds.), Proceedings of the 21st International Conference on Data Engineering, ICDE 2005. 5–8 April 2005, Tokyo, Japan. IEEE Computer Society, pp. 2–11.
- Stonebraker, M., Moore, D., 1996. Object-Relational DBMSs: The Next Great Wave. Morgan Kaufmann Publishers. ISBN 1-55860-397-2.
- Stonebraker, M., Rowe, L.A., 1986. The design of postgres. In: Zaniolo, C. (Ed.), Proceedings of the 1986 ACM SIGMOD International Conference on Management of Data. Washington, DC, USA, May 28–30, 1986. ACM Press, pp. 340–355.
- Stonebraker, M., Wong, E., Kreps, P., Held, G., 1976. The design and implementation of INGRES. *ACM Trans. Database Syst.* 1 (3), 189–222. <https://doi.org/10.1145/320473.320476>.
- Stonebraker, M., Anton, J., Hanson, E.N., 1987. Extending a database system with procedures. *ACM Trans. Database Syst.* 12 (3), 350–376. <https://doi.org/10.1145/27629.27631>.
- Stonebraker, M., Abadi, D.J., Batkin, A., et al., 2005. C-store: a column-oriented DBMS. In: Böhm, K., Jensen, C.S., Haas, L.M., Kersten, M.L., Larson, P., Ooi, B.C. (Eds.), Proceedings of the 31st International Conference on Very Large Data Bases. Trondheim, Norway, August 30–September 2, 2005. ACM, pp. 553–564. <http://www.vldb.org/archives/website/2005/program/paper/thu/p553-stonebraker.pdf>.
- Stonebraker, M., Bear, C., Çetintemel, U., et al., 2007a. One size fits all? Part 2: Benchmarking studies. In: CIDR 2007, Third Biennial Conference on Innovative Data Systems Research, Online Proceedings. Asilomar, CA, USA, January 7–10, 2007, pp. 173–184. [www.cidrdb.org](http://cidrdb.org/cidr2007/papers/cidr07p20.pdf). <http://cidrdb.org/cidr2007/papers/cidr07p20.pdf>.

- Stonebraker, M., Madden, S., Abadi, D.J., Harizopoulos, S., Hachem, N., Helland, P., 2007b. The end of an architectural era (it's time for a complete rewrite). In: Koch, C., Gehrke, J., Garofalakis, M.N., et al. (Eds.), *Proceedings of the 33rd International Conference on Very Large Data Bases*. University of Vienna, Austria, September 23–27, 2007. ACM, pp. 1150–1160. <http://www.vldb.org/conf/2007/papers/industrial/p1150-stonebraker.pdf>.
- Stonebraker, M., Wong, E., Kreps, P., Held, G., 2014. The design and implementation of INGRES. In: Brodie, M. (Ed.), *Making Databases Work: The Pragmatic Wisdom of Michael Stonebraker*. ACM/Morgan & Claypool, pp. 561–605.
- Storer, J.A., 1988. *Data Compression: Methods and Theory*. Principles of Computer Science Series. Computer Science Press. ISBN 978-0716781561.
- Storer, M.W., Greenan, K.M., Miller, E.L., Voruganti, K., 2008. Pergamum: replacing tape with energy efficient, reliable, disk-based archival storage. In: Baker, M., Riedel, E. (Eds.), *6th USENIX Conference on File and Storage Technologies, FAST 2008*. February 26–29, 2008, San Jose, CA, USA. USENIX, pp. 1–16. <http://www.usenix.org/events/fast08/tech/storer.html>.
- Storer, M.W., Greenan, K.M., Miller, E.L., Voruganti, K., 2009. POTSHARDS – a secure, recoverable, long-term archival storage system. *ACM Trans. Storage* 5 (2), 5:1–5:35. <https://doi.org/10.1145/1534912.1534914>.
- Strecker, W.D., 1976. Cache memories for PDP-11 family computers. In: Flynn, M.J., Garcia, O.N., Siewiorek, D.P. (Eds.), *Proceedings of the 3rd Annual Symposium on Computer Architecture*. Clearwater, FL, USA, January 1976. ACM, pp. 155–158.
- Strunk, J.D., 2008. *Using Utility Functions to Control a Distributed Storage System*. PhD thesis. Dept. of Electrical and Computer Engineering, CMU, Pittsburgh, PA.
- Strunk, J.D., Thereska, E., Faloutsos, C., Ganger, G.R., 2008. Using utility to provision storage systems. In: Baker, M., Riedel, E. (Eds.), *6th USENIX Conference on File and Storage Technologies, FAST 2008*. February 26–29, 2008, San Jose, CA, USA. USENIX, pp. 313–328. <http://www.usenix.org/events/fast08/tech/strunk.html>.
- Su, S.Y.W., 1988. *Database Computers Principles, Architectures, and Techniques*. McGraw-Hill, New York, NY. ISBN 0070622957.
- Subbiah, A., Blough, D.M., 2005. An approach for fault tolerant and secure data storage in collaborative work environments. In: Atluri, V., Samarati, P., Yurcik, W., Brumbaugh, L., Zhou, Y. (Eds.), *Proceedings of the 2005 ACM Workshop on Storage Security and Survivability, StorageSS 2005*. Fairfax, VA, USA, November 11, 2005. ACM, pp. 84–93.
- Subramanian, R., Goodman, B.D. (Eds.), 2005. *Peer-to-Peer Computing: The Evolution of a Disruptive Technology*. Computers – Organizations and Data Processing. Idea Group Publishing. ISBN 978-1591404293.
- Sukhwani, B., Thoennes, M., Min, H., Dube, P., Brezzo, B., Asaad, S.W., Dillenberger, D., 2015. A hardware/software approach for database query acceleration with fpgas. *Int. J. Parallel Program.* 43 (6), 1129–1159. <https://doi.org/10.1007/s10766-014-0327-4>.
- Sundaram, R., 2006. The private lives of disk drives. http://www.netapp.com/go/techontap/mat/sample/0206tot_resiliency.htm.
- Surianarayanan, C., Ganapathy, G., Raj, P., 2002. *Essentials of Microservices Architecture: Paradigms, Applications, and Techniques*. CRC Press. ISBN 978-0367249953.
- Takagi, H., 1986. *Analysis of Polling Systems*. MIT Press, Cambridge, MA. ISBN 978-0262200578.
- Takagi, H., 1991. *Queueing Analysis: Foundations of Performance Evaluation, Vol. 1: Vacation and Priority Systems, Part 1*, 3. North Holland, Amsterdam, Holland. ISBN 044488910.
- Tamo, I., Wang, Z., Bruck, J., 2011. Zigzag codes: MDS array codes with optimal rebuilding. *CoRR*, arXiv:1112.0371.
- Tanabe, T., Takayanagi, M., Tatemichi, H., Ura, T., Yamamoto, M., 1999. Redundant optical storage system using DVD-RAM library. In: *16th IEEE Symposium on Mass Storage Systems in cooperation with the 7th NASA Goddard Space Flight Center Conference on Mass Storage Systems and Technologies*. San Diego, California, USA, March 15–18, 1999. IEEE Computer Society, pp. 80–87.
- Tandem Database Group, 1989. NonStop SQL: a distributed, high-performance, high-availability implementation of sql. In: Gawlick, D., Haynie, M.N., Reuter, A. (Eds.), *Proc. 2nd Int'l Workshop on High Performance Transaction Systems (HPTS'87)*. In: *Lecture Notes in Computer Science*, vol. 359. Springer, Pacific Grove, CA, pp. 60–104.
- Tanenbaum, Andrew S., Austin, Todd, 2012. *Structured Computer Organization*, 6th edition. Pearson. ISBN 978-0132916523.
- Tanenbaum, A.S., Bos, H., 2017. *Modern Operating Systems*, 4. Pearson. ISBN 978-9332575776.
- Tanenbaum, A.S., Weaherall, D.J., 2013. *Computer Networks*, 5. Pearson. ISBN 978-8131787571.
- Tanenbaum, A.S., Woodhull, A.S., 2006. *Operating Systems Design and Implementation*, 3. Pearson. ISBN 978-0131429383.
- Taranov, I., et al., 2010. Sedna: native XML database management system (internals overview). In: Elmagarmid, A.K., Agrawal, D. (Eds.), *Proceedings of the ACM SIGMOD International Conference on Management of Data, SIGMOD 2010*. Indianapolis, Indiana, USA, June 6–10, 2010. ACM, pp. 1037–1046.
- Tay, Y.C., Goodman, N., Suri, R., 1985. Locking performance in centralized databases. *ACM Trans. Database Syst.* 10 (4), 415–462. <https://doi.org/10.1145/4879.4880>.
- Taylor, I.J., 2015. *From P2P to Web Services and Grids: Peers in a Client/Server World*. Computer Communications and Networks. Springer. ISBN 978-1852338695.
- Teodosiu, D., Björner, N., Gurevich, Y., Manasse, M., Porkka, J., 2006. Optimizing file replication over limited bandwidth networks using remote differential compression. Technical Report MSR-TR-2006-157. Microsoft Research.
- Teorey, T.J., Pinkerton, T.B., 1972. A comparative analysis of disk scheduling policies. *Commun. ACM* 15 (3), 177–184. <http://doi.acm.org/10.1145/361268.361278>.
- Teubner, J., Woods, L., 2013. *Data Processing on FPGAs. Synthesis Lectures on Data Management*. Morgan & Claypool. ISBN 978-1627050609.
- Thadani, A.J., 1981. Interactive user productivity. *IBM Syst. J.* 20 (4), 407–423. <https://doi.org/10.1147/sj.204.0407>.
- Thereska, E., Abd-El-Malek, M., Wylie, J.J., Narayanan, D., Ganger, G.R., 2006. Informed data distribution selection in a self-predicting storage system. In: *Proceedings of the 3rd International Conference on Autonomic Computing, ICAC 2006*. Dublin, Ireland, 13–16 June 2006. IEEE Computer Society, pp. 187–198. <http://ieeexplore.ieee.org/document/1662398/>.

- Thereska, E., Donnelly, A., Narayanan, D., 2011. Sierra: practical power-proportionality for data center storage. In: Kirsch, C.M., Heiser, G. (Eds.), *European Conference on Computer Systems, Proceedings of the Sixth European Conference on Computer systems, EuroSys 2011*. 2011, Salzburg, Austria, April 10–13, 2011. ACM, pp. 169–182.
- Thiébaud, D., Stone, H.S., 1987. Footprints in the cache. *ACM Trans. Comput. Syst.* 5 (4), 305–329. <https://doi.org/10.1145/29868.32979>.
- Thinking Machines Corp., 1987. Connection machine model cm-2 technical summary. Technical Report HA87-4. Thinking Machines Corp.
- Thomasian, A., 1978. A capacity bound for parallel processing. In: *Proc. Int'l. Conf. on Parallel Processing ICPP*. IEEE, Bellaire, MI.
- Thomasian, A., 1993. Two-phase locking performance and its thrashing behavior. *ACM Trans. Database Syst.* 18 (4), 579–625. <http://doi.acm.org/10.1145/169725.169720>.
- Thomasian, A., 1995a. Rebuild options in RAID5 disk arrays. In: *Proceedings of the Seventh IEEE Symposium on Parallel and Distributed Processing, SPDP 1995*. San Antonio, Texas, USA, October 25–28, 1995. IEEE, pp. 511–518.
- Thomasian, A., 1995b. Checkpointing for optimistic concurrency control methods. *IEEE Trans. Knowl. Data Eng.* 7 (2), 332–339. <https://doi.org/10.1109/69.382303>.
- Thomasian, A., 1998. Concurrency control: methods, performance, and analysis. *ACM Comput. Surv.* 30 (1), 70–119. <https://doi.org/10.1145/274440.274443>.
- Thomasian, A., 2005a. Reconstruct versus read-modify writes in RAID. *Inf. Process. Lett.* 93 (4), 163–168. <https://doi.org/10.1016/j.ipl.2004.10.009>.
- Thomasian, A., 2005b. Clustered RAID arrays and their access costs. *Comput. J.* 48 (6), 702–713. <https://doi.org/10.1093/comjnl/bxh108>.
- Thomasian, A., 2006a. Shortcut method for reliability comparisons in RAID. *J. Syst. Softw.* 79 (11), 1599–1605. <https://doi.org/10.1016/j.jss.2006.02.035>.
- Thomasian, A., 2006b. Mirrored disk routing and scheduling. *Clust. Comput.* 9 (4), 475–484. <https://doi.org/10.1007/s10586-006-0014-3>.
- Thomasian, A., 2006c. Comment on 'RAID5 performance with distributed sparing'. *IEEE Trans. Parallel Distrib. Syst.* 17 (4), 399–400. <https://doi.org/10.1109/TPDS.2006.41>.
- Thomasian, A., 2006d. Multi-level RAID for very large disk arrays. *ACM SIGMETRICS Perform. Eval. Rev.* 33 (4), 17–22. <http://doi.acm.org/10.1145/1138085.1138091>.
- Thomasian, A., 2010a. Storage research in industry and universities. *SIGARCH Comput. Archit. News* 38 (2), 1–48. <http://doi.acm.org/10.1145/1823838.1823840>.
- Thomasian, A., 2010b. Why specialized disks for composite operations may be unnecessary. *SIGARCH Comput. Archit. News* 38 (5), 20–27. <https://doi.org/10.1145/1978907.1978912>.
- Thomasian, A., 2011. Survey and analysis of disk scheduling methods. *SIGARCH Comput. Archit. News* 39 (2), 8–25. <http://doi.acm.org/10.1145/2024716.2024719>.
- Thomasian, A., 2012a. Comment on “DACO: a high performance disk architecture”. *IEEE Trans. Comput.* 61 (4), 588–590. <https://doi.org/10.1109/TC.2011.36>.
- Thomasian, A., 2012b. Rebuild processing in RAID5 with emphasis on the supplementary parity augmentation method[37]. *SIGARCH Comput. Archit. News* 40 (2), 18–27. <https://doi.org/10.1145/2234336.2234340>.
- Thomasian, A., 2013. Improved storage system performance by disk scheduling. *Comput. Syst. Sci. Eng.* 28 (2).
- Thomasian, A., 2014a. Performance evaluation of computer systems. In: Gonzalez, T.F., Diaz-Herrera, J., Tucker, A. (Eds.), *Computing Handbook, Third Edition: Computer Science and Software Engineering*. CRC Press, p. 56.
- Thomasian, A., 2014b. Analysis of fork/join and related queueing systems. *ACM Comput. Surv.* 47 (2), 17:1–17:71. <http://doi.acm.org/10.1145/2628913>.
- Thomasian, A., 2018a. Mirrored and hybrid disk arrays: organization, scheduling, reliability, and performance. *CoRR*, arXiv:1801.08873.
- Thomasian, A., 2018b. Vacationing server model for M/G/1 queues for rebuild processing in RAID5 and threshold scheduling for readers and writers. *Inf. Process. Lett.* 135, 41–46. <https://doi.org/10.1016/j.ipl.2018.02.011>.
- Thomasian, A., 2018c. Performance analysis of transaction processing systems. In: Liu, L., Özsu, M.T. (Eds.), *Encyclopedia of Database Systems, Second Edition*. Springer.
- Thomasian, A., Avizienis, A., 1976. A design study of a shared resource computing system. In: Flynn, M.J., Garcia, O.N., Siewiorek, D.P. (Eds.), *Proceedings of the 3rd Annual Symposium on Computer Architecture*. Clearwater, FL, USA, January 1976. ACM, pp. 105–112.
- Thomasian, A., Bay, P.F., 1986. Analytic queueing network models for parallel processing of task systems. *IEEE Trans. Comput.* 35 (12), 1045–1054. <https://doi.org/10.1109/TC.1986.1676712>.
- Thomasian, A., Blaum, M., 2006. Mirrored disk organization reliability analysis. *IEEE Trans. Comput.* 55 (12), 1640–1644. <https://doi.org/10.1109/TC.2006.201>.
- Thomasian, A., Blaum, M., 2009. Higher reliability redundant disk arrays: organization, operation, and coding. *ACM Trans. Storage* 5 (3), 7:1–7:59. <http://doi.acm.org/10.1145/1629075.1629076>.
- Thomasian, A., Fu, G., 2006. Anticipatory disk arm placement to reduce seek time. *Comput. Syst. Sci. Eng.* 21 (3).
- Thomasian, A., Gargeya, K., 1984. Speeding up computer system simulations using hierarchical modeling. *ACM SIGMETRICS Perform. Eval. Rev.* 12 (4), 34–39. <https://doi.org/10.1145/1041831.1041834>.
- Thomasian, A., Han, C., 2005a. Affinity-based routing in zoned mirrored disks. *Comput. J.* 48 (3), 292–299. <https://doi.org/10.1093/comjnl/bxh085>.
- Thomasian, A., Han, C., 2005b. Heterogeneous disk array architecture and its data allocation policies. In: *Proc. Int'l Symp. on Performance Evaluation of Computer and Telecommunication Systems (SPECTS)*. Cherry-Hill, NJ, July 2005, pp. 617–624.
- Thomasian, A., Liu, C., 2002a. Some new disk scheduling policies and their performance. In: Muntz, R.R., Martonosi, M., de Souza e Silva, E. (Eds.), *Proceedings of the International Conference on Measurements and Modeling of Computer Systems, SIGMETRICS 2002*. June 15–19, 2002, Marina Del Rey, California, USA. ACM, 2002, pp. 266–267.

- Thomasian, A., Liu, C., 2002b. Disk scheduling policies with lookahead. *ACM SIGMETRICS Perform. Eval. Rev.* 30 (2), 31–40. <https://doi.org/10.1145/588160.588165>.
- Thomasian, A., Liu, C., 2004. Empirical performance analysis of the sptf disk scheduling policy. In: *Proc. Int'l Symp. Performance Evaluation Computer and Telecommunication Systems – SPECTS'04*. Society for Modeling and Simulation, pp. 398–401.
- Thomasian, A., Liu, C., 2005a. Comment on “Issues and challenges in the performance analysis of real disk arrays”. *IEEE Trans. Parallel Distrib. Syst.* 16 (11), 1103–1104. <https://doi.org/10.1109/TPDS.2005.132>.
- Thomasian, A., Liu, C., 2005b. Performance comparison of mirrored disk scheduling methods with a shared non-volatile cache. *Distrib. Parallel Databases* 18 (3), 253–281. <https://doi.org/10.1007/s10619-005-4963-y>.
- Thomasian, A., Menon, J., 1994. Performance analysis of RAID5 disk arrays with a vacationing server model for rebuild mode operation. In: *Proceedings of the Tenth International Conference on Data Engineering*. Houston, Texas, USA, February 14–18, 1994. IEEE Computer Society, pp. 111–119.
- Thomasian, A., Menon, J., 1997. RAID5 performance with distributed sparing. *IEEE Trans. Parallel Distrib. Syst.* 8 (6), 640–657. <https://doi.org/10.1109/71.595583>.
- Thomasian, A., Nadji, B., 1981a. Algorithms for queueing network models of multiprogrammed computers using generating functions. *Comput. Perform.* 2 (3), 100–123. <https://doi.org/10.1007/s11042-007-0179-7>.
- Thomasian, A., Nadji, B., 1981b. Aggregation of station in queueing network models of multiprogrammed computers. In: Schwetman, H.D., Sherman, S.W. (Eds.), *Proceedings of the 1981 ACM SIGMETRICS Conference on Measurement and Modeling of Computer Systems*. Las Vegas, Nevada, USA., September 14–16, 1981. ACM, pp. 86–96.
- Thomasian, A., Nicola, V.F., 1993. Performance evaluation of a threshold policy for scheduling readers and writers. *IEEE Trans. Comput.* 42 (1), 83–98. <https://doi.org/10.1109/12.192216>.
- Thomasian, A., Ryu, I.K., 1983. A decomposition solution to the queueing network model of the centralized DBMS with static locking. In: Schwetman, H.D., Bruell, S.C., Dowdy, L.W. (Eds.), *Proceedings of the International Conference on Measurements and Modeling of Computer Systems, SIGMETRICS 1983*. August 29–31, 1983, Minneapolis, Minnesota, USA. ACM, pp. 82–92.
- Thomasian, A., Ryu, I.K., 1985. Analysis of some optimistic concurrency control schemes based on certification. In: Schwetman, H.D., Brumfield, J.A., Reed, D.A., Gaither, B.D. (Eds.), *Proceedings of the 1985 ACM SIGMETRICS Conference on Measurement and Modeling of Computer Systems*. Austin, Texas, USA, August 26–29, 1985. ACM, pp. 192–203.
- Thomasian, A., Ryu, I.K., 1991. Performance analysis of two-phase locking. *IEEE Trans. Softw. Eng.* 17 (5), 386–402. <https://doi.org/10.1109/32.90443>.
- Thomasian, A., Tang, Y., 2011a. Hierarchical raid: organization, operation, reliability, and performance. In: *Proc. 7th IEEE Int'l Workshop on Storage Network Architecture and Parallel I/O (SNAPI)*. Denver, Colorado, May 2011. IEEE.
- Thomasian, A., Tang, Y., 2011b. Performance, reliability, and performability aspects of hierarchical RAID. In: *Sixth International Conference on Networking, Architecture, and Storage, NAS 2011*. Dalian, China, 28–30 July 2011. IEEE Computer Society, pp. 92–101.
- Thomasian, A., Tang, Y., 2012. Performance, reliability, and performability of a hybrid RAID array and a comparison with traditional RAID1 arrays. *Clust. Comput.* 15 (3), 239–253. <https://doi.org/10.1007/s10586-012-0216-9>.
- Thomasian, A., Tantawi, A.N., 1994. Approximate solutions for M/G/1 fork/join synchronization. In: Sadowski, D.A., Seila, A.F., Manivannan, M.S., Tew, J.D. (Eds.), *Proceedings of the 26th Conference on Winter Simulation, WSC 1994*. Lake Buena Vista, FL, USA, December 11–14, 1994. ACM, pp. 361–368.
- Thomasian, A., Xu, J., 2008. Reliability and performance of mirrored disk organizations. *Comput. J.* 51 (6), 615–629. <https://doi.org/10.1093/comjnl/bxm079>.
- Thomasian, Alexander, Xu, Jun, 2011a. RAID level selection for heterogeneous disk arrays. *Clust. Comput.* 14 (2), 115–127. <https://doi.org/10.1007/s10586-010-0129-4>.
- Thomasian, A., Xu, J., 2011b. X-code double parity array operation with two disk failures. *Inf. Process. Lett.* 111 (12), 568–574. <https://doi.org/10.1016/j.ipl.2011.03.001>.
- Thomasian, A., Xu, J., 2011c. Data allocation in heterogeneous disk arrays. In: *Sixth International Conference on Networking, Architecture, and Storage, NAS 2011*. Dalian, China, 28–30 July 2011. IEEE Computer Society, pp. 82–91.
- Thomasian, A., Xu, J., 2016. Data allocation in a heterogeneous disk array (HDA) with multiple RAID levels for database applications. *Comput. Syst. Sci. Eng.* 31 (5).
- Thomasian, A., Zhang, L., 2006. The stepwise dimensionality increasing (SDI) index for high-dimensional data. *Comput. J.* 49 (5), 609–618. <https://doi.org/10.1093/comjnl/bxl022>.
- Thomasian, A., Zhang, L., 2008. Persistent clustered main memory index for accelerating k-nn queries on high dimensional datasets. *Multimed. Tools Appl.* 38 (2), 253–270. <https://doi.org/10.1007/s11042-007-0179-7>.
- Thomasian, A., Castelli, V., Li, C., 1998. Clustering and singular value decomposition for approximate indexing in high dimensional spaces. In: Gardarin, G., French, J.C., Pissinou, N., Makki, K., Bouganim, L. (Eds.), *Proceedings of the 1998 ACM CIKM International Conference on Information and Knowledge Management*. Bethesda, Maryland, USA, November 3–7, 1998. ACM, pp. 201–207.
- Thomasian, A., Han, C., Fu, G., Liu, C., 2004. A performance evaluation tool for RAID disk arrays. In: *1st International Conference on Quantitative Evaluation of Systems (QEST 2004)*. 27–30 September 2004, Enschede, The Netherlands. IEEE Computer Society, pp. 8–17.
- Thomasian, A., Branzoi, B.A., Han, C., 2005. Performance evaluation of a heterogeneous disk array architecture. In: *13th International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS 2005)*. 27–29 September 2005, Atlanta, GA, USA. IEEE Computer Society, pp. 517–520.
- Thomasian, A., Fu, G., Han, C., 2007a. Performance of two-disk failure-tolerant disk arrays. *IEEE Trans. Comput.* 56 (6), 799–814. <https://doi.org/10.1109/TC.2007.1041>.

- Thomasian, A., Fu, G., Ng, S.W., 2007b. Analysis of rebuild processing in RAID5 disk arrays. *Comput. J.* 50 (2), 217–231. <https://doi.org/10.1093/comjnl/bxl064>.
- Thomasian, A., Li, Y., Zhang, L., 2008. Optimal subspace dimensionality for k-nearest-neighbor queries on clustered and dimensionality reduced datasets with SVD. *Multimed. Tools Appl.* 40 (2), 241–259. <https://doi.org/10.1007/s11042-008-0206-3>.
- Thomasian, A., Tang, Y., Hu, Y., 2012. Hierarchical RAID: design, performance, reliability, and recovery. *J. Parallel Distrib. Comput.* 72 (12), 1753–1769. <https://doi.org/10.1016/j.jpdc.2012.07.002>.
- Thornton, J.E., 1970. Design of a Computer: The Control Data 6600. Scott Foresmann and Company. ISBN 978-0673059536.
- Tian, L., Jiang, H., Feng, D., Xin, Q., Shu, X., 2007. Implementation and evaluation of a popularity-based reconstruction optimization algorithm in availability-oriented disk arrays. In: 24th IEEE Conference on Mass Storage Systems and Technologies (MSST 2007). 24–27 September 2007, San Diego, California, USA. IEEE Computer Society, pp. 233–238. <http://doi.ieeecomputersociety.org/10.1109/MSST.2007.20>.
- Tian, L., Cao, Q., Jiang, H., Feng, D., Xie, C., Xin, Q., 2011. Online availability upgrades for parity-based raids through supplementary parity augmentations. *ACM Trans. Storage* 6 (4), 17:1–17:23. <https://doi.org/10.1145/1970338.1970341>.
- Tizno, O., Marshall, A.R., Fernandez-Delgado, N., Herrera, M., Molina, S.I., Hayne, M., 2019. Room-temperature operation of low-voltage, non-volatile, compound-semiconductor memory cells. *Sci. Rep.* 9 (1). <https://doi.org/10.1038/s41598-019-45370-1>.
- To, T.J., Hamidzadeh, B., 1999. Dynamic real-time scheduling strategies for interactive continuous media servers. *Multimed. Syst.* 7 (2), 91–106. <https://doi.org/10.1007/s005300050113>.
- Tomasello, R., Martinez, E., Zivieri, R., et al., 2014. A strategy for the design of skyrmion racetrack memories. *Sci. Rep.* 4 (6784). <https://doi.org/10.1038/srep06784>.
- Tomkins, A., Patterson, R.H., Gibson, G.A., 1997. Informed multi-process prefetching and caching. In: Zahorjan, J., Greenberg, A.G., Leutenegger, S.T. (Eds.), *Proceedings of the 1997 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems*. Seattle, Washington, USA, June 15–18, 1997. ACM, pp. 100–114.
- Towsley, D.F., Chen, S., 1991. Design and evaluation of scheduling policies for two-server fork/join queueing systems. COINS Technical Report 91-39. Dept. of Computer and Information Science, University of Massachusetts, Amherst, MA.
- Towsley, D.F., Chen, S., Yu, S., 1990. Performance analysis of a fault tolerant mirrored disk system. In: King, P.J.B., Mitrani, I., Pooley, R. (Eds.), *Performance '90, Proceedings of the 14th IFIP WG 7.3 International Symposium on Computer Performance Modelling, Measurement and Evaluation*. Edinburgh, Scotland, 12–14 September 1990. North-Holland, pp. 239–253.
- Treiber, K., Menon, J., 1995. Simulation study of cached RAID5 designs. In: *Proceedings of the 1st IEEE Symposium on High-Performance Computer Architecture*. HPCA 1995, Raleigh, North Carolina, USA, January 22–25, 1995. IEEE Computer Society, pp. 186–197.
- Tremaine, R.B., Franaszek, P.A., Robinson, J.T., Schulz, C.O., Smith, T.B., Wazlowski, M.E., Bland, P.M., 2001a. IBM memory expansion technology (MXT). *IBM J. Res. Dev.* 45 (2), 271–286. <https://doi.org/10.1147/rd.452.0271>.
- Tremaine, R., Brett, Smith, T., Basil, Wazlowski, Michael E., Har, David, Mak, Kwok-Ken, Arramreddy, Sujith, 2001b. Pinnacle: IBM MXT in a memory controller chip. *IEEE Micro* 21 (2), 56–68. <https://doi.org/10.1109/40.918003>.
- Trivedi, K., 1982. *Probability and Statistics with Reliability, Queueing, and Computer Science Applications*. Prentice-Hall, Saddle-River, NJ. ISBN 0137115644.
- Trivedi, K.S., 2002. *Probability and Statistics with Reliability, Queueing, and Computer Science Applications*, 2. Wiley. ISBN 978-0471333418.
- Trivedi, K.S., Bobbio, A., 2017. *Reliability and Availability Engineering: Modeling, Analysis, and Applications*. Cambridge University Press. ISBN 978-1107099500.
- Trivedi, K.S., Geist, R.M., 1981. A tutorial on the care iii approach to reliability modeling. Technical Report NAGI-70. NASA Langley Research Center.
- Tuduce, I.C., Gross, T.R., 2005. Adaptive main memory compression. In: *Proceedings of the 2005 USENIX Annual Technical Conference*. April 10–15, 2005, Anaheim, CA, USA. USENIX, pp. 237–250. <http://www.usenix.org/events/usenix05/tech/general/tuduce.html>.
- Tuel Jr., William G., 1976. An analysis of buffer paging in virtual storage systems. *IBM J. Res. Dev.* 20 (5), 518–520. <https://doi.org/10.1147/rd.205.0518>.
- Turnbull, J., 2016. *The Art of Monitoring*. Turnball Press. ISBN 978-0988820241. <https://artofmonitoring.com/>.
- Tzortzatos, E., 2013. System z flash express: introduction, setup, management, uses, and benefits. Technical report. IBM.
- Uhlmann, J.K., 1991. Satisfying general proximity/similarity queries with metric trees. *Inf. Process. Lett.* 40 (4), 175–179. [https://doi.org/10.1016/0020-0190\(91\)90074-R](https://doi.org/10.1016/0020-0190(91)90074-R).
- Uysal, M., Alvarez, G.A., Merchant, A., 2001. A modular, analytical throughput model for modern disk arrays. In: 9th International Workshop on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS 2001). 15–18 August 2001, Cincinnati, OH, USA. IEEE Computer Society, pp. 183–192.
- Uysal, M., Merchant, A., Alvarez, G.A., 2003. Using mems-based storage in disk arrays. In: Chase, J. (Ed.), *Proceedings of the FAST '03 Conference on File and Storage Technologies*. March 31–April 2, 2003, Cathedral Hill Hotel, San Francisco, California. USENIX. <http://www.usenix.org/events/fast03/tech/uysal.html>.
- Vajha, M., Ramkumar, V., Puranik, B., Kini, G.R., Lobo, E., Sasidharan, B., Kumar, P.V., Barg, A., Ye, M., Narayanamurthy, S., Hussain, S., Nandi, S., 2018. Clay codes: moulding MDS codes to yield an MSR code. In: Agrawal, N., Rangaswami, R. (Eds.), 16th USENIX Conference on File and Storage Technologies, FAST 2018. Oakland, CA, USA, February 12–15, 2018. USENIX Association, pp. 139–154. <https://www.usenix.org/conference/fast18/presentation/vajha>.
- Vakali, A., Manolopoulos, Y., 1997. An exact analysis on expected seeks in shadowed disks. *Inf. Process. Lett.* 61 (6), 323–329. [https://doi.org/10.1016/S0020-0190\(97\)00030-6](https://doi.org/10.1016/S0020-0190(97)00030-6).
- Vakali, A., Manolopoulos, Y., 2000. Data placement schemes in replicated mirrored disk systems. *J. Syst. Softw.* 55 (2), 115–128. [https://doi.org/10.1016/S0164-1212\(00\)00065-0](https://doi.org/10.1016/S0164-1212(00)00065-0).

- Varki, E., Merchant, A., Xu, J., Qiu, X., 2004. Issues and challenges in the performance analysis of real disk arrays. *IEEE Trans. Parallel Distrib. Syst.* 15 (6), 559–574. <https://doi.org/10.1109/TPDS.2004.9>.
- Varma, A., Jacobson, Q., 1998. Destage algorithms for disk arrays with nonvolatile caches. *IEEE Trans. Comput.* 47 (2), 228–235. <https://doi.org/10.1109/12.663770>.
- Vasic, B., Kurtas, E.M., 2005. *Coding and Signal Processing for Magnetic Recording Systems*. Computer Engineering Series. CRC Press, Boca Raton, FL. ISBN 9780849315244.
- Venkatesan, V., Iliadis, I., Hu, X., Haas, R., Fragouli, C., 2010. Effect of replica placement on the reliability of large-scale data storage systems. In: *MASCOTS 2010, 18th Annual IEEE/ACM International Symposium on Modeling, Analysis and Simulation of Computer and Telecommunication Systems*. Miami, Florida, USA, August 17–19, 2010. IEEE Computer Society, pp. 79–88.
- Venkatesan, V., Iliadis, I., Fragouli, C., Urbanke, R.L., 2011. Reliability of clustered vs. declustered replica placement in data storage systems. In: *MASCOTS 2011, 19th Annual IEEE/ACM International Symposium on Modeling, Analysis and Simulation of Computer and Telecommunication Systems*. Singapore, 25–27 July 2011. IEEE Computer Society, pp. 307–317.
- Venkatesan, V., Iliadis, I., Haas, R., 2012. Reliability of data storage systems under network rebuild bandwidth constraints. In: *20th IEEE International Symposium on Modeling, Analysis and Simulation of Computer and Telecommunication Systems, MASCOTS 2012*. Washington, DC, USA, August 7–9, 2012. IEEE Computer Society, pp. 189–197.
- Verbitski, A., Gupta, A., Saha, D., Brahmadesam, M., Gupta, K., Mittal, R., Krishnamurthy, S., Maurice, S., Kharatishvili, T., Bao, X., 2017. Amazon aurora: design considerations for high throughput cloud-native relational databases. In: Salihoglu, S., Zhou, W., Chirkova, R., Yang, J., Suciu, D. (Eds.), *Proceedings of the 2017 ACM International Conference on Management of Data, SIGMOD Conference 2017*. Chicago, IL, USA, May 14–19, 2017. ACM, pp. 1041–1052.
- Verbitski, A., Gupta, A., Saha, D., Corey, J., Gupta, K., Brahmadesam, M., Mittal, R., Krishnamurthy, S., Maurice, S., Kharatishvili, T., Bao, X., 2018. Amazon aurora: on avoiding distributed consensus for i/os, commits, and membership changes. In: Das, G., Jermaine, C.M., Bernstein, P.A. (Eds.), *Proceedings of the 2018 International Conference on Management of Data, SIGMOD Conference 2018*. Houston, TX, USA, June 10–15, 2018. ACM, pp. 789–796.
- Verma, A., Koller, R., Useche, L., Rangaswami, R., 2010. Srcmap: energy proportional storage using dynamic consolidation. In: Burns, R.C., Keeton, K. (Eds.), *8th USENIX Conference on File and Storage Technologies*. San Jose, CA, USA, February 23–26, 2010. USENIX, pp. 267–280. http://www.usenix.org/events/fast10/tech/full_papers/verma.pdf.
- Vettiger, P., Despont, M., Drechsler, U., et al., 2000. The “millipede”—more than thousand tips for future Atomic Force Microscopy – AFM storage. *IBM J. Res. Dev.* 44 (3), 323–340. <https://doi.org/10.1147/rd.443.0323>.
- Vitter, J.S., 2001. External memory algorithms and data structures. *ACM Comput. Surv.* 33 (2), 209–271. <https://doi.org/10.1145/384192.384193>.
- Vo, H.T., Chen, C., Ooi, B.C., 2010. Towards elastic transactional cloud storage with range query support. *Proc. VLDB Endow.* 3 (1), 506–517. http://www.vldb.org/pvldb/vldb2010/pvldb_vol3/R45.pdf.
- Vogels, W., 2008. Eventually consistent. *ACM Queue* 6 (6), 14–19. <https://doi.org/10.1145/1466443.1466448>.
- Voigt, D., 1998. HP autoraid field performance in HP World’98. <http://www.openmpe.com/cslproceed/HPW98CD/3000/3354/3354.htm>.
- Vongsathorn, P., Carson, S.D., 1990. A system for adaptive disk rearrangement. *Softw. Pract. Exp.* 20 (3), 225–242. <https://doi.org/10.1002/spe.4380200302>.
- Vu, Q.H., Lupu, M., Ooi, B.C., 2010. *Peer-to-Peer Computing: Principles and Applications*. Springer-Verlag. ISBN 978-3-642-03514-2.
- Vukotic, A., Watt, N., Abedrabbo, T., Fox, D., Partner, J., 2014. *Neo4j in Action*. Manning Publications. ISBN 978-1617290763.
- Vuppapapati, M., Miron, J., Agarwal, R., Truong, D., Motivala, A., Cruanes, T., 2020. Building an elastic query engine on disaggregated storage. In: Bhagwan, R., Porter, G. (Eds.), *17th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2020*. Santa Clara, CA, USA, February 25–27, 2020. USENIX Association, pp. 449–462. <https://www.usenix.org/conference/nsdi20/presentation/vuppapapati>.
- Wah, B.W., Yao, S.B., 1980. DIALOG: a distributed processor organization for database machine. In: *American Federation of Information Processing Societies: 1980 National Computer Conference*. 19–22 May 1980, Anaheim, California, USA. In: *AFIPS Conference Proceedings*, vol. 49. AFIPS Press, pp. 243–253.
- Wallace, G., Douglass, F., Qian, H., Shilane, P., Smaldone, S., Chamness, M., Hsu, W., 2012. Characteristics of backup workloads in production systems. In: Bolosky, W.J., Flinn, J. (Eds.), *Proceedings of the 10th USENIX Conference on File and Storage Technologies, FAST 2012*. San Jose, CA, USA, February 14–17, 2012. USENIX Association, p. 4. <https://www.usenix.org/conference/fast12/characteristics-backup-workloads-production-systems>.
- Wallace, V.L., Rosenberg, R.S., 1966. Markovian models and numerical analysis of computer system behavior. In: *American Federation of Information Processing Societies: Proceedings of the AFIPS ’66 Spring Joint Computer Conference*. April 26–28, 1966, Boston, Massachusetts, USA. In: *AFIPS Conference Proceedings*, vol. 28. AFIPS/Spartan Books/ACM, pp. 141–148.
- Wang, F., Deng, Y., Helian, N., Wu, S., Khare, V., Liao, C., Parker, A., 2007. Evolutionary storage: speeding up a magnetic disk by clustering frequent data. *IEEE Trans. Magn.* 43, 2295–2297.
- Wang, F., Helian, N., Wu, S., Guo, Y., Deng, D., Khare, V., Liao, C., Rashidi, M., Parker, A., 2008a. A case for redundant arrays of hybrid disks (rahd). *IEEE Trans. Magn.* 44, 3738–3741.
- Wang, J., Zhu, H., Li, D., 2008b. eRAID: conserving energy in conventional disk-based RAID system. *IEEE Trans. Comput.* 57 (3), 359–374. <https://doi.org/10.1109/TC.2007.70821>.
- Wang, P., Ma, S., Lu, H., Wang, Z., 2014. A comprehensive comparison between virtual cut-through and wormhole routers for cache coherent network on-chips. *IEICE Electron. Express* 11 (14), 20140496. <https://doi.org/10.1587/elex.11.20140496>.
- Wang, Z., Dimakis, A.G., Bruck, J., 2010. Rebuilding for array codes in distributed storage systems. *CoRR*, arXiv:1009.3291.

- Wang, Z., Tamo, I., Bruck, J., 2011. On codes for optimal rebuilding access. In: 49th Annual Allerton Conference on Communication, Control, and Computing, Allerton 2011. Allerton Park & Retreat Center, Monticello, IL, USA, 28–30 September, 2011. IEEE, pp. 1374–1381.
- Watson, R.W., 2007. Timesharing System Design Concepts. McGraw-Hill. ISBN 9780070684652.
- Watson, T.J., Petre, P., 2000. *Father, Son & Company: My Life at IBM and Beyond*. Bantam.
- Watters, L.J., 1967. Letter to the editor – reduction of integer polynomial programming problems to zero-one linear programming problems. *Oper. Res.* 15 (6), 1171–1174. <https://doi.org/10.1287/opre.15.6.1171>.
- Weatherspoon, H., Kubiatowicz, J., 2002. Erasure coding vs. replication: a quantitative comparison. In: Druschel, P., Kaashoek, M.F., Rowstron, A.I.T. (Eds.), *Peer-to-Peer Systems, First International Workshop, IPTPS 2002, Revised Papers*. Cambridge, MA, USA, March 7–8, 2002. In: *Lecture Notes in Computer Science*, vol. 2429. Springer, pp. 328–338.
- Webb, A.M., Mansell, R., Knight, J.W., Greenspan, S.J., Emmes, D., 2007. Practical software reuse for IBM system z I/O subsystems. *IBM J. Res. Dev.* 51 (1/2), 229–243. <https://doi.org/10.1147/rd.511.0229>.
- Weber, Andreas, Herbst, Nikolas, Groenda, Henning, Kounev, Samuel, 2014. Towards a resource elasticity benchmark for cloud environments. In: *Proceedings of the 2nd International Workshop on Hot Topics in Cloudservice Scalability, HotTopiCS 2014*. Dublin, Ireland, March 22, 2014. ACM, pp. 5:1–5:8.
- Weber, R., Schek, H., Blott, S., 1998. A quantitative analysis and performance study for similarity-search methods in high-dimensional spaces. In: Gupta, A., Shmueli, O., Widom, J. (Eds.), *VLDB'98, Proceedings of 24th International Conference on Very Large Data Bases*. August 24–27, 1998, New York City, New York, USA. Morgan Kaufmann, pp. 194–205. <http://www.vldb.org/conf/1998/p194.pdf>.
- Weddle, C., Oldham, M., Qian, J., Wang, A.A., Reiher, P.L., Kuenning, G., 2007. PARAD: a gear-shifting power-aware RAID. In: Arpaci-Dusseau, A.C., Arpaci-Dusseau, R.H. (Eds.), *5th USENIX Conference on File and Storage Technologies, FAST 2007*. February 13–16, 2007, San Jose, CA, USA. USENIX, pp. 245–260. <http://www.usenix.org/events/fast07/tech/weddle.html>.
- Wehrle, K., Steinmetz, R. (Eds.), 2005. *Peer-to-Peer Systems and Applications*. Lecture Notes in Computer Science. Springer. ISBN 978-3540291923.
- Weikum, G., Zabback, P., 1992. Tuning of striping units in disk-array-based file systems. In: Yu, P.S. (Ed.), *RIDE-TQP '92, Second International Workshop on Research Issues on Data Engineering: Transaction and Query Processing*. Tempe, Arizona, USA, February 2–3, 1992. IEEE Computer Society, pp. 80–87.
- Weil, S.A., Brandt, S.A., Miller, E.L., Long, D.D.E., Maltzahn, C., 2006a. Ceph: a scalable, high-performance distributed file system. In: Bershad, B.N., Mogul, J.C. (Eds.), *7th Symposium on Operating Systems Design and Implementation (OSDI '06)*. November 6–8, Seattle, WA, USA. USENIX Association, pp. 307–320. <http://www.usenix.org/events/osdi06/tech/weil.html>.
- Weil, S.A., Brandt, S.A., Miller, E.L., Maltzahn, C., 2006b. Grid resource management – CRUSH: controlled, scalable, decentralized placement of replicated data. In: *Proceedings of the ACM/IEEE SC2006 Conference on High Performance Networking and Computing*. Tampa, FL, USA, November 11–17, 2006. ACM Press, p. 122.
- Weiser, M.D., Welch, B.B., Demers, A.J., Shenker, S., 1994. Scheduling for reduced CPU energy. In: *Proceedings of the First USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. Monterey, California, USA, November 14–17, 1994. USENIX Association, pp. 13–23. <http://dl.acm.org/citation.cfm?id=1267640>.
- Welch, B., Unangst, M., Abbasi, Z., Gibson, G.A., Mueller, B., Small, J., Zelenka, J., Zhou, B., 2008. Scalable performance of the panasas parallel file system. In: Baker, M., Riedel, E. (Eds.), *6th USENIX Conference on File and Storage Technologies, FAST 2008*. February 26–29, 2008, San Jose, CA, USA. USENIX, pp. 17–33. <http://www.usenix.org/events/fast08/tech/welch.html>.
- Welch, T.A., 1984. A technique for high-performance data compression. *IEEE Computer* 17 (6), 8–19. <https://doi.org/10.1109/MC.1984.1659158>.
- Wexelblat, R.L., 1981. *History of Programming Languages*. Academic Press, New York, NY. ISBN 0-12-745040-8.
- White, D.A., Jain, R.C., 1996. Similarity indexing: algorithms and performance. In: Sethi, I.K., Jain, R.C. (Eds.), *Storage and Retrieval for Still Image and Video Databases IV*. San Diego/La Jolla, CA, USA, January 28–February 2, 1996. In: *SPIE Proceedings*, vol. 2670. SPIE, pp. 62–73.
- White, J., Leuth, C., Bell, J., 2013. Raid-dp: network appliance implementation of raid double parity for data protection. Technical Report No. 3298. Network Appliance Inc., Sunnivala, CA.
- Whitt, W., 1986. Deciding which queue to join: some counterexamples. *Oper. Res.* 34 (1), 55–62. <https://doi.org/10.1287/opre.34.1.55>.
- Widmer, A.X., Franaszek, P.A., 1983. A dc-balanced, partitioned-block, 8b/10b transmission code. *IBM J. Res. Dev.* 27 (5), 440–451. <https://doi.org/10.1147/rd.275.0440>.
- Widom, J., Ceri, S. (Eds.), 1996. *Active Database Systems: Triggers and Rules for Advanced Database Processing*. Data Management. Morgan-Kaufmann. ISBN 978-1558603042.
- Wiederhold, G., 1987. *File Organization for Database Design*. McGraw-Hill Computer Science Series. McGraw-Hill College, New York, NY. ISBN 978-0070701335.
- Wijayarathne, R., Reddy, A.L.N., 1999. Integrated QOS management for disk I/O. In: *IEEE International Conference on Multimedia Computing and Systems, ICMCS 1999, Volume I*. Florence, Italy, June 7–11. IEEE Computer Society, pp. 487–492.
- Wijayarathne, R., Reddy, A.L.N., 2000. Providing QOS guarantees for disk I/O. *Multimed. Syst.* 8 (1), 57–68. <https://doi.org/10.1007/s005300050005>.
- Wilcke, W.W., Garner, R.B., Fleiner, C., Freitas, R.F., et al., 2006. IBM intelligent bricks project – petabytes and beyond. *IBM J. Res. Dev.* 50 (2–3), 181–198. <https://doi.org/10.1147/rd.502.0181>.
- Wildani, A., Schwarz, T.J.E., Miller, E.L., Long, D.D.E., 2009. Protecting against rare event failures in archival systems. In: *17th Annual Meeting of the IEEE/ACM International Symposium on Modelling, Analysis and Simulation of Computer and Telecommunication Systems, MASCOTS 2009*. September 21–23, 2009, South Kensington Campus, Imperial College, London, UK. IEEE Computer Society, pp. 1–11.
- Wilhelm, N.C., 1976. An anomaly in disk scheduling: a comparison of FCFS and SSTF seek scheduling using an empirical model for disk accesses. *Commun. ACM* 19 (1), 13–17. <http://doi.acm.org/10.1145/359970.359977>.

- Wilhelm, N.C., 1977. A general model for the performance of disk systems. *J. ACM* 24 (1), 14–31. <https://doi.org/10.1145/321992.321994>.
- Wilkes, J., 1992a. Predictive power conservation. Technical Report HPL-CSP-92-5. Hewlett-Packard Laboratories, Palo Alto, CA.
- Wilkes, J., 1992b. Datamesh, house-building, and distributed systems technology. In: Banâtre, J. (Ed.), *Proceedings of the 5th ACM SIGOPS European Workshop: Models and Paradigms for Distributed Systems Structuring*. Mont Saint-Michel, France, September 21–23, 1992. ACM.
- Wilkes, J., 1996. The pantheon storage-system simulator. Technical Report HPL-SSP-95-14. Hewlett-Packard Laboratories, Palo Alto, CA.
- Wilkes, J., Golding, R.A., Staelin, C., Sullivan, T., 1996. The HP autoraid hierarchical storage system. *ACM Trans. Comput. Syst.* 14 (1), 108–136. <http://doi.acm.org/10.1145/225535.225539>.
- Wilkes, J., Hoover, C., Keer, B., Mehra, P., Veitch, A., 2008. *Storage, Data, and Information Systems*, 5. HP Press. http://images01.insight.com/media/pdf/Storage_Data_IS_81108.pdf.
- Wilkes, M.V., 1975. *Time-Sharing Computer Systems*, 3. Computer monographs. Elsevier Science Ltd. ISBN 978-0444195258.
- Williams, A.C., Bhandiwad, R.A., 1976. A generating function approach to queueing network analysis of multiprogrammed computers. *Networks* 6 (1), 1–22. <https://doi.org/10.1002/net.3230060102>.
- Williams, S., Waterman, A., Patterson, D.A., 2009. Roofline: an insightful visual performance model for multicore architectures. *Commun. ACM* 52 (4), 65–76. <https://doi.org/10.1145/1498765.1498785>.
- Wilner, A., 2001. Multiple drive failure tolerant raid system. <https://patents.google.com/patent/US6327672>. US patent 6,327,672.
- Wilner, W.T., 1972. Design of the Burroughs B1700. In: *American Federation of Information Processing Societies: Proceedings of the AFIPS '72 Fall Joint Computer Conference – Part I*. December 5–7, 1972, Anaheim, California, USA. In: *AFIPS Conference Proceedings*, vol. 41. AFIPS/ACM/Thomson Book Company, Washington D.C., pp. 489–497.
- Wilson, P.R., Johnstone, M.S., Neely, M., Boles, D., 1995. Dynamic storage allocation: a survey and critical review. In: Baker, H.G. (Ed.), *Memory Management, International Workshop IWMM 95, Proceedings*. Kinross, UK, September 27–29, 1995. In: *Lecture Notes in Computer Science*, vol. 986. Springer, pp. 1–116.
- Wilson, P.R., Kaplan, S.F., Smaragdakis, Y., 1999. The case for compressed caching in virtual memory systems. In: *Proceedings of the 1999 USENIX Annual Technical Conference*. Monterey, California, USA, June 6–11, 1999. USENIX, pp. 101–116. http://www.usenix.org/events/usenix99/full_papers/wilson/wilson.pdf.
- Wolf, J.L., 1989. The placement optimization program: a practical solution to the disk file assignment problem. In: Ferrari, D. (Ed.), *Proceedings of the 1989 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems*. Berkeley, California, USA, May 23–26, 1989. ACM, pp. 1–10.
- Wolf, J.L., Dias, D.M., Iyer, B.R., Yu, P.S., 1988. A hybrid data sharing – data partitioning architecture for transaction processing. In: *Proceedings of the Fourth International Conference on Data Engineering*. February 1–5, 1988, Los Angeles, California, USA. IEEE Computer Society, pp. 520–527.
- Wolf, M., 1995. *High Performance Compilers for Parallel Computing*. Addison-Wesley, Boston, MA.
- Wolff, R.W., 1982. Poisson arrivals see time averages. *Oper. Res.* 30 (2), 223–231. <https://doi.org/10.1287/opre.30.2.223>.
- Wong, C.K., 1980. Minimizing expected head movement in one-dimensional and two-dimensional mass storage systems. *ACM Comput. Surv.* 12 (2), 167–178. <http://doi.acm.org/10.1145/356810.356814>.
- Wong, C.K., 1983. *Algorithmic Studies in Mass Storage Systems*. Computer Science Press, Rockville, MD. ISBN 978-3-642-69354-0.
- Wong, T.M., Golding, R.A., Glider, J.S., Borowsky, E., Becker-Szendy, R.A., Fleiner, C., Hosekote, D.R.K., Zaki, O.A., 2005. Kybos: Self-Management for Distributed Brick-Based Storage. Technical Report RJ10356. IBM Research Center, Almaden, CA. <https://dominoweb.draco.res.ibm.com/reports/rj10356.pdf>.
- Wong, T.M., Golding, R.A., Lin, C., Becker-Szendy, R.A., 2006. Zygaria: storage performance as a managed resource. In: *12th IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS 2006)*. 4–7 April 2006, San Jose, California, USA. IEEE Computer Society, pp. 125–134.
- Worthington, B.L., Ganger, G.R., Patt, Y.N., 1994. Scheduling algorithms for modern disk drives. In: Dowdy, L.W., Bunt, R., Gaither, B.D. (Eds.), *Proceedings of the 1994 ACM SIGMETRICS Conference on Measurement and Modeling of Computer Systems*. Vanderbilt University, Nashville, Tennessee, USA, May 16–20, 1994. ACM, pp. 241–252.
- Wright, T.P., 1936. Factors affecting the costs of airplanes. *J. Aeronaut. Sci.* 10, 302–328.
- Wu, K., Yu, P.S., Gedik, B., et al., 2007. Challenges and experience in prototyping a multi-modal stream analytic and monitoring application on system S. In: Koch, C., Gehrke, J., Garofalakis, M.N., et al. (Eds.), *Proceedings of the 33rd International Conference on Very Large Data Bases*. University of Vienna, Austria, September 23–27, 2007. ACM, pp. 1185–1196. <http://www.vldb.org/conf/2007/papers/industrial/p1185-wu.pdf>.
- Wu, K., Arpaci-Dusseau, A.C., Arpaci-Dusseau, R.H., 2019. Towards an unwritten contract of Intel optane SSD. In: Peek, D., Yadgar, G. (Eds.), *11th USENIX Workshop on Hot Topics in Storage and File Systems, HotStorage 2019*. Renton, WA, USA, July 8–9, 2019. USENIX Association. <https://www.usenix.org/conference/hotstorage19/presentation/wu-kan>.
- Wu, L., Lottarini, A., Paine, T.K., Kim, M.A., Ross, K.A., 2014. Q100: the architecture and design of a database processing unit. In: Balasubramanian, R., Davis, A., Adve, S.V. (Eds.), *Architectural Support for Programming Languages and Operating Systems, ASPLOS '14*. Salt Lake City, UT, USA, March 1–5, 2014. ACM, pp. 255–268.
- Wu, L., Lottarini, A., Paine, T.K., Kim, M.A., Ross, K.A., 2015. The Q100 database processing unit. *IEEE MICRO* 35 (3), 34–46. <https://doi.org/10.1109/MM.2015.51>.
- Wu, S., Jiang, H., Feng, D., Tian, L., Mao, B., 2009. Workout: I/O workload outsourcing for boosting RAID reconstruction performance. In: Seltzer, M.I., Wheeler, R. (Eds.), *7th USENIX Conference on File and Storage Technologies, Proceedings*. February 24–27, 2009, San Francisco, CA, USA. USENIX, pp. 239–252. http://www.usenix.org/events/fast09/tech/full_papers/wu/wu.pdf.
- Wu, Y., Hou, H., Han, Y.S., Lee, P.P.C., Han, G., 2020. Generalized expanded-Blaum-Roth codes and their efficient encoding/decoding. In: *IEEE Global Communications Conference, GLOBECOM 2020, Virtual Event*. Taiwan, December 7–11, 2020. IEEE, pp. 1–6.

- Xia, M., Saxena, M., Blaum, M., Pease, D., 2015. A tale of two erasure codes in HDFS. In: Schindler, J., Zadok, E. (Eds.), *Proceedings of the 13th USENIX Conference on File and Storage Technologies, FAST 2015*. Santa Clara, CA, USA, February 16–19, 2015. USENIX Association, pp. 213–226. <https://www.usenix.org/conference/fast15/technical-sessions/presentation/xia>.
- Xia, W., Jiang, H., Feng, D., Tian, L., Fu, M., Zhou, Y., 2014. Ddelta: a deduplication-inspired fast delta compression approach. *Perform. Eval.* 79, 258–272.
- Xia, W., Jiang, H., Feng, D., Douglass, F., Shilane, P., Hua, Y., Fu, M., Zhang, Y., Zhou, Y., 2016. A comprehensive study of the past, present, and future of data deduplication. *Proc. IEEE* 104 (9), 1681–1710. <https://doi.org/10.1109/JPROC.2016.2571298>.
- Xiang, L., Xu, Y., Lui, J.C.S., Chang, Q., 2010. Optimal recovery of single disk failure in RDP code storage systems. In: Misra, V., Barford, P., Squillante, M.S. (Eds.), *SIGMETRICS 2010, Proceedings of the 2010 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems*. New York, New York, USA, 14–18 June 2010. ACM, pp. 119–130.
- Xiang, L., Xu, Y., Lui, J.C.S., Chang, Q., Pan, Y., Li, R., 2011. A hybrid approach to failed disk recovery using RAID-6 codes: algorithms and performance evaluation. *ACM Trans. Storage* 7 (3), 11:1–11:34. <http://doi.acm.org/10.1145/2027066.2027071>.
- Xie, T., 2008. SEA: a striping-based energy-aware strategy for data placement in raid-structured storage systems. *IEEE Trans. Comput.* 57 (6), 748–761. <https://doi.org/10.1109/TC.2008.27>.
- Xie, T., Sun, Y., 2007. No more energy-performance trade-off: a new data placement strategy for raid-structured storage systems. In: Aluru, S., Parashar, M., Badrinath, R., Prasanna, V.K. (Eds.), *High Performance Computing – HiPC 2007, 14th International Conference, Proceedings*. Goa, India, December 18–21, 2007. In: *Lecture Notes in Computer Science*, vol. 4873. Springer, pp. 35–46.
- Xie, T., Sun, Y., 2008. PEARL: performance, energy, and reliability balanced dynamic data redistribution for next generation disk arrays. In: Miller, E.L., Williamson, C.L. (Eds.), *16th International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS 2008)*. Baltimore, Maryland, USA, September 8–10, 2008. IEEE Computer Society, pp. 199–206.
- Xie, Y. (Ed.), 2014. *Emerging Memory Technologies: Design, Architecture, and Applications*. Springer. ISBN.
- Xin, Q., Miller, E.L., Schwarz, T.J.E., Long, D.D.E., Brandt, S.A., Litwin, W., 2003. Reliability mechanisms for very large storage systems. In: *20th IEEE/11th NASA Goddard Conference on Mass Storage Systems and Technologies. MSS 2003*, San Diego, California, USA, April 7–10, 2003. IEEE Computer Society, pp. 146–156.
- Xu, L., Bruck, J., 1999. X-code: MDS array codes with optimal encoding. *IEEE Trans. Inf. Theory* 45 (1), 272–276. <https://doi.org/10.1109/18.746809>.
- Xu, L., Bohossian, V., Bruck, J., Wagner, D.G., 1999. Low-density MDS codes and factors of complete graphs. *IEEE Trans. Inf. Theory* 45 (6), 1817–1836. <https://doi.org/10.1109/18.782102>.
- Xu, Lianghong, Cipar, James, Krevat, Elie, Tumanov, Alexey, Gupta, Nitin, Kozuch, Michael A., Ganger, Gregory R., 2014a. SpringFS: bridging agility and performance in elastic distributed storage. In: Schroeder, Bianca, Thereska, Eno (Eds.), *Proceedings of the 12th USENIX conference on File and Storage Technologies, FAST 2014*. Santa Clara, CA, USA, February 17–20, 2014. USENIX, pp. 243–255. <https://www.usenix.org/conference/fast14/technical-sessions/presentation/xu>.
- Xu, Lianghong, Cipar, James, Krevat, Elie, Tumanov, Alexey, Gupta, Nitin, Kozuch, Michael A., Ganger, Gregory R., 2014b. Agility and performance in elastic distributed storage. *ACM Trans. Storage* 10 (4), 16:1–16:27. <https://doi.org/10.1145/2668129>.
- Xu, S., Li, R., Lee, P.P.C., Zhu, Y., Xiang, L., Xu, Y., Lui, J.C.S., 2014c. Single disk failure recovery for x-code-based parallel storage systems. *IEEE Trans. Comput.* 63 (4), 995–1007. <https://doi.org/10.1109/TC.2013.8>.
- Yaakobi, E., Grupp, L.M., Siegel, P.H., Swanson, S., Wolf, J.K., 2012. Characterization and error-correcting codes for TLC flash memories. In: *International Conference on Computing, Networking and Communications, ICNC 2012*. Maui, HI, USA, January 30–February 2, 2012. IEEE Computer Society, pp. 486–491.
- Yakovlev, A., Moghadam, H.F., Moharrer, A., et al., 2020. Oracle AutoML: a fast and predictive automl pipeline. *Proc. VLDB Endow.* 13 (12), 3166–3180. <http://www.vldb.org/pvldb/vol13/p3166-yakovlev.pdf>.
- Yan, F., Mountrouidou, X., Riska, A., Smirni, E., 2016. Prefigure: an analytic framework for HDD management. *TOMPECS* 1 (3), 10:1–10:27. <http://doi.acm.org/10.1145/2872331>.
- Yang, J., Sun, F.-B., 1999. A comprehensive review of hard disk drive technology. In: *Proc. Annual Reliability and Maintainability Symposium*.
- Yau, S.S., Fung, H.S., 1977. Associative processor architecture – a survey. *ACM Comput. Surv.* 9 (1), 3–27. <https://doi.org/10.1145/356683.356685>.
- Yazdanbakhsh, A., Angermüller, C., Akin, B., Zhou, Y., Jones, A., Hashemi, M., Swersky, K., Chatterjee, S., Narayanaswami, R., Laudon, J., 2021. Apollo: transferable architecture exploration. *CoRR*, arXiv:2102.01723.
- Yianilos, P.N., 1993. Data structures and algorithms for nearest neighbor search in general metric spaces. In: Ramachandran, V. (Ed.), *Proceedings of the Fourth Annual ACM/SIGACT-SIAM Symposium on Discrete Algorithms*. 25–27 January 1993, Austin, Texas, USA. ACM/SIAM, pp. 311–321. <http://dl.acm.org/citation.cfm?id=313559.313789>.
- Yin, S., Ruan, X., Manzanarez, A., Qin, X., 2009. How reliable are parallel disk systems when energy-saving schemes are involved? In: *Proceedings of the 2009 IEEE International Conference on Cluster Computing*. August 31–September 4, 2009, New Orleans, Louisiana, USA. IEEE Computer Society, pp. 1–9.
- You, L., Pollack, K.T., Long, D.D.E., 2005. Deep store: an archival storage system architecture. In: Aberer, K., Franklin, M.J., Nishio, S. (Eds.), *Proceedings of the 21st International Conference on Data Engineering, ICDE 2005*. 5–8 April 2005, Tokyo, Japan. IEEE Computer Society, 2005, pp. 804–815.
- Yu, C., 2002. *High-Dimensional Indexing: Transformational Approaches to High-Dimensional Range and Similarity Searches*. *Lecture Notes in Computer Science*, vol. 2341. Springer. ISBN 978-3540441991.
- Yu, C., Ooi, B.C., Tan, K., Jagadish, H.V., 2001. Indexing the distance: an efficient method to KNN processing. In: Apers, P.M.G., Atzeni, P., Ceri, S., Paraboschi, S., Ramamohanarao, K., Snodgrass, R.T. (Eds.), *VLDB 2001, Proceedings of 27th International Conference on Very Large Data Bases*. September 11–14, 2001, Roma, Italy. Morgan Kaufmann, pp. 421–430. <http://www.vldb.org/conf/2001/P421.pdf>.

- Yu, C., Zhang, C., Mao, Y., Li, F., 2015. Leap-based content defined chunking—theory and implementation. In: Proc. 31st Symp. Mass Storage System Technologies, pp. 1–12.
- Yu, P.S., Cornell, D.W., Dias, D.M., Thomasian, A., 1989. Performance comparison of IO shipping and database call shipping: schemes in multisystem partitioned databases. *Perform. Eval.* 10 (1), 15–33. [https://doi.org/10.1016/0166-5316\(89\)90003-5](https://doi.org/10.1016/0166-5316(89)90003-5).
- Yu, P.S., Chen, M., Kandlur, D.D., 1993. Grouped sweeping scheduling for dasd-based multimedia storage management. *Multimed. Syst.* 1 (3), 99–109. <https://doi.org/10.1007/BF01213198>.
- Yu, X., Gum, B., Chen, Y., Wang, R.Y., Li, K., Krishnamurthy, A., Anderson, T.E., 2000. Trading capacity for performance in a disk array. In: Jones, M.B., Kaashoek, M.F. (Eds.), 4th Symposium on Operating System Design and Implementation (OSDI 2000). San Diego, California, USA, October 23–25, 2000. USENIX Association, pp. 243–258. <http://dl.acm.org/citation.cfm?id=1251246>.
- Yu, Y., Shin, D., Eom, H., Yeom, H.Y., 2010. NCQ vs. I/O scheduler: preventing unexpected misbehaviors. *ACM Trans. Storage* 6 (1), 2:1–2:37. <http://doi.acm.org/10.1145/1714454.1714456>.
- Yuan, Y., Lee, R., Zhang, X., 2013. The yin and yang of processing data warehousing queries on GPU devices. *Proc. VLDB Endow.* 6 (10), 817–828. <http://www.vldb.org/pvldb/vol6/p817-yuan.pdf>.
- Zabback, P., Menon, J., Riegel, J., 1996. The RAID configuration tool. In: 3rd International Conference on High Performance Computing, HIPC 1996, Proceedings. Trivandrum, India, 19–22 December, 1996. IEEE Computer Society, pp. 55–61.
- Zahorjan, J., Hume, J., Sevcik, K.C., 1978. A queueing model of a rotational position sensing disk system. In: INFOR: Information Systems.
- Zahorjan, J., Sevcik, K.C., Eager, D.L., Galler, B., 1982. Balanced job bound analysis of queueing networks. *Commun. ACM* 25 (2), 134–141. <https://doi.org/10.1145/358396.358447>.
- Zaitsev, G.V., Zonoviev, V.A., Semakov, N.V., 1983. Minimum-check-density codes for correcting bytes of errors, erasures, or defects. *Probl. Inf. Transm.* 197 (1), 197–204. <https://doi.org/10.1023/A%3A1010424208992>.
- Zaki, M.J., 1999. Parallel and distributed association mining: a survey. *IEEE Concurr.* 7 (4), 14–25. <https://doi.org/10.1109/4434.806975>.
- Zaki, M.J., Jr., W.M., 2014. Data Mining and Analysis: Fundamental Concepts and Algorithms. Cambridge University Press. ISBN 978-0521766333.
- Zandi, A., Iyer, B.R., Jr., G.G.L., 1993. Sort order preserving data compression for extended alphabets. In: Storer, J.A., Cohn, M. (Eds.), Proceedings of the IEEE Data Compression Conference. DCC 1993, Snowbird, Utah, USA, March 30–April 1, 1993. IEEE Computer Society, pp. 330–339.
- Zarghamee, M., 2011. Mojtahedi and the founding of the Aryamehr University of Technology. *Iranian Stud.* 44 (5), 767–775. <https://doi.org/10.1080/00210862.2011.570485>.
- Zedlewski, J., Sobti, S., Garg, N., Zheng, F., Krishnamurthy, A., Wang, R.Y., 2003. Modeling hard-disk power consumption. In: Chase, J. (Ed.), Proceedings of the FAST '03 Conference on File and Storage Technologies. March 31–April 2, 2003, Cathedral Hill Hotel, San Francisco, California, USA. USENIX. <http://www.usenix.org/events/fast03/tech/zedlewski.html>.
- Zeuch, S., Breß, S., Rabl, T., et al., 2019. Analyzing efficient stream processing on modern hardware. *Proc. VLDB Endow.* 12 (5), 516–530. <http://www.vldb.org/pvldb/vol12/p516-zeuch.pdf>.
- Zhai, C., Massung, S., 2016. Text Data Management and Analysis. ACM Books. ISBN 978-1-97000-117-4.
- Zhang, C., Lu, J., Xu, P., Chen, Y., 2018a. Unibench: a benchmark for multi-model database management systems. In: Nambiar, R., Poess, M. (Eds.), Performance Evaluation and Benchmarking for the Era of Artificial Intelligence – 10th TPC Technology Conference, TPCTC 2018, Revised Selected Papers. Rio de Janeiro, Brazil, August 27–31, 2018. In: Lecture Notes in Computer Science, vol. 11135. Springer, pp. 7–23.
- Zhang, G., Huang, Z., Ma, X., Yang, S., Wang, Z., Zheng, W., 2018b. RAID+: deterministic and balanced data distribution for large disk enclosures. In: Agrawal, N., Rangaswami, R. (Eds.), 16th USENIX Conference on File and Storage Technologies, FAST 2018. Oakland, CA, USA, February 12–15, 2018. USENIX Association, pp. 279–294. <https://www.usenix.org/conference/fast18/presentation/zhang>.
- Zhang, G., Wang, Z., Ma, X., Yang, S., Huang, Z., Zheng, W., 2020. Determining data distribution for large disk enclosures with 3-d data templates. *ACM Trans. Storage* 15 (4), 27:1–27:38. <https://doi.org/10.1145/3342858>.
- Zhang, L., 2015. High-Dimensional Indexing Methods Utilizing Clustering and Dimensionality Reduction. PhD thesis. New Jersey Institute of Technology, Newark, NJ.
- Zhang, M., Han, S., Lee, P.P.C., 2017. A simulation analysis of reliability in erasure-coded data centers. In: 36th IEEE Symposium on Reliable Distributed Systems, SRDS 2017. Hong Kong, Hong Kong, September 26–29, 2017. IEEE Computer Society, pp. 144–153.
- Zhang, S., He, J., He, B., Lu, M., 2013a. Omnibd: towards portable and efficient query processing on parallel CPU/GPU architectures. *Proc. VLDB Endow.* 6 (12), 1374–1377. <http://www.vldb.org/pvldb/vol6/p1374-he.pdf>.
- Zhang, T., Ramakrishnan, R., Livny, M., 1996. BIRCH: an efficient data clustering method for very large databases. In: Jagadish, H.V., Mumick, I.S. (Eds.), Proceedings of the 1996 ACM SIGMOD International Conference on Management of Data. Montreal, Quebec, Canada, June 4–6, 1996. ACM Press, pp. 103–114.
- Zhang, X., Tune, E., Hagmann, R., Jnagal, R., Gokhale, V., Wilkes, J., 2013b. Cpi²: CPU performance isolation for shared compute clusters. In: Hanzálek, Z., Härtig, H., Castro, M., Kaashoek, M.F. (Eds.), Eighth Eurosys Conference 2013, EuroSys '13. Prague, Czech Republic, April 14–17, 2013. ACM, pp. 379–391.
- Zhang, Y., Rajimwale, A., Arpaci-Dusseau, A.C., Arpaci-Dusseau, R.H., 2010. End-to-end data integrity for file systems: a ZFS case study. In: Burns, R.C., Keeton, K. (Eds.), 8th USENIX Conference on File and Storage Technologies. San Jose, CA, USA, February 23–26, 2010. USENIX, pp. 29–42. http://www.usenix.org/events/fast10/tech/full_papers/zhang.pdf.
- Zhang, Y., Myers, D.S., Arpaci-Dusseau, A.C., Arpaci-Dusseau, R.H., 2013c. Zettabyte reliability with flexible end-to-end data integrity. In: IEEE 29th Symposium on Mass Storage Systems and Technologies, MSST 2013. May 6–10, 2013, Long Beach, CA, USA. IEEE Computer Society, pp. 1–14.
- Zhang, Y., Jiang, H., Feng, D., Xia, W., Fu, M., Huang, F., Zhou, Y., 2015. Ae: an asymmetric extremum content defined chunking algorithm for fast and bandwidth-efficient data deduplication. In: Proc. IEEE INFOCOM, pp. 1–9.

- Zhao, J., Li, S., Chang, J., Byrne, J.L., Ramirez, L.L., Lim, K.T., Xie, Y., Faraboschi, P., 2015. Buri: scaling big-memory computing with hardware-based memory expansion. *ACM Trans. Archit. Code Optim.* 12 (3), 31:1–31:24. <https://doi.org/10.1145/2808233>.
- Zhao, Y., Deshpande, P., Naughton, J.F., 1997. An array-based algorithm for simultaneous multidimensional aggregates. In: Peckham, J. (Ed.), *SIGMOD 1997, Proceedings ACM SIGMOD International Conference on Management of Data*. Tucson, Arizona, USA, May 13–15, 1997. ACM Press, pp. 159–170.
- Zhou, B., Wen, J., 2013. Hysteresis re-chunking based metadata harnessing deduplication of disk images. In: *42nd International Conference on Parallel Processing, ICPP 2013*. Lyon, France, October 1–4, 2013. IEEE Computer Society, pp. 389–398.
- Zhu, B., Li, K., Patterson, R.H., 2008. Avoiding the disk bottleneck in the data domain deduplication file system. In: Baker, M., Riedel, E. (Eds.), *6th USENIX Conference on File and Storage Technologies, FAST 2008*. February 26–29, 2008, San Jose, CA, USA. USENIX, pp. 269–282. <http://www.usenix.org/events/fast08/tech/zhu.html>.
- Zhu, Q., David, F.M., Devaraj, C.F., Li, Z., Zhou, Y., Cao, P., 2004a. Reducing energy consumption of disk storage using power-aware cache management. In: *10th International Conference on High-Performance Computer Architecture (HPCA-10 2004)*. 14–18 February 2004, Madrid, Spain. IEEE Computer Society, pp. 118–129.
- Zhu, Q., Shankar, A., Zhou, Y., 2004b. PB-LRU: a self-tuning power aware storage cache replacement algorithm for conserving disk energy. In: Feautrier, P., Goodman, J.R., Sez nec, A. (Eds.), *Proceedings of the 18th Annual International Conference on Supercomputing, ICS 2004*. Saint Malo, France, June 26–July 01, 2004. ACM, pp. 79–88.
- Zhu, Q., Chen, Z., Tan, L., Zhou, Y., Keeton, K., Wilkes, J., 2005. Hibernator: helping disk arrays sleep through the winter. In: Herbert, A., Birman, K.P. (Eds.), *Proceedings of the 20th ACM Symposium on Operating Systems Principles 2005, SOSP 2005*. Brighton, UK, October 23–26, 2005. ACM, pp. 177–190.
- Zilio, D.C., Rao, J., Lightstone, S., Lohman, G.M., Storm, A.J., Garcia-Arellano, C., Fadden, S., 2004. DB2 design advisor: integrated automatic physical database design. In: Nascimento, M.A., Özsu, M.T., Kossmann, D., Miller, R.J., Blakeley, J.A., Schiefer, K.B. (Eds.), *(e)Proceedings of the Thirtieth International Conference on Very Large Data Bases, VLDB 2004*. Toronto, Canada, August 31–September 3, 2004. Morgan Kaufmann, pp. 1087–1097. <http://www.vldb.org/conf/2004/IND4P1.PDF>.
- Zipf, G.K., 1949. *Human Behavior and the Principle of Least Effort*. MIT Press, Cambridge, MA. ISBN 161427312X.
- Ziv, J., Lempel, A., 1977. A universal algorithm for sequential data compression. *IEEE Trans. Inf. Theory* 23 (3), 337–343. <https://doi.org/10.1109/TIT.1977.1055714>.
- Zivkov, B.T., Smith, A.J., 1997. Disk caching in large database and timeshared systems. In: *MASCOTS 1997, Proceedings of the Fifth International Symposium on Modeling, Analysis and Simulation of Computer and Telecommunication Systems*. Haifa, Israel, January 12–15, 1997. IEEE Computer Society, p. 184.
- Zobel, J., Moffat, A., 2006. Inverted files for text search engines. *ACM Comput. Surv.* 38 (2), 6. <https://doi.org/10.1145/1132956.1132959>.
- Zukowski, M., Boncz, P.A., Nes, N., Héman, S., 2005. Monetdb/x100 – a DBMS in the CPU cache. *IEEE Data Eng. Bull.* 28 (2), 17–22. <http://sites.computer.org/debull/A05june/mzukowski.ps>.

Index

0-9

2-dimensional bin-packing, 568
 2-phase RAID1 scheduling, 235
 2PC - two-phase commit, 616
 2PL - two-phase locking, 42, 500
 2PL analysis, 42
 2's complement arithmetic, 10
 5V value, 520
 5V variety, 520
 5V velocity, 520
 5V veracity, 520
 5V volume, 520
 64-bit addressing, 26
 80-20 rule, 553
 90-10% rule, 406

A

AAC - Advanced Audio Compression, 153
 Abate, Joseph, 213
 ABB - ASEA Brown Boveri, 16
 ABL - Advanced Business Language, 509
 ABR - Affinity Based Routing, 232
 ACATS - Almost Complete Address
 Translation, 308
 access control - attribute-based, 505
 access control - role-based, 505
 access control - rule-based, 505
 access density, 199
 access invariance, 164, 501
 Accumulo, 540
 Acer PCs, 73
 Acharya, Anurag, 413
 ACID compliance, 445
 ACID semantics, 435
 ACM - Association for Computing Machinery,
 629
 Acronis - backup, disaster recovery, 557
 ACS - Automatic Class Selection, 496
 Actifio, 177
 active disks, 412
 Active Memory Expansion, 155
 ADA programming language, 98
 Adabas - multivalued DB, 541
 Adamian, Hovannes, 143
 adaptive disk array, 229
 adaptive disk spindown, 350
 Adleman, Leonard, 192
 Adobe Flash, 152
 ADSL - Asymmetric Digital Subscriber Line,
 140

ADSTAR - ADvanced STorage And Retrieval,
 275
 Advanced Computer Systems - ACS, 20
 Adve, Sarita, 363
 Aerospike real-time data platform, 444
 AES - Advanced Encryption Standard, 156,
 169, 192, 193, 317
 AES256 symmetric key encryption, 193
 AFD - Advanced Format Drive, 165
 AFR Annual Failure Rate, 370
 Aggarwal, Alok, 76
 Agile Methodology, 175
 Agrawal, Rakesh, 501, 509
 AHCI - Advanced Host Controller Interface,
 391
 Ahrens, Matthew, 274
 Akamai, 97
 Akers, John, 275
 Akyurek, Sedat, 208
 Aldridge, Irene, 627
 ALEXnet CNN, 471
 Algol, 4
 Algolia, 536
 Alibaba Realtime compute, 551
 Alibaba tablestore, 541
 Alibaba's X-engine, 454
 ALIS - Automatic Locality Improving Storage,
 208, 372, 569
 Allegrograph, 538
 Alpha microprocessor, 56
 Alphabet Inc., Google's parent, 97
 Altavista, 95
 Alternating/Direct Current - AC/DC, 11
 ALU - Arithmetic Logical Unit, 24
 Alvarez, Guillermo, 213, 231, 304, 309
 Alvisi, Lorenzo, 78
 AMAT - Average Memory Access Time, 45
 Amazon, 536
 Amazon Aurora, 451
 Amazon CloudSearch, 536
 Amazon Company, 536
 Amazon DynamoDB - ADDDB, 247, 432, 524
 Amazon Fire Tablets, 145
 Amazon Glacier, 555
 Amazon Neptune, 538
 Amazon Research, 634
 Amazon S3, 555
 Amazon.com, 94, 145
 AMD APU - Accelerated Processing Unit, 70

AMD CUDA - Compute Unified Device
 Architecture, 472
 AMD ROCm platform, 472
 Amdahl, Gene, 20, 21, 79
 Amdahl Corp., 21
 Amdahl multicore law, 80
 Amdahl rules, 79
 Amdahl's balanced systems law, 79
 Amdahl's parallel processing Law, 79
 Amer, Ahmed, 229, 231
 Amiri, Khalil, 616
 analog data, 516
 analog data reduction, 516
 Anderson, Dave, 364
 Anderson, David, 441, 442
 Anderson, Thomas, 236
 Andreessen, Mark, 94
 Android OS, 146
 Angolia - Search as a Service, 535
 Angus reliability model, 260
 Anticipatory Arm Placement, 209
 AOL - America OnLine, 92
 AONT - All or Nothing Transform, 168
 Apache Accumulo, 540
 Apache Avro, 518
 Apache Beam, 551
 Apache Cassandra, 539
 Apache CouchDB, 526
 Apache Flink, 548, 550, 551
 Apache Hbase, 539
 Apache Hive, 506
 Apache Jackrabbit, 552
 Apache Jena, 537
 Apache Kafka, 519, 543, 548, 550
 Apache Mesos, 530
 Apache NiFi, 548, 550
 Apache Pig, 522
 Apache Salza, 548
 Apache Solr, 535
 Apache Spark Streaming, 548
 Apache Sphinx, 535
 Apache Storm, 519, 543, 550
 Apache Thrift, 518
 APL - A Programming Language, 5
 Apollo Computer, 7
 append and pack, 116
 Apple PCs, 73
 Apple Research, 634
 Application Managed Encryption - AME, 156,
 193

- ArangoDB, 527
 ARBRE - Almaden Research Backend Relational Engine, 395, 428
 ARC - Adaptive Replacement Cache, 39
 architecture efficiency metrics, 35
 Ardent computer, 34
 ARIES - Algorithms for Recovery and Isolation Exploiting Semantics, 501
 ARIES reliability analysis package, 265
 Arista Networks, 461
 arithmetic coding, 149
 ARM - Association Rule Mining, 509
 ARM confidence, 509
 ARM frequent sets, 509
 ARM support, 509
 Armenian illuminated manuscripts, 91
 Arpaci-Dusseau, Andrea, 131, 199
 Arpaci-Dusseau, Remzi, 131, 199, 274
 ARPANET, 92
 ARR - Annual Replacement Rate, 135
 ArrangoDB, 552
 arrival rate, denoted by λ , 6
 arrival theorem, Mean Value Analysis - MVA, 53, 54
 Artificial Neural Network, 466
 As/400, System i, 27
 Asanovic, Krste, 65
 ASATF - Aged Shortest Access Time First Policy, 203
 ASCII code, 9
 ASIC Application Specific Integrated Circuit, 65, 481
 Ask (Jeeves), 95
 ASM - Automatic Storage Management, 228
 Association Rule Mining - ARM, 419
 association rules, 509
 associative cache, 44
 Associative Processors, 408
 Asus PCs, 73
 asymmetric public key cryptography, 193
 asymmetric read/write, 423
 Asymptotic Job Bounds, 53
 AT&T American Telephone & Telegraph Corp., 21, 140, 272
 AT&T Research, 634
 ATA - Advanced Technology Attachment, 199
 Atempo, 160
 Atlas computer, 37
 attribute managed storage, 584
 Audio compression, 153
 audio file formats, 138
 Aurora DB for event streams, 545
 Autocoder, 23
 automatic file migration, 75
 automatic meter reading, 17
 AutoRAID, 171, 277
 Autoscale dynamic server provisioning, 379
 availability zone, 432, 538
 Avamar Dell/EMC, 161
 Avizienis, Algirdas, 5, 10, 265
 AVL tree, 116, 553
 AWS - Amazon Web Services, 145, 185, 432
 AWS Fargate, 530
 AWS Kinesis, 550
 AWS Neptune, 533
 Azure Cognitive Search, 535
 Azure Cosmos DB, 524, 525, 552
 Azure SQL DB, 507
 Azure Table Storage, 540
B
 B+-tree, 16, 351, 494
 B-code, 230
 B5000 computer, 27
 Babbage, Charles, 97
 Bach, Maurice, 200
 Bachman, Charles, 498
 Bachmat, Eitan, 237, 244
 Backblaze, 133, 318
 backup and recovery, 557
 Backus, John, 4
 Baek, Sung Hoon, 616
 Bairavasundaram, Lakshmi, 131, 301
 Baker, Mary, 623
 Balafoutis, Elias, 206
 balancing disk allocations, 568
 Bandic, Zandic, 333
 Banerjee, Prith, 303
 Bansal, Sorav, 39
 Baran, Paul, 99
 Bard, Yonathan, 55
 Barnes & Noble bookstore, 146
 Barracuda backup, 161
 Barroso, Luis, 301, 370
 Barton, Robert, 27
 BaseX - XML database, 543
 Basic Mirroring - BM, 224, 225
 Baskett, Forest, 213, 273
 battery lifetime, 350
 Baum, Rick, 406, 409
 BBS - Bulletin Board System, 94
 BBSRAM, 112
 BCH code, 334
 BCMP theorem, 55
 BCS - Binary Coded Decimal, 15
 Bechtolsheim, Andy, 273
 Becker Szendy, Ralph, 620
 Beehive Erasure Codes, 333
 Beigi, Homayoon, 100
 Belady, Laszlo, 38, 358
 Belady's algorithm, 358
 Belady's anomaly, 38
 Bell, Alan, 106
 Bell, Alexander Graham, 21, 126, 634
 Bell, Gordon, 33, 34
 Bell Labs, 21
 Bellman, Richard, 359, 398, 415
 benchmark STAC, 516
 Benes network, 461
 Benioff, Marc, 535
 Berkeley DB - Oracle, 542
 Berkeley Timesharing, 48
 Bernard, Oliver, 32
 Berners-Lee, Tim, 94
 Bershad, Brian, 350
 best fit, 38
 best path routing, 402
 BEST/1, 55, 279, 388
 BEST/1 capacity planning tool, 47
 Bestavros, Azer, 309
 Beyer, Betsy, 623
 Bezos, Jeff, 145
 BGS Corp., 47, 279
 Bhandarkar, Dileep, 33
 Bhide, Anupam, 215, 235
 Bianchini, Richardo, 354, 359
 BIBD - Balanced Incomplete Block Design, 303
 BIBD desirable properties, 304
 Big Data, 136, 520
 big data clustering, 510
 Big Data five Vs, 520
 big endian, 9
 Bigtable, Google, 540
 Bina, Eric, 94
 Birch clustering method, 510
 Birman, Alex, 47
 bitlocker encryption, 193
 Bitly, 517
 Bitton, Dina, 236
 Blasgen, Michael, 164
 Blaum, Mario, 275, 296, 318, 326, 329, 334, 337, 340, 343, 347
 Blauuw, Gerritt A., 5
 Blazegraph, 538
 block FLP numbers, 8
 block multiplexor channel, 55
 block size choice, 75
 blockchain block time, 561
 blocking records, 18
 Blomer, Johanness, 317
 Bloom filter, 159
 Bloomberg L.P., 141
 BlueDBM, 457
 Blum, Raymond, 623
 Bluray, 126
 Bobbio, Andrea, 249, 256
 Bohossian, Vasken, 230, 272, 347
 Bonwick, Jeff, 118, 274, 333
 book publishers, 630
 book4you.org, 146
 Boral, Haran, 279, 401
 BORG - Block reORGanization, 209
 Bosack, Len, 461
 Bose-Einstein statistic, 411
 bottleneck analysis, 53
 Boxma, Onno, 228, 294
 Bozman, Gerald, 38
 BPaaS - Business Process as a Service, 180
 Brady, Jim, 5, 318
 branch prediction, 24
 branch-and-bound algorithm, 420
 Brandt, Scott, 179
 Brandwajne, Alexandre, 22, 55
 Brewer, Eric, 165, 189, 436
 Broadcom, 498, 502
 Brooks, Frederick P., 5
 Brooks' law, 81, 400
 Bruck, Jehoshua, 230, 318, 320, 328, 340, 347
 Bruck, Shuki, 272
 Bruno, John, 203
 BSATF - Batched Shortest Time First, 203
 Btrfs, 425

- BtrFS - b-tree File System, 495
 - Bubba DBM, 401, 410
 - buffer management, 77
 - buffer retention, 164, 501
 - buffered disk, 74
 - BUNCH, 27
 - Burkhard, Walter, 163, 304
 - Burkhardt III, Henry, 7
 - Burroughs, 27
 - Burroughs A series, 29
 - Burroughs B series, 29
 - Burroughs B1000 series, 29
 - Burroughs B5500, 37
 - Burroughs B6500, 37
 - Burroughs-Wheeler-Transform - BWT, 148
 - Burrows, Michael, 312
 - burst error correcting code, 105
 - burstiness, 355
 - business intelligence, 627
 - busy period in M/G/1 queues, 291
 - Buterin, Vitalik, 562
 - Buzen, Jeff, 47, 54, 56, 279, 284, 388
 - BYNET interconnection network, 402
 - ByteDance, 503
 - Byzantine update commitment, 189
- C**
- C programming language, 6
 - C-FFS Co-locating Fast File System, 201
 - c-store, 539
 - cache coherence, 45
 - cache design, 33
 - cache disks, 75
 - cache miss penalty, 45
 - cache Physically Indexed, Physically Tagged, 46
 - cache Physically Indexed, Virtually Tagged, 46
 - cache Virtually Indexed, Physically Tagged, 46
 - cache Virtually Indexed, Virtually Tagged, 46
 - CacheRAID, 378
 - CAFS - Content-Addressable File Store, 408
 - Calderbank, Robert, 235, 237
 - Call Data Record - CDR, 543
 - calligraphy, 91
 - Cao, Pei, 76, 171, 358, 615
 - CAP Theorem, 190, 436
 - capacity optimized storage, 157
 - capacity planning, 47
 - CAR - Clock with Adaptive Replacement, 39
 - carbon emission, 381
 - Carbonite online backup, 264
 - Carey, Michael, 501
 - Caringo Swarm object storage software, 554
 - Carrera, Enrique, 354
 - Carry Lookahead Adder - CLA, 10
 - Carry Save Adder - CSA, 10
 - Carson, Scott, 208, 285, 313
 - CAS - Content Addressable Storage, 122, 158
 - Cascading, 522
 - Cassandra, 539
 - Cassandra column store, 444
 - CASSM - Context Addressable Cellular System for Non-Numeric Processing, 410
 - Cauchy RS code, 317
 - CAV Constant Angular Velocity, 197
 - CBIR - Content Based Image Retrieval, 73, 417, 510
 - CCD - Charge Coupled Device, 112, 409
 - CD Chained Declustering, 227
 - CD Compact Disk, 197
 - CDC - Control Data Corp., 4, 30
 - CDC 6600, 30
 - CDC 7600, 30
 - CDC Star-100, 30
 - CDN - Content Delivery Networks, 97
 - CDP - Continuous Data Protection, 199
 - Celkoe, Joe, 499
 - Ceph erasure coding, 329
 - Ceph file system, 179
 - Cerebras Swarm fabric, 489
 - Cerebras wafer-scale chip, 487
 - CERN - Conseil Europe'en pour la Recherche Nucle'aire, 94
 - CFA - computer family architecture, 35
 - CFQ - Completely Fair Queueing, 205
 - chaining in Cray 1, 56
 - Chamberlin, Donald, 499
 - Chandy, Mani, 78
 - Chang, Fay, 77
 - Chang, Hsu, 112
 - Charter Communications, 140
 - Chase, Jeff, 353
 - Chaudhary, Alok, 414
 - checking data integrity, 274
 - checkpoint based protocol, 78
 - checkpointing coordinated, 78
 - checkpointing multicomputer, 78
 - checkpointing Petascale, 78
 - checksums, 302
 - Chen, Mon-Song, 203
 - Chen, Peter, 202, 278, 282, 284, 499
 - Chen, Rui, 318
 - Chen, Shenze, 226, 231, 284
 - Chen, Tien Chi, 112
 - Cheng, Edward, 445
 - Chi Corp., 29
 - Chi-square test, 14
 - Chimera hybrid system, 393
 - Chinese Remainder Theorem, 10
 - Ching, Wai-Mee, 5, 516
 - Chiueh, Tzi-cker, 207, 586
 - Cho, Seokhei, 380
 - Chu, Lawrence, 171
 - Chu, Wesley, 39, 40, 396
 - Chubby lock service, 540
 - chunking in deduplication, 157
 - CIBER - Cellular Intercarrier Exchange Roamer, 543
 - CICS - Customer Information Control System, 497
 - Cincom systems, 498
 - circuit switching, 99
 - CISC - Complex Instruction Set Computer, 36
 - CISC microprocessor, 26
 - Cisco, 461
 - Cisco Research, 634
 - Cisco systems, 461
 - CKD - Count Key Data, 197, 496
 - CLARiiON disk array, 7
 - classification, 509
 - Claudio, Fleiner, 617
 - Clay code, 333
 - Clear Analytics, 161, 511
 - Cleversafe Inc., 167
 - CLIQUE clustering, 419
 - CLOCK page replacement algorithm, 38
 - CLOS network, 461
 - closed captioning, 100
 - cloud busting, 181
 - Cloud Object Storage, 555
 - cloud storage, 179
 - cloud: community, 181
 - cloud: hybrid, 181
 - cloud: private, 181
 - cloud: public, 181
 - Cloudera Dataflow, 550
 - Cloudera Inc. enterprise data cloud, 518
 - Cloudflare CDN, 97
 - Cloudian Hyperstore, 554
 - Cloudify, 530
 - Cloudify container, 530
 - CloudSearch Amazon, 536
 - Club of Rome, 102
 - cluster computing, 394
 - Clustered Placement - CP, 264
 - clustered RAID, 288, 566, 574
 - clustered RAID layouts, 307
 - clustering - agglomerative, 509
 - clustering - hierarchical, 509
 - clustering algorithms, 419
 - clustering groups items, 509
 - CLV Constant Linear Velocity, 197
 - CM2 Connection Machine, 280
 - CMG - Computer Measurement Group, 630
 - CMOS - Complementary Metal Oxide Semiconductor, 20, 103
 - CMRR - Center Memory & Recording Research, 632
 - CMU Carnegie Mellon Univ., 271
 - CMU/PDL DIXTRAC, 575
 - CMU/PDL self* storage projects, 586
 - CMU/PDL URSA minor project, 586
 - CN - Cartoon Network, 140
 - CNC - Computer Numerical Control, 98
 - CNN - Cable News Network, 140
 - Cobalt Iron backup, 178
 - COBOL programming language, 5, 498
 - Codd, Ted, 499
 - code - error correcting, 626
 - code ASCII, 9
 - code diff-MDS, 104
 - code EBCDIC, 9
 - code Unicode, 9
 - code utf8, 9
 - Coffman, Edward, 41, 201, 213, 235, 237
 - CoGaDB - Column Oriented, GPU accelerated, 468
 - Cognitive Search, Azure, 535
 - Cohen, Jacob, 228, 294
 - coherency control, 393
 - Cohesity - backup, disaster recovery, 557
 - Colab, 460
 - Colarelli, Dennis, 356

- Colgrove, John, 22, 122, 282
- color TV, 143
- columnar data, 450
- columnstores, 450
- COMA - Cache-Only Memory Architecture, 59
- Comcast Corp., 140
- commercial computer, 3
- Common Data Bus, 23
- Commvault, 557
- Commvault Hyperscale, 177
- compact code, 329
- compaction of data, 75
- Compaq Corp., 33, 72, 271
- competitive ratio, 374
- Complex event processing, 543
- computational memory, 125
- computational storage, 136
- computer algorithms, 628
- computer architecture, 625
- computer arithmetic, 628
- Computer Associates, 498, 502
- computer Atlas, 35
- computer bugs, 11
- computer company ranking, 34
- computer history, 629
- Computer History Museum, 23
- computer mediated txns, 136
- computer organization, 625
- Computer Sciences Corp. - CSC, 22
- Computing Correlation Connected Clusters, 419
- concurrency control in databases, 393, 616
- concurrency control optimistic, 79
- Concurrent computer corporation, 36
- conformed dimensions, 514
- Cong, Jason (UCLA), 84
- Connection Machine, 59
- Conner Chinook Disk, 236
- containers, 529
- context switching in multiprogramming, 53
- continuous archiving, 189
- Convergent Technologies Inc., 29
- Convex computer, 45, 57
- Convolutional Neural Net - CNN, 467
- coordinated checkpointing algorithm, 78
- coordinator log protocol, 428
- Copan Systems, 350, 357
- Copeland, George, 410
- copy on write buffering, 78
- copyback mode, 296
- Coraid Etherdrive, 178
- Corbato, Fernando, 7
- Corbett, Peter, 179, 318, 324, 340, 347
- Corona discharge effect, 11
- Cortado, James, 19
- Cortney, Jim, 235, 285
- cost of downtime, 243
- Couchbase, 520, 526
- CouchDB, 526
- Courtright, William, 178, 179, 212, 285, 310, 586
- coverage parameter, 266
- COW: Copy On Write, 274
- CPC - Cartesian Perceptual Compression, 151
- CPM - Critical Path Method, 16
- CPU cache, 44
- crate.io CrateDB, 552
- crater company, 45
- Cray, Seymour, 3, 56, 86
- Cray 1 supercomputer, 56
- CRC - Cyclic Redundancy Check, 165
- Cringley, Robert X., 20
- Cristian, Flaviu, 309, 428
- CRM - Customer Relationship Management, 535
- crosshatch disk array, 264
- Crunchbase, 559
- CryptDB, 192
- CSM - Central Server Model, 54, 388
- CSP - communicating sequential processes, 528
- CSRCMPSC macro - z/OS compress/expand, 156
- CSVD - Clustered SVD, 416
- CTMC - Continuous Time Markov Chain, 18, 253, 256
- CTMC transient analysis, 256
- CTSS - Compatible Time Sharing System, 7, 48
- cube operator - data warehouse, 512
- CUBRID RDBMS, 507
- Culler, David, 57
- Cullinet, 498
- Cumulo cluster, 556
- curse-of-dimensionality, 415, 421
- CVE - Common Vulnerabilities & Exposures, 11
- CVS - Concurrent Version System, 204
- CXL - Compute Express Link, 74, 391
- Cyclical Scan disk scheduling, 201
- D**
- D-GRAID - gracefully degrading RAID, 199
- D-SPTF - Distributed Shortest Positioning Time First, 205, 234
- DACO - Disk Architecture for Composite Operations, 285, 573
- DAD - Disk Array Designer HP Labs, 171
- Dagger algorithm, 562
- DAM - Direct Access Method, 495
- Damped Working Set - DWS, 39
- Dan, Asit, 40
- DAnA - Database Acceleration of Advanced Analytics, 479
- dark silicon, 85, 479, 480
- DARPA, 57
- DAS - Direct Attached Storage, 277, 461
- DASD - Direct Access Storage Device, 108, 278
- data access skew, 228
- data allocation model, 396
- data allocation problem, 398
- data at rest, 147
- data centric computing, 523
- data cleaning, 623
- data cleansing, 15, 623
- Data Comp DEFLATE, 148
- Data Comp LZ4, 148
- Data Comp LZ4r, 148
- Data Comp LZ77, 148
- Data Comp LZ78, 148
- Data Comp LZ0, 148
- Data Comp LZW, 148
- Data Comp PNG, 148
- Data Comp ZIP, 148
- Data Comp Zstandard, 148
- Data Comp zstd, 148
- data compression, 147
- data compression - irreversible, 147
- data compression - lossless, 147
- data compression - lossy, 147
- data compression - reversible, 147
- data compression ratio, 147
- data deduplication, 157
- Data Direct Networks - DDN, 555
- Data Domain, 159, 272, 634
- data encryption, 191
- Data General, 7
- data in transit, 147
- data lake, 460, 515
- data lineage, 623
- data mining, 509
- data partitioning index, 421
- data preservation, 623
- data provenance, 623
- data reduction, 157, 516
- data replication, 208
- data science, 627
- data set placement, 75
- data transfer rate, 200
- Data Warehouse design, 513
- data warehousing, 500, 513
- data-centric computing, 462
- Data-Crusher for compression, 153
- database buffer, 77
- database clustering, 445
- database hierarchical, 497
- database machine mtngs, 632
- database reorganization, 351
- database snapshot: readonly static view, 274
- database systems, 627
- database triggers, 433, 546
- Databricks, 460
- dataflow paradigm, 482
- dataflow scheduling, 409
- DataMesh project at HP Labs, 171
- DataNode in Hadoop, 523
- Datastax, 539
- Datastax Enterprise, 540
- Datastore, Google Cloud, 527
- Date, Chris, 497
- Datrium, 159
- DATUM disk array, 168, 309
- David, Herbert, 279
- DB call shipping, 395
- DB engine rankings, 503
- DB2 Parallel Edition, 395, 429
- DB2 relational database, 500
- Db4o, 534
- DBIM - DataBase In Memory, 449
- dblp, 630
- DBMIN buffer management, 78
- DBMS hierarchical, 497
- DBMS network, 498
- DCT - Discrete Cosine Transform, 140, 151
- DDA - Data Deduplication Appliance, 162

- DDN - Data Direct Networks, 178
 - de Castro, Edson, 7
 - DEA - Data Encryption Algorithm, 192
 - deadline scheduler, 205
 - debit/credit benchmark, 400
 - DEC - Digital Equipment Corp., 6, 33
 - DEC Alpha microprocessor, 33
 - DEC PDP/11, 35
 - DEC Research Center SRC, 33
 - DEC VAX - Virtual Address eXtension, 33
 - decimal packed, 8
 - decimal unpacked, 8
 - Decision Support System - DSS, 413
 - decision tree, 509
 - Declobbered Placement - DP, 264
 - declustering ratio in RAID5, 566, 591
 - decoupled storage, 454
 - dedicate on write, 172, 277
 - dedicated sparing, 295
 - dedupe aliasing, 160
 - dedupe inline, 160
 - dedupe offline, 160
 - dedupe with chunks, 160
 - deduplication companies, 160
 - deduplication granularity, 160
 - deduplication indexing options, 160
 - Deep Learning - DL, 466, 627
 - Deep Neural Network - DNN, 459, 477
 - degraded mode operation in RAID, 242, 281
 - delay cycle analysis in M/G/1 queues, 291
 - delay line memories, 101
 - delayed writeback, 351
 - Dell Corp., 72
 - Dell, Michael, 175
 - Dell Powerproject Cyber Recovery, 557
 - Dell/EMC, 172
 - Dell/EMC Isilon, 555
 - delta differencing, 157
 - delta encoding data compression, 160
 - Δ -tree high-dimensional index, 416
 - Dempsey, 356
 - Deng, Yuhui, 369
 - Dennard, Robert, 85, 103
 - Dennard scaling rule, 85
 - Denning, Peter, 36, 41, 201
 - Dennis, Jack, 482
 - denormalization of relational tables, 500
 - denormalized relational tables, 445
 - Department of Energy - DoE, 637
 - Desnoyers, Peter, 211
 - destaging locality, 287
 - deterministic mapping, 583
 - Deuteronomy storage engine, 451
 - Devlin, Barry, 513
 - DevOps, 175
 - DeWitt, David, 78, 227, 401, 409
 - DFSMS Data Facility Storage Management Subsystem (z/OS), 198
 - DFTL - Demand-based Flash Translation, 380
 - Dholakia, Ajay, 260, 296, 351
 - DHT - Distributed Hash Table, 434
 - DIAM - Device Independent Access Method, 499
 - Dias, Dan, 215, 447
 - Dias, Daniel, 235
 - diff-MDS code, 9
 - differential evolution, 398
 - digital archive, 245
 - Digital Equipment Corp. - DEC, 271
 - digital permanence, 623
 - Digital Public Library of America, 146
 - Dimakis, Alexandros, 320, 328
 - DiME - Dynamic in-Memory Expression, 450
 - DIMM - Dual Inline Memory Module, 103
 - Ding, Xiaoning, 204
 - DIRECT database machine, 409
 - Direct to Wire Protocol - DoWP, 449
 - direct-mapped CPU cache, 44
 - directory based cache coherence in multiprocessors, 45
 - DirecTV, 140, 144
 - disaggregated infrastructure, 472
 - disaggregated storage, 437
 - Disaster Recovery - DR, 248
 - DISH network, 141, 144
 - disk access time, 200
 - disk allocation policy, 200
 - disk arm scheduling methods, 201
 - Disk Array Controller - DAC, 272
 - disk cache - onboard, 75
 - disk cache design, 75
 - disk consistency checker, 132, 301
 - disk cooling, 587
 - disk defragmentation, 38
 - disk file placement, 208
 - disk scheduling, 75
 - disk scrubbing, 260, 298, 351
 - Disk Scrubbing Algorithm - DSA, 298
 - disk seek time, 199
 - disk shuffling, 171, 208
 - Disk Working Set - DWS, 310
 - Disk-Oriented Rebuild - DOR, 289
 - disk-resident, 421
 - disk-to-disk backup, 158
 - Diskgroup - energy efficient data layout, 365
 - DiskReduce - replication than erasure coding, 225
 - DiskSeen data placement, 204
 - DiskSim simulator, 212, 353
 - disruptive innovation, 33
 - distributed message queue, 543
 - distributed NoSQL, 523
 - distributed sparing, 295
 - Ditzel, David, 36
 - Diverted Accesses - save energy via redundancy, 364
 - division by multiplication (S/360 Model 91), 11
 - DIXTRAC tool to extract disk parameters, 212
 - DjVu file format, 139, 151
 - DNA - DeoxyriboNucleic Acid Storage, 128
 - Docker container, 529, 530
 - document-oriented DB, 525
 - Domain Driven Design, 545
 - domain specific architecture - DSA, 84
 - domain well memory, 126
 - Donnelly, Austin, 369, 378
 - Dorwood, Sean, 158
 - Doshi, Bharat, 289
 - dot-com bubble, 273
 - dot.com domains, 94
 - double buffering, 18
 - double helix, 128
 - double paging anomaly, 77
 - Douglis, Fred, 153, 163, 301, 311, 350, 351
 - Dowdy, Lawrence, 216
 - downcoding in HACFS, 329
 - DPM - Data Protection Manager, 162
 - DPU - Data Processing Unit, 461
 - DRaaS: Disaster Recovery as a Service - DRaaS, 177
 - DRAM - Dynamic Random Access Memory, 43, 85, 103, 225
 - Drapeau/Chervenak, Ann, 271
 - DRDtool, 529
 - DRPM - Dynamic RPM scheme, 354
 - drum memory, 37
 - Druschel, Peter, 209
 - Druva backup, 161, 557
 - Druva Phoenix, 161
 - DS4800 IBM RAID array, 166
 - DSN - Dispersed Storage Network, 167
 - DSP - Digital Signal Processing unit, 473
 - DSSD flash company, 118
 - DTM Dynamic Thermal Management, 364
 - DTMC Discrete Time Markov Chain, 257
 - dual ported devices, 56
 - dual striping RAID1, 228
 - Dubnicki, Cezary, 363
 - Dupecatcher, 161
 - duplication, 523
 - durability, 432
 - DVFS - Dynamic Voltage and Frequency Scaling, 361, 379
 - DWDM - Dense Wavelength Division Multiplexing, 40
 - DXC Tech.: CSC + HP-IT, 22
 - Dykes, John, 364
 - dynamic hot spares, 224
 - dynamic memory allocation, 6
 - Dynamic Path Reconnection - DPR, 56
 - Dynamic Path selection, 56
 - dynamic programming, 359, 415, 499
 - dynamic routing, 410
 - dynamic server provisioning, 378
 - dynamic speed control, 354
 - Dynamic Storage Allocation - DSA, 38
 - Dynamic Tier Manager - DTM, 587
 - DYNAMO simulation language, 102
 - DynamoDB Amazon NoSQL Key-Value DB, 525
 - Dynix - DYNAmic Unix, 7
- ## E
- e-books, 146
 - E-RAID - energy efficient RAID, 368
 - EAFDL - Expected Annual Fraction of Data Loss, 262, 264, 566
 - eager replication, 190
 - EAMR - Energy Assisted Magnetic Recording, 109, 200
 - EBCDIC code, 9
 - EBDT - Extent Based Dynamic Tiering, 587

- ECC - Error Correcting Code, 165, 302
- ECKD - Extended Count Key Data, 197, 224, 272, 496
- Eckert, J. Presper, 29
- Eckert, John, 105
- ECL - Emitter Coupled Logic, 20
- economic lifespan, 253
- ecStore - elastic cloud storage, 435
- EDF - Earliest Deadline First, 205
- edge computing, 47, 101, 550
- EDI - Electronic Data Interchange, 519
- Edison, Thomas Alva, 11
- EDS - Electronic Data Systems, 22
- EDSAC computer, 24
- EELRU - Early Eviction LRU, 39
- EEPROM, 103
- Effelsberg, Wolfgang, 77
- eigenvalues, 418
- eigenvectors, 418
- elastic cloud, 184
- Elastic Cloud Storage - ECS, 555
- elasticity, 436
- Elasticsearch, 534
- Elastras, 435
- Eleftheriou, Evangelos, 118
- elevator scheduling for disks, 201
- ELI programming language, 5, 516
- elliptic key cryptography, 193
- Ellison, Larry/Lawrence (Oracle), 502
- Elnozahy, Mootaz, 78, 361
- embedded inodes - Unix file system, 201
- EMC Centera, 585
- EMC Corp., 272
- EMC Greenplum, 508
- EMC SRDF Symmetric Remote Data Facility, 248
- EMC Symmetrix, 224, 272
- emulation mode - IBM S/360, 15
- Encore computer, 34
- ENIAC computer, 29
- enterprise bus matrix, 514
- Enterprise Storage - ES, 199
- EnterpriseDB based on PostgreSQL, 552
- ephemeral data, 137
- EPIC (Explicitly Parallel Instruction Computing) processor, 31
- EPMR - Energy-enhanced Perpendicular Magnetic Recording, 109
- Epstein, Bob, 502
- ER - Entity Relationship model for logical DB design, 499
- erasure coding, 223
- Ercegovac, Milos, 628
- Erlang probability distribution, 242
- Escher, Maurits, 91
- escrow protocol, 497
- EsperTech streaming analytics, 551
- Essbase Hyperion/Oracle, 511
- Estridge, Philip, 71
- eternal memory, 128
- ETHash algorithm, 561
- ETHash ASIC-resistant algorithm, 562
- Ethereum blockchain, 561
- Ethernet, 388
- ETL: Extract, Transform, Load, 507
- Euclidean distance, 510
- Euler method for ODEs, 256
- Europeana digitized cultural heritage, 146
- EVENODD code, 296
- EVENODD parity code, 318
- event store, 543
- EventStoreDB, 544
- eventual consistency, 432
- evolutionary computing, 398
- exactly once semantics, 443
- Exagrid EX, 161
- ExaPlan data placement, 587
- EXCP - EXecute Channel Program, 25, 406
- Expanded Storage, 26
- expectation discord, 204
- experience curve, 182
- explicit grouping - Unix file system, 201
- exponential distribution memoryless, 253
- ext3 file system, 132
- ext4 - 4th extended files, 425, 495
- extensible hashing, 496
- external memory algorithm, 75
- Extreme value probability distribution, 242
- F**
- F-score statistics, 133
- F2FS - File System for Flash Storage, 425
- FAB Federated Array of Bricks at HP, 617
- Fabry, Robert, 40, 200
- Facebook Haystack, 247
- Facebook Inc., 536
- Facebook Research, 634
- Facebook Terragraph, 93
- Falconstor VTL - Virtual Tape Library, 162
- Falkoff, Aldin, 5
- Faloutsos, Christos, 413
- FAMISH - FAir MIXed-scan Scheduling, 206
- Fano, Robert, 147
- Fargate AWS container, 530
- Farrell, Patrick, 343
- fast code runtime library, 329
- fast write to NVS - NonVolatile Storage, 235
- FAT - File Access Table for MS/DOS, 198
- fault injection, 274, 289
- fault-tolerance, 625
- FAWN - Fast Array of Wimpy Nodes, 441
- FBA - Fixed Block Architecture, 165, 275
- FBLeaRner Flow FB's AI backbone, 459
- FCC - Federal Communications Commissions, 100
- FCFS scheduling, 201
- FedEx - Federal Express, 93
- feedback loop, 362
- Feitelson, Dror, 75, 179, 628
- Felten, Ed, 76
- Feng, Dan, 163, 290, 316
- Feng, Gui Liang, 340
- Fernandez, Eduardo, 77, 410
- Ferrari, Domenico, 35, 41, 414
- Ferroelectric RAM, 112
- fsck - fast file system consistency check, 132, 301
- FGCS - Fifth Gen. Computer Systems, 410
- FHE - Fully Homomorphic Encryption, 193
- FIFO page replacement, 38
- file Btrfs, 495
- file ext4, 495
- file migration, 75
- file systems, 494
- Filemaker, 507
- Filemaker cross platform relational DB, 507
- finite field arithmetic, 340
- Fiorina, Carly (HP), 32, 271, 400
- FIPS - Federal Information Processing Standard, 192
- Firestore, 526
- Firefox web browser from Mozilla, 94
- Firestore, Google Cloud, 526
- first fit storage space allocation, 38
- Fixed Block Architecture - FBA, 224
- Flash compression, 152
- flash erase before write, 424
- flash memory, 24, 115
- flash SSD reliability, 134
- flash wearout, 423
- Flatto, Leopold, 235, 237, 241
- Fletcher's checksum for error detection, 274
- floating parity, 285
- FLP - Floating Point, 3, 8
- FLP IEEE 754, 8
- Flynn's classification, 387
- Ford, Daniel, 106
- foreign key for relational tables, 500
- fork-join queueing systems, 283
- Forrester, Jay, 33, 102
- Fortran programming language, 4
- FoundationDB multi-model distr. NoSQL DB, 444
- Fox Corp., 141
- FPGA - Field Programmable Gate Array, 65, 473
- FPGA accelerated compaction, 455
- fractal compression, 151
- fractal tree index, 495
- fragmentation, external, 37
- fragmentation, internal, 37
- frames and resolvable designs, 310
- Franaszek, Peter, 153, 288, 618
- Franke, Hubertus, 354
- Franklin, Jason, 442
- freeblock scheduling in disks, 202, 413
- Freitas, Richard, 110
- Frequency Scaling, 442
- Frequency-Based (page) Replacement - FBR, 39
- FSCAN, 201
- fsck - file system consistency check, 132, 274
- FTL - Flash Translation Layer, 122, 424
- FTL address mapping, 424
- FTL garbage collection, 424
- FTL out of place update, 424
- FTL wear leveling, 424
- Fu, Gang/Thomas, 209, 237, 351
- Fuja, Thomas, 337
- Fujita, Hachiro, 340
- Fujitsu Corp., 21, 273
- full stripe write for RAID6/6/7, 274, 276, 278

Fuller, Samuel, 37, 213
 fully-associative cache, 44
 Functional Level - FL, 5
 Functional MSR code, 331
 Functional Programming - FP, 4
 Fungible storage architecture, 461
 Furino, Steven, 310
 Future Systems Project at IBM, 20

G

Gad, Eyal, 333
 Gain-cell embedded DRAM, 103
 Gallagher, Robert G., 341
 Gallagher codes, 341
 Galois Counter Mode - GCM, 193
 Galois field arithmetic, 317
 Gamma DBM, 401, 409
 Gandhi, Anshul, 378, 379
 Ganger, Greg, 179, 201, 203, 212, 232, 271, 279, 371
 garbage collection, 314
 Garcia-Molina, Hector, 190, 279
 Garner, Harvey, 10
 Garner, Richard, 617
 Gartner group, 243
 Gartner Magic Quadrant, 554
 Gateway, 72
 Gawlick, Dieter, 22, 282, 445, 497
 GCLOCK - Generalized Clock page replacement, 40
 GE - General Electric Corp., 7, 31
 GE MULTICS, 7
 Gehani, Narain, 21
 Geist, Robert, 202, 238
 general purpose computer, 3
 General Purpose Register - GPR S/360, 23
 Georgiadis, Leonid, 284
 Gerald, Jeanette, 41
 Gerstner, Louis, 275
 GI/M/1 queueing system, 241
 Gibson, Garth, 76, 77, 131, 178, 179, 212, 249, 270, 278, 285, 289, 310, 337, 340, 413, 586, 616, 626
 GIF compression scheme, 148
 Gifford, David, 451
 GIN - Generalized Inverted Index, 505
 GiST Generalized Search Tree, 505
 Gladney, Henry, 623
 Glaser, Ted, 29
 glass storage, 128
 Glider, Joseph, 618
 Go programming language, 528
 Goel, Atul, 318, 340
 Goldberg, Robert, 530
 Golding, Richard, 205, 234, 295, 616, 618, 620
 Goldman encoding, 129
 Goodman, Rodney, 340
 Goodson, Garth, 131
 Goodson, Gregory, 301
 Google Adwords, 97
 Google BigQuery, 507
 Google BigTable, 430, 540
 Google book scanning effort, 147
 Google Cloud Datastore, 527

Google Cloud Firestore, 526
 Google Cloud Platform, 185
 Google F1 DB, 430
 Google File System - GFS, 247, 429, 539
 Google Firebase, 526
 Google Hummingbird, 95
 Google Kubernetes, 530
 Google pagerank algorithm, 95
 Google Research, 634
 Google Search Engine, 95, 536
 Google's Spanner, 430
 Gossling, James, 273
 Goyal, Ambuj, 165, 447
 GPFS - Global Parallel File Systems, 228
 GPU - Graphics Processing Unit, 66, 68, 408, 467, 470
 GPUDB, 468
 GPUQP, 468
 GPUDx, 469
 Graefe, Goetz, 83, 279, 494
 Graham, G. Scott, 41, 55
 Graham, Ronald, 374
 graph databases, 531
 GraphDB, 538
 Graphite time series data, 529
 Gray, Jim, 57, 82, 199, 236, 283, 399, 512
 Gray/Shenoy predictions, 82
 Gray's 5-minute rule, 83
 GRD - Group Rotate Decustering, 226
 greedy rebuild in mirrored disks, 244
 Green, Paul, 40
 Green AI, 381
 GreenAI efficiency, 381
 Greenan, Kevin, 318, 367, 369
 grid codes, 616
 Grosch's Law, 86
 Gross, Thomas, 153
 Group Sweeping (disk) Scheduling - GSS, 203
 Grunwald, Dirk, 356
 GSTF - Grouped Shortest Seek Time First, 202
 GT.M key-value DB, 534
 Guerra, Jorge, 587
 Gunawi, Haryadi, 132
 Gupta, Anoop, 57
 Gupta, Anurag, 451
 Gupta, Ashish, 450
 Gupta, Varun, 379
 Gurumurthi, Sudhanva, 236, 353, 354, 364
 Gustafson, John, 9

H

H-store DBMS, 127
 H-store main memory DB, 554
 H2O ML framework, 454
 HACFS - Hadoop Adaptively-Coded Distributed File System, 329
 Haderle, Don, 501
 Hadoop DataNode, 523
 Hadoop Distributed File System - HDFS, 518
 Hadoop software for reliable distributed computing, 521
 HADP - Human-Active, DBMS-Passive model, 432, 545
 Haerder, Theo, 380
 Hafner, James, 230, 295, 301, 334, 340, 370, 617, 618
 Hager cached RAID controller, 164
 Hagmann, Robert, 414
 Hahn, S., 241
 Hal Varian, 136
 Hamming, Richard, 280
 Hamming-code utilized in RAID2, 280
 HAMR - Heat Assisted Magnetic Recording, 109, 200
 Han, Chunqi, 232, 586
 HANA, SAP, 553
 handwriting recognition, 100
 Hanson, Eric, 450
 Hao, Mingzhe, 132
 Hard Disk Drive - HDD, 197
 Harder, Theo, 77, 279
 Harris Corp., 498
 Hartman, John, 315
 Harvard architecture, 35
 hash joins, 409
 Hashimoto algorithm, 562
 HashTag Erasure Code - HTEC, 329
 Haskin, Roger, 179, 228
 Hatfield, Donald J., 41
 Hazelcast in-memory data grid, 525
 hB-tree multi-dimensional index, 421
 Hbase - column-oriented nonrelational DBMS, 538
 HBM - High Bandwidth Memory, 74, 470
 HBO - Home Box Office, 140
 HCL Technologies - India, 501
 HDA - Hybrid Disk Array, 223
 HDA 2-level allocation, 586
 HDA allocation methods, 569
 HDA clustered RAID5, 580
 HDA data allocation simulation, 575
 HDA degraded load, 574
 HDA Heterogeneous Disk array, 567
 HDA normal load, 572
 HDA rebuild processing, 582
 HDA response time, 570
 HDA workload, 572
 HDD - Hard Disk Drive, 108, 225, 567
 HDFS - Hadoop Distributed File System, 224, 247, 518, 521
 HDFS adaptive coding, 329
 HDFS Xorbas, 328
 heat clustering, 372
 hedged requests are deferred, 233
 Heegard, Chris, 337
 Hellerstein, Joseph, 413, 505
 Hellerstein, Lisa, 337
 Heptagon code, 331, 332
 heterogeneous processors, 284
 Hetzler, Steven, 334
 HFT - high frequency trading, 546, 627
 HGST - Hitachi Global Storage Technologies, 108, 276
 HGST Research, 634
 hibernator power saving disk array, 362
 Hierarchical RAID, 593
 High Frequency Trading - HFT, 546
 High Level Language computer, 36

- Hisashi, Kobayashi, 627
- histograms for RDBMS, 502
- Hitachi Vantara, 173, 555
- Hitchhiker code, 329
- Hitz, David, 277
- Hive, 522
- Hive, Apache data warehouse, 506
- Hive Apache, 506
- Hofri, Micha, 201, 213, 236
- Holland, Mark, 285, 289, 290, 304, 310
- Hollerith, Herman, 98
- Hollerith cards, 98
- holographic data storage, 126
- homographic resolution, 517
- Honeywell Corp., 7, 31
- Hopper, Grace, 5
- horizontal placement, 587
- horizontal scaling, 523
- horizontally partitioned, 409
- Horner's method, 159
- hot mirroring, 290
- hot standby, 428
- hotset algorithm, 77
- hotset buffer management, 78
- HP - Hewlett Packard, 31
- HP AutoRAID, 584
- HP CASA Continuous Access Storage Appliance, 248
- HP Corp., 271
- HP DAD Disk Array Designer, 584
- HP Forum, 584
- HP Labs, 634
- HP Minerva, 584
- HP The Machine, 32
- HP Zerto, 557
- HP3000 computer, 29
- HPE 3PAR, 172
- HPE HP Enterprise, 272
- HPE Omnistack Accelerator Card, 175
- HPE Simplivity, 175
- HRAID, 593
- HRAID concurrency control, 596
- HRAID redundancy level, 594
- HRAID workload, 599
- Hsiao, David, 163, 406, 409
- Hsiao, Hui-I, 227
- Hsu, Jiwu, 573
- Hsu, Windsor, 199, 204, 208, 301, 371, 414, 569
- HTAP - Hybrid Transaction Analytic Processing, 447
- http - hypertext transfer protocol, 519
- Huang, Cheng, 316, 326, 327
- Huang, Chiang, 326
- Huang, Jensen, 471
- Huawei OceanStor 900 V5, 555
- Huawei Tech., 461
- Huawei Technologies Co. Ltd., 173, 461
- Huffman, David, 147
- Huffman code, 8, 92, 129, 147
- Hui, Ken, 193
- Hunter, David, 55
- HVSC High Voltage Direct Current, 11
- Hwang, Kai, 57
- hybrid architecture, 393
- Hybrid Disk Drive - HDD, 369
- HYDRAsstor storage platform, 363
- hype curve, 185
- HyPer, 554
- HyperConverged Infrastructure - HCI, 174
- hypercube machine, 60
- Hyperion, Essbase, 511
- HyperLogLog, 487
- hyperscalar, 165
- Hypertable, 538
- HYRISE main memory database, 553
- I**
- i-Distance high-dimensional index, 416
- I/O formatted, 8
- I/O multipathing, 75
- I/O request shipping, 395
- I/O trace, 238
- I/O unformatted, 8
- I/O workload outsourcing, 290
- IAAS Infrastructure as Service, 180
- IBM - International Business Machines, 3
- IBM 1130, 3, 4, 14
- IBM 1410, 23
- IBM 1620, 3
- IBM 1800, 4
- IBM 2311 drives, 16
- IBM 2314 drives, 16
- IBM 2315 disk, 14
- IBM 5100 APL computer, 5
- IBM AIX OS, 74
- IBM Cognos, 558
- IBM Corp., 274
- IBM Db2, 506
- IBM Db2 Event Store, 545
- IBM eXtended Remote Copy - XRC, 248
- IBM GPFS - General Parallel File System, 179, 555
- IBM Harvest, 30
- IBM IceCube intelligent bricks project, 295
- IBM Infosphere, 501
- IBM Millipede, 113
- IBM MVS - Multiple Virtual Storage OS, 500
- IBM Netezza, 508
- IBM OS/2, 72
- IBM ProtecTier data deduplication, 162
- IBM PS/2, 72
- IBM Research Labs, 635
- IBM RP3 - Research Parallel Processor Project, 5
- IBM RS/6000, 5
- IBM RS/6000 workstation, 74
- IBM RT/PC, 73
- IBM S/360 91, 30
- IBM SCRIPT text editor and formatter, 144
- IBM Spectrum Protect Plus, 557
- IBM Spectrum Scale, 179, 555
- IBM Spectrum storage, 167
- IBM storage, 172
- IBM Streaming Analytics, 547, 549
- IBM Stretch, 30
- IBM System R, 500
- IBM System/360 family, 3, 19, 23, 35
- IBM Telum processor for Z mainframes, 19
- IBM TPF - Transaction Processing Facility, 199
- IBM Ultrastar disks, 575
- IBM XRC - eXtended Remote Copy, 249
- IBM zIIP - z Integrated Information Processor, 541
- IBM's ADSTAR, 108
- IBM's Blue Gene Supercomputer, 61
- IBM's Peer-to-Peer Remote Copy, 248
- IBM's Public Cloud, 185
- ICL - Int'l Computers Limited, 408
- IDA - Information Dispersal Algorithm, 168, 309
- IDE - Intelligent/Integrated Data Electronics, 199
- IDEMA - Int'l Disk Drive Equipment and Materials Association, 378
- identity management, 505
- IDISK - Intelligent Disks project at Berkeley, 413
- idle detectors, 351
- idle period, 291
- IDMG - In Memory Data Grids, 525
- IDMS - Integrated Database Management Systems, 498
- IDR IntraDisk Redundancy, 260, 296, 351
- IEEE - Inst. of Electrical and Electronics Engineers, 629
- Iliadis, Ilias, 106, 260, 262, 264, 296, 587
- ILLIAC IV computer, 57
- ILP - Instruction Level Parallelism, 31
- image compression methods, 151
- IMDB - In-Memory Data Base, 525
- IMDG - In-Memory Data Grid, 525
- Imelienski, Tomasz, 509
- Impetus Stream Analytix, 550
- IMPICA - In Memory PoInter Chasing Accelerator, 125
- importance sampling, 266
- IMS - Information Management System, 497
- IMS Fast-Path, 497
- IMT - Interleaved MultiThreading, 69
- in-chip analytics, 560
- INaaS INformation as a Service, 180
- inclusion property, 39, 584
- Independent Reference Model- IRM, 39
- index B+-tree, 494
- index fractal-tree, 494
- index sequential files, 494
- InfiniBand, 465
- InfiniBand high-speed interconnect standard, 388, 448
- InfluxDB for time series, 528
- information entropy, 148
- Information Retrieval, 518
- Informix Corp., 501, 508
- Infotrend, 178
- Infosys - India, 501
- Ingres DBMS histograms, 502
- Ingres relational database, 502
- Inktomi Corp., 189
- Inmon, Bill, 513
- Inmon method, 513
- InnoDB storage engine, 127
- Inspur Active Storage, 555

INSPur AS130000, 555
 instance store, 455
 instruction registers, 44
 instruction set orthogonal, 33
 integer linear programming, 396
 integer programming, 115
 Intel CXL Compute eXpress Link, 392
 Intel iSPC/2 hypercube, 409
 Intel Itanium, 31, 32
 Intel Omnipath, 465
 Intel Optane, 123
 Intel Pentium FDUV bug, 11
 Intel Research, 635
 Intel RST - Rapid Storage Technology, 118
 Intel/Micron 3D-Xpoint, 123
 intelligent compression, 157
 Intelligent Secondary Storage Devices, 408
 interactive platforms, 137
 interconnection networks, 388
 Interdata 8/32, 35
 Interleaved Declustering - ID, 227
 Interleaved Declustering layout for mirroring, 272
 Internet, 92
 Internet Archive, 146
 internet registry, 95
 internode coding in HRAID, 594
 Intersystems Cache[®] for unstructured data analysis, 508
 Intersystems IRIS, 534
 intradisk parallelism, 236
 intranode coding in HRAID, 594
 inverted index DB, 541
 IO density IOPS/TB, 198
 Iomega zip drives, 72
 IoT - Internet of Things, 69
 Ipad tablet from Apple, 197
 Iphone - smartphone from Apple, 197
 Ipod portable media player, 197
 iPSC - Intel Personal SuperComputer, 61
 IRAW - Idle Read After Write, 302
 IRM - Independent Reference Model, 40
 IRON - Internal RObustNess file system, 302
 IronWorker distributed work-on-demand container, 530
 ISA - Instruction Set Architecture, 3, 23
 ISAM - Index Sequential Access Method, 351, 494
 iSeries (IBM AS/400), 27
 ISP - Internet Service Provider, 92
 ISTC - Information Storage Technology Consortium, 108
 Ithaka Harbors, digital technology for higher ed, 147
 Iverson, Kenneth, 5
 IWARP - Internet Wide Area RDMA Protocol, 392
 Iyer, Balakrishna, 149, 232, 351

J

Jacobs, Bruce, 44, 103
 Java bytecode, 36
 Java programming language, 273, 503
 Java Virtual Machine - JVM, 551

jBase - multivalue database, 542
 JBIG2 image compression standard, 151
 JBOD - Just a Bunch of Disks, 279
 JCA - Java Cryptography Architecture, 520
 JDBC - Java Data Base Connectivity, 520
 Jena - Apache, 537
 JES - Job Entry Subsystem for IBM mainframes, 18
 Ji, Minwen, 248
 Jiang, Hong, 163, 290, 316
 job tracker distributes mapreduce tasks, 523
 Johnson, Reynold, 108
 Join the Shortest Queue - JSQ, 232
 Joukov, Nikolai, 352, 586
 Jouppi, Norman, 84, 85
 Joy, Bill, 200, 273
 JPEG - Joint Photographic Experts Group, 151
 JPEG 2000, 151
 JPL STAR, 399
 JPL STAR computer, 101
 JSON - JavaScript Object Notation, 520
 JSON DB, 520
 Juniper Networks, 461

K

k-d tree main memory index, 415
 k-d-B trees for disk storage, 415
 k-means clustering, 510
 k-nearest neighbors - k-NN, 510
 k-replicated seek distance, 236
 k-step SCAN, 201
 Kaashoek, Frans, 201, 264
 Kahle, Brewster (Internet Archive), 146
 Kaminario all flash storage, 174
 Kaminsky, Michael, 441, 442
 Kandemir, Mahmut, 354, 359, 414
 Kandlur, Dilip, 203
 Kaplan, Scott, 153
 Kari, Hannu, 298, 351
 Karlin, Anna, 76
 Karp, Richard, 337
 Kasson, James, 285
 Katz, Randy, 270, 271, 278, 281–283
 kdb+, 528
 Keccak algorithm, 562
 Keeton, Kimberly, 32, 362, 413
 Kemurdjian, Alexander, 152
 Kenchammana-Hosekote, Deepak T., 616
 Keras Python interface to ANNs, 459
 Kerberos Authentication protocol, 556
 Kermani, Parviz, 99
 Kernel Virtual Memory - KVM, 448
 Key Performance Indicator - KPI, 514
 Key-Value KV SSD, 136
 Key-Value store, 435, 524
 Khoshafian, Setrag, 279
 Khosla, Vinod, 273
 Kim, Chang Soo, 110
 Kim, Han-Joon, 377
 Kim, Michelle, 279
 Kim, Youngjae, 364
 Kimball, Ralph, 513
 Kimball method, 514
 King, Richard, 237, 351

Kioxia - Toshiba flash spinoff, 121
 Kitsuregawa, Masaru, 163, 290, 410
 Kiviat graph, 35
 KLDDiscovery tool for eDiscovery, 162
 Kleinrock, Leonard, 99, 163, 627
 KLT - Karhunen-Loève Transform, 417
 Knight, Joshua, 278
 Knuth, Donald, 105
 Knuth, Donald E., 628
 Kobayashi, Hisashi, 55
 Kogan, Jakob, 510
 Kollias, John, 235
 Konheim, Alan, 192
 Kontena classic, 530
 Kooi, Bob, 502
 Koomey's law, 85
 Koren, Israel, 628
 Kozuch, Michael, 379
 KPI - Key Performance Indicator, 181
 Kravelska, Katina, 329
 Krawczyk, Hugo, 168
 Krishnamurthy, Arvind, 236
 Krishnan, Arvind, 20, 186
 Krishnan, Nikhil, 331
 Kryder, Mark, 110
 Kryder's law, 86, 297
 KSR-1 computer, 59
 Kubernetes, 529
 Kubiawicz, John, 189, 264
 Kuck, David, 3
 Kuenning, Geoff, 365
 Kung, H.T., 409
 Kung's law, 81
 Kx systems, 516
 Kybos, 617, 618

L

L2P - Logical to Physical space, 120
 Lagrange multiplier, 284
 Lakshmi, K.V., 329
 Lam, T.K., 237
 Lamport, Leslie, 78, 144
 Lang, Tomas, 77, 410, 628
 Langdon, Glen, 149, 408
 LANL - Los Alamos Nat'l Lab, 31
 Lantz, Mark, 107
 Laplace Stieltjes Transform - LST, 102
 Larson, Per-Ake, 554
 latch-free data structure, 554
 latency reduction, 236
 latency tail, 234
 latency vs redundancy, 234
 Latex, 144
 Latin squares data allocation, 583
 Lau, Edwin, 41
 Lavenberg, Steve, 55, 164
 Lazowska, Ed, 41, 55, 388
 lazy replication, 190
 LBG - Linde-Buzo-Gray algorithm, 419
 LCP Linearly Compressed Pages, 156
 LDR - Local Dimensionality Reduction, 416, 419
 leaky bucket control, 206
 Lee, Edward, 228, 278, 281–283

Lee, Patrick P.C., 213, 302, 636
 Lee, Sang-goo, 377
 Leffler, Samuel, 200
 left symmetric layout, 281
 Leighton, F. Thomson, 97
 Lempel-Ziv - LZ encoding, 149
 Lenovo, 72, 165
 Levandoski, Justin, 451
 levels of SSD parallelism, 117
 Leventhal, Adam, 297
 Lewin, Daniel, 97
 LFS - Log-structured File System, 76, 311, 315
 LFS data compression, 312
 LFS defragmentation, 312
 LFU - Least Frequently Used, 39
 Li, Kai, 76, 78, 159, 236, 272
 Li, Mingqiang, 285, 302, 573, 616
 Li, Yan, 379
 Licklider, J.C.R., 179
 LightStore, 457
 Limits of Growth Studies, 102
 linear hashing, 496
 linear programming, 14
 linear scaleup, 400
 Lipovski, Jack, 410
 little endian, 9
 Little's result (queueing theory), 238
 Liu, Jane, 205
 Livny, Miron, 279, 510
 load shedding, 546
 Local Reconstruction Code - LRC, 327
 location-independent routing, 189
 lock contention, 501
 Log Structured Array - LSA, 276
 log-based protocol, 78
 Loge indirect disk, 171
 Lohman, Guy, 398
 Lomet, David, 451
 Long, Darrell, 212, 229, 231, 351, 369, 379
 LOOK scheduling, 201
 Looker LookML, 559
 Lorie, Raymond, 428, 623
 Low Density Parity-Check - LPDC, 341
 Lower Bounding Property - LBP, 420
 LPAR - Logical PARTitioning, 47
 LRU page replacement, 38
 LRU stack model, 40
 LRU-K page replacement, 39
 LRW - Least Recently Written cleaning, 117
 LSA - Log Structured Array, 315
 LSE - Latent Sector Error, 260
 LSI Logic RAID, 229
 LSM Log-Structured Merge tree, 445
 LTFS - Linear File Tape System, 106
 LTO - Linear Tape Open, 105
 LTO Ultrium, 105
 Lucas, Caro (U. Tehran), 502
 Luckham, David, 543
 Lui, John C.S., 164, 303, 636
 Lumb, Christopher, 205, 232, 234, 413
 Luo, Jiangqiang, 326
 Lustre File System, 179, 555
 LVM - Logical Volume Management, 228

Lycos Web search engine, 95
 Lynch, William, 29, 74, 201

M

M-disc, 126
 M/G/1 queueing system, 213
 M/M/1 queueing system, 388, 570
 machine data (IoT), 137
 Machine Learning - ML, 466, 477, 627
 machine repairman model, 48
 MacWilliams, Florence, 316
 Madnick, Stuart, 144
 magnetic bubble memory, 112
 magnetic core memory, 33, 101
 magnetic tape, 105, 108
 Mahalanobis distance, 416
 MAID Massive Array of Independent Disks, 350, 356
 main memory compression, 153
 main-memory-resident, 421
 makespan (operations research), 374
 MAMR - Microwave Assisted Magnetic Recording, 109, 200
 Manber, Udi, 159
 Manhattan Project, 31
 Manolopoulos, Yannis, 235, 237
 MAP capacity planning tool, 47
 MapD, 469
 MapR in-Hadoop NoSQL DBMS, 518, 540
 MapReduce, 521
 Marconi, Guglielmo, 92
 MariaDB (opensource MySQL), 506
 market basket (data mining), 509
 market capitalizations, 3
 Marklogic enterprise NoSQL DB, 537, 552
 Markov model, 619
 Markovian dependence, 238
 Markovian reliability models, 271
 Marz, Nathan, 543
 MasPar minisupercomputer, 60
 Massively Parallel Processing - MPP DB, 521
 Matick, Richard, 108
 Matisse DB - object-oriented, 534
 Matloff, Norman, 237
 Matrix Geometric Method, 232
 Matrix Geometric Method - queueing analysis, 201
 Matrix recovery method for RAID, 370
 Matthews Correlation Coefficient - MCC, 133
 Mattson, Richard, 39, 295, 296, 303
 Mauchly, John, 29, 105
 maximum entropy principle, 55
 Maxwell-Boltzmann statistic, 412
 MBR - Minimum Bandwidth Regenerating, 330
 MBR - Minimum Bounding Rectangle, 415
 MCC - MicroElectronics and Computer Technology Corp., 401, 410
 McEliece, Robert, 340
 McKusick, Marshall, 200
 McNeally, Scott, 273
 McNutt, Bruce, 279, 312, 314
 MCU - MicroController Unit, 472
 MD5 - Message Digest (128b crypto), 156, 193

MDBS - Multi-Backend Database System, 409
 MDP - Markov Decision Process, 352
 MDS - Maximum Distance Separable, 230, 251, 337
 MDSV - Multi-Dimensional Storage Virtualization, 207
 Mean Value Analysis - MVA, 53
 Megastore - highly available storage for interactive service, 435
 Megido, Nimrod, 39
 Meiko, 60
 MELP Mixed Excitation Linear Prediction, 100
 memcached distributed object caching, 525
 Memik, Gokhan, 414
 Memorex Corp., 22
 memory compaction, 38
 Memory Driven Computing - MDC, 32
 memory leak, 6
 memory resiliency, 104
 memory space fragmentation, 38
 memristor nonvolatile memory, 114
 MEMS - Micro-Electro-Mechanical System, 112, 586
 MemSQL, SingleStore, 450
 Menon, Jai, 163, 185, 235, 243, 283, 285, 295, 296, 313, 314, 318
 Merchant, Arif, 56, 213, 215, 228, 244, 294, 306
 Merkle tree hash-based data structure, 560
 Mesarovic, Mihajlo (Club of Rome), 102
 Mesos Apache container, 530
 Message Digest - MD5, 158
 message switching, 99
 metric tree, 416
 Michelangelo ML platform, 459
 Micro-Focus enterprise software provider, 558
 microbatching grouping tasks, 543
 Micron Technology, 117
 microprogramming controls the CPU, 23
 microservices, 462
 Microsoft, Excel, 511
 Microsoft Access, 506
 Microsoft Azure, 185, 327, 431
 Microsoft Azure SQL DB, 507
 Microsoft Azure Storage - WAS, 247
 Microsoft Azure Stream Analytics, 547
 Microsoft Bing web search engine, 536
 Microsoft Data Protection Manager - DPM, 162
 Microsoft Flat Datacenter Storage, 247
 Microsoft Hekaton - main memory DB, 554
 Microsoft IIS - Internet Information Service, 540
 Microsoft OLE DB - Object Linking and Embedding, 559
 Microsoft Research, 635
 Microsoft SQL Server, 505
 Microsoft SQL server, 505
 Microsoft Streaming Analytics, 549
 Microsoft Word, 145
 Miller, Ethan, 122, 231, 367, 369
 MIMD computers, 387
 MIN optimal page replacement algorithm, 38
 min-max policy, 284
 minimizing disk latency, 208

- Minimum Storage Regenerating - MSR code, 331
 - MIPS - Microprocessor without Interlocked Pipeline Stages, 67
 - MIPS - Millions of Instructions per Second, 23
 - MIPS RISC microprocessor, 67
 - mirrored disk array, 223, 272
 - Mirzakhani, Maryam, 4
 - MIT Whirlwind project, 102
 - Mitzenmacher, Michael, 234
 - MLdp Machine Learning data platform, 459
 - mobile OS, 146
 - Model 204 DBMS, 542
 - model-based RAID level selection, 585
 - Modeshape, 552
 - Modha, Dharmendra, 39
 - Mogi, Kazuhiko, 290
 - Mohan, Chandrasekaran, 501
 - Mohawk Data Sciences Corp., 98
 - MOLAP - multidimensional OLAP, 511, 512
 - MonetDB, 469
 - MongoDB NoSQL for Clouds, 520, 525
 - Monte-Carlo simulation, 604
 - Moore, Gordon, 84
 - Moore's law, 84
 - Morris, Robert J.T., 106, 164, 264
 - Morse, Samuel, 92
 - Morse code, 92
 - Mozilla, 94
 - MPEG, 152
 - MPEG2, 152
 - MPEG4, 152
 - MPI - Message Processing Interface, 523
 - MPP - Massively Parallel Processing, 57
 - MRAM - Magnetic Random Access Memory, 104
 - MRAM - Magnetorestrictive RAM, 112
 - MRM - Machine Repairman Model, 368
 - MSR - Minimum Storage Regeneration, 329, 330, 332
 - MSRP greedy algorithm, 353
 - MSS - Mass Storage System, 105
 - MTTDL - Mean Time to Data Loss, 566
 - MTTF - Mean Time to Failure, 102
 - Multi Layer Perceptron - MLP, 467
 - multi-actuator disk drive, 200
 - multi-arm disk, 74
 - multicloud management, 187
 - multicore chips, 45, 69
 - MULTICS OS, 6, 144
 - multimodel DB, 552
 - Multiple Choice Knapsack Problem, 359
 - multiple exposure, 198
 - multiple R/W heads per arm, 235
 - multiplier recoding (computer arithmetic), 10
 - multiprogramming, 47
 - multitasking, 53
 - multithreading, 45, 69, 387
 - multivalued database, 541
 - multiversion concurrency control, 435
 - Mumick, Inderpal, 450
 - MUMPS scripting language, 534
 - Muntz, Richard, 53, 303, 636
 - Murphy, Paul, 513
 - Music Download, 197
 - Muth, Peter, 206
 - Mutlu, Onur, 125
 - Mutually Orthogonal Latin Squares, 583
 - MVA - Mean Value Analysis, 55
 - MVCL - Move Character Long (S/360), 25
 - MVS - Multiple Virtual Storage OS, 25, 49, 500
 - MXT - Memory eXpansion Technology, 153
 - MySQL open-sources RDBMS, 505
 - Mythical man month book, 81
- ## N
- Nadji, Behzad, 21, 55
 - Nakayama, Marvin, 266
 - NameNode, 523
 - NameNode in Hadoop, 523
 - Namespace in Hadoop, 523
 - NAND flash memory, 116
 - Narayanan, Dushyanth, 369, 378
 - NAS - Network Attached Storage, 273
 - National Semiconductor Manufacturing (now Texas Instr. West), 7
 - Native Command Queueing - NCQ (SATA drives), 204
 - NCR - National Cash Register Corp., 30, 272
 - NCR 6298 disk array, 30
 - NCUBE, 60
 - nearest-neighbor queries, 417
 - nearest-neighbor search, 416
 - NEC Hydrastor, 162
 - Neches, Philip, 30, 401
 - Neimat, Marie-Anne, 554
 - Nelder-Mead algorithm, 397
 - Nelson, Randolph, 232, 241
 - Neo4j graph database, 531
 - Neptune Amazon graph DB, 538
 - Nerjes, Guido, 206
 - nested RAID levels, 225
 - NetApp - Network Appliance Corp., 172, 277, 316, 318
 - NetApp SnapMirror, 248
 - NetApp StorageGrid, 555
 - NetApp's OnCommand Insight, 198
 - Netflix streaming entertainment service, 139
 - network codes, 328
 - network database, 498
 - Network on Chip - NOC, 395
 - network RAID protocols, 616
 - Neuts, Marcel, 201, 232
 - NEventStore persistence library .NET, 545
 - New Julfa, Iran, 91
 - Newell, Alan, 34
 - Newsmax TV, 144
 - newspapers, 141
 - newspapers oldest, 91
 - Nexsan DDA, 162
 - NFS - Network File System, 273, 312, 315, 556
 - Ng, Spencer, 44, 103, 108, 197, 208, 218, 236, 263, 290, 303, 626
 - Ng, Ying-Wah, 265
 - Ni, Lionel, 57
 - NiagaraFiles software, 548
 - NIC - Network Interface Controller, 456
 - Nicola, Victor, 40, 266
 - Nimble Storage, 118, 172, 277
 - nine track tape, 105
 - NIST - National Institute of Science and Technology, 108, 192
 - NLP - Natural Language Processing, 517
 - NN - Neural Network, 509
 - NN inference, 467
 - NN training, 467
 - NoForce logging, 235
 - Nomad, 530
 - Nomad container, 530
 - Non-Fungible Token - NFT, 562
 - Non-Von DBM, 409
 - nonhierarchical clustering, 510
 - nonlinear integer 0/1 programming, 396
 - NonStop as a Service - NaaS, 224
 - Nook tablet (Barnes & Noble), 146
 - NOOP FIFO scheduler, 205
 - NOR flash memory, 116
 - normalizing relational DBs, 500
 - Northrop/Page Corp., 19
 - NoSQL database, 432, 521, 523
 - not-in-place-write, 446
 - Noyce, Robert (Intel), 84
 - NTIA - National Telecomm & Information Assoc., 95
 - NUMA - Non-Uniform Memory Access, 7, 127
 - Nutanix cloud computing company, 175
 - Nvidia, 471
 - NVMe specification, 391
 - NVMexpress, 424
 - NVquery accelerator, 381
 - NVRAM, 235
 - nvSRAM, 103, 111
 - NYU Ultracomputer, 5
- ## O
- Object Exchange Model - OEM, 519
 - Object Relational DBMS, 501
 - object storage market, 167
 - Object-Oriented DB, 519, 533
 - ObjectStore OO DBMS interface to C++, 534
 - Occam programming language, 60
 - OceanStore - Global-Scale Persistent Storage, 189
 - Ocelot, OpenCL MonetDB, 469
 - OCR - optical character reader, 98
 - ODBC - Open DB Connectivity, 559
 - ODE - Ordinary Differential Equation, 256, 258
 - Offline Power Greedy algorithm, 358
 - offloading DB functions, 414
 - Ogg Theora (lossy audio coding format), 152
 - Ohanyan, Alexis (Reddit), 96
 - Ohanyan, Victor, 251
 - OLAP - OnLine Analytical Processing, 510
 - OLAP Hybrid - HOLAP, 512
 - OLAP Multidimensional - MOLAP, 511
 - OLAP relational - ROLAP, 512
 - OLAP vendors, 510
 - OLE DB - Object Linking and Embedding Database, 559
 - Olsen, Ken (Digital Equipment Corp.), 33
 - OLTP - OnLine Txn Processing, 47, 200
 - Omega Interconnection Network, 410

OmniCube - SimpiVity hyperconverged infrastructure, 176
 OmniDB based on GPUQP, 470
 onboard disk cache, 76
 One Size Fits All databases, 425
 O'Neil, Elizabeth, 39, 445
 O'Neil, Patrick, 39, 445, 497
 online algorithm, 374
 ONTAP - NetApp OS, 172
 ontology, 518
 Opderbeck, Holger, 39
 open channel SSD, 425
 open source projects, 518
 OpenEdge ABL, 509
 Openstack Swift, 556
 OpenVMS operating system from DEC, 36
 operating systems, 626
 Operation Eagle Claw - US in Iran, 28
 Optical Carrier, 93
 optical disc storage, 126
 optimal data reduction for *k*-NN search, 421
 optimal offline algorithm, 374
 Optimal page replacement, 39
 optimal strip size, 283
 optimal write batch size for destage, 313
 Optimistic Concurrency Control, 434, 436
 optimistic die option, 164
 optimistic kill option, 164
 Oracle, Autonomous DB, 447
 Oracle, Exadata, 447
 Oracle AutoML, 452
 Oracle AutoML benchmarking, 453
 Oracle business intelligence suite, 558
 Oracle Corp., 502
 Oracle DB, 505
 Oracle Exadata Data Base Machine, 503
 Oracle Express Server, 511
 Oracle Graph DB, 532
 Oracle in Memory Option, 449
 Oracle Linux Virtualization Manager - OLVM, 449
 Oracle RAC - Real Application Cluster, 122
 Oracle Relational Database, 505
 Oracle Spatial & Graph DB, 533
 Oracle ZFS, 173
 Oracle's many core, main memory processing, 486
 ORCLUS clustering, 419
 ORDBMS - Object Relational DBMS, 507
 order statistics, 279
 Ordered Partition - OP main-memory index, 417
 organ pipe file organization, 208, 372, 569
 Organick, Elliott, 27
 OrientDB open source NoSQL DBMS, 552
 Orji, Cyril, 235
 Ouchi, Ken, 275, 340
 Ousterhout, John, 153, 202, 311, 351, 442
 out-of-core algorithm, 75
 out-of-order execution, 23
 overlay multicast network, 189
 overlay programming, 4
 overload control, 585
 OWL - Web Ontology Language, 537
 Ozkarahan, Esen, 411

P

PaaS Platform as a Service, 180
 packet switching, 99
 page fault in virtual memory, 37
 page level locking, 501
 page replacement algorithm, 37
 paging system analysis, 41
 Panasas - NAS for HPC, 178
 Panasas Parallel File System - PPFS, 179, 586
 Pandey, Dheeraj, 175
 Papadopoulos, Greg, 387
 paper tape, 98
 papermaking, 91
 papyrus from plant for writing, 91
 PARaid - Power Aware RAID, 365
 Parallel File System - PFS, 179, 586
 parallel sysplex - mainframes sharing expanded storage, 387
 PARC - Palo Alto Research Center, 273
 parchment - animal skin for writing, 91
 Pareto self-similar/fractal traffic, 354
 Parhami, Behrooz, 410, 628
 Paris, Jehan-Francois, 212, 229, 231, 338
 parity logging - reducing small-write penalty in RAID5, 285
 parity sparing - data reconstruction space, 296
 parity striping to preserve data sequentiality, 198, 283
 Park, Chan-Ik, 230, 324, 347
 ParsyTec - transputer-based parallel computers, 61
 PASTA - Poisson Arrivals See Time Averages, 238, 292
 Pasupathy, Shankar, 131
 Patel, A. M., 337
 Paterakis, Michael, 206
 Patt, Yale, 212, 271, 279
 Patterson, David, 36, 273, 278, 282, 413
 Patterson, R. Hugo, 76, 77, 159
 Paxos consensus protocol, 456
 Paxos protocol, 430, 455
 PC - Personal Computer, 71
 PCA - Principal Component Analysis, 417
 PClexpress high-speed bus standard, 424
 PCM - Permanent Customer Model, 244, 294
 PCM - Phase-Change Memory, 114
 PCM - Plug-Compatible Machine, 55, 93
 PCM - Pulse Code Modulation, 32
 PCRAM - Phase Change RAM, 110
 PDDL: Permutation Development Data Layout for RAID, 310
 PDF - portable document format, 138, 139, 151
 PDL - Parallel Data Laboratory at CMU, 271
 PDS - Pavilion Data Systems, 178
 PDS - Swarm, 178
 Pease, David, 329
 peer-to-peer computing, 439, 628
 Pekhimenko, Gennady, 156
 Peng, Gang, 207, 586
 pentagon code, 331
 performability combines performance & availability, 254
 performance analysis, 627
 performance isolation, 439
 Pergamum: energy-efficient archival storage, 367
 periodic update for destaging, 285
 Perkin-Elmer, 36
 Permanent Customer Model - PCM, 228
 permanent sparing - EMC Symmetrix, 224
 Perot, Ross, 22
 Perot Systems, 22
 persistent memory programming, 136
 Personal Storage - PS, 199
 PERT - Program Evaluation and Review Technique, 16
 pervasive computing, 180
 Petri nets, 482
 PFA - Predictive Failure Analysis, 243, 301
 PFF - Page Fault Frequency, 39
 Pfister, Greg, 5, 388, 394
 Pi calculation, 71
 Pierce, John, 32
 Pinheiro, Eduardo, 132, 301, 354, 359, 370
 PinK KV-store, 457
 Pinnacle data compression chip, 153
 pipelining, 11
 Pivot 3, 175
 pivot point - delineate inner & outer disk cylinders, 232
 pivoting in datawarehousing, 511
 PL/I programming language, 6
 Plank, James, 78, 316, 318, 340
 Plug Compatible Machine - PCM, 3, 22
 Plug Compatible Mainframe, 20
 PMDS - Partial Coding for SSDs, 334
 PMR - Perpendicular Magnetic Recording, 109
 PMS - Processor Memory Switch, 35
 PNFS Parallel NFS, 273
 PNUTS - Yahoo!'s data serving platform, 434
 Poduska, William - Prime Computers, 7
 Point Access Method for multidimensional indexing, 415
 Poisson arrivals, 238, 388
 POJO - Plain Old Java Object, 522
 PolarDB, 456
 PolarFS, 456
 Poliafito, Antonio, 266
 Pollaczek-Khintchine formula, 213, 238
 polling system, 106
 Polyzois, Christos, 215, 235
 Pond - an OceanStore prototype, 189
 Pony express, 92
 pooled storage, 274
 Popek, Gerald, 530
 Popular Data Concentration - PDC to save energy, 359
 populate process - builds columnar indices for DBIM, 450
 PostgreSQL - open source RDBMS, 505, 506
 POWER - Performance Optimized With Enhanced RISC IBM microprocessor, 73
 Power Flow Study, 16
 power proportionality, 378
 power to frequency relationship, 379
 Power10 inception clustering, 27
 PowerProtect Data Domain, 162

powersaving mode, 233
 PPDFS - power proportional distributed file systems, 371
 Prabhakaran, Vijayan, 199
 Precisely, formerly Syncsort, 406
 precision=% retrieved which are relevant, 418
 preemptive disk scheduling, 293
 prefetching, 18, 74, 76
 prefetching informed, 77
 prefetching programmer inserted, 77
 PREFigURE, 378
 Prime computer, 7
 Prime data layout, 305
 Principles of Operation Manual, 24
 printers, 92
 printing - inkjet, 92
 printing - laser, 92
 printing 3D, 92
 printing press Gutenberg, 91
 printing press Mideast, 91
 PRO - Popularity-based Reconstruction Optimization, 290
 probabilistic routing of disk requests, 232
 probability of deadlock in transaction processing, 42
 Processing in Memory - PIM, 125
 ProClus, 419
 project Genie, 48
 Project Gutenberg, 146
 Project MAC, 7
 Prolegomenon NVM DB, 127
 Prometheus event monitoring, 529
 Proof-of-Work - PoW, 561
 Property Graph Model - PGM, 532
 Proteous simulator for RAID performance, 212
 Protocol buffers to serialize structured data, 518
 ProveDB, 459
 Proximal I/O - disk scheduling, 205
 PSW - Program Status Word for S/360, 25
 public ledger, 560
 PUE - Power Usage Effectiveness, 383
 punched cards, 98
 punched or perforated paper tape, 98
 Pure Storage, 121, 122, 173
 Pure Storage Flashblade, 556
 Pure Storage Purity, 122
 push-based update, 189
 PVFS - Parallel Virtual File System, 179
 PwC Consulting, 275
 Pyramid codes, 327
 pyramid tree, 415
 PyTorch, 459

Q

Q100 Data Processing Unit, 485
 Qlik provides business analytics platform, 559
 Qlik Sense, 559
 Qlik View, 559
 Qlik's Associative Engine, 559
 QLISM - Query Locality Set Model, 78
 QNM - Queueing Network Model, 47, 54
 QNN - Quantum Neural Network, 485
 QoS - Quality of Service, 585
 QoS guarantees, 206

Quad-trees, 415
 Qualcomm Research, 635
 Quality of Service - QoS, 206
 quantization, 10, 467
 Quantum Computing, 485
 Quantum Neural Network, 485
 queries per joule, 441
 query optimizer for relational DBMS, 499
 Quest Data Recovery, 162
 Quinlan, Sean, 158
 Qumulo cluster, 556
 quorum systems, 451

R

R-tree for multi-dimensional data, 415
 R*-tree for multi-dimensional data, 415
 Rabin, Michael, 168, 309
 Rabinow, Jacob, 108
 race to idle, 379
 racetrack memory, 126
 rack awareness, 225
 Raft consensus protocol, 455, 456
 Rahm, Erhard, 393
 RAID - Redundant Array of Independent / Inexpensive Disks, 270
 RAID level selection, 585
 RAID+, 583
 RAID-Z, 274
 RAID0, 278
 RAID1, 223
 RAID1 performance analysis, 243
 RAID2, 280
 RAID3, 281
 RAID4, 281
 RAID5, 281, 282
 RAID5 normal mode performance, 214
 RAID5 rebuild processing, 288
 RAID6, 315
 RAID7, 326
 RAIDframe, 212, 310
 RAIDP: 2 replicas+ check blocks, 247
 RAIDshield, 301
 RAIF - Redundant Array of Independent Filesystems, 586
 RAIL - Redundant Array of Independent Libraries, 106
 RAIM - Redundant Array of Independent Memories, 103
 Rainfinity - File Management Appliance, 272
 RAMAC - Random Access Method of Accounting and Control, 108
 RAMAC Virtual Array - RVA, 165, 276
 Ramakrishnan, K.K., 238
 Ramakrishnan, Raghu, 510
 Ramamorthy, Chitoor, 315
 Ramchandran, Kannan, 329
 RAMCloud - Distributed DRAM-based KV-store, 442
 Rancher, 531
 Rancher container, 531
 Rand Corp., 99
 Random Access NoSQL DB, 519
 Random page replacement, 39
 random permutation, 307
 randomization / uniformization, 257
 Rangaswami, Raju, 209, 370
 ranking methodology of FB engines, 503
 Rao, K.K., 295, 301, 618
 RAP - Relational Associative Processor, 411
 RAPID - Rotating Associative Processor for Info. Diss., 410
 rare event failures, 245
 RARES: Rotating Associative Relational Store, 410
 rasterization, 138
 Raven RDBMS at Microsoft, 459
 raw usable capacity, 160
 RBA - Rule Based Algorithm, 509
 RBER - Raw Bit error rate, 134
 RCA Corp., 22
 RDF - Resource Description Framework, 537
 RDF graph model, 532
 RDMA - Remote Data Memory Access, 274, 392, 461
 RDP - Rotated Diagonal Parity, 318
 RDS - Relational Database Service - Amazon, 451
 Read After Write - RAW, 302
 read routing in RAID1, 240
 Read-Modify-Write - RMW in RAID5/6/7, 229
 Real Application Cluster - RAC, 449, 450
 Real-time Compression Appliance, 162
 real-time requirements, 433, 546
 real-time stream processing, 543
 Realm DB object-oriented for mobile devices, 526
 rebuild analysis, 257
 rebuild analysis simplified, 259
 rebuild baseline with no read-redirection, 289
 rebuild mode, 281
 rebuild piggybacking, 290
 Rebuild Unit - RU, 289
 Rebuild with read redirection, 290
 recall=fraction relevant retrieved, 418
 ReConstruct Write - RCW, 229
 Recurrent Neural Network - RNN, 467
 Recursive Queue Analyzer - RQA, 18
 RED AI, 382
 Red Hat Ceph Storage, 556
 reddit - news aggregation, rating, discussion website, 96
 Reddy, Narasimha, 205, 206, 303
 RedHat Linux, 186, 433
 RedHat OpenShift, 531
 Redis, 524
 Reed, Irving, 316
 Reed-Solomon - RS code, 296, 316
 Regional(1) files, 495
 Regression Analysis, 509
 Reiher, Peter, 365
 Reiser, Martin, 55
 relational algebra, 499
 relational database, 505
 relational database model, 499
 Relational Database Service - RDS (AWS), 432
 relational DBMS +/-s, 504
 Relational DBMS - RDBMS, 504
 Relational Technology Incorporated, 534

- relative files, 495
 - reliability analysis, 627
 - reliability function, 102
 - RELPR Relatively Prime, 306
 - remote mirroring for data recovery, 248
 - removable disk packs, 14
 - REO - RAID Engine and Optimizer, 567, 586
 - replica placement, 264
 - replicating warm data, as in HDFS, 247
 - replication, 223, 523
 - ReRAM, 111
 - RESAR - Robust, Efficient, Scalable, Autonomous, Reliable Exabyte Storage, 231
 - residue number system, 10
 - resolvable design, 310
 - Resource Description Framework - RDF, 537
 - resource disaggregation, 464
 - REST - Representational State Transfer principles, 520
 - RESTful Web API, 536
 - restriping, 295, 582
 - RethinkDB, 444
 - Reuter, Andreas, 400
 - Riedel, Eric, 203, 213, 364, 412
 - Rijndael - Advanced Encryption Standard, 317
 - Ripple Carry Adder - RCA, 10
 - RISC - Reduced Instruction Set Computer, 7, 36, 273
 - RISC POWER processor, 73
 - Riska, Alma, 203, 213
 - Rissanen, Jorma, 149
 - Rivest, Ronald, 192
 - RLE - Run-Length Encoding, 149
 - RM2 disk array, 324
 - RMF - Resource Management Facility (MVS), 567
 - Robertson, James, 10
 - Robinson, John, 39, 153, 288
 - robotic tape libraries, 106
 - Roche, James, 285
 - Rocket D3, 542
 - Rocket Software, 541, 542
 - RocksDB, 456
 - rollback-recovery techniques, 78
 - ROM - Read-Only Memory, 23
 - Rometti, Ginni, 20
 - roofline bound, 82
 - Rosenblum, Mendel, 351
 - rotational delay or latency, 199
 - rotational latency or delay, 199
 - Roth, Ron, 340, 347
 - round-robin routing of disk requests, 232
 - routing disk requests, 231
 - rowstores vs columnstores as in MemSQL, 450
 - Rowstron, Antony, 369
 - RPM - Rotations Per Minute, 199
 - RPO - Recovery Point Objective, 166
 - RPS - Rotational Position Sensing, 55, 236
 - RPS reconnect delay, 56
 - RRDtool, 529
 - RSA - Rivest Shamir Adleman encryption, 193
 - RTI - Relational Technologies Inc., 502
 - Rubrik cloud data management, 557
 - Rucio Scientific Data Management, 521
 - Ruemmler, Chris, 208
 - rule-based RAID level selection, 585
 - run clustering, 372
 - Runge-Kutta method for ODEs, 256
 - Ryu, In Kyung, 164, 400
- ## S
- S/360 emulation, 24
 - S/360 simulation, 24
 - SaaS - Software as a Service, 180, 431, 507, 535
 - Sacco, Giovanni, 77
 - Saha, Debanjan, 451
 - Sahner, Robin, 266
 - Salem, Kenneth, 208, 279
 - Salesforce cloud-based software company, 535
 - Salinger, Patricia, 499
 - SALSA - SoftwARE Log Structured Array, 118
 - SALSA SCM - Storage Capacity Manager, 120
 - Saltz, Joel, 413
 - Sammet, Jean, 5
 - Sanders, D.A., 355
 - SanDisk/Western Digital Flash SSDs, 117
 - SAP ASE - Sybase SQL Server, 507
 - SAP Business Intelligence, 511
 - SAP business objects, 558
 - SAP HANA, 507, 552
 - SAP OrientDB, 527
 - SAPTF - Shortest Adjusted Processing Time First, 203
 - SAS event stream, 547
 - SAS Event Stream Processing, 550
 - SAS Serial Attached SCSI, 424
 - satellite communication networks, 93
 - SATF - Shortest Access Time First, 171, 232
 - SATF with lookahead, 203
 - SATF-CP: Conditional Priority, 203
 - Satoshi Nakamoto, 560
 - Satyanarayanan, Mahadev, 76
 - Sauer, Caetano, 279
 - savepoint, 79
 - Saxena, Mohit, 329
 - Sayano, Masahiro, 340
 - Sayood, Khalid, 149
 - Scality Ring - software-defined object storage, 556
 - SCAN scheduling, 201
 - SCAN-EDF, 205
 - scheduling multiple arms, 236
 - schema constraint, 505
 - Schema Definition Language - SDL, 518
 - schema-agnostic, 539
 - Scherr, Alan, 48
 - Scheuermann, Peter, 373, 587
 - Schindler, Jiri, 131, 199, 205, 232, 244, 301
 - Schkolnick, Mario, 77, 500
 - Schlosser, Steven, 199
 - Schmuck, Frank, 179, 228
 - Schroeder, Bianca, 131, 249, 298, 425
 - Schwabe, Eric, 304
 - Schwan, Karsten, 371
 - Schwarz, Thomas, 212, 229, 231, 308, 369
 - scientific computer, 3
 - SCM Storage Class Memory, 171, 567
 - Scotch parallel file system, 271
 - scrubbing period, 298
 - SCSI - Small Computer System Interface, 199, 302
 - SCSI LUN - Logical UNit, 585
 - Scylla, 540
 - Scylla datastore, 540
 - Scylla DB, 539, 540
 - SDC - System Development Corp., 99
 - SDC Software Defined Compute, 177
 - SDC Software Defined Environment, 177
 - SDI - Stepwise Dimensionality Increasing tree, 422
 - SDK - Showa Denko Corp., 109
 - SDK - Software Development Kit, 558
 - SDN Software Defined Networking, 177
 - SDS - Software Defined Storage, 118, 175, 462
 - SEA placement, 376
 - SEA Striping-based energy aware, 376
 - Seagate Mach2, 200
 - Seagate Technology Holdings plc, 109
 - search engines, 95
 - search engines for text, 534
 - Sears catalog, 93
 - SEC/DED, 281
 - second chance page replacement, 38
 - sector cache, 44
 - Secure Digital - SD-Card, 118
 - Secure Hash algorithm - SHA1, 158
 - Sedna - DBMS for native XML storage, 543
 - Seeger, Bernhard, 494
 - segmentation, 36
 - segmented main memory, 27
 - Self* storage system, 179
 - self-describing data, 519
 - Self-Encrypting Drive - SED, 336
 - Seltzer, Jerome, 144
 - Seltzer, Margo, 200, 202, 623
 - semantic web, 537
 - semi-structured data, 519
 - Seneca remote mirroring for disaster recovery, 248
 - SEQUEL, 499
 - sequencer, 130
 - Sequent Balance, 7
 - Sequent computer systems (symmetric multiprocessing), 45
 - Sequent Symmetry, 7
 - sequential files, 494
 - sequentiality of access, 74
 - Sequin, Carlo, 273
 - serialization framework, e.g. Protocol buffers, 518
 - Service Message Block - SMB, 556
 - service rate, μ , 6
 - set-associative CPU cache, 44
 - Setia, Sanjeev, 285, 313
 - Sevcik, Kenneth, 41, 55, 411
 - seven dwarfs, 27
 - SHA - Secure Hash Logarithm, 193
 - shadow IT, 180
 - Shahabuddin, Perwez, 266
 - Shamgunov, Nikita (cofounded MemSQL), 450
 - Shamir, Adi, 192

- Shamir secret sharing algorithm, 168
- Shannon, Claude, 32, 147
- Shannon-Fano coding, 147
- Shannon's theorem, 148
- shared everything system, 387
- Shared Memory Multiprocessor - SMP, 428
- shared nothing system, 388
- shared-disk or data sharing system, 387, 388, 393
- Sharif/Aryamehr University, 4
- Shaw, David (D. E. Shaw & Co), 409
- Shenoy, Prashant, 82, 199, 206
- Sherpa - Yahoo!'s cloud storage platform, 434
- Shifted Declustering - SD, 231
- Shin, Kang, 208
- Shingled Magnetic Recording - SMR, 199
- short stroking - use few outer cylinders, 204, 214
- shorthand - stenography, 100
- Shriver, Elizabeth, 56
- Shu, Jiwu, 285, 616
- Shum, Annie, 56, 279
- Siewiorek, Daniel, 34, 289
- signed-digit computer arithmetic, 10
- Silberschatz, Avi, 626
- silent data corruption, 274
- Silvester, John, 279
- simplex reflection method - optimization, 397
- simulated annealing - finding probabilistic global optimum, 238, 393
- simulation mode - IBM S/360, 15
- Sing, Siang Sun, 409
- Singh, Jaswinder Pal, 57
- Single Instance Storage - deduplication, 158
- single pass restore, 279
- SingleStore, MemSQL, 450
- Singleton bound, 337
- singular values, 418
- SIS - Single Instance Storage, 157
- Sisense - BI, AI-driven, software, 559
- Sisense Pulse, 560
- Sites, Richard, 56
- Sivasubramaniam, Anand, 354, 364
- Sivathanu, Muthian, 199
- six-sigma - quality control methodology, 183
- SJF - Shortest Job First, 47, 202, 232
- SK Hynix, 118
- skew parameter, 376
- sklearn estimator, 454
- SkyLaTeX, 138
- SLA - Service Level Agreement, 353
- SLED - Single Large Expensive Disk, 278
- Sloane, Neil, 316
- Slootman, Frank, 158, 367
- Slotnick, Daniel, 57
- small write penalty to update RAID5 parity, 276
- Smaragdakis, Yannis, 153
- Smart Fusion Block Transfer (Oracle Exadata), 449
- SMART: Self-Monitoring Analysis and Reporting Technology, 301
- SmartSTOR project, 414
- SMB - Server Message Block, 556
- SMD - Storage Module Drive, 31
- SMF - Systems Management Facility (MVS), 567
- Smith, Alan Jay, 39, 40, 74, 199, 201, 204, 208, 371
- Smith, Keith, 200, 205
- Smotherman, Mark, 20
- SMP - Symmetric MultiProcessor, 45
- SMPP - Short Message Peer_to_Peer, 543
- SMR - Shingled Magnetic Recording, 210
- SMS - Short Message Service, 543
- SMS - Storage Management System, 496
- SMSC - Short Message Service Center, 543
- SMT - Simultaneous MultiThreading, 69
- SMTP - Simple Mail Transfer Protocol, 519
- Snappy data compression, 148
- SNIA - Storage Networking Industry Association, 135
- snooping cache coherence in multiprocessors, 34, 45
- Snowflake - cloud computing based data warehousing, 158, 436
- SOAP - Simple Object Access Protocol, 519
- social media, 137
- Sockut, Gary, 351
- soft computing handles imprecision, 398
- Softlayer - cloud computing provider, 186, 433
- Software AG Apama streaming analytix, 549
- Software Defined Data Center - SSDC, 175
- Software Defined Storage, 555
- Software Development Kit - SDK, 452
- Solaris OS (Sun Microsystems), 273
- Soljanin, Emina, 234
- Solomon, David, 149
- Solomon, Gustave, 316
- Solr search platform, 535
- solver-based, 585
- Solworth, Jon, 235
- Solyndra solar panel, 45
- Sony Group Corp., 140
- Sorenson video codec, 152
- sort-order preserving data compression, 149, 451
- sorting, 105
- sound-ex, 517
- sources of big data, 137
- SPA - Supplementary Parity Augmentation, 316
- space partitioning index, 421
- SPARC - Scalable Processor ARChitecture, 66, 273
- sparing by merging RAID5 arrays, 594
- sparing by restriping in RAID5/6/7, 594
- sparing dedicated in RAID5/6/7, 594
- sparing distributed in RAID5/6/7, 594
- Spark - Hadoop enhancement to MapReduce, 523
- SPARQL for RDF databases, 537
- sparse indexing vs dense for all keys, 160
- Spatial Access Method, 415
- spatial dedupe locality, 160
- spatial locality, 43, 372
- SPC - Storage Performance Council, 404
- SPEC - Standard Performance Evaluation Corporation, 404
- SPEC benchmark, 69
- Spectra Logic's nTier, 162
- speech recognition, 100
- Sphinx search engine, 535
- spin transfer torque MRAM technology, 104
- spindle motor for disk rotation, 364
- Spivey, Mike, 251
- SPL - Stream Processing Language, 547
- split seek option for rebuild reads, 290
- Splunk - log data mining, 534, 556
- SPMD - Single Program Multiple Data, 474
- spooling - multiprogramming to print, 19
- SPRTF - Shortest Processing Time First, 47
- SPSS - Statistical Package for Social Sciences, 14
- SQLite, 506
- SQLite RDBMS in C-library, 506
- SR Sphere/Rectangle tree for high-D, 415
- SRA - Shortcut Reliability Analysis, 254
- SRAM - Static RAM, 43, 103
- SRMap Sample-Replicate-Consolidate Mapping, 370
- SRPT - Shortest Remaining Processing Time First, 47
- SS - Similarity Search tree, 415
- SSD - Solid State Drive, 24, 225
- SSD parallelism, 116
- SSD vs RAID, 116
- SSF - Shortest Seek First, 202
- SSMS - Secret Sharing Made Short, 168
- SSPiRAL disk array, 229
- SSTF - Shortest Seek Time First scheduling, 201
- stack algorithms for page replacement, 39
- stack computers, 27
- staircase join operator, 543
- stale data, 433, 546
- Stallman, Richard, 144
- Stamos, James, 428
- STAR coding, 326
- star plot, 35
- star schema for data warehousing, 500, 511
- STAR Self-Testing & Repairing, 101
- Stardog RDF store, 538
- static routing, 232
- statistical multiplexing, 40
- steganography - hiding messages, 193
- stemming, 517
- stenography - shorthand, 100
- Stewart, William, 628
- STK Iceberg, 276
- Stockmeyer, Larry, 314
- Stodolsky, Daniel, 285
- Stonebraker, Michael, 425, 502
- Stonebraker's classification, 387
- storage - maintenance-free - OceanStore, 189
- Storage Area Network, 273
- storage brick environment - Ursa Minor PDL, 179
- storage bricks, 586
- Storage Class Memory - SCM, 110
- storage fragmentation, 314
- storage lifespan, 126
- Storage Technology Corp. - STK, 276
- storage thrashing - excessive paging, 584

storage virtualization, 586
 StorageTek, 503
 StorageTek archival tape system, 173
 StorageTek Iceberg, 315
 stored procedure for relational databases, 450, 554
 Storer, James W., 149
 Storer, Mark, 367
 StorOne, 178
 Storwize array with compressed data, 173
 stranded resources, unused local memory, 463
 Stratus computers, 402
 Stream Processing Engine, 551
 streaming analytics, 545
 Strecker, Bill, 33
 stretched cluster mode - automatic datacenter failover, 167
 strip or striping unit, 278
 Stripe and Mirror Everywhere - SAME, 585
 Stripe-Oriented Rebuild - SOR, 289
 striping - partitioning files into strips, 278, 279, 565
 Striping Unit - SU, 278
 STTRAM - Spin Transfer Torque RAM, 110, 111
 Su, Stanley, 407, 410
 submarine cables, 93
 subspace clustering, 418
 subtitle translator, 100
 Sum of Squared Distances, 510
 Sun Microsystems, 273, 503
 Sun ZFS, 274
 superparamagnetic limit, 109
 superscalar processor, 73
 Support Vector Machine - SVM, 509
 Suprenum, 60
 Surestore, HP based on AutoRAID, 277
 surveillance data, 137
 SUSE Enterprise Storage - SES, 557
 Sutherland, Ian M., 304
 SVC Storage Virtualization Code, 167
 SVD - Singular Value Decomposition, 416
 Svoboda, Antonin, 10
 Swami, Arun, 509
 SWIFT - Society for Worldwide Interbank Financial Telecommunication, 28
 SwiftStack software-driven data storage, 556
 SWP - Small Write Penalty, 284
 Symantec - Cybersecurity software, 177
 symmetric shared key cryptography, 193
 synchronized disk interleaving, 279
 synchronized disks, 280, 281
 synchronous replication, 432
 Syncsort now Precisely, 406
 System R characteristics, 425
 System R relational DB prototype, 499
 system shared disk, 387
 systematic code encoding includes input, 347
 systolic arrays, 409

T

table scan sequential access, 413
 Table storage, Azure, 540
 Tableau, 535
 Tag Command Queueing - TCQ, 204
 tagging-based, 585
 Takacs, Lajos, 291
 Takagi, Hideaki, 106, 289
 Tandem Corp., 271, 399
 Tandem NonStop SQL, 271, 399
 Tandem's NonStop SQL - TNS, 224
 Tanenbaum, Andrew (Vrije U.), 626
 Tantawi, Asser, 241, 279
 tao - 9-track, 105
 tape - 7 track, 15
 tape density records, 107
 tape library analysis, 106
 tape mounting policies, 106
 Tata Consulting Services - India, 501
 Tauschek, Gustav, 37
 Tcl - Tool command language, 542
 TDP - Thermal Design Power, 479
 Tech Mahindra - India, 501
 technology adoption phases, 561
 Tegile/DataDirect Networks, 174
 telegraph, 92
 Temporal databases, 545
 temporal dedupe locality, 160
 temporal locality, 44
 TenCent WeChat, 92
 Tensorflow, 459
 Teorey, Toby, 201
 Teradata, 30
 Teradata Corp., 272, 401, 506
 Teradata DBC/1012, 224, 272
 Teradata File Segment cache, 553
 tertiary storage, 105
 Tesla, Nikola, 11, 92
 test and write (synchronization), 434
 TEX typesetting system, 144
 text data pond, 517
 text editor, 144
 text-editor emacs, 144
 text-editor pico/nano, 144
 text-editor SCRIPT (IBM), 144
 text-editor TECO, 144
 textual data, 517
 textual disambiguation, 517
 Thekkath, Chandramohan, 228
 Thereska, Eno, 378
 thermal design, 364
 thin clients, 47
 thin provisioning, 172, 178
 Thinking Machines, 59
 Thinking Machines CM2, 280
 Thinking Machines DataVault, 280
 Thomasian, Alex, 55, 57, 203, 209, 232, 235, 237, 251, 279, 284, 285, 288, 290, 322, 351, 395, 585, 586
 Thomason, Michael, 341
 Thomson Reuters, 141
 Thorp shuffle, 306
 ThoughtSpot, 559
 thrashing external, 77
 thrashing internal, 77
 Tian, Lei, 290, 316
 TIBCO Software, 550
 TIBCO Streambase, 547
 TickerTaip, 171, 615
 tied requests deferred to 95th percentile, 233
 TIFF - Tagged Image File Format, 147
 Tiger Shark Parallel File System, 179
 TikTok video sharing from ByteDance, 503
 time fault-tolerance, 132
 time to disk failure, 271
 Time-series DB, 527
 time-sliders feature, 274
 timesharing system, 47
 Timestamp Ordering Method, 452
 TimesTen main-memory DB, 554
 TinyURL, 517
 TIP - Transparent Informed Prefetching, 76
 TLB - Translation Lookaside Buffer, 38, 46, 75
 TMR - Triple Modular Redundancy, 101
 TNT - Turner Network Television, 140
 TokuDB high-performance storage engine, 495
 TokuTek/Percona high-performance DB, 444
 Tomasulo algorithm - S/360 Model 91, 23
 Torvalds, Linus, 65
 Total DBMS, Cincom, 498
 Towsley, Donald, 40, 226, 231, 284
 TPC - Transaction Processing Council, 402
 TPC-C OLTP benchmark, 353
 TPC-H decision support benchmark, 353
 TPF - Transaction Processing Facility, 25, 497
 TPM - Traditional Power Mode, 354
 TPOT - Tree-Based Pipeline Optimization Tool, 454
 TPU - Tensor Processing Unit, 477, 482
 trade publications, 633
 training time in ML, 381
 Transact-SQL Microsoft/Sybase extension, 554
 transactional data - info about txns, 137
 Translation Layer - TL, 118
 Transmeta Corp. low-power x86 on VLIW, 31
 transparent informed prefetching, 76
 transposed mirror organization, 232
 transputer - micro with 4 serial links, 60
 TREC - Tehran Regional Electric Company, 15
 Treiber, Kent, 235, 285
 Tremaine, R. Brett, 153
 Triantafillou, Peter, 206
 Trill - high performance query processor, 548
 trilogy systems - wafer scale integration, 45
 Trivedi, Kishor, 249, 251, 256, 266, 627
 troff typesetting for UNIX, 144
 TSO - Time-Sharing Option, 49
 TSO TimeStamp Ordering, 616
 TTL - Time-To-Live, 525
 Tuduce, Irina, 153
 Turing award, 499, 502
 Turner, Ted - CNN founder, 143
 Turner Broadcasting Systems, 140
 TV stations (US), 142
 two level queueing, 206
 two-headed disk optimal placement, 236
 two-phase commit - 2PC distr. txn processing, 78
 tye.io, 162
 typesetting, 91

U

UBER - Uncorrectable Bit Error Rate, 134
 ubiquitous computing, 180
 Ubuntu OS version of Linux, 273
 UDE - Undetected Disk error, 301
 UltraRAM - universal memory technology, 125
 UML - Unified Modeling Language, 616
 UN - United Nations, 92
 UniBench benchmark for multimodel databases, 553
 Unicode, 9
 UniData/UniVerse - multivalued DB, 541
 uniform routing, 232
 Unisys=Burrhoughs+UNIVAC, 27
 unit record equipment, 98
 unit record machine, 98
 Unitrends, 557
 UNIVAC computers, 27, 29
 Universal Postal Union, 92
 UNIX file system, 200
 UNIX operating system, 6
 Unum FLP, 9
 upcoding in HACFS, 329
 UPS - Uninterruptible Power Supply, 24, 110, 225
 UPS - United Parcel Service, 93
 URE - Undetected Read Error, 302
 URL - Uniform Resource Locator, 94
 URL shortener, 150
 Ursa Minor PDL brick prototype, 179
 USENIX, 630
 User Datagram Protocol - UDP, 519
 USP - User Service Platform, 523
 USPS - United States Postal Service, 92
 utf8 code, 9
 utility function, 353
 utility functions for storage design, 587
 utilization factor, ρ , 6, 238
 UWE - Undetected Write Error, 302
 Uysal, Mustafa, 213, 413, 586

V

Vacationing Server Model in M/G/1 queues, 163
 Vahdat, Amin, 353
 Vakali, Athena, 236, 237
 van Tilborg, Henk, 343
 Vandermonde matrix, 316
 Vantage-point tree, 416
 Vardy, Alexander, 340, 347
 Variable Neighborhood Search, 398
 variable page sizes, 38
 variable scope parity protection, 288
 variable width stripes, 274
 Varian, Hal, 136
 Varki, Elizabeth, 215
 Varma, Anujan, 235, 285
 Varman, Peter, 365
 VAs Virtual Arrays, 566
 VAST data, 167
 VAX-cluster, 388
 VC1 video compression, 152
 Vector Approximation-file, 416
 vector-packing, 568

Veeam Cloud Connect for Service Providers, 557
 Veitch, Alistair, 248
 Venkatesan, Vinodh, 260, 262, 264
 Venti, 158
 Verbitski, Alexandre, 451
 Verify command, 298
 Verisign, 95
 Veritas, 177, 557
 Veritas backup Appliance, 163
 Veritas Flex Appliance, 177
 Veritas Netbackup, 177
 Veritas Volume Replicator, 248
 Versant Object Database - VOD, 534
 Vertica analytic DB, 539
 vertical placement, 587
 vertical scaling, 523
 Vesta parallel file system, 179
 VIA - Virtual Interface Architecture, 392
 ViacomCBS, 141
 video compression, 152
 video file formats, 138
 Vingralek, Radek, 373
 Violin memory, 118
 Violin Systems, 174
 Virginian GPU accelerated DB, 470
 Virtual Array (VA) width in HRAID, 572
 virtual cut-through, 99, 100, 395
 Virtual Machine, 529
 Virtual Machine VM/370, 26
 virtual memory, 4, 24, 36
 virtual memory snapshot, 554
 virtual memory thrashing, 41
 Virtual Storage Appliance - VSA, 174
 virtualization application, 180
 virtualization requirements, 26
 virtualization server, 180
 Virtuoso Universal Server, 509, 552
 Vitria Operational Intelligence, 547
 Vitter, Jeffrey, 76
 VLIW - Very Large Instruction Word, 31
 VM - Virtual Machine, 26, 529
 VMDS - Version Managed Data Store, 509
 VMIN page replacement, 40
 VoD - Video on Demand, 60
 Vogels, Werner, 190, 432
 Voice Coil Motor - VCM, 364
 Voltage Scaling, 442
 VoltDB, 554
 von Neumann architecture, 35
 Voruganti, Kaladhar, 367
 V(R) scheduling combining SCAN & SSTF, 202
 VSA - Virtual Storage Appliance, 174
 VSAM - Virtual Sequential Access Method, 16, 494
 VSM - Vacationing Server Model, 243, 289
 VTL - Virtual Tape Library, 162
 VTOC - Volume Table of Contents, 198

W

Wagner, David, 347
 WakandaDB DB, 509
 Walt Disney, 140

Walter Burkhard, 307
 Wang, An, 102
 Wang, Andy, 365
 Wang, David T., 44, 103
 Wang, Jun, 231, 365, 368
 Wang, Randolph, 236
 Wang, Zhiyang, 320
 Wang Institute, 102
 Wang Labs, 102
 Warner Bros., 140
 Watson, Jr., Thomas J., 3
 Watson, Sr., Thomas J., 3
 wavelet compression, 151
 way parallelism, 380
 Wayback Machine, 73, 146
 WCF - Windows Communication Function, 540
 WDL - Wait-Depth-Limited method, 501
 Weatherspoon, Hakim, 264
 WEAVER codes, 230
 web browser Apple Safari, 94
 web browser Google Chrome, 94
 web browser Microsoft Edge, 94
 web browser Microsoft Internet Explorer, 94
 web browser Mozilla Firefox, 94
 web browser Netscape, 94
 web browsers, 94
 web crawler, 96
 web crawling, 536
 Web search engines, 536
 Web Services - WS, 519
 Weber, Wolf, 370
 Weber, Wolf-Dietrich, 132, 301
 Weibull distribution, 249, 261
 Weikum, Gerhard, 39, 206, 283, 587
 Weil, Sage, 179
 Welch, Brent, 179
 Welch, Jack, 7
 Western Digital ActiveScale, 557
 Western Digital Corp. - WDC, 109, 117, 276
 Whirlwind computer, 33
 Whitehurst, James, 186
 Whitehurst, Jim, 20
 Whitman, Meg, 32
 Whitt, Ward, 232
 wide column store, 538
 Widmer, Alfred, 618
 Wikidata knowledge base, 538
 Wikidata SPARQL, 538
 Wilcke, Winfred, 110, 295, 617
 Wilhelm, Neil, 55, 201
 Wilhite, David, 149
 Wilkes, John, 56, 171, 202, 208, 248, 350, 351, 362, 584, 615, 626
 Wilkes, Maurice, 24
 Wilner, Alden, 229
 Wilson, Paul, 153
 Window-based SPTF, 203
 Windows Azure Storage - WAS, 436
 Wipro - India, 501
 Wolf, Joel, 284
 Wolff, Ronald, 292
 Wong, C.K., 112, 208, 569
 Wong, Eugene, 502
 Wong, Theodore, 618, 620

Wong, Wing Shing, 164
 working set, 37
 Working Set algorithm, 39
 World Bank, 16
 WORM - Write Once, Read Many, 128
 wormhole routing, 100, 395
 Worthington, Bruce, 203, 212, 279
 WOS - Web Object Scalar, 555
 Wright's law, 84, 120
 write amplification, 445
 write anywhere, 235
 write hole error updating parity in RAID5, 274
 write verify approach, 302
 write-invalidate for cache coherence, 46
 write-offloading to active disks for spindown
 disks, 369
 Write-Through cache, 7
 write-update for cache coherence, 46
 WSTF - Weighted Shortest Seek time First, 202
 WTC - World Trade Corp., 3
 Wu, Suzhen, 290
 WWW - World Wide Web, 94
 Wylie, Jay, 369
 Wylie, James, 205
 WYSIWYG - What You See Is What You Get,
 144, 145

X

X-code, 321

X-tree, 415
 XEROX Alto computer, 273
 XFS files, 495
 XFS SGI/Red-Hat, 495
 Xia, Mingyuan, 329
 Xie, Tao, 376, 377
 Xilinx, 66
 Xilinx FPGAs, 473
 xivstorage, 166
 XKL, 461
 XML databases, 542
 XML markup language, 519
 XQuery, 520
 Xu, Jun, 322, 585
 Xu, Lihao, 148, 157, 230, 272, 318, 326, 340,
 347

Y

Y2K bug, 5
 Yahoo, 189
 Yale, Patt, 203
 Yanai, Moshe, 166
 Yandex, 536
 Yania, Moshe, 272
 YARN - Yet Another Resource Negotiator, 521
 YCSB - Yahoo! Cloud Serving Benchmark, 405
 YCSB benchmark, 540
 Yellowfin, 511
 Yet-another Fair Queueing - YFQ, 203

Yin, Shu, 370
 Young, Honesty, 208, 371
 Youtube, 536
 Yu, Philip, 228, 244, 294, 306, 395

Z

z/Linux, 25
 z/OS, 25
 z/VM, 25
 z/VSE, 25
 Zabback, Peter, 283, 587
 Zadok, Erez, 586
 Zahorjan, John, 41, 55
 ZBR - Zoned Bit Recording, 199
 Zebra NFS, 315
 zEDC zEnterprise Data Compression, 156
 Zelenka, Jim, 310
 Zero Latency Read - ZLR, 199, 291, 574
 ZFS files, 495
 Zhang, Xiadong, 204
 Zhou, Qingbo, 358
 Zhou, Yuanyuan, 358, 362, 363
 Zipf distribution, 376
 Zipf's power law, 364
 zoned bit recording, 289
 zoning - zoned bit recording, 289
 Zoomdata - real-time data visualization, 548
 Zygaria managing storage resources, 620
 ZZFS - Zettabyte File System, 274

Storage Systems

Organization, Performance, Coding, Reliability and Their Data Processing

Alexander Thomasian

Storage Systems: Organization, Performance, Coding, Reliability and Their Data Processing was motivated by the 1988 Redundant Array of Inexpensive/Independent Disks proposal to replace large form factor mainframe disks with an array of commodity disks. Disk loads are balanced by striping data into strips—with one strip per disk—and storage reliability is enhanced via replication or erasure coding, which at best dedicates k strips per stripe to tolerate k disk failures. Flash memories have resulted in a paradigm shift with Solid State Drives (SSDs) replacing Hard Disk Drives (HDDs) for high performance applications. RAID and Flash have resulted in the emergence of new storage companies, namely EMC, NetApp, SanDisk, and Purestorage, and a multibillion-dollar storage market. Key new conferences and publications are reviewed in this book.

The goal of the book is to expose students, researchers, and IT professionals to the more important developments in storage systems, while covering the evolution of storage technologies, traditional and novel databases, and novel sources of data. We describe several prototypes: FAWN at CMU, RAMCloud at Stanford, and Lightstore at MIT; Oracle's Exadata, AWS' Aurora, Alibaba's PolarDB, Fungible Data Center; and author's paper designs for cloud storage, namely heterogeneous disk arrays and hierarchical RAID.

Key features

- Surveys storage technologies and lists sources of data: measurements, text, audio, images, and video
- Familiarizes with paradigms to improve performance: caching, prefetching, log-structured file systems, and merge-trees (LSMs)
- Describes RAID organizations and analyzes their performance and reliability
- Conserves storage via data compression, deduplication, compaction, and secures data via encryption
- Specifies implications of storage technologies on performance and power consumption
- Exemplifies database parallelism for big data, analytics, deep learning via multicore CPUs, GPUs, FPGAs, and ASICs, e.g., Google's Tensor Processing Units

About the author

Dr. Alexander Thomasian is the founder and CEO of Thomasian Associates consulting, in Pleasantville, NY, USA. As a former IBM Systems Engineer, he did a PhD in Computer Science at UCLA. Dr. Thomasian has held teaching and research positions at Case Western Reserve U., U. Southern California, Burroughs Corp., IBM T.J. Watson Research Center, and New Jersey Institute of Technology. At IBM's Almaden Research Center, he developed the analysis to predict the performance of IBM's RAID5 product under development. His storage research was funded by National Science Foundation Hitachi Global Storage Technologies, and AT&T. He was a visiting scientist of Chinese Academy of Sciences at Shenzhen and a Fulbright Fellow at the American University of Armenia in Yerevan. He is a Life Fellow of IEEE for fundamental contributions to the performance analysis of computer systems. He was an Editor of IEEE Transactions on Parallel and Distributed Systems, a monograph on database concurrency control, and 150 papers, more recently on storage systems.



ELSEVIER

elsevier.com/books-and-journals

ISBN 978-0-323-90796-5



9 780323 907965